

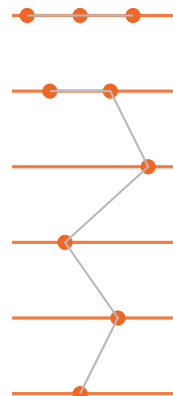
Mastering APIs

From Zero to Production Guru

A 16-Week Intensive Production-Grade Textbook

API Engineering Education Team

2026



API design as an engineering discipline

Consumption, design, creation, security, deployment, and lifecycle management

HTTP wire protocols • REST / GraphQL / gRPC • resilience • enterprise security

Mastering APIs: From Zero to Production Guru

© 2026 API Engineering Education Team. All rights reserved.

This textbook is released under an academic license. It is intended for educational and research purposes.

All LaTeX source code and Python implementations are provided as-is for learning and reference.

Contents

1	Introduction: The API Economy & How to Use This Book	1
	Executive Summary	1
1.1	Learning Objectives & Prerequisites	2
1.1.1	Learning Objectives	2
1.1.2	Prerequisites	3
1.1.3	How to Use This Textbook	4
1.1.4	Reader Pathways	4
1.1.5	Conventions Used in This Book	4
1.2	Deep Technical Mechanics & Architectural Concepts	6
1.2.1	What an API Is: A Formal Definition	6
1.2.2	The API Value Chain	6
1.2.3	Three Mental Models: Product, Contract, Failure Surface	7
1.2.4	The Five-Category API Failure Taxonomy	8
1.2.5	The API Maturity Model (L0–L4)	9
1.2.6	Map of the Book: Four Modules and a Synthesis	9
1.2.7	Reference Architecture Variants	11
1.2.8	Cost and Latency Budgeting	11
1.2.9	Governance, SLOs, and Threat Modeling: The Baseline	12
1.2.10	Scope Boundaries, Assumptions, and Risks	13
1.3	Production-Ready Code Implementation	14
1.3.1	Environment Setup Baseline	15
1.3.2	Evaluation Primer: Measuring Latency Correctly	17
1.3.3	A Complete Offline API and Client in the Standard Library	23
1.4	Common Pitfalls & Anti-Patterns	29
1.4.1	Consumer-Side Pitfalls	29
1.4.2	Provider-Side Pitfalls	29
1.4.3	Organizational Anti-Patterns	30
1.4.4	Common Myths About APIs	30
1.4.5	Common Mistakes & Debugging Gallery	31
1.5	Hands-On Production Lab & Real-World Case Study	31

1.5.1	Quick-Start Lab: Build, Run, Measure (2–3 Hours, Offline)	31
1.5.2	Production War Story: The Missing Timeout	33
1.5.3	Diagnostic Playbook: “The API Is Slow” / “The API Is Broken”	35
1.6	Self-Assessment Exercises & Deep-Dive Questions	35
1.6.1	Readiness Self-Assessment	35
1.6.2	Exercises	36
1.6.3	Deep-Dive Questions	37
1.7	Interview Q&A Vault	37
1.8	External Bibliography & Further Reading	42
1.8.1	Foundational Books	42
1.8.2	Standards and Specifications	43
1.8.3	Online Documentation and Courses	43
1.8.4	Recommended Datasets (Synthetic Only)	43
1.9	Capstone Projects	44
1.9.1	Project 0.1 — Contract Archaeology [junior]	44
1.9.2	Project 0.2 — Honest Benchmarks [mid]	44
1.9.3	Project 0.3 — Failure Taxonomy Field Guide [mid]	45
1.9.4	Project 0.4 — Budgets and Breakers for the Mini API [senior]	46
1.9.5	Project 0.5 — API Platform Governance Starter [staff]	47
1.10	Python Dependencies Appendix	47
1.10.1	fastapi	48
1.10.2	uvicorn	48
1.10.3	httpx	48
1.10.4	pydantic	49
1.10.5	pytest	49
1.11	Chapter Summary & Key Takeaways	49
1.12	Quick-Reference Card	50
1.13	Standardized Evidence Addendum	50
1.13.1	Benchmark Snapshot Template	50
1.13.2	Request Trace Template	50
1.13.3	Cost and Latency Budget Snapshot	50
1.14	Configuration Preset Template	50
1.15	CI Quality Gate Specification	51
1.16	Incident Runbook Template	51
1.17	Cross-Chapter Decision Card	52
1.18	ROI Model and Build-vs-Buy	52
1.19	Disaster Recovery Notes (RPO/RTO)	52

2	The HTTP Wire Protocol: HTTP/1.1, HTTP/2 & HTTP/3	59
2.1	Learning Objectives & Prerequisites	60
2.2	Deep Technical Mechanics & Architectural Concepts	61
2.2.1	HTTP/1.1 Message Anatomy on the Wire	61
2.2.2	Method Semantics per RFC 9110	63
2.2.3	Status-Code Taxonomy and Selection Guidance	64
2.2.4	Header Registry Essentials	65
2.2.5	HTTP/2: The Binary Framing Layer	66
2.2.6	HTTP/3 and QUIC	69
2.2.7	TLS 1.3: The Handshake Behind Every HTTPS Request	70
2.3	Production-Ready Code Implementation	73
2.3.1	Listing 1 — Raw-Socket HTTP/1.1 Client with Byte-Level Tracing	73
2.3.2	Listing 2 — Chunked Transfer Encoding and Keep-Alive, By Hand	76
2.3.3	Listing 3 — TLS Inspector: Context Configuration, ALPN, and Ciphers	80
2.3.4	Listing 4 — HTTP/2 Frame Parser over a Captured Byte Stream	82
2.4	Common Pitfalls & Anti-Patterns	86
2.5	Hands-On Production Lab & Real-World Case Study	87
2.5.1	Lab A — Dissect Real Traffic with curl.exe	87
2.5.2	Lab B — Your Raw Client vs. the Stdlib Server	88
2.5.3	Lab C — Mint a Self-Signed Certificate and Inspect TLS	88
2.5.4	Lab D — HTTP/2 Frame Dissection	89
2.5.5	Real-World Case Study: The 60-Second Mystery	89
2.6	Self-Assessment Exercises & Deep-Dive Questions	89
2.7	Interview Q&A Vault	91
2.8	External Bibliography & Further Reading	96
2.9	Capstone Projects	97
2.9.1	Capstone 1 — From-Scratch HTTP/1.1 Server [junior]	97
2.9.2	Capstone 2 — Chunked Streaming Proxy [mid]	98
2.9.3	Capstone 3 — HPACK-Lite Header Encoder [mid]	99
2.9.4	Capstone 4 — Protocol Fingerprinting & Frame Dissection Toolkit [senior]	99
2.9.5	Capstone 5 — HTTP Version Latency Profiler & Wire-Debugging Workbench [staff]	100
2.10	Python Dependencies Appendix	100
2.10.1	h2 ($\geq 4.1.0$)	101
2.11	Chapter Summary & Key Takeaways	101
2.12	Quick-Reference Card	102
2.13	Standardized Evidence Addendum	102

2.13.1	Benchmark Snapshot Template	102
2.13.2	Request Trace Template	102
2.13.3	Cost and Latency Budget Snapshot	103
2.14	Configuration Preset Template	103
2.15	CI Quality Gate Specification	103
2.16	Incident Runbook Template	104
2.17	Cross-Chapter Decision Card	105
2.18	ROI Model and Build-vs-Buy	105
2.19	Disaster Recovery Notes (RPO/RTO)	105
3	Consuming APIs in Python: From urllib to httpx	107
3.1	Learning Objectives & Prerequisites	108
3.1.1	Learning Objectives	108
3.1.2	Prerequisites	108
3.2	Deep Technical Mechanics & Architectural Concepts	109
3.2.1	The Evolution of Python HTTP Clients	109
3.2.2	Anatomy of a Robust Client	111
3.2.3	Timeout Taxonomy: Bounding Every Wait	112
3.2.4	Connection Pooling, Keep-Alive, and the Cost of Churn	114
3.2.5	Retry Engineering: Backoff, Jitter, Budgets, Idempotency	115
3.2.6	Rate-Limit-Aware Consumption	117
3.2.7	Pagination and Streaming	118
3.2.8	An Error Taxonomy for Consumers	118
3.3	Production-Ready Code Implementation	119
3.3.1	Baseline: A stdlib urllib Consumer	119
3.3.2	A Production-Grade httpx Client	122
3.3.3	Retry Engineering with tenacity	128
3.3.4	Pagination Iterators and a Streaming Downloader	131
3.3.5	Hermetic Offline Tests with httpx.MockTransport	135
3.4	Common Pitfalls & Anti-Patterns	140
3.4.1	Missing Timeouts	140
3.4.2	Retry Storms	140
3.4.3	Swallowing Exceptions	140
3.4.4	Parsing Before the Status Check	141
3.4.5	Leaking Credentials into Logs	141
3.4.6	Unbounded Decompression and Body Reads	141
3.5	Hands-On Production Lab & Real-World Case Study	142
3.5.1	Lab Setup: The Northwind Robotics Parts API Stub	142
3.5.2	Lab Steps	146
3.5.3	War Story: The Missing Read Timeout and the Retry Storm	147
3.6	Self-Assessment Exercises & Deep-Dive Questions	148

3.6.1	Exercises	148
3.6.2	Deep-Dive Questions	148
3.7	Interview Q&A Vault	149
3.7.1	Junior	149
3.7.2	Mid-Level	150
3.7.3	Senior	152
3.7.4	Staff	153
3.8	External Bibliography & Further Reading	154
3.9	Capstone Projects	155
3.9.1	Capstone 2.1 [junior] — Robust CLI Downloader	155
3.9.2	Capstone 2.2 [mid] — Rate-Limit-Aware Catalog Crawler	156
3.9.3	Capstone 2.3 [mid] — Typed SDK Wrapper with Full Test Matrix	157
3.9.4	Capstone 2.4 [senior] — Chaos-Client Harness	157
3.9.5	Capstone 2.5 [staff] — Async Concurrent Fetcher with Bounded Concurrency	158
3.10	Python Dependencies Appendix	159
3.10.1	httpx (≥ 0.27)	160
3.10.2	tenacity (≥ 8.3)	160
3.10.3	pydantic (≥ 2.7)	160
3.10.4	pytest (≥ 8.0)	161
3.11	Chapter Summary & Key Takeaways	161
3.12	Quick-Reference Card	162
3.12.1	Timeout Cheat-Sheet	162
3.12.2	Retry Decision Table	163
3.12.3	Status-Class Handling Table	163
3.13	Standardized Evidence Addendum	163
3.13.1	Benchmark Snapshot Template	163
3.13.2	Request Trace Template	163
3.13.3	Cost and Latency Budget Snapshot	163
3.14	Configuration Preset Template	163
3.15	CI Quality Gate Specification	163
3.16	Incident Runbook Template	164
3.17	Cross-Chapter Decision Card	164
3.18	ROI Model and Build-vs-Buy	164
3.19	Disaster Recovery Notes (RPO/RTO)	164
4	The REST Architectural Style & Resource Design	167
4.1	Learning Objectives & Prerequisites	168
4.2	Deep Technical Mechanics & Architectural Concepts	169
4.2.1	REST Is an Architectural Style, Not a Protocol	169
4.2.2	The Six Constraints, Formalized	170

4.2.3	“JSON over HTTP Is Not REST”: The Richardson Maturity Model	174
4.2.4	Resource Modeling Discipline	177
4.2.5	Uniform Interface Semantics	180
4.2.6	HATEOAS and Hypermedia	183
4.2.7	Representation Design	184
4.2.8	API as a Product	185
4.3	Production-Ready Code Implementation	185
4.3.1	A Stdlib-Only REST Mini-Framework: The Parts Catalog	186
4.3.2	A URI Design Validator	200
4.3.3	A HAL-Style Hypermedia Serializer	205
4.3.4	Design-First Contract: OpenAPI 3.1 Excerpt	209
4.4	Common Pitfalls & Anti-Patterns	213
4.5	Hands-On Production Lab & Real-World Case Study	215
4.5.1	Lab Part A: Design the Warehouse Domain as REST Resources	215
4.5.2	Lab Part B: Constraint Critique of a Deliberately Bad API	216
4.5.3	Case Study: The Redesign That Broke the Mobile Fleet	217
4.6	Self-Assessment Exercises & Deep-Dive Questions	217
4.7	Interview Q&A Vault	218
4.7.1	Junior (Q1–Q10)	218
4.7.2	Mid-Level (Q11–Q20)	220
4.7.3	Senior (Q21–Q28)	221
4.7.4	Staff (Q29–Q36)	223
4.8	External Bibliography & Further Reading	224
4.9	Capstone Projects	225
4.9.1	Capstone 3.1 [junior] — Model the Library Domain	226
4.9.2	Capstone 3.2 [junior] — Run and Extend the Stdlib Catalog	226
4.9.3	Capstone 3.3 [mid] — Productionize the URI Linter	227
4.9.4	Capstone 3.4 [senior] — Port an RPC API to Resource Orientation	228
4.9.5	Capstone 3.5 [staff] — Design-Review Report on a Public API	229
4.10	Python Dependencies Appendix	229
4.10.1	pyyaml (≥ 6.0)	230
4.10.2	jsonschema (≥ 4.21)	230
4.11	Chapter Summary & Key Takeaways	230
4.12	Quick-Reference Card	231
4.13	Standardized Evidence Addendum	232
4.13.1	Benchmark Snapshot Template	232

4.13.2	Request Trace Template	232
4.13.3	Cost and Latency Budget Snapshot	232
4.14	Configuration Preset Template	232
4.15	CI Quality Gate Specification	232
4.16	Incident Runbook Template	233
4.17	Cross-Chapter Decision Card	233
4.18	ROI Model and Build-vs-Buy	233
4.19	Disaster Recovery Notes (RPO/RTO)	233
5	Data on the Wire: Serialization Formats & Data Contracts	237
5.1	Learning Objectives & Prerequisites	238
5.2	Deep Technical Mechanics & Architectural Concepts	240
5.2.1	JSON per RFC 8259: The Grammar You Think You Know	240
5.2.2	JSON Schema 2020-12: From Hopes to Machine-Checked Contracts	243
5.2.3	Protocol Buffers: The Wire Format, Byte by Byte	244
5.2.4	Avro and MessagePack: Two Other Bets	247
5.2.5	The Format Trade-Off Matrix	248
5.2.6	Content Negotiation: Agreeing on the Bytes	249
5.2.7	Contract Engineering: Registries, Compatibility, Consumers	252
5.3	Production-Ready Code Implementation	253
5.3.1	A Hand-Rolled Protobuf Codec (Pure Standard Library)	254
5.3.2	JSON Schema Validation Pipeline with RFC 9457 Errors	258
5.3.3	A Content-Negotiation Selector (Pure Standard Library)	261
5.3.4	A Deterministic Serialization Benchmark Harness	264
5.3.5	Streaming NDJSON Writer/Reader	267
5.4	Common Pitfalls & Anti-Patterns	269
5.5	Hands-On Production Lab & Real-World Case Study	271
5.5.1	Lab: Benchmark Three Formats and Produce a Decision Table	271
5.5.2	Case Study: The 2 ⁵³ Order-ID Corruption Incident	272
5.6	Self-Assessment Exercises & Deep-Dive Questions	272
5.7	Interview Q&A Vault	274
5.8	External Bibliography & Further Reading	280
5.9	Capstone Projects	281
5.9.1	Capstone 1: NDJSON Inventory Log Shipper [junior]	281
5.9.2	Capstone 2: Schema-Validation Middleware [mid]	282
5.9.3	Capstone 3: Hand-Rolled Protobuf Codec Extension [senior]	282
5.9.4	Capstone 4: Negotiation-Aware Cache Key Tool [senior]	283

5.9.5	Capstone 5: Format-Migration CLI for a Dataset [staff]	284
5.10	Python Dependencies Appendix	284
5.10.1	jsonschema	285
5.10.2	msgpack	285
5.10.3	orjson (optional)	285
5.11	Chapter Summary & Key Takeaways	286
5.12	Quick-Reference Card	286
5.13	Standardized Evidence Addendum	287
5.13.1	Benchmark Snapshot Template	287
5.13.2	Request Trace Template	287
5.13.3	Cost and Latency Budget Snapshot	287
5.14	Configuration Preset Template	287
5.15	CI Quality Gate Specification	288
5.16	Incident Runbook Template	288
5.17	Cross-Chapter Decision Card	288
5.18	ROI Model and Build-vs-Buy	288
5.19	Disaster Recovery Notes (RPO/RTO)	289
6	Client-Side Security: API Keys, OAuth 2.0/OIDC & JWT for Consumers	297
6.1	Learning Objectives & Prerequisites	298
6.2	Deep Technical Mechanics & Architectural Concepts	300
6.2.1	The Credential Taxonomy	300
6.2.2	Anatomy of a SigV4-Style Signature	302
6.2.3	OAuth 2.0 Decoded (RFC 6749)	303
6.2.4	OpenID Connect: The Identity Layer	307
6.2.5	JWT Internals (RFC 7519)	308
6.2.6	Secret Storage for Consumers	310
6.3	Production-Ready Code Implementation	311
6.3.1	Listing 5.1 — SigV4-Style HMAC Signer with Verifying Stub Server	311
6.3.2	Listing 5.2 — Complete PKCE Flow Against a Local Stub IdP	319
6.3.3	Listing 5.3 — Hardened JWT Verifier with Adversarial Gauntlet	326
6.3.4	Listing 5.4 — Token Cache: Single-Flight Proactive Refresh and Redaction-Safe Logging	333
6.3.5	Listing 5.5 — Educational Attack Demo: RS256→HS256 Confusion	337
6.4	Common Pitfalls & Anti-Patterns	342
6.5	Hands-On Production Lab & Real-World Case Study	343
6.5.1	Lab: PKCE End-to-End and the JWT Hardening Gauntlet	343

6.5.2	War Story: The CI Log Leak and the 41-Minute Rotation	344
6.6	Self-Assessment Exercises & Deep-Dive Questions	345
6.7	Interview Q&A Vault	346
6.8	External Bibliography & Further Reading	354
6.9	Capstone Projects	355
6.10	Python Dependencies Appendix	358
6.10.1	pyjwt	358
6.10.2	cryptography	359
6.10.3	httpx	359
6.10.4	keyring (optional)	359
6.11	Chapter Summary & Key Takeaways	360
6.12	Quick-Reference Card	361
6.13	Standardized Evidence Addendum	361
6.13.1	Benchmark Snapshot Template	361
6.13.2	Request Trace Template	361
6.13.3	Cost and Latency Budget Snapshot	362
6.14	Configuration Preset Template	362
6.15	CI Quality Gate Specification	363
6.16	Incident Runbook Template	363
6.17	Cross-Chapter Decision Card	364
6.18	ROI Model and Build-vs-Buy	364
6.19	Disaster Recovery Notes (RPO/RTO)	364
7	Building Production REST APIs with FastAPI	365
7.1	Learning Objectives & Prerequisites	366
7.2	Deep Technical Mechanics & Architectural Concepts	368
7.2.1	Design-First vs. Code-First: Who Writes the Contract?	368
7.2.2	OpenAPI 3.1 as the Backbone	369
7.2.3	ASGI and the Server Stack	370
7.2.4	Sync vs. Async Endpoints: Threadpool Semantics	371
7.2.5	The Dependency-Injection Machine	372
7.2.6	Pydantic v2: The Validation Pipeline	373
7.2.7	Configuration, Logging, and the Operational Surface	374
7.2.8	Response Engineering	375
7.3	Production-Ready Code Implementation	376
7.3.1	The Centerpiece: The Northwind Robotics Orders API	377
7.3.2	Middleware Deep-Dive: Correlation IDs and Timing	394
7.3.3	The DI Showcase: Chains, Yield-Cleanup, Caching, Overrides	398
7.3.4	The Test Suite: Offline, In-Process, Deterministic	401
7.4	Common Pitfalls & Anti-Patterns	409
7.5	Hands-On Production Lab & Real-World Case Study	411

7.5.1	Lab: Stand Up the Orders API and Interrogate Its Contract	411
7.5.2	Case Study: The Threadpool Exhaustion Incident	412
7.6	Self-Assessment Exercises & Deep-Dive Questions	413
7.7	Interview Q&A Vault	414
7.8	External Bibliography & Further Reading	421
7.9	Capstone Projects	422
7.9.1	Capstone 1: Todo API with a Full Test Suite [junior]	422
7.9.2	Capstone 2: Inventory API with a SQLite Repository [mid]	423
7.9.3	Capstone 3: Plugin-Style Dependency-Injection Architecture [senior]	424
7.9.4	Capstone 4: Async Migration of a Blocking Service [senior]	424
7.9.5	Capstone 5: OpenAPI-First Generator Round-Trip [staff]	425
7.10	Python Dependencies Appendix	426
7.10.1	fastapi	426
7.10.2	uvicorn[standard]	427
7.10.3	pydantic	427
7.10.4	pydantic-settings	427
7.10.5	httpx	427
7.10.6	pytest	428
7.11	Chapter Summary & Key Takeaways	428
7.12	Quick-Reference Card	429
7.13	Standardized Evidence Addendum	430
7.13.1	Benchmark Snapshot Template	430
7.13.2	Request Trace Template	430
7.13.3	Cost and Latency Budget Snapshot	431
7.14	Configuration Preset Template	431
7.15	CI Quality Gate Specification	431
7.16	Incident Runbook Template	432
7.17	Cross-Chapter Decision Card	432
7.18	ROI Model and Build-vs-Buy	432
7.19	Disaster Recovery Notes (RPO/RTO)	432
8	Errors, Validation & Problem Details (RFC 9457)	435
8.1	Learning Objectives & Prerequisites	436
8.2	Deep Technical Mechanics & Architectural Concepts	437
8.2.1	A Taxonomy of Failure	437
8.2.2	RFC 9457: Problem Details for HTTP APIs	439
8.2.3	Status-Code Precision for Error Cases	441
8.2.4	Validation Engineering with Pydantic v2	444

8.2.5	Error Envelope Architecture	446
8.2.6	Operational Concerns	447
8.3	Production-Ready Code Implementation	447
8.3.1	A Reusable Problem-Details Library	448
8.3.2	Strict Validation Showcase	454
8.3.3	Redaction Middleware and Correlation IDs	458
8.3.4	The Service, Wired	461
8.3.5	Contract Tests: Every Error Path Is Schema-Valid	464
8.3.6	A Machine-Readable Error-Code Registry	469
8.4	Common Pitfalls & Anti-Patterns	472
8.5	Hands-On Production Lab & Real-World Case Study	474
8.5.1	Lab: Retrofitting a Messy Service to Uniform RFC 9457	474
8.5.2	War Story: The PII Leak That Hid in detail	474
8.6	Self-Assessment Exercises & Deep-Dive Questions	475
8.7	Interview Q&A Vault	476
8.8	External Bibliography & Further Reading	482
8.9	Capstone Projects	483
8.9.1	Project 1 [junior] — Problem-Details Retrofit	483
8.9.2	Project 2 [mid] — Error-Code Registry & Documentation Generator	484
8.9.3	Project 3 [mid] — Redaction Library with Fuzz Corpus	484
8.9.4	Project 4 [senior] — Error-Observability Dashboard Specification	485
8.9.5	Project 5 [staff] — Chaos Error-Injection Test Harness	486
8.10	Python Dependencies Appendix	487
8.10.1	fastapi (≥ 0.110)	487
8.10.2	pydantic (≥ 2.7)	487
8.10.3	jsonschema (≥ 4.21)	488
8.10.4	httpx (≥ 0.27)	488
8.10.5	pytest (≥ 8)	488
8.11	Chapter Summary & Key Takeaways	488
8.12	Quick-Reference Card	489
8.13	Standardized Evidence Addendum	490
8.13.1	Benchmark Snapshot Template	490
8.13.2	Request Trace Template	490
8.13.3	Cost and Latency Budget Snapshot	491
8.14	Configuration Preset Template	491
8.15	CI Quality Gate Specification	492
8.16	Incident Runbook Template	492
8.17	Cross-Chapter Decision Card	492
8.18	ROI Model and Build-vs-Buy	493
8.19	Disaster Recovery Notes (RPO/RTO)	493

9	Pagination, Filtering, Sorting & Field Selection	495
	Executive Summary	495
9.1	Learning Objectives & Prerequisites	496
9.1.1	Learning Objectives	496
9.1.2	Prerequisites	497
9.2	Deep Technical Mechanics & Architectural Concepts	498
9.2.1	The Four Dimensions of Collection Design	498
9.2.2	Offset Pagination and Its True Cost	498
9.2.3	The Concurrency Anomaly, Formalized	499
9.2.4	Cursor and Keyset (Seek) Pagination	500
9.2.5	Measured Performance: Offset vs. Keyset	502
9.2.6	Choosing a Strategy: A Decision Tree	502
9.2.7	Cursor Token Design	503
9.2.8	Hypermedia Controls: RFC 8288 Link Headers	505
9.2.9	Page Metadata: <code>has_more</code> versus <code>total_count</code>	505
9.2.10	Filtering: A Safe Query-Parameter Grammar	506
9.2.11	Sorting: Multi-Key Orderings Done Safely	508
9.2.12	Field Selection and Expansion	509
9.2.13	Database Interaction: Indexes, EXPLAIN, Covering	510
9.3	Production-Ready Code Implementation	510
9.3.1	Listing 1: Deterministic Dataset Generator	511
9.3.2	Listing 2: Three Pagination Strategies in One FastAPI Service	514
9.3.3	Listing 3: Allow-Listed Filter/Sort Compiler	523
9.3.4	Listing 4: Benchmark Harness	530
9.3.5	Listing 5: Concurrency-Anomaly Demonstration	533
9.4	Common Pitfalls & Anti-Patterns	537
9.4.1	Offset at Depth	537
9.4.2	Unsigned, Guessable Cursors	537
9.4.3	<code>total_count</code> on Every Page	538
9.4.4	Missing Tie-Breaker	538
9.4.5	Unbounded Page Sizes	538
9.4.6	Filter Parameters Interpolated into SQL	538
9.4.7	Exposing Internal Column Names	538
9.5	Hands-On Production Lab & Real-World Case Study	539
9.5.1	Lab: Three Strategies, One Dataset, Zero Excuses	539
9.5.2	Case Study: The 2 a.m. Reporting API	541
9.6	Self-Assessment Exercises & Deep-Dive Questions	542
9.7	Interview Q&A Vault	543
9.8	External Bibliography & Further Reading	550
9.9	Capstone Projects	551
9.9.1	Capstone 1: Cursor-Paginated Ledger API [junior]	551
9.9.2	Capstone 2: Filter Compiler with Property Tests [mid]	552

9.9.3	Capstone 3: Pagination Benchmark Report [mid]	553
9.9.4	Capstone 4: Infinite-Scroll Client over Signed Cursors [senior]	553
9.9.5	Capstone 5: Multi-Tenant Keyset Design [staff]	554
9.10	Python Dependencies Appendix	555
9.10.1	fastapi (fastapi>=0.110)	556
9.10.2	pydantic (pydantic>=2.7)	556
9.10.3	httpx (httpx>=0.27)	556
9.10.4	pytest (pytest>=8)	557
9.11	Chapter Summary & Key Takeaways	557
9.12	Quick-Reference Card	558
9.13	Standardized Evidence Addendum	559
9.13.1	Benchmark Snapshot Template	559
9.13.2	Request Trace Template	559
9.13.3	Cost and Latency Budget Snapshot	559
9.14	Configuration Preset Template	559
9.15	CI Quality Gate Specification	559
9.16	Incident Runbook Template	560
9.17	Cross-Chapter Decision Card	560
9.18	ROI Model and Build-vs-Buy	560
9.19	Disaster Recovery Notes (RPO/RTO)	560
10	API Versioning & Evolution Strategies	565
10.1	Learning Objectives & Prerequisites	566
10.2	Deep Technical Mechanics & Architectural Concepts	569
10.2.1	A Taxonomy of Change	569
10.2.2	The Robustness Principle and Its Limits	570
10.2.3	Versioning Strategies: A Rigorous Comparison	570
10.2.4	Evolution Mechanics: Expand-and-Contract and Dual-Serving	572
10.2.5	Deprecation Policy Engineering	574
10.2.6	Schema Evolution Rules	575
10.2.7	Version Policy at Scale	576
10.3	Production-Ready Code Implementation	577
10.3.1	Listing 9.1 — Dual-Serving Catalog API (v1 + v2 over One Core)	577
10.3.2	Listing 9.2 — OpenAPI Breaking-Change Gate (CI in a Box)	581
10.3.3	Listing 9.3 — Media-Type Version Negotiator	590
10.3.4	Listing 9.4 — Consumer-Tolerance Test Suite	594
10.3.5	Listing 9.5 — Retirement-Readiness Report Generator	596
10.4	Common Pitfalls & Anti-Patterns	600
10.5	Hands-On Production Lab & Real-World Case Study	602

10.6 Self-Assessment Exercises & Deep-Dive Questions	603
10.7 Interview Q&A Vault	604
10.8 External Bibliography & Further Reading	609
10.9 Capstone Projects	610
10.10 Python Dependencies Appendix	613
10.10.1 fastapi (≥ 0.110)	614
10.10.2 pydantic (≥ 2.7)	614
10.10.3 pyyaml (≥ 6.0)	614
10.10.4 jsonschema (≥ 4.21)	614
10.10.5 httpx (≥ 0.27)	614
10.10.6 pytest (≥ 8)	615
10.11 Chapter Summary & Key Takeaways	615
10.12 Quick-Reference Card	616
10.13 Standardized Evidence Addendum	616
10.13.1 Benchmark Snapshot Template	616
10.13.2 Request Trace Template	616
10.13.3 Cost and Latency Budget Snapshot	617
10.14 Configuration Preset Template	617
10.15 CI Quality Gate Specification	617
10.16 Incident Runbook Template	617
10.17 Cross-Chapter Decision Card	618
10.18 ROI Model and Build-vs-Buy	618
10.19 Disaster Recovery Notes (RPO/RTO)	618
11 Idempotency, Caching & Concurrency Control	623
11.1 Learning Objectives & Prerequisites	624
11.2 Deep Technical Mechanics & Architectural Concepts	625
11.2.1 Idempotency: A Formal Definition	625
11.2.2 The Idempotency-Key Protocol	627
11.2.3 The Exactly-Once Illusion	629
11.2.4 HTTP Caching Machinery per RFC 9110/9111	629
11.2.5 Concurrency Control: The Lost-Update Problem	631
11.2.6 Retries Meet Idempotency: The Money-Transfer Argument	633
11.3 Production-Ready Code Implementation	634
11.3.1 Listing 10.1 — SQLite-Backed Idempotency-Key Middleware	634
11.3.2 Listing 10.2 — Conditional Requests: Strong ETags and the 304 Flow	649
11.3.3 Listing 10.3 — Optimistic Locking with If-Match on Inventory	655
11.3.4 Listing 10.4 — Double-Submit Storm Harness	663
11.3.5 Listing 10.5 — Cache-Policy Simulator	667

11.4	Common Pitfalls & Anti-Patterns	678
11.4.1	Storing the Response Before Committing the Mutation	678
11.4.2	Reusing Idempotency Keys Across Endpoints	679
11.4.3	Hashing Only the Body, Not the Route	679
11.4.4	Retrying POST Without Keys	679
11.4.5	Weak ETags on Byte-Sensitive Resources	679
11.4.6	The no-store / no-cache Confusion	679
11.4.7	Client-Generated ETags	680
11.4.8	Distributed Locks Without Fencing Tokens	680
11.5	Hands-On Production Lab & Real-World Case Study	680
11.5.1	Lab: An Idempotent Payments API That Survives Retry Storms	680
11.5.2	War Story: The Double-Charged Customers	681
11.6	Self-Assessment Exercises & Deep-Dive Questions	681
11.7	Interview Q&A Vault	682
11.8	External Bibliography & Further Reading	687
11.9	Capstone Projects	688
11.10	Python Dependencies Appendix	692
11.10.1	fastapi (≥ 0.110)	692
11.10.2	pydantic (≥ 2.7)	692
11.10.3	httpx (≥ 0.27)	692
11.10.4	pytest (≥ 8)	693
11.11	Chapter Summary & Key Takeaways	693
11.12	Quick-Reference Card	694
11.13	Standardized Evidence Addendum	694
11.13.1	Benchmark Snapshot Template	694
11.13.2	Request Trace Template	694
11.13.3	Cost and Latency Budget Snapshot	695
11.14	Configuration Preset Template	695
11.15	CI Quality Gate Specification	695
11.16	Incident Runbook Template	696
11.17	Cross-Chapter Decision Card	696
11.18	ROI Model and Build-vs-Buy	696
11.19	Disaster Recovery Notes (RPO/RTO)	697
12	Asynchronous & Event-Driven APIs: Webhooks, SSE & WebSockets	699
12.1	Learning Objectives & Prerequisites	701
12.2	Deep Technical Mechanics & Architectural Concepts	703
12.2.1	The Interaction-Pattern Taxonomy	703
12.2.2	Webhooks: The Reverse-API Pattern	705
12.2.3	Server-Sent Events: <code>text/event-stream</code> on the Wire	709
12.2.4	WebSockets: RFC 6455 Precisely	711

12.2.5	AsyncAPI and the Governance of Event Contracts . .	714
12.3	Production-Ready Code Implementation	715
12.3.1	Webhooks: Signed Receiver and Bounded-Retry De- livery Worker	715
12.3.2	Server-Sent Events: Producer, Parser, and Resume .	726
12.3.3	WebSockets: Handshake Math and an Echo/Broadcast Demo	731
12.3.4	The Event Contract in AsyncAPI	734
12.3.5	Contrast: Long Polling in Twelve Lines	736
12.4	Common Pitfalls & Anti-Patterns	738
12.4.1	Unsigned webhooks	738
12.4.2	Retrying permanent failures forever	738
12.4.3	Assuming ordering	738
12.4.4	SSE through buffering proxies	739
12.4.5	WebSocket for plain CRUD	739
12.4.6	Non-idempotent receivers	739
12.4.7	Leaking event payloads in logs	739
12.4.8	Sleep-based async tests	739
12.5	Hands-On Production Lab & Real-World Case Study	740
12.5.1	Lab: The Northwind Robotics Real-Time Notifica- tions Service	740
12.5.2	War Story 1: The Retry Loop That DDoSed a Customer	741
12.5.3	War Story 2: The Unsigned-Webhook Spoofing Incident	741
12.6	Self-Assessment Exercises & Deep-Dive Questions	742
12.7	Interview Q&A Vault	743
12.8	External Bibliography & Further Reading	748
12.9	Capstone Projects	750
12.9.1	Capstone 11.1 [junior] — Signed Webhook Receiver for Robot Alerts	750
12.9.2	Capstone 11.2 [mid] — Webhook Delivery Service with DLQ and Replay	750
12.9.3	Capstone 11.3 [mid] — Resumable SSE Fleet Dash- board Backend	751
12.9.4	Capstone 11.4 [senior] — Bidirectional Robot Control Channel over WebSockets	752
12.9.5	Capstone 11.5 [staff] — Event-Contract Governance Platform	753
12.10	Python Dependencies Appendix	754
12.10.1	fastapi	754
12.10.2	websockets	754
12.10.3	httpx	755
12.10.4	pydantic	755
12.10.5	pytest	755

12.11	Chapter Summary & Key Takeaways	755
12.12	Quick-Reference Card	756
12.13	Standardized Evidence Addendum	757
12.13.1	Benchmark Snapshot Template	757
12.13.2	Request Trace Template	757
12.13.3	Cost and Latency Budget Snapshot	758
12.14	Configuration Preset Template	758
12.15	CI Quality Gate Specification	759
12.16	Incident Runbook Template	759
12.17	Cross-Chapter Decision Card	759
12.18	ROI Model and Build-vs-Buy	760
12.19	Disaster Recovery Notes (RPO/RTO)	760
13	GraphQL: Schema-Driven APIs	761
	Executive Summary	761
13.1	Learning Objectives & Prerequisites	762
13.1.1	Learning Objectives	762
13.1.2	Prerequisites	763
13.2	Deep Technical Mechanics & Architectural Concepts	764
13.2.1	What GraphQL Actually Is	764
13.2.2	The Type System	764
13.2.3	Schema Definition Language and Introspection	766
13.2.4	Operations: Queries, Mutations, Subscriptions	767
13.2.5	The Execution Pipeline and the Wire Shape	768
13.2.6	Resolver Architecture and the N+1 Problem	769
13.2.7	Pagination: The Relay Connection Spec	770
13.2.8	The Error Model	771
13.2.9	API Cost Control	771
13.2.10	Security: The GraphQL-Specific Attack Surface	772
13.2.11	GraphQL vs. REST vs. gRPC: An Honest Matrix	773
13.2.12	Federation Primer (Scope Note)	774
13.3	Production-Ready Code Implementation	774
13.3.1	Listing 1: Instrumented Repository	775
13.3.2	Listing 2: The Strawberry Schema	778
13.3.3	Listing 3: The N+1 Demonstration	787
13.3.4	Listing 4: The Cost-Control Layer	789
13.3.5	Listing 5: The FastAPI Host	798
13.3.6	Listing 6: The httpx Client	800
13.3.7	Listing 7: The Test Suite	805
13.4	Common Pitfalls & Anti-Patterns	815
13.4.1	N+1 by Construction	815
13.4.2	Open Introspection in Production	815
13.4.3	Unlimited Depth and Complexity	816

13.4.4	Treating GraphQL Errors as HTTP 500 Only	816
13.4.5	Schema Equals Database Schema	816
13.4.6	God-Queries from Your Own Clients	816
13.4.7	DataLoader Shared Across Requests	816
13.4.8	Bypassing the HTTP Cache Without a Plan	817
13.5	Hands-On Production Lab & Real-World Case Study	817
13.5.1	Lab: Build, Break, and Harden the Catalog API	817
13.5.2	Case Study: The Introspection-Enabled Enumeration Incident	818
13.6	Self-Assessment Exercises & Deep-Dive Questions	819
13.6.1	Recall and Comprehension	819
13.6.2	Application	820
13.6.3	Deep-Dive Questions	820
13.7	Interview Q&A Vault	821
13.8	External Bibliography & Further Reading	826
13.9	Capstone Projects	827
13.9.1	Capstone 1: Catalog Schema from Scratch [junior]	827
13.9.2	Capstone 2: N+1 Hunter [mid]	828
13.9.3	Capstone 3: Cost-Control Gateway [mid]	828
13.9.4	Capstone 4: APQ Production Gateway [senior]	829
13.9.5	Capstone 5: Federated Robotics Graph [staff]	830
13.10	Python Dependencies Appendix	831
13.10.1	Pinned Requirements	831
13.10.2	strawberry-graphql[fastapi]	831
13.10.3	fastapi	831
13.10.4	httpx	832
13.10.5	pytest	832
13.11	Chapter Summary & Key Takeaways	832
13.12	Quick-Reference Card	833
13.12.1	Type System at a Glance	833
13.12.2	Wire Shapes	834
13.12.3	Cost-Control Checklist	834
13.13	Standardized Evidence Addendum	835
13.13.1	Benchmark Snapshot Template	835
13.13.2	Request Trace Template	835
13.13.3	Cost and Latency Budget Snapshot	835
13.14	Configuration Preset Template	835
13.15	CI Quality Gate Specification	836
13.16	Incident Runbook Template	836
13.17	Cross-Chapter Decision Card	836
13.18	ROI Model and Build-vs-Buy	836
13.19	Disaster Recovery Notes (RPO/RTO)	837

14 gRPC & High-Performance RPC	839
14.1 Learning Objectives & Prerequisites	840
14.2 Deep Technical Mechanics & Architectural Concepts	842
14.2.1 The RPC Conceptual Model: Stubs, Skeletons, Marshalling	842
14.2.2 Why REST+JSON Loses on Hot Internal Paths	843
14.2.3 HTTP/2 as the Transport	843
14.2.4 Protocol Buffers as the Interface Definition Language	845
14.2.5 The Four RPC Modes	848
14.2.6 Deadlines, Cancellation, and Metadata	848
14.2.7 Interceptors: The Cross-Cutting Layer	849
14.2.8 The Error Model	850
14.2.9 Operational gRPC: Load Balancing, Connections, Tooling	852
14.3 Production-Ready Code Implementation	853
14.3.1 Listing 1 — Interface Definition: <code>telemetry.proto</code> .	853
14.3.2 Listing 2 — Server: All Four Modes with Deadline and Cancellation Discipline	856
14.3.3 Listing 3 — Error-Mapping Middleware with Rich Details	863
14.3.4 Listing 4 — Client: Metadata, Deadline Budgets, Interceptor, Retry Policy	865
14.3.5 Listing 5 — Bidirectional Streaming with Flow-Control-Aware Production	870
14.3.6 Listing 6 — Offline Test Suite: Deadline, Cancellation, and Stream Error Paths	873
14.4 Common Pitfalls & Anti-Patterns	877
14.5 Hands-On Production Lab & Real-World Case Study	879
14.5.1 Lab: The Telemetry Service End to End	879
14.5.2 War Story: The L4 Hotspot That Pinned All Traffic to One Pod	880
14.6 Self-Assessment Exercises & Deep-Dive Questions	881
14.7 Interview Q&A Vault	882
14.8 External Bibliography & Further Reading	889
14.9 Capstone Projects	890
14.9.1 Capstone 1 [junior] — Unary Status Service with Deadline Discipline	890
14.9.2 Capstone 2 [mid] — Server-Streaming Telemetry Monitor	891
14.9.3 Capstone 3 [mid] — Bulk Ingestion with Rich Validation Errors	892
14.9.4 Capstone 4 [senior] — Bidirectional Control Plane with a Full Interceptor Stack	892

14.9.5	Capstone 5 [staff] — Edge Gateway: Transcoding plus Load-Balancing Postmortem Lab	893
14.10	Python Dependencies Appendix	894
14.10.1	grpcio	895
14.10.2	grpcio-tools	895
14.10.3	protobuf	895
14.10.4	pytest	896
14.11	Chapter Summary & Key Takeaways	896
14.12	Quick-Reference Card	897
14.13	Standardized Evidence Addendum	899
14.13.1	Benchmark Snapshot Template	899
14.13.2	Request Trace Template	899
14.13.3	Cost and Latency Budget Snapshot	899
14.14	Configuration Preset Template	899
14.15	CI Quality Gate Specification	900
14.16	Incident Runbook Template	900
14.17	Cross-Chapter Decision Card	900
14.18	ROI Model and Build-vs-Buy	900
14.19	Disaster Recovery Notes (RPO/RTO)	901
15	Rate Limiting, Throttling & Quota Systems	903
15.1	Learning Objectives & Prerequisites	904
15.2	Deep Technical Mechanics & Architectural Concepts	906
15.2.1	Why Limit at All: Four Distinct Failures, Five Dis- tinct Tools	906
15.2.2	Fixed Window Counter — and Its Provable 2x Flaw	907
15.2.3	Sliding Window Log: Exactness at $O(N)$ Memory . .	908
15.2.4	Sliding Window Counter: The Weighted Approximation	909
15.2.5	Token Bucket: Burst Capacity as a Conserved Quantity	909
15.2.6	Leaky Bucket: Meter versus Queue	910
15.2.7	GCRA: One Timestamp to Rule Them All	911
15.2.8	Choosing an Algorithm	912
15.2.9	Distribution: The Race, the Cell, and the Failure Policy	912
15.2.10	Client-Facing Semantics: Making Limits Legible . . .	914
15.2.11	Multi-Dimensional Limiting	916
15.3	Production-Ready Code Implementation	917
15.3.1	Listing 14.1 — Five Algorithms, One Contract	917
15.3.2	Listing 14.2 — The Theorems, as Executable Tests .	923
15.3.3	Listing 14.3 — Simulation Harness: One Timeline, Five Verdicts	929
15.3.4	Listing 14.4 — Distributed Sliding Window: Redis + Lua	932

15.3.5 Listing 14.6 — FastAPI Middleware: Headers Are the Product	940
15.3.6 Listing 14.8 — Cost-Based Limiting: Budgeting Work, Not Requests	949
15.4 Common Pitfalls & Anti-Patterns	954
15.5 Hands-On Production Lab & Real-World Case Study	955
15.5.1 Case Study: The Login Endpoint the Per-IP Limit Could Not Save	956
15.6 Self-Assessment Exercises & Deep-Dive Questions	957
15.7 Interview Q&A Vault	958
15.8 External Bibliography & Further Reading	965
15.9 Capstone Projects	967
15.10 Python Dependencies Appendix	970
15.10.1 fakeredis (≥ 2.23)	970
15.10.2 fastapi (≥ 0.110)	971
15.10.3 httpx (≥ 0.27)	971
15.10.4 pytest (≥ 8)	971
15.11 Chapter Summary & Key Takeaways	971
15.12 Quick-Reference Card	972
15.13 Standardized Evidence Addendum	973
15.13.1 Benchmark Snapshot Template	973
15.13.2 Request Trace Template	973
15.13.3 Cost and Latency Budget Snapshot	973
15.14 Configuration Preset Template	973
15.15 CI Quality Gate Specification	974
15.16 Incident Runbook Template	974
15.17 Cross-Chapter Decision Card	975
15.18 ROI Model and Build-vs-Buy	975
15.19 Disaster Recovery Notes (RPO/RTO)	975
16 Microservices & Resilient Inter-Service Communication	979
16.1 Learning Objectives & Prerequisites	980
16.2 Deep Technical Mechanics & Architectural Concepts	982
16.2.1 A Taxonomy of Inter-Service Communication	982
16.2.2 Coupling Dimensions and the Fallacies of Distributed Computing	983
16.2.3 Timeouts Everywhere, and How to Budget Them . .	985
16.2.4 The Mathematics of Retry Amplification	985
16.2.5 The Circuit Breaker, Specified Formally	987
16.2.6 Bulkheads, Hedged Requests, and Backpressure . . .	988
16.2.7 Anatomy of a Cascading Failure	989
16.2.8 Service Discovery and Load Balancing	990
16.2.9 Service Mesh: Data Plane versus Control Plane . . .	991

16.2.10 Distributed Transactions without 2PC	992
16.3 Production-Ready Code Implementation	994
16.3.1 Circuit Breaker: Full State Machine, Sync and Async	994
16.3.2 Retry-Budget Client: Token Bucket, Full Jitter, Dead- lines	1007
16.3.3 Three-Service Chaos Drill: Catalog → Inventory → Warehouse	1017
16.3.4 The Transactional Outbox with Relay and Idempotent Consumer	1027
16.3.5 Saga Orchestrator with Compensating Actions	1035
16.4 Common Pitfalls & Anti-Patterns	1045
16.5 Hands-On Production Lab & Real-World Case Study	1047
16.6 Self-Assessment Exercises & Deep-Dive Questions	1049
16.7 Interview Q&A Vault	1050
16.8 External Bibliography & Further Reading	1057
16.9 Capstone Projects	1058
16.10 Python Dependencies Appendix	1061
16.10.1 fastapi (≥ 0.110)	1061
16.10.2 httpx (≥ 0.27)	1061
16.10.3 pydantic (≥ 2.7)	1062
16.10.4 pytest (≥ 8)	1062
16.10.5 tenacity (≥ 8.3)	1062
16.11 Chapter Summary & Key Takeaways	1062
16.12 Quick-Reference Card	1064
16.13 Standardized Evidence Addendum	1065
16.13.1 Benchmark Snapshot Template	1065
16.13.2 Request Trace Template	1065
16.13.3 Cost and Latency Budget Snapshot	1066
16.14 Configuration Preset Template	1066
16.15 CI Quality Gate Specification	1066
16.16 Incident Runbook Template	1066
16.17 Cross-Chapter Decision Card	1067
16.18 ROI Model and Build-vs-Buy	1067
16.19 Disaster Recovery Notes (RPO/RTO)	1067
17 API Security Engineering (OWASP API Security Top 10)	1069
17.1 Learning Objectives & Prerequisites	1070
17.2 Deep Technical Mechanics & Architectural Concepts	1071
17.2.1 The Trust Model: Where Requests Cross Boundaries	1071
17.2.2 The OWASP API Security Top 10 (2023) at a Glance	1072
17.2.3 API1:2023 — Broken Object Level Authorization (BOLA/IDOR)	1072
17.2.4 API2:2023 — Broken Authentication	1073

17.2.5	API3:2023 — Broken Object Property Level Authorization	1073
17.2.6	API4:2023 — Unrestricted Resource Consumption	1074
17.2.7	API5:2023 — Broken Function Level Authorization	1075
17.2.8	API6:2023 — Unrestricted Access to Sensitive Business Flows	1075
17.2.9	API7:2023 — Server-Side Request Forgery (SSRF)	1076
17.2.10	API8:2023 — Security Misconfiguration	1076
17.2.11	API9:2023 — Improper Inventory Management	1077
17.2.12	API10:2023 — Unsafe Consumption of APIs	1077
17.2.13	Server-Side OAuth 2.0 & OIDC: The Token Validation Pipeline	1078
17.2.14	mTLS for Service Identity	1080
17.2.15	Threat Modeling APIs	1080
17.2.16	Injection and Input Attacks	1082
17.2.17	SSRF Defense-in-Depth	1082
17.2.18	Response Security	1083
17.2.19	Secrets Management	1084
17.3	Production-Ready Code Implementation	1084
17.3.1	Listing 16.1 — Simulated Network (<code>fake_network.py</code>)	1085
17.3.2	Listing 16.2 — Local JWKS Fixture (<code>jwks_fixture.py</code>)	1087
17.3.3	Listing 16.3 — The Deliberately Vulnerable API (<code>vulnerable_app.py</code>)	1090
17.3.4	Listing 16.4 — The Hardened API (<code>hardened_app.py</code>)	1097
17.3.5	Listing 16.5 — Token Validation Middleware (<code>token_validation.py</code>)	1106
17.3.6	Listing 16.6 — Authorization Guards (<code>authz.py</code>)	1110
17.3.7	Listing 16.7 — SSRF-Safe Outbound Fetcher (<code>ssrf_fetcher.py</code>)	1114
17.3.8	Listing 16.8 — STRIDE Worksheet Generator (<code>stride_worksheet.py</code>)	1118
17.3.9	Listing 16.9 — Exploit Suite Wiring (<code>confest.py</code>)	1122
17.3.10	Listing 16.10 — The Exploit Suite (<code>test_exploit_suite.py</code>)	1123
17.3.11	Listings 16.11–16.13 — Library Test Suites	1130
17.4	Common Pitfalls & Anti-Patterns	1138
17.5	Hands-On Production Lab & Real-World Case Study	1139
17.5.1	Lab: The Exploit-Then-Fix Gauntlet	1139
17.5.2	War Story: The BOLA Breach That Walked the Primary Keys	1140
17.6	Self-Assessment Exercises & Deep-Dive Questions	1141
17.7	Interview Q&A Vault	1142
17.8	External Bibliography & Further Reading	1148
17.9	Capstone Projects	1149
17.10	Python Dependencies Appendix	1152
17.10.1	fastapi	1152
17.10.2	pyjwt[crypto]	1153

17.10.3 httpx	1153
17.10.4 pytest	1153
17.10.5 pydantic	1154
17.11 Chapter Summary & Key Takeaways	1154
17.12 Quick-Reference Card	1155
17.13 Standardized Evidence Addendum	1155
17.13.1 Benchmark Snapshot Template	1155
17.13.2 Request Trace Template	1155
17.13.3 Cost and Latency Budget Snapshot	1155
17.14 Configuration Preset Template	1155
17.15 CI Quality Gate Specification	1156
17.16 Incident Runbook Template	1156
17.17 Cross-Chapter Decision Card	1156
17.18 ROI Model and Build-vs-Buy	1156
17.19 Disaster Recovery Notes (RPO/RTO)	1157

18 Deployment & Infrastructure: Containers, Gateways & Edge 1163

18.1 Learning Objectives & Prerequisites	1164
18.2 Deep Technical Mechanics & Architectural Concepts	1166
18.2.1 The Immutable Artifact: What a Container Image Actually Is	1166
18.2.2 Process Discipline: PID 1, Signals, and the Four-Act Shutdown	1168
18.2.3 The API Gateway: Policy Enforcement Point for North-South Traffic	1169
18.2.4 Kubernetes for APIs: The Semantics That Bite	1170
18.2.5 Progressive Delivery: Migrating Traffic, Not Versions	1172
18.2.6 Edge & TLS: Where Bytes Get Terminated	1173
18.3 Production-Ready Code Implementation	1175
18.3.1 Listing 17.1 — Production Multi-Stage Dockerfile	1175
18.3.2 Listing 17.2 — Kubernetes Manifest Set (Reviewed, Not Applied)	1177
18.3.3 Listing 17.3 — The Server Under Test (<code>orders_server.py</code>)	1182
18.3.4 Listing 17.4 — The Graceful-Shutdown Proof (<code>shutdown_demo.py</code>)	1184
18.3.5 Listing 17.5 — Probe-Semantics Simulator (<code>probe_simulator.py</code>)	1189
18.3.6 Listing 17.6 — Canary Analyzer (<code>canary_analysis.py</code>)	1197
18.4 Common Pitfalls & Anti-Patterns	1204
18.5 Hands-On Production Lab & Real-World Case Study	1206
18.5.1 Lab: Signals, Probes, and a Manifest Design Review	1206
18.5.2 Case Study: The Rolling Deploy That Dropped In- Flight Requests	1206
18.6 Self-Assessment Exercises & Deep-Dive Questions	1207

18.7 Interview Q&A Vault	1209
18.8 External Bibliography & Further Reading	1217
18.9 Capstone Projects	1218
18.10 Python Dependencies Appendix	1222
18.10.1 fastapi (≥ 0.110)	1222
18.10.2 uvicorn (≥ 0.29)	1222
18.10.3 httpx (≥ 0.27)	1223
18.10.4 pytest (≥ 8)	1223
18.10.5 pyyaml (≥ 6.0)	1223
18.11 Chapter Summary & Key Takeaways	1224
18.12 Quick-Reference Card	1225
18.13 Standardized Evidence Addendum	1225
18.13.1 Benchmark Snapshot Template	1225
18.13.2 Request Trace Template	1225
18.13.3 Cost and Latency Budget Snapshot	1226
18.14 Configuration Preset Template	1226
18.15 CI Quality Gate Specification	1226
18.16 Incident Runbook Template	1226
18.17 Cross-Chapter Decision Card	1227
18.18 ROI Model and Build-vs-Buy	1227
18.19 Disaster Recovery Notes (RPO/RTO)	1227
19 Observability & SRE for APIs	1233
19.1 Learning Objectives & Prerequisites	1234
19.2 Deep Technical Mechanics & Architectural Concepts	1236
19.2.1 The Three Pillars, Applied to APIs	1236
19.2.2 OpenTelemetry Architecture: API, SDK, Exporter, Collector	1237
19.2.3 The W3C Trace Context: <code>traceparent</code> , Byte by Byte	1238
19.2.4 Span Kinds and Semantic Conventions for HTTP	1239
19.2.5 Sampling: Head, Tail, and the Errors You Cannot Afford to Lose	1240
19.2.6 Metrics Engineering: Histograms, Percentiles, Cardi- nality	1241
19.2.7 SLO Engineering: SLIs, Error Budgets, Burn Rates	1242
19.2.8 Operational Practice: From Telemetry to Response	1245
19.3 Production-Ready Code Implementation	1246
19.3.1 Listing 18.1 — The W3C Trace Context, by Hand	1246
19.3.2 Listing 18.2 — The Header Mechanics, Proven	1251
19.3.3 Listing 18.3 — An Instrumented FastAPI Service, Offline	1254
19.3.4 Listing 18.4 — Histograms: Percentiles That Survive the Tail	1265

19.3.5 Listing 18.5 — The Burn-Rate Simulator	1271
19.3.6 Listing 18.6 — Runbook and Postmortem Generator	1277
19.4 Common Pitfalls & Anti-Patterns	1284
19.5 Hands-On Production Lab & Real-World Case Study	1286
19.5.1 Case Study: The Invisible Retry Storm	1287
19.6 Self-Assessment Exercises & Deep-Dive Questions	1287
19.7 Interview Q&A Vault	1289
19.8 External Bibliography & Further Reading	1295
19.9 Capstone Projects	1296
19.10 Python Dependencies Appendix	1299
19.10.1 opentelemetry-sdk (≥ 1.25)	1299
19.10.2 opentelemetry-instrumentation-fastapi ($\geq 0.46b0$)	1300
19.10.3 fastapi (≥ 0.110)	1300
19.10.4 pytest (≥ 8)	1300
19.11 Chapter Summary & Key Takeaways	1301
19.12 Quick-Reference Card	1302
19.13 Standardized Evidence Addendum	1303
19.13.1 Benchmark Snapshot Template	1303
19.13.2 Request Trace Template	1303
19.13.3 Cost and Latency Budget Snapshot	1303
19.14 Configuration Preset Template	1303
19.15 CI Quality Gate Specification	1303
19.16 Incident Runbook Template	1304
19.17 Cross-Chapter Decision Card	1304
19.18 ROI Model and Build-vs-Buy	1304
19.19 Disaster Recovery Notes (RPO/RTO)	1304
20 Testing & Quality Engineering for APIs	1307
20.1 Learning Objectives & Prerequisites	1308
20.2 Deep Technical Mechanics & Architectural Concepts	1310
20.2.1 The API Test Pyramid	1310
20.2.2 Consumer-Driven Contract Testing	1312
20.2.3 Property-Based Testing and API Fuzzing	1313
20.2.4 Test Doubles, Record–Replay, and Virtual Clocks	1314
20.2.5 Load and Performance Testing	1315
20.2.6 Chaos Engineering Basics	1317
20.2.7 CI Quality Gates as Code	1318
20.3 Production-Ready Code Implementation	1319
20.3.1 Listing 19.1 — The App Under Test with Deliberate-Defect Knobs	1319
20.3.2 Listing 19.2 — A Mini Pact: Recorder, Pact File, Provider Verifier	1323

20.3.3	Listing 19.3 — Contract Tests, Including the Breaking Change	1331
20.3.4	Listing 19.4 — Property-Based Suite with Hypothesis	1333
20.3.5	Listing 19.5 — A Stdlib OpenAPI Fuzzer with Shrinking	1337
20.3.6	Listing 19.6 — An asyncio Load Harness with Knee-Point Sweep	1344
20.3.7	Listing 19.7 — Fault-Injection Middleware and Resilient Client	1351
20.3.8	Listing 19.8 — The Quality Gate	1358
20.3.9	Listing 19.9 — The Pipeline as Code	1361
20.4	Common Pitfalls & Anti-Patterns	1362
20.5	Hands-On Production Lab & Real-World Case Study	1364
20.5.1	Lab: Run the Gate, Then Break the API	1364
20.5.2	War Story: The Deploy That Passed Everything Except the Contract	1365
20.6	Self-Assessment Exercises & Deep-Dive Questions	1365
20.7	Interview Q&A Vault	1366
20.8	External Bibliography & Further Reading	1372
20.9	Capstone Projects	1373
20.10	Python Dependencies Appendix	1376
20.10.1	pytest	1377
20.10.2	hypothesis	1377
20.10.3	fastapi	1377
20.10.4	httpx	1377
20.10.5	pydantic	1378
20.10.6	jsonschema	1378
20.11	Chapter Summary & Key Takeaways	1378
20.12	Quick-Reference Card	1379
20.13	Standardized Evidence Addendum	1380
20.13.1	Benchmark Snapshot Template	1380
20.13.2	Request Trace Template	1380
20.13.3	Cost and Latency Budget Snapshot	1380
20.14	Configuration Preset Template	1380
20.15	CI Quality Gate Specification	1381
20.16	Incident Runbook Template	1381
20.17	Cross-Chapter Decision Card	1382
20.18	ROI Model and Build-vs-Buy	1382
20.19	Disaster Recovery Notes (RPO/RTO)	1382
21	API Lifecycle, Governance & Developer Experience	1383
21.1	Learning Objectives & Prerequisites	1384
21.2	Deep Technical Mechanics & Architectural Concepts	1386
21.2.1	The API Lifecycle as a State Machine	1386

21.2.2	Design-First Governance	1388
21.2.3	Developer Experience Engineering	1390
21.2.4	API as a Product	1392
21.2.5	Deprecation & Sunsetting at Scale	1393
21.2.6	Regulatory & Data Governance	1395
21.2.7	Platform Economics	1396
21.3	Production-Ready Code Implementation	1397
21.3.1	Listing 20.1 — The Ruleset as Data	1397
21.3.2	Listing 20.2 — <code>mini_spectral.py</code> : A Spectral-Style Linter	1398
21.3.3	Listing 20.3 — Linter Tests, Including the Non-Compliant Spec	1408
21.3.4	Listing 20.4 — <code>docs_generator.py</code> : OpenAPI → Markdown	1412
21.3.5	Listing 20.5 — <code>ttfc_funnel.py</code> : Activation Analytics	1418
21.3.6	Listing 20.6 — <code>deprecation_tracker.py</code> : Campaign Management	1423
21.3.7	Listing 20.7 — <code>catalog_manifest.py</code> : Backstage- Style Entries	1431
21.3.8	Listing 20.8 — Smoke Tests for the Platform Tools .	1435
21.4	Common Pitfalls & Anti-Patterns	1437
21.4.1	Governance as Gatekeeping Theater	1437
21.4.2	Style Guide Without Linter Enforcement	1437
21.4.3	The Portal Nobody Can Search	1438
21.4.4	SDKs Promised but Unmaintained	1438
21.4.5	SLA Looser Than SLO, Reversed	1438
21.4.6	Sunsetting Without Telemetry	1438
21.4.7	Docs Drifting From Spec	1438
21.5	Hands-On Production Lab & Real-World Case Study	1439
21.5.1	Lab: Stand Up the Mini Platform	1439
21.5.2	Case Study: The Shadow API Nobody Owned	1440
21.6	Self-Assessment Exercises & Deep-Dive Questions	1440
21.7	Interview Q&A Vault	1442
21.8	External Bibliography & Further Reading	1448
21.9	Capstone Projects	1449
21.10	Python Dependencies Appendix	1452
21.10.1	PyYAML (<code>pyyaml>=6.0</code>)	1453
21.10.2	jsonschema (<code>jsonschema>=4.21</code>)	1453
21.10.3	Pydantic (<code>pydantic>=2.7</code>)	1453
21.10.4	pytest (<code>pytest>=8</code>)	1453
21.11	Chapter Summary & Key Takeaways	1454
21.12	Quick-Reference Card	1455
21.13	Standardized Evidence Addendum	1455

21.13.1 Benchmark Snapshot Template	1455
21.13.2 Request Trace Template	1455
21.13.3 Cost and Latency Budget Snapshot	1456
21.14 Configuration Preset Template	1456
21.15 CI Quality Gate Specification	1457
21.16 Incident Runbook Template	1457
21.17 Cross-Chapter Decision Card	1457
21.18 ROI Model and Build-vs-Buy	1458
21.19 Disaster Recovery Notes (RPO/RTO)	1458
22 Synthesis, Reference Architectures & Future Directions	1459
22.1 Learning Objectives & Prerequisites	1460
22.2 Deep Technical Mechanics & Architectural Concepts	1461
22.2.1 The Master Decision Card: Choosing an Interaction Profile	1461
22.2.2 The Northwind Robotics Reference Architecture	1465
22.2.3 The Grand Capstone: Platform Specification	1468
22.2.4 Cross-Cutting Synthesis	1471
22.2.5 Emerging Frontiers, Honestly Assessed	1471
22.2.6 Career Pathways and the Mastery Checklist	1476
22.3 Production-Ready Code Implementation	1478
22.3.1 Listing 1 — The Protocol-Selection Advisor	1478
22.3.2 Listing 2 — The Northwind Platform Slice	1490
22.3.3 Listing 3 — The Architecture Conformance Checker	1503
22.4 Common Pitfalls & Anti-Patterns	1514
22.4.1 Resume-Driven Protocol Selection	1514
22.4.2 One-Protocol-Fits-All Mandates	1514
22.4.3 Big-Bang Platform Builds	1515
22.4.4 Skipping Governance Until Fifty APIs Exist	1515
22.4.5 Adopting Frontiers Without Operational Maturity	1515
22.4.6 Synthesis Without Ownership	1515
22.5 Hands-On Production Lab & Real-World Case Study	1515
22.5.1 Lab: Run the Full Toolkit	1515
22.5.2 Case Study: Three Years of an Enterprise API Platform	1517
22.6 Self-Assessment Exercises & Deep-Dive Questions	1518
22.6.1 Concept Review	1518
22.6.2 Architecture-Design Drills	1519
22.7 Interview Q&A Vault	1519
22.8 External Bibliography & Further Reading	1524
22.9 Capstone Projects	1525
22.9.1 [junior] Project 1: Extend the Decision Advisor	1526
22.9.2 [mid] Project 2: Governance Toolchain	1526
22.9.3 [senior] Project 3: Multi-Protocol Read/Write Split	1527

22.9.4 [senior] Project 4: Webhook Notification Pipeline with Outbox Replay	1528
22.9.5 [staff] Project 5: The Grand Capstone — Northwind Robotics API Platform	1529
22.10 Python Dependencies Appendix	1530
22.10.1 fastapi	1530
22.10.2 strawberry-graphql[fastapi]	1530
22.10.3 pydantic	1531
22.10.4 pyyaml	1531
22.10.5 httpx	1531
22.10.6 pytest	1531
22.11 Chapter Summary & Key Takeaways	1532
22.12 Quick-Reference Card	1532
22.13 Closing Thoughts	1532
22.14 Standardized Evidence Addendum	1534
22.14.1 Benchmark Snapshot Template	1534
22.14.2 Request Trace Template	1534
22.14.3 Cost and Latency Budget Snapshot	1534
22.15 Configuration Preset Template	1535
22.16 CI Quality Gate Specification	1535
22.17 Incident Runbook Template	1535
22.18 Cross-Chapter Decision Card	1536
22.19 ROI Model and Build-vs-Buy	1536
22.20 Disaster Recovery Notes (RPO/RTO)	1536
A Additional Resources	1537
A.1 Glossary of Terms	1537
A.2 Recommended Learning Path	1540
B Solutions, Model Answers & Grading Rubrics	1541
B.1 Chapter 0: Orientation, Failure Taxonomies & Engineering Judgment	1542
B.1.1 Self-Assessment Model Answers	1542
B.1.2 Capstone Grading Rubrics	1545
B.2 Chapter 1: HTTP on the Wire	1547
B.2.1 Self-Assessment Model Answers	1547
B.2.2 Capstone Grading Rubrics	1549
B.3 Chapter 2: Consuming APIs in Python	1552
B.3.1 Self-Assessment Model Answers	1552
B.3.2 Capstone Grading Rubrics	1553
B.4 Chapter 3: The REST Architectural Style	1555
B.4.1 Self-Assessment Model Answers	1555
B.4.2 Capstone Grading Rubrics	1557

B.5	Chapter 4: Serialization & Contracts	1559
B.5.1	Self-Assessment Model Answers	1559
B.5.2	Capstone Grading Rubrics	1561
B.6	Chapter 5: Client-Side Security	1563
B.6.1	Self-Assessment Model Answers	1563
B.6.2	Capstone Grading Rubrics	1565
B.7	Chapter 6: Building APIs with FastAPI	1567
B.7.1	Self-Assessment Model Answers	1567
B.7.2	Capstone Grading Rubrics	1568
B.8	Chapter 7: Errors, Validation & Problem Details	1571
B.8.1	Self-Assessment Model Answers	1571
B.8.2	Capstone Grading Rubrics	1573
B.9	Chapter 8: Pagination, Filtering & Sorting	1575
B.9.1	Self-Assessment Model Answers	1575
B.9.2	Capstone Grading Rubrics	1577
B.10	Chapter 9: Versioning & Evolution	1579
B.10.1	Self-Assessment Model Answers	1579
B.10.2	Capstone Grading Rubrics	1581
B.11	Chapter 10: Idempotency, Caching & Concurrency	1583
B.11.1	Self-Assessment Model Answers	1583
B.11.2	Capstone Grading Rubrics	1585
B.12	Chapter 11: Async & Event-Driven APIs	1587
B.12.1	Self-Assessment Model Answers	1587
B.12.2	Capstone Grading Rubrics	1589
B.13	Chapter 12: GraphQL	1591
B.13.1	Self-Assessment Model Answers	1591
B.13.2	Capstone Grading Rubrics	1593
B.14	Chapter 13: gRPC & High-Performance RPC	1596
B.14.1	Self-Assessment Model Answers	1596
B.14.2	Capstone Grading Rubrics	1598
B.15	Chapter 14: Rate Limiting & Throttling	1600
B.15.1	Self-Assessment Model Answers	1600
B.15.2	Capstone Grading Rubrics	1602
B.16	Chapter 15: Microservices & Resilience	1604
B.16.1	Self-Assessment Model Answers	1604
B.16.2	Capstone Grading Rubrics	1606
B.17	Chapter 16: API Security Engineering	1608
B.17.1	Self-Assessment Model Answers	1608
B.17.2	Capstone Grading Rubrics	1610
B.18	Chapter 17: Deployment & Infrastructure	1612
B.18.1	Self-Assessment Model Answers	1612
B.18.2	Capstone Grading Rubrics	1614
B.19	Chapter 18: Observability & SRE	1617

B.19.1 Self-Assessment Model Answers	1617
B.19.2 Capstone Grading Rubrics	1618
B.20 Chapter 19: Testing & Quality Engineering	1620
B.20.1 Self-Assessment Model Answers	1620
B.20.2 Capstone Grading Rubrics	1622
B.21 Chapter 20: Lifecycle, Governance & Developer Experience .	1624
B.21.1 Self-Assessment Model Answers	1624
B.21.2 Capstone Grading Rubrics	1626
B.22 Chapter 21: Synthesis & Future Directions	1628
B.22.1 Self-Assessment Model Answers	1628
B.22.2 Capstone Grading Rubrics	1630

List of Figures

1.1	The API Maturity Model. Each level is a prerequisite for the next; the right margin shows which module of this book builds that level's capabilities.	10
1.2	Reference architecture (variant V3 shown with V4 event leg). Variant V1 uses only the client against a stubbed provider; V2 removes the gateway and second service.	12
2.1	HTTP/2 stream state machine (RFC 9113 §5.1). Push-related transitions are retained for completeness; push itself is deprecated.	68
2.2	Three generations of HTTP concurrency: parallel connections (1.1), multiplexed streams over one TCP connection (2), and independent streams over QUIC (3).	70
2.3	TLS 1.3 full handshake flight diagram (braces = encrypted under handshake keys).	73
3.1	Client request lifecycle pipeline. Every phase between pool acquisition and response receipt is bounded by an explicit timeout; failures route through a retry gate that enforces budget, jitter, and idempotency; telemetry hooks observe every stage.	112
3.2	Backoff schedules for $b = 1$ s. Deterministic backoff grows exponentially but synchronizes clients; full jitter halves the expected delay and, crucially, decorrelates concurrent retrievers (one seeded realization shown). Production policy caps both at c (here 30 s).	116
3.3	Retry timeline for one request. Transient 500s consume full-jitter backoff slots; a 429 overrides the schedule with the server's Retry-After ; the budget caps total attempts and elapsed time.	116

4.1	Request lifecycle through a layered, constraint-compliant deployment. Each layer sees only its immediate neighbors (layered system); the shared cache short-circuits revalidation (cacheability); the service holds no session state (statelessness).	173
4.2	The Richardson Maturity Model. Each step adopts one more web technology: resource URIs, then HTTP method/status semantics, then hypermedia controls [RAR13].	175
4.3	Resource-relationship graph for the Northwind Robotics warehouse domain. Containment (solid) appears in the path; independent aggregates (purchase orders) get top-level URIs and relate via query filters and links (dashed), keeping nesting at or below two levels.	179
5.1	Wire sizes for three payload shapes. JSON and MessagePack bars are measured by the benchmark harness in Section 5.3; the protobuf small-record bar is measured (the hand-rolled codec), and the protobuf nested/wide bars are projected from packed-field arithmetic — your lab run will refine them. Binary formats win consistently; the margin grows with numeric and wide payloads.	249
5.2	Format selection decision tree. Defaults, not laws: a public API with a performance-critical mobile tier may still serve protobuf via a vendor media type (Section 5.2.6).	250
5.3	Server-side negotiation decision flow. The Vary header is what keeps shared caches honest across negotiated variants. .	293
5.4	Encode/decode pipeline. Symmetric validation at both trust boundaries is deliberate: the producer’s bug must not become the consumer’s incident.	293
6.1	Authorization code grant with PKCE. The verifier never leaves the back channel; the browser-visible front channel carries only the challenge and the opaque code.	305
6.2	Access-token lifecycle state machine. The skew window converts the expiry cliff into a managed transition; reuse detection on the refresh path can drop the client back to <i>No token</i> at any time.	307
6.3	JWT compact anatomy. Only the green segment requires the key; the red and blue segments are plaintext to every holder of the token.	308
7.1	The two contract workflows converge on the same discipline: a reviewed OpenAPI artifact and a CI gate. They differ only in which side of the arrow is edited by hand.	370

7.2	The ASGI request lifecycle. Left of the router is the server; the router and middleware are Starlette; DI, validation, and response-model serialization are FastAPI.	371
7.3	The Orders API dependency graph. Providers (white) resolve per-request and are cached once per request; resources (gray) are process-lifetime singletons constructed once in the lifespan and merely <i>looked up</i> by providers. Overrides replace any provider node at test time.	373
8.1	Status-code decision tree for API error responses. Auth failures (401/403) and throttling (429) are orthogonal overlays: they can interrupt any branch before application logic runs.	444
8.2	Single error-handling pipeline: exception $\rightarrow$ problem mapper $\rightarrow$ redaction $\rightarrow$ response, with full-fidelity server-side logging forked off at the mapper under the correlation ID.	446
9.1	Offset versus keyset page-fetch latency at increasing depth. Measured values from Listing 9.4; illustrative and machine-dependent — the <i>shape</i> , not the absolute numbers, is the finding.	504
9.2	Pagination-strategy decision tree. Random access is the only property offset provides that keyset cannot; it is also the property that costs $O(k)$ and drifts under writes.	561
9.3	Cursor lifecycle: minting (top row) and redemption (bottom row). Verification precedes any use of payload fields; the one-extra-row fetch (<code>LIMIT p+1</code>) computes <code>has_more</code> without a <code>COUNT(*)</code> query.	562
9.4	Write-concurrency anomaly timeline. Above: offset windows are positional, so each front-of-ordering insert replays one row into the next page. Below: keyset windows are anchored on boundary <i>values</i> ; writes ahead of the anchor never enter the predicate. Compare Listing 9.5’s measured output.	562
10.1	Version-selection routing flow. An unrecognized <i>signal</i> and an unsupported <i>version</i> are different failures; both surface as 406 (or 404 for unknown paths) with pointers to supported versions.	571
10.2	Expand-and-contract migration timeline. The dual-serving window between T_1 and T_3 is where all consumer risk is absorbed; contraction before the window closes is what causes outages.	573

10.3	Dual-serving architecture. Both public versions are surfaces over one service core; v1 is a pure adapter with deprecation metadata, and the gateway feeds per-version telemetry that later justifies retirement.	574
11.1	Idempotency-key decision flow. Exactly one path executes business logic; every other path returns a deterministic, side-effect-free answer.	628
11.2	Conditional request sequence. The revalidation round trip still costs latency, but the 304 transfers zero representation bytes; with max-age still fresh, the round trip disappears entirely.	631
11.3	The lost-update interleaving. Reads interleave harmlessly; the second blind write is the corruption. A conditional write converts the race into a deterministic rejection.	632
12.1	The four interaction patterns. Request/response and short polling put the client in charge of timing; long polling holds the request until the server has something to say; push keeps a channel open (or calls back out, for webhooks) so the server initiates delivery.	705
12.2	Webhook delivery state machine. Every terminal failure is either a permanent rejection (do not retry a payload the consumer has told you is invalid) or budget exhaustion into the dead-letter queue, from which only an explicit operator action restores the event.	708
12.3	SSE versus WebSocket topology. Both pin one socket per client; SSE's channel is a plain HTTP response with standardized resume, while WebSocket's is a symmetric frame channel requiring application-level reconnection.	714
13.1	Request lifecycle. Static cost control sits between parse and validate: cheap to compute, before any field executes.	768
13.2	DataLoader batching: per-field loads within one tick collapse into a single keyed batch read; the per-request cache dedupes repeated keys.	770
13.3	Request topology: REST distributes a screen's data need across endpoints; GraphQL concentrates it into one operation whose cost the server must bound.	774

14.1	Anatomy of a unary gRPC call over one HTTP/2 stream. Client frames in orange, server frames in blue. Every protobuf message is preceded by the 5-byte prefix (compression flag + big-endian length); the definitive outcome arrives in the <code>grpc-status</code> trailer.	846
14.2	Message timelines for the four RPC modes. Client-to-server messages in orange, server-to-client in blue. Every arrow is a length-prefixed message on the same HTTP/2 stream; the stream closes with the <code>grpc-status</code> trailer.	849
14.3	Interceptor chain. Client interceptors wrap outbound calls (attach metadata, observe latency, drive retries); server interceptors wrap inbound handlers (enforce authz, map domain errors to status codes, emit metrics).	850
15.1	Token bucket mechanics. Left: refill at r tokens/s into a bucket capped at B ; admission consumes one token, empty buckets reject. Right: token level over time — a full bucket absorbs a burst of B instantly (vertical drop), refills linearly at r , then admits only as fast as tokens drip in. The area discipline of Proposition 15.4 is visible: burst height B , sustained slope r	910
15.2	Algorithm selection flow. The first question is topological (shared state?), the second semantic (exactness?), the third behavioral (bursts?). “No bursts tolerated” with $O(1)$ memory means accepting the fixed or sliding counter’s window artifacts consciously.	912
15.3	Distributed limiter architecture. Edge replicas consult one shared cell per key, chosen by consistent hashing so cell loss remaps only $1/C$ of keyspace; the Lua script makes check-and-admit atomic inside a cell. The dashed policy annotation is mandatory, not decorative: every replica must know what it does when the cell is unreachable (Section 15.2.9).	914
15.4	Burst behavior at the same nominal rate (10 requests/second per key), measured by Listing 15.3.3. The fixed window admits 20 requests across the $t=1.0$ boundary — Theorem 15.1 made visible — while the sliding log pays the burst penalty back in full before resuming. The token bucket and GCRA series coincide point for point, as Proposition 15.5 requires; the sliding counter’s weighted estimate coincidentally matches them on this timeline.	933

16.1	Circuit breaker state machine with the parameters of Definition 16.4. The half-open state is the recovery probe mechanism: a strictly limited number of trial calls decide whether the dependency has healed before normal traffic resumes. . .	987
16.2	The cascading-failure death spiral. Each stage feeds the next, and the retry stage feeds back into the first. Every stage has an observable early-warning signal and a pattern that breaks the chain (Table 16.2).	989
16.3	Service mesh anatomy. The data plane (sidecar proxies) executes traffic policy on the request path: mTLS identity, retries, timeouts, load balancing, telemetry. The control plane distributes configuration and certificates but never touches a request.	991
16.4	Saga orchestration and the transactional outbox. Top: the orchestrator drives steps in order and compensates in reverse on failure. Bottom: the domain write and the outbox insert commit atomically; the relay publishes at-least-once; the idempotent consumer absorbs replays.	993
17.1	Request trust-boundary and validation pipeline. Each gate is independent and can reject on its own evidence; the egress guard applies the same suspicion to traffic <i>leaving</i> the service (API7, API10).	1072
17.2	Token validation sequence. Steps 2–5 are <i>authentication</i> ; only after them may claims be read; step 7 is <i>authorization</i> and is a separate decision per function, object, and property. . .	1078
17.3	STRIDE attack-surface map for the Northwind Robotics fleet API. Edges are labeled by the STRIDE category they expose and the OWASP item that weaponizes it.	1081
18.1	Probe lifecycle. The startup probe alone is evaluated during boot, protecting slow starts from the liveness budget. After its first success, readiness (routing consequence) and liveness (restart consequence) run on their own periods.	1170
18.2	Canary rollout timeline. Each step holds for a fixed analysis window while the gate (error-rate SPRT plus latency guardrails, Listing 18.3.6) evaluates; evidence crossing the upper boundary rolls back automatically, the lower boundary promotes.	1173
18.3	Request path through edge, gateway, and mesh. TLS terminates at the provider edge (public certificate, WAF inspection), is re-encrypted to the gateway, and becomes mTLS east-west; each box owns exactly the policies its position can see.	1174

19.1	The observability pipeline. Application code depends only on the OTel API; the SDK assembles spans and metric aggregations; the exporter ships OTLP to the collector, which batches, redacts, optionally tail-samples, and fans out to one backend per signal. Vendor changes happen at the collector, never in application code.	1238
19.2	One trace across three services. Bars are spans on a shared time axis; every span carries trace id <code>4bf9...4736</code> ; each hop injects a fresh span id into the outgoing <code>traceparent</code> , which the next service records as its parent. This is exactly the tree Listing 18.3 exports and asserts.	1240
19.3	The long tail, plotted from Listing 18.4's seeded distribution. Half of all requests finish in 25 ms, yet the <i>average</i> is 761 ms — pulled 30× above the median by a tail the naked eye almost misses — and p99 is three buckets past the average. Any dashboard or SLO keyed on the mean is measuring a user nobody has.	1242
19.4	Burn-rate alert logic. The long window (blue brace) filters noise by requiring a sustained burn; the short window (orange brace) requires the burn to be active <i>at this instant</i> , so alerts clear promptly after mitigation instead of paging on history. Both must hold (Equation (19.4)).	1244
20.1	The API test pyramid. Width encodes recommended <i>volume</i> ; height encodes cost and environmental fidelity. Contract tests sit above component tests because they assume a working provider component, yet far below integration tests in environmental cost.	1311
20.2	The consumer-driven contract loop (Pact model [Pac24]). The pact file is written by the <i>consumer</i> , owned by <i>both</i> teams, and executed by the <i>provider's</i> build.	1312
20.3	Load-test methodology. The CI gate runs a short structural smoke; deep sweeps are scheduled offline and re-baselined on architectural change.	1317
21.1	The API lifecycle as a state machine. Each transition is an audited event with named preconditions; <code>published</code> is the dwell state where normal operation (and only backward-compatible evolution) happens.	1387
21.2	The paved path: design → lint → review → publish → portal. Compliance is the default because the compliant route is the one with tooling.	1390

21.3	The governance feedback loop: standards flow to consumers through tooling; observed consumer behavior flows back as evidence that revises the standards.	1394
22.1	The Master Decision Card as a decision tree. Question numbers refer to Table 22.1; delivery is tested first because it is structural. Edge cases (gRPC-Web for browsers, GraphQL subscriptions over WebSockets) are refinements, not new branches.	1464
22.2	The Northwind Robotics API Platform reference architecture. Solid arrows are request/event flow; dashed arrows are control-plane and telemetry flow. Every box names the chapter that teaches it.	1466
22.3	The API engineer mastery pathway. Levels are scopes of responsibility, not years of tenure.	1477

List of Tables

1.1	Reader pathways. Chapter numbers refer to the dependency map in Table 1.3.	5
1.2	The API failure taxonomy used throughout this book.	8
1.3	Hard prerequisite edges between chapters.	53
1.4	Starter latency budget decomposition (share of p99 target).	54
1.5	Initial risk register for the curriculum itself.	54
1.6	Symptom-to-cause gallery for the most common lab failures.	55
1.7	Readiness self-assessment (target: total ≥ 14 , no zeros).	55
1.8	Chapter 0 quick-reference card.	56
1.9	Minimum benchmark table to keep per chapter.	56
1.10	Operational budget template for this chapter’s technique.	56
1.11	Decision card to complete per chapter.	57
2.1	Method properties per RFC 9110 §9. *Cacheable by default heuristics. [†] POST responses are cacheable only with explicit freshness information. [‡] PATCH <i>can</i> be made idempotent by construction, but the spec does not require it.	64
2.2	Status-code selection for the scenarios production teams argue about.	65
2.3	HTTP/2 frame types (RFC 9113 §6).	67
2.4	TLS 1.2 versus TLS 1.3 at a glance [Res18].	72
2.5	Method cheat sheet.	102
2.6	Status-code cheat sheet.	103
2.7	HTTP/2 frame-type cheat sheet (header: 24-bit length, 8-bit type, 8-bit flags, 1 reserved bit, 31-bit stream ID).	104
2.8	Minimum benchmark table to keep per chapter.	104
2.9	Operational budget template for this chapter’s technique.	105
2.10	Decision card to complete per chapter.	105
3.1	HTTP client support matrix for Python 3.11+ consumers.	111
3.2	Consumer error taxonomy. Each layer maps to a distinct exception type and remediation path.	119
3.3	Timeout dimensions: always set all four; never leave any infinite.	162

3.4	Retry decisions by failure signal.	165
3.5	Status-class handling for consumers (RFC 9110 [Fie22a]). . .	165
3.6	Minimum benchmark table to keep per chapter.	166
3.7	Operational budget template for this chapter's technique. . .	166
3.8	Decision card to complete per chapter.	166
4.1	Fielding's six constraints as an exchange: constraints in, properties out [Fie00].	174
4.2	Method properties per RFC 9110 [Fie22a]. * PATCH is not <i>guaranteed</i> idempotent, but a specific patch document applied twice usually is — design for it when you can.	181
4.3	Precise status codes for CRUD lifecycles. 404 in response to DELETE is informative, not an error the client must handle differently; 204 versus 200 signals whether the response carries a representation.	234
4.4	Hypermedia media types for JSON APIs (see Section 4.8). .	235
4.5	Six-constraint compliance checklist.	235
4.6	Method semantics at a glance [Fie22a].	236
4.7	Richardson Maturity Model diagnostic [RAR13].	236
4.8	Minimum benchmark table to keep per chapter.	236
4.9	Operational budget template for this chapter's technique. . .	236
4.10	Decision card to complete per chapter.	236
5.1	Core JSON Schema 2020-12 keywords used throughout this chapter.	290
5.2	Protocol Buffers wire types (proto3).	291
5.3	Serialization format trade-off matrix. "Fast" and "small" are relative; measure with your own payloads (Section 5.3). . . .	292
5.4	Format selection at a glance (details: Table 5.3).	294
5.5	Negotiation headers cheat-sheet.	294
5.6	Minimum benchmark table to keep per chapter.	294
5.7	Operational budget template for this chapter's technique. . .	295
5.8	Decision card to complete per chapter.	295
6.1	Registered JWT claims and the consumer-side validation rule for each.	309
6.2	Credential storage matrix for client applications.	311
6.3	Grant-selection table.	361
6.4	The only safe JWT validation order.	361
6.5	Credential storage matrix.	362
6.6	Minimum benchmark table to keep per chapter.	363
6.7	Operational budget template for this chapter's technique. . .	363
6.8	Decision card to complete per chapter.	364

7.1	Design-first vs. code-first: the honest trade-off.	369
7.2	Probe semantics. Conflating readiness with liveness causes boot loops: a dependency outage makes liveness fail, the orchestrator restarts the pod, which changes nothing.	375
7.3	Endpoint and execution-model decisions at a glance.	429
7.4	DI rules that prevent the common incidents.	430
7.5	Minimum benchmark table to keep per chapter.	430
7.6	Operational budget template for this chapter's technique. . .	431
7.7	Decision card to complete per chapter.	432
8.1	Standard problem detail members per RFC 9457 [Not23]. . .	440
8.2	Error status selection cheat sheet.	490
8.3	Problem-details member cheat sheet.	491
8.4	Minimum benchmark table to keep per chapter.	491
8.5	Operational budget template for this chapter's technique. . .	491
8.6	Decision card to complete per chapter.	492
9.1	Measured single-page latency by strategy and depth (median of 15; illustrative, machine-dependent; page size 50, $n =$ 200,000 rows, SQLite with composite index).	503
9.2	Pagination strategies at a glance ($p =$ page size, $n =$ rows). .	563
9.3	Minimum benchmark table to keep per chapter.	563
9.4	Operational budget template for this chapter's technique. . .	564
9.5	Decision card to complete per chapter.	564
10.1	Change taxonomy under Definition 10.1.	619
10.2	Versioning strategy trade-off matrix.	620
10.3	Minimum benchmark table to keep per chapter.	620
10.4	Operational budget template for this chapter's technique. . .	621
10.5	Decision card to complete per chapter.	621
11.1	Prerequisite map for Chapter 10.	626
11.2	Safety and idempotency per RFC 9110 [Fie22a]. PATCH is idempotent only if the patch document encodes an absolute, not relative, transformation.	627
11.3	Core Cache-Control response directives (RFC 9111).	630
11.4	Concurrency-control strategy selection.	633
11.5	Method idempotency and retry policy [Fie22a].	694
11.6	Cache-Control cheat-sheet.	694
11.7	Status-code selection for this chapter's machinery.	694
11.8	Minimum benchmark table to keep per chapter.	695
11.9	Operational budget template for this chapter's technique. . .	695
11.10	Decision card to complete per chapter.	696

12.1	Pattern selection matrix. “Latency” is data-arrival latency seen by the consumer; “fan-out cost” is the marginal cost of the N -th consumer.	704
12.2	Minimum benchmark table to keep per chapter.	758
12.3	Operational budget template for this chapter’s technique. . .	758
12.4	Decision card to complete per chapter.	759
13.1	The four list/nullability combinations as error-containment policy.	765
13.2	Decision matrix. No column is “best”; each optimizes a different constraint.	773
13.3	GraphQL type system cheat sheet.	833
13.4	Minimum benchmark table to keep per chapter.	835
13.5	Operational budget template for this chapter’s technique. . .	835
13.6	Decision card to complete per chapter.	837
14.1	The 17 canonical gRPC status codes and their conventional HTTP mapping when transcoding to JSON APIs (the mapping used by Google’s <code>google/rpc/code.proto</code> conventions).851	
14.2	Abridged; the full 17-code table with HTTP mappings is Table 14.1.	898
14.3	Minimum benchmark table to keep per chapter.	899
14.4	Operational budget template for this chapter’s technique. . .	899
14.5	Decision card to complete per chapter.	901
15.1	Admission algorithm comparison. All rows enforce the same nominal rate; they differ in memory, burst envelope, and boundary behavior. GCRA and token bucket are provably equivalent (Proposition 15.5); the choice between them is operational, not behavioral.	976
15.2	Failure-policy decision matrix. Degraded-local dominates when a cheap local limiter is available; the fail-open/fail-closed binary is for endpoints where either abuse or availability is existentially worse.	977
15.3	Minimum benchmark table to keep per chapter.	977
15.4	Operational budget template for this chapter’s technique. . .	977
15.5	Decision card to complete per chapter.	977
16.1	Synchronous versus asynchronous inter-service communication.983	
16.2	Failure-mode analysis: each stage of the spiral, its observable precursor, and the pattern that breaks the chain.	990
16.3	Discovery and load-balancing topologies.	991

16.4	Measured chaos-drill results, Listing 16.3.3 (deterministic: counts, not wall-clock). Resilience patterns do not convert failure into success; they convert expensive failure into cheap failure and buy recovery headroom.	1025
16.5	Symptom-to-pattern selection.	1064
16.6	Starting points for Definition 16.4’s parameters. Every number is a hypothesis until a chaos drill confirms it.	1065
16.7	Minimum benchmark table to keep per chapter.	1065
16.8	Operational budget template for this chapter’s technique. . .	1066
16.9	Decision card to complete per chapter.	1067
17.1	OWASP API Security Top 10 (2023) [OWA23]: mechanism summary. Every row is executable in this chapter’s exploit suite.	1158
17.2	STRIDE per endpoint: worked row set for the robot patch route. Every mitigations cell names an executable control, not a policy aspiration.	1159
17.3	OWASP API Top 10 mitigation matrix [OWA23].	1159
17.4	The only safe token-validation order.	1160
17.5	Secure-headers table for JSON APIs.	1160
17.6	Minimum benchmark table to keep per chapter.	1160
17.7	Operational budget template for this chapter’s technique. . .	1160
17.8	Decision card to complete per chapter.	1161
18.1	Gateway capabilities matrix. Envoy is the substrate many gateways and meshes (Istio, Consul, Contour, Gloo) are built from; Kong is a batteries-included gateway on NGINX; NGINX itself is the minimal, static-configuration baseline. .	1228
18.2	Probe semantics. The load-bearing row is the second: readiness failure changes <i>routing</i> , liveness failure changes <i>process state</i> . A readiness probe that also gates liveness restarts Pods for other services’ outages; a liveness probe that checks dependencies turns a database blip into a fleet-wide restart storm.	1229
18.3	Rolling-update arithmetic for a 4-replica Deployment. Percentages are allowed for both fields (rounded: surge up, unavailable down), which keeps the policy correct as HPA moves <i>R</i>	1230
18.4	Minimum benchmark table to keep per chapter.	1230
18.5	Operational budget template for this chapter’s technique. . .	1231
18.6	Decision card to complete per chapter.	1231
19.1	<code>traceparent</code> field reference (W3C Trace Context).	1239

19.2	Error budgets by SLO target. The budget is a <i>spending allowance</i> shared by outages, risky deploys, and experiments — when it is spent, the roadmap tilts to reliability [Bey+16].	1243
19.3	Multi-window multi-burn-rate alerting for a 99.9% SLO (SRE Workbook parameters) [Bey+16]. Listing 18.5 implements and tests exactly this table.	1244
19.4	Severity levels. The response column is part of the definition: a severity without a response contract is a label, not a level.	1305
19.5	Minimum benchmark table to keep per chapter.	1306
19.6	Operational budget template for this chapter’s technique.	1306
19.7	Decision card to complete per chapter.	1306
20.1	Test-layer responsibility matrix.	1379
20.2	Fault taxonomy and how to assert on it deterministically.	1379
20.3	Little’s Law and saturation cheat sheet.	1380
20.4	Minimum benchmark table to keep per chapter.	1380
20.5	Operational budget template for this chapter’s technique.	1381
20.6	Decision card to complete per chapter.	1382
21.1	Lifecycle states with evidence-based entry/exit criteria.	1387
21.2	The consumer-observable surfaces a style guide must pin down.	1388
21.3	Deprecation policy: internal versus external consumers.	1395
21.4	Lifecycle states: the contract each side is under.	1455
21.5	Style-guide core rules and their enforcement.	1455
21.6	DX and governance metrics cheat sheet.	1456
21.7	Minimum benchmark table to keep per chapter.	1456
21.8	Operational budget template for this chapter’s technique.	1456
21.9	Decision card to complete per chapter.	1457
22.1	The six criteria of the Master Decision Card.	1462
22.2	Interaction profile comparison matrix (tendencies, not laws).	1463
22.3	Grand Capstone API surfaces. Each surface cites its governing spec artifact.	1469
22.4	Grand Capstone four-milestone delivery plan.	1470
22.5	The consolidated failure taxonomy: one row per chapter’s core enemy.	1472
22.6	Module I synthesis: speak the protocol fluently before building.	1473
22.7	Module II synthesis: correctness is a designed property.	1473
22.8	Module III synthesis: pick profiles per constraint, scale deliberately.	1474
22.9	Module IV synthesis: production is a discipline, not a destination.	1474
22.10	Frontier adoption postures. Nothing here exempts you from the operational maturity of Module IV.	1476

22.11The mastery checklist, by level.	1477
22.12The one-page Master Decision Card. Photocopy freely. . . .	1533
22.13Minimum benchmark table to keep per chapter.	1534
22.14Operational budget template for this chapter's technique. . .	1534
22.15Decision card to complete per chapter.	1536

List of Tables

Chapter 1

Introduction: The API Economy & How to Use This Book

Executive Summary

Every system you will build, operate, or integrate over the next decade will either expose an Application Programming Interface (API) or depend on one — usually both. The modern enterprise is no longer a monolith behind a firewall; it is a mesh of services communicating over versioned contracts, where the interface *is* the product and the contract *is* the engineering artifact of record. Industry analysts have estimated for years that the large majority of enterprise workloads now traverse an API boundary at some point in their lifecycle, and the economic consequences are measurable: entire companies (payments processors, communications platforms, mapping providers) sell nothing except well-designed interfaces over HTTP. This is the *API economy*: interfaces priced, versioned, secured, and observed with the same rigor once reserved for physical supply chains.

This book is an engineering curriculum for that reality. It treats APIs not as an afterthought bolted onto an application but as **products**, **contracts**, and **failure surfaces** — three mental models that recur in every chapter. As products, APIs have consumers, developer experience, pricing, and lifecycles. As contracts, they have formal, machine-checkable semantics: operations, representations, status codes, and failure behavior, all versioned over time. As failure surfaces, they are the exact location where distributed systems hurt you: timeouts, retries, partial failures, and misaligned assumptions between provider and consumer. Mastering APIs means mastering all three perspectives simultaneously.

You are reading Chapter 0, the orientation chapter. Its job is fourfold. First, it defines terms *formally* — we will not wave our hands at words like

“REST,” “contract,” or “latency percentile.” Second, it maps the territory: four modules, twenty-one technical chapters, and the dependency structure between them, so you can plan a path whether you are a consumer of APIs, a producer of them, an architect governing a platform, or a data engineer moving records across boundaries. Third, it installs the evaluation discipline used throughout the book: latency measured in percentiles rather than averages, error budgets derived from service-level objectives, and benchmark protocols that a skeptical reviewer can reproduce. Fourth, it gets your hands dirty within the first three hours: an offline, stdlib-only lab in which you build a working HTTP API and client on your own machine and measure its latency distribution.

Key Concept

An API is a **versioned contract** between a provider and its consumers. The contract specifies *operations* (what you may ask), *representations* (the shape of data exchanged), and *failure semantics* (what happens when things go wrong), all carried over a *transport protocol*. Everything else in this book — frameworks, gateways, meshes, observability — exists to make that contract reliable, secure, and evolvable at scale.

The tone throughout is pragmatic and production-focused. Every technique is introduced because it prevents a real failure mode, and every chapter closes with measurable success criteria rather than vague encouragement. Code is Python 3.11+, fully typed, stdlib-first, and runnable offline — no cloud accounts, no API keys, no network access required. Where third-party services appear (payment gateways, identity providers), they are simulated with deterministic local stubs. All datasets are synthetic, drawn from the fictional *Northwind Robotics* universe used across this curriculum.

By the end of this chapter you will have a precise vocabulary, a working local environment, a verified benchmark harness, and a map of the road ahead. Let us begin.

1.1 Learning Objectives & Prerequisites

1.1.1 Learning Objectives

After completing this chapter and its lab, you will be able to:

- LO-1.** Define an API formally as a versioned contract (operations, representations, failure semantics, transport) and explain the API value chain from provider to consumer to end user.

- LO-2.** Apply the three mental models — API as product, API as contract, API as failure surface — to reason about design decisions.
- LO-3.** Classify real incidents into the five-category API failure taxonomy (transport, contract, semantic, operational, security) and select an appropriate detection strategy for each.
- LO-4.** Locate any system on the five-level API Maturity Model (L0–L4) and articulate the concrete capabilities required to advance one level.
- LO-5.** Measure client-observed latency correctly using percentiles (p50/p95/p99), explain why arithmetic means mislead, and run the minimal benchmark protocol defined in this chapter.
- LO-6.** Decompose a request’s latency budget across TLS setup, validation, business logic, and serialization, and compute an error budget from an SLO target (e.g. 99.9% monthly availability = 43.2 minutes of downtime).
- LO-7.** Set up a reproducible Python development environment on Windows PowerShell and verify it with an automated sanity-check script.
- LO-8.** Build, run, and measure a minimal stdlib-only HTTP API and client offline, forming the baseline for every subsequent chapter.
- LO-9.** Choose the correct reader pathway (consumer, producer, architect, data engineer) and locate prerequisite chapters for any topic.
- LO-10.** State the governance, threat-modeling (STRIDE), and ethics baselines that frame all production work in this book.

1.1.2 Prerequisites

This is an intensive curriculum aimed at practicing engineers. We assume:

- **Python 3.11 or newer.** You should be comfortable with functions, classes, type hints (PEP 484), context managers, and virtual environments. You do *not* need prior web-framework experience; FastAPI is taught from first principles in Chapter 7.
- **Basic networking literacy.** You should know what an IP address, a TCP connection, a DNS lookup, and a port are. We teach HTTP from the wire up in Chapter 2, so deep protocol knowledge is *not* a prerequisite — but nothing in that chapter should be your first exposure to the word “packet.”

- **Git.** Cloning, branching, committing. All sample code is versioned, and the labs assume you can diff and revert.
- **A Windows workstation with PowerShell.** All setup and run instructions are given in PowerShell idioms. Everything also works on macOS/Linux, but those commands are intentionally out of scope.

1.1.3 How to Use This Textbook

The book supports three modes of use, and you may move between them freely:

Self-paced study (16 weeks). One module per four weeks, one to two chapters per week. Do every lab; the labs are where the learning consolidates. Budget 6–10 hours per week including exercises.

Team workshops. Each chapter’s lab and capstone are scoped for a half-day workshop. A common pattern is a “guild” format: one engineer presents the chapter’s failure taxonomy section, the team then runs the lab against a shared repository, and the session closes with the self-assessment questions answered collaboratively.

Reference. Chapters are cross-referenced rather than strictly linear. If you arrive with production experience, jump directly to the chapter you need and use the dependency map (Section 1.2.6) to backfill prerequisites.

1.1.4 Reader Pathways

Four curated tracks through the same twenty-one chapters. Every track reads Chapters 0–4; divergence is deliberate, and each track rejoins the shared core where its role demands it.

1.1.5 Conventions Used in This Book

- **Code listings** are complete and runnable. Nothing is elided; there are no “left as an exercise” gaps in production listings. Python listings are typed (PEP 484) and lint-clean (PEP 8).
- **Boxes.** *Key Concept* boxes state load-bearing ideas you must internalize. *Warning* boxes mark mistakes that cause production incidents. *Example* boxes ground abstractions in the Northwind Robotics universe.

Track	Chapter sequence
Consumer-only	0–5, 8 (as a client), 10, 14 (as a client), 19 (consumer contract tests). Skip provider build chapters; read their “contract” sections only.
Producer	0–10, 16–20. Add 11–13 selectively per your product’s protocol surface. This is the default full-stack track.
Architect	0–1, 3–4, 9–15, 17–21. Skim implementation detail in 2 and 6–8; go deep on versioning, resilience, observability, and governance.
Data engineer	0–5, 8, 10–11, 14–15, 18. Emphasis on pagination, idempotent pipelines, webhooks/streams, and backpressure; GraphQL (12) if your warehouse feeds BI consumers.

Table 1.1: Reader pathways. Chapter numbers refer to the dependency map in Table 1.3.

- **Formal definitions** appear in numbered Definition blocks and are the canonical meaning of the term for the rest of the book.
- **Math notation.** Latency percentiles are written p_{50}, p_{95}, p_{99} ; rates are written as λ (arrival rate, requests/second) and μ (service rate); queueing formulas are stated precisely when used. Scalars are italic (x), vectors bold ($\mathbf{x}$).
- **Paths and commands** are Windows-native. In prose, a path such as the virtual-environment activation script is written `.venv\Scripts\Activate.ps1`; inside code blocks, backslashes are literal.
- **Citations** refer to the bibliography; foundational references such as Fielding’s dissertation [Fie00] and the HTTP semantics standard [Fie22a] are cited at first substantive use.

Warning

Offline-first is a hard constraint, not a suggestion. Every listing, lab, and test in this book runs with the network cable unplugged. If you find yourself reaching for a live third-party API while doing an exercise, stop: the chapter provides a deterministic local stub for it. This discipline is deliberate — it forces you to understand the protocol rather than the vendor SDK, and it makes every result in the book reproducible on a fresh machine.

1.2 Deep Technical Mechanics & Architectural Concepts

1.2.1 What an API Is: A Formal Definition

The word “API” is used so loosely in industry that it has nearly lost its meaning. In this book it has exactly one meaning, and we will use it consistently for twenty-one chapters.

Definition 1.1 (Application Programming Interface). An *API* is a **versioned contract** between a *provider* and one or more *consumers*, specifying:

1. a set of **operations** $O = \{o_1, o_2, \dots, o_n\}$, each with a name, input parameters, and preconditions;
2. **representations** R : the serialized data shapes exchanged in requests and responses (schemas, media types, encodings);
3. **failure semantics** F : for each operation, the complete set of failure modes (status codes, error representations, retryability, and side-effect guarantees); and
4. a **transport protocol** T over which operations are invoked (e.g. HTTP/1.1 or HTTP/2 per RFC 9110 [Fie22a], gRPC over HTTP/2, or a message broker binding).

A *version* v of the contract is an immutable snapshot $\langle O_v, R_v, F_v, T \rangle$; evolving any component produces a new version governed by a compatibility policy (Chapter 10).

Three consequences of Definition 1.1 are worth stating now because they drive everything that follows. First, an API is *not* the implementation; two services with identical contracts are substitutable, which is precisely what makes mocking, contract testing, and the offline-first discipline of this book possible. Second, failure semantics are part of the contract — a provider that changes which errors it returns has broken its contract just as surely as one that renames a field. Third, versioning is not optional metadata; it is part of the definition. An unversioned API is an incomplete contract whose terms can change under the consumer at any moment.

1.2.2 The API Value Chain

APIs exist to move value across organizational and machine boundaries. The canonical value chain has five links:

1. **Capability.** Something the provider can do: charge a card, forecast demand, retrieve a robot’s telemetry.
2. **Interface.** The versioned contract exposing the capability.
3. **Distribution.** How consumers discover and integrate: documentation, catalogs, SDKs, developer portals.
4. **Consumption.** Client applications composed from one or more APIs.
5. **End-user value.** The business outcome, measured in revenue, cost avoided, or risk reduced.

Value leaks at every seam. A capability without a contract is unreachable; a contract without distribution is undiscovered; consumption without observability is unbillable and un-debuggable. The maturity model in Section 1.2.5 is, at heart, a checklist for sealing those seams. Treating the API *as a product* [GH21] means assigning an owner to the whole chain, not merely to link two.

Example

Northwind Robotics. Northwind builds warehouse robots. Its telemetry platform (capability) exposes `GET /v1/robots/{id}/telemetry` (interface), published through an internal developer portal with an OpenAPI 3.1 document [Ope21] (distribution), consumed by the fleet-dashboard web app and a nightly data-pipeline job (consumption), which together cut unplanned robot downtime by 18% last quarter (end-user value). When the pipeline team complains that “the API is slow,” every link of this chain is a suspect — and this chapter’s diagnostic playbook (Section 1.5.3) tells you how to isolate which one.

1.2.3 Three Mental Models: Product, Contract, Failure Surface

API as product. The API has customers (developers), a funnel (discovery → first successful call → production traffic), competitors, and a lifecycle from beta to deprecation and sunset (Chapter 21). Product thinking forces you to budget for documentation, changelogs, and developer experience (DX) as first-class engineering deliverables.

API as contract. The contract is machine-checkable. OpenAPI documents, JSON Schemas, and protocol-buffer definitions are not documentation *about* the system; they are artifacts *of* the system, diffed

in code review and enforced in CI (Chapters 5 and 20). Contract thinking turns “we changed a field, hope nobody noticed” into a build failure.

API as failure surface. Every API call is a distributed-systems event: it can be delayed, duplicated, dropped, reordered, or partially applied. The consumer and provider hold independent, possibly inconsistent views of the world [Kle17]. Failure-surface thinking is why this book spends entire chapters on idempotency (Chapter 11), rate limiting (Chapter 15), and resilience patterns (Chapter 16).

1.2.4 The Five-Category API Failure Taxonomy

Debugging is classification before it is repair. Nearly every API incident you will ever meet belongs to exactly one of five categories. Learn the taxonomy now; every subsequent chapter deepens one or more rows of it.

Category	Example	Primary detection strategy
Transport	TCP connect timeout; TLS handshake failure; DNS resolution error; HTTP 502 from an intermediate proxy.	Connection-level metrics, synthetic probes, TCP/TLS handshake timing, load-balancer logs.
Contract	Provider renames <code>unit_price</code> to <code>unitPrice</code> ; consumer deserializer throws; HTTP 400 on previously valid payload.	Schema-diff gates in CI, consumer-driven contract tests, canary traffic replay.
Semantic	Call succeeds (HTTP 200) but means the wrong thing: price returned in cents instead of dollars; “deleted” flag inverted.	Property-based tests, reconciliation jobs, business-metric anomaly detection.
Operational	Latency p99 degrades from 120 ms to 4 s under load; retry storm after a partial outage; connection-pool exhaustion.	Percentile latency dashboards, saturation metrics, distributed tracing, load tests.
Security	Broken object-level authorization (BOLA); leaked API key in a client bundle; unsanitized input reaching a SQL query.	Authz fuzzing, secret scanning, WAF/IDS alerts, penetration tests, OWASP API Top 10 audits.

Table 1.2: The API failure taxonomy used throughout this book.

Two rules make the taxonomy actionable. Rule one: **classify before you mitigate**. Retrying a contract failure is waste; retrying a transport failure with exponential backoff is correct; retrying a non-idempotent semantic

operation is a corruption vector. Rule two: **each category has a natural home in the stack**. Transport failures live below HTTP semantics, contract failures at the schema layer, semantic failures in business logic, operational failures in capacity and dependency management, and security failures at every boundary — which is why Chapter 17 treats the OWASP API Security Top 10 as a cross-cutting concern rather than a bolt-on.

1.2.5 The API Maturity Model (L0–L4)

Organizations do not leap from shell scripts to governed platforms; they climb. The five-level model below is the book’s yardstick — you will be asked to place real systems on it in the exercises.

- L0 — Ad-hoc scripts.** Calls are made with one-off scripts, copied curl commands, and hardcoded credentials. No versioning, no tests, no runbooks. Characteristic quote: “it works on my machine.”
- L1 — Documented REST.** Endpoints follow REST constraints [Fie00], ship human-readable documentation, and use structured errors. Contracts exist but are enforced by convention, not by machines.
- L2 — Contract-tested in CI.** OpenAPI/JSON Schema artifacts are generated or validated in CI; breaking changes fail the build; consumer contract tests run before deploys (Chapter 20).
- L3 — Resilient, observable services.** Timeouts, retries with budgets, circuit breakers, and idempotency keys are standard [Nyg18]; SLOs with error budgets are defined and alerted on [Bey+16]; traces and structured logs make every request explainable (Chapters 19 and 16).
- L4 — Governed API platform.** A developer portal, catalog, lifecycle policies, automated governance linting, security baselines, and paved-road templates make the compliant path the easy path (Chapter 21).

1.2.6 Map of the Book: Four Modules and a Synthesis

The curriculum is a 16-week intensive organized into four modules plus a synthesis chapter.

Module 1 — Foundations (Chapters 1–5). Chapter 2 opens the wire: requests, responses, headers, and status codes per RFC 9110 [Fie22a], captured and decoded by hand. Chapter 3 builds disciplined HTTP clients in Python: sessions, timeouts, retries, and connection pooling. Chapter 4

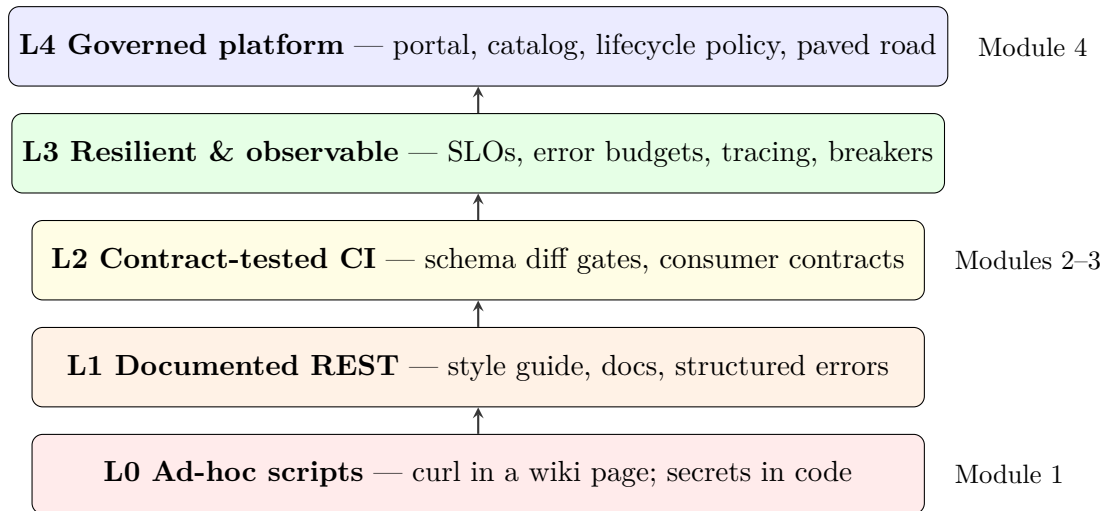


Figure 1.1: The API Maturity Model. Each level is a prerequisite for the next; the right margin shows which module of this book builds that level’s capabilities.

formalizes REST as an architectural style [Fie00] — resources, representations, hypermedia, and the constraints that actually matter. Chapter 5 covers JSON, schema languages, and contract artifacts. Chapter 6 secures the consumer side: credential storage, TLS verification, and token handling.

Module 2 — Designing & Building (Chapters 6–10). Chapter 7 builds production services with FastAPI. Chapter 8 standardizes errors with RFC 9457 problem details. Chapter 9 solves pagination, filtering, and sorting over large collections. Chapter 10 is the contract evolution chapter: compatibility classes, version negotiation, deprecation policy. Chapter 11 closes the module with the trinity of safe repetition: idempotency keys, HTTP caching, and concurrency control.

Module 3 — Advanced Architecture (Chapters 11–15). Chapter 12 covers webhooks, Server-Sent Events, and WebSockets. Chapters 13 and 14 teach the two major alternative paradigms — GraphQL’s client-driven queries and gRPC’s schema-first, high-performance RPC. Chapter 15 derives rate limiting and throttling algorithms from queueing first principles. Chapter 16 assembles resilience patterns for service-to-service calls [New21].

Module 4 — Enterprise Production & Security (Chapters 16–20). Chapter 17 works through the OWASP API Security Top 10 with exploitable labs and fixes. Chapter 18 covers deployment, gateways, and Kubernetes. Chapter 19 is SRE applied to APIs: SLIs, SLOs, error budgets,

tracing [Bey+16]. Chapter 20 is quality engineering: contract, property-based, chaos, and load testing. Chapter 21 closes with governance, lifecycle management, and developer experience.

Synthesis (Chapter 21). Chapter 22 integrates everything into a capstone architecture and looks ahead at emerging protocol and platform trends.

Chapter dependency map. Solid knowledge of a chapter requires its ancestors. The edges below are the *hard* prerequisites; all other cross-references are enrichment.

1.2.7 Reference Architecture Variants

Four deployment topologies recur across the book. Each chapter states which variant its listings target; the evaluation protocol (Section 1.3.2) is defined identically for all four so that results are comparable.

V1 — Client-only consumer. Your code calls an external (simulated) provider. Used in Chapters 2–6.

V2 — Single FastAPI service. One process, one datastore, local clients. Used in Chapters 7–11.

V3 — Gateway-fronted microservices. An edge gateway routes to multiple services with resilience policies. Used in Chapters 15–19.

V4 — Event-driven. Producers and consumers decoupled through topics; webhooks and streams. Used in Chapter 12 and parts of 21.

1.2.8 Cost and Latency Budgeting

A request’s end-to-end latency L decomposes into additive stages:

$$L = L_{\text{TLS}} + L_{\text{queue}} + L_{\text{validate}} + L_{\text{logic}} + L_{\text{serialize}} + L_{\text{egress}} \quad (1.1)$$

where L_{TLS} is connection setup (amortized to near zero by keep-alive), L_{queue} is time waiting for a worker, L_{validate} is schema and authz checking, L_{logic} is business logic including downstream calls, and $L_{\text{serialize}} + L_{\text{egress}}$ is response encoding and transmission. A *latency budget* assigns each stage a target share of the SLO; Table 1.4 shows the starter budget used in labs. Cost budgeting mirrors the same decomposition: TLS and egress are dominated by network, validation and serialization by CPU, logic by I/O and downstream fan-out. When p99 violates the SLO, Equation (1.1) tells

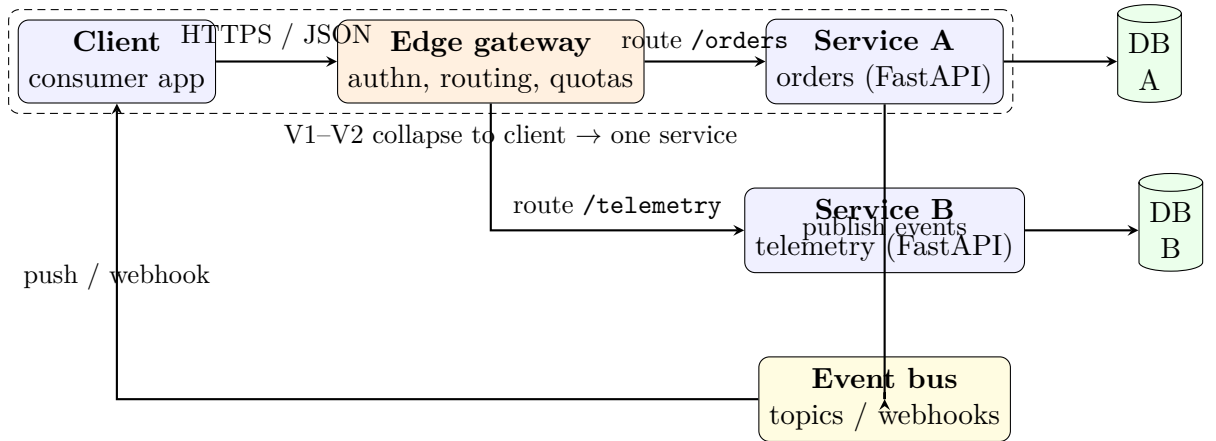


Figure 1.2: Reference architecture (variant V3 shown with V4 event leg). Variant V1 uses only the client against a stubbed provider; V2 removes the gateway and second service.

you where to spend optimization effort *before* you profile anything: measure each stage, compare to its budget, and fix the largest overrun first.

1.2.9 Governance, SLOs, and Threat Modeling: The Baseline

Governance baseline. Every API in this book, including toy ones, carries five artifacts: an owner, a version, a machine-readable contract, a changelog, and a deprecation policy. That is the minimum bar for L1 on the maturity ladder, and it costs almost nothing at small scale. Governance at L4 adds catalogs, automated linting of contracts, and lifecycle state machines (Chapter 21).

SLO and error-budget starter. A service-level objective states a target for a measurable indicator. The canonical first SLO is availability: the fraction of requests answered with a non-5xx status within the latency threshold. The *error budget* is the allowed shortfall. Worked example: a 99.9% monthly availability SLO over a 30-day month permits

$$(1 - 0.999) \times 30 \times 24 \times 60 = 43.2 \text{ minutes of downtime per month. (1.2)}$$

The budget converts reliability from a vibe into a spendable resource: when more than half the budget is gone mid-month, you freeze feature releases and spend engineering on reliability instead [Bey+16]. We compute budgets for every lab from Chapter 19 onward.

Threat model primer (STRIDE). Before any endpoint ships, ask six questions — one per STRIDE category [GH21]:

- **Spoofing:** can a caller impersonate another principal? (Control: authentication on every request; no anonymous writes.)
- **Tampering:** can a caller modify data in transit or at rest? (Control: TLS everywhere; integrity checks on stored payloads.)
- **Repudiation:** can an actor deny an action? (Control: immutable audit logs with actor identity.)
- **Information disclosure:** can a caller read data beyond its authorization? (Control: object-level authorization on every resource fetch; minimal response fields.)
- **Denial of service:** can a caller exhaust capacity? (Control: rate limits, quotas, payload-size caps.)
- **Elevation of privilege:** can a low-privilege caller perform high-privilege operations? (Control: server-side role checks; never trust client-supplied roles.)

Chapter 17 turns each row into executable tests.

Ethics and responsible use. APIs move power: they decide whose data flows where, at what price, under whose surveillance. The professional baseline adopted throughout this book: collect the minimum data the operation requires; never log credentials, tokens, or personal data; honor the provider’s terms and the user’s consent when consuming; publish honest deprecation timelines when producing; and treat synthetic-data discipline (never real PII in examples, tests, or datasets) as non-negotiable. All datasets in this curriculum are fictional, set in the Northwind Robotics universe.

1.2.10 Scope Boundaries, Assumptions, and Risks

Scope boundaries. This book teaches API engineering, not: frontend frameworks, database administration, Kubernetes cluster operation (Chapter 18 uses it, does not teach it), language-agnostic theory for its own sake, or vendor-specific cloud certifications. Where a vendor service is the industry-standard reference (e.g. a managed gateway), we name it but implement with local, offline equivalents.

Assumption register. Assumptions the rest of the book makes, stated once so later chapters can build on them:

- A1.** Python 3.11+ on Windows with PowerShell; commands are PowerShell-native.
- A2.** All network dependencies are simulated locally; no cloud accounts or API keys are available or required.
- A3.** Readers control a machine where they may open localhost ports and install packages into a virtual environment.
- A4.** Synthetic data only; the Northwind Robotics datasets in `datasets\` are the canonical fixtures.
- A5.** Latency claims are always percentile claims; no claim is made from averages alone.

Risk register (top entries).

Success criteria template. Every chapter closes with measurable criteria of this form: *Given (prerequisites), when (procedure), then (observable outcome with a number)*. Example for this chapter: “Given a fresh `venv`, when the quick-start lab is executed per Section 1.5, then the client completes 200 measured requests with zero errors and reports p50, p95, and p99, and the learner can state which latency stage dominates on localhost.”

Terminology normalization. To keep twenty-one chapters unambiguous: *endpoint* means one operation on one path; *resource* means a domain entity addressable by URI; *representation* means a serialized form of a resource; *consumer/provider* are roles, not machines (one service is often both); *error* is any non-success outcome; *fault* is a defect that causes errors; *failure* is user-visible loss of service. Latency is always client-observed unless marked *server-side*.

1.3 Production-Ready Code Implementation

This section installs the two disciplines every later chapter assumes: a *reproducible environment* and a *reproducible measurement protocol*. All three listings are complete, typed, `stdlib`-only, and offline.

1.3.1 Environment Setup Baseline

Minimal local stack (used from Chapter 1). Python 3.11+, a virtual environment, and the packages in this chapter’s appendix. From the repository root in PowerShell:

```
1 python -m venv .venv
2 .\venv\Scripts\Activate.ps1
3 python -m pip install --upgrade pip
4 pip install -r requirements.txt
```

If activation is blocked by policy, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, or invoke `.\venv\Scripts\python.exe` directly without activating.

Cloud-ready stack (used from Chapter 17). The local stack plus Docker Desktop (for container labs), a local Kubernetes distribution, and a tunnel-free mindset: all “cloud” deployments in this book target local clusters and local registries so the exercises remain offline. The cloud-ready stack adds no Python dependencies beyond the minimal stack.

Sanity check. Run Listing 1.3.1 once per machine and once after any environment change. It verifies the interpreter version, importability of required packages, and availability of the localhost ports used by the labs — and it never touches the network beyond loopback.

```
1 """Environment sanity check for the API curriculum(
   offline, stdlib-only).
2
3 Verifies: interpreter version, required packages,
   localhost port
4 availability, and writes a structured report via
   logging.
5 Run: python sanity_check.py
6 Exit code 0 means the environment is ready for every
   lab in the book.
7 """
8 from __future__ import annotations
9
10 import importlib.util
11 import logging
12 import socket
13 import sys
14 from dataclasses import dataclass, field
15
16 MIN_PYTHON: tuple[int, int] = (3, 11)
```

```
17 REQUIRED_PACKAGES: tuple[str, ...] = (  
18     "fastapi", "uvicorn", "httpx", "pydantic", "pytest"  
19 )  
20 REQUIRED_PORTS: tuple[int, ...] = (8000, 8001, 8765)  
21  
22 logging.basicConfig(  
23     level=logging.INFO,  
24     format="%(levelname)-7s %(message)s",  
25 )  
26 logger = logging.getLogger("sanity_check")  
27  
28  
29 @dataclass  
30 class CheckReport:  
31     """Accumulates named check results for a final  
32     verdict."""  
33  
34     failures: list[str] = field(default_factory=list)  
35  
36     def record(self, name: str, ok: bool, detail: str)  
37         -> None:  
38         status = "PASS" if ok else "FAIL"  
39         logger.info("[%s] %s: %s", status, name, detail  
40             )  
41         if not ok:  
42             self.failures.append(name)  
43  
44  
45     def check_python_version(report: CheckReport) -> None:  
46         current = sys.version_info[:2]  
47         ok = current >= MIN_PYTHON  
48         detail = f"found {current[0]}.{current[1]}, require  
49             >={MIN_PYTHON[0]}.{MIN_PYTHON[1]}"  
50         report.record("python-version", ok, detail)  
51  
52  
53     def check_packages(report: CheckReport) -> None:  
54         for package in REQUIRED_PACKAGES:  
55             found = importlib.util.find_spec(package) is  
56                 not None  
57             detail = "importable" if found else "NOT  
58                 INSTALLED"  
59             report.record(f"package:{package}", found,  
60                 detail)
```

```

55
56 def check_port_available(port: int) -> tuple[bool, str
    ]:
57     """A port is 'available' if we can bind it on
        loopback right now."""
58     try:
59         with socket.socket(socket.AF_INET, socket.
            SOCK_STREAM) as probe:
60             probe.setsockopt(socket.SOL_SOCKET, socket.
                SO_REUSEADDR, 1)
61             probe.bind(("127.0.0.1", port))
62             return True, "free"
63     except OSError as exc:
64         return False, f"unavailable_{exc.strerror}_or_
            exc}"
65
66
67 def check_ports(report: CheckReport) -> None:
68     for port in REQUIRED_PORTS:
69         ok, detail = check_port_available(port)
70         report.record(f"port:{port}", ok, detail)
71
72
73 def main() -> int:
74     report = CheckReport()
75     check_python_version(report)
76     check_packages(report)
77     check_ports(report)
78     if report.failures:
79         logger.error("Environment NOT ready; failed_
            checks: %s",
80                     ", ".join(report.failures))
81         return 1
82     logger.info("Environment ready for all labs.")
83     return 0
84
85
86 if __name__ == "__main__":
87     raise SystemExit(main())

```

1.3.2 Evaluation Primer: Measuring Latency Correctly

Why averages lie. Latency distributions of network services are heavy-tailed: most requests are fast, a few are catastrophically slow, and the tail

is where user pain lives. The arithmetic mean of a heavy-tailed distribution is dominated by rare outliers in small samples and hides them in large ones — either way it describes no actual user’s experience. The standard remedy is *percentiles*: the value below which a given fraction of observations falls. We always report three:

Definition 1.2 (Latency percentile). Given n sorted latency samples $t_{(1)} \leq t_{(2)} \leq \dots \leq t_{(n)}$, the q -th percentile p_q is $t_{(\lceil qn/100 \rceil)}$. Thus p_{50} (the median) is the typical request, p_{95} is what the slowest 5% experience at best, and p_{99} exposes the tail that averages conceal.

Throughput vs. latency. Throughput λ (requests/second) and latency L are not independent. By Little’s Law, concurrency $C = \lambda \cdot L$; as load approaches capacity, queueing theory predicts latency grows non-linearly even while throughput still rises. Benchmark claims must therefore state the offered load: “p99 = 45 ms at 200 rps” is a measurement; “p99 = 45 ms” is marketing. Throughout this book, benchmarks pin the load first.

Minimal benchmark protocol. Every performance claim in this book is produced by this protocol:

1. **Warmup:** issue 50 unmeasured requests to stabilize caches, JIT-like effects, and connection pools.
2. **Measure:** issue $n \geq 200$ sequential requests, timing each with `time.perf_counter_ns()` on the client.
3. **Record:** keep raw samples; never aggregate before storing.
4. **Report:** p50, p95, p99, max, error count, achieved throughput, and the exact configuration (variant V1–V4, payload size, machine class).
5. **Repeat:** results quoted in prose are medians of 3 runs.

Warning

Two measurement sins invalidate benchmarks silently. First, measuring with `time.time()` or `datetime.now()`: wall clocks can jump backwards under NTP corrections; always use a monotonic clock (`time.perf_counter_ns`). Second, timing server-side only: the user experiences the client-observed latency, which includes queueing and network — server logs systematically under-report it.

The benchmark harness. Listing 1.3.2 is the harness used (and extended) throughout the book. It is transport-agnostic: pass any callable and it returns a typed `BenchmarkResult`. It deliberately uses only the standard library so it runs in every chapter’s environment.

```

1  """Reusable latency benchmark harness (stdlib-only,
   deterministic protocol).
2
3  Implements the book's minimal benchmark protocol:
   warmup, timed samples via
4  a monotonic clock, raw-sample retention, percentile
   reporting. No network
5  code lives here; the caller supplies a 'send' callable
   so the same harness
6  measures urllib clients, httpx clients, or in-process
   fakes.
7
8  Run the built-in self-test: python bench.py --selftest
9  """
10 from __future__ import annotations
11
12 import argparse
13 import logging
14 import math
15 import statistics
16 import time
17 from collections.abc import Callable
18 from dataclasses import dataclass, field
19
20 logger = logging.getLogger("bench")
21
22 MIN_SAMPLES = 200
23 DEFAULT_WARMUP = 50
24
25
26 class BenchmarkError(RuntimeError):
27     """Raised when the benchmark configuration is
   invalid."""
28
29
30 @dataclass(frozen=True)
31 class BenchmarkResult:
32     """Immutable outcome of one benchmark run."""
33
34     samples_ms: tuple[float, ...]
35     errors: int

```

```

36     wall_seconds: float
37     label: str = ""
38
39     @property
40     def n(self) -> int:
41         return len(self.samples_ms)
42
43     @property
44     def throughput_rps(self) -> float:
45         if self.wall_seconds <= 0.0:
46             raise BenchmarkError("wall_time_must_be_
47                                     positive")
48         return (self.n + self.errors) / self.
49                 wall_seconds
50
51     def percentile(self, q: float) -> float:
52         """Nearest-rank_percentile_per_the_book's_
53             Definition."""
54         if not self.samples_ms:
55             raise BenchmarkError("no_samples_collected"
56                                 )
57         if not 0.0 < q <= 100.0:
58             raise BenchmarkError(f"q_must_be_in_(0,
59                                   100],_got_{q}")
60         ordered = sorted(self.samples_ms)
61         rank = max(1, math.ceil(q * len(ordered) /
62                                100.0))
63         return ordered[rank - 1]
64
65     def summary(self) -> dict[str, float | int | str]:
66         return {
67             "label": self.label,
68             "n": self.n,
69             "errors": self.errors,
70             "p50_ms": round(self.percentile(50), 3),
71             "p95_ms": round(self.percentile(95), 3),
72             "p99_ms": round(self.percentile(99), 3),
73             "max_ms": round(max(self.samples_ms), 3),
74             "throughput_rps": round(self.throughput_rps
75                                     , 1),
76         }
77
78 def run_benchmark(
79     send: Callable[[], None],
80     *,

```

```

75     samples: int = MIN_SAMPLES,
76     warmup: int = DEFAULT_WARMUP,
77     label: str = "",
78 ) -> BenchmarkResult:
79     """Execute the warmup+measured-runs protocol
80     against 'send'.
81
82     'send' must perform exactly one request and raise
83     on failure; raised
84     exceptions are counted as errors and excluded from
85     latency samples.
86
87     """
88     if samples < MIN_SAMPLES:
89         raise BenchmarkError(
90             f"samples must be >={MIN_SAMPLES} for
91             stable percentiles")
92     if warmup < 0:
93         raise BenchmarkError("warmup must be >=0")
94
95     for _ in range(warmup):
96         try:
97             send()
98         except Exception: # noqa: BLE001 -- warmup
99             failures are tolerated
100             logger.debug("warmup request failed",
101                           exc_info=True)
102
103     timings: list[float] = []
104     errors = 0
105     start_ns = time.perf_counter_ns()
106     for _ in range(samples):
107         t0 = time.perf_counter_ns()
108         try:
109             send()
110         except Exception: # noqa: BLE001 -- errors are
111             data, not crashes
112             errors += 1
113             logger.debug("measured request failed",
114                           exc_info=True)
115             continue
116         timings.append((time.perf_counter_ns() - t0) /
117                        1_000_000.0)
118     wall_seconds = (time.perf_counter_ns() - start_ns)
119                   / 1_000_000_000.0
120
121     return BenchmarkResult(

```

```

111         samples_ms=tuple(timings), errors=errors,
112         wall_seconds=wall_seconds, label=label,
113     )
114
115
116 def _selftest() -> int:
117     """Deterministic offline self-test against an in-
118         process fake."""
119     state = {"calls": 0}
120
121     def fake_send() -> None:
122         state["calls"] += 1
123         time.sleep(0.0005) # deterministic-ish work,
124                             # no wall-clock asserts
125         if state["calls"] % 97 == 0:
126             raise ConnectionError("simulated transient
127                                     failure")
128
129     result = run_benchmark(fake_send, samples=200,
130                             warmup=10,
131                             label="selftest")
132     report = result.summary()
133     for key, value in report.items():
134         logger.info("%-16s %s", key, value)
135     assert result.n + result.errors == 200, "sample
136         accounting broken"
137     assert report["p50_ms"] <= report["p95_ms"] <=
138         report["p99_ms"]
139     assert result.errors > 0, "expected simulated
140         errors"
141     logger.info("selftest OK")
142     return 0
143
144 if __name__ == "__main__":
145     logging.basicConfig(level=logging.INFO, format="%(
146         levelname)-7s %(message)s")
147     parser = argparse.ArgumentParser(description=
148         __doc__)
149     parser.add_argument("--selftest", action="
150         store_true")
151     args = parser.parse_args()
152     if args.selftest:
153         raise SystemExit(_selftest())
154     parser.error("pass --selftest (this module is a
155         library otherwise)")

```

Note the deliberate design choices: errors are *counted*, *not raised*, because a benchmark that crashes on the first failure measures nothing; exceptions are caught broadly only at this boundary, logged with `exc_info`, and converted into data; and the self-test asserts ordering invariants ($p_{50} \leq p_{95} \leq p_{99}$) rather than absolute timings, keeping it deterministic and machine-independent.

1.3.3 A Complete Offline API and Client in the Standard Library

Before frameworks, you must see the primitives. Listing 1.3.3 implements a working JSON API with `http.server` — the Northwind Robotics parts catalog — plus a client built on `urllib`. It enforces the disciplines this book demands everywhere: explicit timeouts on every call, structured JSON errors, input validation with specific exceptions, and logging instead of `print`. Chapter 7 will replace the plumbing with FastAPI; the disciplines remain.

```

1  """Northwind_Robotics_parts_API+_client,_stdlib-only_
   and_fully_offline.
2
3  Server: GET_/healthz->{"status":_ok
   "}
4  GET_/v1/parts->{"items":_
   [...]}
5  GET_/v1/parts/<sku>->part_object_or
   _RFC-shaped_error
6  Client: typed_functions_with_explicit_timeouts_and_
   error_mapping.
7
8  Run_server: python_mini_api.py_serve--port_8000
9  Run_client_demo(spawns_server_in-process): python_
   mini_api.py_demo
10 """
11 from __future__ import annotations
12
13 import argparse
14 import json
15 import logging
16 import threading
17 import urllib.error
18 import urllib.request
19 from dataclasses import dataclass
20 from http.server import BaseHTTPRequestHandler,
   ThreadingHTTPServer

```

```
21 from typing import Any
22
23 logger = logging.getLogger("mini_api")
24
25 DEFAULT_HOST = "127.0.0.1"
26 DEFAULT_PORT = 8000
27 CLIENT_TIMEOUT_S = 2.0 # every network call in this
    book carries a timeout
28
29 # Synthetic fixture data (Northwind Robotics universe).
    No real PII.
30 PARTS: dict[str, dict[str, Any]] = {
31     "NR-AXLE-001": {"sku": "NR-AXLE-001", "name": "
        Drive_axle,Mk_IV",
32                     "unit_price_cents": 4599, "stock":
                        37},
33     "NR-LIDAR-204": {"sku": "NR-LIDAR-204", "name": "
        LiDAR_puck,16-channel",
34                     "unit_price_cents": 189900, "stock":
                        4},
35     "NR-GRIP-011": {"sku": "NR-GRIP-011", "name": "
        Parallel_gripper_jaw",
36                     "unit_price_cents": 12350, "stock":
                        120},
37 }
38
39
40 def problem(status: int, title: str, detail: str) ->
    bytes:
41     """RFC_9457-flavored_error_body(formalized_in_
        Chapter_7)."""
42     body = {
43         "type": "about:blank",
44         "title": title,
45         "status": status,
46         "detail": detail,
47     }
48     return json.dumps(body).encode("utf-8")
49
50
51 class PartsHandler(BaseHTTPRequestHandler):
52     server_version = "NorthwindParts/0.1"
53
54     def log_message(self, fmt: str, *args: Any) -> None
        :
55         logger.info("%s-%s", self.address_string(),
```

```

        fmt % args)

56
57 def _send_json(self, status: int, payload: bytes)
    -> None:
58     self.send_response(status)
59     self.send_header("Content-Type", "application/
        json")
60     self.send_header("Content-Length", str(len(
        payload)))
61     self.end_headers()
62     self.wfile.write(payload)
63
64 def do_GET(self) -> None: # noqa: N802 -- stdlib
    naming contract
65     if self.path == "/healthz":
66         self._send_json(200, json.dumps({"status":
            "ok"}).encode())
67         return
68     if self.path == "/v1/parts":
69         payload = json.dumps({"items": list(PARTS.
            values())}).encode()
70         self._send_json(200, payload)
71         return
72     if self.path.startswith("/v1/parts/"):
73         sku = self.path.removeprefix("/v1/parts/").
            strip()
74         part = PARTS.get(sku)
75         if part is None:
76             self._send_json(404, problem(
77                 404, "Not_Found", f"no_part_with_
                    sku_{sku}"))
78             return
79         self._send_json(200, json.dumps(part).
            encode())
80         return
81     self._send_json(404, problem(
82         404, "Not_Found", f"unknown_path_{self.
            path}"))
83
84
85 def make_server(host: str = DEFAULT_HOST,
86                 port: int = DEFAULT_PORT) ->
    ThreadingHTTPServer:
87     return ThreadingHTTPServer((host, port),
        PartsHandler)
88

```

```

89
90 # ----- client side
91 -----
92
93 class ApiClientError(RuntimeError):
94     """Base class for client-side failures."""
95
96 class ApiHttpError(ApiClientError):
97     """The server answered with a non-2xx status."""
98
99     def __init__(self, status: int, detail: str) ->
100         None:
101         super().__init__(f"HTTP{status}: {detail}")
102         self.status = status
103         self.detail = detail
104
105 @dataclass(frozen=True)
106 class PartsClient:
107     """Minimal typed client for the parts API."""
108
109     base_url: str = f"http://{DEFAULT_HOST}:{
110         DEFAULT_PORT}"
111     timeout_s: float = CLIENT_TIMEOUT_S
112
113     def _get_json(self, path: str) -> Any:
114         if not path.startswith("/"):
115             raise ValueError(f"path must start with
116                 '/' , got {path!r}")
117         request = urllib.request.Request(self.base_url
118             + path, method="GET")
119         try:
120             with urllib.request.urlopen(request,
121                 timeout=self.timeout_s)
122                 as resp:
123                 return json.loads(resp.read().decode("
124                     utf-8"))
125         except urllib.error.HTTPError as exc:
126             detail = exc.read().decode("utf-8", errors=
127                 "replace")
128             raise ApiHttpError(exc.code, detail) from
129                 exc
130         except (urllib.error.URLError, TimeoutError,
131             OSError) as exc:

```



```

124         raise ApiClientError(f"transport_failure:_{exc}") from exc
125
126     def health(self) -> bool:
127         return self._get_json("/healthz")["status"] == "ok"
128
129     def list_parts(self) -> list[dict[str, Any]]:
130         return list(self._get_json("/v1/parts")["items"])
131
132     def get_part(self, sku: str) -> dict[str, Any]:
133         if not sku:
134             raise ValueError("sku_must_be_non-empty")
135         return dict(self._get_json(f"/v1/parts/{sku}"))
136
137
138     def demo() -> int:
139         """Start the server in-process and exercise the client. Offline."""
140         server = make_server()
141         thread = threading.Thread(target=server.serve_forever, daemon=True)
142         thread.start()
143         try:
144             client = PartsClient()
145             logger.info("health:_%s", client.health())
146             parts = client.list_parts()
147             logger.info("catalog_size:_%d", len(parts))
148             part = client.get_part("NR-LIDAR-204")
149             logger.info("fetched:_%s_(%d_cents)",
150                         part["name"], part["unit_price_cents"])
151
152             try:
153                 client.get_part("NO-SUCH-SKU")
154             except ApiHttpError as exc:
155                 logger.info("expected_404_mapped_cleanly:_status=%d", exc.status)
156
157         finally:
158             server.shutdown()
159             server.server_close()
160             thread.join(timeout=2.0)
161         return 0
162
163     def main() -> int:

```

```

163     logging.basicConfig(level=logging.INFO,
164                         format="%(levelname)-7s%(
165                             message)s")
166     parser = argparse.ArgumentParser(description=
167         __doc__)
168     sub = parser.add_subparsers(dest="command",
169                                 required=True)
170     serve = sub.add_parser("serve", help="run the
171         server in the foreground")
172     serve.add_argument("--port", type=int, default=
173         DEFAULT_PORT)
174     sub.add_parser("demo", help="in-process server +
175         client round trip")
176     args = parser.parse_args()
177
178     if args.command == "serve":
179         if not 1024 <= args.port <= 65535:
180             parser.error("port must be in [1024, 65535]
181                 ")
182             server = make_server(port=args.port)
183             logger.info("serving on http://%s:%d",
184                 DEFAULT_HOST, args.port)
185             try:
186                 server.serve_forever()
187             except KeyboardInterrupt:
188                 logger.info("shutting down")
189             finally:
190                 server.server_close()
191             return 0
192     return demo()
193
194 if __name__ == "__main__":
195     raise SystemExit(main())

```

Key Concept

Every listing in this book obeys five rules, visible already in these three: (1) explicit timeouts on every network call; (2) typed function signatures and dataclasses; (3) specific exceptions with chained causes (`raise ... from exc`); (4) logging instead of `print`; (5) offline determinism — no listing requires the internet, a cloud account, or an API key.

1.4 Common Pitfalls & Anti-Patterns

These are the failure patterns this book's authors see most often in code reviews and incident post-mortems. Each is stated with its detection signal and its fix; the chapters referenced contain the full treatment.

1.4.1 Consumer-Side Pitfalls

Missing timeouts (the cardinal sin). Any network call without a timeout can block *forever*; TCP's default retransmission behavior happily waits minutes. Detection: thread dumps full of blocked I/O threads; fix: a connect *and* read timeout on every call (Section 1.5.2 shows what happens otherwise).

Retrying everything, immediately, forever. Unbounded retries turn a partial outage into a self-inflicted denial of service. Fix: bounded retries with exponential backoff and jitter, only on retryable (idempotent or explicitly safe) operations (Chapters 3, 11, 15).

Parsing errors by string-matching. Matching on "something went wrong" in a response body is a contract bug waiting to be renamed. Fix: branch on status codes and machine-readable error types (Chapter 8).

Ignoring the contract until runtime. Discovering a renamed field in production is a process failure. Fix: schema validation in CI (Chapters 5, 20).

Secrets in source or logs. API keys committed to git or printed in debug logs are the most common real-world API breach vector. Fix: environment variables, secret stores, log redaction (Chapter 6).

1.4.2 Provider-Side Pitfalls

Leaky abstraction responses. Returning stack traces, ORM errors, or internal hostnames in error bodies leaks your architecture to attackers. Fix: RFC 9457 problem details with safe, curated fields.

Breaking changes in place. "It's an internal API" is how distributed monoliths are born; consumers couple silently. Fix: version every contract and automate compatibility checks (Chapter 10).

Offset pagination on mutating collections. Items inserted while a consumer pages shift every window, duplicating or skipping records. Fix: cursor-based pagination (Chapter 9).

200 OK with an error inside. Wrapping failures in success envelopes defeats HTTP intermediaries, caches, and every monitoring tool. Fix: correct status codes; bodies carry detail, statuses carry semantics.

No resource limits. Unbounded page sizes, payload sizes, and query complexity are invitations to accidental (or deliberate) denial of service. Fix: caps plus rate limiting (Chapter 15).

1.4.3 Organizational Anti-Patterns

The API as afterthought. Designing the UI first and exposing whatever it needed produces chatty, anemic APIs. Fix: contract-first design reviews.

Docs as a wiki page. Unversioned prose documentation decays within weeks. Fix: machine-readable contracts as the source of truth, docs generated from them [Ope21].

No owner. An API owned by “the team” is owned by no one; deprecation, incidents, and consumer questions all stall. Fix: a named owner per contract (governance baseline, Section 1.2.9).

1.4.4 Common Myths About APIs

“If it returns JSON over HTTP, it is REST.” False. REST is a set of architectural constraints [Fie00]; most JSON-over-HTTP APIs are RPC-flavored HTTP interfaces. That is often fine — but call things by their names, because the constraints you actually honor determine which guarantees (cacheability, statelessness) you may assume.

“HTTP 200 means it worked.” It means the server produced a response it considers successful. Semantic failures ride inside 200 bodies all the time; monitoring that counts only status codes is blind to them.

“Internal APIs don’t need contracts or versioning.” Internal consumers are still consumers, and they couple silently. The blast radius of an unversioned internal API is often *larger* than a public one because nobody tracks who depends on it.

“Timeouts make systems slower.” Timeouts cap the blast radius of slow dependencies; the slow path exists whether or not you bound it. The unbounded version is strictly worse (Section 1.5.2).

“Retries fix flakiness.” Retries fix *transient transport* failures and amplify everything else: they multiply load during outages and duplicate

non-idempotent effects. A retry policy is a designed artifact, not a default.

“The gateway will handle security.” Gateways terminate TLS and enforce coarse policies; they cannot fix broken object-level authorization or injection inside business logic. Security is layered, and the innermost layer is your code (Chapter 17).

“Average latency is a fine health metric.” Averages hide the tail where user pain lives. Percentiles under stated load, or it did not happen (Section 1.3.2).

“Documentation keeps itself up to date.” Only generated artifacts stay honest. Hand-written prose decays; machine-readable contracts with docs rendered from them do not [Ope21].

1.4.5 Common Mistakes & Debugging Gallery

A field guide to symptoms you will meet in the labs:

1.5 Hands-On Production Lab & Real-World Case Study

1.5.1 Quick-Start Lab: Build, Run, Measure (2–3 Hours, Offline)

Objective. Stand up the reference stack at variant V1+V2, run the benchmark protocol against it, and produce your first honest percentile report.

Prerequisites. Sanity check green (Listing 1.3.1).

Step 1 — Environment (15 min). Create and activate the virtual environment and install dependencies exactly as in Section 1.3.1. Run the sanity check and keep its output; labs in later chapters assume it.

Step 2 — Save the code (15 min). Save Listing 1.3.3 as `mini_api.py` and Listing 1.3.2 as `bench.py` in a lab folder. Verify the harness self-test:

```
1 python bench.py --selftest
```

Expect the summary keys (`p50_ms`, `p95_ms`, `p99_ms`, `errors`, `throughput_rps`) and a final `selftest OK`.

Step 3 — Run the server (10 min). In one PowerShell window:

```
1 python mini_api.py serve --port 8000
```

In a second window, smoke-test with the stdlib client:

```
1 python mini_api.py demo
```

Step 4 — Measure (45 min). Write `lab0_measure.py` that imports both modules and measures the real client against the real server:

```
1 """Lab0 measurement driver: benchmark the stdlib
   client end to end."""
2 from __future__ import annotations
3
4 import logging
5 import threading
6
7 from bench import run_benchmark
8 from mini_api import PartsClient, make_server
9
10 logger = logging.getLogger("lab0")
11
12
13 def main() -> int:
14     logging.basicConfig(level=logging.INFO,
15                         format="%(levelname)-7s%(
16                             message)s")
17     server = make_server(port=8000)
18     thread = threading.Thread(target=server.
19                               serve_forever, daemon=True)
20     thread.start()
21     try:
22         client = PartsClient(base_url="http
23                               ://127.0.0.1:8000")
24         result = run_benchmark(
25             lambda: client.get_part("NR-AXLE-001"),
26             samples=500, warmup=50, label="stdlib-local
27             ",
28         )
29         for key, value in result.summary().items():
30             logger.info("%-16s%s", key, value)
31     if result.errors != 0:
32         logger.error("expected zero errors on
33                       localhost, got %d",
34                       result.errors)
35     return 1
```

```

31     finally:
32         server.shutdown()
33         server.server_close()
34         thread.join(timeout=2.0)
35     return 0
36
37
38 if __name__ == "__main__":
39     raise SystemExit(main())

```

Step 5 — Analyze (45 min). Answer in writing: (a) Which latency stage of Equation (1.1) dominates on localhost, and why does L_{TLS} not appear? (b) What is the ratio p_{99}/p_{50} ? What does a ratio above 3 suggest? (c) Repeat with the server under a second concurrent client; how do p_{50} and p_{99} move relative to each other?

Step 6 — Break it deliberately (30 min). Introduce one controlled fault at a time and record which taxonomy category (Table 1.2) each produces: stop the server mid-benchmark (transport); request a nonexistent SKU and count 404s (contract/semantic boundary — argue which); remove the client timeout and kill the server before answering (transport, and observe the hang); send a malformed path (contract). This fault-injection habit is the seed of Chapter 19’s chaos testing.

Acceptance criteria.

- ☐ Sanity check reports all PASS.
- ☐ 500-request benchmark completes with zero errors.
- ☐ Written answers name the dominant latency stage and the p_{99}/p_{50} ratio.
- ☐ Fault-injection table maps each induced failure to a taxonomy category.

1.5.2 Production War Story: The Missing Timeout

Names are fictionalized; the failure pattern is one of the most common in the industry [Nyg18].

Background. Northwind Robotics’ fleet dashboard (consumer) called the inventory service (provider) over HTTP to render each robot’s parts manifest. The client library defaulted to *no timeout* — “we’ll add timeouts

later” had been on the backlog for two quarters. The inventory service, in turn, depended on a catalog database.

Trigger. At 14:07 on a Tuesday, a routine database failover made catalog queries slow: not failing, just slow — 30 seconds instead of 30 milliseconds. Slow failures are the dangerous kind; nothing alarms because nothing has “failed.”

Cascade. Inventory-service workers blocked on catalog queries. Its thread pool (64 threads) exhausted in under a minute, so inventory requests from the dashboard queued, then blocked. Dashboard server workers, holding open requests with no timeout, blocked in turn; its pool (128 threads) exhausted four minutes later. The load balancer, seeing all dashboard workers busy, began returning 503s for *every* route — including `/healthz` and the static marketing pages that had no dependency on inventory at all. By 14:19, a slow SELECT statement had taken down the entire customer-facing site. Retry logic in a third system (the mobile app’s aggressive immediate-retry policy) tripled the offered load during the incident, deepening the queue.

Detection and recovery. The on-call engineer’s percentile dashboards told the story in one glance: p50 dashboard latency flat until 14:07, then p99 vertical. Recovery was manual thread-pool drainage via rolling restarts; the catalog failover completed on its own at 14:31. Total customer-visible outage: 24 minutes — more than half a 99.9% monthly error budget (Equation (1.2)) spent in one afternoon.

Root cause and fixes. The post-mortem classified the incident across three taxonomy categories: an *operational* trigger (slow dependency), amplified by a *transport* defect (missing timeouts — no failure semantics defined between the systems), worsened by a *semantic* one (immediate unbounded retries). Fixes, in order of leverage: (1) connect and read timeouts on every call (500 ms / 2 s); (2) bounded retries with exponential backoff and jitter on idempotent reads only; (3) bulkheads — separate thread pools per dependency so one slow dependency cannot starve unrelated routes; (4) circuit breakers to fail fast while the dependency recovers; (5) a load test reproducing the scenario in CI. Chapters 3, 11, 15, and 16 implement each fix; Chapter 19 builds the dashboards that caught it.

Warning

The default timeout of most HTTP client libraries is *infinite* or absurdly long. Treat any client configuration without an explicit timeout as a Sev-2 defect. There is no safe default; there is only your

latency budget.

1.5.3 Diagnostic Playbook: “The API Is Slow” / “The API Is Broken”

When the page arrives, run this sequence before forming theories:

1. **Define the symptom precisely.** Which operation, which consumers, since when? Get a failing request ID or a reproducible call. “Slow” and “broken” are different playbooks from here.
2. **Reproduce from a neutral client.** A single measured call from outside the complaining system (your Chapter 2 client) isolates consumer-side causes.
3. **Classify into the taxonomy.** Transport (cannot connect, TLS errors, 502s)? Contract (400/422, schema mismatches)? Semantic (200 with wrong data)? Operational (latency tail, saturation)? Security (401/403, blocked IPs)? Table 1.2 maps each to its detection strategy.
4. **Localize with the budget.** Decompose latency per Equation (1.1); the stage that exceeds its budget is your suspect. For errors, walk the request path hop by hop.
5. **Check the tail, not the mean.** p99 first; if p99 is fine, the complaint is about a specific consumer segment, not the fleet.
6. **Form one hypothesis, test one change.** Change a single variable, re-measure with the benchmark protocol, keep the raw samples.
7. **Write it down.** Every incident ends with the taxonomy classification, the fix, and the test that prevents recurrence.

1.6 Self-Assessment Exercises & Deep-Dive Questions

1.6.1 Readiness Self-Assessment

Score yourself honestly (0 = no idea, 1 = heard of it, 2 = can do it) before starting Chapter 1. Total ≥ 14 means you are ready; any 0 identifies a topic to shore up first.

1.6.2 Exercises

- E0.1. [junior] Classify the incidents.** For each: (a) a proxy returns 502 during a deploy; (b) a field changes type from string to number; (c) prices are returned in the wrong currency with HTTP 200; (d) p99 triples every day at batch-job time; (e) a customer reads another customer's invoice by incrementing an ID. Map each to the taxonomy and name its detection strategy.
- E0.2. [junior] Percentiles by hand.** Given sorted latencies (ms): 4, 5, 5, 6, 7, 9, 12, 18, 40, 220 — compute p_{50} , p_{90} , the mean, and explain which single number you would put on a dashboard.
- E0.3. [junior] Extend the mini API.** Add GET `/v1/parts?in_stock=true` filtering to Listing 1.3.3, returning a 400 problem body for unknown query parameters. Keep it stdlib-only and typed.
- E0.4. [mid] Error budget arithmetic.** Northwind's fleet API has a 99.95% monthly SLO. Compute the downtime budget in minutes for a 30-day month, then convert it into “allowed 5xx responses per million requests.” Which unit is easier to alert on, and why?
- E0.5. [mid] Harden the benchmark.** Modify Listing 1.3.2 to interleave three payload sizes in a fixed, seeded pattern and report per-size percentiles. Why must the pattern be seeded rather than random?
- E0.6. [mid] Maturity audit.** Take the last API you worked on (or Listing 1.3.3) and score it L0–L4. List the three cheapest changes that would raise it one level.
- E0.7. [senior] Budget decomposition.** Instrument Listing 1.3.3 to timestamp request receipt, handler start, and response flush; compute a per-stage latency table over 500 requests and reconcile it with Equation (1.1). Which stage's budget would you tighten first?
- E0.8. [senior] Timeout theorem.** Using the war story's numbers, estimate the maximum request rate the dashboard could have sustained with 128 workers and 30-second dependency stalls. Then show how a 2-second timeout changes the answer (Little's Law, $C = \lambda L$).
- E0.9. [senior] STRIDE the mini API.** Produce a six-row STRIDE table for Listing 1.3.3 as written. Which threats does the code actually control today? Which does it ignore?
- E0.10. [staff] Design the governance gate.** Draft the CI checks (as a

checklist, not code) that would keep every Northwind API at L2 minimum: what fails a build, what warns, and who can override. Compare with the addendum’s CI quality-gate template.

1.6.3 Deep-Dive Questions

- D0.1.** Definition 1.1 makes failure semantics part of the contract. Argue for and against treating a change in retryability guidance (“clients may now retry POST X”) as a breaking change.
- D0.2.** The maturity model is linear; real organizations are not. Describe a plausible system that is simultaneously L4 in governance and L0 in resilience, and the risk that combination creates.
- D0.3.** Little’s Law ($C = \lambda L$) assumes a stable system. Which assumptions break during a retry storm, and what does the law then mislead you about?
- D0.4.** Percentile latency SLOs can be gamed by shedding load (returning fast errors). Design an SLI definition that resists this gaming.
- D0.5.** Every lab in this book is offline. What does the offline constraint *prevent* you from learning, and how would you compensate before touching a real provider?

1.7 Interview Q&A Vault

Thirty-two questions, ordered junior → staff. Answers are intentionally concise; interviews reward precision over volume.

Q1. [junior] What is an API?

A versioned contract between provider and consumers specifying operations, representations, and failure semantics over a transport protocol (Definition 1.1). Not the implementation — the agreement.

Q2. [junior] What is the difference between a library and an API?

A library executes in your process and address space; an API crosses a process or network boundary, so it has transport, versioning, and partial-failure concerns a library never faces.

Q3. [junior] Name the parts of an HTTP request.

Request line (method, target, version), headers, optional body. The response mirrors this: status line, headers, optional body [Fie22a].

Q4. [junior] What do the HTTP status-code classes mean?

1xx informational, 2xx success, 3xx redirection, 4xx client fault, 5xx server fault. The class alone localizes responsibility before you read a single body byte.

Q5. [junior] GET vs. POST?

GET retrieves a representation and must be safe and idempotent; POST submits an operation the server defines, with no such guarantee. Caches and retries treat them differently because the semantics differ.

Q6. [junior] What is JSON and why did it win over XML for web APIs?

A text serialization of objects, arrays, numbers, strings, booleans, and null. It won on parsing simplicity, direct mapping to programming language data structures, and payload size — at the cost of no native schema or comment support, which Chapter 4 addresses.

Q7. [junior] What is a timeout and why does every call need one?

A bound on how long you wait for a connection or a response. Without it, a silent peer blocks your worker forever; at scale that exhausts pools and cascades (Section 1.5.2).

Q8. [junior] What is an idempotent operation?

One where performing it n times has the same server-side effect as performing it once. GET, PUT, and DELETE are specified idempotent; POST is not, which is exactly why idempotency keys exist (Chapter 11).

Q9. [junior] What is a status 404 vs. 410?

404: not found, possibly transient. 410: gone permanently, a deliberate deprecation signal. Returning the right one lets consumers automate their response.

Q10. [junior] What does TLS give an API connection?

Server authentication, confidentiality, and integrity in transit — and with mutual TLS, client authentication too. It does nothing for authorization or input validation.

Q11. [mid] What is REST, precisely?

An architectural style [Fie00]: client-server, stateless, cacheable, uniform interface (resources identified by URIs, manipulated through representations, self-descriptive messages, hypermedia), layered sys-

tem, optional code-on-demand. A JSON-over-HTTP CRUD endpoint is not automatically REST.

Q12. [mid] Explain the difference between authentication and authorization.

Authentication establishes *who* the caller is; authorization decides *what* that caller may do. The most damaging modern API flaw — broken object-level authorization — is an authenticated caller reaching objects it should not, i.e. an authorization failure.

Q13. [mid] Why do averages mislead for latency? What do you report instead?

Latency is heavy-tailed; the mean describes no real user and hides the tail where pain concentrates. Report percentiles: p50 for the typical experience, p95/p99 for the tail, always with sample size and offered load.

Q14. [mid] What is an API contract test?

A test that verifies provider behavior against the published contract (schema, status codes, error shapes), often generated from an OpenAPI document, and consumer tests that pin the fields the consumer actually uses. They run in CI so a breaking change fails a build, not a customer.

Q15. [mid] Your consumer gets HTTP 429. What happened and what do you do?

The provider rate-limited you. Read `Retry-After`, back off with exponential delay and jitter, reduce concurrency, and if the limit is chronic, negotiate quota or cache more. Never retry immediately (Chapter 15).

Q16. [mid] What is connection pooling and why does it matter?

Reusing TCP/TLS connections across requests. Handshakes cost round trips (budget stage L_{TLS}); pooling amortizes them, often the single largest latency win for chatty clients (Chapter 3).

Q17. [mid] When should you paginate, and offset vs. cursor?

Whenever a collection can outgrow one response. Offset pagination is simple but unstable under concurrent inserts and gets slower deeper in; cursor (keyset) pagination is stable and index-friendly. Default to cursors for large or mutating collections (Chapter 9).

Q18. [mid] What belongs in an error response body?

RFC 9457 fields: `type` (machine-readable URI), `title`, `status`,

detail, instance, plus curated extensions. Never stack traces, SQL, internal hostnames, or secrets.

Q19. [mid] Semantic versioning for APIs: what breaks what?

Removing or renaming fields, tightening types, changing semantics: breaking, new major. Adding optional fields or new operations: backward compatible, minor. Deprecating with a sunset date: policy, not schema (Chapter 10).

Q20. [mid] What is the N+1 problem in API composition?

One list call followed by n detail calls. It multiplies latency and load; fix with batch endpoints, embedded expansions, or GraphQL-style selection — chosen consciously, since each trades server complexity differently.

Q21. [senior] Design retries correctly. What is the full checklist?

(1) Only retry on retryable failures: transport errors, 429, 5xx — never 4xx semantics; (2) only idempotent operations, or add idempotency keys; (3) bounded attempts (typically 3); (4) exponential backoff with full jitter; (5) a retry budget so retries stay a small fraction of traffic; (6) respect **Retry-After**; (7) the total retry deadline must fit the caller's own latency budget.

Q22. [senior] Explain Little's Law and one operational use.

$C = \lambda L$: concurrency equals arrival rate times latency. Use: if a dependency slows from 30 ms to 30 s at 100 rps, in-flight requests grow from 3 to 3000 — you can compute the exact pool size at which you die, which is why timeouts and bulkheads are sized from this law.

Q23. [senior] What is a circuit breaker and what state machine does it run?

Closed (normal) $\rightarrow$ Open (failing fast) after an error threshold $\rightarrow$ Half-Open (probe with limited traffic) $\rightarrow$ Closed on success or Open on failure. It converts slow failures into fast ones, protecting both caller pools and the recovering dependency [Nyg18].

Q24. [senior] Error budget: what is it and how does it change behavior?

The allowed unreliability: $1 - \text{SLO}$. At 99.9% monthly, that is 43.2 minutes (Equation (1.2)). Budget remaining $\Rightarrow$ ship features; budget burning fast $\Rightarrow$ freeze releases, invest in reliability. It turns the reliability-vs-velocity argument into arithmetic [Bey+16].

Q25. [senior] How do you make a POST safely retryable?

Client generates an idempotency key per logical operation; server stores (key, response) with a TTL and replays the stored response on duplicate keys; unique constraints enforce exactly-once effects in storage. Watch the races: concurrent first-delivery and retry must serialize on the key (Chapter 11).

Q26. [senior] GraphQL vs. REST vs. gRPC: how do you choose?

REST: public contracts, caching, broad interoperability. GraphQL: many clients with divergent data needs, aggregating several backends, at the cost of query-complexity governance. gRPC: internal service-to-service, strict schemas, streaming, and low latency, at the cost of browser and tooling friction. Choose per consumer, not per fashion (Chapters 13–14).

Q27. [senior] What is backpressure and where does it appear in APIs?

A fast producer overwhelming a slow consumer. It appears as queue growth: in connection pools, in async streams (SSE/WebSockets), and in broker consumer lag. Controls: bounded queues, flow-control windows, 429/503 load shedding, and rate limiting upstream (Chapter 12).

Q28. [senior] Walk through a webhook delivery system’s failure modes.

Delivery is at-least-once, so consumers must be idempotent; ordering is not guaranteed; the consumer’s endpoint can be down (retry schedule with backoff, dead-letter after n attempts); payloads can be spoofed (sign with HMAC, verify before parsing); and slow consumer responses are backpressure on your fleet.

Q29. [staff] How would you design an API platform’s governance so teams don’t route around it?

Make the compliant path the fastest path: templates that pass linting by default, a catalog that gives discoverability in exchange for metadata, CI gates that are fast and actionable, and an explicit, time-boxed exception process. Governance that only says “no” gets bypassed; governance that ships value gets adopted (Chapter 21).

Q30. [staff] A partner integrates against your API and you must change a field’s meaning. Walk the full process.

You do not change meaning in place. Add the new field alongside the old; publish the deprecation with a sunset date; measure usage of the old field per consumer; contact stragglers through the developer

portal; dual-write/dual-read during transition; remove only after sunset with usage at zero. Contract thinking turns a breaking change into a managed migration (Chapters 10, 21).

- Q31. [staff] Your CEO asks for “five-nines.” What do you say?**
Five-nines is 5.26 minutes of downtime *per year*. Price it: multi-region active-active, chaos-tested failover, mature on-call, and accepting that any single dependency with a weaker SLO caps you. Show the marginal cost curve from 99.9 to 99.95 to 99.99 and let the business buy the point it needs — per user journey, not uniformly.
- Q32. [staff] How do you sunset a widely used public API with minimal ecosystem damage?**
Announce early with a date and migration guide; sunset headers on every response; staged brownouts (scheduled short outages) to flush out silent consumers; direct outreach keyed off logs; a final hard-stop date you actually honor. The goal is zero surprised consumers, because surprised consumers write blog posts.

1.8 External Bibliography & Further Reading

1.8.1 Foundational Books

- Roy T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures* (2000) [Fie00] — the REST dissertation; read Chapter 5 before Chapter 3 of this book.
- Martin Kleppmann, *Designing Data-Intensive Applications* [Kle17] — the distributed-systems foundation beneath every failure mode in our taxonomy.
- Betsy Beyer et al., *Site Reliability Engineering* [Bey+16] — SLOs, error budgets, and the operational culture behind Module 4.
- Michael T. Nygard, *Release It!*, 2nd ed. [Nyg18] — stability patterns and the canonical timeout/bulkhead/circuit-breaker case studies.
- Sam Newman, *Building Microservices*, 2nd ed. [New21] — service decomposition and inter-service communication for Chapters 11–15.
- Lukas Gehl, *API Design Patterns* [GH21] — product-thinking and pattern language for API designers.

1.8.2 Standards and Specifications

- RFC 9110, *HTTP Semantics* [Fie22a] — the normative reference for methods, status codes, and header fields.
- RFC 9111, *HTTP Caching*; RFC 9112, *HTTP/1.1* — companions to RFC 9110 for Chapters 1 and 10.
- RFC 9457, *Problem Details for HTTP APIs* — the error-format standard implemented from Chapter 7 onward.
- OpenAPI Specification 3.1 [Ope21] — the contract language used from Chapter 4 onward.
- The Twelve-Factor App [Wig17] — configuration and operational principles assumed by our deployment chapters.
- OWASP API Security Top 10 (2023) — the threat catalog of Chapter 16.

1.8.3 Online Documentation and Courses

- FastAPI official documentation and tutorial — the framework of Module 2.
- Python `http.server`, `urllib`, and `logging` documentation — the `stdlib` surfaces used in this chapter’s listings.
- Google SRE Workbook (online) — worked SLO and alerting examples.
- MDN Web Docs, *HTTP* reference — quick, accurate header and status-code lookups.
- *Designing APIs* (public university API-economy courses and conference tracks such as API City and apidays) — current practice in API product management and governance.

1.8.4 Recommended Datasets (Synthetic Only)

- **Northwind Robotics catalog** (`datasets\`): parts, robots, and telemetry fixtures used throughout the labs; entirely fictional, no real PII.
- **Generated latency traces**: produce your own with the benchmark harness (Listing 1.3.2) — real measurements from your machine are the best dataset for evaluation exercises.

- **Synthetic event streams** for Chapter 11: generated with seeded randomness per that chapter’s fixtures; never scraped or real customer data.

1.9 Capstone Projects

Five projects, one per difficulty band. Each is offline, stdlib-first, and built on the Northwind Robotics universe. These are portfolio pieces: keep them in git with measured results committed alongside the code.

1.9.1 Project 0.1 — Contract Archaeology [junior]

Objective. Reverse-engineer the complete contract of Listing 1.3.3 by observation alone, then write it down formally.

Inputs/Datasets. The running mini API; the Northwind parts fixture embedded in it.

Steps.

1. Probe every path and method combination; record status codes, headers, and bodies.
2. Draft an informal OpenAPI-style description: paths, parameters, response schemas, error shapes.
3. Deliberately trigger each failure mode and record its body.
4. Diff your contract against the source; note everything observation could not reveal.

Acceptance Criteria.

- ☐ Written contract covers 100% of paths and error bodies.
- ☐ Each operation’s failure semantics are enumerated.
- ☐ A one-paragraph reflection names what black-box probing cannot discover (e.g. retryability, rate limits).

Stretch Goals. Hand-write a valid OpenAPI 3.1 document for the API and validate it with a local linter.

1.9.2 Project 0.2 — Honest Benchmarks [mid]

Objective. Produce a defensible latency study of the mini API under three load shapes, exposing where averages and percentiles diverge.

Inputs/Datasets. Listings 1.3.2 and 1.3.3; self-generated latency traces.

Steps.

1. Extend the harness to support concurrency via threads (keep the protocol: warmup, $n \geq 200$, raw samples).
2. Run three configurations: sequential, 4 concurrent clients, 16 concurrent clients. Three repetitions each.
3. Report mean, p50, p95, p99, max, throughput, and error counts per configuration in the addendum’s benchmark-table format.
4. Quantify the divergence between mean and p99 as load rises.

Acceptance Criteria.

- ☐ Nine runs (3 configs $\times$ 3 reps) with raw samples retained in CSV.
- ☐ Written analysis: at what concurrency does p99 detach from p50, and which budget stage (Equation (1.1)) is responsible?
- ☐ One paragraph on why the mean understated user pain.

Stretch Goals. Add payload-size as a variable and fit a simple model $L = a + b \cdot \text{bytes}$.

1.9.3 Project 0.3 — Failure Taxonomy Field Guide [mid]

Objective. Build a runnable gallery: one reproducible, scripted demonstration per taxonomy category, with detection evidence for each.

Inputs/Datasets. The mini API; a second stdlib service you write to play “flaky dependency.”

Steps.

1. Script five demos: transport (connection refused / hang), contract (schema change), semantic (wrong-unit response), operational (injected latency under load), security (missing authz check on an object).
2. For each, capture the client-visible symptom and the telemetry that detects it (status codes, percentiles, logs).
3. Write the playbook entry an on-call engineer would follow.

Acceptance Criteria.

- ☐ Five scripts, each deterministic and offline.
- ☐ Each demo maps symptom $\rightarrow$ category $\rightarrow$ detection strategy per Table 1.2.
- ☐ Playbook entries follow Section 1.5.3.

Stretch Goals. Package the flaky dependency as a reusable “chaos stub” with configurable failure probabilities (seeded).

1.9.4 Project 0.4 — Budgets and Breakers for the Mini API [senior]

Objective. Turn the mini API plus a slow dependency into a resilience demonstrator: timeouts, retry policy, circuit breaker, and an SLO with a live error-budget burn calculation.

Inputs/Datasets. Mini API; your chaos stub from Project 0.3 (or a fresh one with configurable delay/failure injection).

Steps.

1. Add a `/v1/price-check` endpoint that calls the dependency.
2. Implement client-side: 500 ms connect / 2 s read timeouts, 3-attempt backoff with jitter, and a closed/open/half-open breaker.
3. Define an SLI (non-5xx within 300 ms) and a 99.9% SLO; compute the budget from Equation (1.2).
4. Run a 10-minute scripted failure scenario; plot or tabulate budget burn over time and show the breaker’s effect.

Acceptance Criteria.

- ☐ Identical fault scenario run with and without protections; the protected run shows materially lower p99 and error rate.
- ☐ Error-budget computation shown from raw samples.
- ☐ A post-incident report in the war-story format of Section 1.5.2.

Stretch Goals. Add a bulkhead (separate executor for the dependency call) and demonstrate isolation of an unrelated endpoint.

1.9.5 Project 0.5 — API Platform Governance Starter [staff]

Objective. Design (and partially automate) the minimum governance plane that would hold a 20-team organization at maturity L2: catalogs, contract gates, ownership, and deprecation policy.

Inputs/Datasets. Three or more of your own prior lab APIs as the “fleet”; the governance baseline of Section 1.2.9.

Steps.

1. Define the metadata schema for the catalog (owner, version, lifecycle state, contract URI, SLO).
2. Write a linter (stdlib Python is fine) that scores each API’s contract document against your rules: has version, has problem details, has pagination caps, declares auth scheme.
3. Author the deprecation policy: timelines, consumer notification, sunset mechanics.
4. Produce a one-page “maturity report” per API with L-level and next actions.

Acceptance Criteria.

- ☐ Catalog holds every lab API with complete metadata.
- ☐ Linter runs offline and exits non-zero on violations (CI-ready).
- ☐ Deprecation policy includes measurable sunset criteria.
- ☐ Maturity reports agree with a manual audit on at least one API.

Stretch Goals. Generate a static HTML developer portal from the catalog (no frameworks; template strings are fine).

1.10 Python Dependencies Appendix

This chapter’s own listings are stdlib-only. The file below is the baseline `requirements.txt` for the whole book’s companion environment; later chapters extend, never replace, it. Pin exactly as shown; the sanity check (Listing 1.3.1) verifies importability, not versions, so minor-version flexibility is acceptable in practice.

```
1 # requirements.txt -- baseline environment for "
  Mastering APIs"
```

```
2 # Python 3.11+ ; install with:  pip install -r
   requirements.txt
3 fastapi==0.115.6
4 uvicorn==0.34.0
5 httpx==0.28.1
6 pydantic==2.10.4
7 pytest==8.3.4
```

1.10.1 fastapi

Description. The ASGI web framework used from Chapter 7 onward for all provider-side services.

Why included. Type-driven request validation, automatic OpenAPI 3.1 generation [Ope21], and first-class async support make it the cleanest vehicle for teaching contract-first API design; its generated schemas plug directly into the contract-testing toolchain of Chapter 20.

Alternatives. Flask (simpler, but validation and OpenAPI are add-ons), Django REST Framework (heavier, ORM-coupled), Litestar (comparable, smaller ecosystem).

Installation.

```
1 pip install fastapi==0.115.6
```

1.10.2 uvicorn

Description. A minimal ASGI server that runs FastAPI applications.

Why included. The reference server for every lab: one process, one command, deterministic startup on 127.0.0.1. Chapter 17 replaces it behind a production gateway, where its single-process model is discussed honestly.

Alternatives. Hypercorn (HTTP/2 support), Gunicorn with Uvicorn workers (POSIX-oriented process management), Daphne.

Installation.

```
1 pip install uvicorn==0.34.0
```

1.10.3 httpx

Description. A typed HTTP client (sync and async) with connection pooling, HTTP/2 support, and per-request timeouts.

Why included. It is the consumer-side workhorse from Chapter 3: explicit timeout configuration, transport plug-ins for offline stubbing, and

an API surface that mirrors the protocol concepts taught in Chapter 2 rather than hiding them.

Alternatives. `urllib` (stdlib, used in this chapter to show the primitives), `requests` (synchronous only, less explicit about timeouts), `aiohttp` (async-first).

Installation.

```
1 pip install httpx==0.28.1
```

1.10.4 pydantic

Description. Data-validation library that parses untrusted input into typed Python models using standard type annotations.

Why included. It is the concrete implementation of Chapter 5’s core idea — validation at the system boundary with typed models — and it powers FastAPI’s request/response contracts. Its error output is also the raw material for RFC 9457 responses in Chapter 8.

Alternatives. `dataclasses` + manual validation (shown in this chapter for contrast), `marshmallow`, `msgspec` (faster, narrower).

Installation.

```
1 pip install pydantic==2.10.4
```

1.10.5 pytest

Description. The test runner used by every chapter’s test suite.

Why included. Deterministic, fixture-based tests are the book’s acceptance mechanism; labs end with `pytest -q`, and Chapter 20 builds the full quality-engineering stack (contract, property-based, load) on top of it.

Alternatives. `unittest` (stdlib; used in early offline drills where zero dependencies matter), `nose2` (unmaintained — avoid).

Installation.

```
1 pip install pytest==8.3.4
```

1.11 Chapter Summary & Key Takeaways

- An API is a **versioned contract**: operations, representations, and failure semantics over a transport (Definition 1.1). Hold the product, contract, and failure-surface mental models simultaneously.

- Incidents classify into five categories — transport, contract, semantic, operational, security — each with its own detection strategy (Table 1.2). Classify before you mitigate.
- Maturity climbs L0→L4: scripts, documented REST, contract-tested CI, resilient observable services, governed platform. Know where your systems sit.
- Latency is reported in percentiles under a stated load, never as a bare average; budgets decompose per Equation (1.1), and error budgets follow from SLOs (99.9% monthly = 43.2 minutes).
- Every network call carries an explicit timeout; the war story of Section 1.5.2 is what happens when one does not.
- The environment is offline-first, deterministic, typed, and PowerShell-native; the sanity check gates every lab.
- You now have a working stdlib API, client, and benchmark harness — the seed from which twenty-one chapters grow.

1.12 Quick-Reference Card

1.13 Standardized Evidence Addendum

1.13.1 Benchmark Snapshot Template

1.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

1.13.3 Cost and Latency Budget Snapshot

1.14 Configuration Preset Template


```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

1.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

1.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

1.17 Cross-Chapter Decision Card

1.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

1.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter	Requires	Why
1 HTTP wire	0	Baseline environment and vocabulary.
2 Consuming APIs	1	Clients are built on raw protocol semantics.
3 REST style	1	Constraints are stated in protocol terms.
4 Serialization & contracts	2–3	Schemas describe REST payloads.
5 Client-side security	2, 4	Secures typed, contract-aware clients.
6 FastAPI	3–4	Framework routes map to resources and schemas.
7 Problem details	6	Error media types plug into the framework.
8 Pagination/filter/sort	4, 6	Collection contracts over real services.
9 Versioning	4, 7	Compatibility is defined over schemas & errors.
10 Idempotency/caching	1, 6–7	Safe retries need protocol & error semantics.
11 Async APIs	1, 6	Webhooks/SSE/WebSockets extend the HTTP mental model.
12 GraphQL	3–4	A different contract style over the same wire.
13 gRPC	4	Schema-first RPC; protobuf is a serialization contract.
14 Rate limiting	2, 10	Builds on client behavior and safe retries.
15 Microservices resilience	10, 14	Breakers and budgets compose earlier patterns.
16 API security	5, 6	Threats are mapped onto real services and clients.
17 Deployment & gateways	6, 15	Ships resilient services behind gateways.
18 Observability & SRE	14–15	SLOs sit on rate-limited, resilient services.
19 Testing & quality	4, 6–7	Contract tests need schemas and error models.
20 Lifecycle & governance	9, 16, 19	Governance composes versioning, security, testing.
21 Synthesis	all	Integrative capstone.

Table 1.3: Hard prerequisite edges between chapters.

Stage	Starter budget	Dominant cost driver
TLS / connection setup	$\leq 10\%$	Handshake round trips; keep-alive hit rate.
Queueing	$\leq 10\%$	Worker pool saturation; arrival bursts.
Request validation	$\leq 10\%$	Schema complexity; CPU per request.
Business logic & I/O	$\leq 55\%$	Downstream fan-out; query cost.
Serialization & egress	$\leq 15\%$	Payload size; compression ratio.

Table 1.4: Starter latency budget decomposition (share of p99 target).

Risk	Likelihood	Impact	Mitigation
Reader environment diverges from baseline	Medium	High	Sanity-check script (Listing 1.3.1) gates every lab.
Percentile measurements misread due to tiny samples	High	Medium	Benchmark protocol fixes $n \geq 200$ and warmup discipline.
Learners copy toy auth patterns into real systems	Medium	High	Warning boxes flag every non-production pattern in place.
Offline stubs create false confidence about real services	Low	Medium	Chapters state exactly which behaviors the stub does not model.

Table 1.5: Initial risk register for the curriculum itself.

Symptom	Likely cause	First check
<code>ConnectionRefusedError</code> on localhost	Server not running, or bound to a different port/interface	<code>Test-NetConnection 127.0.0.1 -Port 8000</code> in PowerShell.
Request hangs, no exception	Missing client timeout; server accepted but never answered	Add/verify <code>connect + read</code> timeouts; check server logs.
Intermittent <code>RemoteDisconnected</code>	Server closed an idle keep-alive connection the client reused	Retry once on idempotent methods; shorten client keep-alive.
HTTP 415 Unsupported Media Type	Missing or wrong <code>Content-Type</code> header	Compare request headers against the contract's media type.
JSON decode error on error paths	Error body is not JSON (proxy HTML page, empty body)	Log status + first 200 bytes of body before decoding.
p99 spike with flat p50	Queueing at saturation, GC pauses, or one slow downstream	Decompose per Equation (1.1); check pool metrics.
Works in demo, fails under load test	Connection exhaustion, thread-pool starvation, retry storms	Count open sockets; check retry policy and pool sizing.

Table 1.6: Symptom-to-cause gallery for the most common lab failures.

Skill	Score (0–2)
Create and activate a Python virtual environment in PowerShell	
Write a typed Python function with a dataclass return value	
Explain what happens between <code>urlopen</code> and receiving bytes	
Read an HTTP request/response pair in raw text form	
Branch on HTTP status codes and content types	
Explain the difference between TCP and HTTP	
Compute a percentile from a sorted list by hand	
Use git to clone, branch, commit, and diff	
Explain what TLS authenticates and encrypts	
State one reason an average is a bad latency summary	

Table 1.7: Readiness self-assessment (target: total ≥ 14 , no zeros).

Item	Value / rule
API definition	Versioned contract: operations + representations + failure semantics over a transport.
Failure taxonomy	Transport / contract / semantic / operational / security — classify, then mitigate.
Maturity levels	L0 scripts; L1 documented; L2 contract-tested CI; L3 resilient + observable; L4 governed platform.
Latency reporting	p50 / p95 / p99, $n \geq 200$, warmup 50, state the load; never averages alone.
Latency budget	$L = L_{TLS} + L_{queue} + L_{validate} + L_{logic} + L_{serialize} + L_{egress}$.
Error budget	1 – SLO; 99.9%/month = 43.2 min; 99.95% = 21.6 min; 99.99% = 4.32 min.
Timeouts	Explicit connect + read timeout on every call; no defaults trusted.
Retries	Bounded (3), exponential backoff + jitter, idempotent ops only, honor Retry-After .
Little’s Law	$C = \lambda L$: concurrency = rate $\times$ latency.
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
Environment	<code>python -m venv .venv</code> then <code>.\venv\Scripts\Activate.ps1</code> ; run the sanity check before any lab.
Golden rules	Offline-first; deterministic; stdlib-first; synthetic data only; PowerShell-native docs.

Table 1.8: Chapter 0 quick-reference card.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 1.9: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 1.10: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 1.11: Decision card to complete per chapter.

Chapter 2

The HTTP Wire Protocol: HTTP/1.1, HTTP/2 & HTTP/3

Every API you will ever design, consume, debug, or rescue at 3 a.m. ultimately reduces to bytes moving across a socket according to the rules of the Hypertext Transfer Protocol. Frameworks, gateways, and client libraries exist to hide those bytes from you—and that is precisely why production incidents so often baffle teams: when an abstraction leaks, the only ground truth left is the wire. This chapter rebuilds HTTP from the socket upward. We begin with the textual, line-oriented framing of HTTP/1.1 [Fie22b], including the exact byte layout of requests and responses, the two legal ways to delimit a message body (**Content-Length** versus **Transfer-Encoding: chunked**), persistent connections, and the head-of-line (HOL) blocking problem that made HTTP/1.1 pipelining a historical dead end. We then formalize the semantics layer [Fie22a]: method properties (safe, idempotent, cacheable), the status-code taxonomy with precise selection guidance, and the essential header registry that governs content negotiation, conditional requests, caching, and cross-origin access.

From there we move to the binary era. HTTP/2 [Tho22] replaces text with a 9-byte frame header and multiplexed streams, compresses headers with HPACK [Peo15], and introduces credit-based flow control—yet it merely relocates HOL blocking from the application layer to TCP. HTTP/3 [Bis22] completes the arc by abandoning TCP for QUIC [Iye21] over UDP, giving each stream independent delivery and folding the transport handshake into TLS 1.3 [Res18]. Because no HTTPS request exists without TLS, we dissect the TLS 1.3 handshake message by message: **ClientHello**, **ServerHello**, **EncryptedExtensions**, **Certificate**, **CertificateVerify**, and **Finished**, along with ALPN negotiation, SNI, certificate chain validation, and the 2-RTT (TLS 1.2) versus 1-RTT (TLS 1.3) economics that drive connection

setup latency.

Why does this matter in production? Because the most expensive API defects are protocol-level misunderstandings: request smuggling born of **Content-Length/Transfer-Encoding** ambiguity, connection churn born of a client keep-alive timeout that is one second longer than the load balancer’s idle timeout, retry storms born of treating non-idempotent **POST** as safe to replay, and latency cliffs born of TCP-level HOL blocking under packet loss. An engineer who can read the wire—who can hexdump a chunked body, decode an HTTP/2 frame header by hand, and narrate a TLS 1.3 flight—debugs in minutes what others debug in days. This chapter gives you that literacy, with complete, offline, stdlib-only Python laboratories that let you observe every byte yourself.

2.1 Learning Objectives & Prerequisites

Prerequisites

This chapter assumes you have completed **Chapter 0** (Introduction), which establishes the workspace conventions: Python 3.11+, PowerShell on Windows, offline-first execution, and the fictional *Northwind Robotics* universe used for all synthetic data. You should be comfortable with basic Python (functions, classes, context managers) and with running scripts from a terminal. No prior networking knowledge is assumed; sockets, TLS, and framing are built from first principles.

Learning Objectives

After completing this chapter, you will be able to:

- LO1.1** Read and construct raw HTTP/1.1 messages at the byte level, including request-line/status-line grammar, CRLF framing, and both body-framing mechanisms (**Content-Length** and chunked transfer coding) [Fie22b].
- LO1.2** Classify every standard method by its *safe*, *idempotent*, and *cacheable* properties, and select the correct status code for ambiguous production scenarios (200 vs. 201 vs. 204; 400 vs. 422; 401 vs. 403 vs. 404; 409 vs. 412; 429; 503 with **Retry-After**) [Fie22a].
- LO1.3** Explain the HTTP/2 binary framing layer bit by bit: the 9-byte frame header, all ten frame types, the stream state machine, HPACK compression, and two-level flow control [Tho22; Peo15].

- LO1.4** Articulate why HTTP/3 moved to QUIC over UDP, how QUIC connection IDs enable migration, and what head-of-line trade-offs remain [Bis22; Iye21].
- LO1.5** Narrate the TLS 1.3 handshake message by message, explain 1-RTT versus 2-RTT setup and 0-RTT resumption, and configure ALPN/SNI correctly in a Python `ssl.SSLContext` [Res18].
- LO1.6** Build, run, and extend stdlib-only tooling: a raw-socket HTTP/1.1 client, a chunked/keep-alive demonstration server, a TLS configuration inspector, and an HTTP/2 frame parser.
- LO1.7** Diagnose the classic wire-level failure modes—request smuggling, keep-alive race conditions, HOL blocking, and blind retries of non-idempotent methods—and design mitigations.

Key Concept

The wire is the contract. Specifications such as OpenAPI describe what an API *should* do; the wire shows what it *actually* does. Every debugging session that matters ends at the byte stream. Train yourself to always ask: *what did the bytes say?*

2.2 Deep Technical Mechanics & Architectural Concepts

2.2.1 HTTP/1.1 Message Anatomy on the Wire

HTTP/1.1 is a *textual*, request–response, stateless application protocol carried over a reliable byte stream, historically TCP [Fie22b]. “Textual” is literal: the bytes on the wire are ASCII (with body octets of arbitrary encoding), lines are terminated by the two-byte sequence CRLF (0x0D 0x0A), and you can read an entire exchange with nothing more than a hex dump.

Definition 2.1 (HTTP/1.1 message). An HTTP/1.1 *message* is the sequence: *start-line*, zero or more *header fields* each terminated by CRLF, an empty line (CRLF alone) marking the end of the header section, and an optional *message body* whose length is determined by the framing rules of RFC 9112 §6. A request’s start-line is the *request-line* (`method SP request-target SP HTTP-version CRLF`); a response’s start-line is the *status-line* (`HTTP-version SP status-code SP reason-phrase CRLF`).

A minimal, complete request as raw bytes—note the visible `\r\n` escape annotations we will use throughout for readability—looks like this:

```
1 GET /robots/42 HTTP/1.1\r\n
2 Host: api.northwind-robotics.example\r\n
3 Accept: application/json\r\n
4 Connection: keep-alive\r\n
5 \r\n
```

And the corresponding response, with a 27-byte JSON body:

```
1 HTTP/1.1 200 OK\r\n
2 Content-Type: application/json\r\n
3 Content-Length: 27\r\n
4 Connection: keep-alive\r\n
5 \r\n
6 {"id":42,"name":"Weld-Bot"}\r\n    <- trailing CRLF
   shown for clarity;
7                                     only the 27 body
                                       bytes are
                                       counted
```

Key Concept

The empty line is the protocol. The single CRLF that separates the header section from the body is the most important two bytes in HTTP/1.1. Every parser—from a browser to an embedded proxy—splits head from body at the first CRLF CRLF sequence. Anything ambiguous about where the body starts or ends is a security incident waiting to happen (Section 2.4).

Message Body Framing: Content-Length vs. Transfer-Encoding

Because HTTP/1.1 reuses connections, a receiver must know exactly where one message ends and the next begins. There are exactly two legal delimiting mechanisms for bodies [Fie22b]:

1. **Content-Length:** n — the body is exactly n octets. The receiver reads n bytes and the next byte starts the next message. A message with both **Transfer-Encoding** and **Content-Length** is malformed; **Transfer-Encoding** wins, and the discrepancy is the root of *request smuggling* (Section 2.4).
2. **Transfer-Encoding: chunked** — the body is a sequence of chunks, each prefixed by its size in hexadecimal. The exact byte layout:

```
1 HTTP/1.1 200 OK\r\n
2 Content-Type: text/plain\r\n
3 Transfer-Encoding: chunked\r\n
4 \r\n
5 7\r\n          <- chunk size, ASCII hex (0x37 0
   x0D 0x0A)
6 Weld-Bo\r\n    <- 7 data octets + CRLF
7 6\r\n
8 t #42\r\n
9 0\r\n          <- terminal zero-length chunk
10 \r\n          <- end of trailers (none here)
```

Each chunk is `chunk-size [CRLF chunk-data CRLF]`; the stream ends with a zero-size chunk followed by an optional trailer section and a final CRLF. Chunked encoding exists so a sender can stream a body whose length is not known in advance (log tails, generated CSVs, Server-Sent Events). If neither header is present in a response, the body runs until connection close—which destroys keep-alive for that connection.

Connection Reuse, Keep-Alive, and Pipelining

HTTP/1.0 closed the TCP connection after every response; HTTP/1.1 makes connections *persistent by default* [Fie22b]. A client declares `Connection: close` only to opt out. Reuse amortizes TCP slow start and the TLS handshake across many requests—at high request rates this is the difference between thousands and tens of requests per second per connection.

Pipelining went further: send request 2 before response 1 arrives. It died for three compounding reasons: (1) responses must still be delivered in request order, so one slow response blocks all later ones (application-layer HOL blocking); (2) intermediaries (proxies, ancient load balancers) routinely mangled pipelined traffic; (3) a non-idempotent pipelined request cannot be safely retried if the connection dies mid-pipeline. Every major browser shipped with pipelining disabled and eventually removed the code entirely. The industry answer to the same problem was HTTP/2 multiplexing.

2.2.2 Method Semantics per RFC 9110

Method semantics are not conventions; they are contract terms that intermediaries (caches, retries, pre-fetchers) rely on [Fie22a].

Definition 2.2 (Safe, idempotent, cacheable). A method is *safe* if it is read-only: the client does not request, and cannot be held accountable for, any state change on the server. A method is *idempotent* if repeating the identical request $n \geq 1$ times has the same *intended effect on server state*

as one request (side effects like logging aside). A method is *cacheable* if responses to it may be stored for reuse, subject to freshness directives.

Method	Safe	Idempotent	Cacheable	Intended semantics
GET	yes	yes	yes*	Retrieve a representation of the target resource.
HEAD	yes	yes	yes	GET without the body; metadata only.
POST	no	no	rare [†]	Process the enclosed representation per resource semantics (create, action).
PUT	no	yes	no	Replace the target resource's state with the enclosed representation.
PATCH	no	no [‡]	no	Apply a partial modification described by the enclosed document.
DELETE	no	yes	no	Remove the target resource.
OPTIONS	yes	yes	no	Discover communication options (CORS preflight).
TRACE	yes	yes	no	Loop-back diagnostic echo; widely disabled.
CONNECT	no	no	no	Establish a tunnel (e.g., TLS through a proxy).

Table 2.1: Method properties per RFC 9110 §9. *Cacheable by default heuristics. [†]POST responses are cacheable only with explicit freshness information. [‡]PATCH *can* be made idempotent by construction, but the spec does not require it.

The engineering consequences are direct: a client library may transparently retry a PUT after a timeout, but must *never* blindly retry a POST; a shared cache may serve a GET from cache but must invalidate on PUT/PATCH/DELETE to the same URI; a crawler may freely GET and HEAD but nothing else.

2.2.3 Status-Code Taxonomy and Selection Guidance

Status codes are three-digit integers grouped by class [Fie22a]: **1xx** informational (interim, e.g. 100 Continue), **2xx** success, **3xx** redirection, **4xx** client error (the *request* is at fault), **5xx** server error (the *server* failed a valid request). The discrimination between 4xx and 5xx is operationally

vital: 4xx errors are caller bugs and must not page anyone; 5xx errors burn the provider’s error budget.

Dilemma	Choose	Why
200 vs. 201 vs. 204	201	Resource created $\Rightarrow$ 201 with Location ; action returning content $\Rightarrow$ 200; success with nothing to say $\Rightarrow$ 204 (no body allowed).
400 vs. 422	422	400 = malformed syntax / framing; 422 = syntactically valid, semantically wrong (schema violations).
401 vs. 403 vs. 404	varies	401 = unauthenticated (send WWW-Authenticate); 403 = authenticated but forbidden; 404 may <i>mask</i> existence to avoid resource enumeration.
409 vs. 412	varies	409 = state conflict (duplicate, version mismatch on plain writes); 412 = a conditional header precondition (If-Match) failed.
429	429	Rate limited; always pair with Retry-After .
503	503	Transient unavailability (overload, maintenance); pair with Retry-After ; signals “retry later,” unlike 500.

Table 2.2: Status-code selection for the scenarios production teams argue about.

2.2.4 Header Registry Essentials

Headers are name–value pairs, case-insensitive, order-insensitive (with Set-Cookie the classic exception). The essentials for API engineers [Fie22a]:

- **Host** — mandatory in HTTP/1.1 requests; enables virtual hosting and SNI-based TLS routing. Missing **Host** $\Rightarrow$ 400.
- **Accept** / **Content-Type** — **Accept** declares what the client can parse; **Content-Type** declares what the body *is*. Both use media types with optional parameters.
- **Content negotiation with q-values** — weighting in $[0, 1]$: **Accept**: `application/json;q=1.0, text/csv;q=0.5, */*;q=0.1`. The server picks the highest-*q* representation it can produce, or answers 406 **Not Acceptable**.
- **Conditional requests** — **Etag** is an opaque validator for a representation. **If-None-Match**: `"v17"` on a GET yields 304 **Not**

Modified (saving bandwidth); If-Match: "v17" on a PUT yields 412 Precondition Failed on a lost update—the foundation of optimistic concurrency (Chapter 10).

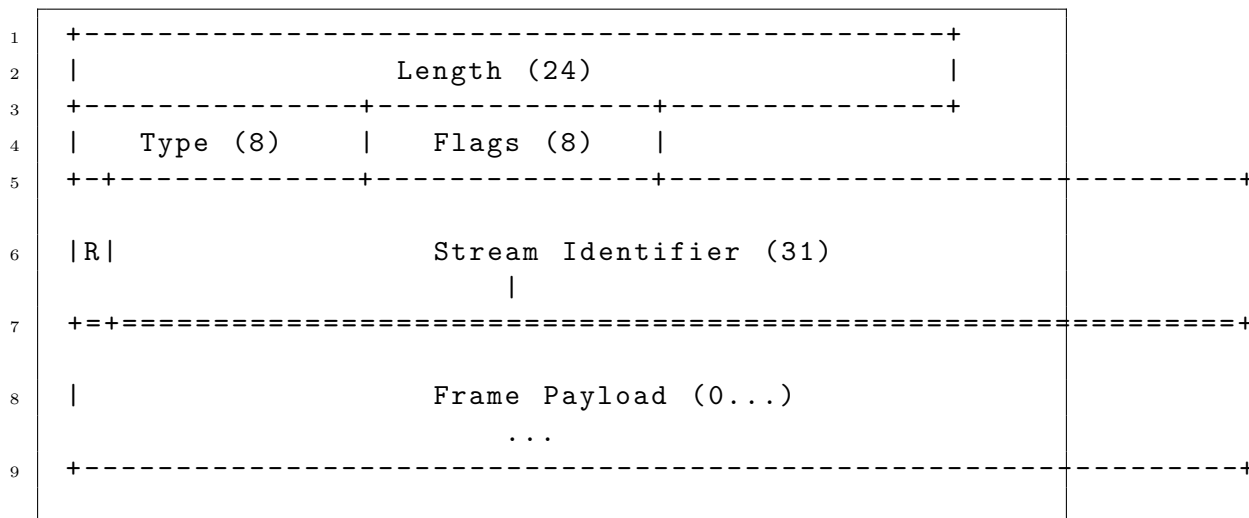
- **Cache-Control** — directives such as `no-store` (never persist), `no-cache` (revalidate every time), `max-age=60`, `private` vs. `public`, `must-revalidate`. `no-cache` is chronically misread as “do not cache”; it means “cache, but revalidate.”
- **CORS preflight** — browsers send `OPTIONS` with `Origin` and `Access-Control-Request-Method`; the server answers with `Access-Control-Allow-Origin`, `-Allow-Methods`, `-Allow-Headers`, `-Max-Age`. CORS is a browser-enforced relaxation of same-origin policy, *not* an authorization mechanism—non-browser clients ignore it entirely.
- **Connection management** — `Connection: keep-alive|close`, `Keep-Alive: timeout=5, max=100` (hop-by-hop, stripped by proxies), and in HTTP/2 the outright prohibition of connection-specific headers.

2.2.5 HTTP/2: The Binary Framing Layer

HTTP/2 keeps the semantics of HTTP (methods, status codes, headers) but replaces the wire format with a length-prefixed binary frame protocol over a single TCP connection [Tho22]. A connection begins with the client *connection preface*—the 24-octet magic string `PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n` followed immediately by a **SETTINGS** frame.

The 9-Byte Frame Header

Every frame starts with a fixed 9-octet header:



That is: a **24-bit** unsigned payload length (maximum default $2^{14} = 16384$ octets, negotiated up to $2^{24} - 1$ via `SETTINGS_MAX_FRAME_SIZE`), an **8-bit** type, an **8-bit** flag field whose meaning is type-specific, a **1-bit reserved** field, and a **31-bit** stream identifier (0 means the frame applies to the whole connection).

Type	Code	Stream	Purpose (key flags)
DATA	0x0	>0	Body octets (<code>END_STREAM=0x1</code> , <code>PADDED=0x8</code>).
HEADERS	0x1	>0	HPACK header block (<code>END_STREAM=0x1</code> , <code>END_HEADERS=0x4</code> , <code>PADDED=0x8</code> , <code>PRIORITY=0x20</code>).
PRIORITY	0x2	>0	(Deprecated in RFC 9113) stream dependency/weight hint.
RST_STREAM	0x3	>0	Abruptly terminate one stream; 32-bit error code.
SETTINGS	0x4	0	Connection parameters; <code>ACK=0x1</code> echoes acceptance.
PUSH_PROMISE	0x5	>0	Server push reservation; now deprecated in practice.
PING	0x6	0	8-octet opaque liveness/RTT probe (<code>ACK=0x1</code>).
GOAWAY	0x7	0	Initiate shutdown; carries last-stream-id and error code.
WINDOW_UPDATE	0x8	any	Increment flow-control window by $1..2^{31} - 1$.
CONTINUATION	0x9	>0	Continue a header block too large for one HEADERS frame (<code>END_HEADERS=0x4</code>).

Table 2.3: HTTP/2 frame types (RFC 9113 §6).

Streams, Multiplexing, and the State Machine

A *stream* is an independent, bidirectional sequence of frames within the connection, identified by a 31-bit ID: client-initiated streams use *odd* IDs (1, 3, 5, ...), server-initiated (push) streams use *even* IDs. Because frames carry their stream ID, requests and responses for many streams interleave freely—one slow response no longer blocks the others. That is the multiplexing that pipelining could never deliver. Figure 2.1 gives the lifecycle.

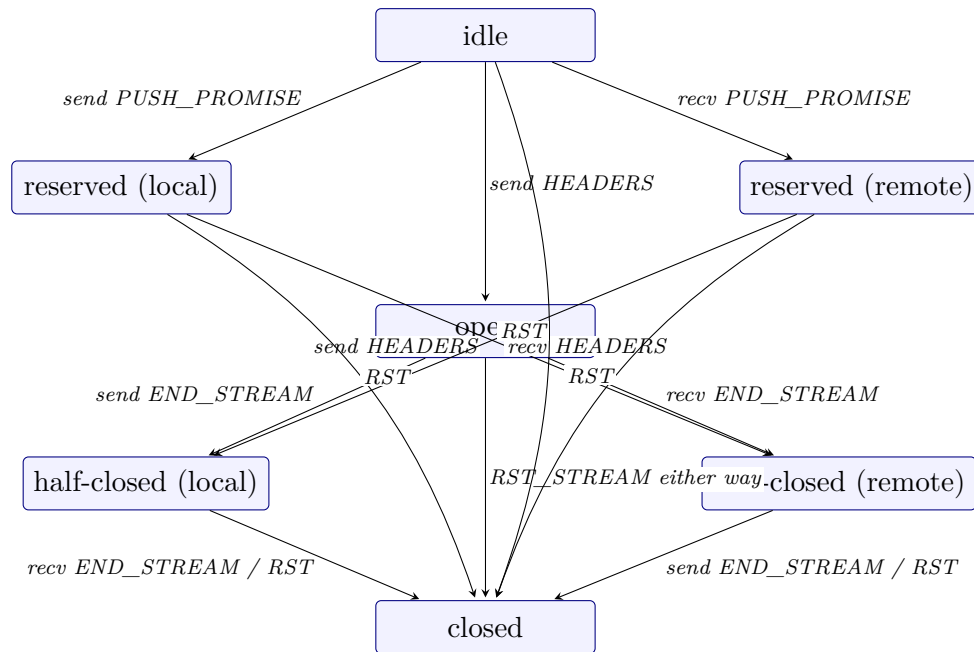


Figure 2.1: HTTP/2 stream state machine (RFC 9113 §5.1). Push-related transitions are retained for completeness; push itself is deprecated.

HPACK Header Compression

Sending textual headers per request wastes hundreds of bytes (cookies, `User-Agent`, `Accept`) and historically defeated TCP’s initial congestion window. HPACK [Peo15] compresses the header block with three mechanisms: (1) a **static table** of 61 predefined name/value pairs (index 1 is `:authority`, index 2 is `:method: GET, ...`); (2) a per-connection **dynamic table** (default 4096 octets, set by `SETTINGS_HEADER_TABLE_SIZE`) of recently seen headers, maintained identically on both sides so an index reference suffices on repeat; (3) a static **Huffman code** for string literals. Because the dynamic table is ordered state, header blocks must be decoded in order—one reason header blocks can span `HEADERS + CONTINUATION` frames but cannot interleave with other streams’ header frames. QPACK (HTTP/3) relaxes exactly this constraint.

Flow Control and Push Deprecation

HTTP/2 flow control is credit-based and hop-by-hop: each `DATA` octet consumes credit from *both* the per-stream window and the connection-wide window (both default to 65,535 octets). Receivers grant more credit with `WINDOW_UPDATE`; senders change peer stream windows with `SETTINGS_INITIAL_WINDOW_SIZE`. The two-level design lets a proxy protect the whole connection from one greedy stream while still letting fast streams run. Only `DATA` frames count

against windows. *Server push*, designed to preemptively send subresources, was deprecated in RFC 9113 and disabled by browsers: in practice it wasted bandwidth pushing assets clients already had cached, and 103 **Early Hints** plus good caching won.

HTTP/2's Remaining Head-of-Line Problem

Multiplexing eliminated *application-layer* HOL blocking, but HTTP/2 runs over a single ordered TCP byte stream. If one TCP segment is lost, every stream stalls until retransmission—*transport-layer* HOL blocking. On lossy mobile networks a 1–2% packet loss rate can make HTTP/2 slower than six parallel HTTP/1.1 connections. This is the empirical motivation for HTTP/3.

2.2.6 HTTP/3 and QUIC

HTTP/3 [Bis22] keeps HTTP semantics and the frame/stream model but runs over QUIC [Iye21], a transport built on **UDP**. Why UDP? Not because UDP is inherently faster, but because it is the only substrate that lets a userspace transport evolve: TCP is ossified in kernels and middleboxes that inspect, rate-limit, and mangle anything that is not classic TCP. QUIC reimplements reliability, congestion control, and—critically—*independent* streams in userspace on top of UDP datagrams.

- **Stream independence:** each QUIC stream has its own ordered delivery; a lost packet stalls only the stream(s) whose data it carried. Transport HOL blocking is gone.
- **Connection IDs and migration:** QUIC connections are identified by Connection IDs, not the 4-tuple. When a phone moves from Wi-Fi to LTE and its source IP changes, the connection *survives*—no re-handshake, no broken downloads.
- **Integrated TLS 1.3:** QUIC encrypts everything above the packet number by design; the transport and crypto handshakes are fused, giving **1-RTT** connection establishment and **0-RTT** resumption for repeat clients (with replay caveats for non-idempotent requests).
- **QPACK:** HPACK's ordered dynamic table would reintroduce stream coupling; QPACK allows out-of-order header-block delivery with explicit acknowledgements and encoder-stream coordination.

Deployment realities: HTTP/3 requires **Alt-Svc** advertisement or HTTPS DNS records for discovery, UDP 443 must be unblocked (many enterprise networks still throttle it), CPU cost per packet is higher than kernel TCP,

and every stack keeps HTTP/2+TCP as fallback. Figure 2.2 summarizes the three generations.

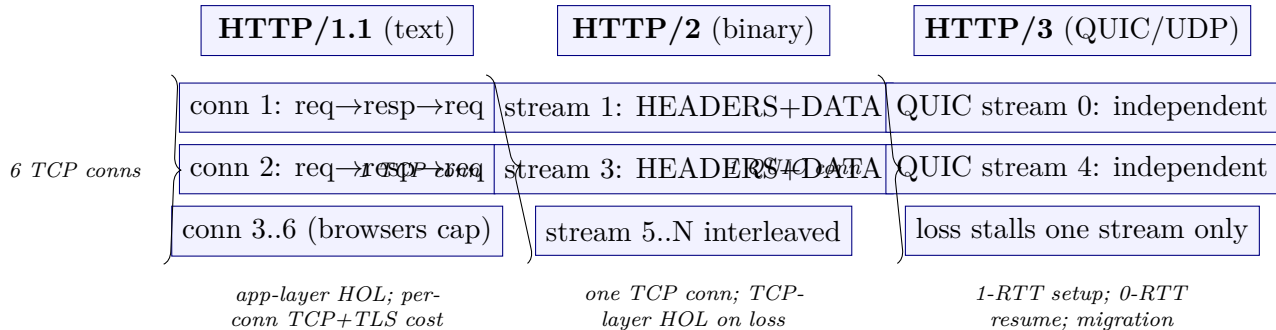


Figure 2.2: Three generations of HTTP concurrency: parallel connections (1.1), multiplexed streams over one TCP connection (2), and independent streams over QUIC (3).

2.2.7 TLS 1.3: The Handshake Behind Every HTTPS Request

HTTP semantics travel inside TLS records (record type 22 = handshake, 23 = application data). TLS 1.3 [Res18] is a ground-up simplification of TLS 1.2: it removes RSA key transport and static DH (forward secrecy is now mandatory), removes renegotiation, shrinks the cipher list to five AEAD suites (e.g. TLS_AES_128_GCM_SHA256), and—the headline—cuts the handshake from **2-RTT** to **1-RTT**, with **0-RTT** resumption for returning clients.

Message-by-Message

Handshake message types are single octets: **ClientHello**=1, **ServerHello**=2, **EncryptedExtensions**=8, **Certificate**=11, **CertificateVerify**=15, **Finished**=20.

1. **ClientHello** — supported versions (**supported_versions** extension listing 1.3), 32-byte random, cipher suites, **key_share** (the client’s ephemeral (EC)DHE public key, sent *speculatively*—this is what buys the 1-RTT), **server_name** (SNI), and **application_layer_protocol_negotiation** (ALPN) listing e.g. h2, http/1.1.
2. **ServerHello** — chosen cipher suite, server random, and the server’s **key_share**. From this point both sides derive handshake traffic keys: *everything after ServerHello is encrypted*.

3. **EncryptedExtensions** — responses to extensions, including the selected ALPN protocol.
4. **Certificate** — the server's certificate chain (leaf first, intermediates after).
5. **CertificateVerify** — a signature over the handshake transcript with the certificate's private key; proves possession.
6. **Finished** — an HMAC over the transcript using the finished key; authenticates the entire handshake. The client answers with its own **Finished**, and application data flows.

```

1 Client
  Server
2
3 ClientHello
4 + key_share, signature_algorithms,
5   psk_key_exchange_modes, SNI, ALPN
6           ----->
7
8                                     ServerHello
9                                     +
10                                    key_share
11
12                                    {
13                                    EncryptedExtensions
14                                    }
15                                    {
16                                    Certificate
17                                    }
18                                    {
19                                    CertificateVerify
20                                    }
21                                    {Finished}
22 <----- ^ server
23         flight (1 RTT total
24         )
25 {Finished}
26
27 ----->
28 [Application Data]
29 Application Data]
30
31 ^ { } = encrypted with handshake keys; [ ] =
32   application keys

```

The *key schedule* is a chain of HKDF extracts/expands: an **early secret** (from the PSK, or zeros) → **handshake secret** (mixed with the ECDHE shared secret) → **master secret**, from which client/server handshake traffic keys, application traffic keys, and resumption secrets are derived. Each stage is bound to the transcript hash, so tampering anywhere upstream poisons every key downstream.

TLS 1.2 vs. 1.3: The Round-Trip Economics

Property	TLS 1.2	TLS 1.3
Full handshake	2-RTT	1-RTT
Resumed handshake	1-RTT	0-RTT (replayable early data!)
Key exchange	RSA transport allowed	(EC)DHE only; forward secrecy mandatory
Handshake encryption	mostly cleartext	everything after ServerHello encrypted
Cipher suites	dozens, incl. CBC/RC4 legacy	five AEAD suites only

Table 2.4: TLS 1.2 versus TLS 1.3 at a glance [Res18].

At 80 ms RTT (typical cross-continent), a cold TLS 1.2 + TCP setup costs ≈ 240 ms before the first request byte; TLS 1.3 cuts this to ≈ 160 ms, QUIC to ≈ 80 ms, and 0-RTT to zero. This is why keep-alive hit rate is a first-class SLO (see the cost template in the standard addendum).

SNI, ALPN, and Certificate Chain Validation

SNI (`server_name`) lets one IP host many TLS identities; the server picks the certificate matching the requested name. **ALPN** selects the application protocol inside the handshake—`h2` or `http/1.1`—so no extra round trip is needed to discover HTTP/2 support; an ALPN mismatch (client offers only `h2`, server speaks only `http/1.1`) fails the connection with `no_application_protocol`. **Chain validation** walks leaf → intermediates → a locally trusted root, checking at each hop: signatures, validity windows (`notBefore/notAfter`), hostname match against SANs, revocation where enforced, and that intermediates carry the `basicConstraints: CA=TRUE` extension. Session **resumption** (tickets issued in `NewSessionTicket` post-handshake) skips the certificate flight; in TLS 1.3 resumption *is* a PSK handshake, optionally with fresh DHE.

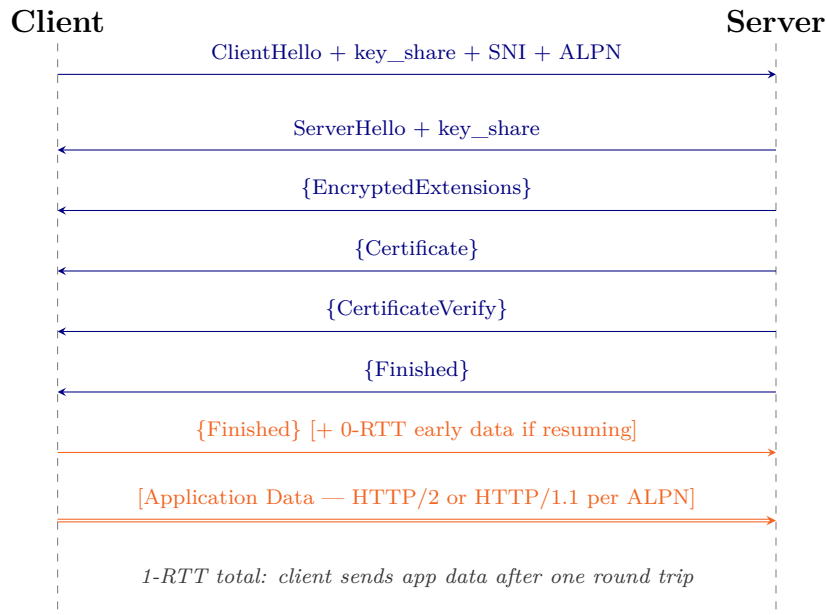


Figure 2.3: TLS 1.3 full handshake flight diagram (braces = encrypted under handshake keys).

2.3 Production-Ready Code Implementation

All listings are Python 3.11+, fully typed, stdlib-only, offline (localhost), and deterministic. Run them from PowerShell with `python <file>.py`; each starts its own local server, exchanges traffic, prints the exact bytes, and shuts down cleanly.

2.3.1 Listing 1 — Raw-Socket HTTP/1.1 Client with Byte-Level Tracing

Design decisions. We deliberately avoid `http.client` and open a raw `socket`: the pedagogical goal is to see that an HTTP request *is* bytes. The server side is the stdlib `http.server` running in a daemon thread on an ephemeral port (`("127.0.0.1", 0)` — the OS assigns a free port, eliminating port-collision flakiness). **Defensive notes:** the escaped-byte view uses `bytes.decode("ascii", "backslashreplace")`-style rendering so arbitrary binary never corrupts the terminal; the client sets an explicit timeout so a wedged server fails fast instead of hanging; the response is read until `Content-Length` bytes of body arrive, never with a blind `recv` loop (which would block forever on a keep-alive connection).

```

1  """Raw-socket HTTP/1.1 client against a local stdlib
2  server.
```

```

3 Prints the exact bytes sent and received, with CRLF
   framing rendered
4 visibly. Offline, deterministic, stdlib only (Python
   3.11+).
5 """
6 from __future__ import annotations
7
8 import socket
9 import threading
10 from http.server import BaseHTTPRequestHandler,
   ThreadingHTTPServer
11
12 HOST = "127.0.0.1"
13 BODY = b'{"id":42,"name":"Weld-Bot"}'
14
15
16 def visible(data: bytes) -> str:
17     """Render bytes with escapes so CRLF framing is
   visible."""
18     return "".join(
19         {0x0D: r"\r", 0x0A: r"\n"}.get(b) or (chr(b) if
20         32 <= b < 127 else rf"\x{b:02x}")
21         for b in data
22     )
23
24 class RobotHandler(BaseHTTPRequestHandler):
25     protocol_version = "HTTP/1.1" # enable keep-alive
   responses
26
27     def do_GET(self) -> None: # noqa: N802 (stdlib
   naming)
28         if self.path != "/robots/42":
29             self.send_error(404, "Not Found")
30             return
31         self.send_response(200)
32         self.send_header("Content-Type", "application/
   json")
33         self.send_header("Content-Length", str(len(BODY
   )))
34         self.end_headers()
35         self.wfile.write(BODY)
36
37     def log_message(self, fmt: str, *args: object) ->
   None:
38         pass # keep stdout deterministic

```



```

39
40
41 def raw_get(host: str, port: int, path: str) -> tuple[
    bytes, bytes]:
42     request = (
43         f"GET_{path}_HTTP/1.1\r\n"
44         f"Host:_{host}:{port}\r\n"
45         "Accept:_application/json\r\n"
46         "Connection:_close\r\n" # close => read-until-
            EOF is unambiguous
47         "\r\n"
48     ).encode("ascii")
49     with socket.create_connection((host, port), timeout
        =5.0) as sock:
50         sock.sendall(request)
51         chunks: list[bytes] = []
52         while True:
53             chunk = sock.recv(4096)
54             if not chunk:
55                 break
56             chunks.append(chunk)
57     return request, b"".join(chunks)
58
59
60 def main() -> None:
61     server = ThreadingHTTPServer((HOST, 0),
        RobotHandler)
62     thread = threading.Thread(target=server.
        serve_forever, daemon=True)
63     thread.start()
64     try:
65         port = server.server_address[1]
66         sent, received = raw_get(HOST, port, "/robots
            /42")
67         print("===_BYTES_SENT_===")
68         print(visible(sent))
69         print("===_BYTES_RECEIVED_===")
70         print(visible(received))
71         assert b"HTTP/1.1_200_OK\r\n" in received
72         assert received.endswith(BODY)
73         print("===_OK:_status_line_and_body_verified_
            ===")
74     finally:
75         server.shutdown()
76         server.server_close()
77         thread.join(timeout=5.0)

```

```

78
79
80 if __name__ == "__main__":
81     main()

```

2.3.2 Listing 2 — Chunked Transfer Encoding and Keep-Alive, By Hand

Design decisions. Here we implement *both* ends at the socket level: a tiny `socketserver` server that emits a `Transfer-Encoding: chunked` response (three chunks) and honors keep-alive for a second request on the same connection, and a client that parses chunks incrementally. **Defensive notes:** the chunk parser validates hex sizes and enforces a maximum chunk size (a classic fuzzing target); every socket has a timeout; the server handles `Connection: close` correctly; all threads are daemons and joined.

```

1  """Chunked transfer encoding + keep-alive demo over raw
   sockets.
2
3  The server streams a body as three chunks, then serves
   a second request
4  on the SAME connection (keep-alive). The client parses
   chunks by hand.
5  Offline, deterministic, stdlib only (Python 3.11+).
6  """
7  from __future__ import annotations
8
9  import socket
10 import socketserver
11 import threading
12
13 HOST = "127.0.0.1"
14 MAX_CHUNK = 1 << 20 # 1 MiB sanity bound against
   hostile size lines
15
16
17 def recv_until(sock: socket.socket, marker: bytes,
18               limit: int = 65536) -> tuple[bytes,
19               bytes]:
20     """Read until marker (inclusive); return (head,
21         leftover bytes).
22
23     Keeping the leftover is essential: a single recv()
24     may return header

```

```

22  """and body bytes together, and discarding the tail
    desyncs framing.
23  """
24  buf = b""
25  while marker not in buf:
26      chunk = sock.recv(4096)
27      if not chunk:
28          raise ConnectionError("EOF before marker")
29      buf += chunk
30      if len(buf) > limit:
31          raise ValueError("header section exceeds
                                limit")
32  head, _, rest = buf.partition(marker)
33  return head + marker, rest
34
35
36  def read_chunked_body(sock: socket.socket, buffered:
    bytes) -> bytes:
37      """Parse a chunked body. 'buffered' holds bytes
        already read past
38  """the header terminator (maybe empty). Returns
    decoded body."""
39      body = bytearray()
40      pending = bytearray(buffered)
41
42      def read_line() -> bytes:
43          while b"\r\n" not in pending:
44              data = sock.recv(4096)
45              if not data:
46                  raise ConnectionError("EOF inside
                                        chunked body")
47              pending.extend(data)
48          line, _, rest = bytes(pending).partition(b"\r\n")
49          pending.clear()
50          pending.extend(rest)
51          return line
52
53      while True:
54          size = int(read_line().split(b";")[0], 16) #
                ignore extensions
55          if size > MAX_CHUNK:
56              raise ValueError(f"chunk size {size}
                                exceeds bound")
57          if size == 0:
58              while read_line() != b"": # consume

```

```

        trailers
        pass
    return bytes(body)
while len(pending) < size + 2: # data +
    trailing CRLF
    data = sock.recv(4096)
    if not data:
        raise ConnectionError("EOF inside chunk
                                data")
    pending.extend(data)
    body.extend(pending[:size])
    if pending[size : size + 2] != b"\r\n":
        raise ValueError("chunk missing trailing
                           CRLF")
    del pending[:size + 2]

class ChunkedHandler(socketserver.BaseRequestHandler):
    def handle(self) -> None:
        sock: socket.socket = self.request
        sock.settimeout(5.0)
        for _ in range(2): # serve two requests on one
            connection
            head, _ = recv_until(sock, b"\r\n\r\n") #
                requests are head-only
            request_line = head.split(b"\r\n", 1)[0].
                decode("ascii")
            keep_alive = b"connection: close" not in
                head.lower()
            if request_line.startswith("GET/stream"):
                parts = [b"Weld-", b"Bot", b"#42
                        online"]
                payload = b"".join(
                    b"%X\r\n%s\r\n" % (len(p), p) for p
                    in parts
                ) + b"0\r\n\r\n"
                sock.sendall(
                    b"HTTP/1.1 200 OK\r\n"
                    b"Content-Type: text/plain\r\n"
                    b"Transfer-Encoding: chunked\r\n\r\n"
                    n" + payload
                )
            else:
                body = b"pong"
                sock.sendall(
                    b"HTTP/1.1 200 OK\r\nContent-Length

```

```

94         :_4\r\n\r\n" + body
95     )
96     if not keep_alive:
97         break
98
99 def main() -> None:
100     server = socketserver.ThreadingTCPServer((HOST, 0),
101         ChunkedHandler)
102     server.daemon_threads = True
103     thread = threading.Thread(target=server.
104         serve_forever, daemon=True)
105     thread.start()
106     try:
107         port = server.server_address[1]
108         with socket.create_connection((HOST, port),
109             timeout=5.0) as sock:
110             sock.sendall(
111                 b"GET_/stream_HTTP/1.1\r\nHost:_
112                 localhost\r\n\r\n"
113             )
114             head, buffered = recv_until(sock, b"\r\n\r\n")
115             print("===_RESPONSE_HEAD_(request_1)_===")
116             print(head.decode("ascii"), end="")
117             body = read_chunked_body(sock, buffered)
118             print("===_DECODED_CHUNKED_BODY_===")
119             print(repr(body))
120             # Second request on the SAME socket: keep-
121             # alive in action.
122             sock.sendall(
123                 b"GET_/ping_HTTP/1.1\r\nHost:_localhost
124                 \r\n"
125                 b"Connection:_close\r\n\r\n"
126             )
127             # Server closes after this response: read
128             # until EOF.
129             reply2 = b""
130             while True:
131                 chunk = sock.recv(4096)
132                 if not chunk:
133                     break
134                 reply2 += chunk
135             print("===_RESPONSE_(request_2,_same_
136                 connection)_===")
137             print(reply2.decode("ascii"))

```

```

130         assert body == b"Weld-Bot_#42_online"
131         assert b"pong" in reply2
132         print("==_OK:_chunked_decode+_keep-alive_
              reuse_verified==")
133     finally:
134         server.shutdown()
135         server.server_close()
136         thread.join(timeout=5.0)
137
138
139 if __name__ == "__main__":
140     main()

```

2.3.3 Listing 3 — TLS Inspector: Context Configuration, ALPN, and Ciphers

Design decisions. Generating a self-signed certificate purely from stdlib Python is not supported (the `ssl` module consumes but does not mint certificates), so the lab (Section 2.5) mints one with a documented `openssl.exe` command. This listing is split into two halves that work with or without that certificate: (1) a *certificate fixture decoder* using the documented stdlib hook `ssl._ssl._test_decode_cert`, which parses a PEM file without opening any connection; (2) a *context auditor* that builds a hardened `SSLContext`, pins TLS 1.3, sets ALPN to `h2/http/1.1`, and prints the effective cipher list — all offline. If the PEM fixture from the lab exists next to the script, the decoded fields (subject, issuer, SANs, validity) are printed too; otherwise the script explains exactly how to create it and still runs to completion.

```

1  """TLS_inspector:_certificate_decoding,_context_
   _hardening,_ALPN,_ciphers.
2
3  Offline,_stdlib_only_(Python_3.11+)._If_a_PEM_fixture_(
   _created_in_the
4  chapter_lab_with_openssl)_exists_beside_this_script,_
   _its_fields_are
5  decoded_with_ssl._ssl._test_decode_cert;_otherwise_the_
   _context_audit
6  still_runs_and_the_script_prints_the_fixture_
   _instructions.
7
8  """
9
10 from __future__ import annotations
11
12 import ssl
13 from pathlib import Path

```

```

13 CERT_PATH = Path(__file__).with_name("northwind_lab.pem
    ")
14
15
16 def decode_certificate(path: Path) -> None:
17     """Parse a PEM certificate file into a dict and
        print key fields."""
18     decoded = ssl._ssl._test_decode_cert(str(path)) #
        stdlib test hook
19     print("== CERTIFICATE FIELDS ==")
20     for field in ("subject", "issuer", "notBefore", "
        notAfter"):
21         print(f"{field}: {decoded.get(field)}")
22     san = decoded.get("subjectAltName", ())
23     print(f"subjectAltName: {san}")
24     serial = decoded.get("serialNumber", "<unknown>")
25     print(f"serialNumber: {serial}")
26
27
28 def build_client_context() -> ssl.SSLContext:
29     """Hardened TLS 1.3 client context with ALPN."""
30     ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
31     ctx.minimum_version = ssl.TLSVersion.TLSv1_3
32     ctx.check_hostname = True
33     ctx.verify_mode = ssl.CERT_REQUIRED
34     ctx.set_alpn_protocols(["h2", "http/1.1"])
35     if CERT_PATH.exists():
36         ctx.load_verify_locations(cafile=str(CERT_PATH)
            )
37     return ctx
38
39
40 def audit_ciphers(ctx: ssl.SSLContext) -> None:
41     """Print the negotiated-cipher candidates of a
        context."""
42     print("== TLS 1.3+ CIPHER SUITES AVAILABLE ==")
43     for cipher in ctx.get_ciphers():
44         print(
45             f"{cipher['name']:<32} protocol={cipher['
                protocol']:<8}"
46             f"bits={cipher['strength_bits']}"
47         )
48
49
50 def main() -> None:
51     if CERT_PATH.exists():

```

```

52         decode_certificate(CERT_PATH)
53     else:
54         print(f"NOTE: {CERT_PATH.name} not found; run
55               the chapter lab's
56               \"openssl command to create it, then re-
57               run.\")
58
59     ctx = build_client_context()
60     print(f"minimum_version: {ctx.minimum_version}")
61     print(f"ALPN offered: h2, http/1.1")
62     audit_ciphers(ctx)
63     # NOTE: OpenSSL's cipher API may omit TLS 1.3
64     # suites (they are not
65     # configurable); an empty/short list here is
66     # expected, not an error.
67     assert ctx.minimum_version == ssl.TLSVersion.
68         TLSv1_3
69     print("== OK: context hardened and cipher audit
70           complete ==")
71
72 if __name__ == "__main__":
73     main()

```

2.3.4 Listing 4 — HTTP/2 Frame Parser over a Captured Byte Stream

Design decisions. The parser consumes the 9-octet header with `struct` (network byte order), validates the 31-bit stream mask, decodes the type-specific flags for DATA/HEADERS/SETTINGS, and prints a table. The “captured stream” is an embedded fixture built byte-by-byte in the script itself — a client connection preface plus SETTINGS, SETTINGS-ACK, HEADERS, and DATA frames — so the output is fully deterministic and reproducible offline. **Defensive notes:** the parser raises `ValueError` on truncation rather than slicing garbage, enforces `SETTINGS_MAX_FRAME_SIZE` (2^{14} default), and treats unknown frame types as skip-able per RFC 9113 §5.5 (extensions must be ignorable).

```

1  """HTTP/2 frame parser: decode a captured byte stream,
2     stdlib only.
3
4  The fixture is constructed in-script (client preface +
5     SETTINGS,
6     SETTINGS-ACK, HEADERS, DATA) so results are
7     deterministic and offline.
8
9  """

```



```

6  from __future__ import annotations
7
8  import struct
9  from dataclasses import dataclass
10
11 MAX_FRAME_SIZE = 2**14 # RFC 9113 default
12     SETTINGS_MAX_FRAME_SIZE
13 CLIENT_PREFACE = b"PRI*\r\nHTTP/2.0\r\n\r\nSM\r\n\r\n"
14
15 FRAME_TYPES = {
16     0x0: "DATA", 0x1: "HEADERS", 0x2: "PRIORITY", 0x3:
17         "RST_STREAM",
18     0x4: "SETTINGS", 0x5: "PUSH_PROMISE", 0x6: "PING",
19     0x7: "GOAWAY",
20     0x8: "WINDOW_UPDATE", 0x9: "CONTINUATION",
21 }
22
23 FLAG_NAMES = {
24     0x0: {0x1: "END_STREAM", 0x8: "PADDED"},
25     0x1: {0x1: "END_STREAM", 0x4: "END_HEADERS", 0x8: "
26         PADDED",
27         0x20: "PRIORITY"},
28     0x4: {0x1: "ACK"},
29     0x6: {0x1: "ACK"},
30     0x9: {0x4: "END_HEADERS"},
31 }
32
33 @dataclass(frozen=True)
34 class Frame:
35     length: int
36     type_code: int
37     flags: int
38     stream_id: int
39     payload: bytes
40
41     @property
42     def type_name(self) -> str:
43         return FRAME_TYPES.get(self.type_code, f"
44             UNKNOWN(0x{self.type_code:02x})")
45
46     @property
47     def flag_names(self) -> str:
48         known = FLAG_NAMES.get(self.type_code, {})
49         names = [v for bit, v in known.items() if self.
50             flags & bit]
51         return "|".join(names) if names else "-"

```

```

46
47
48 def parse_frame(buf: bytes, offset: int) -> tuple[Frame
49 , int]:
50     """Parse one frame starting at offset. Raises on
51     truncation."""
52     if len(buf) - offset < 9:
53         raise ValueError("truncated frame header")
54     length = int.from_bytes(buf[offset : offset + 3], "
55         big")
56     type_code, flags = buf[offset + 3], buf[offset + 4]
57     raw_sid = struct.unpack_from(">I", buf, offset + 5)
58     [0]
59     stream_id = raw_sid & 0x7FFFFFFF # strip reserved
60     bit
61     end = offset + 9 + length
62     if length > MAX_FRAME_SIZE:
63         raise ValueError(f"frame length {length}
64         exceeds MAX_FRAME_SIZE")
65     if len(buf) < end:
66         raise ValueError("truncated frame payload")
67     return Frame(length, type_code, flags, stream_id,
68         buf[offset + 9 : end]), end
69
70
71 def build_fixture() -> bytes:
72     """Preface SETTINGS SETTINGS-ACK HEADERS DATA on stream 1."""
73     def frame(t: int, f: int, sid: int, payload: bytes)
74     -> bytes:
75         return (len(payload).to_bytes(3, "big") + bytes
76             ([t, f])
77             + (sid & 0x7FFFFFFF).to_bytes(4, "big")
78             + payload)
79
80     settings = struct.pack(">HI", 0x3, 100) #
81         MAX_CONCURRENT_STREAMS=100
82     headers_block = bytes([0x82, 0x86, 0x84]) # HPACK:
83         :method GET,
84
85         # :
86         scheme
87         http
88         , :
89         path
90         /
91
92     data = b'{"robot": "Weld-Bot"}'

```

2.3. Production-Ready Code Implementation

```
75     return b"".join([
76         CLIENT_PREFACE,
77         frame(0x4, 0x0, 0, settings),          # SETTINGS
78         frame(0x4, 0x1, 0, b""),              # SETTINGS
79         ack
80         frame(0x1, 0x4 | 0x1, 1, headers_block), #
81         HEADERS | END_HEADERS | END_STREAM
82         frame(0x0, 0x1, 1, data),              # DATA
83         END_STREAM
84     ])
85
86 def main() -> None:
87     stream = build_fixture()
88     assert stream[:24] == CLIENT_PREFACE
89     print(f"client_preamble(24 octets): {CLIENT_PREFACE
90         !r}")
91     print(f"{'len':>5} {'type':<14} {'flags':<28} {'
92         stream':>6}")
93     offset = len(CLIENT_PREFACE)
94     frames: list[Frame] = []
95     while offset < len(stream):
96         frame_obj, offset = parse_frame(stream, offset)
97         frames.append(frame_obj)
98         print(f"{frame_obj.length:>5} {frame_obj.
99             type_name:<14}
100             f"{frame_obj.flag_names:<28} {frame_obj.
101                 stream_id:>6}")
102     assert [f.type_code for f in frames] == [0x4, 0x4,
103         0x1, 0x0]
104     assert frames[2].stream_id == 1 and frames[2].flags
105         & 0x4
106     assert frames[3].payload == b'{"robot":"Weld-Bot"}'
107     print(f"== {OK:4} frames parsed, {flags} and {stream}
108         IDs verified ==")
109
110 if __name__ == "__main__":
111     main()
```

2.4 Common Pitfalls & Anti-Patterns

P1 — Content-Length / Transfer-Encoding Ambiguity (Request Smuggling)

Warning

If a request carries *both* Content-Length and Transfer-Encoding: chunked, RFC 9112 §6.3 says the message is malformed and Transfer-Encoding overrides. When a front-end proxy honors Content-Length and a back-end honors Transfer-Encoding (or vice versa), an attacker can desynchronize the two parsers and *smuggle* a second request into the connection — the classic **CL.TE / TE.CL request smuggling** attack. Mitigations: reject ambiguous requests outright (400), normalize at the front proxy, and prefer HTTP/2 end-to-end (binary framing removes the ambiguity class entirely).

P2 — Assuming GET Never Has a Body

RFC 9110 does not forbid a body on GET; it declares that a GET body has *no generally defined semantics*. Some stacks (Elasticsearch historically) used GET bodies; many proxies drop or reject them. **Rule:** never put semantics-bearing content in a GET/HEAD/DELETE body; use POST if you need a request body on a read-like operation.

P3 — Mishandling 100-continue

A client sending Expect: 100-continue must be prepared for: (a) an interim 100 followed by the final response, (b) a final error (e.g. 413) *without* sending the body, or (c) a server that never sends 100 — in which case the client should transmit the body after a timeout (curl waits 1 s). Clients that block forever waiting for 100, or that treat the interim 100 as the final status, are both broken.

P4 — Keep-Alive Timeout Mismatch

If the client's idle-connection reuse window is *longer* than the server/LB idle timeout, the client will race a stale connection: it sends a request onto a socket the middlebox just closed, producing intermittent ECONNRESET/"connection reset by peer" errors under load. **Rule of thumb:** client idle timeout < LB idle timeout < server keep-alive timeout. This is the single most common cause of "random" production connection failures.

P5 — Blindly Retrying Non-Idempotent Methods

A timeout is ambiguous: the request may have been processed. Retrying a `POST /orders` can double-charge a customer. Only safe/idempotent methods (Table 2.1) may be retried transparently; everything else needs application-level idempotency keys (Chapter 10) before any retry layer is switched on.

P6 — Reading Until Close on a Keep-Alive Connection

A client that reads an HTTP/1.1 response “until EOF” without honoring `Content-Length` or chunked framing hangs forever when the server keeps the connection open. Always parse framing headers first; read exactly the framed length.

P7 — Ignoring Retry-After and Treating 5xx as Identical

`503 + Retry-After` is an explicit back-pressure signal; hammering it converts overload into outage. `429` without honoring `Retry-After` gets clients blacklisted. Distinguish transient (`503`, `429`) from permanent (`500`, `501`) and respect the server’s stated delay with jittered backoff.

P8 — Forgetting Hop-by-Hop Headers

`Connection`, `Keep-Alive`, `TE`, `Trailer`, `Transfer-Encoding`, `Upgrade` are *hop-by-hop*: proxies must not forward them. A hand-rolled reverse proxy that copies all headers verbatim leaks connection state across hops and breaks framing.

2.5 Hands-On Production Lab & Real-World Case Study

2.5.1 Lab A — Dissect Real Traffic with curl.exe

All commands run in PowerShell on Windows; use `curl.exe` explicitly so PowerShell does not alias `curl` to `Invoke-WebRequest`.

1. **Start a local server.** In one terminal: `python -m http.server 8765 -bind 127.0.0.1`. It serves the current directory over plain HTTP/1.1.
2. **Capture the verbose exchange.** In a second terminal:

```
1 curl.exe -v http://127.0.0.1:8765/README.md -o NUL
```

Lines prefixed > are the exact request bytes; < the response. Confirm: the request-line, the mandatory **Host** header, the HTTP/1.1 200 OK status-line, **Content-Length**, and the blank line separating head from body.

3. Force protocol versions and observe.

```
1 curl.exe -v --http1.1 http://127.0.0.1:8765/ -o NUL
2 curl.exe -v -I http://127.0.0.1:8765/README.md
```

The **-I** issues a **HEAD**: identical headers, zero body bytes — verify **Content-Length** is still present but no body arrives.

4. Conditional request. Note the **ETag** (or **Last-Modified**) from step 2, then:

```
1 curl.exe -v -H "If-None-Match: <etag>" http
  ://127.0.0.1:8765/README.md -o NUL
```

(The stdlib `SimpleHTTPRequestHandler` does not implement validators — observe that it ignores the header and returns 200; repeat against any conditional-capable server, or your Listing-1 server extended with an **ETag**, to see 304 Not Modified.)

2.5.2 Lab B — Your Raw Client vs. the Stdlib Server

Run Listing 1, then re-run it with the request modified to omit **Connection: close** and observe that the client now hangs unless it honors **Content-Length** — Pitfall P6 reproduced on demand. Then run Listing 2 and match each printed chunk against the byte layout diagram in this chapter.

2.5.3 Lab C — Mint a Self-Signed Certificate and Inspect TLS

```
1 # openssl.exe ships with Git for Windows:
2 # C:\Program Files\Git\usr\bin\openssl.exe
3 openssl.exe req -x509 -newkey rsa:2048 -keyout
   northwind_lab.key '
4 -out northwind_lab.pem -days 365 -nodes '
5 -subj "/CN=localhost/O=Northwind Robotics"
```

Place `northwind_lab.pem` beside Listing 3 and run it: the fixture decoder prints subject, issuer, SANs, and validity; the context audit prints the TLS 1.3 cipher list and ALPN set. (Self-signed certificates are for local labs only — never disable verification in production to make one work.)

2.5.4 Lab D — HTTP/2 Frame Dissection

Run Listing 4 and confirm the parsed sequence: `SETTINGS`, `SETTINGS(ACK)`, `HEADERS(END_STREAM|END_HEADERS)`, `DATA(END_STREAM)` on stream 1. Then mutate the fixture: flip a flag bit, truncate a payload, and watch the parser’s `ValueError` fire — you now own a differential fuzzing harness.

2.5.5 Real-World Case Study: The 60-Second Mystery

War story (composite of common industry incidents, Northwind Robotics setting). Northwind Robotics’ telemetry API sat behind a cloud load balancer with a **60-second idle timeout**; the Python fleet’s HTTP client pooled connections with a **300-second** keep-alive. Under bursty load, clients routinely grabbed connections the LB had silently reaped mid-idle. The first byte of the next request disappeared into a half-open socket; the client saw `ConnectionResetError` on ~0.3% of requests — always the first request after an idle gap. Retries masked most failures, but a non-idempotent `POST /jobs` retry caused duplicate job dispatch (Pitfall P5 compounding P4). Diagnosis took one packet capture: the LB’s `FIN` arriving 60s after the last request, followed seconds later by the client’s next request on the dead connection and an immediate `RST`. **Fix:** set the client pool’s idle timeout to 45s (below the LB’s 60s), raise the LB timeout to 75s, and gate `POST` retries behind idempotency keys. Errors dropped to zero; p99 latency fell 18ms because connection reuse actually worked instead of gambling.

2.6 Self-Assessment Exercises & Deep-Dive Questions

E1.1 Byte archaeology. Extend Listing 1 to send a `POST` with a `Content-Length` body, then deliberately declare a `Content-Length` one byte too small. Observe how the leftover byte corrupts the next exchange on a keep-alive connection. Explain the desynchronization in terms of framing.

E1.2 Chunked boundaries. Modify Listing 2’s server to emit a chunked body where a chunk straddles two `recv` calls. Verify the client’s pending buffer logic handles it; then reduce `MAX_CHUNK` below a chunk’s size and confirm the defensive `ValueError`.

E1.3 Status-code audit. For each endpoint of a fictional Northwind Robotics API (`POST /robots`, `GET /robots/{id}`, `PUT /robots/{id}/firmware`,

DELETE /robots/{id}), choose the full set of success and error status codes you would return, with one-line justifications referencing Table 2.2.

- E1.4 Frame arithmetic.** A HEADERS frame carries a 70,000-octet header block. With the default `SETTINGS_MAX_FRAME_SIZE`, how many frames are emitted, of which types, and with which flags set on the final frame?
- E1.5 HPACK reasoning.** A client sends 100 requests to the same API with identical headers except a per-request `x-trace-id`. Estimate the header bytes sent under HTTP/1.1 versus HTTP/2+HPACK, stating your assumptions about dynamic-table hits.
- E1.6 Flow-control deadlock hunt.** A client sets `SETTINGS_INITIAL_WINDOW_SIZE` = 0 mid-connection. Describe precisely which frames can still flow and why this is legal; then explain how a buggy server that never sends `WINDOW_UPDATE` manifests to users.
- E1.7 TLS transcript.** From memory, list the TLS 1.3 handshake messages in order, mark which are encrypted, and identify the exact message after which the client could theoretically begin sending 0-RTT data on a resumption.
- E1.8 Smuggling sketch.** Draw the byte-level layout of a CL.TE smuggling payload against a front-end that trusts `Content-Length` and a back-end that trusts `Transfer-Encoding`. Then state the two mitigations that would have prevented it.
- E1.9 HOL cost model.** Given 1% uniform packet loss, 50 ms RTT, and 10 parallel small requests, argue qualitatively when HTTP/2 over one connection loses to HTTP/1.1 over six, and why HTTP/3's outcome differs.
- E1.10 Deep dive — 0-RTT danger.** Explain why 0-RTT early data is replayable, why that makes `POST /payments` over 0-RTT unacceptable, and which defences (single-use tickets, early-data allowlists of safe methods) are appropriate.

2.7 Interview Q&A Vault

Junior Level

- Q1. What are the three parts of an HTTP/1.1 request on the wire?** The request-line (METHOD SP target SP HTTP/1.1), zero or more header fields each ending in CRLF, a mandatory empty line (CRLF), and an optional body delimited by Content-Length or chunked framing.
- Q2. What bytes terminate every HTTP/1.1 header line?** CRLF — carriage return + line feed, 0x0D 0x0A. Not LF alone; bare-LF tolerance is a parser quirk that smuggling attacks exploit.
- Q3. Why is the Host header mandatory in HTTP/1.1?** It enables virtual hosting: one IP serves many domains, and the server (and TLS SNI routing) needs the name to pick the right site. Absence $\Rightarrow$ 400 Bad Request.
- Q4. GET vs. POST in one sentence?** GET retrieves a representation and is safe, idempotent, and cacheable; POST asks the resource to process the enclosed body and has none of those guarantees.
- Q5. What does idempotent mean, precisely?** Repeating the identical request n times has the same intended effect on server state as one request. It says nothing about identical *responses* (a retried DELETE may return 404 the second time and still be idempotent).
- Q6. What is the difference between 401 and 403?** 401 = unauthenticated: credentials missing or invalid, and the response must carry WWW-Authenticate. 403 = authenticated (or authentication irrelevant) but not allowed; re-authenticating will not help.
- Q7. When do you return 201 instead of 200?** When the request created one or more new resources; include a Location header pointing at the primary new resource. 200 is for generic success with a payload.
- Q8. What is 204 used for?** Success with no representation to return (e.g. a DELETE or an idempotent save). A 204 response must not carry a body.
- Q9. What does Content-Type declare vs. Accept?** Content-Type describes the body actually present in this message; Accept is a request header describing what the client can parse in the response.

- Q10. What is a q-value?** A relative weight in $[0, 1]$ on an **Accept** media range, e.g. `application/json;q=0.9`. The server serves the highest-*q* representation it can produce, else 406.
- Q11. What is keep-alive?** HTTP/1.1's default persistent-connection behavior: the TCP connection stays open after the response so subsequent requests skip connection setup. It is why message framing (**Content-Length/chunked**) matters.
- Q12. What does **Connection: close** do?** Tells the peer to close the TCP connection after this message; for responses it also makes “read until EOF” a legal body framing.

Mid Level

- Q13. Why did HTTP/1.1 pipelining die?** Responses must arrive in request order, so one slow response blocks the whole pipeline (HOL blocking); intermediaries corrupted pipelined traffic; and a failed pipeline cannot safely retry non-idempotent requests. Browsers shipped it disabled and removed it.
- Q14. Explain chunked transfer encoding byte layout.** Each chunk: ASCII-hex size, CRLF, that many data octets, CRLF. Terminated by a zero-size chunk, optional trailers, final CRLF. It enables streaming bodies of unknown length over persistent connections.
- Q15. 400 vs. 422?** 400: the server could not parse the request — malformed syntax, bad framing. 422: the request parsed fine but its semantics fail (e.g. JSON schema violations, a negative quantity). 400 blames the bytes; 422 blames the meaning.
- Q16. 409 vs. 412?** 409: the request conflicts with current resource state (duplicate key, version mismatch on an unconditional write). 412: a conditional request's precondition header (**If-Match**, **If-Unmodified-Since**) evaluated false — the client asked for precondition checking and lost.
- Q17. Why send **Retry-After** with 429 and 503?** It converts client retry behavior from a thundering herd into a scheduled drain: the server tells clients the earliest sensible retry moment. Without it, retry storms deepen the overload that caused the 503.
- Q18. How does **If-None-Match** save bandwidth?** The client echoes a stored ETag; if the representation is unchanged the server returns 304 Not Modified with no body — headers only. On a safe method it is

a cache revalidation; with `If-Match` on PUT it becomes optimistic concurrency control.

- Q19. What does `Cache-Control: no-cache` actually mean?** “Store is allowed, but revalidate with the origin before every reuse.” It is chronically confused with `no-store` (never persist at all). Quoting the wrong one either leaks sensitive data or destroys cache efficiency.
- Q20. What is CORS preflight and who enforces it?** For non-simple cross-origin requests the browser sends `OPTIONS` with `Origin + Access-Control-Request-Method`; the server’s `Access-Control-Allow-*` headers authorize the real request. The *browser* enforces it — CORS is not a server-side security boundary; curl ignores it.
- Q21. Describe the HTTP/2 frame header.** Nine octets: 24-bit payload length, 8-bit type, 8-bit flags, 1 reserved bit + 31-bit stream identifier. Everything else in HTTP/2 is payload typed by those fields.
- Q22. Name all ten HTTP/2 frame types and their codes.** `DATA=0x0`, `HEADERS=0x1`, `PRIORITY=0x2`, `RST_STREAM=0x3`, `SETTINGS=0x4`, `PUSH_PROMISE=0x5`, `PING=0x6`, `GOAWAY=0x7`, `WINDOW_UPDATE=0x8`, `CONTINUATION=0x9`.
- Q23. What is the HTTP/2 connection preface?** The client magic `PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n` (24 octets) followed by a `SETTINGS` frame; it lets a server distinguish h2c from HTTP/1.1 on a shared port and seeds connection parameters.
- Q24. How does HTTP/2 multiplexing work?** One TCP connection carries many streams; every frame is tagged with a stream ID, so requests/responses interleave at frame granularity and one slow response no longer blocks others. Concurrency limits come from `SETTINGS_MAX_CONCURRENT_STREAMS`.
- Q25. What is HPACK and why does HTTP/2 need it?** HPACK compresses header blocks using a 61-entry static table, a per-connection dynamic table of recently used headers (default 4096 octets), and static Huffman coding. Without it, per-request textual headers (cookies alone can be kilobytes) would dominate small-payload traffic and defeat TCP’s initial congestion window.
- Q26. Why must HPACK header blocks be decoded in order, and how does QPACK fix it?** HPACK’s dynamic table is a shared ordered state machine; decoding block *n* requires having applied blocks $< n$. On QUIC’s independent streams that ordering would

re-couple streams, so QPACK moves table updates onto separate encoder streams and tracks acknowledgements, tolerating out-of-order delivery at the cost of possible encoder-delay stalls.

Senior Level

- Q27. Explain HTTP/2 flow control.** Credit-based, hop-by-hop: every DATA octet consumes credit from both the stream's window and the connection window (both default 65,535). Receivers replenish with WINDOW_UPDATE increments ($1..2^{31} - 1$); senders resize peer stream windows via SETTINGS_INITIAL_WINDOW_SIZE. Only DATA counts; a zero window stalls DATA but never HEADERS/PING/RST, so deadlocks are recoverable.
- Q28. What HOL blocking remains in HTTP/2?** TCP's ordered byte stream: one lost segment stalls all streams until retransmission, because TCP cannot deliver later bytes early. Multiplexing removed application-layer HOL only; the transport layer re-imposed it. On lossy links this can make h2 slower than parallel h1 connections.
- Q29. Why does HTTP/3 use UDP — isn't UDP unreliable?** UDP is deliberately minimal: QUIC reimplements reliability, ordering, and congestion control *in userspace* on top of it. Kernel TCP and middlebox ossification make deploying a new TCP-like transport impossible; UDP is the only ubiquitous substrate that permits transport evolution, per-stream independence, and fused TLS 1.3.
- Q30. How does QUIC connection migration work?** Connections are bound to Connection IDs, not the 5-tuple. When a client's IP/port changes (Wi-Fi→LTE), packets with the same Connection ID still identify the connection; the peer validates the new path (path validation challenge) and continues — no re-handshake, streams resume where they left off.
- Q31. TLS 1.3 handshake: which messages exist and which are encrypted?** ClientHello and ServerHello are cleartext (plus legacy record wrapping); from EncryptedExtensions onward — Certificate, CertificateVerify, Finished — everything is encrypted under handshake traffic keys derived from the ECDHE key share exchange completed by ServerHello.
- Q32. TLS 1.2 vs. 1.3 round trips?** 1.2 full handshake: 2-RTT; 1.3: 1-RTT because the client speculatively sends its `key_share` in ClientHello. Resumption: 1.2 gives 1-RTT; 1.3 gives 0-RTT early data —

which is replayable and therefore must be restricted to idempotent requests.

- Q33. What do SNI and ALPN each negotiate?** SNI (`server_name`) selects which certificate identity the server presents on a shared IP. ALPN selects the application protocol (`h2`, `http/1.1`) inside the handshake, avoiding a post-handshake negotiation round trip; failure to agree aborts with `no_application_protocol`.
- Q34. Walk certificate chain validation.** Verify the leaf's hostname against SANs, validity window, and signature using the presented intermediate(s); recurse until a locally trusted root anchor; enforce `basicConstraints CA=TRUE` and key-usage on intermediates; optionally check revocation (OCSP/CRL). Any failure aborts before the first request byte.
- Q35. What is request smuggling and how does HTTP/2 help?** Disagreement between two HTTP/1.1 parsers about message boundaries (CL.TE/TE.CL desync) lets an attacker prefix a hidden request onto the next victim's request. HTTP/2's binary length-prefixed framing removes textual ambiguity — though `h2→h1` downgrades reintroduce risk, so gateways must normalize strictly.
- Q36. Why was HTTP/2 server push deprecated?** Adoption was negligible and often harmful: servers pushed assets clients already cached, wasting bandwidth; cache-aware push never materialized; priority interactions were buggy. RFC 9113 deprecates it, browsers removed support, and 103 `Early Hints` covers the honest use case.
- Q37. How would you debug intermittent connection-reset errors behind an LB?** Suspect a keep-alive race first: compare client pool idle timeout, LB idle timeout, and server keep-alive timeout. Confirm with a packet capture showing FIN from the LB followed by a client request and RST. Fix by making the client timeout strictly shorter than the LB's.
- Q38. Design the retry policy for POST /payments over flaky HTTP/2.** Never transport-retry blindly. Require an idempotency key, retry only on connection-refused/pre-send failures without it, bound retries, honor `Retry-After/429`, jitter backoff, and make the server deduplicate on the key. The protocol tells you POST is not idempotent; the application must restore the property.
- Q39. When does 0-RTT help and when does it hurt?** It elimi-

nates handshake latency for resumed connections — valuable for mobile/high-RTT clients. It hurts when early data is replayed by a network attacker: restrict 0-RTT to safe, idempotent requests, use single-use tickets, and keep anti-replay windows tight.

Q40. Why do proxies strip `Connection` and `Keep-Alive`? They are hop-by-hop headers describing one link, not the path. Forwarding them would assert connection properties across hops the header knows nothing about, corrupting framing and tunnel semantics (RFC 9110 §7.6.1; HTTP/2 forbids them outright).

2.8 External Bibliography & Further Reading

Normative Specifications (read these first)

- RFC 9110, *HTTP Semantics* [Fie22a] — methods, status codes, header fields, content negotiation, conditional requests, caching semantics.
- RFC 9112, *HTTP/1.1* [Fie22b] — message syntax, framing, chunked coding, connection management.
- RFC 9113, *HTTP/2* [Tho22] — binary framing, streams, flow control, push deprecation.
- RFC 9114, *HTTP/3* [Bis22] — HTTP semantics over QUIC streams.
- RFC 9000, *QUIC* [Iye21] — the transport: packets, streams, loss recovery, connection IDs, migration.
- RFC 8446, *TLS 1.3* [Res18] — handshake messages, key schedule, 0-RTT.
- RFC 7541, *HPACK* [Peo15] — header compression for HTTP/2.
- RFC 9204, *QPACK* (inline; not in the shared bibliography) — field compression for HTTP/3.
- RFC 7231 and RFC 7540 (superseded by 9110/9113 but ubiquitous in older literature; inline reference).

Books

- Barry Pollard, *HTTP/2 in Action*, Manning, 2019 (inline reference) — the most practical h2 operations book; covers framing, HPACK, and real deployment pitfalls.
- Ilya Grigorik, *High Performance Browser Networking*, O'Reilly, 2013 (free at hpbn.co; inline reference) — TCP/TLS/HTTP latency mechanics; the canonical treatment of why round trips dominate.
- Martin Kleppmann, *Designing Data-Intensive Applications* [Kle17] — Chapter 1 and the distributed-systems framing for retries, idempotence, and exactly-once illusions.
- Michael T. Nygard, *Release It!*, 2nd ed. [Nyg18] — timeout, connection-pool, and stability patterns; the source of the keep-alive-mismatch failure genus.
- Roy T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures* [Fie00] — the dissertation; Chapter 5 defines REST, Chapter 6 motivates HTTP's design.

Documentation, Tools & Courses

- Mozilla MDN Web Docs, *HTTP* reference (inline) — best-maintained header/status-code reference.
- Wireshark TLS/QUIC dissectors and the `sslkeylog` (SSLKEYLOGFILE) workflow (inline) — decrypt your own lab traffic.
- `nghttp2` (`nghttp`, `h2load`) and `curl -http2/-http3` documentation (inline) — reference clients for protocol-version experiments.
- Cloudflare Learning Center and the *Smashing Magazine*/Cloudflare QUIC deployment post-mortems (inline) — real-world HTTP/3 rollout data.

2.9 Capstone Projects

2.9.1 Capstone 1 — From-Scratch HTTP/1.1 Server [junior]

Objective: Implement a minimal but correct HTTP/1.1 server over raw `socketserver`, serving the fictional Northwind Robotics parts catalog.

Inputs/Datasets: an in-repo JSON file `parts.json` (synthetic catalog, ≤ 50 entries).

Steps: (1) parse request-line and headers until CRLF; (2) implement GET/HEAD for `/parts` and `/parts/{id}`; (3) emit correct `Content-Length`, `Content-Type`, `Connection` semantics; (4) support keep-alive for at least 5 sequential requests per connection; (5) return 404/400/405 correctly.

Acceptance Criteria:

- ☐ `curl.exe -v` shows byte-correct status-line, headers, and framing.
- ☐ Ten sequential keep-alive requests on one connection all succeed.
- ☐ Malformed request-line yields 400; unknown path yields 404; POST yields 405.
- ☐ HEAD returns headers identical to GET with zero body bytes.

Stretch Goals: conditional GET with `ETag/If-None-Match` $\rightarrow$ 304; graceful Connection: `close`.

2.9.2 Capstone 2 — Chunked Streaming Proxy [mid]

Objective: Build a localhost reverse proxy that relays responses using `Transfer-Encoding: chunked` while enforcing defensive parsing.

Inputs/Datasets: Listing-2-style local origin server emitting chunked telemetry; `telemetry.csv` synthetic dataset.

Steps: (1) accept client connections; (2) forward request head verbatim; (3) re-chunk origin bytes in 256-octet chunks; (4) enforce max chunk bounds and CRLF validation; (5) log per-request byte counts with `logging`.

Acceptance Criteria:

- ☐ Client receives identical decoded body as direct origin fetch.
- ☐ Oversized or malformed chunk from origin is rejected with 502 and logged.
- ☐ Hop-by-hop headers are stripped per RFC 9110 §7.6.1.
- ☐ Runs fully offline; deterministic test script included.

Stretch Goals: trailer support (`Trailer: X-Checksum`); backpressure via bounded buffers.

2.9.3 Capstone 3 — HPACK-Lite Header Encoder [mid]

Objective: Implement a pure-stdlib encoder for a subset of HPACK: static-table indexing, literal-with-incremental-indexing, and integer prefix coding (RFC 7541 §5).

Inputs/Datasets: a fixed request-header corpus (`headers_corpus.json`, synthetic Northwind Robotics API calls).

Steps: (1) implement RFC 7541 integer encoding (N -bit prefixes); (2) match against the 61-entry static table; (3) maintain a bounded dynamic table with eviction; (4) emit hex dumps per header block; (5) measure bytes saved vs. HTTP/1.1 text across the corpus.

Acceptance Criteria:

- ☐ Integer encoder round-trips all corpus values, including prefix-overflow cases.
- ☐ Second occurrence of an identical header block encodes as a single indexed reference.
- ☐ Dynamic table never exceeds its configured size bound.
- ☐ Report shows $\geq 60\%$ byte reduction on the repeat-heavy corpus.

Stretch Goals: static Huffman coding; a decoder that round-trips your encoder.

2.9.4 Capstone 4 — Protocol Fingerprinting & Frame Dissection Toolkit [senior]

Objective: Extend Listing 4 into a passive analysis CLI that ingests captured byte streams (files), classifies the protocol (HTTP/1.1 text, h2 preface, TLS records), and emits a structured report.

Inputs/Datasets: a `captures\` folder of synthetic fixtures generated by your own scripts (HTTP/1.1 exchanges, h2 frame sequences, TLS record headers).

Steps: (1) sniff protocol by magic bytes/preface; (2) dispatch to per-protocol parsers; (3) for h2, emit per-frame CSV (type, flags, stream, length); (4) detect anomalies: CL+TE ambiguity, frames exceeding 16 KiB, RST floods; (5) exit codes suitable for CI.

Acceptance Criteria:

- ☐ Correctly classifies all provided fixtures with zero false positives.
- ☐ Flags a planted CL.TE smuggling capture and a truncated frame.

- ☐ Output is deterministic (stable ordering, no timestamps) for diff-based tests.
- ☐ `-json` mode emits machine-readable reports.

Stretch Goals: HPACK state reconstruction across HEADERS/CONTINUATION sequences; entropy-based flags for non-HTTP payloads.

2.9.5 Capstone 5 — HTTP Version Latency Profiler & Wire-Debugging Workbench [staff]

Objective: Build a local benchmark harness that quantifies connection-setup and per-request costs across HTTP/1.1 configurations (cold connection, keep-alive warm, pipelining simulation, chunked vs. fixed-length), producing the standard addendum’s benchmark table from real measurements.

Inputs/Datasets: your own local servers from Capstones 1–2; synthetic payload set (1 KB, 64 KB, 1 MB) in `payloads\`.

Steps: (1) harness spins up servers on ephemeral ports; (2) measure p50/p99 for each configuration with seeded, fixed iteration counts (no wall-clock assertions in tests — record, don’t assert); (3) simulate loss/latency with a socket-level delay shim; (4) emit the benchmark table as Markdown + CSV; (5) document how the methodology would extend to h2/h3 with the `h2/aioquic` libraries.

Acceptance Criteria:

- ☐ Harness runs offline end-to-end with one command and seeds all randomness.
- ☐ Report quantifies keep-alive benefit (warm vs. cold) for each payload size.
- ☐ Delay shim demonstrates HOL-blocking cost growth under injected latency.
- ☐ Benchmark table conforms to the chapter addendum template.

Stretch Goals: pluggable transport interface so a real h2 client (via `h2`) drops into the same harness; flame-graph-friendly structured logs.

2.10 Python Dependencies Appendix

The core listings in this chapter are **stdlib-only**. One optional dependency extends the laboratories:

```
1 # requirements.txt (chapter 1 - optional tooling only;  
   core is stdlib)  
2 h2>=4.1.0
```

2.10.1 h2 ($\geq 4.1.0$)

Description: A pure-Python, in-memory HTTP/2 protocol state machine (hyper-h2). It handles framing, HPACK, and flow control; you supply sockets and TLS.

Why included: Listings 1–4 teach the wire by hand; **h2** is the bridge to speaking *real* h2 against local servers in the stretch goals and Capstone 5 without implementing the full protocol stack.

Alternatives: **httpx**[**http2**] (full client, hides the frames — pedagogically backwards here); **aioquic** (if you extend to HTTP/3); raw **hyperframe** (lower level than needed).

Installation (PowerShell):

```
1 python -m venv .venv  
2 .\.venv\Scripts\Activate.ps1  
3 pip install -r requirements.txt
```

2.11 Chapter Summary & Key Takeaways

- HTTP/1.1 is a textual protocol framed by CRLF; every message boundary derives from **Content-Length**, chunked coding, or connection close — and ambiguity between the first two is a security vulnerability class (request smuggling), not a style question [Fie22b].
- Method properties (safe/idempotent/cacheable) and status-code precision (201 vs. 200 vs. 204; 400 vs. 422; 401/403/404; 409 vs. 412; 429/503 + **Retry-After**) are contracts that caches, retries, and intermediaries enforce — get them right and infrastructure works *for* you [Fie22a].
- HTTP/2 replaces text with a 9-octet frame header (24-bit length, 8-bit type, 8-bit flags, 31-bit stream ID), multiplexes streams over one TCP connection, compresses headers with HPACK, and flow-controls with dual windows — but inherits TCP’s transport-layer HOL blocking [Tho22; Peo15].
- HTTP/3/QUIC moves streams into userspace over UDP: per-stream loss independence, Connection-ID-based migration, and a fused 1-RTT

TLS 1.3 handshake with 0-RTT resumption for repeat clients [Bis22; Iye21; Res18].

- TLS 1.3’s six-message handshake, mandatory forward secrecy, and ALPN-negotiated protocol selection define both the security and the latency floor of every HTTPS call; keep-alive hit rate is therefore an economic metric, not a nicety.
- The dominant production failure modes are protocol misunderstandings: keep-alive timeout mismatches, blind non-idempotent retries, read-until-close framing bugs, and ignored back-pressure signals. Each is preventable with the mental models — and the four runnable tools — built in this chapter.

2.12 Quick-Reference Card

Method	Safe/Idem./Cache	Use for
GET	Y / Y / Y	Read a resource.
HEAD	Y / Y / Y	Metadata only (no body).
POST	N / N / rare	Create / process; needs idempotency key to retry.
PUT	N / Y / N	Full replacement at known URI.
PATCH	N / N / N	Partial update.
DELETE	N / Y / N	Remove; 204 on success.
OPTIONS	Y / Y / N	Capabilities / CORS preflight.
TRACE	Y / Y / N	Echo diagnostics (usually disabled).
CONNECT	N / N / N	Tunnel establishment.

Table 2.5: Method cheat sheet.

2.13 Standardized Evidence Addendum

2.13.1 Benchmark Snapshot Template

2.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.

Code	Meaning / when to emit
200 / 201 / 204	OK with body / created (+Location) / OK, no body.
301 / 302 / 307 / 308	Permanent / temporary redirect; 307/308 preserve method+body.
304	Not modified (conditional GET hit).
400 / 401 / 403 / 404	Malformed / unauthenticated / forbidden / not found (or masked).
405 / 406 / 409 / 412 / 413 / 415 / 422 / 429	Method not allowed / not acceptable / conflict / precondition failed / payload too large / unsupported media / semantic error / rate limited (+Retry-After).
500 / 502 / 503 / 504	Server fault / bad gateway / unavailable (+Retry-After) / gateway timeout.

Table 2.6: Status-code cheat sheet.

- **Result:** what changed in correctness, resilience, latency, and operability.

2.13.3 Cost and Latency Budget Snapshot

2.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

2.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,

Code	Frame	One-liner
0x0	DATA	Body octets; counts against both flow-control windows.
0x1	HEADERS	HPACK block; opens a stream.
0x2	PRIORITY	Deprecated dependency/weight hint.
0x3	RST_STREAM	Kill one stream (32-bit error code).
0x4	SETTINGS	Connection parameters; must be ACKed.
0x5	PUSH_PROMISE	Push reservation (deprecated in practice).
0x6	PING	Liveness / RTT (8 octets, ACK echo).
0x7	GOAWAY	Graceful shutdown; last-stream-id.
0x8	WINDOW_UPDATE	Flow-control credit increment.
0x9	CONTINUATION	Header-block spillover; needs END_HEADERS.

Table 2.7: HTTP/2 frame-type cheat sheet (header: 24-bit length, 8-bit type, 8-bit flags, 1 reserved bit, 31-bit stream ID).

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 2.8: Minimum benchmark table to keep per chapter.

- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

2.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 2.9: Operational budget template for this chapter’s technique.

5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

2.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 2.10: Decision card to complete per chapter.

2.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

2.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.

- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 3

Consuming APIs in Python: From urllib to httpx

Consuming an HTTP API looks deceptively simple: build a URL, send a request, read some JSON. That mental model survives exactly until the first production incident. A consumer that omits timeouts will, under a slow dependency, exhaust its worker pool and take down a healthy service. A consumer that retries blindly will multiply load on an already struggling server and convert a brown outage into a full one. A consumer that parses a response body before checking the status code will turn a perfectly meaningful 503 into a confusing `JSONDecodeError` three stack frames away from the real cause. This chapter treats API consumption as the engineering discipline it actually is.

We trace the evolution of Python HTTP clients — from the standard library’s `urllib`, through the ergonomic revolution of `requests` [Pyt24], to the modern `httpx` library with its unified sync/async API and HTTP/2 support [Enc24] — and extract the design lessons each generation teaches. We then build, piece by piece, a production-grade client: explicit connect/read/write/pool timeouts, bounded connection pools with keep-alive, retry logic with exponential backoff and full jitter (with the underlying mathematics derived, not hand-waved), honoring of `Retry-After` and `RateLimit` headers per RFC 9110 [Fie22a], typed pagination iterators, streaming downloaders with cancellation, and an error taxonomy that maps transport failures, status failures, and contract failures to distinct, actionable exceptions. Every listing is fully typed, runs on Python 3.11+, and is validated *offline* against a local stub server or `httpx.MockTransport`: no network, no cloud accounts, no API keys, no flakiness.

The capstone of the chapter is a resilient SDK wrapper around the fictional *Northwind Robotics Parts API*: typed `pydantic` response models [Col24], telemetry hooks, transport injection for hermetic tests, and a complete fault-matrix test suite simulating 500s, 429s with `Retry-After`,

and slow responses. The resilience philosophy throughout follows Nygard’s *Release It!* [Nyg18]: every integration point is a betrayal waiting to happen, so engineer the client to fail fast, fail visibly, and recover gracefully.

3.1 Learning Objectives & Prerequisites

3.1.1 Learning Objectives

After completing this chapter, you will be able to:

1. **Select** the appropriate Python HTTP client library (`urllib`, `requests`, `httpx`) for a given workload, justifying the choice in terms of sync vs. async support, HTTP/2 capability, and dependency footprint.
2. **Design** a robust client anatomy: base-URL joining, header and authentication injection, event hooks, and transport abstraction.
3. **Specify** a complete timeout policy (connect, read, write, pool) and explain why each dimension exists and what failure it bounds.
4. **Size** connection pools correctly, reasoning about keep-alive, DNS caching, TLS session reuse, and the cost of connection churn.
5. **Derive and implement** exponential backoff with full jitter, compute expected-delay bounds, enforce retry budgets, honor **Retry-After**, and gate retries behind idempotency analysis.
6. **Implement** client-side rate limiting (token bucket) that composes cleanly with server-driven 429 signals.
7. **Consume** offset, cursor, and link-header paginated endpoints as lazy typed iterators, and stream large payloads with progress reporting and cancellation.
8. **Classify** consumer-side errors into transport, status, and contract categories and map each to an actionable exception hierarchy.
9. **Build and test** a typed SDK wrapper with telemetry hooks and hermetic offline tests via `httpx.MockTransport`.

3.1.2 Prerequisites

This chapter assumes mastery of:

- **Chapter 0:** the curriculum’s tooling baseline — Python 3.11+, a virtual environment, `pip`, and PowerShell on Windows.

- **Chapter 1:** the HTTP wire protocol — methods, status-code classes, headers, message bodies, content negotiation, and the semantics of idempotent vs. non-idempotent methods as standardized in RFC 9110 [Fie22a].
- Core Python: type hints, context managers (`with`), iterators and generators, and the `logging` module.

No third-party package knowledge is assumed; each library is introduced from first principles. All code runs offline against the local stub server provided in Section 3.5 or against `httpx.MockTransport`.

Key Concept

The consumer’s contract. A production API client is not “code that calls an endpoint.” It is a managed boundary between your process and an unreliable network dependency. Its obligations are: bound every wait (timeouts), bound every resource (pools, concurrency, memory), bound every retry (budgets, jitter, idempotency), and make every failure observable and actionable. Everything in this chapter serves one of those four bounds.

3.2 Deep Technical Mechanics & Architectural Concepts

3.2.1 The Evolution of Python HTTP Clients

`urllib`: the stdlib baseline

Python ships with `urllib.request`, a functional but austere client. Its virtues are real: zero third-party dependencies, availability on every interpreter, and therefore suitability for constrained environments (installers, bootstrap scripts, single-file tools, frozen binaries). Its sharp edges are equally real:

- **A single global timeout** passed per call — no distinction between connect and read phases, and no pool-acquisition bound.
- **No session abstraction.** Every call re-resolves DNS, re-opens TCP, and re-performs the TLS handshake unless you hand-manage an `OpenerDirector`; keep-alive is not the default behavior of the high-level API.

- **Exceptions for non-2xx statuses.** `HTTPError` is raised for 4xx/5xx, which is defensible but surprises developers trained on `requests`, and the error doubles as a response object, a notorious source of confusion.
- **Manual everything:** JSON encoding/decoding, query-string construction, redirect policy, decompression negotiation, and proxy configuration are all the caller’s problem.

The lesson `urllib` teaches is foundational: *an HTTP client is a state machine over sockets*, and if the library does not manage that state for you, your application code will — badly, and usually at 3 a.m.

requests: the ergonomic revolution

`requests` [Pyt24] re-founded Python HTTP consumption on two ideas: a humane API (“HTTP for Humans”) and the **Session** object. A **Session** persists cookies, default headers, and — critically — a connection pool across requests, making keep-alive the default. Its limitations only became pressing a decade later: it is synchronous only, HTTP/1.1 only, built on `urllib3`, and its timeout model (a single value or a `(connect, read)` pair) does not cover write or pool-acquisition phases. For a synchronous, HTTP/1.1, request/response workload, `requests` remains a correct and battle-tested choice; its deficit is architectural headroom, not correctness.

httpx: the modern synthesis

`httpx` [Enc24] keeps the `requests`-shaped API and adds the features the modern era demands:

- **Dual sync/async clients** (`httpx.Client` and `httpx.AsyncClient`) sharing one API surface — the same instrumentation, retry, and pagination code paths serve threads and `asyncio` alike.
- **HTTP/2 support** (via the optional `h2` package), enabling multiplexed streams over a single connection.
- **A four-dimensional timeout model:** connect, read, write, and pool timeouts are independently configurable.
- **Transport injection:** `httpx.MockTransport` lets tests exercise the *entire* client stack — pooling, redirects, timeouts, hooks — without a socket ever being opened.
- **Event hooks:** per-client middleware for logging, tracing, and telemetry.

Capability	<code>urllib</code>	<code>requests</code>	<code>httpx</code>	Notes
Stdlib (no deps)	Yes	No	No	<code>urllib</code> wins constrained environments
Session / pooling	Manual	Yes	Yes	keep-alive default in both libraries
Async support	No	No	Yes	<code>httpx</code> : <code>asyncio</code> and <code>trio</code>
HTTP/2	No	No	Opt-in	requires <code>h2</code> extra
Timeout dimensions	1	2	4	<code>httpx</code> adds write + pool
Mock transport	No	Adapter-based	First-class	<code>MockTransport</code> runs the full stack
Type hints	Partial	Partial	Full	<code>httpx</code> ships <code>py.typed</code>

Table 3.1: HTTP client support matrix for Python 3.11+ consumers.

Selection heuristic. Use `urllib` only when third-party dependencies are forbidden and the call count is tiny. Use `requests` when the codebase is synchronous, HTTP/1.1 suffices, and the dependency already exists. Use `httpx` for anything new: async workloads, HTTP/2 upstreams, or any system where hermetic testing matters — which is to say, any system. The remainder of this chapter standardizes on `httpx`.

3.2.2 Anatomy of a Robust Client

A well-built client is a pipeline, not a function call. Figure 3.1 shows the stages every request traverses in the production client we build in Section 3.3.

The pipeline’s constituent responsibilities:

Base URL handling. Store the base URL once; join paths with a `httpx.URL` (or `urljoin` semantics) rather than string concatenation. Trailing-slash bugs (`api.example.com/v1` + `/parts`) are the single most common client configuration defect. `httpx.Client(base_url=...)` implements the correct merge semantics.

Header management. Set protocol headers (`Accept`, `Accept-Encoding`, `User-Agent`) at client construction. A descriptive `User-Agent` (product, version, contact) is professional courtesy — and often a provider

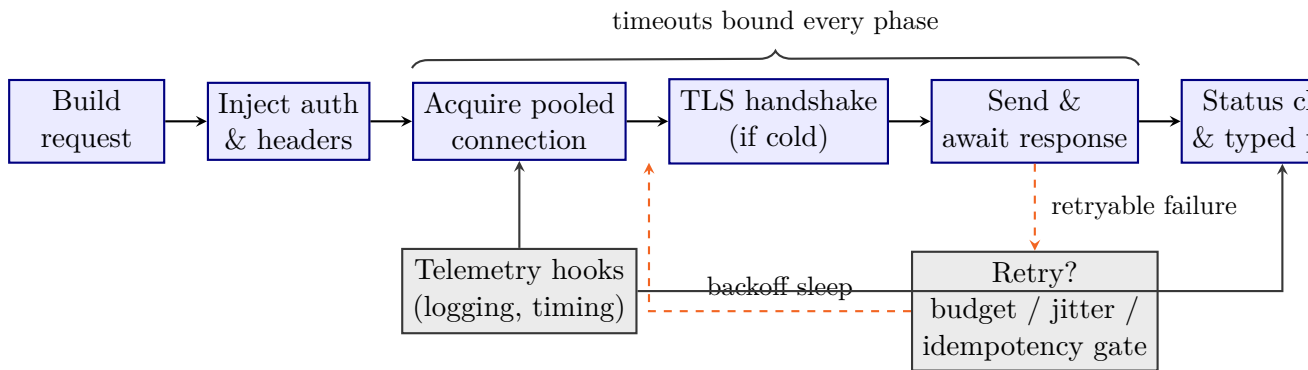


Figure 3.1: Client request lifecycle pipeline. Every phase between pool acquisition and response receipt is bounded by an explicit timeout; failures route through a retry gate that enforces budget, jitter, and idempotency; telemetry hooks observe every stage.

requirement — so your traffic is identifiable when you are accidentally the source of an incident.

Authentication injection. Credentials live in exactly one place: an `auth` callable or a constructor-level header. Never sprinkle `headers={"Authorization": ...}` across call sites, and never log the `Authorization` header (see Section 3.4).

Hooks / middleware. `httpx` event hooks (`request`, `response`) are the seam for telemetry, trace-context propagation, and cross-cutting policy — without polluting business logic.

3.2.3 Timeout Taxonomy: Bounding Every Wait

Definition 3.1 (Timeout). A *timeout* is an upper bound on the wall-clock time a specific phase of an operation may consume before the operation is aborted and the phase’s resources are reclaimed. A timeout is a correctness property, not a performance tuning knob: it converts an unbounded wait (a hang) into a bounded failure (an exception).

A request without a timeout is a thread (or task) that can be held *forever*. Nygard documents the canonical failure mode [Nyg18]: a dependency degrades until responses slow to a crawl; every worker in the calling service blocks awaiting sockets that will never answer; the caller’s pool exhausts; the caller stops serving its own traffic; the outage propagates upstream. The dependency’s partial failure becomes the consumer’s total failure — a cascading failure enabled entirely by the missing timeout. Hence the industry adage we elevate to law:

Warning

There is no safe default of “no timeout.” `urllib`’s global default is *infinite*; `requests` defaults to *infinite*; `httpx` defaults to 5 seconds and must be explicitly disabled to remove. Any code review that finds a bare `requests.get(url)` or `urlopen(url)` with no timeout has found a latent severity-1 incident. “It has never hung” is a statement about the past, not a property of the system.

The four timeout dimensions

`httpx.Timeout` exposes the complete taxonomy [Enc24]:

connect — time to establish the connection: DNS resolution, TCP handshake, and TLS negotiation. Bounds a dead or firewalled host. Typical: 1–5 s. Connects to a healthy host on a healthy network complete in tens of milliseconds; a 5 s connect timeout is already generous.

read — maximum gap while awaiting bytes from the peer: time to first byte *and* inter-chunk gaps while streaming a body. Bounds a hung application behind a healthy load balancer — the failure mode TCP cannot see, because the connection is fine and the peer simply never sends. This is the timeout whose absence causes the war story in Section 3.5.3. Typical: 5–30 s, aligned to the endpoint’s documented p99.

write — maximum gap while sending the request body. Matters for uploads and large POST bodies; bounds a peer that stops reading. Typical: 5–30 s.

pool — time to wait for a connection from a saturated pool. Bounds queueing delay when concurrency exceeds pool size, turning silent queueing (head-of-line blocking that looks like a hang) into a loud, actionable `PoolTimeout`. Typical: 1–5 s.

Per-call vs. client defaults. Set a conservative default on the client (e.g., connect 2 s, read 10 s, write 10 s, pool 2 s) and tighten or loosen per call where the endpoint contract justifies it — a report-generation endpoint may legitimately need a 120 s read timeout, but that exception must be explicit at the call site, not global. A default of “whatever the library ships” is an abdication; a default of “infinite” is a defect.

3.2.4 Connection Pooling, Keep-Alive, and the Cost of Churn

Establishing a fresh HTTPS connection costs: one DNS lookup (typically cached, 1–50 ms when not), one TCP handshake (1 RTT), and one TLS 1.3 handshake (1 RTT more, plus certificate validation and CPU for asymmetric cryptography). On a cross-country path at 60 ms RTT, a cold connection adds ~120–200 ms before the first request byte flows — and the full handshake’s key-exchange cost is paid again for every request. HTTP keep-alive amortizes all of it: the pool holds idle connections open and hands them to subsequent requests, reducing per-request overhead to the request/response exchange itself.

`httpx.Limits` exposes the two sizing levers [Enc24]:

`max_connections` — total connections the pool will hold. Size it to the peak concurrent requests your process will issue to *all* hosts. Default: 100.

`max_keepalive_connections` — connections retained idle for reuse. Size it to your steady-state concurrency against this client. Default: 20. A value too small causes *churn*: connections are closed on response completion and rebuilt on the next request, re-paying handshake costs under load.

Three subtleties separate a correctly tuned pool from a cargo-culted one:

1. **DNS caching.** The OS resolver cache is outside Python’s control; long-lived pools naturally avoid re-resolution because keep-alive connections skip DNS entirely. For short-lived connections, resolution happens per connection, and a flaky resolver becomes a latency multiplier.
2. **TLS session reuse.** TLS 1.3 session tickets allow *resumed* handshakes (0-RTT/1-RTT reduction) on new connections to the same host; libraries built on the stdlib `ssl` module benefit when a shared `SSLContext` is used. Keep-alive still dominates: the cheapest handshake is the one you never perform.
3. **One client per base URL, long-lived.** Create the `Client` once (module scope or dependency-injected), reuse it for the process lifetime, and close it deterministically (context manager or shutdown hook). Creating a client per request discards the pool — the single most common `httpx` misuse — while never closing clients leaks sockets until the OS reclaims them.

3.2.5 Retry Engineering: Backoff, Jitter, Budgets, Idempotency

Retries exist because a large class of failures is *transient*: packet loss, a restarting pod, a load balancer draining a node, a brief 502 from a proxy mid-deploy. Retrying converts transient failure into success. Retrying *wrong* converts a partial outage into a total one — the retry storm of Section 3.5.3.

Exponential backoff and full jitter: the math

Let attempts be numbered $n = 1, 2, \dots, N$ after the initial call, with base delay b and cap c . Uncapped exponential backoff waits

$$W_n = b \cdot 2^{n-1}, \quad (3.1)$$

so with $b = 1$ s the schedule is 1, 2, 4, 8, 16, ... seconds. Exponential spacing buys the server recovery time, but it has a fatal flaw under correlated failure: when an outage ends, *every* queued client wakes on the same schedule and re-hammers the server in synchronized waves — the *thundering herd*.

Full jitter (popularized by the AWS Architecture Blog, see Section 3.8) draws each delay uniformly from a range:

$$W_n \sim \text{Uniform}(0, \min(c, b \cdot 2^{n-1})). \quad (3.2)$$

Because a uniform variable on $[0, m]$ has mean $m/2$, the expected per-attempt delay while the cap is inactive is

$$\mathbb{E}[W_n] = \frac{b \cdot 2^{n-1}}{2}, \quad \mathbb{E}\left[\sum_{n=1}^N W_n\right] = \frac{b}{2} (2^N - 1), \quad (3.3)$$

exactly half the deterministic schedule, with the same exponential growth profile. Jitter sacrifices no meaningful aggressiveness while decorrelating clients: collision probability across k concurrent retriers drops from ~ 1 (deterministic) to $\sim 1/(\text{range width})$. A variant, *decorrelated jitter*, samples $W_n \sim \text{Uniform}(b, 3W_{n-1}) \wedge c$, coupling consecutive waits instead of attempt indices; it spreads load comparably and converges faster after short blips. Both are vastly superior to unjittered or “equal jitter” schedules; full jitter is the simplest to reason about and is what we implement.

Figure 3.3 shows the same policy as a timeline for a single request’s life.

Retry budgets

Per-request caps ($N \leq 3\text{--}5$ attempts, a total-time ceiling) bound the blast radius of one call. They do not bound the *fleet*: if 30% of all calls are failing, a retry-everything policy multiplies offered load by up to N and finishes

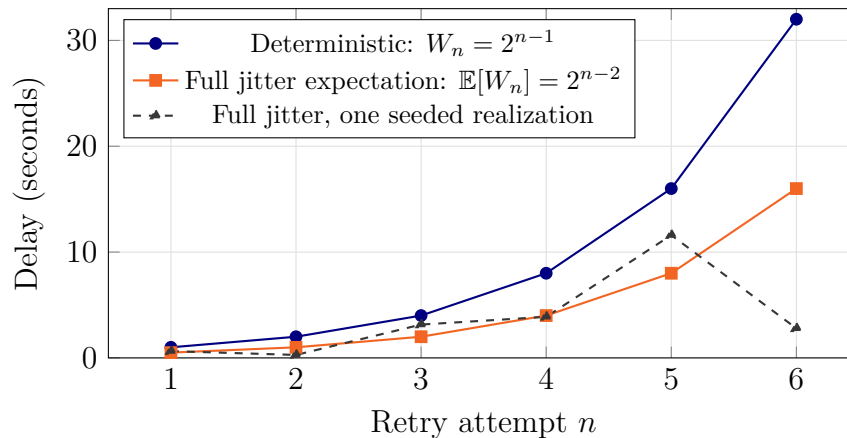


Figure 3.2: Backoff schedules for $b = 1$ s. Deterministic backoff grows exponentially but synchronizes clients; full jitter halves the expected delay and, crucially, decorrelates concurrent retrievers (one seeded realization shown). Production policy caps both at c (here 30 s).

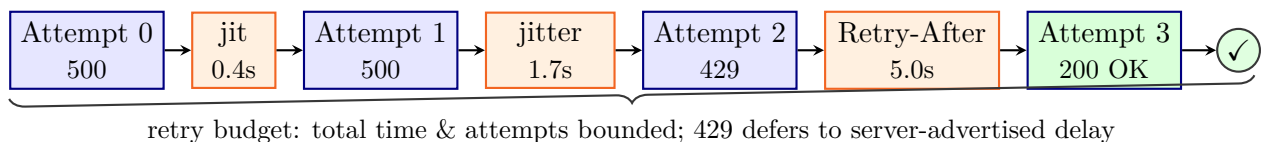


Figure 3.3: Retry timeline for one request. Transient 500s consume full-jitter backoff slots; a 429 overrides the schedule with the server’s **Retry-After**; the budget caps total attempts and elapsed time.

the server off. A **retry budget** rate-limits retries themselves — e.g., “no more than 10% of issued requests per minute may be retries” — typically implemented as a shared token bucket. When the budget is empty, fail fast and let the circuit breaker (Chapter 15) take over; the two patterns compose: retries handle transient blips, the breaker handles sustained failure, the budget prevents retries from feeding the fire.

Retry-After and which statuses merit retry

RFC 9110 [Fie22a] defines **Retry-After** (on 429 and 503) as the server’s explicit request for a delay, in seconds or HTTP-date. A server that tells you when to come back is giving you better information than any locally computed backoff; **always honor it** (clamped to a sane ceiling) when present. Retry policy by status class:

- **Retry:** 408, 425, 429, and 5xx (with skepticism for 501/505), plus transport errors (connect failures, connection resets, read timeouts).

- **Never retry (as-is):** 400, 401, 403, 404, 409, 422 — these are deterministic; the identical request will fail identically.

The idempotency gate

A retry replays a request. Replaying is safe only when replay cannot change the outcome: per RFC 9110 [Fie22a], `GET`, `HEAD`, `PUT`, `DELETE`, and `OPTIONS` are idempotent by method semantics. `POST` is not — a retried `POST /orders` may create two orders, and a read timeout is the treacherous case: the server may have processed the request *and then* the response was lost, so the client’s “failure” is the server’s “success.” The professional escape hatches, in order of preference:

1. **Idempotency keys:** the API accepts a client-generated `Idempotency-Key` header and deduplicates replays server-side (Chapter 10 covers the provider side). With a key, retrying a `POST` becomes safe.
2. **Natural idempotency:** the operation is semantically repeatable (“set status to X”), verifiable from the endpoint contract.
3. **No retry:** when neither holds, propagate the failure and let a human or a reconciliation job decide. A client that guesses wrong here causes data corruption, the worst failure class there is.

3.2.6 Rate-Limit-Aware Consumption

Server-side rate limiting (Chapter 14) protects the provider; client-side throttling is the consumer’s half of the contract. A well-behaved client:

1. **Proactively paces itself** below the published limit with a token bucket: a bucket of capacity B refills at rate r tokens/s; each request consumes one token; an empty bucket blocks until a token accrues. Sustained throughput converges to r with burst tolerance B .
2. **Reacts to signals:** on 429, read `Retry-After` and the `RateLimit-*` header family (limit, remaining, reset), back the bucket off, and retry only after the advertised window.
3. **Never compensates for a 429 by raising parallelism** — the queue is not shorter because you knocked louder.

Composing the two layers (proactive bucket, reactive retry policy) yields a client that stays under the limit in steady state and recovers gracefully when it mis-estimates.

3.2.7 Pagination and Streaming

APIs paginate to bound response size; consumers must abstract the mechanics behind a lazy iterator so call sites write `for part in client.parts():` and nothing more. The three dominant patterns (Chapter 8 designs them; here we consume them):

Offset/limit — `?offset=100&limit=50`. Simple, stateless, but unstable under concurrent writes (rows shift between pages) and increasingly expensive for deep offsets. Termination: a short page or an empty one.

Cursor — `?cursor=eyJpZCI6...`. Opaque, stable, cheap at depth. The cursor must be treated as an opaque token — never parsed, never fabricated. Termination: absent `next_cursor`.

RFC 8288 Link headers — `Link: <...?page=3>; rel="next"`. Self-describing and cache-friendly; termination is the absence of `rel="next"`.

For large single payloads (catalog exports, firmware images), do not buffer: `client.stream(...)` plus `response.iter_bytes()` yields chunks as they arrive, keeping memory $O(\text{chunk size})$ instead of $O(\text{payload size})$. Three production details: (1) `httpx` transparently negotiates and applies `gzip/br` decompression when the codecs are installed — verify `Content-Encoding` handling rather than assuming it; (2) progress reporting must be driven by bytes received against `Content-Length`, tolerating its absence (chunked responses have none); (3) cancellation must propagate — closing the stream or cancelling the task must release the connection promptly.

3.2.8 An Error Taxonomy for Consumers

“The API call failed” is not an error message; it is a shrug. Actionable consumers classify failure along three orthogonal layers, because each layer has a different owner and a different remedy:

We therefore define a small exception hierarchy — `ApiClientError` at the root with `ApiTransportError`, `ApiStatusError` (subclassed as `ApiClientError4xx`, `ApiServerError5xx`, `ApiRateLimited` carrying `retry_after`), and `ApiContractError` — so callers can *except* precisely the category they can act on. Kleppmann’s systems lens [Kle17] generalizes the point: at a network boundary, distinguish “the request failed,” “the request succeeded,” and “*we cannot know*” — and design each path explicitly.

Layer	Examples	Meaning	Action
Transport	ConnectTimeout, ReadTimeout, ConnectError, pool exhaustion	Request never completed, or response never arrived	Retry (if idempotent); check network, DNS, TLS; alert on sustained rate
Status (HTTP)	4xx, 5xx after a complete exchange	Server answered and refused/failed	4xx: fix the request (deterministic). 5xx/429: retry per policy; page if sustained
Contract / parse	JSON decode failure, schema violation, missing field	200 OK but the payload broke the contract	Never retry. Raise loudly; version skew or a provider defect — treat as a bug

Table 3.2: Consumer error taxonomy. Each layer maps to a distinct exception type and remediation path.

3.3 Production-Ready Code Implementation

All listings target Python 3.11+, are fully type-annotated, and run offline. The demonstration server is the local stub in Listing 3.6; every test uses `httpx.MockTransport`. Run them with `python` (listings) and `pytest -q` (test suite) from a virtual environment configured per Section 3.10.

3.3.1 Baseline: A `stdlib urllib` Consumer

Listing 3.1 is the deliberate “before” picture: correct, dependency-free, and studded with the sharp edges catalogued in Section 3.2.1. Read it and enumerate what production would demand that it lacks.

Example

Reading the baseline. As you read Listing 3.1, annotate: Where are the timeout dimensions? Where is keep-alive? What happens on a 429? Where would authentication live? What is logged? The gaps you find are the design brief for the `httpx` client that follows.

Listing 3.1: `ch02_urllib_baseline.py` — `stdlib` baseline.

```

1 """ch02_urllib_baseline.py -- stdlib urllib consumer (
   baseline, with sharp edges).
2

```

```

3 Run against the local stub: python ch02_stub_server.py
  (in another shell)
4 Then: python ch02_urllib_baseline.py
5 """
6 from __future__ import annotations
7
8 import json
9 import logging
10 import urllib.error
11 import urllib.request
12 from dataclasses import dataclass
13 from typing import Any
14
15 logger = logging.getLogger("ch02.urllib_baseline")
16
17 BASE_URL = "http://127.0.0.1:8765" # local stub only
  -- never the internet
18
19
20 @dataclass(frozen=True)
21 class Part:
22     part_id: str
23     name: str
24     unit_price_cents: int
25
26
27 def fetch_part(part_id: str, timeout_s: float = 5.0) ->
  Part:
28     """Fetch one part with urllib. NOTE the single,
  flat timeout.
29
30     Sharp edges on display:
31     * one timeout covers DNS+connect+read; no write/
  pool dimension;
32     * no session -> no keep-alive: DNS/TCP/TLS are re-
  -paid per call;
33     * HTTPError is raised for 4xx/5xx AND doubles as
  a response object;
34     * JSON, status classification, and retry are all
  manual.
35     """
36     url = f"{BASE_URL}/parts/{part_id}"
37     req = urllib.request.Request(url, headers={"Accept":
  "application/json"})
38     try:

```

```

39         with urllib.request.urlopen(req, timeout=
40             timeout_s) as resp:
41             status: int = resp.status
42             raw: bytes = resp.read() # unbounded: a
43                                     hostile peer can exhaust RAM
44     except urllib.error.HTTPError as exc:
45         # HTTPError is simultaneously the exception and
46         the response.
47         logger.error("HTTP%s_for%s", exc.code, url)
48         raise RuntimeError(f"status_{exc.code}") from
49             exc
50     except urllib.error.URLError as exc:
51         # Transport failure (DNS, refused connection,
52         timeout -> socket.timeout
53         # wrapped inside .reason). Nothing
54         distinguishes the sub-causes here.
55         logger.error("transport_failure_for%s:%s",
56             url, exc.reason)
57         raise RuntimeError(f"transport:_{exc.reason}")
58             from exc
59
60     if status != 200:
61         raise RuntimeError(f"unexpected_status_{status}"
62             ")
63     payload: dict[str, Any] = json.loads(raw.decode("
64         utf-8"))
65     return Part(
66         part_id=str(payload["part_id"]),
67         name=str(payload["name"]),
68         unit_price_cents=int(payload["unit_price_cents"
69             ]),
70     )
71
72 def main() -> None:
73     logging.basicConfig(level=logging.INFO, format="%(
74         levelname)s %(name)s: %(message)s")
75     part = fetch_part("NR-1001")
76     logger.info("got_part:_%s", part)
77
78 if __name__ == "__main__":
79     main()

```

3.3.2 A Production-Grade httpx Client

Listing 3.2 implements the pipeline of Figure 3.1: four-dimension timeouts, bounded pooling, single-point auth injection, event-hook telemetry, the error taxonomy of Table 3.2, and transport injection so the identical object runs against production URLs or an in-memory mock.

Listing 3.2: `ch02_httpx_client.py` — production client core.

```
1  """ch02_httpx_client.py -- production-grade httpx
   client for the
2  Northwind Robotics Parts API.
3
4  Design contract:
5  ** explicit 4-dimension timeouts (connect/read/write/
   pool); nothing infinite;
6  ** bounded connection pool with keep-alive; one long-
   lived client instance;
7  ** auth injected in exactly one place, never logged;
8  ** typed error taxonomy: transport vs status vs
   contract;
9  ** telemetry via event hooks (latency + status, header
   -safe logging);
10 ** transport injection (httpx.MockTransport) for
   hermetic offline tests.
11 """
12 from __future__ import annotations
13
14 import logging
15 import time
16 from typing import Any
17
18 import httpx
19
20 logger = logging.getLogger("ch02.httpx_client")
21
22 RETRYABLE_STATUSES: frozenset[int] = frozenset({408,
23         425, 429, 500, 502, 503, 504})
24
25 # ---- Error taxonomy
26
27 class ApiClientError(Exception):
28     """Root of all errors raised by this client."""
29
```



```

30
31 class ApiTransportError(ApiClientError):
32     """The request never completed: DNS, TCP, TLS,
33         timeout, pool exhaustion."""
34
35 class ApiStatusError(ApiClientError):
36     """The server answered with a non-2xx status."""
37
38     def __init__(self, message: str, *, status_code:
39         int) -> None:
40         super().__init__(message)
41         self.status_code = status_code
42
43 class ApiClientError4xx(ApiStatusError):
44     """Deterministic caller fault; retrying as-is is
45         futile."""
46
47 class ApiServerError5xx(ApiStatusError):
48     """Transient server fault; a candidate for retry
49         per policy."""
50
51 class ApiRateLimited(ApiStatusError):
52     """429: carries the server-advertised delay when
53         present."""
54
55     def __init__(self, message: str, *, status_code:
56         int, retry_after: float | None) -> None:
57         super().__init__(message, status_code=
58             status_code)
59         self.retry_after = retry_after
60
61 class ApiContractError(ApiClientError):
62     """2xx received, but the payload violates the
63         expected contract."""
64
65 # ---- Telemetry hooks
66 -----
67
68 def _log_request(request: httpx.Request) -> None:

```

```
66     # NEVER log request.headers: Authorization lives
67     # there.
68     logger.info("request_%s_%s", request.method,
69                 request.url.path)
70
71 def _log_response(response: httpx.Response) -> None:
72     # Latency is timed in _request() with time.
73     # monotonic(): response.elapsed
74     # is only populated after the body is read, which a
75     # hook must not force
76     # (it would buffer streaming responses).
77     logger.info(
78         "response_%s_%s->%d",
79         response.request.method, response.request.url.
80         path, response.status_code,
81     )
82
83 # ---- The client
84 -----
85
86 class NorthwindPartsClient:
87     """Typed, resilient consumer for the Northwind
88     Robotics Parts API."""
89
90     def __init__(
91         self,
92         base_url: str,
93         api_key: str,
94         *,
95         transport: httpx.BaseTransport | None = None,
96         # inject MockTransport in tests
97         timeout: httpx.Timeout | None = None,
98         limits: httpx.Limits | None = None,
99     ) -> None:
100         if not base_url:
101             raise ValueError("base_url must be non-
102                               empty")
103         if not api_key:
104             raise ValueError("api_key must be non-empty
105                               ")
106         self._client = httpx.Client(
107             base_url=base_url.rstrip("/"),
108             # Auth lives HERE and nowhere else.
```

```

101         headers={
102             "Authorization": f"Bearer_{api_key}",
103             "Accept": "application/json",
104             "User-Agent": "northwind-parts-sdk/1.0_
              (ops@northwind-robotics.example)",
105         },
106         timeout=timeout or httpx.Timeout(
107             connect=2.0, read=10.0, write=10.0,
108             pool=2.0
109         ),
110         limits=limits or httpx.Limits(
111             max_connections=20,
112             max_keepalive_connections=10,
113             keepalive_expiry=30.0,
114         ),
115         event_hooks={"request": [_log_request], "
              response": [_log_response]},
116         transport=transport,
117     )
118
119     # Context-manager protocol: deterministic socket
120     lifecycle.
121     def __enter__(self) -> "NorthwindPartsClient":
122         return self
123
124     def __exit__(self, *exc_info: object) -> None:
125         self.close()
126
127     def close(self) -> None:
128         self._client.close()
129
130     # Core request path
131     -----
132
133     def _request(self, method: str, path: str, **kwargs
134         : Any) -> httpx.Response:
135         started = time.monotonic()
136         try:
137             response = self._client.request(method,
138                 path, **kwargs)
139         except httpx.TimeoutException as exc:
140             kind = type(exc).__name__ # ConnectTimeout
141                 / ReadTimeout / PoolTimeout / ...
142             logger.warning("timeout_{%s}_on_{%s}_{%s}",
143                 kind, method, path)

```

```

136         raise ApiTransportError(f"{kind}_on_{method}
137                                _{path}") from exc
138     except httpx.TransportError as exc:
139         logger.warning("transport_error_on_%s_%s:_%s",
140                        method, path, exc)
141         raise ApiTransportError(f"transport_error_on_{method}_{path}:_{exc}") from exc
142     finally:
143         logger.debug(
144             "call_%s_%s_took_%.1fms",
145             method, path, (time.monotonic() -
146                           started) * 1000.0,
147         )
148     self._raise_for_status(response)
149     return response
150
151 @staticmethod
152 def _raise_for_status(response: httpx.Response) ->
153     None:
154     code = response.status_code
155     if code < 400:
156         return
157     message = f"{response.request.method}_{response
158                .request.url.path}_->_{code}"
159     if code == 429:
160         retry_after = _parse_retry_after(response.
161                                           headers.get("Retry-After"))
162         raise ApiRateLimited(message, status_code=
163                               code, retry_after=retry_after)
164     if code >= 500:
165         raise ApiServerError5xx(message,
166                                  status_code=code)
167     raise ApiClientError4xx(message, status_code=
168                              code)
169
170 # Public API
171 -----
172
173 def get_json(self, path: str, **kwargs: Any) ->
174     dict[str, Any]:
175     response = self._request("GET", path, **kwargs)
176     try:
177         payload = response.json()
178     except ValueError as exc: # 200 OK but not
179         JSON: contract breach.

```

3.3. Production-Ready Code Implementation

```
168         raise ApiContractError(f"{path}: invalid JSON body") from exc
169     if not isinstance(payload, dict):
170         raise ApiContractError(f"{path}: expected JSON object, got {type(payload).__name__}")
171     return payload
172
173     def get_part(self, part_id: str) -> dict[str, Any]:
174         if not part_id:
175             raise ValueError("part_id must be non-empty")
176         return self.get_json(f"/parts/{part_id}")
177
178
179     def _parse_retry_after(value: str | None, *, ceiling_s: float = 60.0) -> float | None:
180         """Parse a delta-seconds Retry-After header, clamped to a ceiling."""
181         if value is None:
182             return None
183         try:
184             seconds = float(value) # delta-seconds form;
185                                     HTTP-date form omitted for brevity
186         except ValueError:
187             return None
188         return max(0.0, min(seconds, ceiling_s))
189
190     def main() -> None:
191         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(name)s: %(message)s")
192         # The demo key is synthetic; the stub accepts any non-empty bearer token.
193         with NorthwindPartsClient("http://127.0.0.1:8765", api_key="demo-key-nr-001") as client:
194             part = client.get_part("NR-1001")
195             logger.info("part: %s", part)
196
197
198     if __name__ == "__main__":
199         main()
```

3.3.3 Retry Engineering with tenacity

Listing 3.3 layers retry policy onto the client using `tenacity`. The policy encodes exactly the rules of Section 3.2.5: full jitter via `wait_random_exponential` (mean $b/2$ doubling to the cap, matching Equation 3.3), `Retry-After` honored and clamped when a 429 advertises it, a combined attempt-plus-elapsed budget, and an idempotency gate that refuses to replay non-idempotent methods lacking an idempotency key. Determinism for tests is achieved by injecting the random source and a no-op sleep.

Listing 3.3: `ch02_retry.py` — jitter, `Retry-After`, idempotency gate.

```

1  """ch02_retry.py -- retry wrapper: full jitter + Retry-
   After + idempotency gate.
2
3  Policy(matches Section 3.5):
4  ** retry on ApiTransportError, ApiServerError5xx,
   ApiRateLimited;
5  ** full jitter: wait ~ Uniform(0, min(cap, base * 2**(
   n-1))), base=0.5s, cap=8s;
6  ** on 429 with Retry-After, the server-advertised
   delay OVERRIDES the jitter;
7  ** budget: at most 4 attempts AND at most 20s total
   elapsed;
8  ** idempotency gate: non-idempotent methods retry only
   with an
9  Idempotency-Key header;
10 ** deterministic tests: inject 'rng' and 'sleep'.
11 """
12 from __future__ import annotations
13
14 import logging
15 import random
16 from collections.abc import Callable
17
18 import httpx
19 from tenacity import (
20     RetryCallState,
21     Retrying,
22     retry_if_exception_type,
23     stop_after_attempt,
24     stop_after_delay,
25 )
26
27 from ch02_httpx_client import (
28     ApiRateLimited,
29     ApiServerError5xx,

```

```

30     ApiTransportError,
31     NorthwindPartsClient,
32 )
33
34 logger = logging.getLogger("ch02.retry")
35
36 BASE_DELAY_S = 0.5
37 CAP_DELAY_S = 8.0
38 MAX_ATTEMPTS = 4
39 MAX_ELAPSED_S = 20.0
40
41 SAFE_METHODS: frozenset[str] = frozenset({"GET", "HEAD",
42     , "OPTIONS", "PUT", "DELETE"})
43
44 RETRYABLE = (ApiTransportError, ApiServerError5xx,
45     ApiRateLimited)
46
47
48 def wait_policy(state: RetryCallState, *, rng: random.
49     Random) -> float:
50     """Retry-After from a 429 wins; otherwise full
51         jitter on exponential backoff.
52
53     tenacity calls this between attempts with the just-
54         failed attempt's state,
55     so the server-advertised delay is read straight off
56     the raised exception.
57     """
58     exc = state.outcome.exception() if state.outcome is
59         not None else None
60     if isinstance(exc, ApiRateLimited) and exc.
61         retry_after is not None:
62         logger.info("429 received: honoring Retry-After
63             of %.1fs", exc.retry_after)
64         return exc.retry_after
65     upper = min(CAP_DELAY_S, BASE_DELAY_S * (2 ** (
66         state.attempt_number - 1)))
67     delay = rng.uniform(0.0, upper)
68     logger.info("attempt %d failed; full-jitter backoff
69         of %.2fs",
70             state.attempt_number, delay)
71     return delay
72
73
74 class ResilientPartsClient:
75     """NorthwindPartsClient + retry policy. Retryable
76         failures only."""

```

```
64
65     def __init__(
66         self,
67         inner: NorthwindPartsClient,
68         *,
69         rng: random.Random | None = None,
70         sleep: Callable[[float], None] | None = None,
71     ) -> None:
72         self._inner = inner
73         # Seeded RNG and injectable sleep keep tests
74         # deterministic & instant.
75         self._rng = rng or random.Random(1337)
76         self._sleep = sleep # None -> tenacity's real
77                             # sleep (production)
78
79     def request_with_retry(
80         self,
81         method: str,
82         path: str,
83         *,
84         idempotency_key: str | None = None,
85         **kwargs: object,
86     ) -> httpx.Response:
87         method = method.upper()
88         if method not in SAFE_METHODS and not
89             idempotency_key:
90             raise ValueError(
91                 f"refusing to auto-retry non-idempotent
92                   {method} without
93                   "an idempotency key"
94             )
95         if idempotency_key:
96             headers = dict(kwargs.pop("headers", {}) or
97                               {})
98             headers["Idempotency-Key"] =
99                 idempotency_key
100             kwargs["headers"] = headers
101
102         retrying = Retrying(
103             retry=retry_if_exception_type(RETRYABLE),
104             wait=lambda state: wait_policy(state, rng=
105                 self._rng),
106             stop=stop_after_attempt(MAX_ATTEMPTS) |
107                 stop_after_delay(MAX_ELAPSED_S),
108             sleep=self._sleep if self._sleep is not
109                 None else _real_sleep,
```



```

101         reraise=True,
102     )
103     return retrying(self._inner._request, method,
104                     path, **kwargs)
105
106 def _real_sleep(seconds: float) -> None:
107     import time
108
109     time.sleep(seconds)
110
111
112 def main() -> None:
113     logging.basicConfig(level=logging.INFO, format="%(
114         levelname)s %(name)s: %(message)s")
115     inner = NorthwindPartsClient("http://127.0.0.1:8765
116         ", api_key="demo-key-nr-001")
117     resilient = ResilientPartsClient(inner)
118     try:
119         response = resilient.request_with_retry("GET",
120         "/parts/NR-1001")
121         logger.info("status=%d body=%s", response.
122             status_code, response.text[:80])
123     finally:
124         inner.close()
125
126 if __name__ == "__main__":
127     main()

```

3.3.4 Pagination Iterators and a Streaming Downloader

Listing 3.4 hides pagination mechanics behind lazy typed iterators (cursor and link-header variants) and adds a streaming downloader with byte-level progress and prompt cancellation.

Listing 3.4: ch02_pagination_stream.py — iterators and streaming.

```

1  """ch02_pagination_stream.py -- typed pagination
2     iterators + streaming download.
3
4  Cursor iterator: follows opaque 'next_cursor' tokens;
5     never parses them.
6  Link iterator: follows RFC 8288 'Link: <url>; rel="next"
7     ' headers.

```

```
5 | Downloader: streams via iter_bytes(bounded_memory),
   | reports progress,
6 | and honours cancellation by closing the
   | response promptly.
7 | """
8 | from __future__ import annotations
9 |
10 | import logging
11 | from collections.abc import Callable, Iterator
12 | from pathlib import Path
13 | from typing import Any
14 |
15 | import httpx
16 |
17 | from ch02_httpx_client import ApiContractError,
   | NorthwindPartsClient
18 |
19 | logger = logging.getLogger("ch02.pagination")
20 |
21 | CHUNK_SIZE = 64 * 1024 # 64 KiB: memory stays O(chunk)
   | , never O(payload)
22 |
23 |
24 | def iter_parts_cursor(
25 |     client: NorthwindPartsClient,
26 |     *,
27 |     page_size: int = 50,
28 |     max_pages: int = 1000, # runaway-guard: a broken
   | server must not loop us forever
29 | ) -> Iterator[dict[str, Any]]:
30 |     """Yield every part from a cursor-paginated /parts
   | endpoint."""
31 |     cursor: str | None = None
32 |     for _ in range(max_pages):
33 |         params: dict[str, Any] = {"limit": page_size}
34 |         if cursor is not None:
35 |             params["cursor"] = cursor # opaque token:
   | pass through, never parse
36 |         page = client.get_json("/parts", params=params)
37 |         items = page.get("items")
38 |         if not isinstance(items, list):
39 |             raise ApiContractError("/parts: 'items'
   | must be a list")
40 |         yield from items
41 |         cursor = page.get("next_cursor")
42 |         if not cursor: # termination: absent
```

```

        next_cursor
        return
43     raise ApiContractError(f"/parts:␣exceeded␣{
44         max_pages}␣pages␣(server␣bug?)")
45
46
47 def iter_parts_link_header(
48     client: NorthwindPartsClient, *, max_pages: int =
49         1000
50 ) -> Iterator[dict[str, Any]]:
51     """Yield␣every␣part␣by␣following␣RFC␣8288␣Link:␣rel
52     =␣"next"␣headers."""
53     response = client._request("GET", "/parts-link")
54     for _ in range(max_pages):
55         try:
56             page: dict[str, Any] = response.json()
57             except ValueError as exc:
58                 raise ApiContractError("/parts-link:␣
59                     invalid␣JSON␣body") from exc
60             items = page.get("items")
61             if not isinstance(items, list):
62                 raise ApiContractError("/parts-link:␣'items
63                     '␣must␣be␣a␣list")
64             yield from items
65             next_url = _parse_link_next(response.headers.
66                 get("Link"))
67             if next_url is None: # termination: no rel="
68                 next"
69                 return
70             response = client._request("GET", next_url)
71             raise ApiContractError(f"/parts-link:␣exceeded␣{
72                 max_pages}␣pages")
73
74 def _parse_link_next(header: str | None) -> str | None:
75     """Extract␣the␣rel="next"␣target␣from␣an␣RFC␣8288␣
76     Link␣header."""
77     if not header:
78         return None
79     for segment in header.split(","):
80         parts = segment.split(";")
81         if len(parts) == 2 and parts[1].strip() == 'rel
82             ="next"':
83             return parts[0].strip().strip("<>")
84     return None

```

```

78
79 def stream_download(
80     client: NorthwindPartsClient,
81     path: str,
82     destination: Path,
83     *,
84     on_progress: Callable[[int, int | None], None] |
85         None = None,
86     cancel: Callable[[], bool] = lambda: False,
87 ) -> int:
88     """Stream path to destination; return bytes
89     written.
90
91     Memory stays 0(CHUNK_SIZE). Cancellation is
92     cooperative: when cancel()
93     returns True we abort, close the stream, and delete
94     the partial file so
95     no corrupt artifact survives.
96     """
97     received = 0
98     tmp = destination.with_suffix(destination.suffix +
99         ".part")
100     try:
101         with client._client.stream("GET", path) as
102             response:
103             client._raise_for_status(response)
104             total_header = response.headers.get("
105                 Content-Length")
106             total = int(total_header) if total_header
107                 else None
108             with tmp.open("wb") as fh:
109                 for chunk in response.iter_bytes(
110                     CHUNK_SIZE):
111                     if cancel():
112                         raise InterruptedError("
113                             download cancelled by caller
114                             ")
115                     fh.write(chunk)
116                     received += len(chunk)
117                     if on_progress:
118                         on_progress(received, total)
119             tmp.replace(destination) # atomic rename: all-
120                 or-nothing artifact
121     except BaseException:
122         tmp.unlink(missing_ok=True) # never leave a

```

```

112         partial file behind
113         raise
114
115 def main() -> None:
116     logging.basicConfig(level=logging.INFO, format="%(
117         levelname)s %(name)s: %(message)s")
118     with NorthwindPartsClient("http://127.0.0.1:8765",
119         api_key="demo-key-nr-001") as client:
120         count = sum(1 for _ in iter_parts_cursor(client
121             , page_size=2))
122         logger.info("cursor-paginated_parts: %d", count
123             )
124         written = stream_download(
125             client, "/catalog/export",
126             Path("catalog_export.jsonl"),
127             on_progress=lambda done, total: logger.info(
128                 "progress: %d/%s bytes", done, total),
129             )
130         logger.info("downloaded %d bytes", written)
131
132 if __name__ == "__main__":
133     main()

```

3.3.5 Hermetic Offline Tests with httpx.MockTransport

The final listing is the discipline that makes everything above trustworthy: a fault-matrix suite that exercises the full client stack — timeouts, pooling, hooks, retry policy, taxonomy — with zero sockets, zero wall-clock dependence, and seeded randomness. `MockTransport` routes requests to an in-process handler while preserving the entire `httpx` pipeline [Enc24]; the injected `sleep` records delays instead of consuming real time.

Listing 3.5: `test_ch02_client.py` — offline fault-matrix test suite.

```

1  """test_ch02_client.py --hermetic tests: no network, no
2     wall clock, seeded RNG.
3
4  Run: pytest -q
5  Simulates: 500s then success, 429 with Retry-After,
6     deterministic 4xx,
7     transport timeouts, slow responses (mocked), contract
8     breaches, and the
9     idempotency gate.
10 """

```

```
8 from __future__ import annotations
9
10 import random
11
12 import httpx
13 import pytest
14
15 from ch02_httpx_client import (
16     ApiClientError4xx,
17     ApiContractError,
18     ApiRateLimited,
19     ApiServerError5xx,
20     ApiTransportError,
21     NorthwindPartsClient,
22 )
23 from ch02_retry import ResilientPartsClient
24
25 BASE_URL = "http://stub.local" # never resolves:
    MockTransport intercepts
26
27
28 def make_client(handler, **kwargs) ->
    NorthwindPartsClient:
29     return NorthwindPartsClient(
30         BASE_URL, api_key="test-key",
31         transport=httpx.MockTransport(handler), **
            kwargs,
32     )
33
34
35 # --- Status handling & taxonomy
    -----
36
37 def test_get_part_success() -> None:
38     def handler(request: httpx.Request) -> httpx.
        Response:
39         assert request.headers["Authorization"] == "
            Bearer test-key"
40         return httpx.Response(200, json={"part_id": "NR
            -1001", "name": "Servo"})
41     with make_client(handler) as client:
42         assert client.get_part("NR-1001")["name"] == "
            Servo"
43
44
45 def test_404_maps_to_typed_client_error() -> None:
```

```

46     with make_client(lambda r: httpx.Response(404, json
47         ={"error": "no_such_part"})) as c:
48         with pytest.raises(ApiClientError4xx) as
49             exc_info:
50             c.get_part("NOPE-1")
51         assert exc_info.value.status_code == 404
52
53 def test_500_maps_to_server_error() -> None:
54     with make_client(lambda r: httpx.Response(503)) as
55         c:
56         with pytest.raises(ApiServerError5xx):
57             c.get_part("NR-1001")
58
59 def test_429_carries_retry_after() -> None:
60     def handler(request: httpx.Request) -> httpx.
61         Response:
62         return httpx.Response(429, headers={"Retry-
63             After": "7"})
64     with make_client(handler) as c:
65         with pytest.raises(ApiRateLimited) as exc_info:
66             c.get_part("NR-1001")
67         assert exc_info.value.retry_after == 7.0
68
69 def test_transport_timeout_maps_to_transport_error() ->
70     None:
71     def handler(request: httpx.Request) -> httpx.
72         Response:
73         raise httpx.ReadTimeout("simulated_hung_peer",
74             request=request)
75     with make_client(handler) as c:
76         with pytest.raises(ApiTransportError):
77             c.get_part("NR-1001")
78
79 def test_200_with_broken_body_is_contract_error() ->
80     None:
81     with make_client(lambda r: httpx.Response(200,
82         content=b"<html>oops</html>")) as c:
83         with pytest.raises(ApiContractError):
84             c.get_part("NR-1001")
85
86 # --- Retry policy

```

```
-----
82
83 def test_retry_recovers_from_transient_500s() -> None:
84     calls = {"n": 0}
85     def handler(request: httpx.Request) -> httpx.
      Response:
86         calls["n"] += 1
87         if calls["n"] < 3:
88             return httpx.Response(500)
89         return httpx.Response(200, json={"part_id": "NR
      -1001"})
90     inner = make_client(handler)
91     sleeps: list[float] = []
92     resilient = ResilientPartsClient(
93         inner, rng=random.Random(42), sleep=sleeps.
      append
94     )
95     response = resilient.request_with_retry("GET", "/"
      parts/NR-1001")
96     assert response.status_code == 200
97     assert calls["n"] == 3
98     assert len(sleeps) == 2 # two retries, two
      jittered sleeps, zero wall time
99
100
101 def test_retry_honors_retry_after_over_jitter() -> None
      :
102     calls = {"n": 0}
103     def handler(request: httpx.Request) -> httpx.
      Response:
104         calls["n"] += 1
105         if calls["n"] == 1:
106             return httpx.Response(429, headers={"Retry-
      After": "5"})
107             return httpx.Response(200, json={"ok": True})
108     inner = make_client(handler)
109     sleeps: list[float] = []
110     resilient = ResilientPartsClient(
111         inner, rng=random.Random(42), sleep=sleeps.
      append
112     )
113     resilient.request_with_retry("GET", "/parts/NR-1001
      ")
114     assert sleeps == [5.0] # server-advertised delay,
      not jitter
```



```

115
116
117 def test_4xx_is_never_retried() -> None:
118     calls = {"n": 0}
119     def handler(request: httpx.Request) -> httpx.
120         Response:
121         calls["n"] += 1
122         return httpx.Response(422, json={"detail": "
123             invalid"})
124     inner = make_client(handler)
125     resilient = ResilientPartsClient(inner, rng=random.
126         Random(42),
127         sleep=lambda s:
128             None)
129     with pytest.raises(ApiClientError4xx):
130         resilient.request_with_retry("GET", "/parts/NR
131             -1001")
132     assert calls["n"] == 1
133
134 def test_post_without_idempotency_key_is_refused() ->
135     None:
136     inner = make_client(lambda r: httpx.Response(200,
137         json={}))
138     resilient = ResilientPartsClient(inner)
139     with pytest.raises(ValueError, match="idempotency")
140         :
141         resilient.request_with_retry("POST", "/orders",
142             json={"qty": 1})
143
144 def
145 test_post_with_idempotency_key_retries_and_sends_header
146 () -> None:
147     seen: list[str | None] = []
148     calls = {"n": 0}
149     def handler(request: httpx.Request) -> httpx.
150         Response:
151         seen.append(request.headers.get("Idempotency-
152             Key"))
153         calls["n"] += 1
154         if calls["n"] == 1:
155             return httpx.Response(503)
156         return httpx.Response(201, json={"order_id": "
157             ORD-9"})
158     inner = make_client(handler)

```

```
147     resilient = ResilientPartsClient(inner, rng=random.  
148         Random(42),  
149         sleep=lambda s:  
150             None)  
151     response = resilient.request_with_retry(  
152         "POST", "/orders", idempotency_key="key-abc-123",  
153         json={"qty": 1}  
    )  
    assert response.status_code == 201  
    assert seen == ["key-abc-123", "key-abc-123"] #  
        key stable across replays
```

Note what the suite does *not* do: open a socket, sleep in real time, assert on wall-clock, or depend on test ordering. Those four absences are the definition of a deterministic, offline-first consumer test suite.

3.4 Common Pitfalls & Anti-Patterns

Each anti-pattern below is a real production failure mode, stated with its mechanism and its fix.

3.4.1 Missing Timeouts

Anti-pattern: `requests.get(url)` — infinite wait by default. **Mechanism:** a hung peer holds a worker forever; under load the pool exhausts and the healthy caller falls over (the cascading failure of Section 3.5.3). **Fix:** explicit four-dimension timeouts on the client, tighter or looser per call only with justification. Treat any timeout-free call found in review as a defect, full stop.

3.4.2 Retry Storms

Anti-pattern: unbounded `while True:` `try again immediately`, or fixed-interval retries with no cap, no jitter, no budget. **Mechanism:** when the dependency degrades, every client retries in lockstep; offered load multiplies exactly when capacity shrinks; the outage deepens and self-sustains. **Fix:** capped attempts *plus* elapsed budget, full jitter, **Retry-After** honored, and a retry budget (Section 3.2.5); circuit breaking for sustained failure (Chapter 15).

3.4.3 Swallowing Exceptions

Anti-pattern: `except Exception:` `pass`, or logging and returning `None` from a function whose contract promises a resource. **Mechanism:** failures

become silent data corruption — a “missing part” that was really a 503; downstream code manufactures defaults; the root cause is invisible to operators. **Fix:** catch specific types (the taxonomy of Section 3.2.8), add context, re-raise; let only code that can *act* on a failure catch it.

3.4.4 Parsing Before the Status Check

Anti-pattern: `resp.json()` unconditionally. **Mechanism:** a 503 with an HTML body from a proxy surfaces as `JSONDecodeError`, three frames from the truth; on-call engineers debug serialization when the real story is capacity. **Fix:** classify the status first (our `_raise_for_status`), then parse; a 2xx with an unparseable body is a distinct `ApiContractError`.

3.4.5 Leaking Credentials into Logs

Anti-pattern: logging `request.headers`, full URLs with query tokens, or response bodies on the error path. **Mechanism:** bearer tokens and API keys land in log aggregators with broad read access and long retention — a credential-leak incident from a debugging convenience. **Fix:** log method, path, status, latency; never headers, bodies, or auth material (see `_log_request` in Listing 3.2). The twelve-factor config discipline [Wig17] applies: credentials arrive via environment, never via source, and never leave via logs.

3.4.6 Unbounded Decompression and Body Reads

Anti-pattern: `response.read()` / `response.text` on untrusted or unbounded endpoints; accepting `gzip/br` without size discipline. **Mechanism:** a 2 MB gzip bomb expands to gigabytes in memory; a streaming endpoint with no `Content-Length` reads until RAM is gone. **Fix:** stream with `iter_bytes` (Listing 3.4) for anything large or untrusted; enforce a maximum decompressed size; write partial downloads to a `.part` file and rename atomically on success.

Warning

The compound anti-pattern. These failures compose: no timeout + aggressive retry + swallowed exception is a client that hangs forever, retries forever, and reports nothing. Review consumers as systems, not as lines.

3.5 Hands-On Production Lab & Real-World Case Study

3.5.1 Lab Setup: The Northwind Robotics Parts API Stub

The lab consumes a fictional supplier API — the *Northwind Robotics Parts API* — served entirely by a local stub on 127.0.0.1:8765. The stub (Listing 3.6) uses only the standard library and implements five behaviors: a part lookup, cursor pagination over a small synthetic catalog, a 429 + **Retry-After** fault for the retry demo, a flaky endpoint that fails twice before succeeding, and a streaming catalog export.

Listing 3.6: `ch02_stub_server.py` — local stub for the Northwind Robotics Parts API.

```

1  """ch02_stub_server.py -- offline stub for the
   Northwind Robotics Parts API.
2
3  Stdlib only. Deterministic. Listens on 127.0.0.1:8765 (
   localhost only).
4
5  Endpoints:
6  GET /parts/{id} -> 200 JSON part, or 404
7  GET /parts?limit&cursor -> cursor-paginated catalog (5
   synthetic parts)
8  GET /parts-link -> RFC 8288 Link-header
   pagination (2 pages)
9  GET /rate-limited -> 429 with Retry-After: 2 (
   always)
10 GET /flaky -> 500, 500, then 200 (per-
   process state)
11 GET /catalog/export -> streamed JSONL catalog
12
13 Run: python ch02_stub_server.py
14 """
15 from __future__ import annotations
16
17 import json
18 import logging
19 import threading
20 from http.server import BaseHTTPRequestHandler,
   ThreadingHTTPServer
21 from urllib.parse import parse_qs, urlparse
22
23 logger = logging.getLogger("ch02.stub")

```

```

24
25 # Synthetic Northwind Robotics catalog: fictional parts
    , fictional prices.
26 CATALOG: list[dict[str, object]] = [
27     {"part_id": "NR-1001", "name": "Servo_Motor_SG-90",
        "unit_price_cents": 450},
28     {"part_id": "NR-1002", "name": "LiDAR_Unit_LR-2", "
        unit_price_cents": 8999},
29     {"part_id": "NR-1003", "name": "Chassis_Plate_AL
        -7075", "unit_price_cents": 3200},
30     {"part_id": "NR-1004", "name": "Wheel_Encoder_WE
        -512", "unit_price_cents": 1250},
31     {"part_id": "NR-1005", "name": "Battery_Pack_BP-4S"
        , "unit_price_cents": 5400},
32 ]
33
34 _flaky_calls = {"n": 0}
35 _state_lock = threading.Lock()
36
37
38 class StubHandler(BaseHTTPRequestHandler):
39     server_version = "NorthwindStub/1.0"
40
41     def _send_json(self, status: int, payload: dict[str
        , object],
42                   extra_headers: dict[str, str] | None
                        = None) -> None:
43         body = json.dumps(payload).encode("utf-8")
44         self.send_response(status)
45         self.send_header("Content-Type", "application/
            json")
46         self.send_header("Content-Length", str(len(body)
            )))
47         for key, value in (extra_headers or {}).items():
48             :
49             self.send_header(key, value)
50         self.end_headers()
51         self.wfile.write(body)
52
53     def do_GET(self) -> None: # noqa: N802 (stdlib
        naming)
54         parsed = urlparse(self.path)
55         route, query = parsed.path, parse_qs(parsed.
            query)
56
57         if route.startswith("/parts/"):

```

```

57         part_id = route.rsplit("/", 1)[-1]
58         part = next((p for p in CATALOG if p["
59             part_id"] == part_id), None)
60         if part is None:
61             self._send_json(404, {"error": f"no_
62                 such_part:{part_id}"})
63         else:
64             self._send_json(200, part)
65         return
66
67     if route == "/parts":
68         limit = int(query.get("limit", ["50"])[0])
69         cursor = query.get("cursor", [None])[0]
70         start = int(cursor) if cursor is not None
71             else 0 # stub: cursor = index
72         items = CATALOG[start:start + limit]
73         next_index = start + limit
74         payload: dict[str, object] = {"items":
75             items}
76         if next_index < len(CATALOG):
77             payload["next_cursor"] = str(next_index
78                 )
79         self._send_json(200, payload)
80         return
81
82     if route == "/parts-link":
83         if query.get("page") == ["2"]:
84             # Final page: no Link header ->
85             iterator terminates.
86             self._send_json(200, {"items": CATALOG
87                 [3:]})
88         else:
89             self._send_json(
90                 200, {"items": CATALOG[:3]},
91                 extra_headers={"Link": '</parts-
92                     link?page=2>;rel="next"',

```

```

93         if route == "/flaky":
94             with _state_lock:
95                 _flaky_calls["n"] += 1
96                 n = _flaky_calls["n"]
97             if n < 3:
98                 self._send_json(500, {"error": "
99                     transient_blip"})
100             else:
101                 self._send_json(200, {"part_id": "NR
102                     -1001", "ok": True})
103             return
104
105         if route == "/catalog/export":
106             lines = "".join(json.dumps(p) + "\n" for p
107                 in CATALOG).encode("utf-8")
108             self.send_response(200)
109             self.send_header("Content-Type", "
110                 application/x-ndjson")
111             self.send_header("Content-Length", str(len(
112                 lines)))
113             self.end_headers()
114             self.wfile.write(lines)
115             return
116
117         self._send_json(404, {"error": f"unknown_route:
118             {route}"})
119
120     def log_message(self, fmt: str, *args: object) ->
121         None:
122         logger.info("stub: " + fmt, *args)
123
124     def main() -> None:
125         logging.basicConfig(level=logging.INFO, format="%(
126             levelname)s %(name)s: %(message)s")
127         server = ThreadingHTTPServer(("127.0.0.1", 8765),
128             StubHandler)
129         logger.info("Northwind_stub_listening_on_http
130             ://127.0.0.1:8765 (Ctrl+C to stop)")
131         try:
132             server.serve_forever()
133         except KeyboardInterrupt:
134             server.shutdown()
135
136 if __name__ == "__main__":

```

```
main()
```

3.5.2 Lab Steps

Work from a project folder, e.g. `C:\Training\ch02_lab`, containing the five listing files. All commands are PowerShell.

1. **Environment.** Create and activate the virtual environment, then install the pinned dependencies:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

2. **Start the stub** in a first terminal: `python ch02_stub_server.py`. Confirm it answers: `curl.exe http://127.0.0.1:8765/parts/NR-1001`.
3. **Baseline.** In a second terminal run `python ch02_urllib_baseline.py`. Note in the stub's log that every request is a fresh connection (`Connection: close` semantics) — the churn of Section 3.2.4 made visible.
4. **Production client.** Run `python ch02_httpx_client.py`. Observe the telemetry hooks emitting per-request latency lines.
5. **Retry demo.** Point the demo at the flaky endpoint (edit `main()` to call `/flaky`), run `python ch02_retry.py`, and watch two jittered retries precede success. Then call `/rate-limited` and confirm the client waits the server-advertised 2 seconds.
6. **Pagination and streaming.** Run `python ch02_pagination_stream.py`; verify the iterator yields all five parts across three pages and that `catalog_export.jsonl` contains five NDJSON lines.
7. **Offline fault matrix.** Stop the stub and run `pytest -q`. All tests must pass with the stub down — proof the suite is hermetic.
8. **Break it on purpose.** Set the client's read timeout to 0.05 against the stub's `/flaky` endpoint, observe `ApiTransportError`, then restore. Change the stub's 429 to omit `Retry-After` and confirm the client falls back to full jitter.

Key Concept

Exit criteria. The lab is complete when: (1) the retry demo shows server-advertised waits overriding jitter; (2) the pagination iterator terminates without a `max_pages` tripwire firing; (3) the pytest suite passes with the network stack provably unused.

3.5.3 War Story: The Missing Read Timeout and the Retry Storm

A fulfillment service (names fictionalized, physics real) called a warehouse inventory API on every order. The call was a one-liner: `requests.get(url)`. No timeout — `requests` defaults to infinite. A hand-rolled retry loop sat above it: `for attempt in range(20): try: ...; time.sleep(1)`.

At 14:07 on a Tuesday, a bad deploy left the inventory service’s worker threads deadlocked on a database lock. The load balancer kept passing health checks (the `/healthz` handler was a separate thread), so traffic kept flowing into a process that accepted connections and then *said nothing, forever*. TCP was healthy; HTTP was catatonic. Only a read timeout can see this failure — and there was none.

The cascade ran exactly as Nygard’s playbook predicts [Nyg18]. By 14:12 every fulfillment worker held a socket awaiting bytes that would never come. By 14:15 the retry loops — one-second fixed interval, no jitter, twenty attempts — had multiplied offered load on the inventory service by roughly 20×, ensuring that even when the lock was manually killed at 14:22, the service drowned instantly in synchronized retries: a self-inflicted retry storm. The circuit breaker that would have shed the load did not exist. Checkout error rates peaked at 94%; full recovery took another 40 minutes after the root cause was fixed, because the storm outlived its trigger.

The remediation diff was embarrassingly small: a four-dimension timeout (connect 2 s, read 5 s, write 5 s, pool 1 s), three jittered retries with an elapsed-time budget, **Retry-After** honored, and a fleet-wide retry budget feeding a circuit breaker (Chapter 15). The lesson this chapter encodes: **the cheap resilience work is done in the client, before the incident, or it is done in the incident review, after.**

3.6 Self-Assessment Exercises & Deep-Dive Questions

3.6.1 Exercises

1. **Timeout audit.** Grep a codebase you know for `requests.get()`, `requests.post()`, and `urlopen()` lacking a `timeout=` argument. For each hit, state which of the four timeout dimensions is unbounded and what failure it permits.
2. **Jitter analysis.** With $b = 0.5\text{ s}$ and $N = 4$ retries, compute (a) the deterministic exponential total wait, (b) the expected total wait under full jitter, and (c) the worst-case total wait under full jitter with cap $c = 8\text{ s}$.
3. **Idempotency triage.** For each operation, state whether a blind retry after a read timeout is safe, and why: `GET /parts/NR-1001`; `POST /orders`; `PUT /parts/NR-1001/status` with body `{"status": "reserved"}`; `POST /payments` with an `Idempotency-Key` header.
4. **Pool sizing.** A service runs 32 worker threads; each order makes at most 2 concurrent calls to one upstream. Propose `max_connections` and `max_keepalive_connections` and justify both.
5. **Extend the taxonomy.** Add `ApiAuthError` (401/403) to Listing 3.2, wire it into `_raise_for_status`, and add a `MockTransport` test proving it is raised and never retried.
6. **Offset walker.** The iterators in Listing 3.4 cover cursor and link-header pagination. Implement `iter_parts_offset(client, limit)` for an offset/limit endpoint, with correct termination and a `max_pages` tripwire, and test it against `MockTransport`.

3.6.2 Deep-Dive Questions

1. Why does a pool timeout exist separately from the read timeout? What distinct failure does each bound, and what does `PoolTimeout` tell you about your configuration that `ReadTimeout` never could?
2. Full jitter halves expected total wait relative to deterministic backoff (Equation 3.3). Is that a latency cost or a latency saving for the caller, and why is it a fleet-level win regardless?
3. A server sends `Retry-After: 120` but your retry budget caps elapsed

time at 20 s. Reconcile the two policies. Which wins, and what should the client do when they conflict irreconcilably?

4. Explain why `requests` cannot support HTTP/2 without a rewrite of its connection core, and what architectural property of `httpx` made HTTP/2 an *option* rather than a fork.
5. Under what conditions is cursor pagination strictly more correct than offset pagination for a catalog that changes while you read it? Construct the interleaving that corrupts an offset walk.
6. Your `MockTransport` suite passes and production fails on TLS. What layer did the mock legitimately exclude, and what is the minimum additional test that covers it without internet access?

3.7 Interview Q&A Vault

Thirty-two questions ordered junior → staff. Answers are deliberately compressed to what a strong candidate says out loud.

3.7.1 Junior

1. **Q: What does an HTTP client library actually do for you?**
A: Manages the socket lifecycle (DNS, TCP, TLS), formats the request on the wire, parses the response, and — in session-based libraries — pools connections for reuse. Without it you are writing a state machine over sockets by hand.
2. **Q: `urllib` vs. `requests` — why does anyone still use `urllib`?**
A: Zero dependencies: it is always present in the `stdlib`, so installers, bootstrap scripts, and frozen single-file tools use it. For anything sustained, its flat timeout, missing session/pool, and manual JSON make `requests` or `httpx` the right call.
3. **Q: What is keep-alive and why does it matter?**
A: Reusing one TCP/TLS connection across requests instead of re-opening per request. It eliminates repeated handshake latency (~2 RTTs) and crypto CPU on every call — often the single biggest client-side latency win.
4. **Q: What is the default timeout of `requests.get()`?**
A: None — it waits forever. That is a trap: the library’s ergonomic default is a production hazard, which is why timeouts must always be explicit.

5. **Q: A call returns status 500. What do you do?**

A: Classify it as a transient server fault: retry per policy (bounded, jittered, idempotency-aware). Never parse the body as if it were a success payload.

6. **Q: A call returns 404. Retry it?**

A: No. 4xx is deterministic — the same request fails identically. Fix the request (the URL, the ID, the payload), not the plumbing.

7. **Q: What is JSON deserialization doing at the client boundary?**

A: Converting the response body bytes into Python objects. It belongs *after* the status check, and its failure on a 2xx is a contract error, not a transport error.

8. **Q: Why should a client send a User-Agent?**

A: Identifiability. When your traffic misbehaves, the provider can reach you instead of blocking an entire IP range — and many providers contractually require it.

3.7.2 Mid-Level

9. **Q: Why is a missing timeout a severity-1 risk?**

A: Because it converts a dependency's partial failure into your total failure: every worker blocks on a socket that never answers, the pool exhausts, and your healthy service stops serving. Unbounded waits are unbounded resource holds — the textbook cascading failure [Nyg18].

10. **Q: Name the four httpx timeout dimensions and what each bounds.**

A: connect (DNS/TCP/TLS establishment), read (waiting for response bytes, including mid-stream gaps), write (sending the request body), pool (waiting for a free connection in a saturated pool). Each bounds a distinct hang a single flat timeout cannot express.

11. **Q: Connect timeout vs. read timeout — which failure does each catch?**

A: Connect catches a dead, firewalled, or TLS-broken host. Read catches a live connection with a hung application behind it — the failure TCP keepalive cannot see.

12. **Q: What does PoolTimeout tell you?**

A: Your concurrency exceeds your pool: requests are queueing for connections. The fix is sizing (`max_connections`) or concurrency

control — not a longer pool timeout, which just makes the queue quieter.

13. **Q: Explain exponential backoff with full jitter.**

A: On attempt n , wait a *random* value drawn uniformly from $[0, \min(c, b \cdot 2^{n-1})]$. Expected wait is $b \cdot 2^{n-2}$: exponential growth for server recovery, randomized phase so correlated clients stop colliding.

14. **Q: Why decorrelated jitter?**

A: It samples the next wait from $[b, 3 \times \text{previous}]$ rather than the attempt index, so a client's schedule adapts to its own history. Both it and full jitter break lockstep; decorrelated jitter converges faster after short blips. The essential property either way is randomness — without it, fleets retry in synchronized waves.

15. **Q: What is a retry storm and how do you prevent it?**

A: Retries multiplying offered load on an already-degraded dependency, deepening the outage. Prevention: capped attempts, elapsed budgets, jitter, honoring **Retry-After**, a fleet-wide retry budget, and a circuit breaker for sustained failure.

16. **Q: When is it safe to retry a POST?**

A: Only when the API accepts an idempotency key (server-side dedup) or the operation is naturally idempotent per its contract. Otherwise a read timeout is ambiguous — the server may have succeeded — and replaying risks duplicate side effects.

17. **Q: requests vs. httpx trade-offs?**

A: **requests**: older, sync-only, HTTP/1.1-only, two-dimension timeouts, huge install base. **httpx**: same API shape plus sync *and* async clients, optional HTTP/2, four-dimension timeouts, first-class Mock-Transport, full type hints. New code: httpx; existing sync codebases: requests is defensible.

18. **Q: What does Retry-After do and where does it appear?**

A: Server-advertised minimum delay before retrying, defined by RFC 9110 [Fie22a] for 429 and 503 as delta-seconds or an HTTP-date. It outranks any client-computed backoff because the server knows its own recovery timeline.

19. **Q: How do you consume a paginated endpoint cleanly?**

A: Hide the mechanics behind a lazy iterator: the caller writes one **for** loop; the walker follows cursors or **Link: rel="next"** headers, treats cursors as opaque, and terminates on a missing next pointer —

with a page-count tripwire against server bugs.

20. **Q: Why stream downloads instead of `response.read()`?**

A: Memory stays $O(\text{chunk})$ rather than $O(\text{payload})$; you get progress reporting, cooperative cancellation, and immunity to decompression bombs. Buffering is only acceptable for small, trusted, size-known payloads.

3.7.3 Senior

21. **Q: Design the exception hierarchy for an SDK wrapper.**

A: One root (`ApiClientError`); three branches matching failure layers: transport (timeout/DNS/TLS — retryable), status (split 4xx deterministic from 5xx transient; 429 carries `retry_after`), and contract (2xx with a broken body — never retry, alert). Callers *except* the layer they can act on.

22. **Q: How do you test retry logic without slow, flaky tests?**

A: Inject everything time-like: `httpx.MockTransport` for the wire, a seeded `random.Random` for jitter, and an injected `sleep` that records durations instead of sleeping. Assert on call counts and recorded delays — zero wall-clock assertions.

23. **Q: What is a retry budget and why aren't per-request caps enough?**

A: Per-request caps bound one call; a budget bounds the fleet — e.g., retries limited to 10% of issued requests per minute. When a dependency fails broadly, every request hitting its per-request cap still multiplies total load; the budget converts that into controlled fail-fast.

24. **Q: Your client sees 429 with Retry-After: 120 but an elapsed budget of 20 s. Design the resolution.**

A: The server delay wins within the budget; on conflict, fail the operation with `ApiRateLimited` carrying the delay so the *caller* can requeue the work at the application layer. Never silently clamp to 20 s and retry into a still-limited server.

25. **Q: How do event hooks differ from subclassing the client?**

A: Hooks are cross-cutting seams (logging, tracing, auth refresh) applied uniformly without touching the request path; subclassing embeds policy in the call path and fragments across overrides. Prefer composition (hooks, wrapped transports) over inheritance.

26. **Q: A 2xx response contains HTML from a captive proxy.**

Which layer failed and how should the client react?

A: Contract layer. `Content-Type` is wrong and JSON parsing fails; raise `ApiContractError`, alert, and never retry — replaying cannot fix a middlebox interposing.

27. Q: Why is creating a new `httpx.Client` per request an anti-pattern?

A: It discards the connection pool: every call re-pays DNS, TCP, and TLS, and sockets accumulate in `TIME_WAIT` under load. One long-lived client per base URL, closed deterministically, is the correct shape.

28. Q: What telemetry belongs on the client hot path, and what must never appear in logs?

A: Method, path, status class, latency, retry count, exception type. Never: `Authorization` headers, API keys, cookies, or full bodies — credentials in log aggregation is a reportable leak.

29. Q: HTTP/2 changes what for a client?

A: Multiplexing: many concurrent streams share one connection, eliminating HTTP/1.1 head-of-line blocking at the connection level and shrinking pool pressure. It does *not* change timeout, retry, or idempotency semantics — those are application-layer concerns.

30. Q: When would you choose async consumption, and what does it actually buy?

A: When a single process must sustain high concurrent outbound calls (crawlers, fan-out aggregators). It buys massive connection concurrency without thread-per-connection memory and context-switch costs. It buys nothing for a sequential script — and adds correctness hazards (unbounded gather, blocking calls in the loop).

3.7.4 Staff

31. Q: Design an organization-wide HTTP client policy. What is in it and how is it enforced?

A: Mandated: four-dimension timeouts with ceilings, jittered bounded retries plus a shared retry budget, idempotency gates on mutation, header-safe telemetry, hermetic `MockTransport` tests. Enforced as a shared internal client library (policy as code), CI lint rules banning bare timeout-free calls, and fault-injection tests in staging. Policy documents that CI cannot check are fiction.

32. Q: A downstream provider publishes a 100 req/s limit. Your

fleet of 200 pods each has a local token bucket set to 1 req/s. What goes wrong and how do you architect past it?

A: Local buckets under-utilize (fairness is emergent, not guaranteed) and can't react to provider-side repartitioning; pod autoscaling silently raises aggregate rate. Architecture: make local limits a function of live fleet size (or a central rate-limiting sidecar), treat 429s as control signals that back the whole fleet off, and negotiate limits as an SLO with the provider rather than reverse-engineering them from error rates.

3.8 External Bibliography & Further Reading

- **httpx documentation** [Enc24] — the authoritative reference for the four-dimension timeout model, `Limits` pooling, event hooks, HTTP/2 opt-in, and `MockTransport`. Read the “Timeouts” and “Advanced” pages in full.
- **Requests documentation** [Pyt24] — the Session abstraction and the historical baseline every Python engineer inherits; its “Advanced Usage” chapter remains excellent on connection pooling semantics.
- **RFC 9110, *HTTP Semantics*** [Fie22a] — the normative definitions of method idempotency (§9.2.2), status-code classes, 429/503, and `Retry-After` (§15.5.4, §10.2.3). The primary source for every retry rule in this chapter.
- **RFC 8288, *Web Linking*** — the `Link` header and `rel="next"` consumed in Listing 3.4.
- Michael T. Nygard, *Release It!*, 2nd ed. [Nyg18] — the canonical treatment of cascading failure, timeouts, and circuit breakers. Chapter 4 (“Stability Antipatterns”) is the war story’s playbook.
- Marc Brooker, AWS Architecture Blog, “*Exponential Back-off and Jitter*” and “*Timeouts, retries, and backoff with jitter*” — the industry-standard derivations of full jitter and decorrelated jitter that Section 3.2.5 formalizes. <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>
- **Tenacity documentation** — the retry library used in Listing 3.3: composable stop/wait/retry strategies. <https://tenacity.readthedocs.io/>

- **pydantic documentation** [Col24] — typed response models and validation for the SDK-wrapper pattern; Chapter 4 develops contract-first serialization in depth.
- **The Twelve-Factor App** [Wig17] — factor III (config in the environment) underpins the credential-handling rules of Section 3.4.
- **Martin Kleppmann, *Designing Data-Intensive Applications*** [Kle17] — Chapter 8’s treatment of partial failure and “we cannot know” outcomes at network boundaries; the theoretical frame for our error taxonomy.

3.9 Capstone Projects

Five progressive projects. All run offline against the Northwind stub (Listing 3.6) or `httpx.MockTransport`; all require a passing pytest suite; all follow the workspace conventions (typed Python 3.11+, PowerShell docs, synthetic data only).

3.9.1 Capstone 2.1 [junior] — Robust CLI Downloader

Objective. Build `nrget`: a PowerShell-friendly CLI that downloads a part’s datasheet (`/catalog/export`) from the stub to a local file, correctly and observably.

Inputs/Datasets. The local stub’s `/catalog/export` endpoint (synthetic Northwind catalog, NDJSON).

Steps.

1. Wrap `httpx.Client` with explicit four-dimension timeouts and a typed error taxonomy (reuse Listing 3.2 patterns).
2. Stream the body with `iter_bytes`, a `.part` temp file, and atomic rename on success.
3. Print byte-level progress to the console; support a `-max-bytes` guard.
4. Exit codes: 0 success, 2 client error, 3 server/transient error after bounded retries.

Acceptance Criteria.

- No call lacks an explicit timeout (verified by code inspection and a test asserting the client’s `timeout` fields).

- ☐ A killed download leaves no partial artifact.
- ☐ MockTransport tests cover 200, 404, 500-then-200, and malformed-body paths; suite passes with the stub stopped.
- ☐ Logs contain method/path/status/latency and no headers.

Stretch Goals. Resume support via `Range` requests; a `-dry-run` mode that prints the request plan without sending.

3.9.2 Capstone 2.2 [mid] — Rate-Limit-Aware Catalog Crawler

Objective. Crawl the entire paginated catalog of an instrumented stub that enforces a published limit of 5 req/s and answers `429 + Retry-After` when exceeded — without ever being throttled in steady state.

Inputs/Datasets. Extended stub: cursor-paginated `/parts` over a 500-part synthetic catalog, a token-bucket limiter, and `RateLimit-Policy/Retry-After` headers.

Steps.

1. Implement a client-side token bucket ($r = 4$ tokens/s, $B = 4$) gating every request.
2. Wrap calls in the retry policy of Listing 3.3; `Retry-After` overrides jitter.
3. Consume pages through the cursor iterator of Listing 3.4.
4. Record per-request timestamps to a CSV; compute achieved rate.

Acceptance Criteria.

- ☐ Full crawl completes; zero `429`s in the steady state (allow one cold-start burst violation, documented).
- ☐ Achieved sustained rate ≤ 5 req/s, evidenced from the timestamp CSV (not wall-clock test assertions).
- ☐ A forced-`429` MockTransport test proves `Retry-After` overrides jitter exactly.
- ☐ Deterministic suite: seeded RNG, injected sleep, no network.

Stretch Goals. Adaptive bucket that lowers r on `429` and recovers slowly (additive-increase); parallel crawlers sharing one process-wide bucket.

3.9.3 Capstone 2.3 [mid] — Typed SDK Wrapper with Full Test Matrix

Objective. Publish (locally) a `northwind-parts-sdk` package: typed pydantic response models, the resilient client, pagination iterators, and a complete offline fault matrix.

Inputs/Datasets. The Northwind stub endpoints; a hand-written JSON Schema for the `Part` resource used to generate/validate the pydantic model.

Steps.

1. Model `Part`, `PartPage`, and `ApiError` as pydantic models with strict types [Col24].
2. Parse responses into models; map validation failures to `ApiContractError` with field-level context.
3. Compose timeouts, pooling, auth injection, hooks, and the tenacity retry wrapper behind a clean public API.
4. Build the `MockTransport` matrix: every status class, transport timeouts, mid-stream reset, truncated JSON, wrong `Content-Type`, oversized page count.

Acceptance Criteria.

- ☐ Public API is fully typed; `mypy`-clean (or equivalent checked annotations) on the package.
- ☐ Fault matrix covers ≥ 15 scenarios including every taxonomy branch; all deterministic and offline.
- ☐ Contract errors are never retried; transient errors retry within budget; idempotency gate enforced on mutations.
- ☐ README (PowerShell-only instructions) documents install, usage, and error semantics.

Stretch Goals. Generate the model from the JSON Schema at build time; add an async facade reusing the same core via `httpx.AsyncClient`.

3.9.4 Capstone 2.4 [senior] — Chaos-Client Harness

Objective. Build a fault-injection transport that sits between your resilient client and the stub and proves, empirically, that the resilience policy works: the client survives a gauntlet of injected faults with predictable behavior.

Inputs/Datasets. A `ChaosTransport` wrapping `httpx.MockTransport` with a scripted fault schedule: connect resets, read stalls, truncation, 500 bursts, 429s with/without `Retry-After`, gigantic `Content-Length` lies, and gzip bombs (synthetic, capped).

Steps.

1. Implement `ChaosTransport(BaseTransport)` reading a YAML/JSON fault schedule; deterministic via seeded RNG.
2. Define the expected behavior per fault (recover, fail-fast with typed error, abort with bounded time) as an executable contract.
3. Run the full SDK against each schedule; assert both outcome *and* bounds: total attempts, total sleep, bytes buffered.
4. Produce a coverage report mapping faults $\rightarrow$ observed client behavior.

Acceptance Criteria.

- ☐ ≥ 8 distinct fault classes, each with an executable expectation.
- ☐ Every run is reproducible from its seed; no wall-clock dependence.
- ☐ At least one fault deliberately exceeds the retry budget and the client demonstrably fails fast with the correct typed error.
- ☐ Decompression-bomb test proves the streaming path's size guard engages.

Stretch Goals. Latency distributions (injected p99 spikes) with a fake clock; chaos *schedules* replayed from recorded production incident timelines (synthetic).

3.9.5 Capstone 2.5 [staff] — Async Concurrent Fetcher with Bounded Concurrency

Objective. Design and validate an `httpx.AsyncClient`-based fetcher that drains a 10,000-item work queue from the stub at maximum throughput *without* tripping the provider's rate limit, exhausting the pool, or losing backpressure — and write the design memo defending every knob.

Inputs/Datasets. Instrumented stub with a 200-part synthetic catalog, per-IP token bucket at 50 req/s, and injected latency jitter (5–200 ms, seeded).

Steps.

1. Bounded concurrency via `asyncio.Semaphore` (not unbounded `gather`); pool limits aligned to the semaphore.

2. Process-wide token bucket below the published limit; 429 handling that backs off the *whole* fetcher, not one task.
3. Retry policy per Listing 3.3 semantics, adapted for async sleep; elapsed and attempt budgets enforced.
4. Cancellation: a single Ctrl+C (or task cancel) drains in-flight work, closes streams, and exits cleanly with a partial-results manifest.
5. The memo: Little's-law sizing of concurrency vs. rate limit; pool vs. semaphore alignment; failure-mode table.

Acceptance Criteria.

- ☐ Full queue drains with zero unhandled exceptions and zero pool timeouts at the chosen settings.
- ☐ Sustained rate ≤ 50 req/s evidenced from stub-side logs; 429 rate $< 0.1\%$ after warm-up.
- ☐ Deterministic tests via `MockTransport` + injected async sleep; no wall-clock assertions.
- ☐ Cancellation test proves no orphan tasks and no partial artifacts.
- ☐ Design memo quantifies every knob; trade-offs and rejected alternatives documented.

Stretch Goals. Adaptive concurrency controller (AIMD on 429/latency signals); fairness across multiple upstream hosts with per-host buckets.

3.10 Python Dependencies Appendix

The chapter's code pins four third-party packages. Python 3.11+ is required throughout. Install on Windows/PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

Listing 3.7: `requirements.txt` for Chapter 2.

```
1 httpx>=0.27,<1.0
2 tenacity>=8.3
3 pydantic>=2.7,<3.0
4 pytest>=8.0
```

3.10.1 httpx (≥ 0.27)

Description. A fully typed HTTP client offering a `requests`-compatible API with synchronous and asynchronous clients, optional HTTP/2, four-dimension timeouts, connection pooling via `Limits`, event hooks, and `MockTransport` for hermetic tests [Enc24].

Why included. It is the chapter’s foundation: the only mainstream Python client that unifies sync/async, expresses the full timeout taxonomy of Section 3.2.3, and makes offline testing a first-class feature rather than a monkey-patch.

Alternatives. `requests` [Pyt24] (sync-only, HTTP/1.1-only, two-dimension timeouts); `aiohttp` (async-only, its own API shape); stdlib `urllib` (dependency-free but austere, see Section 3.2.1).

Installation. `pip install "httpx>=0.27,<1.0"` — pin the upper bound while the library remains pre-1.0.

3.10.2 tenacity (≥ 8.3)

Description. A general-purpose retrying library: composable `stop`, `wait`, and `retry` strategies, with support for coroutines, statistics, and custom sleep injection.

Why included. Correct retry loops are subtle (budget arithmetic, exception filtering, jitter distribution, sleep abstraction). Tenacity makes the policy declarative and — decisive for this book — accepts an injected `sleep`, enabling the deterministic fault-matrix tests of Section 3.3.5.

Alternatives. `backoff` (decorator-oriented, less composable); `urllib3.util.Retry` (only within `requests/urllib3`, and its `respect_retry_after_header` defaults helpfully apply there); hand-rolled loops (acceptable for one-off scripts, error-prone at scale).

Installation. `pip install "tenacity>=8.3"`.

3.10.3 pydantic (≥ 2.7)

Description. Data-validation library using Python type annotations: declares typed models, validates and coerces inbound data, and produces structured errors with field-level paths [Col24].

Why included. The SDK-wrapper pattern (Capstone 2.3) needs a contract layer that turns “200 OK with a surprising body” into a precise, actionable `ApiContractError`. `pydantic v2`’s Rust core makes validation cheap enough for the hot path.

Alternatives. `dataclasses` + manual checks (stdlib, no validation); `msgspec` (faster, smaller ecosystem); `attrs` + `cattrs` (mature, more assembly required).

Installation. `pip install "pydantic>=2.7,<3.0"`.

3.10.4 pytest (≥ 8.0)

Description. The standard Python test framework: fixtures, parameterization, rich assertion introspection, and a plugin ecosystem.

Why included. Every listing in this chapter is validated by the fault-matrix suite (Listing 3.5); pytest’s `raises` introspection and fixtures make exception-taxonomy tests terse and readable.

Alternatives. `unittest` (stdlib, more boilerplate); `nose2` (smaller ecosystem). For async tests add `pytest-asyncio` — deliberately excluded here to keep the chapter’s suite purely synchronous and deterministic.

Installation. `pip install "pytest>=8.0"`.

3.11 Chapter Summary & Key Takeaways

- **Client choice is architectural.** `urllib` for zero-dependency bootstrap code; `requests` for legacy synchronous HTTP/1.1 estates; `httpx` for everything new — async, HTTP/2, four-dimension timeouts, and `MockTransport` testability [Enc24; Pyt24].
- **Every wait is bounded or it is a bug.** Connect, read, write, and pool timeouts bound four distinct hangs; “no timeout” is the enabling defect of cascading failure [Nyg18].
- **Pools are capacity.** One long-lived client per base URL; size `max_connections` to peak concurrency and `max_keepalive_connections` to steady state; churn re-pays DNS/TCP/TLS per request.
- **Retries are load.** Cap attempts *and* elapsed time, jitter fully (expected total wait $\frac{b}{2}(2^N - 1)$), honor `Retry-After`, budget the fleet, and gate non-idempotent methods behind idempotency keys [Fie22a].
- **Consume pagination lazily and payloads as streams.** Iterators hide offset/cursor/link-header mechanics; `iter_bytes` keeps memory $O(\text{chunk})$; cancellation and atomic rename keep artifacts clean.
- **Classify before you react.** Transport, status, and contract failures have different owners and remedies; parse only after the status check; never retry deterministic failures.
- **Test hermetically.** `MockTransport`, seeded randomness, and injected sleep yield a deterministic, offline fault matrix — the only kind of test suite a consumer library deserves.

3.12 Quick-Reference Card

3.12.1 Timeout Cheat-Sheet

Timeout	Bounds	Failure it catches	Starting value
connect	DNS + TCP + TLS setup	dead/brickwalled host, TLS mis-configuration	10 s
read	waiting for response bytes	app behind healthy LB; stalled stream	10 s (align to endpoint p99)
write	sending request body	peer that stopped reading (uploads)	10 s
pool	waiting for a free connection	concurrency > pool size (queueing)	infinite

Table 3.3: Timeout dimensions: always set all four; never leave any infinite.

3.12.2 Retry Decision Table

3.12.3 Status-Class Handling Table

3.13 Standardized Evidence Addendum

3.13.1 Benchmark Snapshot Template

3.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

3.13.3 Cost and Latency Budget Snapshot

3.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

3.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),

- policy version and rollback plan present in release metadata.

3.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

3.17 Cross-Chapter Decision Card

3.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

3.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Signal	Retry?	Conditions
Connect/Read timeout, connection reset	Yes	idempotent method or idempotency key; jittered backoff
408, 425	Yes	same as above
429	Yes	after Retry-After (clamped); back off the bucket
500, 502, 503, 504	Yes	bounded attempts + elapsed budget; 503 may carry Retry-After [Fie22a]
400, 401, 403, 404, 409, 422	No	deterministic: fix the request, rotate credentials, or reconcile state
2xx with unparseable/invalid body	No	ApiContractError : provider bug or version skew — alert
POST/PUT-side-effect without idempotency key	No	replay may duplicate effects; escalate to caller

Table 3.4: Retry decisions by failure signal.

Class	Meaning	Consumer action
1xx	Interim	handled by the transport; never surfaced to callers
2xx	Success	then (and only then) parse; validate against the contract
3xx	Redirect	follow per client policy; preserve method semantics; cap hops
4xx	Caller fault	typed ApiClientError4xx ; fix request; never retry as-is
5xx	Server fault	typed ApiServerError5xx ; retry per policy; alert on sustained rate

Table 3.5: Status-class handling for consumers (RFC 9110 [Fie22a]).

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 3.6: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 3.7: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 3.8: Decision card to complete per chapter.

Chapter 4

The REST Architectural Style & Resource Design

Executive Summary

Representational State Transfer (REST) is the most influential — and the most misunderstood — architectural style in modern software engineering. Coined by Roy Fielding in his 2000 doctoral dissertation [Fie00], REST is not a protocol, not a specification, and emphatically not “JSON over HTTP.” It is a *style*: a named set of architectural constraints that, when applied together, induce a specific, desirable set of system properties — scalability, reliability, visibility, portability, and independent evolvability. An HTTP API earns the label “RESTful” only to the degree that it honors those constraints; an HTTP API that ignores them is merely an HTTP API, which may still be a perfectly defensible engineering choice — provided the choice is made deliberately rather than by accident of vocabulary.

This chapter formalizes Fielding’s six constraints (client–server, statelessness, cacheability, uniform interface, layered system, and the optional code-on-demand), stating each precisely, enumerating the properties it induces, and identifying what each forbids. We then confront the industry’s chronic mislabeling problem through the Richardson Maturity Model [RAR13], a diagnostic ladder from the “swamp of POX” (Plain Old XML) through resources and HTTP verbs up to hypermedia controls. The middle of the chapter is devoted to the discipline of resource design: identifying resources as nouns, distinguishing collections, documents, stores, and controllers, and engineering URI grammars that are hierarchical, predictable, and opaque to clients. We give the uniform interface its full semantic treatment — method safety and idempotency, precise status codes for every CRUD lifecycle transition, PUT versus PATCH (including JSON Merge Patch and JSON Patch), and content negotiation — before turning to HATEOAS and the practical economics of hypermedia media types (HAL,

JSON:API, Collection+JSON). Representation design closes the theory: embedding versus linking, partial representations, and the anti-corruption discipline that keeps your API from becoming a serialized database dump.

Three complete, offline, stdlib-first Python 3.11 implementations anchor the theory in practice: a zero-framework REST parts-catalog service for the fictional *Northwind Robotics* universe that implements ETags, conditional requests, content negotiation, and correct method semantics with nothing but `http.server`; a URI design linter that flags style violations in route tables; and a HAL-style hypermedia serializer. A worked OpenAPI 3.1 excerpt [Ope21] demonstrates the design-first workflow. The chapter closes with thirty-six interview questions, five progressive capstone projects, and a quick-reference card. Mastery objective: by the end of this chapter you should be able to *state*, *defend*, and *enforce* REST's constraints — and to recognize, with surgical precision, when an API is RESTful, when it is merely HTTP, and why the difference matters for evolvability.

4.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its labs, and at least two capstone projects, you will be able to:

1. **Formalize** REST as an architectural style: state Fielding's six constraints verbatim-adjacent, name the properties each induces, and name what each forbids [Fie00].
2. **Diagnose** any HTTP API on the Richardson Maturity Model and articulate precisely which constraints it satisfies or violates [RAR13].
3. **Model** a business domain as resources: distinguish collections, documents, stores, and controllers; choose nouns over verbs; and design a coherent URI grammar (hierarchy, casing, pluralization, query versus path).
4. **Apply** the uniform interface with precision: safety and idempotency of every method, correct status codes for each CRUD transition, conditional requests, and PUT versus PATCH semantics [Fie22a].
5. **Evaluate** hypermedia (HATEOAS) as an engineering trade-off: compare HAL, JSON:API, and Collection+JSON, and decide when hypermedia pays for itself [Not17].
6. **Design** representations for evolution: embedding versus linking, par-

tial representations, and anti-corruption layering between the API and the database schema [Kle17].

7. **Implement** constraint-compliant REST services with the Python standard library alone, and **govern** API style with linting, design-first OpenAPI workflows, and review gates [Ope21; GH21].

Prerequisites

This chapter assumes you have completed Chapters 0–2 of this book:

- **Chapter 0** — the course contract, environment setup, and the Northwind Robotics fictional universe used for all synthetic data.
- **Chapter 1** — the HTTP wire protocol: requests, responses, headers, methods, status codes, and message bodies, as specified in RFC 9110 [Fie22a].
- **Chapter 2** — consuming HTTP APIs from Python: `urllib` and `http.client`, parsing JSON, handling errors, and reading response headers.

You should be comfortable reading an HTTP request–response exchange in raw text form and writing typed Python 3.11. No third-party packages are required for the core listings; the optional route-table validator uses `pyyaml` when YAML input is desired (Section 4.10). Everything in this chapter runs offline on a fresh Windows machine with PowerShell.

4.2 Deep Technical Mechanics & Architectural Concepts

4.2.1 REST Is an Architectural Style, Not a Protocol

The single most consequential sentence in Fielding’s dissertation is also the most ignored: REST is “an architectural style for distributed hypermedia systems” [Fie00]. An *architectural style*, in Fielding’s formal usage, is a named, coordinated set of architectural constraints. A *constraint* is a rule that restricts the design space — it *removes* options. Styles earn their keep because constraints, applied in combination, *induce properties*: emergent, measurable qualities of the resulting system such as scalability, reliability, visibility, and evolvability.

Definition 4.1 (Architectural Style). An *architectural style* is a coordinated set of constraints on the elements (components, connectors, data) of an

architecture and on the relationships among them, chosen so that the constraints jointly induce a desired set of system properties. REST is one such style; pipe-and-filter, event-driven integration, and the layered monolith are others.

Three practical consequences follow immediately:

1. **REST is not HTTP.** HTTP is a transfer protocol whose design *embodies* many REST constraints (Fielding co-authored it [Fie22a]), which makes HTTP an unusually convenient substrate. But REST could in principle be realized over another transfer mechanism, and — more importantly — an HTTP API can violate every REST constraint while remaining perfectly valid HTTP.
2. **“RESTful” is gradated, not binary.** An API satisfies individual constraints to a degree. The honest engineering question is never “is it REST?” but “which constraints does it honor, which does it trade away, and was that trade deliberate?”
3. **Compliance is bought, not declared.** Each constraint has a cost (for example, statelessness moves session data onto the wire; cacheability risks staleness). The style is a negotiated exchange: constraints in, properties out.

Key Concept

REST is a *style* (constraints $\rightarrow$ properties), not a protocol, product, or specification. “JSON over HTTP” describes two data formats in transit; it says nothing about architecture. An API is RESTful only insofar as it honors Fielding’s constraints — and the most commonly skipped one is the uniform interface’s hypermedia clause.

4.2.2 The Six Constraints, Formalized

Fielding derives REST incrementally, starting from the “null style” (no constraints) and adding one constraint at a time [Fie00]. We follow the same derivation, stating each constraint, its induced properties, and what it forbids.

Constraint 1: Client–Server

Statement. Separate the user-interface concerns (client) from the data-storage concerns (server) so that the two may evolve independently, communicating only through a well-defined interface.

Induced properties. *Portability* of the user interface across platforms; *scalability* by simplification of server components; and, most valuably, *independent evolvability* — the server can change its storage technology, and the client its rendering technology, without coordination.

Forbids. Shared-nothing violations in the small: the client may not reach into server storage (no database links exposed to consumers), and the server may not assume anything about client presentation.

Constraint 2: Statelessness

Definition 4.2 (Statelessness (Fielding’s sense)). Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. *Session state is therefore kept entirely on the client.*

Note the precise meaning: the server may of course hold *resource* state (the parts catalog itself). What it may not hold is *session* state — memory of where a particular client is in an interaction. A request authenticated with a bearer token, carrying its pagination cursor and all parameters, is stateless even though the token validates against server-side data.

Induced properties. *Visibility* (a monitoring system can understand a request in isolation), *reliability* (recovery from partial failure: any request can simply be retried), and *scalability* (no session affinity; any replica can serve any request, and memory is freed between requests).

Forbids. Server-side session contexts, “sticky” conversational state, and protocols whose requests are only interpretable relative to previous requests on the same connection. The cost: repetitive data per request and reduced server control over consistent application behavior — the classic trade Fielding acknowledges explicitly [Fie00].

Constraint 3: Cacheability

Statement. The data within a response to a request must be implicitly or explicitly labeled as cacheable or non-cacheable.

Induced properties. When a response is cacheable, a client cache may reuse it for later, equivalent requests: *efficiency*, *scalability*, and *user-perceived performance* improve, sometimes by eliminating network interactions entirely. HTTP realizes this constraint with validators (ETag, Last-Modified) and freshness directives (Cache-Control) [Fie22a].

Forbids. Ambiguity: a response that says nothing about its cacheability forces intermediaries to guess, and guessed caching is how stale inventory gets sold twice. The admitted risk is decreased *reliability* if stale cached data differs significantly from what a direct request would have returned.

Constraint 4: Uniform Interface

This is the constraint that distinguishes REST from generic HTTP usage, and it decomposes into four sub-constraints [Fie00]:

1. **Identification of resources.** Each resource is named by a stable identifier (in HTTP, a URI). The resource is the conceptual entity; the identifier is its name.
2. **Manipulation of resources through representations.** Clients never touch the resource itself; they exchange *representations* (e.g., a JSON document) of its current or desired state. A representation plus its metadata is sufficient to modify or delete the resource, given permission.
3. **Self-descriptive messages.** Each message carries enough metadata (media type, method, status code, cache directives) to describe its own processing. Intermediaries can act on messages without private knowledge.
4. **Hypermedia as the engine of application state (HATEOAS).** The client navigates the application by following links and forms discovered in representations — not by constructing URIs from out-of-band knowledge.

Definition 4.3 (Resource). A *resource* is any named conceptual target of a hypertext reference: a document, a temporal service (“today’s weather in Lodz”), a collection of other resources, a non-virtual object (a person), and so forth. A resource is a mapping to a set of entities; the set may vary over time, provided the *semantics* of the mapping remain stable. “The current price of part NX-1001” is a fine resource even though its value changes hourly.

Definition 4.4 (Representation). A *representation* is a sequence of bytes plus metadata (media type, validators, links) that captures the current or intended state of a resource at the time of a request. One resource admits many representations (JSON, CSV, HTML); representations are negotiated, not assumed.

Induced properties. *Simplicity* of the overall architecture, *visibility* for intermediaries, and *independent evolvability*, because implementations are decoupled from the services they access. **The cost is efficiency:** a uniform interface transfers information in a standardized form rather than a form specialized to one application’s needs.

Constraint 5: Layered System

Statement. The architecture is composed of hierarchical layers; each component cannot “see” beyond the immediate layer with which it interacts. Intermediaries — gateways, load balancers, shared caches, content delivery networks — may be introduced at any layer without changing component semantics.

Induced properties. *Scalability* (load balancing, shared caching at organizational boundaries), *security policy enforcement* at the edge, and *legacy encapsulation*. **The cost** is added latency and data overhead per hop, which the cache constraint is expected to compensate for.

Constraint 6: Code-on-Demand (Optional)

Statement. The server may temporarily extend client functionality by transferring executable code (scripts, applets) which the client executes.

Induced properties. *Extensibility* and simplified clients (features deploy without client installation). **Forbids/reduces:** *visibility* — an intermediary cannot understand what arbitrary code will do — which is precisely why Fielding marks this constraint optional. JavaScript delivered to browsers is its canonical realization; for machine-to-machine APIs it is almost always omitted.

Figure 4.1 traces one conditional GET through a concrete layered deployment, annotating which constraint each layer exploits.

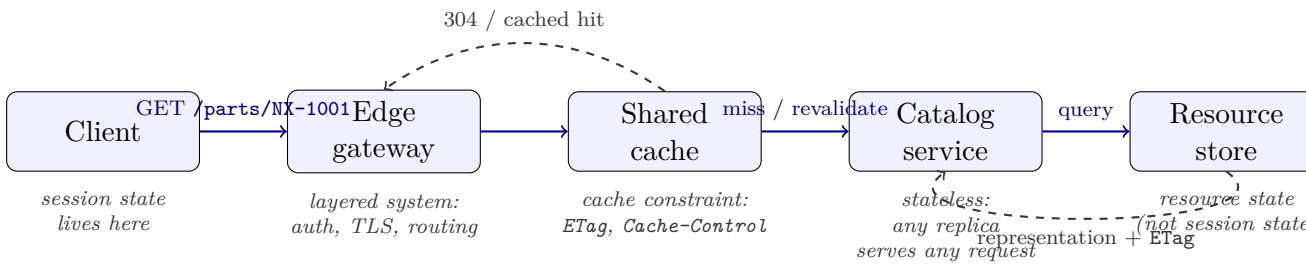


Figure 4.1: Request lifecycle through a layered, constraint-compliant deployment. Each layer sees only its immediate neighbors (layered system); the shared cache short-circuits revalidation (cacheability); the service holds no session state (statelessness).

Table 4.1 summarizes the exchange each constraint offers.

Constraint	Induced properties	Cost / forbids
Client-server	portability, independent evolution	no shared storage access
Statelessness	visibility, reliability, scalability	repeated context per request
Cacheability	efficiency, perceived performance	staleness risk; must label responses
Uniform interface	simplicity, visibility, decoupling	less efficient than be-spoke forms
Layered system	scalability, security at boundaries	per-hop latency
Code-on-demand (opt.)	extensibility, thin clients	reduced visibility

Table 4.1: Fielding’s six constraints as an exchange: constraints in, properties out [Fie00].

4.2.3 “JSON over HTTP Is Not REST”: The Richardson Maturity Model

Because the industry adopted the word “REST” long before it absorbed Fielding’s constraints, a corrective vocabulary became necessary. Leonard Richardson’s *Maturity Model* [RAR13] classifies HTTP APIs into four levels according to which web technologies they actually exploit. It is best read not as a moral hierarchy but as a diagnostic: each level names the constraints an API has *started* to honor. Figure 4.2 shows the ladder.

Level 0: The Swamp of POX

A single endpoint receives every operation; the HTTP method and status code carry no application meaning. This is HTTP used as a tunnel — SOAP and many legacy “XML-RPC over HTTP” services live here.

```

1 POST /rpc HTTP/1.1
2 Content-Type: application/json
3
4 {"action": "getPart", "sku": "NX-1001"}
5
6 HTTP/1.1 200 OK
7 Content-Type: application/json
8
9 {"status": "ok", "part": {"sku": "NX-1001", "name": "
    Servo Motor 12V"}}
10
11 POST /rpc HTTP/1.1
12 Content-Type: application/json

```

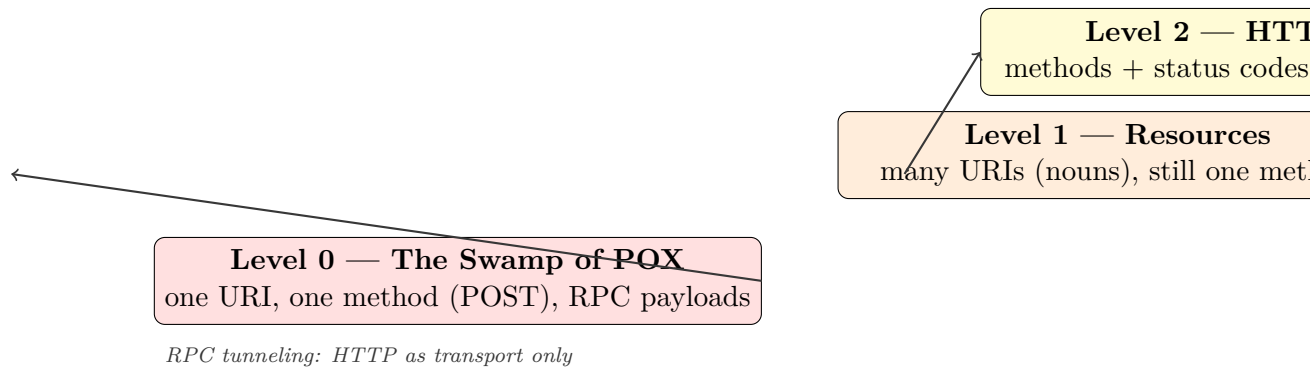


Figure 4.2: The Richardson Maturity Model. Each step adopts one more web technology: resource URIs, then HTTP method/status semantics, then hypermedia controls [RAR13].

```

13
14 {"action": "getPart", "sku": "NOPE"}
15
16 HTTP/1.1 200 OK
17 Content-Type: application/json
18
19 {"status": "error", "message": "part not found"}

```

Every failure is a 200 OK; no proxy, cache, or monitoring tool can distinguish success from failure, safe from unsafe. Nothing about the interaction is visible to the web’s infrastructure.

Level 1: Resources

The API introduces individually addressable resources — URIs are nouns — but still tunnels all operations through one method.

```

1 POST /parts/NX-1001 HTTP/1.1
2 Content-Type: application/json
3
4 {"action": "restock", "quantity": 40}
5
6 HTTP/1.1 200 OK
7 {"status": "ok", "stock": 180}

```

Resources now exist as first-class addressable things (a genuine improvement: URIs can be bookmarked, logged, and secured individually), but safety, idempotency, and cacheability semantics remain unavailable because every request is a POST.

Level 2: HTTP Verbs

Methods and status codes are used as RFC 9110 intends [Fie22a]: GET for retrieval (safe, cacheable), PUT/PATCH for update, DELETE for removal, POST for subordinate creation, with status codes that tell the truth.

```
1 GET /parts/NX-1001 HTTP/1.1
2 Accept: application/json
3
4 HTTP/1.1 200 OK
5 ETag: "v3-9f2c"
6 Content-Type: application/json
7
8 {"sku": "NX-1001", "name": "Servo Motor 12V", "stock":
9   140}
10
11 GET /parts/NOPE HTTP/1.1
12
13 HTTP/1.1 404 Not Found
14 Content-Type: application/problem+json
15
16 {"type": "about:blank", "title": "Not Found", "status":
   404,
   "detail": "No part with sku 'NOPE' exists."}
```

Level 2 is where caches, conditional requests, idempotent retries, and honest monitoring become possible. The overwhelming majority of well-run production APIs live here.

Level 3: Hypermedia Controls (HATEOAS)

Representations carry *controls* — links and forms — that advertise the client's legal next steps, so application state is driven by server-supplied hypermedia rather than client-side URI construction [Not17].

```
1 {
2   "sku": "NX-1001",
3   "name": "Servo Motor 12V",
4   "stock": 140,
5   "_links": {
6     "self": { "href": "/parts/NX-1001" },
7     "collection": { "href": "/parts" },
```

```
8     "category": { "href": "/categories/actuators" },
9     "restock": { "href": "/parts/NX-1001/stock-
10                 adjustments",
11                 "method": "POST",
12                 "title": "Record a stock adjustment"
13             }
14 }
15 }
```

A Level 3 client that needs to restock a part does not *know* the adjustment URI; it discovers it. The server can therefore restructure its URI space — insert a gateway, shard by region, rename collections — without breaking clients, because clients treat URIs as opaque (Section 4.2.4). Section 4.2.6 examines why this highest level remains rare and when it is worth the investment.

Warning

Richardson’s levels are diagnostic, not aspirational dogma. Level 3 buys evolvability at real cost in tooling and client complexity. The engineering sin is not stopping at Level 2; it is calling a Level 0 service “RESTful” and thereby importing expectations (caches, safe retries, intermediaries) the service cannot meet.

4.2.4 Resource Modeling Discipline

REST’s central design act is deciding *what the things are*. Get the resource model right and methods, status codes, caching, and security policies fall into place; get it wrong and no amount of framework sophistication will save the API.

Nouns, Not Verbs

A resource URI names a *thing* (a noun); the HTTP method supplies the *operation* (the verb). The discipline exists because the uniform interface already provides a fixed, universally understood verb vocabulary — embedding verbs in URIs duplicates that vocabulary with an unbounded, vendor-specific one, destroying visibility: `GET /getPart?id=7` tells an intermediary nothing about safety, while `GET /parts/NX-1001` declares itself safe and cacheable by construction.

When a domain concept genuinely *is* an action (“restock,” “authorize,” “translate”), the RESTful resolution is to *reify the action’s result* as a resource: instead of `POST /parts/NX-1001/restock`, create `POST /parts/NX-1001/stock-adjustments` — each adjustment is a record with

its own identity, audit trail, and lifecycle. The verb becomes a noun, and the audit log you would have built anyway becomes the resource.

The Four Resource Archetypes

Following the taxonomy popularized by Massé’s *REST API Design Rulebook* (see Section 4.8), resources come in four archetypes:

- **Document** — a singular, object-like resource: `/parts/NX-1001`. The REST analogue of a row or object instance.
- **Collection** — a server-managed directory of documents: `/parts`. Clients `GET` it; `POST` proposes new members.
- **Store** — a *client*-managed resource repository: the client decides when an entry is created, updated, or deleted (`PUT /parts/NX-1001` with a client-chosen SKU is store-like).
- **Controller** — the sanctioned escape hatch: a verb-like resource standing in for an operation that cannot be reified: `POST /catalog/exports` (creates an export job) is better modeled as a collection of *jobs*; `POST /password-resets` similarly turns a procedure into a resource. Use controllers only when reification fails.

Sub-Resources and Relationship Modeling

Hierarchy in the path should express *containment* or *ownership* that outlives either participant’s individual addressing: a stock level exists only in the context of a warehouse *and* a part, so `/warehouses/WH-01/stock/NX-1001` is defensible. Independent aggregates with their own lifecycles deserve top-level URIs linked by hypermedia or foreign identifiers, not deep nesting:

1	<code>GET /warehouses/WH-01</code>	<code># document</code>
2	<code>GET /warehouses/WH-01/stock</code> collection scoped by parent	<code>#</code>
3	<code>GET /warehouses/WH-01/stock/NX-1001</code> resource (contained)	<code># sub-</code>
4	<code>GET /purchase-orders?warehouse=WH-01</code> relationship via query filter	<code>#</code>
5	<code>GET /purchase-orders/PO-2026-0417</code> independent aggregate, top level	<code>#</code>

Rule of thumb, enforced by the linter in Listing 4.3.2: nest at most two levels. Beyond that, prefer a top-level collection with a query filter. Deep paths encode a *single* navigational path into a graph that has many; they ossify one query pattern and complicate authorization, caching keys,

and refactoring. Figure 4.3 shows the resource graph for the Northwind Robotics warehouse domain developed in the lab (Section 4.5): solid edges are containment (path hierarchy), dashed edges are associations (query filters or hypermedia links).

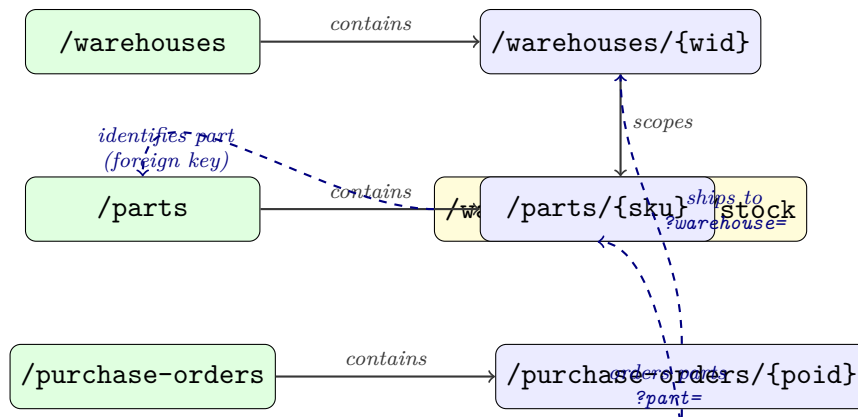


Figure 4.3: Resource-relationship graph for the Northwind Robotics warehouse domain. Containment (solid) appears in the path; independent aggregates (purchase orders) get top-level URIs and relate via query filters and links (dashed), keeping nesting at or below two levels.

URI Design Grammar

A URI decomposes as `scheme://authority/path?query#fragment`; path segments left-to-right descend a hierarchy. Production conventions, settled with arguments rather than taste:

- **Hierarchy.** Forward slashes express containment; `/warehouses/WH-01/stock` reads as a path *into* the domain. Do not use hierarchy to encode *queries*: filters belong after `?`.
- **Casing.** Lowercase *kebab-case* for multi-word segments (`/purchase-orders`). URIs are case-sensitive per RFC 9110 [Fie22a], but hosts and humans are not; camelCase paths (`/purchaseOrders`) produce duplicate-resource bugs behind case-insensitive proxies and normalize differently across gateways. Kebab-case is also what Zalando’s and Microsoft’s published guidelines converge on (Section 4.8).
- **Pluralization.** The honest answer: the debate is noise; the invariant is *consistency*. Plural collections win on three arguments: (1) the collection URI `/parts` and the document URI `/parts/NX-1001`

then share a stem, making containment legible; (2) plural reads correctly for both `GET /parts` (many) and `GET /parts/NX-1001` (one member *of* many); (3) singular resources remain expressible as singleton documents (`/catalog/profile`) without forcing every collection into grammatical contortions. Mixed schemes (`/part/NX-1001` but `/warehouses`) are the only unambiguously wrong answer, and the linter in Section 4.3 treats them as defects.

- **Trailing slashes.** `/parts` and `/parts/` are *different URIs* [Fie22a]; permitting both doubles cache keys and splits analytics. Pick the bare form, and answer the slashed form with `301 Moved Permanently` — a redirect is cacheable and self-healing; silent aliasing is not.
- **Query versus path.** Path segments identify; query parameters *modulate*: filtering (`?category=sensors`), sorting (`?sort=-unitPrice`), pagination (`?cursor=...`), partial representation (`?fields=sku,name`). If removing the parameter changes *which* resource is addressed, it belongs in the path; if it changes only *how much* or *which view* of that resource is returned, it belongs in the query.

URI Opacity

Definition 4.5 (URI Opacity). Clients must treat URIs as opaque identifiers: they may store, compare, and dereference them, but must not parse, construct, or infer semantics from their internal structure. Knowledge of how to *form* a URI lives on the server and is communicated through hypermedia or (pragmatically) a versioned contract such as OpenAPI [Ope21].

Opacity is what makes Level 3 restructuring safe and what makes even Level 2 APIs evolvable: the day a client string-templates `/parts/ + sku`, your URI grammar has become an unchangeable public contract whether you intended it or not. Fielding’s formulation is blunt — the only types of URI a RESTful system may mint are those embedded in representations or constructed through server-supplied templates [Fie00].

4.2.5 Uniform Interface Semantics

Safety and Idempotency

RFC 9110 assigns every standard method two formal properties [Fie22a]. *Safety* means the request is essentially read-only: the client does not request, and cannot be held accountable for, any state change. *Idempotency* means the intended effect of multiple identical requests is the same as that of a single request — the property that makes automatic retries of timed-out requests a sound engineering decision rather than a gamble.

Method	Safe	Idempotent	Typical semantics on <code>/parts</code>
GET	yes	yes	retrieve a representation; cacheable by default
HEAD	yes	yes	GET without the body; metadata probe
OPTIONS	yes	yes	discover allowed methods (Allow)
POST	no	no	create subordinate; server assigns identity
PUT	no	yes	full replacement (or creation) at a known URI
PATCH	no	no *	partial modification; semantics defined by media type
DELETE	no	yes	remove; repeating yields 404, state unchanged

Table 4.2: Method properties per RFC 9110 [Fie22a]. * PATCH is not *guaranteed* idempotent, but a specific patch document applied twice usually is — design for it when you can.

Idempotency is defined over *server state*, not responses: repeating DELETE `/parts/NX-1001` returns 404 the second time, yet the method is idempotent because the resulting state (no such part) is identical. This is why load balancers and client libraries may transparently retry idempotent methods and must never transparently retry POST.

Status Codes for the CRUD Lifecycle

Status codes are the server’s half of the uniform interface: they let generic components — caches, gateways, monitors, retry loops — react correctly without private knowledge. Table 4.3 gives the lifecycle map used throughout this book [Fie22a].

Two habits deserve special emphasis. First, 201 **Created** without a **Location** header is half an answer: the client learns *that* a resource was created but not *where* it lives. Second, conditional requests — **If-None-Match** on GET (yielding 304 **Not Modified**) and **If-Match** on PUT/DELETE (yielding 412 **Precondition Failed** on mismatch) — turn caching and optimistic concurrency from application-level schemes into generic HTTP machinery that every intermediary already understands [Fie22a]. Listing 4.3.1 implements both.

PUT versus PATCH: Replacement versus Modification

PUT is *full replacement*: the enclosed representation becomes the new state of the resource; omitted fields are reset, not preserved. It is idempotent by construction. PATCH is *partial modification*: the body describes a *change*,

and its semantics are defined by the patch media type. Two standardized patch formats dominate:

- **JSON Merge Patch (RFC 7386).** The patch is itself a JSON document; it is recursively merged into the target: provided members replace, *null* members are *removed*, absent members are untouched. Trivial to produce (any JSON object is a valid patch), but arrays are replaced wholesale and *null* can never be a stored value — you cannot distinguish “absent” from “set to null.”

```
1 PATCH /parts/NX-1001          Content-Type:
   application/merge-patch+json
2 { "unitPrice": { "currency": "USD", "amountCents":
   2799 }, "legacyNote": null }
```

- **JSON Patch (RFC 6902).** The patch is an *array of operations* — add, remove, replace, move, copy, **test** — applied in order with transaction semantics: if any operation fails, none is applied. The **test** operation doubles as a lightweight precondition (“apply only if /stock is still 140”). Verbose, but precise, array-aware, and atomically safe.

```
1 PATCH /parts/NX-1001          Content-Type:
   application/json-patch+json
2 [
3   { "op": "test",      "path": "/stock", "value":
   140 },
4   { "op": "replace", "path": "/stock", "value":
   180 }
5 ]
```

Choose Merge Patch for forgiving, human-authored configuration updates; choose JSON Patch for collaborative, concurrent editing where atomicity and preconditions matter. Chapter 10 develops idempotency and concurrency control in depth.

Content Negotiation in Resource Design

Because resources are manipulated *through representations*, the same resource may be rendered as JSON, CSV, or a vendor-specific media type. Server driven negotiation uses the **Accept** header with quality values; the server answers with the best available representation or **406 Not Acceptable** [Fie22a]:

```
1 GET /parts HTTP/1.1
```

```

2 Accept: application/hal+json;q=1.0, text/csv;q=0.5
3
4 HTTP/1.1 200 OK
5 Content-Type: application/hal+json
6 Vary: Accept

```

Design consequences: (1) always send **Vary: Accept** so shared caches key variants separately; (2) negotiate *format*, not *semantics* — a **text/csv** rendering of **/parts** must describe the same resource state, merely shaped differently; (3) media-type parameters (**application/vnd.northwind.part+json;version=2**) are a legitimate versioning surface, examined against URI versioning in Chapter 9.

4.2.6 HATEOAS and Hypermedia

Links and Affordances

RFC 8288 standardizes *web linking*: typed links between resources, serializable in **Link** headers or in the body, each carrying a relation type (**rel**) that names its semantics (**self**, **next**, **edit**, or an extension relation) [Not17]. A link is a *navigation affordance*; a link plus method plus expected payload (HAL forms, Collection+JSON templates) is an *action affordance* — the hypermedia equivalent of an HTML form. Together they let the server, at runtime, advertise the client’s legal state transitions: the **restock** link in our earlier example can simply be *absent* when the part is discontinued, and the client needs no ‘if discontinued’ logic.

Three Hypermedia Media Types Compared

HAL is the pragmatic minimum: two reserved members buy link-driven navigation with near-zero disruption [Not17]. JSON:API standardizes nearly every recurring design decision — pagination, filtering, error objects, compound documents — which eliminates bikeshedding across teams but constrains resource shapes. Collection+JSON is the purest realization of Fielding’s constraint: its **template** tells clients how to *write* without any out-of-band schema.

Why Hypermedia Is Rare, and When It Pays Off

Honesty about adoption matters for engineering judgment. Hypermedia APIs are rare because (1) most API consumers are first-party web and mobile clients whose release cadence matches the server’s, so compile-time coupling to URIs is tolerable; (2) mainstream tooling (OpenAPI generators, SDK pipelines) is built around static contracts, not runtime-discovered affordances; (3) hypermedia shifts complexity onto client authors, who must

write generic link-followers instead of calling endpoints. Hypermedia pays off when the cost asymmetry flips: long-lived public APIs whose servers must restructure without breaking integrations they do not control; workflow-heavy domains (order state machines, approval chains) where the legal next actions genuinely vary per state; and generic clients (crawlers, API browsers, agentic consumers) that cannot embed per-API path knowledge. The professional default for this book: Level 2 with *selective* hypermedia — `self/next/prev` pagination links and state-transition links on workflow resources — capturing most of the evolvability at a fraction of the cost [RAR13].

4.2.7 Representation Design

Embedding versus Linking

Should `GET /purchase-orders/P0-2026-0417` embed its line items, link to them, or both? Embedding saves round trips and atomicity of read; linking saves payload size, enables independent caching and freshness per resource, and keeps the representation stable as the related collection grows. The disciplined default: embed *identity and summary* (id, name, and a link) of related resources; offer full embedding behind an explicit opt-in (`?embed=lines`) so the base representation stays small and cacheable. Never embed unbounded collections in a document representation — that is a pagination accident waiting to happen (Chapter 8).

Partial Representations

Sparse fieldsets (`?fields=sku,name`) let bandwidth-sensitive clients (mobile, high-fan-out aggregators) pay only for what they render. Keep the mechanism simple, deterministic, and documented in the contract; projection must never change *which* resource is addressed — that is what paths are for.

API Is Not Database Schema: Anti-Corruption Thinking

The fastest way to freeze your database forever is to serialize it. When `GET /parts/NX-1001` is an ORM object passed through a JSON encoder, every column rename, normalization, or storage migration becomes a breaking API change — and every breaking API change is now a schema change negotiated with every consumer. The corrective is an *anti-corruption layer*: a deliberate translation between the storage model and the published representation. The catalog in Section 4.3 stores `unit_price_cents` (an integer) but publishes a structured `unitPrice` object with currency — the representation is designed for consumers, the storage column for accountants,

and neither dictates the other. Kleppmann’s treatment of schema evolution gives the theoretical grounding: systems evolve safely when writers and readers are decoupled by explicit encodings with defined compatibility rules [Kle17].

Evolution-Friendly Representations

Four rules carry most of the weight [GH21]: (1) *additive only* — new optional fields are safe; removing or renaming fields is a breaking change requiring a version transition (Chapter 9); (2) *tolerant readers* — clients must ignore unknown members, and the contract must say so; (3) *never reuse names* — a retired field’s name stays retired, or old clients will misread new data; (4) *explicit nulls and enums* — document whether absent and null differ, and treat enum extension as a breaking change for strict consumers. Applied consistently, these rules let a Level 2 API evolve for years without version banners.

4.2.8 API as a Product

An API is a product whose customers are developers. Consumer-first design inverts the usual implementation order: write the contract a consumer would *want* to integrate against *before* writing the code that fulfills it. The design-first workflow is concrete: draft the OpenAPI 3.1 document [Ope21]; review it with a representative consumer; lint it against the organization’s style guide; generate mocks so consumers can integrate against the contract immediately; only then implement, with the contract driving tests (Chapters 4 and 19). Consistency at scale is a governance problem, not a talent problem: published style guides (Zalando, Microsoft — see Section 4.8), automated linting in CI, and a review gate for contract changes turn “we value consistency” from a sentiment into a mechanism [GH21]. The API Design Patterns corpus [GH21] catalogues the recurring decisions — resource layout, pagination tokens, long-running operations — that such governance should standardize once and reuse everywhere.

4.3 Production-Ready Code Implementation

The three listings in this section are complete, typed, and runnable offline on Python 3.11+ with no third-party dependencies. All data is synthetic, drawn from the fictional Northwind Robotics universe. Run each from PowerShell, e.g. `python catalog_api.py --selftest`.

4.3.1 A Stdlib-Only REST Mini-Framework: The Parts Catalog

Listing 4.3.1 implements the catalog domain on nothing but `http.server`. It is deliberately framework-free so that every constraint is visible in code: statelessness (no session state exists anywhere), the uniform interface (method dispatch, status codes, `Allow` headers), cacheability (ETags, conditional requests, `Cache-Control`), self-descriptive messages (`Content-Type`, content negotiation, `Vary`), and layered-system friendliness (it speaks plain HTTP intermediaries can cache and route).

```

1  """Northwind_Robotics_parts_catalog_API_(stdlib-only,
    offline).
2
3  Demonstrates_Fielding's_constraints_without_frameworks:
4  **_uniform_interface_-_correct_methods,_status_codes,
    _Allow_headers
5  **_cacheability_____ETag,_If-None-Match_(304),_If-
    Match_(412),
6  _____Cache-Control_on_every_
    representation
7  **_statelessness_____zero_session_state;_every_
    request_self-contained
8  **_self-descriptiveness_-_content_negotiation_(JSON/_
    CSV),_Vary:_Accept
9
10 Python_3.11+._Run:
11 _____python_catalog_api.py_--selftest_____#_
    deterministic_in-process_checks
12 _____python_catalog_api.py_--port_8080_____#_serve;_try_
    curl.exe_-i_...
13
14 from __future__ import annotations
15
16 import argparse
17 import hashlib
18 import json
19 import logging
20 import re
21 from dataclasses import dataclass
22 from http import HTTPStatus
23 from http.server import BaseHTTPRequestHandler,
    ThreadingHTTPServer
24 from typing import Any
25 from urllib.parse import parse_qs, urlparse
26

```



```

27 logger = logging.getLogger("northwind.catalog")
28
29 #
    -----
30 # Domain layer (anti-corruption: storage shape !=
    representation shape)
31 #
    -----
32
33 SKU_PATTERN = re.compile(r"^[A-Z]{2}-\d{4}$")
34 VALID_CATEGORIES = {"actuators", "sensors", "frames", "
    mobility"}
35
36
37 class ValidationError(Exception):
38     """Raised when a representation fails boundary
    validation(400)."""
39
40
41 class ConflictError(Exception):
42     """Raised when a write conflicts with existing
    state(409)."""
43
44
45 @dataclass
46 class Part:
47     sku: str
48     name: str
49     category: str
50     unit_price_cents: int
51     stock: int
52     version: int = 1
53
54     def to_representation(self) -> dict[str, Any]:
55         # Published shape is designed for consumers,
            not for the table:
56         # money is a structured object; the storage
            column is an int.
57         return {
58             "sku": self.sku,
59             "name": self.name,
60             "category": self.category,
61             "unitPrice": {
62                 "currency": "USD",

```

```

63         "amountCents": self.unit_price_cents,
64     },
65     "stock": self.stock,
66 }
67
68
69 def parse_part(body: bytes, *, sku_from_path: str |
None = None) -> Part:
70     """Validate a client representation into a Part.
Fail fast, specifically."""
71     try:
72         data = json.loads(body.decode("utf-8"))
73     except (UnicodeDecodeError, json.JSONDecodeError)
as exc:
74         raise ValidationError(f"request body is not
valid JSON: {exc}") from exc
75     if not isinstance(data, dict):
76         raise ValidationError("request body must be a
JSON object")
77
78     price = data.get("unitPrice")
79     if not isinstance(price, dict) or not isinstance(
price.get("amountCents"), int
80 ):
81         raise ValidationError("unitPrice.amountCents(
integer) is required")
82     if price["amountCents"] < 0:
83         raise ValidationError("unitPrice.amountCents
must be >= 0")
84
85
86     name = data.get("name")
87     category = data.get("category")
88     stock = data.get("stock")
89     if not isinstance(name, str) or not name.strip():
90         raise ValidationError("name (non-empty string)
is required")
91     if category not in VALID_CATEGORIES:
92         raise ValidationError(
93             f"category must be one of {sorted(
VALID_CATEGORIES)}"
94         )
95     if not isinstance(stock, int) or stock < 0:
96         raise ValidationError("stock (integer >= 0) is
required")
97
98     sku = sku_from_path or data.get("sku", "")

```

```

99     if not SKU_PATTERN.match(sku):
100         raise ValidationError("sku_must_match_pattern_
            AA-0000")
101     if sku_from_path and data.get("sku") not in (None,
        sku_from_path):
102         raise ValidationError("body_sku_must_match_the_
            URI_sku")
103
104     return Part(
105         sku=sku,
106         name=name.strip(),
107         category=category,
108         unit_price_cents=price["amountCents"],
109         stock=stock,
110     )
111
112
113 class CatalogStore:
114     """In-memory repository seeded with deterministic
        synthetic data."""
115
116     def __init__(self) -> None:
117         self._parts: dict[str, Part] = {}
118         self._seed()
119
120     def _seed(self) -> None:
121         for part in (
122             Part("NX-1001", "Servo_Motor_12V", "
                actuators", 2599, 140),
123             Part("NX-1002", "LIDAR_Module_R2", "sensors
                ", 18999, 32),
124             Part("NX-1003", "Aluminum_Chassis_Plate", "
                frames", 7450, 58),
125             Part("NX-1004", "Omni_Wheel_96mm", "
                mobility", 3250, 210),
126         ):
127             self._parts[part.sku] = part
128
129     def list(self, category: str | None = None) -> list
        [Part]:
130         parts = sorted(self._parts.values(), key=lambda
            p: p.sku)
131         if category is not None:
132             parts = [p for p in parts if p.category ==
                category]
133         return parts

```

```

134
135     def get(self, sku: str) -> Part | None:
136         return self._parts.get(sku)
137
138     def create(self, part: Part) -> None:
139         if part.sku in self._parts:
140             raise ConflictError(f"part_{part.sku}_
141                                 already_exists")
142         self._parts[part.sku] = part
143
144     def replace(self, sku: str, part: Part) -> bool:
145         """PUT semantics: full replacement. Returns
146         True if it existed."""
147         existed = sku in self._parts
148         old = self._parts.get(sku)
149         part.version = (old.version + 1) if old else 1
150         self._parts[sku] = part
151         return existed
152
153     def delete(self, sku: str) -> bool:
154         return self._parts.pop(sku, None) is not None
155
156 #
157 -----
158
159 # HTTP layer
160 #
161 -----
162
163
164
165 COLLECTION_RE = re.compile(r"^/parts$")
166 DOCUMENT_RE = re.compile(r"^/parts/(?P<sku>[A-Za-z0
167 -9-]+)$")
168 JSON = "application/json"
169 CSV = "text/csv"
170
171
172 def etag_for(part: Part) -> str:
173     digest = hashlib.sha256(
174         json.dumps(part.to_representation(), sort_keys=
175                     True).encode()
176     ).hexdigest()[:12]
177     return f'p{part.version}-{digest}'

```

```

172 def negotiate(accept: str | None) -> str | None:
173     """Pick the best supported media type, honoring q-
        values. None=>406."""
174     if not accept or accept.strip() in ("", "*/*"):
175         return JSON
176     best: tuple[float, str] | None = None
177     for item in accept.split(","):
178         media, _, params = item.partition(";")
179         media = media.strip().lower()
180         quality = 1.0
181         for param in params.split(";"):
182             key, _, value = param.partition("=")
183             if key.strip() == "q":
184                 try:
185                     quality = float(value)
186                 except ValueError:
187                     quality = 0.0
188             if media in (JSON, CSV) and quality > 0:
189                 if best is None or quality > best[0]:
190                     best = (quality, media)
191     return best[1] if best else None
192
193
194 def to_csv(reps: list[dict[str, Any]]) -> bytes:
195     lines = ["sku,name,category,unit_price_cents,stock"]
196     for r in reps:
197         name = r["name"].replace("'", '"')
198         lines.append(
199             f'{{r["sku"]}},{{name}},{{r["category"]}},'
200             f'{{r["unitPrice"]["amountCents"]}},{{r["stock"]}}'
201         )
202     return ("\r\n".join(lines) + "\r\n").encode("utf-8")
203
204
205 class CatalogHandler(BaseHTTPRequestHandler):
206     server_version = "NorthwindCatalog/1.0"
207     store: CatalogStore # injected by make_server
208
209     def log_message(self, fmt: str, *args: Any) -> None:
210         :
211         logger.info("%s-%s", self.address_string(),
            fmt % args)

```

```

212 # -- response helpers
213 -----
214
215 def _send(
216     self,
217     status: HTTPStatus,
218     body: bytes = b"",
219     content_type: str = JSON,
220     extra: dict[str, str] | None = None,
221 ) -> None:
222     self.send_response(status)
223     self.send_header("Content-Type", content_type)
224     self.send_header("Content-Length", str(len(body)))
225     self.send_header("Cache-Control", "no-cache")
226     self.send_header("Vary", "Accept")
227     for key, value in (extra or {}).items():
228         self.send_header(key, value)
229     self.end_headers()
230     if body and self.command != "HEAD":
231         self.wfile.write(body)
232
233 def _problem(self, status: HTTPStatus, detail: str)
234     -> None:
235     body = json.dumps(
236         {"title": status.phrase, "status": int(
237             status), "detail": detail}
238     ).encode()
239     self._send(status, body, "application/problem+
240         json")
241
242 def _read_body(self) -> bytes:
243     length = int(self.headers.get("Content-Length")
244         or 0)
245     return self.rfile.read(length) if length else b
246         ""
247
248 def _check_json_content_type(self) -> None:
249     ctype = (self.headers.get("Content-Type") or "")
250         .split(";")[0].strip()
251     if ctype not in (JSON, "application/merge-patch
252         +json"):
253         raise UnsupportedMediaType(ctype)
254
255 # -- method entry points

```

```

248 -----
249 def do_GET(self) -> None: # noqa: N802 (http.
    server naming)
250     self._dispatch("GET")
251
252 def do_HEAD(self) -> None: # noqa: N802
253     self._dispatch("GET")
254
255 def do_POST(self) -> None: # noqa: N802
256     self._dispatch("POST")
257
258 def do_PUT(self) -> None: # noqa: N802
259     self._dispatch("PUT")
260
261 def do_PATCH(self) -> None: # noqa: N802
262     self._dispatch("PATCH")
263
264 def do_DELETE(self) -> None: # noqa: N802
265     self._dispatch("DELETE")
266
267 def do_OPTIONS(self) -> None: # noqa: N802
268     parsed = urlparse(self.path)
269     allowed = (
270         "GET, HEAD, POST, OPTIONS"
271         if COLLECTION_RE.match(parsed.path)
272         else "GET, HEAD, PUT, PATCH, DELETE, OPTIONS"
273     )
274     self._send(HTTPStatus.NO_CONTENT, extra={"Allow": allowed})
275
276 # -- router
277 -----
278
279 def _dispatch(self, method: str) -> None:
280     parsed = urlparse(self.path)
281     path = parsed.path.rstrip("/") or "/"
282     query = parse_qs(parsed.query)
283     doc = DOCUMENT_RE.match(path)
284     try:
285         if COLLECTION_RE.match(path):
286             self._collection(method, query)
287         elif doc:
288             self._document(method, doc.group("sku"))

```

```

    )
288     else:
289         self._problem(HTTPStatus.NOT_FOUND, f"
            no_resource_at_{path}")
290     except ValidationError as exc:
291         self._problem(HTTPStatus.BAD_REQUEST, str(
            exc))
292     except ConflictError as exc:
293         self._problem(HTTPStatus.CONFLICT, str(exc)
            )
294     except UnsupportedMediaType as exc:
295         self._problem(
296             HTTPStatus.UNSUPPORTED_MEDIA_TYPE,
297             f"unsupported_media_type:{exc}",
298         )
299     except Exception: # never leak internals over
        the wire
300         logger.exception("unhandled_error_
            processing_%s_%s", method, path)
301         self._problem(HTTPStatus.
            INTERNAL_SERVER_ERROR, "internal_error")
302
303     def _reject(self, allowed: str) -> None:
304         body = json.dumps(
305             {
306                 "title": "Method_Not_Allowed",
307                 "status": int(HTTPStatus.
                    METHOD_NOT_ALLOWED),
308                 "detail": "method_not_allowed_on_this_
                    resource",
309             }
310         ).encode()
311         self._send(
312             HTTPStatus.METHOD_NOT_ALLOWED,
313             body,
314             "application/problem+json",
315             extra={"Allow": allowed},
316         )
317
318     # -- /parts (collection)
    -----
319
320     def _collection(self, method: str, query: dict[str,
        list[str]]) -> None:
321         if method == "GET":

```



```

322         category = query.get("category", [None])[0]
323         reps = [p.to_representation() for p in self
324                 .store.list(category)]
325         media = negotiate(self.headers.get("Accept"
326                                           ))
327         if media is None:
328             self._problem(
329                 HTTPStatus.NOT_ACCEPTABLE, "
330                 supported: application/json,
331                 text/csv"
332             )
333         return
334     body = (
335         json.dumps({"parts": reps, "count": len
336                    (reps)}).encode()
337         if media == JSON
338         else to_csv(reps)
339     )
340     self._send(HTTPStatus.OK, body, media)
341 elif method == "POST":
342     self._check_json_content_type()
343     part = parse_part(self._read_body())
344     self.store.create(part)
345     body = json.dumps(part.to_representation())
346     .encode()
347     self._send(
348         HTTPStatus.CREATED,
349         body,
350         extra={
351             "Location": f"/parts/{part.sku}",
352             "ETag": etag_for(part),
353         },
354     )
355 else:
356     self._reject("GET, HEAD, POST, OPTIONS")
357
358 # -- /parts/{sku} (document)
359 -----
360
361 def _document(self, method: str, sku: str) -> None:
362     part = self.store.get(sku)
363     if method == "GET": # HEAD is dispatched here
364         with body suppressed
365         if part is None:
366             self._problem(HTTPStatus.NOT_FOUND, f"
367                             no part with sku {sku!r}")

```

```

359         return
360     etag = etag_for(part)
361     if self.headers.get("If-None-Match") ==
362         etag:
363         self._send(HTTPStatus.NOT_MODIFIED,
364             extra={"ETag": etag})
365         return
366     media = negotiate(self.headers.get("Accept"
367 ))
368     if media is None:
369         self._problem(HTTPStatus.NOT_ACCEPTABLE
370             , "supported: json, csv")
371         return
372     rep = part.to_representation()
373     body = (
374         json.dumps(rep).encode() if media ==
375         JSON else to_csv([rep])
376     )
377     self._send(HTTPStatus.OK, body, media,
378         extra={"ETag": etag})
379 elif method == "PUT":
380     self._check_json_content_type()
381     if part and self.headers.get("If-Match")
382     not in (None, etag_for(part)):
383         self._problem(
384             HTTPStatus.PRECONDITION_FAILED,
385             "If-Match does not match current
386             ETag; re-read and retry",
387         )
388         return
389     new_part = parse_part(self._read_body(),
390         sku_from_path=sku)
391     existed = self.store.replace(sku, new_part)
392     etag = etag_for(self.store.get(sku)) #
393     type: ignore[arg-type]
394     status = HTTPStatus.OK if existed else
395     HTTPStatus.CREATED
396     extra = {"ETag": etag}
397     if not existed:
398         extra["Location"] = f"/parts/{sku}"
399     self._send(status, b"", extra=extra)
400 elif method == "PATCH":
401     if part is None:
402         self._problem(HTTPStatus.NOT_FOUND, f"
403             no part with sku {sku!r}")
404         return

```

```

393         ctype = (self.headers.get("Content-Type")
394                  or "").split(";")[0]
395         if ctype.strip() != "application/merge-
396             patch+json":
397             raise UnsupportedMediaType(ctype or "<
398                 missing>")
399         patch = json.loads(self._read_body().decode
400             ("utf-8") or "{}")
401         merged = merge_patch(part.to_representation
402             (), patch)
403         updated = parse_part(
404             json.dumps(merged).encode(),
405             sku_from_path=sku
406         )
407         self.store.replace(sku, updated) # bumps
408             the version counter
409         stored = self.store.get(sku)
410         assert stored is not None # just written
411             by replace()
412         body = json.dumps(stored.to_representation
413            ()).encode()
414         self._send(HTTPStatus.OK, body, extra={"
415             ETag": etag_for(stored)})
416     elif method == "DELETE":
417         if self.store.delete(sku):
418             self._send(HTTPStatus.NO_CONTENT)
419         else:
420             self._problem(HTTPStatus.NOT_FOUND, f"
421                 no part with sku {sku!r}")
422     else:
423         self._reject("GET, HEAD, PUT, PATCH, DELETE
424             , OPTIONS")
425
426 class UnsupportedMediaType(Exception):
427     """Raised when Content-Type is not one the resource
428         understands (415)."""
429
430 def merge_patch(target: dict[str, Any], patch: dict[str
431     , Any]) -> dict[str, Any]:
432     """RFC 7386 JSON Merge Patch: merge keys; null
433         deletes the key."""
434     result = dict(target)
435     for key, value in patch.items():
436         if value is None:

```

```
424         result.pop(key, None)
425     elif isinstance(value, dict) and isinstance(
426         result.get(key), dict):
427         result[key] = merge_patch(result[key],
428             value)
429     else:
430         result[key] = value
431     return result
432
433 def make_server(port: int) -> ThreadingHTTPServer:
434     CatalogHandler.store = CatalogStore()
435     return ThreadingHTTPServer(("127.0.0.1", port),
436         CatalogHandler)
437
438 def run_selftest() -> None:
439     """Deterministic in-process checks; no sockets, no
440         network."""
441     from http.client import HTTPConnection
442     import threading
443
444     server = make_server(0)
445     thread = threading.Thread(target=server.
446         serve_forever, daemon=True)
447     thread.start()
448     host, port = server.server_address
449     conn = HTTPConnection(host, port, timeout=5)
450     try:
451         conn.request("GET", "/parts/NX-1001")
452         resp = conn.getresponse()
453         assert resp.status == 200 and resp.getheader("
454             ETag")
455         etag = resp.getheader("ETag")
456         resp.read()
457
458         conn.request("GET", "/parts/NX-1001", headers={
459             "If-None-Match": etag})
460         assert conn.getresponse().status == 304
461
462         conn.request("GET", "/parts/NOPE")
463         assert conn.getresponse().status == 404
464
465         conn.request(
466             "POST",
467             "/parts",
```

```

463         body=json.dumps({
464             "sku": "NX-1005", "name": "Torque_
            Wrench_Arm",
465             "category": "actuators",
466             "unitPrice": {"currency": "USD", "
            amountCents": 9900},
467             "stock": 12,
468         }),
469         headers={"Content-Type": "application/json"
            },
470     )
471     resp = conn.getresponse()
472     assert resp.status == 201
473     assert resp.getheader("Location") == "/parts/NX
        -1005"
474     resp.read()
475
476     conn.request("DELETE", "/parts/NX-1005")
477     assert conn.getresponse().status == 204
478     conn.request("DELETE", "/parts/NX-1005")
479     assert conn.getresponse().status == 404
480
481     conn.request(
482         "GET", "/parts", headers={"Accept": "
            application/xml"}
483     )
484     assert conn.getresponse().status == 406
485     print("selftest: all checks passed_
        (200/304/404/201/204/404/406)")
486 finally:
487     conn.close()
488     server.shutdown()
489     server.server_close()
490
491 if __name__ == "__main__":
492     parser = argparse.ArgumentParser(description=
        __doc__)
493     parser.add_argument("--port", type=int, default
        =8080)
494     parser.add_argument("--selftest", action="
        store_true")
495     args = parser.parse_args()
496     logging.basicConfig(level=logging.INFO, format="%(
        levelname)s_%(message)s")
497     if args.selftest:

```

```

499         run_selftest()
500     else:
501         logger.info("serving Northwind parts catalog on
                    127.0.0.1:%d", args.port)
502         make_server(args.port).serve_forever()

```

Listing 3.1 — catalog_api.py: constraint-compliant REST service on http.server.

Note what the framework-free implementation makes unavoidable: the ETag is derived from representation bytes plus a version counter; If-Match on PUT implements optimistic concurrency for free; 405 responses carry Allow; 415 guards the content-type boundary; and the Cache-Control: no-cache plus ETag combination means caches must revalidate but may reuse bodies — correctness without giving up the network savings [Fie22a]. The one deliberate simplification is storage: the in-memory store stands in for a repository layer that Chapter 6 replaces with a real persistence stack behind the identical interface.

4.3.2 A URI Design Validator

Style guides die in wikis unless they are executable. Listing 4.3.2 implements the URI grammar rules of Section 4.2.4 as a linter over a route table — a JSON (stdlib) or YAML (requires pyyaml, Section 4.10) document shaped like a simplified OpenAPI `paths` object. It exits non-zero when it finds violations, so it can gate CI exactly like a code linter.

```

1  """uri_lint.py — enforce the URI design grammar on a
   route_table.
2
3  Usage (PowerShell):
4  python uri_lint.py#####_#_lint_the_
   embedded_sample_table
5  python uri_lint.py routes.json#####_#_JSON_route_
   table_(stdlib_only)
6  python uri_lint.py routes.yaml#####_#_YAML_route_
   table_(needs_pyyaml)
7
8  Exit code 0 = clean, 1 = violations found, 2 = input
   could not be parsed.
9  Deterministic: findings are sorted; no clock, no
   network, no randomness.
10 """
11 from __future__ import annotations
12
13 import json
14 import logging

```

```

15 import re
16 import sys
17 from dataclasses import dataclass
18 from pathlib import Path
19 from typing import Any
20
21 logger = logging.getLogger("northwind.uri_lint")
22
23 MAX_NESTING_DEPTH = 2 # collection/document pairs, per
24 style guide
25
26 # Verb lexicon: segments that signal RPC-style
27 tunneling (rule R001).
28 VERB_LEXICON = {
29     "get", "fetch", "read", "list", "query", "search",
30     "find",
31     "create", "add", "insert", "register", "make", "new",
32     "update", "edit", "modify", "change", "set", "patch",
33     "delete", "remove", "destroy", "purge",
34     "do", "process", "execute", "run", "submit", "trigger",
35     "calculate", "compute", "export", "import", "sync",
36 }
37
38 KEBAB_RE = re.compile(r"^[a-z0-9]+(-[a-z0-9]+)*$")
39 PARAM_RE = re.compile(r"^\{[a-zA-Z][a-zA-Z0-9_]*\}$")
40
41 @dataclass(frozen=True)
42 class Finding:
43     rule: str
44     path: str
45     message: str
46
47     def render(self) -> str:
48         return f"[{self.rule}] {self.path}: {self.message}"
49
50 def static_segments(path: str) -> list[str]:
51     """Path segments minus template parameters like {sku}."""
52     return [s for s in path.split("/") if s and not
53             PARAM_RE.match(s)]

```

```

52
53
54 def lint_path(path: str) -> list[Finding]:
55     findings: list[Finding] = []
56
57     if path != "/" and path.endswith("/"):
58         # R005
59         findings.append(Finding(
60             "R005", path,
61             "trailing_slash_creates_a_duplicate_URI;_
62             use_the_bare_form_"
63             "and_301-redirect_the_slashed_form",
64         ))
65
66     segments = [s for s in path.split("/") if s]
67     for seg in segments:
68         if PARAM_RE.match(seg):
69             continue
70         if seg.lower() in VERB_LEXICON:
71             # R001
72             findings.append(Finding(
73                 "R001", path,
74                 f"verb_{seg!r}_in_path;_the_HTTP_method_
75                 _supplies_the_verb_-_"
76                 f"reify_the_action_as_a_resource_
77                 instead",
78             ))
79         if not KEBAB_RE.match(seg):
80             # R004
81             findings.append(Finding(
82                 "R004", path,
83                 f"segment_{seg!r}_is_not_lowercase_
84                 kebab-case_"
85                 f"(no_camelCase,_snake_case,_or_mixed_
86                 case)",
87             ))
88
89     depth = (len(segments) + 1) // 2
90     # R002
91
92     if depth > MAX_NESTING_DEPTH:
93         findings.append(Finding(
94             "R002", path,
95             f"nesting_depth_{depth}_exceeds_{
96                 MAX_NESTING_DEPTH};_"
97             f"prefer_a_top-level_collection_with_a_
98             query_filter",

```



```

87         ))
88     return findings
89
90
91 def lint_pluralization(paths: list[str]) -> list[
92     Finding]:
93     """R003: collection segments must be consistently plural."""
94     singular: list[str] = []
95     plural: list[str] = []
96     for path in paths:
97         segs = static_segments(path)
98         if not segs:
99             continue
100         first = segs[0]
101         (plural if first.endswith("s") else singular).
102         append(first)
103     if singular and plural:
104         return [Finding(
105             "R003", ", ".join(sorted(set(singular))),
106             "singular collection segment(s) mixed with plural",
107             f"{sorted(set(plural))}; pick plural everywhere",
108         )]
109     return []
110
111 def lint_route_table(table: dict[str, Any]) -> list[
112     Finding]:
113     paths = table.get("paths")
114     if not isinstance(paths, dict) or not paths:
115         raise ValueError("route table must contain a non-empty 'paths' object")
116     findings: list[Finding] = []
117     for path in paths:
118         if not isinstance(path, str) or not path.startswith("/"):
119             raise ValueError(f"invalid path key: {path!r}")
120         findings.extend(lint_path(path))
121     findings.extend(lint_pluralization(list(paths)))
122     return sorted(findings, key=lambda f: (f.path, f.rule))

```

```

123 def load_table(path: Path) -> dict[str, Any]:
124     text = path.read_text(encoding="utf-8")
125     if path.suffix in (".yaml", ".yml"):
126         try:
127             import yaml # type: ignore[import-untyped]
128         except ImportError as exc:
129             raise RuntimeError(
130                 "YAML input requires pyyaml: pip
131                 install pyyaml"
132             ) from exc
133         data = yaml.safe_load(text)
134     else:
135         data = json.loads(text)
136     if not isinstance(data, dict):
137         raise ValueError("route table must be a JSON/
138         YAML object")
139     return data
140
141 SAMPLE_TABLE: dict[str, Any] = {
142     "paths": {
143         "/parts": {},
144         "/parts/{sku}": {},
145         "/parts/{sku}/restock": {}, # action-like;
146         reify as stock-adjustments
147         "/getPart": {}, # R003 + R004 (
148         getPart: camelCase, singular)
149         "/warehouses/{wid}/stock/{sku}/history": {}, #
150         R002: depth 3
151         "/purchaseOrders/{id}": {}, # R004:
152         camelCase
153         "/suppliers/": {}, # R005:
154         trailing slash
155     }
156 }
157
158 def main(argv: list[str]) -> int:
159     logging.basicConfig(level=logging.WARNING, format="
160         %(levelname)s %(message)s")
161     if len(argv) > 1:
162         try:
163             table = load_table(Path(argv[1]))
164         except (OSError, ValueError, RuntimeError,
165             json.JSONDecodeError) as exc:
166             logger.error("cannot load route table: %s",

```

```

161         exc)
162         return 2
163     else:
164         table = SAMPLE_TABLE
165
166         findings = lint_route_table(table)
167         for finding in findings:
168             print(finding.render())
169             print(f"\n{len(findings)}_violation(s)_in_{len(
170                 table['paths'])}_path(s)")
171         return 1 if findings else 0
172
173 if __name__ == "__main__":
174     sys.exit(main(sys.argv))

```

Listing 3.2 — uri_lint.py: executable URI style guide.

Run against its embedded sample, the linter reports five findings across seven paths (R002–R005 all fire) — style defects caught mechanically, before a single line of service code exists. The lexicon deliberately contains only generic verbs; domain-specific action words like `restock` are caught by human review guided by the reification rule, not by an ever-growing word list. This is design governance as code: the same rule IDs can be referenced in review comments and waivers.

4.3.3 A HAL-Style Hypermedia Serializer

Listing 4.3.3 closes the loop on Section 4.2.6: it renders the catalog domain as HAL-like documents with `_links` and `_embedded`, keeping serialization separate from both storage and transport.

```

1  """hal_serializer.py - render catalog resources as HAL-
2  like documents.
3
4  Stdlib-only and deterministic (sorted keys, fixed
5  sample data). The point:
6  links are DATA emitted by the server, not strings
7  concatenated by clients.
8
9  """
10 from __future__ import annotations
11
12 import json
13 from dataclasses import dataclass, field
14 from typing import Any

```

```

12 BASE = "https://api.northwind-robotics.example" #
    fictional host, never called
13
14
15 @dataclass(frozen=True)
16 class Link:
17     href: str
18     method: str = "GET"
19     title: str | None = None
20     templated: bool = False
21
22     def render(self) -> dict[str, Any]:
23         out: dict[str, Any] = {"href": self.href}
24         if self.method != "GET":
25             out["method"] = self.method
26         if self.title:
27             out["title"] = self.title
28         if self.templated:
29             out["templated"] = True
30         return out
31
32
33 @dataclass
34 class HalResource:
35     state: dict[str, Any]
36     links: dict[str, Link | list[Link]] = field(
37         default_factory=dict)
38     embedded: dict[str, list["HalResource"]] = field(
39         default_factory=dict)
40
41     def to_dict(self) -> dict[str, Any]:
42         doc: dict[str, Any] = dict(self.state)
43         doc["_links"] = {
44             rel: ([lnk.render() for lnk in link]
45                  if isinstance(link, list) else link.
46                  render())
47             for rel, link in sorted(self.links.items())
48         }
49         if self.embedded:
50             doc["_embedded"] = {
51                 rel: [res.to_dict() for res in
52                      resources]
53                 for rel, resources in sorted(self.
54                     embedded.items())
55             }
56         return doc

```

```

52
53
54 def part_resource(part: dict[str, Any]) -> HalResource:
55     sku = part["sku"]
56     links: dict[str, Link | list[Link]] = {
57         "self": Link(f"{BASE}/parts/{sku}"),
58         "collection": Link(f"{BASE}/parts", title="All
59             parts"),
60         "category": Link(f"{BASE}/categories/{part['
61             category']}"),
62     }
63     # Action affordances are state-dependent: only in-
64     # stock, active parts
65     # advertise the restock transition. Clients need no
66     # business rule for this.
67     if part["stock"] > 0 and part.get("status", "active
68         ") == "active":
69         links["restock"] = Link(
70             f"{BASE}/parts/{sku}/stock-adjustments",
71             method="POST",
72             title="Record a stock adjustment",
73         )
74     return HalResource(state={k: v for k, v in part.
75         items() if k != "status"},
76         links=links)
77
78 def parts_collection(parts: list[dict[str, Any]], *,
79     cursor: str | None,
80     next_cursor: str | None) ->
81     HalResource:
82     collection = HalResource(
83         state={"count": len(parts)},
84         links={"self": Link(f"{BASE}/parts" + (f"?
85             cursor={cursor}" if cursor
86                 else ""))
87         },
88         embedded={"item": [part_resource(p) for p in
89             parts]},
90     )
91     # Pagination links are the minimum viable
92     # hypermedia: the server owns the
93     # cursor format, so it can change storage without
94     # breaking clients.
95     if next_cursor is not None:
96         collection.links["next"] = Link(f"{BASE}/parts?

```

```

85         cursor={next_cursor}")
86     return collection
87
88 CATALOG_SAMPLE: list[dict[str, Any]] = [
89     {"sku": "NX-1001", "name": "Servo_Motor_12V", "
90      category": "actuators",
91      "unitPrice": {"currency": "USD", "amountCents":
92       2599}, "stock": 140},
93     {"sku": "NX-1002", "name": "LIDAR_Module_R2", "
94      category": "sensors",
95      "unitPrice": {"currency": "USD", "amountCents":
96       18999}, "stock": 32},
97     {"sku": "NX-1003", "name": "Aluminum_Chassis_Plate"
98      , "category": "frames",
99      "unitPrice": {"currency": "USD", "amountCents":
100       7450}, "stock": 0,
101      "status": "discontinued"},
102 ]
103
104 def main() -> None:
105     document = part_resource(CATALOG_SAMPLE[0]).to_dict
106     ()
107     listing = parts_collection(
108         CATALOG_SAMPLE, cursor=None, next_cursor="
109         c3RlcDoy"
110     ).to_dict()
111     print("---_document_---")
112     print(json.dumps(document, indent=2, sort_keys=True
113     ))
114     print("---_collection_---")
115     print(json.dumps(listing, indent=2, sort_keys=True
116     ))
117
118 if __name__ == "__main__":
119     main()

```

Listing 3.3 — hal_serializer.py: HAL-style serializer with state-dependent action affordances.

Two details reward attention. First, the `restock` affordance is *computed from resource state*: a discontinued part simply stops advertising the transition, and a hypermedia-driven client disables the feature without a code change. Second, the pagination `next` link is an opaque cursor the server owns — the client stores and dereferences it but never constructs it, which

is URI opacity (Section 4.2.4) doing real work.

4.3.4 Design-First Contract: OpenAPI 3.1 Excerpt

Section 4.2.8 argued for contract-first development. Listing 4.3.4 is the worked excerpt for the catalog; the prose commentary follows in the block's design notes.

```
1 openapi: 3.1.0
2 info:
3   title: Northwind Robotics Parts Catalog API
4   version: 1.0.0
5   summary: Resource-oriented catalog of robot parts (
6     synthetic training data).
7   license: { name: Proprietary }
8 jsonSchemaDialect: https://json-schema.org/draft
9   /2020-12/schema
10 servers:
11   - url: http://localhost:8080
12     description: Offline development server (Listing
13       3.1)
14 tags:
15   - name: parts
16     description: Catalog part documents and the parts
17       collection.
18 paths:
19   /parts:
20     get:
21       summary: List parts
22       operationId: listParts
23       tags: [parts]
24       parameters:
25         - name: category
26           in: query
27           description: Filter by category. Query
28             modulates; path identifies.
29       schema:
30         type: string
31         enum: [actuators, sensors, frames, mobility
32           ]
33       responses:
34         "200":
35           description: A page of part representations.
36           headers:
37             Cache-Control:
```

```
33         schema: { type: string }
34     content:
35         application/json:
36             schema: { $ref: "#/components/schemas/
37                 PartList" }
38         text/csv:
39             schema: { type: string }
40     "406": { $ref: "#/components/responses/
41         NotAcceptable" }
42 post:
43     summary: Create a part (server assigns nothing;
44         client supplies SKU)
45     operationId: createPart
46     tags: [parts]
47     requestBody:
48         required: true
49         content:
50             application/json:
51                 schema: { $ref: "#/components/schemas/
52                     PartInput" }
53     responses:
54         "201":
55             description: Created; Location header
56                 addresses the new document.
57             headers:
58                 Location: { schema: { type: string } }
59                 ETag: { schema: { type: string } }
60         "400": { $ref: "#/components/responses/
61             BadRequest" }
62         "409": { $ref: "#/components/responses/Conflict
63             " }
64         "415": { $ref: "#/components/responses/
65             UnsupportedMediaType" }
66 /parts/{sku}:
67     parameters:
68         - name: sku
69           in: path
70           required: true
71           schema: { type: string, pattern: "^[A-Z
72               ]{2}-[0-9]{4}$" }
73     get:
74         summary: Retrieve a part; supports conditional
75             requests
76         operationId: getPart
77         tags: [parts]
```



```

69     parameters:
70       - name: If-None-Match
71         in: header
72         schema: { type: string }
73     responses:
74       "200":
75         description: The part representation, with
76           validators.
77         headers:
78           ETag: { schema: { type: string } }
79         content:
80           application/json:
81             schema: { $ref: "#/components/schemas/
82               Part" }
83       "304": { description: Not modified; re-use the
84         cached copy. }
85       "404": { $ref: "#/components/responses/NotFound
86         " }
87   put:
88     summary: Full replacement (idempotent); If-Match
89       guards lost updates
90     operationId: replacePart
91     tags: [parts]
92     parameters:
93       - name: If-Match
94         in: header
95         schema: { type: string }
96     requestBody:
97       required: true
98       content:
99         application/json:
100           schema: { $ref: "#/components/schemas/
101             PartInput" }
102     responses:
103       "200": { description: Replaced. }
104       "201": { description: Created at a client-known
105         URI (store semantics). }
106       "412": { description: ETag precondition failed;
107         re-read and retry. }
108   patch:
109     summary: Partial update via JSON Merge Patch (RFC
110       7386)
111     operationId: mergePatchPart
112     tags: [parts]
113     requestBody:
114       required: true

```

```
106         content:
107             application/merge-patch+json:
108                 schema: { type: object }
109         responses:
110             "200": { description: Updated representation
111                     returned. }
111             "415": { $ref: "#/components/responses/
112                     UnsupportedMediaType" }
112     delete:
113         summary: Remove a part (idempotent effect)
114         operationId: deletePart
115         tags: [parts]
116         responses:
117             "204": { description: Removed; no body. }
118             "404": { $ref: "#/components/responses/NotFound
119                     " }
120 components:
121     schemas:
122         Money:
123             type: object
124             required: [currency, amountCents]
125             properties:
126                 currency: { type: string, minLength: 3,
127                             maxLength: 3 }
127                 amountCents: { type: integer, minimum: 0 }
128             additionalProperties: false
129         PartInput:
130             type: object
131             required: [sku, name, category, unitPrice, stock]
132             properties:
133                 sku: { type: string, pattern: "^[A-Z
134                       ]{2}-[0-9]{4}$" }
135                 name: { type: string, minLength: 1 }
136                 category: { type: string, enum: [actuators,
137                                                   sensors, frames, mobility] }
138                 unitPrice: { $ref: "#/components/schemas/Money"
139                               }
140                 stock: { type: integer, minimum: 0 }
141             additionalProperties: false
142         Part:
143             allOf:
144                 - $ref: "#/components/schemas/PartInput"
145             description: Server representation; identical
146                         shape here by choice,
147                         but the contract (not the database) is the
```

```

144         source of truth.
145     PartList:
146         type: object
147         required: [parts, count]
148         properties:
149             parts:
150                 type: array
151                 items: { $ref: "#/components/schemas/Part" }
152                 count: { type: integer }
153     responses:
154         BadRequest:
155             description: Malformed or invalid representation.
156             content:
157                 application/problem+json:
158                     schema: { $ref: "#/components/schemas/Problem" }
159         NotFound: { description: Unknown resource. }
160         Conflict: { description: State conflict (duplicate
161                     SKU, stale edit). }
162         NotAcceptable: { description: No supported
163                         representation matches Accept. }
164         UnsupportedMediaType: { description: Content-Type
165                                 not supported. }
166     # Problem schema elided for brevity: title/status/
167     detail per problem+json.

```

Listing 3.4 — openapi.yaml (excerpt): design-first contract for the catalog.

Design commentary: every response declares its headers (ETag, Location) because headers are contract, not decoration; error responses are reusable components so the 400s across resources can never drift apart; `additionalProperties: false` makes the contract closed by default, forcing field additions through deliberate review (the evolution discipline of Section 4.2.7); and conditional-request headers are first-class parameters, so generated SDKs expose optimistic concurrency to every consumer. Chapter 4 develops serialization contracts; Chapter 9 extends this document with a versioning strategy.

4.4 Common Pitfalls & Anti-Patterns

Each anti-pattern below is stated with its symptom, the constraint it breaks, and the corrective. All are detected by Listing 4.3.2 where they appear in URIs.

1. **Verbs in URIs** (`/getPart`, `/createOrder`). *Breaks*: uniform interface — the URI vocabulary duplicates the method vocabulary, so

nothing is safe, idempotent, or cacheable by inspection. *Fix*: noun for the thing, method for the operation; reify true actions as resources (`/stock-adjustments`).

2. **Tunneling everything through POST.** *Breaks*: safety, idempotency, cacheability, visibility — intermediaries must assume the worst about every request. *Fix*: Level 2 method discipline; `GET` for reads so caches and conditional requests can work.
3. **Returning 200 for errors** (error objects in a 200 body). *Breaks*: self-descriptive messages — monitors, retry loops, and caches all read status codes; a 200-wrapped failure may be *cached*. *Fix*: the Table 4.3 lifecycle, with `application/problem+json` bodies for detail.
4. **Leaking the database schema** (ORM objects serialized directly). *Breaks*: independent evolvability — schema migrations become breaking API changes. *Fix*: an anti-corruption layer between storage and representation (Section 4.2.7).
5. **Inconsistent naming** (mixed singular/plural, camelCase beside kebab-case, `/order` beside `/warehouses`). *Breaks*: the uniform interface’s simplicity property — every consumer now carries a lookup table of your exceptions. *Fix*: one written grammar; lint it in CI (Listing 4.3.2).
6. **Ignoring caching semantics** (no `Cache-Control`, no validators). *Breaks*: cacheability — intermediaries guess, and guessed caching is either absent (wasted scale) or wrong (stale data sold twice). *Fix*: validators on every `GET` representation; explicit freshness policy, even if the policy is `no-cache` [Fie22a].
7. **Deep nesting beyond two levels** (`/warehouses/WH-01/zones/Z-3/bins/B-17/parts/M`). *Breaks*: evolvability — the path ossifies one navigational route through a many-pathed graph; authorization and cache keys multiply. *Fix*: top-level collections with query filters; containment paths only where lifetime truly depends on the parent.
8. **Idempotency theater** (`PUT` that increments a counter). *Breaks*: `PUT`’s contract; safe retry becomes corruption. *Fix*: if the operation is intrinsically non-idempotent (increment, append), use `POST` to a subordinate collection.
9. **Versioned filenames-as-APIs** (`/api/v2/final/REAL/`). *Breaks*: URI opacity and cool-URI discipline; version policy belongs in the contract, not in path folklore (Chapter 9).

10. **Silent query-parameter divergence** (`?format=json` beside `Accept`).
Breaks: self-description and cache correctness (the URI key now varies by parameter a cache may ignore). *Fix:* negotiate via headers with `Vary`, or make the variant a distinct URI deliberately — never both.

Warning

The most expensive anti-pattern is not on this list because it is meta: *labeling a Level 0/1 service “REST”* in documentation. Consumers then assume safety, idempotent retries, and cacheability that the service does not provide, and build those assumptions into production traffic. Name your architecture honestly; an “HTTP RPC API” is a respectable, supportable thing.

4.5 Hands-On Production Lab & Real-World Case Study

4.5.1 Lab Part A: Design the Warehouse Domain as REST Resources

Work in the Northwind Robotics universe (all data synthetic). Target: 60–90 minutes, paper or a scratch OpenAPI file; no implementation required.

1. **Inventory the domain.** Northwind’s warehouse system tracks: warehouses; the parts catalog (Listing 4.3.1); stock levels (how many of a part sit in a warehouse); suppliers; purchase orders sent to suppliers; and inbound shipments against those orders.
2. **Classify every concept** into the four archetypes (Section 4.2.4): collections, documents, stores, controllers. Expect zero controllers; if you invented one, try to reify it.
3. **Draw the resource graph.** Decide which relations are *containment* (path hierarchy) versus *association* (query filter or link). Compare with Figure 4.3. Specifically: is `/warehouses/{wid}/stock/{sku}` justified? (Yes — a stock level’s identity is the pair (warehouse, part); it cannot exist without both.) Is `/suppliers/{sid}/purchase-orders/{poid}` justified? (No — orders outlive any single navigational path; make `/purchase-orders?supplier=SID` instead.)
4. **Write the URI table** for at least eight routes and lint it with `python uri_lint.py yourtable.json` until clean.

5. **Assign the uniform interface.** For each route, tabulate allowed methods, success codes, the three most likely failure codes, and the caching policy (validator, freshness) for each GET.
6. **Design one hypermedia transition.** Purchase orders move **draft** → **submitted** → **received** → **closed**. Express the legal transitions as state-dependent links on the order representation, as Listing 4.3.3 does for **restock**.

Exit criteria: a lint-clean URI table, a completed method/status/cache matrix, and a defensible two-sentence justification for every nested route.

4.5.2 Lab Part B: Constraint Critique of a Deliberately Bad API

The following route table ships from a fictional vendor, “RoboDirect” (any resemblance to production APIs you have suffered is coincidental). Audit it against every constraint and rule in this chapter before reading the key.

1	POST	/api/getPartDetails	-> 200 always; errors in body
2	POST	/api/getPartDetails?id=NX-1001	-> same endpoint, query id
3	POST	/api/updateStockLevel	-> body: {partId, warehouseId, qty}
4	GET	/api/Parts/ListAll	-> 200, no ETag, no Cache-Control
5	POST	/api/warehouses/WH-01/zones/Z3/bins/B17/ getContents	
6	DELETE	/api/purchaseOrder/delete/P0-2026-0417	-> 200 {" ok":true}
7	GET	/api/v2_final/parts/NX-1001.json	

Answer key (abridged). Every route tunnels through POST or embeds a verb (R001), so nothing is safe, idempotent, or cacheable; errors hidden in 200 bodies defeat monitors and caches; **ListAll** returns an uncacheable, unbounded collection with no validators; the five-segment warehouse path (R002) ossifies one navigation path; **delete** in a DELETE path doubles the verb; **v2_final** and the **.json** suffix hard-code version and format into the URI, defeating content negotiation and cool-URI discipline; and the mixed **Parts/parts/purchaseOrder** naming fails R003 three ways. Estimated Richardson level: 0, trending 1. Rewrite left as Self-Assessment Exercise 7.1.

4.5.3 Case Study: The Redesign That Broke the Mobile Fleet

Context. Northwind Robotics’ field-technician app (a deployed fleet of rugged Android devices scanning warehouse stock) consumed `GET /api/inventory/{sku}`, which returned stock across all warehouses. A platform team, having read a book much like this one, redesigned the API “properly”: `/api/inventory/{sku}` became `/api/v2/stock?part={sku}`, launched in six weeks, and the old route was deleted the same day. Within hours, the entire device fleet returned 404s; technicians reverted to paper; two warehouses missed shipping cutoffs.

Failure analysis. The new URI design was, in isolation, better. The failure was a discipline failure, not a design failure. (1) *URIs are contracts*: clients had hard-coded paths (violating opacity, as clients inevitably do), so renaming was a breaking change regardless of how “clean” the new grammar was. (2) *No coexistence window*: mobile fleets upgrade on app-store timescales of months; the old route must live until measured traffic reaches zero, returning deprecation signals (`Deprecation` and `Sunset` headers per RFC 8594 and RFC 9745 practice). (3) *No consumer observability*: nobody could answer “who still calls the old route?” because access logs were never keyed by client version. (4) *Versioning as renaming*: `v2` was minted for a change that a new *resource* (`/stock-levels`) alongside the old one would have made non-breaking.

Recovery and doctrine. The old route was restored behind a 308 redirect to a compatibility shim; a deprecation schedule with dated sunset headers was published; per-client-version request logging was added at the gateway; and the governance rule adopted was: *additive changes ship immediately; breaking changes ship only behind measured coexistence*. Chapters 9 and 20 turn this war story into process.

4.6 Self-Assessment Exercises & Deep-Dive Questions

1. **(7.1) Rewrite RoboDirect.** Produce a lint-clean route table for the Lab Part B API, with a method/status/cache matrix for every route.
2. **(7.2)** State statelessness precisely enough to decide: is a request carrying a session cookie stateless? A bearer token? An OAuth access token validated against a token-introspection endpoint? Defend each answer from the definition.
3. **(7.3)** Northwind sells “today’s promotions” — a resource whose value changes daily. Design its URI, freshness headers, and validators. What

breaks if a shared cache serves it eight hours stale?

4. **(7.4)** Extend Listing 4.3.1 with HEAD correctness verified by the self-test, and with a POST /parts-side Idempotency-Key header. Which constraint does each addition strengthen?
5. **(7.5)** Implement an RFC 6902 JSON Patch evaluator (operations add, remove, replace, test) and wire it into the catalog’s PATCH alongside Merge Patch. Which media type selects which evaluator?
6. **(7.6) Deep dive.** Fielding calls the uniform interface trade-off “efficiency versus evolvability.” Construct one concrete Northwind scenario where the uniform interface is measurably less efficient than a bespoke RPC, and state what is purchased with that inefficiency.
7. **(7.7) Deep dive.** Argue both sides: should /warehouses/{wid}/stock/{sku} expose DELETE, and if so, what does it mean — zero stock, or no record? What do 404 and 410 each communicate here?
8. **(7.8) Deep dive.** Your organization mandates JSON:API. Which decisions in this chapter does it make for you, which does it forbid, and where does its opinion cost you flexibility the catalog used?

4.7 Interview Q&A Vault

Thirty-six questions, ordered junior → staff. Answers are calibrated to what a strong candidate says in two to six sentences.

4.7.1 Junior (Q1–Q10)

Q1. Is REST a protocol or a standard?

Answer: Neither. REST is an architectural *style* — a set of constraints defined in Fielding’s dissertation [Fie00] — that induces properties like scalability and evolvability. HTTP is a protocol; OpenAPI is a description standard; REST constrains how you use them.

Q2. State Fielding’s six constraints.

Answer: Client–server, statelessness, cacheability, uniform interface, layered system, and (optional) code-on-demand. Bonus depth: the uniform interface decomposes into resource identification, manipulation through representations, self-descriptive messages, and HATEOAS.

Q3. What properties does statelessness induce?

Answer: Visibility (any request is understandable in isolation), reliability (requests can simply be retried after failure), and scalability (no session affinity; any replica serves any request). Cost: session data travels on every request.

Q4. What is a resource?

Answer: Any named conceptual target of a hypertext reference — a document, a collection, a service, even “today’s price of NX-1001.” Clients never touch the resource, only representations of its state.

Q5. Name the four resource archetypes.

Answer: Document (singular object), collection (server-managed directory), store (client-managed entry repository), and controller (verb-like escape hatch, to be minimized by reifying actions as resources).

Q6. Why “nouns, not verbs” in URIs?

Answer: The HTTP method already supplies the verb. Verbs in URIs duplicate that vocabulary with unbounded private words, destroying the safety, idempotency, and cacheability signals that generic infrastructure depends on.

Q7. What do safe and idempotent mean? Give one example each.

Answer: Safe: read-only, no accountability for state change (GET). Idempotent: repeating the request has the same effect as one (PUT, DELETE). Note idempotency is about server *state*, not the response — a repeated DELETE returns 404 but is still idempotent.

Q8. What status code should a successful POST that creates a resource return, and with which header?

Answer: 201 Created with a `Location` header pointing at the new resource’s URI; ideally also its representation and `ETag`.

Q9. What is the difference between 400 and 422?

Answer: 400: the request is malformed — unparseable JSON, missing required fields, wrong types. 422: syntactically fine but semantically unprocessable — a well-formed request whose values violate business rules (e.g., negative stock).

Q10. What does a 405 response require?

Answer: An `Allow` header listing the methods the resource does support, so clients can self-correct without documentation archaeology.

4.7.2 Mid-Level (Q11–Q20)

Q11. Explain the Richardson Maturity Model.

Answer: Four levels diagnosing which web technologies an HTTP API exploits [RAR13]: L0 tunnels RPC through one URI and POST; L1 introduces resource URIs; L2 uses methods and status codes with correct semantics; L3 adds hypermedia controls (HATEOAS). It is diagnostic, not a moral ladder.

Q12. PUT versus PATCH?

Answer: PUT is full replacement — the body becomes the resource’s new state; omitted fields are reset; idempotent by construction. PATCH is partial modification whose semantics come from the patch media type — JSON Merge Patch (RFC 7386: merge, null deletes) or JSON Patch (RFC 6902: ordered atomic operations with a `test` precondition).

Q13. When is a repeated DELETE not an error?

Answer: Always, by design. The first DELETE returns 204; the second returns 404. Both leave the server in the same state, which is exactly why DELETE is idempotent and safe for retry loops — the client should treat 404-after-DELETE as confirmation, not failure.

Q14. How do ETags and conditional requests work?

Answer: The server attaches a validator (ETag) to a representation. A client revalidating sends `If-None-Match`; a match yields 304 with no body (cache efficiency). A client writing sends `If-Match`; a mismatch yields 412 (optimistic concurrency — no lost updates) [Fie22a].

Q15. Design a URI scheme for: parts, stock per warehouse, purchase orders to suppliers.

Answer: `/parts`, `/parts/{sku}`; `/warehouses/{wid}/stock/{sku}` (containment: stock’s identity is the pair); `/purchase-orders` top-level with `?supplier=` and `?warehouse=` filters because orders outlive any single navigation path. Depth ≤ 2 , kebab-case, plural collections.

Q16. Query string or path segment — how do you decide?

Answer: Path identifies *which* resource; query modulates *how* it is returned: filtering, sorting, pagination, projection. If deleting the parameter addresses a different thing, it belongs in the path.

Q17. Singular or plural collection names?

Answer: The debate is noise; consistency is the law. Plural wins on three arguments: collection and member URIs share a stem (`/parts`, `/parts/NX-1001`); it reads correctly for both many and one-of-many; and singletons stay expressible as documents. Mixing is the only indefensible answer.

Q18. Are `/parts` and `/parts/` the same resource?

Answer: No — URIs are compared character-wise, so they are distinct identifiers with distinct cache keys. Publish the bare form and answer the slashed form with a cacheable 301 redirect; never alias silently.

Q19. What is URI opacity and what does it buy?

Answer: Clients treat URIs as opaque: store, compare, dereference — never parse or construct. It buys the server freedom to restructure (gateways, sharding, renaming) without breaking consumers, and it is the precondition for hypermedia-driven evolution.

Q20. When would you return 410 instead of 404?

Answer: 404 says “not here, maybe never was, maybe elsewhere”; 410 says “was here, deliberately retired, do not ask again.” Use 410 for sunset resources so crawlers, caches, and clients can permanently drop references.

4.7.3 Senior (Q21–Q28)

Q21. What does the layered-system constraint permit, and what does it cost?

Answer: It permits intermediaries — gateways, shared caches, CDNs — that components cannot see past, enabling load balancing, edge security, and legacy encapsulation. It costs per-hop latency, which the cache constraint is expected to amortize [Fie00].

Q22. Why is code-on-demand optional, and where do you actually see it?

Answer: It reduces visibility — intermediaries cannot reason about downloaded behavior — so Fielding brackets it as the one optional constraint. Its canonical realization is JavaScript to browsers; machine-to-machine APIs almost always omit it.

Q23. When does hypermedia pay off, and when is it over-engineering?

Answer: It pays when consumers outnumber your coordination bandwidth: long-lived public APIs, server-driven workflows whose legal transitions vary per state, generic clients. It is over-engineering

for first-party clients shipping in lockstep, where static contracts (OpenAPI) plus selective links (pagination, state transitions) capture most of the value [RAR13].

Q24. Compare HAL, JSON:API, and Collection+JSON.

Answer: HAL is minimal — `_links`, `_embedded`, curies; reads only. JSON:API is a full protocol: resource objects, relationships, compound documents, pagination and error conventions, write semantics. Collection+JSON is the purest hypermedia: the representation carries queries and a write template, enabling fully generic clients.

Q25. How do you keep an API from becoming a serialized database?

Answer: An anti-corruption layer: representations designed for consumers (structured money, curated fields), storage designed for integrity, explicit translation between. Signals you have failed: column names in JSON, ORM objects passed to the encoder, schema migrations breaking clients [Kle17].

Q26. Design PATCH for a resource edited concurrently by many clients.

Answer: JSON Patch (RFC 6902) with `test` operations as preconditions and `If-Match` on the ETag; apply atomically or not at all; return 409/412 with a problem detail so the client re-reads and re-derives its patch. Merge Patch is the wrong tool — no atomicity, no preconditions.

Q27. A stakeholder demands `?format=csv` instead of content negotiation. Trade-offs?

Answer: It works and is browser-testable, but it forks the identifier space (two URIs, one resource), complicates cache keys unless `Vary`-equivalent discipline is applied to the query, and bypasses `q`-values. Acceptable as a documented alias; the `Accept` header with `Vary: Accept` remains the canonical mechanism [Fie22a].

Q28. Your API must expose a batch “revalue inventory” operation. RESTful design?

Answer: Reify the operation: `POST /inventory-revaluations` creates a revaluation resource with its own URI, status, and audit trail; `GET` on it reports progress. You get idempotent submission (via keys), observable lifecycle, and cancelability — none of which a `POST /revalueInventory` verb-endpoint can offer.

4.7.4 Staff (Q29–Q36)

Q29. “REST doesn’t scale to our read patterns — clients need aggregates from five resources.” Respond.

Answer: REST never forbids server-composed resources: a `/dashboards/warehouse-summary` resource is a first-class, cacheable document. What REST forbids is letting clients dictate per-request join graphs ad hoc. Compose on the server behind a stable resource, or admit you need a query paradigm (GraphQL, Chapter 12) — the honest trade, not a tunnelled query POST pretending to be REST.

Q30. How do you evolve a Level 2 API for five years without /v2?

Answer: Additive-only changes, tolerant readers, never reuse retired field names, closed-by-default schemas in review (`additionalProperties: false` as a forcing function), deprecation headers with measured coexistence, and consumer-keyed access logging so every removal decision is data-driven. Version banners become a rare, deliberate event (Chapter 9) [GH21].

Q31. Define a governance model for URI and contract consistency across forty teams.

Answer: A short written style guide (archetypes, grammar, status matrix), executable linting in CI with stable rule IDs (the Listing 4.3.2 pattern), a review gate only for cross-cutting changes, and published exception/waiver mechanics. Consistency enforced by tooling scales; consistency enforced by meetings does not [GH21].

Q32. Statelessness versus connection-oriented protocols: when do you abandon the constraint?

Answer: When the interaction *is* a session: streams, collaborative editing, subscriptions. The disciplined move is to declare the boundary — REST for command/control resources, WebSocket or gRPC streaming (Chapters 11, 13) for the conversational edge — rather than smuggling session state into “REST” endpoints.

Q33. A cache served stale stock levels and inventory was oversold. Root-cause through the constraints.

Answer: The cacheability constraint was violated by omission: responses carried no freshness metadata, so an intermediary guessed. Fix at the origin: validators plus explicit `Cache-Control` (`no-cache` revalidation for stock), and treat any response without a caching policy as a defect in review [Fie22a].

Q34. When is RPC legitimately preferable to REST? Say so without hedging.

Answer: Low-latency internal service-to-service calls with schema-first contracts and no intermediaries (gRPC's home); operations that are intrinsically computations over transient inputs (`calculateRoute`); and teams whose consumers all ship in lockstep. REST buys evolvability and interoperability with the web's infrastructure; pay for it only when you need it.

Q35. Design the migration from a Level 0 SOAP estate to resource orientation without a big bang.

Answer: Strangle at the edge: stand up a gateway translating the ten highest-traffic RPC operations into resources; publish OpenAPI for the new surface; dual-run with reconciliation dashboards; migrate consumers by cohort with sunset headers on the legacy endpoint; never rename and restructure in the same release (the Lab case study).

Q36. What would you instrument to prove REST's claimed properties are actually materializing?

Answer: Cache hit ratio and revalidation rate (cacheability); cross-replica request distribution without affinity (statelessness); share of 4xx/5xx correctly classified by status code alone (visibility/self-description); consumer breakage rate per contract change (evolvability); and p99 added latency per intermediary layer (the layered system's bill coming due).

4.8 External Bibliography & Further Reading

- **Roy T. Fielding**, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, UC Irvine, 2000 [Fie00]. Chapter 5 is the primary source for everything in Section 4.2.2; read it before any secondary source, including this book.
- **Leonard Richardson, Mike Amundsen, Sam Ruby**, *RESTful Web APIs*, O'Reilly, 2013 [RAR13]. The maturity model, Collection+JSON, and the most honest treatment of hypermedia economics in print.
- **Mark Massé**, *REST API Design Rulebook*, O'Reilly, 2011. The WRML-influenced four-archetype taxonomy (document, collection, store, controller) and URI grammar conventions used in Section 4.2.4.

- **JJ Geewax**, *API Design Patterns*, Manning, 2021 [GH21]. A pattern catalogue for resource layout, pagination, long-running operations, and versioning; the vocabulary of Section 4.2.8’s governance discussion.
- **RFC 9110**, *HTTP Semantics*, IETF, 2022 [Fie22a]. The normative source for method properties, status codes, content negotiation, and validators; cite it in design reviews, not folklore.
- **RFC 8288**, *Web Linking*, IETF, 2017 [Not17]. Typed links and relation types — the mechanical foundation of HATEOAS.
- **RFC 7386** (*JSON Merge Patch*) and **RFC 6902** (*JSON Patch*), IETF. The two patch grammars compared in Section 4.2.5; both short enough to read in one sitting.
- **OpenAPI Specification 3.1**, OpenAPI Initiative [Ope21]. The contract format for the design-first workflow; note that 3.1 aligns fully with JSON Schema 2020-12.
- **JSON:API specification** (jsonapi.org). Even if you never adopt it, its pagination, error-object, and compound-document conventions are a free design review of your own conventions.
- **Zalando RESTful API Guidelines** and **Microsoft REST API Guidelines**. Two battle-tested corporate style guides; read them to see how the debates of Section 4.2.4 get settled in practice (kebab-case, plural collections, explicit null policy).
- **Martin Kleppmann**, *Designing Data-Intensive Applications*, O’Reilly, 2017 [Kle17]. Chapter on encoding and evolution: the theoretical basis for anti-corruption representations and tolerant readers.
- **The Twelve-Factor App** [Wig17]. Process disposability and backing-services discipline: operational cousins of statelessness.

4.9 Capstone Projects

Five progressive projects. Each produces artifacts suitable for a design-review portfolio. All are offline and use synthetic Northwind Robotics data only.

4.9.1 Capstone 3.1 [junior] — Model the Library Domain

Objective. Practice pure resource modeling on an unfamiliar domain before touching code.

Inputs/Datasets. A fictional circulating library: books (with editions and copies), members, loans, holds, fines. Invent all data; at least ten books, four members, three active loans.

Steps.

1. Classify every concept into the four archetypes; reify at least two “actions” (returning a book, placing a hold) as resources.
2. Draw the resource graph; justify every containment edge in one sentence.
3. Write the full route table and lint it with Listing 4.3.2 until clean.
4. Produce the method/status/cache matrix for every route.

Acceptance Criteria.

- ☐ Zero lint findings (rules R001–R005).
- ☐ No controller archetype without a written reification failure analysis.
- ☐ Nesting depth never exceeds two.
- ☐ Every GET route has a stated validator and freshness policy.

Stretch Goals. Add sparse fieldsets and an `?embed=` policy; write the JSON Merge Patch document that renews a loan.

4.9.2 Capstone 3.2 [junior] — Run and Extend the Stdlib Catalog

Objective. Operate Listing 4.3.1 and prove its constraint compliance empirically.

Inputs/Datasets. `catalog_api.py`; `curl.exe` (ships with Windows 10+); PowerShell.

Steps.

1. Run `python catalog_api.py --selftest` and map each assertion to the constraint it demonstrates.
2. Serve on port 8080; with `curl -i`, demonstrate: 200 with ETag; 304 via If-None-Match; 201 with Location; 409 on duplicate SKU; 412

via a stale `If-Match` on `PUT`; 406 via `Accept: application/xml`;
CSV via `Accept: text/csv`.

3. Add one category, **batteries**, end-to-end (validation set, seed data, one new selftest assertion).

Acceptance Criteria.

- ☐ All seven demonstrations captured as transcripts in a `lab-notes.md`.
- ☐ Selftest passes after your extension.
- ☐ A one-paragraph memo: which constraint would break first if you added server-side login sessions, and why.

Stretch Goals. Implement `HEAD`-only verification; add `Sunset/Deprecation` headers to a deliberately retired part.

4.9.3 Capstone 3.3 [mid] — Productionize the URI Linter

Objective. Turn Listing 4.3.2 into a governance tool a real platform team could run in CI.

Inputs/Datasets. `uri_lint.py`; three route tables you author: one clean, one failing each rule, one extracted by hand from a public API's documentation (retyped, synthetic base URLs).

Steps.

1. Add a waiver mechanism (`# lint-waive: R002 reason` in the table's metadata) with expiry dates.
2. Add rules: R006 (identifier segments must pair with a collection), R007 (query parameters documented in the table must be kebab-case).
3. Emit machine-readable output (`--format json`) alongside the text report.
4. Write `pytest`-style unit tests (`stdlib unittest` is acceptable) covering every rule, including false-positive cases.

Acceptance Criteria.

- ☐ Exit codes 0/1/2 behave as documented.
- ☐ Waivers suppress only their cited rule and path, and expire.
- ☐ Full rule test coverage, all green and deterministic.

- ☐ JSON report validates against a schema you write.

Stretch Goals. Accept a real OpenAPI 3.1 document as input; diff two route tables and classify each delta as breaking or additive.

4.9.4 Capstone 3.4 [senior] — Port an RPC API to Resource Orientation

Objective. Execute the Section 4.5 migration doctrine on a realistic legacy surface.

Inputs/Datasets. The RoboDirect API from Lab Part B, extended by you to twelve endpoints (add shipping, invoicing, and technician scheduling RPCs); a synthetic two-week access log you generate showing per-endpoint traffic.

Steps.

1. Model the full RoboDirect domain as resources; produce the target route table and lint it.
2. Write the OpenAPI 3.1 contract for the new surface, design-first.
3. Implement compatibility shims (301/308 redirects or proxy routes) mapping every legacy call to a new resource, with **Deprecation** and **Sunset** headers.
4. Write a migration plan keyed by the access log: consumer cohorts, coexistence windows, measurable exit criteria per cohort.

Acceptance Criteria.

- ☐ Every legacy endpoint has a mapped resource or a written reification failure analysis.
- ☐ Contract passes Listing 4.3.2 with zero findings.
- ☐ Shims demonstrably preserve behavior for the twelve legacy calls (before/after transcripts).
- ☐ Migration plan names dates, owners, and kill criteria.

Stretch Goals. Add hypermedia transition links to the two workflow-heavy resources (shipments, invoices); quantify payload and round-trip deltas versus the RPC originals.

4.9.5 Capstone 3.5 [staff] — Design-Review Report on a Public API

Objective. Produce a staff-grade architectural critique of a real, publicly documented API (documentation review only — no production traffic; a captured, synthetic replay of documented examples is the dataset).

Inputs/Datasets. The public documentation and OpenAPI document of one large API of your choice; this chapter’s constraint tables; the Listing 4.3.2 linter run against its published paths.

Steps.

1. Score the API against all six constraints with evidence (captured responses, documented behaviors).
2. Place it on the Richardson scale per subsystem — large APIs are rarely uniform.
3. Quantify: lint findings by rule; status-code usage distribution; cacheability of GET responses; hypermedia coverage.
4. Write a prioritized remediation backlog with effort/impact estimates and a coexistence-safe migration approach for the top three items.

Acceptance Criteria.

- ☐ Every claim cites observable evidence (header capture, documented example, lint output).
- ☐ At least one finding credits the API for a sophisticated choice — pure hatchet jobs fail review.
- ☐ Remediation backlog survives the case-study doctrine: no rename-and-restructure in a single step.
- ☐ Report is eight pages or fewer and readable by a non-specialist executive.

Stretch Goals. Repeat the review five years apart using archived documentation; measure whether the vendor’s evolvability claims materialized.

4.10 Python Dependencies Appendix

The chapter is stdlib-first: Listings 4.3.1 and 4.3.3 run on a bare Python 3.11 installation. Only the route-table validator has an optional third-party need.

```
1 # requirements.txt -- Chapter 3 (Python 3.11+)
2 # Core listings (catalog API, HAL serializer): standard
   library only.
3 pyyaml>=6.0      # uri_lint.py: YAML route tables (
   JSON input works without it)
4 jsonschema>=4.21 # optional: structurally validate
   OpenAPI 3.1 documents
```

4.10.1 pyyaml (≥ 6.0)

Description. The de-facto YAML 1.1/1.2 parser for Python; used by `uri_lint.py` only when the route table is authored in YAML (the native format of most OpenAPI documents).

Why included. YAML is the authoring format of the API-design ecosystem; rejecting it would make the linter useless against real contracts. It is a mature, widely pinned dependency with no runtime services.

Alternatives. `ruamel.yaml` (round-trip preservation; heavier); the standard library alone (accept JSON only — the linter already does).

Installation (PowerShell).

```
1 python -m pip install "pyyaml>=6.0"
```

4.10.2 jsonschema (≥ 4.21)

Description. A JSON Schema validator implementing the 2020-12 draft that OpenAPI 3.1 adopts [Ope21].

Why included. Optional belt-and-braces: validate that an OpenAPI document's schemas are well-formed and that example payloads conform, before the contract reaches consumers. Not required by any listing.

Alternatives. `openapi-spec-validator` (validates the OpenAPI document itself, built on `jsonschema`); `check-jsonschema` (CLI wrapper).

Installation (PowerShell).

```
1 python -m pip install "jsonschema>=4.21"
```

4.11 Chapter Summary & Key Takeaways

- **REST is a style, not a protocol.** Six constraints — client-server, statelessness, cacheability, uniform interface, layered system, optional code-on-demand — are traded for scalability, visibility, reliability, and evolvability [Fie00].

- **Diagnose before you judge.** The Richardson Maturity Model (POX → resources → verbs → hypermedia) tells you which constraints an API actually honors [RAR13]. “JSON over HTTP” tells you nothing.
- **Model things, not operations.** Nouns in URIs, the four archetypes, reified actions, containment-only nesting (depth ≤ 2), kebab-case, plural collections, bare trailing-slash policy, path for identity and query for modulation.
- **Let the uniform interface work.** Safety and idempotency are contracts your retries and caches depend on; status codes are the server’s half of the conversation; ETags plus conditional requests give caching and optimistic concurrency for free [Fie22a].
- **PATCH means choosing a patch grammar.** Merge Patch for forgiving updates, JSON Patch for atomic, preconditioned, concurrent editing.
- **Hypermedia is an investment decision.** Links and affordances [Not17] buy restructuring freedom; adopt selectively unless your consumers genuinely outnumber your coordination bandwidth.
- **Representations are products, not dumps.** Anti-corruption layering, additive evolution, tolerant readers, and never reusing field names keep APIs evolving for years [Kle17; GH21].
- **Govern with code.** A linted route table and a design-first OpenAPI contract [Ope21] beat a wiki of good intentions.

4.12 Quick-Reference Card

```
1 URI grammar: lowercase kebab-case . plural collections
   . depth <= 2 .
2 no verbs . no trailing slash (301 the slashed form) .
3 path = identity, query = modulation . URIs are opaque
   to clients
```

4.13 Standardized Evidence Addendum

4.13.1 Benchmark Snapshot Template

4.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

4.13.3 Cost and Latency Budget Snapshot

4.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

4.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

4.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

4.17 Cross-Chapter Decision Card

4.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

4.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Transition	Success	Failure semantics
Create (POST/PUT)	201 + <code>Location</code>	400 invalid body; 409 conflict (duplicate SKU); 415 wrong media type; 422 semantically invalid
Read (GET)	200 (304 on conditional hit)	404 unknown resource; 410 permanently retired; 406 no acceptable representation
Replace (PUT)	200 or 204 (201 if created)	412 precondition failed (<code>If-Match</code> mismatch); 409 concurrent-edit conflict; 400/422 invalid
Modify (PATCH)	200 or 204	400 malformed patch document; 409 conflicting state; 422 patch not applicable; 415 wrong patch media type
Remove (DELETE)	204 (200 with body)	404 already absent; 409 referenced by other resources
Any	—	405 method not allowed (+ <code>Allow</code> header); 429 rate limited; 500/503 server-side

Table 4.3: Precise status codes for CRUD lifecycles. 404 in response to DELETE is informative, not an error the client must handle differently; 204 versus 200 signals whether the response carries a representation.

	HAL	JSON:API	Collection+JSON
Media type	application/hal+json	application/vnd.api+json	application/vnd.collection+json
Core model	<code>_links</code> , <code>_embedded</code> , curies	<code>data/attributes/relationships</code> compound documents	<code>relationships</code> + <code>queries</code> + write <code>template</code>
Write support	out of scope (HAL Forms extension)	full CRUD semantics specified	template-driven POST/PUT bodies
Philosophy	minimalism; bring your own conventions	opinionated full protocol (pagination, errors, sparse fields)	pure hypermedia; the representation <i>is</i> the contract
Best fit	adding links to an existing JSON API	greenfield APIs wanting everything standardized	reference designs, hypermedia research, generic clients

Table 4.4: Hypermedia media types for JSON APIs (see Section 4.8).

Constraint	One-line test
Client-server	Can the UI and the datastore each change without the other?
Statelessness	Is every request self-contained — no server session context?
Cacheability	Does every response say whether and how it may be cached?
Uniform interface	Resources identified; manipulated via representations; messages self-describing; hypermedia drives state?
Layered system	Could an intermediary be inserted without semantic change?
Code-on-demand (opt.)	If scripts are served, is the visibility loss accepted in writing?

Table 4.5: Six-constraint compliance checklist.

Method	Safe	Idempotent	Success
GET/HEAD	yes	yes	200 / 304
POST	no	no	201 (+ Location) or 200
PUT	no	yes	200 / 204 / 201 (412 on stale If-Match)
PATCH	no	(design for it)	200 / 204
DELETE	no	yes	204 (404 on repeat is fine)
OPTIONS	yes	yes	204 + Allow

Table 4.6: Method semantics at a glance [Fie22a].

RMM level	Signature	What you gain
0 — POX	one URI, POST, 200-always	nothing; HTTP as tunnel
1 — Resources	many noun URIs, one verb	addressability
2 — Verbs	methods + status codes + validators	caching, safe retry, visibility
3 — Hypermedia	representations links/forms	carry restructuring freedom, run-time discovery

Table 4.7: Richardson Maturity Model diagnostic [RAR13].

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 4.8: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 4.9: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 4.10: Decision card to complete per chapter.

Chapter 5

Data on the Wire: Serialization Formats & Data Contracts

Every API call is, at its core, an act of translation. Inside your process, a *Part* is a graph of objects living in heap memory; on the wire it must become an ordered sequence of bytes; on the far side it must be reconstructed into an equivalent structure without loss, ambiguity, or surprise. The rules governing that translation — the *serialization format* — and the rules governing what the translated bytes are allowed to mean — the *data contract* — are the twin subjects of this chapter. They are also the two most underestimated sources of production incidents in distributed systems. Numbers silently truncated at the 2⁵³ IEEE 754 boundary, a reused Protocol Buffers tag number that reinterprets a customer discount as a warehouse identifier, a **Content-Type** header that lies about the body it labels: each of these is a serialization or contract failure, and each has caused real, expensive outages.

We begin with JSON, the lingua franca of web APIs, studied with wire-level rigor against RFC 8259 [Bra17]: its six value types, its grammar, its Unicode and escaping rules, and — critically — its known ambiguities and sharp edges, including the number-precision pitfall and the unspecified semantics of duplicate object keys. We then formalize JSON payloads with **JSON Schema** (2020-12), learning the core vocabulary, the distinction between assertions and annotations, and how schemas become machine-checkable contracts that drive validation, documentation, and code generation. Next we descend to the binary layer: **Protocol Buffers**, with a complete byte-by-byte varint encoding walkthrough; **Avro**, with its writer/reader schema resolution model; and **MessagePack**, a schemaless binary JSON. We compare all of them on size, speed, schema evolution, and debuggability. Because bytes alone are not enough — a server must also *agree* with the client on which representation to send — we study HTTP content negotiation in full mechanical depth: **Accept** q-values, media type

parameters, vendor types, and compression via **Accept-Encoding**. Finally, we elevate the discussion to *contract engineering*: consumer-driven contracts, schema registries, and the precise meaning of backward, forward, and full compatibility.

By the end of this chapter you will be able to hand-encode a Protocol Buffers message byte by byte, implement a standards-correct content-negotiation selector, validate payloads against a JSON Schema and emit RFC 9457 problem-details error reports [Not23], benchmark competing formats with honest methodology, and defend every one of those choices in a design review or an interview room.

Key Concept

Serialization converts in-memory structures to bytes; a **data contract** constrains what those bytes may mean. Formats (JSON, Protobuf, Avro, MessagePack) are interchangeable in principle; contracts (schemas, compatibility rules, negotiated media types) are what make interchange safe in practice. Treat the contract as the API and the format as an implementation detail you can benchmark, swap, and version.

5.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Describe the complete RFC 8259 JSON grammar — the six value types, string escaping, and Unicode/UTF-8 rules — and explain precisely which behaviors the specification leaves undefined (number range, duplicate keys) and why those gaps cause production defects [Bra17].
- LO-2.** Demonstrate, with exact arithmetic, why IEEE 754 double-precision numbers cannot safely represent integers beyond 2^{53} , and design payloads that carry 64-bit identifiers without corruption.
- LO-3.** Author JSON Schema (2020-12) documents using `type`, `properties`, `required`, `additionalProperties`, `$ref`, `oneOf/anyOf/allOf`, and `unevaluatedProperties`, and distinguish assertions from annotations [JSO22].
- LO-4.** Encode and decode Protocol Buffers wire format by hand:

variants, zigzag mapping, wire types 0/1/2/5, and the tag computation $\text{tag} = (\text{field} \ll 3) | \text{wire_type}$; state the schema-evolution rules (never reuse tag numbers, use **reserved**) and explain the catastrophe that follows from violating them [Goo24].

- LO-5.** Contrast Avro’s writer/reader schema resolution with Protobuf’s tag-based evolution and MessagePack’s schemaless self-describing binary encoding, and select a format using a quantitative trade-off matrix.
- LO-6.** Implement HTTP content negotiation correctly: parse **Accept** q-values, compute the best available representation, honor media type parameters and vendor types, and reason about **Accept-Encoding** compression trade-offs including CPU cost and decompression-bomb risk [Fie22a].
- LO-7.** Define backward, forward, and full schema compatibility precisely, and explain how schema registries and consumer-driven contracts enforce them (deepened in Chapter 19).
- LO-8.** Build, in typed Python 3.11, a minimal Protobuf codec, a JSON Schema validation pipeline emitting RFC 9457 errors, a negotiation selector, a deterministic benchmark harness, and a streaming NDJSON reader/writer.

Prerequisites

This chapter assumes you have completed:

- **Chapter 0** — environment setup: Python 3.11+, a virtual environment, and the ability to run `pytest` and `pip` from PowerShell.
- **Chapter 1** — the HTTP wire protocol: requests, responses, header fields, status codes, and message bodies. We reuse that vocabulary constantly, especially in the content-negotiation sections.
- **Chapter 2** — consuming APIs from Python: `json` module basics, reading response bodies, and structured error handling.

You should also be comfortable with binary/hexadecimal notation (Chapter 1’s byte dump exercises) and with reading railroad-diagram-style grammar descriptions. No network access is required anywhere in this chapter: every fixture is synthetic, embedded, and deterministic, set in the fictional *Northwind Robotics* parts and logistics universe.

Warning

All benchmark numbers printed by this chapter’s code are **illustrative**, not authoritative. They vary by machine, interpreter build, and payload. The chapter’s harness is deterministic in *structure* — fixed seeds, fixed payloads, fixed iteration counts — so that you can reproduce relative comparisons on your own hardware, but never hard-code timing assertions into tests.

5.2 Deep Technical Mechanics & Architectural Concepts

5.2.1 JSON per RFC 8259: The Grammar You Think You Know

JSON (JavaScript Object Notation) is defined by RFC 8259 as “a text format for the serialization of structured data” [Bra17]. Its entire grammar recognizes exactly **six value types**:

Definition 5.1 (JSON value). A JSON value is exactly one of: an *object* (an unordered collection of name/value pairs), an *array* (an ordered list of values), a *string* (a sequence of Unicode code points), a *number* (decimal, optional fraction and exponent), *true*, *false*, or *null*.

Whitespace is permitted only around structural tokens (`{`, `}`, `[`, `]`, `:`, `,`). There are **no comments** and **no trailing commas** — both are syntax errors, and the trailing comma is a perennial source of bugs when payloads are assembled by string templating rather than by a serializer.

Numbers: the specified syntax and the unspecified semantics. RFC 8259 fully specifies number *syntax* (optional minus, integer part with no leading zeros, optional fraction, optional exponent) but deliberately declines to specify *range or precision*:

This specification allows implementations to set limits on the range and precision of numbers accepted. — RFC 8259, Section 6 [Bra17]

This is the single most consequential sentence in the JSON specification, because virtually every implementation — JavaScript’s `JSON.parse`, Python’s `json`, Java’s common parsers — defaults to the IEEE 754 binary64 (“double”) floating-point representation. A double stores a sign bit,

an 11-bit biased exponent, and a 52-bit significand. The integers it can represent *exactly* are those with magnitude at most

$$2^{53} = 9\,007\,199\,254\,740\,992.$$

Beyond that boundary, the spacing between adjacent representable values exceeds 1, so integers begin to alias. The canonical failure, which you can reproduce in any browser console or Python REPL:

```
1 >>> import json
2 >>> payload = json.dumps({"order_id":
   9007199254740993}) # 2**53 + 1
3 >>> payload
4 '{"order_id": 9007199254740993}'
5 >>> # Simulate the consumer: a JavaScript-style double-
   only parser
6 >>> float(9007199254740993) == float(9007199254740992)
7 True
8 >>> int(float(9007199254740993))
9 9007199254740992
```

The consumer asked for order $2^{53} + 1$ and silently received order 2^{53} — a *different real order* in the database. No exception is raised anywhere; the corruption is lossless-looking, which is precisely what makes it dangerous. We return to this incident pattern in the case study (Section 5.5). The defensive rules:

1. Transmit 64-bit integers as **JSON strings** (`"order_id": "9007199254740993"`) and parse them explicitly.
2. If you control both ends, prefer a binary format with explicit integer widths.
3. If you must emit JSON numbers, document a safe range contract ($|n| < 2^{53}$) and validate it in your schema with `minimum` and `maximum` keywords.

Strings, Unicode, and escaping. JSON text exchanged between systems **must be encoded in UTF-8**; RFC 8259 removed the older UTF-16/UTF-32 options for interoperability [Bra17]. Within strings, only five characters have mandatory short escapes (`\"`, `\\`, `\b`, `\f`, `\n`, `\r`, `\t`), plus `\/` as an optional escape, and any code point may be written `\uXXXX`. Two subtleties matter in production:

- **Surrogate pairs.** Code points above the Basic Multilingual Plane (emoji, rare CJK ideographs) are written as a *pair* of `\uXXXX` escapes

in UTF-16 surrogate form. A parser that validates each escape independently can accept an unpaired surrogate and produce a string that later blows up an encoder or a database column with a strict UTF-8 check.

- **Equivalence is not byte equality.** The strings "café" (composed) and "café" with a combining accent (decomposed) are visually identical, semantically equivalent, and byte-level different. Key lookups that mix the two fail. Normalize (NFC) at system boundaries when keys carry human-entered text.

Duplicate object keys. RFC 8259 says object names “SHOULD be unique” but explicitly declines to define behavior when they are not [Bra17]. Implementations variously keep the last value (Python, JavaScript), keep the first, or reject the document. Attackers exploit the divergence: a request smuggled through a last-wins proxy into a first-wins backend presents a *different document* to each hop — a classic parser differential. Defensive rule: when payloads cross trust boundaries, use a parser that detects duplicates (Python: `json.loads(..., object_pairs_hook=...)` and raise on repeats).

JSON Lines / NDJSON for streaming. A single JSON document must be buffered to its closing brace before it parses — fatal for multi-gigabyte feeds. **JSON Lines** (a.k.a. NDJSON, `application/x-ndjson`) solves this structurally: each line is one complete, self-contained JSON value, terminated by `\n`.

```
1 {"part_id": "NR-1001", "name": "Servo Motor 42mm", "qty": 120}
2 {"part_id": "NR-1002", "name": "Planetary Gearbox 10:1", "qty": 45}
3 {"part_id": "NR-1003", "name": "LiDAR Puck 16ch", "qty": 8}
```

The format buys you: (1) constant-memory, line-at-a-time processing; (2) cheap append semantics for logs; (3) resumability — truncate at a line boundary and every preceding record is still valid. It costs you: no top-level schema anchor, no random access, and no guarantee that every line shares one shape (validate per line). Section 5.3 implements a streaming NDJSON pipeline.

JSON Pointer and JSON Path. Two complementary notations locate values inside a JSON document:

- **JSON Pointer** (RFC 6901): a strict, unambiguous path string such as `/parts/0/sku`, where `~0` escapes `~` and `~1` escapes `/`. It identifies exactly one value; it cannot express queries.
- **JSON Path**: a query language (think XPath for JSON) with wildcards, slices, and filters, e.g. `$.parts[?(@.qty < 10)].sku`. Powerful, but historically divergent across implementations; the IETF standardization (RFC 9535) is recent enough that you should pin your library version in contracts that expose JSON Path expressions to clients.

Use JSON Pointer for *addressing* (error locations, patch operations, schema `$ref` fragments) and JSON Path for *selection* (filtering, extraction).

5.2.2 JSON Schema 2020-12: From Hopes to Machine-Checked Contracts

An undocumented JSON payload is a rumor. **JSON Schema** turns it into a machine-checkable contract: a JSON document that describes which JSON documents are acceptable [JSO22]. The 2020-12 dialect is the version assumed throughout this book (and is the dialect embedded in OpenAPI 3.1).

Definition 5.2 (JSON Schema assertion vs. annotation). An *assertion* keyword constrains validity: if the instance violates it, validation *fails* (`type`, `required`, `minimum`). An *annotation* keyword attaches metadata that validators collect but never enforce (`title`, `description`, `default`, `examples`). Confusing the two — e.g. assuming `default` fills in missing values — is a top-ten JSON Schema misconception.

The core vocabulary you must command fluently:

The subtlest row is the last structural one. `additionalProperties: false` only “sees” members declared in the *same* schema object. When you compose with `allOf`, members declared in sibling subschemas are invisible to it, so a “closed” base schema wrongly rejects extensions. `unevaluatedProperties: false` (new in 2019-09, stabilized in 2020-12) closes that hole: it applies after all in-place applicators have run, rejecting any member no subschema ever evaluated. The pattern for extensible-but-closed contracts is therefore:

```
1 {  
2   "allOf": [{ "$ref": "#/$defs/partBase" }],  
3   "properties": { "firmware": { "type": "string" } },  
4   "unevaluatedProperties": false  
5 }
```

Validation semantics. Validation walks the instance top-down; each applicator produces either a boolean result (fast, “fail-fast” mode) or an *annotation collection plus error tree* (basic/detailed/verbose output formats). Production validators should collect *all* errors, not stop at the first — a client fixing one error per round trip is a support-ticket generator. Section 5.3 maps the error tree onto an RFC 9457 `application/problem+json` body [Not23].

Code generation and contract-first thinking. A schema that only validates is half-used. The same document can drive: (a) generated client/server models (Python dataclasses, TypeScript interfaces, Java records); (b) generated documentation; (c) generated test fixtures (schema-guided fuzzers); (d) mock servers. This inverts the legacy workflow — implementation first, documentation later, contract never — into **contract-first**: author the schema, review it as an API artifact, then generate everything else from it. The schema becomes the single source of truth and the diff that code review actually guards.

Versioning schemas. Schemas evolve; the discipline is making evolution legible:

- Give every schema a stable \$id URI containing a version segment (`https://schemas.northwind-robotics.example/parts/v2/part.json`).
- Prefer *compatible* edits (Section 5.2.7): add optional fields, widen enums only when consumers tolerate unknowns, never narrow a type in place.
- For breaking changes, publish a new \$id alongside the old one and run both validators during a migration window.
- Record *deprecated* members with the `deprecated: true` annotation instead of deleting them mid-version.

5.2.3 Protocol Buffers: The Wire Format, Byte by Byte

Protocol Buffers (protobuf) is Google’s language-neutral binary serialization system [Goo24]. You declare messages in an Interface Definition Language (IDL); a compiler generates codecs; at runtime, a message is a compact byte stream with *no field names, no type names, no delimiters between fields beyond what the encoding itself carries*. Everything the decoder knows about meaning comes from two small integers per field: a **field number** and a **wire type**, packed into one varint called the **tag**:

$\text{tag} = (\text{field_number} \ll 3) | \text{wire_type}, \quad \text{field_number} = \text{tag} \gg 3, \quad \text{wire_type} = \text{tag} \& 0b11$

Varints, worked byte by byte. A varint encodes an unsigned integer 7 bits at a time, *least significant group first*. Each byte's high bit is a continuation flag: 1 means “more bytes follow”, 0 means “this is the last byte”. The 300 encoding is the textbook example:

$$300 = 256 + 44 = (2 \ll 7) + 44 \implies \text{groups} = [44, 2] \implies \text{bytes} = [\underbrace{44 | 0x80}_{0xAC}, \underbrace{2}_{0x02}]$$

So the decimal integer 300 serializes as the two bytes 0xAC 0x02. Decoding reverses the process: strip continuation bits, then reassemble $\sum_i b_i \ll (7i)$.

```

1  Value 300 = 0b1_0101100
2              |  |
3              |  +-- low 7 bits: 0101100 (44) -> 0
                x80|44 = 0xAC (MSB set: more)
4              +----- high bits: 10          (2) -> 0x02
                (MSB clear: done)
5
6  Wire bytes: [0xAC] [0x02]
7              ^      ^
8              |      +-- 0000 0010 -> contributes 2
                << 7 = 256
9              +----- 1010 1100 -> contributes 44
                (continuation bit stripped)
10             Total: 256 + 44 = 300

```

Putting a whole message on the wire. Consider a slice of the Northwind Robotics catalog:

```

1  syntax = "proto3";
2  message Part {
3      string part_id = 1;    // LEN    (wire type 2)
4      uint32 qty       = 2;  // VARINT (wire type 0)
5      sint32 delta     = 3;  // VARINT, zigzag-encoded
6      double unit_kg   = 4;  // I64    (wire type 1)
7      float  width_mm  = 5;  // I32    (wire type 5)
8  }

```

With `part_id = "A1"`, `qty = 300`, `delta = -3`, `unit_kg = 1.5`, `width_mm = 42.0`, the complete encoding is 23 bytes:

Offset	Bytes	Field	Tag byte	Meaning
0	0x0A	1	1<<3 2=10	field
1	0x02	1	--	1, LEN
2-3	0x41 0x31	1	--	length = 2 "A1" (UTF-8)
4	0x10	2	2<<3 0=16	field
5-6	0xAC 0x02	2	--	2, VARINT
7	0x18	3	3<<3 0=24	varint
8	0x05	3	--	300 (worked above)
9	0x21	4	4<<3 1=33	field
10-17	0x00 00 00 00 00 00	4	--	4, I64
18	0x2D	5	5<<3 5=45	field
19-22	0x00 00 28 42	5	--	as IEEE-754 double, LE
				1.5
				as IEEE-754 float, LE
				42.0
Equivalent JSON: {"part_id":"A1","qty":300,"delta":-3, "unit_kg":1.5,"width_mm":42.0} -> 67 bytes (compact)				

Zigzag: why sint exists. Varints are length-proportional to magnitude, so a negative `int32` like `-1`, sign-extended to 64 bits, costs the maximum *ten bytes*. Zigzag mapping folds signed integers into unsigned ones so that small magnitudes — positive or negative — stay cheap:

$$\text{zigzag}(n) = (n \ll 1) \oplus (n \gg 63) \quad \Longleftrightarrow \quad 0 \mapsto 0, -1 \mapsto 1, 1 \mapsto 2, -2 \mapsto 3, 2 \mapsto 4, \dots$$

In the worked message above, `delta = -3` encodes as `zigzag(-3) = 5`, one byte instead of ten. Rule of thumb: if a signed field will frequently hold negative values, declare it `sint32/sint64`.

Schema evolution: the rules that keep you employed. Because the wire carries only numbers and wire types, evolution is governed by a short list of iron rules [Goo24; Kle17]:

1. **Never reuse a tag number.** A tag is a permanent identity. If field 7 once meant `discount_ppm` and you recycle it as `warehouse_id`, a stale consumer will read warehouse IDs as discounts — silently, with a plausible-looking type match (the case study in Section 5.5 dramatizes this).
2. **Never change a field's wire-compatible type.** Safe: `int32` → `int64`. Unsafe: `string` → `bytes` across semantics, or any change that alters the wire type.
3. **Delete fields by reserving, not by removing bare.** Write `reserved 7; reserved "discount_ppm";` so the compiler rejects any future reuse.
4. **Adding fields is always safe** — old readers ignore unknown tags (proto3 discards them by default; proto2/unknown-fields-aware runtimes can retain them).
5. **Field presence changed in proto3:** scalars default to zero and are omitted on the wire; use `optional` to regain explicit presence.

5.2.4 Avro and MessagePack: Two Other Bets

Avro: schema rides with (or beside) the data. Apache Avro defines schemas in JSON and a compact binary encoding that, unlike protobuf, writes *no tags and no field numbers at all*: fields are serialized in schema declaration order, and the decoder navigates purely by schema. Integers use the same varint+zigzag technique you just learned. The design consequence is profound: an Avro payload is meaningless without its **writer schema**, so Avro systems ship schemas alongside data (in an Object Container File header) or resolve them from a registry by fingerprint. Evolution is then a formal problem — **schema resolution**: given writer schema W and reader schema R , match fields *by name*, apply promotion rules (`int` → `long` → `float` → `double`), supply reader **defaults** for missing fields, and fail when a required reader field has no writer counterpart and no default. Compared with protobuf's tag matching, name matching is more forgiving of renumbering but less forgiving of renames.

MessagePack: schemaless binary JSON. MessagePack asks: what if JSON, but binary? Every value begins with a one-byte **type prefix**; small

values are embedded directly in the prefix byte itself (*fixint*, *fixstr*, *fixarray*, *fixmap*).

1	MessagePack byte map for {"part_id": "A1", "qty": 300}			
2				
3	Offset	Byte(s)	Prefix family	Meaning
4	-----	-----	-----	
5	0	0x82 with 2 pairs	fixmap (1000xxxx)	map
6	1	0xA7 length 7	fixstr (101xxxxx)	string,
7	2-8	"part_id" bytes	--	UTF-8
8	9	0xA2 length 2	fixstr	string,
9	10-11	"A1" bytes	--	UTF-8
10	12	0xA3 length 3	fixstr	string,
11	13-15	"qty" bytes	--	UTF-8
12	16	0xCD unsigned 16-bit follows	uint 16 (0xCD)	
13	17-18	0x01 0x2C big-endian	--	300,
14				
15	Total: 19 bytes. Compact JSON of same data: 26 bytes.			
16	Key families: 0x00-0x7F positive fixint 0xE0-0xFF			
17	negative fixint			
18	0x80-0x8F fixmap 0x90-0x9F fixarray 0xA0-0xBF			
	fixstr			
	0xC0 nil 0xC2/0xC3 bool 0xCA float32 0xCB			
	float64 0xCD uint16			

The trade is explicit: MessagePack keeps JSON's flexibility — schema-less, self-describing, dynamically typed — while buying maybe 10–30% size reduction and fast parse times. It gives up field-name elision (names are on the wire, as the byte map shows) and gives up compile-time contracts. It is a serialization *format*, not a contract system; pair it with JSON Schema if you need the latter.

5.2.5 The Format Trade-Off Matrix

Figure 5.1 shows measured wire sizes for the three Northwind payload shapes you will benchmark yourself in the lab; Figure 5.2 compresses the

matrix into a selection decision tree.

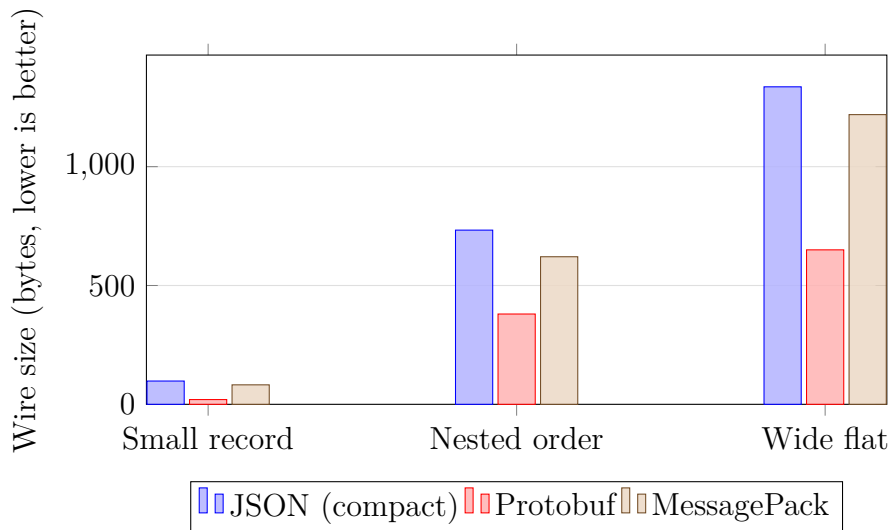


Figure 5.1: Wire sizes for three payload shapes. JSON and MessagePack bars are measured by the benchmark harness in Section 5.3; the protobuf small-record bar is measured (the hand-rolled codec), and the protobuf nested/wide bars are projected from packed-field arithmetic — your lab run will refine them. Binary formats win consistently; the margin grows with numeric and wide payloads.

5.2.6 Content Negotiation: Agreeing on the Bytes

Serialization answers “how do I encode this object?” Content negotiation answers the prior question: “*which* encoding, in which language, compressed how, does this response carry?” HTTP semantics (RFC 9110, Section 12) define proactive negotiation via request headers the server evaluates against the representations it can produce [Fie22a].

Accept and q-values: the selection algorithm. The Accept header is a comma-separated list of *media ranges* with optional quality values:

```

1 Accept: application/vnd.northwind.parts+json;v=2,
2       application/json;q=0.9,
3       text/html;q=0.8,
4       application/*;q=0.5,
5       */*;q=0.1

```

A media range matches a concrete media type by specificity: an exact **type/subtype** with parameters beats **type/***, which beats ***/***. The server’s algorithm — which Section 5.3 implements — is:

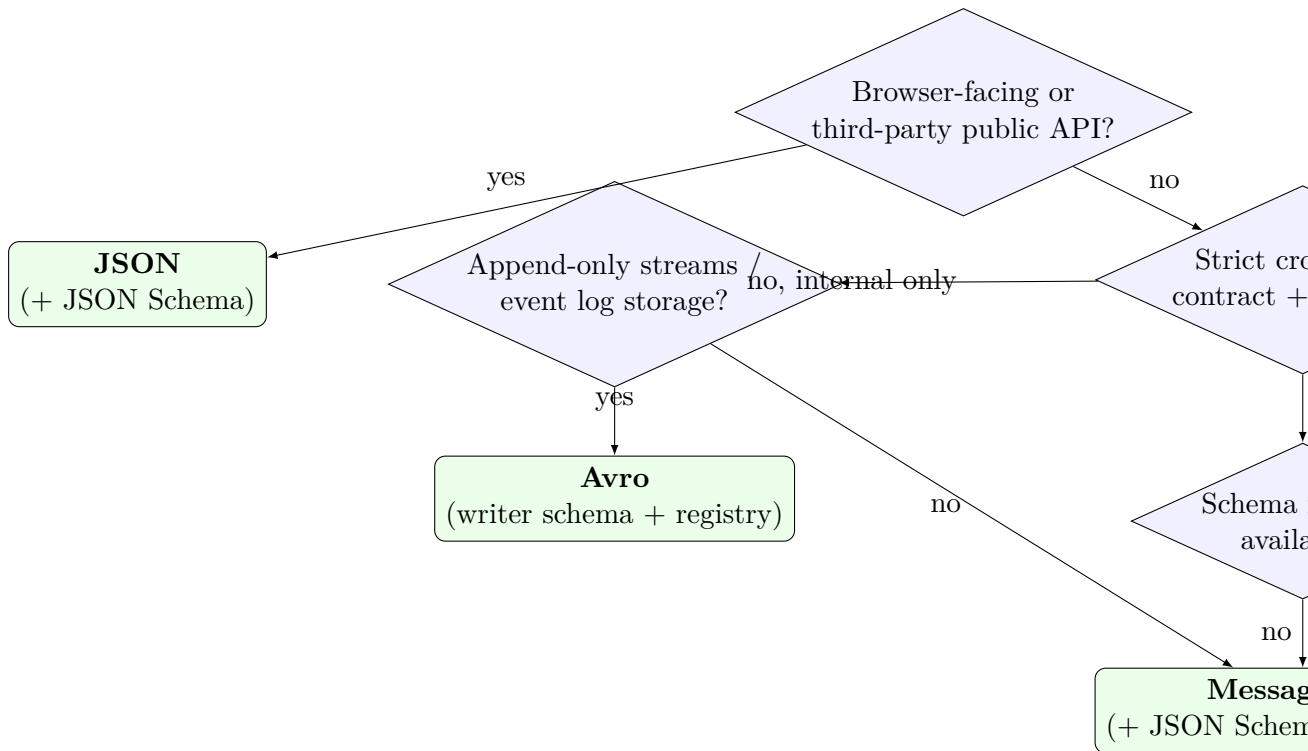


Figure 5.2: Format selection decision tree. Defaults, not laws: a public API with a performance-critical mobile tier may still serve protobuf via a vendor media type (Section 5.2.6).

1. Parse each entry into (type, subtype, parameters, q); a missing q means $q = 1.0$; a missing **Accept** header means $/*$ at $q = 1.0$.
2. For each *available representation*, find the most specific matching range; assign it that range's q . Entries with $q = 0$ mean “explicitly refused”.
3. Order candidates by descending q ; break ties by media-range specificity (exact > type-wildcard > full wildcard), then by server preference.
4. If nothing matches with $q > 0$, return 406 Not Acceptable.

Worked example: suppose the server can produce `application/json` and `text/html` for the header above. `application/json` matches both the `v=2` vendor range (no — parameters don't match, the vendor subtype differs), `application/json` at $q=0.9$, `application/*` at $q=0.5$, and `/*` at $q=0.1$; the most specific matching range is `application/json` itself, so the score is 0.9. `text/html` scores 0.8. The server selects JSON. Note what did *not* happen: the vendor range with its `v=2` parameter did not

match plain `application/json` — parameter-bearing ranges match only representations that satisfy the parameters.

Content-Type, charset, and media type parameters. `Accept` is the request side; `Content-Type` is the response’s declaration of what the body *actually is*:

```
1 Content-Type: application/json; charset=utf-8
```

Parameters after the semicolon are part of the type. `charset` governs how bytes map to characters; omitting it from a text type invites the receiver to guess (legacy defaults bite here — e.g. ISO-8859-1 assumptions in old stacks). JSON in UTF-8 needs no charset at all per RFC 8259 [Bra17], but emitting `charset=utf-8` explicitly is harmless and silences buggy intermediaries.

Vendor media types. Public APIs increasingly version *representations* instead of URLs via vendor trees:

```
1 application/vnd.northwind.parts+json;v=2
```

The `vnd.` tree marks a company-owned type; the `+json` suffix tells generic tooling the body follows JSON structure (so a JSON Schema validator or a pretty-printer works unmodified); the `v=2` parameter carries the contract version. This keeps URLs stable (`GET /parts/NR-1001`) while the representation evolves — the purest expression of REST’s representation-orientation, and the pattern behind GitHub’s and Stripe-style content-negotiated versioning. The operational cost: caches must key on `Accept` (via the `Vary` header), and intermediaries that ignore parameters can serve a v1 body to a v2 client. Test your CDN.

Accept-Encoding and compression trade-offs. Independently of media type, the client advertises acceptable *content codings*:

```
1 Accept-Encoding: br;q=1.0, gzip;q=0.8, zstd;q=0.9,
  identity;q=0.1
```

`gzip` (DEFLATE) is universal; `br` (Brotli) compresses text 15–25% smaller at similar decode cost; `zstd` offers tunable levels with excellent decode speed and is spreading through CDN and gRPC stacks. The engineering trade-offs:

- **CPU vs. bytes.** Compression pays when the link is the bottleneck (mobile, cross-region) and the payload is large and repetitive (JSON arrays). For sub-kilobyte payloads, compression *adds* bytes (frame overhead) and CPU; most servers enforce a minimum size threshold (~860 bytes is a common nginx default; 1 KiB is a sane rule).

- **Compression bombs.** A 10 KB gzip body can expand to gigabytes of zeros. Any service that decompresses untrusted input must cap the *decompressed* size mid-stream (Section 5.4); Python's `gzip.decompress` has no built-in limit — `gzip.GzipFile` with bounded `read(n)` loops does.
- **Streaming compression.** For large or unbounded bodies, compress chunk-by-chunk (`Content-Encoding: gzip` with chunked or HTTP/2 framing) rather than buffering the whole payload; the client's memory stays flat and time-to-first-byte drops. Never combine with `Content-Length` — the compressed length is unknown until done.
- **Double compression.** Binary formats and already-compressed media (images, `.parquet`, `.zip`) gain nothing; skip encoding for them to save CPU.

5.2.7 Contract Engineering: Registries, Compatibility, Consumers

A schema only creates value if every producer and consumer agrees on *which* schema and on *how it may change*. That agreement system has three pillars.

Schema registries. A registry (Confluent Schema Registry for Avro/Protobuf/JSON Schema, or an internal artifact store keyed by `$id`) is a versioned, addressable store of schemas plus a *compatibility gate*: every new schema version is checked against the configured compatibility mode before it may be published. Producers reference schemas by ID (Avro's wire convention prefixes each message with a 4-byte registry ID), so payloads stay lean and consumers fetch-and-cache schemas out of band.

Definition 5.3 (Compatibility modes). Given existing consumers C and producers P of a schema S , and a candidate new schema S' :

- **Backward compatible:** consumers written for S' can read data written with S (the *new code reads old data*). Typical allowed change: add a field *with a default*, delete a field. Required for consumers to upgrade first.
- **Forward compatible:** consumers written for S can read data written with S' (the *old code reads new data*). Typical allowed change: delete a field with a default, add a field that old readers ignore. Required for producers to upgrade first.

- **Full compatible:** both directions simultaneously — changes are restricted to optional-field additions/removals with defaults. The only mode that permits arbitrary upgrade order in a fleet.

The modes compose transitively across versions (*backward-transitive* checks S' against *all* earlier versions, not just the latest) — essential when old data lives for years in a log or a data lake.

Consumer-driven contracts. Compatibility modes answer “is this schema change structurally safe?” They cannot answer “will I break the mobile app’s checkout screen?” — that requires knowing what consumers actually *use*. **Consumer-driven contract testing** (CDC) records each consumer’s expectations as executable contracts and runs them against provider builds in CI, flipping the power balance: providers discover breaking changes before merge, and consumers own the definition of “breaking”. We treat CDC in full — Pact-style workflows, contract storage, provider verification — in Chapter 19 (Testing & Quality Engineering). For now, hold the principle: the contract is co-owned, executable, and versioned alongside the code.

Example

Contract-first change workflow at Northwind Robotics (parts service). (1) Schema author proposes `part-v3.json` adding optional `firmware_channel`. (2) CI validates the schema itself, then runs the registry check: `backward-compatible?` `yes` (optional field, default `"stable"`). (3) Provider CI runs recorded consumer contracts — one consumer’s contract forbids unknown top-level keys, so the change is negotiated to arrive under `additionalProperties`-tolerant semantics first. (4) Schema published; producer and consumer deploy in either order. No runtime surprises, because every gate was executable.

5.3 Production-Ready Code Implementation

Five complete, typed, offline programs. All run under Python 3.11+; only the JSON Schema pipeline and (optionally) the benchmark need third-party packages (Section 5.10). All fixtures are synthetic Northwind Robotics data; all demos use deterministic seeds.

5.3.1 A Hand-Rolled Protobuf Codec (Pure Standard Library)

The pedagogical centerpiece: a working encoder/decoder for the `Part` message from Section 5.2.3, using nothing but `struct` and `dataclasses`. Reading this code *is* reading the wire format specification [Goo24].

```

1  """mini_protobuf.py -- a hand-rolled Protocol Buffers
   wire-format codec.
2
3  Implements(proto3 semantics) for a single message:
4  message Part {
5  string part_id = 1; // LEN wire type 2
6  uint32 qty = 2; // VARINT wire type 0
7  sint32 delta = 3; // VARINT wire type 0 (
   zigzag)
8  double unit_kg = 4; // I64 wire type 1
9  float width_mm = 5; // I32 wire type 5
10 }
11 Pure standard library; offline; deterministic. Python
   3.11+.
12
13 from __future__ import annotations
14
15 import struct
16 from dataclasses import dataclass
17
18 WIRE_VARINT = 0
19 WIRE_I64 = 1
20 WIRE_LEN = 2
21 WIRE_I32 = 5
22 _MAX_VARINT_BYTES = 10 # a 64-bit varint never exceeds
   10 bytes
23
24
25 class DecodeError(ValueError):
26     """Raised when a byte stream is not a valid Part
   encoding."""
27
28
29 def encode_varint(value: int) -> bytes:
30     """Encode an unsigned 64-bit integer as a varint (7
   bits/byte, LSB first)."""
31     if not 0 <= value < 1 << 64:
32         raise ValueError(f"varint out of uint64 range: {value}")

```

```

33     out = bytearray()
34     while True:
35         bits = value & 0x7F
36         value >>= 7
37         out.append(bits | 0x80 if value else bits) #
            MSB set => more bytes
38         if not value:
39             return bytes(out)
40
41
42 def decode_varint(buf: bytes, pos: int) -> tuple[int,
int]:
43     """Decode a varint at buf[pos]; return (value,
next_pos)."""
44     result, shift = 0, 0
45     for i in range(_MAX_VARINT_BYTES):
46         if pos + i >= len(buf):
47             raise DecodeError("truncated varint")
48         byte = buf[pos + i]
49         result |= (byte & 0x7F) << shift
50         if not byte & 0x80: # continuation bit clear
            => final byte
51             return result, pos + i + 1
52         shift += 7
53     raise DecodeError("varint exceeds 64 bits")
54
55
56 def zigzag_encode(n: int) -> int:
57     """Map signed n to unsigned so small magnitudes
stay short."""
58     return (n << 1) ^ (n >> 63)
59
60
61 def zigzag_decode(u: int) -> int:
62     return (u >> 1) ^ -(u & 1)
63
64
65 def make_tag(field_number: int, wire_type: int) ->
bytes:
66     if not 1 <= field_number < 1 << 29:
67         raise ValueError(f"field_number out of range: {
field_number}")
68     return encode_varint((field_number << 3) |
wire_type)
69
70

```

```

71 @dataclass(frozen=True)
72 class Part:
73     part_id: str = ""
74     qty: int = 0
75     delta: int = 0
76     unit_kg: float = 0.0
77     width_mm: float = 0.0
78
79     def encode(self) -> bytes:
80         """Serialize; proto3 rule: default-valued
81            scalar fields are omitted."""
82         out = bytearray()
83         if self.part_id:
84             raw = self.part_id.encode("utf-8")
85             out += make_tag(1, WIRE_LEN) +
86                 encode_varint(len(raw)) + raw
87         if self.qty:
88             out += make_tag(2, WIRE_VARINT) +
89                 encode_varint(self.qty)
90         if self.delta:
91             out += make_tag(3, WIRE_VARINT) +
92                 encode_varint(zigzag_encode(self.delta))
93         if self.unit_kg != 0.0:
94             out += make_tag(4, WIRE_I64) + struct.pack(
95                 "<d", self.unit_kg)
96         if self.width_mm != 0.0:
97             out += make_tag(5, WIRE_I32) + struct.pack(
98                 "<f", self.width_mm)
99         return bytes(out)
100
101     @classmethod
102     def decode(cls, buf: bytes) -> "Part":
103         """Parse wire bytes. Unknown fields are skipped
104            (forward compat)."""
105         part_id, qty, delta, unit_kg, width_mm = "", 0,
106             0, 0.0, 0.0
107         pos = 0
108         while pos < len(buf):
109             tag, pos = decode_varint(buf, pos)
110             field_number, wire_type = tag >> 3, tag & 0
111             b111
112             if wire_type == WIRE_VARINT:
113                 value, pos = decode_varint(buf, pos)
114                 if field_number == 2:
115                     qty = value
116                 elif field_number == 3:

```

```

108         delta = zigzag_decode(value)
109     elif wire_type == WIRE_LEN:
110         length, pos = decode_varint(buf, pos)
111         if pos + length > len(buf):
112             raise DecodeError("LEN_field_
113                               overruns_buffer")
114         payload = buf[pos:pos + length]
115         pos += length
116         if field_number == 1:
117             part_id = payload.decode("utf-8")
118     elif wire_type == WIRE_I64:
119         (value,) = struct.unpack_from("<d", buf
120                                     , pos)
121         pos += 8
122         if field_number == 4:
123             unit_kg = value
124     elif wire_type == WIRE_I32:
125         (value,) = struct.unpack_from("<f", buf
126                                     , pos)
127         pos += 4
128         if field_number == 5:
129             width_mm = value
130     else:
131         raise DecodeError(f"unsupported_wire_
132                           type_{wire_type}")
133     return cls(part_id=part_id, qty=qty, delta=
134               delta,
135               unit_kg=unit_kg, width_mm=width_mm)
136
137 def _demo() -> None:
138     part = Part(part_id="A1", qty=300, delta=-3,
139                 unit_kg=1.5, width_mm=42.0)
140     wire = part.encode()
141     print(f"Encoded_{len(wire)}_bytes:")
142     for i, byte in enumerate(wire):
143         print(f"_{i:2d}:_{0x{byte:02X}}_{0b{byte
144               :08b}")
145
146     roundtrip = Part.decode(wire)
147     assert roundtrip == part, "round-trip_mismatch"
148     # The varint for qty=300 must be exactly the
149     # textbook pair 0xAC 0x02:
150     assert wire[5:7] == bytes([0xAC, 0x02])
151     # zigzag(-3) == 5 must occupy a single byte after
152     # the field-3 tag:
153     assert wire[7] == 0x18 and wire[8] == 0x05

```

```

145     # Unknown-field tolerance: append field 99 (VARINT)
        and re-decode.
146     tolerant = Part.decode(wire + make_tag(99,
        WIRE_VARINT) + encode_varint(7))
147     assert tolerant == part
148     print("Round-trip OK; varint, zigzag, and unknown-
        field checks passed.")
149
150
151 if __name__ == "__main__":
152     _demo()

```

Expected output (abridged): the encoder emits 23 bytes; offset 5–6 print 0xAC, 0x02 (the worked varint for 300); offset 8 prints 0x05 (zigzag of −3); all assertions pass.

5.3.2 JSON Schema Validation Pipeline with RFC 9457 Errors

Validating is easy; producing *actionable, standard-shaped* errors is the production part. This pipeline validates Northwind Part payloads against a 2020-12 schema and renders failures as an RFC 9457 application/problem+json document [Not23; JSO22].

```

1  """validate_parts.py -- JSON Schema 2020-12 validation
    -> RFC 9457 errors.
2
3  Requires: jsonschema >= 4.21 (see Python Dependencies
    Appendix).
4  Offline: the schema is an embedded fixture; no registry
    , no network.
5  """
6  from __future__ import annotations
7
8  import json
9  import logging
10 from typing import Any
11
12 from jsonschema import Draft202012Validator
13 from jsonschema.exceptions import SchemaError
14
15 logger = logging.getLogger("northwind.validation")
16
17 PART_SCHEMA: dict[str, Any] = {
18     "$schema": "https://json-schema.org/draft/2020-12/
        schema",

```



```

19     "$id": "https://schemas.northwind-robotics.example/
20         parts/v2/part.json",
21     "title": "Northwind_Robotics_Part",
22     "type": "object",
23     "properties": {
24         "part_id": {
25             "type": "string",
26             "pattern": "^NR-[0-9]{4}$",
27             "description": "Catalog identifier; string to
28                 avoid the 2^53 trap.",
29         },
30         "name": {"type": "string", "minLength": 1, "
31             maxLength": 120},
32         "qty": {"type": "integer", "minimum": 0},
33         "unit_price_usd": {"type": "number", "
34             exclusiveMinimum": 0},
35         "status": {"enum": ["active", "deprecated", "
36             eol"]},
37         "firmware": {"type": "string"},
38     },
39     "required": ["part_id", "name", "qty", "status"],
40     "additionalProperties": False,
41 }
42
43 def build_validator(schema: dict[str, Any]) ->
44     Draft202012Validator:
45     """Compile the schema once; fail fast if the schema
46         itself is invalid."""
47     try:
48         Draft202012Validator.check_schema(schema)
49     except SchemaError as exc:
50         raise ValueError(f"schema is not valid 2020-12:
51             {exc.message}") from exc
52     return Draft202012Validator(schema)
53
54 def to_problem_details(
55     validator: Draft202012Validator, instance: Any
56 ) -> dict[str, Any] | None:
57     """Return None if valid, else an RFC 9457 problem
58         document (status 422)."""
59     errors = sorted(validator.iter_errors(instance),
60         key=lambda e: list(e.path))
61     if not errors:
62         return None

```

```

55     return {
56         "type": "https://problems.northwind-robotics.
           example/schema-validation",
57         "title": "Payload does not conform to the Part
           schema",
58         "status": 422,
59         "detail": f"{len(errors)} schema violation(s)
           found.",
60         "instance": "/v2/parts",
61         "errors": [
62             {
63                 "pointer": "/" + "/".join(str(p) for p
           in e.absolute_path),
64                 "keyword": e.validator,
65                 "message": e.message,
66             }
67             for e in errors
68         ],
69     }
70
71
72 def _demo() -> None:
73     logging.basicConfig(level=logging.INFO, format="%(
           levelname)s %(message)s")
74     validator = build_validator(PART_SCHEMA)
75
76     good = {
77         "part_id": "NR-1001", "name": "Servo_Motor_42mm
           ",
78         "qty": 120, "unit_price_usd": 18.75, "status":
           "active",
79     }
80     bad = {
81         "part_id": "servo-1",           # violates
           pattern
82         "name": "",                     # violates
           minLength
83         "qty": -4,                       # violates
           minimum
84         "status": "retired",             # not in enum
85         "debug_note": "leftover",       #
           additionalProperties: false
86     }
87     assert to_problem_details(validator, good) is None
88     problem = to_problem_details(validator, bad)
89     assert problem is not None and problem["status"] ==

```

```

422
90 body = json.dumps(problem, indent=2)
91 logger.info("responding_422_with_application/
    problem+json")
92 print(body)
93 assert len(problem["errors"]) == 5 # one per
    violation, all collected
94
95
96 if __name__ == "__main__":
97     _demo()

```

Note the deliberate choices: the validator is compiled once (schema compilation is expensive; per-request compilation is an anti-pattern), `iter_errors` collects *all* violations, and each violation carries a JSON Pointer (`pointer`) so clients can highlight the offending field.

5.3.3 A Content-Negotiation Selector (Pure Standard Library)

The Accept algorithm from Section 5.2.6, implemented exactly, with unit-test-style asserts covering q-values, specificity, and q=0 refusal [Fie22a].

```

1  """negotiation.py -- RFC 9110 proactive content
    negotiation, pure stdlib."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass
5
6
7  @dataclass(frozen=True)
8  class MediaRange:
9      type: str
10     subtype: str
11     q: float = 1.0
12
13     def specificity(self) -> int:
14         """exact(2) > type/*(1) > */*(0); drives tie-
            breaking."""
15         if self.type == "*":
16             return 0
17         if self.subtype == "*":
18             return 1
19         return 2
20
21     def matches(self, media_type: str) -> bool:

```

```

22         t, _, s = media_type.partition("/")
23         return (self.type in ("*", t)) and (self.
24             subtype in ("*", s))
25
26 def parse_accept(header: str | None) -> list[MediaRange
27     ]:
28     """Parse an Accept header; absent or blank means
29     '*'/*' at q=1.0."""
30     if not header or not header.strip():
31         return [MediaRange("*", "*")]
32     ranges: list[MediaRange] = []
33     for raw in header.split(","):
34         parts = [p.strip() for p in raw.split(";")]
35         type_, _, subtype = parts[0].partition("/")
36         if not subtype:
37             continue # malformed entry: skip, do not
38                       # crash on client junk
39         q = 1.0
40         for param in parts[1:]:
41             key, _, value = param.partition("=")
42             if key.strip().lower() == "q":
43                 try:
44                     q = float(value)
45                 except ValueError:
46                     q = 0.0 # a non-numeric q is
47                           # treated as 'refuse'
48         q = min(max(q, 0.0), 1.0)
49         ranges.append(MediaRange(type_.strip().lower(),
50             subtype.strip().lower(), q))
51     return ranges or [MediaRange("*", "*")]
52
53 def select_representation(
54     accept_header: str | None, available: list[str]
55 ) -> str | None:
56     """Return the best available media type, or None
57     (-> HTTP 406).
58
59     Score = q of the MOST SPECIFIC matching range; ties
60     break toward
61     the server's own preference (the order of '
62     available').
63
64     """
65     if not available:

```

```

59         raise ValueError("available must contain at least one media type")
60     ranges = parse_accept(accept_header)
61     scored: list[tuple[float, int, int, str]] = []
62     for server_rank, media_type in enumerate(available):
63         matches = [r for r in ranges if r.matches(media_type)]
64         if not matches:
65             scored.append((0.0, 0, -server_rank, media_type))
66             continue
67         best = max(matches, key=lambda r: (r.specificity(), r.q))
68         scored.append((best.q, best.specificity(), -server_rank, media_type))
69     scored.sort(reverse=True)
70     top_q, _, _, winner = scored[0]
71     return winner if top_q > 0.0 else None
72
73
74 def _demo() -> None:
75     header = ("application/vnd.northwind.parts+json;v=2,"
76              "application/json;q=0.9,text/html;q=0.8,*/*;q=0.1")
77     server = ["application/json", "text/html"]
78     assert select_representation(header, server) == "application/json"
79     assert select_representation("text/html", server) == "text/html"
80     # q=0 on everything the server can do -> 406 (None)
81     assert select_representation("image/png,application/json;q=0",
82                                server) is None
83     # No Accept header -> server default (first listed)
84     assert select_representation(None, server) == "application/json"
85     # Specificity trap: JSON is scored by the MORE SPECIFIC application/*
86     # range (q=0.4), not by the /* range (q=0.9) -> HTML wins at 0.9.
87     assert select_representation("application/*;q=0.4,*/*;q=0.9",
88                                server) == "text/html"
89     print("All negotiation selector checks passed.")

```

```

90
91
92 if __name__ == "__main__":
93     _demo()

```

Warning

The final assert encodes the subtlest rule in RFC 9110 negotiation: a representation’s quality is taken from the *most specific* media range that matches it — not the highest-q range. Given `application/*;q=0.4`, `*/*;q=0.9`, JSON is scored 0.4 (the type-wildcard range) even though the full wildcard offers 0.9; HTML, matched only by `*/*`, keeps 0.9 and wins. A “max q anywhere” implementation silently inverts client intent here. This is exactly the kind of edge case your own selector must be unit-tested against.

5.3.4 A Deterministic Serialization Benchmark Harness

Benchmarks are where format debates go to be settled honestly. The harness below builds three deterministic payload shapes (small record, nested order, wide flat), round-trips each through JSON, the hand-rolled protobuf codec, and — if installed — MessagePack, and prints a size + timing table. Structure is fully deterministic (seeded `random`); timings are reported, never asserted.

```

1  """bench_serializers.py -- JSON vs hand-rolled Protobuf
   vs MessagePack.
2
3  Deterministic fixtures (seeded), fixed iteration counts
   , illustrative timing.
4  MessagePack is optional: if msgpack is not installed,
   its column is skipped.
5  Run: python bench_serializers.py
6  """
7  from __future__ import annotations
8
9  import json
10 import random
11 import time
12 from dataclasses import dataclass
13 from typing import Any, Callable
14
15 from mini_protobuf import Part # the hand-rolled codec
   from Listing 1

```

```

16 try:
17     import msgpack # type: ignore[import-not-found]
18 except ImportError: # pragma: no cover - optional
19     dependency
20     msgpack = None # type: ignore[assignment]
21
22 ITERATIONS = 5_000
23
24
25 def build_fixtures(seed: int = 42) -> dict[str, dict[
26     str, Any]]:
27     rng = random.Random(seed)
28     small = {"part_id": "NR-1001", "name": "Servo_Motor
29         _42mm",
30             "qty": 120, "unit_price_usd": 18.75, "
31             status": "active"}
32     nested = {
33         "order_id": "9007199254740991", # string on
34         purpose: see 2^53 rule
35         "customer": {"id": "C-88213", "region": "MX-NLE
36             "},
37         "lines": [{"part_id": f"NR-{1000_+_i}", "qty":
38             rng.randint(1, 50),
39             "unit_price_usd": round(rng.uniform
40                 (1, 200), 2)}
41             for i in range(12)],
42     }
43     wide = {f"metric_{i:02d}": round(rng.uniform(0,
44         1000), 4) for i in range(64)}
45     return {"small_record": small, "nested_order":
46         nested, "wide_flat": wide}
47
48 @dataclass(frozen=True)
49 class Codec:
50     name: str
51     encode: Callable[[dict[str, Any]], bytes]
52     decode: Callable[[bytes], Any]
53
54 def _json_codecs() -> Codec:
55     return Codec("json",
56         lambda d: json.dumps(d, separators=(",",
57             ":")).encode("utf-8"),
58         lambda b: json.loads(b.decode("utf-8"))
59     )

```

```

51         ))
52
53 def _msgpack_codecs() -> Codec | None:
54     if msgpack is None:
55         return None
56     return Codec("msgpack",
57                  lambda d: msgpack.packb(d,
58                                          use_bin_type=True),
59                  lambda b: msgpack.unpackb(b, raw=False)
60                  ))
61
62 def _proto_small_codec() -> Codec:
63     """Protobuf comparison for the small-record shape (
64     Part fields only)."""
65     def enc(d: dict[str, Any]) -> bytes:
66         return Part(part_id=d["part_id"], qty=d["qty"],
67                     unit_kg=d["unit_price_usd"]).encode()
68
69     def dec(b: bytes) -> Part:
70         return Part.decode(b)
71     return Codec("mini-protobuf", enc, dec)
72
73 def timed(fn: Callable[[], Any], iterations: int =
74     ITERATIONS) -> float:
75     """Median-ish wall time per call in microseconds (
76     illustrative only)."""
77     start = time.perf_counter()
78     for _ in range(iterations):
79         fn()
80     return (time.perf_counter() - start) * 1e6 /
81         iterations
82
83 def main() -> None:
84     fixtures = build_fixtures()
85     generic = [c for c in (_json_codecs(),
86                           _msgpack_codecs()) if c]
87     header = f"{'shape':<14}{'codec':<15}{'bytes':>8}{'
88         enc_us':>10}{'dec_us':>10}"
89     print(header)
90     print("-" * len(header))
91     for shape, doc in fixtures.items():

```



```

87     codecs = list(generic)
88     if shape == "small_record":
89         codecs.append(_proto_small_codec())
90     for codec in codecs:
91         wire = codec.encode(doc)
92         assert codec.decode(wire) is not None
93         enc_us = timed(lambda: codec.encode(doc))
94         dec_us = timed(lambda: codec.decode(wire))
95         print(f"{shape:<14}{codec.name:<15}{len(wire):>8}"
96               f"{enc_us:>10.2f}{dec_us:>10.2f}")
97
98
99 if __name__ == "__main__":
100     main()

```

The protobuf row covers only the small-record shape because our hand-rolled codec models exactly one message — extending it is Capstone 3. Sizes in Figure 5.1 come from this harness; expect the same ordering and different absolute numbers on your machine.

5.3.5 Streaming NDJSON Writer/Reader

A line-delimited pipeline with bounded memory: the writer emits one JSON value per line; the reader is a generator, so the consumer controls the pace — backpressure-friendly by construction. A malformed line is logged and skipped, not fatal: stream consumers must survive one poisoned record.

```

1  """ndjson_stream.py -- streaming JSON Lines writer/
   reader (stdlib only)."""
2  from __future__ import annotations
3
4  import io
5  import json
6  import logging
7  from typing import Any, Iterator, TextIO
8
9  logger = logging.getLogger("northwind.ndjson")
10
11
12 def write_ndjson(records: Iterator[dict[str, Any]],
13                 sink: TextIO) -> int:
14     """Write records as NDJSON; returns the number of
15     lines written."""
16     count = 0
17     for record in records:

```

```

16         line = json.dumps(record, separators=(",", ":"),
17                               ensure_ascii=False)
18         if "\n" in line: # json.dumps never emits raw
19                               newlines; guard anyway
20             raise ValueError("record_would_break_line_framing")
21         sink.write(line + "\n")
22         count += 1
23     sink.flush()
24     return count
25
26 def read_ndjson(source: TextIO, max_line_bytes: int = 1
27               _000_000) -> Iterator[Any]:
28     """Yield one decoded value per line. Bounded lines;
29       bad lines are skipped.
30
31       Backpressure: this is a generator -- if the
32       consumer is slow, no records
33       are pulled from 'source', so memory stays O(1) in
34       the number of records.
35     """
36     for lineno, raw in enumerate(source, start=1):
37         if len(raw) > max_line_bytes:
38             logger.error("line %d exceeds %d bytes;
39                           skipping",
40                           lineno, max_line_bytes)
41             continue
42         line = raw.strip()
43         if not line:
44             continue
45         try:
46             yield json.loads(line)
47         except json.JSONDecodeError as exc:
48             logger.error("line %d is not valid JSON (%s);
49                           skipping",
50                           lineno, exc.msg)
51
52 def _demo() -> None:
53     logging.basicConfig(level=logging.INFO, format="%(
54       levelname)s %(message)s")
55     records = (
56         {"part_id": f"NR-{1000+i}", "qty": i * 7, "
57           status": "active"}
58         for i in range(1, 4)

```

```

51     )
52     buffer = io.StringIO() # stands in for a socket or
    a log file
53     written = write_ndjson(records, buffer)
54     # Simulate one poisoned record appended after the
    clean batch:
55     buffer.write('{ "part_id": "BROKEN", "qty": ' )
56     buffer.seek(0)
57     decoded = list(read_ndjson(buffer))
58     assert written == 3
59     assert [r["part_id"] for r in decoded] == ["NR-1001",
    "NR-1002", "NR-1003"]
60     assert sum(r["qty"] for r in decoded) == 7 + 14 +
    21
61     print(f"Wrote {written} lines; recovered {len(
    decoded)} of "
62           f"{written+1} (1 poisoned line skipped).")
63
64
65 if __name__ == "__main__":
66     _demo()

```

Key Concept

Backpressure in a pull-based pipeline is free: because `read_ndjson` is a generator, records flow only as fast as the consumer iterates. The expensive resources — the file handle, the socket buffer — are held open but never inflated. The dual discipline is on the writer: flush deliberately, and never emit a record you have not finished framing (a torn write mid-line is the classic NDJSON corruption when a process is killed; write-then-fsync per batch, or use atomic file rotation as in Capstone 1).

5.4 Common Pitfalls & Anti-Patterns

Each entry below is a defect class observed repeatedly in production systems, with its mechanism and its fix.

1. **Floats for money.** `unit_price_usd: 19.99` is a binary approximation: 19.99 has no finite binary expansion, and `json.loads("19.99") == 19.99` is true only because Python's repr round-trips; arithmetic (`0.1 + 0.2 != 0.3`) does not. **Fix:** transmit money as integer minor units

(`unit_price_cents: 1999`) or as a decimal string parsed into `decimal.Decimal`; in protobuf use a `Money`-style message, never `float/double`.

2. Trusting Content-Type without sniffing the body. The header is a claim, not a fact. A compromised or buggy upstream can label executable HTML as `application/json` and hand your parser — or your user's browser — a different document than promised. **Fix:** validate the body against the declared type (parse strictly, schema-validate at trust boundaries), and send `X-Content-Type-Options: nosniff` on responses so browsers do not second-guess you.

3. Reusing a protobuf tag number. The decoder resolves meaning solely by tag; a recycled tag makes old data silently parse as a new field (Section 5.5 case study). **Fix:** `reserved` statements, lint rules in CI, and a schema registry that diffs tag allocations.

4. Unbounded decompression. `gzip.decompress(body)` on attacker-controlled input is a memory-exhaustion primitive: a 42 KB bomb expands to 4.5 PB in the famous `42.zip`; even honest 10 GB log archives arrive mislabeled. **Fix:** stream through `gzip.GzipFile/zstandard` readers and abort when the cumulative decompressed size crosses a configured ceiling; reject `Content-Encoding` you did not negotiate.

5. Assuming JSON object key order. RFC 8259 defines objects as *unordered* [Bra17]. Python's `dict` preserves insertion order, which makes order-dependent tests pass in Python and fail against a Go intermediary that re-serializes with sorted keys, or a CDN that normalizes payloads. **Fix:** never key logic on member order; if a signature must cover the body, canonicalize (e.g. JCS, RFC 8785) before hashing.

6. Silent numeric coercion of identifiers. Serializing a database `BIGINT` primary key as a JSON number invites the 2^{53} corruption from Section 5.2.1. ORM defaults do this to you without asking. **Fix:** identifiers are strings by policy; schemas pin `"type": "string"` with a pattern; linters reject `integer` schema types on `*_id` fields unless `maximum ≤ 253 - 1` is declared.

7. default as a data-filling mechanism. JSON Schema's `default` is an *annotation*; validators do not mutate instances (Section 5.2.2). Pipelines that assume defaults were applied read `None` downstream. **Fix:** apply defaults in application code or with an explicit coercion pass; never rely on the validator to fix data.

8. Catching everything around the parser. `except Exception:` `return {}` converts a malformed-payload bug into a phantom empty document, corrupting state silently. **Fix:** catch `json.JSONDecodeError` (or your codec's specific error), map to a 400/422 problem document [Not23], and let truly unexpected exceptions surface.

5.5 Hands-On Production Lab & Real-World Case Study

5.5.1 Lab: Benchmark Three Formats and Produce a Decision Table

Objective. Replace opinion with measurement: quantify JSON vs. protobuf vs. MessagePack on *your* payload shapes and turn the result into a selection table your team could actually adopt.

Setup (PowerShell).

```
1 cd "C:\Training\Training Areas\Data jobs learning\APIs
   - Usage and development\code"
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 pip install -r requirements.txt # jsonschema, msgpack
   (orjson optional)
5 python bench_serializers.py
```

Steps.

1. Run the harness from Listing in Section 5.3.4 unmodified; record the size column for all three shapes.
2. Extend the fixtures: add a `deep_tree` shape (6 levels of nesting) and a `text_heavy` shape (8 KB of log message strings). Re-run.
3. Extend the protobuf comparison: write a second hand-rolled codec for the `wide_flat` shape as a `repeated double` packed field (wire type 2, one tag, then a length, then contiguous 8-byte values). Observe what packed encoding does to size and decode time.
4. Add gzip on top of each format (`gzip.compress(wire, 9)`) and record the compressed sizes. Note which format compresses *least* proportionally and explain why from the byte layouts you now understand.
5. Produce a decision table with one row per shape and columns: raw size per format, gzip size per format, encode/decode time rank, debugga-

bility note, and your recommended format for that shape *in a named Northwind service* (e.g. public catalog API vs. internal telemetry bus).

Acceptance criteria. Your table covers ≥ 5 shapes $\times$ 3 formats; every claim in the “recommended” column cites a number from your own run; the report includes one paragraph explaining a result that surprised you.

5.5.2 Case Study: The 2^{53} Order-ID Corruption Incident

Northwind Robotics’ orders service stores `order_id` as a 64-bit integer (a `BIGINT` identity column). The Java backend serializes it as a JSON number — correctly, per its own parser, which handles 64-bit integers natively. Years pass. Then order volumes cross 2^{53} , and the React storefront begins silently routing customers to the wrong order-status pages: JavaScript’s `JSON.parse` reads every ID through an IEEE 754 double, and 9007199254740993 and 9007199254740992 collapse to the same value (Section 5.2.1). No errors, no alerts; the corruption is discovered by a customer who reports seeing a stranger’s shipment.

The postmortem’s lessons, in the team’s own remediation order: (1) identifiers are *opaque strings* — the API contract changed to `"order_id": "9007199254740993"`, shipped behind a vendor media type bump `application/vnd.northwind.` with the negotiation machinery of Section 5.2.6 serving both versions during the migration window; (2) the JSON Schema now declares every `*_id` as a string with a pattern, enforced in CI by the pipeline from Section 5.3.2; (3) a synthetic *canary assertion* in staging posts IDs near $2^{53} \pm k$ and diffs what the browser receives; (4) the team adopted a policy that numeric JSON fields carry explicit `minimum`/`maximum` bounds, making the safe range part of the reviewed contract rather than an implicit hope.

Warning

The incident’s root cause was not “JavaScript is bad at integers.” It was a contract that encoded a 64-bit semantic type into a format whose common denominator cannot carry it, with no schema and no negotiation to catch the mismatch. Serialization choices are contract choices.

5.6 Self-Assessment Exercises & Deep-Dive Questions

E4.1 Write out, by hand, the varint encoding of 1, 127, 128, and 16,384. Verify each with `encode_varint` from the hand-rolled codec.

- E4.2** Compute `zigzag(-64)` and `zigzag-1(127)` without a computer; then check with `zigzag_encode/zigzag_decode`.
- E4.3** A colleague proposes changing field 4 of `Part` from `double` (I64) to `int64` (VARINT) “to save bytes.” Using only the wire-type table, explain what old consumers will do with new data and why this violates the evolution rules.
- E4.4** Extend the JSON Schema fixture so that `qty` is required *only when* `status` is `"active"` (hint: `if/then`). Validate three test instances covering both branches.
- E4.5** Show that `additionalProperties: false` inside an `allOf`-composed schema rejects legitimate extended fields, then fix it with `unevaluatedProperties`. Capture before/after validator output.
- E4.6** Given `Accept: text/*;q=0.3, text/html;q=0.7, */*;q=0.5`, compute by hand the score a server should assign to `text/html` and to `application/json`; confirm with the negotiation selector.
- E4.7** Instrument the NDJSON reader to measure lines-per-second on a 1,000,000 record file you generate deterministically; then make the generator *stop early* after 100 records and confirm the file is never read past line 101.
- E4.8** Construct the smallest NDJSON stream that corrupts under naive string-concatenation writing (torn final line) and show that the reader from Section 5.3.5 recovers all intact records.
- E4.9** Write a function that detects duplicate keys in a JSON document using `object_pairs_hook`, and use it to demonstrate a first-wins vs. last-wins parser differential on `{"a": 1, "a": 2}`.
- E4.10** Prove with a one-line Python expression that $2^{53} + 1$ and $2^{53} + 2$ are distinguishable as Python ints but not after a round-trip through `float`. Explain which JSON parser default this simulates.
- E4.11** Avro promotes `int` $\rightarrow$ `long` during schema resolution. Explain why the reverse (`long` $\rightarrow$ `int`) is rejected even when all live values fit, connecting your answer to the definition of forward compatibility.
- E4.12** A CDN serves a cached v1 JSON body to a client that asked for v=2 via a media type parameter. Which response header, missing at the origin, permitted this, and what value should it carry?

5.7 Interview Q&A Vault

Thirty-six questions, ordered junior → staff. Answer aloud before reading.

Q1. What are the six JSON value types?

Object, array, string, number, `true`, `false`, `null` — that is the entire grammar of values per RFC 8259 [Bra17]. There is no date, no binary, no undefined, no comment type; every richer type is a convention layered on strings or numbers.

Q2. Why can't JSON represent all 64-bit integers safely?

RFC 8259 standardizes number *syntax*, not precision; mainstream parsers (JavaScript, and anything round-tripping through doubles) use IEEE 754 binary64, whose exact-integer ceiling is 2^{53} . Integers beyond it alias to a representable neighbor — silently. Contracts therefore carry 64-bit IDs as strings, or constrain numeric ranges explicitly.

Q3. What does RFC 8259 say about duplicate object keys?

Names SHOULD be unique, and behavior when they are not is undefined. Python and JavaScript keep the last value; some parsers keep the first; strict parsers reject. Crossing a proxy/backend pair with different policies yields a parser differential — an attack surface. Detect duplicates with `object_pairs_hook` at trust boundaries.

Q4. Does JSON allow comments or trailing commas?

No. Both are syntax errors. Config-file dialects (JSONC, JSON5) add them, but a strict RFC 8259 parser — which is what your API clients run — rejects both. Never hand-assemble payloads with string templating; use a serializer.

Q5. What encoding must JSON use on the wire?

UTF-8. RFC 8259 removed UTF-16/UTF-32 for interchange. Byte-order marks are forbidden, and strings may carry `\uXXXX` escapes, with non-BMP code points expressed as UTF-16 surrogate pairs — which strict receivers must validate as pairs.

Q6. JSON vs. NDJSON — when do you choose each?

One JSON document buffers to its closing brace; NDJSON frames one value per line, giving constant-memory streaming, append-friendly logs, and resumable partial reads. Choose a single document for request/response bodies with one coherent shape; choose NDJSON for feeds, logs, and bulk export. Validate per line, since lines share no enforced shape.

Q7. What is the difference between JSON Pointer and JSON Path?

JSON Pointer (RFC 6901) is a strict address of exactly one value (`/lines/0/part_id`); JSON Path is a query language with wildcards and filters (`$.lines[?(@.qty>10)]`). Use the former for error locations and references, the latter for selection — and pin the library version, since JSON Path semantics only recently standardized.

Q8. What is JSON Schema, and what is the 2020-12 dialect?

A JSON vocabulary for constraining JSON instances; 2020-12 is the stable dialect used by OpenAPI 3.1 [[JSO22](#)]. It distinguishes assertions (enforce), applicators (compose subschemas), and annotations (metadata).

Q9. Explain `required` vs. `properties`.

`properties` constrains values *if the key is present*; `required` constrains presence. A key listed in `properties` but not `required` may be absent; a required key with no `properties` entry is unconstrained. Both are needed for a tight object contract.

Q10. `additionalProperties: false` vs. `unevaluatedProperties: false`?

The former sees only keys declared in the same schema object, so it breaks under `allOf` composition; the latter runs after all applicators and sees the union of evaluated keys. For closed-but-composable schemas, use `unevaluatedProperties`.

Q11. Is the `default` keyword applied during validation?

No — it is an annotation. Validators report validity; they never mutate the instance. Apply defaults in application code or a separate coercion pass.

Q12. What is `$ref` and how do you avoid its common traps?

`$ref` inlines another schema by URI/fragment, enabling reuse via `$defs`. Traps: sibling keywords alongside `$ref` are ignored in older drafts (allowed in 2020-12 but confusing); unresolvable remote references make validation network-dependent — keep references local in offline pipelines.

Q13. `oneOf` vs. `anyOf` vs. `allOf`?

Exactly one, at least one, and all subschemas must validate, respectively. `oneOf` is the right tool for tagged unions; misusing `anyOf` for unions accepts instances matching multiple branches and hides ambiguity.

Q14. Walk me through protobuf varint encoding of the number 300.

$300 = 2 \cdot 2^7 + 44$; low 7 bits first: 44 with the continuation bit set (0xAC), then 2 as the final byte (0x02). Decoder strips continuation bits and reassembles $\sum b_i \cdot 2^{7i}$.

Q15. Why does protobuf have zigzag encoding and the sint types?

Varint cost grows with magnitude; a negative int32 sign-extends to 64 bits and costs ten bytes. Zigzag folds sign into magnitude ($n \mapsto (n \ll 1) \oplus (n \gg 63)$) so -1 encodes as 1, one byte. Use sint32/sint64 for signed fields that go negative.

Q16. How is a protobuf tag computed, and what are wire types 0/1/2/5?

$\text{tag} = (\text{field_number} \ll 3) | \text{wire_type}$. Type 0: varint (ints, bools, enums); 1: 64-bit fixed (double, fixed64); 2: length-delimited (strings, bytes, sub-messages, packed repeated); 5: 32-bit fixed (float, fixed32). Types 3/4 (groups) are deprecated [Goo24].

Q17. What breaks when you reuse a protobuf tag number?

Decoders resolve meaning by tag alone. A consumer built against the old schema reads the new field's bytes *as the old field* — no error, wrong data. With a compatible wire type the corruption is silent and plausible-looking. Mitigations: never reuse; reserved declarations; registry-enforced diff checks.

Q18. Adding vs. removing fields in proto3 — what's safe?

Adding a field is always safe: old readers skip unknown tags. Removing is safe only if no consumer depends on it and the tag is reserved afterward. Renaming a field is wire-safe (names aren't on the wire) but breaks generated code and JSON mappings.

Q19. How does Avro's schema evolution differ from protobuf's?

Protobuf matches fields by tag number; Avro matches by name and resolves a writer schema against a reader schema with promotion rules ($\text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$) and reader defaults. Avro payloads carry no tags, so the writer schema must be available out of band (container file header or registry ID).

Q20. Describe MessagePack's encoding strategy in one minute.

Self-describing binary JSON: every value starts with a type-prefix byte; small integers, short strings, arrays, and maps embed their

length or value in the prefix itself (fixint/fixstr/fixarray/fixmap families, e.g. 0x80–0x8F for maps up to 15 pairs). Larger values use dedicated prefix codes (0xCD = uint16 follows). It keeps JSON’s schema-less flexibility, drops some text overhead, and keeps field names on the wire.

Q21. Protobuf vs. Avro for a Kafka pipeline?

Both work with a registry. Avro’s name-based resolution and defaults are idiomatic in the Kafka ecosystem and handle reader-driven evolution smoothly; protobuf gives smaller, tag-driven payloads and first-class gRPC synergy. The deciding factors are usually ecosystem (what your registry and stream tooling support) and whether you need protobuf’s RPC toolchain [Kle17].

Q22. Why is JSON still the right default for public APIs?

Ubiquity: browsers parse it natively, every language has a parser, humans can debug it with `curl`, and tooling (OpenAPI, JSON Schema) is deepest. Its costs — size, parse speed, number precision — are real but rarely dominant at public-API latencies, and can be bounded with schema discipline and compression.

Q23. How would you debug a protobuf message you captured on the wire?

Without the schema you can still walk tags: split each varint into field number and wire type, decode varints and length-delimited chunks, and render a structural dump (`protoc --decode_raw` does exactly this). Field *names* are unrecoverable — they were never transmitted; semantic debugging needs the `.proto` that matches the tag allocation.

Q24. Explain the Accept q-value selection algorithm.

Parse ranges with q (default 1.0). For each producible representation, take the most specific matching range — exact beats `type/*` beats `*/*` — and assign its q ; $q=0$ refuses. Sort by q , tie-break by specificity, then server preference; no positive- q match yields 406. The subtlety: a specific low- q range overrides a wildcard high- q range *for that representation* [Fie22a].

Q25. What is 406 Not Acceptable and when should you return it?

When no producible representation has $q > 0$ against the client’s `Accept`. Return it with a body listing available types rather than silently serving a default the client may mishandle. Many public

APIs choose the pragmatic default instead; if so, document it.

Q26. What's in a media type beyond type/subtype?

Parameters after semicolons: `charset`, `profile`, and vendor parameters like `v=2`. They are part of the type for matching; a range with parameters does not match a representation lacking them. The `+json` structured-syntax suffix lets generic JSON tooling handle unknown vendor types.

Q27. Design a vendor media type for a versioned parts API.

`application/vnd.northwind.parts+json;v=2`: vendor tree for ownership, `+json` suffix for tooling, parameter for contract version. Keep the URL stable; negotiate representation via `Accept`; emit `Vary: Accept` so caches key variants correctly; document the deprecation schedule per version.

Q28. gzip vs. Brotli vs. zstd — how do you choose?

`gzip` is universal and adequate; `Brotli` compresses text 15–25 % smaller at similar decode cost, ideal for static/cacheable web payloads; `zstd` offers fast decode and tunable levels, strong for service-to-service and log pipelines. The header `Accept-Encoding` plus `q`-values lets the client decide; enforce minimum body sizes so small payloads aren't *grown* by framing overhead.

Q29. What is a decompression bomb and how do you defend against it?

A small compressed payload expanding to gigabytes, exhausting memory. Defend by streaming decompression with a cumulative byte ceiling, capping compression ratio, and refusing `Content-Encoding` values you didn't negotiate. Never call one-shot `decompress` on untrusted input.

Q30. Why must caches see `Vary: Accept` on negotiated resources?

Otherwise a shared cache may replay a stored JSON variant to a client that requested HTML or a `v2` vendor type — a silent contract violation. `Vary` lists the request headers that select the variant; get it wrong and every downstream cache becomes a correctness bug.

Q31. Define backward, forward, and full compatibility precisely.

Backward: new consumers read old data. Forward: old consumers read new data. Full: both. Each permits a different edit set (e.g. adding a field with a default is backward; deleting an unused field is forward), and the mode your registry enforces dictates the safe

deployment order of producers and consumers.

Q32. How does a schema registry enforce compatibility?

It versions schemas, assigns IDs referenced by messages (e.g. a registry-ID prefix on each Avro record), and gates publication: a new version is rejected unless it passes the configured mode (backward/forward/full, optionally transitive) against prior versions. This turns evolution policy into a CI check instead of a wiki page.

Q33. JSON Schema vs. protobuf IDL — when each?

JSON Schema validates documents you already exchange as JSON: public REST APIs, config files, NDJSON feeds; it's descriptive, rich in string/numeric constraints, and needs no codegen. Protobuf IDL is generative: it defines the wire encoding itself, yields typed codecs in many languages, and suits internal high-throughput RPC. Many organizations use both — JSON Schema at the edge, protobuf on the inside [[JSO22](#); [Goo24](#)].

Q34. Your p99 latency regressed after switching a hot endpoint to a “faster” binary format. What do you check?

Whether the benchmark measured steady state or cold JIT/warmup; allocation churn in the codec; gzip on top of binary erasing gains while adding CPU; payload shape (text-heavy payloads compress to parity); Nagle/flush behavior on smaller writes; and whether validation moved, not vanished — e.g. schema validation now happens after decode instead of during parse.

Q35. How would you migrate a public API's payload from JSON to protobuf without breaking existing clients?

You don't migrate — you *add*. Serve both representations behind content negotiation with a vendor media type, keep JSON the default, publish the protobuf type as opt-in, measure adoption via negotiated-type logs, and set a sunset date. The negotiation machinery is the migration machinery.

Q36. (Staff) A consumer team's parser keeps the *first* duplicate key; yours keeps the *last*. An auditor shows a payload where the difference changes a price. Who owns the fix and how do you close the class?

The producer owns it: emitting duplicates is already outside RFC 8259's interoperability guidance. Immediate: reject duplicates at the ingress boundary (`object_pairs_hook` raising on repeats) so no document is ambiguous internally. Structural: make signing/canonicalization

cover the raw bytes, add a duplicate-detection contract test to every consumer registry entry, and publish a parser-conformance note in the API’s versioning policy.

Q37. (Staff) When would you *forbid* a binary format even though it’s smaller and faster?

When the consuming ecosystem can’t carry the contract: browsers without a protoc runtime, third parties who must debug with curl, regulatory requirements for human-readable archives, or teams without schema-registry discipline — a protobuf pipeline without tag governance is a corruption engine. Format choice is a socio-technical decision; the matrix includes your organization’s ability to operate the contract machinery.

5.8 External Bibliography & Further Reading

- T. Bray, *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017 — the normative grammar; read Section 6 (numbers) and Section 8 (strings) until you can recite their limits [Bra17].
- R. Fielding, M. Nottingham, J. Reschke, *HTTP Semantics*, RFC 9110, IETF, 2022 — Section 12 (Content Negotiation) and Section 8 (Content Codings) are the reference for everything in Section 5.2.6 [Fie22a].
- *JSON Schema Specification (2020-12)* and documentation, <https://json-schema.org/specification> — the validation and applicator vocabularies, plus the official test suite [JSO22].
- Google, *Protocol Buffers Documentation*, <https://protobuf.dev/> — the “Encoding” and “Proto3 Language Guide” pages; the varint/zigzag tables there are the authority behind Section 5.2.3 [Goo24].
- M. Nottingham, E. Wilde, S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023 — the error-report shape emitted by the validation pipeline [Not23].
- M. Kleppmann, *Designing Data-Intensive Applications*, O’Reilly, 2017 — Chapter 4 (“Encoding and Evolution”) is the canonical treatment of schema evolution across protobuf, Avro, and friends [Kle17].
- *Apache Avro Specification*, <https://avro.apache.org/docs/> — bi-

nary encoding and the formal Schema Resolution rules (not in this book's shared bibliography; read online).

- *MessagePack Specification*, <https://github.com/msgpack/msgpack/blob/master/spec.md> — the complete type-prefix byte table used in Section 5.2.4.
- IANA, *Media Types Registry*, <https://www.iana.org/assignments/media-types/> — the authoritative registry for `type/subtype` names, the `vnd.` tree, and structured syntax suffixes.
- *JSON Lines* (<https://jsonlines.org/>) and RFC 7464 (*JSON Text Sequences*) — the two line/sequence framing conventions for streaming JSON.
- A. Böhm, M. Sporny et al., *JSON Canonicalization Scheme (JCS)*, RFC 8785 — for the signing-before-hashing use case raised in Section 5.4.

5.9 Capstone Projects

Exactly five projects, progressive. All are offline, deterministic, and use only synthetic Northwind Robotics data. Place deliverables under `capstones\capstone_NN_<slug>\` per the workspace conventions.

5.9.1 Capstone 1: NDJSON Inventory Log Shipper [junior]

Objective. Build a robust local log shipper: watch a directory of NDJSON inventory snapshots, validate every line against a `stock-event` JSON Schema, and copy clean batches into an output directory with atomic rotation.

Inputs/Datasets. A generated fixture of 10,000 synthetic stock events (seeded), with 0.5% deliberately poisoned lines (bad enum, torn JSON, wrong type).

Steps. (1) Generate fixtures deterministically. (2) Write the schema and validator. (3) Implement line-at-a-time reading with a byte ceiling. (4) Quarantine bad lines to `quarantine.ndjson` with line numbers. (5) Rotate output files at 1,000 records, writing `.tmp` then renaming.

Acceptance Criteria.

- All 9,950 clean records shipped; exactly the poisoned set quarantined; run is reproducible from the seed.

- ☐ Memory stays bounded (process RSS flat over the run).
- ☐ Killing the process mid-batch never leaves a torn final line in a `.ndjson` file (only in `.tmp`).

Stretch Goals. Gzip each rotated batch; emit a summary problem document (RFC 9457 shape) for the quarantine set.

5.9.2 Capstone 2: Schema-Validation Middleware [mid]

Objective. Package the Section 5.3.2 pipeline as a reusable WSGI-style middleware that validates request bodies against per-route JSON Schemas and returns RFC 9457 problems with JSON Pointers.

Inputs/Datasets. A route table mapping three synthetic endpoints (`/parts`, `/orders`, `/warehouses`) to embedded schemas; a deterministic corpus of 200 valid and 200 invalid bodies.

Steps. (1) Compile validators once at startup. (2) Intercept `POST/PUT` bodies; enforce a body-size cap. (3) Map error trees to problem documents. (4) Log validation outcomes structurally (route, keyword, pointer) without logging bodies.

Acceptance Criteria.

- ☐ Every invalid body yields 422 with `application/problem+json` and a complete `errors` array.
- ☐ Valid bodies pass through byte-identical; zero false rejections on the corpus.
- ☐ Validation overhead measured and reported; no per-request schema recompilation.

Stretch Goals. Content-negotiate the error body (JSON vs. HTML problem rendering) using the Section 5.3.3 selector.

5.9.3 Capstone 3: Hand-Rolled Protobuf Codec Extension [senior]

Objective. Extend the mini codec into a two-message system: add a `StockEvent` message with `repeated uint32 warehouse_ids` (packed, wire type 2), an `enum` status, and an embedded `Part` sub-message; then implement unknown-field retention for forward compatibility.

Inputs/Datasets. The `Part` message from this chapter; a seeded generator producing 5,000 stock events.

Steps. (1) Design the `.proto`-style IDL and document tag allocations. (2) Implement packed-repeated encode/decode. (3) Implement `enum` and

sub-message paths. (4) Store unknown fields and re-emit them on re-encode. (5) Write a schema-evolution test: a “v1 decoder” reading v2 bytes must not crash and must round-trip unknown fields.

Acceptance Criteria.

- ☐ Byte-level golden tests for at least five messages (expected hex committed).
- ☐ Unknown fields survive a decode/encode round trip byte-identical.
- ☐ `reserved` rules documented; a CI-style lint rejects tag reuse in the IDL document.

Stretch Goals. Add zigzag `sint64` deltas and benchmark packed vs. non-packed repeated encodings.

5.9.4 Capstone 4: Negotiation-Aware Cache Key Tool [senior]

Objective. Build a CLI that, given recorded HTTP request/response pairs, computes correct cache keys that account for `Accept`, `Accept-Encoding`, and the response’s `Vary` header — and flags responses whose `Vary` is missing or wrong for their negotiated dimensions.

Inputs/Datasets. A synthetic capture file (NDJSON) of 1,000 exchanges across JSON/HTML/vendor types and gzip/br/identity codings, with 5 % deliberately mis-`Varyed` responses.

Steps. (1) Parse captures. (2) Normalize negotiation headers per RFC 9110. (3) Derive the variant key from `Vary`. (4) Report collisions: two responses with the same key but different bodies. (5) Emit a fix-list of missing `Vary` directives.

Acceptance Criteria.

- ☐ All seeded mis-`Varyed` responses detected; no false positives on the clean set.
- ☐ Variant keys are stable across runs (deterministic ordering).
- ☐ Report explains each collision in terms of the negotiation algorithm from Section 5.2.6.

Stretch Goals. Simulate a second-level cache and quantify hit-rate loss from overly broad `Vary` (e.g. `Vary: User-Agent`).

5.9.5 Capstone 5: Format-Migration CLI for a Dataset [staff]

Objective. Design and implement a migration tool that converts the Northwind parts catalog dataset between JSON, NDJSON, MessagePack, and the hand-rolled protobuf encoding — with a versioned schema manifest, compatibility checking (backward/forward/full), and a dry-run impact report.

Inputs/Datasets. A 50,000-record synthetic catalog (seeded) in v1 JSON; a v2 schema adding an optional field and deprecating another; your Capstone 3 codec.

Steps. (1) Author v1/v2 schemas and a manifest recording compatibility mode per pair. (2) Implement converters for all four format pairs with streaming (constant memory). (3) Run the compatibility gate before any conversion. (4) Produce a diff report: record count, size delta, field-level changes, incompatibilities blocked. (5) Add a **verify** mode that round-trips and asserts semantic equality.

Acceptance Criteria.

- ☐ All 4×4 conversions run offline and stream in bounded memory.
- ☐ The gate correctly blocks a seeded backward-incompatible edit and permits the compatible one, with precise reasons.
- ☐ **verify** proves semantic equality (not byte equality) across every format pair, including float-tolerant comparison.
- ☐ The impact report's size deltas match the benchmark harness within measurement noise.

Stretch Goals. Emit the manifest as a machine-readable registry document; add a chaos mode that truncates the output mid-stream and prove **verify** catches it.

5.10 Python Dependencies Appendix

Everything in this chapter pins to a minimal `requirements.txt`:

```
1 jsonschema>=4.21,<5.0
2 msgpack>=1.0,<2.0
3 orjson>=3.9,<4.0      # optional: fast JSON for the
                        benchmark's extra row
```

Install on Windows/PowerShell:

```
1 python -m venv .venv
```

```
2 | .\venv\Scripts\Activate.ps1
3 | pip install -r requirements.txt
```

5.10.1 jsonschema

Description. The reference Python implementation of JSON Schema, supporting drafts through 2020-12 with pluggable format checkers.

Why included. Powers the validation pipeline (Section 5.3.2) and Capstones 1–2; its `Draft202012Validator` and `iter_errors` give the complete error tree needed for RFC 9457 reporting.

Alternatives. `fastjsonschema` (compiles schemas to code; faster, narrower error detail); `pydantic` (model-centric validation — excellent, but a different paradigm from document-centric JSON Schema).

Installation. `pip install "jsonschema>=4.21,<5.0"`.

5.10.2 msgpack

Description. Binary MessagePack encoder/decoder with a C-accelerated backend.

Why included. Supplies the MessagePack column of the benchmark harness and the format-migration capstone; the chapter’s code degrades gracefully if it is absent.

Alternatives. `msgpack` is effectively the standard; pure-Python fallbacks exist but skew benchmarks. For protobuf comparisons on real projects, `protobuf` (generated code) is the counterpart — deliberately *not* used here so the wire format stays visible.

Installation. `pip install "msgpack>=1.0,<2.0"`.

5.10.3 orjson (optional)

Description. A Rust-backed JSON library, typically 2–5× faster than `json` on realistic payloads, with strict UTF-8 and RFC 8259 conformance.

Why included. Lets the benchmark answer “is my bottleneck the format or the library?” — an honest performance conversation requires a fast JSON baseline. Optional: the harness skips it if missing.

Alternatives. `ujson` (fast but historically laxer about edge conformance); the stdlib `json` module (correct, universal, slowest).

Installation. `pip install "orjson>=3.9,<4.0"`.

5.11 Chapter Summary & Key Takeaways

- JSON (RFC 8259) is six types, UTF-8-only on the wire, no comments, no trailing commas — and deliberately silent on number range and duplicate keys. Those two silences cause real incidents; design contracts that close them [Bra17].
- IEEE 754 doubles pin JSON’s exact-integer ceiling at 2^{53} . Identifiers and money are strings or scaled integers; never trust a JSON number to carry a 64-bit identity.
- JSON Schema 2020-12 turns payloads into machine-checkable contracts: assertions enforce, annotations describe, `unevaluatedProperties` closes composed objects, and validators collect complete error trees that map cleanly to RFC 9457 problems [JSO22; Not23].
- Protobuf’s wire format is a triumph of minimalism: $\text{tag} = (\text{field} \ll 3) \mid \text{wire_type}$, varints, zigzag, and four live wire types. Its evolution rules — never reuse tags, reserve deleted ones, add freely — are non-negotiable [Goo24].
- Avro resolves writer against reader schemas by name; MessagePack is schema-less binary JSON. Choose with the trade-off matrix, then verify with your own benchmark harness.
- Content negotiation is an algorithm, not a vibe: q-values, specificity, 406, Vary, vendor media types, and bounded decompression are the moving parts [Fie22a].
- Compatibility modes (backward/forward/full), schema registries, and consumer-driven contracts (Chapter 19) are how contracts survive contact with change.

5.12 Quick-Reference Card

Format Comparison Matrix

Negotiation Headers Cheat-Sheet

Protobuf Evolution Rules

1. Never reuse a tag number — tags are permanent identities.
2. Delete fields via `reserved N;` and `reserved "name";`.

3. Add fields freely; old readers skip unknown tags.
4. Change types only within wire-compatible families (`int32` $\rightarrow$ `int64`; never across wire types).
5. Use `sint32/sint64` for values that go negative (zigzag).
6. In `proto3`, scalars default to zero and vanish from the wire; use `optional` for explicit presence.
7. Let a registry or lint gate enforce all of the above in CI.

5.13 Standardized Evidence Addendum

5.13.1 Benchmark Snapshot Template

5.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

5.13.3 Cost and Latency Budget Snapshot

5.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

5.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

5.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

5.17 Cross-Chapter Decision Card

5.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

5.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Keyword	Kind	Semantics
<code>type</code>	assertion	Instance must be one of the six JSON types (or a list of them).
<code>properties</code>	applicator	Per-key subschemas for named object members.
<code>required</code>	assertion	Array of member names that must be present.
<code>additionalProperties</code>	applicator	Schema for members not matched by <code>properties/patternProperties</code> ; <code>false</code> forbids extras.
<code>\$ref</code>	reference	Inline the schema at a URI/fragment (reusable definitions via <code>\$defs</code>).
<code>allOf</code>	applicator	Instance must validate against <i>every</i> subschema (conjunction; used for composition).
<code>anyOf</code>	applicator	Instance must validate against <i>at least one</i> subschema.
<code>oneOf</code>	applicator	Instance must validate against <i>exactly one</i> subschema.
<code>unevaluatedProperties</code>	applicator	Schema for members not evaluated by any in-place applicator — the <code>allOf</code> -aware way to close an object.
<code>enum / const</code>	assertion	Membership in a value set / equality to one value.
<code>minimum, maximum, multipleOf</code>	assertion	Numeric bounds and divisibility.
<code>minLength, maxLength, pattern</code>	assertion	String length and regular-expression constraints.
<code>format</code>	annotation*	Vocabulary hint (<code>date-time</code> , <code>uri</code> , <code>uuid</code>); in 2020-12 it is an annotation unless the implementation opts into assertion.
<code>title, description, examples</code>	annotation	Documentation only; never affect validity.

Table 5.1: Core JSON Schema 2020-12 keywords used throughout this chapter.

Wire type	Name	Used for	Pay
0	VARINT	int32, int64, uint32, uint64, sint32, sint64, bool, enum	1–10 vari byte Exa 8 byte little endi
1	I64	fixed64, sfixed64, double	Var leng <i>n</i> , ther <i>n</i> byte Do not use. Exa 4 byte little endi
2	LEN	string, bytes, embedded messages, packed repeated fields	
3, 4	SGROUP, EGROUP	Deprecated groups	
5	I32	fixed32, sfixed32, float	

Table 5.2: Protocol Buffers wire types (proto3).

Dimension	JSON	Protobuf	Avro	MessagePack
Size (typical)	Largest; key names + text numbers	Smallest; tags + varints	Smallest; no tags at all	Small; key names re- main
Encode/decode speed	Slowest (text parse)	Fast (gen- erated code)	Fast	Fast
Schema required?	No (JSON Schema op- tional, ex- ternal)	Yes (IDL, compiled)	Yes (JSON schema, resolved at read)	No
Evolution mechanism	Convention + JSON Schema compat rules	Tag num- bers + re- served	Name matching + defaults + promo- tion	None built- in
Human debuggability	Excellent (readable)	Poor (needs protoc decode)	Poor (needs schema)	Poor (bi- nary)
Ecosystem fit	Everywhere; browsers native	gRPC, Google stack, polyglot codegen	Kafka, Hadoop, streaming pipelines	Redis, game/IoT, polyglot
Unknown-field handling	Ignored by most parsers	Ignored/dropped (proto3)	Resolved via schema	Preserved as data
64-bit integer safety	Unsafe be- yond 2^{53}	Safe (ex- plicit widths)	Safe (long)	Safe (uint64/int64)

Table 5.3: Serialization format trade-off matrix. “Fast” and “small” are relative; measure with your own payloads (Section 5.3).

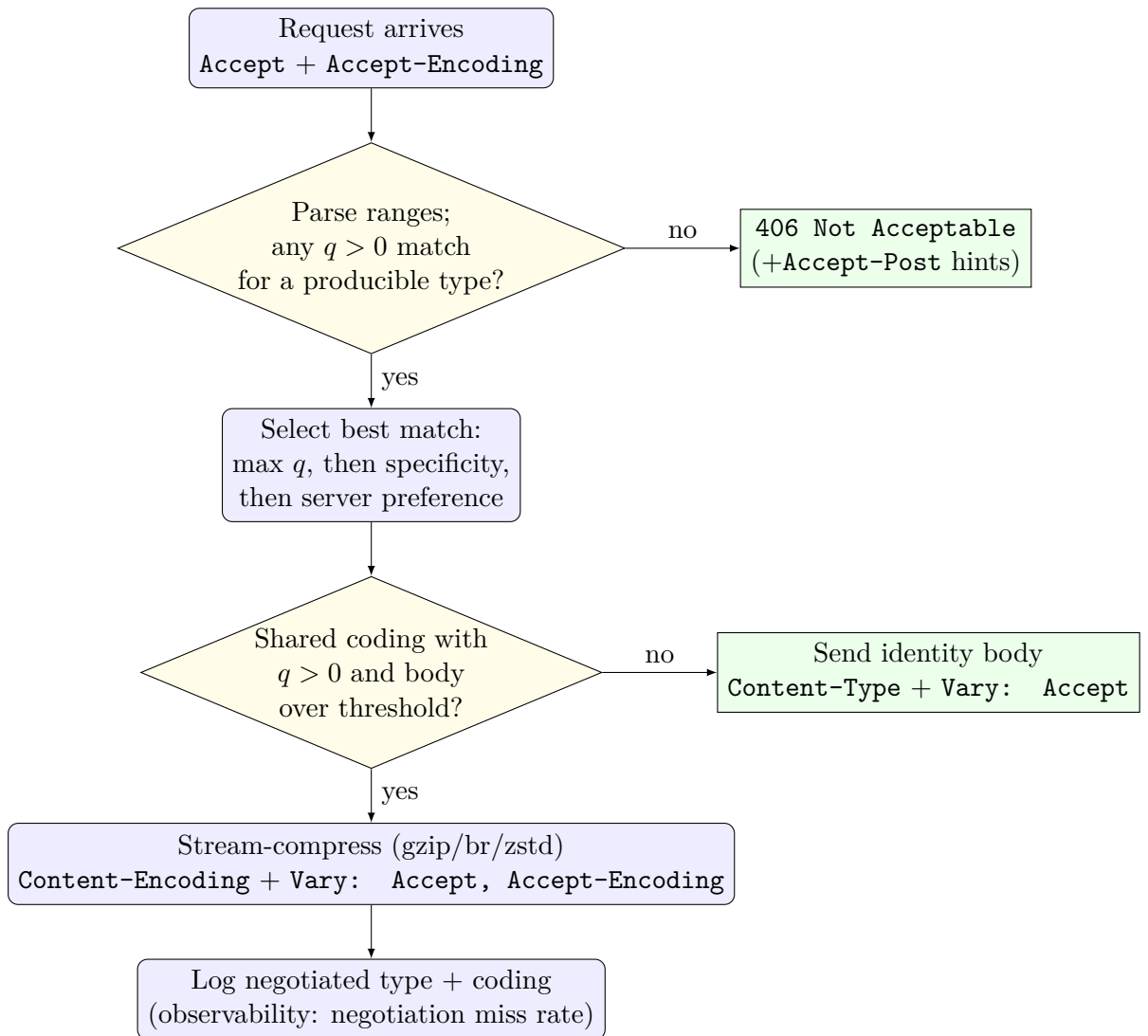


Figure 5.3: Server-side negotiation decision flow. The **Vary** header is what keeps shared caches honest across negotiated variants.

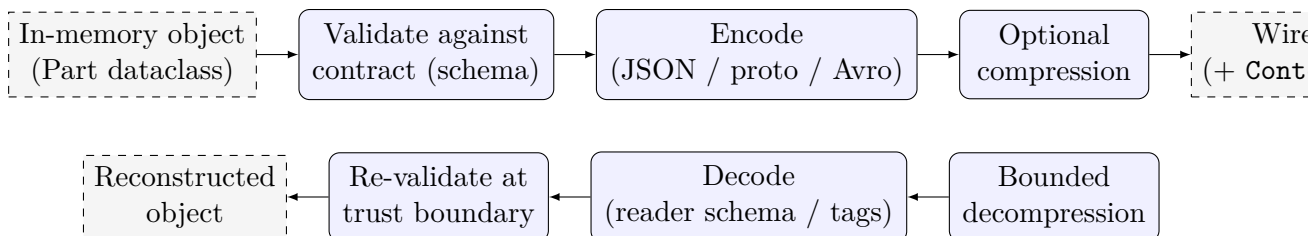


Figure 5.4: Encode/decode pipeline. Symmetric validation at both trust boundaries is deliberate: the producer's bug must not become the consumer's incident.

Format	Size	Speed	Schema	Reach for it when
JSON	large	slow	external	Public APIs, browser clients, human debugging.
Protobuf	smallest	fast	required (IDL)	Internal RPC, strict contracts, codegen.
Avro	smallest	fast	required (JSON)	Event logs, Kafka, writer/reader evolution.
MessagePack	small	fast	none	Schemaless internal pipes, IoT, caches.
NDJSON	large	slow	external	Feeds, logs, bulk export, resumable streams.

Table 5.4: Format selection at a glance (details: Table 5.3).

Header	Rule to remember
Accept	Missing $q = 1.0$; most specific matching range sets the score; $q=0$ refuses; no match $\rightarrow 406$.
Content-Type	Declares what the body <i>is</i> ; parameters (<code>charset</code> , <code>v=2</code>) are part of the type.
Accept-Encoding	Negotiates coding (gzip/br/zstd); enforce a minimum body size; never buffer-decompress untrusted input.
Content-Encoding	What coding was applied; incompatible with a guessed Content-Length .
Vary	Lists request headers that select the variant; omit it and caches corrupt negotiated resources.
X-Content-Type-Options: nosniff	Stops browsers from second-guessing the declared type.

Table 5.5: Negotiation headers cheat-sheet.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 5.6: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 5.7: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 5.8: Decision card to complete per chapter.

Chapter 6

Client-Side Security: API Keys, OAuth 2.0/OIDC & JWT for Consumers

Every credential your client application holds is a loaded weapon pointed at your own production environment. An API key pasted into a CI log, a bearer token cached one second past its expiry, a JWT verifier that trusts the attacker-controlled `alg` header — each of these is a small, invisible defect that converts an ordinary consumer application into a breach vector. Chapters 1 and 2 taught you how to put requests on the wire; this chapter teaches you how to prove *who is calling* without ever leaking the proof. We treat client-side security as an engineering discipline with specifications, failure modes, and testable invariants — not as a set of cargo-culted snippets copied from a vendor quickstart.

We begin with the credential taxonomy every consumer must distinguish: long-lived **API keys** (with real entropy requirements and the header-versus-query placement debate, settled by log-forensics reality), **HTTP Basic** (Base64 is an encoding, never an encryption), **Bearer tokens** per RFC 6750, and **HMAC-signed requests**, studied through a complete anatomy of AWS Signature Version 4 — canonical URI, canonical query string, canonical headers, string-to-sign, and the four-stage signing-key derivation chain. We then decode **OAuth 2.0** per RFC 6749 [Har12]: the four roles, the authorization-code grant in full sequence detail, the PKCE extension per RFC 7636 [Sak15] with its 43–128 character verifier and the S256 transform arithmetic, the client-credentials and device-code grants, refresh-token rotation with reuse detection, and a post-mortem of why the implicit grant is deprecated. The **OpenID Connect** layer follows: ID token versus access token, discovery documents, JWKS key rotation via `kid`, nonces, and the `userinfo` endpoint. We then dissect **JWT** per RFC 7519 [Jon15]: compact serialization, `base64url`, the seven registered

claims, the non-negotiable validation order (signature first, claims second), the `alg=none` and RS256→HS256 confusion attacks, `kid` injection, clock-skew leeway, why JWS is not encryption, and the revocation problem that haunts stateless tokens. The final mechanics section covers what most texts ignore: **where secrets live on the client** — environment variables, OS keychains, secret managers — with rotation runbooks and log-redaction discipline.

Every claim in this chapter is backed by runnable, offline, stdlib-only Python 3.11 code: a SigV4-style signer with a verifying stub server that proves tamper detection, a complete PKCE flow against a local in-memory identity provider, a hardened JWT verifier that survives a six-token adversarial gauntlet, a single-flight proactive token cache with redaction-safe logging, and a clearly labeled educational demonstration of the RS256→HS256 confusion attack succeeding against a naive verifier and dying against a hardened one.

Key Concept

A credential is a *liability with an expiry date*, not an asset. The client-side security problem decomposes into four questions you must answer for every credential your application touches: **entropy** (can it be guessed?), **placement** (where does it travel, and what records it?), **validation** (who checks it, in what order, against which allow-lists?), and **lifecycle** (how is it stored, rotated, revoked, and scrubbed from logs?). If any answer is “we do not know,” you have a vulnerability with a known CVE-shaped future.

6.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Classify client credentials — API keys, HTTP Basic, bearer tokens, HMAC signatures — by entropy, placement, replay resistance, and revocation story, and defend a placement choice using the log-leakage argument against query-string credentials.
- LO-2.** Reconstruct the AWS SigV4 signing chain by hand: canonical URI and query rules, canonical header normalization, string-to-sign assembly, and the four-stage key derivation $k_{\text{signing}} = \text{HMAC}(\text{HMAC}(\text{HMAC}(\text{HMAC}(\text{“NWR4”} + k, d), r), s), \text{“request”})$.

- LO-3.** Explain the four OAuth 2.0 roles and implement the authorization-code grant with PKCE — generating a 43–128 character `code_verifier`, deriving `code_challenge` = `BASE64URL(SHA256(v))`, and exchanging codes offline against a stub authorization server [Har12; Sak15].
- LO-4.** Select the correct grant (authorization code + PKCE, client credentials, device code) for a given client topology, and argue precisely why the implicit grant is deprecated and why `client_secret` can never protect a public client.
- LO-5.** Implement refresh discipline: proactive refresh inside a skew window, single-flight refresh under concurrency, refresh-token rotation, and reuse detection with token-family revocation.
- LO-6.** Distinguish the OIDC ID token from the OAuth access token, consume a discovery document and JWKS endpoint, and follow key rotation through `kid` headers without downtime.
- LO-7.** Validate a JWT in the only safe order — structural parse, algorithm allow-list, signature verification, then temporal and audience claims with bounded leeway — and explain how `alg=none`, RS256→HS256 confusion, and `kid` injection each defeat naive verifiers [Jon15; OWA23].
- LO-8.** Choose a client-side secret storage mechanism (environment variable, OS keychain, secret manager) using a threat-model matrix, write a rotation runbook, and install log redaction so credentials can never reach an aggregator.

Prerequisites

This chapter assumes you have completed:

- **Chapter 0** — Python 3.11+ environment, virtual environments, and running scripts from PowerShell.
- **Chapter 1** — the HTTP wire protocol: methods, status codes, header fields, and message bodies. We reuse **Authorization**, **Location**, and redirect semantics constantly.
- **Chapter 2** — consuming APIs from Python: `urllib` and structured error handling, which every listing here extends.

You should be comfortable reading cryptographic *constructions* (no proofs) and with the idea that a hash is one-way while an encoding is not.

Everything runs offline against loopback stub servers; all keys are synthetic fixtures in the fictional *Northwind Robotics* universe.

Warning

The attack demonstrations in this chapter (Sections 6.3–6.5) exist so you can recognize and kill these bug classes in code review. Run them only against the local stubs provided. Pointing adversarial tokens at systems you do not own is neither education nor research; it is unauthorized access.

6.2 Deep Technical Mechanics & Architectural Concepts

6.2.1 The Credential Taxonomy

Definition 6.1 (Credential). A *credential* is a byte string whose presentation to a verifier establishes an authentication claim. Credentials differ along five axes: **entropy** (bits of guessing resistance), **bearer vs. proof-of-possession** (does mere possession suffice?), **lifetime** (seconds to years), **revocability** (can the issuer kill it before expiry?), and **placement** (which protocol field carries it, and what infrastructure records that field?).

API Keys: Entropy and Placement

An API key is a shared secret, typically issued out-of-band from a developer portal, that identifies a *project* or *application* — rarely an end user. Two engineering questions dominate: how much entropy, and where does it ride on the wire?

Entropy. A key that humans type is not a key; it is a password with delusions. Machine credentials must carry at least 128 bits of cryptographically generated entropy, conventionally rendered as 32 hex characters or roughly 22–26 base64url characters. The relevant quantity is the min-entropy of the generation process, not the length of the string: `aaaaaaaa-...` is 36 characters and zero security. Generate with `secrets.token_urlsafe(32)` (256 bits), never with `random`, timestamps, or UUIDv4s alone (122 bits, and only if the UUID generator is a CSPRNG).

Placement: header vs. query. RFC 6750 permits bearer tokens in the `Authorization` header, a form body field, or the query parameter `access_token`. Only the first is acceptable in production, and the reason is not ideological — it is forensic:

- **Query strings are logged everywhere.** Web server access logs (nginx, Apache, IIS), load balancers, CDNs, APM agents, and browser

history all record the full request-target, query included, by default. Headers are *not* logged by default.

- **Query strings leak via Referer.** If the response page links anywhere, the full URL — key included — can flow to third parties.
- **Query strings survive in analytics and caches.** A cache key computed from the full URL stores the credential; a CDN edge log ships it to a third party's bucket.

The OWASP API Security Top 10 lists exactly this failure mode under broken authentication [OWA23]. The one-sentence rule: *a credential in a URL is a credential in a log file you do not control*.

HTTP Basic: Base64 Is Not Encryption

HTTP Basic (RFC 7617) sends `Authorization: Basic base64(username ":" password)`. Base64 is a *transfer encoding*: it is injective, keyless, and trivially reversible by every proxy, log shipper, and junior analyst on earth. Basic over plaintext HTTP is a password broadcast; Basic over TLS is a password on every request, giving the server an ongoing obligation to protect a reusable long-lived secret and giving every intermediary a readable credential inside the TLS termination boundary. It survives today for internal tooling and machine-to-machine calls behind mTLS — contexts where its simplicity outweighs its exposure. For public APIs it should be considered a legacy smell.

Bearer Tokens

A bearer token is a credential where *possession is proof*: whoever presents the string is trusted, exactly like cash. OAuth 2.0 access tokens are bearer tokens carried as `Authorization: Bearer <token>`. The design consequences follow immediately: TLS is non-negotiable (anyone who sees the token owns the token); tokens must be short-lived to bound the theft window; and the client must treat the token string as radioactive — never logged, never in URLs, never in browser storage accessible to third-party scripts without an explicit, documented threat analysis.

HMAC-Signed Requests: Proof Without Disclosure

HMAC request signing breaks the bearer assumption: the credential (a shared secret) never crosses the wire at all. Instead the client computes $\text{HMAC}_k(\text{canonical request})$ and sends the *signature*; the server, holding the same k , recomputes and compares. A replayed signature is useless against a modified body, and the secret itself is never exposed to logs, proxies,

or referers. This is the design behind AWS Signature Version 4, whose anatomy we now walk precisely, because the same construction appears in Azure, GCP, and countless webhook schemes (GitHub, Stripe, Slack).

6.2.2 Anatomy of a SigV4-Style Signature

SigV4’s core insight is that signing “an HTTP request” is meaningless until the request is *canonicalized* — reduced to exactly one byte string per logical request. Two requests that differ only in header casing, header order, or query-parameter order must produce the same canonical form, or verification becomes a coin flip. The construction proceeds in four stages.

Stage 1: the canonical request. Concatenate, newline-separated:

1. **HTTP method**, uppercased: POST.
2. **Canonical URI**: the absolute path with each segment URI-encoded per RFC 3986 (unreserved characters `A-Za-z0-9-._~` pass through; everything else becomes `%XX` uppercase hex; slashes are preserved as delimiters).
3. **Canonical query string**: parameters sorted first by encoded name, then by encoded value, joined as `name=value` pairs with `&` — each name and value individually URI-encoded.
4. **Canonical headers**: header names lowercased, values trimmed with internal runs of whitespace collapsed to one space, sorted by name, rendered as `name:value\n`. At minimum `host` and the date/payload headers must be signed.
5. **Signed headers**: the semicolon-joined, sorted list of the header names included above — this binds the signature to a specific header set and prevents an intermediary from injecting an unsigned header the application might honor.
6. **Hashed payload**: `SHA256(body)` in lowercase hex. The body is hashed, not copied, so the canonical request stays bounded in size.

Stage 2: the string-to-sign. A second newline-separated document: the algorithm identifier (`NWR4-HMAC-SHA256` in our teaching dialect; AWS uses `AWS4-HMAC-SHA256`), the request timestamp in `yyyyMMdd'T'HHmmss'Z'` form, the *credential scope* `date/region/service/nwr4_request`, and the hex SHA-256 of the canonical request from Stage 1.

Stage 3: the signing-key derivation chain. Rather than using the long-term secret directly, SigV4 derives a scoped, short-lived key through a four-stage HMAC cascade:

$$\begin{aligned}k_{\text{date}} &= \text{HMAC}(\text{"NWR4"} + k_{\text{secret}}, d) \\k_{\text{region}} &= \text{HMAC}(k_{\text{date}}, r) \\k_{\text{service}} &= \text{HMAC}(k_{\text{region}}, s) \\k_{\text{signing}} &= \text{HMAC}(k_{\text{service}}, \text{"nwr4_request"})\end{aligned}$$

Each stage is domain-separated: a key derived for the **parts** service in **test-region-1** on 2026-03-01 is cryptographically unrelated to any other scope, so a signature extracted from one context cannot be transplanted into another.

Stage 4: the signature and the Authorization header. $sig = \text{hex}(\text{HMAC}(k_{\text{signing}}, \text{string-to-sign}))$, delivered as a structured header naming the credential scope, the signed header list, and the signature. The server repeats Stages 1–4 with its copy of the secret and compares with a constant-time equality check (`hmac.compare_digest` — never `==`, whose early-exit leaks byte-position information through timing).

Listing 6.3.1 implements the whole chain and a verifying stub server; run it and observe that flipping one JSON field in the body flips the response from 200 to 403, because the payload hash embedded in the canonical request no longer matches.

Example

Why does the signed-headers list exist? Imagine a proxy adds **X-Debug: true** in transit and the application, finding the header, skips an authorization check. If **x-debug** was not in **SignedHeaders**, the signature still verifies — the signer never saw it — yet the application behaves as if the client requested debug mode. Binding the header *set* into the signature makes such injection detectable: any added signed-name breaks verification, and a disciplined verifier ignores unsigned hop-by-hop additions by policy.

6.2.3 OAuth 2.0 Decoded (RFC 6749)

OAuth 2.0 is a delegation framework, not an authentication protocol: it lets a *client* obtain limited access to a *resource owner's* data held by a *resource server*, with consent mediated by an *authorization server* [Har12]. Memorize the four roles as actors with distinct trust boundaries:

- **Resource owner** — the human (or organization) whose data is at stake; grants consent.

- **Client** — your application; wants access. *Public* clients (SPAs, mobile apps, CLIs) cannot keep a secret; *confidential* clients (server-side daemons) can.
- **Authorization server (AS)** — authenticates the owner, captures consent, mints tokens.
- **Resource server (RS)** — the API; validates access tokens and enforces scopes.

Definition 6.2 (Grant type). A *grant type* is the choreography by which a client obtains an access token. Choosing a grant is choosing a threat model: each grant exists because some client topology makes the others unsafe or impossible.

Authorization Code Grant

The workhorse for any client with a redirect-capable front channel. The client sends the user's browser to the AS `/authorize` endpoint with `response_type=code`, `client_id`, `redirect_uri`, `scope`, and a one-time `state` (CSRF defense). The AS authenticates the owner, records consent, and 302-redirects back with a short-lived, single-use `code`. The client then exchanges the code at the back-channel `/token` endpoint — authenticated with its `client_secret` if confidential — for an access token (and usually a refresh token). The design's elegance is the channel split: the browser (front channel, logged, bookmarked, referer-leaky) carries only the opaque code; the actual token travels server-to-server.

PKCE (RFC 7636): Proof Key for Code Exchange

The code grant has a hole for public clients: anyone who intercepts the code (malicious app registering the same custom URI scheme on a phone, referer leakage, log scraping) can exchange it, because the public client has no secret to prove the exchange request comes from the same party that started the flow. PKCE closes the hole with a per-flow ephemeral secret [Sak15]:

1. The client generates a high-entropy `code_verifier`: a random string of **43–128 characters** drawn from the unreserved URI set `[A-Za-z0-9-._~]`. (The 43-character floor corresponds to 256 bits of entropy in base64url: $43 \times 6 \approx 258$.)
2. The client derives the challenge via the S256 transform:

```
code_challenge = BASE64URL-ENCODE(SHA256(ASCII(code_verifier)))
```

and sends it, with `code_challenge_method=S256`, on the authorize request. (The legacy `plain` method sends the verifier itself and defeats the purpose; reject servers that offer only `plain`.)

3. The AS stores the challenge beside the code. At token exchange, the client presents the *verifier*; the AS recomputes S256 and compares. An interceptor who stole the code but never saw the verifier (it never travels until the back-channel exchange) cannot redeem it.

Figure 6.1 shows the full sequence, and Listing 6.3.2 implements both endpoints plus the client.

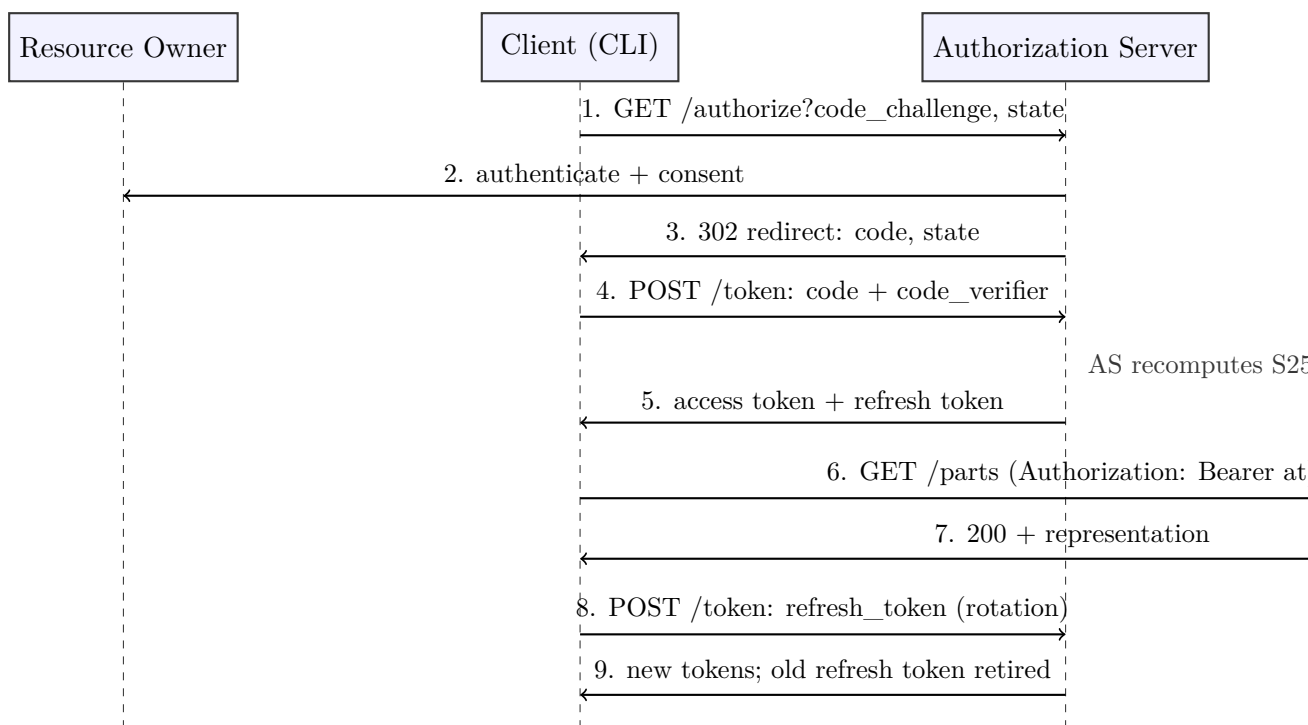


Figure 6.1: Authorization code grant with PKCE. The verifier never leaves the back channel; the browser-visible front channel carries only the challenge and the opaque code.

Client Credentials Grant

For machine-to-machine calls with no resource owner at all: the confidential client POSTs `grant_type=client_credentials` with its `client_id/client_secret` directly to `/token` and receives an access token scoped to itself. There is no refresh token — the client simply re-authenticates. If you find a `client_secret` in shipped browser or mobile code, the architecture has

already failed: a secret distributed to every device is a public value wearing a trench coat.

Device Code Grant (RFC 8628)

For input-constrained devices (CLIs on headless boxes, TVs, IoT): the device POSTs to a `/device_authorization` endpoint and receives a `device_code`, a short human-typable `user_code`, a `verification_uri`, and a polling `interval`. The human visits the URI on a real browser, enters the code, and consents; meanwhile the device polls `/token` with the `device_code`, honoring `authorization_pending` and `slow_down` responses until consent lands. The security property: the credential-conferring channel (the human's browser) is disjoint from the constrained device, and polling rate limits blunt guessing of user codes.

Implicit Grant: A Post-Mortem

The implicit grant (`response_type=token`) delivered access tokens directly through the browser redirect URI fragment. It is deprecated — the OAuth Security Best Current Practice and the OAuth 2.1 draft both remove it — because it put bearer tokens into URLs (logs, history, referers), offered no client authentication, no refresh path, and no way to bind the token to the client that requested it. PKCE made its only legitimate use case (browser apps) obsolete. If you encounter it in the wild, treat the migration to authorization code + PKCE as a security remediation, not a refactor.

Refresh Discipline: Rotation, Reuse Detection, Clock Skew

Long-lived refresh tokens are the crown jewels of the client. Three disciplines make them survivable:

1. **Proactive refresh.** Never ride a token to its death: refresh when $now \geq exp - skew$ with $skew \geq$ the p99 request latency plus clock error (60 s is a sane default). A token that expires mid-flight yields the dreaded “200 on send, 401 on retry” flapping.
2. **Single-flight refresh.** Under concurrency, ten threads seeing an expired token must produce *one* refresh call, not ten — ten parallel refreshes against a rotating AS looks exactly like token replay and can trip reuse detection, revoking your own session. Listing 6.3.4 implements the double-checked-locking pattern.
3. **Rotation with reuse detection.** The AS issues a new refresh token on every use and retires the old one. If a retired token is ever presented again, one of two copies (yours, an attacker's) is stolen — so the AS

revokes the entire token *family*, forcing re-authentication. Clients must therefore persist rotated tokens atomically: crash between “received new token” and “wrote it to disk” and you have bricked your own session.

Figure 6.2 summarizes the access-token lifecycle as a state machine.

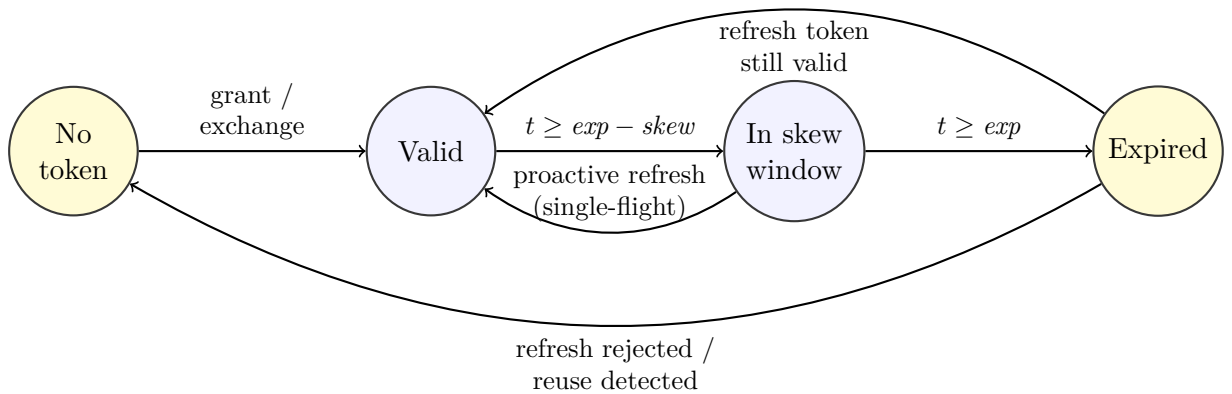


Figure 6.2: Access-token lifecycle state machine. The skew window converts the expiry cliff into a managed transition; reuse detection on the refresh path can drop the client back to *No token* at any time.

6.2.4 OpenID Connect: The Identity Layer

OAuth 2.0 delegates *access*; it deliberately says nothing about *who* the user is. OpenID Connect (OIDC) layers identity on top:

- **ID token vs. access token.** The ID token is a JWT whose audience (*aud*) is *your client* — it is proof of authentication *for you to consume locally*. The access token is for the resource server; your client should treat it as opaque and never parse it. The canonical client bug — sending the ID token to an API — fails the API’s audience check when the API is implemented correctly, and creates a confused deputy when it is not.
- **Discovery.** OIDC providers publish `/.well-known/openid-configuration`: issuer, authorization and token endpoints, JWKS URI, supported algorithms and scopes. Hard-coding endpoint URLs is a fragility; discover them.
- **JWKS and kid.** The AS publishes its public signing keys as a JSON Web Key Set. Tokens carry a *kid* (key ID) header; verifiers select

the matching key. During rotation, the AS signs new tokens with the new key while the JWKS advertises both — clients that cache JWKS must re-fetch on unknown `kid` (with a rate limit, or a flood of forged `kid` values becomes a self-inflicted DoS).

- **Nonce.** For front-channel ID token delivery, the client embeds a random `nonce` in the authorize request and requires the same value inside the ID token, binding the token to this authentication round trip and defeating token replay.
- **Userinfo.** A protected endpoint that returns claims about the authenticated user, checked with the access token — the OIDC answer to “who is this session?”

6.2.5 JWT Internals (RFC 7519)

A JSON Web Token in compact serialization is three `base64url` segments joined by dots: `header.payload.signature` [Jon15]. The header is a JSON object naming the algorithm (`alg`), type (`typ=JWT`), and key id (`kid`); the payload carries *claims*; the signature covers the first two segments exactly as transmitted. Base64url is Base64 with `+//` replaced by `-/_` and padding omitted — an *encoding*, so the payload of every JWT you have ever issued is readable by anyone holding the token. Never put secrets, PII beyond need, or authorization policy you would not publish into a payload.

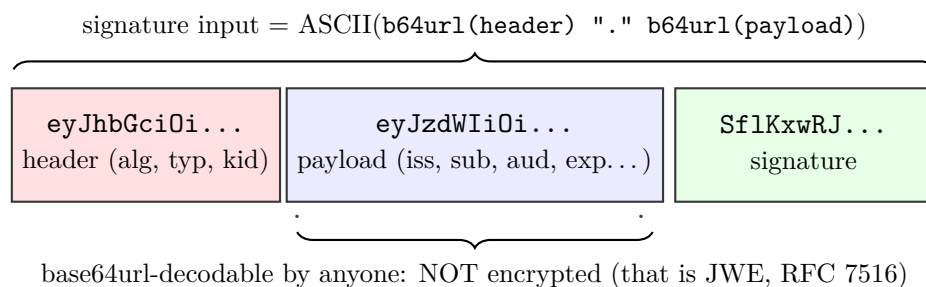


Figure 6.3: JWT compact anatomy. Only the green segment requires the key; the red and blue segments are plaintext to every holder of the token.

Registered Claims

RFC 7519 registers seven claim names [Jon15]. Three are temporal, four are identity/routing:

Claim	Type	Validation rule for consumers
<code>iss</code>	string	Exact match against the configured issuer.
<code>sub</code>	string	Principal identifier; log it, never trust it before signature verification.
<code>aud</code>	string/list	Must contain <i>your</i> audience; reject “any-audience” tokens.
<code>exp</code>	NumericDate	Required; reject when $now > exp + leeway$.
<code>nbft</code>	NumericDate	Reject when $now + leeway < nbft$.
<code>iat</code>	NumericDate	Reject future <code>iat</code> beyond leeway (clock-attack signal).
<code>jti</code>	string	Unique token id; the hook for replay caches and revocation lists.

Table 6.1: Registered JWT claims and the consumer-side validation rule for each.

The Validation Order Is the Security Boundary

A verifier that inspects claims before verifying the signature is reading attacker-controlled JSON. The only safe order is:

1. **Structural parse:** exactly three segments; base64url decodable; header and payload are JSON objects.
2. **Algorithm allow-list:** `alg` must be in a *configured* set (often a single entry), `none` must always be rejected, and the algorithm must be consistent with the key type you configured (no HMAC with RSA public keys).
3. **Key selection:** by `kid` from a *trusted* key set (JWKS you fetched), never by a URL or path inside the token.
4. **Signature verification** with constant-time comparison.
5. **Claim validation:** `exp`, `nbft`, `iat` with a bounded leeway (30–60 s; you are compensating for clock skew, not writing tokens a second life), then `iss` and `aud`.

The Attack Zoo

- **alg=none.** Early libraries accepted `{"alg":"none"}` with an empty signature. Every modern library rejects it — but only if you do not hand-roll verification or pass attacker-influenced algorithm choices.
- **RS256→HS256 confusion.** If the verifier expects RS256 but picks its verification method from the token header, an attacker signs

with HMAC-SHA256 using the *public key* (which is public — JWKS publishes it) as the HMAC secret. The naive verifier computes HMAC with its configured key bytes and the forgery verifies. The fix is architectural: bind algorithm to key configuration, never to the token. Listing 6.3.5 demonstrates both the kill and the cure.

- **kid injection.** If `kid` is used to build a file path or SQL query (`SELECT key FROM keys WHERE id='$kid'`), the attacker injects `../../../../etc/passwd` or `' OR '1'='1`. Treat `kid` as an opaque lookup key into an in-memory map you populated from JWKS — nothing else.
- **Clock-skew abuse.** Leeway exists for distributed clock error, but an unbounded leeway resurrects expired tokens. Cap it, apply it symmetrically to `exp/nbf/iat`, and alert on skew beyond the cap.

JWS Is Not Encryption, and Revocation Is the Price of Statelessness

A signed JWT (JWS, RFC 7515) guarantees integrity and origin — not confidentiality. If claims must be hidden, that is JWE (RFC 7516), a different and rarer beast. More important operationally: a valid JWT is valid until `exp` no matter what happens to the user — password reset, termination, role change. Stateless verification bought you horizontal scale and paid for it with a revocation gap. Mitigations, in order of practicality: short access-token lifetimes (5–15 min) plus refresh rotation; a `jti` blocklist checked on the highest-value endpoints; token exchange/downscoping at the edge. Any design that says “we will just revoke the JWT” without one of these mechanisms has not finished designing.

6.2.6 Secret Storage for Consumers

Where the client keeps credentials between runs is a threat-model decision:

Whatever the store, three disciplines apply. **Least privilege:** scope every credential to the minimum permission set — a read-only key for a read-only job — so a leak’s blast radius is pre-bounded. **Rotation runbooks:** every credential class needs a written, rehearsed procedure — issue new, deploy, verify, revoke old — with the rotation deadline tied to the credential’s lifetime, and break-glass steps for a leak (revoke first, investigate second; Section 6.5 walks an incident). **Never log credentials:** install a redaction filter on the root logger (Listing 6.3.4), scrub `Authorization` headers in HTTP client middleware, and treat “a credential appeared in a log” as a compromise requiring rotation, because log aggregation is exfiltration with extra steps.

Mechanism	Strengths	Failure modes
Environment variables	Twelve-factor native [Wig17]; trivially injectable in CI; no code coupling	Readable via <code>/proc</code> , crash dumps, <code>docker inspect</code> ; inherited by child processes
OS keychain (keyring)	Encrypted at rest by the OS; per-user ACL; survives reboots	Desktop-only; headless CI needs a fallback; API differs per OS
Secret manager (cloud vault)	Central audit, rotation, fine-grained ACL, dynamic secrets	Network dependency; bootstrap-credential problem; cost
Files on disk	Simplest possible	World-readable by default; committed to git by accident; visible in backups

Table 6.2: Credential storage matrix for client applications.

6.3 Production-Ready Code Implementation

All five listings share the chapter’s golden rules: Python 3.11+ standard library only, fully offline (loopback stubs, ephemeral ports), deterministic (seeded PRNGs, injected clocks, fixed timestamps), typed, and defensive (specific exceptions, input validation, redaction-aware logging). Every key is a synthetic Northwind Robotics fixture. Save each listing as its own file and run with `python listing_5_N_*.py` from PowerShell.

6.3.1 Listing 5.1 — SigV4-Style HMAC Signer with Verifying Stub Server

This is the canonicalization pipeline of Section 6.2.2 rendered in code, plus a stub `http.server` that re-derives the signature and proves tamper detection: the happy path returns 200, while a tampered body and a wrong secret each return 403.

```

1  """Listing 5.1 -- SigV4-style HMAC request signer with
   a verifying stub server.
2
3  Educational re-implementation of the AWS Signature
   Version 4 construction
4  (canonical request -> string-to-sign -> key derivation
   chain -> signature)
5  using only the Python 3.11+ standard library. Runs
   fully offline: the

```

```
6 verifying_server_is_a_stdlib_‘‘http.server‘‘_bound_to_
   the_loopback
7 interface.
8
9 All_keys_in_this_file_are_SYNTHETIC_TEST_FIXTURES_for_
   the_fictional
10 Northwind_Robotics_universe._Never_use_them_anywhere_
   else.
11 """
12
13 from __future__ import annotations
14
15 import hashlib
16 import hmac
17 import json
18 import threading
19 import urllib.request
20 from http.server import BaseHTTPRequestHandler,
   HTTPServer
21
22 # --- Synthetic test fixtures (NOT real credentials)
   -----
23 ACCESS_KEY_ID = "NWRTESTACCESSKEY0001"
24 SECRET_ACCESS_KEY = "nwr-test-secret-key-0123456789
   abcdef0123456789abcdef"
25 REGION = "test-region-1"
26 SERVICE = "parts"
27 ALGORITHM = "NWR4-HMAC-SHA256"
28 FIXED_AMZ_DATE = "20260301T120000Z" # fixed for
   deterministic output
29 FIXED_DATE_STAMP = "20260301"
30
31
32 def _sha256_hex(data: bytes) -> str:
33     return hashlib.sha256(data).hexdigest()
34
35
36 def _hmac_sha256(key: bytes, msg: str) -> bytes:
37     return hmac.new(key, msg.encode("utf-8"), hashlib.
   sha256).digest()
38
39
40 def _uri_encode(value: str, safe: str = "~") -> str:
41     """RFC_3986_URI_encoding;_unreserved_characters_are
   _left_as-is."""
42     unreserved = "
```

```

        ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789
        -._"
43     keep = unreserved + safe
44     out: list[str] = []
45     for ch in value:
46         if ch in keep:
47             out.append(ch)
48         else:
49             out.extend(f"%{b:02X}" for b in ch.encode("
                utf-8"))
50     return "".join(out)
51
52
53 def canonical_uri(path: str) -> str:
54     """Normalize the path: each segment is URI-encoded,
55     slashes preserved."""
56     if not path.startswith("/"):
57         raise ValueError("path must be absolute")
58     return "/" + ".join(_uri_encode(seg) for seg in path.
59         split("/"))
60
61 def canonical_query(query: dict[str, str]) -> str:
62     """Sort parameters by (name, value); both are URI-
63     encoded."""
64     pairs = sorted((_uri_encode(k), _uri_encode(v)) for
65         k, v in query.items())
66     return "&".join(f"{k}={v}" for k, v in pairs)
67
68 def canonical_headers(headers: dict[str, str]) -> tuple
69     [str, str]:
70     """Lowercase names, trim/collapse whitespace, sort;
71     return (block, list)."""
72     normalized = {
73         name.strip().lower(): " ".join(value.strip().
74             split())
75         for name, value in headers.items()
76     }
77     ordered = sorted(normalized.items())
78     block = "".join(f"{name}:{value}\n" for name, value
79         in ordered)
80     signed = ";".join(name for name, _ in ordered)
81     return block, signed

```

```
78 def build_canonical_request(  
79     method: str,  
80     path: str,  
81     query: dict[str, str],  
82     headers: dict[str, str],  
83     payload: bytes,  
84 ) -> tuple[str, str]:  
85     """Return (canonical_request, signed_headers)."""  
86     header_block, signed_headers = canonical_headers(  
87         headers)  
88     canonical = "\n".join(  
89         [  
90             method.upper(),  
91             canonical_uri(path),  
92             canonical_query(query),  
93             header_block,  
94             signed_headers,  
95             _sha256_hex(payload),  
96         ]  
97     )  
98     return canonical, signed_headers  
99  
100 def derive_signing_key(secret: str, date_stamp: str,  
101     region: str, service: str) -> bytes:  
102     """SigV4-style derivation chain: date->region->  
103     service->request."""  
104     k_date = _hmac_sha256(("NWR4" + secret).encode("utf  
105     -8"), date_stamp)  
106     k_region = _hmac_sha256(k_date, region)  
107     k_service = _hmac_sha256(k_region, service)  
108     return _hmac_sha256(k_service, "nwr4_request")  
109  
110 def sign_request(  
111     method: str,  
112     path: str,  
113     query: dict[str, str],  
114     payload: bytes,  
115     host: str,  
116     access_key: str,  
117     secret_key: str,  
118     amz_date: str = FIXED_AMZ_DATE,  
119     date_stamp: str = FIXED_DATE_STAMP,  
120 ) -> dict[str, str]:  
121     """Produce the signed header set for an outgoing
```



```

120     request."""
121     scope = f"{date_stamp}/{REGION}/{SERVICE}/
122           nwr4_request"
123     headers = {
124         "host": host,
125         "x-nwr-date": amz_date,
126         "x-nwr-content-sha256": _sha256_hex(payload),
127     }
128     canonical, signed_headers = build_canonical_request
129     (method, path, query, headers, payload)
130     string_to_sign = "\n".join([ALGORITHM, amz_date,
131                                scope, _sha256_hex(canonical.encode("utf-8"))])
132     signature = _hmac_sha256(
133         derive_signing_key(secret_key, date_stamp,
134                             REGION, SERVICE), string_to_sign
135     ).hex()
136     headers["Authorization"] = (
137         f"{ALGORITHM} Credential={access_key}/{scope},
138         "
139         f"SignedHeaders={signed_headers}, Signature={
140             signature}"
141     )
142     return headers
143
144 class VerifyingHandler(BaseHTTPRequestHandler):
145     """Stub parts API that recomputes the signature and
146     rejects tampering."""
147
148     def do_POST(self) -> None: # noqa: N802 (stdlib
149         naming)
150         length = int(self.headers.get("Content-Length",
151                                       "0"))
152         payload = self.rfile.read(length)
153         path, _, raw_query = self.path.partition("?")
154         query = dict(
155             pair.split("=", 1) for pair in raw_query.
156             split("&") if pair
157         ) if raw_query else {}
158
159         try:
160             auth = self.headers["Authorization"]
161             credential = auth.split("Credential=")[1].
162                 split(",")[0]
163             access_key, date_stamp, region, service, _
164                 = credential.split("/")

```

```
153         amz_date = self.headers["x-nwr-date"]
154         headers = {
155             "host": self.headers["host"],
156             "x-nwr-date": amz_date,
157             "x-nwr-content-sha256": self.headers["x-
158                 -nwr-content-sha256"],
159         }
160         canonical, _ = build_canonical_request("
161             POST", path, query, headers, payload)
162         scope = f"{date_stamp}/{region}/{service}/
163             nwr4_request"
164         string_to_sign = "\n".join(
165             [ALGORITHM, amz_date, scope,
166              _sha256_hex(canonical.encode("utf-8"
167              ))])
168         expected = _hmac_sha256(
169             derive_signing_key(SECRET_ACCESS_KEY,
170             date_stamp, region, service),
171             string_to_sign,
172             ).hex()
173         presented = auth.split("Signature=")[1]
174         if not hmac.compare_digest(expected,
175             presented):
176             raise PermissionError("signature_
177                 mismatch")
178         self._respond(200, {"status": "accepted", "
179             access_key": access_key})
180     except (KeyError, IndexError, ValueError,
181             PermissionError) as exc:
182         self._respond(403, {"error": type(exc).
183             __name__})
184
185     def _respond(self, status: int, body: dict[str, str
186         ]) -> None:
187         data = json.dumps(body).encode("utf-8")
188         self.send_response(status)
189         self.send_header("Content-Type", "application/
190             json")
191         self.send_header("Content-Length", str(len(data
192             )))
193         self.end_headers()
194         self.wfile.write(data)
195
196     def log_message(self, format: str, *args: object)
197         -> None:
```

```

184         pass # keep demo output deterministic
185
186
187 def _post(path: str, query: dict[str, str], payload:
    bytes, headers: dict[str, str], port: int) -> tuple[
    int, str]:
188     qs = "&".join(f"{k}={v}" for k, v in query.items())
189     url = f"http://127.0.0.1:{port}{path}" + (f"?{qs}"
        if qs else "")
190     req = urllib.request.Request(url, data=payload,
        method="POST")
191     for name, value in headers.items():
192         if name.lower() != "host":
193             req.add_header(name, value)
194     try:
195         with urllib.request.urlopen(req) as resp:
196             return resp.status, resp.read().decode("utf
                -8")
197     except urllib.error.HTTPError as exc:
198         return exc.code, exc.read().decode("utf-8")
199
200
201 def main() -> None:
202     server = HTTPServer(("127.0.0.1", 0),
        VerifyingHandler)
203     port = server.server_address[1]
204     threading.Thread(target=server.serve_forever,
        daemon=True).start()
205     try:
206         host = f"127.0.0.1:{port}"
207         path, query = "/api/v1/parts", {"warehouse": "
            east", "priority": "high"}
208         payload = json.dumps({"sku": "NWR-ARM-9000", "
            quantity": 3}).encode("utf-8")
209
210         # 1. Happy path: properly signed request is
            accepted.
211         headers = sign_request("POST", path, query,
            payload, host, ACCESS_KEY_ID,
            SECRET_ACCESS_KEY)
212         status, body = _post(path, query, payload,
            headers, port)
213         print(f"[1] signed request -> {status
            } {body}")
214
215         # 2. Tampered body: signature no longer matches

```

```

    the payload hash.
216     evil_payload = json.dumps({"sku": "NWR-ARM-9000", "quantity": 99999}).encode("utf-8")
217     status, body = _post(path, query, evil_payload, headers, port)
218     print(f"[2] tampered body -> {status} {body}")
219
220     # 3. Wrong secret: derived signing key differs, verification fails.
221     bad = sign_request("POST", path, query, payload, host, ACCESS_KEY_ID, "wrong-secret")
222     status, body = _post(path, query, payload, bad, port)
223     print(f"[3] wrong secret key -> {status} {body}")
224
225     # 4. Show the deterministic signing internals for inspection.
226     canonical, signed_headers = build_canonical_request(
227         "POST", path, query, {"host": host, "x-nwr-date": FIXED_AMZ_DATE,
228                                "x-nwr-content-sha256": _sha256_hex(payload)},
229         payload,
230     )
231     print(f"[4] canonical request (lines): {len(canonical.splitlines())}, signed headers: {signed_headers}")
232     finally:
233         server.shutdown()
234
235 if __name__ == "__main__":
236     main()
237
238
239
```

Observed output (deterministic):

```

1 [1] signed request -> 200 {"status": "
    accepted", "access_key": "NWRTESTACCESSKEY0001"}
2 [2] tampered body -> 403 {"error": "
    PermissionError"}
3 [3] wrong secret key -> 403 {"error": "
    PermissionError"}
4 [4] canonical request (lines): 9 | signed headers: host
```

```
;x-nwr-content-sha256;x-nwr-date
```

6.3.2 Listing 5.2 — Complete PKCE Flow Against a Local Stub IdP

A runnable authorization-code + PKCE implementation per RFC 6749 and RFC 7636 [Har12; Sak15]: the stub IdP holds in-memory grants, enforces S256, rotates refresh tokens, and revokes the whole token family on reuse; the client generates the verifier/challenge pair, checks `state`, exchanges, and refreshes. In production the verifier comes from `secrets.token_urlsafe`; here a seeded PRNG keeps output deterministic.

```
1  """Listing 5.2 -- Complete OAuth 2.0 Authorization Code
   2      + PKCE flow, offline.
   3
   4  A stdlib-only, fully local demonstration:
   5  * 'StubIdP' -- an in-memory authorization server (
   6      authorize + token
   7      endpoints) implementing PKCE verification, refresh-
   8      token rotation, and
   9      reuse detection, per RFC 6749 / RFC 7636 semantics.
  10  * 'PkceClient' -- a public client that generates a
  11      code_verifier /
  12      code_challenge (S256), obtains a code, exchanges it
  13      , and refreshes.
  14
  15  Deterministic: the verifier and codes come from a
  16      seeded PRNG. In
  17  production, use 'secrets.token_urlsafe' instead of '
  18      random.Random'.
  19
  20  All identifiers are synthetic fixtures for the
  21      fictional Northwind Robotics
  22      universe. No real network is used: the server binds to
  23      the loopback
  24      interface on an ephemeral port.
  25  """
  26
  27  from __future__ import annotations
  28
  29  import base64
  30  import hashlib
  31  import json
  32  import random
  33  import threading
```

```
25 import urllib.parse
26 import urllib.request
27 from dataclasses import dataclass
28 from http.server import BaseHTTPRequestHandler,
    HTTPServer
29
30 CLIENT_ID = "nwr-parts-cli"
31 REDIRECT_URI = "http://127.0.0.1/callback"
32 VERIFIER_ALPHABET = "
    ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789
    -._~"
33
34
35 def s256_challenge(verifier: str) -> str:
36     """RFC 7636: BASE64URL-ENCODE(SHA256(ASCII(
        code_verifier)))"""
37     digest = hashlib.sha256(verifier.encode("ascii")).
        digest()
38     return base64.urlsafe_b64encode(digest).rstrip(b"=")
        .decode("ascii")
39
40
41 def make_verifier(rng: random.Random, length: int = 64)
    -> str:
42     """RFC 7636: 43-128 chars from the unreserved set.
        Use secrets in prod."""
43     if not 43 <= length <= 128:
44         raise ValueError("code_verifier length must be
            in [43, 128]")
45     return "".join(rng.choice(VERIFIER_ALPHABET) for _
        in range(length))
46
47
48 @dataclass
49 class PendingGrant:
50     challenge: str
51     method: str
52
53
54 class StubIdP:
55     """In-memory authorization-server state shared by
        the HTTP handler."""
56
57     def __init__(self, seed: int = 20260301) -> None:
58         self._rng = random.Random(seed)
59         self._grants: dict[str, PendingGrant] = {}
```

```

60     self._active: dict[str, str] = {}    #
        refresh_token -> family id
61     self._retired: dict[str, str] = {}    # rotated-
        out token -> family id
62     self._counter = 0
63
64     def _mint(self, prefix: str) -> str:
65         self._counter += 1
66         return f"{prefix}-{self._counter:04d}-nwr-
            synthetic"
67
68     def create_grant(self, challenge: str, method: str)
        -> str:
69         code = self._mint("code")
70         self._grants[code] = PendingGrant(challenge=
            challenge, method=method)
71         return code
72
73     def exchange_code(self, code: str, verifier: str)
        -> dict[str, object]:
74         grant = self._grants.pop(code, None)    # one-
            time use
75         if grant is None:
76             raise PermissionError("invalid_grant:
                unknown_or_reused_code")
77         if grant.method != "S256" or s256_challenge(
            verifier) != grant.challenge:
78             raise PermissionError("invalid_grant:
                PKCE_verification_failed")
79         return self._issue_tokens(family=self._mint("
            fam"))
80
81     def refresh(self, refresh_token: str) -> dict[str,
        object]:
82         if refresh_token in self._retired:
83             # Reuse of an already-rotated token ==
                theft signal:
84             # revoke the ENTIRE token family (RFC 6749
                section 10.4 pattern).
85             family = self._retired[refresh_token]
86             self._active = {t: f for t, f in self.
                _active.items() if f != family}
87             raise PermissionError("invalid_grant:
                refresh_token_reuse_detected; family
                revoked")
88         family = self._active.pop(refresh_token, None)

```

```
89         if family is None:
90             raise PermissionError("invalid_grant:␣
91                                     unknown␣refresh␣token")
92         self._retired[refresh_token] = family # rotate
93             : old token retires
94         return self._issue_tokens(family=family)
95
96     def _issue_tokens(self, family: str) -> dict[str,
97         object]:
98         refresh = self._mint("rt")
99         self._active[refresh] = family
100         return {
101             "access_token": self._mint("at"),
102             "token_type": "Bearer",
103             "expires_in": 300,
104             "refresh_token": refresh,
105         }
106
107 class IdPHandler(BaseHTTPRequestHandler):
108     idp: StubIdP # injected by make_server()
109
110     def do_GET(self) -> None: # noqa: N802 -- /
111         authorize
112         params = urllib.parse.parse_qs(urllib.parse.
113             urlsplit(self.path).query)
114         try:
115             if not self.path.startswith("/authorize"):
116                 raise ValueError("unknown␣endpoint")
117             if params["client_id"][0] != CLIENT_ID:
118                 raise PermissionError("
119                     unauthorized␣client")
120             if params.get("code_challenge_method", [""])
121                 [0] != "S256":
122                 raise PermissionError("invalid_request:
123                     ␣only␣S256␣is␣accepted")
124             code = self.idp.create_grant(params["
125                 code_challenge"][0], "S256")
126             location = REDIRECT_URI + "?" + urllib.
127                 parse.urlencode(
128                     {"code": code, "state": params["state"]
129                         [0]})
130             self.send_response(302)
131             self.send_header("Location", location)
132             self.end_headers()
```



```

124         except (KeyError, ValueError, PermissionError)
125             as exc:
126                 self._json(400, {"error": str(exc)})
127
128     def do_POST(self) -> None: # noqa: N802 -- /token
129         length = int(self.headers.get("Content-Length",
130                                     "0"))
131         form = urllib.parse.parse_qs(self.rfile.read(
132             length).decode("utf-8"))
133         try:
134             if not self.path.startswith("/token"):
135                 raise ValueError("unknown_endpoint")
136             grant_type = form["grant_type"][0]
137             if grant_type == "authorization_code":
138                 tokens = self.idp.exchange_code(form["
139                     code"][0], form["code_verifier"][0])
140             elif grant_type == "refresh_token":
141                 tokens = self.idp.refresh(form["
142                     refresh_token"][0])
143             else:
144                 raise ValueError(f"
145                     unsupported_grant_type: {grant_type}
146                     ")
147             self._json(200, tokens)
148         except (KeyError, ValueError) as exc:
149             self._json(400, {"error": str(exc)})
150         except PermissionError as exc:
151             self._json(403, {"error": str(exc)})
152
153     def _json(self, status: int, body: dict[str, object]
154              ] -> None:
155         data = json.dumps(body).encode("utf-8")
156         self.send_response(status)
157         self.send_header("Content-Type", "application/
158                             json")
159         self.send_header("Content-Length", str(len(data)
160                                     )))
161         self.end_headers()
162         self.wfile.write(data)
163
164     def log_message(self, format: str, *args: object)
165         -> None:
166         pass
167
168 class PkceClient:

```

```
159     def __init__(self, base_url: str, rng: random.  
160         Random) -> None:  
161         self.base_url = base_url  
162         self.rng = rng  
163  
164     def _post_form(self, path: str, fields: dict[str,  
165         str]) -> dict[str, object]:  
166         req = urllib.request.Request(  
167             self.base_url + path,  
168             data=urllib.parse.urlencode(fields).encode(  
169                 "utf-8"),  
170             method="POST",  
171         )  
172         try:  
173             with urllib.request.urlopen(req) as resp:  
174                 return json.loads(resp.read().decode(  
175                     "utf-8"))  
176         except urllib.error.HTTPError as exc:  
177             raise PermissionError(exc.read().decode(  
178                 "utf-8")) from exc  
179  
180     def authorize_and_exchange(self) -> dict[str,  
181         object]:  
182         verifier = make_verifier(self.rng)  
183         params = {  
184             "response_type": "code",  
185             "client_id": CLIENT_ID,  
186             "redirect_uri": REDIRECT_URI,  
187             "scope": "parts:read",  
188             "state": make_verifier(self.rng, 43),  
189             "code_challenge": s256_challenge(verifier),  
190             "code_challenge_method": "S256",  
191         }  
192         # A real CLI opens the system browser here; we  
193         # follow the 302 locally.  
194         url = self.base_url + "/authorize?" + urllib.  
195             parse.urlencode(params)  
196         opener = urllib.request.build_opener(NoRedirect  
197             ())  
198         try:  
199             opener.open(url)  
200             raise RuntimeError("authorize endpoint must  
201                 answer 302, not 200")  
202         except urllib.error.HTTPError as exc:  
203             if exc.code != 302:  
204                 raise
```

```

195         location = exc.headers["Location"]
196         query = urllib.parse.parse_qs(urllib.parse.
197             urlsplit(location).query)
198         if query["state"][0] != params["state"]:
199             raise PermissionError("state_mismatch:
200                 possible CSRF")
201         return self._post_form(
202             "/token",
203             {
204                 "grant_type": "authorization_code",
205                 "code": query["code"][0],
206                 "redirect_uri": REDIRECT_URI,
207                 "client_id": CLIENT_ID,
208                 "code_verifier": verifier,
209             },
210         )
211
212     def refresh(self, refresh_token: str) -> dict[str,
213         object]:
214         return self._post_form(
215             "/token",
216             {"grant_type": "refresh_token", "
217                 refresh_token": refresh_token,
218                 "client_id": CLIENT_ID},
219         )
220
221
222     class NoRedirect(urllib.request.HTTPRedirectHandler):
223         def redirect_request(self, req, fp, code, msg,
224             headers, newurl): # type: ignore[override]
225             return None
226
227
228     def main() -> None:
229         idp = StubIdP(seed=20260301)
230         handler = type("BoundIdPHandler", (IdPHandler,), {"
231             idp": idp})
232         server = HTTPServer(("127.0.0.1", 0), handler)
233         threading.Thread(target=server.serve_forever,
234             daemon=True).start()
235         client = PkceClient(f"http://127.0.0.1:{server.
236             server_address[1]}",
237             random.Random(42))
238
239         try:
240             tokens = client.authorize_and_exchange()
241             print(f"[1] code_exchanged->access={

```

```
233         tokens['access_token']}]_]"
234         f"refresh={tokens['refresh_token']}")
235
236     rotated = client.refresh(str(tokens["
237         refresh_token"])))
238     print(f"[2]_refresh_rotated_>_access={
239         rotated['access_token']}]_]"
240         f"refresh={rotated['refresh_token']}")
241
242     try:
243         client.refresh(str(tokens["refresh_token"])
244             ) # replay old token
245     except PermissionError as exc:
246         print(f"[3]_reuse_of_old_refresh_>_
247             REJECTED:_{json.loads(exc.args[0])['
248             error']}")
249
250     try:
251         client.refresh(str(rotated["refresh_token"
252             ])) # family was revoked
253     except PermissionError as exc:
254         print(f"[4]_family_after_reuse_>_
255             REVOKED:_{json.loads(exc.args[0])['error'
256             ']}")
257
258     finally:
259         server.shutdown()
260
261 if __name__ == "__main__":
262     main()
```

Observed output (deterministic):

```
1 [1] code exchanged      -> access=at-0004-nwr-
   synthetic refresh=rt-0003-nwr-synthetic
2 [2] refresh rotated     -> access=at-0006-nwr-
   synthetic refresh=rt-0005-nwr-synthetic
3 [3] reuse of old refresh -> REJECTED: invalid_grant:
   refresh token reuse detected; family revoked
4 [4] family after reuse  -> REVOKED: invalid_grant:
   unknown refresh token
```

6.3.3 Listing 5.3 — Hardened JWT Verifier with Adversarial Gauntlet

The validation order of Section 6.2.5 as executable policy: allow-list first, signature second (constant-time), claims third (with leeway). The `SignatureVerifier`

protocol marks the exact seam where an RS256 backend built on cryptography plugs in.

```

1  """Listing 5.3--Hardened JWT (RFC 7519) verifier with
   _adversarial tests.
2
3  Stdlib-only HS256 implementation built on 'hmac.
   compare_digest', plus a
4  'SignatureVerifier' protocol showing exactly where an
   RS256 backend (e.g.
5  'cryptography') plugs in. The verifier:
6
7  1. rejects 'alg=None' and any algorithm outside an
   explicit allow-list;
8  2. selects the verification key by *configured*
   algorithm, never by the
9  token's 'alg' header alone (defeats RS256->HS256
   confusion);
10 3. verifies the signature BEFORE trusting a single
   claim;
11 4. validates exp/nbf/iat with a bounded clock-skew
   leeway, plus iss/aud.
12
13 All keys are synthetic test fixtures for the fictional
   Northwind Robotics
14 universe. The embedded adversarial tokens are for
   education only.
15 """
16
17 from __future__ import annotations
18
19 import base64
20 import hashlib
21 import hmac
22 import json
23 import time
24 from typing import Callable, Mapping, Protocol
25
26 # --- Synthetic test fixtures (NOT real keys)
   -----
27 HS256_SECRET = b"nwr-test-hs256-secret-do-not-use-in-
   production"
28 ISSUER = "https://idp.northwind-robotics.example/"
29 AUDIENCE = "nwr-parts-api"
30 ALLOWED_ALGS = frozenset({"HS256"})
31 LEEWAY_SECONDS = 30

```

```
32
33
34 class JWTErrror(Exception):
35     """Base class for all token validation failures."""
36
37
38 def b64url_encode(data: bytes) -> str:
39     return base64.urlsafe_b64encode(data).rstrip(b"=").
40         decode("ascii")
41
42 def b64url_decode(segment: str) -> bytes:
43     if not segment or "=" in segment:
44         raise JWTErrror("malformed_base64url_segment")
45     padding = "=" * (-len(segment) % 4)
46     try:
47         return base64.urlsafe_b64decode(segment +
48             padding)
49     except (ValueError, base64.binascii.Error) as exc:
50         raise JWTErrror("malformed_base64url_segment")
51         from exc
52
53 def encode_jwt(header: Mapping[str, object], payload:
54     Mapping[str, object], key: bytes) -> str:
55     """Sign a compact JWS with HS256 (test helper /
56         fixture factory)."""
57     signing_input = f"{b64url_encode(json.dumps(header,
58         separators=(',', ','), ': '))}. " \
59         f"{b64url_encode(json.dumps(payload,
60             separators=(',', ','), ': '))}. "
61     signature = hmac.new(key, signing_input.encode("
62         ascii"), hashlib.sha256).digest()
63     return f"{signing_input}.{b64url_encode(signature)}"
64
65
66 class SignatureVerifier(Protocol):
67     """Pluggable crypto backend. An RS256
68         implementation would call
69     """
70     """ cryptography.hazmat.primitives.asymmetric.rsa """
71     verify_with_PSS/PKCS1v15
72     """ here; the HS256 implementation below uses stdlib
73         HMAC. """
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```

65     def verify(self, signing_input: bytes, signature:
66         bytes) -> bool: ...
67
68 class HS256Verifier:
69     def __init__(self, secret: bytes) -> None:
70         if len(secret) < 32:
71             raise ValueError("HS256_keys_must_be_>=256
72                 bits")
73         self._secret = secret
74
75     def verify(self, signing_input: bytes, signature:
76         bytes) -> bool:
77         expected = hmac.new(self._secret, signing_input
78             , hashlib.sha256).digest()
79         return hmac.compare_digest(expected, signature)
80             # constant time
81
82 class JWTVerifier:
83     def __init__(
84         self,
85         verifiers: Mapping[str, SignatureVerifier],
86         issuer: str,
87         audience: str,
88         leeway: int = LEEWAY_SECONDS,
89         now: Callable[[], float] = time.time,
90     ) -> None:
91         self._verifiers = dict(verifiers)
92         self._issuer = issuer
93         self._audience = audience
94         self._leeway = leeway
95         self._now = now
96
97     def decode(self, token: str) -> dict[str, object]:
98         parts = token.split(".")
99         if len(parts) != 3:
100             raise JWTError("token_must_have_exactly_
101                 three_segments")
102         header_b64, payload_b64, signature_b64 = parts
103         try:
104             header = json.loads(b64url_decode(
105                 header_b64))
106             payload = json.loads(b64url_decode(
107                 payload_b64))
108         except (json.JSONDecodeError,

```

```
UnicodeDecodeError) as exc:
103     raise JWTError("unparsable_header_or_payload") from exc
104 if not isinstance(header, dict) or not
    isinstance(payload, dict):
105     raise JWTError("header_and_payload_must_be_JSON_objects")
106
107 # --- Step 1: algorithm allow-list, BEFORE
    touching anything else. ---
108 alg = header.get("alg")
109 if not isinstance(alg, str) or alg == "none" or
    alg not in self._verifiers:
110     raise JWTError(f"algorithm_not_allowed:_{
        alg!r}")
111 if header.get("crit"):
112     raise JWTError("unsupported_critical_extensions")
113
114 # --- Step 2: verify the signature with the
    CONFIGURED algorithm. ---
115 signature = b64url_decode(signature_b64)
116 signing_input = f"{header_b64}.{payload_b64}".
    encode("ascii")
117 if not self._verifiers[alg].verify(
    signing_input, signature):
118     raise JWTError("invalid_signature")
119
120 # --- Step 3: only now are claims trustworthy
    enough to evaluate. ---
121 self._validate_claims(payload)
122 return payload
123
124 def _validate_claims(self, claims: Mapping[str,
    object]) -> None:
125     now = int(self._now())
126     exp = claims.get("exp")
127     if not isinstance(exp, int):
128         raise JWTError("missing_or_non-numeric_exp")
129
130     if now > exp + self._leeway:
131         raise JWTError("token_expired")
132     nbf = claims.get("nbf")
133     if nbf is not None and (not isinstance(nbf, int)
        ) or now + self._leeway < nbf:
        raise JWTError("token_not_yet_valid")
```



```

134         iat = claims.get("iat")
135         if iat is not None and (not isinstance(iat, int)
136             or iat > now + self._leeway):
137             raise JWTError("iat_is_in_the_future")
138         if claims.get("iss") != self._issuer:
139             raise JWTError("issuer_mismatch")
140         aud = claims.get("aud")
141         audiences = aud if isinstance(aud, list) else [aud]
142         if self._audience not in audiences:
143             raise JWTError("audience_mismatch")
144
145     def _fixture_claims(now: int) -> dict[str, object]:
146         return {
147             "iss": ISSUER,
148             "sub": "svc-parts-cli@northwind-robotics.
149                 example",
150             "aud": AUDIENCE,
151             "exp": now + 300,
152             "nbf": now - 5,
153             "iat": now - 10,
154             "jti": "nwr-jti-0001-synthetic",
155         }
156
157     def main() -> None:
158         FIXED_NOW = 1_800_000_000 # deterministic clock
159         for the demo
160         verifier = JWTVerifier(
161             verifiers={"HS256": HS256Verifier(HS256_SECRET)},
162             issuer=ISSUER,
163             audience=AUDIENCE,
164             now=lambda: FIXED_NOW,
165         )
166
167     def attempt(label: str, token: str) -> None:
168         try:
169             claims = verifier.decode(token)
170             print(f"[{label}]_ACCEPTED_sub={claims['sub']}")
171         except JWTError as exc:
172             print(f"[{label}]_rejected({exc})")
173
174     good = encode_jwt({"alg": "HS256", "typ": "JWT"},

```

```

    _fixture_claims(FIXED_NOW), HS256_SECRET)
174 attempt("1_valid", good)
175
176 # Adversarial 1: alg=none with an empty signature.
177 none_token = (
178     b64url_encode(json.dumps({"alg": "none", "typ":
179         "JWT"}).encode())
180     + "." + b64url_encode(json.dumps(
181         _fixture_claims(FIXED_NOW)).encode()) + "."
182 )
183 attempt("2_alg=none", none_token)
184
185 # Adversarial 2: expired by one hour.
186 expired = _fixture_claims(FIXED_NOW)
187 expired["exp"] = FIXED_NOW - 3600
188 attempt("3_expired", encode_jwt({"alg": "
189     HS256"}, expired, HS256_SECRET))
190
191 # Adversarial 3: minted for a different audience (
192     token confused deputy).
193 wrong_aud = _fixture_claims(FIXED_NOW)
194 wrong_aud["aud"] = "nwr-billing-api"
195 attempt("4_wrong_audience", encode_jwt({"alg": "
196     HS256"}, wrong_aud, HS256_SECRET))
197
198 # Adversarial 4: payload tampered after signing (
199     privilege escalation).
200 header_b64, payload_b64, signature_b64 = good.split
201     (".")
202 evil = _fixture_claims(FIXED_NOW)
203 evil["scope"] = "admin"
204 tampered = f"{header_b64}.{b64url_encode(json.dumps
205     (evil).encode())}.{signature_b64}"
206 attempt("5_tampered_payload", tampered)
207
208 # Adversarial 5: not-before in the future (pre-
209     played token).
210 future = _fixture_claims(FIXED_NOW)
211 future["nbf"] = FIXED_NOW + 3600
212 attempt("6_future_nbf", encode_jwt({"alg": "
213     HS256"}, future, HS256_SECRET))
214
215 if __name__ == "__main__":
216     main()
```

Observed output (deterministic):

```

1 [1 valid          ] ACCEPTED  sub=svc-parts-
   cli@northwind-robotics.example
2 [2 alg=none       ] rejected  (algorithm not allowed:
   'none')
3 [3 expired        ] rejected  (token expired)
4 [4 wrong audience ] rejected  (audience mismatch)
5 [5 tampered payload] rejected  (invalid signature)
6 [6 future nbf     ] rejected  (token not yet valid)

```

6.3.4 Listing 5.4 — Token Cache: Single-Flight Proactive Refresh and Redaction-Safe Logging

The refresh discipline of Section 6.2.3 as a component: an injected clock (deterministic), a double-checked lock (single-flight under concurrency), a skew window (proactive refresh), and a `logging.Filter` that scrubs credential-shaped substrings from every record before any handler can emit them.

```

1  """Listing 5.4 -- Token cache with single-flight
   proactive refresh and
2  redaction-safe logging.
3
4  ‘TokenCache’ guarantees three production properties:
5  * proactive refresh -- tokens are renewed ‘skew’
   seconds BEFORE expiry,
6  * so in-flight requests never ride a token into its
   death window;
7  * single-flight -- no matter how many threads hit an
   expired cache at
8  * once, exactly ONE refresh call reaches the token
   endpoint;
9  * fail-closed -- a refresh error propagates; a
   poisoned cache never
10 * serves a known-dead token as fresh.
11
12 ‘RedactionFilter’ is a ‘logging.Filter’ that scrubs
   bearer tokens,
13 client secrets, and API keys from every record --
   install it on the root
14 logger before any code can log a credential.
15
16 Deterministic: the clock is injected. Synthetic
   Northwind Robotics fixtures
17 only; no network access.

```

```
18 """
19
20 from __future__ import annotations
21
22 import logging
23 import re
24 import threading
25 from dataclasses import dataclass
26 from typing import Callable
27
28 REDACTION_PATTERNS: tuple[re.Pattern[str], ...] = tuple
29 (
30     re.compile(p, re.IGNORECASE)
31     for p in (
32         r"Bearer\s+[A-Za-z0-9\-\._~+/\+=*]",
33         r"(api[_-]?key|client[_-]?secret|access[_-]?token|refresh[_-]?token)",
34         r"[\"]'=\s+[\^\\s\\\"',,}]+\s+[\"]'?",
35     )
36 )
37
38 class RedactionFilter(logging.Filter):
39     """Drop-in logging filter that redacts credential-shaped substrings."""
40
41     def filter(self, record: logging.LogRecord) -> bool:
42         :
43         record.msg = self._scrub(record.getMessage())
44         record.args = ()
45         return True
46
47     @staticmethod
48     def _scrub(text: str) -> str:
49         for pattern in REDACTION_PATTERNS:
50             text = pattern.sub("[REDACTED]", text)
51         return text
52
53 @dataclass(frozen=True)
54 class Token:
55     access_token: str
56     refresh_token: str
57     expires_at: float # epoch seconds
58
59
```

```

60 class TokenCache:
61     def __init__(
62         self,
63         fetch: Callable[[], Token],
64         refresh: Callable[[str], Token],
65         now: Callable[[], float],
66         skew: float = 60.0,
67     ) -> None:
68         self._fetch = fetch
69         self._refresh = refresh
70         self._now = now
71         self._skew = skew
72         self._lock = threading.Lock()
73         self._token: Token | None = None
74         self.refresh_calls = 0 # instrumentation for
                               tests/demos
75
76     def get(self) -> str:
77         token = self._token
78         if token is not None and not self.
           _expiring_soon(token):
79             return token.access_token # fast
           path: no lock
80         with self._lock:
81             token = self._token
82             if token is not None and not self.
               _expiring_soon(token):
83                 return token.access_token # double
               -checked under lock
84             self.refresh_calls += 1 # single
               -flight: one call only
85             self._token = (
86                 self._refresh(token.refresh_token) if
                   token else self._fetch()
87             )
88             return self._token.access_token
89
90     def _expiring_soon(self, token: Token) -> bool:
91         return self._now() >= token.expires_at - self.
           _skew
92
93
94 def main() -> None:
95     # --- Redaction demo
           -----

```

```

96     root = logging.getLogger()
97     root.setLevel(logging.INFO)
98     handler = logging.StreamHandler()
99     handler.addFilter(RedactionFilter())
100    root.addHandler(handler)
101    log = logging.getLogger("nwr.demo")
102
103    # --- Deterministic clock and token factory
104    -----
105    state = {"now": 1_000.0, "counter": 0}
106
107    def make_token(kind: str) -> Token:
108        state["counter"] += 1
109        return Token(
110            access_token=f"at-{{kind}}-{{state['counter']}:02d}-nwr-synthetic",
111            refresh_token=f"rt-{{kind}}-{{state['counter']}:02d}-nwr-synthetic",
112            expires_at=state["now"] + 300.0,
113        )
114
115    cache = TokenCache(
116        fetch=lambda: make_token("initial"),
117        refresh=lambda rt: make_token("refresh"),
118        now=lambda: state["now"],
119        skew=60.0,
120    )
121
122    # 1. First call fetches; second call is served from
123    the cache.
124    first, second = cache.get(), cache.get()
125    print(f"[1]_cached_reuse:_{{first==second}}_({first})")
126
127    # 2. Advance into the proactive-refresh window:
128    expiry-59s <= now.
129    state["now"] += 300.0 - 59.0
130    third = cache.get()
131    print(f"[2]_proactive_refresh_fired:_{{third!=first}}_({third})")
132
133    # 3. Single-flight: expire, then hammer the cache
134    from 8 threads.
135    state["now"] += 300.0 - 30.0
136    before = cache.refresh_calls
137    results: list[str] = []

```

```

134     threads = [threading.Thread(target=lambda: results.
135                               append(cache.get()))
136                for _ in range(8)]
137     for t in threads:
138         t.start()
139     for t in threads:
140         t.join()
141     print(f"[3] threads=8 distinct tokens={len(set(
142         results))}")
143     f"refresh calls made={cache.refresh_calls -
144         before}"
145
146     # 4. Redaction: this must NOT print the credential.
147     log.warning("retrying with Bearer at-refresh-03-nwr-
148         synthetic after 401")
149     log.info("config client_secret='nwr-super-secret-
150         fixture' loaded")
151
152     if __name__ == "__main__":
153         main()

```

Observed output (deterministic):

```

1 retrying with [REDACTED] after 401
2 [1] cached reuse: True (at-initial-01-nwr-synthetic)
3 [2] proactive refresh fired: True (at-refresh-02-nwr-
4   synthetic)
5 [3] threads=8 distinct tokens=1 refresh calls made=1
6 config [REDACTED] loaded

```

6.3.5 Listing 5.5 — Educational Attack Demo: RS256→HS256 Confusion

The confusion attack of Section 6.2.5, made concrete. Because the standard library ships no RSA, we model the asymmetric relationship with a **toy textbook RSA** (small fixed primes, raw pow math, no padding — a teaching stand-in, not cryptography). The point survives the simplification: the naive verifier lets the *token* choose the algorithm while the *configuration* supplies the key, and the forgery verifies; the hardened verifier binds algorithm to key configuration and the attack dies at the allow-list.

```

1 """Listing 5.5 -- EDUCATIONAL ATTACK DEMO: RS256->
2   HS256 algorithm confusion.

```

```
3 The classic JWT library bug: the verifier picks its
   verification ALGORITHM
4 from the attacker-controlled token header while picking
   the KEY from its own
5 RS256 configuration. An attacker who knows the public
   key (public keys are
6 public -- JWKS endpoints publish them) mints an 'alg=
   HS256' token using
7 the public key bytes as the HMAC secret. The naive
   verifier computes
8 'HMAC(public_key, token)' and the forgery verifies.
9
10 This file demonstrates the attack against a naive
   verifier and its failure
11 against a hardened one. Because the Python standard
   library has no RSA, we
12 use a TOY TEXTBOOK RSA (small fixed primes, raw 'pow'
   math, no padding).
13 It exists ONLY to model the asymmetric/private-public
   key relationship --
14 it is not remotely secure cryptography. Use '
   cryptography' in production.
15
16 Synthetic Northwind Robotics fixtures only. Runs fully
   offline.
17 """
18
19 from __future__ import annotations
20
21 import base64
22 import hashlib
23 import hmac
24 import json
25 from typing import Mapping
26
27 # --- Toy textbook RSA (EDUCATIONAL STAND-IN; never use
   in real code) -----
28 _P = 1_299_709 # fixed small primes for deterministic
   keys
29 _Q = 1_299_721
30 N = _P * _Q
31 _E = 65_537
32 _D = pow(_E, -1, (_P - 1) * (_Q - 1))
33 PUBLIC_KEY_PEM = f"-----BEGIN NWR PUBLIC KEY-----\nn={{N
   }}_e={{_E}}\nn-----END NWR PUBLIC KEY-----".encode()
34 PRIVATE_KEY_PEM = f"-----BEGIN NWR PRIVATE KEY-----\nn
```



```

35     = {N}_d={_D}\n-----END_NWR_PRIVATE_KEY-----".encode()
36
37 def _toy_rsa_sign(signing_input: bytes) -> bytes:
38     h = int.from_bytes(hashlib.sha256(signing_input).
39         digest(), "big") % N
40     return pow(h, _D, N).to_bytes((N.bit_length() + 7)
41         // 8, "big")
42
43 def _toy_rsa_verify(signing_input: bytes, signature:
44     bytes) -> bool:
45     h = int.from_bytes(hashlib.sha256(signing_input).
46         digest(), "big") % N
47     return pow(int.from_bytes(signature, "big"), _E, N)
48         == h
49
50 def _b64(data: bytes) -> str:
51     return base64.urlsafe_b64encode(data).rstrip(b"=").
52         decode("ascii")
53
54 def _unb64(segment: str) -> bytes:
55     return base64.urlsafe_b64decode(segment + "=" * (-
56         len(segment) % 4))
57
58 def _mint(header: Mapping[str, object], payload:
59     Mapping[str, object]) -> str:
60     signing_input = (
61         f"{_b64(json.dumps(header, separators=(',', '\n',
62             ': ')).encode())}."
63         f"{_b64(json.dumps(payload, separators=(',', '\n',
64             ': ')).encode())}"
65     ).encode("ascii")
66     if header["alg"] == "RS256": # legit IdP path:
67         signs with PRIVATE key
68         sig = _toy_rsa_sign(signing_input)
69     else: # attacker path: HMAC with the PUBLIC key as
70         the "secret"
71         sig = hmac.new(PUBLIC_KEY_PEM, signing_input,
72             hashlib.sha256).digest()
73     return signing_input.decode("ascii") + "." + _b64(
74         sig)

```

```
66
67 CLAIMS = {"iss": "https://idp.northwind-robotics.
        example/", "sub": "attacker",
68           "aud": "nwr-parts-api", "role": "admin", "exp
           ": 9_999_999_999}
69
70
71 def naive_verify(token: str, configured_key: bytes) ->
    dict[str, object]:
72     """BUG: trusts the token's alg header to choose the
        verification method,
73     while the KEY comes from local config. Classic
        algorithm confusion."""
74     header_b64, payload_b64, sig_b64 = token.split(".")
75     header = json.loads(_unb64(header_b64))
76     signing_input = f"{header_b64}.{payload_b64}".
        encode("ascii")
77     signature = _unb64(sig_b64)
78     if header["alg"] == "RS256":
79         ok = _toy_rsa_verify(signing_input, signature)
80     elif header["alg"] == "HS256":
81         ok = hmac.compare_digest(
82             hmac.new(configured_key, signing_input,
                        hashlib.sha256).digest(), signature
83         )
84     else:
85         ok = False
86     if not ok:
87         raise PermissionError("invalid signature")
88     return json.loads(_unb64(payload_b64))
89
90
91 def hardened_verify(token: str, configured_key: bytes)
    -> dict[str, object]:
92     """FIX: the algorithm allow-list is part of the KEY
        CONFIGURATION, so the
93     token header can never downgrade RS256 to HS256."""
94     header_b64, payload_b64, sig_b64 = token.split(".")
95     header = json.loads(_unb64(header_b64))
96     if header.get("alg") != "RS256": # allow-list of
        exactly one algorithm
97         raise PermissionError(f"algorithm not allowed: {
            header.get('alg')!r}")
98     signing_input = f"{header_b64}.{payload_b64}".
        encode("ascii")
99     if not _toy_rsa_verify(signing_input, _unb64(
```

```

100         sig_b64)):
101         raise PermissionError("invalid_signature")
102     return json.loads(_unb64(payload_b64))
103
104 def main() -> None:
105     forged = _mint({"alg": "HS256", "typ": "JWT"},
106                   CLAIMS)
107     legit = _mint({"alg": "RS256", "typ": "JWT"},
108                  {**CLAIMS, "sub": "svc-parts-cli", "
109                    role": "reader"})
109
110     for name, verifier in (("naive_{}", naive_verify),
111                           ("hardened", hardened_verify)):
112         try:
113             claims = verifier(legit, PUBLIC_KEY_PEM)
114             print(f"[{name}]_legit_RS256_token_{}->_
115                   ACCEPTED_sub={claims['sub']}")
116         except PermissionError as exc:
117             print(f"[{name}]_legit_RS256_token_{}->_
118                   rejected_{(exc)}")
119         try:
120             claims = verifier(forged, PUBLIC_KEY_PEM)
121             print(f"[{name}]_forged_HS256_(confused)_->_
122                   _ACCEPTED!!_role={claims['role']}_"
123                   f"<--_attack_succeeded")
124         except PermissionError as exc:
125             print(f"[{name}]_forged_HS256_(confused)_->_
126                   _rejected_{(exc)}_<--_attack_failed")
127
128 if __name__ == "__main__":
129     main()

```

Observed output (deterministic):

```

1 [naive  ] legit RS256 token      -> ACCEPTED sub=svc-
   parts-cli
2 [naive  ] forged HS256 (confused) -> ACCEPTED!! role=
   admin <-- attack succeeded
3 [hardened] legit RS256 token      -> ACCEPTED sub=svc-
   parts-cli
4 [hardened] forged HS256 (confused) -> rejected (
   algorithm not allowed: 'HS256') <-- attack failed

```

6.4 Common Pitfalls & Anti-Patterns

Each entry below names the failure, explains why it happens, and gives the fix. These are drawn from real incident post-mortems and the OWASP API Security Top 10 [OWA23].

1. **Tokens and keys in URLs.** `?access_token=...` and `?api_key=...` land in access logs, CDN logs, browser history, and `Referer` headers. *Fix:* credentials ride in `Authorization` headers, period; treat any historic URL exposure as a leak requiring rotation.
2. **Logging the Authorization header.** Debug-level HTTP middleware that dumps headers is a credential exfiltration pipeline into your log aggregator. *Fix:* redaction filters at the logger and at the HTTP client layer (Listing 6.3.4); alert on regex matches for token shapes in log pipelines.
3. **client_secret in browser or mobile code.** A secret shipped to every device is public. *Fix:* public clients use authorization code + PKCE with no secret; confidential clients keep secrets server-side.
4. **Accepting whatever alg the token declares.** Enables `alg=none` and RS256→HS256 confusion (Listing 6.3.5). *Fix:* the algorithm allow-list lives in verifier configuration beside the key, and the token's header can only choose among already-approved kids.
5. **Ignoring exp (or validating with unbounded leeway).** A token checked only for signature is valid forever once stolen. *Fix:* require `exp`, cap leeway at 30–60 s, apply it symmetrically to `nbfiat`.
6. **Storing tokens in localStorage without a threat analysis.** Any XSS payload in any dependency reads `localStorage`; a bearer token there is a bearer token for the attacker's C2 server. The honest alternative trade-off — `HttpOnly`; `Secure`; `SameSite` cookies plus CSRF defenses — must be a documented decision, not a default [OWA23].
7. **Infinite retry on 401.** Retrying a request whose token was *rejected* (not expired) in a tight loop is a self-DoS that also looks like credential-stuffing to the provider's WAF — expect IP bans. *Fix:* one refresh + one retry, then fail fast; circuit-breaker semantics belong here as much as they do in service meshes [Nyg18].
8. **One key across environments.** A single API key for dev/staging/prod means the dev laptop leak is a prod breach. *Fix:* per-environment credentials with distinct prefixes and scopes; prod secrets never touch

developer machines.

9. **Comparing signatures with `==`.** Byte-at-a-time early exit leaks match length through timing. *Fix:* `hmac.compare_digest`, always.
10. **Parsing access tokens.** Access tokens are opaque to clients; the JWT-shaped ones are an implementation detail of the AS/RS pair. Clients that branch on access-token claims break when the provider rotates formats. Only ID tokens (OIDC) are for the client to parse.
11. **Caching JWKS forever.** A stale key cache fails closed after every provider rotation — or fails open if a developer “fixes” it by disabling signature checks. *Fix:* cache with TTL, re-fetch once on unknown kid, rate-limit re-fetches.
12. **Home-rolled crypto beyond educational scope.** Listings 6.3.3 and 6.3.5 are deliberately stdlib to expose the mechanics; production verifiers belong to maintained libraries (`pyjwt`, `cryptography`) with security response teams.

6.5 Hands-On Production Lab & Real-World Case Study

6.5.1 Lab: PKCE End-to-End and the JWT Hardening Gauntlet

Objective. Stand up the full local identity stack, run the complete PKCE flow, then run the adversarial JWT gauntlet — all offline.

Setup (PowerShell). From the directory where you saved the listings:

```
1 python --version          # must report 3.11 or newer
2 python listing_5_1_hmac_signer.py
3 python listing_5_2_pkce_flow.py
4 python listing_5_3_jwt_verifier.py
5 python listing_5_4_token_cache.py
6 python listing_5_5_alg_confusion.py
```

Part A — sign and tamper. Run Listing 6.3.1. Confirm step [2] returns 403. Then modify the client to sign with `warehouse=west` while the request line still says `warehouse=east`; predict the result before running, then explain in one sentence which canonical-request line caught the discrepancy.

Part B — PKCE end-to-end. Run Listing 6.3.2. Then instrument `StubIdP.exchange_code` to print the presented verifier length and confirm

it lies in [43, 128]. Break PKCE deliberately: change the client to send a verifier one character longer than the one it challenged with; observe `invalid_grant` and identify the exact comparison that failed.

Part C — rotation forensics. Extend Listing 6.3.2’s `main` to refresh three times, then replay *the first* refresh token. Verify that all later tokens in the family die. This is the behavior your production client must survive gracefully: on family revocation, fall back to a fresh interactive grant, never to infinite retry.

Part D — the gauntlet. Run Listing 6.3.3; all six adversarial tokens must be rejected. Then remove *one* defense at a time (drop the allow-list check; skip `aud`; set `leeway=10**9`) and watch which gauntlet rows flip to `ACCEPTED`. Record the defense-to-attack mapping — it is Table 6.1 made visceral.

Part E — attack and cure. Run Listing 6.3.5. Then port `hardened_verify`’s policy into Listing 6.3.3’s `JWTVerifier` (it is already there — find the three lines that implement it). Explain why “just check `alg != 'none'`” would still have lost to this attack.

Deliverable. A short lab note: the four observed outputs, the Part B failure explanation, and the defense-to-attack table from Part D.

6.5.2 War Story: The CI Log Leak and the 41-Minute Rotation

At Northwind Robotics (fictional composite, drawn from depressingly common real incidents), a build engineer debugging a failing deployment added `echo "using key $NWR_API_KEY"` to a CI pipeline step. The key — a production, write-scoped parts-inventory credential — landed in the build log. The log was “private,” but the CI vendor’s retention meant it was copied to a cold-storage bucket whose bucket policy the security team had flagged, twice, as overly broad. Three weeks later an automated scanner found the bucket listing, and 02:14 on a Saturday the parts API saw a write burst from an unfamiliar ASN: phantom SKU updates pricing robot arms at \$0.01.

The incident worked because of four stacked client-side failures, each of which maps to a section of this chapter. (1) *The credential was over-scoped*: one key held read and write for every warehouse — least privilege (Section 6.2.6) would have bounded the blast radius to a single environment. (2) *The log pipeline had no redaction*: Listing 6.3.4’s filter, installed at the CI log shipper, would have emitted `[REDACTED]`. (3) *The key was long-lived with no rotation runbook*: the on-call had to invent the rotation procedure during the incident. (4) *Detection came from the attacker’s carelessness*, not from tooling: no alert existed on “write burst from new ASN with a valid key.”

The rotation itself took 41 minutes: 6 to confirm the leak, 9 to mint per-service keys, 14 to redeploy four consumers (one had the key baked into a container image, not an environment variable — the Twelve-Factor violation [Wig17] that ate the night), and 12 to verify traffic recovery before revoking the old key. The post-incident actions read like this chapter’s table of contents: per-environment least-privilege keys, redaction at the shipper, a written rotation runbook with a 15-minute target, secret-scanning on CI logs, and anomaly alerts on valid-credential abuse. The lesson worth memorizing: *credential leaks are “when,” not “if” — the engineering variable under your control is the blast radius and the rotation clock.*

6.6 Self-Assessment Exercises & Deep-Dive Questions

Exercises

1. **Entropy audit.** Compute the min-entropy of these key generation schemes and rank them: (a) 22 base64url chars from `secrets`; (b) 36-char UUIDv4; (c) 32 hex chars from `random.Random(42)`; (d) `sha256(hostname + date)`. Which are acceptable for a production API key?
2. **Canonicalization trap.** Extend Listing 6.3.1’s demo: sign a request whose query contains the key `filter` twice with different values. What does the current `canonical_query` do with multi-valued parameters, what does real SigV4 require, and what is the interoperability bug if client and server disagree?
3. **Leeway calculus.** Your fleet’s clocks skew by ± 20 s and p99 request latency is 8 s. Derive a defensible leeway and refresh skew; then state the residual risk of a token replayed within the leeway window and how `jti` caching closes it.
4. **State and nonce.** In Listing 6.3.2, remove the `state` check and describe the exact CSRF attack that becomes possible against a browser-based client. Why does PKCE not obviate `state`?
5. **Gauntlet extension.** Add three new adversarial tokens to Listing 6.3.3: (a) `exp` as a JSON string (`"1893456000"`); (b) `aud` as a list containing your audience and a foreign one — decide and defend accept/reject; (c) a header with `"crit":["exp"]`. Verify each is rejected for the right reason.

6. **Single-flight proof.** In Listing 6.3.4, raise the thread count to 64 and add a 50 ms sleep inside the fake `refresh`. Prove instrumentally that exactly one refresh occurs; then remove the double-check inside the lock and observe the failure mode.
7. **Redaction coverage.** Write five adversarial log lines designed to defeat `RedactionFilter` (e.g., tokens split across concatenated strings, URL-encoded secrets, JSON with unusual spacing). For each that succeeds, write the defense-in-depth control that catches what regexes cannot.

Deep-Dive Questions

1. SigV4 signs the payload hash, not the payload. What class of attack does this *not* prevent, that full request signing including a timestamp and a nonce would? Design the server-side replay cache that closes the gap.
2. Why does the authorization-code grant split the flow across a front channel and a back channel, and which specific properties of browser URLs (Chapter 1’s wire view) make the split necessary?
3. Refresh-token rotation turns a passive leak into an active alarm. Explain the detection logic, then analyze the false-positive case (client crash between receipt and persistence) and design a client that distinguishes it from theft.
4. “JWTs are self-contained, so revocation is impossible” is half of a true sentence. Reconstruct the full cost model: what does stateless verification buy (latency, scale, blast radius of an RS outage), and what is the precise price (revocation gap, claim staleness)? When does introspection win?
5. An auditor asks why your mobile app uses authorization code + PKCE instead of client credentials with an embedded secret, given that “the secret is obfuscated.” Write the three-paragraph answer.

6.7 Interview Q&A Vault

Junior items test vocabulary and hygiene; mid-level items test mechanism; senior/staff items test judgment under trade-offs.

Junior (Q1–Q10)

Q1. Why should API keys go in a header rather than the query string?

A. Query strings are recorded by default in web-server access logs, load balancers, CDNs, APM agents, browser history, and **Referer** headers; headers are not. A credential in a URL is a credential in a log you do not control. Headers also keep the credential out of cache keys and bookmarks.

Q2. Is Base64 encryption? Why is HTTP Basic dangerous over plain HTTP?

A. No — Base64 is a keyless, reversible *encoding*. **Authorization: Basic** is the password, lightly dressed. Over plaintext HTTP any on-path reader recovers the password; over TLS it still means a long-lived reusable secret is transmitted on every request.

Q3. What does “bearer token” mean, and what follows from it?

A. Possession is proof: whoever presents the string is trusted, like cash. Therefore TLS everywhere, short lifetimes, and fanatical storage/logging hygiene on the client — anyone who copies the token owns the session.

Q4. What are the three segments of a JWT and what protects each?

A. Header, payload, signature, base64url-encoded and dot-joined. Only the signature is cryptographically protected; the header and payload are merely *encoded* — readable by anyone holding the token (see Figure 6.3).

Q5. Name the seven registered JWT claims.

A. **iss** (issuer), **sub** (subject), **aud** (audience), **exp** (expiration), **nbf** (not-before), **iat** (issued-at), **jti** (token id).

Q6. What is the difference between authentication and authorization in OAuth terms?

A. Authentication proves who you are (OIDC’s ID token); authorization determines what you may do (OAuth scopes on an access token). OAuth 2.0 alone is a delegation/authorization framework

— that is why OIDC was layered on top.

Q7. What is state for in the authorization-code flow?

A. CSRF protection: the client plants a random value in the authorize request and requires the same value back on the redirect, proving the redirect belongs to the flow this client instance started.

Q8. Why must HMAC signature comparison use `hmac.compare_digest` instead of `==`?

A. `==` exits at the first differing byte, leaking how many leading bytes matched through response timing; `compare_digest` runs in constant time, denying the oracle.

Q9. Where should a CLI tool store a refresh token?

A. OS keychain (via `keyring`) first; an ACL-restricted file with 0600-style permissions as a fallback; never in the project directory, dotfiles under version control, or the process's logged environment.

Q10. What HTTP status should an API return for an expired vs. an invalid token, and why do clients care?

A. Both are 401 (authentication problem), conventionally distinguished by `WWW-Authenticate` error parameters. Clients care because expiry is retryable via refresh while invalidity (revocation, bad signature) must fail fast — retry loops on the latter self-DoS and look like attacks.

Mid-Level (Q11–Q22)

Q11. Why PKCE for public clients?

A. Public clients cannot hold a `client_secret`, so an intercepted authorization code was historically redeemable by anyone. PKCE binds the exchange to the flow initiator: the client sends SHA-256 of a random verifier on the front channel and reveals the verifier only on the back-channel exchange; the AS recomputes and compares. An interceptor with the code but not the verifier holds nothing.

Q12. Walk through the SigV4 signing chain.

A. (1) Canonicalize: method, URI-encoded path, sorted encoded query, normalized sorted headers, signed-header list, SHA-256

of the body — newline-joined. (2) String-to-sign: algorithm id, timestamp, credential scope (date/region/service/request), SHA-256 of the canonical request. (3) Derive the key: HMAC cascade date → region → service → **request**, seeded with **SCHEME+secret**. (4) Signature: hex HMAC of the string-to-sign under the derived key, shipped in **Authorization** with the scope and signed-header list. The cascade domain-separates keys so a signature is valid for exactly one date/region/service context.

Q13. Why is `alg` allow-listing critical?

A. Because the token header is attacker-controlled. If the verifier derives its algorithm from the token, the attacker chooses the verification method: **none** (no signature at all) or HS256 with the RS256 public key as the HMAC secret — which verifies, because the public key is public. Binding algorithm to key configuration makes both choices impossible (Listing 6.3.5).

Q14. ID token vs. access token?

A. The ID token (OIDC) is a JWT whose audience is the client — proof of authentication for local consumption. The access token targets the resource server; the client must treat it as opaque, never parse it, never forward the ID token in its place. Audience enforcement is what makes the distinction meaningful.

Q15. How do you handle refresh-token reuse detection?

A. Rotate on every refresh: the AS retires the presented token and issues a new one. If a retired token is ever presented, either the real client or a thief holds a stale copy, so the AS revokes the entire token family and forces re-authentication. Client side: persist rotated tokens atomically (write-then-swap), or your own crash becomes indistinguishable from theft and logs you out.

Q16. When would you choose client credentials over authorization code?

A. When there is no resource owner: daemon-to-API, CI-to-registry, service-to-service. The client authenticates as itself with its secret and gets a narrowly scoped token. If a human user's consent or identity is involved, client credentials is the wrong tool.

Q17. Describe the device-code grant and its security properties.

A. (RFC 8628) The device gets a `device_code`, a human-readable `user_code`, and a verification URI; the user authorizes on a real browser; the device polls the token endpoint, honoring `authorization_pending/slow_down`. Security: the consent channel is disjoint from the constrained device, user codes are short-lived, and polling intervals rate-limit guessing.

Q18. Why was the implicit grant deprecated?

A. It delivered bearer tokens through the browser URI fragment: tokens in URLs (history, logs, referers), no client authentication, no refresh path, no proof the token reached the requesting client. PKCE gave browser apps a strictly safer alternative, and OAuth 2.1 removes implicit entirely.

Q19. What is kid injection and the fix?

A. If the verifier uses the token's `kid` to build a filesystem path or SQL query, the attacker steers key lookup (`../.. / ' OR 1=1 -`). Fix: `kid` is an opaque key into an in-memory map populated exclusively from the trusted JWKS document; unknown `kid` triggers one rate-limited JWKS refresh, then rejection.

Q20. Explain clock-skew leeway: why it exists and how to bound it.

A. Distributed clocks disagree; without leeway, honest tokens fail at boundaries. Leeway widens validity by seconds in both directions for `exp/nbf/iat`. Bound it to measured fleet skew plus latency (30–60 s typical) — unbounded leeway silently resurrects expired tokens, which is worse than the flakes it prevents.

Q21. Single-flight refresh: what problem does it solve?

A. *N* concurrent threads observe an expired token; without coordination they fire *N* refreshes. Against a rotating AS, refresh $n > 1$ replays a retired token — which reuse detection reads as theft, revoking the family. A lock with double-checked state guarantees exactly one refresh (Listing 6.3.4).

Q22. Why are JWTs “not encrypted,” and when would you use JWE?

A. JWS signs; anyone can base64url-decode the payload. Use JWE only when claims themselves are confidential in transit at

the application layer — rare, because TLS already protects the wire and the endpoint must decrypt anyway. The common real answer: minimize claims instead of encrypting them.

Senior & Staff (Q23–Q32)

Q23. Design the revocation story for a stateless JWT architecture.

A. Accept that signature-validity and authorization-validity decouple. Layer: (1) short access-token TTLs (5–15 min) bounding the gap; (2) refresh rotation with reuse detection as the kill switch; (3) a `jti` or subject-level blocklist consulted only on high-value endpoints (bounded state where it pays); (4) a security-events channel pushing “user disabled” signals to edge verifiers. State explicitly what residue remains: up to TTL seconds of valid access for low-value endpoints.

Q24. Your CI pipeline leaked a production API key into build logs. Walk the first hour.

A. Revoke/rotate first, investigate second — every minute of debate is exposure. Then: contain log access (the log is now a credential store), enumerate what the key could do (scope audit), review usage telemetry for anomalies across the exposure window, rotate anything that touched the same secret store, and only then write the post-mortem. The durable fixes: redaction at the shipper, per-environment scoped keys, and a rehearsed runbook with a rotation SLO (Section 6.5).

Q25. Compare HMAC request signing with bearer tokens for a service-to-service mesh.

A. HMAC: secret never on the wire, integrity per request, replay needs timestamp/nonce plus a cache; cost is canonicalization fragility and shared-secret management at scale. Bearer+TLS: simple, composes with OAuth scopes and identity propagation, but possession is proof — theft equals impersonation until expiry. Staff answer: bearer for identity propagation inside mTLS, HMAC (or proof-of-possession schemes like DPoP) where the channel or the storage cannot be trusted end-to-end.

Q26. An engineer proposes validating JWTs by decoding claims first to route to the right key, then verifying. Reject or accept?

A. Accept only in a precisely bounded form: reading `iss` or `kid` to *select* a key from your trusted configuration is unavoidable in multi-issuer systems, but every claim remains untrusted input until the signature verifies — so the selected key must come from your allow-listed map, the algorithm from that key’s configuration, and all authorization decisions strictly after verification. Routing on unverified claims for *trust* decisions (not just key lookup) is the confused deputy wearing a routing hat.

Q27. How do you scope credentials under least privilege when the provider offers only coarse scopes?

A. Compensate at layers you control: separate credentials per environment and per job; a thin internal broker that exchanges a coarse token for task-shaped, short-lived downscoped ones; network egress policy bounding where the credential can be used from; and telemetry keyed by credential identity so each key’s behavior has a baseline. Document the residual risk as an accepted-risk item, not a footnote.

Q28. Explain why “we rotate keys annually” fails the entropy argument even with perfect key generation.

A. Entropy answers “can it be guessed”; lifetime answers “how long does a leak stay useful.” A 256-bit key leaked on day one is worth 364 days of access under annual rotation. Rotation cadence is a blast-radius decision: set it from exposure telemetry (how fast can you detect?) and revocation cost, not from calendar aesthetics. Automation is what makes a short cadence survivable.

Q29. Design a token-broker service for untrusted developer laptops.

A. Developers hold only short-lived personal OIDC sessions (SSO with phishing-resistant MFA); the broker — a confidential service — holds real provider credentials, mints task-scoped short-lived tokens after policy checks, logs every issuance with redaction-safe audit records, and rotates its own secrets continuously. Laptops never see production-grade keys, so a lost laptop is an account-disable event, not a fleet rotation.

Q30. What breaks first when you scale refresh-token rotation to 10,000 concurrent client instances?

A. Thundering-herd refresh (shared TTLs synchronize expiries —

add jitter), single-flight failures under partial persistence (instance crashes post-rotation read as reuse — tolerate one grace refresh within seconds of rotation, as major providers do), and family-revocation storms cascading into 10,000 simultaneous interactive logins. SLO the token endpoint for refresh bursts, not steady state.

Q31. Assess “store tokens in localStorage” for an SPA.

A. localStorage is readable by any script in the origin — every npm dependency included. The honest options: BFF pattern (tokens stay server-side; the browser holds an `HttpOnly`; `Secure`; `SameSite` cookie with CSRF defenses), or in-memory tokens with silent refresh and accepted XSS exposure. What is not acceptable is the unexamined default. Cite the trade-off in the design doc; auditors and interviewers both ask.

Q32. How would you detect algorithm-confusion regressions in a large polyglot fleet?

A. Continuous adversarial testing, not audits: a CI gate that replays a corpus of attack tokens (`alg=none`, RS256→HS256, kid traversal, foreign aud, expired-within-huge-leeway) against every service’s verifier build — Listing 6.3.3’s gauntlet productized — plus dependency pinning with security-update SLAs and a policy test asserting each service’s configured allow-list has exactly the intended entries.

Q33. Your provider rotates JWKS keys with zero overlap and your fleet fails closed for 20 minutes monthly. Design the fix.

A. Client-side: cache JWKS with a short TTL, refetch once on unknown kid (rate-limited), and retry verification once with refreshed keys before rejecting. Provider-side escalation: publish-before-sign overlap is the industry norm; raise it as a reliability defect with the failing signature windows as evidence. Fleet-side: alert on kid-miss rate so you see the provider event as an event, not as an outage.

6.8 External Bibliography & Further Reading

- **RFC 6749 — The OAuth 2.0 Authorization Framework** [Har12]. The base specification: roles, grants, token endpoint, error codes. Read sections 1–4 and 10 (security considerations) first.
- **RFC 6750 — OAuth 2.0 Bearer Token Usage**. How bearer tokens are carried (header, body, query) and the WWW-Authenticate error vocabulary.
- **RFC 7636 — PKCE** [Sak15]. Fourteen pages that fixed public-client OAuth; the S256 transform and verifier alphabet are specified precisely enough to implement in an afternoon.
- **RFC 8628 — Device Authorization Grant**. The device-code flow: polling discipline, `slow_down`, user-code design.
- **RFC 7519 — JSON Web Token** [Jon15], with **RFC 7515 (JWS)** for the signature construction and **RFC 7518 (JWA)** for the algorithm registry.
- **OpenID Connect Core 1.0**. ID token semantics, discovery, nonce, userinfo — read alongside RFC 6749, never instead of it.
- **OWASP API Security Top 10 (2023)** [OWA23] and the **OWASP Cheat Sheets** (JSON Web Token, Secrets Management, REST Security). The pitfall taxonomy of Section 6.4 in operational form.
- **Justin Richer & Antonio Sanso, *OAuth 2 in Action* (Manning, 2017)**. The best prose walkthrough of the grants, including the attack history behind PKCE.
- **Auth0 and Okta developer documentation**. Vendor-maintained, pragmatic treatments of refresh rotation, reuse detection, and SPA storage trade-offs; read critically — defaults optimize for time-to-first-token, not blast radius.
- **AWS Signature Version 4 documentation**. The canonical reference for the signing chain reproduced in Section 6.2.2.
- **Michael T. Nygard, *Release It!*, 2nd ed.** [Nyg18]. Stability patterns (circuit breaker, fail fast) that govern what your client does *after* a 401.

- **The Twelve-Factor App, factor III** [Wig17]. Config-in-environment discipline: the baseline for keeping credentials out of images and source trees.

6.9 Capstone Projects

All capstones run offline against local stubs, use synthetic Northwind Robotics fixtures only, and must be documented with PowerShell instructions.

Capstone 5.1 — HMAC-Signed Webhook Consumer [junior]

Objective. Build the receiving half of a signed-webhook contract: a local stub that verifies `X-NWR-Signature` headers on incoming delivery events.

Inputs/Datasets. A synthetic JSONL file of 20 delivery events (order updates for fictional SKUs); a fixed synthetic shared secret; three deliberately corrupted events (tampered body, stale timestamp, wrong secret).

Steps. (1) Implement canonical signing (method + path + timestamp + SHA-256 body) from Section 6.2.2, simplified to a single-stage key. (2) Stand up a stub `http.server` verifier using `hmac.compare_digest` and a 5-minute timestamp window. (3) Replay the 20 events; reject the 3 corrupted ones for the correct distinct reasons. (4) Add a replay cache keyed by signature with a 10-minute TTL.

Acceptance Criteria.

- ☐ All 17 valid events accepted; all 3 corrupted events rejected with distinct error messages.
- ☐ Replay of a previously accepted event rejected by the cache.
- ☐ Comparison uses `hmac.compare_digest`; no `==` on secrets anywhere (grep-verifiable).
- ☐ Runs offline; output is byte-for-byte reproducible.

Stretch Goals. Multi-secret key rollover (accept current and previous secret for 24 h); constant-time behavior measurement across matching/non-matching prefixes.

Capstone 5.2 — Full PKCE CLI [mid]

Objective. Ship a complete command-line OAuth client for the fictional Northwind Robotics parts API: login, call, refresh, logout — against the Listing 6.3.2 stub IdP.

Inputs/Datasets. The stub IdP (extended with `/device_authorization` if attempting the stretch); a token file path under the user profile; a synthetic parts dataset.

Steps. (1) Wrap Listing 6.3.2’s client in a CLI with `login`, `parts list`, `refresh`, `logout` subcommands. (2) Persist tokens atomically (write temp file, then rename) with owner-only ACL. (3) Integrate Listing 6.3.4’s cache for proactive, single-flight refresh. (4) Handle family revocation by falling back to a fresh login flow with a clear message — never a retry loop. (5) Install the redaction filter before any logging.

Acceptance Criteria.

- ☐ `login` performs the full PKCE exchange and stores tokens with owner-only permissions.
- ☐ Expired access tokens are refreshed proactively; 8 concurrent invocations cause exactly one refresh call.
- ☐ Replaying a retired refresh token (simulated) revokes the family and the CLI re-authenticates cleanly.
- ☐ No token material appears in any log output (tested by scraping `stderr`).

Stretch Goals. Device-code login mode; OS keychain storage via `keyring` with file fallback.

Capstone 5.3 — JWT Security Linter [mid]

Objective. Build a static analyzer that flags the JWT-handling anti-patterns of Section 6.4 in Python source.

Inputs/Datasets. A corpus of 12 synthetic “client” files seeded with known violations (accepting any `alg`, missing `exp` check, unbounded leeway, `==` on signatures, tokens logged, tokens in URLs) and 3 clean files.

Steps. (1) Use the `ast` module to find `jwt.decode` calls and verifier constructions; extract the effective algorithm set and claim checks. (2) Emit findings as structured records (file, line, rule id, severity, one-line fix). (3) Define exit codes: 0 clean, 1 findings. (4) Validate against the corpus: every seeded violation found, zero false positives on clean files.

Acceptance Criteria.

- ☐ Detects at least the six named rule categories with file and line attribution.
- ☐ Zero false positives on the clean corpus files.
- ☐ Findings output is deterministic (sorted, stable).

- Each rule cites the chapter subsection or OWASP category it enforces [OWA23].

Stretch Goals. Auto-fix mode for the mechanical rules (`==` $\rightarrow$ `compare_digest`); pre-commit integration instructions.

Capstone 5.4 — Token-Broker Service [senior]

Objective. Implement the broker from Q29: a local service that exchanges developer identity (a stub OIDC login) for short-lived, task-scoped tokens — so developer processes never hold long-lived credentials.

Inputs/Datasets. A stub IdP issuing HS256 ID tokens; a policy file mapping fictional roles (`parts-reader`, `parts-writer`) to allowed scopes; a stub resource API enforcing scopes.

Steps. (1) Broker endpoint validates the ID token with Listing 6.3.3's verifier (issuer, audience, exp, allow-list). (2) Policy engine downscopes: requested scope $\cap$ role scope, TTL capped at 15 minutes. (3) Issue task tokens; log issuance through the redaction filter with `jti` recorded. (4) Resource API verifies task tokens and rejects escalated or expired ones. (5) Chaos test: kill the broker mid-refresh; clients must fail fast with actionable errors, not hang or retry-storm [Nyg18].

Acceptance Criteria.

- End-to-end: login $\rightarrow$ broker $\rightarrow$ scoped call succeeds; scope escalation attempt rejected.
- No long-lived secret exists in any client process or file (demonstrated by inspection test).
- Issuance audit log contains `sub`, `jti`, `scope`, `expiry` — and zero token material.
- Broker outage produces bounded, observable client failure (one retry, then exit code 2).

Stretch Goals. `jti` blocklist revocation endpoint with propagation measurement; RS256 issuance using `cryptography`.

Capstone 5.5 — Red-Team Your Own Client [staff]

Objective. Adversarially review the chapter's own code — and your Capstone 5.2/5.4 artifacts — then land the fixes.

Inputs/Datasets. Listings 6.3.1–6.3.5; your capstone artifacts; a written threat model template (assets, trust boundaries, abuse cases).

Steps. (1) Build the attack corpus: timing oracles, replay within leeway, kid confusion, multi-valued query canonicalization mismatch, refresh-family false positives, redaction bypasses. (2) Execute each against the listings; record succeed/fail with evidence. (3) For each success, land the minimal fix and a regression test. (4) Write the review report: finding, severity, blast radius, fix, residual risk. (5) Convert the corpus into a CI gate that fails the build on regression.

Acceptance Criteria.

- ☐ Threat model document with at least 6 abuse cases traced to code.
- ☐ At least 3 genuine findings in the chapter listings (they exist — the stubs simplify for pedagogy), each with fix and test.
- ☐ The CI gate demonstrably fails on a reintroduced vulnerability and passes on the fixed tree.
- ☐ Report quantifies residual risk for each accepted finding.

Stretch Goals. Differential testing of your verifier against `pyjwt` on 10,000 fuzzed tokens; a short internal “lessons learned” writeup suitable for an engineering blog.

6.10 Python Dependencies Appendix

Every listing in this chapter runs on the Python 3.11+ **standard library alone**; the packages below are what you adopt when graduating the patterns to production. The chapter’s pinned `requirements.txt`:

```
1 # Chapter 5 production stack (chapter listings
   themselves are stdlib-only)
2 pyjwt>=2.8,<3          # JWT encode/decode with JWKS
   support
3 cryptography>=42       # real RSA/EC signatures (RS256/
   ES256 verification)
4 httpx>=0.27,<1         # modern HTTP client with
   timeouts and auth middleware
5 keyring>=25             # optional: OS keychain storage
   for tokens
```

6.10.1 pyjwt

Description. The de-facto Python JWT library: encode/decode, claim validation, algorithm allow-lists, JWKS client.

Why included. Listing 6.3.3 is deliberately hand-rolled to expose the verification order; production code must inherit a maintained library's security response. `pyjwt` makes the safe order the default: `algorithms=["RS256"]` is mandatory, `none` is rejected, and `leeway` is a first-class parameter.

Alternatives. `authlib` (broader: full OAuth/OIDC client and server); `python-jose` (historically common, less actively maintained).

Installation (PowerShell).

```
1 python -m pip install "pyjwt>=2.8,<3"
2 python -m pip install "pyjwt[crypto]"      # adds
    cryptography for RS256
```

6.10.2 cryptography

Description. The Python binding to vetted cryptographic primitives (RSA, EC, Ed25519, X.509).

Why included. The standard library has no asymmetric crypto — Listing 6.3.5's toy RSA exists precisely to mark that gap. Real RS256/ES256 verification, and any JWE work, goes through this package.

Alternatives. `pyca/cryptography` is effectively the consensus choice; `pynacl` covers a different (NaCl) primitive set.

Installation (PowerShell).

```
1 python -m pip install "cryptography>=42"
```

6.10.3 httpx

Description. A modern sync/async HTTP client: connection pooling, strict timeouts, middleware-friendly auth hooks, HTTP/2.

Why included. Chapter 2 used `urllib` for pedagogy; production OAuth clients need enforced timeouts (a hung token endpoint must not hang your refresh path), automatic retries with backoff, and an auth middleware seam where Listing 6.3.4's cache plugs in.

Alternatives. `requests` (sync-only, ubiquitous); `aiohttp` (async-first); stdlib `urllib` (no dependencies, no ergonomics).

Installation (PowerShell).

```
1 python -m pip install "httpx>=0.27,<1"
```

6.10.4 keyring (optional)

Description. Cross-platform access to OS credential stores: Windows Credential Manager, macOS Keychain, Linux Secret Service.

Why included. Section 6.2.6’s matrix ranks the OS keychain above files for CLI token storage; **keyring** is the uniform API. Optional because headless CI environments usually have no keychain — fall back to ACL-restricted files or injected environment variables.

Alternatives. Per-OS APIs via **ctypes**/shelling out (do not); cloud secret managers (heavier, network-bound).

Installation (PowerShell).

```
1 python -m pip install "keyring>=25"
```

6.11 Chapter Summary & Key Takeaways

- **Placement is destiny.** Credentials in URLs end up in logs; credentials in headers do not. Header placement, TLS, and redaction filters are the baseline, not the hardening.
- **Canonicalization is the signature.** HMAC request signing (SigV4) derives its strength from deterministic canonical form plus a domain-separated key-derivation chain; tamper with any byte and verification fails closed (Listing 6.3.1).
- **Match the grant to the topology.** Public clients: authorization code + PKCE (43–128 char verifier, S256 challenge). Daemons: client credentials. Constrained devices: device code. Implicit: never again [Har12; Sak15].
- **Refresh is a distributed-systems problem.** Proactive refresh inside a skew window, single-flight under concurrency, rotation with family revocation on reuse — and atomic persistence, or your own crash reads as theft.
- **Verify in order or not at all.** Parse structure, enforce the *configured* algorithm allow-list, verify the signature with constant-time comparison, then validate claims with bounded leeway. Any other order reads attacker-controlled data as truth (Listings 6.3.3 and 6.3.5).
- **JWTs expire; they do not revoke.** Stateless verification trades a revocation gap for scale. Close the gap with short TTLs, rotation, and targeted **jti** blocklists.
- **Leaks are “when.”** Pre-bound the blast radius (least-privilege, per-environment keys), rehearse the rotation runbook, and never let a credential reach a log pipeline.

6.12 Quick-Reference Card

Client topology	Grant
Server-side web app (can hold a secret)	Authorization code (+PKCE for defense in depth)
SPA / mobile / CLI (public client)	Authorization code + PKCE, no secret
Daemon, CI job, service account	Client credentials, narrowest scope
TV, console, headless device	Device code (RFC 8628)
First-party password prompt	Migrate to code+PKCE; never roll your own
Legacy implicit deployment	Security remediation: migrate to code+PKCE

Table 6.3: Grant-selection table.

#	JWT validation checklist (in order)
1	Exactly three segments; base64url decodes; header/payload are JSON objects.
2	<code>alg</code> in configured allow-list; <code>none</code> rejected; alg matches configured key type.
3	<code>kid</code> resolves in your trusted JWKS map (one rate-limited refetch on miss).
4	Signature verifies, constant-time comparison.
5	<code>exp</code> present, $now \leq exp + leeway$.
6	<code>nbf/iat</code> sane within the same leeway.
7	<code>iss</code> exact match; <code>aud</code> contains you.
8	Only now: authorization decisions from claims.

Table 6.4: The only safe JWT validation order.

6.13 Standardized Evidence Addendum

6.13.1 Benchmark Snapshot Template

6.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.

Context	First choice	Fallback	Never
CI/CD pipeline	Secret manager / OIDC federation	Scoped env var injection	Secrets in repo, logs, images
Developer CLI	OS keychain	0600-style ACL file outside repo	Dotfiles, shell history
Server daemon	Secret manager	Env var via orchestrator	Config files in image
Browser SPA	BFF server-side session	In-memory + silent refresh	localStorage without documented analysis

Table 6.5: Credential storage matrix.

- **Result:** what changed in correctness, resilience, latency, and operability.

6.13.3 Cost and Latency Budget Snapshot

6.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```


Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 6.6: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 6.7: Operational budget template for this chapter’s technique.

6.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

6.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

6.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 6.8: Decision card to complete per chapter.

6.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

6.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 7

Building Production REST APIs with FastAPI

Chapters 1–5 taught you to speak HTTP fluently and to consume other people’s APIs defensively. This chapter turns the table: you are now the *provider*. Building an API that strangers can rely on is a different discipline from writing one that works on your laptop — the contract must be explicit, the runtime must survive hostile input and partial failure, the operational surface (health, logs, metrics, shutdown) must exist *before* the first deploy, and every one of those properties must be testable without a network. **FastAPI** [Ram24] is the framework this book uses for that job: a thin, ruthlessly pragmatic layer over Starlette (routing, middleware, the ASGI runtime) and Pydantic v2 (validation and serialization) that turns Python type annotations into a machine-readable OpenAPI 3.1 contract [Ope21].

We begin where production systems actually begin — with the contract — and confront the design-first versus code-first dilemma head-on, settling on a disciplined answer: the OpenAPI document is the backbone either way; the question is only who writes it. We then descend into the runtime: the ASGI callable protocol (`scope/receive/send`), what uvicorn and hypercorn actually do, where Starlette ends and FastAPI begins, and what the event loop means for your choice between `def` and `async def` endpoints. The centerpiece is a complete, production-grade *Northwind Robotics Orders API*: typed Pydantic v2 models with domain invariants, CRUD routes, a repository boundary with in-memory and SQLite implementations behind dependency injection, pure-ASGI middleware carrying correlation IDs into structured JSON logs, liveness/readiness probes, custom exception handlers that never leak stack traces, an NDJSON streaming endpoint, and a customized OpenAPI document — all verified by an in-process `TestClient` suite that runs offline in under a second.

By the end of this chapter you will be able to explain — at the level of interview-grade precision — exactly what happens between a TCP byte

stream arriving at uvicorn and a validated, serialized, logged response leaving it, and you will be able to build that pipeline yourself.

Key Concept

FastAPI is not a web framework so much as a **compiler from type annotations to contracts**. Declare the shape of your data with Pydantic models, declare how to obtain your collaborators with **Depends**, and declare your endpoint signatures; FastAPI derives validation, serialization, dependency resolution, and the OpenAPI schema from those declarations. Your job shifts from writing plumbing to writing *declarations you can defend*.

7.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Argue the design-first versus code-first trade-off quantitatively, and construct an OpenAPI 3.1 document — **servers**, **paths**, **operations**, **components.schemas**, and **securitySchemes** — that serves as a source of truth for implementation, documentation, and client generation [Ope21].
- LO-2.** Explain the ASGI callable protocol (**scope**, **receive**, **send**), the division of labor between uvicorn/hypercorn, Starlette, and FastAPI, and the event-loop implications of every line of handler code you write.
- LO-3.** Trace the complete FastAPI request lifecycle — routing resolution, middleware stack, dependency resolution, Pydantic validation, handler invocation, response serialization — and predict the ordering guarantees of each stage.
- LO-4.** Design dependency-injection graphs with **Depends**: sub-dependencies, per-request caching, **yield**-based cleanup, and **dependency_overrides** for tests.
- LO-5.** Choose correctly between **def** and **async def** endpoints using threadpool semantics, and between **BackgroundTasks** and a real task queue using durability and latency criteria.
- LO-6.** Apply the Pydantic v2 validation pipeline — **field_validator**,

`model_validator`, serialization aliases, and the Rust-backed core — to enforce domain invariants, and use `response_model` as a security boundary that prevents internal fields from leaking [Col24].

LO-7. Operate the service: 12-factor configuration with `pydantic-settings` [Wig17], structured JSON logging with correlation IDs, liveness versus readiness probes, and graceful shutdown via the lifespan protocol.

LO-8. Build, run, and test the complete Orders API offline with `TestClient`/httpx [Enc24], including 422 validation errors, domain errors, health endpoints, and OpenAPI schema assertions.

Prerequisites

This chapter assumes you have completed:

- **Chapter 0** — environment setup: Python 3.11+, a virtual environment, and the ability to run `pip` and `pytest` from PowerShell.
- **Chapter 1** — the HTTP wire protocol: methods, status codes, header fields, and message bodies. Every route in this chapter is an exercise of that vocabulary.
- **Chapter 2** — consuming APIs from Python: `httpx` basics, which the test suite reuses in its ASGI-transport form.
- **Chapter 3** — the REST architectural style: resources, representations, and the uniform interface that the Orders API implements.
- **Chapter 4** — serialization and data contracts: JSON, NDJSON, and the notion of a machine-checkable contract, which OpenAPI operationalizes here.

No network access is required anywhere in this chapter: every test runs against the in-process ASGI application, and every dataset is synthetic, embedded, and deterministic, set in the fictional *Northwind Robotics* parts and logistics universe.

Warning

Version discipline. The code in this chapter targets FastAPI $\geq$ 0.110, Starlette’s modern lifespan API, and Pydantic v2 ($\geq$ 2.7). Py-

dantic v1 idioms — `@validator`, `.dict()`, `class Config` — appear constantly in old blog posts and in large language model output; they are *wrong* here and several raise hard errors under v2. Pin the versions from Section 7.10 before writing a line of code.

7.2 Deep Technical Mechanics & Architectural Concepts

7.2.1 Design-First vs. Code-First: Who Writes the Contract?

Every HTTP API has a contract — a description of its resources, operations, representations, and failure modes — whether or not anyone writes it down. The only choice is whether that contract is an *artifact you author* or an *archaeological finding* your consumers reconstruct from observed behavior. The two disciplined answers to “who writes it first?” define the two workflows:

Definition 7.1 (Design-first (contract-first)). The OpenAPI document is authored *before* implementation, reviewed as an artifact in its own right, and treated as the source of truth. Server stubs, client SDKs, contract tests, and mock servers are generated from it; the implementation is continuously checked for conformance to it.

Definition 7.2 (Code-first (implementation-first)). The contract is *derived* from annotated code. FastAPI is the canonical code-first toolchain: type annotations on path operations and Pydantic models are compiled into an OpenAPI 3.1 document at runtime (`GET /openapi.json`) [Ram24].

Neither is morally superior; they optimize for different failure modes. Design-first pays a front-loaded cost — you must design representations before you can test an idea — but buys parallel workstreams (consumers build against mocks from day one), independent contract review, and immunity to “accidental contract” drift where a refactor silently changes the wire shape. Code-first pays almost nothing up front and keeps the contract bit-identical to the implementation by construction, but risks shipping an accidental contract: whatever the code happens to emit today becomes, *de facto*, a promise. The pragmatic synthesis used throughout this book: *code-first mechanics, design-first governance*. Author the API in FastAPI, but treat the generated `openapi.json` as a reviewed, versioned, CI-asserted artifact — the test suite in Section 7.3.4 asserts its contents exactly.

	Design-first	Code-first (FastAPI)
Source of truth	Hand-written OpenAPI YAML	Type annotations + Pydantic models
Contract fidelity	Drift possible (code may diverge)	Exact by construction
Consumer start time	Immediately (generated mocks)	After first deployable build
Review surface	Diff of the spec file	Diff of code <i>and</i> pinned spec
Best when	Public API, many external consumers	Internal APIs, small teams, fast iteration
Main hazard	Spec/implementation skew	Accidental contract, unreviewed drift

Table 7.1: Design-first vs. code-first: the honest trade-off.

7.2.2 OpenAPI 3.1 as the Backbone

OpenAPI 3.1 [Ope21] describes an HTTP API as a JSON or YAML document with a small set of top-level keys. Know them cold:

- **openapi** — the version string ("3.1.0"). 3.1's headline change: schemas are now *full JSON Schema 2020-12* (Chapter 4), replacing the 3.0 subset with its awkward **nullable** keyword.
- **info** — **title**, **version**, contact and license metadata. The version here is *your API's* version, not the spec's.
- **servers** — an array of base URLs (**url** plus **description**, optionally with templated variables). This is how generated clients and the interactive docs know where the API lives in each environment.
- **paths** — the heart: a map from path templates (`/orders/{order_id}`) to *path items*, each holding one *operation* object per HTTP method. An operation carries **operationId**, **tags**, **parameters**, **requestBody**, and — most importantly — **responses**: a map from status code to a representation schema. A complete operation documents its error shapes too; FastAPI emits the 422 validation response automatically.
- **components** — the reusable parts bin: **schemas** (Pydantic models land here, referenced by **\$ref**), **parameters**, **responses**, **requestBodies**, **headers**, and **securitySchemes**.
- **securitySchemes** / **security** — declarative descriptions of authentication mechanisms (**apiKey**, **http** bearer/basic, **oauth2**, **openIdConnect**)

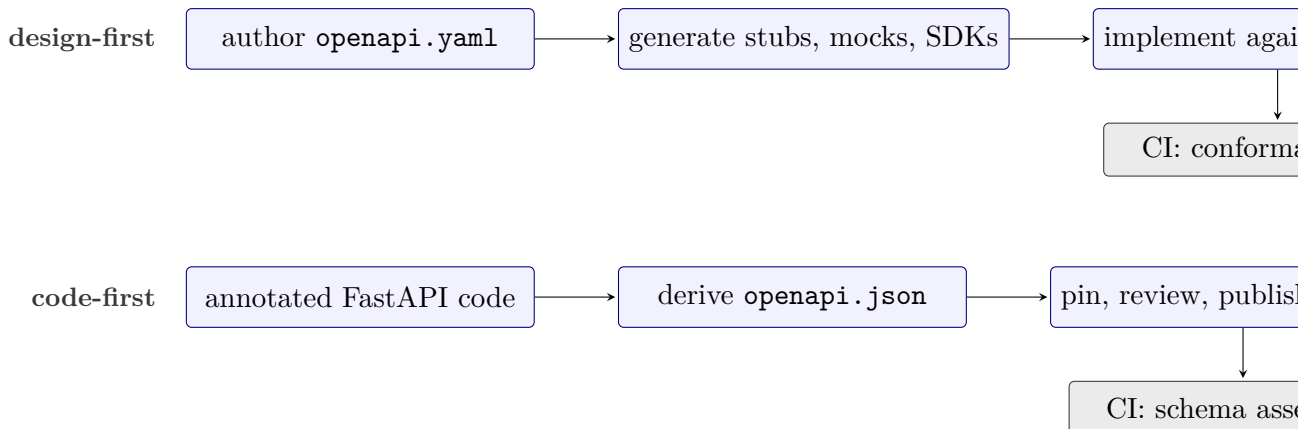


Figure 7.1: The two contract workflows converge on the same discipline: a reviewed OpenAPI artifact and a CI gate. They differ only in which side of the arrow is edited by hand.

and where they apply. Chapter 16 implements enforcement; here we publish the scheme in the contract.

- **tags** — named groups with descriptions; they drive the grouping in generated documentation.

FastAPI generates all of this from your annotations: the `response_model` becomes the success response schema, path and query parameters become `parameters`, the request model becomes `requestBody`, and every Pydantic model is hoisted into `components.schemas`. The customization hook (Section 7.3.1) replaces `app.openapi` to inject `servers`, a `securitySchemes` entry, and extensions such as `x-audience` — the escape hatch that keeps generated contracts publishable as organizational artifacts.

7.2.3 ASGI and the Server Stack

FastAPI code never touches a socket. Beneath it sits a three-layer stack, and knowing exactly where each layer ends is the difference between debugging production and guessing at it.

Definition 7.3 (ASGI). The *Asynchronous Server Gateway Interface*: a callable contract between an HTTP server and a Python application. An ASGI application is `async def app(scope, receive, send):` `scope` is a dict of connection-lifetime metadata (method, path, headers, the `state` bag); `receive` is an awaitable returning the next event (`http.request` body chunks, `http.disconnect`); `send` awaits outbound events (`http.response.start` with status and headers, then `http.response.body` chunks).

1. **The server** (uvicorn or hypercorn) owns TCP sockets, TLS termination, HTTP/1.1 parsing, and — in uvicorn’s [standard] build — the `uvloop` event loop policy and `httptools` parser. Its whole job is translating bytes on the wire into ASGI events and back. Uvicorn is the default choice (C-accelerated, minimal); hypercorn adds HTTP/2 and HTTP/3 (QUIC) support.
2. **Starlette** is the ASGI toolkit: the router that matches `scope["path"]`, the middleware machinery, request/response abstractions, `TestClient`, and the *lifespan* protocol (startup/shutdown as ASGI messages).
3. **FastAPI** is a routing subclass plus a dependency-injection and validation compiler on top of Starlette. When you write `@app.get("/orders")`, you are registering a Starlette route whose endpoint wrapper builds the DI graph, runs Pydantic validation, calls your function, and serializes the result.

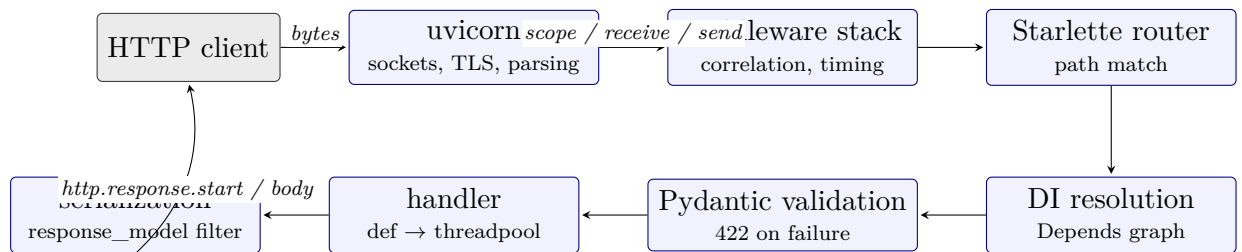


Figure 7.2: The ASGI request lifecycle. Left of the router is the server; the router and middleware are Starlette; DI, validation, and response-model serialization are FastAPI.

Event-loop implications. Uvicorn runs one OS thread per worker process, and that thread runs one event loop. Every coroutine — every middleware, every `async def` endpoint, every streaming generator — shares it. A coroutine that blocks (a synchronous database driver, a `time.sleep`, a CPU-heavy serialization) does not block one request; it blocks *every* request the worker is serving. The escape hatch is the threadpool, discussed next.

7.2.4 Sync vs. Async Endpoints: Threadpool Semantics

FastAPI dispatches on the *declaration* of your endpoint:

- `async def` endpoints are awaited directly **on the event loop**. They are correct when every blocking operation in the body is itself awaitable (async database driver, `httpx` async client, `asyncio.sleep`).

- **def** endpoints are submitted to a threadpool (anyio's limiter, default capacity **40 threads**) and awaited from the loop. The loop stays responsive; the cost is a thread context switch and a hard ceiling of 40 concurrent blocking calls per worker.

This yields a decision rule with no exceptions: *async when the I/O stack is async end-to-end; def when any link in the chain blocks*. The Orders API deliberately declares its CRUD endpoints **def**: the `sqlite3` driver is synchronous, so calling it from **async def** would stall the loop, while the threadpool absorbs it correctly. The corollary is the capacity question: if p99 downstream latency is L ms and arrival rate is λ requests/s, Little's Law says the pool needs $\lambda \cdot L / 1000$ slots; 40 slots at 50 ms latency saturates near 800 requests/s per worker — beyond that, requests *queue inside the pool* and latencies climb without a single error. Section 7.5.2 is the war story of exactly that failure.

When does **async def** genuinely help? When concurrency per worker is the bottleneck and the whole stack is awaitable: proxying, fan-out to several upstream APIs (Chapter 2's async client patterns), WebSockets, and server-streamed responses. For CPU-bound work neither helps — that work belongs in a subprocess or a task queue.

Background tasks vs. task queues. Starlette's `BackgroundTasks` runs callables *after* the response is sent, in the same process: ideal for fire-and-forget side effects that may be lost on deploy (cache warming, a best-effort audit write). It offers no retries, no persistence, no visibility; a worker kill loses the task silently. Anything whose loss is a business event — sending the invoice, provisioning the robot — belongs in a real queue (Celery, arq, a broker-backed outbox; Chapter 11 covers event-driven patterns). The decision is one question: *if this process dies a millisecond after the response, may the work be lost?*

7.2.5 The Dependency-Injection Machine

Depends is FastAPI's second compiler: from callable annotations it builds, per request, a *directed acyclic graph* of providers.

- **Resolution.** A dependency is any callable — function, class, or class instance — whose own signature is analyzed for parameters, so dependencies compose into sub-dependencies. FastAPI resolves the graph depth-first, caching each provider's result *per request*: if a handler and two of its sub-dependencies all request `get_settings`, it executes *once*. The DI showcase (Section 7.3.3) proves the caching empirically with a resolution counter.

- **Yield dependencies and cleanup.** A provider may `yield` its value; code after the `yield` runs after the response has been sent, in LIFO order, and runs *even when the handler raised*. This is where connections close, sessions commit-or-rollback, and locks release. Pre-`yield` code runs during resolution; post-`yield` code is teardown — never put response-affecting logic after the `yield`.
- **Scopes.** Dependencies are request-scoped by default (per-request caching). Process-lifetime singletons belong in `lifespan` state (as the Orders API does with its repository) or in `lru_cache`-backed providers for stateless values such as settings.
- **Overrides.** `app.dependency_overrides` maps the original provider to a replacement, keyed by the callable object itself. It is the canonical test seam: swap the repository for a stub, swap settings for test values, without touching route code — demonstrated in the test suite (Section 7.3.4).

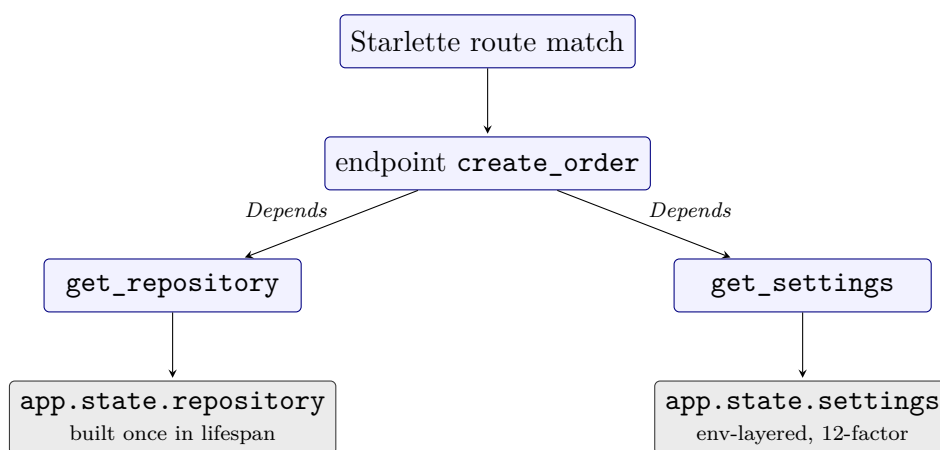


Figure 7.3: The Orders API dependency graph. Providers (white) resolve per-request and are cached once per request; resources (gray) are process-lifetime singletons constructed once in the lifespan and merely *looked up* by providers. Overrides replace any provider node at test time.

7.2.6 Pydantic v2: The Validation Pipeline

Pydantic v2 [Col24] is a rewrite around `pydantic-core`, a Rust validation engine; the Python layer is a thin schema builder. Two consequences matter daily. First, **performance**: validation of a realistic model is commonly 5–20× faster than v1 — fast enough that “validate everything at the boundary” stops being a performance conversation. Second, **the validator taxonomy**:

- `@field_validator("sku")` runs per field, before or after type coercion (`mode="before"` / `mode="after"`), and sees one field in isolation. Use it for per-field normalization and rules — the Orders API lower-cases `customer_id` here.
- `@model_validator(mode="after")` runs once the whole model is coerced, and sees *all* fields. Use it for cross-field domain invariants — the Orders API rejects duplicate SKU lines here, because no single field can see the duplication.
- **Serialization aliases.** `Field(serialization_alias=)` decouples the Python name from the wire name; FastAPI serializes response models with `by_alias=True`, so `customer_id` becomes `customerId` on the wire while Python code keeps idiomatic names. `populate_by_name=True` keeps construction by Python name legal.

The final piece is the one with security consequences: `response_model`. FastAPI does not merely document it — it *re-validates and filters* handler return values through it. Fields absent from `response_model` never reach the wire, which is why the Orders API can return `OrderRecord` objects (carrying `warehouse_hint` and `fraud_score`) from handlers and still guarantee clients only ever see the `OrderRead` contract.

7.2.7 Configuration, Logging, and the Operational Surface

12-factor configuration [Wig17]: config that varies between deploys lives in the environment, never in code. `pydantic-settings` gives this a typed front door: a `BaseSettings` subclass declares fields, defaults, and an `env_prefix` (`NWR_`), and the library layers sources — init arguments, then environment variables, then a `.env` file, then defaults — with earlier sources winning. The layering is the point: tests inject `Settings(repository_backend="memory")` explicitly; production sets `NWR_SQLITE_PATH`; nothing reads `os.environ` ad hoc. **Secrets boundary:** settings objects may *reference* secrets (file paths, key names) but secret material itself enters through the environment at deploy time and is never logged, never returned in a response, and never committed — the full treatment is Chapter 16.

Structured logging with correlation IDs. Text logs are for humans at a laptop; production log pipelines ingest JSON. The Orders API formats every record as one JSON object per line and binds a correlation ID — the incoming `X-Request-Id` or a minted one — into a `contextvars.ContextVar` at the outermost middleware, so every log line emitted anywhere in the request, in any layer, carries the same ID. Chapter 18 builds the full observability stack (metrics, tracing) on this foundation.

Probes. Three distinct questions, three distinct endpoints:

Probe	Question	Checks
Startup	May the process begin receiving probes at all?	Lifespan completion: migrations, cache warmup, config load.
Readiness (<code>/ready</code>)	May we route traffic to this instance?	Dependencies reachable (repository <code>ping()</code>), warmup done.
Liveness (<code>/health</code>)	Is the process wedged beyond recovery?	Nothing external — prove the loop runs; fail $\Rightarrow$ restart.

Table 7.2: Probe semantics. Conflating readiness with liveness causes boot loops: a dependency outage makes liveness fail, the orchestrator restarts the pod, which changes nothing.

Graceful shutdown. Uvicorn’s lifespan protocol sends `lifespan.shutdown` on SIGTERM; the `asyncontextmanager` resumes after its `yield`. The discipline: flip readiness to unavailable *first* (so the load balancer drains the instance), then finish in-flight requests, then close repositories. The Orders API’s `lifespan` implements exactly this ordering.

7.2.8 Response Engineering

The response side of the contract deserves the same rigor as the request side:

- **response_model filtering** — the security boundary from Section 7.2.6: declare the outbound shape, let FastAPI drop everything else.
- **Status codes** — 201 with the created representation for POST, 204 with an empty body for DELETE, 422 for validation failure, 409 for domain-state conflicts (the invalid transition), 404 for missing resources, 503 for unready dependencies. RFC 9110 §15 defines the semantics [Fie22a]; Chapter 7 deepens error bodies into full RFC 9457 problem details.
- **Custom responses** — return a `Response` subclass directly to bypass serialization (the error handlers return `JSONResponse` with `application/problem+json`), or subclass `JSONResponse` for alternative encoders.
- **Streaming** — `StreamingResponse` with an iterator sends body chunks as generated, in constant memory; a *sync* generator runs

on the threadpool, an `async` generator on the loop. The NDJSON endpoint in Section 7.3.1 snapshots the records first, then streams one JSON object per line.

- **ETags and caching** — validators and conditional requests are middleware-level concerns; Chapter 10 builds the ETag middleware this chapter points at.

Example

The accidental-contract trap, concretely. Without `response_model`, returning an `OrderRecord` would serialize `fraud_score` — an internal risk signal — into every client response. No exception is raised; the leak ships silently. Declaring `response_model=OrderRead` makes the leak structurally impossible, and the test suite asserts absence of the field. This is why “always declare `response_model`” is a rule, not a style preference.

7.3 Production-Ready Code Implementation

The chapter ships four complete, runnable Python files. Layout:

```
1 orders_api.py          # the centerpiece: models,
    repositories, DI, routes
2 middleware.py          # pure-ASGI correlation-ID +
    timing middleware
3 di_showcase.py         # DI mechanics: chains, yield
    cleanup, caching, overrides
4 test_orders_api.py     # offline TestClient suite (17
    tests)
```

All four run offline; the test suite completes in about a second. Save them in one directory, then (PowerShell):

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install "fastapi>=0.110" "uvicorn[standard]>=0.29"
    "pydantic>=2.7" '
4         "pydantic-settings>=2.2" "httpx>=0.27" "
        pytest>=8"
5 pytest test_orders_api.py -q
```

7.3.1 The Centerpiece: The Northwind Robotics Orders API

Read the file top to bottom in the order it is written: settings, logging, domain models, repositories, lifespan, dependencies, error mapping, then the application factory. Each layer only depends on layers above it — that ordering discipline is what keeps the DI graph acyclic and the file readable in one pass.

```

1  r"""NorthwindRoboticsOrdersAPI--production-grade
    FastAPIreference.service.
2
3  Everythingrunsofflineanddeterministically.
    Persistenceiseitheran
4  in-memoryrepository(default)orSQLite,selectedby
    settings.
5
6  Runlocally(PowerShell):
7  uvuvicornorders_api:app--port8000
8  Exerciseit:
9  curl.exehttp://localhost:8000/health
10 curl.exe-XPOSThttp://localhost:8000/orders-H"
    Content-Type: application/json"-d'{"customer_id":
    acme","lines":[{"sku":"NWR-1001","quantity":2,"
    unit_price_cents":4999}]}'
12 Testitfullyin-process(no network):
13 uvuvicornpytesttest_orders_api.py-q
14 """
15 from __future__ import annotations
16
17 import json
18 import logging
19 import sqlite3
20 import threading
21 from collections.abc import AsyncIterator, Iterator
22 from contextlib import asynccontextmanager
23 from datetime import datetime, timezone
24 from enum import StrEnum
25 from itertools import count
26 from typing import Annotated, Any, Protocol
27
28 from fastapi import Depends, FastAPI, Query, Request,
    Response, status
29 from fastapi.exceptions import RequestValidationError
30 from fastapi.openapi.utils import get_openapi

```

```
31 from fastapi.responses import JSONResponse,
    StreamingResponse
32 from pydantic import BaseModel, ConfigDict, Field,
    field_validator, model_validator
33 from pydantic_settings import BaseSettings,
    SettingsConfigDict
34
35 from middleware import CorrelationIdMiddleware,
    TimingMiddleware, request_id_var
36
37 logger = logging.getLogger("northwind.orders")
38
39 #
    -----
40 # Configuration (12-factor: everything environment-
    driven, nothing hardcoded)
41 #
    -----
42
43
44 class Settings(BaseSettings):
45     """Environment-layered configuration; NWR_* env
        vars override defaults."""
46
47     model_config = SettingsConfigDict(
48         env_prefix="NWR_", env_file=".env", extra="
            ignore"
49     )
50
51     app_name: str = "northwind-orders-api"
52     app_version: str = "1.0.0"
53     environment: str = "dev" # dev | staging | prod
54     repository_backend: str = "memory" # memory |
        sqlite
55     sqlite_path: str = "orders.db"
56     log_level: str = "INFO"
57
58
59 #
    -----
60 # Structured JSON logging (deepened in Chapter 18)
61 #
    -----
```



```

62
63
64 class JsonFormatter(logging.Formatter):
65     """One JSON object per line, with the correlation
66         ID of the request."""
67
68     def format(self, record: logging.LogRecord) -> str:
69         payload: dict[str, Any] = {
70             "ts": datetime.fromtimestamp(record.created
71                 , tz=timezone.utc).isoformat(),
72             "level": record.levelname,
73             "logger": record.name,
74             "msg": record.getMessage(),
75             "request_id": request_id_var.get(),
76         }
77         for key in ("method", "path", "status_code", "
78             elapsed_ms"):
79             value = getattr(record, key, None)
80             if value is not None:
81                 payload[key] = value
82         if record.exc_info:
83             payload["exc"] = self.formatException(
84                 record.exc_info)
85         return json.dumps(payload, separators=(",", ":"),
86             sort_keys=True)
87
88
89 def configure_logging(level: str) -> None:
90     handler = logging.StreamHandler()
91     handler.setFormatter(JsonFormatter())
92     root = logging.getLogger()
93     root.handlers[:] = [handler]
94     root.setLevel(level.upper())
95
96 #
97 -----
98
99 # Domain model (Pydantic v2)
100 #
101 -----
102
103 class OrderStatus(StrEnum):

```

```

98     PENDING = "pending"
99     CONFIRMED = "confirmed"
100    SHIPPED = "shipped"
101    CANCELLED = "cancelled"
102
103
104    ALLOWED_TRANSITIONS: dict[OrderStatus, frozenset[
105        OrderStatus]] = {
106        OrderStatus.PENDING: frozenset({OrderStatus.
107            CONFIRMED, OrderStatus.CANCELLED}),
108        OrderStatus.CONFIRMED: frozenset({OrderStatus.
109            SHIPPED, OrderStatus.CANCELLED}),
110        OrderStatus.SHIPPED: frozenset(), # terminal
111        OrderStatus.CANCELLED: frozenset(), # terminal
112    }
113
114
115    class OrderLine(BaseModel):
116        """Immutable value object inside an order."""
117
118        model_config = ConfigDict(frozen=True)
119
120        sku: str = Field(pattern=r"^NWR-[0-9]{4}$",
121            description="Northwind part SKU")
122        quantity: int = Field(ge=1, le=1000)
123        unit_price_cents: int = Field(ge=0, le=10_000_000)
124
125
126    class OrderCreate(BaseModel):
127        """Inbound payload. Edge validation here; domain
128            invariants in the model."""
129
130        customer_id: str = Field(min_length=3, max_length
131            =64)
132        lines: list[OrderLine] = Field(min_length=1,
133            max_length=50)
134
135        @field_validator("customer_id")
136        @classmethod
137        def customer_id_must_be_a_slug(cls, value: str) ->
138            str:
139            normalized = value.strip().lower()
140            if not normalized.replace("-", "").replace("_",
141                "").isalnum():
142                raise ValueError(
143                    "customer_id must be alphanumeric(

```

```

135         dashes/underscores_allowed)"
136     )
137     return normalized
138
139     @model_validator(mode="after")
140     def reject_duplicate_skus(self) -> "OrderCreate":
141         skus = [line.sku for line in self.lines]
142         if len(skus) != len(set(skus)):
143             raise ValueError(
144                 "duplicate_SKU_lines_are_not_allowed;_
145                 merge_quantities_instead"
146             )
147         return self
148
149 class OrderStatusUpdate(BaseModel):
150     status: OrderStatus
151
152 class OrderRead(BaseModel):
153     """Outbound_contract:_exactly_what_clients_are_
154     allowed_to_see."""
155
156     model_config = ConfigDict(populate_by_name=True)
157
158     id: str
159     customer_id: str = Field(serialization_alias="
160         customerId")
161     lines: list[OrderLine]
162     status: OrderStatus
163     total_cents: int
164     created_at: datetime
165     version: int
166
167 class OrderRecord(OrderRead):
168     """Stored_record:_a_superset_of_the_outbound_
169     contract.
170
171     """
172     """'warehouse_hint' and 'fraud_score' are internal
173     -only; they never
174     leave the process because every route declares '
175     response_model=OrderRead'
176     and the streaming endpoint excludes them explicitly
177     .
178     """

```

```

173     warehouse_hint: str = "auto"
174     fraud_score: float = 0.0
175
176
177
178 class OrderNotFoundError(LookupError):
179     def __init__(self, order_id: str) -> None:
180         super().__init__(f"order_{order_id!r} not found")
181         self.order_id = order_id
182
183
184 class InvalidTransitionError(ValueError):
185     def __init__(self, current: OrderStatus, target:
186         OrderStatus) -> None:
187         super().__init__(f"cannot transition order from
188             {current} to {target}")
189         self.current = current
190         self.target = target
191
192
193 def transition(record: OrderRecord, target: OrderStatus
194 ) -> OrderRecord:
195     """Pure domain function: the ONLY place a status is
196     allowed to change."""
197     if target not in ALLOWED_TRANSITIONS[record.status
198 ]:
199         raise InvalidTransitionError(record.status,
200             target)
201     return record.model_copy(
202         update={"status": target, "version": record.
203             version + 1}
204     )
205
206 #
207 -----
208
209 # Repository: the persistence boundary, injected as a
210 # dependency
211 #
212 -----
213
214
215 class OrderRepository(Protocol):

```

```

206     def new_id(self) -> str: ...
207     def insert(self, record: OrderRecord) ->
        OrderRecord: ...
208     def get(self, order_id: str) -> OrderRecord: ...
209     def list(self, *, limit: int, offset: int) -> list[
        OrderRecord]: ...
210     def replace(self, record: OrderRecord) ->
        OrderRecord: ...
211     def delete(self, order_id: str) -> None: ...
212     def ping(self) -> bool: ...
213     def close(self) -> None: ...
214
215
216 class InMemoryOrderRepository:
217     """Deterministic process-local store; IDs are
        sequential: ord-0001, ..."""
218
219     def __init__(self) -> None:
220         self._lock = threading.Lock()
221         self._orders: dict[str, OrderRecord] = {}
222         self._sequence = count(1)
223         self._closed = False
224
225     def new_id(self) -> str:
226         with self._lock:
227             return f"ord-{{next(self._sequence):04d}}"
228
229     def insert(self, record: OrderRecord) ->
        OrderRecord:
230         with self._lock:
231             self._orders[record.id] = record
232         return record
233
234     def get(self, order_id: str) -> OrderRecord:
235         try:
236             return self._orders[order_id]
237         except KeyError:
238             raise OrderNotFoundError(order_id) from
                None
239
240     def list(self, *, limit: int, offset: int) -> list[
        OrderRecord]:
241         with self._lock:
242             records = list(self._orders.values()) #
                insertion order = stable
243         return records[offset : offset + limit]

```

```

244
245     def replace(self, record: OrderRecord) ->
246         OrderRecord:
247         with self._lock:
248             if record.id not in self._orders:
249                 raise OrderNotFoundError(record.id)
250             self._orders[record.id] = record
251             return record
252
253     def delete(self, order_id: str) -> None:
254         with self._lock:
255             if self._orders.pop(order_id, None) is None:
256                 :
257                 raise OrderNotFoundError(order_id)
258
259     def ping(self) -> bool:
260         return not self._closed
261
262     def close(self) -> None:
263         self._closed = True
264
265 class SQLiteOrderRepository:
266     """SQLite-backed store. sqlite3 is a BLOCKING
267     driver: call it only from
268     """
269     """def endpoints, which FastAPI runs on the
270     threadpool (Section 6.3)."""
271
272     def __init__(self, path: str) -> None:
273         self._lock = threading.Lock()
274         self._conn = sqlite3.connect(path,
275             check_same_thread=False)
276         self._conn.execute(
277             "CREATE TABLE IF NOT EXISTS orders ("
278             "id TEXT PRIMARY KEY, seq INTEGER NOT NULL,
279             doc TEXT NOT NULL)"
280         )
281         self._conn.commit()
282
283     def new_id(self) -> str:
284         with self._lock:
285             row = self._conn.execute(
286                 "SELECT COALESCE(MAX(seq), 0) + 1 FROM
287                 orders"
288             ).fetchone()
289             return f"ord-{row[0]:04d}"

```

```

283
284 def insert(self, record: OrderRecord) ->
    OrderRecord:
285     seq = int(record.id.rsplit("-", 1)[1])
286     with self._lock:
287         self._conn.execute(
288             "INSERT INTO orders (id, seq, doc)
                VALUES (?, ?, ?)",
289             (record.id, seq, record.model_dump_json(
                )),
290         )
291         self._conn.commit()
292     return record
293
294 def _row_to_record(self, row: tuple[str, str],
    order_id: str) -> OrderRecord:
295     if row is None:
296         raise OrderNotFoundError(order_id)
297     return OrderRecord.model_validate_json(row[1])
298
299 def get(self, order_id: str) -> OrderRecord:
300     row = self._conn.execute(
301         "SELECT id, doc FROM orders WHERE id = ?",
302         (order_id,))
303     ).fetchone()
304     return self._row_to_record(row, order_id)
305
306 def list(self, *, limit: int, offset: int) -> list[
    OrderRecord]:
307     rows = self._conn.execute(
308         "SELECT doc FROM orders ORDER BY seq LIMIT
                ? OFFSET ?", (limit, offset)
309     ).fetchall()
310     return [OrderRecord.model_validate_json(row[0])
311             for row in rows]
312
313 def replace(self, record: OrderRecord) ->
    OrderRecord:
314     with self._lock:
315         cursor = self._conn.execute(
316             "UPDATE orders SET doc = ? WHERE id = ?
                ",
317             (record.model_dump_json(), record.id),
318         )
319         self._conn.commit()
320     if cursor.rowcount == 0:

```

```

319         raise OrderNotFoundError(record.id)
320     return record
321
322     def delete(self, order_id: str) -> None:
323         with self._lock:
324             cursor = self._conn.execute(
325                 "DELETE FROM orders WHERE id=?", (
326                     order_id,)
327             )
328             self._conn.commit()
329             if cursor.rowcount == 0:
330                 raise OrderNotFoundError(order_id)
331
332     def ping(self) -> bool:
333         try:
334             self._conn.execute("SELECT 1")
335             return True
336         except sqlite3.Error:
337             return False
338
339     def close(self) -> None:
340         self._conn.close()
341
342 #
343
344 -----
345
346 # Lifespan: startup and graceful shutdown
347 #
348 -----
349
350 @asynccontextmanager
351 async def lifespan(app: FastAPI) -> AsyncIterator[None]:
352     settings: Settings = app.state.settings
353     configure_logging(settings.log_level)
354     if settings.repository_backend == "sqlite":
355         repository: OrderRepository =
356             SQLiteOrderRepository(settings.sqlite_path)
357     else:
358         repository = InMemoryOrderRepository()
359     app.state.repository = repository
360     app.state.ready = True # startup work done;
361     readiness probe may pass

```



```

357     logger.info(
358         "startup_complete",
359         extra={"backend": settings.repository_backend,
360              "env": settings.environment},
361     )
362     try:
363         yield # the application serves requests here
364     finally:
365         app.state.ready = False # stop passing
366         # readiness before teardown
367         repository.close()
368         logger.info("shutdown_complete")
369
370 #
371
372 -----
373
374 # Dependencies
375 #
376 -----
377
378 def get_settings(request: Request) -> Settings:
379     return request.app.state.settings
380
381 def get_repository(request: Request) -> OrderRepository:
382     return request.app.state.repository
383
384 SettingsDep = Annotated[Settings, Depends(get_settings)]
385 RepoDep = Annotated[OrderRepository, Depends(get_repository)]
386
387 #
388
389 -----
390
391 # Error mapping: domain errors -> RFC 9457-shaped
392 # problem documents (ch. 7)
393 #
394 -----

```

```
389
390
391 def problem(
392     status_code: int, title: str, detail: str, **extra:
393         Any
394 ) -> JSONResponse:
395     body = {
396         "type": "about:blank",
397         "title": title,
398         "status": status_code,
399         "detail": detail,
400         **extra,
401     }
402     return JSONResponse(
403         status_code=status_code,
404         content=body,
405         media_type="application/problem+json",
406     )
407
408 async def order_not_found_handler(
409     request: Request, exc: OrderNotFoundError
410 ) -> JSONResponse:
411     return problem(404, "Order not found", str(exc),
412                    order_id=exc.order_id)
413
414 async def invalid_transition_handler(
415     request: Request, exc: InvalidTransitionError
416 ) -> JSONResponse:
417     return problem(
418         409,
419         "Invalid state transition",
420         str(exc),
421         current=exc.current.value,
422         target=exc.target.value,
423     )
424
425
426 async def validation_error_handler(
427     request: Request, exc: RequestValidationError
428 ) -> JSONResponse:
429     # Sanitize: never forward raw Pydantic ctx objects
430     # to clients.
431     errors = [
432         {
```

```

432         "loc": ".".join(str(part) for part in error
433             ["loc"]),
434         "msg": error["msg"],
435         "type": error["type"],
436     }
437     for error in exc.errors()
438 ]
439 return problem(422, "Validation_␣failed", "Request_␣
440     payload_␣is_␣invalid", errors=errors)
441
442 async def unhandled_error_handler(request: Request, exc
443     : Exception) -> JSONResponse:
444     # Log the full stack trace server-side; return NONE
445     # of it to the client.
446     logger.error("unhandled_␣error_␣on_␣%s", request.url.
447         path, exc_info=exc)
448     return problem(500, "Internal_␣server_␣error", "An_␣
449         unexpected_␣error_␣occurred")
450
451 #
452 -----
453
454 # Application factory
455 #
456 -----
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```
466     app.add_middleware(CorrelationIdMiddleware)
467
468     app.add_exception_handler(OrderNotFoundError,
469                               order_not_found_handler)
469     app.add_exception_handler(InvalidTransitionError,
470                               invalid_transition_handler)
470     app.add_exception_handler(RequestValidationError,
471                               validation_error_handler)
471     app.add_exception_handler(Exception,
472                               unhandled_error_handler)
472
473     # -- operational endpoints
473     -----
474
475     @app.get("/health", tags=["ops"])
476     async def health() -> dict[str, str]:
477         # Liveness: is the process alive and able to
478         # run a coroutine?
479         return {"status": "ok"}
479
480     @app.get("/ready", tags=["ops"])
481     def readiness(request: Request, repo: RepoDep) ->
482     JSONResponse:
482         # Readiness: MAY we receive traffic? Checks
483         # dependencies, not the loop.
483         if not request.app.state.ready or not repo.ping
484         ():
484             return JSONResponse(
485                 status_code=status.
486                 HTTP_503_SERVICE_UNAVAILABLE,
486                 content={"status": "unavailable"},
487             )
488         return JSONResponse(content={"status": "ready"
489                                     })
489
490     # -- orders CRUD
490     -----
491
492     @app.post(
493         "/orders",
494         response_model=OrderRead,
495         status_code=status.HTTP_201_CREATED,
496         tags=["orders"],
497     )
498     def create_order(payload: OrderCreate, repo:
```

```

RepoDep) -> OrderRecord:
499     record = OrderRecord(
500         id=repo.new_id(),
501         customer_id=payload.customer_id,
502         lines=payload.lines,
503         status=OrderStatus.PENDING,
504         total_cents=sum(
505             line.quantity * line.unit_price_cents
506             for line in payload.lines
507         ),
508         created_at=datetime.now(timezone.utc),
509         version=1,
510     )
511     return repo.insert(record)
512
513 @app.get("/orders", response_model=list[OrderRead],
514         tags=["orders"])
515 def list_orders(
516     repo: RepoDep,
517     limit: Annotated[int, Query(ge=1, le=100)] =
518         20,
519     offset: Annotated[int, Query(ge=0)] = 0,
520 ) -> list[OrderRecord]:
521     return repo.list(limit=limit, offset=offset)
522
523 @app.get("/orders/{order_id}", response_model=
524         OrderRead, tags=["orders"])
525 def get_order(order_id: str, repo: RepoDep) ->
526         OrderRecord:
527     return repo.get(order_id)
528
529 @app.patch("/orders/{order_id}", response_model=
530         OrderRead, tags=["orders"])
531 def update_order_status(
532     order_id: str, payload: OrderStatusUpdate, repo
533     : RepoDep
534 ) -> OrderRecord:
535     current = repo.get(order_id)
536     return repo.replace(transition(current, payload
537         .status))
538
539 @app.delete("/orders/{order_id}", status_code=
540         status.HTTP_204_NO_CONTENT,
541         tags=["orders"])
542 def delete_order(order_id: str, repo: RepoDep) ->
543         Response:

```

```

534         repo.delete(order_id)
535     return Response(status_code=status.
536                     HTTP_204_NO_CONTENT)

536
537 @app.get("/orders-stream", tags=["orders"])
538 def stream_orders(repo: RepoDep) ->
539     StreamingResponse:
540     # Snapshot first; the sync generator below runs
541     # on the threadpool.
542     records = repo.list(limit=10_000, offset=0)

543     def ndjson() -> Iterator[bytes]:
544         for record in records:
545             # Internal-only fields must not cross
546             # the wire on ANY path.
547             line = record.model_dump_json(
548                 by_alias=True, exclude={"
549                     warehouse_hint", "fraud_score"}
550             )
551             yield line.encode("utf-8") + b"\n"

552     return StreamingResponse(ndjson(), media_type="
553         application/x-ndjson")

554
555 # -- OpenAPI customization
556 -----

557
558 def custom_openapi() -> dict[str, Any]:
559     if app.openapi_schema:
560         return app.openapi_schema
561     schema = get_openapi(
562         title="Northwind_Robotics_Orders_API",
563         version=settings.app_version,
564         description=(
565             "Fictional_order_management_for_the_
566             Northwind_Robotics_
567             parts_universe._Synthetic_data_only."
568         ),
569         routes=app.routes,
570         servers=[
571             {"url": "http://localhost:8000", "
572              description": "local_dev"},
573             {"url": "https://api.northwind-robotics
574              .example",
575              "description": "production_(fictional)"}

```

```

569         },
570     )
571     # Document the API-key scheme used by the edge
572     # gateway. Enforcement
573     # itself is Chapter 16's subject; here we only
574     # publish the contract.
575     schema.setdefault("components", {}).setdefault(
576         "securitySchemes", {})[
577         "ApiKeyAuth"
578     ] = {"type": "apiKey", "in": "header", "name":
579         "X-API-Key"}
580     schema["info"]["x-audience"] = "internal"
581     app.openapi_schema = schema
582     return app.openapi_schema
583
584 app.openapi = custom_openapi # type: ignore[method
585                               -assign]
586 return app
587
588 # Module-level instance for 'uvicorn orders_api:app'.
589 app = create_app()

```

Walk the file once more with the mechanics of Section 7.2 in mind; six design decisions deserve a second look:

1. **The factory, not the module global.** `create_app` takes a `Settings` and returns a configured `FastAPI`. Tests get isolated applications with deterministic state; `uvicorn` still gets a module-level `app`. This is the single most important structural habit in the file.
2. **OrderCreate vs. OrderRead vs. OrderRecord.** Three models, three audiences: inbound validation (with a cross-field `model_validator`), outbound contract (with a serialization alias), and the internal super-set. The `response_model=OrderRead` declaration on every route is the firewall between the second and third.
3. **Repositories behind a Protocol.** Routes never name a storage engine; they name the protocol and receive whatever the lifespan constructed (memory or `SQLite`) — or whatever a test injected via `dependency_overrides`. Deterministic, sequential IDs (`ord-0001`, `ord-0002`, ...) keep every test assertion exact; a production repository would mint `UUIDv7s` at the same seam.
4. **def endpoints, on purpose.** `sqlite3` is a blocking driver, so

the CRUD handlers are plain `def` and run on the threadpool (Section 7.2.4). Only `/health` — which touches nothing blocking — is `async def`.

5. **Errors are mapped, not raised ad hoc.** Domain exceptions (`OrderNotFoundError`, `InvalidTransitionError`) are raised deep in the call stack and translated to RFC 9457-shaped problem documents by registered handlers; the catch-all handler logs the full traceback server-side and returns none of it.
6. **The OpenAPI hook publishes the contract.** `custom_openapi` adds `servers`, an `ApiKeyAuth` security scheme (enforcement is Chapter 16’s subject), and an `x-audience` extension, then caches the schema on `app.openapi_schema` so generation happens once.

Key Concept

The lifespan is the composition root. Everything process-lifetime — the settings object, the repository, the logging configuration — is constructed in exactly one place (`lifespan/create_app`) and *looked up* everywhere else. Dependencies look up; they do not build. When you find a dependency constructing its own database pool per request, the composition root has been bypassed.

7.3.2 Middleware Deep-Dive: Correlation IDs and Timing

Both middlewares are written against the raw ASGI protocol — no FastAPI or Starlette imports — which is exactly the point: at this layer the framework *is* the protocol. Study how `send` is wrapped to intercept `http.response.start` (the only moment headers can still be added), and how the correlation middleware guards non-HTTP scopes (lifespan, websocket) before touching anything.

```

1  """Pure-ASGI middleware: correlation IDs and request
   timing.
2
3  Both classes implement the ASGI callable protocol
   directly --
4  ‘ ‘ __call__ (scope, receive, send) ‘ ‘ -- so nothing here
   depends on FastAPI
5  or Starlette. Registration order matters: the LAST
   middleware added is the
6  OUTERMOST (runs first on the request path, last on the
   response path):

```



```

7
8 app.add_middleware(TimingMiddleware)#####
   added_1st->inner
9 app.add_middleware(CorrelationIdMiddleware)###
   added_2nd->outer
10
11 Because CorrelationIdMiddleware is outermost, the
   request-id context
12 variable is already bound when TimingMiddleware logs,
   so every access-log
13 line carries the correlation ID.
14 """
15 from __future__ import annotations
16
17 import logging
18 import time
19 import uuid
20 from contextvars import ContextVar
21 from typing import Any, Awaitable, Callable,
   MutableMapping
22
23 # The three callables that make up the ASGI protocol
   surface.
24 ASGIScope = MutableMapping[str, Any]
25 ASGIMessage = MutableMapping[str, Any]
26 ASGIReceive = Callable[[], Awaitable[ASGIMessage]]
27 ASGISend = Callable[[ASGIMessage], Awaitable[None]]
28 ASGIApp = Callable[[ASGIScope, ASGIReceive, ASGISend],
   Awaitable[None]]
29
30 # Bound once per request by CorrelationIdMiddleware;
   read by the JSON log
31 # formatter so every log line inside the request
   carries the same ID.
32 request_id_var: ContextVar[str] = ContextVar("
   request_id", default="-")
33
34 logger = logging.getLogger("northwind.access")
35
36
37 class CorrelationIdMiddleware:
38     """Extract (or mint) X-Request-Id, bind it, and
   echo it on the response."""
39
40     def __init__(self, app: ASGIApp) -> None:
41         self.app = app

```

```

42
43     async def __call__(
44         self, scope: ASGIScope, receive: ASGIReceive,
45         send: ASGISend
46     ) -> None:
47         if scope["type"] != "http": # pass lifespan/
48             websocket through untouched
49             await self.app(scope, receive, send)
50             return
51
52         headers = {k.lower(): v for k, v in scope.get("
53             headers", [])}
54         request_id = (
55             headers.get(b"x-request-id", b "").decode("
56                 latin-1")
57             or uuid.uuid4().hex[:16]
58         )
59         token = request_id_var.set(request_id)
60         scope["request_id"] = request_id # visible to
61             the app via request.scope
62
63     async def send_with_request_id(message:
64         ASGIMessage) -> None:
65         if message["type"] == "http.response.start"
66             :
67             message.setdefault("headers", [])
68             message["headers"].append(
69                 (b"x-request-id", request_id.encode(
70                     "latin-1"))
71             )
72             await send(message)
73
74     try:
75         await self.app(scope, receive,
76             send_with_request_id)
77     finally:
78         request_id_var.reset(token) # never leak
79             IDs across requests
80
81 class TimingMiddleware:
82     """Measure downstream wall time, add X-Process-Time
83         -Ms, log one JSON line."""
84
85     def __init__(self, app: ASGIApp) -> None:
86         self.app = app

```

```

77
78     async def __call__(
79         self, scope: ASGIScope, receive: ASGIReceive,
80         send: ASGISend
81     ) -> None:
82         if scope["type"] != "http":
83             await self.app(scope, receive, send)
84             return
85
86         started = time.perf_counter()
87         status_code = 0
88
89         async def send_with_timing(message: ASGIMessage
90             ) -> None:
91             nonlocal status_code
92             if message["type"] == "http.response.start"
93                 :
94                 status_code = message["status"]
95                 elapsed_ms = (time.perf_counter() -
96                     started) * 1000.0
97                 message.setdefault("headers", [])
98                 message["headers"].append(
99                     (b"x-process-time-ms", f"{
100                         elapsed_ms:.2f}".encode("ascii"))
101                 )
102             )
103             await send(message)
104
105         try:
106             await self.app(scope, receive,
107                 send_with_timing)
108         finally:
109             # Logs even on unhandled exceptions (
110                 status_code stays 0).
111             elapsed_ms = (time.perf_counter() - started
112                 ) * 1000.0
113             logger.info(
114                 "request_completed",
115                 extra={
116                     "method": scope["method"],
117                     "path": scope["path"],
118                     "status_code": status_code,
119                     "elapsed_ms": round(elapsed_ms, 2),
120                     "request_id": request_id_var.get(),
121                 },
122             )

```

Ordering semantics. `add_middleware` wraps the application like layers of an onion: the *last* middleware added is the *outermost* — it runs first on the request path and last on the response path. The Orders API adds `TimingMiddleware` first and `CorrelationIdMiddleware` second, so correlation wraps timing: the context variable is already bound when the timing middleware emits its access-log line, and that line carries `request_id`. Reverse the two `add_middleware` calls and every access log would show `request_id="-"`. The `finally` in both classes matters for the same reason: an unhandled exception downstream must still reset the context variable and must still produce a log line (with `status_code` left at 0 — a silent, greppable marker of “never reached response start”).

Warning

BaseHTTPMiddleware vs. pure ASGI. Starlette’s `BaseHTTPMiddleware` is convenient (request/response objects instead of raw messages) but historically buffered streaming responses, broke `request.context` propagation into background tasks, and added a task-hop of overhead per request. Pure-ASGI middleware, as above, has none of those costs and forces you to learn the protocol you are operating on. Reach for `BaseHTTPMiddleware` only when the request/response API genuinely simplifies the code.

7.3.3 The DI Showcase: Chains, Yield-Cleanup, Caching, Overrides

A minimal, self-checking application that isolates the four DI mechanics. Run it directly — `python di_showcase.py` — and it proves its own claims with `TestClient` assertions.

```

1  """Dependency-injection showcase: chains, yield cleanup
   , caching, overrides.
2
3  Self-checking demo (offline, in-process):
4  """python di_showcase.py
5  """
6  from __future__ import annotations
7
8  import sqlite3
9  from collections.abc import Iterator
10 from typing import Annotated, Protocol
11
12 from fastapi import Depends, FastAPI
13 from fastapi.testclient import TestClient
14

```

```

15
16 # --- a tiny domain: a quote-of-the-day service
17     -----
18
19 class QuoteSource(Protocol):
20     def quote(self) -> str: ...
21
22
23 class HandbookQuoteSource:
24     """The 'real' implementation: reads the Northwind
25        SRE handbook."""
26
27     def quote(self) -> str:
28         return "Robots do not panic. -- Northwind SRE
29            handbook"
30
31
32
33 QUOTE_RESOLUTIONS = {"count": 0} # observable side
34     effect for the caching demo
35
36
37
38 def get_quote_source() -> HandbookQuoteSource:
39     QUOTE_RESOLUTIONS["count"] += 1
40     return HandbookQuoteSource()
41
42
43
44 QuoteDep = Annotated[QuoteSource, Depends(
45     get_quote_source)]
46
47
48
49 def get_audit_trail(source: QuoteDep) -> list[str]:
50     """Sub-dependency: depends on the SAME QuoteSource
51        the handler requests."""
52     return [f"audit: served quote of length {len(source
53         .quote())}"]
54
55
56
57 # --- a yield dependency with guaranteed cleanup
58     -----
59
60
61
62 def get_connection() -> Iterator[sqlite3.Connection]:
63     """One in-memory SQLite connection per request,
64        closed afterwards."""
65     conn = sqlite3.connect(":memory:",

```

```

        check_same_thread=False)
52 conn.execute("CREATE TABLE visits (path TEXT)")
53 conn.execute("INSERT INTO visits (path) VALUES ('/
    demo')")
54 try:
55     yield conn # the endpoint (and sub-
        dependencies) run here
56 finally:
57     conn.close() # runs even when the endpoint
        raises
58
59
60 ConnectionDep = Annotated[sqlite3.Connection, Depends(
    get_connection)]
61
62
63 def create_app() -> FastAPI:
64     app = FastAPI(title="di-showcase")
65
66     @app.get("/demo")
67     def demo(
68         direct: QuoteDep, # requested here AND inside
            get_audit_trail ...
69         audit: Annotated[list[str], Depends(
            get_audit_trail)],
70         conn: ConnectionDep,
71     ) -> dict[str, object]:
72         rows = conn.execute("SELECT COUNT(*) FROM
            visits").fetchone()[0]
73         return {
74             "quote": direct.quote(),
75             "audit": audit,
76             "visits": rows,
77             "quote_resolutions": QUOTE_RESOLUTIONS["
                count"],
78         }
79
80     return app
81
82
83 class FallbackQuoteSource:
84     """Test double injected via dependency overrides."""
85
86     def quote(self) -> str:
87         return "synthetic fallback quote"

```

```

88
89
90 if __name__ == "__main__":
91     # 1) Per-request caching: the dependency graph
92     #     requests QuoteSource twice
93     #     (directly + via the audit sub-dependency) yet
94     #     resolves it ONCE.
95     with TestClient(create_app()) as client:
96         first = client.get("/demo").json()
97         second = client.get("/demo").json()
98         assert first["quote_resolutions"] == 1, first
99         assert second["quote_resolutions"] == 2, second
100         print("per-request_caching_OK:", first["
101             quote_resolutions"],
102             "->", second["quote_resolutions"])
103
104     # 2) Overrides: swap the implementation without
105     #     touching route code.
106     app = create_app()
107     app.dependency_overrides[get_quote_source] =
108         FallbackQuoteSource
109     with TestClient(app) as client:
110         body = client.get("/demo").json()
111         assert body["quote"] == "synthetic_fallback_quote",
112             body
113     print("override_OK:", body["quote"])

```

Three observations. First, the demo handler requests `QuoteSource` directly *and* transitively through `get_audit_trail`, yet `quote_resolutions` increments once per request — per-request caching in action. Second, `get_connection` is a generator dependency: the connection lives exactly one request and its finally runs even when the handler raises; this is the pattern for sessions, transactions, and locks. Third, `dependency_overrides` is keyed by the *original callable*, not by name — override `get_quote_source` and every dependent (including `get_audit_trail`) sees the replacement, because resolution happens through the graph, not through imports.

7.3.4 The Test Suite: Offline, In-Process, Deterministic

`TestClient` [Enc24] runs the real ASGI application in-process — routing, middleware, DI, validation, serialization — with no socket and no network. Used as a context manager it also drives the *lifespan*, so startup and graceful shutdown are under test too, not just the routes. The suite covers the four contract surfaces that matter: happy-path CRUD, validation errors (422),

domain errors (404/409), and the generated OpenAPI document itself.

```

1  """TestClient suite for the Orders API: fully offline,
   fully deterministic."""
2  from __future__ import annotations
3
4  import json
5
6  import pytest
7  from fastapi.testclient import TestClient
8
9  import di_showcase
10 from orders_api import (
11     InMemoryOrderRepository,
12     OrderNotFoundError,
13     Settings,
14     create_app,
15     get_repository,
16 )
17
18 VALID_ORDER = {
19     "customer_id": "Acme-Robotics",
20     "lines": [{"sku": "NWR-1001", "quantity": 2, "
21               unit_price_cents": 4999}],
22 }
23
24 @pytest.fixture()
25 def client() -> TestClient:
26     # The context manager drives the lifespan: startup
27     # runs, readiness flips,
28     # and shutdown/close is exercised on exit.
29     app = create_app(Settings(repository_backend="
30                           memory"))
31     with TestClient(app) as test_client:
32         yield test_client
33
34 # -- operational endpoints
35 -----
36
37 def test_health_and_readiness(client: TestClient) ->
   None:
38     assert client.get("/health").json() == {"status": "
   ok"}

```



```

38     assert client.get("/ready").json() == {"status": "
        ready"}
39
40
41 def test_readiness_fails_closed(client: TestClient) ->
    None:
42     client.app.state.ready = False # simulate mid-
        shutdown / failed warmup
43     response = client.get("/ready")
44     assert response.status_code == 503
45     assert response.json() == {"status": "unavailable"}
46
47
48 # -- CRUD happy paths
    -----

49
50
51 def test_create_order_filters_internal_fields(client:
    TestClient) -> None:
52     response = client.post("/orders", json=VALID_ORDER)
53     assert response.status_code == 201
54     body = response.json()
55     assert body["id"] == "ord-0001" # deterministic
        IDs
56     assert body["status"] == "pending"
57     assert body["total_cents"] == 9998
58     assert body["customerId"] == "acme-robotics" #
        alias + normalization
59     assert "customer_id" not in body
60     # response_model=OrderRead strips internal-only
        fields:
61     assert "warehouse_hint" not in body
62     assert "fraud_score" not in body
63
64
65 def test_get_and_list_orders(client: TestClient) ->
    None:
66     client.post("/orders", json=VALID_ORDER)
67     client.post(
68         "/orders",
69         json={
70             "customer_id": "globex",
71             "lines": [{ "sku": "NWR-2002", "quantity":
                1, "unit_price_cents": 100}],
72         },

```

```

73     )
74     fetched = client.get("/orders/ord-0002")
75     assert fetched.status_code == 200
76     assert fetched.json()["customerId"] == "globex"
77
78     page = client.get("/orders", params={"limit": 1, "
79         offset": 1})
80     assert [o["id"] for o in page.json()] == ["ord-0002
81         "]
82
83 def test_status_transition_happy_path(client:
84     TestClient) -> None:
85     client.post("/orders", json=VALID_ORDER)
86     confirmed = client.patch("/orders/ord-0001", json={
87         "status": "confirmed"})
88     assert confirmed.status_code == 200
89     assert confirmed.json()["status"] == "confirmed"
90     assert confirmed.json()["version"] == 2
91     shipped = client.patch("/orders/ord-0001", json={
92         "status": "shipped"})
93     assert shipped.json()["status"] == "shipped"
94
95 def test_delete_order_then_404(client: TestClient) ->
96     None:
97     client.post("/orders", json=VALID_ORDER)
98     assert client.delete("/orders/ord-0001").
99         status_code == 204
100     assert client.get("/orders/ord-0001").status_code
101         == 404
102
103 # -- validation and domain errors
104 -----
105
106 def test_invalid_sku_is_a_422_with_sanitized_errors(
107     client: TestClient) -> None:
108     bad = {"customer_id": "acme", "lines": [
109         {"sku": "not-a-sku", "quantity": 1, "
110             unit_price_cents": 100}]}
111     response = client.post("/orders", json=bad)
112     assert response.status_code == 422
113     body = response.json()
114     assert response.headers["content-type"].startswith(

```

```

    "application/problem+json")
1108 assert body["title"] == "Validation␣failed"
1109 assert body["errors"][0]["loc"] == "body.lines.0.
    sku"
1110 assert "ctx" not in body["errors"][0] # no raw
    internals leak
1111
1112
1113 def test_duplicate_skus_rejected_by_model_validator(
    client: TestClient) -> None:
1114     dup = {"customer_id": "acme", "lines": [
1115         {"sku": "NWR-1001", "quantity": 1, "
            unit_price_cents": 100},
1116         {"sku": "NWR-1001", "quantity": 3, "
            unit_price_cents": 100}]}
1117     response = client.post("/orders", json=dup)
1118     assert response.status_code == 422
1119     assert "duplicate␣SKU" in response.json()["errors"
        ][0]["msg"]
1120
1121
1122 def test_missing_order_is_a_404_problem(client:
    TestClient) -> None:
1123     response = client.get("/orders/ord-9999")
1124     assert response.status_code == 404
1125     body = response.json()
1126     assert body["title"] == "Order␣not␣found"
1127     assert body["order_id"] == "ord-9999"
1128
1129
1130 def test_invalid_transition_is_a_409(client: TestClient
    ) -> None:
1131     client.post("/orders", json=VALID_ORDER)
1132     response = client.patch("/orders/ord-0001", json={"
        status": "shipped"})
1133     assert response.status_code == 409
1134     body = response.json()
1135     assert body["current"] == "pending"
1136     assert body["target"] == "shipped"
1137
1138
1139 # -- middleware
    -----
1140
1141

```

```

142 def test_correlation_id_echoed(client: TestClient) ->
    None:
143     response = client.get("/health", headers={"X-
        Request-Id": "req-abc-123"})
144     assert response.headers["x-request-id"] == "req-abc
        -123"
145
146
147 def test_correlation_id_minted_when_absent(client:
    TestClient) -> None:
148     response = client.get("/health")
149     assert len(response.headers["x-request-id"]) == 16
150     assert "x-process-time-ms" in response.headers
151
152
153 # -- OpenAPI contract
    -----
154
155
156 def test_openapi_schema_is_customized_and_complete(
    client: TestClient) -> None:
157     schema = client.get("/openapi.json").json()
158     assert schema["info"]["title"] == "Northwind_
        Robotics_Orders_API"
159     assert schema["info"]["x-audience"] == "internal"
160     assert {p for p in schema["paths"]} >= {
161         "/orders", "/orders/{order_id}", "/health", "/"
        ready", "/orders-stream"}
162     assert schema["components"]["securitySchemes"]["
        ApiKeyAuth"] == {
163         "type": "apiKey", "in": "header", "name": "X-
        API-Key"}
164     assert schema["servers"][0]["url"] == "http://
        localhost:8000"
165     create_op = schema["paths"]["/orders"]["post"]
166     assert "201" in create_op["responses"] # status
        codes documented for free
167     assert "422" in create_op["responses"]
168
169
170 # -- streaming
    -----
171
172

```

```

173 def test_stream_orders_as_ndjson(client: TestClient) ->
    None:
174     client.post("/orders", json=VALID_ORDER)
175     client.post("/orders", json={
176         "customer_id": "globex",
177         "lines": [{"sku": "NWR-2002", "quantity": 5, "
            unit_price_cents": 50}]}))
178     response = client.get("/orders-stream")
179     assert response.headers["content-type"].startswith(
        "application/x-ndjson")
180     lines = [json.loads(line) for line in response.text
        .splitlines()]
181     assert [line["id"] for line in lines] == ["ord-0001",
        "ord-0002"]
182     assert all("fraud_score" not in line for line in
        lines)
183
184
185 # -- the SQLite backend
    -----

186
187
188 def test_sqlite_repository_roundtrip(tmp_path) -> None:
189     settings = Settings(
190         repository_backend="sqlite", sqlite_path=str(
            tmp_path / "orders.db")
191     )
192     with TestClient(create_app(settings)) as client:
193         assert client.post("/orders", json=VALID_ORDER)
            .status_code == 201
194         body = client.get("/orders/ord-0001").json()
195         assert body["total_cents"] == 9998
196         # lifecycle: shutdown closed the DB, readiness
            now fails closed
197         assert client.app.state.ready is False
198
199
200 # -- dependency overrides
    -----

201
202
203 class EmptyRepository(InMemoryOrderRepository):
204     def get(self, order_id: str): # every lookup
        misses

```

```

205         raise OrderNotFoundError(order_id)
206
207
208 def test_dependency_override_swaps_the_repository() ->
None:
209     app = create_app(Settings(repository_backend="
memory"))
210     app.dependency_overrides[get_repository] =
EmptyRepository
211     with TestClient(app) as client:
212         assert client.get("/orders/ord-0001").
status_code == 404
213     app.dependency_overrides.clear()
214
215
216 def test_di_showcase_caching_and_override() -> None:
217     app = di_showcase.create_app()
218     with TestClient(app) as client:
219         first = client.get("/demo").json()
220         assert first["quote_resolutions"] == 1 # resolved
once per request
221
222     app = di_showcase.create_app()
223     app.dependency_overrides[di_showcase.
get_quote_source] = (
224         di_showcase.FallbackQuoteSource)
225     with TestClient(app) as client:
226         assert client.get("/demo").json()["quote"] == "
synthetic_fallback_quote"

```

Note what the suite asserts and what it refuses to assert. It asserts the *contract*: status codes, problem-document shapes, the aliased wire name `customerId`, the *absence* of internal fields, the correlation header, and the contents of `/openapi.json`. It does not assert log timestamps, elapsed times, or minted request-ID values — those are runtime facts, not contract facts, and pinning them would make the suite non-deterministic. Deterministic repository IDs (`ord-0001`) are what make every other assertion exact.

Example

Two seams, two styles. The suite demonstrates both test seams from this chapter. The `client` fixture uses the *factory seam* — `create_app(Settings(repository_backend="memory"))` — which is the right choice when the whole environ-

ment changes. The override tests use the *override seam* — `app.dependency_overrides[get_repository] = EmptyRepository` — which is the right choice when one collaborator changes and everything else should stay real. Choosing per test keeps suites fast *and* honest.

7.4 Common Pitfalls & Anti-Patterns

Each of these has caused a real incident or a real data leak. They are ordered roughly by how expensive they are to discover late.

Blocking the event loop in `async def` handlers

The single most damaging FastAPI mistake. A synchronous database driver, a `requests` call, or a `time.sleep` inside `async def` does not block one request — it blocks the worker’s entire event loop: every in-flight request stalls, health checks time out, and the orchestrator concludes the pod is dead. The rule is structural, not stylistic: if any call in the handler chain blocks, the handler is `def`. Audit dependencies too — a blocking *dependency* under an `async` endpoint blocks exactly the same loop.

Leaking stack traces in 500s

Starlette’s debug mode renders beautiful tracebacks — including local variables, which means connection strings and customer data — into the HTTP response. Production runs with debug off *and* a registered `Exception` handler that logs the traceback server-side and returns a generic problem document. “It only happens on unexpected errors” is exactly when you cannot predict what the frames contain.

Mutable default dependencies

`def endpoint(cache: dict = {})` and module-level dependency instances shared across requests are the FastAPI flavor of Python’s classic mutable-default-argument bug, with concurrency on top: two requests mutating one dict is a data race that survives your test suite and detonates under load. Providers should construct fresh state per request, or delegate shared state to a deliberately synchronized process-lifetime resource built in the lifespan (like the repository’s `threading.Lock`).

Validation only at the edge, with no domain invariants

Field constraints (`ge=1`, regex patterns) catch malformed *shapes*; they cannot catch malformed *meaning*. “Pending orders cannot ship” and “an order may not contain the same SKU twice” are domain invariants: the first lives in the `transition` function, the second in a `model_validator`. Systems that validate only shapes end up enforcing invariants in the database’s error messages — which is to say, not enforcing them at all in any way a client can act on.

`response_model` drift

Declaring `response_model` once and never auditing it rots: new internal fields get added to the stored model, someone forgets the model is returned from a route, and the field leaks. Defenses: keep the outbound model (`OrderRead`) a distinct class from the stored record (`OrderRecord`), and assert field absence in tests — the suite’s “`fraud_score`” not in body line is a security control, not a style test.

Catching broad Exception in handlers

`except Exception: return JsonResponse(...)` inside a handler swallows *programming* errors (a typo’d attribute, a violated invariant) together with *operational* errors, converting bugs into plausible-looking 200s or 500s with no traceback anywhere. Raise specific domain exceptions; let the registered handlers map them; let everything else reach the catch-all, where it is logged with full context. Broad catches belong at exactly one layer: the outermost error handler.

Heavy work at import time / module scope

Connecting to a database, reading files, or building a repository at module import means: tests cannot import the module without side effects, every uvicorn worker re-does the work on fork, and configuration is frozen before anyone could inject it. Everything expensive belongs in `create_app/lifespan`. The module level should contain only declarations — and the single, side-effect-free `app = create_app()` line for the server’s benefit.

7.5 Hands-On Production Lab & Real-World Case Study

7.5.1 Lab: Stand Up the Orders API and Interrogate Its Contract

Objective. Run the Orders API locally, exercise it with `curl.exe` and `TestClient`, and inspect the generated OpenAPI document — offline end to end.

Setup (PowerShell):

```

1 mkdir orders-lab; cd orders-lab
2 # save orders_api.py, middleware.py, di_showcase.py,
   test_orders_api.py here
3 python -m venv .venv
4 .\.venv\Scripts\Activate.ps1
5 pip install "fastapi>=0.110" "uvicorn[standard]>=0.29"
   "pydantic>=2.7" '
6     "pydantic-settings>=2.2" "httpx>=0.27" "
   pytest>=8"

```

Step 1 — run the suite. `pytest test_orders_api.py -q` should report 17 passed in about a second, with no network access. Read one JSON log line from the output and find its `request_id` field.

Step 2 — run the server.

```

1 uvicorn orders_api:app --port 8000

```

Step 3 — exercise it (second PowerShell window; note `curl.exe`, not the `curl` alias):

```

1 curl.exe http://localhost:8000/health
2 curl.exe http://localhost:8000/ready
3 curl.exe -X POST http://localhost:8000/orders -H "
   Content-Type: application/json" '
4   -H "X-Request-Id: lab-001" '
5   -d '{"customer_id":"acme","lines":[{"sku":"NWR
   -1001","quantity":2,"unit_price_cents":4999}]}'
6 curl.exe http://localhost:8000/orders/ord-0001
7 curl.exe -X PATCH http://localhost:8000/orders/ord-0001
   -H "Content-Type: application/json" '
8   -d '{"status":"shipped"}'

```

Observe: the PATCH returns a 409 problem document (pending cannot go to shipped); every response echoes `x-request-id: lab-001`; and the response body contains `customerId` — never `fraud_score`.

Step 4 — inspect the contract.

```
1 curl.exe http://localhost:8000/openapi.json -o openapi.json
```

Open `openapi.json` and locate: the `servers` array injected by `custom_openapi`; the `ApiKeyAuth` entry under `components.securitySchemes`; the 422 response on `POST /orders`; and the `OrderRead` schema — confirm `warehouse_hint` and `fraud_score` appear nowhere in it. Then open `http://localhost:8000/docs` and drive the API from the generated Swagger UI.

Step 5 — flip the backend. Stop the server, set `$env:NWR_REPOSITORY_BACKEND="sqlite"` and `$env:NWR_SQLITE_PATH="lab.db"`, restart, and repeat Step 3. No code changed; the environment did. That is the 12-factor property made visible. Stop with `Ctrl+C` and watch the shutdown log line — the lifespan’s post-yield code running.

Acceptance criteria. You are done when: the suite passes; the 409 transition error reproduces; a request without `X-Request-Id` gets a minted 16-hex-digit ID back; and you can point at the exact line in `openapi.json` where the alias `customerId` is declared.

7.5.2 Case Study: The Threadpool Exhaustion Incident

Context. A robotics-parts fulfillment service — one uvicorn worker class per pod, four pods — served a warehouse scanning fleet. Its hottest endpoint, `POST /shipments`, was declared `async def` because “async is faster,” but its body called the synchronous PostgreSQL driver (`psycopg2` without the `async` wrapper) three times per request.

Symptom. Latency dashboards showed p99 climbing from 40 ms to 30 s every day at the 09:00 scanning shift-change, with *zero* 5xx errors — then mass pod restarts as liveness probes (served by the same event loop) began timing out. Restarts made it worse: warming pods re-entered a queue that was already saturated.

Diagnosis. The blocking driver calls ran *on the event loop*, so each of the four workers could make progress on exactly one request at a time. Effective concurrency per pod was one, not the hundreds the autoscaler assumed; Little’s Law did the rest. The tell was in the metrics: event-loop lag spiked *before* latency did, and per-pod CPU sat near one core — four busy loops, zero parallelism.

The botched first fix. An engineer converted the endpoint to `def`, moving the driver calls to `anyio`’s threadpool — correct in principle. But the 40-thread default was sized for incidental blocking, not for the whole workload: at shift change the pool filled, requests queued inside it, and latencies still climbed — now without loop lag to explain them. The second-order lesson: the threadpool is a *bounded resource with a queue*, and the

queue is invisible unless you instrument pool occupancy.

Resolution. Three changes, in order of leverage: (1) the endpoint stayed `def` and the three driver calls were batched into one round-trip, cutting thread-seconds per request by three; (2) the `anyio` thread limiter was raised to match the measured concurrency need, with pool occupancy exported as a metric and alerted; (3) the roadmap item — an `async def` honestly — was scheduled rather than assumed free. Post-incident p99 at shift change: 180 ms.

Takeaways. `async` is a property of your I/O stack, not your function keyword; threadpools are capacity you must size and instrument; and liveness probes sharing the event loop turn overload into restart loops — one more reason `/health` checks nothing external.

7.6 Self-Assessment Exercises & Deep-Dive Questions

Work these before the interview vault; they are designed to be answered with code, not prose.

- E1. Header dependency.** Add a dependency `get_api_version` that reads an `X-API-Version` header, defaults to "1", and is available to every route via a typed alias. Write a test asserting the default and the override.
- E2. Idempotent create.** Extend `POST /orders` to accept an `Idempotency-Key` header: a repeated key with an identical body returns the original 201 response; the same key with a different body returns 409. (Chapter 10 generalizes this.)
- E3. Blocking-proof the loop.** Add a middleware that measures event-loop lag (schedule `asyncio.sleep(0)` and time the delay) and emits it as a log field. Demonstrate the lag with a deliberately blocking endpoint, then fix the endpoint.
- E4. Scope audit.** Write a websocket-free pure-ASGI middleware that counts requests per path and exposes the counts on `/metrics` (Prometheus text format). Why must the counter live on `app.state` rather than as a middleware attribute? (Hint: middleware instances are per-application; be precise about which *object* would be wrong and why.)
- E5. Validator choreography.** Move the duplicate-SKU rule from `OrderCreate`'s `model_validator` to the repository's `insert`, raising a domain error mapped to 422. Which location is correct, and

what does your answer imply about invariants that span *multiple* aggregates (e.g. “a customer may have at most five open orders”)?

- E6. Streaming discipline.** Modify `/orders-stream` to page the repository (`limit/offset` batches of 100) inside the generator instead of snapshotting first. What failure mode does snapshotting prevent during a long stream with concurrent writes?
- E7. Shutdown honesty.** Write a test that opens a streaming response, exits the `TestClient` context mid-stream, and asserts the repository’s `close()` ran. What does `uvicorn` do differently from `TestClient` here, and where is that documented?
- E8. Contract pinning.** Add a test that diffs `/openapi.json` against a checked-in snapshot and fails on any change. Then make a deliberate breaking change (rename a response field) and watch it catch you. This is design-first governance on a code-first toolchain.

7.7 Interview Q&A Vault

Thirty-six questions, ordered junior → staff. Answer aloud before reading.

Q1. What is ASGI, and how does it differ from WSGI?

The Asynchronous Server Gateway Interface is the contract between an HTTP server and a Python application: a single awaitable callable `app(scope, receive, send)`. WSGI is a synchronous, two-argument callable (`environ, start_response`) that handles exactly one request per call and cannot represent websockets, server push, or long-lived connections. ASGI’s event-channel design supports all of those and lets one process interleave thousands of concurrent connections on a single event loop.

Q2. Explain the ASGI `scope/receive/send` protocol.

`scope` is a connection-lifetime dict: `type ("http", "websocket", "lifespan")`, `method`, `path`, `headers`, plus the `state` bag shared with the lifespan. `receive()` is an awaitable yielding the next inbound event — `http.request` body chunks (with `more_body`) or `http.disconnect`. `send()` awaits outbound events: exactly one `http.response.start` (`status + headers`) followed by `http.response.body` chunks. Every header mutation must happen before `response.start` is sent — that is why middleware wraps `send` and intercepts that one message.

Q3. Where does FastAPI end and Starlette begin?

Starlette owns the ASGI surface: routing, middleware, requests, responses, lifespan, `TestClient`. FastAPI is a Starlette subclass plus a compiler: it analyzes your type annotations to build the dependency graph, run Pydantic validation, serialize through `response_model`, and emit OpenAPI. If a feature touches sockets, paths, or the lifespan, it is Starlette; if it touches your function signature, it is FastAPI.

Q4. What happens when a `def` endpoint runs?

FastAPI detects the plain function and, instead of awaiting it, submits it to anyio's threadpool and awaits the *future* on the event loop. The loop stays responsive; the handler runs concurrently with other requests on a worker thread. The pool is bounded (default 40 tokens), so at most 40 blocking calls run concurrently per worker — beyond that, requests queue inside the pool.

Q5. When should an endpoint be `async def`?

When every blocking operation in its call chain — including dependencies — is awaitable: an async database driver, an async `httpx` client, `asyncio.sleep`. If any link blocks, the endpoint must be `def`. The keyword describes your I/O stack, not your performance ambitions.

Q6. What is the default threadpool size for `def` endpoints, and how do you change it?

Forty threads per worker (anyio's default limiter). Raise it in the lifespan via anyio's thread limiter API (`anyio.to_thread.current_default_thread_limiter = N`) and treat the number as a capacity decision you instrument, not a magic constant.

Q7. What happens if you call a blocking driver inside `async def`?

The event loop stalls for the duration of the call. Every concurrent request on that worker — including liveness probes — freezes, latencies climb without errors, and orchestrators start killing healthy-looking pods. It is the classic FastAPI production incident; the fix is `def` (threadpool) or a genuinely async driver.

Q8. Walk the full request lifecycle of `POST /orders`.

Uvicorn parses bytes and builds the HTTP scope; the middleware stack runs outermost-first (correlation, then timing); the Starlette router matches `/orders + POST`; FastAPI resolves the DI graph (`get_settings`, `get_repository`, cached per request); Pydantic

validates `OrderCreate` (422 on failure, short-circuiting the handler); the `def` handler runs on the threadpool; the returned `OrderRecord` is filtered through `response_model=OrderRead` and serialized by alias; middleware wraps up on the way out (timing logs, correlation echoes the ID); uvicorn writes bytes.

Q9. How does FastAPI's DI resolve sub-dependencies?

Recursively: a dependency's own signature is analyzed, so `Depends` can appear inside providers. FastAPI flattens the resulting DAG at route-registration time and, per request, executes it depth-first with a per-request cache keyed by the provider callable — a shared sub-dependency requested twice runs once and both consumers see the same instance.

Q10. What is per-request caching of dependencies, and when does it not apply?

Within one request, each provider executes at most once; the result is reused everywhere it is depended upon. It does *not* apply across requests (no cross-request cache), and you can opt out per use with `Depends(provider, use_cache=False)` — needed, for example, when a provider mutates state and two consumers genuinely need independent instances.

Q11. How do yield dependencies work, and when does cleanup run?

The provider runs to its `yield`, the yielded value is injected, the handler and response complete, and then the code after `yield` runs — in LIFO order across the graph, and in a `finally` sense: it runs even when the handler raised. Use it for closing connections, committing or rolling back transactions, and releasing locks. Never put logic after the `yield` that must affect the response — the response is already sent.

Q12. How do `dependency_overrides` work?

`app.dependency_overrides` maps the original provider callable to a replacement callable with a compatible signature. Resolution consults the map first, so overriding a provider replaces it everywhere in the graph, including inside sub-dependencies. Clear the map after the test (`app.dependency_overrides.clear()`); leaking overrides between tests is a classic source of order-dependent suites.

Q13. Pydantic v1 vs. v2: what changed, and why is validation faster?

v2 moved the validation engine to `pydantic-core`, written in Rust, making typical model validation 5–20× faster. The Python API was modernized: `@field_validator/@model_validator` replace `@validator/@root_validator`, `model_dump()/model_validate()` replace `.dict()/parse_obj()`, `model_config = ConfigDict(...)` replaces `class Config`, and v1 shims raise hard errors rather than silently changing behavior.

Q14. `field_validator` vs. `model_validator` — when each?

Field validators see one field, before or after type coercion; use them for per-field normalization and rules (trim, lowercase, format checks). Model validators see the coerced whole model; use them for cross-field invariants (duplicate detection, “end after start”). Putting a cross-field rule in a field validator fails because sibling fields may not exist yet.

Q15. What is `serialization_alias` and when do you use it?

It decouples the Python attribute name from the wire name: Python code uses `customer_id`; the JSON body carries `customerId`. FastAPI serializes response models with `by_alias=True`, so the alias is what clients see. Use it when the contract’s naming convention (camel-Case) differs from Python’s, without giving up either.

Q16. Why does `response_model` matter for security?

Because it is a filter, not just documentation: FastAPI re-validates handler return values against it and drops undeclared fields. Returning an internal record that happens to carry `fraud_score` or password hashes is safe only because the declared outbound model omits them. Without `response_model`, whatever the ORM object grew last sprint ships to clients — a leak that raises no exception.

Q17. What happens if you omit `response_model`?

FastAPI serializes whatever the handler returns (via its JSON encoder) and documents the response as unconstrained. Three costs: internal fields can leak, the OpenAPI contract loses its response schema (breaking client generation), and serialization bugs surface at runtime instead of at response-validation time.

Q18. How are 422 errors produced, and how do you customize them?

When Pydantic validation of path, query, header, or body parameters fails, FastAPI raises `RequestValidationError` before the handler runs, and the default handler returns 422 with the error array.

Register your own handler to reshape it (the Orders API emits an RFC 9457-shaped problem with a sanitized `errors` array); sanitize — raw Pydantic errors can carry internal context objects that do not belong on the wire.

Q19. Liveness vs. readiness vs. startup probes — distinguish them.

Startup: has initialization finished (gates the other two). Liveness: is the process wedged — fail it and the orchestrator restarts you; check nothing external. Readiness: may traffic be routed here — checks dependencies and warmup; fail it and you are drained, not restarted. Conflating readiness into liveness turns a database outage into a restart loop.

Q20. How does graceful shutdown work in uvicorn?

On SIGTERM, uvicorn stops accepting connections, sends the ASGI `lifespan.shutdown` message, and waits (bounded) for in-flight requests. Your lifespan's `post-yield` code runs: flip readiness off first so the load balancer drains you, then close repositories and flush logs. SIGKILL after the grace period takes what it takes — anything unflushed is lost.

Q21. BackgroundTasks vs. a real task queue?

`BackgroundTasks` runs callables after the response, in-process: no retries, no persistence, lost on deploy. Suitable for best-effort side effects (cache warming, metrics flushes). Work whose loss is a business event — billing, provisioning, notification — needs a durable queue with retries and dead-lettering (Celery, `arq`, outbox pattern). The deciding question: may this work vanish if the process dies one millisecond after the response?

Q22. Design-first vs. code-first — argue both sides.

Design-first: the OpenAPI document is authored and reviewed before code; consumers start against mocks immediately; the contract cannot drift silently — at the cost of front-loaded design and a spec/implementation skew risk. Code-first: annotations generate the contract, so it is exact by construction and iteration is cheap — at the cost of an “accidental contract” that changes whenever code does. The synthesis: code-first mechanics, design-first governance — pin and review the generated schema in CI.

Q23. Name the top-level keys of an OpenAPI 3.1 document.

`openapi` (spec version), `info` (title, API version), `servers`, `paths`

(path items holding per-method operations), `components` (`schemas`, `parameters`, `responses`, `securitySchemes`, ...), `security`, `tags`, and `externalDocs` [Ope21].

Q24. How does FastAPI generate the OpenAPI schema, and how do you customize it?

At request time for `/openapi.json`, `app.openapi()` walks the registered routes and compiles annotations into the document, caching the result on `app.openapi_schema`. Customize by assigning a replacement callable to `app.openapi`: call `get_openapi(...)`, mutate the dict (servers, security schemes, `x-*` extensions), cache, and return it. Respect the cache or you regenerate on every docs request.

Q25. What changed between OpenAPI 3.0 and 3.1?

3.1 aligned `components.schemas` with full JSON Schema 2020-12: real type arrays (`type: [string, "null"]`) replace the 3.0 nullable keyword; `prefixItems`, `unevaluatedProperties`, and `$defs` become legal; and the `examples` arrays semantics were clarified. For API authors the practical headline is that one schema dialect now serves validation, documentation, and codegen.

Q26. Explain 12-factor configuration and pydantic-settings' source layering.

Config that varies per deploy lives in the environment, not in code [Wig17]. `BaseSettings` declares typed fields with defaults and an `env_prefix`; resolution layers sources with precedence: explicit init arguments, environment variables, `.env` file, then field defaults. Tests inject init arguments; production injects the environment; nobody parses `os.environ` ad hoc.

Q27. Where do secrets live, and what must never be logged?

Secret material enters via the environment (or a secrets manager the environment points to) at deploy time; never in source control, never in `.env` files committed to the repo, never in log lines, exception messages, or error responses. Settings objects may reference secrets (paths, key names); the debug-mode 500 page and careless `repr` of settings are the two classic leak paths.

Q28. Why an application factory instead of module-level app construction?

A factory (`create_app(settings)`) gives every test an isolated, deterministically configured application and makes the composition root explicit. Module-level construction freezes configuration at

import time, couples import to side effects, and makes parallel tests share state. Keep exactly one module-level line — `app = create_app()` — for `uvicorn module:app`.

Q29. How do you test FastAPI offline?

With `TestClient` (or `httpx`'s `ASGITransport`): the real ASGI app runs in-process — routing, middleware, DI, validation, serialization — over no socket. Use the client as a context manager so the lifespan (startup/shutdown) runs; inject test state through the factory or `dependency_overrides`; and assert the contract — status codes, error shapes, header echoes, and the OpenAPI document itself.

Q30. How would you put a correlation ID on every log line of a request?

Outermost middleware extracts or mints `X-Request-Id` and binds it to a `contextvars.ContextVar`; a logging `Formatter` reads the variable for every record; the middleware echoes the ID on the response and resets the variable in a `finally`. Context variables (not thread-locals) because the request is a coroutine, not a thread.

Q31. Explain middleware ordering semantics.

`add_middleware` wraps the app like an onion: the *last* added is *outermost* — first on the request path, last on the response path. Order matters for causality: correlation must wrap timing if the timing log should carry the request ID; authentication must wrap anything that trusts identity.

Q32. `BaseHTTPMiddleware` vs. pure-ASGI middleware?

`BaseHTTPMiddleware` trades the raw protocol for request/response objects — convenient, but it historically buffered streaming responses, added a task hop, and had context-propagation bugs with background tasks. Pure-ASGI middleware (implement `__call__` (`scope`, `receive`, `send`)) has none of those costs and composes at exactly one layer. Prefer pure ASGI unless the request/response API materially simplifies the code.

Q33. Why is catching broad `Exception` inside handlers an anti-pattern?

It collapses programming errors and operational errors into one path: a typo returns a plausible 500 (or worse, a 200-with-fallback) with the traceback swallowed, and the error budget lies. Raise specific domain exceptions, register handlers that map them to precise statuses, and let a single catch-all log-and-sanitize everything unexpected. Broad

catch at exactly one layer.

Q34. How does `StreamingResponse` work, and what are its constraints?

It takes an iterator of byte chunks and sends one `http.response.body` message per chunk with `more_body=True`, keeping memory constant regardless of payload size. A sync generator runs on the threadpool; an `async` generator on the loop — so an `async` generator must not block. Headers (including `Content-Type`) are committed at `response.start`, before the first chunk: you cannot change your mind mid-stream, and mid-stream errors can only truncate, not re-status.

Q35. How would you size the threadpool for a def-heavy service?

With Little’s Law: required slots $\approx \lambda \cdot L$, with λ the peak arrival rate and L the mean blocking-service time per request in seconds. Validate empirically: instrument pool occupancy and queue depth, load-test at $2\times$ expected peak, and set a limit that saturates *downstream* before it saturates memory — a 400-thread pool in front of a 50-connection database just moves the queue.

Q36. How would you migrate a blocking production service to `async` safely?

Instrument first: event-loop lag, threadpool occupancy, downstream latency. Migrate the leaves, not the trunk: replace the driver with its `async` equivalent behind the repository protocol, keeping `def` endpoints. Only when the whole chain is awaitable do you flip handlers to `async def`, one route at a time, behind a config flag, with the blocking variant still deployable. Gate the rollout on loop-lag SLOs; the anti-pattern is the big-bang rewrite where a single missed blocking call takes down the loop.

7.8 External Bibliography & Further Reading

- **FastAPI documentation** [Ram24] — the reference for path operations, dependencies, middleware, testing, and OpenAPI customization. Read the *Advanced User Guide* sections on dependencies with `yield` and lifespan; they are this chapter’s mechanics in the official voice.
- **Starlette documentation** — the ASGI toolkit underneath: routing, middleware, `TestClient`, lifespan, and the pure-ASGI middleware

style used in Section 7.3.2.

- **Uvicorn documentation** — deployment, workers, the `[standard]` extras (`uvloop`, `httptools`), and the lifespan/timeout settings that govern graceful shutdown.
- **Pydantic v2 documentation** [Col24] — validators, serialization aliases, `ConfigDict`, and the migration guide from v1. The *pydantic-core* architecture notes explain where the 5–20× speedup comes from.
- **pydantic-settings documentation** — source layering, `env_prefix`, `.env` files, and secrets handling.
- **The OpenAPI 3.1 specification** [Ope21] — the normative contract grammar; read the *Paths*, *Operation*, *Components*, and *Security Scheme* objects alongside Section 7.2.2.
- **The Twelve-Factor App** [Wig17] — factor III (config in the environment) is the configuration section’s backbone; factor IX (disposability) is the graceful-shutdown section’s.
- **RFC 9110 (HTTP Semantics)** [Fie22a] — §15 for the status-code semantics this chapter’s routes implement; the status registry entries for 201, 204, 404, 409, and 422.
- **Building Microservices**, 2nd ed., Sam Newman [New21] — the organizational frame: service boundaries, contract thinking, and why “the API is the product” outlives any framework choice.
- **httpx documentation** [Enc24] — the `ASGITransport` mechanism underneath `TestClient`, and the async client patterns that Chapter 2 introduced.

7.9 Capstone Projects

Exactly five projects, progressive. All are offline, deterministic, and use only synthetic Northwind Robotics data. Place deliverables under `capstones\capstone_NN_<slug>\` per the workspace conventions.

7.9.1 Capstone 1: Todo API with a Full Test Suite [junior]

Objective. Build a minimal but *complete* FastAPI service: a todo-list API with create, read, list, complete, and delete operations, an in-memory repository behind a protocol, and a `TestClient` suite that covers every route.

Inputs/Datasets. A seeded fixture of 20 synthetic todos (titles from the Northwind Robotics maintenance log, e.g. “recalibrate arm 7”).

Steps. (1) Model `TodoCreate`/`TodoRead` with constraints (title 3–80 chars; priority 1–5). (2) Implement the repository protocol and an in-memory implementation with deterministic IDs. (3) Wire routes with `response_model` on all of them and correct status codes (201/204/404/422). (4) Write at least 10 tests: every happy path, a 404, and two distinct 422s.

Acceptance Criteria.

- ☐ `pytest -q` passes offline; IDs are deterministic per fresh app.
- ☐ Every route declares `response_model`; no internal field appears in any response body.
- ☐ The OpenAPI schema documents a 422 response for every route that accepts a body.

Stretch Goals. Add the correlation-ID middleware and assert the header echo; add a `/todos-stream` NDJSON endpoint with a streaming test.

7.9.2 Capstone 2: Inventory API with a SQLite Repository [mid]

Objective. Rebuild the Orders pattern for warehouse inventory: parts, stock levels, and reservations — with *both* repositories (in-memory and SQLite) behind one protocol, selected by `pydantic-settings`, and a readiness probe that actually exercises the database.

Inputs/Datasets. A generated fixture of 500 synthetic parts (NWR-0001...NWR-0500) with stock counts; 1% deliberately invalid rows for negative tests.

Steps. (1) Model `Part`, `Reservation`, and their create/read counterparts. (2) Implement both repositories; the SQLite one must create its schema idempotently. (3) Implement reservation as a *domain transition* with invariants (cannot reserve more than on-hand; cannot cancel a shipped reservation) mapped to 409. (4) Make `/ready` fail when the database file is unreadable. (5) Test both backends through the factory seam.

Acceptance Criteria.

- ☐ The same suite passes against `memory` and `sqlite` settings with zero code changes.
- ☐ Oversubscribing a part returns 409 with a problem document naming the part and the shortfall.
- ☐ Readiness returns 503 when the SQLite path is a directory (probing the failure path, not mocking it).

Stretch Goals. Add optimistic concurrency: a `version` field checked on update (412 Precondition Failed on mismatch; Chapter 10 generalizes).

7.9.3 Capstone 3: Plugin-Style Dependency-Injection Architecture [senior]

Objective. Refactor the Orders API so that *repositories*, *notifiers*, and *pricing policies* are *plugins*: implementations register themselves in a module-level registry, settings select them by name, and the lifespan composes the chosen set — with a test suite that swaps each plugin via `dependency_overrides` without touching route code.

Inputs/Datasets. The chapter’s Orders API; two synthetic notifier plugins (“webhook-log” and “audit-ndjson”) and two pricing policies (“list” and “fleet-discount”).

Steps. (1) Define protocols for `Notifier` and `PricingPolicy`. (2) Build a registry (name → factory) with fail-fast errors on unknown names. (3) Extend `Settings` with `notifier: str` and `pricing: str`; the lifespan instantiates the selection. (4) Thread both through DI into the order lifecycle (price on create, notify on status change). (5) Prove isolation: each plugin has tests using overrides, and one test boots every registry entry.

Acceptance Criteria.

- ☐ Adding a new plugin touches exactly one new file plus one registry line — no route or factory edits.
- ☐ Booting with `NWR_NOTIFIER=bogus` fails at startup with an actionable error, not at first request.
- ☐ Override tests demonstrate that notifications and pricing are replaceable per test, per request graph.

Stretch Goals. Emit the effective plugin set and versions into `/health` and into a startup log line; add a contract test that every registered notifier satisfies the protocol against a shared behavioral suite.

7.9.4 Capstone 4: Async Migration of a Blocking Service [senior]

Objective. Take a deliberately blocking service (synchronous SQLite repository, `time.sleep`-simulated latency) and migrate it to correct-concurrency form, *proving* each step with an event-loop-lag middleware and a concurrency benchmark — the Section 7.5.2 incident, reproduced and fixed in miniature.

Inputs/Datasets. A provided baseline service where `async def` handlers

call blocking code (the bug); a seeded dataset of 1,000 orders; a benchmark script issuing 200 concurrent requests through `ASGITransport`.

Steps. (1) Write the loop-lag middleware and the benchmark; capture baseline p50/p99 and loop lag. (2) Fix variant A: handlers become `def` (threadpool). Re-measure. (3) Fix variant B: replace the repository with an `asyncio.to_thread`-wrapped implementation behind the same protocol and restore `async def`. Re-measure. (4) Write the decision memo: which variant ships, and what limiter setting, with the numbers.

Acceptance Criteria.

- ☐ Baseline shows loop lag > 5 s under load; both fixed variants show lag < 50 ms.
- ☐ p99 under 200-way concurrency improves by an order of magnitude; results are reproducible across three runs.
- ☐ The memo cites pool occupancy evidence, not folklore.

Stretch Goals. Add a third variant using a genuinely async store (an `asyncio.Queue`-backed fake) and explain the remaining differences; instrument and graph threadpool queue depth during the transition.

7.9.5 Capstone 5: OpenAPI-First Generator Round-Trip [staff]

Objective. Close the design-first loop: author `orders.openapi.yaml` by hand for a superset of the chapter’s API (adding shipments), generate a FastAPI skeleton and a typed Python client from it, implement the skeleton, and build a CI-style gate that fails when the running app’s schema diverges from the authored contract — then perform a governed contract change end to end.

Inputs/Datasets. The authored spec; the chapter’s Orders API as the seed implementation; a “change request” adding an optional `priority` field to orders (backward compatible) and one renaming a field (breaking).

Steps. (1) Author the 3.1 spec with `servers`, `components`, `securitySchemes`, and complete error responses. (2) Generate or hand-synthesize the skeleton from the spec; implement against it. (3) Implement the diff gate: fetch `/openapi.json`, compare against the pinned spec, classify every difference as breaking or compatible. (4) Apply the compatible change: spec first, code second, gate green. (5) Attempt the breaking change and document how the gate stops it.

Acceptance Criteria.

- ☐ The gate classifies the seeded compatible and breaking changes correctly, with human-readable reasons.

- The running service passes its own gate; the pinned spec and the generated schema round-trip with zero diffs.
- The typed client consumes the service in a `TestClient` integration test with no hand-edited models.
- A written policy states who may bump which version number and when (feeding Chapter 9 and Chapter 20).

Stretch Goals. Emit the diff report as an RFC 9457 problem document; extend the gate to detect *undocumented* endpoints (in code, absent from spec) as a distinct violation class.

7.10 Python Dependencies Appendix

Everything in this chapter pins to a minimal `requirements.txt`:

```
1 fastapi>=0.110
2 uvicorn[standard]>=0.29
3 pydantic>=2.7
4 pydantic-settings>=2.2
5 httpx>=0.27
6 pytest>=8
```

Install on Windows/PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

7.10.1 fastapi

Description. The framework itself: annotation-driven routing, dependency injection, Pydantic integration, and OpenAPI generation on top of Starlette [Ram24].

Why included. The subject of the chapter; every listing runs on it.

Alternatives. **Flask** (synchronous WSGI, vast ecosystem, no native validation or contract generation), **Django Ninja** (FastAPI-style typed APIs inside the Django universe — choose when you already pay for Django), **Litestar** (comparable typed-ASGI framework with a batteries-included posture), **aiohttp** (lower-level async framework for when you want to own the stack).

Installation. `pip install "fastapi>=0.110"`.

7.10.2 uvicorn[standard]

Description. The ASGI server: sockets, TLS, HTTP parsing, and the event loop. The `[standard]` extra pulls in `uvloop` and `httptools` — the C-accelerated loop policy and parser used in production.

Why included. Runs the app in the lab; its lifespan and shutdown semantics are part of the chapter's operational surface. Not needed by the test suite — `TestClient` speaks ASGI directly, a fact worth noticing.

Alternatives. `hypercorn` (HTTP/2 and HTTP/3 support), `daphne` (Channels ecosystem), `granian` (Rust-based server).

Installation. `pip install "uvicorn[standard]>=0.29"` (quote the brackets in PowerShell).

7.10.3 pydantic

Description. Data validation and settings-independent modeling: the `BaseModel`, validators, and serialization machinery the whole contract is built from [Col24]. v2's `pydantic-core` is a Rust engine; validation speed stops being a design constraint.

Why included. Every inbound payload, outbound contract, and stored record in the chapter is a Pydantic model; `response_model` filtering is a Pydantic behavior.

Alternatives. `attrs` + `cattrs` (class authoring plus explicit (de)serialization; more assembly, less magic), `msgspec` (extremely fast, schema-first, stricter), `dataclasses` + `manual checks` (the honest baseline you now know how to beat).

Installation. `pip install "pydantic>=2.7"`.

7.10.4 pydantic-settings

Description. The `BaseSettings` layer: typed, environment-layered configuration with prefixes, `.env` files, and explicit precedence — 12-factor factor III as a library [Wig17].

Why included. The Orders API's `Settings` is its composition root input; the test suite's determinism depends on injecting it explicitly.

Alternatives. `dynaconf` (multi-format, heavier), `enviroms` (thin env parsing), `os.environ` plus `discipline` (viable until the first time you need types or layering — usually week two).

Installation. `pip install "pydantic-settings>=2.2"`.

7.10.5 httpx

Description. The HTTP client whose `ASGITransport` powers `TestClient` [Enc24]: requests are dispatched straight into the ASGI callable, no socket involved.

Why included. Offline testing is this chapter’s non-negotiable; `httpx` is also the client library of Chapter 2, so one mental model serves both directions of the wire.

Alternatives. `requests` (synchronous, no ASGI transport — fine for scripting, wrong here), `aiohttp` client (async, different API).

Installation. `pip install "httpx>=0.27"`.

7.10.6 `pytest`

Description. The test runner: fixtures, parametrization, and the `tmp_path` fixture the SQLite test uses for hermetic file state.

Why included. The chapter’s entire verification story — 17 tests, offline, one second — is a `pytest` suite; Chapters 19 builds the full quality-engineering practice on it.

Alternatives. `unittest` (stdlib, zero install, more ceremony), `nose2` (largely historical).

Installation. `pip install "pytest>=8"`.

7.11 Chapter Summary & Key Takeaways

- **The contract is the product.** Design-first and code-first differ in who authors the OpenAPI document, not in whether it exists; pin, review, and CI-assert the generated schema and you get code-first speed with design-first governance.
- **ASGI is three callables.** `scope`, `receive`, `send` — `uvicorn` translates bytes to events, `Starlette` routes them, `FastAPI` compiles your annotations around them, and one event loop per worker runs the whole show.
- **`async` describes your I/O stack, not your wishes.** Any blocking link makes the endpoint `def`; the 40-thread pool is capacity you must size and instrument, as the threadpool exhaustion incident demonstrates.
- **DI is a per-request DAG with seams.** Sub-dependencies resolve depth-first, cache once per request, clean up after the response via `yield`, and swap wholesale via `dependency_overrides`.
- **`Pydantic v2` is the validation boundary and `response_model` is the security boundary.** Field validators normalize, model validators enforce invariants, aliases decouple names, and the response model guarantees internal fields never reach the wire.

- **Operations are designed, not bolted on.** Settings through the environment, JSON logs with correlation IDs, three probes with three distinct meanings, and a lifespan that flips readiness before it closes anything.
- **Test the contract in-process.** `TestClient` drives the real app — middleware, DI, validation, lifespan — offline; assert status codes, error shapes, header echoes, and the OpenAPI document itself.

7.12 Quick-Reference Card

Endpoint Decision Table

Situation	Reach for
Async I/O end-to-end (async driver, <code>httpx</code>)	<code>async def</code> endpoint, awaited on the loop.
Any blocking call (sync driver, <code>requests</code>)	<code>def</code> endpoint — the threadpool absorbs it.
CPU-bound work	Neither: subprocess or task queue; the loop and the pool are both wrong.
Best-effort side effect after response	<code>BackgroundTasks</code> .
Work that must survive a deploy	Durable task queue / outbox (Chapter 11).
Constant-memory large payload	<code>StreamingResponse</code> ; sync generator = threadpool, async = loop.

Table 7.3: Endpoint and execution-model decisions at a glance.

Dependency-Injection Cheat-Sheet

Lifecycle Ordering

1. Uvicorn parses bytes → builds the HTTP scope.
2. Middleware, outermost first (last `add_middleware` is outermost).
3. Starlette router matches path and method.
4. DI graph resolves (per-request cache populated).
5. Pydantic validates inputs — 422 short-circuits here.

Mechanism	Rule to remember
<code>Annotated[T, Depends(p)]</code> Sub-dependencies	Declare once as a type alias; reuse everywhere. Providers can <code>Depends</code> too; the graph is flattened and resolved depth-first.
Per-request cache	Each provider runs at most once per request; <code>use_cache=False</code> opts out.
<code>yield</code> cleanup	Post- <code>yield</code> runs after the response, LIFO, even on exceptions.
Process-lifetime state	Build in <code>lifespan</code> , store on <code>app.state</code> , look up via providers.
Test seam	<code>app.dependency_overrides[original] = fake</code> ; clear after the test.

Table 7.4: DI rules that prevent the common incidents.

6. Handler runs: `async def` on the loop, `def` on the pool.
7. Return value filtered through `response_model`, serialized by alias.
8. Middleware unwinds (headers already committed at `http.response.start`).
9. `yield`-dependency cleanup runs (LIFO).
10. Background tasks run; on SIGTERM the lifespan's post-`yield` code runs last.

7.13 Standardized Evidence Addendum

7.13.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 7.5: Minimum benchmark table to keep per chapter.

7.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.

- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

7.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 7.6: Operational budget template for this chapter’s technique.

7.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

7.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

7.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

7.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 7.7: Decision card to complete per chapter.

7.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

7.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.

- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 8

Errors, Validation & Problem Details (RFC 9457)

Executive Summary

Every API has two interfaces: the success contract and the failure contract. Teams routinely invest weeks polishing the former and minutes on the latter, and then discover—during a 3 a.m. incident—that clients cannot tell a validation mistake from a melted database, cannot retry safely, and cannot page the right owner because every error looks like **500 Internal Server Error** with a stack trace taped to it. This chapter fixes that.

We treat errors as a first-class design surface. You will learn a rigorous taxonomy that separates transport, protocol, and application failures, and a repeatable decision procedure that classifies each failure as client-fixable (4xx), server-owned (5xx), or dependency-caused, and as retrievable or non-retrievable. You will master RFC 9457 *Problem Details for HTTP APIs* [Not23], the standard that replaces ad-hoc error payloads with a single, extensible, machine-readable shape carried in `application/problem+json`. You will learn status-code precision—why 422 means *syntactically valid but semantically invalid*, why 404 and 410 tell different stories, why 200-with-an-error-body is an anti-pattern that corrupts caches, retries, and monitoring.

On the engineering side, you will build a production-grade error pipeline in FastAPI: strict Pydantic v2 validation at the boundary [Col24], a `ProblemException` hierarchy mapped to RFC 9457 responses by a single handler, PII-redaction middleware, correlation IDs end-to-end, a machine-readable error-code registry, and a contract test suite that schema-validates *every* error path with JSON Schema [JSO22]. Everything runs offline, deterministically, in process. By the end, “what did the API say when it failed?” will have one answer everywhere in your system: a well-formed problem document.

Key Concept

Errors are part of the contract. Version them, document them, test them, and alert on them exactly as you do for success responses. A consumer should be able to handle every failure mode of your API without ever reading a log line or calling your support desk.

8.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, you will be able to:

1. Classify any failure into the transport/protocol/application taxonomy and justify a status code, retriability verdict, and ownership assignment for it.
2. Author RFC 9457 problem documents, including extension members, and explain the semantics of `type`, `title`, `status`, `detail`, and `instance`.
3. Choose precisely between 400 and 422, 404 and 410, 401, 403 and 407, and among 500, 502, 503, and 504.
4. Engineer validation with Pydantic v2: strict mode, `field_validator` versus `model_validator`, `Annotated` custom types, and coercion pitfalls.
5. Implement a single error-handling pipeline—domain exceptions → problem mapper → redaction → response—with correlation IDs and server-side structured logging.
6. Schema-validate every error response in contract tests and generate a machine-readable error-code registry and its public documentation.
7. Operate errors in production: error-rate SLIs per code and endpoint, alert thresholds, PII redaction, and error-budget interaction (Chapter 18).

Prerequisites

This chapter assumes you have completed Chapters 0–6. In particular, you should be comfortable with:

- HTTP semantics: methods, status-code classes, header negotiation (Chapter 1, [Fie22a]).
- Consuming APIs from Python with `httpx` (Chapter 2).
- REST resource modeling and representation design (Chapter 3).
- Serialization contracts, JSON Schema, and OpenAPI 3.1 documents (Chapter 4, [Ope21]).
- Client-side security hygiene: where secrets may and may not appear (Chapter 5).
- Building FastAPI services: routers, dependency injection, Pydantic models, and `TestClient` (Chapter 6, [Ram24]).

All code targets Python 3.11+, runs fully offline, and uses only the dependencies in the Python Dependencies Appendix. Commands are given for Windows PowerShell.

8.2 Deep Technical Mechanics & Architectural Concepts

8.2.1 A Taxonomy of Failure

Before choosing a status code or a payload shape, you must know *what kind of thing went wrong*. We use a three-layer taxonomy.

Definition 8.1 (Transport Error). A failure below HTTP semantics: the bytes never formed a complete, routable request or response. Examples: TCP connection refused, TLS handshake failure, DNS resolution failure, connection reset mid-body. Transport errors have *no HTTP status code at all*—there is nothing to attach one to. From the client’s perspective they surface as exceptions (e.g. `httpx.ConnectError`), and they are almost always retrievable with backoff because no application state was necessarily touched.

Definition 8.2 (Protocol Error). A failure of HTTP itself: the message arrived but is malformed or violates protocol expectations. Examples: syntactically invalid JSON in the request body, a `Content-Type` the server does not support (415), a missing mandatory header, an unacceptable `Accept` header (406). The server understood nothing reliably enough to run business logic, so the response must not imply that it did.

Definition 8.3 (Application (Domain) Error). A failure inside valid protocol traffic: the request was well-formed HTTP carrying well-formed JSON, but the domain rejected it. Examples: a SKU that does not exist, a state transition the shipment lifecycle forbids, insufficient funds. Application errors are where APIs earn or lose trust, because they are the errors clients handle programmatically.

The taxonomy matters because each layer has different handling machinery. Transport errors are handled by client retry policies and circuit breakers [Nyg18]. Protocol errors are handled by the framework’s request parsing. Application errors are handled by *your* domain code, and they deserve the most design effort.

Ownership: Who Can Fix It?

The 4xx/5xx split in RFC 9110 [Fie22a] is fundamentally about *ownership of the fix*:

- **Client-fixable (4xx).** The client sent something the server can never accept as-is. Retrying the identical request is useless; something about the request must change. Examples: 400, 404, 409, 422.
- **Server-owned (5xx).** The request was fine; the server failed to fulfill it. Retrying the identical request may succeed later. Examples: 500, 503.
- **Dependency-caused (also 5xx, but distinguishable).** The server failed *because a downstream dependency failed*. 502 (bad response from upstream), 504 (upstream timeout), and 503 (deliberate unavailability) communicate this. Distinguishing dependency failures from internal bugs is operationally crucial: the page goes to a different team and the retry semantics differ.

Warning

Never blame the client for your outage. Returning 400 when a database is down teaches clients to stop retrying, corrupts their error metrics, and hides your incident behind their dashboards. If the request was valid and you could not serve it, the answer is in the 5xx family—always.

A Decision Procedure for Retriability

Clients need one bit of information above all: *may I send this again?* Encode the decision explicitly, for both client and server authors:

1. **Did the request reach application logic?** If not (transport error, 408, 429, 502, 503, 504), the request is retrievable *if it is idempotent or idempotency-protected* (Chapter 10).
2. **Is the failure about the request content?** 400, 401, 403, 404, 409, 415, 422: not retrievable without modification. 409 is the nuance: retrievable *after* the client refreshes state and re-evaluates preconditions.
3. **Is the failure an unhandled server fault (500)?** Treat as *not* retrievable for mutations unless the operation is idempotent; retrying a non-idempotent POST after a 500 risks double-execution because you cannot know whether the server committed before failing.
4. **Did the server say when?** 429 and 503 with Retry-After are retrievable *at or after* the indicated time, with jitter to avoid thundering herds.

Publish this classification per error code (we build exactly such a registry in Listing 7.6) so no client engineer ever has to guess.

8.2.2 RFC 9457: Problem Details for HTTP APIs

Before RFC 7807 (2016), every API invented its own error payload: `{"error": "oops"}`, `{"message": ..., "code": 1234}`, bare strings, HTML pages from middleware you forgot existed. Client code became a museum of per-API parsers. RFC 7807 standardized a shape; RFC 9457 (March 2024) [Not23] *obsoletes* RFC 7807, clarifying extension member handling, the `application/problem+xml` deprecation of XML-specific guidance, and alignment with modern HTTP (RFC 9110 [Fie22a]). If a document or library says “7807,” read it as 9457 with a historical accent.

Definition 8.4 (Problem Detail Object). A JSON (or XML) object carrying *machine-readable details of one HTTP error response*, served with the media type `application/problem+json` and an error status code. It is a deliberately minimal kernel—five standard members—extended by the API with additional members.

The Five Standard Members

The division of labor is the design’s genius: `type/title` describe the *class* of problem (stable, documentable, matchable); `detail/instance` describe the *occurrence* (specific, variable, not for matching). Most broken client integrations come from confusing these two levels—matching on `detail` strings is the classic offender.

Member	Required	Semantics
<code>type</code>	yes (default <code>about:blank</code>)	A URI reference that identifies the problem <i>type</i> . When dereferenced, it should lead to human documentation. It is the stable identifier clients match on.
<code>title</code>	recommended	Short, human-readable summary of the problem <i>type</i> . Stable per type; never per-occurrence. Localizable but should not change between occurrences of the same type.
<code>status</code>	recommended	The HTTP status code, echoed into the body for convenience of intermediaries and logs. Must match the actual response status.
<code>detail</code>	optional	Human-readable explanation of <i>this occurrence</i> . May differ per occurrence; never write client logic against it.
<code>instance</code>	optional	A URI reference identifying <i>this occurrence</i> ; often the request path or a per-error URN.

Table 8.1: Standard problem detail members per RFC 9457 [Not23].

Extension Members and Documentation-as-Contract

RFC 9457 explicitly permits *extension members*: additional keys defined by your API, such as `code`, `trace_id`, or an `errors` array of per-field failures. Two rules keep extensions healthy:

1. Clients **must ignore unknown members**. This makes extension members additive-evolvable: you can ship new members without a version bump, exactly like new fields in a success representation.
2. Every extension member you emit **must be documented at the type URI**. The **type URI** is the contract anchor: *documentation-as-contract*. When a client dereferences `https://docs.example.invalid/problems/validat`, they should find the member list, the status code, retriability, and remediation guidance—generated from the same registry your code uses (Section 8.3, Listing 7.6).

RFC 9457 also defines an IANA registry of problem *types*, though in practice most APIs mint their own type URIs under a domain they control.

The operational requirement is not registration; it is *stability*: a `type` URI, once shipped, must keep meaning the same problem class forever.

Why Must `type` Be a URI?

Three reasons. First, *global uniqueness without coordination*: URIs minted under your domain cannot collide with another vendor’s “`validation_error`.” Second, *dereferenceability*: a URI can double as a documentation link, collapsing identifier and manual into one artifact. Third, *opacity*: because the spec says clients match on the URI string itself (not on fetched content), you may serve `about:blank` for generic errors and reserve rich type URIs for documented classes. Note that `about:blank` with a `title` matching the status’s reason phrase is the spec-sanctioned “I have nothing more specific to say” fallback.

Security Considerations

RFC 9457’s security section is blunt, and production experience blunter: **problem details are exfiltration channels if you let them be**. Rules we enforce in this chapter’s code:

- **No internals.** No stack traces, framework versions, SQL fragments, hostnames, or file paths in `detail`. Each is reconnaissance data for an attacker probing your error paths.
- **No PII.** `detail` must never echo submitted personal data. “Email jane.doe@example.invalid already registered” both leaks an account-existence oracle and lands PII in every log aggregator and APM tool that captures response bodies.
- **No secret material.** Failed-payload logging must redact credentials, tokens, and keys *before* writing (Chapter 5).
- **Defense in depth.** Even with disciplined handlers, add a redaction middleware as the last gate (Listing 7.3): humans make mistakes at 3 a.m.; middleware does not.

8.2.3 Status-Code Precision for Error Cases

Status codes are the coarsest, most universally understood part of your error contract; caches, load balancers, retry middleware, and monitoring all key on them before anyone reads a body. Imprecision here is amplified everywhere. This section walks the pairs that production systems most often confuse; all semantics are per RFC 9110 §15 [Fie22a].

400 versus 422

400 Bad Request means the server *cannot or will not process the request due to something perceived to be a client error*—with the canonical reading that the *framing or syntax* itself is wrong: malformed JSON, a missing required header, a query parameter that fails to parse as an integer. The request could not be interpreted as a valid instance of your contract.

422 Unprocessable Content means the request *was* syntactically valid—the JSON parsed, the schema shape held—but *semantic* instructions could not be followed: an end date before a start date, a quantity of zero, a SKU failing its checksum. The mnemonic: **400 = I cannot read you; 422 = I read you and reject what you said**. FastAPI emits 422 for Pydantic validation failures by default [Ram24], which is the convention this chapter keeps. Using 400 for semantic failures is not fatal, but it is lossy: clients distinguish “fix my serializer” from “fix my business values” only if you do.

404 versus 410

404 Not Found says *nothing* about history: the resource is not here, and the server does not say whether it ever was. **410 Gone** is a deliberate tombstone: the resource existed and is permanently retired—a deleted account, an expired export, a sunset endpoint version. 410 is cacheable by default and tells crawlers and sync clients to stop asking and purge local copies. Use it when you know; use 404 otherwise. Be aware that 410 (like any distinct response to existence probes) can be an information-disclosure oracle for sensitive resources; for those, uniform 404 is the privacy-preserving choice.

409 Conflict and Its Domains

409 means the request is fine but *conflicts with current resource state*. It appears in at least four distinct domains, and your problem **type** should say which:

- **State-machine conflict:** cancelling an already-shipped shipment (our Listing 7.4 example).
- **Uniqueness conflict:** creating a resource whose natural key is taken. (Some APIs use 422 here; 409 is the better fit because the payload is valid in isolation.)
- **Concurrent-modification conflict:** an ETag / If-Match precondition failed—though 412 is the precise code for failed preconditions (Chapter 10).

- **Reference conflict:** deleting a warehouse that still holds inventory.

In all domains, 409 is *conditionally retrievable*: the client should re-read state, re-decide, and only then retry.

401 versus 403 versus 407

401 Unauthorized is badly named: it means *unauthenticated*— “I do not know who you are (or your credentials failed).” It *must* carry a WWW-Authenticate challenge header; a 401 without it is a spec violation. 403 Forbidden means *authenticated but not authorized*: “I know who you are, and no.” Re-authenticating will not help. 407 Proxy Authentication Required is the same challenge as 401 but issued by an intermediary proxy, with Proxy-Authenticate; API authors rarely emit it, but gateway operators must not strip it. One more subtlety: for resources whose *existence* is sensitive, authorization failure should collapse to 404, not 403, to avoid an existence oracle.

429 and Retry-After

429 Too Many Requests says the client itself is the load problem (Chapter 14 develops the limiting algorithms). It is only useful when paired with Retry-After (delay-seconds or an HTTP date) and, ideally, rate-limit headers describing the policy window. A 429 without Retry-After invites clients to invent their own backoff—usually badly, usually synchronized into thundering herds. In problem details, carry the retry budget as an extension member so programmatic clients do not parse headers at all.

500, 502, 503, 504

The 5xx family differentiates *where* the failure lives:

- 500 Internal Server Error: your code faulted. Generic, uncatchable-by-design, always a bug or an unmodeled failure.
- 502 Bad Gateway: a gateway/upstream returned something invalid. The dependency answered—wrongly.
- 503 Service Unavailable: deliberate unavailability—overload shedding, maintenance, a blown circuit breaker [Nyg18]. Almost always retrievable; pair with Retry-After when the duration is known.
- 504 Gateway Timeout: the dependency never answered in time.

The operational value is paging precision: 500 pages your team; 502/504 page the integration owner; 503 should mostly page nobody—it is the system

protecting itself. If your error mapper collapses all four into 500, every dependency outage becomes your incident.

The 200-with-Error-Payload Anti-Pattern

Some APIs return 200 OK with `{"success": false, "error": ...}` in the body. Every layer of the HTTP stack is then actively misled: caches may store the failure as a success; retry middleware sees success and does nothing; CDN and load-balancer metrics report 100% availability during a total outage; monitoring must parse bodies to see the fire. The only defensible carve-outs are protocols that deliberately tunnel errors above transport semantics (GraphQL over HTTP is the notable one, Chapter 12)—and even there, the community is moving toward distinguishing transport success from application errors explicitly. For REST-style HTTP APIs: the status code must reflect the outcome. Always.

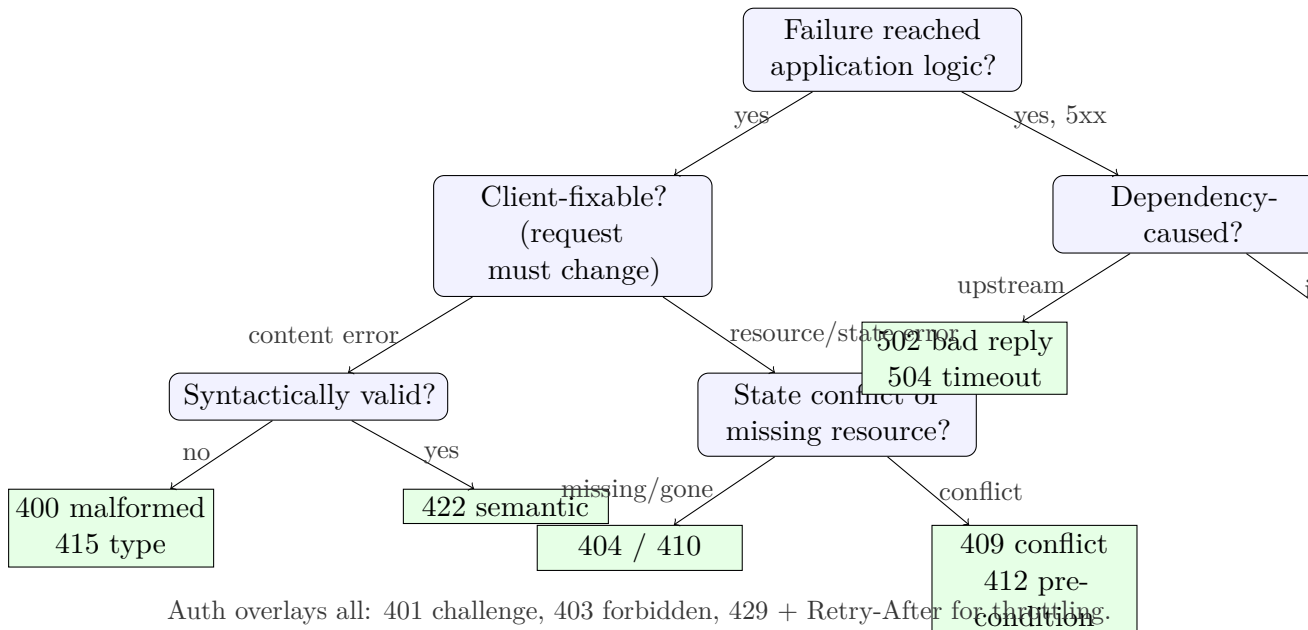


Figure 8.1: Status-code decision tree for API error responses. Auth failures (401/403) and throttling (429) are orthogonal overlays: they can interrupt any branch before application logic runs.

8.2.4 Validation Engineering with Pydantic v2

Validation is where error contracts meet user experience: a validation error is the error a human being most often has to read, understand, and act on. Pydantic v2 [Col24] gives us the machinery; discipline turns it into a contract.

Validation at Boundaries versus Domain Invariants

Boundary validation answers: is this payload a legal instance of the input schema? Types, ranges, patterns, string lengths. It is mechanical, declarative, and belongs in the model layer at the edge of the system. *Domain invariants* answer: is this payload legal *given the current state of the world*? “Express shipments under 30 kg” is an invariant expressible in the model; “this SKU is stocked at the destination warehouse” is an invariant that requires a database and therefore belongs in the service layer, where its failure becomes a domain exception (409/422), not a schema error. Keeping the two apart keeps models reusable and keeps service errors semantically typed instead of masquerading as typos.

field_validator versus model_validator

A `field_validator` sees exactly one field (plus, if requested, already-validated sibling values via `ValidationInfo`) and answers single-field questions: is this SKU pattern valid, is this quantity within review limits. A `model_validator` with `mode="after"` sees the whole, already-coerced model and answers cross-field questions: priority versus total weight. Use field validators for local rules—they compose and localize error pointers naturally—and reserve model validators for genuine invariants. A model validator’s error has no natural field location, which is why our mapping emits the JSON Pointer / (the whole document) for it.

Coercion Strictness

Pydantic’s default *lax* mode coerces: the string `"12"` becomes the integer `12`; `"true"` becomes `True`. At a public API boundary this is a hazard class: silent coercion converts client bugs into corrupted records—the client *thought* it sent a string, you stored an integer, and their audit diff never matches. *Strict mode* (`model_config = ConfigDict(strict=True)`) accepts only values of the declared type, surfacing the type mismatch as a 422 the client can fix. The nuance, demonstrated in Listing 7.2: JSON has no decimal literal, so blanket strictness makes a `Decimal` field unusable. The correct move is *per-field* strictness opt-out (`Field(strict=False, ...)`) hardened with a validator (finite, quantized). Strictness is a dial per field, not a religion.

Error Message UX

A validation message is read by a tired integrator at 2 a.m. Good messages name the rule and the expectation (“priority must be one of [express, freight, standard]”), never echo the raw offending value (it may be a credential they

pasted by mistake), and never expose internal field or table names that differ from the public contract. When you translate Pydantic’s error array into problem-details extensions, locate each failure with a JSON Pointer (RFC 6901): `/lines/0/quantity` is unambiguous to both humans and machines, and it survives localization of the human message.

8.2.5 Error Envelope Architecture

The cardinal rule: **one pipeline, one shape**. Every error that leaves the service—validation failure, domain rejection, unhandled exception, framework-generated error—passes through the same stages and emerges as `application/problem+json`. Figure 8.2 shows the pipeline our code implements.

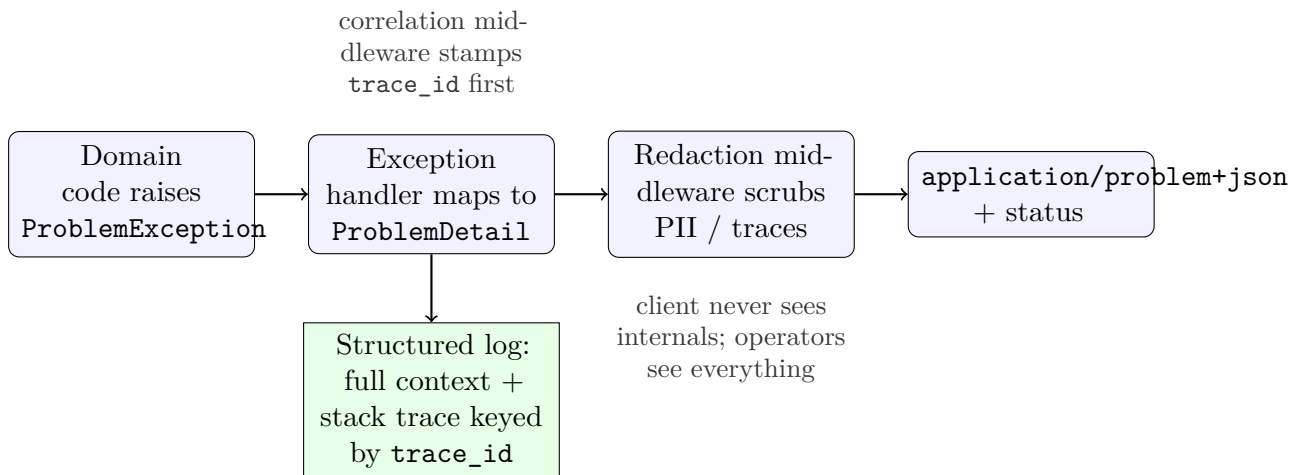


Figure 8.2: Single error-handling pipeline: exception → problem mapper → redaction → response, with full-fidelity server-side logging forked off at the mapper under the correlation ID.

Machine-Readable Codes

The RFC 9457 `type` URI is the primary machine identifier; we add a `code` extension member (`NRW-SHIPMENT-NOT-FOUND`) because operations teams alert, dashboard, and page on short codes far more easily than on URIs, and because codes survive documentation-host moves. Codes must be: unique, immutable once shipped, prefixed per system, and backed by a registry that validates itself at import time (Listing 7.6). Never let a developer invent a code inline in a handler—that is how `ERR_2` happens.

Localization

`title` and `detail` are human-facing and may be localized via `Accept-Language`; `type` and `code` are machine-facing and must *never* be localized. The practical pattern: treat `title` as a lookup key into a message catalogue per locale, keep `code` ASCII, and keep interpolation values (limits, field names) as extension members so localized strings are pure templates. Clients that match on errors must match on `type/code` only—a localized `title` is a rendering, not an identifier.

8.2.6 Operational Concerns

Correlation IDs End-to-End

Every request carries a trace identifier: accepted from an inbound `X-Trace-Id` header if well-formed (bounded length, character allowlist—never trust and reflect arbitrary header bytes), minted otherwise, echoed in the response header, embedded in the problem body as `trace_id`, and attached to every log line the request produces. This collapses the support loop from “describe what you saw” to “give me the `trace_id`.” Propagate the same ID to downstream calls so a single customer report reconstructs the whole request tree (Chapter 18 generalizes this to full distributed tracing).

Error Observability and Alerting

The error contract is your observability taxonomy: emit metrics as `errors_total{endpoint, status, code}` and you can answer, per deploy, per endpoint, per error code: is this new, is it ours, is it a dependency, is it client abuse. Suggested alert thresholds as a starting point: 5xx rate > 1% of requests over 5 minutes pages; a single `code` spiking > 10× its 7-day baseline pages; 4xx rate > 25% sustained opens a ticket (usually a broken client deploy, not your bug). Error budget interaction is direct: 5xx responses burn the budget of Chapter 18. Formally, over a window W :

$$\text{burn}(W) = \frac{|\{r \in W : \text{status}(r) \geq 500\}|}{|W|} \quad \text{budget}(W) = 1 - \text{SLO} \quad (8.1)$$

and when $\text{burn}(W)$ approaches $\text{budget}(W)$, Chapter 18’s policy freezes feature deploys in favor of reliability work. A precise error taxonomy is what makes that arithmetic actionable instead of theatrical.

8.3 Production-Ready Code Implementation

The reference implementation is a small FastAPI service for the fictional *Northwind Robotics* universe (all data synthetic). Six files form one coherent

system:

- `problem_details.py` — the reusable RFC 9457 library: models, exception hierarchy, handlers, JSON Pointer mapping.
- `shipment_models.py` — strict Pydantic v2 validation showcase with a coercion-pitfall demo.
- `redaction.py` — correlation-ID and PII-redaction middleware.
- `app.py` — the service wiring, with deliberately interesting failure endpoints.
- `test_contract.py` — contract tests schema-validating every error path.
- `error_codes.py` — the machine-readable error-code registry and its documentation generator.

Setup (PowerShell, offline after install):

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt      # see the
   Dependencies Appendix
4 pytest -q                          # 10 contract tests,
   all in-process
5 python shipment_models.py           # coercion pitfall
   demo
6 python error_codes.py               # regenerate the
   error catalogue
```

8.3.1 A Reusable Problem-Details Library

Listing 7.1 is the heart of the chapter. `ProblemDetail` models the RFC 9457 kernel plus our extension members (`code`, `trace_id`, `errors[]`). `ProblemException` is the single bridge between domain code and HTTP: domain code raises semantically named subclasses; one handler renders them. The `RequestValidationError` handler converts Pydantic’s error array into RFC 6901 JSON Pointers, and the catch-all `Exception` handler proves the invariant that *nothing* leaves the service unshaped—the client gets a redacted 500 with a `trace_id`; the log gets the truth.

```
1 """problem_details.py -- Reusable RFC 9457 problem-
   details library.
```

```

3 Listing_7.1.Offline-first,stdlib,pydantic,fastapi
  only.
4 """
5 from __future__ import annotations
6
7 import re
8 import uuid
9 from typing import Any, Optional
10
11 from fastapi import FastAPI, Request
12 from fastapi.exceptions import RequestValidationError
13 from fastapi.responses import JSONResponse
14 from pydantic import BaseModel, ConfigDict, Field
15
16 PROBLEM_MEDIA_TYPE = "application/problem+json"
17
18 # Synthetic, obviously fictional documentation
19 # namespace (RFC 9457 type URIs).
20 TYPE_BASE = "https://docs.example.invalid/problems"
21
22 class ErrorLocation(BaseModel):
23     """One-field-level validation failure, located by
24     JSON Pointer (RFC 6901)."""
25
26     pointer: str = Field(description="JSON Pointer to
27     the offending value")
28     code: str = Field(description="Machine-readable
29     error code")
30     message: str = Field(description="Human-readable,
31     redacted message")
32
33 class ProblemDetail(BaseModel):
34     """RFC 9457 problem detail object with Northwind
35     Robotics extensions."""
36
37     model_config = ConfigDict(populate_by_name=True)
38
39     type: str = Field(default="about:blank")
40     title: str
41     status: int
42     detail: Optional[str] = None
43     instance: Optional[str] = None
44     # Extension members.
45     code: Optional[str] = None

```

```

42     trace_id: Optional[str] = None
43     errors: list[ErrorLocation] = Field(default_factory=
44         =list)
45
46 class ProblemException(Exception):
47     """Base class: a domain exception that maps 1:1 to
48         a ProblemDetail."""
49
49     status: int = 500
50     title: str = "Internal_Server_Error"
51     type_slug: str = "internal"
52     code: str = "NRW-INTERNAL"
53
54     def __init__(
55         self,
56         detail: Optional[str] = None,
57         *,
58         instance: Optional[str] = None,
59         errors: Optional[list[ErrorLocation]] = None,
60     ) -> None:
61         super().__init__(detail)
62         self.detail = detail
63         self.instance = instance
64         self.errors = errors or []
65
66     def to_problem(self, trace_id: str) ->
67         ProblemDetail:
68         return ProblemDetail(
69             type=f"{TYPE_BASE}/{self.type_slug}",
70             title=self.title,
71             status=self.status,
72             detail=self.detail,
73             instance=self.instance,
74             code=self.code,
75             trace_id=trace_id,
76             errors=self.errors,
77         )
78
79 class ShipmentNotFound(ProblemException):
80     status = 404
81     title = "Shipment_Not_Found"
82     type_slug = "shipment-not-found"
83     code = "NRW-SHIPMENT-NOT-FOUND"
84

```



```

85
86 class ShipmentStateConflict(ProblemException):
87     status = 409
88     title = "Shipment␣State␣Conflict"
89     type_slug = "shipment-state-conflict"
90     code = "NRW-SHIPMENT-STATE-CONFLICT"
91
92
93 class WarehouseUnavailable(ProblemException):
94     status = 503
95     title = "Dependency␣Unavailable"
96     type_slug = "warehouse-unavailable"
97     code = "NRW-WAREHOUSE-UNAVAILABLE"
98
99
100 class RateLimitExceeded(ProblemException):
101     status = 429
102     title = "Rate␣Limit␣Exceeded"
103     type_slug = "rate-limit-exceeded"
104     code = "NRW-RATE-LIMIT-EXCEEDED"
105
106     def __init__(self, retry_after: int, **kwargs: Any)
107         -> None:
108         super().__init__(
109             detail=f"Retry␣after␣{retry_after}␣seconds.",
110             ", **kwargs
111         )
112         self.retry_after = retry_after
113
114
115 def json_pointer(loc: list[Any]) -> str:
116     """Convert␣a␣Pydantic␣error␣'loc'␣tuple␣to␣an␣RFC
117     6901␣JSON␣Pointer."""
118     parts = []
119     for seg in loc:
120         if seg == "body": # FastAPI wraps body params
121             under "body"
122             continue
123         parts.append(str(seg).replace("~", "~0").
124             replace("/", "~1"))
125     return "/" + "/".join(parts)

```

```

126 ) -> JSONResponse:
127     payload = problem.model_dump(exclude_none=True)
128     return JSONResponse(
129         status_code=problem.status,
130         content=payload,
131         media_type=PROBLEM_MEDIA_TYPE,
132         headers=headers,
133     )
134
135
136 def register_problem_handlers(app: FastAPI) -> None:
137     """Single pipeline: domain exception -> problem
138     mapper -> redaction -> response."""
139
140     @app.exception_handler(ProblemException)
141     async def handle_problem(
142         request: Request, exc: ProblemException
143     ) -> JSONResponse:
144         trace_id = getattr(request.state, "trace_id",
145                             str(uuid.uuid4()))
146         problem = exc.to_problem(trace_id)
147         headers = None
148         if isinstance(exc, RateLimitExceeded):
149             headers = {"Retry-After": str(exc.retry_after)}
150         return problem_response(problem, headers=headers)
151
152     @app.exception_handler(RequestValidationError)
153     async def handle_validation(
154         request: Request, exc: RequestValidationError
155     ) -> JSONResponse:
156         trace_id = getattr(request.state, "trace_id",
157                             str(uuid.uuid4()))
158         # 422: the payload was syntactically valid JSON
159         # but semantically invalid.
160         locs = [
161             ErrorLocation(
162                 pointer=json_pointer(list(err.get("loc",
163

```

```

163         for err in exc.errors()
164     ]
165     problem = ProblemDetail(
166         type=f"{TYPE_BASE}/validation-failed",
167         title="Validation␣Failed",
168         status=422,
169         detail="Request␣payload␣failed␣semantic␣
170             validation.",
171         instance=str(request.url.path),
172         code="NRW-VALIDATION-FAILED",
173         trace_id=trace_id,
174         errors=locs,
175     )
176     return problem_response(problem)
177
178 @app.exception_handler(Exception)
179 async def handle_unexpected(
180     request: Request, exc: Exception
181 ) -> JSONResponse:
182     trace_id = getattr(request.state, "trace_id",
183         str(uuid.uuid4()))
184     # Full context goes to the server log; the
185     # client gets a redacted body.
186     print(f"trace_id={trace_id}␣unhandled={exc!r}")
187     problem = ProblemDetail(
188         type=f"{TYPE_BASE}/internal",
189         title="Internal␣Server␣Error",
190         status=500,
191         detail="An␣unexpected␣error␣occurred.␣"
192         "Reference␣the␣trace_id␣when␣contacting␣
193         support.",
194         instance=str(request.url.path),
195         code="NRW-INTERNAL",
196         trace_id=trace_id,
197     )
198     return problem_response(problem)
199
200 SECRET_PATTERNS: list[tuple[re.Pattern[str], str]] = [
201     (re.compile(r"[\w.+~]+@[ \w-]+\.[\w.]+"), "<email>"),
202     ,
203     (re.compile(r"\b\d{3}-\d{2}-\d{4}\b"), "<tax-id>"),
204     (re.compile(r"(?i)(api[_-]?key|token|password)=[^&\s]+"), r"\1=<redacted>"),
205 ]

```

```

203
204 def redact_text(text: str) -> str:
205     """Scrub PII-like patterns and secrets from
        arbitrary text."""
206     for pattern, replacement in SECRET_PATTERNS:
207         text = pattern.sub(replacement, text)
208     return text

```

Example

A client handles this without string matching. Given any error from the shipments API, a Python client switches on the stable extension member:

```

1 import httpx
2
3 resp = httpx.post("https://api.example.invalid/
    shipments", json=payload)
4 if resp.is_error:
5     problem = resp.json() if resp.headers["content
        -type"].startswith(
6         "application/problem+json") else {}
7     match problem.get("code"):
8         case "NRW-VALIDATION-FAILED":
9             fix_fields(problem["errors"])
                # JSON Pointers per field
10        case "NRW-RATE-LIMIT-EXCEEDED":
11            sleep(float(resp.headers["Retry-After"]
                )))
12        case _:
13            escalate(problem.get("trace_id"))
                # never match on "detail"

```

Note what the client never does: parse `detail`, match English prose, or guess retriability from the status alone.

8.3.2 Strict Validation Showcase

Listing 7.2 models the Northwind Robotics `Shipment` resource. Study the Annotated pattern: `Sku` and `CountryCode` are named, reusable types whose constraints travel with the type itself—the self-documenting alternative to scattering `Field(...)` magic at every use site. Note also the deliberate `strict=False` opt-out on `unit_weight_kg`: JSON has no decimal literal, so a strict `Decimal` would reject *every* legitimate JSON number; the companion validator restores the safety (finiteness, quantization) that strictness was

buying.

```

1  """shipment_models.py--StrictPydanticv2validation
   showcase.
2
3  Listing 7.2. Northwind Robotics "Shipment" resource.
   Demonstrates strict mode,
4  field_validator vs model_validator, and Annotated
   custom types.
5  """
6  from __future__ import annotations
7
8  from decimal import Decimal
9  from typing import Annotated, Optional
10
11  from pydantic import (
12      BaseModel,
13      ConfigDict,
14      Field,
15      StringConstraints,
16      field_validator,
17      model_validator,
18  )
19
20  # --- Custom types via Annotated: reusable, composable,
   self-documenting. ----
21  Sku = Annotated[
22      str,
23      StringConstraints(pattern=r"^[NRW-[A-Z]{2}-\d{4}$",
24                          min_length=11, max_length=11),
25  ]
26  CountryCode = Annotated[str, StringConstraints(pattern=
27      r"^[A-Z]{2}$")]
28
29  class Address(BaseModel):
30      model_config = ConfigDict(strict=True, extra="
31          forbid")
32
33      street: str = Field(min_length=1, max_length=120)
34      city: str = Field(min_length=1, max_length=60)
35      country: CountryCode
36
37  class PackageLine(BaseModel):
38      model_config = ConfigDict(strict=True, extra="

```

```

    forbid")
38
39     sku: Sku
40     quantity: int = Field(ge=1, le=10_000)
41     # Decimal avoids binary float money errors. JSON
42     # has no decimal literal,
43     # so this field opts OUT of strict mode (strict=
44     # False) and is hardened
45     # by a validator instead -- blanket strictness
46     # breaks legitimate input.
47     unit_weight_kg: Decimal = Field(strict=False, gt=
48     Decimal("0"), le=Decimal("500"))
49
50     @field_validator("quantity")
51     @classmethod
52     def quantity_not_suspiciously_round(cls, value: int
53     ) -> int:
54         # Business heuristic: quantities over 5000
55         # require explicit review.
56         if value > 5000:
57             raise ValueError("quantity_above_5000_
58             requires_manual_review")
59         return value
60
61     @field_validator("unit_weight_kg")
62     @classmethod
63     def weight_is_finite(cls, value: Decimal) ->
64     Decimal:
65         if not value.is_finite():
66             raise ValueError("unit_weight_kg_must_be_a_
67             finite_number")
68         return value.quantize(Decimal("0.001"))
69
70 class Shipment(BaseModel):
71     """Strict at the boundary: no silent coercion, no
72     unknown fields."""
73
74     model_config = ConfigDict(strict=True, extra="
75     forbid")
76
77     shipment_id: Optional[str] = Field(default=None,
78     pattern=r"shp_\d{8}$")
79     destination: Address
80     lines: list[PackageLine] = Field(min_length=1)
81     priority: str = Field(default="standard")

```

```

71
72     @field_validator("priority")
73     @classmethod
74     def priority_is_known(cls, value: str) -> str:
75         allowed = {"standard", "express", "freight"}
76         if value not in allowed:
77             raise ValueError(f"priority must be one of {sorted(allowed)}")
78         return value
79
80     @model_validator(mode="after")
81     def express_shipments_stay_light(self) -> "Shipment":
82         # Cross-field invariant: express parcels cannot
83         # exceed 30 kg total.
84         if self.priority == "express":
85             total = sum(
86                 line.unit_weight_kg * line.quantity for
87                 line in self.lines
88             )
89             if total > Decimal("30"):
90                 raise ValueError(
91                     f"express shipments must weigh <= 30 kg, got {total} kg"
92                 )
93             return self
94
95     def demo_coercion_pitfall() -> dict[str, str]:
96         """Show why lax mode is dangerous at the API boundary."""
97         outcome: dict[str, str] = {}
98
99         class LaxPackage(BaseModel): # default: lax
100             coercion
101             quantity: int
102
103         coerced = LaxPackage(quantity="12") # type: ignore
104         [arg-type]
105         outcome["lax_accepts_string"] = f"{coerced.quantity}!r} (silently coerced)"
106
107         try:
108             PackageLine(sku="NRW-AR-1001", quantity="12",
109                         unit_weight_kg=Decimal("1.0")) # type:
110             ignore[arg-type]

```

```

106     except Exception as exc:
107         first = str(exc).splitlines()[0]
108         outcome["strict_rejects_string"] = first
109     return outcome
110
111
112 if __name__ == "__main__":
113     for key, value in demo_coercion_pitfall().items():
114         print(f"{key}: {value}")

```

Running `python shipment_models.py` prints:

```

1 lax_accepts_string: 12 (silently coerced)
2 strict_rejects_string: 1 validation error for
  PackageLine

```

The lax model accepted a string where an integer was declared, without a murmur; the strict model refused. In lax mode the corruption would have entered the system wearing a disguise; in strict mode it became a 422 with pointer `/lines/0/quantity` that the client can fix.

8.3.3 Redaction Middleware and Correlation IDs

Listing 7.3 is the last line of defense. Two middlewares bracket the application: `CorrelationIdMiddleware` stamps or mints the trace ID (inner, so handlers can read `request.state.trace_id`), and `ProblemRedactionMiddleware` (outer) re-reads every `application/problem+json` body and scrubs it. Middleware ordering in Starlette is last-added-equals-outermost, so the redactor wraps the correlator wraps the app. The redactor is deliberately pessimistic: if a body smells like a stack trace, it replaces the detail entirely; the full payload goes only to the server log, keyed by the trace ID, where operators with access control can see it.

```

1 """redaction.py -- Correlation IDs and response
   redaction middleware.
2
3 Listing 7.3. Guarantees: problem+json bodies never leak
   PII patterns or
4 stack traces; full context is logged server-side under
   the trace_id.
5 """
6 from __future__ import annotations
7
8 import json
9 import logging
10 import re
11 import uuid

```



```

12 from typing import Awaitable, Callable
13
14 from fastapi import Request, Response
15 from starlette.middleware.base import
16     BaseHTTPMiddleware
17 from starlette.types import ASGIApp
18
19 logger = logging.getLogger("northwind.errors")
20
21 TRACE_HEADER = "X-Trace-Id"
22 PROBLEM_MEDIA_TYPE = "application/problem+json"
23
24 # Deterministic, conservative scrub rules. Extend via
25 registry, not edits.
26 PII_PATTERNS: list[tuple[re.Pattern[str], str]] = [
27     (re.compile(r"[\w.+~]+@[ \w-]+\.[\w.]+"), "<email>"),
28     ,
29     (re.compile(r"\b\d{3}-\d{2}-\d{4}\b"), "<tax-id>"),
30     (re.compile(r"\b(?:\d[ ]-){13,16}\b"), "<card-
31         number>"),
32     (re.compile(r"(?i)(api[_-]?key|token|password|
33         secret)[\"']?\s*[:=]\s*[\"']?[^&\s\"']+"), r"
34         \1=<redacted>"),
35 ]
36
37 STACK_TRACE_MARKER = re.compile(r"(Traceback\[(most\|
38     recent\|call\|last\)|File\["^"]+\",\|line\|d+)")
39
40 def scrub(text: str) -> str:
41     """Remove PII-like patterns and stack-trace
42     fragments from text."""
43     for pattern, replacement in PII_PATTERNS:
44         text = pattern.sub(replacement, text)
45     if STACK_TRACE_MARKER.search(text):
46         return "An internal error occurred. Contact
47             support with the trace_id."
48     return text
49
50 class CorrelationIdMiddleware(BaseHTTPMiddleware):
51     """Accept or mint a trace ID and expose it on
52     request.state and response."""
53
54     async def dispatch(
55         self,

```

```

48         request: Request,
49         call_next: Callable[[Request], Awaitable[
50             Response]],
51     ) -> Response:
52         incoming = request.headers.get(TRACE_HEADER, "")
53         # Accept only safe trace IDs: bounded length,
54         # no control chars.
55         trace_id = incoming if re.fullmatch(r"[\w
56             \-]{1,64}", incoming) else str(uuid.uuid4())
57         request.state.trace_id = trace_id
58         response = await call_next(request)
59         response.headers[TRACE_HEADER] = trace_id
60         return response
61
62 class ProblemRedactionMiddleware(BaseHTTPMiddleware):
63     """Last line of defense: scrub every problem+json
64     body before it leaves."""
65
66     async def dispatch(
67         self,
68         request: Request,
69         call_next: Callable[[Request], Awaitable[
70             Response]],
71     ) -> Response:
72         response = await call_next(request)
73         content_type = response.headers.get("content-
74             type", "")
75         if not content_type.startswith(
76             PROBLEM_MEDIA_TYPE):
77             return response
78
79         body = b""
80         async for chunk in response.body_iterator: #
81             type: ignore[attr-defined]
82             body += chunk if isinstance(chunk, bytes)
83             else chunk.encode()
84
85         original = body.decode("utf-8", errors="replace")
86         cleaned = scrub(original)
87         if cleaned != original:
88             trace_id = getattr(request.state, "trace_id", "unknown")
89             # Server-side keeps the full payload for

```

```

82         forensics; client gets scrubbed.
83         logger.warning(
84             "problem_response_redacted",
85             extra={"trace_id": trace_id, "raw_body"
86                 : original},
87         )
88         headers = dict(response.headers)
89         headers["content-length"] = str(len(cleaned.
90             encode()))
91         return Response(
92             content=cleaned,
93             status_code=response.status_code,
94             headers=headers,
95             media_type=PROBLEM_MEDIA_TYPE,
96         )

```

Warning

Regex redaction is a floor, not a ceiling. Pattern lists catch the shapes you remembered; they do not catch names, free-text addresses, or the PII your schema did not anticipate. Treat regex scrubbing as the emergency brake behind the primary defense—*never putting PII into error payloads in the first place*—and fuzz your scrubber with an adversarial corpus (Capstone Project 3) before you trust it.

8.3.4 The Service, Wired

Listing 7.4 assembles the pieces into a FastAPI application with an in-memory store (offline, deterministic). It deliberately includes endpoints for every failure mode we need to contract-test: missing resources, state conflicts, a dependency failure, a throttle, and a “leaky” endpoint simulating a developer who echoed PII into an error detail.

```

1  """app.py--NorthwindRoboticsShipmentsService,
   wired_for_RFC9457_errors.
2
3  Listing 7.4. In-memory store only: offline,
   deterministic, no network.
4  """
5  from __future__ import annotations
6
7  from decimal import Decimal
8
9  from fastapi import FastAPI, Request
10

```

```

11 from problem_details import (
12     ProblemException,
13     RateLimitExceeded,
14     ShipmentNotFound,
15     ShipmentStateConflict,
16     WarehouseUnavailable,
17     register_problem_handlers,
18 )
19 from redaction import CorrelationIdMiddleware,
20     ProblemRedactionMiddleware
21 from shipment_models import Shipment
22
23 # In-memory store keyed by shipment_id. Synthetic data
24 # only.
25 STORE: dict[str, Shipment] = {
26     "shp_00000001": Shipment(
27         shipment_id="shp_00000001",
28         destination={
29             "street": "1_Gearbox_Lane",
30             "city": "Springfield",
31             "country": "US",
32         },
33         lines=[{"sku": "NRW-AR-1001", "quantity": 2, "
34             unit_weight_kg": Decimal("4.5")}],
35         priority="standard",
36     )
37 }
38 SEQUENCE = {"next": 2}
39
40 def create_app() -> FastAPI:
41     app = FastAPI(title="Northwind_Robotics_Shipments",
42         version="1.0.0")
43     # Middleware order matters: correlation first,
44     # redaction outermost-last.
45     app.add_middleware(ProblemRedactionMiddleware)
46     app.add_middleware(CorrelationIdMiddleware)
47     register_problem_handlers(app)
48
49     @app.post("/shipments", status_code=201)
50     def create_shipment(shipment: Shipment, request:
51         Request) -> Shipment:
52         new_id = f"shp_{SEQUENCE['next']:08d}"
53         SEQUENCE["next"] += 1
54         stored = shipment.model_copy(update={"
55             shipment_id": new_id})

```

```

50     STORE[new_id] = stored
51     return stored
52
53 @app.get("/shipments/{shipment_id}")
54 def get_shipment(shipment_id: str) -> Shipment:
55     if shipment_id not in STORE:
56         raise ShipmentNotFound(
57             detail=f"No shipment with id {
58                 shipment_id!r} exists.",
59             instance=f"/shipments/{shipment_id}",
60         )
61     return STORE[shipment_id]
62
63 @app.post("/shipments/{shipment_id}/cancel")
64 def cancel_shipment(shipment_id: str) -> Shipment:
65     shipment = STORE.get(shipment_id)
66     if shipment is None:
67         raise ShipmentNotFound(detail="Cannot
68             cancel a missing shipment.")
69     if shipment.priority == "freight":
70         # Domain rule: freight contracts lock at
71         # booking time.
72         raise ShipmentStateConflict(
73             detail="Freight shipments cannot be
74                 cancelled after booking.",
75             instance=f"/shipments/{shipment_id}",
76         )
77     return shipment
78
79 @app.get("/warehouse/ping")
80 def warehouse_ping() -> dict[str, str]:
81     # Simulated dependency failure: retrievable,
82     # server-owned.
83     raise WarehouseUnavailable(detail="Warehouse
84         API timed out after 3 attempts.")
85
86 @app.get("/shipments-throttled")
87 def throttled() -> dict[str, str]:
88     raise RateLimitExceeded(retry_after=30)
89
90 @app.get("/debug/leaky")
91 def leaky_failure() -> dict[str, str]:
92     # A careless developer echoes request context
93     # into the error detail.
94     raise ProblemException(
95         detail="Failed to notify jane.doe@example.

```

```

89         invalid;␣"
90         "tax_id␣123-45-6789␣on␣file."
91     )
92     return app
93
94
95 app = create_app()

```

8.3.5 Contract Tests: Every Error Path Is Schema-Valid

Listing 7.5 is what turns the error contract from aspiration into CI gate. Two ideas carry the weight. First, a `PROBLEM_SCHEMA` (JSON Schema draft 2020-12, [JSO22]) with `additionalProperties: false` that defines exactly what a problem document from this service may contain—including the extension members. Second, one assertion helper, `assert_problem`, applied to *every* error test: correct status, correct media type, schema-valid body, body status matching the header, and trace-ID round-trip. Adding a new error path without a contract test becomes impossible to do quietly.

The tests run fully in-process via `TestClient` [Ram24]—no sockets, no network, deterministic on a fresh machine. Note `raise_server_exceptions=False`: we want to verify the 500 *response*, not have the test harness re-raise.

```

1  """test_contract.py␣--␣Contract␣tests:␣every␣error␣path
   ␣is␣valid␣problem+json.
2
3  Listing␣7.5.␣In-process␣ASGI␣tests␣via␣httpx␣TestClient
   ␣Fully␣offline.
4  """
5  from __future__ import annotations
6
7  import jsonschema
8  import pytest
9  from fastapi.testclient import TestClient
10
11  from app import app
12
13  PROBLEM_SCHEMA: dict = {
14      "$schema": "https://json-schema.org/draft/2020-12/
   ␣schema",
15      "type": "object",
16      "required": ["type", "title", "status"],
17      "properties": {

```

```

18         "type": {"type": "string", "format": "uri-
19             reference"},
20         "title": {"type": "string", "minLength": 1},
21         "status": {"type": "integer", "minimum": 400, "
22             maximum": 599},
23         "detail": {"type": "string"},
24         "instance": {"type": "string"},
25         "code": {"type": "string", "pattern": "^NRW-[A-
26             Z-]+${}"},
27         "trace_id": {"type": "string", "minLength": 1},
28         "errors": {
29             "type": "array",
30             "items": {
31                 "type": "object",
32                 "required": ["pointer", "code", "
33                     message"],
34                 "properties": {
35                     "pointer": {"type": "string", "
36                         pattern": "^/"},
37                     "code": {"type": "string"},
38                     "message": {"type": "string"},
39                 },
40             },
41         },
42         "additionalProperties": False,
43     }
44
45 PII_FRAGMENTS = ("@", "123-45-6789", "Traceback", 'File
46     □"')
47
48 @pytest.fixture()
49 def client() -> TestClient:
50     return TestClient(app, raise_server_exceptions=
51         False)
52
53 def assert_problem(response, expected_status: int) ->
54     dict:
55     assert response.status_code == expected_status
56     assert response.headers["content-type"].startswith(
57         "application/problem+json")
58     body = response.json()
59     jsonschema.validate(body, PROBLEM_SCHEMA)
60     assert body["status"] == expected_status

```

```

55     assert "X-Trace-Id" in response.headers
56     assert body["trace_id"] == response.headers["X-
    Trace-Id"]
57     return body
58
59
60 def test_404_is_problem_json(client: TestClient) ->
    None:
61     body = assert_problem(client.get("/shipments/
    shp_999999999"), 404)
62     assert body["code"] == "NRW-SHIPMENT-NOT-FOUND"
63     assert body["type"].endswith("/shipment-not-found")
64
65
66 def test_422_maps_pydantic_errors_to_json_pointers(
    client: TestClient) -> None:
67     payload = {
68         "destination": {"street": "", "city": "
    Springfield", "country": "USA"},
69         "lines": [{"sku": "bad-sku", "quantity": 0, "
    unit_weight_kg": "not-a-number"}],
70         "priority": "hyperdrive",
71     }
72     body = assert_problem(client.post("/shipments",
    json=payload), 422)
73     assert body["code"] == "NRW-VALIDATION-FAILED"
74     pointers = {err["pointer"] for err in body["errors"]
    ]}
75     # Each failure is located by an RFC 6901 JSON
    Pointer.
76     assert "/destination/street" in pointers
77     assert "/destination/country" in pointers
78     assert "/lines/0/sku" in pointers
79     assert "/lines/0/quantity" in pointers
80     assert "/lines/0/unit_weight_kg" in pointers #
    unparseable decimal
81     assert "/priority" in pointers
82
83
84 def test_422_cross_field_invariant(client: TestClient)
    -> None:
85     payload = {
86         "destination": {"street": "1_Gearbox_Lane", "
    city": "Springfield", "country": "US"},
87         "lines": [{"sku": "NRW-AR-1001", "quantity":
    10, "unit_weight_kg": 4.5}],

```



```

88         "priority": "express",
89     }
90     body = assert_problem(client.post("/shipments",
91                                     json=payload), 422)
92     assert body["errors"][0]["pointer"] == "/"
93     assert "express" in body["errors"][0]["message"]
94
95 def test_409_domain_conflict(client: TestClient) ->
96     None:
97     create = client.post(
98         "/shipments",
99         json={
100             "destination": {"street": "9_Servo_Rd", "
101                             city": "Springfield", "country": "US"},
102             "lines": [{"sku": "NRW-FR-9000", "quantity"
103                       : 1, "unit_weight_kg": 120}],
104             "priority": "freight",
105         },
106     )
107     assert create.status_code == 201
108     shipment_id = create.json()["shipment_id"]
109     body = assert_problem(client.post(f"/shipments/{
110                                     shipment_id}/cancel"), 409)
111     assert body["code"] == "NRW-SHIPMENT-STATE-CONFLICT"
112
113
114
115
116
117
118
119 def test_500_is_redacted_and_never_leaks_pii(client:
120     TestClient) -> None:
121     response = client.get("/debug/leaky")
122     body = assert_problem(response, 500)
123     for fragment in PII_FRAGMENTS:
124         assert fragment not in response.text
125     # The generic shape is preserved; the caller gets
126     # an actionable trace_id.
127     assert body["code"] == "NRW-INTERNAL"
128
129
130
131
132
133
134
135 def test_429_carries_retry_after(client: TestClient) ->
136     None:
137     response = client.get("/shipments-throttled")
138     assert_problem(response, 429)
139     assert response.headers["Retry-After"] == "30"

```

```

125 def test_503_dependency_failure_is_retriable(client:
    TestClient) -> None:
126     body = assert_problem(client.get("/warehouse/ping")
        , 503)
127     assert body["code"] == "NRW-WAREHOUSE-UNAVAILABLE"
128
129
130 def test_client_supplied_trace_id_is_echoed(client:
    TestClient) -> None:
131     response = client.get(
132         "/shipments/shp_99999999", headers={"X-Trace-Id": "req-abc-123"}
133     )
134     assert response.headers["X-Trace-Id"] == "req-abc-123"
135     assert response.json()["trace_id"] == "req-abc-123"
136
137
138 def test_malformed_trace_id_is_replaced(client:
    TestClient) -> None:
139     response = client.get(
140         "/shipments/shp_99999999", headers={"X-Trace-Id": "a" * 500}
141     )
142     trace_id = response.headers["X-Trace-Id"]
143     assert trace_id != "a" * 500
144     assert len(trace_id) <= 64
145
146
147 def test_happy_path_unaffected(client: TestClient) ->
    None:
148     response = client.get("/shipments/shp_00000001")
149     assert response.status_code == 200
150     assert response.json()["shipment_id"] == "shp_00000001"

```

Running the suite (all ten tests pass, in-process, in about a second):

```

1 pytest -q
2 .....
3
   [100%]
3 10 passed in 1.05s

```

8.3.6 A Machine-Readable Error-Code Registry

Listing 7.6 closes the loop between code, documentation, and operations. The registry is a frozen dataclass table—one row per error code—that *validates itself at import time* (a duplicate code or missing remediation crashes the deploy, not the customer) and renders the public Markdown catalogue that the type URIs promise. Clients, docs, and alert rules all derive from this single artifact; that is what “machine-readable” buys.

```

1  """error_codes.py--Machine-readable error-code
   registry with doc generation.
2
3  Listing 7.6. One source of truth: clients, docs, and
   alerts all derive from
4  this registry. Run directly to print the generated
   Markdown catalogue.
5  """
6  from __future__ import annotations
7
8  from dataclasses import dataclass
9  from enum import Enum
10
11
12  class Retriable(Enum):
13      YES = "yes"
14      NO = "no"
15      CONDITIONAL = "conditional"
16
17
18  @dataclass(frozen=True)
19  class ErrorCode:
20      code: str          # stable, machine-readable,
        never reused
21      http_status: int
22      type_slug: str     # suffix of the RFC 9457 type
        URI
23      title: str        # short, stable, localizable
        key
24      retrieable: Retriable
25      owner: str        # team accountable for this
        failure mode
26      remediation: str  # one-line client-actionable
        guidance
27
28
29  REGISTRY: tuple[ErrorCode, ...] = (
30      ErrorCode(

```

```

31         code="NRW-VALIDATION-FAILED",
32         http_status=422,
33         type_slug="validation-failed",
34         title="Validation␣Failed",
35         retrieable=Retriable.NO,
36         owner="api-platform",
37         remediation="Fix␣the␣fields␣named␣in␣errors␣[␣.
           pointer␣and␣resend.",
38     ),
39     ErrorCode(
40         code="NRW-SHIPMENT-NOT-FOUND",
41         http_status=404,
42         type_slug="shipment-not-found",
43         title="Shipment␣Not␣Found",
44         retrieable=Retriable.NO,
45         owner="fulfillment",
46         remediation="Verify␣the␣shipment_id;␣do␣not␣
           retry␣blindly.",
47     ),
48     ErrorCode(
49         code="NRW-SHIPMENT-STATE-CONFLICT",
50         http_status=409,
51         type_slug="shipment-state-conflict",
52         title="Shipment␣State␣Conflict",
53         retrieable=Retriable.CONDITIONAL,
54         owner="fulfillment",
55         remediation="Refresh␣the␣resource,␣re-evaluate␣
           preconditions,␣retry␣once.",
56     ),
57     ErrorCode(
58         code="NRW-RATE-LIMIT-EXCEEDED",
59         http_status=429,
60         type_slug="rate-limit-exceeded",
61         title="Rate␣Limit␣Exceeded",
62         retrieable=Retriable.YES,
63         owner="api-platform",
64         remediation="Back␣off␣for␣Retry-After␣seconds,␣
           then␣retry␣with␣jitter.",
65     ),
66     ErrorCode(
67         code="NRW-WAREHOUSE-UNAVAILABLE",
68         http_status=503,
69         type_slug="warehouse-unavailable",
70         title="Dependency␣Unavailable",
71         retrieable=Retriable.YES,
72         owner="warehouse-integrations",

```

```

73         remediation="Retry with exponential backoff;
           alert if it persists > 5 min.",
74     ),
75     ErrorCode(
76         code="NRW-INTERNAL",
77         http_status=500,
78         type_slug="internal",
79         title="Internal Server Error",
80         retrieable=Retriable.NO,
81         owner="api-platform",
82         remediation="Report the trace_id to support; do
           not retry mutations.",
83     ),
84 )
85
86
87 def validate_registry(registry: tuple[ErrorCode, ...] =
    REGISTRY) -> None:
88     """Fail fast at import time if the registry is
           internally inconsistent."""
89     codes = [entry.code for entry in registry]
90     assert len(codes) == len(set(codes)), "duplicate
           error codes"
91     for entry in registry:
92         assert entry.code.startswith("NRW-"), f"bad
           prefix: {entry.code}"
93         assert 400 <= entry.http_status <= 599, f"bad
           status: {entry.code}"
94         assert entry.remediation, f"missing remediation
           : {entry.code}"
95         if entry.http_status < 500:
96             # Client-facing codes must explain what the
               CLIENT can do.
97             assert "support" not in entry.remediation.
               lower() or entry.http_status == 500
98
99
100 def render_markdown(registry: tuple[ErrorCode, ...] =
    REGISTRY) -> str:
101     """Generate the public error catalogue page from
           the registry."""
102     lines = [
103         "# Northwind Robotics API -- Error Code
           Catalogue",
104         "",
105         "| Code | HTTP | Title | Retriable | Owner |

```

```

106         Remediation_|",
107         "|---|---|---|---|---|---|",
108     ]
109     for entry in registry:
110         lines.append(
111             f"|_{entry.code}|_{entry.http_status}|_{entry.title}|"
112             f"|_{entry.retriable.value}|_{entry.owner}"
113             f"|_{entry.remediation}|_"
114         )
115     return "\n".join(lines)
116
117 validate_registry()
118
119 if __name__ == "__main__":
120     print(render_markdown())

```

8.4 Common Pitfalls & Anti-Patterns

Each entry below is a failure mode observed in real production estates, with its cost and its cure.

200 OK with an Error Body

Symptom: `{"success": false}` rides a 200. *Cost:* caches store failures, retry middleware sleeps through outages, and availability dashboards show green during the fire. *Cure:* the status code reflects the outcome; the body explains it. No exceptions for REST-style HTTP.

Leaking Stack Traces and PII

Symptom: a 500 body contains `Traceback (most recent call last)`, a SQL fragment, or the caller's email address. *Cost:* reconnaissance data for attackers, regulatory exposure, PII smeared across every log pipeline that captures bodies. *Cure:* generic client-facing `detail`, full context in server logs under the trace ID, redaction middleware as the last gate (Listing 7.3).

Inconsistent Error Shapes per Endpoint

Symptom: `/orders` returns `{"error": ...}`, `/shipments` returns `{"message": ...}`, the gateway adds an HTML page for good measure. *Cost:* every consumer writes N error parsers and still misses the gateway's. *Cure:*

one pipeline, one media type, schema-validated in CI for *every* endpoint—including framework-generated 404s and 405s.

Stringly-Typed Client Error Matching

Symptom: consumer code does `if "insufficient" in response["message"]`.
Cost: your copy edit is their breaking change; localization is a breaking change; fixing a typo is a breaking change. *Cure:* match on `type` and `code` only; treat `title/detail` as display text. Providers: never change `type/code` semantics in place.

500 for Validation Failures

Symptom: a bad payload raises `KeyError` deep in a handler and surfaces as 500. *Cost:* clients give up (5xx reads as “not my fault, retry later”), your error budget burns for their typo, and your on-call investigates non-incidents. *Cure:* validate at the boundary with a real schema; unreachable-by-validation code paths may still 500, and that is fine—that is what 500 is for.

Swallowing Exceptions into Generic 400

Symptom: `except Exception: return 400`. *Cost:* the mirror image of the previous sin: real outages masquerade as client errors, clients stop retrying during your incident, and paging logic that keys on 5xx never fires. *Cure:* catch specific exceptions, map each to its true class, and let the genuinely unexpected fall through to the 500 handler.

Logging Secrets from Failed Payloads

Symptom: a debug log line dumps the raw request body on validation failure—including `password` and `api_key` fields. *Cost:* credential material in log aggregators with broad read access and long retention. *Cure:* log the *redacted* error summary, the JSON Pointers of failing fields, and the trace ID—never the raw payload; where payload capture is unavoidable, redact before writing (Chapter 5).

8.5 Hands-On Production Lab & Real-World Case Study

8.5.1 Lab: Retrofitting a Messy Service to Uniform RFC 9457

You inherit `legacy_quotes.py`, a FastAPI service for Northwind Robotics' spare-parts quoting. It is a museum of error sins: one endpoint returns 200 with `{"ok": false}`; another returns a bare string; a third lets a `sqlite3.IntegrityError` escape as an HTML 500; validation errors use FastAPI's default (non-problem) shape; and a "helpful" handler echoes the submitted customer email into its error detail.

Objective. Without changing any success-path behavior, make every error the service can emit conform to the RFC 9457 pipeline of Section 8.3. **Steps.**

1. **Inventory.** Exercise every endpoint's failure modes with `TestClient` and record the current (inconsistent) shapes in a table. This is your regression baseline.
2. **Install the pipeline.** Drop in `problem_details.py` and `redaction.py` unmodified; register handlers and middleware in the app factory.
3. **Convert domain failures.** Replace ad-hoc returns with `ProblemException` subclasses; choose precise statuses (409 for the duplicate-part-number case, 422 for semantic quote errors).
4. **Fix the 200-with-error endpoint last**—it requires consumer coordination; document the change in the OpenAPI **responses** section [Ope21].
5. **Write the contract tests first, per endpoint**, using the `PROBLEM_SCHEMA` helper; run them red, then green.
6. **Prove redaction.** Add the intentionally leaky handler back behind a test-only flag and assert no PII fragment survives.

Deliverable. A diff touching no success-path logic, a contract test per error path, and a one-page "error contract" section for the API's README generated from your registry.

8.5.2 War Story: The PII Leak That Hid in detail

A fulfillment API (composite of several real incidents) validated customer notification preferences against a downstream directory. When the directory

rejected an address, the developer—debugging an integration under deadline pressure—returned the directory’s error verbatim in the problem **detail**: “*mailbox jane.doe@example.invalid not provisioned for account 123-45-6789.*” The response was a correctly-typed, correctly-shaped 422. Every control that looked at *structure* passed.

The blast radius was invisible for weeks because the leak rode inside a “valid” error contract. Three systems made it worse: the CDN logged response bodies for 422s during a debugging flag that was never turned off; the APM tool captured error bodies as “context”; and a third-party integrator helpfully surfaced **detail** in their end-user-facing UI—so end users occasionally saw *other* customers’ contact data when batch jobs interleaved failures.

Discovery came from an audit query, not an alert: a routine scan of APM payloads for the regex `\S+@\S+` lit up thousands of error bodies. Remediation proceeded in four moves, in this order, and the order is the lesson: (1) *stop the bleeding at the edge*—deploy redaction middleware the same day, before the code fix, because the fix had other callers; (2) *fix the producer*—the directory client now maps upstream errors to typed exceptions with generic details; (3) *purge and rotate*—body capture disabled at the CDN, APM payload retention shortened, affected logs expunged per policy; (4) *make it a test, not a memory*—a contract test now asserts a PII-fragment denylist against every error path, which is precisely `test_500_is_redacted_and_never_leaks_pii` in Listing 7.5.

The post-incident review concluded that the RFC 9457 adoption had been *structurally* complete and *semantically* hollow: shape without content policy. Every control the team added afterward—registry-reviewed details, denylist contract tests, edge redaction—is about content. Shape is necessary; it is nowhere near sufficient.

8.6 Self-Assessment Exercises & Deep-Dive Questions

1. A request with malformed JSON arrives at `POST /shipments`. Which layer of the taxonomy fires, what status results, and why does the answer *not* involve Pydantic?
2. Your teammate proposes returning 404 for unauthorized access to a resource. Construct the threat model where this is correct, and the one where it leaks information anyway.
3. The registry in Listing 7.6 marks 409 as *conditionally* retrievable. Write the precise client algorithm (pseudo-code) for a safe 409 retry.

4. Why does Listing 7.2 declare `unit_weight_kg` with `strict=False`? What breaks if you remove the opt-out, and what breaks if you also remove the companion validator?
5. Design the JSON Pointer for a failure inside `lines[2].dimensions.width`. What RFC 6901 escaping must you apply if a key itself contains `/`?
6. A client reports “your API returns 500s.” The dashboard shows 5xx-dominated traffic that is actually 503 from your circuit breaker. What did the missing distinction cost you, and what metric label prevents recurrence?
7. Argue both sides: should `title` be localized? What invariant must hold for localization to be safe?
8. Your redaction middleware must not buffer 50 MB streaming problem bodies. Adapt Listing 7.3’s design; what trade-off do you accept?
9. Extend the contract suite so that a *new* undocumented extension member fails CI. Which schema keyword already does this, and what workflow does that imply for adding members deliberately?
10. Map the chapter’s alert thresholds onto the error-budget policy of Chapter 18. Which single error code should *never* page, and why?

8.7 Interview Q&A Vault

Thirty-four questions, junior through staff, with model answers calibrated to what a strong candidate says out loud.

Junior

- Q1.** *What is the difference between 400 and 422?* **A.** 400 means the request is malformed at the protocol/syntax level—unparseable JSON, bad framing. 422 means the request was syntactically valid but semantically unacceptable: the JSON parsed, the schema shape held, but the values violate domain rules. Mnemonic: 400 = *can’t read you*; 422 = *read you, reject what you said*.
- Q2.** *What media type does RFC 9457 define, and why a distinct media type at all?* **A.** `application/problem+json`. The distinct type lets clients, gateways, and tooling identify error bodies without content sniffing, and lets the same endpoint serve a success schema and an error schema under one URL.

- Q3.** *Name the five standard members of a problem detail object.* **A.** `type` (URI identifying the problem class), `title` (stable short summary of the class), `status` (the HTTP status, echoed), `detail` (human explanation of this occurrence), `instance` (URI identifying this occurrence).
- Q4.** *Why is returning 200 with an error in the body harmful?* **A.** Every HTTP intermediary trusts the status code: caches may store the failure, retry logic sees success, monitoring reports availability during an outage. The status must reflect the outcome; the body elaborates.
- Q5.** *What is the difference between 401 and 403?* **A.** 401 = unauthenticated (“prove who you are”) and must carry `WWW-Authenticate`; 403 = authenticated but not authorized (“I know you; no”). Re-login fixes the first, never the second.
- Q6.** *Should a client match on the `detail` string to decide handling?* **A.** Never. `detail` is human-facing, per-occurrence, and free to change (or be localized). Clients match on `type` and machine-readable extension members like `code`.
- Q7.** *What does `Retry-After` do and which statuses pair with it?* **A.** It tells the client how long to wait before retrying (delay-seconds or an HTTP date), and pairs with 429 and 503. Without it, clients invent backoff—often synchronized, often harmful.
- Q8.** *In Pydantic v2, what is the difference between `field_validator` and `model_validator`?* **A.** A field validator sees one field and enforces local rules (pattern, range); a model validator (especially `mode="after"`) sees the whole validated model and enforces cross-field invariants like “total weight under 30 kg for express.”

Mid-Level

- Q9.** *What belongs in `detail` versus `title`?* **A.** `title` is per-*type*: stable, short, localizable, safe to display as a heading; it must not vary between occurrences. `detail` is per-*occurrence*: what happened this time, with the specifics a human needs—but never internals, secrets, or PII, and never anything a client would parse.
- Q10.** *How do you make errors machine-actionable?* **A.** Stable `type` URIs plus extension members: a short `code` for alerting and dashboards, a `retriable` classification (or per-code registry entry), structured field errors with JSON Pointers for 422s, `trace_id` for support, and

remediation hints where the client can act.

- Q11.** *Why must **type** be a URI?* **A.** Global uniqueness without a central registry (mint under your domain), dereferenceability (the identifier doubles as the documentation link), and opacity (clients match the string, not fetched content). `about:blank` is the sanctioned generic fallback.
- Q12.** *When do you return **410** instead of **404**?* **A.** When you *know* the resource existed and is permanently gone—deleted exports, sunset versions. **410** is cacheable and tells sync clients to purge. Use **404** when you do not know or do not want to reveal history; for sensitive resources, uniform **404** avoids an existence oracle.
- Q13.** *Give three distinct **409** conflict domains.* **A.** State-machine conflict (cancel a shipped shipment), uniqueness conflict (natural key taken), reference conflict (delete a warehouse holding inventory); a fourth is precondition/concurrency failure, though **412** is more precise for **If-Match**-style checks.
- Q14.** *Distinguish **500**, **502**, **503**, and **504** operationally.* **A.** **500**: my code faulted—page my team. **502**: upstream answered invalidly—page the integration owner. **503**: deliberate unavailability (shedding, maintenance, open breaker)—usually self-healing, pair with **Retry-After**. **504**: upstream never answered in time. The distinction drives paging, retry policy, and which dashboard lights up.
- Q15.** *Why is strict mode at the API boundary recommended, and what is the catch?* **A.** Strict mode rejects silent coercion ("**12**" → **12**), so client type bugs surface as fixable **422**s instead of corrupted data. The catch: JSON cannot express some Python types (`Decimal`, `datetime` as literals), so those fields need per-field `strict=False` plus hardening validators.
- Q16.** *What is a JSON Pointer doing in an error response?* **A.** It locates the exact failing value in the submitted document (`/lines/0/quantity`, RFC 6901), so client tooling can highlight the offending field programmatically—unambiguous for machines, immune to message localization.
- Q17.** *Your service's default framework **404** returns HTML. Is that a problem?* **A.** Yes: it violates the uniform error contract—clients parsing `problem+json` get a surprise. Framework and gateway-generated errors (**404**, **405**, **413**) must pass through the same pipeline; contract tests should include at least one framework-generated error.

Senior

- Q18.** *Design an error taxonomy for a payments API.* **A.** Three axes: layer (transport/protocol/application), ownership (client-fixable 4xx, server-owned 5xx, dependency-caused 502/503/504 from the processor), and retriability—which for payments must compose with idempotency keys, because retrying a capture after a 504 without a key risks double-charging. Concrete classes: 422 semantic card data errors with per-field pointers; 402-style declined payment as a distinct, documented type (not a generic 400); 409 for duplicate idempotency keys with different payloads; 503 + **Retry-After** for processor outages. Every class gets a registry row with owner, retriability, and remediation.
- Q19.** *How do you evolve an error contract without breaking clients?* **A.** Additively: new extension members are safe (clients must ignore unknowns); new **type** URIs for new classes are safe (clients must have a default branch). Never change the meaning of an existing **type/code**, never remove a documented member, and treat **title** copy edits as safe only because clients were told never to match on it.
- Q20.** *How do you keep PII out of error responses at scale?* **A.** Defense in depth: (1) type the pipeline so **detail** strings are built from curated templates, not exception messages; (2) never interpolate raw input values into messages; (3) redaction middleware as the last gate with a denylist of patterns and stack-trace markers; (4) contract tests asserting a PII-fragment denylist on every error path; (5) log-side redaction before writing failed payloads. Each layer assumes the previous one fails.
- Q21.** *How do you handle validation that requires a database—say, “SKU must be stocked at destination”?* **A.** That is a domain invariant, not boundary validation: the payload is well-formed either way. Check it in the service layer and raise a typed domain exception (409 or 422 with a specific **type** like `.../sku-not-stocked`). Do not smuggle database calls into Pydantic validators—models become untestable, and request latency becomes validation latency with no caching story.
- Q22.** *What should the correlation/trace ID story be across a multi-service request?* **A.** Mint or accept at the edge (validated: bounded length, character allowlist), propagate on every downstream call, embed in every problem body and log line, echo in response headers. Support

flow: customer reports trace ID → one query reconstructs the request tree. Never reflect unvalidated inbound IDs—header smuggling and log injection ride on that.

- Q23.** *A client asks you to add the offending value to each 422 error for debugging convenience. Do you?* **A.** No as a default: the offending value is frequently a credential or PII pasted into the wrong field, and error bodies flow to logs, APMs, and client UIs. Offer the JSON Pointer and the rule; if a value echo is truly needed, gate it behind an authenticated debug-mode header that is disabled in production and redaction-aware.
- Q24.** *How do you test that your error contract holds, beyond unit tests?* **A.** Schema-validate every error path in contract tests with `additionalProperties: false` so undocumented members fail CI; property/fuzz test the redactor with an adversarial corpus; chaos-inject dependency failures and assert the mapped status family; and add a gateway-level test so intermediary errors match the contract too.
- Q25.** *When is 503 preferable to just letting requests time out?* **A.** Always, when you know you are shedding: a fast, explicit 503 + `Retry-After` preserves client goodwill, frees connections immediately, feeds accurate metrics, and lets well-behaved clients back off on schedule. A timeout costs the client the full deadline and teaches nothing.
- Q26.** *How does the error contract interact with caching?* **A.** Error statuses have distinct cacheability: 404/410 are cacheable by default (heuristic), and that is often desirable; 429/503 must not be cached beyond their validity—set `Cache-Control: no-store` on error responses that are occurrence-specific. A cached 500 replayed for an hour is an outage amplifier.

Staff

- Q27.** *You are standardizing errors across forty services and six languages. What do you ship first?* **A.** The registry and the schema, not a library: a language-neutral problem schema, a governed error-code registry with owners and retrievability, and a contract test kit each service can run in CI. Libraries per language follow, generated from or validated against those artifacts. Standardizing code before standardizing data produces forty idiomatic dialects of inconsistency.
- Q28.** *How do you decide what is a new type versus a new detail of an existing type?* **A.** Ask whether a client should *branch* on it. If

handling differs—different remediation, different retrievability—it is a new type. If only the human explanation differs, it is the same type with a different **detail**. Type proliferation is real: every type is a documented, versioned, supported promise.

- Q29.** *Design the alert policy on top of this chapter’s taxonomy.* **A.** Metrics as `errors_total{endpoint, status, code}`. Page on 5xx rate > 1% over 5 min and on any single code spiking > 10× baseline; ticket on sustained 4xx > 25% (usually a broken consumer). Suppress 503-shedding pages when the breaker is working as designed. Wire 5xx burn into the Chapter-18 error budget so deploys freeze on budget exhaustion.
- Q30.** *A regulator asks you to prove error responses never contain PII. What is your evidence chain?* **A.** Typed pipeline (details are template-built), middleware redaction (defense in depth), CI contract tests with PII denylists on every error path, a fuzzed redaction corpus with regression history, log-side redaction with access-controlled raw capture, and periodic audit scans of captured payloads. Process evidence (the pipeline) plus empirical evidence (the scans); neither alone is convincing.
- Q31.** *Should error responses be part of your OpenAPI document? What exactly do you publish?* **A.** Yes [Ope21]: for each operation, the 4xx/5xx responses with media type `application/problem+json`, the problem schema including your extension members, and **default** responses for the catch-alls. Per-operation, enumerate the **type** values that operation can emit—that enumeration is the actionable part of the contract.
- Q32.** *How do you keep gateway-generated errors (auth, throttling, body-size limits) inside the contract?* **A.** The gateway is part of the API as clients see it, so its errors must be mapped to `problem+json` at the edge: configure the gateway’s error templates, mint gateway-scoped codes in the same registry (`NRW-GATEWAY-...`), and run contract tests through the gateway, not only against the app in-process.
- Q33.** *Your company acquires another firm whose API returns XML-only errors. Draft the migration.* **A.** Contract-first: publish the target `problem+json` schema and registry, stand up a translation shim at the edge that emits both (content negotiation on **Accept**), instrument which clients still negotiate XML, migrate them with sunset headers and a deadline, and keep `application/problem+xml` only until telemetry shows zero dependers. RFC 9457 retains the XML media

type, so the shim is standards-legal throughout.

Q34. *What is the deepest mistake teams make with error design?* **A.** Treating errors as output formatting rather than as *behavioral contract*: clients build retry loops, reconciliation jobs, and compliance evidence on your failures. The shape is the easy half; the hard half is semantics—ownership, retrievability, stability guarantees, and the discipline to keep content policy (no PII, no internals) as tested code rather than wiki aspiration. Teams that get this right page less, integrate faster, and survive audits; teams that do not are debugging other people’s string matching forever.

8.8 External Bibliography & Further Reading

- **RFC 9457 — Problem Details for HTTP APIs** [Not23]. The primary source. Read §3 (members), §4 (extension members), and the security considerations; note its obsolescence of RFC 7807 and what changed (extension handling clarifications, modern HTTP alignment).
- **RFC 7807 (historical)**. The 2016 original. Still cited in older libraries and articles; read it only to recognize legacy references, and follow 9457 for anything new.
- **RFC 9110 — HTTP Semantics, §15 (Status Codes)** [Fie22a]. The authoritative definition of every status code discussed here, including cacheability defaults and the `WWW-Authenticate/Retry-After` header requirements.
- **RFC 6901 — JSON Pointer**. The path syntax used for per-field error locations, including the `~0/ ~1` escaping rules.
- **Pydantic v2 documentation** [Col24]. Validators, strict mode, Annotated constraints, and the error taxonomy (`loc/type/msg`) our mapper consumes.
- **FastAPI documentation** [Ram24]. Exception handlers, middleware, `RequestValidationError`, and `TestClient` for in-process contract tests.
- **OpenAPI 3.1 specification** [Ope21]. Documenting `application/problem+json` responses per operation and the `default` response.

- **JSON Schema documentation** [JSO22]. Draft 2020-12 validation semantics used by the contract tests.
- **Stripe API error documentation**. An exemplary industrial error contract: stable `type`, per-language handling guides, and 402-class payment-specific codes worth studying as a design reference.
- **Google AIP-193 (Errors)**. A mature enterprise standard for error payloads and codes; compare its decisions (canonical codes, `ErrorInfo` details) with RFC 9457's.
- **Microsoft REST API Guidelines — error response section**. A second industrial reference point, useful for the `error.code/error.details` nesting pattern.
- **Nygard, *Release It!*** [Nyg18]. Failure modes, circuit breakers, and why 503-as-shedding is a feature of healthy systems.

8.9 Capstone Projects

Five projects, progressive in difficulty. All are offline-first, deterministic, and use synthetic Northwind Robotics data only.

8.9.1 Project 1 [junior] — Problem-Details Retrofit

Objective. Take the intentionally messy `legacy_quotes.py` from this chapter's lab (or any small FastAPI service you wrote for Chapter 6) and retrofit uniform RFC 9457 errors without touching success-path behavior.

Inputs/Datasets. The lab service; this chapter's `problem_details.py`; a synthetic parts catalogue (10 fictional SKUs).

Steps.

1. Inventory current error shapes with `TestClient`.
2. Register the problem handlers; convert ad-hoc errors to `ProblemException` subclasses.
3. Write one contract test per error path using the shared `PROBLEM_SCHEMA`.
4. Update the OpenAPI `responses` documentation for two endpoints.

Acceptance Criteria.

- ☐ Every error response (including framework 404) is schema-valid `application/problem+json`.
- ☐ No success-path test changed; all still pass.

☐ 422s carry JSON-Pointer-located `errors[]`.

☐ `pytest -q` green from a fresh `.venv`.

Stretch Goals. Add Accept-Language-driven localization of `title` for two locales; add a client helper that branches on `code`.

8.9.2 Project 2 [mid] — Error-Code Registry & Documentation Generator

Objective. Build a standalone registry package for a (fictional) multi-service Northwind Robotics platform and generate both human docs and a machine-readable artefact from it.

Inputs/Datasets. Listing 7.6 as a seed; a list of 15–20 invented error codes across three services (shipments, warehouse, billing).

Steps.

1. Model the registry with self-validation at import (duplicates, status ranges, remediation presence, retrievability vocabulary).
2. Generate a Markdown catalogue per service and a single `errors.json` for client SDKs.
3. Write a CI-style check: any exception class whose `code` is absent from the registry fails the build (introspect the exception hierarchy).
4. Emit a changelog diff between two registry versions.

Acceptance Criteria.

- ☐ Import-time validation catches an injected duplicate and an injected missing remediation (tests prove both).
- ☐ Generated Markdown renders a complete catalogue; JSON validates against a schema you publish.
- ☐ Orphan-code check fails a deliberately broken service.
- ☐ Registry diff output is deterministic (stable ordering).

Stretch Goals. Per-code deep links resolvable as `type` URIs; deprecation workflow (mark, never delete, never reuse).

8.9.3 Project 3 [mid] — Redaction Library with Fuzz Corpus

Objective. Harden Listing 7.3’s scrubber into a standalone, tested library and attack it yourself before an attacker does.

Inputs/Datasets. A hand-built adversarial corpus: 50+ synthetic strings mixing fake emails (`*@example.invalid`), fake tax IDs, fake card numbers, fake stack traces, near-misses (valid text that merely *looks* like PII), and Unicode edge cases.

Steps.

1. Extract `scrub` into a package with a pluggable rule list.
2. Build the corpus as data files; every corpus entry asserts redact/not-redact expectations.
3. Add a deterministic property-style sweep: seeded generation of pattern-like mutations around each rule.
4. Measure and bound false-positive rate on a benign-text corpus.

Acceptance Criteria.

- ☐ 100% of malicious corpus entries redacted.
- ☐ Documented, bounded false-positive rate on benign corpus; regressions fail CI.
- ☐ All generation seeded; suite is deterministic and offline.
- ☐ Rule list extensible without editing library code.

Stretch Goals. Streaming redaction for chunked bodies; structured (JSON-aware) scrubbing that redacts by field name as well as by pattern.

8.9.4 Project 4 [senior] — Error-Observability Dashboard Specification

Objective. Specify—and prototype offline—the error observability layer for the shipments API: metrics, alert rules, and a dashboard definition, all derived from the error-code registry.

Inputs/Datasets. A synthetic, seeded log generator producing a week of request outcomes with known injected incidents (a dependency outage, a bad client deploy, a throttling event).

Steps.

1. Define the metric taxonomy (`errors_total{endpoint, status, code}`) and exemplar queries (error rate per code, per endpoint).
2. Encode the chapter’s alert thresholds as executable rules over the synthetic dataset.

3. Prototype an offline aggregator that replays the log stream and prints alert firing timelines.
4. Write the dashboard spec: panels, drill-down from code to trace IDs, error-budget burn panel referencing Chapter 18.

Acceptance Criteria.

- ☐ Replay deterministically detects all three injected incidents with zero false pages on the baseline day.
- ☐ Alert rules are data (declarative config), not code.
- ☐ Dashboard spec traces every panel to a registry code.
- ☐ Burn-rate panel matches hand-computed values on a 100-request worked example.

Stretch Goals. Burn-rate windows (fast/slow) per Chapter 18; anomaly baseline per code using the synthetic history.

8.9.5 Project 5 [staff] — Chaos Error-Injection Test Harness

Objective. Build an in-process harness that injects failures (timeouts, malformed upstream payloads, connection resets, slow leaks) at the seams of a two-service synthetic system, and proves the *entire chain*—status mapping, problem shape, redaction, correlation, metrics labels—survives chaos.

Inputs/Datasets. A two-service toy (gateway service calling a warehouse stub); a seeded fault schedule; this chapter’s full pipeline installed in both services.

Steps.

1. Implement fault injectors as dependency overrides: deterministic schedule from a seed, one fault per run or scripted mixes.
2. Assert, per fault: correct status family (502/504/503, never 500), schema-valid problem body, trace ID continuity across both services, PII denylist holds.
3. Produce a fault $\times$ contract matrix showing which faults each test catches; eliminate blind spots.
4. Write the runbook entry template each fault class maps to.

Acceptance Criteria.

- ☐ Every injected fault yields a schema-valid problem response with the correct status family—no raw 500s, no HTML.
- ☐ Trace IDs join both services' logs for every scenario.
- ☐ Full suite deterministic under a fixed seed; runs offline in under a minute.
- ☐ Matrix reviewed: every fault class has at least one failing-before/passing-after test story.

Stretch Goals. Composed faults (timeout *plus* redaction bypass attempt); a “contract score” metric across the matrix trending over registry versions.

8.10 Python Dependencies Appendix

Everything in this chapter installs from the following `requirements.txt`:

```
1 fastapi >=0.110
2 pydantic >=2.7
3 jsonschema >=4.21
4 httpx >=0.27
5 pytest >=8
```

Install on Windows with PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

8.10.1 fastapi (≥ 0.110)

Description. The ASGI web framework used for the service, its exception-handler hooks, and `TestClient`. **Why included.** Chapter-6 continuity; its `RequestValidationError` carries the Pydantic error array our 422 mapper consumes, and its handler registration is the seam where the single pipeline attaches. **Alternatives.** Starlette alone (lower-level, same machinery); Flask or Django REST framework (sync ecosystems with their own exception patterns—the RFC 9457 design transfers, the code does not). **Installation.** `pip install "fastapi>=0.110"`.

8.10.2 pydantic (≥ 2.7)

Description. Data-validation library powering both the boundary models (`Shipment`) and the `ProblemDetail` response model. **Why included.**

Strict-mode enforcement, `field_validator` / `model_validator`, `Annotated` constraints, and a structured error array with `loc` tuples that map cleanly to JSON Pointers. **Alternatives.** `dataclasses` plus hand-rolled checks (verbose, error-prone); `attrs+cattrs` (capable, less integrated with FastAPI); `Marshmallow` (mature but schema-parallel to your models). **Installation.** `pip install "pydantic>=2.7"`.

8.10.3 `jsonschema` (≥ 4.21)

Description. JSON Schema validation for Python, used to contract-test every error response against `PROBLEM_SCHEMA` [JSO22]. **Why included.** Turns “our errors have a consistent shape” from a hope into a CI gate; `additionalProperties: false` makes undocumented extension members a build failure. **Alternatives.** Hand-written assertions per field (brittle); validating via OpenAPI tooling (heavier; OpenAPI 3.1 embeds JSON Schema 2020-12 anyway [Ope21]). **Installation.** `pip install "jsonschema>=4.21"`.

8.10.4 `httpx` (≥ 0.27)

Description. The HTTP client underneath FastAPI’s `TestClient` and the client used in this book’s consumption chapters. **Why included.** Enables fully in-process ASGI tests—no sockets, no network, deterministic—while exercising the real request/response stack. **Alternatives.** `requests` (no ASGI transport; Chapter 2 discusses the trade-offs); `aiohttp` (different async model). **Installation.** `pip install "httpx>=0.27"`.

8.10.5 `pytest` (≥ 8)

Description. Test runner and fixture machinery for the contract suite. **Why included.** Fixture-scoped `TestClient`, expressive assertion output, and the ecosystem standard; the suite is ten plain test functions with one shared assertion helper. **Alternatives.** `unittest` (stdlib, more ceremony); `nose2` (effectively unmaintained). **Installation.** `pip install "pytest>=8"`.

8.11 Chapter Summary & Key Takeaways

- **Errors are contract.** Design, version, document, and test the failure surface with the same rigor as the success surface.
- **Classify before you code.** Transport versus protocol versus application; client-fixable (4xx) versus server-owned (5xx) versus dependency-

caused; retrievable versus not—with a written decision procedure, not vibes.

- **RFC 9457 is the shape.** `application/problem+json` with `type/title/status/detail/instance` plus documented extension members; `type` is the stable, URI-identified, documentation-anchored class; RFC 7807 is history.
- **Status precision pays.** 400 (syntax) versus 422 (semantics); 404 versus 410; 401 versus 403 versus 407; 429+**Retry-After**; 500 versus 502/503/504; and never 200-with-error.
- **One pipeline.** Domain exceptions → problem mapper → redaction → response, with correlation IDs and full-fidelity server-side logging. No endpoint improvises.
- **Strict at the boundary, invariant-aware in the domain.** Pydantic strict mode with deliberate per-field exceptions; model validators for cross-field rules; service layer for stateful invariants.
- **Content policy is code.** No internals, no PII, no secrets in error bodies or logs—enforced by typed templates, denylist contract tests, and redaction middleware, not by convention.
- **Operate the taxonomy.** Metrics per endpoint, status, and code; page on 5xx burn; feed it into the Chapter-18 error budget.

8.12 Quick-Reference Card

Status-Code Selection

Problem-Details Members

Redaction Checklist

- ☐ `detail` built from curated templates, not raw exception text.
- ☐ No stack traces, SQL, hostnames, or file paths in any client body.
- ☐ No PII (emails, tax IDs, card numbers) echoed from payloads.
- ☐ No secrets (tokens, keys, passwords) in bodies *or* logs.
- ☐ Redaction middleware is the last gate on every `problem+json` response.

Code	Retriable	Use when
400	no	malformed syntax/framing; unreadable request
401	after auth	missing/failed credentials; send WWW-Authenticate
403	no	authenticated, not authorized (or use 404 to hide existence)
404	no	not found, or existence must not be revealed
407	after auth	proxy demands its own authentication
409	conditional	conflict with current resource state; refresh then retry
410	no	known-permanently-gone; clients should purge
412	conditional	precondition (If-Match etc.) failed
415	no	unsupported Content-Type
422	no	syntactically valid, semantically invalid; include field pointers
429	yes + Retry-After	client exceeds rate policy
500	only if idempotent	your code faulted; page your team
502	yes	upstream returned an invalid response
503	yes + Retry-After	deliberate shedding/maintenance/open breaker
504	yes	upstream timed out

Table 8.2: Error status selection cheat sheet.

- ☐ Full context logged server-side under `trace_id`, access-controlled.
- ☐ Contract tests assert a PII-fragment denylist on every error path.
- ☐ Redactor fuzzed against an adversarial corpus; suite deterministic.

8.13 Standardized Evidence Addendum

8.13.1 Benchmark Snapshot Template

8.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

Member	Rule of thumb
<code>type</code>	URI, stable forever, documents the class; clients match on it
<code>title</code>	short, stable per type, localizable, display-only
<code>status</code>	must equal the response status
<code>detail</code>	per-occurrence, human-only, no PII/internals, never parsed
<code>instance</code>	identifies this occurrence (path or URN)
<code>code</code> (ext.)	short machine code from the registry; for alerts and SDKs
<code>trace_id</code> (ext.)	correlation ID; the support conversation starter
<code>errors[]</code> (ext.)	per-field failures with RFC 6901 JSON Pointers

Table 8.3: Problem-details member cheat sheet.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 8.4: Minimum benchmark table to keep per chapter.

8.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 8.5: Operational budget template for this chapter's technique.

8.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

8.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

8.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

8.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 8.6: Decision card to complete per chapter.

8.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

8.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 9

Pagination, Filtering, Sorting & Field Selection

Executive Summary

Every list endpoint you will ever ship is a liability until you answer four questions precisely: *how does the client walk the collection, how does the client narrow it, how does the client order it, and how much of each resource does the client actually need?* These are the questions of pagination, filtering, sorting, and field selection, and they are deceptively deep. The naive answers — `OFFSET 100000`, a `WHERE` clause string-concatenated from query parameters, an `ORDER BY` on whatever column name the client sent, and a response body containing every column the ORM happened to load — are the four horsemen of the reporting-API apocalypse that this chapter’s case study reconstructs in painful detail.

The chapter’s spine is a single idea with three incarnations: **position is fragile, value is stable**. Offset pagination addresses rows by their *position* in an ordering; positions shift under concurrent writes, and reaching a deep position costs work proportional to the position itself. Cursor and keyset pagination address rows by their *value* at the boundary of the previous page; values are stable under inserts and deletes elsewhere in the ordering, and reaching a boundary value costs an index lookup — $O(\log n)$, not $O(n)$. We derive both costs formally, then demonstrate the concurrency anomaly with a deterministic, scripted writer/reader interleaving that reproduces duplicates and skips under offset pagination while keyset pagination remains exact.

The second half of the chapter treats the query surface around pagination: filtering and sorting. Here the governing principle is **never let the client speak SQL**. Clients speak a small, documented, allow-listed grammar of fields and operators; the server parses that grammar into a typed abstract syntax tree and compiles the tree into parameterized SQL, rejecting unknown

fields, unknown operators, and type mismatches with a clean 400. Sorting gets the same treatment, plus one iron rule: *every ordering ends with a unique tie-breaker*, because a non-deterministic `ORDER BY` silently corrupts every pagination strategy built on top of it. We close with field selection (`?fields=`) and expansion (`?expand=`), where the naive implementation reintroduces the $N+1$ query problem at the API boundary, and with the database craft that makes all of it fast: composite indexes aligned with keyset predicates, covering indexes, and how to read an `EXPLAIN` plan well enough to know whether you got the index right.

Everything is runnable offline. Five complete Python listings — a seeded dataset generator for the fictional Northwind Robotics parts catalog, a FastAPI service implementing all three pagination strategies with correct headers and HMAC-signed cursors, an allow-listed filter/sort compiler, a benchmark harness, and the concurrency-anomaly demo — form a laboratory you can execute end to end on a Windows workstation with PowerShell in under a minute.

Key Concept

A page is not a *place* in a list; it is a *predicate* on values. `OFFSET 150000` asks the database to compute and discard 150,000 rows and breaks when the list shifts. `WHERE (cost, id) > (4200, 918273)` asks the index to seek directly to a stable boundary and is immune to writes anywhere else in the ordering. Design list endpoints around boundary values, and design filter and sort surfaces around allow-lists compiled to bound parameters — never around client-supplied SQL fragments.

9.1 Learning Objectives & Prerequisites

9.1.1 Learning Objectives

After completing this chapter and its lab, you will be able to:

- LO-1.** Derive the asymptotic cost of offset pagination ($O(k)$ scanned rows at depth k) and of keyset pagination ($O(\log n)$ seek plus $O(p)$ page fetch), and predict latency growth qualitatively from first principles.
- LO-2.** Formalize the offset-pagination concurrency anomaly — duplicates and skips caused by inserts or deletes ahead of the reader's window — and explain why keyset pagination is immune to it.
- LO-3.** Implement all three pagination strategies (offset, transparent keyset, opaque signed cursor) in FastAPI with correct HTTP semantics,

including RFC 8288 **Link** headers with **rel** values **next**, **prev**, **first**, and **last**.

- LO-4.** Design cursor tokens that are opaque, tamper-evident (HMAC-SHA256), versioned, and expiring, and choose deliberately between **has_more** and **total_count** page metadata.
- LO-5.** Write a keyset predicate using tuple comparison — **WHERE (sort_col, id) > (?, ?)** — and align a composite index with it so the predicate stays sargable.
- LO-6.** Build an allow-listed filter grammar (operators **eq/ne/gt/gte/lt/lte/in/contains**) that parses into a typed AST and compiles to parameterized SQL, rejecting anything outside the grammar.
- LO-7.** Enforce deterministic ordering: multi-key sorts, explicit **NULL** ordering, and a unique tie-breaker column appended to every sort.
- LO-8.** Implement sparse fieldsets and resource expansion without the $N+1$ query problem, using DataLoader-style batching at the API boundary.
- LO-9.** Read an **EXPLAIN QUERY PLAN** output and judge whether a composite or covering index actually serves a keyset query.
- LO-10.** Measure and report offset-versus-keyset latency growth with a reproducible benchmark protocol, and interpret the resulting curve.

9.1.2 Prerequisites

This chapter assumes Chapters 0–6. Concretely:

- **Chapter 0:** the latency-measurement discipline (percentiles, reproducible benchmark protocols, synthetic Northwind Robotics data).
- **Chapters 1–2:** HTTP request/response mechanics, header semantics, and consuming APIs from Python clients.
- **Chapter 3:** REST resource modeling — collections, identifiers, and why representations are projections of state.
- **Chapter 4:** serialization contracts; the response shapes here are exactly the JSON contracts of that chapter under pagination.
- **Chapters 5–6:** client-side security hygiene and building FastAPI services with dependency-injected configuration, which this chapter’s reference service extends [Ram24; Col24].

- **Working knowledge of SQL:** `SELECT`, `WHERE`, `ORDER BY`, `LIMIT/OFFSET`, and what a B-tree index is. We teach *index alignment* for pagination, but not index internals from zero; *Designing Data-Intensive Applications* [Kle17] is the recommended companion.
- **Environment:** Python 3.11+ on Windows with PowerShell, a virtual environment, and the dependencies from this chapter’s appendix installed.

9.2 Deep Technical Mechanics & Architectural Concepts

9.2.1 The Four Dimensions of Collection Design

A collection endpoint such as `GET /parts` is a function from client intent to a projected, ordered, bounded subset of a relation. Decompose that function into four orthogonal dimensions and every design decision in this chapter has a home:

1. **Pagination** — which *window* of the ordered collection? (offset, cursor, keyset)
2. **Filtering** — which *subset* of the relation? (allow-listed predicates)
3. **Sorting** — which *order*? (multi-key, deterministic)
4. **Field selection** — which *projection* of each resource? (sparse fieldsets, expansion)

The dimensions are orthogonal in design but *not* in implementation: filtering changes the relation being paginated, sorting defines what “next” means, and field selection changes what each row costs to serialize. A change to any dimension between page one and page two of a client’s walk invalidates the walk — a consistency requirement we will formalize in Section 9.2.10.

9.2.2 Offset Pagination and Its True Cost

Offset pagination is the strategy everyone reaches for first:

```
1 SELECT id, sku, name, unit_cost_cents
2 FROM parts
3 ORDER BY unit_cost_cents, id
4 LIMIT 50 OFFSET 100000;
```


The semantics are simple: sort the relation, discard the first k rows, return the next p . The problem is the word *discard*. A B-tree index on (`unit_cost_cents`, `id`) lets the engine find the *start* of the ordering in $O(\log n)$, but it provides no random access to the k -th entry: index leaf entries do not carry their ordinal position. The engine must therefore **visit** entries $1 \dots k$ to count its way to entry $k+1$, and in the general case — when the query selects columns not present in the index — each visited entry costs a heap or clustered-index lookup to fetch the row, only for the row to be thrown away.

Definition 9.1 (Offset page-fetch cost). Let n be the number of rows matching the filter, k the offset, p the page size, c_s the per-row cost of scanning (and, where required, fetching) a row that will be discarded, and c_f the per-row cost of fetching and returning a row that will be emitted. Then the expected cost of one offset page fetch is

$$T_{\text{offset}}(k, p) = c_{\text{seek}} + k \cdot c_s + p \cdot c_f = O(\log n) + O(k) + O(p).$$

With k up to n , worst-case page cost is $O(n)$: *the deepest page costs as much as reading the entire table*.

Two consequences follow. First, latency *grows linearly with depth*, so the pagination API has a built-in denial-of-wallet vector: a crawler (or a buggy retry loop) walking to depth n consumes $\sum_k O(k) = O(n^2/p)$ total row visits. Second, aggregate user experience degrades exactly where your most valuable queries live — auditors, backfill jobs, and reporting exports are the clients that walk deep.

Example

Northwind Robotics ships 200,000 parts. A page of 50 at offset 0 costs one index seek and 50 row fetches — sub-millisecond on any hardware. The same page at offset 150,000 must first scan past 150,000 index entries (and, without a covering index, fetch 150,000 rows that are immediately discarded). On the reference hardware used for this chapter’s benchmark that page costs ~ 6 ms against ~ 0.07 ms for the equivalent keyset seek — a $\sim 90\times$ gap that widens linearly with depth. Section 9.2.5 gives the full measured table.

9.2.3 The Concurrency Anomaly, Formalized

Cost is only half the indictment. Offset pagination is also *incorrect* under concurrent writes, in a way that no isolation level fixes, because the anomaly spans multiple transactions (one per page request).

Model the collection as a sequence $S = \langle r_1, r_2, \dots, r_n \rangle$ ordered by the sort key. A reader requests pages of size p : page j is $S[jp \dots jp+p-1]$. Between two page requests, a writer commits. Define $\Delta_{\text{ins}}(j)$ as the number of rows inserted at positions $\leq jp$ (ahead of or inside the window boundary) and $\Delta_{\text{del}}(j)$ the number of such rows deleted.

Definition 9.2 (Offset window drift). After writer activity ahead of the boundary, the next offset page returns

$$S'[jp \dots jp+p-1] = S[jp-\Delta \dots jp-\Delta+p-1], \quad \Delta = \Delta_{\text{ins}}(j) - \Delta_{\text{del}}(j).$$

Every insertion ahead of the boundary ($\Delta > 0$) replays Δ rows the reader already saw (**duplicates**); every deletion ($\Delta < 0$) conceals $|\Delta|$ rows the reader never saw (**skips**).

The reader did nothing wrong; each individual page was a correct answer to the query as executed. The anomaly is a property of *positional addressing*: an offset is a coordinate in a coordinate system that the writer keeps renumbering. Section 9.3.5 proves this with a deterministic interleaving in which three concurrent front-of-list inserts cause the offset walker to emit rows 5, 9, and 13 twice and never emit rows 18, 19, and 20 at all.

Warning

Read-committed isolation — the default in PostgreSQL, and effectively what SQLite gives each statement — makes this *worse* to reason about, not better: every page request sees the latest committed state, so the window boundary moves underneath the reader between every pair of requests. Snapshot isolation across the whole walk would fix consistency but requires holding a transaction (and its snapshot) open across an unbounded number of HTTP requests — an operational anti-pattern of its own. Keyset pagination sidesteps the entire dilemma.

9.2.4 Cursor and Keyset (Seek) Pagination

Keyset pagination replaces the positional coordinate with a *value coordinate*: the sort-key values of the last row of the previous page. The next page is defined by a predicate, not a position:

```

1 SELECT id, sku, name, unit_cost_cents
2 FROM parts
3 WHERE (unit_cost_cents, id) > (4200, 918273)    -- the
   keyset
4 ORDER BY unit_cost_cents, id
5 LIMIT 50;
```

Tuple comparison semantics

The predicate $(a, b) > (x, y)$ is a *row-value* (tuple) comparison with lexicographic semantics:

$$(a, b) > (x, y) \equiv a > x \vee (a = x \wedge b > y).$$

This is exactly the condition “strictly after the boundary row in the (a, b) ordering,” including rows that tie on a but follow on b . The second disjunct is why the tie-breaker column is not optional: without `id`, all rows sharing the boundary value of `unit_cost_cents` would be either replayed or skipped. SQL standardizes row-value comparison and both SQLite (3.15+) and PostgreSQL implement it; where a dialect lacks it, the expanded disjunction above is the portable equivalent — write it exactly, because a hand-rolled `a >= x AND b > y` is subtly wrong (it drops the $a = x \wedge b > y$ survivors of the tie on one side and replays $a > x$ ties on the other).

Why the seek is $O(\log n)$

Given a composite index on `(unit_cost_cents, id)` whose column order matches the sort order, the keyset predicate is *sargable*: the engine descends the B-tree directly to the first entry satisfying the tuple predicate — $O(\log n)$ comparisons — then reads p consecutive leaf entries, already in order:

Definition 9.3 (Keyset page-fetch cost).

$$T_{\text{keyset}}(p) = c_{\text{seek}} + p \cdot c_f = O(\log n) + O(p),$$

independent of depth. Page 1 and page 10,000 cost the same.

The index-alignment requirement is precise: the index’s leading columns must equal the `ORDER BY` columns in the same order and direction (or all reversed, which a B-tree can scan backward). An index on `(id, unit_cost_cents)` is useless for this query; an index on `(unit_cost_cents)` alone serves the seek on the first column but forces an in-memory sort for the tie-break — check with `EXPLAIN` (Section 9.2.13).

Stable ordering is a correctness requirement

Both strategies presuppose a *total* order. If the sort key is not unique, rows that tie can be emitted in different orders by different executions of the same query (the optimizer is free to choose), and neither offsets nor keysets mean anything stable. The rule is absolute:

Key Concept

Every paginated ordering must end in a unique, immutable tie-breaker column — almost always the primary key. `ORDER BY unit_cost_cents` is a bug waiting for a tie; `ORDER BY unit_cost_cents, id` is a contract. The same rule makes keyset predicates total, makes offset walks at least deterministic, and makes multi-column sorts (Section 9.2.11) composable.

Cursor pagination: keyset with an envelope

Cursor pagination is keyset pagination with the boundary values packaged into an opaque token the server hands to the client: `next_cursor` in, `cursor` out. The underlying seek is identical; what changes is the encapsulation. The server can now hide internal column names, version the payload, sign it against tampering, and expire it — Section 9.2.7 develops the full design space. Stripe, Slack, and most large public APIs converge on this shape precisely because it decouples the client contract from the storage schema.

9.2.5 Measured Performance: Offset vs. Keyset

Theory predicts linear-versus-constant growth; measurement confirms it. Listing 9.4 benchmarks single-page fetches (page size 50, median of 15 repetitions) at increasing depths over a 200,000-row Northwind Robotics parts table in SQLite, backed by the composite index `idx_parts_cost_id`. Results from one run on the reference workstation (Windows, Python 3.13, warm page cache):

Read the curve as the two cost formulas made visible: T_{offset} is the line $k \cdot c_s$, and T_{keyset} is the constant $c_{\text{seek}} + p \cdot c_f$. The slight super-linearity at depth 150,000 is cache behavior — discarded rows still flow through the buffer pool — which makes the production case *worse* than the asymptotic one, since production offset queries rarely enjoy a covering index.

9.2.6 Choosing a Strategy: A Decision Tree

The honest summary: offset is acceptable when the collection is small, walks are shallow, and random access is a genuine product requirement — an internal admin grid is the canonical case. Everything user-facing, machine-consumed, exported, or infinite-scrolled should be keyset, and the cursor envelope should be opaque the moment the API is public [GH21].

Depth	Offset (ms)	Keyset (ms)	Speedup	Observation
0	0.060	0.062	1.0x	Identical: offset 0 <i>is</i> a seek.
1,000	0.074	0.065	1.1x	Scan cost still below timer noise.
10,000	0.214	0.069	3.1x	Linear regime clearly visible.
50,000	0.937	0.079	11.9x	Offset now dominated by discards.
100,000	1.853	0.076	24.4x	Keyset flat within noise.
150,000	5.908	0.065	91.5x	Offset super-linear: cache pressure.

Table 9.1: Measured single-page latency by strategy and depth (median of 15; illustrative, machine-dependent; page size 50, $n = 200,000$ rows, SQLite with composite index).

9.2.7 Cursor Token Design

Opaque versus transparent

A *transparent* cursor exposes the boundary values as ordinary query parameters (`?after_cost=4200&after_id=918273`). Clients can construct them, debug them, and bookmark them — and they couple your public contract to your physical schema forever. An *opaque* cursor wraps the same values in an encoded, signed token. The client treats it as an unanalyzable string; the server retains freedom to rename columns, change the sort, or rotate the encoding. For public APIs, opacity is the default recommendation; for internal APIs where debuggability dominates, transparent keysets are defensible.

A reference token format

The token used by the reference service (Listing 9.2) is

$$\underbrace{\text{base64url}(\text{JSON})}_{\text{payload}} . \underbrace{\text{HMAC-SHA256}_{32 \text{ hex chars}}(\text{payload})}_{\text{truncated tag}}$$

with payload

```
1 {"v":1,"exp":1735686000,"cost":4200,"id":918273}
```

- **Version (v).** Cursors outlive deploys. A version field lets the server reject (or migrate) tokens minted by a previous contract instead of misinterpreting them as corrupt.

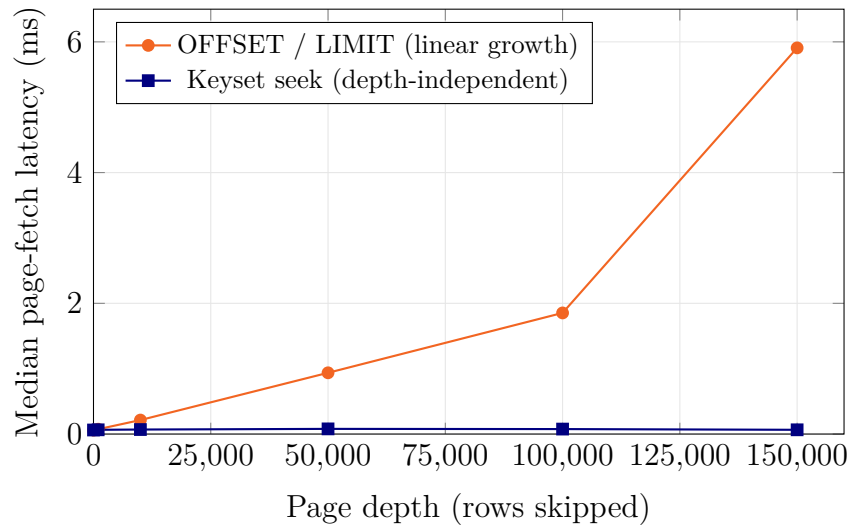


Figure 9.1: Offset versus keyset page-fetch latency at increasing depth. Measured values from Listing 9.4; illustrative and machine-dependent — the *shape*, not the absolute numbers, is the finding.

- **Expiry (exp).** A cursor is a capability: it encodes a position in a potentially sensitive ordering. Expiry bounds replay windows, bounds how long a leaked token is useful, and gives you a principled answer to “how long may a client pause mid-walk?”
- **Signature.** HMAC-SHA256 over the payload, truncated to 128 bits, verified with a constant-time comparison (`hmac.compare_digest`). Verification runs *before* any payload field is trusted. Tamper handling is uniform: any malformed, forged, expired, or wrong-version token receives the same 400 problem-details response, revealing nothing about which check failed beyond what the legitimate client needs to recover (restart the walk).
- **Key management.** The signing key is server-side configuration loaded from the environment (a secrets manager in production), never committed, never logged. The demo key in Listing 9.2 is explicitly marked demo-only.

Warning

Unsigned or merely *encoded* cursors (plain base64 JSON) are guessable and forgeable: a client can mint a cursor pointing anywhere in your ordering, bypassing any access control you keyed off earlier pages, and can probe field values by binary-searching the boundary. Signing

is not hardening; it is the minimum bar. Similarly, never put user-input-derived SQL fragments in the token — the payload is decoded into a typed structure and re-bound as parameters, exactly like any other external input.

9.2.8 Hypermedia Controls: RFC 8288 Link Headers

Page navigation is a hypermedia concern, and HTTP already has a standard surface for it: the Link header [Not17]. Each link is a target URI plus a relation type:

```

1 HTTP/1.1 200 OK
2 Link: </parts/offset?limit=50&offset=100>; rel="next",
3       </parts/offset?limit=50&offset=0>; rel="first",
4       </parts/offset?limit=50&offset=50>; rel="prev"
5 Content-Type: application/json

```

The registered relations that matter here are **next**, **prev**, **first**, and **last**. Emitting them buys three things: clients can be *driven by the hypermedia* instead of constructing URLs (a client that follows **rel="next"** never needs to know whether you paginate by offset, keyset, or opaque cursor — strategy migration becomes a server-side decision); intermediaries and crawlers understand the semantics; and the absence of **rel="next"** is an unambiguous, parseable end-of-collection signal. Body-level fields (**next_cursor**) and Link headers coexist peacefully — the reference service emits both — but if you implement only one, prefer the header: it is the standard, content-negotiation-friendly surface [Fie22a].

Two construction rules matter. First, links must be *complete and self-contained*: the **next** URI reproduces every parameter that defined the current page (limit, filters, sort) so the walk is resumable without client-side state. Second, **rel="last"** requires knowing the total, which ties it to the cost discussion that follows — many APIs deliberately omit it.

9.2.9 Page Metadata: **has_more** versus **total_count**

Clients want to know two different things: “is there another page?” and “how many rows are there in total?” The first is cheap; the second is not.

has_more costs one row: fetch **LIMIT** $p + 1$, emit p rows, and set **has_more** = **true** if a $(p+1)$ -th row exists. The cursor lifecycle in Figure 9.3 builds this in.

total_count costs a **COUNT(*)** over the full filtered relation. At small scale that is a fast index scan; at scale it is an $O(n)$ operation *per page request*, executed again and again by every client on every page — the

database recomputes a number almost no client acts upon, while holding buffer-pool pages that latency-sensitive queries need. On a 200,000-row SQLite table the count alone costs multiples of a page fetch; on a 200-million-row PostgreSQL table under concurrent load it is the query your on-call learns to hate (Section 9.5.2).

When product requirements genuinely need a total, choose deliberately:

- **Opt-in counting** — `?include_total=true`, as in the reference service — so casual clients never pay for it.
- **Count once, not per page** — return the total only on page one of a walk, or in a separate `/parts/stats` resource.
- **Estimate** — PostgreSQL’s planner keeps `reltuples` estimates; `EXPLAIN` row estimates, or a periodically refreshed `COUNT` materialized into a stats table, give “~200,000 results” good enough for UI badges.
- **Exact counts only over small filtered sets** — enforce a threshold: compute the exact count only when a capped probe (`SELECT ... LIMIT 10001`) returns few enough rows.

9.2.10 Filtering: A Safe Query-Parameter Grammar

Filtering is where list endpoints most often become injection vehicles. The failure pattern is always the same shape: the server treats query parameters as *SQL text* rather than as *values*:

```
1 # NEVER do this. One crafted ?min_cost=1 OR 1=1-- later
  ...
2 sql = f"SELECT * FROM parts WHERE unit_cost_cents >= {min_cost}"
```

The defense has three layers, all present in Listing 9.3:

Layer 1: an explicit, allow-listed grammar

Define the public surface as a mini-grammar, not as SQL:

```
1 filter    := clause (";" clause)*
2 clause    := field ":" op ":" value
3 field     := "sku" | "name" | "category" | "cost" | "weight" | "active"
4 op        := "eq" | "ne" | "gt" | "gte" | "lt" | "lte" | "in" | "contains"
5 value     := scalar | scalar ("," scalar)* ;
             comma list for "in"
```



```

6 | sort      := ["-"] field ("," ["-"] field)*      ; "-"
   | = descending

```

Note what the grammar achieves by construction. The field set is a *public naming layer*: clients say `cost`; the column is `unit_cost_cents`. Internal columns that must never be filtered or leaked (password hashes, tenant keys, margin data) simply do not exist in the grammar. The operator set is closed and semantics are documented: `in` takes a comma list, `contains` is case-insensitive substring on text fields only, ordering operators reject text fields, and `contains` escapes LIKE wildcards in user input so `%` in a search string is data, not a pattern.

Layer 2: parse to a typed AST, then validate

Each clause becomes a typed node — `Comparison`, `InList`, `Contains` — with the value already coerced to the field’s declared Python type. Anything unparseable, unknown, or mistyped raises `FilterError`, which the HTTP layer maps to a 400 problem-details response [Fie22a]. Validation happens on the AST, where context is rich, rather than on the SQL string, where context is gone.

Layer 3: compile the AST to parameterized SQL

Compilation resolves each public field name to its column via the allow-list mapping — *never* via the input string — emits a fixed skeleton of SQL with `?` placeholders, and returns the values as a bound parameter list. User input never touches SQL text; an `OR '1'='1` probe becomes a string literal compared against a column and matches nothing. This is the AST-based translation pattern: the grammar bounds what can be expressed, the AST bounds what can be validated, and parameter binding bounds what can be executed.

Key Concept

Filter safety is an architecture, not a regex. **Allow-list the grammar** (fields and operators), **parse to a typed AST** (validate with full context, reject with 400s), and **compile with bound parameters** (user values never become SQL text). Any one layer alone has a bypass; the composition is robust.

Filter consistency with pagination

A page is only meaningful relative to the filter and sort that defined the walk. Two rules follow. First, the filter and sort parameters are part of the *cursor*: either they travel verbatim in every `Link` target and the server

rejects a mid-walk change, or their hash is embedded in the signed cursor payload so a mismatched continuation fails fast instead of silently returning pages from a different query. Second, keyset predicates and filters compose by conjunction — `WHERE <filters> AND (unit_cost_cents, id) > (?, ?)` — which keeps the composite index usable when the filter’s leading column aligns with the index (the `idx_parts_cat_cost_id` index exists precisely so that `category = ? AND (cost, id) > (?, ?)` seeks instead of scans). Filtering by a column you did not index and then keyset-paginating the result reintroduces the scan you worked to eliminate; check the plan.

9.2.11 Sorting: Multi-Key Orderings Done Safely

Sorting composes with everything above, so its rules are short and strict:

- **Allow-list sortable fields and directions**, exactly as for filters. `?sort=-cost,sku` parses to `ORDER BY unit_cost_cents DESC, sku ASC`; an unknown field is a 400, and direction accepts only the two documented tokens. Direction must be a parsed token, not interpolated text — `ORDER BY {user_input}` is SQL injection wearing a trench coat.
- **Append the unique tie-breaker automatically.** The compiler in Listing 9.3 always ends the key list with `id ASC`. A client asking for `-cost` receives `ORDER BY unit_cost_cents DESC, id ASC`: deterministic, total, and keyset-compatible (the keyset tuple then spans *all* sort keys: `(cost, sku, id) > (?, ?, ?)`).
- **Pin NULL ordering explicitly.** SQL leaves NULL placement in `ORDER BY` implementation-defined (SQLite: NULLs first on `ASC`; PostgreSQL: NULLs last). If any sortable column is nullable, state it: `ORDER BY discontinued_at ASC NULLS LAST, id ASC` — and make the keyset predicate NULL-aware (`(col IS NULL)` disjuncts), or exclude NULL rows from censored endpoints. Ambiguous NULL handling is a silent page-boundary corruptor.
- **Descending keysets invert the predicate.** Walking `ORDER BY cost DESC, id DESC` pages forward with `(cost, id) < (?, ?)`. Mixed directions (`cost DESC, id ASC`) are legal but the tuple predicate no longer maps to one comparison — expand to the disjunctive form. Prefer uniform direction for cursor endpoints.
- **Multi-column sorts need multi-column indexes.** The index column order must match the `ORDER BY` order left-to-right; an index on `(cost, sku, id)` cannot serve `ORDER BY sku, cost, id`.

9.2.12 Field Selection and Expansion

Sparse fieldsets

?fields=sku,name,cost lets the client declare the projection it actually uses. The win is threefold: smaller payloads (egress cost, mobile bandwidth), less serialization CPU, and — the quiet giant — *covering-index eligibility*: `SELECT sku, unit_cost_cents` with a restricted projection can be answered entirely from an index without touching the table heap. The safety rules mirror filtering: allow-list selectable fields, resolve through the public-name mapping, default to a documented field set when the parameter is absent, and always include `id` (clients need identity even when they forget to ask).

Expansion and the N+1 boundary problem

?expand=category_detail,supplier asks the server to embed related resources inline. The naive implementation issues one query for the page of 50 parts and then *one supplier query per part* — 51 round trips for one HTTP request, the classic $N+1$ transplanted from ORM-land to the API boundary. The fix is DataLoader-style batching applied to REST: collect the distinct foreign keys of the page, issue *one* `WHERE id IN (...)` query per expanded relation, and stitch results in memory:

```

1 def expand_suppliers(conn: sqlite3.Connection,
2                       parts: list[dict]) -> dict[int,
3                                     dict]:
4     """One batched query for all suppliers referenced
5       by this page."""
6     keys = sorted({p["supplier_id"] for p in parts})
7     placeholders = ", ".join("?" for _ in keys)
8     rows = conn.execute(
9         f"SELECT id, name, lead_time_days FROM suppliers"
10        f"WHERE id IN ({placeholders})", keys,
11    ).fetchall()
12    return {r["id"]: dict(r) for r in rows}

```

Batching converts $1 + N \cdot k$ queries into $1 + k$ for k expanded relations. Guard the expansion surface with the same allow-list discipline (expandable relations are enumerated; each has a bounded batch size), cap expansion depth (no `expand=a.b.c` transitive graphs in a REST endpoint — that way lies GraphQL, Chapter 12), and remember the trade-off: expansion couples cacheability and payload size to the *largest* consumer's needs. Sparse fieldsets shrink responses; expansion grows them. Offer both, and let `Vary` and your cache keys reflect the parameters [Fie22a].

9.2.13 Database Interaction: Indexes, EXPLAIN, Covering

All of the above stands or falls on physical design:

- **Composite index alignment.** For the ordering (`unit_cost_cents`, `id`), the index `idx_parts_cost_id` ON `parts(unit_cost_cents, id)` makes both the seek and the ordered page read index-native. The column order in the index must match the sort order; equality-prefixed filters extend the rule: `category = ? AND (cost, id) > (?,?)` wants (`category`, `unit_cost_cents`, `id`) — equality columns first, range/sort columns after.
- **Reading EXPLAIN.** `EXPLAIN QUERY PLAN` on the keyset query reports `SEARCH parts USING INDEX idx_parts_cost_id (unit_cost_cents>?...)` — a `SEARCH`, i.e. a B-tree seek. The offset variant reports the same index but *scans* through *k* entries; at scale a planner may even choose `SCAN parts` plus an in-memory sort, which is the “wrong index” smell. Words to fear in any plan for a paginated endpoint: `SCAN` (full), `USE TEMP B-TREE FOR ORDER BY` (sort not served by an index), and a rows-examined estimate that grows with your offset.
- **Covering indexes.** If the page projection is (`id`, `sku`, `unit_cost_cents`), an index on (`unit_cost_cents`, `id`) *covering* `sku` (via `INCLUDE` in PostgreSQL, or a third index column in SQLite) answers the whole page from the index — `USING COVERING INDEX` in the plan — eliminating *p* heap fetches per page and, for offset, *k* wasted heap fetches per deep page.
- **Write amplification is the bill.** Every index is paid for on every insert and update. The two indexes in the reference schema are justified by two real query shapes; index-per-hope is how write latency dies.

9.3 Production-Ready Code Implementation

Five complete, runnable listings form the chapter’s laboratory. All are Python 3.11+, fully typed, PEP 8, and offline-first: the only datastore is local SQLite (stdlib `sqlite3`); the only third-party packages are FastAPI and its test client (which needs `httpx`). Everything runs on Windows PowerShell in seconds. Setup:

```
1 cd "C:\Training\Training Areas\Data jobs learning\APIs  
  - Usage and development\code"
```

```

2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 pip install fastapi pydantic httpx pytest
5 python gen_dataset.py --rows 200000 --db parts.db

```

9.3.1 Listing 1: Deterministic Dataset Generator

The generator (Listing 9.1) builds the Northwind Robotics parts catalog: 200,000 synthetic rows with a skewed cost distribution (many cheap fasteners, few expensive actuators), deterministic per-row content derived from a seeded PRNG, and the two composite indexes that Section 9.2.13 justified: `idx_parts_cost_id` for the unfiltered keyset walk and `idx_parts_cat_cost_id` for category-filtered walks. **ANALYZE** gives the query planner real statistics. Generation is pure CPU and one bulk insert — about 3.5 seconds for 200,000 rows on the reference workstation; scale to a million rows by passing `--rows 1000000` if you want the benchmark’s linear region to hurt properly.

Listing 9.1: Deterministic synthetic dataset generator (`gen_dataset.py`).

```

1  """Deterministic synthetic dataset generator for
   Chapter 8.
2
3  Generates the fictional *Northwind Robotics* parts
   catalog into a local
4  SQLite database. Offline-first: no network, no third-
   party packages.
5
6  Usage (PowerShell):
7  python gen_dataset.py --rows 200000 --db parts.db
8  """
9
10 from __future__ import annotations
11
12 import argparse
13 import random
14 import sqlite3
15 import time
16 from pathlib import Path
17
18 SCHEMA = """
19 CREATE TABLE IF NOT EXISTS parts (
20     id INTEGER PRIMARY KEY,
21     sku TEXT NOT NULL UNIQUE,
22     name TEXT NOT NULL,
23     category TEXT NOT NULL,
24     unit_cost_cents INTEGER NOT NULL,

```

```

25     weight_grams INTEGER NOT NULL,
26     active INTEGER NOT NULL,
27     updated_at TEXT NOT NULL
28 );
29 -- Composite index aligned with the keyset ordering (
    unit_cost_cents, id).
30 CREATE INDEX IF NOT EXISTS idx_parts_cost_id
31 ON parts (unit_cost_cents, id);
32 -- Composite index aligned with (category,
    unit_cost_cents, id) keysets.
33 CREATE INDEX IF NOT EXISTS idx_parts_cat_cost_id
34 ON parts (category, unit_cost_cents, id);
35 ""
36
37 CATEGORIES: list[str] = [
38     "actuator", "sensor", "chassis", "drivetrain", "
    power",
39     "controller", "fastener", "optic",
40 ]
41
42 NAME_PARTS: list[str] = [
43     "servo", "gyro", "lidar", "torque", "chassis", "
    rail", "bearing",
44     "gearbox", "battery", "inverter", "encoder", "
    bracket", "harness",
45 ]
46
47 NAME_SUFFIXES: list[str] = [
48     "Alpha", "Beta", "Prime", "XL", "Micro", "HD", "Mk2
    ", "Pro",
49 ]
50
51
52 def generate_rows(count: int, seed: int) -> list[tuple
53 ]:
54     """Build 'count' deterministic synthetic part
    rows."""
55     rng = random.Random(seed)
56     rows: list[tuple] = []
57     for i in range(1, count + 1):
58         name = f"{rng.choice(NAME_PARTS)}-{rng.choice(
    NAME_SUFFIXES)}"
59         category = rng.choice(CATEGORIES)
60         # Skewed cost distribution: many cheap parts,
    few expensive ones.
        cost = int(rng.expovariate(1 / 4000.0)) + 50

```

```

61     weight = rng.randint(2, 25_000)
62     active = 1 if rng.random() < 0.93 else 0
63     # Deterministic pseudo-timestamp derived from
        the row number only.
64     minute = (i * 7) % (60 * 24 * 365)
65     updated = f"2025-01-01T00:00:00Z+{minute:07d}m"
66     rows.append((f"NWR-{i:07d}", name, category,
        cost, weight, active, updated))
67     return rows
68
69
70 def build_database(db_path: Path, rows: int, seed: int)
    -> None:
71     """Create (or replace) the SQLite database with '
        rows' parts."""
72     if db_path.exists():
73         db_path.unlink()
74     conn = sqlite3.connect(db_path)
75     try:
76         conn.executescript(SCHEMA)
77         started = time.perf_counter()
78         data = generate_rows(rows, seed)
79         with conn:
80             conn.executemany(
81                 "INSERT INTO parts"
82                 "(sku, name, category, unit_cost_cents,
83                  weight_grams,
84                  active, updated_at) VALUES (?, ?, ?,
85                  ?, ?, ?, ?)",
86                 data,
87             )
88             conn.execute("ANALYZE")
89             conn.commit()
90             elapsed = time.perf_counter() - started
91             count = conn.execute("SELECT COUNT(*) FROM
                parts").fetchone()[0]
92             print(f"generated {count:,} rows into {db_path}
                in {elapsed:.2f}s")
93         finally:
94             conn.close()
95
96 def main() -> None:
97     parser = argparse.ArgumentParser(description=
        __doc__)
98     parser.add_argument("--rows", type=int, default=200

```

```

    _000)
98     parser.add_argument("--db", type=Path, default=Path
        ("parts.db"))
99     parser.add_argument("--seed", type=int, default=42)
100    args = parser.parse_args()
101    if args.rows < 1:
102        raise SystemExit("--rows must be >= 1")
103    build_database(args.db, args.rows, args.seed)
104
105
106 if __name__ == "__main__":
107     main()
```

9.3.2 Listing 2: Three Pagination Strategies in One FastAPI Service

The service (Listing 9.2) implements the full design space of Section 9.2 behind three endpoints that share one ordering — `(unit_cost_cents, id)` — so their results can be compared row-for-row:

- GET `/parts/offset` — LIMIT/OFFSET with RFC 8288 Link headers (`first/next/prev`, plus `last` only when the client opts into the count via `include_total=true`).
- GET `/parts/keyset` — transparent keyset pagination: `after_cost/after_id` parameters, the tuple predicate `WHERE (unit_cost_cents, id) > (?, ?)`, and the LIMIT `p+1` trick driving `has_more` and the `rel="next"` link.
- GET `/parts/cursor` — opaque cursors: versioned JSON payload with expiry, base64url-encoded, HMAC-SHA256-signed and verified with `hmac.compare_digest` *before* any payload field is read. Any malformed, forged, wrong-version, or expired token receives the same 400 problem-details body.

Limits are clamped server-side (`MAX_LIMIT = 200`): an unbounded limit is a resource-exhaustion invitation. The `_self_check` entry point pages 250 rows through all three strategies offline via `TestClient`, asserts byte-identical id sequences, and confirms a forged cursor is rejected — run it with `python pagination_service.py`.

Listing 9.2: FastAPI service implementing offset, keyset, and HMAC-signed cursor pagination (`pagination_service.py`).

```

1  """Chapter_8_reference_service: three pagination
   strategies over one table.
2
3  FastAPI application exposing the Northwind Robotics
   parts catalog with:
4
5  * GET /parts/offset -- classic LIMIT/OFFSET with
   RFC_8288 Link headers.
6  * GET /parts/keyset -- keyset (seek) pagination
   with transparent params.
7  * GET /parts/cursor -- opaque, HMAC-signed,
   versioned, expiring cursors.
8
9  All three order by '(unit_cost_cents, id)' which is
   backed by the
10 composite index 'idx_parts_cost_id' created by '
   gen_dataset.py'.
11
12 Run the self-check (offline, uses FastAPI TestClient):
13
14 python gen_dataset.py --rows 50000 --db parts.db
15 python pagination_service.py
16 """
17
18 from __future__ import annotations
19
20 import base64
21 import hashlib
22 import hmac
23 import json
24 import os
25 import sqlite3
26 import time
27 from pathlib import Path
28 from typing import Any
29
30 from fastapi import FastAPI, Query, Request, Response
31 from fastapi.responses import JSONResponse
32
33 DB_PATH = Path(os.environ.get("PARTS_DB", "parts.db"))
34 # Demo-only key. In production load from a secrets
   manager, never hardcode.
35 CURSOR_SECRET = os.environ.get("CURSOR_SECRET", "
   northwind-robotics-demo-key")
36 CURSOR_VERSION = 1
37 CURSOR_TTL_SECONDS = 3600

```

```

38 MAX_LIMIT = 200
39 DEFAULT_LIMIT = 50
40
41 SELECT_COLS = "id, sku, name, category, unit_cost_cents
42               , weight_grams, active"
43 ORDER_BY = "unit_cost_cents, id"
44
45 app = FastAPI(title="Northwind_Robotics_Parts_API",
46               version="1.0.0")
47
48 def get_db() -> sqlite3.Connection:
49     conn = sqlite3.connect(DB_PATH)
50     conn.row_factory = sqlite3.Row
51     return conn
52
53 def row_to_part(row: sqlite3.Row) -> dict[str, Any]:
54     return {
55         "id": row["id"],
56         "sku": row["sku"],
57         "name": row["name"],
58         "category": row["category"],
59         "unit_cost_cents": row["unit_cost_cents"],
60         "weight_grams": row["weight_grams"],
61         "active": bool(row["active"]),
62     }
63
64
65 def clamp_limit(limit: int) -> int:
66     if limit < 1:
67         return DEFAULT_LIMIT
68     return min(limit, MAX_LIMIT)
69
70
71 def link_header(base_url: str, links: dict[str, str])
72     -> str:
73     """Render an RFC 8288 Link header value from rel ->
74         URL mappings."""
75     return ", ".join(f'<{base_url}{target}>; rel="{rel}"'
76                     for rel, target in links.items())
77
78 #

```

```

77 # Strategy 1: offset pagination
78 #
79 -----
80 @app.get("/parts/offset")
81 def list_parts_offset(
82     request: Request,
83     response: Response,
84     limit: int = Query(DEFAULT_LIMIT, ge=1, le=
85         MAX_LIMIT),
86     offset: int = Query(0, ge=0),
87     include_total: bool = Query(False),
88 ) -> dict[str, Any]:
89     limit = clamp_limit(limit)
90     conn = get_db()
91     try:
92         rows = conn.execute(
93             f"SELECT_{SELECT_COLS}_FROM_parts"
94             f"ORDER_BY_{ORDER_BY}_LIMIT_{limit}_OFFSET_{offset}",
95             (limit, offset),
96         ).fetchall()
97         body: dict[str, Any] = {"items": [row_to_part(r)
98             for r in rows]}
99         if include_total:
100             # Deliberately opt-in: COUNT(*) is a full
101             # index scan at scale.
102             total = conn.execute("SELECT_COUNT(*)_FROM_parts").fetchone()[0]
103             body["total_count"] = total
104
105         base = str(request.url.path)
106         links: dict[str, str] = {"first": f"{base}?
107             limit={limit}&offset=0"}
108         if rows:
109             links["next"] = f"{base}?limit={limit}&
110                 offset={offset+limit}"
111         if offset > 0:
112             prev_offset = max(0, offset - limit)
113             links["prev"] = f"{base}?limit={limit}&
114                 offset={prev_offset}"
115         if "total_count" in body:
116             last_offset = max(0, ((body["total_count"]
117                 - 1) // limit) * limit)
118             links["last"] = f"{base}?limit={limit}&
119                 offset={last_offset}"
120         response.headers["Link"] = link_header(base,

```

```

112         links)
113         return body
114     finally:
115         conn.close()
116
117 #
118 -----
119 # Strategy 2: keyset (seek) pagination -- transparent
120 # parameters
121 -----
122
123 @app.get("/parts/keyset")
124 def list_parts_keyset(
125     request: Request,
126     response: Response,
127     limit: int = Query(DEFAULT_LIMIT, ge=1, le=
128         MAX_LIMIT),
129     after_cost: int | None = Query(None, ge=0),
130     after_id: int | None = Query(None, ge=1),
131 ) -> dict[str, Any]:
132     limit = clamp_limit(limit)
133     conn = get_db()
134     try:
135         if after_cost is None or after_id is None:
136             rows = conn.execute(
137                 f"SELECT_{SELECT_COLS}_FROM_parts_ORDER
138                 _BY_{ORDER_BY}_LIMIT_",
139                 (limit + 1,), # fetch one extra row to
140                             compute has_more
141             ).fetchall()
142         else:
143             # Tuple comparison keeps the predicate
144             # sargable on the
145             # composite index (unit_cost_cents, id).
146             rows = conn.execute(
147                 f"SELECT_{SELECT_COLS}_FROM_parts_"
148                 f"WHERE_(unit_cost_cents, id)>_(?, ?)"
149                 "
150                 f"ORDER_BY_{ORDER_BY}_LIMIT_",
151                 (after_cost, after_id, limit + 1),
152             ).fetchall()
153
154     has_more = len(rows) > limit

```

```

147     page = rows[:limit]
148     body: dict[str, Any] = {
149         "items": [row_to_part(r) for r in page],
150         "has_more": has_more,
151     }
152     if has_more and page:
153         last = page[-1]
154         base = str(request.url.path)
155         nxt = (
156             f"{base}?limit={limit}"
157             f"&after_cost={last['unit_cost_cents']}"
158             f"&after_id={last['id']}"
159         )
160         response.headers["Link"] = link_header(base, {"next": nxt})
161     return body
162 finally:
163     conn.close()
164
165 #
166 -----
167 # Strategy 3: opaque signed cursors (HMAC-SHA256,
168 # versioned, expiring)
169 #
170 -----
171
172 def _b64url_encode(raw: bytes) -> str:
173     return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")
174
175
176 def _b64url_decode(text: str) -> bytes:
177     padding = "=" * (-len(text) % 4)
178     return base64.urlsafe_b64decode(text + padding)
179
180
181 def encode_cursor(last_cost: int, last_id: int) -> str:
182     payload = {
183         "v": CURSOR_VERSION,
184         "exp": int(time.time()) + CURSOR_TTL_SECONDS,
185         "cost": last_cost,
186         "id": last_id,
187     }
188     raw = json.dumps(payload, separators=(",", ":"),

```

```

        sort_keys=True).encode()
185 body = _b64url_encode(raw)
186 tag = hmac.new(CURSOR_SECRET.encode(), body.encode
        (), hashlib.sha256).hexdigest()
187 return f"{body}.{tag[:32]}"
188
189
190 class CursorError(ValueError):
191     """Raised for any malformed, forged, or expired
        cursor."""
192
193
194 def decode_cursor(token: str) -> tuple[int, int]:
195     try:
196         body, tag = token.rsplit(".", 1)
197     except ValueError as exc:
198         raise CursorError("cursor is malformed") from
            exc
199     expected = hmac.new(
200         CURSOR_SECRET.encode(), body.encode(), hashlib.
            sha256
201     ).hexdigest()[:32]
202     if not hmac.compare_digest(expected, tag):
203         raise CursorError("cursor signature mismatch")
204     try:
205         payload = json.loads(_b64url_decode(body))
206     except (ValueError, UnicodeDecodeError) as exc:
207         raise CursorError("cursor payload is not valid
            JSON") from exc
208     if payload.get("v") != CURSOR_VERSION:
209         raise CursorError("unsupported cursor version")
210     if int(payload.get("exp", 0)) < int(time.time()):
211         raise CursorError("cursor has expired")
212     return int(payload["cost"]), int(payload["id"])
213
214
215 @app.get("/parts/cursor")
216 def list_parts_cursor(
217     request: Request,
218     response: Response,
219     limit: int = Query(DEFAULT_LIMIT, ge=1, le=
        MAX_LIMIT),
220     cursor: str | None = Query(None),
221 ) -> dict[str, Any]:
222     limit = clamp_limit(limit)
223     position: tuple[int, int] | None = None

```

```

224     if cursor is not None:
225         try:
226             position = decode_cursor(cursor)
227         except CursorError as exc:
228             return JsonResponse(
229                 status_code=400,
230                 content={
231                     "type": "about:blank",
232                     "title": "Invalid cursor",
233                     "status": 400,
234                     "detail": str(exc),
235                 },
236             )
237
238     conn = get_db()
239     try:
240         if position is None:
241             rows = conn.execute(
242                 f"SELECT_{SELECT_COLS}_FROM_parts_ORDER
243                     _BY_{ORDER_BY}_LIMIT_",
244                 (limit + 1, ),
245             ).fetchall()
246         else:
247             rows = conn.execute(
248                 f"SELECT_{SELECT_COLS}_FROM_parts_"
249                 f"WHERE_(unit_cost_cents, id)_(?, ?)"
250                 f"ORDER_BY_{ORDER_BY}_LIMIT_",
251                 (position[0], position[1], limit + 1),
252             ).fetchall()
253         has_more = len(rows) > limit
254         page = rows[:limit]
255         body: dict[str, Any] = {
256             "items": [row_to_part(r) for r in page],
257             "has_more": has_more,
258         }
259         if has_more and page:
260             last = page[-1]
261             token = encode_cursor(last["unit_cost_cents"], last["id"])
262             body["next_cursor"] = token
263             base = str(request.url.path)
264             response.headers["Link"] = link_header(
265                 base, {"next": f"{base}?limit={limit}&cursor={token}"}

```

```

266         return body
267     finally:
268         conn.close()
269
270
271 def _self_check() -> None:
272     """Offline verification: page through all three
273     strategies and compare."""
274     from fastapi.testclient import TestClient
275
276     if not DB_PATH.exists():
277         raise SystemExit(f"database {DB_PATH} missing;
278         run gen_dataset.py first")
279     client = TestClient(app)
280
281     expected = get_db().execute(
282         "SELECT id FROM parts ORDER BY unit_cost_cents,
283         id LIMIT 250"
284     ).fetchall()
285     expected_ids = [r[0] for r in expected]
286
287     # Offset: walk five pages of 50.
288     seen: list[int] = []
289     offset = 0
290     for _ in range(5):
291         res = client.get(f"/parts/offset?limit=50&
292         offset={offset}")
293         assert res.status_code == 200, res.text
294         assert "next" in res.headers["Link"]
295         seen.extend(item["id"] for item in res.json()["
296         items"])
297         offset += 50
298     assert seen == expected_ids, "offset walk diverged"
299
300     # Keyset: walk with transparent after_* parameters.
301     seen = []
302     params: dict[str, Any] = {"limit": 50}
303     for _ in range(5):
304         res = client.get("/parts/keyset", params=params
305         )
306         assert res.status_code == 200, res.text
307         payload = res.json()
308         seen.extend(item["id"] for item in payload["
309         items"])
310         last = payload["items"][-1]
311         params = {

```



```

305         "limit": 50,
306         "after_cost": last["unit_cost_cents"],
307         "after_id": last["id"],
308     }
309     assert seen == expected_ids, "keyset␣walk␣diverged"
310
311     # Cursor: walk with opaque signed tokens; then
312     # tamper and expect 400.
313     seen = []
314     token: str | None = None
315     for _ in range(5):
316         params = {"limit": 50} if token is None else {"limit": 50, "cursor": token}
317         res = client.get("/parts/cursor", params=params)
318         assert res.status_code == 200, res.text
319         payload = res.json()
320         seen.extend(item["id"] for item in payload["items"])
321         token = payload["next_cursor"]
322         assert seen == expected_ids, "cursor␣walk␣diverged"
323
324         forged = token[:-2] + ("00" if not token.endswith("00") else "11")
325         res = client.get("/parts/cursor", params={"limit": 50, "cursor": forged})
326         assert res.status_code == 400, res.text
327
328         res = client.get("/parts/offset?limit=50&include_total=true")
329         assert res.json()["total_count"] > 0
330         assert "last" in res.headers["Link"]
331         print("self-check␣OK:␣offset,␣keyset,␣and␣cursor␣strategies␣agree;"
332               "␣forged␣cursor␣rejected␣with␣400")
333
334 if __name__ == "__main__":
335     _self_check()

```

9.3.3 Listing 3: Allow-Listed Filter/Sort Compiler

The compiler (Listing 9.3) is the three-layer defense of Section 9.2.10 as working code. The public grammar (`field:op:value` clauses joined by `;`) parses into frozen dataclass AST nodes; values are coerced to the field's

declared type at parse time; compilation resolves public names through the allow-list and emits ? placeholders plus a bound parameter list. The demo exercises the happy path (a three-clause filter with a two-key descending sort), four rejection classes (unknown field, unknown operator, type mismatch, malformed clause), and an injection probe — `name:eq:x' OR '1'='1` — which compiles to a bound string literal and correctly matches zero rows. Note the public/private naming split: clients filter on `cost`, the column is `unit_cost_cents`, and internal columns are simply absent from `ALLOWED_FIELDS`.

Listing 9.3: Filter/sort compiler: allow-listed grammar, typed AST, parameterized SQL (`filter_compiler.py`).

```

1  """Allow-listed filter/sort compiler: query params ->
   parameterized SQL.
2
3  Grammar(public, documented surface):
4
5  ?filter=unit_cost_cents:gte:500;category:eq:sensor;
   name:contains:servo
6  ?sort=-unit_cost_cents,sku????????(leading '-'
   means descending)
7
8  The compiler never interpolates user input into SQL
   text. It builds a small
9  AST of typed nodes, then renders that AST to a WHERE
   clause plus a bound
10 parameter list. Unknown fields, unknown operators, and
   type mismatches are
11 rejected with a descriptive 'FilterError' -- clients
   get a 400, never a 500.
12
13 Offline demo: python filter_compiler.py
14 """
15
16 from __future__ import annotations
17
18 import sqlite3
19 from dataclasses import dataclass
20 from pathlib import Path
21 from typing import Any, Literal
22
23 # The public allow-list: API name -> (column, python
   type).
24 # Internal column names are deliberately *not* the
   public names everywhere:
25 # "cost" is the public name for the internal column

```

```

    unit_cost_cents.
26 ALLOWED_FIELDS: dict[str, tuple[str, type]] = {
27     "sku": ("sku", str),
28     "name": ("name", str),
29     "category": ("category", str),
30     "cost": ("unit_cost_cents", int),
31     "weight": ("weight_grams", int),
32     "active": ("active", bool),
33 }
34
35 ALLOWED_OPERATORS = {"eq", "ne", "gt", "gte", "lt", "lte", "in", "contains"}
36 ORDERING_OPERATORS = {"gt", "gte", "lt", "lte"}
37
38
39 class FilterError(ValueError):
40     """Raised for any filter/sort input outside the documented grammar."""
41
42
43 @dataclass(frozen=True)
44 class Comparison:
45     field: str
46     operator: Literal["eq", "ne", "gt", "gte", "lt", "lte"]
47     value: Any
48
49
50 @dataclass(frozen=True)
51 class InList:
52     field: str
53     values: tuple[Any, ...]
54
55
56 @dataclass(frozen=True)
57 class Contains:
58     field: str
59     needle: str
60
61
62 @dataclass(frozen=True)
63 class SortKey:
64     field: str
65     descending: bool
66
67

```

```

68 FilterNode = Comparison | InList | Contains
69
70
71 def _coerce(field: str, raw: str) -> Any:
72     column, kind = ALLOWED_FIELDS[field]
73     del column # coercion only needs the declared type
74     try:
75         if kind is bool:
76             if raw.lower() not in {"true", "false"}:
77                 raise ValueError(raw)
78             return 1 if raw.lower() == "true" else 0
79         return kind(raw)
80     except (TypeError, ValueError) as exc:
81         raise FilterError(
82             f"value_{raw!r} is not a valid {kind.
83                 __name__}_for_field_{field!r}"
84         ) from exc
85
86 def parse_filter(expr: str) -> list[FilterNode]:
87     """Parse the 'field:op:value[;field:op:value...]' '
88         _mini-grammar.'"""
89     nodes: list[FilterNode] = []
90     for clause in expr.split(";"):
91         clause = clause.strip()
92         if not clause:
93             continue
94         parts = clause.split(":", 2)
95         if len(parts) != 3:
96             raise FilterError(f"malformed_clause_{
97                 clause!r};_expected_field:op:value")
98         field, operator, raw = parts
99         if field not in ALLOWED_FIELDS:
100             raise FilterError(f"unknown_field_{field!r}
101                 };_allowed:_{sorted(ALLOWED_FIELDS)}")
102         if operator not in ALLOWED_OPERATORS:
103             raise FilterError(
104                 f"unknown_operator_{operator!r};_
105                     allowed:_{sorted(ALLOWED_OPERATORS)}"
106             )
107         if operator == "in":
108             values = tuple(_coerce(field, v) for v in
109                 raw.split(",") if v != "")
110             if not values:
111                 raise FilterError("in_operator_requires

```

```

107         _at_least_one_value")
108         nodes.append(InList(field=field, values=
109             values))
110     elif operator == "contains":
111         _, kind = ALLOWED_FIELDS[field]
112         if kind is not str:
113             raise FilterError(f"contains_is_only_
114                 valid_on_text_fields,not_{field!r}"
115                 )
116         nodes.append(Contains(field=field, needle=
117             raw))
118     else:
119         _, kind = ALLOWED_FIELDS[field]
120         if kind is str and operator in
121             ORDERING_OPERATORS:
122             raise FilterError(f"operator_{operator!
123                 r}_is_not_valid_on_text_field_{field
124                 !r}")
125         nodes.append(
126             Comparison(field=field, operator=
127                 operator, value=_coerce(field, raw))
128         )
129     return nodes
130
131 def parse_sort(expr: str | None) -> list[SortKey]:
132     """Parse '-cost,sku' into sort keys; always
133     append the id tie-breaker."""
134     keys: list[SortKey] = []
135     if expr:
136         for token in expr.split(","):
137             token = token.strip()
138             if not token:
139                 continue
140             descending = token.startswith("-")
141             field = token[1:] if descending else token
142             if field not in ALLOWED_FIELDS:
143                 raise FilterError(f"cannot_sort_by_
144                     unknown_field_{field!r}")
145             keys.append(SortKey(field=field, descending
146                 =descending))
147     # Deterministic tie-breaker: id is unique, so the
148     # total order is stable.
149     keys.append(SortKey(field="id", descending=False))
150     return keys

```

```

140
141 def compile_where(nodes: list[FilterNode]) -> tuple[str
142 , list[Any]]:
143     """Render filter nodes to a WHERE clause plus bound
144     parameters."""
145     fragments: list[str] = []
146     params: list[Any] = []
147     for node in nodes:
148         column = ALLOWED_FIELDS[node.field][0] # allow
149         -list lookup, not input
150         if isinstance(node, Comparison):
151             symbol = {"eq": "=", "ne": "!=", "gt": ">",
152                     "gte": ">=", "lt": "<", "lte": "<="}[
153                 node.operator]
154             fragments.append(f"{column}_{symbol}_?")
155             params.append(node.value)
156         elif isinstance(node, InList):
157             placeholders = ", ".join("?" for _ in node.
158                 values)
159             fragments.append(f"{column}_IN_{(
160                 placeholders)}")
161             params.extend(node.values)
162         elif isinstance(node, Contains):
163             # Escape LIKE wildcards in user input;
164             # ESCAPE clause pins the marker.
165             needle = node.needle.replace("\\", "\\").
166                 replace("%", "\\%").replace("_", "\\_")
167             fragments.append(f"{column}_LIKE_{(
168                 ESCAPE
169                 '\\\'')}")
170             params.append(f"%{needle}%")
171         else: # pragma: no cover - exhaustive by
172             construction
173             raise FilterError(f"unsupported node {node!
174                 r}")
175     clause = " _AND_ ".join(fragments) if fragments else
176     "1=1"
177     return clause, params
178
179 def compile_query(
180     filter_expr: str | None,
181     sort_expr: str | None,
182     limit: int,
183 ) -> tuple[str, list[Any]]:
184     """Compile public params into a complete, fully
185     parameterized SELECT."""

```

```

172     if limit < 1 or limit > 200:
173         raise FilterError("limit must be between 1 and
174                             200")
175     nodes = parse_filter(filter_expr) if filter_expr
176     else []
177     where, params = compile_where(nodes)
178     order = ", ".join(
179         f"{ALLOWED_FIELDS.get(k.field, ('id',))[0]}{'
180         DESC' if k.descending else 'ASC'}"
181         for k in parse_sort(sort_expr)
182     )
183     sql = (
184         "SELECT id, sku, name, category,
185             unit_cost_cents, weight_grams, active"
186         f"FROM parts WHERE {where} ORDER BY {order}
187         LIMIT ?"
188     )
189     return sql, [*params, limit]
190
191 def _demo() -> None:
192     db = Path("parts.db")
193     if not db.exists():
194         raise SystemExit("parts.db missing; run
195                             gen_dataset.py first")
196     conn = sqlite3.connect(db)
197     try:
198         sql, params = compile_query(
199             "cost:gte:500;category:in:sensor,actuator;
200             name:contains:servo",
201             "-cost,sku",
202             limit=5,
203         )
204         print(f"SQL: {sql}")
205         print(f"params: {params}")
206         for row in conn.execute(sql, params):
207             print(row)
208
209         for bad in (
210             "password_hash:eq:x",      # unknown field (
211                 internal column leaked?)
212             "cost:drop:1",              # unknown operator
213             "cost:gte:abc",             # type mismatch
214             "cost:eq:1;sort:evil",     # malformed clause
215         ):
216             try:

```

```

210         compile_query(bad, None, 10)
211     except FilterError as exc:
212         print(f"rejected_{bad!r}:_{exc}")
213     else:
214         raise SystemExit(f"input_{bad!r}_should_
                _have_been_rejected")
215
216     # Injection attempt is inert: it becomes a
        bound string parameter.
217     sql, params = compile_query("name:eq:x' _OR_
        '1'='1", None, 5)
218     rows = conn.execute(sql, params).fetchall()
219     assert rows == [], "injection_probe_must_match_
        nothing"
220     print("injection_probe_neutralized_as_bound_
        parameter;_demo_OK")
221 finally:
222     conn.close()
223
224
225 if __name__ == "__main__":
226     _demo()

```

9.3.4 Listing 4: Benchmark Harness

The harness (Listing 9.4) measures what Section 9.2.5 plotted: median single-page latency for offset and keyset fetches at depths from 0 to 150,000, page size 50, 15 repetitions, warm cache. Two details keep it honest. First, the keyset boundary for each depth is discovered with a one-off positional query (`boundary_at`) so both strategies fetch the *same* page. Second, medians rather than means suppress scheduler noise. The printed table is the source of Table 9.1 and Figure 9.1; per Chapter 0's discipline, the numbers are illustrative artifacts of one machine — the deliverable is the curve shape and your ability to reproduce it.

Listing 9.4: Offset-versus-keyset latency benchmark harness (`benchmark.py`).

```

1  """Benchmark_harness:_offset_vs_keyset_page-fetch_
        latency_at_depth.
2
3  Measures_the_median_wall-clock_time_to_fetch_one_page_
        of_50_rows_at
4  increasing_logical_depths_into_the_(unit_cost_cents,_id
        )_ordering.
5

```



```

6  Numbers are machine-dependent and therefore *
   illustrative*: the textbook
7  reports the shape of the curve, never a promised
   absolute latency.
8
9  Run: python gen_dataset.py --rows 200000 --db parts.db
10 python benchmark.py
11 """
12
13 from __future__ import annotations
14
15 import sqlite3
16 import statistics
17 import time
18 from pathlib import Path
19
20 PAGE_SIZE = 50
21 REPEATS = 15
22 DEPTHS = [0, 1_000, 10_000, 50_000, 100_000, 150_000]
23
24 OFFSET_SQL = (
25     "SELECT id, sku, name FROM parts"
26     "ORDER BY unit_cost_cents, id LIMIT ? OFFSET ?"
27 )
28 KEYSET_SQL = (
29     "SELECT id, sku, name FROM parts"
30     "WHERE (unit_cost_cents, id) > (?, ?)"
31     "ORDER BY unit_cost_cents, id LIMIT ?"
32 )
33
34
35 def time_query(conn: sqlite3.Connection, sql: str,
36               params: tuple) -> float:
37     """Return median seconds over REPEATS executions of
38     one page fetch."""
39     samples: list[float] = []
40     for _ in range(REPEATS):
41         start = time.perf_counter()
42         rows = conn.execute(sql, params).fetchall()
43         samples.append(time.perf_counter() - start)
44     assert len(rows) == PAGE_SIZE
45     return statistics.median(samples)
46
47 def boundary_at(conn: sqlite3.Connection, depth: int)
48     -> tuple[int, int]:

```

```

47     """Return the (cost, id) tuple of the row at
        logical position 'depth'."""
48     row = conn.execute(
49         "SELECT unit_cost_cents, id FROM parts"
50         "ORDER BY unit_cost_cents, id LIMIT 1 OFFSET ?"
51         ,
52         (depth, ),
53     ).fetchone()
54     if row is None:
55         raise ValueError(f"depth {depth} beyond table
56                             size")
57     return int(row[0]), int(row[1])
58
59 def main() -> None:
60     db = Path("parts.db")
61     if not db.exists():
62         raise SystemExit("parts.db missing; run
63                             gen_dataset.py first")
64     conn = sqlite3.connect(db)
65     try:
66         total = conn.execute("SELECT COUNT(*) FROM
67                             parts").fetchone()[0]
68         print(f"rows={total:}, page_size={PAGE_SIZE},
69               repeats={REPEATS}")
70         print(f"{'depth':>8} | {'offset_ms':>10} | {'
71               keyset_ms':>10} | speedup")
72         print("-" * 48)
73         for depth in DEPTHS:
74             if depth + PAGE_SIZE >= total:
75                 continue
76             cost, pid = boundary_at(conn, depth)
77             t_offset = time_query(conn, OFFSET_SQL, (
78                 PAGE_SIZE, depth))
79             t_keyset = time_query(conn, KEYSET_SQL, (
80                 cost, pid, PAGE_SIZE))
81             ratio = t_offset / t_keyset if t_keyset
82                     else float("inf")
83             print(
84                 f"{'depth':>8} | {'t_offset*1000:>10.3f
85                       } | "
86                 f"{'t_keyset*1000:>10.3f'} | {'ratio
87                       :>6.1f}x"
88             )
89     finally:
90         conn.close()

```

```

81
82
83 if __name__ == "__main__":
84     main()

```

9.3.5 Listing 5: Concurrency-Anomaly Demonstration

Finally, the anomaly made tangible (Listing 9.5). A single-threaded, fully deterministic script interleaves a reader walking four pages of five rows with a writer that inserts one *cheap* part — landing at the front of the (`unit_cost_cents`, `id`) ordering — between every pair of page fetches. This is a schedule two real connections can produce; scripting it in one process makes it reproducible offline. The script runs both walkers against a pristine catalog (baseline) and against the interleaved catalog, then diffs:

```

1 offset: with concurrent inserts -> duplicates=[5, 9,
    13], skipped=[18, 19, 20]
2 keyset: with concurrent inserts -> duplicates=None,
    skipped=None

```

Read the offset line against the drift formula of Section 9.2.3: each front insert ($\Delta_{\text{ins}} = 1$ at every boundary) shifts the window back by one, replaying one previously emitted row per page (the duplicates) and pushing three tail rows past the final page (the skips). The keyset walk anchors on values; front inserts are strictly *before* the anchor and therefore invisible to every subsequent predicate. Twenty rows, five-line output, one permanent intuition.

Listing 9.5: Deterministic writer/reader interleaving proving offset drift and keyset stability (`anomaly_demo.py`).

```

1 """Concurrency-anomaly_demo: offset pagination
    duplicates, keyset does not.
2
3 A deterministic, single-threaded script that*
    interleaves* a reader (walking
4 pages of a parts list) with a writer (inserting cheap
    parts at the front of
5 the (unit_cost_cents, id) ordering between page fetches
    ). This is exactly the
6 schedule two concurrent connections can produce;
    scripting it in one process
7 keeps the demo reproducible and offline.
8
9 Run: python anomaly_demo.py

```

```
10 """
11
12 from __future__ import annotations
13
14 import random
15 import sqlite3
16 from dataclasses import dataclass, field
17
18 DB = ":memory:"
19 PAGE = 5
20 TOTAL_PAGES = 4
21
22
23 def seed_catalog(conn: sqlite3.Connection) -> None:
24     conn.execute(
25         "CREATE TABLE parts ("
26         "id INTEGER PRIMARY KEY, sku TEXT,
27         unit_cost_cents INTEGER)"
28     )
29     rng = random.Random(7)
30     rows = [
31         (f"NWR-{i:05d}", 1000 + i * 100) for i in range
32         (1, 21)
33     ]
34     del rng # rows are literal: fully deterministic
35     conn.executemany(
36         "INSERT INTO parts (sku, unit_cost_cents)
37         VALUES (?, ?)", rows
38     )
39     conn.commit()
40
41
42 @dataclass
43 class WalkResult:
44     seen_ids: list[int] = field(default_factory=list)
45
46     @property
47     def duplicates(self) -> list[int]:
48         seen: set[int] = set()
49         dupes: list[int] = []
50         for pid in self.seen_ids:
51             if pid in seen and pid not in dupes:
52                 dupes.append(pid)
53             seen.add(pid)
54         return dupes
```

```

53
54 def writer_insert_cheap_part(conn: sqlite3.Connection,
55                               n: int) -> None:
56     """Insert one part cheaper than everything else (
57         lands at the front)."""
58     conn.execute(
59         "INSERT INTO parts (sku, unit_cost_cents)
60         VALUES (?, ?)",
61         (f"NWR-NEW-{n:02d}", 10 + n),
62     )
63     conn.commit()
64
65 def walk_offset(conn: sqlite3.Connection, writer: bool)
66     -> WalkResult:
67     """Paginate with LIMIT/OFFSET while the writer
68         shifts the window."""
69     result = WalkResult()
70     for page_no in range(TOTAL_PAGES):
71         rows = conn.execute(
72             "SELECT id FROM parts ORDER BY
73             unit_cost_cents, id"
74             "LIMIT ? OFFSET ?",
75             (PAGE, page_no * PAGE),
76         ).fetchall()
77         result.seen_ids.extend(r[0] for r in rows)
78         if writer:
79             # Between requests, another client inserts
80             # a cheap part.
81             writer_insert_cheap_part(conn, page_no)
82     return result
83
84 def walk_keyset(conn: sqlite3.Connection, writer: bool)
85     -> WalkResult:
86     """Paginate with a (cost, id) seek predicate under
87         the same writer."""
88     result = WalkResult()
89     position: tuple[int, int] | None = None
90     for page_no in range(TOTAL_PAGES):
91         if position is None:
92             rows = conn.execute(
93                 "SELECT id, unit_cost_cents FROM parts
94                 "
95                 "ORDER BY unit_cost_cents, id LIMIT ?",
96                 (PAGE, ),

```

```

89         ).fetchall()
90     else:
91         rows = conn.execute(
92             "SELECT id, unit_cost_cents FROM parts "
93             "WHERE (unit_cost_cents, id) > (?, ?) "
94             "ORDER BY unit_cost_cents, id LIMIT ?",
95             (position[0], position[1], PAGE),
96         ).fetchall()
97     result.seen_ids.extend(r[0] for r in rows)
98     if rows:
99         position = (int(rows[-1][1]), int(rows
100                        [-1][0]))
101     if writer:
102         writer_insert_cheap_part(conn, 100 +
103                                page_no)
104     return result
105
106 def main() -> None:
107     for strategy, walker in (("offset", walk_offset), ("
108         "keyset", walk_keyset)):
109         conn = sqlite3.connect(DB)
110         seed_catalog(conn)
111         baseline = walker(conn, writer=False)
112         conn.close()
113
114         conn = sqlite3.connect(DB)
115         seed_catalog(conn)
116         shifted = walker(conn, writer=True)
117         dupes = shifted.duplicates
118         # Rows of the original catalog the reader *
119         # skipped* entirely.
120         skipped = sorted(set(baseline.seen_ids) - set(
121             shifted.seen_ids))
122         print(
123             f"{strategy:>7}: with concurrent inserts ->
124             "
125             f"duplicates={dupes or 'none'}, skipped={
126                 skipped or 'none'}"
127         )
128         conn.close()
129
130     print(
131         "Conclusion: offset windows are positional, so
132         front inserts replay"

```

```

126         "rows;\nkeyset\windows\are\value-anchored,\so\
127         the\walk\stays\stable."
128     )
129
130 if __name__ == "__main__":
131     main()

```

Example

Verification record. Every listing above was executed offline before this chapter shipped: dataset generation (200,000 rows in ~ 3.5 s), the service self-check (offset, keyset, and cursor walks agree on 250 ids; forged cursor rejected with 400), the compiler demo (four rejection classes plus a neutralized injection probe), the anomaly demo (output above), and the benchmark (Table 9.1). Reproduce all of it with five PowerShell commands in Section 9.5.

9.4 Common Pitfalls & Anti-Patterns

Each entry below names the failure, explains the mechanism, and states the fix. Most production pagination incidents are one of these seven.

9.4.1 Offset at Depth

Failure: a list endpoint that is fast in testing and degrades linearly in production as the table grows, because tests walk page 1 and users (or crawlers) walk page 4,000. **Mechanism:** $T_{\text{offset}} = O(k)$ — Section 9.2.2; every deep page re-scans and discards everything before it. **Fix:** keyset or cursor pagination for any collection that can outgrow memory; if the product genuinely needs random access, cap the depth (`offset + limit` ≤ 10000) and route true exports to an async job.

9.4.2 Unsigned, Guessable Cursors

Failure: base64-encoded (but unsigned) cursors whose payload a curious client edits — replaying walks, jumping arbitrarily, probing the ordering, or injecting unexpected types into payload fields. **Mechanism:** encoding is not authentication; the server trusted client-carried state. **Fix:** HMAC-sign the payload, verify with a constant-time comparison *before* parsing, version the payload, expire it, and answer every failure mode with the same undifferentiated 400 (Section 9.2.7).

9.4.3 `total_count` on Every Page

Failure: each page response embeds an exact `COUNT(*)`, so every request of every client’s walk re-runs an $O(n)$ aggregate. **Mechanism:** the count is computed even though the client needed it once (or never); under concurrency it is also instantly stale, so the precision was illusory anyway. **Fix:** default to `has_more` via `LIMIT p+1`; make totals opt-in, estimate from planner statistics when an approximation suffices, and never compute `rel="last"` links by default (Section 9.2.9).

9.4.4 Missing Tie-Breaker

Failure: `ORDER BY unit_cost_cents` on a column full of duplicates; pagination “mostly works” and occasionally replays or drops rows near ties — differently on each deploy, plan change, or vacuum. **Mechanism:** ties make the order non-total, so neither offsets nor keysets name a stable boundary. **Fix:** always append a unique, immutable column: `ORDER BY unit_cost_cents, id`. Enforce it in the sort compiler, not in documentation.

9.4.5 Unbounded Page Sizes

Failure: the server honors `?limit=1000000`; one client allocates a megabyte-scale JSON array per request and starves the pool. **Mechanism:** `limit` is a resource reservation request from an untrusted peer. **Fix:** a hard server-side ceiling (`MAX_LIMIT = 200` in the reference service) with silent clamping or a 400; document the ceiling in the contract.

9.4.6 Filter Parameters Interpolated into SQL

Failure: `f"WHERE col >= {param}"` or, worse, `f"ORDER BY {sort_param}"`. **Mechanism:** query parameters are attacker-controlled text; interpolation makes them executable (Section 9.2.10). This remains the top API injection vector in the wild. **Fix:** allow-listed grammar $\rightarrow$ typed AST $\rightarrow$ bound parameters. There is no safe-regex shortcut, and “internal API” is not a mitigation — internal callers get compromised too.

9.4.7 Exposing Internal Column Names

Failure: filter/sort/field parameters accepted as raw column names, leaking schema internals (`password_hash`, `tenant_key`) to API consumers and coupling the public contract to physical refactors. **Mechanism:** the public surface *is* the schema. **Fix:** a public naming layer resolved through an

allow-list map (`cost` $\rightarrow$ `unit_cost_cents`); unknown names get a 400 that lists the *allowed* fields, never the schema.

Warning

A composite anti-pattern ties the list together: *offset pagination* + *per-page COUNT(*)* + *client-controlled sort on a non-unique column*. Each is defensible alone in a demo; combined at scale they produce an endpoint that is slow, incorrect under writes, and non-deterministic — simultaneously. When you inherit such an endpoint, migrate consumers to cursors behind the `Link` header (clients following `rel="next"` need not notice) rather than breaking the contract outright.

9.5 Hands-On Production Lab & Real-World Case Study

9.5.1 Lab: Three Strategies, One Dataset, Zero Excuses

Time: 45–60 minutes. **Prerequisites:** the environment from Section 9.3. All commands are PowerShell, run from the chapter’s code directory.

Part A — Build and verify the dataset (5 min)

```
1 python gen_dataset.py --rows 200000 --db parts.db
2 python -c "import sqlite3; c=sqlite3.connect('parts.db'); print(c.execute('SELECT COUNT(*), MIN(unit_cost_cents), MAX(unit_cost_cents) FROM parts').fetchone())"
```

Expected: generated 200,000 rows into `parts.db` in a few seconds, then a count of 200,000. Re-run with the same `--seed`: the database is byte-comparable deterministic in content (same rows, same order).

Part B — Exercise all three strategies (15 min)

```
1 python pagination_service.py
```

The self-check walks 250 ids through `/parts/offset`, `/parts/keyset`, and `/parts/cursor`, asserts the three id sequences are identical, and forges a cursor to confirm the 400 path. Then probe by hand with the server running (`uvicorn pagination_service:app -port 8000` in a second terminal):

```
1 # First page, opaque cursor strategy; note the Link
   header and next_cursor.
2 (Invoke-WebRequest "http://localhost:8000/parts/cursor?
   limit=3").Headers["Link"]
3 (Invoke-WebRequest "http://localhost:8000/parts/cursor?
   limit=3").Content
4
5 # Tamper with the cursor: change the final character;
   expect HTTP 400.
6 Invoke-WebRequest "http://localhost:8000/parts/cursor?
   limit=3&cursor=<forged>"
7
8 # Keyset, transparently: choose your own boundary.
9 Invoke-RestMethod "http://localhost:8000/parts/keyset?
   limit=3&after_cost=5000&after_id=100000"
```

Deliverable: in your notes, record the Link header of each strategy and state which relations appear and why (last only under `include_total=true`; no prev for cursors — why?).

Part C — Break filtering, then fail to (10 min)

```
1 python filter_compiler.py
```

Confirm the four rejection classes print, then write three attack strings of your own (an `OR 1=1` variant, a semicolon chain, a `UNION SELECT`) and verify each either parses to an inert bound parameter or is rejected at the grammar. Deliverable: the compiled SQL and parameter list for one attack, annotated with the layer that neutralized it.

Part D — Reproduce the anomaly (10 min)

```
1 python anomaly_demo.py
```

Expected output (deterministic):

```
1 offset: with concurrent inserts -> duplicates=[5, 9,
   13], skipped=[18, 19, 20]
2 keyset: with concurrent inserts -> duplicates=None,
   skipped=None
```

Deliverable: modify the writer to *delete* rows ahead of the window instead of inserting, predict the outcome using the drift formula ($\Delta < 0 \Rightarrow$ skips), and confirm. Then explain, in two sentences, why no transaction isolation level on the reader's individual page queries prevents this.

Part E — Measure the curve (10 min)

```
1 python benchmark.py
```

Compare against Table 9.1. Your absolute numbers will differ (hardware, SQLite version, cache state); the *shape* must not: offset roughly linear in depth, keyset flat within noise. Deliverables: (i) your table; (ii) re-run with `--rows 1000000` and report where offset crosses 50 ms; (iii) run `EXPLAIN QUERY PLAN` on both `OFFSET_SQL` and `KEYSET_SQL` and quote the line that proves the keyset is a `SEARCH` using `idx_parts_cost_id`.

9.5.2 Case Study: The 2 a.m. Reporting API

Names and numbers are anonymized composites drawn from real post-mortems; the failure physics is exact.

A mid-size robotics-parts distributor — call it the production twin of our fictional Northwind Robotics — exposed an internal reporting API used by the finance team’s nightly reconciliation export. The endpoint was the composite anti-pattern incarnate: `GET /api/ledger?offset=&limit=1000`, sorted by `posted_at` (non-unique, millisecond timestamps from a batch importer), every response carrying `total_count`, filters accepted as `?where=...` fragments “because only finance uses it.”

At 2 a.m. the export job began its walk over what had quietly become an 8-million-row ledger after an acquisition data load. The walk is the $O(n^2/p)$ trap in production form: 8,000 page requests, each scanning an average of 4 million rows to reach its window — about 3.2×10^{10} row visits for the night — and each request *also* running `COUNT(*)` to fill `total_count` and the `rel="last"` link. Page fetches that cost 2 ms at offset 0 cost 40+ seconds by the middle of the walk. The connection pool saturated; the health-check endpoint — served by the same pool — began timing out; the orchestrator concluded the service was dead and began killing and rescheduling pods, whose cold caches made the next pages slower still. The morning dashboard showed a cascading failure across services that shared the database; the root cause was one `OFFSET` clause.

And the report was *wrong anyway*. The batch importer was still inserting back-dated rows ahead of the walker’s window; the export contained duplicated postings (finance caught ~11,000 double-counted line items in review) and a silent gap where a midnight delete had shifted a window forward. Non-unique `posted_at` ordering meant the duplicates were not even deterministic.

The fix, deployed over the following sprint, maps one-to-one onto this chapter: (1) keyset pagination over (`posted_at`, `ledger_id`) with a composite index to match — walk time fell from 9 hours to 11 minutes and

became flat with respect to depth; (2) `total_count` removed from pages, replaced by `has_more` and a nightly-refreshed stats table for the UI badge; (3) the `?where=` fragment replaced by an allow-listed filter grammar compiled to bound parameters (an audit found the fragment interface was reachable by any authenticated internal user); (4) a deterministic tie-breaker added to every ordered endpoint. The postmortem’s action item worth stealing: *“No list endpoint ships without a named pagination strategy, a maximum depth or page size, and an EXPLAIN plan in the PR description.”*

9.6 Self-Assessment Exercises & Deep-Dive Questions

Attempt each before consulting the references. Exercises marked [code] extend the chapter’s listings; [proof] items demand a derivation or a counterexample; [design] items demand a defensible trade-off.

- E8.1.** [proof] Starting from $T_{\text{offset}}(k, p) = c_{\text{seek}} + kc_s + pc_f$, show that a full walk of an n -row table in pages of p costs $\Theta(n^2/p)$ row visits, and compute the constant factor. What does this imply for the *worst* page size from the server’s point of view?
- E8.2.** [proof] Give a precise writer schedule (inserts and deletes with positions) over a 20-row, page-size-5 walk such that the offset reader emits *exactly* the multiset $\{r_1, \dots, r_{19}\}$ with r_{20} never emitted and r_7 emitted twice.
- E8.3.** [code] Extend `anomaly_demo.py` with a third walker that paginates by offset but re-reads the last row of the previous page to detect drift. Show it detects duplicates but cannot recover the skipped rows. Why is detection-without-recovery the best a purely positional scheme can do?
- E8.4.** [code] Add a `/parts/cursor` variant whose payload embeds a SHA-256 hash of the active filter and sort parameters, rejecting continuations whose parameters no longer match. Explain which attack or bug class this closes (Section 9.2.10).
- E8.5.** [design] Your product requires “jump to page 47” in an admin grid over 3 million rows. Design the endpoint: what do you cap, what do you count, what do you cache, and what do you tell the PM about the page-47 experience?
- E8.6.** [proof] Show that the tuple predicate $(a, b) > (x, y)$ and its disjunctive expansion $a > x \text{ OR } (a = x \text{ AND } b > y)$ are logically equivalent,

and construct a 6-row table on which the hand-rolled predicate `a >= x AND b > y` returns a different result set. Name the two distinct error modes.

- E8.7.** [code] Add `NULLS LAST` ordering to a nullable `discontinued_at` sort and write the NULL-aware keyset predicate for it. Verify against `filter_compiler.py`'s demo database.
- E8.8.** [design] Design the expansion policy for `GET /orders ?expand=customer,lines.part:` batch sizes, depth cap, cache headers, and the point at which you tell the consumer to use the GraphQL gateway (Chapter 12) instead.
- E8.9.** [code] Using `EXPLAIN QUERY PLAN`, produce one index definition that turns the chapter's keyset query from `SEARCH ... USING INDEX` into `SCAN` plus a temporary B-tree, and explain in one paragraph what the planner is telling you.
- E8.10.** [design] A compliance rule requires an *exact* total of matching records on an audited search endpoint. Reconcile this with Section 9.2.9's cost analysis; give at least two architectures and their failure modes.

9.7 Interview Q&A Vault

Ordered junior → staff. Answers are model responses at the depth a strong candidate gives verbally.

- Q1. What is pagination and why can't we just return everything?** Pagination returns a collection in bounded pages. Unbounded responses blow up memory on both sides, serialize for seconds, blow through timeouts, and turn one curious client into a denial-of-service event. Bounding page size is a resource-governance decision as much as a UX one.
- Q2. How does LIMIT/OFFSET work?** `ORDER BY` materializes the ordering, the engine skips `OFFSET` rows, then returns up to `LIMIT` rows. The skip is the problem: the engine must visit the skipped rows to count past them.
- Q3. What is a cursor in API pagination?** An opaque token the server returns with a page; the client sends it back to get the next page. Internally it usually encodes the boundary row's sort-key values — a bookmark by value, not by position.
- Q4. What is the difference between `has_more` and `total_count`?**

`has_more` answers “is there a next page” and costs one extra row (`LIMIT p+1`). `total_count` answers “how many altogether” and costs a full `COUNT(*)` over the filtered relation on every request that includes it.

Q5. What HTTP header is designed for page navigation links?

The `Link` header, RFC 8288 [Not17], with relation types `next`, `prev`, `first`, `last`. Clients can follow `rel="next"` without knowing the pagination strategy.

Q6. What does `?fields=id,name` do and why offer it?

Sparse field selection: the response includes only the requested attributes. It cuts payload size and serialization cost, and can make queries answerable from a covering index. Fields must come from an allow-list, not raw column names.

Q7. Why should every sort end with a unique column?

Because a non-unique sort key leaves ties in an unspecified order, and page boundaries through a tie are non-deterministic. Appending the primary key makes the order total, which both offset and keyset pagination require to be stable.

Q8. Name three safe filter operators you’d expose and one you would not.

`eq`, `gte`, `in` — all parameterizable and index-friendly. I would not expose a raw `where` or free-text SQL fragment parameter: it cannot be made safe by sanitization, only by replacement with a grammar.

Q9. Why is offset pagination $O(n)$ at depth while keyset is $O(\log n)$?

A B-tree supports lookup by *value*, not by *position*. Reaching offset k requires visiting k index entries to count them — $O(k)$, up to $O(n)$. A keyset predicate `(cost,id) > (?,?)` is a value lookup: $O(\log n)$ descent plus $O(p)$ for the page itself, independent of depth.

Q10. Explain the duplicates-and-skips anomaly of offset pagination.

Between page requests, inserts ahead of the reader’s window shift every later position down by one, so the next `OFFSET` window replays rows already emitted (duplicates); deletes shift positions up, hiding unemitted rows (skips). Each page is individually correct; the composition is not, because the offset is a coordinate in a system the writer keeps renumbering.

Q11. Why doesn’t snapshot isolation fix it?

It would — if the whole walk ran in one transaction. But the walk spans unbounded HTTP

requests; holding a snapshot open that long pins versions, bloats the database, and creates its own failure modes. Keyset anchoring fixes consistency without long-lived snapshots.

- Q12. Write the keyset predicate for ORDER BY cost, id and expand the tuple comparison.** WHERE (cost, id) > (?, ?) — equivalent to cost > ? OR (cost = ? AND id > ?). The second disjunct is what keeps ties on cost correct, which is why the unique tie-breaker is part of the contract.
- Q13. What index serves that query, and what must align?** A composite index on (cost, id) — same column order and direction as the ORDER BY. An index on (id, cost) is useless for it; (cost) alone forces a sort of ties. Verify with EXPLAIN: you want SEARCH ... USING INDEX, not SCAN or a temp B-tree.
- Q14. Design a tamper-proof cursor token.** JSON payload with version, expiry, and boundary values; base64url encode; append a truncated HMAC-SHA256 keyed with a server secret; verify with a constant-time comparison before parsing; on any failure return one undifferentiated 400. Payload fields are still re-bound as query parameters — a valid signature does not make the contents trusted SQL.
- Q15. Why version the cursor payload?** Tokens outlive deploys. A client mid-walk during a rollout can hold a token minted by the old build; a version field lets the new build reject it cleanly (client restarts the walk) instead of mis-parsing it as corrupt or, worse, valid-but-different.
- Q16. When is offset pagination actually fine?** Small or slowly-changing collections, shallow walks, and a real random-access requirement — an internal admin grid over ten thousand rows is the canonical case. Cap the maximum offset, and don't compute COUNT(*) per page.
- Q17. How do you implement rel="last" and what does it cost?** You need the total: $last = \lfloor (n - 1) / p \rfloor \cdot p$. That means COUNT(*) per request that emits it — an $O(n)$ aggregate — which is why many APIs omit last or compute totals from cached estimates.
- Q18. How would you estimate total_count cheaply?** Planner statistics (e.g. PostgreSQL reltuples), the row estimate from EXPLAIN of the same query, or a periodically refreshed stats table. All are approximate; present them as approximate (“~200k results”) and never mix an estimate into a contract that promises exactness.
- Q19. How do you keep client-supplied sort direction safe?** Direction

is parsed as a token from a two-element set (`asc/desc` or a leading `-`), never interpolated. `ORDER BY {user_input}` is SQL injection; the field allow-list protects the column, the direction token protects the keyword position, which bind parameters cannot reach.

- Q20. How do filter parameters and cursors interact?** The filter and sort define the walk, so they're part of the cursor contract: either they must be resent verbatim on every page (and the server rejects mismatches) or their hash is embedded in the signed payload. A cursor silently continuing a *different* query is a subtle correctness bug.
- Q21. What is the N+1 problem at an API boundary and how do you fix it in REST?** `?expand=supplier` on a 50-item page becomes 1 query for the page plus 50 supplier queries. Fix with DataLoader-style batching: collect the page's foreign keys, run one `WHERE id IN (...)` per expanded relation, stitch in memory. $1+N$ queries become $1+k$ for k relations.
- Q22. How do keyset pagination and multi-column sort interact?** The keyset tuple spans *all* sort keys in order — `(a, b, id) > (?, ?, ?)` — and the composite index must match column-for-column. Mixed sort directions break the single tuple comparison (you need the expanded disjunctive form with per-column operators), which is why cursor endpoints usually fix or constrain direction.
- Q23. How do you paginate a feed ranked by score?** Score is the sort key: order by `(score DESC, id DESC)` and keyset on `(score, id) < (?, ?)`. The hard part is that scores *change*: the walk then sees a time-varying ranking, so either snapshot the ranking (materialize a ranked list keyed by a snapshot id, cursor over positions *within the immutable snapshot*) or accept drift and document it. Hot feeds (Reddit-style) snapshot; warm feeds accept drift.
- Q24. Your keyset query suddenly regressed to seconds. How do you diagnose?** `EXPLAIN` first: look for `SCAN` replacing `SEARCH`, a temp B-tree for `ORDER BY`, or a rows-examined estimate that tracks the table size. Usual causes: a new filter column not prefixed in the index, a sort direction flip, stale planner statistics, or an `OR`-ed predicate defeating sargability. Then check whether the payload outgrew the covering index.
- Q25. Transparent keyset vs. opaque cursor — how do you choose?** Transparent keysets (`after_cost=...`) are debuggable and book-markable but couple the public contract to the physical schema and

let clients construct arbitrary boundaries. Opaque signed cursors decouple contract from schema, enable expiry and rotation, and prevent boundary forgery — at the price of undebuggability-from-the-outside. Public API: opaque. Internal tools where engineers live in logs: transparent is defensible.

- Q26. How do you paginate consistently across multiple database shards?** You don't get a global cheap ordering; you merge. Options: scatter keyset queries to all shards and k -way merge the pages (cost grows with shard count per page), pin a tenant to a shard so pagination is single-shard (the multi-tenant answer), or maintain a denormalized global ordering table. Offsets across shards compound the $O(k)$ scan k ways — keyset is essentially mandatory.
- Q27. What are the security considerations of cursor design?** Cursors are capabilities: sign them (integrity), expire them (replay window), keep tenant/scope claims out unless enforced server-side per request (a valid cursor must not widen authorization), never log raw tokens, and treat payload fields as untrusted input even after signature verification. And the signing key rotates without invalidating the world only if you version keys.
- Q28. How would you migrate a live public API from offset to cursors without breaking consumers?** Ship cursor support alongside offset on the same endpoint; emit `Link: rel="next"` in cursor form even for offset requests so hypermedia-driven clients migrate transparently; announce deprecation with a sunset header; add depth caps to offset first (they fail loudly for the crawlers you most want off it); measure strategy mix from logs; remove offset only at the next major version per the versioning policy (Chapter 9).
- Q29. When does `COUNT(*)` actually become expensive, and what's the production alternative?** When the filtered relation stops fitting comfortably in cache or the count runs per page per client under concurrency — at millions of rows it dominates endpoint latency. Alternatives: opt-in counts, counts on page one only, planner estimates, pre-aggregated stats tables refreshed out of band, or threshold counting (exact up to a cap, "10000+" beyond it, computed with a `LIMIT`d probe).
- Q30. Design pagination for a multi-tenant SaaS where tenant sizes span six orders of magnitude.** Per-tenant keyset over (`tenant_id`, `sort_key`, `id`) with a composite index led by `tenant_id`: the equality prefix keeps every tenant's walk a local index range, so the 10-row

tenant and the 100-million-row tenant share code without sharing performance failure modes. Depth caps and page-size ceilings apply per tenant; exports for whale tenants go to an async job with resumable cursors stored server-side.

- Q31. How do you make a paginated export resumable across server restarts and client crashes?** Persist the cursor: the export job checkpoints the signed cursor (plus the filter/sort hash) after each page, so a crash resumes at the last boundary instead of page one. Keyset makes this sound — the resume point is a value, immune to intervening writes; offset checkpoints resume into a shifted coordinate system and silently corrupt the export.
- Q32. A client demands stable pagination over data being bulk backfilled. Options?** Four honest answers: (1) snapshot semantics — materialize the query result or a snapshot id up front, paginate the frozen snapshot; (2) keyset with documented drift semantics (fine for appends at the tail, wrong for front inserts); (3) quiesce or fence the backfill behind a high-watermark column (`WHERE created_at <= ?` baked into the walk) so the walked relation is effectively immutable; (4) change feed / CDC instead of pagination when the requirement is really “see everything exactly once” (Chapter 11).
- Q33. Where do filtering capabilities belong architecturally — gateway, service, or database?** Enforcement of the grammar and allow-list belongs at the service boundary (it is contract logic); compilation targets the database (it is execution logic); the gateway may enforce coarse limits (parameter count, page-size ceilings) but should not own filter semantics, or you cannot evolve the contract without redeploying infrastructure. The deeper the predicate is pushed into the query (sargable, index-aligned), the less data crosses every layer.
- Q34. How do you test pagination correctness systematically?** Property tests, not examples: for random table contents, page sizes, and write schedules, assert the concatenation of pages equals the expected ordered multiset (keyset) or detect drift (offset). Add determinism tests (same walk twice $\Rightarrow$ same pages), boundary tests (empty table, single row, page size 1, exact multiples, ties spanning boundaries), security tests (forged, expired, and version-mismatched cursors), and a benchmark gate in CI so a missing index fails the build, not the 2 a.m. pager.
- Q35. What consistency guarantees would you document for a public cursor-paginated endpoint, and why those?** Per-page

serializability (each page is a consistent answer at some instant), monotonic value-progression (no page revisits a boundary it passed), and documented drift semantics for mutations of already-visited versus not-yet-visited regions. You explicitly do *not* promise a stable total or a walk-level snapshot unless you sell snapshot cursors — promising more than the mechanism delivers is how you end up in someone else’s postmortem.

- Q36. Is cursor pagination compatible with relational JOINS and aggregation endpoints?** With care. The keyset predicate must reference columns of the driving relation and the index must cover the join’s access path; joins multiply rows per key value, so the boundary tuple must be unique at the *joined* row granularity or you split a group across pages. Aggregates generally should not paginate by keyset over grouped keys without a covering index on the grouping columns — and pagination over a `GROUP BY` whose groups change under concurrent writes has the same drift physics as the base case, one level up.
- Q37. What would you put in an organization-wide pagination standard?** One envelope shape (`items`, `has_more`, `next_cursor`), one cursor format (signed, versioned, expiring), `Link` headers mandatory, `total_count` opt-in only, `MAX_LIMIT` set platform-wide, a unique tie-breaker on every ordered endpoint, an allow-listed filter grammar with a fixed operator set, an `EXPLAIN` plan attached to every list-endpoint PR, and a benchmark gate on depth-scaling. Standards exist so that the wrong thing is the hard thing.
- Q38. Critique: “We’ll just paginate with `WHERE id > last_id`”. When is that right, and when does it silently lie?** It is a keyset over the primary key — optimal when the product ordering *is* insertion order (ledgers, event logs, audit trails) because the clustered index serves it for free. It silently lies when consumers believe they’re seeing a business ordering — recency by `updated_at`, price, relevance — because `id` order is not any of those; it also breaks under `id` reassignment, merged imports with re-keyed `ids`, and any future requirement to re-sort. Right tool for append-only streams, wrong abstraction for ranked collections.

9.8 External Bibliography & Further Reading

- **RFC 8288, *Web Linking*** [Not17] — the normative reference for Link headers and relation types (`next/prev/first/last`). Short, precise, and the reason hypermedia pagination is interoperable.
- **RFC 9110, *HTTP Semantics*** [Fie22a] — status codes for the failure modes in this chapter (400 for grammar rejections), `Vary` for parameterized responses, and the caching semantics that interact with field selection.
- **Markus Winand, *Use The Index, Luke*** (use-the-index-luke.com) — the canonical free reference on index anatomy; its “Pagination Done the Right Way” chapter is the keyset method’s best-known exposition, with per-database dialect notes.
- **Martin Kleppmann, *Designing Data-Intensive Applications*** [Kle17] — B-tree internals, snapshot isolation, and the read/write-path trade-offs behind every cost claim in this chapter.
- **JJ Geewax, *API Design Patterns*** [GH21] — opinionated, battle-tested treatment of page tokens, filtering, and resource projection as repeatable design patterns.
- **PostgreSQL Documentation, *LIMIT and OFFSET and Indexes*** — authoritative semantics for row skipping, row estimates used in counting strategies, and `NULLS FIRST/LAST` behavior.
- **SQLite Documentation, *SELECT and EXPLAIN QUERY PLAN*** — row-value comparison support and the query-plan output vocabulary (`SEARCH` vs. `SCAN` vs. covering index) used throughout the lab.
- **JSON:API specification, *Pagination and Sparse Fieldsets profiles*** — a widely adopted convention family (`page[size]`, `page[after]`, `fields[type]`) worth aligning with when you do not control both ends.
- **Stripe API documentation, *Pagination*** — the industry reference implementation of opaque cursor pagination (`starting_after/ending_before`, `has_more`); read it as a case study in minimal, stable contracts.
- **Slack API documentation, *Pagination*** — cursor pagination

at scale (`next_cursor` in `response_metadata`), including the operational rationale for retiring offset-based methods.

- **FastAPI and Pydantic documentation** [Ram24; Col24] — query-parameter validation, response modeling, and the `TestClient` used by the reference service’s offline self-check.
- *The Datadog/Shopify engineering blogs on cursor-based pagination and GraphQL Connections* (Relay Cursor Connections spec) — for the `edges/node/pageInfo` vocabulary you will meet again in Chapter 12.

9.9 Capstone Projects

Five projects, progressive. Each names its difficulty tier; acceptance criteria are checkable facts, not vibes.

9.9.1 Capstone 1: Cursor-Paginated Ledger API [junior]

Objective. Re-implement the case study correctly: a `/ledger` endpoint over a synthetic append-only postings table, paginated by opaque signed cursor.

Inputs/Datasets. `gen_dataset.py` extended with a `ledger(posted_at, ledger_id, amount_cents, account)` table, seeded with 100,000 postings including deliberate timestamp ties.

Steps.

1. Order by `(posted_at, ledger_id)`; build the composite index to match.
2. Implement the keyset predicate with tuple comparison and the `LIMIT p+1 has_more` trick.
3. Wrap the boundary in an HMAC-signed, versioned, expiring cursor; reject forged tokens with a uniform 400.
4. Emit `Link: rel="next"` on every non-final page.

Acceptance Criteria.

- A scripted full walk emits every posting exactly once, verified by set comparison against the source table.

- ☐ `EXPLAIN QUERY PLAN` shows `SEARCH` using the composite index for every page query.
- ☐ Forged, expired, and version-mismatched cursors each return 400 with a problem-details body.
- ☐ Timestamp ties spanning a page boundary are handled without duplicates or skips (test included).

Stretch Goals. Add `rel="prev"` via a reversed-order query; add per-account filtering with an index-led equality prefix.

9.9.2 Capstone 2: Filter Compiler with Property Tests [mid]

Objective. Harden Listing 9.3 into a library with a documented grammar and adversarial test coverage.

Inputs/Datasets. The `parts` table; a corpus of 50 attack strings (injection probes, wildcard floods, encoding tricks, type confusions) you curate yourself.

Steps.

1. Specify the grammar in the README-style module docstring (fields, operators, types, precedence-free conjunction).
2. Extend the AST with parentheses-free `OR` groups (|-separated clauses) compiled with correct parentheses.
3. Write `pytest` suites: unit tests per node type, golden SQL/params tests, and property tests asserting that (a) compiled SQL contains no substring of any raw input value, and (b) every attack string is either rejected or executes as inert data.
4. Add a FastAPI adapter mapping `FilterError` to a 400 problem-details response listing allowed fields and operators.

Acceptance Criteria.

- ☐ `pytest -q` green, including the property tests.
- ☐ All 50 attack strings rejected or provably inert (asserted by executing them against the dataset).
- ☐ No public filter/sort/field parameter name equals an internal column name it was not intended to expose.

- ☐ Compiled queries use bound parameters exclusively — verified by inspection test on the compiler output.

Stretch Goals. Case-insensitive `contains` with correct collation; filter-dependent index recommendation report driven by `EXPLAIN`.

9.9.3 Capstone 3: Pagination Benchmark Report [mid]

Objective. Produce a reproducible, reviewer-grade benchmark comparing offset, keyset, and cursor page fetches across table sizes and page sizes.

Inputs/Datasets. `gen_dataset.py` at 100k, 200k, and 1M rows; `benchmark.py` extended with cursor-token overhead measurement and p50/p95 columns.

Steps.

1. Parameterize the harness by row count, page size, and depth list; record environment metadata (Python, SQLite version, CPU).
2. Add percentile statistics (p50/p95) and a cold-cache variant using a fresh connection per repetition.
3. Measure cursor mint/verify overhead per request and report it as a latency line item.
4. Write the report: tables, one growth-curve figure per row count, and a stated verdict on where offset crosses your latency budget.

Acceptance Criteria.

- ☐ One command reproduces every table in the report (numbers flagged illustrative, shapes asserted).
- ☐ Offset growth is demonstrably linear (fit reported); keyset is flat within stated noise bounds.
- ☐ Cursor overhead is quantified and shown to be constant with depth.
- ☐ The report names the exact index used by each query and quotes the `EXPLAIN` evidence.

Stretch Goals. Covering-index variant showing the offset penalty shrinking but not vanishing; PowerShell runner script for the whole matrix.

9.9.4 Capstone 4: Infinite-Scroll Client over Signed Cursors [senior]

Objective. Build a terminal-based infinite-scroll parts browser that consumes *only* `rel="next"` links, proving the client never learns the pagination

strategy.

Inputs/Datasets. The chapter's FastAPI service (cursor mode) over the 200k-row dataset; `httpx` for the client.

Steps.

1. Implement the client loop: follow `Link rel="next"` until absent; render pages with running totals and elapsed time per page.
2. Add resume: persist the last cursor and the filter/sort state to disk; restart mid-walk and continue exactly.
3. Add failure drills: expired cursor (sleep past TTL), forged cursor, server restart — each handled with a documented recovery (restart walk vs. resume).
4. Run the client against a live writer inserting rows mid-walk; record and explain the observable behavior.

Acceptance Criteria.

- ☐ Client source contains no offset arithmetic and no cursor parsing — only header following.
- ☐ A kill-and-resume test completes the walk with zero duplicates (asserted client-side).
- ☐ Expired and forged cursors produce a controlled, user-visible recovery path, not a crash.
- ☐ Mid-walk inserts produce a written behavior note consistent with the chapter's drift model.

Stretch Goals. Conditional requests with `ETag` per page (previews Chapter 10); strategy negotiation where the server silently switches the client from offset links to cursor links.

9.9.5 Capstone 5: Multi-Tenant Keyset Design [staff]

Objective. Design and validate the pagination layer for a multi-tenant SaaS whose tenants span six orders of magnitude in row count, including isolation, index design, and org-wide standards.

Inputs/Datasets. A synthetic three-tenant dataset (1k, 200k, and 5M rows) in one `parts` table with `tenant_id`; the chapter's service as the starting point.

Steps.

1. Add `tenant_id` as the index-leading equality column: `(tenant_id, unit_cost_cents, id)`; prove every tenant's walk stays a local `SEARCH`.
2. Embed a tenant-scope claim *outside* the cursor (server-side auth context) and prove a valid cursor from tenant A cannot page tenant B.
3. Design per-tenant policy: page-size ceilings, depth caps, and an async export path with server-side resumable cursors for the whale tenant.
4. Write the org standard as a one-page ADR: envelope shape, cursor format, error contract, and the EXPLAIN-in-the-PR gate.
5. Load-test: concurrent walks from all three tenants; demonstrate the whale tenant's walk does not move the small tenants' p99.

Acceptance Criteria.

- ☐ Cross-tenant cursor replay is impossible, with a test proving it.
- ☐ All three tenants' page latency is depth-independent within stated noise, measured and tabulated.
- ☐ The whale tenant's export resumes correctly after a simulated crash (checkpoint test).
- ☐ The ADR names every trade-off explicitly rejected (offsets, per-page counts, transparent cursors) with reasons.

Stretch Goals. Row-level security in SQLite views or a second connection role; snapshot-id cursors for compliance exports requiring a frozen relation.

9.10 Python Dependencies Appendix

The chapter's laboratory pins the following environment. Everything else is the Python 3.11+ standard library — notably `sqlite3`, `hmac`, `hashlib`, `base64`, and `random` — consistent with the curriculum's offline-first, stdlib-first rules.

```
1 fastapi>=0.110
2 pydantic>=2.7
3 httpx>=0.27
4 pytest>=8
```

Install on Windows PowerShell from the chapter's `code` directory:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

9.10.1 fastapi (fastapi>=0.110)

Description. The ASGI web framework used for the reference service [Ram24]: routing, request parsing, and automatic query-parameter validation via type hints.

Why included. Chapters 6–7 standardized the book’s API implementations on FastAPI; this chapter’s three pagination endpoints use its typed `Query` parameters (with `ge/le` bounds enforcing page-size ceilings) and its `TestClient` for the offline self-check.

Alternatives. Flask (simpler, but validation is hand-rolled); Django REST Framework (heavier, ORM-centric); a stdlib `http.server` implementation (fine for demos, no validation ecosystem).

Installation. `pip install "fastapi>=0.110"` — pulls `starlette` and `pydantic`; add `uvicorn` only if you want to serve it live (`pip install uvicorn`).

9.10.2 pydantic (pydantic>=2.7)

Description. Data-validation library [Col24] powering FastAPI’s parameter coercion and the natural home for typed cursor-payload models in larger codebases.

Why included. It is a FastAPI transitive dependency and the contract-validation layer established in Chapter 4; this chapter’s service relies on its `Query(..., ge=1, le=200)` coercion to reject bad page parameters before handler code runs.

Alternatives. `dataclasses` plus manual coercion (stdlib, more code); `attrs` with validators; `marshmallow`.

Installation. `pip install "pydantic>=2.7"` — installed automatically with FastAPI; pin explicitly because v1/v2 semantics differ.

9.10.3 httpx (httpx>=0.27)

Description. Modern HTTP client for Python, sync and async.

Why included. FastAPI’s `TestClient` is built on `httpx`; the offline self-check in Listing 9.2 and the infinite-scroll client in Capstone 4 both speak `httpx`. It also matches Chapter 2’s client-side toolchain.

Alternatives. `requests` (sync-only, ubiquitous); `urllib` (stdlib, verbose); `aiohttp` (async, different API).

Installation. `pip install "httpx>=0.27"`.

9.10.4 pytest (pytest>=8)

Description. The standard Python test runner and assertion ecosystem.

Why included. Capstones 2–5 specify `pytest` suites (property tests for the filter compiler, checkpoint and isolation tests for the capstone services); it is the curriculum-wide test framework validated by every track's `pytest -q` gate.

Alternatives. `unittest` (stdlib, no fixtures/plugins); `nose2` (legacy); `hypothesis` as a *complement* for property-based testing if your track permits the extra dependency.

Installation. `pip install "pytest>=8"`.

9.11 Chapter Summary & Key Takeaways

- **Position is fragile; value is stable.** Offset pagination costs $O(k)$ per page at depth k and drifts (duplicates/skips) under concurrent writes; keyset pagination costs $O(\log n) + O(p)$ at any depth and anchors on boundary values immune to writes elsewhere in the ordering.
- **The tuple predicate is the contract.** `WHERE (sort_col, id) > (?, ?)` with a composite index in matching column order turns page n into the same B-tree seek as page 1. Verify with `EXPLAIN: SEARCH ... USING INDEX`, never a temp B-tree sort.
- **Cursors are capabilities.** Opaque, HMAC-signed, versioned, expiring tokens, verified constant-time before parsing, rejected with a uniform 400. `Link` headers (RFC 8288) make navigation hypermedia-driven and strategy-migration possible.
- **Counting is not free.** Default to `has_more (LIMIT p+1)`; make `total_count` opt-in, estimated, or out-of-band — never a per-page $O(n)$ tax on every client.
- **Never let the client speak SQL.** Allow-listed grammar $\rightarrow$ typed AST $\rightarrow$ bound parameters, with public names decoupled from physical columns; direction tokens parsed, not interpolated.
- **Determinism is enforced, not hoped for.** Every ordering ends in a unique tie-breaker; `NULL` placement is stated explicitly; page sizes are capped server-side.
- **Projection and expansion are budget tools.** Sparse fieldsets

shrink payloads and unlock covering indexes; expansion requires DataLoader-style batching or the $N+1$ problem moves to your API boundary.

9.12 Quick-Reference Card

Strategy Comparison Matrix

Cursor Design Checklist

1. Payload: version field, expiry timestamp, boundary values — nothing else.
2. Integrity: HMAC-SHA256 (truncated), verified with `compare_digest` *before* parsing.
3. Failures: malformed, forged, expired, wrong-version $\Rightarrow$ one uniform 400 problem-details response.
4. Key management: secret from environment/secrets manager; never logged; rotation plan via key version.
5. Contract: `next_cursor` in body *and* `Link: rel="next"`; filter/sort state bound to the cursor (resent or hashed).
6. Payload fields re-bound as SQL parameters — signature $\neq$ trust.

Filter Grammar (Reference Implementation)

```
1 filter    := clause (";" clause)*
2 clause    := field ":" op ":" value
3 field     := "sku" | "name" | "category" | "cost" | "
   weight" | "active"
4 op        := "eq" | "ne" | "gt" | "gte" | "lt" | "lte" |
   "in" | "contains"
5 value     := scalar | scalar ("," scalar)*      # comma
   list for "in"
6 sort      := ["-"] field ("," ["-"] field)*    # "-" =
   descending
7 fields    := field ("," field)*                # sparse
   fieldset
8 rules     : unknown field/op -> 400; values coerced to
   declared types;
9           contains escapes LIKE wildcards; sort always
           appends ", id";
```

SQL emitted with bound parameters only.

9.13 Standardized Evidence Addendum

9.13.1 Benchmark Snapshot Template

9.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

9.13.3 Cost and Latency Budget Snapshot

9.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

9.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

9.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

9.17 Cross-Chapter Decision Card

9.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

9.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

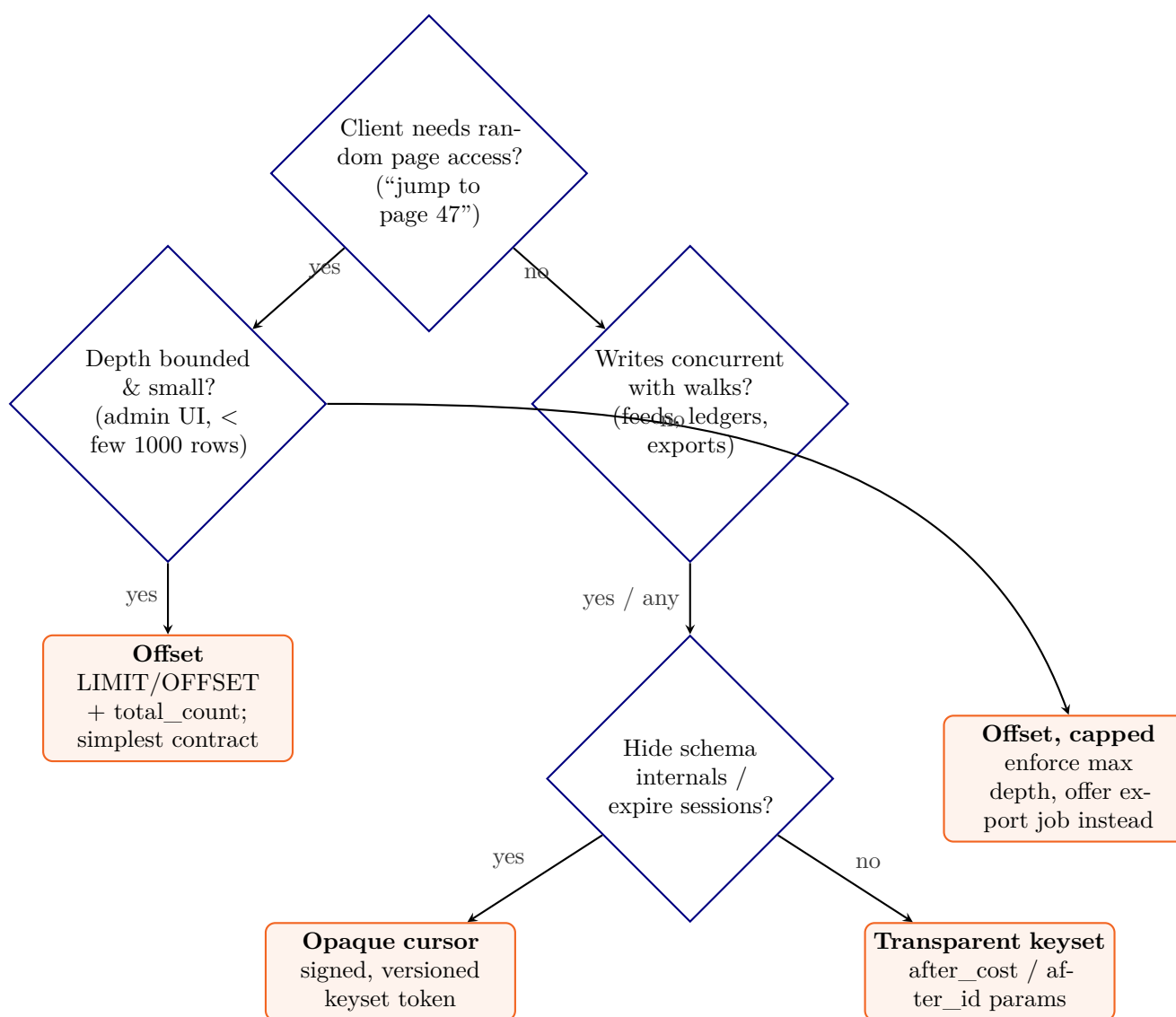


Figure 9.2: Pagination-strategy decision tree. Random access is the only property offset provides that keyset cannot; it is also the property that costs $O(k)$ and drifts under writes.

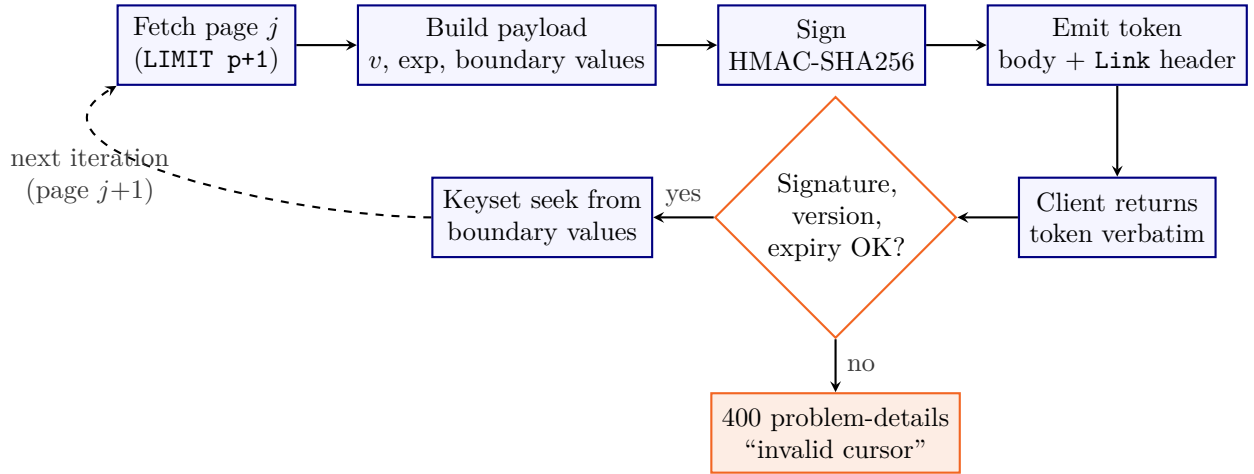


Figure 9.3: Cursor lifecycle: minting (top row) and redemption (bottom row). Verification precedes any use of payload fields; the one-extra-row fetch (LIMIT $p+1$) computes `has_more` without a `COUNT(*)` query.

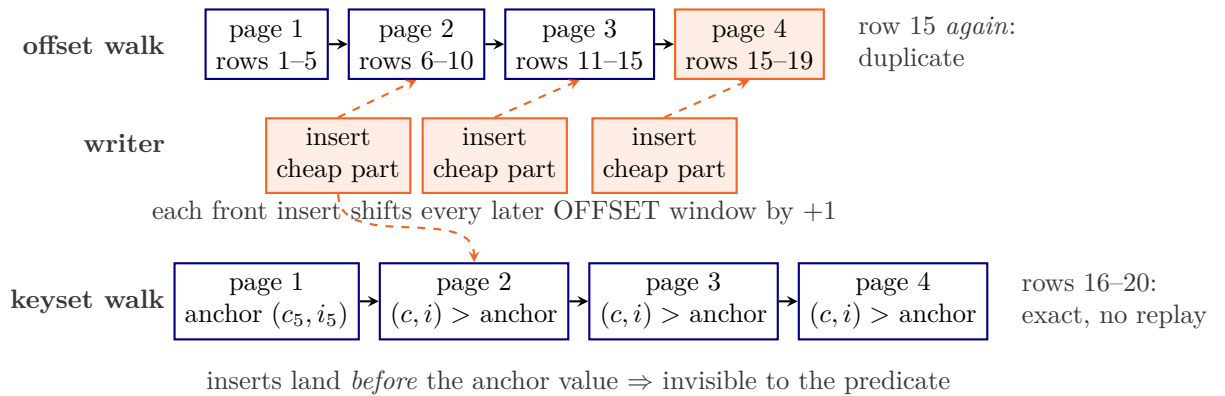


Figure 9.4: Write-concurrency anomaly timeline. Above: offset windows are positional, so each front-of-ordering insert replays one row into the next page. Below: keyset windows are anchored on boundary *values*; writes ahead of the anchor never enter the predicate. Compare Listing 9.5’s measured output.

Strategy	Cost at depth k	Write-stable?	Random access?	Use when
Offset (LIMIT/OFFSET)	$O(k)$	No — duplicates/skips	Yes	Small collections; admin grids; capped depth
Keyset (seek)	$O(\log n) + O(p)$	Yes	No	Large collections; exports; feeds; internal APIs.
Opaque cursor	$O(\log n) + O(p)$	Yes	No	Public APIs: schema hiding, expiry, tamper evidence

Table 9.2: Pagination strategies at a glance (p = page size, n = rows).

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 9.3: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 9.4: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 9.5: Decision card to complete per chapter.

Chapter 10

API Versioning & Evolution Strategies

An API is a promise. The moment a single external consumer integrates against your endpoints, every field name, every enum value, every error code, and every default becomes a load-bearing part of someone else’s production system — and you can no longer change any of it unilaterally without a plan. Yet APIs *must* change: businesses launch new products, regulations demand new fields, performance work restructures aggregates, and yesterday’s design decisions become today’s constraints. The central question of this chapter is therefore not *whether* to change an API, but *how to change it without breaking the consumers who trusted you*. Versioning is the discipline that answers that question; evolution strategy is the engineering program that makes the discipline affordable.

We begin by formalizing the vocabulary. A *breaking change* is defined precisely — not by intuition but by the existence of a formerly valid client interaction that the new contract rejects, mishandles, or renders uninterpretable — and we classify the everyday catalog of changes (removing or renaming fields, tightening validation, changing types or semantics, adding required fields, adding enum values, changing URLs) against that definition. We examine the robustness principle (Postel’s law) as a design posture for APIs, and just as carefully examine its limits, because blind tolerance fossilizes bugs into the contract. We then compare the four dominant versioning mechanisms — URI path, query parameter, custom header, and **Accept** media-type negotiation — with a rigorous trade-off matrix covering caches, gateways, documentation, client pinning, and hypermedia, and we study what Stripe, GitHub, and Google actually do in production and the reasoning behind each choice [GH21].

Mechanics follow policy. We study the expand-and-contract (parallel change) pattern as the canonical zero-downtime migration shape, adapter and anti-corruption layers that translate between version surfaces, dual-

serving architectures that run n public versions over one service core, consumer migration campaigns, and the usage telemetry that tells you when a version is finally safe to retire. Deprecation is treated as an engineering discipline with wire-level artifacts: the **Deprecation** and **Sunset** response headers of RFC 8594, updated by RFC 9745 [Wil19], **Link** relations that point at successor versions, sunset timelines, contractual stability SLAs, SemVer discipline for API contracts, and changelogs that consumers can actually plan against. Schema evolution gets its own rulebook — additive-only changes, never reusing names, enum extensibility strategies, and the sharply different unknown-field tolerance rules of JSON versus Protocol Buffers (Chapter 4) [Goo24] — and we close the loop with compatibility testing in CI: OpenAPI diffing in the style of `oasdiff` as a breaking-change gate that fails the build before a bad contract ever ships [Ope21].

The code is deliberately production-shaped. You will build a dual-serving FastAPI catalog API in which v1 is a pure adapter over the v2 domain model and carries live **Deprecation**/**Sunset** headers; a stdlib-plus-YAML OpenAPI diff gate that classifies changes as breaking or non-breaking and exits non-zero on breakage; a media-type version negotiator with correct 406 behavior; a consumer-tolerance test suite that *proves* old clients survive additive change; and a retirement-readiness report generator that turns access logs into a decommission decision. Everything is offline, deterministic, typed Python 3.11, set in the fictional Northwind Robotics parts-catalog universe.

Key Concept

A **version** is not a number on a URL; it is a *supported contract* with a lifecycle: introduced, current, deprecated, sunset, retired. Breaking changes demand a new version; non-breaking changes ride the existing one. The cheapest version is the one you never had to mint — prefer additive, tolerant-reader evolution and reserve explicit versions for changes that are genuinely incompatible. Retirement, not introduction, is the expensive part of the lifecycle, so instrument usage from day one.

10.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Classify any concrete API change — field removal, rename, type change, semantic change, validation tightening, required-field addition, enum-value addition, error-code change, URL

change — as breaking or non-breaking, and defend the classification with the formal definition of a breaking change, including the subtle enum-addition caveat.

- LO-2.** State the robustness principle (Postel’s law) as it applies to API design, apply tolerant-reader and additive-only disciplines, and articulate the principle’s failure modes: bug fossilization, ambiguity amplification, and security surface growth.
- LO-3.** Compare URI path, query-parameter, custom-header, and **Accept** media-type versioning across caching, gateway routing, documentation, client pinning, hypermedia, and debuggability; explain why Stripe uses date-based versions, why GitHub versions via a header, and why Google versions in the URI [\[GH21\]](#).
- LO-4.** Execute an expand-and-contract migration end to end: deploy the new contract alongside the old, dual-serve through an adapter layer, migrate consumers with telemetry-driven campaigns, and contract by retiring the old version only when the data permits it.
- LO-5.** Engineer a deprecation policy: emit RFC 8594/RFC 9745 **Deprecation** and **Sunset** headers, publish sunset timelines and stability SLAs, maintain a SemVer-disciplined changelog, and select communication channels proportionate to blast radius [\[Wil19\]](#).
- LO-6.** Apply schema-evolution rules — additive-only changes, never reusing names, enum extensibility with fallback values, unknown-field tolerance — and contrast JSON’s name-based tolerance with Protocol Buffers’ tag-based rules [\[Goo24\]](#).
- LO-7.** Build, in typed Python 3.11, a dual-serving FastAPI application with a v1-to-v2 adapter layer, an OpenAPI breaking-change CI gate, a media-type version negotiator, a consumer-tolerance test suite, and a retirement-readiness report generator [\[Ram24; Ope21\]](#).
- LO-8.** Design a version-support policy at scale: decide how many versions to run concurrently, quantify the per-version carrying cost, and differentiate policy for internal versus external APIs.

Prerequisites

This chapter assumes you have completed:

- **Chapter 0** — environment setup: Python 3.11+, a virtual environment, and the ability to run `pip` and `pytest` from PowerShell.
- **Chapter 1** — the HTTP wire protocol: methods, status codes, and header fields, especially `Accept`, `Content-Type`, `Link`, and cache-related headers [Fie22a].
- **Chapter 2** — consuming APIs from Python: parsing JSON responses and handling HTTP errors.
- **Chapter 3** — the REST architectural style: resources, representations, and hypermedia, which inform the URI-versus-media-type debate.
- **Chapter 4** — serialization formats and data contracts: JSON Schema, Protocol Buffers tag rules, content negotiation, and the backward/forward/full compatibility vocabulary we extend here [Goo24].
- **Chapter 5** — client-side security fundamentals, and **Chapter 6** — building APIs with FastAPI: routing, Pydantic models, and `TestClient` [Ram24].

No network access, cloud account, or API key is required anywhere in this chapter: every fixture is synthetic, embedded, and deterministic, set in the fictional *Northwind Robotics* parts and logistics universe.

Warning

Versioning policy is a **business commitment** encoded in engineering artifacts. A `Sunset` header, a changelog entry, and a support SLA are promises your consumers will plan around. Do not publish a deprecation date you are not prepared to honor, and do not retire a version while consumers you have not contacted still depend on it. The code in this chapter is simple; the discipline around it is the actual subject.

10.2 Deep Technical Mechanics & Architectural Concepts

10.2.1 A Taxonomy of Change

Every versioning decision reduces to one question: *does this change break an existing consumer?* Answering it honestly requires a definition precise enough to classify borderline cases, not a vibe.

Definition 10.1 (Breaking change). Let C be an API contract and $\mathcal{K}(C)$ the set of client interactions (request plus response interpretation) that are valid and succeed under C . A change producing contract C' is *breaking* if and only if there exists at least one interaction $k \in \mathcal{K}(C)$ such that under C' , k is rejected, fails, or returns a response that a client written against C cannot interpret with its existing parsing and handling logic. A change with no such k is *non-breaking* (compatible).

Definition 10.2 (Compatibility modes). A new contract (or schema) version C' is *backward compatible* with C if consumers written against C work unchanged against C' (new server, old clients). C' is *forward compatible* with C if consumers written against C' can meaningfully process data produced under C (old readers, new data). A change that is both is *fully compatible*. Web API evolution is overwhelmingly concerned with backward compatibility; durable data formats (Chapter 4) must care about both directions.

With Definition 10.1 in hand, the everyday catalog of changes classifies cleanly — with two notorious edge cases that we flag explicitly.

The enum-addition caveat. Adding a value to an enumeration is the most underestimated breaking change in API design. Producers think of enums as closed sets they own; consumers frequently compile them into closed `switch/match` statements, strict deserializers, or database check constraints. When Northwind Robotics adds `battery` to the catalog `category` enum, every mobile client whose generated model rejects unknown enum values does not degrade — it *crashes*, typically inside its JSON decoding layer, on an otherwise perfectly valid response. The mitigation is contractual: document enums as *open* (extensible) from day one, require clients to define fallback behavior for unknown values, and provide a sentinel such as `unknown/other` in the published schema. Protocol Buffers codifies exactly this posture: unrecognized enum values are preserved and surfaced as unknown values rather than rejected [Goo24]. If your existing consumer base already treats the enum as closed, adding a value *is* a breaking change and belongs in a new version.

10.2.2 The Robustness Principle and Its Limits

Postel’s law — *be conservative in what you send, be liberal in what you accept* — translates into two concrete API disciplines. On the producer side, **conservative sending** means additive-only evolution, stable field names, documented-open enums, and never reusing a retired name for a new meaning. On the consumer side, **liberal acceptance** means the *tolerant reader*: ignore fields you do not recognize, do not fail on unexpected enum values, do not depend on object member ordering, and treat absence of an optional field as a normal case. A consumer base built on tolerant readers makes most additive producer changes free, which is why the tolerant-reader posture is the single highest-leverage compatibility investment an ecosystem can make.

The principle has real limits, and a principal engineer must know them. First, **bug fossilization**: every quirk you silently tolerate — the misspelled field you alias, the malformed date you repair — becomes a de facto contract you can never remove, because some consumer now depends on your tolerance. Second, **ambiguity amplification**: accepting **price** as either a string or a number forces every consumer to guess which semantics apply, multiplying interoperability bugs rather than preventing them. Third, **security surface**: liberal parsers are historically fertile ground for injection, confusion, and denial-of-service attacks; validate strictly at the trust boundary (Chapter 5) and be liberal only about *ignoring* the unrecognized, never about *accepting* the malformed. The mature reading of Postel is: strict about structure and safety, tolerant about extensibility.

10.2.3 Versioning Strategies: A Rigorous Comparison

When a change is genuinely breaking, you must mint a new version, and the version identifier must live somewhere on the wire. Four placements dominate practice; Figure 10.1 shows the selection flow they share.

URI path versioning. The version is a path segment: `/v1/catalog/WR-1042`. Strengths: maximally visible and debuggable (the version is obvious in every log line and browser address bar); routing is trivial for any gateway; per-version cache keys emerge naturally because the URI is the primary cache key in HTTP [Fie22a]; client pinning is effortless. Weaknesses: it violates the REST ideal that a URI identifies a *resource*, not a representation of it — `/v1/catalog/WR-1042` and `/v2/catalog/WR-1042` are the same part wearing two addresses; links embedded in stored data (bookmarks, database columns, emails) pin consumers to versions; hypermedia payloads must be regenerated per version.

Query-parameter versioning. `/catalog/WR-1042?version=2`. Largely the URI strategy with worse ergonomics: caches *do* key on query strings, so cacheability survives, but intermediaries and logging pipelines more often normalize or drop query parameters than path segments, and the version reads as optional decoration rather than structural commitment.

Custom header versioning. `X-API-Version: 2`. Keeps URIs stable and pure, but suffers at the edges: browser debugging is awkward, many gateways require explicit configuration to route on custom headers, documentation tooling rarely models it, and — critically — naive caches keyed only on URI will serve a v1 body to a v2 request unless the server emits **Vary: X-API-Version** and every intermediary honors it [Fie22a]. Cache poisoning across versions is a real, observed failure mode (Section 10.4).

Media-type (Accept header) versioning. `Accept: application/vnd.northwind.catalog.version=2`. This is the most REST-orthodox placement: the URI identifies the resource; the version is a property of the negotiated *representation*, exactly as content negotiation intends (Chapter 4) [Fie22a]. It composes naturally with hypermedia and permits resource-by-resource versioning granularity. Its costs are the sharpest: the highest complexity floor for consumers, near-invisibility in logs and browsers, gateway routing that must parse media-type parameters, the same **Vary: Accept** cache requirement, and documentation tooling that assumes path-based APIs. Listing 9.3 implements a correct negotiator, including the 406 response for unsupported versions.

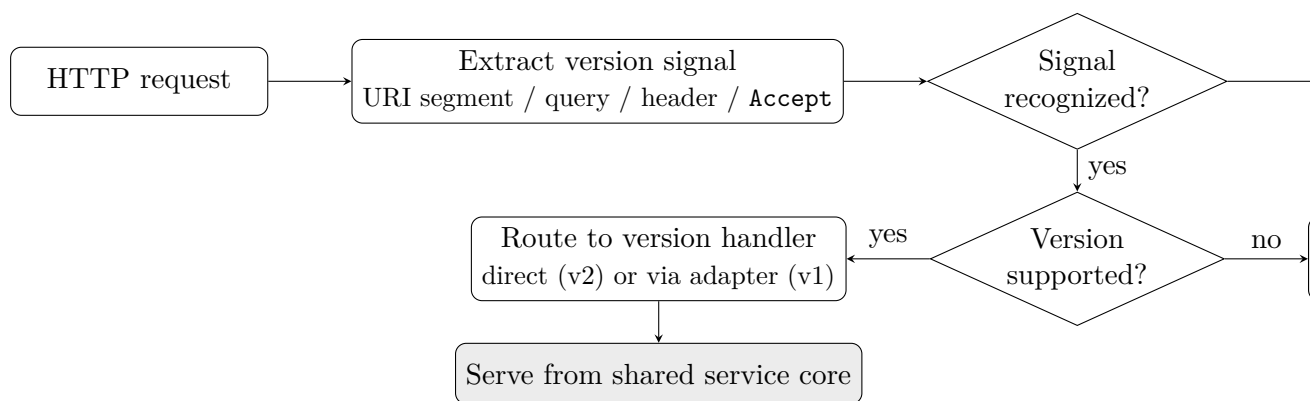


Figure 10.1: Version-selection routing flow. An unrecognized *signal* and an unsupported *version* are different failures; both surface as 406 (or 404 for unknown paths) with pointers to supported versions.

What major providers do, and why. **Stripe** versions by *date*: every account is pinned to a version like 2024-06-20, sent per request via the **Stripe-Version** header, and each response is rendered through a cascade of backward-compatibility transformation layers from current internals down to the caller's pinned date. The payoff is that a consumer's integration *never changes underneath them* unless they explicitly upgrade, while Stripe ships continuous internal change. **GitHub** versions via a media type plus an **X-GitHub-API-Version** date header, keeping resource URIs stable across versions — the REST-orthodox choice, suited to a hypermedia-rich API. **Google** versions in the URI (/v1/, /v2/) and codifies the policy in its API Improvement Proposals: discoverability, gateway simplicity, and cache clarity outweigh URI purity at Google's operational scale [GH21]. The pattern to internalize is not *which* mechanism won but *why*: each provider chose the placement whose costs its consumer base and infrastructure could bear, and each pairs the mechanism with a strict non-breaking-change policy so that new versions are rare events.

10.2.4 Evolution Mechanics: Expand-and-Contract and Dual-Serving

The **expand-and-contract** pattern (also called *parallel change*) is the canonical zero-downtime migration shape for any shared contract, from database schemas [Kle17] to public APIs. Its discipline is that the system passes through an intermediate state in which *both* the old and new contracts are simultaneously valid, so no consumer ever observes a discontinuity. The pattern has three phases, shown in Figure 10.2:

1. **Expand.** Add the new contract (endpoint version, field, or representation) alongside the old one. Nothing is removed. Old consumers are unaffected by construction; new consumers can opt in.
2. **Migrate.** Move consumers from old to new, at their pace but on your schedule: announce deprecation, publish a migration guide, instrument usage, and run a migration campaign (Section 10.2.4).
3. **Contract.** When telemetry shows the old surface is unused (or below a contractual threshold with stragglers individually engaged), retire it. Only now is the old code deleted.

Adapters and anti-corruption layers. Dual-serving does *not* mean maintaining two implementations of the business logic. The architecture that keeps it affordable is one service core expressed in the *newest* domain model, with each older public version implemented as a thin **adapter** — a

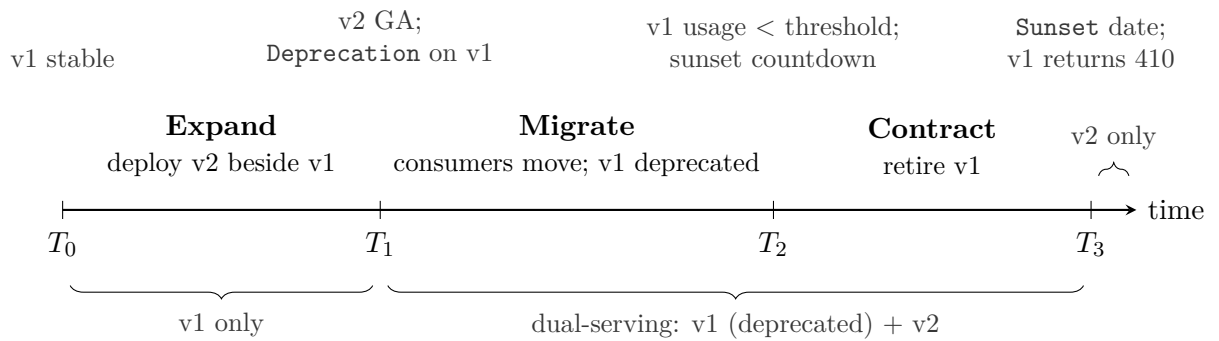


Figure 10.2: Expand-and-contract migration timeline. The dual-serving window between T_1 and T_3 is where all consumer risk is absorbed; contraction before the window closes is what causes outages.

pure translation layer mapping between the old wire contract and the current model. The adapter is an anti-corruption layer in the domain-driven-design sense: it prevents legacy vocabulary (`price` in float dollars, `in_stock` as a boolean) from leaking into the current model (`unit_price_cents`, `stock count`). Adapters must be pure, total, and tested as contracts: every field in the old surface must be derivable from the new model, and the test suite (Listing 9.4) must prove old clients cannot observe the translation. When a field *cannot* be faithfully derived — say the v2 model drops a concept v1 exposed — that is the signal that retirement of v1, not another adapter hack, is the honest path. Figure 10.3 shows the serving architecture; Listing 9.1 implements it.

Consumer migration campaigns. A deprecation announcement is not a migration. A migration campaign is a managed program with an owner, a consumer registry, and a deadline. Identify consumers from telemetry (API keys, OAuth client IDs, IP ranges as a last resort); segment by traffic and criticality; contact the long tail individually — an email to a consumer doing 40% of your v1 traffic is worth more than a banner seen by everyone; publish a migration guide with a field-level mapping table and copy-pasteable diffs; offer a feedback channel and a published escalation path; and schedule brownouts (short, announced windows in which the old version returns errors) as a forcing function for consumers who do not read email. Brownouts convert silent dependency into visible, survivable failure while there is still time to act.

Usage telemetry as the retirement oracle. The contract phase can only begin when you can *prove* who still depends on the old version. Log, at minimum: version, endpoint, consumer identity, and timestamp on every

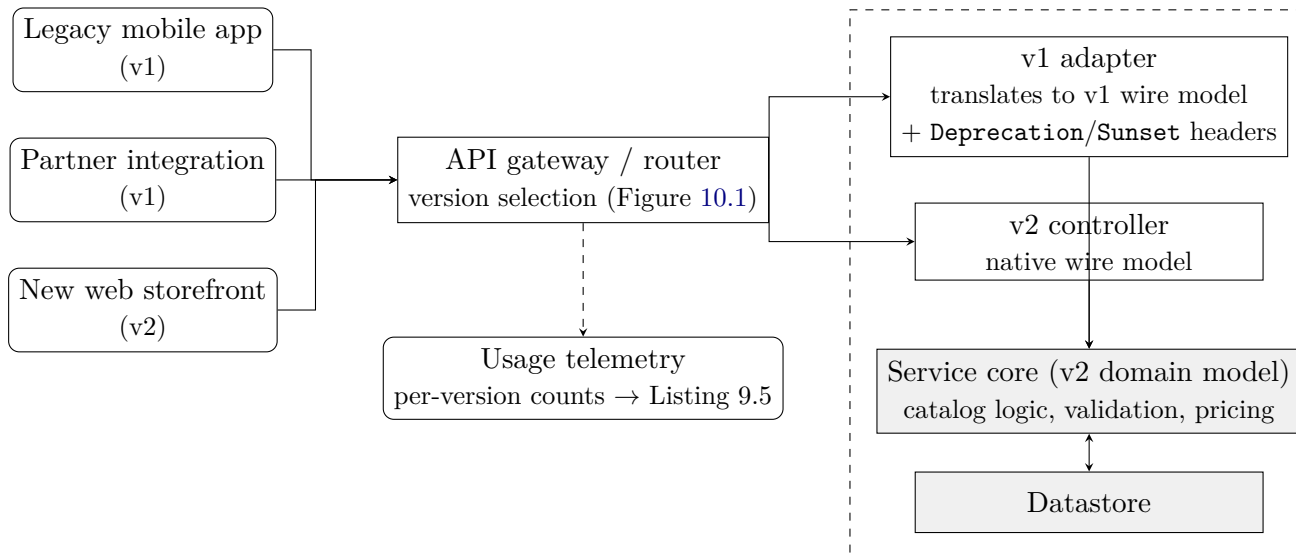


Figure 10.3: Dual-serving architecture. Both public versions are surfaces over one service core; v1 is a pure adapter with deprecation metadata, and the gateway feeds per-version telemetry that later justifies retirement.

request. From that stream compute per-version request share, distinct-consumer counts, and trend. Listing 9.5 turns such a log into a retirement-readiness verdict. A common threshold policy: schedule sunset when the old version drops below 5% of traffic *and* every remaining consumer has been contacted; execute sunset only when remaining consumers have migrated or formally accepted the cutoff.

10.2.5 Deprecation Policy Engineering

Deprecation is a state, not a deletion. A deprecated version still works; it is no longer recommended, receives no enhancements, and has a published end-of-life. The wire-level vocabulary is standardized. RFC 8594 introduced the **Deprecation** and **Sunset** response header fields [Wil19]; RFC 9745 (2025) subsequently revised **Deprecation**, replacing the original HTTP-date value with a Unix-timestamp form `@<seconds>`, while **Sunset** retains an HTTP-date. A well-behaved deprecated endpoint therefore answers like Listing 9.1’s v1 surface:

- **Deprecation:** `@1788220800` — the moment deprecation began (RFC 9745 form; RFC 8594-era deployments used an HTTP-date here).
- **Sunset:** `Sat, 01 May 2027 00:00:00 GMT` — the committed moment the version becomes unresponsive.

- **Link:** `<https://api.example/v2/catalog>; rel="successor-version"` and `rel="deprecation"` pointing at the migration guide — machine-followable pointers, not just prose.

Around those headers sits the policy. **Sunset timelines** must be generous and public: twelve months is a common floor for external APIs with managed consumers; six may suffice for a fast-moving partner API; internal APIs can contract in weeks. **Stability SLAs** put the promise in the contract: “no breaking changes within a major version,” “deprecated versions receive security fixes until sunset,” “at least 12 months between deprecation and sunset.” **SemVer for APIs** gives the changelog its grammar: MAJOR for breaking changes (a new version), MINOR for backward-compatible additions (new field, new endpoint, new *documented open* enum value), PATCH for compatible fixes. **Changelog discipline** means every change is dated, classified by that grammar, and written for the consumer (“*field X added to resource Y*”), not the implementer. **Communication channels** scale with blast radius: headers and changelogs for the attentive, developer-portal banners and newsletters for the reachable, direct contact for the identified, brownouts for the rest.

10.2.6 Schema Evolution Rules

Within a version, the schema may only move in compatible directions. The rulebook:

1. **Additive only.** New fields, new endpoints, new optional parameters. Never remove, never rename, never retype within a version.
2. **Never reuse names.** A retired name is reserved forever; reintroducing `price` with new semantics corrupts every stored payload and cached response that still carries the old meaning. Protocol Buffers encodes this as `reserved` field numbers and names — adopt the same discipline in JSON contracts even though the format cannot enforce it [Goo24].
3. **Enums are open by contract.** Publish them as extensible, ship a sentinel value, and require client fallback behavior. Adding a value to an enum documented as closed is a breaking change.
4. **Unknown fields are tolerated, not errors.** This is where the wire format matters. In JSON, tolerance is a *client discipline*: nothing in the format stops a strict deserializer from rejecting unknown members, so the contract must ask clients to ignore them (and generated models must be configured to do so). In Protocol Buffers, tolerance is a

format property: unknown tags are skipped by the wire parser itself, and since proto3 they are even retained on reserialize, so a proxy can pass through fields it does not understand [Goo24]. JSON gives you tolerance by convention; Protobuf gives it to you by construction (Chapter 4).

Compatibility testing in CI. Rules without enforcement are wishes. The enforcement mechanism is a breaking-change gate in the pipeline: diff the proposed OpenAPI description against the published one, classify every delta, and fail the build on any breaking change that is not accompanied by a version bump [Ope21]. This is what tools such as `oasdiff` automate; Listing 9.2 builds a minimal but real gate with `stdlib` plus `PyYAML` so the classification logic is fully inspectable. The gate belongs on the pull request, where the cost of the conversation is lowest — discovering a break in code review costs minutes; discovering it from a consumer’s pager costs trust.

10.2.7 Version Policy at Scale

Every concurrently supported version carries a recurring cost: adapter code and its tests, a documentation surface, a compatibility CI lane, on-call knowledge, and the combinatorial test matrix against every other moving part of the system. A useful mental model:

$$C_{\text{carry}}(n) = n \cdot c_{\text{adapter}} + c_{\text{test}} \cdot \binom{n}{2} + c_{\text{docs}} \cdot n + c_{\text{incident}}(n) \quad (10.1)$$

where n is the number of live versions. The quadratic term — pairwise interaction testing — is why “just keep every version forever” collapses operationally long before it collapses financially. Practical policies: **two live versions** (current plus previous) for most external APIs; **three** for platforms with slow-moving enterprise consumers, with the third on an accelerated sunset; date-based schemes like Stripe’s bound n by time rather than count (“all versions younger than 24 months”), which caps cost automatically. **Internal APIs** may adopt much shorter windows — weeks to months — because consumers are enumerable, reachable, and often deployable by the same organization; coordinated expand-and-contract with a two-week dual-serving window is routine. **External APIs** must assume unenumerable consumers, cached copies of old documentation, and contracts you cannot see; timelines stretch to quarters or years, and retirement requires the full campaign machinery of Section 10.2.4. The universal rule: the number of versions you support is a product decision with an engineering budget, not an engineering convenience.

10.3 Production-Ready Code Implementation

All listings are typed Python 3.11, offline, deterministic, and set in the Northwind Robotics catalog universe. Install the dependencies first (Section 10.10), place each listing in a working directory such as `C:\Training\ch09`, and run from PowerShell.

10.3.1 Listing 9.1 — Dual-Serving Catalog API (v1 + v2 over One Core)

One service core holds the v2 domain model; the v1 surface is a pure adapter with live `Deprecation`, `Sunset`, and `Link` headers. Running the module executes a deterministic demonstration against FastAPI's `TestClient` — no server, no network [Ram24].

```

1  """Dual-serving Catalog API (v1+v2) for Northwind
   Robotics.
2
3  Chapter 9, Listing 9.1 -- one v2 domain model serves
   two public versions.
4  The v1 surface is a pure adapter over the v2 model and
   carries RFC 8594 /
5  RFC 9745 deprecation metadata. Offline: run the module
   for a deterministic
6  demonstration via FastAPI's TestClient.
7  """
8
9  from __future__ import annotations
10
11 import enum
12 from datetime import datetime, timezone
13
14 from fastapi import FastAPI, Request, Response
15 from fastapi.responses import JSONResponse
16 from fastapi.testclient import TestClient
17 from pydantic import BaseModel, Field
18
19 # --- Deprecation policy constants (dates are fixed =>
   deterministic output) ---
20 DEPRECATED_AT = datetime(2026, 9, 1, tzinfo=timezone.
   utc)
21 SUNSET_AT = datetime(2027, 5, 1, tzinfo=timezone.utc)
22 DEPRECATION_HEADER = f"@{int(DEPRECATED_AT.timestamp())
   }" # RFC 9745

```

```
23 SUNSET_HEADER = "Sat, 01 May 2027 00:00:00 GMT"
                # RFC 8594
24 MIGRATION_GUIDE = "https://docs.northwind-robotics.
    example/catalog/migration"
25
26
27 # --- Single source of truth: the v2 domain model
    -----
28 class Category(str, enum.Enum):
29     """Open enum: consumers MUST tolerate unknown
        values (fallback rule)."""
30
31     ACTUATOR = "actuator"
32     SENSOR = "sensor"
33     CONTROLLER = "controller"
34     BATTERY = "battery" # added mid-lifecycle; safe
        only because enum is open
35
36
37 class PartV2(BaseModel):
38     """Current (v2) domain model. All business logic
        speaks only this type."""
39
40     part_id: str
41     sku: str
42     name: str
43     unit_price_cents: int = Field(ge=0)
44     currency: str = "USD"
45     category: Category
46     stock: int = Field(ge=0)
47     tags: list[str] = []
48     status: str = "active"
49
50
51 class PartV1(BaseModel):
52     """Legacy (v1) wire contract, frozen forever: float
        -dollar price, bool stock."""
53
54     id: str
55     sku: str
56     name: str
57     price: float
58     category: str
59     in_stock: bool
60
61
```



```

62 CATALOG: dict[str, PartV2] = {
63     p.part_id: p
64     for p in [
65         PartV2(part_id="WR-1042", sku="SRV-T450", name=
66             "Torque_Servo_T450",
67             unit_price_cents=12999, category=
68                 Category.ACTUATOR, stock=34,
69                 tags=["servo", "high-torque"]),
70         PartV2(part_id="WR-2210", sku="LDR-M90", name="
71             Lidar_Module_M90",
72             unit_price_cents=45900, category=
73                 Category.SENSOR, stock=0,
74                 tags=["lidar", "navigation"]),
75         PartV2(part_id="WR-3307", sku="CTL-AX12", name=
76             "Axis_Controller_AX12",
77             unit_price_cents=8900, category=Category
78                 .CONTROLLER, stock=112,
79                 tags=["controller", "can-bus"]),
80         PartV2(part_id="WR-4100", sku="BAT-LP38", name=
81             "LiPo_Pack_LP38",
82             unit_price_cents=15450, category=
83                 Category.BATTERY, stock=57,
84                 tags=["battery", "38wh"]),
85     ]
86 }
87
88 # --- Anti-corruption layer: v2 domain model -> frozen
89 # v1 wire contract -----
90
91 def to_v1(part: PartV2) -> PartV1:
92     """Pure, total translation: every v1 field is
93         derivable from the v2 model."""
94
95     return PartV1(
96         id=part.part_id,
97         sku=part.sku,
98         name=part.name,
99         price=round(part.unit_price_cents / 100, 2),
100         category=part.category.value,
101         in_stock=part.stock > 0,
102     )
103
104 def lookup(part_id: str) -> PartV2 | None:
105     return CATALOG.get(part_id.upper())

```

```

98
99 app = FastAPI(title="Northwind_Robotics_Catalog_API",
100               version="2.0.0")
101
102 @app.middleware("http")
103 async def deprecation_middleware(request: Request,
104                                call_next) -> Response:
105     """Stamp RFC 8594/9745 lifecycle metadata on every
106         v1 response."""
107
108     response: Response = await call_next(request)
109     if request.url.path.startswith("/v1/"):
110         response.headers["Deprecation"] =
111             DEPRECATION_HEADER
112         response.headers["Sunset"] = SUNSET_HEADER
113         response.headers["Link"] = (
114             f'<{MIGRATION_GUIDE}>;rel="deprecation", '
115             f'</v2/catalog>;rel="successor-version"'
116         )
117     return response
118
119 def not_found(version: str, part_id: str) ->
120     JSONResponse:
121     return JSONResponse(
122         status_code=404,
123         content={"type": "about:blank", "title": "Part
124                 not found",
125                 "status": 404, "detail": f"no part '{
126                 part_id}' in {version}"},
127     )
128
129 # --- v1 surface (deprecated): adapter over the core
130 -----
131 @app.get("/v1/catalog", response_model=list[PartV1])
132 def list_catalog_v1() -> list[PartV1]:
133     return [to_v1(p) for p in CATALOG.values()]
134
135 @app.get("/v1/catalog/{part_id}", response_model=PartV1)
136 def get_part_v1(part_id: str) -> PartV1 | JSONResponse:
137     part = lookup(part_id)
138     return to_v1(part) if part else not_found("v1",

```

```

    part_id)
135
136
137 # --- v2 surface (current): native domain model
    -----
138 @app.get("/v2/catalog", response_model=list[PartV2])
139 def list_catalog_v2() -> list[PartV2]:
140     return list(CATALOG.values())
141
142
143 @app.get("/v2/catalog/{part_id}", response_model=PartV2
    )
144 def get_part_v2(part_id: str) -> PartV2 | JSONResponse:
145     part = lookup(part_id)
146     return part if part else not_found("v2", part_id)
147
148
149 def demo() -> None:
150     client = TestClient(app)
151     r1 = client.get("/v1/catalog/WR-1042")
152     print("==_v1_(deprecated)_==")
153     print("Deprecation:", r1.headers.get("Deprecation")
    )
154     print("Sunset:_____", r1.headers.get("Sunset"))
155     print("Link:_____", r1.headers.get("Link"))
156     print("Body:_____", r1.json())
157     r2 = client.get("/v2/catalog/WR-1042")
158     print("\n==_v2_(current)_==")
159     print("Deprecation:", r2.headers.get("Deprecation")
    )
160     print("Body:_____", r2.json())
161     r3 = client.get("/v2/catalog/NOPE-1")
162     print("\n==_unknown_part_==", r3.status_code, r3.
        json()["detail"])
163
164
165 if __name__ == "__main__":
166     demo()

```

10.3.2 Listing 9.2 — OpenAPI Breaking-Change Gate (CI in a Box)

The gate diffs two OpenAPI 3.x YAML descriptions, classifies every delta against the taxonomy of Table 10.1, prints a report, and exits non-zero on any breaking change — the exact behavior you wire into a pull-request

pipeline [Ope21]. It deliberately handles the high-signal subset (paths, operations, request bodies, response schemas) and assumes inline schemas; resolving `$ref` is Capstone 3.

```
1  """OpenAPI_diff_gate.py (Chapter 9, Listing 9.2).
2
3  Classifies the delta between an OLD and a NEW OpenAPI
4  3.x YAML spec as
5  BREAKING or NON-BREAKING and exits non-zero when any
6  breaking change exists.
7  Stdlib + PyYAML only. Inline schemas assumed (no $ref
8  resolution).
9
10 Usage (PowerShell):
11 python openapi_diff_gate.py catalog_v1.yaml
12    catalog_v2.yaml
13 """
14
15 from __future__ import annotations
16
17 import argparse
18 import enum
19 import sys
20 from dataclasses import dataclass
21 from pathlib import Path
22 from typing import Any
23
24 import yaml
25
26 HTTP_METHODS = frozenset(
27     {"get", "put", "post", "delete", "patch", "head", "
28     options", "trace"}
29 )
30
31 class Severity(enum.Enum):
32     BREAKING = "BREAKING"
33     NON_BREAKING = "NON-BREAKING"
34
35 @dataclass(frozen=True)
36 class Change:
37     severity: Severity
38     location: str
39     message: str
```

```

37
38     def render(self) -> str:
39         return f"[{self.severity.value:>12}] {self.
40             location}: {self.message}"
41
42 def _operations(path_item: dict[str, Any]) -> dict[str,
43     Any]:
44     return {m: op for m, op in path_item.items() if m.
45         lower() in HTTP_METHODS}
46
47 def _diff_schema(
48     old: dict[str, Any],
49     new: dict[str, Any],
50     location: str,
51     direction: str, # "request" or "response"
52     changes: list[Change],
53 ) -> None:
54     old_props: dict[str, Any] = old.get("properties",
55         {}) or {}
56     new_props: dict[str, Any] = new.get("properties",
57         {}) or {}
58     new_required = set(new.get("required", []) or [])
59
60     for name in old_props:
61         if name not in new_props:
62             changes.append(Change(
63                 Severity.BREAKING, location,
64                 f"property '{name}' removed from {
65                     direction} schema"))
66
67     for name, new_prop in new_props.items():
68         if name not in old_props:
69             if direction == "request" and name in
70                 new_required:
71                 changes.append(Change(
72                     Severity.BREAKING, location,
73                     f"required request property '{name}'
74                         added"))
75             else:
76                 changes.append(Change(
77                     Severity.NON_BREAKING, location,
78                     f"optional {direction} property '{
79                         name}' added"))
80         continue

```

```

74         old_type = old_props[name].get("type")
75         new_type = new_prop.get("type")
76         if old_type != new_type:
77             changes.append(Change(
78                 Severity.BREAKING, location,
79                 f"property_{name}'_type_changed_{old_type!r}->_{new_type!r}"))
80         elif old_type == "object":
81             _diff_schema(old_props[name], new_prop,
82                 f"{location}.{name}",
83                 direction, changes)
84         elif old_type == "array":
85             _diff_schema(old_props[name].get("items",
86                 {})) or {},
87                 new_prop.get("items", {}) or {},
88                 f"{location}.{name}[]",
89                 direction, changes)
90         old_enum = old_props[name].get("enum")
91         new_enum = new_prop.get("enum")
92         if old_enum and new_enum:
93             removed = set(old_enum) - set(new_enum)
94             if removed:
95                 changes.append(Change(
96                     Severity.BREAKING, location,
97                     f"enum_value(s)_removed_from_{name}':_{sorted(removed)}"))
98             added = set(new_enum) - set(old_enum)
99             if added:
100                 changes.append(Change(
101                     Severity.NON_BREAKING, location,
102                     f"enum_value(s)_added_to_{name}':_{sorted(added)}"
103                     "(safe_only_if_the_enum_is_documented_as_open)"))
104
105 def _json_schema(media: dict[str, Any]) -> dict[str, Any]:
106     content = media.get("content", {}) or {}
107     return (content.get("application/json", {}) or {}).get("schema", {}) or {}
108
109 def _diff_operation(
110     old: dict[str, Any],

```

```

110     new: dict[str, Any],
111     location: str,
112     changes: list[Change],
113 ) -> None:
114     old_body = _json_schema(old.get("requestBody", {}))
115     new_body = _json_schema(new.get("requestBody", {}))
116     if new_body and old_body:
117         _diff_schema(old_body, new_body, f"{location}_\nrequestBody",
118                     "request", changes)
119     elif new_body and not old_body and new_body.get("required"):
120         changes.append(Change(
121             Severity.BREAKING, location,
122             "request_body_with_required_properties_\nintroduced"))
123
124     old_responses: dict[str, Any] = old.get("responses", {}) or {}
125     new_responses: dict[str, Any] = new.get("responses", {}) or {}
126     for code in old_responses:
127         if code not in new_responses:
128             changes.append(Change(
129                 Severity.BREAKING, location,
130                 f"response_status_{code}'_removed"))
131     for code in new_responses:
132         if code not in old_responses:
133             changes.append(Change(
134                 Severity.NON_BREAKING, location,
135                 f"response_status_{code}'_added"))
136     for code in old_responses.keys() & new_responses.keys():
137         _diff_schema(_json_schema(old_responses[code]),
138                     _json_schema(new_responses[code]),
139                     f"{location}_response_{code}", "response", changes)
140
141
142 def diff_specs(old: dict[str, Any], new: dict[str, Any]) -> list[Change]:
143     changes: list[Change] = []
144     old_paths: dict[str, Any] = old.get("paths", {}) or {}

```

```

145     new_paths: dict[str, Any] = new.get("paths", {}) or
146         {}
147
148     for path in old_paths:
149         if path not in new_paths:
150             changes.append(Change(Severity.BREAKING,
151                                   path, "path_removed"))
152     for path in new_paths:
153         if path not in old_paths:
154             changes.append(Change(Severity.NON_BREAKING,
155                                   path, "path_added"))
156
157     for path in old_paths.keys() & new_paths.keys():
158         old_ops = _operations(old_paths[path])
159         new_ops = _operations(new_paths[path])
160         for method in old_ops:
161             if method not in new_ops:
162                 changes.append(Change(
163                     Severity.BREAKING, f"{method.upper()}_{path}",
164                     "operation_removed"))
165         for method in new_ops:
166             if method not in old_ops:
167                 changes.append(Change(
168                     Severity.NON_BREAKING, f"{method.upper()}_{path}",
169                     "operation_added"))
170         for method in old_ops.keys() & new_ops.keys():
171             _diff_operation(old_ops[method], new_ops[
172                             method],
173                             f"{method.upper()}_{path}",
174                             changes)
175
176     return changes
177
178
179 def main(argv: list[str] | None = None) -> int:
180     parser = argparse.ArgumentParser(description=
181         __doc__)
182     parser.add_argument("old_spec", type=Path)
183     parser.add_argument("new_spec", type=Path)
184     args = parser.parse_args(argv)
185
186     old = yaml.safe_load(args.old_spec.read_text(
187         encoding="utf-8"))
188     new = yaml.safe_load(args.new_spec.read_text(
189         encoding="utf-8"))

```



```

181     changes = diff_specs(old, new)
182
183     breaking = [c for c in changes if c.severity is
184                 Severity.BREAKING]
185     for change in changes:
186         print(change.render())
187     print(f"\n{len(changes)} change(s): {len(breaking)}
188           breaking, {
189           f'{len(changes)-len(breaking)} non-breaking
190           ")
191     if breaking:
192         print("GATE_RESULT: FAIL -- breaking changes
193               require a new version")
194         return 1
195     print("GATE_RESULT: PASS -- all changes are
196           backward compatible")
197     return 0
198
199 if __name__ == "__main__":
200     sys.exit(main())

```

The two fixture specs below describe the Catalog contract before and after the breaking v1-to-v2 redesign. Save them beside the gate.

```

1 # catalog_v1.yaml -- published contract (the baseline
2   the gate protects)
3 openapi: "3.1.0"
4 info:
5   title: Northwind Robotics Catalog API
6   version: "1.4.0"
7 paths:
8   /catalog:
9     get:
10       operationId: listCatalog
11       responses:
12         "200":
13           description: All parts
14           content:
15             application/json:
16               schema:
17                 type: object
18                 required: [items]
19                 properties:
20                   items:
21                     type: array
22                     items:

```

```
22         type: object
23         required: [id, sku, name, price,
24                   category, in_stock]
25         properties:
26             id: {type: string}
27             sku: {type: string}
28             name: {type: string}
29             price: {type: number}
30             category: {type: string, enum:
31                       [actuator, sensor,
32                        controller]}
33             in_stock: {type: boolean}
34 /catalog/{partId}:
35   get:
36     operationId: getPart
37     responses:
38       "200":
39         description: One part
40         content:
41           application/json:
42             schema:
43               type: object
44               required: [id, sku, name, price,
45                         category, in_stock]
46               properties:
47                 id: {type: string}
48                 sku: {type: string}
49                 name: {type: string}
50                 price: {type: number}
51                 category: {type: string, enum: [
52                           actuator, sensor, controller]}
53                 in_stock: {type: boolean}
54       "404":
55         description: Not found
```

```
1 # catalog_v2.yaml -- proposed contract (renames and
2   retypes => gate must FAIL)
3 openapi: "3.1.0"
4 info:
5   title: Northwind Robotics Catalog API
6   version: "2.0.0"
7 paths:
8   /catalog:
9     get:
10       operationId: listCatalog
11       responses:
```

```

11     "200":
12         description: All parts
13         content:
14             application/json:
15                 schema:
16                     type: object
17                     required: [items]
18                     properties:
19                         items:
20                             type: array
21                             items:
22                                 type: object
23                                 required: [part_id, sku, name,
24                                     unit_price_cents, category,
25                                     stock]
26                                 properties:
27                                     part_id: {type: string}
28                                     sku: {type: string}
29                                     name: {type: string}
30                                     unit_price_cents: {type:
31                                         integer}
32                                     currency: {type: string}
33                                     category: {type: string, enum:
34                                         [actuator, sensor,
35                                         controller, battery]}
36                                     stock: {type: integer}
37 /catalog/{partId}:
38     get:
39         operationId: getPart
40         responses:
41             "200":
42                 description: One part
43                 content:
44                     application/json:
45                         schema:
46                             type: object
47                             required: [part_id, sku, name,
48                                 unit_price_cents, category, stock]
49                             properties:
                                part_id: {type: string}
                                sku: {type: string}
                                name: {type: string}
                                unit_price_cents: {type: integer}
                                currency: {type: string}
                                category: {type: string, enum: [
                                    actuator, sensor, controller,

```

```

50         battery]]
51         stock: {type: integer}
52     "404":
53         description: Not found
54 /catalog/health:
55     get:
56         operationId: catalogHealth
57         responses:
58             "200":
59                 description: Liveness probe

```

10.3.3 Listing 9.3 — Media-Type Version Negotiator

A correct negotiator for `Accept: application/vnd.northwind.catalog+json; version=2`: it parses q-values and parameters per RFC 9110 [Fie22a], selects the best supported version, and yields 406 (with a Link to supported versions) when nothing matches.

```

1  """Media-type version negotiation (Chapter 9, Listing
2     9.3).
3
4  Parses Accept headers carrying vendor media types with
5  a 'version=N'
6  parameter, honors q-values, and returns None (-> HTTP
7  406) when no supported
8  version matches. Run the module for a deterministic
9  negotiation table plus a
10 TestClient demonstration. Stdlib + FastAPI.
11 """
12
13 from __future__ import annotations
14
15 from dataclasses import dataclass
16
17 from fastapi import FastAPI, Request
18 from fastapi.responses import JSONResponse
19 from fastapi.testclient import TestClient
20
21 VENDOR_TYPE = "application/vnd.northwind.catalog+json"
22 SUPPORTED_VERSIONS = (1, 2)
23 DEFAULT_VERSION = 2
24
25 PART_V1 = {"id": "WR-1042", "sku": "SRV-T450", "name":
26             "Torque Servo T450",
27             "price": 129.99, "category": "actuator", "
28             in_stock": True}

```

```

23 PART_V2 = {"part_id": "WR-1042", "sku": "SRV-T450", "
24           name": "Torque_Servo_T450",
25           "unit_price_cents": 12999, "currency": "USD"
26           ,
27           "category": "actuator", "stock": 34}
28
29 @dataclass(frozen=True)
30 class MediaRange:
31     media_type: str
32     q: float
33     version: int | None
34     order: int
35
36 def parse_accept(header: str | None) -> list[MediaRange
37 ]:
38     """Parse an Accept header into q-sorted media
39     ranges (stable on ties)."""
40
41     if not header:
42         return [MediaRange("*/*", 1.0, None, 0)]
43     ranges: list[MediaRange] = []
44     for order, raw in enumerate(header.split(", ")):
45         tokens = [t.strip() for t in raw.split(";")]
46         media_type = tokens[0].lower()
47         q, version = 1.0, None
48         for param in tokens[1:]:
49             if "=" not in param:
50                 continue
51             key, _, value = param.partition("=")
52             key, value = key.strip().lower(), value.
53                 strip().strip('\"')
54             if key == "q":
55                 try:
56                     q = float(value)
57                 except ValueError:
58                     q = 0.0
59             elif key == "version":
60                 try:
61                     version = int(value)
62                 except ValueError:
63                     version = None
64         ranges.append(MediaRange(media_type, q, version
65                                 , order))
66     acceptable = [r for r in ranges if r.q > 0.0]

```

```

63     return sorted(acceptable, key=lambda r: (-r.q, r.
64         order))
65
66 def negotiate(
67     header: str | None,
68     supported: tuple[int, ...] = SUPPORTED_VERSIONS,
69     default: int = DEFAULT_VERSION,
70 ) -> int | None:
71     """Return the best supported version for the header
72     , or None (-> 406).
73
74     A vendor range with an explicit version matches
75     only that version; a bare
76     vendor type, application/json, or */* maps to the
77     server default.
78
79     """
80
81     best_version: int | None = None
82     best_q = -1.0
83     for r in parse_accept(header):
84         if r.media_type == VENDOR_TYPE and r.version is
85             not None:
86             if r.version in supported and r.q > best_q:
87                 best_version, best_q = r.version, r.q
88         elif r.media_type in (VENDOR_TYPE, "application
89             /json", "*/"):
90             if r.q > best_q:
91                 best_version, best_q = default, r.q
92     return best_version
93
94 app = FastAPI(title="Northwind Robotics Catalog (media-
95     type versioned)")
96
97 @app.get("/catalog/{part_id}")
98 def get_part(part_id: str, request: Request) ->
99     JSONResponse:
100     version = negotiate(request.headers.get("accept"))
101     if version is None:
102         return JSONResponse(
103             status_code=406,
104             content={"type": "about:blank", "title": "
105                 Not Acceptable",
106                 "status": 406,

```

```

100         "detail": "supported:_" + ",_".
101         join(
102             f"{VENDOR_TYPE};_version={v}"
103             for v in SUPPORTED_VERSIONS)),
104         headers={"Link": f'</catalog>;_rel="
105                 versions"',
106                 },
107     )
108     body = PART_V1 if version == 1 else PART_V2
109     return JsonResponse(
110         content=body,
111         media_type=f"{VENDOR_TYPE};_version={version}",
112         headers={"Vary": "Accept"},
113     )
114
115 def demo() -> None:
116     cases = [
117         None,
118         "application/json",
119         "application/vnd.northwind.catalog+json;_
120         version=1",
121         "application/vnd.northwind.catalog+json;_
122         version=2",
123         "application/vnd.northwind.catalog+json;_
124         version=1;q=0.4,_",
125         "application/vnd.northwind.catalog+json;_
126         version=2;q=0.9",
127         "application/vnd.northwind.catalog+json;_
128         version=3",
129         "application/xml",
130     ]
131     print(f"{'Accept_header':<78}>_version")
132     for header in cases:
133         print(f"{'{str(header):<78}>_{negotiate(header)}
134               }")
135     client = TestClient(app)
136     r = client.get("/catalog/WR-1042",
137                   headers={"Accept": f"{VENDOR_TYPE};_
138                             version=3"})
139     print("\n406_case:", r.status_code, r.json()["
140           detail"])
141
142 if __name__ == "__main__":
143     demo()

```

10.3.4 Listing 9.4 — Consumer-Tolerance Test Suite

Compatibility is a property you *test*, not *assert*. This pytest suite plays a frozen v1-era client against the live dual-serving app of Listing 9.1 and proves the tolerant-reader guarantees: unknown fields are ignored, additive changes are invisible, new enum values degrade to a fallback instead of crashing, and the v1 surface keeps its promised shape plus its deprecation metadata.

```

1  """Consumer-tolerance contract tests (Chapter 9,
   Listing 9.4).
2
3  Proves that a frozen v1-era client keeps working while
   the API evolves:
4  unknown-field tolerance, additive changes, open-enum
   fallback, and the
5  stability of the deprecated v1 surface. Run: pytest -q
6  """
7
8  from __future__ import annotations
9
10 import pytest
11 from fastapi.testclient import TestClient
12
13 from catalog_api import app
14
15 # A v1-era client knows exactly these fields and
   nothing else.
16 LEGACY_FIELDS = frozenset({"id", "sku", "name", "price",
   "category", "in_stock"})
17 LEGACY_CATEGORIES = frozenset({"actuator", "sensor", "
   controller"})
18
19
20 def legacy_parse_part(payload: dict) -> dict:
21     """Tolerant reader: extract known fields, ignore
   everything else."""
22
23     return {k: payload[k] for k in LEGACY_FIELDS &
   payload.keys()}
24
25
26 def legacy_category(value: str) -> str:
27     """Open-enum fallback rule documented in the v1
   contract."""
28
29     return value if value in LEGACY_CATEGORIES else "
```



```

30         unknown"
31
32 @pytest.fixture(scope="module")
33 def client() -> TestClient:
34     return TestClient(app)
35
36
37 def test_v1_surface_keeps_every_promised_field(client:
TestClient) -> None:
38     response = client.get("/v1/catalog")
39     assert response.status_code == 200
40     parts = response.json()
41     assert parts, "fixture_catalog_must_not_be_empty"
42     for part in parts:
43         assert LEGACY_FIELDS <= set(part), f"v1_
contract_violated:{part}"
44         assert isinstance(part["price"], float)
45         assert isinstance(part["in_stock"], bool)
46
47
48 def test_unknown_fields_are_ignored_by_tolerant_reader
() -> None:
49     future_payload = {
50         "id": "WR-9000", "sku": "NXT-1", "name": "Next-
Gen_Servo",
51         "price": 42.5, "category": "actuator", "
in_stock": True,
52         # Fields a future version might add:
53         "unit_price_cents": 4250, "stock": 7, "tags": [
"servo"],
54         "dimensions": {"w_mm": 40, "h_mm": 20}, "
carbon_footprint_g": 812,
55     }
56     parsed = legacy_parse_part(future_payload)
57     assert parsed == {
58         "id": "WR-9000", "sku": "NXT-1", "name": "Next-
Gen_Servo",
59         "price": 42.5, "category": "actuator", "
in_stock": True,
60     }
61
62
63 def test_new_enum_value_degrades_to_fallback_not_crash
() -> None:
64     # 'battery' was added after this client shipped;

```

```

        fallback keeps it alive.
65     assert legacy_category("battery") == "unknown"
66     assert legacy_category("actuator") == "actuator"
67
68
69 def test_v2_additive_fields_do_not_leak_into_v1(client:
    TestClient) -> None:
70     v1 = client.get("/v1/catalog/WR-1042").json()
71     v2 = client.get("/v2/catalog/WR-1042").json()
72     assert set(v1) == LEGACY_FIELDS
73     assert {"unit_price_cents", "stock", "currency", "
        tags"} <= set(v2)
74     # Adapter fidelity: v1 values are derivable from
        the v2 model.
75     assert v1["price"] == round(v2["unit_price_cents"]
        / 100, 2)
76     assert v1["in_stock"] == (v2["stock"] > 0)
77
78
79 def test_v1_carries_deprecation_metadata_v2_does_not(
    client: TestClient) -> None:
80     v1 = client.get("/v1/catalog/WR-1042")
81     assert v1.headers["Deprecation"].startswith("@")
82     assert "Sunset" in v1.headers
83     assert 'rel="successor-version"' in v1.headers["
        Link"]
84     v2 = client.get("/v2/catalog/WR-1042")
85     assert "Deprecation" not in v2.headers
86     assert "Sunset" not in v2.headers

```

10.3.5 Listing 9.5 — Retirement-Readiness Report Generator

Retirement is a data-driven decision. This tool consumes a JSON-lines access-log fixture (gateway logs exported offline), computes per-version traffic share, distinct consumers, and monthly trend, and emits a verdict against a threshold policy. The embedded fixture shows v1 declining toward the 5% retirement line.

```

1  """Retirement-readiness_report_generator_(Chapter_9,
    Listing_9.5).
2
3  Reads_a_JSON-lines_access_log_(one_event_per_line:uts,
    path,_consumer)_and

```

```

4 reports_per_version_usage_share, distinct_consumers,
   and_monthly_trend, then
5 emits_a_retirement_verdict_against_the_chapter's
   threshold_policy.
6
7 Usage(PowerShell):
8 python retirement_report.py --demo #
   embedded_fixture
9 python retirement_report.py access_log.jsonl #
   your_own_export
10 """
11
12 from __future__ import annotations
13
14 import argparse
15 import json
16 from collections import defaultdict
17 from pathlib import Path
18
19 RETIRE_SHARE_THRESHOLD = 0.05 # retire when v1 < 5% of
   catalog traffic...
20
21 DEMO_LOG = """\
22 {"ts": "2026-05-04T09:13:00Z", "path": "/v1/catalog/WR
   -1042", "consumer": "partner-acme"}
23 {"ts": "2026-05-05T10:01:00Z", "path": "/v2/catalog", "
   consumer": "storefront-web"}
24 {"ts": "2026-05-06T11:40:00Z", "path": "/v2/catalog/WR
   -2210", "consumer": "storefront-web"}
25 {"ts": "2026-06-02T09:00:00Z", "path": "/v2/catalog", "
   consumer": "storefront-web"}
26 {"ts": "2026-06-03T09:05:00Z", "path": "/v2/catalog", "
   consumer": "partner-globex"}
27 {"ts": "2026-06-04T12:31:00Z", "path": "/v2/catalog/WR
   -4100", "consumer": "warehouse-wms"}
28 {"ts": "2026-06-05T15:10:00Z", "path": "/v2/catalog", "
   consumer": "storefront-web"}
29 {"ts": "2026-07-01T09:00:00Z", "path": "/v2/catalog", "
   consumer": "storefront-web"}
30 {"ts": "2026-07-02T09:10:00Z", "path": "/v2/catalog/WR
   -3307", "consumer": "partner-globex"}
31 {"ts": "2026-07-03T10:20:00Z", "path": "/v2/catalog", "
   consumer": "warehouse-wms"}
32 {"ts": "2026-07-04T11:00:00Z", "path": "/v2/catalog", "
   consumer": "mobile-ios-4.0"}
33 {"ts": "2026-07-05T13:45:00Z", "path": "/v2/catalog/WR

```

```

-1042",␣"consumer":␣"storefront-web"}
34 {"ts":␣"2026-08-02T08:30:00Z",␣"path":␣"/v2/catalog",␣"
  consumer":␣"storefront-web"}
35 {"ts":␣"2026-08-02T09:00:00Z",␣"path":␣"/v2/catalog",␣"
  consumer":␣"partner-globex"}
36 {"ts":␣"2026-08-03T09:30:00Z",␣"path":␣"/v2/catalog/WR
  -4100",␣"consumer":␣"warehouse-wms"}
37 {"ts":␣"2026-08-04T10:00:00Z",␣"path":␣"/v2/catalog",␣"
  consumer":␣"mobile-ios-4.0"}
38 {"ts":␣"2026-08-05T11:15:00Z",␣"path":␣"/v2/catalog/WR
  -2210",␣"consumer":␣"storefront-web"}
39 {"ts":␣"2026-08-06T12:00:00Z",␣"path":␣"/v2/catalog",␣"
  consumer":␣"partner-globex"}
40 {"ts":␣"2026-08-07T12:30:00Z",␣"path":␣"/v2/catalog/WR
  -3307",␣"consumer":␣"warehouse-wms"}
41 {"ts":␣"2026-08-08T13:00:00Z",␣"path":␣"/v2/catalog",␣"
  consumer":␣"mobile-ios-4.0"}
42 {"ts":␣"2026-08-09T14:20:00Z",␣"path":␣"/v2/catalog/WR
  -1042",␣"consumer":␣"storefront-web"}
43 ""
44
45
46 def path_version(path: str) -> str | None:
47     for prefix in ("/v1/", "/v2/"):
48         if path.startswith(prefix):
49             return prefix.strip("/")
50     return None
51
52
53 def build_report(lines: list[str]) -> str:
54     total: dict[str, int] = defaultdict(int)
55     consumers: dict[str, set[str]] = defaultdict(set)
56     monthly: dict[str, dict[str, int]] = defaultdict(
57         lambda: defaultdict(int))
58
59     for raw in lines:
60         raw = raw.strip()
61         if not raw:
62             continue
63         event = json.loads(raw)
64         version = path_version(str(event["path"]))
65         if version is None:
66             continue # unversioned traffic is out of
67             scope for this report
68         total[version] += 1
69         consumers[version].add(str(event["consumer"]))

```

```

68         monthly[str(event["ts"])[7:]]["version"] += 1
69
70     grand_total = sum(total.values())
71     if grand_total == 0:
72         return "No versioned traffic found; nothing to report."
73
74     out = ["RETIREMENT-READINESS_REPORT--NorthwindRoboticsCatalogAPI",
75           "=" * 62, f"Total versioned requests: {grand_total}", ""]
76     for version in sorted(total):
77         share = total[version] / grand_total
78         out.append(f"{version}: {total[version]:>4} requests"
79                   f"({share:6.1%}), {len(consumers[version])} consumer(s)"
80                   f"({sorted(consumers[version])}")
81     out += ["", "Monthly trend (v1 share):"]
82     for month in sorted(monthly):
83         m = monthly[month]
84         month_total = sum(m.values())
85         out.append(f"  {month}: v1={m.get('v1', 0):>2} v2={m.get('v2', 0):>2}"
86                   f"({v1_share:=m.get('v1', 0)/month_total:6.1%})")
87
88     months = sorted(monthly)
89     first = monthly[months[0]]
90     last = monthly[months[-1]]
91     first_share = first.get("v1", 0) / sum(first.values())
92     last_share = last.get("v1", 0) / sum(last.values())
93     declining = last_share <= first_share
94     v1_share = total.get("v1", 0) / grand_total
95
96     out += ["", "Verdict:"]
97     if v1_share < RETIRE_SHARE_THRESHOLD and declining:
98         out.append(f"  v1 share {v1_share:.1%} < {RETIRE_SHARE_THRESHOLD:.0%}"
99                   f"and declining: SCHEDULE SUNSET BROWNOUTS, confirm the"
100                   f"remaining consumer(s) {sorted(consumers.get('v1', []))}"
101                   f"then retire on the published date."

```

```

102     else:
103         out.append(f"v1_share_{v1_share:.1%}_(
104             declining={declining}):_KEEP_"
105             "v1_RUNNING;_continue_migration_"
106             "campaign_and_re-evaluate_"
107             "next_month.")
108     return "\n".join(out)
109
110 def main(argv: list[str] | None = None) -> int:
111     parser = argparse.ArgumentParser(description=
112         __doc__)
113     parser.add_argument("logfile", type=Path, nargs="?",
114         help="JSON-lines_access_log_
115             export")
116     parser.add_argument("--demo", action="store_true",
117         help="use_the_embedded_
118             deterministic_fixture")
119     args = parser.parse_args(argv)
120
121     if args.demo or args.logfile is None:
122         lines = DEMO_LOG.splitlines()
123     else:
124         lines = args.logfile.read_text(encoding="utf-8"
125             ).splitlines()
126     print(build_report(lines))
127     return 0
128
129 if __name__ == "__main__":
130     raise SystemExit(main())

```

10.4 Common Pitfalls & Anti-Patterns

Warning

Every anti-pattern below has caused a real production incident class. They are ordered roughly by frequency of appearance in design reviews.

1. Version-per-endpoint chaos. Letting individual endpoints evolve their own versions independently (`/v1/orders`, `/v3/customers`, `/v2/orders/items`) shatters the API into a combinatorial mess: no single document describes

the system, client SDKs must track a version matrix, and gateway routing becomes archaeology. Version the *API* (or at most the bounded context), not the endpoint. Media-type versioning is the one legitimate exception because its per-resource granularity is negotiated, not routed.

2. Silently changing semantics within a version. Renaming nothing, retyping nothing — just quietly redefining what `stock` means from “on hand” to “available to promise.” No client errors fire; every client silently computes wrong answers. Semantic drift is the most damaging break precisely because nothing detects it except business outcomes. The defense is contractual: field *meaning* is part of the contract, documented, and covered by consumer-tolerance tests with fixed fixtures.

3. Breaking changes via “harmless” renames. `price` becomes `unit_price_cents`; `id` becomes `part_id`. Each rename is internally justified (clarity! units in the name!) and externally a removal-plus-addition — a breaking change wearing a refactor’s clothes (Listing 9.2’s fixture pair demonstrates exactly this, and the gate fails the build). If a name must change, add the new name, keep serving the old one, and migrate.

4. Never retiring versions. Versions without sunsets accumulate into permanent carrying cost (Equation 10.1): every adapter, doc set, and CI lane lives forever, and eventually the team maintains contracts nobody remembers designing. A version without a deprecation plan is a mortgage with no amortization schedule.

5. Version in both URI and header. `/v1/catalog` with `X-API-Version: 2` — which wins? Every consumer picks a different answer, support tickets multiply, and the gateway must implement a precedence rule that no document defined. Choose exactly one version signal per API and reject requests that carry conflicting ones.

6. Ignoring cache poisoning across versions. With header- or media-type-based versioning, a shared cache keyed on URI alone will happily serve a v1 body to a v2 request. The mitigations are mandatory, not optional: emit `Vary: X-API-Version` (or `Vary: Accept`) on every versioned response and verify intermediaries honor it [Fie22a] — or use URI versioning and let the URI do the keying. Listing 9.3 emits `Vary: Accept` for this reason.

10.5 Hands-On Production Lab & Real-World Case Study

Lab: Evolve the Catalog API from v1 to v2

You own the Northwind Robotics Catalog API. The v1 contract (float-dollar `price`, boolean `in_stock`) cannot express multi-currency pricing or real stock counts, so a v2 contract has been designed. Execute the full expand-and-contract lifecycle.

Setup (PowerShell):

```
1 cd C:\Training\ch09
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 pip install -r requirements.txt    # see Section 9.11
```

1. **Expand.** Save Listing 9.1 as `catalog_api.py` and run `python catalog_api.py`. Observe that v1 and v2 both answer and that v1 responses carry `Deprecation`, `Sunset`, and `Link` headers while v2 responses do not.
2. **Gate the change.** Save Listing 9.2 as `openapi_diff_gate.py` plus the two fixture specs and run `python openapi_diff_gate.py catalog_v1.yaml catalog_v2.yaml`; confirm it reports the renames and retypes as `BREAKING` and exits 1 (`echo $LASTEXITCODE`). Then `diff catalog_v1.yaml` against itself and confirm the gate passes and exits 0 — this is the “no changes” CI lane.
3. **Prove consumer survival.** Save Listing 9.4 as `test_consumer_tolerance.py` and run `pytest -q`. All five tests must pass: they are the evidence that the expand phase harmed no one.
4. **Negotiate.** Save Listing 9.3 as `media_type_negotiation.py`, run it, and verify the negotiation table: explicit versions select their representation, bare types get the default, unsupported versions yield `None` (HTTP 406).
5. **Contract.** Save Listing 9.5 as `retirement_report.py` and run `python retirement_report.py -demo`. Read the verdict: v1 is below the 5% threshold and declining, with exactly one identified straggler — the campaign’s final contact list.

War Story: The “Harmless” Enum Addition

A robotics parts platform (the kind Northwind Robotics sells into) shipped a seemingly trivial release: the catalog `category` enum gained a new value, `battery`, to support a product-line launch. The API team ran their compatibility checks — no fields removed, no types changed, all endpoints identical — and deployed on a Tuesday. By Wednesday morning, crash dashboards for the two largest mobile consumers were vertical. Both apps had been built from generated models in which the enum was a *closed* native type; their JSON decoders threw on the unknown value, and because decoding happened inside the list parser, a single battery part anywhere in a page of results poisoned the entire response. The apps did not degrade — they died on launch screens that happened to render category chips.

The postmortem lessons are this chapter in miniature. (1) The producer’s compatibility model (additive = safe) did not match the consumers’ parsing model (closed enum = fatal), and under Definition 10.1 the change *was* breaking for that consumer base. (2) The enum had never been documented as open, so consumers reasonably treated it as closed. (3) The fix was not code but contract: the API now publishes every enum as extensible with an `unknown` sentinel, consumer SDKs regenerate with fallback arms, a consumer-tolerance fixture suite (the pattern of Listing 9.4) injects synthetic unknown enum values in CI, and genuinely closed enums can only change in new major versions. The enum addition that crashed the fleet is now the first slide of their API design onboarding.

10.6 Self-Assessment Exercises & Deep-Dive Questions

- E9.1. Classification drill.** Classify each as breaking or non-breaking, citing Definition 10.1: (a) adding an optional `warehouse` field to the part response; (b) making a previously optional request field required; (c) changing `price` from dollars to cents while renaming it `unit_price_cents`; (d) reordering JSON object members; (e) changing a 404 detail message; (f) adding 410 `Gone` as a *new* response for retired resources.
- E9.2. Extend the gate.** Listing 9.2 ignores path and query `parameters`. Extend it so that adding a *required* parameter or removing an existing parameter is flagged breaking. Add fixtures proving both directions.
- E9.3. q-value correctness.** Construct an `Accept` header in which a lower-q explicit version must lose to a higher-q wildcard,

and verify Listing 9.3’s `negotiate` returns the default. Then construct the reverse.

- E9.4. Adapter audit.** Propose a `v2` field that *cannot* be back-translated into the `v1` surface of Listing 9.1 (e.g., a multi-currency price list). What are the three honest options, and which does expand-and-contract prescribe?
- E9.5. Cache audit.** Your API uses `X-API-Version`. Write the exact response headers required for safe shared caching, and explain what breaks if a CDN ignores `Vary`.
- E9.6. Deep dive: closed ecosystems.** Your consumers are all mobile apps with six-month release laggards and generated strict models. Defend or reject media-type versioning for this base, and choose deprecation timelines accordingly.
- E9.7. Deep dive: the exception clause.** Your threat model team demands an immediate breaking change to close an auth bypass. Reconcile “never break within a version” with security response; define the policy you would publish *before* the incident.

10.7 Interview Q&A Vault

Junior (Q1–Q10)

- Q1. What is an API version?** A distinctly identified, supported contract between the API and its consumers — not just a URL prefix. Minting one declares that the contract changed incompatibly; supporting one commits you to its stability SLA.
- Q2. Define a breaking change.** Any change after which at least one formerly valid, successful client interaction is rejected, fails, or becomes uninterpretable by a client written against the old contract (Definition 10.1).
- Q3. Classify these five changes: removing a response field; adding an optional request field; renaming `price` to `unit_price`; adding a new endpoint; changing a field from float to integer.** Breaking; non-breaking; breaking (rename = remove + add); non-breaking; breaking.
- Q4. Is adding a response field breaking?** For tolerant readers, no — unknown members are ignored. For clients with strict schema

validation or closed generated models, it can be. Your contract must state which client posture you support.

- Q5. Why is URI versioning so popular despite being “un-RESTful”?** Operational clarity: visible in logs and browsers, trivially routed by gateways, naturally cache-keyed, and effortless for consumers to pin. Most organizations value those over URI purity.
- Q6. What does HTTP 406 mean in a versioned API?** Not Acceptable — the server cannot produce any representation the client’s `Accept` header permits, e.g. an unsupported `version=N` media-type parameter.
- Q7. What is the difference between deprecation and retirement?** Deprecated means still functional but discouraged, no longer enhanced, and scheduled for removal; retired (sunset) means the surface no longer responds (typically 410 Gone or a redirect).
- Q8. What are the Deprecation and Sunset headers?** Standardized response headers (RFC 8594, with `Deprecation` revised by RFC 9745) announcing when a surface was deprecated and when it will become unresponsive, so clients can detect lifecycle state programmatically [Wil19].
- Q9. Give two examples of non-breaking changes.** Adding an endpoint; adding an optional request field or a response field (for tolerant readers); adding an enum value to a documented-open enum; loosening validation.
- Q10. Why should you never reuse a retired field name?** Stored payloads, caches, and old clients still carry the old meaning; reusing the name silently reinterprets their data. Retired names are reserved forever — Protobuf enforces this with `reserved` [Goo24].

Mid (Q11–Q20)

- Q11. URI vs. header versioning — defend a choice for a public partner API.** URI path: partners debug with curl and browsers, gateways route on paths for free, caches key correctly without `Vary`, and pinning is obvious in code review. I would accept the REST-purity cost and document why. (Full credit answers argue either side with the trade-off matrix, not dogma.)

- Q12. How does Stripe’s date-based versioning work?** Accounts pin an API version by date; requests may override per call with a version header. Internally, responses are rendered from current code through a stack of backward-compatibility transforms down to the pinned date, so integrations never change underneath a consumer until they choose to upgrade.
- Q13. What is expand-and-contract (parallel change)?** A three-phase migration: *expand* (add the new contract beside the old, remove nothing), *migrate* (move consumers with telemetry and campaigns), *contract* (retire the old surface only when usage proves it safe). It keeps an intermediate state where both contracts are valid, so consumers never see a discontinuity [Kle17].
- Q14. What is an anti-corruption layer in versioned APIs?** An adapter that translates between an old public contract and the current domain model, preventing legacy vocabulary from leaking into new code. It must be pure, total, and contract-tested; fields that cannot be faithfully derived signal retirement, not more translation.
- Q15. How do you dual-serve two versions without duplicating business logic?** One service core owns the newest domain model; older versions are thin adapters over it; a gateway routes by version signal; telemetry on the router feeds the retirement decision (Figure 10.3).
- Q16. How do caches interact with header-based versioning?** Caches key primarily on URI; two versions sharing a URI poison each other unless responses carry **Vary** on the versioning header and every intermediary honors it [Fie22a]. URI versioning sidesteps the problem entirely.
- Q17. When may you break compatibility without minting a new version?** Only under pre-published exception clauses: critical security fixes, legally mandated changes, correcting behavior that contradicts the documented contract, or surfaces explicitly labeled unstable/beta. The clause must exist *before* it is invoked.
- Q18. Your consumer base can’t tolerate new enum values. What are your options?** Treat enum growth as breaking: mint a new version; or add a parallel string field with the extended value set and freeze the enum; or run a migration campaign to tolerant SDKs first, then grow the enum. What you may not do is add the

value and hope.

- Q19. What belongs in a migration guide?** A field-level old-to-new mapping table, behavioral deltas (semantics, errors, defaults), copy-pasteable request/response diffs, SDK upgrade steps, the sunset timeline, and a support channel.
- Q20. How does SemVer apply to APIs?** MAJOR = breaking contract change (new version); MINOR = backward-compatible addition (field, endpoint, open-enum value); PATCH = compatible fix. The changelog classifies every dated entry by this grammar.

Senior (Q21–Q27)

- Q21. Design a breaking-change CI gate.** Diff the proposed OpenAPI description against the published baseline; classify deltas (removed paths/operations/properties, type changes, new required request fields, removed enum values) as breaking; fail the pipeline unless the PR also bumps the major version. Keep the classifier inspectable (Listing 9.2) and treat warnings (enum additions) as human-review triggers [Ope21].
- Q22. How do you decide when to retire a version?** Telemetry, not calendar: schedule sunset when the old version falls below a traffic-share threshold (commonly $\sim 5\%$) *and* is trending down; execute only when every remaining consumer has migrated or formally accepted the cutoff. Listing 9.5 automates the verdict.
- Q23. Compare JSON and Protocol Buffers for schema evolution.** JSON tolerance is client discipline — the format cannot stop a strict deserializer from rejecting unknown members, so contracts must mandate tolerant readers. Protobuf tolerance is a wire property: unknown tags are skipped by the parser and retained on reserialize, and `reserved` makes name/tag reuse impossible [Goo24]. JSON evolves by convention; Protobuf by construction (Chapter 4).
- Q24. How many versions should you support concurrently?** As few as the consumer base permits: two (current + previous) is the common external policy; three for slow enterprise bases; date-based schemes cap by age instead of count. Each added version pays adapter, docs, CI, and pairwise-testing costs that grow superlinearly (Equation 10.1).
- Q25. How do internal and external API versioning policies**

differ? Internal consumers are enumerable and often deployable by your own org, so dual-serving windows of weeks and aggressive retirement are feasible. External consumers are unenumerable and uncontractable at scale, so timelines stretch to quarters/years, communication needs full campaign machinery, and brownouts replace direct outreach for the long tail.

Q26. What are brownouts and when are they justified? Scheduled, announced windows in which a deprecated surface returns errors, used late in a migration campaign to surface silent dependencies while rollback is still safe. Justified only after direct outreach, with clear notice, and never near the sunset date itself.

Q27. How do you handle a breaking change to a stored representation (e.g., cached payloads, event log) while versioning the API? Decouple wire version from storage version: persist in the newest schema with upcasting on read, serve old wire versions via adapters, and never let a retired wire contract dictate the storage format. This is the DDIA dual of expand-and-contract applied to data at rest [Kle17].

Staff (Q28–Q34)

Q28. Design a version policy for an organization with 200 internal APIs and 15 external ones. Federated guardrails: a single published policy defining breaking changes, mandatory CI diff gates on every spec repo, internal APIs capped at two versions with 30–90 day dual-serving windows enforced by platform tooling, external APIs on contractual 12-month sunsets with a consumer registry, and an exceptions board owning the security-break clause. Uniform telemetry format so retirement decisions are mechanical everywhere.

Q29. When is versioning the wrong tool? When the change can be expressed additively, when consumers are few and coordinated (migrate in place), or when the real problem is missing tolerant-reader discipline. Every version you can avoid minting saves its full carrying cost forever.

Q30. A principal argues for “versionless” APIs. Evaluate. Versionless means maximal additive discipline plus guaranteed tolerant readers — achievable in controlled ecosystems (internal, strong SDK governance) and effectively what Stripe simulates externally. It fails where you cannot govern client parsing behavior; there,

explicit versions are the honest acknowledgment of uncontrolled consumers.

- Q31. How do you price the cost of a version to justify retirement to leadership?** Quantify Equation 10.1 per version: adapter maintenance hours, CI lane minutes, documentation drift incidents, on-call context-switch cost, and the quadratic test matrix; then put the telemetry (consumers remaining, their traffic share) beside it. Retirement business cases write themselves when both numbers are honest.
- Q32. Design the deprecation-communication architecture for a platform with 50k API consumers.** Layered: wire-level (Deprecation/Sunset/Link headers on every response), artifact-level (changelog, migration guides, versioned docs), channel-level (portal banners, newsletter, status page), and direct (registry-driven outreach to identified top consumers, brownouts for the unresponsive tail). Each layer catches the consumers the previous one missed.
- Q33. Your gateway must route five versions of fifty endpoints at 100k RPS. What breaks first, and how do you design for it?** Per-version route tables and adapter latency compound; the answer is pushing version dispatch to cheap, cacheable rules (URI prefix routing at the edge), keeping adapters in-process and allocation-free on hot paths, and making version count a managed budget precisely because routing and translation cost scales with n .
- Q34. Write the policy for the security-exception clause.** Scope (severity-defined), authority (who may invoke), consumer notification (maximum feasible notice, out-of-band for identified consumers), compatibility duty of care (provide patched old-version endpoint where feasible), and post-incident retrospective updating the contract. A break invoked without this pre-published framework is indistinguishable from an accident.

10.8 External Bibliography & Further Reading

- RFC 8594, *The Sunset HTTP Header Field* (and its original Deprecation definition) — the wire-level deprecation vocabulary [Wil19].

- RFC 9745, *The Deprecation HTTP Header Field* (2025) — revises `Deprecation` to the `@<unix-timestamp>` form; read alongside RFC 8594.
- RFC 9110, *HTTP Semantics* — `Accept`, content negotiation, `Vary`, and cache-key behavior underpinning every versioning placement decision [Fie22a].
- OpenAPI Specification 3.1 — the contract description format our CI gate diffs [Ope21].
- Stripe, *API versioning documentation* and the engineering blog post *APIs as infrastructure: future-proofing Stripe with versioning* — the canonical date-based versioning write-up.
- J. Gehl, *API Design Patterns* — versioning and backward-compatibility chapters [GH21].
- M. Kleppmann, *Designing Data-Intensive Applications*, ch. 4 — schema evolution, rolling upgrades, and the theoretical basis of expand-and-contract [Kle17].
- Google API Improvement Proposals AIP-180 (*Backwards compatibility*) and AIP-185 (*API versioning*) — Google’s codified policy.
- Microsoft REST API Guidelines, *Versioning* section — a second large-scale policy to compare against Google’s.
- Protocol Buffers documentation, *Updating a Message Type* — tag reservation, unknown fields, and enum open/closed semantics [Goo24].
- The `oasdiff` project (Tufin) — production-grade OpenAPI diffing and breaking-change detection that Listing 9.2 miniaturizes.
- FastAPI documentation — routers, middleware, and `TestClient` used throughout the listings [Ram24].

10.9 Capstone Projects

Capstone 9.1 — Add v2 to an Existing API with an Adapter Layer [junior]

Objective: Take any single-version CRUD API (your Chapter 6 project qualifies) and evolve one resource through a breaking redesign without breaking v1 consumers.

Inputs/Datasets: The existing API; the synthetic Northwind Robotics catalog fixture from Listing 9.1 as the domain data.

Steps: (1) Define the v2 contract for the resource (at least one rename and one retype). (2) Implement the v2 model as the service core. (3) Write a pure `to_v1` adapter and mount both surfaces. (4) Add **Deprecation**, **Sunset**, and **Link** middleware on v1. (5) Write adapter-fidelity tests proving v1 values derive from v2.

Acceptance Criteria:

- ☐ v1 and v2 respond correctly for every seeded record.
- ☐ v1 responses carry all three lifecycle headers; v2 carries none.
- ☐ Adapter tests pass deterministically offline.
- ☐ No business logic is duplicated between surfaces.

Stretch Goals: Add a third, experimental version behind an opt-in header; measure adapter overhead with a fixed-iteration timing loop.

Capstone 9.2 — Media-Type Version Negotiation Middleware [mid]

Objective: Generalize Listing 9.3 into reusable FastAPI middleware that version-negotiates *every* endpoint of an API by vendor media type.

Inputs/Datasets: Listing 9.3; the dual-serving app of Listing 9.1 refactored to a single `/catalog` URI.

Steps: (1) Implement the negotiator as middleware that resolves the version and stashes it in request state. (2) Route representations through an adapter registry keyed by version. (3) Emit 406 with a machine-readable supported-versions body on failure. (4) Emit **Vary: Accept** on all versioned responses. (5) Test q-value precedence, unsupported versions, and default selection.

Acceptance Criteria:

- ☐ All negotiation outcomes match a published decision table.
- ☐ 406 bodies enumerate supported **version=N** values.
- ☐ **Vary: Accept** present on every 200.
- ☐ Full pytest suite passes offline.

Stretch Goals: Content-negotiate a second wire format (`+json` vs. `+cbor`-style suffix) orthogonally to version.

Capstone 9.3 — Production Breaking-Change CI Gate [mid]

Objective: Harden Listing 9.2 into a gate you would actually run on pull requests.

Inputs/Datasets: Listing 9.2; the two catalog fixture specs; three additional spec pairs you author (purely additive, parameter-level breaking, \$ref-based).

Steps: (1) Add \$ref resolution for local references. (2) Diff path/query parameters (new required or removed = breaking). (3) Classify enum growth as a *warning* tier between breaking and clean. (4) Emit a machine-readable JSON report alongside the text one. (5) Add a `-allow-breaking-with=VERSION_BUMP` escape hatch that passes only when `info.version`'s major component increased.

Acceptance Criteria:

- ☐ Exit code 1 exactly when breaking changes exist without a major bump.
- ☐ All five fixture pairs produce the expected verdicts.
- ☐ JSON report round-trips through `json.loads`.

Stretch Goals: Detect semantic-risk heuristics (format changes, pattern tightening, `additionalProperties: false` introductions).

Capstone 9.4 — Deprecation Campaign Plan & Retirement Analytics [senior]

Objective: Design and instrument the complete retirement program for a deprecated version, from announcement to 410.

Inputs/Datasets: Listing 9.5 and its log fixture, extended by you to 6 months and 12 synthetic consumers with realistic migration curves.

Steps: (1) Write the one-page deprecation policy (dates, SLAs, channels, brownout schedule). (2) Extend the report generator with per-consumer trend lines and a projected-zero-crossing date via linear fit. (3) Produce the consumer contact list ranked by dependency. (4) Script the brownout simulator: given the log, estimate which consumers would notice first. (5) Define the go/no-go checklist for executing sunset.

Acceptance Criteria:

- ☐ Policy document covers timeline, SLAs, channels, and exceptions.
- ☐ Report projects a retirement date and names remaining consumers deterministically.

- ☐ Go/no-go checklist is mechanical (every item checkable from telemetry).

Stretch Goals: Add a “what-if” mode showing how the projected date shifts if the top two consumers migrate next month.

Capstone 9.5 — Version Governance at Platform Scale [staff]

Objective: Design the versioning and retirement governance system for a fictitious platform org running 40 internal and 6 external APIs.

Inputs/Datasets: A synthetic portfolio registry you author (YAML: per-API versions, consumer counts, traffic shares, owners); this chapter’s cost model (Equation 10.1).

Steps: (1) Codify the policy: breaking-change definition, allowed strategies per audience, sunset floors, exception clauses. (2) Build a registry linter that flags policy violations (version in URI *and* header; third live version without waiver; deprecated version lacking a sunset date). (3) Compute the portfolio’s carrying cost under your model and identify the five most expensive versions. (4) Write the RFC-style decision record proposing your org-wide strategy mix. (5) Define the telemetry schema every API must emit.

Acceptance Criteria:

- ☐ Policy document is complete enough to lint mechanically.
- ☐ Linter flags at least the four violation classes above on a seeded registry.
- ☐ Cost report ranks versions and is reproducible offline.
- ☐ Decision record defends the strategy mix against at least two alternatives.

Stretch Goals: Simulate ten years of portfolio growth and show when the quadratic term of Equation 10.1 forces policy change.

10.10 Python Dependencies Appendix

All code runs offline after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

```
1 fastapi>=0.110
2 pydantic>=2.7
3 pyyaml>=6.0
4 jsonschema>=4.21
5 httpx>=0.27
6 pytest>=8
```

10.10.1 fastapi (≥ 0.110)

Description: The ASGI web framework used for every served API in this chapter. **Why included:** routing, Pydantic integration, middleware (deprecation headers), and `TestClient` for offline demonstrations [Ram24]. **Alternatives:** Flask + manual validation; Starlette directly (FastAPI's base). **Installation:** `pip install "fastapi>=0.110"`.

10.10.2 pydantic (≥ 2.7)

Description: Type-driven data validation and serialization library. **Why included:** defines the v1/v2 wire models and the domain model in Listing 9.1; its strict-by-default validation is the producer-side counterpart to the chapter's consumer tolerance rules. **Alternatives:** `dataclasses` + manual checks; `attrs`; `marshmallow`. **Installation:** `pip install "pydantic>=2.7"`.

10.10.3 pyyaml (≥ 6.0)

Description: YAML parser/emitter. **Why included:** the breaking-change gate (Listing 9.2) reads OpenAPI descriptions authored in YAML, the format real API teams use [Ope21]. **Alternatives:** `ruamel.yaml` (round-trip fidelity); parsing JSON-flavored OpenAPI with `stdlib json`. **Installation:** `pip install "pyyaml>=6.0"`.

10.10.4 jsonschema (≥ 4.21)

Description: JSON Schema validation engine. **Why included:** exercise and capstone work validating consumer fixtures against published schemas (E9.2, Capstone 9.3's semantic checks); not required by the printed listings. **Alternatives:** manual validation; Pydantic's schema export. **Installation:** `pip install "jsonschema>=4.21"`.

10.10.5 httpx (≥ 0.27)

Description: Sync/async HTTP client. **Why included:** the transport underneath FastAPI's `TestClient` — required for Listings 9.1, 9.3, and 9.4

to run offline. **Alternatives:** requests (synchronous, not ASGI-native); urllib (stdlib, low-level). **Installation:** `pip install "httpx>=0.27"`.

10.10.6 pytest (≥ 8)

Description: Test framework. **Why included:** runs the consumer-tolerance suite (Listing 9.4) and every capstone's acceptance checks. **Alternatives:** unittest (stdlib); nose2. **Installation:** `pip install "pytest>=8"`.

10.11 Chapter Summary & Key Takeaways

- A breaking change is defined by its victims: any change that invalidates a formerly successful client interaction. Renames, type changes, semantic drift, validation tightening, and required-field additions are all breaking; additions and loosening are not — except enum growth against closed consumer parsers.
- Postel's law, applied maturely, means strict about structure and safety, tolerant about extensibility. Tolerant readers make additive evolution free; blind tolerance fossilizes bugs into contracts.
- Four versioning placements — URI path, query parameter, custom header, media type — trade cache correctness, gateway simplicity, debuggability, and REST purity differently. Stripe (dates), GitHub (header + media type), and Google (URI) each chose rationally for their consumer base; none is universally right.
- Expand-and-contract is the only honest zero-downtime migration shape: add beside, migrate with telemetry, contract on evidence. Dual-serving keeps n surfaces affordable by putting one domain model behind pure, tested adapters.
- Deprecation is engineered, not announced: **Deprecation**/**Sunset**/**Link** headers (RFC 8594, revised by RFC 9745), published timelines, stability SLAs, SemVer changelogs, and layered communication [Wil19].
- Schema discipline is additive-only with reserved names and open enums; JSON tolerates by convention, Protobuf by construction [Goo24].
- Compatibility belongs in CI: an OpenAPI diff gate fails the build on breaking changes before consumers do.
- Every live version carries recurring, superlinear cost. Retirement is

the goal state; telemetry — not the calendar — grants permission to reach it.

10.12 Quick-Reference Card

Breaking-Change Checklist

Breaking: removed/renamed field • type or semantics change • tightened validation • new required request field • removed endpoint/operation • changed URL • changed error codes/shape • enum value *removed* • enum value *added to a documented-closed enum*. Non-breaking: added endpoint • added optional request field • added response field (tolerant readers) • loosened validation • added enum value to a documented-open enum.

Strategy Matrix (Condensed)

URI path — best ops ergonomics, worst REST purity • query param — URI-like, weaker tooling • custom header — pure URIs, fragile caches, needs Vary • media type — most REST-orthodox, highest consumer complexity, needs Vary: Accept. Pick one signal; never two.

Deprecation-Header Reference

Deprecation: @<unix-ts> (RFC 9745; RFC 8594 used HTTP-date) — when deprecation began. Sunset: <HTTP-date> (RFC 8594) — when the surface dies. Link: <...>; rel="deprecation" — policy document. rel="successor-version" — where to migrate. Lifecycle: introduced → current → deprecated → sunset → retired (410).

10.13 Standardized Evidence Addendum

10.13.1 Benchmark Snapshot Template

10.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.

- **Result:** what changed in correctness, resilience, latency, and operability.

10.13.3 Cost and Latency Budget Snapshot

10.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

10.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

10.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

10.17 Cross-Chapter Decision Card

10.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

10.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Change	Verdict	Why
Remove a response field	Breaking	Clients reading the field fail or misinterpret; violates the old contract's response shape.
Rename a field	Breaking	A rename is a removal plus an addition; the removal half breaks. "Harmless renames" are the most common accidental breakage.
Change a field's type	Breaking	<code>price</code> as float dollars becoming integer cents deserializes wrongly or fails validation on every old client.
Change field semantics	Breaking	Same name, same type, new meaning (e.g., <code>stock</code> switches from on-hand to available-to-promise): old clients silently compute wrong answers. Worst class of break — no error is raised.
Tighten validation	Breaking	Requests that succeeded now return 400. Loosening validation is non-breaking.
Add a required request field	Breaking	Old clients cannot supply it; their formerly valid requests are rejected.
Add an optional request field	Non-breaking	Old requests remain valid; the server defaults the new field.
Add a response field	Non-breaking*	Tolerant readers ignore it; strict-schema clients may not — see below.
Add an endpoint	Non-breaking	Nothing existing changed; new capability only.
Change a URL path	Breaking	Every hard-coded client reference 404s. URL <i>additions</i> are fine; URL <i>moves</i> break.
Change an error code/body	Breaking	Clients branch on status codes and problem-detail members; changing them breaks error-handling paths.
Add an enum value	Conditional	Breaking for clients that validate exhaustively or <code>switch</code> without a default arm; non-breaking for tolerant readers. See the caveat below.

Criteria	URL path	Query param	Custom header	Media type
Cache correctness	Excellent	Good	Fragile (needs Vary)	Fragile (needs Vary)
Gateway/routing	Trivial	Easy	Config required	Config required
Debuggability (browser, logs)	Excellent	Good	Poor	Poor
Client pinning difficulty	Trivial	Easy	Moderate	Hard
Hypermedia fit	Poor	Poor	Good	Excellent
Doc/tooling support	Excellent	Good	Fair	Fair
REST purity	Poor	Fair	Good	Excellent
Version granularity	Per API	Per API	Per API	Per resource

Table 10.2: Versioning strategy trade-off matrix.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 10.3: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 10.4: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 10.5: Decision card to complete per chapter.

Chapter 11

Idempotency, Caching & Concurrency Control

Networks fail in the middle. Not before your request, not after the response arrives, but precisely in the interval where neither party can tell whether the other side acted. Every production API lives inside that interval: a client that times out cannot distinguish *the server never received the request* from *the server executed it and the response was lost*. Three disciplines together turn that ambiguity from a source of corruption into an engineering property you can prove things about. *Idempotency* makes repeating an operation safe. *Caching* makes repeating a read cheap. *Concurrency control* makes simultaneous writes correct. This chapter treats all three as one coherent correctness program, because in production they are never exercised separately: the retry that saves your client from a timeout is the same mechanism that double-charges your customer when the endpoint is not idempotent, and the cache that absorbs your read traffic is the same mechanism that serves a stale balance when validation is sloppy.

We begin formally. Idempotency is defined as a fixed-point property on observable state — $f(f(x)) = f(x)$ — and we map that definition onto the HTTP method vocabulary of RFC 9110, separating what the specification guarantees (GET, PUT, DELETE) from what well-designed APIs *add* (idempotent POST via idempotency keys) [Fie22a]. The IETF Idempotency-Key draft is studied as a wire protocol with precise replay semantics: same key and same payload replays the stored response, same key and different payload is a 422, a concurrent duplicate in flight is a 409, and keys expire on a 24-hour window in the style Stripe popularized. We then dismantle the exactly-once myth: the Two Generals' Problem makes distributed exactly-once delivery impossible, and the achievable substitute — at-least-once delivery combined with idempotent processing — is what the literature calls *effectively-once* [Kle17].

The caching half of the chapter is a guided tour of RFC 9111: the

Cache-Control directive vocabulary (**max-age**, the widely confused **no-cache** versus **no-store**, **private** versus **public**, **stale-while-revalidate**, **stale-if-error**, **must-revalidate**), strong and weak validators, the **If-None-Match**/304 dance, heuristic freshness, surrogate keys for CDN purging, **Vary**, and cache poisoning through unkeyed inputs [Fie22a]. Concurrency control gets equal rigor: the lost-update problem is developed as a worked interleaving, optimistic locking with **If-Match**/ETag is given its precise status-code algebra (428, 412, and when 409 is the honest answer), pessimistic patterns are scoped to where they belong, and distributed locks are presented together with their failure modes and the fencing-token fix [Kle17; Nyg18].

The code is production-shaped and entirely offline. You will build a SQLite-backed **Idempotency-Key** middleware for FastAPI and prove, with fifty concurrent duplicate submits released on a single barrier, that exactly one mutation occurs; a conditional-requests demo that measures the bandwidth a 304 saves; an optimistic-locked inventory resource for the fictional Northwind Robotics warehouse; a double-submit storm harness that reproduces the classic double-charge incident and then cures it; and a pure stdlib cache-policy simulator that makes RFC 9111 directive semantics executable. Everything is typed Python 3.11, deterministic, and tested with in-process ASGI clients [Ram24; Col24].

Key Concept

A retry is a second attempt at a request whose first outcome is *unknown*. Idempotency converts “unknown” into “safe to repeat”: at-least-once delivery plus idempotent processing yields effectively-once semantics. Caching and concurrency control are the read-side and write-side halves of the same discipline: validators (ETag, Last-Modified) let a cache prove freshness with **If-None-Match**, and the same validators let a writer prove it is overwriting the state it actually read with **If-Match**. One mechanism — the validator — underwrites both.

11.1 Learning Objectives & Prerequisites

After completing this chapter you will be able to:

1. Define idempotency formally as a fixed-point property on observable state, and classify every HTTP method by its RFC 9110 safety and idempotency guarantees [Fie22a].
2. Implement the IETF **Idempotency-Key** protocol end to end: client key generation, fingerprinted request storage, response snapshotting,

replay semantics, in-flight deduplication (409), mismatch rejection (422), expiry windows, and per-account scoping.

3. Explain why distributed exactly-once delivery is impossible (Two Generals' Problem) and construct the effectively-once substitute from at-least-once delivery plus idempotent processing [Kle17].
4. Author correct `Cache-Control` policies, distinguish `no-cache` from `no-store`, apply `stale-while-revalidate` and `stale-if-error` deliberately, and choose strong versus weak validators correctly.
5. Diagnose the lost-update problem from an interleaving, and prevent it with optimistic concurrency control using `If-Match`, selecting 428, 412, and 409 precisely.
6. Evaluate pessimistic locking, database constraints, and distributed locks (with fencing tokens) against optimistic control for a given workload [Kle17].
7. Prove, with a deterministic concurrency test, that a retry storm against an idempotent endpoint produces exactly one observable mutation.

Prerequisites. Chapters 0–6: the HTTP wire protocol including methods, status codes, and header semantics (Chapter 1); consuming APIs from Python with `httpx` and retry discipline (Chapter 2); REST resource modeling (Chapter 3); JSON serialization and canonicalization (Chapter 4); the FastAPI application factory, dependency injection, middleware, and `TestClient` patterns (Chapter 6). Comfort with `threading.Barrier`, `sqlite3`, and `hashlib` from the Python standard library is assumed; no database server, network access, or cloud account is required anywhere in this chapter.

11.2 Deep Technical Mechanics & Architectural Concepts

11.2.1 Idempotency: A Formal Definition

Definition 11.1 (Idempotent operation). Let S be the set of observable server states and let an API operation be a function $f : S \rightarrow S$. The operation is *idempotent* iff applying it twice is observationally indistinguishable from applying it once:

$$f(f(x)) = f(x) \quad \text{for all } x \in S.$$

Prerequisite	Where it is used here
HTTP methods, status codes, headers	method idempotency algebra; 304/409/412/422/428 selection
Retry loops with backoff (Ch. 2)	why non-idempotent retry is unsafe; the money-transfer argument
REST resources and representations (Ch. 3)	ETags as representation validators; conditional PUT
JSON canonicalization (Ch. 4)	request fingerprinting for idempotency keys
FastAPI middleware and TestClient (Ch. 6)	the idempotency middleware and all offline tests

Table 11.1: Prerequisite map for Chapter 10.

“Observable” deliberately includes side effects a client can detect — ledger entries, emails sent, robots dispatched — and excludes implementation-internal artifacts such as log lines or monotonic request counters.

Two consequences deserve emphasis. First, idempotency is a property of the *effect*, not of the *response*: a replayed request may legally return the freshly stored receipt rather than re-executing, and it may even return a different status code in degenerate cases (a second DELETE returning 404) while remaining idempotent, because the *state* is unchanged. Second, idempotency is not safety. A safe method (GET, HEAD) changes nothing at all; an idempotent method may change state on the first application but not on repetition. The identity function is the only safe operation; every fixed-point operation is idempotent.

RFC 9110 assigns each standard method a fixed position in this algebra [Fie22a], reproduced in Table 11.2. The subtle row is DELETE: the first call transitions the resource to “absent”; every subsequent call finds the fixed point already reached. POST is the only mainstream method with *no* guarantee — which is precisely why POST is the method that carries charges, bookings, and dispatches, and precisely why the industry layered idempotency keys on top of it.

Example

Fixed points in one inventory endpoint. Consider PUT `/inventory/NWR-AXLE-42` with body `{"quantity": 30}`. Whether executed once or fifty times, the observable state ends at `quantity = 30`: the operation is idempotent. Now consider POST `/inventory/NWR-AXLE-42/adjustments` with body `{"delta": -5}`. Each execution appends a new adjustment; fifty executions re-

Method	Safe	Idempotent	Effect of a retry
GET, HEAD	yes	yes	none; caches may even answer locally
OPTIONS, TRACE	yes	yes	none
PUT	no	yes	same absolute state written again
DELETE	no	yes	resource remains absent (fixed point)
POST	no	no	a second, independent side effect
PATCH	no	no (in general)	depends on the delta encoding

Table 11.2: Safety and idempotency per RFC 9110 [Fie22a]. PATCH is idempotent only if the patch document encodes an absolute, not relative, transformation.

serve 250 units. The POST is not idempotent, and no amount of careful client retry logic can make it safe unless the server adds a deduplication mechanism. That mechanism is the idempotency key.

11.2.2 The Idempotency-Key Protocol

The **Idempotency-Key** pattern, standardized as an IETF draft and popularized by Stripe, upgrades an unsafe POST into an idempotent operation by attaching a client-generated unique key to each logical request. The server stores the outcome keyed by that value and answers duplicates from the store instead of re-executing. The protocol has five design decisions, each of which is a failure mode when gotten wrong.

Key generation. The key must be generated by the *client*, one per logical operation (typically a UUIDv4 minted when the user opens the checkout form, not when the request is sent). All retries of that operation — manual retries, library retries, load-balancer retries — reuse the same key. A new key means a new operation; that is the entire contract.

Storage design. The server persists, per key: a *request fingerprint* (a SHA-256 over method, route, and canonical request body), a *response snapshot* (status code, body, content type), a lifecycle state (`in_flight` or `completed`), and a creation timestamp for expiry. The fingerprint is what makes key reuse detectable: it must cover the route, not only the body, or the same JSON sent to two endpoints will collide.

Replay semantics. Three collision outcomes, three status codes:

- Same key, same fingerprint, state `completed` → replay the stored response byte-for-byte, marked with a header such as `Idempotency-Replayed`:

`true` so observability can distinguish replays from executions.

- Same key, different fingerprint → **422**. The client reused a key for a different logical operation; replaying would silently corrupt intent, and executing would double-apply.
- Same key, same fingerprint, state `in_flight` → **409** with **Retry-After**, because the original is still executing and its outcome does not exist yet. (Some drafts discuss 425 Too Early; 409 is the widely deployed choice because the condition is a genuine state conflict.)

Expiry. Keys cannot be stored forever; Stripe retains them for 24 hours. The expiry window is a correctness parameter, not housekeeping: it bounds how long a duplicate can be recognized as a duplicate, and therefore bounds how late a retry may arrive. Choose it from your client’s worst-case retry horizon, not from storage cost.

Scoping. Keys are unique per *account*, not globally — two tenants must never replay each other’s responses, and key uniqueness pressure should not cross tenant boundaries. Endpoints are bound implicitly by the fingerprint (a key reused on a different route mismatches), which prevents cross-endpoint replay without extra bookkeeping.

Figure 11.1 shows the full decision flow as implemented in Listing 10.1.

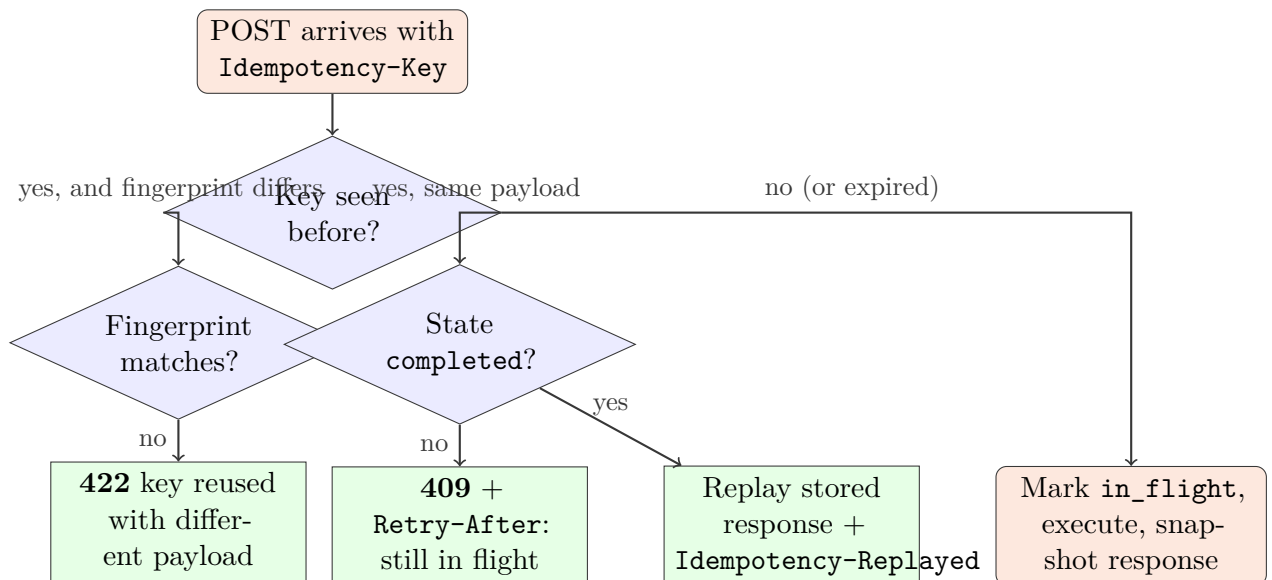


Figure 11.1: Idempotency-key decision flow. Exactly one path executes business logic; every other path returns a deterministic, side-effect-free answer.

Warning

The response snapshot must be written in the same transaction as the state mutation, or the system has a crash window in which the effect happened but the receipt was never recorded. A retry arriving in that window either double-executes (if the key row was rolled back) or deadlocks (if the key row survived as `in_flight` forever). Store first, commit once, replay forever — and never store the response before the mutation is durable.

11.2.3 The Exactly-Once Illusion

Every distributed systems curriculum eventually confronts the Two Generals' Problem: two armies must agree on an attack time using messengers who may be captured. No finite protocol of unreliable messages can guarantee both sides act together, because the last messenger's arrival is itself unknowable to the sender. An HTTP request over a flaky network *is* the Two Generals: after a timeout, the client cannot know whether the request was lost before execution or the response was lost after it. Exactly-once delivery is therefore not a difficult engineering goal; it is a theorem-level impossibility [Kle17].

What is achievable — and what production systems actually ship — is the composition

at-least-once delivery + idempotent processing = effectively-once semantics.

The transport is allowed to deliver duplicates (and will); the processor is built so duplicates are observationally inert. Every deduplication mechanism has a *window*: the idempotency-key TTL, the message-ID retention period, the dedup log's compaction horizon. A duplicate arriving outside the window is indistinguishable from a new request. Sizing the window from the client's maximum retry horizon, and monitoring how full it runs, is the operational discipline that makes “effectively” honest.

11.2.4 HTTP Caching Machinery per RFC 9110/9111

HTTP caching exists to eliminate two things: *latency* (a cache closer than the origin answers faster) and *origin load* (a shared cache answers instead of the origin at all). Both are achieved by the same small machinery: a *freshness lifetime* the origin assigns, and *validators* the cache uses to prove freshness when the lifetime has run out [Fie22a].

Freshness directives. `Cache-Control` is a comma-separated directive vocabulary; the directives that matter most are summarized in Table 11.3. The single most expensive confusion in the vocabulary is `no-cache` versus

no-store. Despite its name, **no-cache** permits *storage* — it forbids *serving* without revalidation. **no-store** forbids storage entirely and is the correct choice for responses containing credentials, account data, or one-time tokens.

Directive	Semantics
max-age=n	fresh for n seconds; a cache may serve without contacting the origin
no-cache	may store, <i>must</i> revalidate before every reuse
no-store	may not store at all; every request reaches the origin
private	only the single-user (browser) cache may store
public	shared caches (CDNs, proxies) may store
stale-while-revalidate=n	may serve stale for n seconds past expiry while refreshing in the background
stale-if-error=n	may serve stale for n seconds if the origin errors
must-revalidate	once stale, never serve without a successful revalidation (overrides stale serving)

Table 11.3: Core **Cache-Control** response directives (RFC 9111).

Validators: strong and weak. When a stored response is stale, the cache revalidates with a *conditional request*. Two validators exist. **Last-Modified** is a one-second-granularity timestamp — *weak*, because two changes inside one second are invisible to it. **ETag** is an opaque tag; a *strong* ETag must change whenever the representation changes by a single byte, while a *weak* ETag (prefixed W/) asserts only semantic equivalence. Strong ETags support range requests and byte-sensitive caches; weak ETags are for representations whose bytes may legitimately vary (e.g. recompressed but semantically identical bodies). The golden rule: a strong ETag is a content address — compute it from the representation bytes (SHA-256 is ideal) or from a version counter when the representation is a pure function of the versioned state. *Never* let a client invent the validator.

The conditional flow is two headers deep: the cache presents its stored validator with **If-None-Match** (or **If-Modified-Since**), and the origin answers **304 Not Modified** with no body when the validator still matches. Figure 11.2 shows the sequence; Listing 10.2 measures the saved bytes.

Heuristic freshness. When the origin sends validators but no explicit lifetime, caches may apply the RFC 9111 heuristic: $\text{freshness} = 0.1 \times (\text{Date} - \text{Last-Modified})$. A document untouched for a week is guessed fresh for about 17 hours. The heuristic is a fallback, not a design; explicit **max-age** is always preferable because it is a decision rather than a guess.

Surrogate keys and CDNs. A surrogate key (e.g. **Surrogate-Key: part:NWR-AXLE-42 catalog**) tags stored responses at the CDN so that one mutation can purge thousands of URLs by tag instead of enumerating

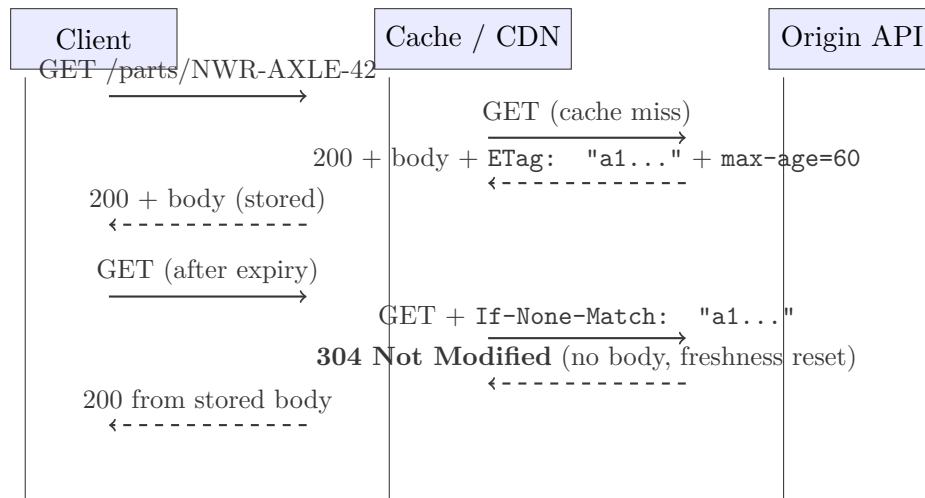


Figure 11.2: Conditional request sequence. The revalidation round trip still costs latency, but the 304 transfers zero representation bytes; with `max-age` still fresh, the round trip disappears entirely.

paths. Tag-based invalidation converts the classic “two hard things” joke into an operational primitive: writes emit tags, purges reference tags, and the cache invalidation problem becomes a bookkeeping problem.

Vary and cache poisoning. The cache key is, by default, method + URI. If the response depends on any request header — `Accept`, `Accept-Encoding`, `Accept-Language`, an authorization scope — the response must declare it with `Vary`, or the cache will serve one client’s representation to another. The hostile version of this mistake is *cache poisoning*: if an unkeyed input (a reflected header, a query parameter ignored by the key) influences the response, an attacker can store a poisoned representation under an innocent key and have the cache deliver it to everyone [Nyg18]. Rule: every byte of the response must be a function of the cache key plus the `Vary` set.

11.2.5 Concurrency Control: The Lost-Update Problem

Caching controls how *readers* see state; concurrency control decides what happens when *writers* collide. The canonical failure is the lost update, worked out interleaving by interleaving in Figure 11.3: two clients read the same inventory row, each computes a new value from its private copy, and the second write silently overwrites the first. Neither client errored. The invariant “reserved $\leq$ on hand” is violated anyway. The bug is not in either request; it is in the *composition* — which is why no amount of single-request validation finds it [Kle17].

Optimistic locking with If-Match. HTTP’s native answer is the

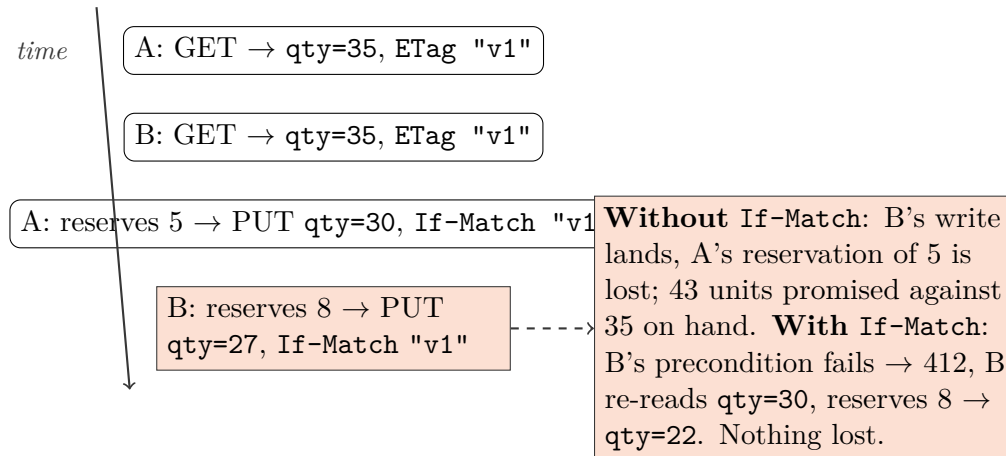


Figure 11.3: The lost-update interleaving. Reads interleave harmlessly; the second blind write is the corruption. A conditional write converts the race into a deterministic rejection.

conditional write. The server stamps every representation with a strong ETag derived from a version counter; a writer echoes it in **If-Match**; the server applies the write inside an atomic compare-and-set:

$$\text{apply} \iff \text{version}_{\text{current}} = \text{version}_{\text{sent}}.$$

The status-code algebra is precise and worth memorizing:

- **428 Precondition Required** — the server *requires* conditional writes and the client sent none. This is the API declaring “blind overwrites are forbidden here.”
- **412 Precondition Failed** — the client sent a precondition and it evaluated false: someone wrote first. The correct client behavior is read–re-decide–retry.
- **409 Conflict** — the request conflicts with server state for *business* reasons (duplicate natural key, invalid state transition), not with a representation version. Use 409 for semantic conflicts and in-flight idempotency collisions; use 412 strictly for failed preconditions. Confusing them sends clients down the wrong recovery path.

Optimistic control shines when contention is rare: zero locks held, no blocking, and the retry cost is paid only by actual collisions. Its price is the read-modify-write retry loop, which under hot contention can livelock; the fix is bounded retries with jittered backoff and escalation.

Pessimistic patterns. When contention is hot or the retry of a failed operation is expensive (a payment re-authorization, a compiled report), take

the lock up front: `SELECT ... FOR UPDATE` in the database, or a serialized queue per aggregate. Pessimism trades throughput for predictability; it is the right tool for short critical sections over hot rows, and the wrong tool for long user think-times, where a held lock outlives the session.

Distributed locks and fencing. A lock service (Redis, ZooKeeper-style consensus) extends mutual exclusion across processes, but a paused lock holder is indistinguishable from a dead one: the lease expires, a second client acquires the lock, and the paused client wakes up and writes anyway. The fix is a *fencing token* — a monotonic number issued with every lock acquisition — which the resource checks on every write, rejecting any token older than the newest it has seen. The lock provides mutual exclusion for efficiency; the fencing token provides correctness for safety. A distributed lock without fencing is a race condition with extra infrastructure [Kle17].

Strategy	Choose when	Cost
Optimistic (If-Match/CAS)	low-to-moderate contention; cheap retries	retry loop; livelock risk on hot keys
Pessimistic (DB row lock)	hot rows; expensive retries; short critical sections	blocking, lock management, deadlock risk
Serialized queue / single writer	per-aggregate ordering is natural	throughput ceiling of one writer
Distributed lock + fencing token	cross-process mutual exclusion	lock service dependency; fencing enforced at the resource

Table 11.4: Concurrency-control strategy selection.

11.2.6 Retries Meet Idempotency: The Money-Transfer Argument

Chapter 2 taught clients to retry with exponential backoff and jitter. Chapter 10 supplies the correctness condition that makes that teaching safe: *a client may retry freely iff the operation is idempotent or carries an idempotency key*. Consider the canonical non-idempotent operation — a money transfer POST — under a timeout:

1. The client sends `POST /transfers {amount_cents: 2500}`.
2. The server executes it; the response is lost in the network.

3. The client retries. Without a key, the server executes *again*. The customer is charged twice; the ledger is authoritative; the duplicate must now be discovered and refunded by a human.

The argument generalizes into a design rule with three clauses. (1) The outcome of a timed-out request is *unknown*, never “failed” — treating it as failed is the bug. (2) Retrying an unknown-outcome non-idempotent request creates exactly one duplicate per ambiguity window; load balancers, proxies, and HTTP libraries all create such windows, often without the application code’s knowledge. (3) The only client-side fix is to make the retry *recognizable*: same key, same payload, so the server’s dedup machinery can collapse the storm to one execution. Idempotency keys are therefore not a server feature; they are the second half of the retry contract, and an SDK that retries POST without minting a key is unsafe by construction.

11.3 Production-Ready Code Implementation

All listings run offline on Python 3.11+ against in-process ASGI test clients; concurrency is exercised with `threading.Barrier` and assertions on final state, never on wall-clock timing. Set up once per machine from PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install fastapi httpx pydantic pytest
4 pytest -q
```

11.3.1 Listing 10.1 — SQLite-Backed Idempotency-Key Middleware

The middleware implements the full decision flow of Figure 11.1: atomic check-and-insert against a SQLite store (the `PRIMARY KEY` constraint *is* the concurrency control), fingerprinting over method + route + canonical JSON body, response snapshotting, 422 on fingerprint mismatch, 409 with `Retry-After` while the twin is in flight, per-account scoping, and a Stripe-style 24-hour TTL. Two deliberate design choices deserve attention. First, keys are claimed *before* execution and the row is *abandoned* on exceptions and 5xx responses, because a server-side failure is an outcome a client must be allowed to retry. Second, the store’s single connection is guarded by one `RLock`; correctness lives in the atomic `INSERT`, not in the volume of the bookkeeping.


```

1  """Listing 10.1--SQLite-backed Idempotency-Key
   middleware for FastAPI.
2
3  Northwind Robotics Payments API. Offline-first, typed
   Python 3.11+.
4  Implements the IETF 'Idempotency-Key' draft semantics:
5
6  * first request with a key executes and snapshots the
   response,
7  * same key + same payload replays the stored response,
8  * same key + different payload is rejected with 422,
9  * a concurrent duplicate while the first is in flight
   gets 409,
10 * keys expire after a configurable TTL (default: 24
   hours, Stripe-style),
11 * keys are scoped per account (header 'X-Account-Id')
   .
12 """
13
14 from __future__ import annotations
15
16 import hashlib
17 import json
18 import sqlite3
19 import threading
20 import time
21 from dataclasses import dataclass
22 from typing import Optional
23
24 from fastapi import FastAPI, Request
25 from fastapi.responses import JSONResponse, Response
26 from pydantic import BaseModel, Field
27 from starlette.middleware.base import (
28     BaseHTTPMiddleware,
29     RequestResponseEndpoint,
30 )
31
32 IDEMPOTENCY_KEY_HEADER = "idempotency-key"
33 REPLAYED_HEADER = "idempotency-replayed"
34 DEFAULT_TTL_SECONDS = 24 * 60 * 60 # Stripe-style 24-
   hour key expiry
35 IDEMPOTENT_METHODS = frozenset({"POST", "PUT", "PATCH"
   })
36
37

```

```

38 #
    -----
39 # Persistence: the idempotency key store
40 #
    -----

41 @dataclass(frozen=True)
42 class StoredResponse:
43     """Immutable snapshot of a completed response."""
44
45     status_code: int
46     body: bytes
47     content_type: str
48
49
50 class IdempotencyStore:
51     """SQLite-backed key store.
52
53     One row per (scope, key). Rows transition '
54     in_flight' -> 'completed'.
55     A single connection guarded by an RLock keeps
56     concurrent handler
57     threads serialized, so 'begin' is an atomic check
58     -and-insert.
59     """
60
61     def __init__(self, path: str = ":memory:",
62                  ttl_seconds: float = DEFAULT_TTL_SECONDS) ->
63     None:
64         self.ttl_seconds = ttl_seconds
65         self._lock = threading.RLock()
66         self._conn = sqlite3.connect(path,
67                                       check_same_thread=False)
68         self._conn.execute(
69             """
70             CREATE TABLE IF NOT EXISTS idempotency_keys
71             (
72                 scope TEXT NOT NULL,
73                 idem_key TEXT NOT NULL,
74                 fingerprint TEXT NOT NULL,
75                 state TEXT NOT NULL CHECK (state
76                 IN ('in_flight', 'completed')),
77                 status_code INTEGER,
78                 response_body BLOB,
79                 content_type TEXT,

```

```

72         created_at=REAL NOT NULL,
73         PRIMARY KEY(scope, idem_key)
74     )
75     """
76 )
77     self._conn.commit()
78
79     def _prune_expired(self, now: float) -> None:
80         self._conn.execute(
81             "DELETE FROM idempotency_keys WHERE
82              created_at < ?",
83             (self.ttl_seconds, now),
84         )
85
86     def begin(
87         self, scope: str, key: str, fingerprint: str,
88         now: Optional[float] = None
89     ) -> tuple[str, Optional[StoredResponse]]:
90         """Atomically claim a key or classify the
91            collision.
92
93            Returns one of:
94            ("started", None) -- caller won the race
95            and must execute,
96            ("replay", stored) -- same fingerprint,
97            completed: replay it,
98            ("mismatch", None) -- same key, different
99            payload: 422,
100            ("in_flight", None) -- same key in flight
101            concurrently: 409.
102
103            """
104         now = time.time() if now is None else now
105         with self._lock:
106             self._prune_expired(now)
107             try:
108                 self._conn.execute(
109                     "INSERT INTO idempotency_keys
110                      (scope, idem_key, fingerprint,
111                       state, created_at)
112                      VALUES (?, ?, ?, 'in_flight', ?)"
113                     ,
114                     (scope, key, fingerprint, now),
115                 )
116                 self._conn.commit()
117                 return ("started", None)
118             except sqlite3.IntegrityError:

```

```

109         row = self._conn.execute(
110             "SELECT fingerprint, state,
111                status_code, response_body,
112                content_type
113            FROM idempotency_keys WHERE scope
114                = ? AND idem_key = ?",
115            (scope, key),
116        ).fetchone()
117
118     if row is None: # pragma: no cover -
119         defensive
120         raise RuntimeError("idempotency row
121             vanished mid-transaction")
122
123     if row[0] != fingerprint:
124         return ("mismatch", None)
125     if row[1] == "in_flight":
126         return ("in_flight", None)
127     return ("replay", StoredResponse(row[2],
128         bytes(row[3]), row[4]))
129
130 def complete(
131     self, scope: str, key: str, stored:
132     StoredResponse
133 ) -> None:
134     """Attach the finished response snapshot to the
135         claimed key."""
136     with self._lock:
137         self._conn.execute(
138             "UPDATE idempotency_keys
139             SET state = 'completed', status_code =
140             ?,
141             response_body = ?, content_type =
142             ?
143             WHERE scope = ? AND idem_key = ? AND
144             state = 'in_flight'",
145             (stored.status_code, stored.body,
146              stored.content_type, scope, key),
147         )
148     self._conn.commit()
149
150 def abandon(self, scope: str, key: str) -> None:
151     """Release the key after a 5xx/exception so the
152         client may retry."""
153     with self._lock:
154         self._conn.execute(
155             "DELETE FROM idempotency_keys
156             WHERE scope = ? AND idem_key = ? AND

```

```

143         state=_ 'in_flight',
144         (scope, key),
145         )
146         self._conn.commit()
147
148     def close(self) -> None:
149         with self._lock:
150             self._conn.close()
151
152     def fingerprint_request(method: str, path: str, body:
153     bytes) -> str:
154         """SHA-256 over method+route+canonical body.
155
156         Hashing only the body is a classic bug: the same
157         JSON payload sent to
158         two different endpoints must not collide, so the
159         route is part of the
160         fingerprint. JSON bodies are canonicalized so key
161         order is irrelevant.
162
163         """
164         try:
165             canonical = json.dumps(
166                 json.loads(body.decode("utf-8")), sort_keys
167                 =True, separators=(",", ":")
168             ).encode("utf-8")
169         except (ValueError, UnicodeDecodeError):
170             canonical = body
171         digest = hashlib.sha256()
172         digest.update(method.upper().encode("ascii"))
173         digest.update(b"\x00")
174         digest.update(path.encode("utf-8"))
175         digest.update(b"\x00")
176         digest.update(canonical)
177         return digest.hexdigest()
178
179     def _problem(status: int, title: str, detail: str,
180     type_slug: str) -> JSONResponse:
181         return JSONResponse(
182             status_code=status,
183             media_type="application/problem+json",
184             content={
185                 "type": f"https://api.northwind-robotics.
186                 example/problems/{type_slug}",
187                 "title": title,

```

```

181         "status": status,
182         "detail": detail,
183     },
184 )
185
186
187 class IdempotencyMiddleware(BaseHTTPMiddleware):
188     """Applies Idempotency-Key semantics to unsafe
189     methods."""
190
191     def __init__(
192         self,
193         app: FastAPI,
194         store: IdempotencyStore,
195         methods: frozenset[str] = IDEMPOTENT_METHODS,
196     ) -> None:
197         super().__init__(app)
198         self.store = store
199         self.methods = methods
200
201     async def dispatch(
202         self, request: Request, call_next:
203         RequestResponseEndpoint
204     ) -> Response:
205         key = request.headers.get(
206             IDEMPOTENCY_KEY_HEADER)
207         if request.method not in self.methods or not
208             key:
209             return await call_next(request)
210
211         scope = request.headers.get("x-account-id", "
212             anonymous")
213         body = await request.body()
214         fingerprint = fingerprint_request(request.
215             method, request.url.path, body)
216
217         outcome, stored = self.store.begin(scope, key,
218             fingerprint)
219         if outcome == "mismatch":
220             return _problem(
221                 422,
222                 "Idempotency-key payload mismatch",
223                 "This Idempotency-Key was already used
224                 with a different request"
225                 " payload. Keys identify one logical
226                 request; mint a new key"

```

```

218         "for a new operation.",
219         "idempotency-key-mismatch",
220     )
221     if outcome == "in_flight":
222         response = _problem(
223             409,
224             "Request with this idempotency key is in flight",
225             "A concurrent request carrying the same Idempotency-Key is",
226             "still being processed. Retry after a short backoff.",
227             "idempotency-key-in-flight",
228         )
229         response.headers["Retry-After"] = "1"
230         return response
231     if outcome == "replay" and stored is not None:
232         return Response(
233             content=stored.body,
234             status_code=stored.status_code,
235             media_type=stored.content_type,
236             headers={REPLAYED_HEADER: "true"},
237         )
238
239     # We claimed the key: execute and snapshot, or abandon on failure.
240     try:
241         downstream = await call_next(request)
242         body_bytes = b""
243         async for chunk in downstream.body_iterator:
244             body_bytes += chunk
245     except Exception:
246         self.store.abandon(scope, key)
247         raise
248
249     content_type = downstream.headers.get("content-type", "application/json")
250     if downstream.status_code < 500:
251         self.store.complete(
252             scope, key, StoredResponse(downstream.status_code, body_bytes, content_type)
253         )
254     else:
255         # Server-side failures are never replayed;

```

```

        free the key.
        self.store.abandon(scope, key)
256     return Response(
257         content=body_bytes,
258         status_code=downstream.status_code,
259         media_type=content_type,
260     )
261
262
263
264 #
    -----
265 # Demo application: Northwind Robotics payments ledger
266 #
    -----

267 class TransferRequest(BaseModel):
268     """One funds movement between internal accounts."""
269
270     from_account: str = Field(min_length=1)
271     to_account: str = Field(min_length=1)
272     amount_cents: int = Field(gt=0)
273
274
275 class TransferReceipt(BaseModel):
276     transfer_id: str
277     from_account: str
278     to_account: str
279     amount_cents: int
280     sequence: int
281
282
283 class Ledger:
284     """Thread-safe in-memory ledger; the observable side effect under test."""
285
286     def __init__(self) -> None:
287         self._lock = threading.Lock()
288         self._transfers: list[TransferReceipt] = []
289
290     def post(self, request: TransferRequest) -> TransferReceipt:
291         with self._lock:
292             receipt = TransferReceipt(
293                 transfer_id=f"tr_{len(self._transfers)}_
                    +1:06d}",

```



```

294         from_account=request.from_account,
295         to_account=request.to_account,
296         amount_cents=request.amount_cents,
297         sequence=len(self._transfers) + 1,
298     )
299     self._transfers.append(receipt)
300     return receipt
301
302     def count(self) -> int:
303         with self._lock:
304             return len(self._transfers)
305
306     def total_cents(self) -> int:
307         with self._lock:
308             return sum(t.amount_cents for t in self._transfers)
309
310
311 def create_app(store: Optional[IdempotencyStore] = None
312 ) -> FastAPI:
313     """Application factory; each test gets an isolated
314     store + ledger."""
315     app = FastAPI(title="Northwind Robotics Payments",
316                   version="1.0.0")
317     app.state.store = store if store is not None else
318         IdempotencyStore()
319     app.state.ledger = Ledger()
320     app.add_middleware(IdempotencyMiddleware, store=app
321                       .state.store)
322
323     @app.post("/api/v1/payments/transfers", status_code
324              =201)
325     def create_transfer(payload: TransferRequest) ->
326         TransferReceipt:
327         return app.state.ledger.post(payload)
328
329     return app

```

The test suite proves every branch of the decision flow, culminating in the 50-thread duplicate storm: fifty workers released on one barrier submit the identical keyed payload; the assertion battery checks that the ledger holds exactly one transfer, that all fifty callers received the byte-identical receipt, and that exactly one response was a non-replayed origin execution.

```

1  """Listing 10.1 test suite -- replay, mismatch, scope,
2  and 50-thread storm.

```

```

3  Concurrency_tests_assert_on_final_state_and_response_
    equality_only;
4  threading.Barrier_maximizes_overlap_without_any_timing_
    dependence.
5  """
6
7  from __future__ import annotations
8
9  import threading
10
11 import pytest
12 from fastapi.testclient import TestClient
13
14 from listing_10_1_idempotency_middleware import (
15     REPLAYED_HEADER,
16     IdempotencyStore,
17     create_app,
18     fingerprint_request,
19 )
20
21 PAYLOAD = {
22     "from_account": "ops-checking",
23     "to_account": "vendor-mecha-parts",
24     "amount_cents": 2500,
25 }
26 KEY = "9b6f3c1e-6d3a-4a2f-9f6d-0d0f5f3d2a01"
27
28
29 @pytest.fixture()
30 def client() -> TestClient:
31     app = create_app()
32     with TestClient(app) as test_client:
33         yield test_client
34
35
36 def test_first_request_executes_and_second_replays(
37     client: TestClient) -> None:
38     first = client.post(
39         "/api/v1/payments/transfers", json=PAYLOAD,
40         headers={"Idempotency-Key": KEY}
41     )
42     assert first.status_code == 201
43     assert REPLAYED_HEADER not in first.headers
44
45     replay = client.post(
46         "/api/v1/payments/transfers", json=PAYLOAD,

```

```

45         headers={"Idempotency-Key": KEY}
46     )
47     assert replay.status_code == 201
48     assert replay.headers[REPLAYED_HEADER] == "true"
49     assert replay.json() == first.json()
50     assert client.app.state.ledger.count() == 1 #
51         exactly-once effect
52
53 def test_same_key_different_payload_is_422(client:
54     TestClient) -> None:
55     client.post(
56         "/api/v1/payments/transfers", json=PAYLOAD,
57         headers={"Idempotency-Key": KEY}
58     )
59     tampered = dict(PAYLOAD, amount_cents=9999)
60     response = client.post(
61         "/api/v1/payments/transfers", json=tampered,
62         headers={"Idempotency-Key": KEY}
63     )
64     assert response.status_code == 422
65     assert response.headers["content-type"].startswith(
66         "application/problem+json")
67     assert client.app.state.ledger.count() == 1
68
69 def test_same_key_different_route_is_422(client:
70     TestClient) -> None:
71     """The fingerprint covers the route, not just the
72     body."""
73     client.post(
74         "/api/v1/payments/transfers", json=PAYLOAD,
75         headers={"Idempotency-Key": KEY}
76     )
77     response = client.post(
78         "/api/v1/payments/refunds", json=PAYLOAD,
79         headers={"Idempotency-Key": KEY}
80     )
81     assert response.status_code == 422
82
83 def test_keys_are_scoped_per_account(client: TestClient
84 ) -> None:
85     """The same key under another account is a
86     different logical request."""
87     client.post(

```

```

79         "/api/v1/payments/transfers",
80         json=PAYLOAD,
81         headers={"Idempotency-Key": KEY, "X-Account-Id"
82                 : "acct-alpha"},
83     )
84     other = client.post(
85         "/api/v1/payments/transfers",
86         json=PAYLOAD,
87         headers={"Idempotency-Key": KEY, "X-Account-Id"
88                 : "acct-beta"},
89     )
90     assert other.status_code == 201
91     assert REPLAYED_HEADER not in other.headers
92     assert client.app.state.ledger.count() == 2
93
94 def test_new_key_means_new_operation(client: TestClient)
95     -> None:
96     for key in ("key-a", "key-b"):
97         response = client.post(
98             "/api/v1/payments/transfers", json=PAYLOAD,
99             headers={"Idempotency-Key": key}
100         )
101         assert response.status_code == 201
102         assert client.app.state.ledger.count() == 2
103
104 def test_in_flight_key_returns_409(client: TestClient)
105     -> None:
106     """Deterministically seed an in-flight row, then
107        collide with it."""
108     import json as _json
109     import time as _time
110
111     store: IdempotencyStore = client.app.state.store
112     body = _json.dumps(PAYLOAD).encode("utf-8")
113     fingerprint = fingerprint_request("POST", "/api/v1/
114         payments/transfers", body)
115     outcome, _ = store.begin("anonymous", KEY,
116                             fingerprint, now=_time.time())
117     assert outcome == "started" # row is now '
118         in_flight', never completed
119
120     response = client.post(
121         "/api/v1/payments/transfers", json=PAYLOAD,
122         headers={"Idempotency-Key": KEY}

```

```

115     )
116     assert response.status_code == 409
117     assert response.headers["Retry-After"] == "1"
118     assert client.app.state.ledger.count() == 0
119
120
121 def test_expired_key_is_collected_and_reused(client:
TestClient) -> None:
122     store = IdempotencyStore(ttl_seconds=60)
123     app = create_app(store=store)
124     with TestClient(app) as expired_client:
125         old = expired_client.post(
126             "/api/v1/payments/transfers", json=PAYLOAD,
127             headers={"Idempotency-Key": KEY}
128         )
129         assert old.status_code == 201
130         # Simulate time travel: age every row beyond
131         # the TTL.
132         store._conn.execute("UPDATE idempotency_keys
SET created_at=created_at-120")
133         store._conn.commit()
134         again = expired_client.post(
135             "/api/v1/payments/transfers", json=PAYLOAD,
136             headers={"Idempotency-Key": KEY}
137         )
138         assert again.status_code == 201
139         assert REPLAYED_HEADER not in again.headers #
140         # expired: treated as new
141         assert expired_client.app.state.ledger.count()
142         == 2
143
144
145 def test_fifty_concurrent_duplicate_submits_mutate_once
(client: TestClient) -> None:
146     """50 threads, identical key+payload, released on
147     one barrier.
148
149     Exactly one transfer may exist at the end; every
150     client must observe
151     the same receipt body; exactly one response is a
152     non-replayed origin
153     response. Duplicates that arrive while the winner
154     is in flight get
155     409 and retry until the snapshot is available.
156     """
157     workers = 50

```

```

149     barrier = threading.Barrier(workers)
150     results: list[tuple[int, bytes, bool]] = []
151     results_lock = threading.Lock()
152
153     def submit() -> None:
154         barrier.wait()
155         attempts = 0
156         while True:
157             attempts += 1
158             assert attempts < 200, "retry_storm_did_not
159                 _converge"
160             response = client.post(
161                 "/api/v1/payments/transfers",
162                 json=PAYLOAD,
163                 headers={"Idempotency-Key": KEY},
164             )
165             if response.status_code == 409:
166                 continue # winner still in flight;
167                 poll for the snapshot
168             with results_lock:
169                 results.append(
170                     (
171                         response.status_code,
172                         response.content,
173                         response.headers.get(
174                             REPLAYED_HEADER) == "true",
175                     )
176                 )
177             return
178
179     threads = [threading.Thread(target=submit) for _ in
180                 range(workers)]
181     for thread in threads:
182         thread.start()
183     for thread in threads:
184         thread.join()
185
186     assert client.app.state.ledger.count() == 1 # one
187         observable mutation
188     assert client.app.state.ledger.total_cents() ==
189         2500
190     assert len(results) == workers
191     statuses = {status for status, _, _ in results}
192     bodies = {body for _, body, _ in results}
193     assert statuses == {201}
194     assert len(bodies) == 1 # every caller received

```

```

    the identical receipt
189 non_replayed = sum(1 for _, _, replayed in results
    if not replayed)
190 assert non_replayed == 1 # exactly one origin
    execution

```

11.3.2 Listing 10.2 — Conditional Requests: Strong ETags and the 304 Flow

A strong validator is a content address: SHA-256 over the exact representation bytes, rendered from a canonical serializer (sorted keys, compact separators — Chapter 4’s canonicalization discipline doing double duty). The demo fetches a part, repeats the fetch conditionally, mutates the state, and prints the byte ledger: 200 with the full body, 304 with zero bytes, then a rotated ETag after the write.

```

1  """Listing 10.2 -- Conditional requests: strong ETags
   and the 304 flow.
2
3  Northwind Robotics parts-catalog API. The ETag is a
   strong validator:
4  SHA-256 over the exact representation bytes, so any
   byte-level change of
5  the representation yields a new tag. The demo measures
   the bandwidth a
6  304 Not Modified saves on a repeat fetch. Offline,
   deterministic.
7  """
8
9  from __future__ import annotations
10
11  import hashlib
12  import json
13  from dataclasses import dataclass
14
15  from fastapi import FastAPI, Request
16  from fastapi.responses import Response
17
18
19  def strong_etag(representation: bytes) -> str:
20      """A strong validator: the quoted SHA-256 of the
       exact response bytes."""
21      return '"' + hashlib.sha256(representation).
       hexdigest() + '"'
22

```

```

23
24 @dataclass
25 class PartRecord:
26     sku: str
27     name: str
28     unit_price_cents: int
29     stock: int
30
31     def render(self) -> bytes:
32         """Canonical JSON bytes: one state maps to
33             exactly one byte string."""
34         return json.dumps(
35             {
36                 "sku": self.sku,
37                 "name": self.name,
38                 "unit_price_cents": self.
39                     unit_price_cents,
40                 "stock": self.stock,
41             },
42             sort_keys=True,
43             separators=(",", ":"),
44             ).encode("utf-8")
45
46 CATALOG: dict[str, PartRecord] = {
47     "NWR-AXLE-42": PartRecord("NWR-AXLE-42", "Heavy-
48         duty_drive_axle", 189_99, 35),
49     "NWR-SERVO-7": PartRecord("NWR-SERVO-7", "Precision
50         servo_motor", 74_50, 212),
51 }
52
53 def create_app() -> FastAPI:
54     app = FastAPI(title="Northwind_Robotics_Catalog",
55         version="1.0.0")
56
57     @app.get("/parts/{sku}")
58     def get_part(sku: str, request: Request) ->
59         Response:
60         part = CATALOG.get(sku)
61         if part is None:
62             return Response(status_code=404)
63         body = part.render()
64         etag = strong_etag(body)
65         headers = {
66             "ETag": etag,

```



```

63         "Cache-Control": "public, max-age=60",
64     }
65     if request.headers.get("if-none-match") == etag:
66         :
67         # Validator matches: the client's copy is
68         # current. Send no body.
69         return Response(status_code=304, headers=
70             headers)
71     return Response(
72         content=body,
73         media_type="application/json",
74         headers=headers,
75     )
76
77 @app.post("/parts/{sku}/restock", status_code=200)
78 def restock(sku: str, units: int) -> dict[str,
79     object]:
80     part = CATALOG[sku]
81     part.stock += units # state change -> new
82     representation -> new ETag
83     return {"sku": sku, "stock": part.stock}
84
85 return app
86
87 def measure_conditional_savings() -> dict[str, int |
88     float | bool]:
89     """Fetch a part twice; the second fetch is a
90     conditional GET.
91
92     Returns byte counts proving the 304 path transfers
93     no representation.
94
95     """
96     from fastapi.testclient import TestClient
97
98     snapshot = CATALOG["NWR-AXLE-42"].stock # keep the
99     module fixture stable
100     try:
101         with TestClient(create_app()) as client:
102             first = client.get("/parts/NWR-AXLE-42")
103             assert first.status_code == 200
104             etag = first.headers["etag"]
105             first_bytes = len(first.content)
106
107             second = client.get(
108                 "/parts/NWR-AXLE-42", headers={"If-None

```

```

100         -Match": etag}
101     )
102     assert second.status_code == 304
103     assert second.headers["etag"] == etag
104     second_bytes = len(second.content)
105
106     # The origin state changes: the old
107     # validator no longer matches.
108     client.post("/parts/NWR-AXLE-42/restock",
109                params={"units": 5})
110     third = client.get(
111         "/parts/NWR-AXLE-42", headers={"If-None-
112         Match": etag}
113     )
114     assert third.status_code == 200
115     assert third.headers["etag"] != etag
116
117     saved_pct = 100.0 * (1 - second_bytes /
118                        first_bytes)
119     return {
120         "first_status": first.status_code,
121         "first_bytes": first_bytes,
122         "second_status": second.status_code,
123         "second_bytes": second_bytes,
124         "saved_pct": round(saved_pct, 1),
125         "etag_rotated_after_restock": third.
126         headers["etag"] != etag,
127     }
128 finally:
129     CATALOG["NWR-AXLE-42"].stock = snapshot
130
131 if __name__ == "__main__":
132     stats = measure_conditional_savings()
133     print("Conditional-request bandwidth measurement")
134     print("-----")
135     print(f"initial GET: {stats['first_status']}
136           "
137           f"({stats['first_bytes']} bytes)")
138     print(f"conditional GET: {stats['second_status']}
139           "
140           f"({stats['second_bytes']} bytes)")
141     print(f"bandwidth saved: {stats['saved_pct']}%")
142     print(f"ETag rotated on write: {stats['
143           etag_rotated_after_restock']}")

```

```

1  """Listing 10.2 tests -- ETag stability, 304 flow,
   rotation on mutation."""
2
3  from __future__ import annotations
4
5  import pytest
6  from fastapi.testclient import TestClient
7
8  from listing_10_2_conditional_requests import (
9      CATALOG,
10     create_app,
11     measure_conditional_savings,
12     strong_etag,
13 )
14
15
16 @pytest.fixture()
17 def client() -> TestClient:
18     with TestClient(create_app()) as test_client:
19         yield test_client
20     CATALOG["NWR-AXLE-42"].stock = 35 # restore the
   seeded fixture
21
22
23 def test_strong_etag_is_content_addressed() -> None:
24     assert strong_etag(b"payload") == strong_etag(b"
   payload")
25     assert strong_etag(b"payload") != strong_etag(b"
   payloae")
26     assert strong_etag(b"x").startswith('') and
   strong_etag(b"x").endswith('')
27
28
29 def test_repeat_get_yields_identical_etag(client:
   TestClient) -> None:
30     one = client.get("/parts/NWR-AXLE-42")
31     two = client.get("/parts/NWR-AXLE-42")
32     assert one.headers["etag"] == two.headers["etag"]
33     assert one.headers["cache-control"] == "public, max
   -age=60"
34
35
36 def test_if_none_match_short_circuits_to_304(client:
   TestClient) -> None:
37     etag = client.get("/parts/NWR-SERVO-7").headers["

```

```

    etag"]
38     response = client.get("/parts/NWR-SERVO-7", headers
    ={"If-None-Match": etag})
39     assert response.status_code == 304
40     assert response.content == b""
41     assert response.headers["etag"] == etag # 304
    still carries validators
42
43
44 def test_mutation_rotates_etag_and_invalidates_cache(
    client: TestClient) -> None:
45     before = client.get("/parts/NWR-AXLE-42")
46     client.post("/parts/NWR-AXLE-42/restock", params={"
    units": 10})
47     after = client.get(
48         "/parts/NWR-AXLE-42", headers={"If-None-Match":
    before.headers["etag"]}
49     )
50     assert after.status_code == 200
51     assert after.headers["etag"] != before.headers["
    etag"]
52     assert after.json()["stock"] == 45
53
54
55 def test_unknown_part_is_404(client: TestClient) ->
    None:
56     assert client.get("/parts/NOPE-1").status_code ==
    404
57
58
59 def test_demo_reports_full_savings() -> None:
60     stats = measure_conditional_savings()
61     assert stats["second_status"] == 304
62     assert stats["second_bytes"] == 0
63     assert stats["saved_pct"] == 100.0
64     assert stats["etag_rotated_after_restock"] is True

```

Running the demo prints the measured savings:

```

1 Conditional-request bandwidth measurement
2 -----
3 initial GET      : 200 (88 bytes)
4 conditional GET  : 304 (0 bytes)
5 bandwidth saved  : 100.0%
6 ETag rotated on write: True

```

11.3.3 Listing 10.3 — Optimistic Locking with If-Match on Inventory

The inventory resource stamps every representation with a version-derived strong ETag ("v3") and refuses blind writes: no If-Match is 428, a stale If-Match is 412, and only an exact match reaches the atomic compare-and-swap. The status codes are not decoration — each selects a different client recovery strategy (add the header; re-read and re-decide; resolve a business conflict).

```

1  """Listing 10.3 -- Optimistic locking with If-Match on
   an inventory API.
2
3  Northwind Robotics warehouse inventory. Every
   representation carries a
4  version-derived ETag ("v3"). Writes must present '
   If-Match ':
5
6  * header missing -> 428 Precondition Required
   ,
7  * header does not match -> 412 Precondition Failed (
   lost update refused),
8  * header matches -> write applied, version
   incremented.
9
10 The version counter is a strong validator here because
   the representation
11 is a pure function of the stored state: equal versions
   imply byte-identical
12 bodies. Offline, deterministic.
13 """
14
15 from __future__ import annotations
16
17 import json
18 import threading
19 from dataclasses import dataclass
20
21 from fastapi import FastAPI, Request
22 from fastapi.responses import JSONResponse, Response
23 from pydantic import BaseModel, Field
24
25
26 @dataclass
27 class InventoryRecord:
28     sku: str

```

```

29     quantity: int
30     version: int
31
32     def render(self) -> bytes:
33         return json.dumps(
34             {"sku": self.sku, "quantity": self.quantity
35              , "version": self.version},
36             sort_keys=True,
37             separators=(",", ":"),
38             ).encode("utf-8")
39
40     @property
41     def etag(self) -> str:
42         return f'v{self.version}'
43
44 class InventoryStore:
45     """Thread-safe in-memory inventory keyed by SKU."""
46
47     def __init__(self) -> None:
48         self._lock = threading.Lock()
49         self._records = {
50             "NWR-AXLE-42": InventoryRecord("NWR-AXLE-42",
51                                             quantity=35, version=1),
52             "NWR-SERVO-7": InventoryRecord("NWR-SERVO-7",
53                                             quantity=212, version=1),
54         }
55
56     def get(self, sku: str) -> InventoryRecord | None:
57         with self._lock:
58             record = self._records.get(sku)
59             if record is None:
60                 return None
61             return InventoryRecord(record.sku, record.
62                                   quantity, record.version)
63
64     def compare_and_swap(
65         self, sku: str, expected_etag: str,
66         new_quantity: int
67     ) -> tuple[str, InventoryRecord | None]:
68         """Atomic check-and-set. Returns ('not_found' |
69         mismatch' | 'updated', record)."""
70         with self._lock:
71             record = self._records.get(sku)
72             if record is None:
73                 return ("not_found", None)

```

```

69         if record.etag != expected_etag:
70             return (
71                 "mismatch",
72                 InventoryRecord(record.sku, record.
73                                quantity, record.version),
74             )
75         updated = InventoryRecord(sku, new_quantity
76                                , record.version + 1)
77         self._records[sku] = updated
78         return ("updated", updated)
79
80 def _problem(status: int, title: str, detail: str, slug
81 : str) -> JSONResponse:
82     return JSONResponse(
83         status_code=status,
84         media_type="application/problem+json",
85         content={
86             "type": f"https://api.northwind-robotics.
87                    example/problems/{slug}",
88             "title": title,
89             "status": status,
90             "detail": detail,
91         },
92     )
93
94 class InventoryUpdate(BaseModel):
95     quantity: int = Field(ge=0)
96
97 def create_app() -> FastAPI:
98     app = FastAPI(title="Northwind_Robotics_Inventory",
99                  version="1.0.0")
100     app.state.store = InventoryStore()
101
102     @app.get("/inventory/{sku}")
103     def get_inventory(sku: str) -> Response:
104         record = app.state.store.get(sku)
105         if record is None:
106             return Response(status_code=404)
107         return Response(
108             content=record.render(),
109             media_type="application/json",
110             headers={"ETag": record.etag},
111         )

```

```

110 @app.put("/inventory/{sku}")
111 def put_inventory(
112     sku: str, payload: InventoryUpdate, request:
113     Request
114 ) -> Response:
115     if_match = request.headers.get("if-match")
116     if if_match is None:
117         return _problem(
118             428,
119             "Precondition␣required",
120             "Writes␣to␣this␣resource␣must␣be␣
121             conditional:␣send␣If-Match"
122             "␣with␣the␣ETag␣of␣the␣representation␣
123             you␣based␣your␣edit␣on.",
124             "precondition-required",
125         )
126     outcome, record = app.state.store.
127     compare_and_swap(
128         sku, if_match, payload.quantity
129     )
130     if outcome == "not_found":
131         return Response(status_code=404)
132     if outcome == "mismatch" and record is not None
133     :
134         return _problem(
135             412,
136             "Precondition␣failed",
137             "The␣resource␣changed␣since␣your␣copy:␣
138             current␣ETag␣is"
139             f"␣{record.etag},␣you␣sent␣If-Match:␣{
140             if_match}.␣Re-read,"
141             "␣re-decide,␣and␣retry.",
142             "precondition-failed",
143         )
144     assert record is not None
145     return Response(
146         content=record.render(),
147         media_type="application/json",
148         headers={"ETag": record.etag},
149     )
150
151 return app

```

```

1 """Listing␣10.3␣tests␣--␣428/412␣selection␣and␣lost-
2 update␣prevention.

```



```

2
3 The concurrency test drives a deterministic read-modify
  -write storm:
4 every writer retries with a fresh read until its
  compare-and-set wins,
5 so the final state must equal the sum of all intended
  increments.
6 """
7
8 from __future__ import annotations
9
10 import threading
11
12 import pytest
13 from fastapi.testclient import TestClient
14
15 from listing_10_3_optimistic_locking import create_app
16
17 SKU = "NWR-AXLE-42"
18
19
20 @pytest.fixture()
21 def client() -> TestClient:
22     with TestClient(create_app()) as test_client:
23         yield test_client
24
25
26 def test_write_without_if_match_is_428(client:
    TestClient) -> None:
27     response = client.put(f"/inventory/{SKU}", json={"
        quantity": 30})
28     assert response.status_code == 428
29     assert response.headers["content-type"].startswith(
        "application/problem+json")
30     assert client.get(f"/inventory/{SKU}").json()["
        quantity"] == 35
31
32
33 def test_stale_if_match_is_412_and_state_is_untouched(
    client: TestClient) -> None:
34     etag_v1 = client.get(f"/inventory/{SKU}").headers["
        etag"]
35     # Someone else writes first.
36     winner = client.put(
37         f"/inventory/{SKU}", json={"quantity": 30},
        headers={"If-Match": etag_v1}

```

```

38     )
39     assert winner.status_code == 200
40     # Our stale write must be rejected.
41     loser = client.put(
42         f"/inventory/{SKU}", json={"quantity": 25},
43         headers={"If-Match": etag_v1}
44     )
45     assert loser.status_code == 412
46     state = client.get(f"/inventory/{SKU}").json()
47     assert state["quantity"] == 30 # the winner's
48     value, not the stale one
49     assert state["version"] == 2
50
51 def test_lost_update_scenario_is_prevented(client:
52     TestClient) -> None:
53     """The textbook interleaving: A and B both read v1;
54     A writes; B must lose."""
55     read_a = client.get(f"/inventory/{SKU}")
56     read_b = client.get(f"/inventory/{SKU}")
57     assert read_a.headers["etag"] == read_b.headers["
58         etag"] == 'v1'
59
60     # A decides from its copy: reserve 5 of 35 -> 30.
61     ok = client.put(
62         f"/inventory/{SKU}",
63         json={"quantity": 35 - 5},
64         headers={"If-Match": read_a.headers["etag"]},
65     )
66     assert ok.status_code == 200
67     assert ok.headers["etag"] == 'v2'
68
69     # B decides from the same stale copy: reserve 8 of
70     35 -> 27. Refused.
71     refused = client.put(
72         f"/inventory/{SKU}",
73         json={"quantity": 35 - 8},
74         headers={"If-Match": read_b.headers["etag"]},
75     )
76     assert refused.status_code == 412
77
78     # B re-reads, re-decides against current truth, and
79     retries.
80     fresh = client.get(f"/inventory/{SKU}")
81     retried = client.put(
82         f"/inventory/{SKU}",

```

```

77         json={"quantity": fresh.json()["quantity"] -
78             8},
79         headers={"If-Match": fresh.headers["etag"]},
80     )
81     assert retried.status_code == 200
82     final = client.get(f"/inventory/{SKU}").json()
83     assert final["quantity"] == 35 - 5 - 8 # both
84         reservations, none lost
85     assert final["version"] == 3
86
87 def test_unknown_sku_is_404(client: TestClient) -> None:
88     assert client.get("/inventory/NOPE").status_code ==
89         404
90     assert (
91         client.put(
92             "/inventory/NOPE", json={"quantity": 1},
93             headers={"If-Match": '"v1"'})
94         .status_code
95         == 404
96     )
97
98 def test_negative_quantity_is_rejected(client:
99     TestClient) -> None:
100     etag = client.get(f"/inventory/{SKU}").headers["
101         etag"]
102     response = client.put(
103         f"/inventory/{SKU}", json={"quantity": -1},
104         headers={"If-Match": etag}
105     )
106     assert response.status_code == 422
107
108 def
109     test_concurrent_read_modify_write_storm_loses_nothing
110     (client: TestClient) -> None:
111         """20 threads each increment by one under
112             optimistic retry.
113
114         A writer loops: read -> compute -> conditional
115             write; on 412 it
116             re-reads and retries. Loop termination is state-
117             driven (a successful
118             compare-and-set), never timing-driven.

```

```

110 """
111     workers = 20
112     barrier = threading.Barrier(workers)
113     attempts: list[int] = []
114     attempts_lock = threading.Lock()
115
116     def increment_once() -> None:
117         barrier.wait()
118         tries = 0
119         while True:
120             tries += 1
121             assert tries < 500, "optimistic_retry_loop_
122                 did_not_converge"
123             current = client.get(f"/inventory/{SKU}")
124             etag = current.headers["etag"]
125             target = current.json()["quantity"] + 1
126             written = client.put(
127                 f"/inventory/{SKU}",
128                 json={"quantity": target},
129                 headers={"If-Match": etag},
130             )
131             if written.status_code == 200:
132                 with attempts_lock:
133                     attempts.append(tries)
134                 return
135             assert written.status_code == 412 # only
136                 contention may fail us
137
138     threads = [threading.Thread(target=increment_once)
139                 for _ in range(workers)]
140     for thread in threads:
141         thread.start()
142     for thread in threads:
143         thread.join()
144
145     final = client.get(f"/inventory/{SKU}").json()
146     assert final["quantity"] == 35 + workers # every
147         increment landed
148     assert final["version"] == 1 + workers # one
149         version per accepted write
150     assert len(attempts) == workers

```

11.3.4 Listing 10.4 — Double-Submit Storm Harness

This harness reproduces the incident shape — fifty identical POST `/transfers` requests fired from one barrier — twice: once as the unsafe legacy client (no key, every duplicate executes) and once with the `Idempotency-Key`. The output is the chapter's thesis in two lines of measured output.

```

1  """Listing 10.4 -- Double-submit storm harness.
2
3  Reproduces the classic checkout bug -- a user double-
4  clicks "Pay", the
5  load balancer retries a timed-out request, or a mobile
6  client re-sends on
7  flaky Wi-Fi -- and proves that an Idempotency-Key turns
8  a storm of N
9  duplicate requests into exactly one observable mutation
10
11 The harness is deterministic: workers are released on
12 one
13 threading.Barrier, retries on 409 are state-driven (
14 poll until the winner
15 publishes its snapshot), and every assertion is on
16 final state, never on
17 timing. Run directly for a printed before/after
18 comparison.
19
20 """
21
22 from __future__ import annotations
23
24 import threading
25 from dataclasses import dataclass, field
26
27 from fastapi.testclient import TestClient
28
29 from listing_10_1_idempotency_middleware import
30     REPLAYED_HEADER, create_app
31
32 TRANSFERS_URL = "/api/v1/payments/transfers"
33 PAYLOAD = {
34     "from_account": "ops-checking",
35     "to_account": "vendor-mecha-parts",
36     "amount_cents": 2500,
37 }
38
39 @dataclass(frozen=True)

```

```

32 class StormResult:
33     """What a storm of duplicate submits produced."""
34
35     workers: int
36     status_codes: list[int] = field(default_factory=
37         list)
38     bodies: set[bytes] = field(default_factory=set)
39     replayed_count: int = 0
40     ledger_entries: int = 0
41     ledger_total_cents: int = 0
42
43 def run_double_submit_storm(
44     workers: int = 50, use_idempotency_key: bool = True
45 ) -> StormResult:
46     """Fire 'workers' identical POSTs at the API from
47         a single barrier.
48
49     With 'use_idempotency_key=False' this simulates
50     the unsafe legacy
51     client: every request executes. With the key,
52     exactly one executes.
53     """
54     app = create_app()
55     key = "storm-" + ("keyed" if use_idempotency_key
56         else "naive")
57
58     with TestClient(app) as client:
59         barrier = threading.Barrier(workers)
60         status_codes: list[int] = []
61         bodies: set[bytes] = set()
62         replayed = 0
63         lock = threading.Lock()
64
65         def submit() -> None:
66             nonlocal replayed
67             headers = (
68                 {"Idempotency-Key": key} if
69                 use_idempotency_key else {}
70             )
71             barrier.wait()
72             attempts = 0
73             while True:
74                 attempts += 1
75                 assert attempts < 200, "storm did not
76                     converge"

```

```

71         response = client.post(TRANSFERS_URL,
72                                json=PAYLOAD, headers=headers)
73         if response.status_code == 409 and
74            use_idempotency_key:
75             continue # winner in flight; poll
76                      for its snapshot
77         with lock:
78             status_codes.append(response.
79                                status_code)
80             bodies.add(response.content)
81             if response.headers.get(
82                REPLAYED_HEADER) == "true":
83                 replayed += 1
84         return
85
86     threads = [threading.Thread(target=submit) for
87                _ in range(workers)]
88     for thread in threads:
89         thread.start()
90     for thread in threads:
91         thread.join()
92
93     return StormResult(
94         workers=workers,
95         status_codes=status_codes,
96         bodies=bodies,
97         replayed_count=replayed,
98         ledger_entries=app.state.ledger.count(),
99         ledger_total_cents=app.state.ledger.
100            total_cents(),
101     )
102
103 def main() -> None:
104     print("Double-submit storm: 50 identical POST /
105           transfers")
106     print("=" * 52)
107     naive = run_double_submit_storm(workers=50,
108                                     use_idempotency_key=False)
109     print(f"without Idempotency-Key: {naive.
110           ledger_entries} ledger entries, "
111           f"{naive.ledger_total_cents} cents moved --
112           the incident")
113     keyed = run_double_submit_storm(workers=50,
114                                     use_idempotency_key=True)
115     print(f"with Idempotency-Key: {keyed.

```

```

105         ledger_entries}␣ledger␣entry,"
106         f"␣{keyed.ledger_total_cents}␣cents␣moved,␣"
107         f"{keyed.replayed_count}␣replayed␣responses")
108
109 if __name__ == "__main__":
110     main()

```

```

1  """Listing␣10.4␣tests␣--␣the␣storm␣harness␣proves␣
   effectively-once."""
2
3  from __future__ import annotations
4
5  from listing_10_4_double_submit_storm import
   run_double_submit_storm
6
7
8  def test_naive_client_executes_every_duplicate() ->
   None:
9      """The␣unsafe␣baseline:␣no␣key,␣so␣all␣50␣
   duplicates␣charge␣through."""
10     result = run_double_submit_storm(workers=50,
   use_idempotency_key=False)
11     assert result.ledger_entries == 50
12     assert result.ledger_total_cents == 50 * 2500
13     assert set(result.status_codes) == {201}
14     assert result.replayed_count == 0
15
16
17  def test_keyed_storm_mutates_exactly_once() -> None:
18     result = run_double_submit_storm(workers=50,
   use_idempotency_key=True)
19     assert result.ledger_entries == 1
20     assert result.ledger_total_cents == 2500
21     assert set(result.status_codes) == {201}
22     assert len(result.bodies) == 1 # identical receipt
   for every caller
23     # Exactly one origin execution; the other 49
   observed the replay.
24     assert result.replayed_count == 49
25
26
27  def test_small_storms_also_collapse_to_one_mutation()
   -> None:
28     for workers in (2, 3, 7):
29         result = run_double_submit_storm(workers=

```



```

workers, use_idempotency_key=True)
30 assert result.ledger_entries == 1
31 assert result.replayed_count == workers - 1

```

```

1 Double-submit storm: 50 identical POST /transfers
2 =====
3 without Idempotency-Key : 50 ledger entries, 125000
   cents moved <-- the incident
4 with Idempotency-Key : 1 ledger entry, 2500 cents
   moved, 49 replayed responses

```

11.3.5 Listing 10.5 — Cache-Policy Simulator

The simulator makes RFC 9111 executable: parse a `Cache-Control` policy, script an origin's behavior over a logical timeline (representations change at $t = 70$, the origin falls over at $t = 130$), and compute the exact outcome of every request — HIT, MISS, revalidation hit or miss, stale-while-revalidate, stale-if-error, bypass. Because time is a logical integer and the origin is a sorted script, every run is bit-for-bit reproducible, which makes the simulator a teaching instrument and a policy test harness at once.

```

1 """Listing 10.5 -- Cache-policy simulator (pure stdlib)
2
3 Given a response's cache policy (Cache-Control
   directives + validators)
4 and a deterministic request timeline against a scripted
   origin, compute
5 the outcome of every request: HIT, MISS, revalidation,
   stale-serve, or
6 bypass. Teaches RFC 9111 freshness semantics without a
   network.
7
8 Time is a logical integer (seconds); the origin is a
   script of
9 (t, state) changes, so every run is bit-for-bit
   reproducible.
10 """
11
12 from __future__ import annotations
13
14 from collections import Counter
15 from dataclasses import dataclass
16 from typing import Optional
17

```

```

18 HEURISTIC_FRACTION = 0.1 # RFC 9111 heuristic: 10% of
   (Date - Last-Modified)
19
20
21 #
   -----
22 # Policy parsing
23 #
   -----

24 @dataclass(frozen=True)
25 class CachePolicy:
26     """The cache-relevant slice of one origin response.
27     """
28
29     no_store: bool = False
30     no_cache: bool = False
31     must_revalidate: bool = False
32     public: bool = False
33     private: bool = False
34     max_age: Optional[float] = None
35     stale_while_revalidate: Optional[float] = None
36     stale_if_error: Optional[float] = None
37     etag: Optional[str] = None
38     last_modified: Optional[float] = None
39     date: float = 0.0
40
41     def freshness_lifetime(self) -> float:
42         """Explicit max-age wins; otherwise the RFC
43         9111 heuristic."""
44         if self.max_age is not None:
45             return self.max_age
46         if self.last_modified is not None:
47             return HEURISTIC_FRACTION * max(0.0, self.
48                 date - self.last_modified)
49         return 0.0
50
51     def parse_cache_control(value: str) -> dict[str,
52         Optional[str]]:
53         """'public, max-age=60' -> {'public': None, 'max-
54             age': '60'}."""
55         directives: dict[str, Optional[str]] = {}
56         for token in value.split(","):
57             name, _, arg = token.strip().partition("=")

```

```

54         if name:
55             directives[name.lower()] = arg.strip(',')
56             if arg else None
57         return directives
58
59 def policy_from_headers(
60     cache_control: str,
61     *,
62     etag: Optional[str] = None,
63     last_modified: Optional[float] = None,
64     date: float = 0.0,
65 ) -> CachePolicy:
66     d = parse_cache_control(cache_control)
67     return CachePolicy(
68         no_store="no-store" in d,
69         no_cache="no-cache" in d,
70         must_revalidate="must-revalidate" in d,
71         public="public" in d,
72         private="private" in d,
73         max_age=float(d["max-age"]) if d.get("max-age")
74             is not None else None,
75         stale_while_revalidate=(
76             float(d["stale-while-revalidate"])
77             if d.get("stale-while-revalidate") is not
78                 None
79             else None
80         ),
81         stale_if_error=(
82             float(d["stale-if-error"])
83             if d.get("stale-if-error") is not None
84             else None
85         ),
86         etag=etag,
87         last_modified=last_modified,
88         date=date,
89     )
90
91 #
92
93 # Scripted origin
94 #

```

```

93 @dataclass(frozen=True)
94 class OriginState:
95     status: int # 200 or 500
96     etag: str
97     body_version: int
98
99
100 class ScriptedOrigin:
101     """A deterministic origin: a sorted script of (t,
102         OriginState) changes."""
103
104     def __init__(self, script: list[tuple[float,
105         OriginState]]) -> None:
106         if not script:
107             raise ValueError("origin script needs at
108                 least one state")
109         self._script = sorted(script, key=lambda item:
110             item[0])
111         self.fetch_count = 0
112
113     def state_at(self, t: float) -> OriginState:
114         state = self._script[0][1]
115         for change_at, candidate in self._script:
116             if change_at <= t:
117                 state = candidate
118             else:
119                 break
120         return state
121
122     def fetch(self, t: float) -> OriginState:
123         self.fetch_count += 1
124         return self.state_at(t)
125
126 #
127 -----
128
129 # The cache
130 #
131 -----
132
133 @dataclass
134 class StoredEntry:
135     policy: CachePolicy
136     etag: str
137     body_version: int

```

```

131     stored_at: float
132
133
134 HIT = "HIT"
135 MISS = "MISS"
136 REVALIDATED_HIT = "REVALIDATED_HIT" # conditional
    GET -> 304
137 REVALIDATED_MISS = "REVALIDATED_MISS" # conditional
    GET -> 200 (new ETag)
138 STALE_WHILE_REVALIDATE = "STALE_WHILE_REVALIDATE"
139 STALE_IF_ERROR = "STALE_IF_ERROR"
140 ERROR = "ERROR" # origin down, nothing servable
141 BYPASS = "BYPASS" # no-store: never reused
142
143
144 class SimulatedCache:
145     """Single-entry shared cache executing RFC 9111
    freshness decisions."""
146
147     def __init__(self, origin: ScriptedOrigin) -> None:
148         self.origin = origin
149         self.stored: Optional[StoredEntry] = None
150         self.stats: Counter[str] = Counter()
151
152     def request(self, t: float) -> str:
153         outcome = self._serve(t)
154         self.stats[outcome] += 1
155         return outcome
156
157     def _serve(self, t: float) -> str:
158         entry = self.stored
159         if entry is None or entry.policy.no_store:
160             return self._fetch_and_store(t, miss_label=
                MISS if entry is None else BYPASS)
161
162         policy = entry.policy
163         age = t - entry.stored_at
164         lifetime = policy.freshness_lifetime()
165         fresh = age <= lifetime and not policy.no_cache
166
167         if fresh:
168             return HIT
169
170         # Stale-while-revalidate: serve stale now,
            refresh in the background.
171         if (

```

```

172         policy.stale_while_revalidate is not None
173         and not policy.no_cache
174         and not policy.must_revalidate
175         and age <= lifetime + policy.
            stale_while_revalidate
176     ):
177         self._background_revalidate(t, entry)
178         return STALE_WHILE_REVALIDATE
179
180     return self._revalidate(t, entry, age, lifetime
        )
181
182     def _revalidate(
183         self, t: float, entry: StoredEntry, age: float,
            lifetime: float
184     ) -> str:
185         policy = entry.policy
186         origin_state = self.origin.fetch(t)
187         if origin_state.status >= 500:
188             if (
189                 policy.stale_if_error is not None
190                 and not policy.must_revalidate
191                 and age <= lifetime + policy.
                    stale_if_error
192             ):
193                 return STALE_IF_ERROR
194             return ERROR
195         if origin_state.etag == entry.etag:
196             # 304: freshen the stored entry in place.
197             entry.stored_at = t
198             return REVALIDATED_HIT
199         # 200: a new representation replaces the entry.
200         self.stored = StoredEntry(
201             policy=entry.policy,
202             etag=origin_state.etag,
203             body_version=origin_state.body_version,
204             stored_at=t,
205         )
206         return REVALIDATED_MISS
207
208     def _background_revalidate(self, t: float, entry:
        StoredEntry) -> None:
209         """Model the async refresh synchronously for
            determinism."""
210         origin_state = self.origin.fetch(t)
211         if origin_state.status < 500:

```

```

212         self.stored = StoredEntry(
213             policy=entry.policy,
214             etag=origin_state.etag,
215             body_version=origin_state.body_version,
216             stored_at=t,
217         )
218
219     def _fetch_and_store(self, t: float, miss_label:
220         str) -> str:
221         origin_state = self.origin.fetch(t)
222         if origin_state.status >= 500:
223             return ERROR
224         policy = CachePolicy(
225             max_age=60, etag=origin_state.etag, date=t
226         ) # demo default; scenarios override via
227         inject()
228         if self.stored is not None and self.stored.
229             policy.no_store:
230             return miss_label # BYPASS: policy forbids
231                 storage entirely
232         self.stored = StoredEntry(
233             policy=policy,
234             etag=origin_state.etag,
235             body_version=origin_state.body_version,
236             stored_at=t,
237         )
238         return miss_label
239
240     def inject(self, policy: CachePolicy, t: float) ->
241         None:
242         """Prime the cache with an entry under an
243             explicit policy."""
244         state = self.origin.state_at(t)
245         self.stored = StoredEntry(
246             policy=policy,
247             etag=state.etag,
248             body_version=state.body_version,
249             stored_at=t,
250         )
251
252     def run_timeline(cache: SimulatedCache, times: list[
253         float]) -> list[str]:
254         return [cache.request(t) for t in times]

```

```

251 if __name__ == "__main__":
252     origin = ScriptedOrigin(
253         [
254             (0, OriginState(200, 'v1', 1)),
255             (70, OriginState(200, 'v2', 2)),
256             (130, OriginState(500, 'v2', 2)),
257         ]
258     )
259     cache = SimulatedCache(origin)
260     cache.request(0) # MISS, stores v1 with max-age=60
261     print("t=10:", cache.request(10)) # HIT
262     print("t=80:", cache.request(80)) # revalidate
263     # -> new etag v2
264     print("t=150:", cache.request(150)) # stale,
265     # origin down
266     print("stats:", dict(cache.stats))

```

```

1  """Listing 10.5 tests -- directive semantics, proven on
2  scripted timelines."""
3
4  from __future__ import annotations
5
6  from listing_10_5_cache_simulator import (
7      BYPASS,
8      ERROR,
9      HIT,
10     MISS,
11     OriginState,
12     REVALIDATED_HIT,
13     REVALIDATED_MISS,
14     STALE_IF_ERROR,
15     STALE_WHILE_REVALIDATE,
16     SimulatedCache,
17     ScriptedOrigin,
18     parse_cache_control,
19     policy_from_headers,
20     run_timeline,
21 )
22
23 def make_cache(
24     script: list[tuple[float, OriginState]] | None =
25     None,
26 ) -> SimulatedCache:
27     if script is None:
28         script = [(0, OriginState(200, 'v1', 1))]

```



```

28     return SimulatedCache(ScriptedOrigin(script))
29
30
31 def test_parse_cache_control() -> None:
32     d = parse_cache_control('public,␣max-age=60,␣stale-
33         while-revalidate=30')
34     assert d == {"public": None, "max-age": "60", "
35         stale-while-revalidate": "30"}
36
37
38 def test_max_age_governs_freshness() -> None:
39     cache = make_cache()
40     cache.inject(policy_from_headers("public,␣max-age
41         =60", etag=' "v1" '), t=0)
42     assert run_timeline(cache, [10, 60]) == [HIT, HIT]
43     assert cache.request(61) == REVALIDATED_HIT #
44         stale -> conditional GET -> 304
45     assert cache.request(70) == HIT # freshened by the
46         304
47
48
49 def test_no_store_never_caches() -> None:
50     cache = make_cache()
51     cache.inject(policy_from_headers("no-store", etag='
52         "v1" '), t=0)
53     assert run_timeline(cache, [1, 2, 3]) == [BYPASS,
54         BYPASS, BYPASS]
55     assert cache.origin.fetch_count == 3 # every
56         request hits the origin
57
58
59 def test_no_cache_revalidates_every_time() -> None:
60     cache = make_cache()
61     cache.inject(
62         policy_from_headers("no-cache", etag=' "v1" ',
63             date=0), t=0
64     )
65     # Validators make each revalidation cheap (304),
66         but nothing is served
67     # without contacting the origin first.
68     assert run_timeline(cache, [1, 2]) == [
69         REVALIDATED_HIT, REVALIDATED_HIT]
70     assert cache.origin.fetch_count == 2
71
72
73 def test_heuristic_freshness_without_max_age() -> None:

```

```

63     cache = make_cache()
64     # Last-Modified 1000s before Date -> heuristic
        freshness = 100s.
65     policy = policy_from_headers(
66         "public", etag='v1', last_modified=0.0, date
            =1000.0
67     )
68     cache.inject(policy, t=1000.0)
69     assert cache.request(1050.0) == HIT
70     assert cache.request(1200.0) == REVALIDATED_HIT
71
72
73 def test_changed_etag_yields_revalidated_miss() -> None
    :
74     cache = make_cache(
75         [
76             (0, OriginState(200, 'v1', 1)),
77             (50, OriginState(200, 'v2', 2)),
78         ]
79     )
80     cache.inject(policy_from_headers("public", max-age
        =10", etag='v1'), t=0)
81     assert cache.request(60) == REVALIDATED_MISS # 200
        with new body
82     assert cache.stored is not None and cache.stored.
        etag == 'v2'
83     assert cache.request(65) == HIT # the fresh
        replacement now serves
84
85
86 def
    test_stale_while_revalidate_serves_stale_then_refreshes
    () -> None:
87     cache = make_cache(
88         [
89             (0, OriginState(200, 'v1', 1)),
90             (40, OriginState(200, 'v2', 2)),
91         ]
92     )
93     cache.inject(
94         policy_from_headers(
95             "public", max-age=30, stale-while-revalidate
                =60", etag='v1'
96         ),
97         t=0,
98     )

```

```

99     assert cache.request(50) == STALE_WHILE_REVALIDATE
       # stale served...
100     assert cache.stored is not None and cache.stored.
       etag == '"v2"' # ...refreshed
101     assert cache.request(55) == HIT
102
103
104 def test_stale_if_error_rescues_an_origin_outage() ->
None:
105     cache = make_cache(
106         [
107             (0, OriginState(200, '"v1"', 1)),
108             (100, OriginState(500, '"v1"', 1)),
109         ]
110     )
111     cache.inject(
112         policy_from_headers("public,␣max-age=30,␣stale-
           if-error=300", etag='"v1"'),
113         t=0,
114     )
115     assert cache.request(120) == STALE_IF_ERROR #
       origin down, stale saved us
116
117
118 def test_must_revalidate_forbids_stale_if_error() ->
None:
119     cache = make_cache(
120         [
121             (0, OriginState(200, '"v1"', 1)),
122             (100, OriginState(500, '"v1"', 1)),
123         ]
124     )
125     cache.inject(
126         policy_from_headers(
127             "public,␣max-age=30,␣must-revalidate,␣stale-
           if-error=300", etag='"v1"'
128         ),
129         t=0,
130     )
131     assert cache.request(120) == ERROR # must-
       revalidate wins: no stale serve
132
133
134 def test_full_demo_timeline_is_deterministic() -> None:
135     def run() -> tuple[list[str], dict[str, int]]:
136         origin = ScriptedOrigin(

```

```

137         [
138             (0, OriginState(200, 'v1', 1)),
139             (70, OriginState(200, 'v2', 2)),
140             (130, OriginState(500, 'v2', 2)),
141         ]
142     )
143     cache = SimulatedCache(origin)
144     outcomes = run_timeline(cache, [0, 10, 80, 100,
145                               150])
146     return outcomes, dict(cache.stats)
147
148 first_outcomes, first_stats = run()
149 second_outcomes, second_stats = run()
150 assert first_outcomes == second_outcomes # bit-for
151     -bit reproducible
152 assert first_stats == second_stats
153 assert first_outcomes == [MISS, HIT,
154                           REVALIDATED_MISS, HIT, ERROR]

```

Key Concept

All five listings share one architectural motif: *correctness lives in an atomic compare-and-set at a storage boundary*. The idempotency store claims keys with a primary-key INSERT; the inventory store swaps state under one lock with the expected version in hand; the cache revalidates against a validator rather than trusting a clock. Distribution changes the storage, not the shape of the argument.

11.4 Common Pitfalls & Anti-Patterns

11.4.1 Storing the Response Before Committing the Mutation

Writing the idempotency snapshot before the business transaction commits inverts the failure window: a crash now leaves a stored success response for a mutation that rolled back. The retry then replays a receipt for a transfer that never happened. Rule: claim the key early (*in_flight*), record the response only with (or after) the durable commit, and abandon the claim on failure so the client can retry cleanly.

11.4.2 Reusing Idempotency Keys Across Endpoints

A key must identify one logical operation on one endpoint. Reusing a key on a second endpoint either replays the wrong response (if fingerprints are not checked) or 422s (if they are). Clients should mint keys per *user intent* — one per checkout form instance — and servers must enforce the fingerprint so sloppy reuse is loud, not silent.

11.4.3 Hashing Only the Body, Not the Route

A fingerprint computed from the request body alone lets the same key + identical JSON body collide across `/transfers` and `/refunds`. Listing 10.1's `fingerprint_request` hashes method and path alongside the canonical body for exactly this reason; the test `test_same_key_different_route_is_422` is the regression gate.

11.4.4 Retrying POST Without Keys

An HTTP client library that transparently retries POST on timeout is an undocumented double-charge generator. Audit every layer — your SDK, your proxy, your load balancer's retry policy — and either mint an idempotency key per logical operation or disable retries on unsafe methods. Retry budgets (Chapter 2) limit *how often*; keys limit *what repeats mean*.

11.4.5 Weak ETags on Byte-Sensitive Resources

A weak validator (`W/"abc"`) asserts semantic equivalence, not byte equality. Using one on a resource that feeds range requests, delta encodings, or cryptographic verification lets a cache combine stale bytes with fresh offsets. Rule: content-addressed (hash) or version-addressed ETags for anything byte-sensitive; weak ETags only for representations that legitimately vary in bytes while staying semantically equal.

11.4.6 The no-store / no-cache Confusion

`no-cache` *stores* the response and revalidates on every use; `no-store` never stores. Teams that write `no-cache` on responses containing personal data have built a compliant-looking privacy leak: the bytes sit in every intermediary on the path. When the data is sensitive, the directive is `no-store`, full stop; add `private` when per-user browser caching is acceptable.

11.4.7 Client-Generated ETags

Validators are issued by the authority that owns the state — the server. A client that invents an ETag (hashing its own serialization, guessing a version) couples the cache protocol to a serialization the server never committed to, and optimistic locking degenerates into a coin flip. Clients treat ETags as opaque; servers mint them; nobody else touches them.

11.4.8 Distributed Locks Without Fencing Tokens

A lease-based lock makes a paused holder and a crashed holder indistinguishable; when the lease lapses, two clients believe they hold the lock. Without fencing tokens checked at the resource, the second acquisition does not restore mutual exclusion — it merely narrows the race window. If the resource cannot check tokens, the lock is advisory and must be treated as a throughput optimization, never as a correctness mechanism [Kle17].

11.5 Hands-On Production Lab & Real-World Case Study

11.5.1 Lab: An Idempotent Payments API That Survives Retry Storms

Scenario. Northwind Robotics exposes `POST /api/v1/payments/transfers` to its dealer network. Dealers run flaky warehouse Wi-Fi; their integration library retries aggressively. Your job: ship an endpoint on which *any* number of duplicate submissions produces exactly one transfer.

Build order.

1. Start from Listing 10.1. Run `pytest test_listing_10_1.py -q` and confirm all eight tests pass.
2. Extend `TransferRequest` with a `reference` field (dealer-supplied order number) and prove by test that canonical JSON fingerprinting treats reordered keys as the same payload.
3. Lower the TTL to 5 minutes and write a test (via the `now` parameter, not `sleep`) showing an aged-out key is collected and the operation executes again.
4. Add metrics: count executions, replays, mismatches, and in-flight rejections; expose them at `GET /api/v1/_meta/idempotency`.

5. Run Listing 10.4's storm at 200 workers against your extended app. Acceptance: one ledger entry, 200 identical receipts.

Deliverable. The extended middleware, the metrics endpoint, and the passing storm output pasted into your lab notes.

11.5.2 War Story: The Double-Charged Customers

A payments team (the details are composited from several public incident reviews) shipped a new load balancer configured to retry requests on connection reset — a well-intentioned resilience default. Retries were enabled for *all* methods, including POST, on the theory that “the API is mostly idempotent anyway.” The charges endpoint was not. For three days, a small fraction of checkouts — those whose responses coincided with a connection reset — executed twice. Because both executions returned 201 with distinct charge IDs, no client-side error ever fired; the first symptom was customers reporting duplicate card charges, hours later, through the support queue. Forensics was hampered because the two charges had different IDs and timestamps seconds apart: they looked like two deliberate purchases. Remediation cost: a weekend of ledger forensics, thousands of manual refunds, and a lasting support burden.

Three structural fixes followed. (1) Retries at every infrastructure layer were restricted to idempotent methods unless the request carries an idempotency key. (2) The charges endpoint adopted idempotency keys with a 24-hour TTL — the client SDK mints one per checkout session. (3) Replay telemetry (**Idempotency-Replayed** counts) became a dashboard: a replay spike is now an early-warning signal for exactly the network conditions that used to produce double charges. The lesson generalizes: *infrastructure retries are invisible application behavior*, and any retry you did not design is a retry you have not made safe.

11.6 Self-Assessment Exercises & Deep-Dive Questions

1. **[junior]** Classify each as safe, idempotent, both, or neither: GET; PUT; DELETE; POST; PATCH with body `{"op": "set", "field": "qty", "value": 5}`; PATCH with body `{"op": "inc", "field": "qty"}`.
2. **[junior]** Your endpoint returns `Cache-Control: no-cache`. A colleague claims the response is never stored anywhere. Correct them precisely.

3. **[junior]** Given `max-age=60` and a response stored at $t = 0$, which of the requests at $t \in \{30, 60, 61\}$ hit the cache? Verify with Listing 10.5.
4. **[mid]** A client reuses one idempotency key for its whole batch job, varying only the payload. What happens, request by request, against Listing 10.1? Which status codes appear and why?
5. **[mid]** Listing 10.1 abandons the key row on downstream 5xx. Argue both sides: should a deterministic 422 (validation failure) be stored and replayed, or abandoned? What does Stripe’s design choose, and why?
6. **[mid]** Extend Listing 10.5 with a **Vary** model: the cache key becomes (URI, Accept-Language). Show a timeline in which omitting **Vary** serves a French representation to an English requester.
7. **[senior]** Design the database schema for idempotency keys when the mutation and the snapshot must commit atomically in one relational transaction. Where does the `in_flight` row live, and what does a crashed transaction leave behind?
8. **[senior]** A hot inventory SKU suffers livelock under optimistic retries: 95% of conditional writes return 412. Compare three escapes — bounded retry with jitter, pessimistic row locks, and per-SKU command queues — and pick one with numbers.
9. **[staff]** Your CDN supports surrogate keys. Design the tagging and purge scheme for the Northwind catalog so that a price change on one part invalidates exactly the affected URLs across product, search, and recommendation surfaces.
10. **[staff]** Prove or refute: “If every mutation endpoint requires an idempotency key, we no longer need unique constraints in the database.” Treat the key TTL as the crux of your argument.

11.7 Interview Q&A Vault

Junior (Q1–Q10)

Q1. Define idempotency in one sentence. Repeating an operation any number of times leaves the observable state exactly where one application left it: $f(f(x)) = f(x)$.

Q2. Which HTTP methods are idempotent by specification?

GET, HEAD, OPTIONS, TRACE (also safe), plus PUT and DELETE. POST is not; PATCH is not in general [Fie22a].

- Q3. Prove POST is not idempotent by spec.** Exhibit a counterexample permitted by the spec: `POST /orders` creates a new order per request. Two identical requests produce two orders with distinct IDs — $f(f(x)) \neq f(x)$ on the observable state (order count). Since the specification assigns POST no fixed-point guarantee, no conforming server is required to deduplicate, QED.
- Q4. Safe versus idempotent — what is the difference?** Safe methods change nothing (zero applications $\equiv$ one). Idempotent methods may change state once but not additionally on repetition. GET is both; PUT is idempotent but not safe.
- Q5. Is a second DELETE returning 404 still idempotent?** Yes. Idempotency constrains the *state*, not the status code: after the first DELETE the resource is absent, and it stays absent no matter how often the request repeats.
- Q6. What is an idempotency key and who generates it?** A unique token (typically UUIDv4) generated by the *client*, once per logical operation, sent on every retry of that operation so the server can recognize and collapse duplicates.
- Q7. What does Cache-Control: no-store mean?** Caches must not store any part of the response; every request must reach the origin. It is the correct directive for credentials and personal data.
- Q8. What does a 304 Not Modified response contain?** No body — only headers, including refreshed validators and cache directives. It tells the cache its stored representation is still current.
- Q9. What is an ETag?** An opaque, server-issued validator identifying one version of a representation. A strong ETag changes when a single byte changes; a weak one (W/ prefix) asserts only semantic equivalence.
- Q10. What status code answers a failed If-Match?** 412 Precondition Failed: the resource changed since the client's copy; the client should re-read, re-decide, and retry.

Mid (Q11–Q22)

- Q11. no-cache versus no-store?** `no-cache` permits storage but requires revalidation before every reuse; `no-store` forbids storage entirely. The names are famously misleading — memorize the semantics, not the words.
- Q12. Design idempotency for a payment endpoint.** Client mints a UUID per checkout intent and sends it as `Idempotency-Key`. Server: atomic check-and-insert of (account, key) with a fingerprint of method+route+canonical body; execute once; snapshot status+body in the same transaction; replay on duplicate; 422 on fingerprint mismatch; 409 while the twin is in flight; TTL of 24h matched to the client’s retry horizon.
- Q13. 412 versus 409 versus 428?** 428: the server requires conditional requests and none was sent. 412: a precondition (`If-Match`) was sent and evaluated false. 409: the request conflicts with server *state* — business-rule conflicts and in-flight idempotency collisions. Precondition problems are 412/428; state conflicts are 409.
- Q14. Why is exactly-once delivery impossible?** The Two Generals’ Problem: over an unreliable channel, no finite message exchange lets both parties be certain the other will act. After a timeout the sender cannot distinguish “request lost” from “response lost,” so no protocol guarantees exactly-once. Systems ship at-least-once delivery plus idempotent processing: effectively-once [Kle17].
- Q15. What are stale-while-revalidate semantics?** Within the directive’s window past expiry, the cache serves the stale representation *immediately* and refreshes it from the origin in the background. Latency stays at cache speed; staleness is bounded by the window; the next request gets the fresh copy.
- Q16. What is stale-if-error for?** Availability during origin outages: if revalidation fails with an error, the cache may serve the stale entry within the window. `must-revalidate` disables this escape hatch — choose it when a stale answer is worse than an error (e.g. account balances).
- Q17. Strong versus weak ETags — when does the difference matter?** Range requests and byte-level combining: a weak validator cannot prove two byte strings identical, so caches must not use it to satisfy partial-content requests. Content-addressed (hash) ETags are strong by construction.

- Q18. How would you fingerprint an idempotent request?** SHA-256 over method, route template, and the canonicalized body (JSON with sorted keys and fixed separators). Hashing the body alone misses cross-endpoint collisions; hashing raw bytes misses semantically identical JSON with reordered keys.
- Q19. What is the lost-update problem?** Two transactions read the same state, compute new states privately, and write; the second write overwrites the first, silently discarding one update. Prevented by conditional writes (compare-and-set on a version) or by serializing access with locks.
- Q20. What is a fencing token?** A monotonic token issued at lock acquisition and checked by the resource on every write; writes bearing an older token are rejected. It closes the paused-lock-holder hole that lease expiry alone cannot [Kle17].
- Q21. Your client retries timed-out POSTs. What can go wrong and how do you fix it?** A timeout means the outcome is unknown; a blind retry can apply the operation twice (double charge). Fix: mint an idempotency key per logical operation so the server collapses the retry into a replay, or move the operation behind a naturally idempotent PUT.
- Q22. What is cache poisoning via unkeyed input?** When a response depends on data not present in the cache key or the **Vary** set (a reflected header, an ignored query parameter), an attacker can store a malicious representation under an innocent key and have the cache serve it to other users. Defense: every response byte is a function of key plus **Vary**; never reflect unkeyed input.

Senior (Q23–Q28)

- Q23. Why must the idempotency snapshot and the business mutation commit in one transaction?** Otherwise a crash between the two leaves an unrecoverable skew: a replayable receipt for a rolled-back mutation, or a mutation with no receipt whose `in_flight` key blocks retries forever. Atomicity converts the pair into one fact [Kle17].
- Q24. Where should the dedup check live: middleware, handler, or database?** The database owns correctness (the unique constraint is the atomic compare-and-set); middleware owns the protocol mapping (422/409/ replay); the handler should stay ig-

norant. Pushing the check into application code above the store reintroduces TOCTOU races.

- Q25. How do you size the idempotency-key TTL?** From the client’s worst-case retry horizon: maximum backoff schedule times maximum attempts, plus clock skew, plus a margin for human-in-the-loop replays (support re-runs). Stripe’s 24 hours is a pragmatic upper bound for interactive clients; batch reconcilers may want days.
- Q26. A shared cache serves one user’s account page to another. Root-cause it.** A personalized response reached a shared cache without `private/no-store`, or the key omitted the authorization dimension (missing `Vary: Authorization`). The fix is directive hygiene plus key review; the prevention is a policy lint: no 200 with `Set-Cookie` or per-user data may be `public`.
- Q27. Optimistic versus pessimistic control for a hot inventory row — decide.** Measure collision rate. Below a few percent, optimistic wins (no lock overhead, collisions pay the retry). At high collision rates the retry churn dominates: move to a per-aggregate queue or row lock, trading latency for throughput stability. There is no doctrinaire answer — only a measured one.
- Q28. Design the recovery path for a client that receives 409 “idempotency key in flight.”** Bounded poll with backoff (the server hints via `Retry-After`) until the twin completes and the replay becomes available; on expiry of the bound, surface an explicit “outcome unknown, reconcile via GET” state rather than guessing either success or failure.

Staff (Q29–Q33)

- Q29. Design the end-to-end duplicate-defense architecture for a payment platform: client, edge, service, database.** Client mints one key per intent; edge terminates retry storms and never retries unsafe methods itself; service middleware maps the key protocol; the database enforces (account, key) uniqueness and commits effect+snapshot atomically; downstream consumers get effectively-once from the same keys propagated as message IDs. Each layer assumes the others fail.
- Q30. Your CDN bill tripled after a “cache everything” initiative. How do you find the waste?** Attribute misses by cause: unkeyed

variance (accidental `Vary`-like inputs such as per-user query tokens), lifetime mis-tuning (one-second `max-age`), non-cacheable methods, and key explosion from ignored parameters. Then rebuild the policy matrix per surface — immutable assets long-cached with content-hashed URLs, API responses short-cached with validators, personalized surfaces `private` or `no-store`.

Q31. When is serving stale data unacceptable, and what is the mechanism that enforces freshness under failure?

When a wrong answer is worse than no answer: balances, entitlement checks, safety interlocks. `must-revalidate` (and refusal of `stale-if-error`) converts origin failure into an explicit error rather than a confident lie; that is a product decision encoded in a header.

Q32. How do idempotency keys interact with event-driven consumers downstream of the API?

The key (or a derived operation ID) must travel with the emitted event as its dedup identity; consumers apply the same effectively-once pattern with their own dedup stores and windows. The API's replay store protects the synchronous boundary only — the async boundary needs its own [Kle17].

Q33. Governance: how do you make 40 teams adopt one idempotency and caching policy?

Codify it where it cannot be skipped: shared middleware for the key protocol (with conformance tests every service must pass), a gateway-level lint for `Cache-Control` legality, and golden dashboards for replay rate and hit ratio. Then make the paved road faster than the detour — a platform library doing the right thing by default beats a policy document every time [GH21; Nyg18].

11.8 External Bibliography & Further Reading

- **RFC 9110 — HTTP Semantics** [Fie22a]. The normative source for method safety and idempotency, status codes 304/409/412/ 428, conditional request headers (`If-Match`, `If-None-Match`, `If-Modified-Since`), ETag strength, and `Vary`.
- **RFC 9111 — HTTP Caching**. The companion specification: `Cache-Control` directives, freshness lifetimes, heuristic freshness, `stale-while-revalidate` and `stale-if-error` (registered exten-

sion directives), and cache-invalidation-on-unsafe-methods. (Read alongside RFC 9110 [Fie22a].)

- **IETF Draft — The Idempotency-Key HTTP Header Field.** The standardization effort behind the header: key generation guidance, fingerprinting, and the replay/mismatch/in-flight taxonomy implemented in Listing 10.1.
- **Stripe API Documentation — Idempotent Requests.** The canonical production deployment: client-generated keys, 24-hour retention, response replay, and the parameter-mismatch error shape.
- **Kleppmann, *Designing Data-Intensive Applications* [Kle17].** Chapters on transactions and distributed systems: lost updates, serializability, fencing tokens, and the definitive treatment of why exactly-once is impossible and effectively-once is the real target.
- **Nygard, *Release It!* [Nyg18].** Stability patterns and war stories: how retries, timeouts, and caches behave under failure in real production systems.
- **Gehl, *API Design Patterns* [GH21].** Design-pattern treatment of idempotency, request identifiers, and standard methods as building blocks.
- **Google AIP-155 (Request identification) and AIP-194 (Retries).** Google’s API Improvement Proposals for request IDs and retry-safe design: which methods may be retried and how request identifiers make retries safe.
- **FastAPI and Pydantic documentation [Ram24; Col24].** Middleware ordering, `TestClient` usage, and the validation behavior the chapter’s listings rely on.

11.9 Capstone Projects

Capstone 10.1 — Idempotency-Key Client SDK [junior]

Objective: Build the client half of the protocol: a small Python class wrapping `httpx` that mints one key per logical operation and replays it across retries, speaking to Listing 10.1’s server.

Inputs/Datasets: Listing 10.1 (server); the Northwind Robotics transfer payload fixture from the test suite.

Steps: (1) Implement `Operation` objects that mint a UUIDv4 key at

construction. (2) Implement `submit()` with bounded retry: retry on connection error and on 409 (honoring **Retry-After**), never on 422. (3) Prove a retried submit yields exactly one ledger entry. (4) Prove a tampered payload under the same key surfaces the 422 to the caller. (5) Log every attempt with attempt number and outcome.

Acceptance Criteria:

- ☐ One **Operation** retries up to 5 times and produces exactly one server-side mutation.
- ☐ 422 is never retried; 409 is retried with backoff.
- ☐ A second **Operation** mints a fresh key and executes a second transfer.
- ☐ All tests pass offline and deterministically.

Stretch Goals: Persist in-flight operations to a local file so a crashed client resumes with the same key.

Capstone 10.2 — Conditional-GET Sync Agent [junior]

Objective: Write a catalog sync agent that polls Listing 10.2's parts API on a fixed schedule and transfers bytes only when representations actually changed.

Inputs/Datasets: Listing 10.2; a scripted mutation schedule (restock events at chosen logical steps).

Steps: (1) Keep per-SKU validator state (ETag + body) in a local JSON file. (2) On each poll, send **If-None-Match** when a validator exists. (3) On 304, keep the stored body and count a zero-byte sync; on 200, replace state. (4) Run 20 polls over the scripted schedule; report bytes transferred versus a naive always-GET baseline. (5) Prove a mid-run restock is observed exactly once.

Acceptance Criteria:

- ☐ 304 polls transfer zero representation bytes.
- ☐ The agent's local state always equals the server's current representation after each poll.
- ☐ The savings report is reproducible from the scripted schedule.

Stretch Goals: Add **Last-Modified/If-Modified-Since** as a fallback when no ETag is present.

Capstone 10.3 — Optimistic-Locking Order Service [mid]

Objective: Extend Listing 10.3 into an order-reservation service in which reserving stock and releasing stock are both conditional writes, with a client that performs the read-decide-write-retry loop.

Inputs/Datasets: Listing 10.3; a seeded catalog of 10 SKUs; a deterministic order script (seeded random choice of SKU and quantity).

Steps: (1) Model reservations as quantity transitions guarded by If-Match. (2) Implement the client retry loop with bounded attempts and deterministic (seeded) jitter. (3) Drive 100 concurrent order workers from the seeded script. (4) Assert global conservation: final quantity equals initial minus sum of accepted reservations. (5) Emit a contention report: attempts histogram, 412 rate per SKU.

Acceptance Criteria:

- ☐ No lost updates under the 100-worker storm (conservation invariant holds exactly).
- ☐ Blind writes receive 428; stale writes receive 412.
- ☐ The contention report is identical across runs.

Stretch Goals: Add an adaptive escalation: SKUs whose 412 rate exceeds a threshold switch to a serialized per-SKU queue; show the throughput/latency trade-off in the report.

Capstone 10.4 — Cache-Policy Workbench [senior]

Objective: Turn Listing 10.5 into a policy evaluation tool: given a traffic timeline and a library of candidate policies, rank them by origin load, staleness window, and error resilience.

Inputs/Datasets: Listing 10.5; a synthetic one-hour request timeline (request arrivals, origin mutation events, one scripted outage); five candidate policies from `no-store` to `max-age=300`, `stale-while-revalidate=60`, `stale-if-error=600`.

Steps: (1) Load timelines and policies from a JSON fixture. (2) Simulate every (policy, timeline) pair. (3) Produce a comparison table: origin fetches, bytes saved, longest stale serve, requests failed during the outage. (4) Write a one-page recommendation memo choosing a policy per surface (product page, pricing API, account page). (5) Add a regression test asserting the ranking is stable.

Acceptance Criteria:

- ☐ The table is reproducible bit-for-bit from the fixtures.

- ☐ At least one policy is shown to serve zero failed requests during the scripted outage, and the memo explains why.
- ☐ The account page is assigned **no-store** or **private** with a one-sentence justification.

Stretch Goals: Model a shared cache with a **Vary** dimension and demonstrate a misconfiguration that leaks across user segments.

Capstone 10.5 — Correctness Audit of a Retry-Capable Gateway [staff]

Objective: Design and execute a full duplicate-safety audit for a fictitious API platform: enumerate every retry source (client SDK, proxy, load balancer, queue redelivery), classify every mutation endpoint by idempotency posture, and produce the remediation plan.

Inputs/Datasets: A synthetic platform registry you author (YAML: 20 endpoints with methods, retry sources, and current dedup posture); Listings 10.1–10.5 as the reference implementation kit.

Steps: (1) Build the retry-source inventory: for each endpoint, list every layer that can duplicate a request. (2) Classify endpoints: spec-idempotent, key-protected, unprotected. (3) Write a linter that flags unprotected unsafe methods reachable by any retry source. (4) For the top three risk endpoints, specify the fix (key protocol adoption, PUT redesign, or constraint-backed dedup) with migration steps. (5) Define the ongoing telemetry: replay rate, 409 rate, 422 rate, and the alert thresholds that would have caught the war story of Section 11.5.2.

Acceptance Criteria:

- ☐ The registry linter flags exactly the seeded violations, with zero false positives on the protected endpoints.
- ☐ Every remediation references the mechanism that enforces it (unique constraint, fingerprint, fencing token) — not just a header.
- ☐ The telemetry plan includes at least one alert tied to each failure mode in the war story.
- ☐ The plan states the blast radius of the idempotency-key store being unavailable and how the platform degrades.

Stretch Goals: Extend the audit across the async boundary: model queue redelivery for each endpoint’s emitted events and specify the consumer-side dedup windows.

11.10 Python Dependencies Appendix

All code runs offline after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

```
1 fastapi>=0.110
2 pydantic>=2.7
3 httpx>=0.27
4 pytest>=8
```

`sqlite3`, `hashlib`, `threading`, and `dataclasses` are Python standard library and need no installation.

11.10.1 `fastapi` (≥ 0.110)

Description: The ASGI web framework hosting every served API in this chapter. **Why included:** routing, the middleware extension point used by the idempotency layer (Listing 10.1), header access for conditional requests (Listings 10.2, 10.3), and the application-factory pattern that gives every test an isolated store [Ram24]. **Alternatives:** Starlette directly (FastAPI's base); Flask plus manual request validation. **Installation:** `pip install "fastapi>=0.110"`.

11.10.2 `pydantic` (≥ 2.7)

Description: Type-driven validation and serialization library. **Why included:** defines the `TransferRequest` and `InventoryUpdate` wire contracts; its declarative constraints (`gt=0`, `ge=0`) are the boundary validation the correctness arguments assume [Col24]. **Alternatives:** `dataclasses` with hand-rolled checks; `attrs`; `marshmallow`. **Installation:** `pip install "pydantic>=2.7"`.

11.10.3 `httpx` (≥ 0.27)

Description: Synchronous and asynchronous HTTP client. **Why included:** it is the transport underneath FastAPI's `TestClient`; every offline test in this chapter runs through it, and Capstone 10.1 builds the retrying client SDK on it. **Alternatives:** `requests` (synchronous only, not ASGI-native); `urllib` (`stdlib`, low-level). **Installation:** `pip install "httpx>=0.27"`.

11.10.4 pytest (≥ 8)

Description: Test framework. **Why included:** runs all 33 tests in this chapter, including the barrier-synchronized concurrency suites whose assertions prove exactly-once effect and lost-update prevention. **Alternatives:** unittest (stdlib); nose2. **Installation:** `pip install "pytest>=8"`.

11.11 Chapter Summary & Key Takeaways

- **Idempotency is a fixed-point property** — $f(f(x)) = f(x)$ on observable state. GET/PUT/DELETE have it by specification; POST gets it by design, via idempotency keys [Fie22a].
- **The Idempotency-Key protocol is a small state machine:** client-minted key, server-side fingerprint (method + route + canonical body), atomic claim, response snapshot, replay on duplicate, 422 on mismatch, 409 while in flight, expiry matched to the client's retry horizon.
- **Exactly-once delivery is impossible** (Two Generals); at-least-once delivery plus idempotent processing is the effectively-once substitute, and every dedup mechanism has a window that must be sized and monitored [Kle17].
- **Caching is a contract:** the origin assigns a freshness lifetime (`max-age`) and validators (strong ETag); the cache revalidates with `If-None-Match` and serves 304s; `no-cache` $\neq$ `no-store`; `stale-while-revalidate` buys latency, `stale-if-error` buys availability, `must-revalidate` vetoes both.
- **Concurrency control is compare-and-set at the storage boundary.** Conditional writes with `If-Match` convert lost updates into 412s; 428 demands the precondition; 409 is for state conflicts. Distributed locks need fencing tokens or they are throughput hints, not correctness.
- **Retries without idempotency are unsafe by construction.** Every infrastructure layer that retries POST silently is a latent double-charge; the fix is architectural (keys, idempotent methods), not procedural ("be careful").
- **Prove it, don't hope:** the chapter's tests collapse 50-thread storms into single mutations and 20-writer races into exact arithmetic — deterministic assertions on final state, never on timing.

11.12 Quick-Reference Card

Method	Safe	Idempotent	Retry policy
GET / HEAD	yes	yes	always safe to retry
OPTIONS / TRACE	yes	yes	always safe to retry
PUT	no	yes	safe to retry
DELETE	no	yes	safe to retry
POST	no	no	retry only with an idempotency key
PATCH	no	no	retry only if the delta is absolute or keyed

Table 11.5: Method idempotency and retry policy [Fie22a].

Directive	One-line meaning
max-age= <i>n</i>	serve fresh from cache for <i>n</i> seconds
no-cache	store OK, but revalidate before every reuse
no-store	never store; always go to origin
private / public	browser-only vs. shared-cacheable
stale-while-revalidate= <i>n</i>	serve stale $\leq n$ s while refreshing
stale-if-error= <i>n</i>	serve stale $\leq n$ s when origin errors
must-revalidate	no stale serving once expired, ever

Table 11.6: Cache-Control cheat-sheet.

Status	Select when
304	validator still matches; no body retransmission needed
409	state conflict: business rule or in-flight idempotency twin
412	If-Match/precondition evaluated false (lost update refused)
422	idempotency key reused with a different payload fingerprint
428	server requires conditional writes; precondition header missing

Table 11.7: Status-code selection for this chapter’s machinery.

11.13 Standardized Evidence Addendum

11.13.1 Benchmark Snapshot Template

11.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 11.8: Minimum benchmark table to keep per chapter.

- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

11.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 11.9: Operational budget template for this chapter's technique.

11.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

11.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,

- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

11.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

11.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 11.10: Decision card to complete per chapter.

11.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.

- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

11.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 12

Asynchronous & Event-Driven APIs: Webhooks, SSE & WebSockets

Every chapter up to this point has treated HTTP as a telephone call: the client dials, the server answers, the connection hangs up. That request/response shape is the right default for the overwhelming majority of API work — but it structurally cannot express a large and growing class of interactions in which *the server knows something first*. A robot’s battery crosses a critical threshold; a payment settles three minutes after checkout; a colleague edits the document you are viewing. In each case the interesting event originates server-side, and the client wants to know about it in milliseconds, not at the next polling interval. This chapter is about the three industrial-strength answers to that problem — **webhooks**, **Server-Sent Events (SSE)**, and **WebSockets** — and about the engineering discipline each one demands before it can survive production.

We begin with the interaction-pattern taxonomy, because the most expensive mistake in this space is choosing the wrong pattern, not implementing the right one badly. Request/response, short polling, long polling, and server push are compared on the axes that actually decide architectures: latency, fan-out cost, infrastructure compatibility, cacheability, and failure semantics. Webhooks are then treated with the seriousness they deserve as *reverse APIs*: you, the provider, become an HTTP client calling endpoints you do not operate, across networks you do not control, with delivery guarantees you must engineer explicitly. We specify the de-facto industry contract wire by wire — at-least-once delivery, bounded retries with exponential backoff, HMAC-SHA256 signatures over timestamp-plus-body in the Stripe-style `t=...,v1=...` header, timestamp tolerance windows for replay protection, endpoint verification handshakes, secret rotation, idempotent receivers, and dead-letter queues — and we give the provider and consumer

checklists that turn that contract into an operational habit.

Server-Sent Events get the same wire-level treatment: the exact `text/event-stream` grammar (the `event`, `data`, `id`, and `retry` fields, comment keep-alives, multi-line `data` folding), the `EventSource` reconnection protocol with `Last-Event-ID`, and the two production killers that rarely appear in tutorials — buffering intermediaries that swallow your stream (`X-Accel-Buffering: no`) and the HTTP/1.1 six-connections-per-origin limit that HTTP/2 removes. WebSockets are specified per RFC 6455 with the full opening handshake — including a worked derivation of `Sec-WebSocket-Accept` as `Base64(SHA-1(key || GUID))` — the frame format byte by byte, masking and the cache-poisoning rationale for it, opcodes, ping/pong keep-alive, close codes, and subprotocol negotiation. Just as importantly, we enumerate when WebSockets is the *wrong* tool: stateless CRUD, cacheable reads, and anything that must traverse hostile intermediaries. The conceptual frame throughout is the event-driven architecture canon [Kle17] and its microservice application [New21], with HTTP semantics as specified in [Fie22a].

The code is production-shaped and entirely offline. You will build a webhook receiver that verifies Stripe-style HMAC signatures with a timestamp tolerance window and an idempotent event store; a delivery worker with bounded exponential-backoff retries, permanent-4xx short-circuiting, and a dead-letter log; an SSE producer emitting well-formed `text/event-stream` with `id` and `retry` fields plus a dependency-free client parser that resumes with `Last-Event-ID`; a WebSocket echo/broadcast demo with protocol-level ping/pong and a clean close handshake, alongside a pure-stdlib proof of the `Sec-WebSocket-Accept` derivation; and an AsyncAPI 3.0 document describing the Northwind Robotics telemetry contract [Asy23]. Every listing is typed Python 3.11, deterministic, and covered by tests that run with no network, no credentials, and no clock dependence.

Key Concept

Polling asks “*has anything happened?*” on the client’s schedule; push delivers “*this happened*” on the server’s schedule. The three push mechanisms are not interchangeable: **webhooks** push between *servers* (durable, signed, retried, at-least-once), **SSE** pushes a one-way text stream to *browsers and simple clients* over plain HTTP with automatic reconnection, and **WebSockets** open a full-duplex frame-oriented channel for genuinely bidirectional interaction. Choose by direction of data flow and who the consumer is — then engineer delivery semantics, authentication, and backpressure explicitly, because none of the three gives you those for free.

12.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Classify any real-time requirement — dashboards, notifications, chat, collaborative editing, server-to-server eventing — into request/response, short polling, long polling, SSE, WebSockets, or webhooks, and defend the classification with a latency / fan-out / infrastructure trade-off matrix.
- LO-2.** Specify a production webhook contract end to end: at-least-once delivery semantics, bounded retry schedules with exponential backoff, HMAC-SHA256 signature headers in the `t=...,v1=...` scheme, timestamp tolerance windows, endpoint verification handshakes, secret rotation with overlapping `v1` entries, ordering caveats, and dead-letter handling.
- LO-3.** Implement an idempotent webhook receiver that verifies Stripe-style HMAC signatures with a timestamp tolerance window, and a delivery worker that distinguishes retryable from permanent failures — in typed Python 3.11 with FastAPI [\[Ram24\]](#).
- LO-4.** Read and write the `text/event-stream` wire format exactly: `event/data/id/retry` fields, multi-line `data` folding, comment keep-alives; explain `EventSource` reconnection with `Last-Event-ID`; and neutralize buffering proxies with `X-Accel-Buffering: no`.
- LO-5.** Derive `Sec-WebSocket-Accept` from `Sec-WebSocket-Key` using SHA-1 and the RFC 6455 GUID; parse a WebSocket frame header byte by byte (FIN, opcode, mask bit, 7/16/64-bit payload lengths); explain why client-to-server masking exists; and use ping/pong and close codes correctly.
- LO-6.** Decide when WebSockets is the wrong choice — stateless CRUD, cacheable content, hostile intermediaries, horizontally scaled stateless services — and articulate what SSE, long polling, or plain REST does better in each case.
- LO-7.** Describe an event-driven contract in AsyncAPI 3.0 (channels, messages, operations, bindings) and govern event schemas with the same rigor as OpenAPI-governed REST contracts [\[Asy23\]](#).

- LO-8.** Design consumer-side processing for at-least-once streams: idempotent consumers, poison-message quarantine, and the operational telemetry that detects retry storms before they become incidents.

Prerequisites

This chapter assumes you have completed:

- **Chapter 0** — environment setup: Python 3.11+, a virtual environment, and `pip/pytest` under PowerShell.
- **Chapter 1** — the HTTP wire protocol: methods, status codes, header fields, connection management, and the Upgrade mechanism [Fie22a].
- **Chapter 2** — consuming APIs from Python, including streaming response bodies with `httpx` [Enc24].
- **Chapters 3–4** — REST resource modeling and serialization contracts; events are just representations pushed instead of pulled.
- **Chapter 5** — client-side security fundamentals: TLS, bearer tokens [Har12], and HMAC concepts, which this chapter extends to message signing.
- **Chapter 6** — building APIs with FastAPI: routing, Pydantic models, dependency injection, and `TestClient` [Ram24].
- **Chapter 7** — error modeling with problem details, reused here for webhook rejection responses.

No network access, cloud account, or API key is required anywhere in this chapter. Every server binds to `127.0.0.1`; every fixture is synthetic, embedded, and deterministic, set in the fictional *Northwind Robotics* fleet-telemetry universe.

Warning

Webhooks invert the trust relationship you are used to. When you *send* webhooks you are an HTTP client exposed to arbitrary customer endpoints (SSRF, slow receivers, retry storms); when you *receive* them you are an unauthenticated-by-default server that the public internet will probe within minutes of deployment. Both directions require explicit security engineering — signatures, tolerances, timeouts, and bounded retries — none of which frameworks provide by default.

12.2 Deep Technical Mechanics & Architectural Concepts

12.2.1 The Interaction-Pattern Taxonomy

Strip away branding and every real-time mechanism reduces to one question: *who initiates, and how long does the connection live?*

Definition 12.1 (Interaction pattern). An *interaction pattern* is the contract about initiation and connection lifetime between a consumer and a producer of API data. The four canonical patterns are: **request/response** (consumer initiates; one response per request; connection reusable but logically stateless); **short polling** (consumer re-issues request/response on a fixed interval, usually receiving “nothing new”); **long polling** (consumer initiates; producer *holds* the request open until data is available or a deadline passes, then the consumer immediately re-issues); and **server push** (producer initiates delivery over a long-lived or provider-initiated channel: SSE, WebSocket, or an outbound webhook call).

Short polling is request/response with a timer, and its economics are brutal at scale: if a dashboard polls every 5 s for events that arrive twice an hour, over 99% of requests are empty work — yet the load balancer, TLS terminator, auth middleware, and application worker pay full price for every one. Long polling converts that waste into connection occupancy: fewer, longer requests, each of which returns the instant data arrives. It works through every proxy and firewall on earth because it *is* plain HTTP, but each notification still costs a full request/response cycle, and a “thundering reconnect” after every publish can stampede the server.

Server push inverts initiation. SSE keeps one HTTP response open forever and appends events to it — strictly server-to-client, text-only, but otherwise an ordinary HTTP response that enjoys CORS, cookies, and standard auth. WebSockets perform an HTTP Upgrade and then abandon HTTP semantics entirely for a bidirectional frame protocol. Webhooks abandon the shared connection altogether: the *provider* makes a fresh outbound HTTP POST to an endpoint the *consumer* registered, making webhooks the only pattern that works server-to-server with no consumer-side listening socket.

Figure 12.1 draws the four shapes side by side.

Two cells of Table 12.1 decide most real architectures. First: *is the consumer a browser or a server?* Browsers cannot receive inbound connections, so server-to-server push (webhooks) is out; browsers speak SSE and WebSocket natively. Second: *is the traffic one-way or two-way?* A telemetry dashboard is one-way — SSE wins on simplicity, reconnection semantics, and proxy friendliness. A collaborative editor is two-way — WebSocket is

Pattern	Typical latency	Fan-out cost	Infrastructure fit	Failure semantics
Request/response	one round trip	one request per consumer per read	universal; cacheable	clean: status code per call
Short polling	up to interval t	N consumers $\times$ $1/t$ requests, mostly empty	universal; cacheable in principle	clean; but load spikes
Long polling	near-instant per event	one held connection per consumer; reconnect stampede per event	excellent; passes any proxy	held-request timeouts; retry is client logic
SSE	near-instant, continuous	one socket per consumer; cheap frames	good; blocked by buffering proxies; HTTP/1.1 six-per-origin cap	automatic reconnect + Last-Event-ID
WebSocket	near-instant, both directions	one socket per consumer; frame overhead minimal	fair; Upgrade blocked by some proxies; not cacheable	manual reconnect; app-level heartbeat
Webhooks	seconds (queued, retried)	provider makes one outbound POST per subscriber per event	server-to-server only; consumer needs a public endpoint	at-least-once; DLQ after bounded retries

Table 12.1: Pattern selection matrix. “Latency” is data-arrival latency seen by the consumer; “fan-out cost” is the marginal cost of the N -th consumer.

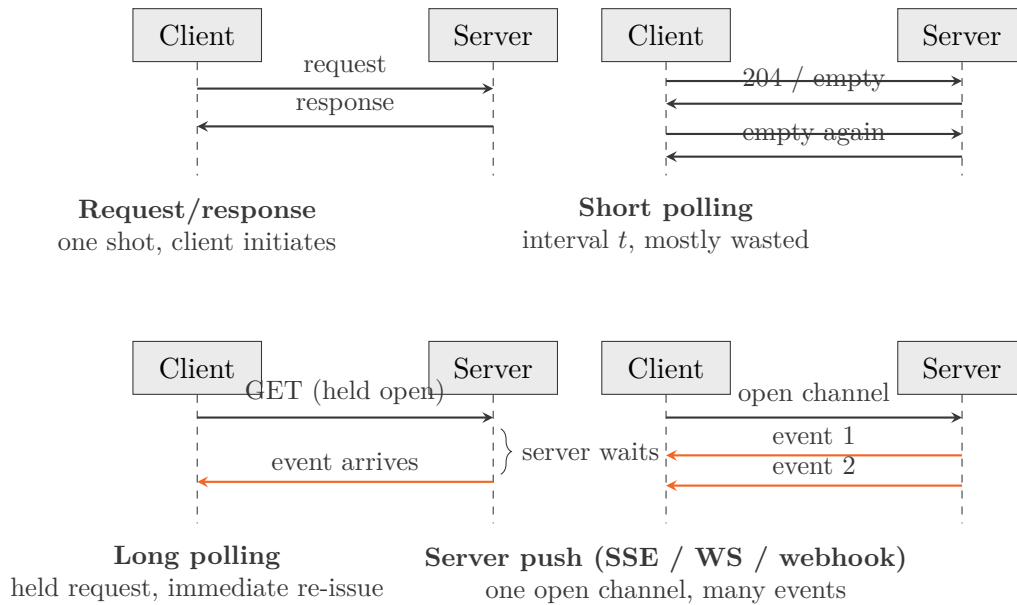


Figure 12.1: The four interaction patterns. Request/response and short polling put the client in charge of timing; long polling holds the request until the server has something to say; push keeps a channel open (or calls back out, for webhooks) so the server initiates delivery.

the only honest fit. Polling survives everywhere as the zero-infrastructure fallback, and Chapter 10’s caching machinery can make short polling of *cacheable* resources nearly free, which is why “just poll with ETag” remains the right answer more often than fashion suggests.

12.2.2 Webhooks: The Reverse-API Pattern

Definition 12.2 (Webhook). A *webhook* is a provider-initiated HTTP callback: the consumer registers an endpoint URL with the provider; when a subscribed event occurs, the provider serializes the event and POSTs it to that URL. Delivery is *at-least-once*: the provider retries until it receives a 2xx or exhausts its retry budget, so duplicates are a contractual certainty, not an edge case.

Webhooks look trivial — “just POST JSON when something happens” — and that illusion is responsible for a disproportionate share of production incidents. The provider is now an HTTP client making unsolicited calls to arbitrary internet endpoints; the consumer is now an unauthenticated server accepting unsolicited calls from the internet. Every guarantee that plain REST gets for free must be rebuilt explicitly.

Delivery semantics and retry schedules

At-least-once delivery is not a lazy implementation choice; it is the only semantics compatible with unreliable networks. Exactly-once delivery over a network partition is provably impossible (the Two Generals problem); the provider cannot distinguish “consumer crashed before processing” from “consumer processed but the 200 was lost in transit.” Since the second case *must* be retried, the first case is retried too — hence duplicates. The honest contract is: *at-least-once transport, exactly-once processing*, where the consumer achieves the second half via idempotency (Chapter 10).

A production retry schedule is bounded, exponential, and jittered. A typical shape: attempts at 0, 1, 2, 4, 8 minutes, then hourly for up to 24 or 72 hours, then dead-letter. Jitter (random $\pm 20\%$) prevents synchronized retry storms when a popular consumer recovers from an outage and ten thousand queued deliveries fire simultaneously. Crucially, the schedule distinguishes *retryable* failures — connection errors, timeouts, 408, 409, 425, 429, and 5xx — from *permanent* ones: a 400 or 422 means the payload itself is rejected and retrying it ten thousand times is an own-goal DDoS (Section 12.5 tells that war story).

Signature verification: the `t=...,v1=...` scheme

A webhook endpoint is a public URL that accepts POSTs from anyone who finds it. Without authentication, whoever guesses the URL can inject fabricated “payment succeeded” or “robot docked” events. Bearer tokens in a configured header help, but the industry standard — popularized by Stripe and now near-universal — is symmetric **HMAC-SHA256 message signing**:

$$\sigma = \text{HMAC-SHA256}(k, t \parallel "." \parallel b) \quad (12.1)$$

where k is the per-endpoint signing secret, t is the Unix timestamp at signing time, and b is the *raw, unparsed* request body. The provider sends the timestamp and signature together in one header:

```
1 POST /webhooks/telemetry HTTP/1.1
2 Host: ops.customer.example
3 Content-Type: application/json
4 X-Northwind-Signature: t=1700000000,v1=5257
                        a869e7ecebed32affa62cdca3fa51cad7e77a0e56ff536d0ce8e108d8bd
5
                        \_____/ \
                        /
6                        timestamp    hex HMAC-SHA256
                        over "1700000000.<raw-body>"
```


7	(replay	(proves possession
	of the shared secret k;	
8	anchor)	multiple v1=
	entries allowed during	
	rotation)	

The receiver recomputes Equation (12.1) over the raw body and the header’s own t , then compares with constant-time semantics (`hmac.compare_digest`), because a naive `==` on hex strings leaks prefix-match timing that can be iterated into a forgery oracle. Three independent checks, three independent failure modes:

1. **Freshness:** $|now - t| \leq \text{tolerance}$ (typically 300 s). This is the replay-protection window: a captured request replayed later carries a stale t and is rejected regardless of signature validity. The tolerance exists only to absorb clock skew and delivery latency; keep it tight.
2. **Authenticity:** at least one `v1` candidate matches the recomputed HMAC. Multiple `v1` values in one header are the secret-rotation mechanism: during rotation the provider signs with both the old and new secret, and receivers configured with either accept.
3. **Integrity:** the HMAC covers the timestamp and body together, so neither can be altered without detection. Signing the raw bytes — not a re-serialized object — matters: JSON re-serialization is not byte-stable (key order, whitespace, Unicode escapes), so verify *before* parsing.

Endpoint verification, rotation, ordering, and the DLQ

Before a provider sends real events, it must prove the registrant actually controls the endpoint — otherwise webhooks become an amplification vector for attacking third parties. The standard *verification handshake*: at registration time the provider POSTs a challenge (a random token) and requires the endpoint to echo it back, or requires the registrant to complete an out-of-band confirmation. Slack-style `url_verification` challenges and WebSub-style hub-challenge GETs are the canonical shapes.

Secret rotation follows the overlap pattern above: issue k' , sign with both k and k' for one tolerance window, then retire k . Providers must also support consumer-triggered rotation with immediate effect — when a consumer suspects a leak, “rotate within 24 hours” is not an answer.

Ordering is not part of the contract. Retries, parallel delivery workers, and per-endpoint queues conspire to deliver `created` after `updated`, or to deliver the same event twice with a third event in between. Consumers must either be order-insensitive (each event carries full current state — the

state-transfer style) or must buffer and reorder by a monotonic sequence number or `occurred_at` timestamp the provider includes in every payload. Northwind Robotics events carry both an `id` and an `occurred_at` for exactly this reason.

Finally, the **dead-letter queue**: when the retry budget is exhausted, the event and its full attempt history must land somewhere durable and alerted, with a replay tool. A webhook provider without a DLQ silently loses customer data on the first extended consumer outage; a DLQ without alerting is a write-only graveyard.

Key Concept

Provider checklist: verify endpoint ownership before first delivery; sign every delivery (HMAC-SHA256 over timestamp + raw body); retry only retryable failures, with bounded exponential backoff plus jitter; honor consumer-driven secret rotation immediately; include a unique event id and timestamp in every payload; dead-letter with alerting and a replay tool; document that ordering is best-effort. **Consumer checklist:** verify freshness *then* signature, in constant time, over raw bytes, before parsing; respond 2xx only after the event is durably recorded (or 2xx-fast with a queue in front); make processing idempotent on event id; never log raw payloads or secrets; return 4xx for malformed requests so the provider stops retrying; monitor duplicate rates — a spike means a retry storm is in progress.

Figure 12.2 captures the provider-side delivery lifecycle as a state machine; the code in Section 12.3.1 implements it exactly.

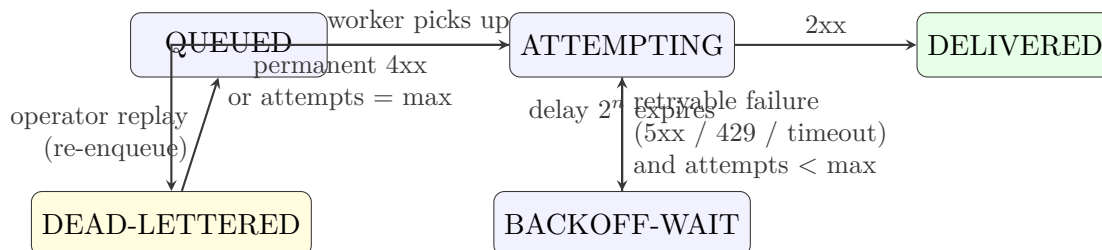


Figure 12.2: Webhook delivery state machine. Every terminal failure is either a permanent rejection (do not retry a payload the consumer has told you is invalid) or budget exhaustion into the dead-letter queue, from which only an explicit operator action restores the event.

12.2.3 Server-Sent Events: text/event-stream on the Wire

Server-Sent Events are the most underrated mechanism in this chapter: a completely standard HTTP response whose body simply never ends, carrying a tiny text protocol that browsers parse natively via the `EventSource` API. There is no upgrade, no new framing layer, no new attack surface — the stream is cache-control-compatible, CORS-compatible, cookie-authenticated, and readable with `curl`.

Definition 12.3 (Server-Sent Events). *Server-Sent Events* (SSE) is a W3C/WHATWG protocol in which a server responds to a GET with `Content-Type: text/event-stream` and then writes a sequence of UTF-8 *event blocks* separated by blank lines. Each block consists of `field: value` lines; the defined fields are `event` (named type, default `message`), `data` (the payload; multiple `data` lines are joined with `\n`), `id` (the last-event-id cursor used for reconnection), and `retry` (reconnection delay in milliseconds). Lines beginning with a colon are comments and serve as keep-alives.

The wire format, exactly as bytes on the connection:

```

1 HTTP/1.1 200 OK
2 Content-Type: text/event-stream
3 Cache-Control: no-cache
4 X-Accel-Buffering: no
5
6 : connected to northwind telemetry (comment line = keep
   -alive; proxies flush on bytes)
7
8 event: telemetry
9 data: {"robot": "NWR-07",
10 data: "battery": 0.82}
11 id: 128
12 retry: 2000
13
14 event: alert
15 data: {"robot": "NWR-07", "alert": "dock requested"}
16 id: 129
17
18 : keep-alive (sent every ~15 s so idle proxies do not
   reap the connection)

```

The grammar rules that bite in production: field name and value are separated by a colon plus one *optional* leading space; a blank line **dispatches** the buffered event; a block with no `data` field is discarded; a `data` value

split across lines is rejoined with newlines in order; `id` must not contain newlines and *persists* — the client’s last-event-id is sticky until replaced; `retry` is parsed as ASCII digits only, everything else is ignored.

Reconnection and Last-Event-ID

SSE’s killer feature is that reconnection is *protocol*, not application code. When the TCP connection drops, the browser’s `EventSource` waits the most recent `retry` interval (default three seconds), re-issues the GET, and automatically sets the `Last-Event-ID` request header to the id of the last dispatched event. A correct server reads that header and resumes the stream *after* it — from a ring buffer, an event log, or a broker offset. This gives SSE something WebSocket does not have: a standardized, zero-code resumability story. The contract has one sharp edge: the server decides how much history to retain, so “resume” is bounded by the retention window; a client that reconnects after the window has scrolled past must receive a signal (a `control` event, or an HTTP 204 telling `EventSource` to stop) and fall back to a full re-fetch.

Example

The browser side is deliberately unremarkable — that is the point:

```
1 const source = new EventSource("/stream");  
    // cookies flow automatically  
2 source.addEventListener("telemetry", (e) => render  
    (JSON.parse(e.data)));  
3 source.addEventListener("alert",      (e) =>  
    escalate(JSON.parse(e.data)));  
4 source.onerror = () => { /* browser retries  
    automatically with Last-Event-ID */ };  
5 // e.lastEventId is the id: field; server-set "  
    retry: 2000" controls the backoff
```

No reconnection logic, no heartbeat logic, no framing logic. Everything that WebSocket leaves to you, the SSE user agent already does.

Proxy buffering and HTTP/2

Two infrastructure facts kill more SSE deployments than any protocol bug. First, **buffering intermediaries**: many reverse proxies (nginx in default proxy mode, some CDNs, some corporate egress proxies) buffer response bodies before forwarding — which turns your real-time stream into a not-real-time dump delivered when the buffer fills or the connection closes. The defenses: set `X-Accel-Buffering: no` (honored by nginx

and Envoy), send `Cache-Control: no-cache`, flush after every event, and emit comment keep-alives every 10–15 seconds so idle-timeout middleboxes see traffic. Second, **the HTTP/1.1 connection cap**: browsers allow only six concurrent connections per origin, and each `EventSource` holds one for its entire lifetime — two open dashboard tabs can starve every other request to that origin. Over HTTP/2 the cap effectively disappears (streams are multiplexed over one connection), so SSE *wants* HTTP/2 — with the caveat that a single TCP connection’s head-of-line blocking then applies to everything multiplexed on it.

12.2.4 WebSockets: RFC 6455 Precisely

WebSocket is not “HTTP but faster.” It is a distinct, frame-oriented, bidirectional protocol that borrows HTTP only for its opening handshake — deliberately, so that the same port 80/443 infrastructure, TLS termination, and Upgrade machinery [Fie22a] can be reused.

The opening handshake

The client sends an ordinary HTTP/1.1 GET with `Upgrade: websocket`, `Connection: Upgrade`, a protocol version (`Sec-WebSocket-Version: 13`), and a randomly generated 16-byte nonce, Base64-encoded, in `Sec-WebSocket-Key`. The server proves it understood the WebSocket upgrade — and was not merely an HTTP server echoing headers — by computing:

$$\text{Sec-WebSocket-Accept} = \text{Base64}\left(\text{SHA-1}\left(\text{Sec-WebSocket-Key} \parallel \text{"258EAF5E-E914-47DA-95CA-C5AB0DC85B11"}\right)\right) \quad (12.2)$$

The magic GUID is fixed by RFC 6455 section 1.3; it guarantees the concatenated string can never be a plausible HTTP artifact, so a non-WebSocket server cannot accidentally produce a valid accept value. The worked example from the RFC itself — which Listing 11.6 reproduces and asserts in pure stdlib Python — is:

```

1 Client:  Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
2 Server:  SHA-1("dGhlIHNhbXBsZSBub25jZQ==258EAF5E-E914
          -47DA-95CA-C5AB0DC85B11")
3          = 0xb3 0x7a 0x4f 0x2c 0xc0 0x62 0x4f 0x16 0x90 0
          xf6
4          0x46 0x06 0xcf 0x38 0x59 0x45 0xb2 0xbe 0xc4 0
          xea
5          -> Base64 -> Sec-WebSocket-Accept:
          s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

```


Why masking exists is the most-asked WebSocket interview question, and the answer is not confidentiality. Client-to-server frames are XOR-masked with a fresh random 32-bit key per frame to defend *intermediaries*, not endpoints: researchers showed that a hostile script in a browser could otherwise craft bytes that, to a misbehaving transparent proxy, look like a smuggled HTTP request — poisoning shared caches (the 2010 “cache poisoning” attack that motivated the mask). Because the key is chosen by the sender’s stack after the application supplied the bytes, the attacker cannot predict what the masked output will look like on the wire. Masking costs a few cycles; server-to-client frames skip it.

Ping/pong (opcodes 0x9/0xA) is protocol-level keep-alive and liveness detection: either peer may ping; the other must pong “as soon as practical” with identical payload. TCP itself will happily keep a dead connection “open” for hours, so applications that care about zombie connections ping every 20–30 seconds and close after a missed pong. **Close** (0x8) is a handshake, not a hang-up: the initiator sends a close frame with a 2-byte status code and optional UTF-8 reason; the peer echoes a close frame; then the TCP connection drops. Key codes: 1000 normal, 1001 going away (server restart / page navigation), 1002 protocol error, 1003 unsupported data, 1006 is *reserved* for abnormal closure and must never appear on the wire, 1007 invalid payload, 1008 policy violation, 1009 message too big, 1011 internal server error.

When WebSockets is the wrong choice

WebSocket’s power is exactly its cost: it abandons HTTP semantics, so nothing in the HTTP toolchain applies — no caching, no ETag, no idempotency vocabulary, no status codes, no content negotiation, no meaningful gateway routing. Reach for something else when:

- **The work is stateless CRUD.** Create/read/update/delete with request/response semantics wants REST: you get caching, retries, tracing, and firewall friendliness for free [Fie22a].
- **The data is cacheable.** A WebSocket frame cannot be cached by a CDN; an ETag-pollled resource can be nearly free at scale.
- **The traffic is one-way.** SSE is simpler, resumable, and proxy-friendlier; WebSocket buys you a send channel you will never use and bills you in connection-management code.
- **Hostile intermediaries are in the path.** Some corporate proxies and older load balancers mangle or refuse Upgrade. SSE and long polling degrade gracefully; WebSocket just fails.

- **You need horizontal statelessness.** Each WebSocket pins a client to a server process; broadcast across a fleet requires a backplane (broker fan-out, Redis pub/sub). SSE has the same pinning but simpler recovery, because reconnection with **Last-Event-ID** is built in.

Figure 12.3 contrasts the two push topologies directly.

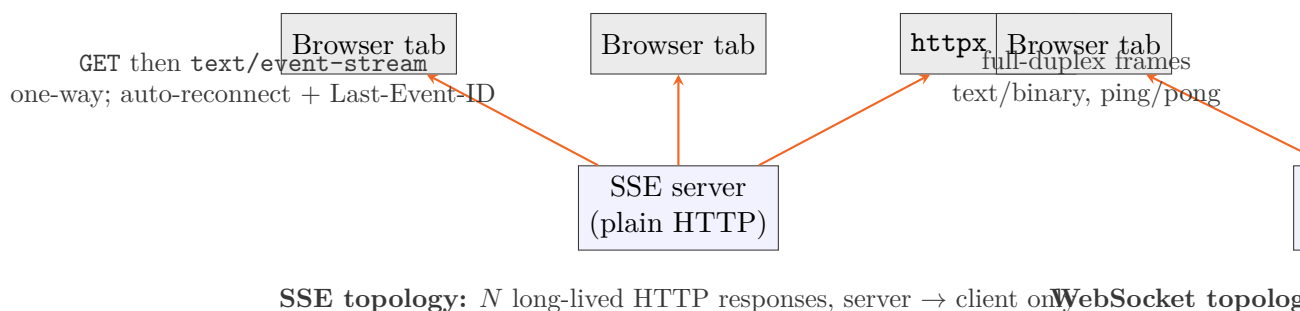


Figure 12.3: SSE versus WebSocket topology. Both pin one socket per client; SSE’s channel is a plain HTTP response with standardized resume, while WebSocket’s is a symmetric frame channel requiring application-level reconnection.

12.2.5 AsyncAPI and the Governance of Event Contracts

REST APIs got their contract discipline from OpenAPI; event-driven APIs get theirs from **AsyncAPI** [Asy23]. The conceptual mapping is direct: where OpenAPI has paths and operations, AsyncAPI has *channels* (addressable destinations such as a Kafka topic `northwind.robots.{robot_id}.telemetry`) and *operations* (send/receive over a channel); where OpenAPI has request/response schemas, AsyncAPI has *messages* with payload schemas; *bindings* attach protocol-specific configuration (Kafka partitions, MQTT QoS, WebSocket subprotocols) without polluting the logical contract. Listing 11.8 shows the Northwind Robotics telemetry contract in AsyncAPI 3.0.

Governance is the point, not the YAML. Event schemas evolve under exactly the same compatibility algebra as REST representations (Chapter 9): additive changes are safe, removals and renamings break consumers you have never met — and in event-driven systems you *cannot* meet them, because the whole point of the broker is that producers do not know their consumers. Discipline therefore means: a schema registry or repository as the single source of truth, CI gates that reject breaking schema diffs, explicit compatibility modes (backward for consumers-first rollout), and a

deprecation workflow for event types exactly as for endpoints. The consumer-side corollaries mirror the webhook contract from Section 12.2.2: assume **at-least-once** delivery and make every consumer idempotent on the event id; treat **poison messages** — events that crash or reject deterministically — as a routing problem (quarantine to a dead-letter topic after n attempts) rather than an infinite retry; and never trust the payload beyond the schema, because a broker is a trust boundary like any other [Kle17; New21].

12.3 Production-Ready Code Implementation

All listings run on Python 3.11+ (verified on 3.13) with the dependencies in Section 12.10. Everything binds to 127.0.0.1 or uses in-process ASGI transport: no network, no credentials, no wall-clock dependence. PowerShell setup:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
4 pytest -q
```

12.3.1 Webhooks: Signed Receiver and Bounded-Retry Delivery Worker

Listing 11.1 implements the consumer side of Section 12.2.2: freshness check, constant-time signature comparison over the raw body, and an idempotency store that turns at-least-once delivery into exactly-once processing. The clock is injectable so tests are deterministic.

```
1 """Webhook receiver with Stripe-style HMAC-SHA256
   signature verification."""
2 from __future__ import annotations
3
4 import hashlib
5 import hmac
6 import json
7 import time
8 from collections.abc import Callable
9 from dataclasses import dataclass, field
10 from typing import Any
11
12 from fastapi import FastAPI, Request
13 from fastapi.responses import JSONResponse
14
```

```
15 SIGNATURE_HEADER = "x-northwind-signature"
16 TOLERANCE_SECONDS = 300
17
18
19 def compute_signature(secret: str, timestamp: int, body
20 : bytes) -> str:
21     """Return the hex HMAC-SHA256 of <timestamp>.<body>"""
22     signed_payload = str(timestamp).encode("utf-8") + b
23     "." + body
24     digest = hmac.new(secret.encode("utf-8"),
25     signed_payload, hashlib.sha256)
26     return digest.hexdigest()
27
28
29 def parse_signature_header(header: str) -> tuple[int,
30 list[str]]:
31     """Parse t=...,v1=... into (timestamp, [
32     signatures]); ValueError if malformed."""
33     timestamp: int | None = None
34     signatures: list[str] = []
35     for part in header.split(","):
36         key, _, value = part.partition("=")
37         if key.strip() == "t":
38             try:
39                 timestamp = int(value)
40             except ValueError as exc:
41                 raise ValueError("non-integer timestamp") from exc
42         elif key.strip() == "v1":
43             signatures.append(value.strip())
44     if timestamp is None or not signatures:
45         raise ValueError("malformed signature header")
46     return timestamp, signatures
47
48
49 @dataclass
50 class IdempotencyStore:
51     """In-memory set of processed event ids (swap for Redis/Postgres in prod)."""
52
53     processed: set[str] = field(default_factory=set)
54
55     def is_duplicate(self, event_id: str) -> bool:
56         return event_id in self.processed
```

```

53     def mark_processed(self, event_id: str) -> None:
54         self.processed.add(event_id)
55
56
57 def _problem(status: int, title: str, detail: str) ->
JSONResponse:
58     return JSONResponse(
59         status_code=status,
60         media_type="application/problem+json",
61         content={"type": "about:blank", "title": title,
62                 "status": status, "detail": detail},
63     )
64
65
66 def create_app(
67     secret: str,
68     store: IdempotencyStore,
69     now: Callable[[], float] = time.time,
70     tolerance_seconds: int = TOLERANCE_SECONDS,
71 ) -> FastAPI:
72     app = FastAPI(title="Northwind_Webhook_
Receiver")
73
74     @app.post("/webhooks/telemetry")
75     async def receive_telemetry(request: Request) ->
JSONResponse:
76         body = await request.body()
77         raw_header = request.headers.get(
SIGNATURE_HEADER, "")
78         try:
79             timestamp, signatures =
parse_signature_header(raw_header)
80         except ValueError:
81             return _problem(400, "Malformed_Signature",
"Signature_header_is_
82                 missing_or_malformed.")
83
84         # Replay protection: refuse events signed too
85         # far from our clock.
86         if abs(now() - timestamp) > tolerance_seconds:
87             return _problem(400, "Stale_Timestamp",
"Signature_timestamp_
88                 outside_tolerance_window
89                 .")
89
90         expected = compute_signature(secret, timestamp,

```

```

    body)
90     if not any(hmac.compare_digest(expected, c) for
    c in signatures):
91         return _problem(400, "Invalid_Signature",
92                           "No_v1_signature_matches_
    the_payload.")
93
94     try:
95         event: dict[str, Any] = json.loads(body)
96         event_id = str(event["id"])
97     except (json.JSONDecodeError, KeyError,
    TypeError):
98         return _problem(400, "Malformed_Event",
99                           "Body_is_not_a_JSON_event_
    with_an_id_field.")
100
101     # Idempotent receiver: at-least-once delivery
    means duplicates WILL arrive.
102     if store.is_duplicate(event_id):
103         return JSONResponse(status_code=200,
104                             content={"status": "
    duplicate", "id":
    event_id})
105     store.mark_processed(event_id)
106     return JSONResponse(status_code=200,
107                         content={"status": "
    processed", "id":
    event_id})
108
109     return app
```

Listing 11.2 is the provider side: the state machine of Figure 12.2 in code. Transport, clock, and sleeper are injected; backoff delays are pure functions of the attempt number, so the test suite observes the schedule without sleeping.

```

1  """Webhook_delivery_worker: bounded_retries, backoff,
    and_a_dead-letter_log."""
2  from __future__ import annotations
3
4  import json
5  import time
6  from collections.abc import Callable
7  from dataclasses import dataclass
8  from enum import Enum
9  from typing import Any
10
```

```

11 from webhook_receiver import compute_signature
12
13 RETRYABLE_STATUSES = frozenset({408, 409, 425, 429,
14                                500, 502, 503, 504})
15
16 class DeliveryOutcome(str, Enum):
17     DELIVERED = "delivered"
18     DEAD_LETTERED = "dead_lettered"
19     REJECTED_PERMANENT = "rejected_permanent"
20
21
22 @dataclass(frozen=True)
23 class WebhookEvent:
24     event_id: str
25     event_type: str
26     occurred_at: int
27     payload: dict[str, Any]
28
29     def body_bytes(self) -> bytes:
30         doc = {"id": self.event_id, "type": self.
31               event_type,
32               "occurred_at": self.occurred_at, "data":
33                 self.payload}
34         return json.dumps(doc, sort_keys=True).encode("
35               utf-8")
36
37
38 @dataclass(frozen=True)
39 class Attempt:
40     number: int
41     status: int | None # None => transport raised (
42                       connection refused, DNS, ...)
43     retryable: bool
44
45
46 @dataclass(frozen=True)
47 class DeadLetter:
48     event_id: str
49     endpoint: str
50     attempts: tuple[Attempt, ...]
51     reason: str
52
53
54 Transport = Callable[[str, dict[str, str], bytes], int]

```

```
52
53 class WebhookDeliverer:
54     def __init__(
55         self,
56         secret: str,
57         transport: Transport,
58         *,
59         max_attempts: int = 5,
60         base_delay_seconds: float = 1.0,
61         now: Callable[[], float] = time.time,
62         sleeper: Callable[[float], None] = time.sleep,
63     ) -> None:
64         if max_attempts < 1:
65             raise ValueError("max_attempts must be >= 1")
66
67         self._secret = secret
68         self._transport = transport
69         self._max_attempts = max_attempts
70         self._base_delay = base_delay_seconds
71         self._now = now
72         self._sleeper = sleeper
73         self.dead_letters: list[DeadLetter] = []
74
75     def backoff_delay(self, attempt_number: int) ->
76         float:
77         """Exponential backoff with a deterministic cap
78         : 1, 2, 4, 8, ... capped at 60s."""
79         return min(self._base_delay * (2 ** (
80             attempt_number - 1)), 60.0)
81
82     def deliver(self, endpoint: str, event:
83         WebhookEvent) -> DeliveryOutcome:
84         body = event.body_bytes()
85         attempts: list[Attempt] = []
86         for number in range(1, self._max_attempts + 1):
87             timestamp = int(self._now())
88             headers = {
89                 "content-type": "application/json",
90                 "x-northwind-signature": (
91                     f"t={timestamp},v1={
92                         compute_signature(self._secret,
93                             timestamp, body)}"
94                 ),
95             }
96             try:
97                 status = self._transport(endpoint,
```

```

        headers, body)
91         retryable = status in
            RETRYABLE_STATUSES
92     except OSError:
93         status, retryable = None, True
94         attempts.append(Attempt(number=number,
            status=status, retryable=retryable))
95
96     if status is not None and 200 <= status <
        300:
97         return DeliveryOutcome.DELIVERED
98     if status is not None and not retryable:
99         # 4xx (other than 408/409/425/429)
            means "stop calling me";
100        # retrying forever is how webhook loops
            turn into a DDoS.
101        self._dead_letter(event, endpoint,
            attempts, f"permanent_{status}")
102        return DeliveryOutcome.
            REJECTED_PERMANENT
103    if number < self._max_attempts:
104        self._sleeper(self.backoff_delay(number
            ))
105
106    self._dead_letter(event, endpoint, attempts, "
        max_attempts_exhausted")
107    return DeliveryOutcome.DEAD_LETTERED
108
109    def _dead_letter(
110        self, event: WebhookEvent, endpoint: str,
            attempts: list[Attempt], reason: str
111    ) -> None:
112        self.dead_letters.append(
113            DeadLetter(event.event_id, endpoint, tuple(
                attempts), reason)
114        )

```

The test suite (Listing 11.3) covers the attacks and failure modes that matter: tampered bodies, stale timestamps, wrong secrets, replayed deliveries, retry-then-succeed, budget exhaustion into the DLQ, and the permanent-4xx short-circuit.

```

1  """Offline, deterministic tests for the webhook
    receiver and delivery worker."""
2  from __future__ import annotations
3
4  import json

```

```
5
6 import pytest
7 from fastapi.testclient import TestClient
8
9 from webhook_delivery import DeliveryOutcome,
    WebhookDeliverer, WebhookEvent
10 from webhook_receiver import (
11     IdempotencyStore, compute_signature, create_app,
12     parse_signature_header,
13 )
14 SECRET = "whsec_northwind_test_only"
15 NOW = 1_700_000_000 # fixed clock: no wall-time
    dependence
16
17
18 def signed_headers(secret: str, timestamp: int, body:
    bytes) -> dict[str, str]:
19     sig = compute_signature(secret, timestamp, body)
20     return {"x-northwind-signature": f"t={timestamp},v1
        ={sig}"}
21
22
23 def event_body(event_id: str = "evt_1001") -> bytes:
24     return json.dumps({"id": event_id, "type": "robot.
        telemetry",
25                        "data": {"robot": "NWR-07", "
        battery": 0.82}}).encode()
26
27
28 @pytest.fixture()
29 def client() -> TestClient:
30     app = create_app(SECRET, IdempotencyStore(), now=
        lambda: float(NOW))
31     return TestClient(app)
32
33
34 def test_valid_signature_is_processed(client:
    TestClient) -> None:
35     body = event_body()
36     resp = client.post("/webhooks/telemetry", content=
        body,
37                        headers=signed_headers(SECRET,
        NOW, body))
38     assert resp.status_code == 200
39     assert resp.json() == {"status": "processed", "id":
```



```

    "evt_1001"}
40
41
42 def test_tampered_body_is_rejected(client: TestClient)
    -> None:
43     body = event_body()
44     headers = signed_headers(SECRET, NOW, body)
45     tampered = body.replace(b"0.82", b"0.01") #
        attacker flips the battery level
46     resp = client.post("/webhooks/telemetry", content=
        tampered, headers=headers)
47     assert resp.status_code == 400
48     assert resp.json()["title"] == "Invalid_Signature"
49
50
51 def test_stale_timestamp_is_rejected(client: TestClient
    ) -> None:
52     body = event_body()
53     old = NOW - 3600 # signed an hour ago: outside the
        300 s tolerance
54     resp = client.post("/webhooks/telemetry", content=
        body,
55
        headers=signed_headers(SECRET,
        old, body))
56     assert resp.status_code == 400
57     assert resp.json()["title"] == "Stale_Timestamp"
58
59
60 def test_wrong_secret_is_rejected(client: TestClient)
    -> None:
61     body = event_body()
62     headers = signed_headers("whsec_attacker_guess",
        NOW, body)
63     resp = client.post("/webhooks/telemetry", content=
        body, headers=headers)
64     assert resp.status_code == 400
65
66
67 def test_replayed_delivery_is_idempotent(client:
    TestClient) -> None:
68     body = event_body()
69     headers = signed_headers(SECRET, NOW, body)
70     first = client.post("/webhooks/telemetry", content=
        body, headers=headers)
71     replay = client.post("/webhooks/telemetry", content
        =body, headers=headers)

```

```
72     assert first.json()["status"] == "processed"
73     assert replay.status_code == 200 # ack again, but
       do not reprocess
74     assert replay.json()["status"] == "duplicate"
75
76
77 def test_header_parsing_supports_rotation_overlap() ->
  None:
78     ts, sigs = parse_signature_header("t=1700000000,v1=
       aaa,v1=bbb")
79     assert ts == NOW
80     assert sigs == ["aaa", "bbb"]
81
82
83 def make_event() -> WebhookEvent:
84     return WebhookEvent("evt_1001", "robot.telemetry",
       NOW, {"robot": "NWR-07"})
85
86
87 class FakeTransport:
88     """Scripted statuses; records every call. OSError
       simulates connection failure."""
89
90     def __init__(self, script: list[int | type[OSError
       ]]) -> None:
91         self.script = script
92         self.calls: list[tuple[str, dict[str, str],
       bytes]] = []
93
94     def __call__(self, url: str, headers: dict[str, str
       ], body: bytes) -> int:
95         self.calls.append((url, headers, body))
96         outcome = self.script.pop(0)
97         if isinstance(outcome, type):
98             raise outcome("connection_refused")
99         return outcome
100
101
102 def make_deliverer(transport: FakeTransport) -> tuple[
  WebhookDeliverer, list[float]]:
103     sleeps: list[float] = []
104     deliverer = WebhookDeliverer(
105         SECRET, transport, max_attempts=5,
106         base_delay_seconds=1.0,
107         now=lambda: float(NOW), sleeper=sleeps.append,
```

```

108     return deliverer, sleeps
109
110
111 def test_delivery_succeeds_first_try() -> None:
112     transport = FakeTransport([200])
113     deliverer, sleeps = make_deliverer(transport)
114     outcome = deliverer.deliver("https://ops.northwind-
115         robotics.example/hook", make_event())
116     assert outcome is DeliveryOutcome.DELIVERED
117     assert len(transport.calls) == 1 and sleeps == []
118     and deliverer.dead_letters == []
119
120 def
121     test_delivery_retries_5xx_with_backoff_then_succeeds
122     () -> None:
123         transport = FakeTransport([500, 503, 200])
124         deliverer, sleeps = make_deliverer(transport)
125         outcome = deliverer.deliver("https://ops.northwind-
126             robotics.example/hook", make_event())
127         assert outcome is DeliveryOutcome.DELIVERED
128         assert len(transport.calls) == 3
129         assert sleeps == [1.0, 2.0] # deterministic
130             exponential backoff, no real waiting
131
132 def test_delivery_dead_letters_after_max_attempts() ->
133     None:
134         transport = FakeTransport([500, 500, 500, 500,
135             500])
136         deliverer, sleeps = make_deliverer(transport)
137         outcome = deliverer.deliver("https://ops.northwind-
138             robotics.example/hook", make_event())
139         assert outcome is DeliveryOutcome.DEAD_LETTERED
140         assert sleeps == [1.0, 2.0, 4.0, 8.0]
141         assert len(deliverer.dead_letters) == 1
142         assert deliverer.dead_letters[0].reason == "max_
143             attempts_exhausted"
144
145 def test_delivery_does_not_retry_permanent_4xx() ->
146     None:
147         transport = FakeTransport([422])
148         deliverer, sleeps = make_deliverer(transport)
149         outcome = deliverer.deliver("https://ops.northwind-
150             robotics.example/hook", make_event())

```

```
142     assert outcome is DeliveryOutcome.  
        REJECTED_PERMANENT  
143     assert len(transport.calls) == 1 and sleeps == []  
144     assert deliverer.dead_letters[0].reason == "  
        permanent_422"  
145  
146  
147 def test_network_error_is_retryable() -> None:  
148     transport = FakeTransport([OSError, 200])  
149     deliverer, _ = make_deliverer(transport)  
150     outcome = deliverer.deliver("https://ops.northwind-  
        robotics.example/hook", make_event())  
151     assert outcome is DeliveryOutcome.DELIVERED  
152     assert transport.calls[0][1]["x-northwind-signature"  
        "].startswith(f"t={NOW},v1=")
```

12.3.2 Server-Sent Events: Producer, Parser, and Resume

Listing 11.4 is both halves of the SSE contract. The producer is a FastAPI `StreamingResponse` [Ram24] emitting well-formed `text/event-stream` — `event/data/id/ retry` fields, multi-line data folding, comment keep-alives, the `X-Accel-Buffering: no escape hatch` — and honoring `Last-Event-ID` resume from the scripted feed. The client parser is a pure generator over lines implementing the WHATWG dispatch rules, so it works over `httpx` [Enc24], the `stdlib`, or a test fixture.

```
1  """Server-Sent Events: FastAPI producer + a dependency-  
    free client parser."""  
2  from __future__ import annotations  
3  
4  from collections.abc import AsyncIterator, Iterable,  
    Iterator  
5  from dataclasses import dataclass  
6  
7  from fastapi import FastAPI, Request  
8  from fastapi.responses import StreamingResponse  
9  
10 # The scripted Northwind Robotics telemetry feed (  
    synthetic, deterministic).  
11 TELEMETRY_FEED: list[tuple[int, str, str]] = [  
12     (1, "telemetry", '{"robot": "NWR-07", "battery":  
        0.82}'),  
13     (2, "telemetry", '{"robot": "NWR-07", "battery":  
        0.79}'),
```

```

14         (3, "alert", '{"robot": "NWR-07", "alert": "dock_
15             requested"}'),
16     ]
17     RETRY_MS = 2000
18
19     def format_sse(data: str, *, event: str | None = None,
20                   event_id: int | None = None, retry: int
21                   | None = None) -> str:
22         """Serialize one SSE event block. Multi-line data
23         gets one "data:" per line."""
24         lines: list[str] = []
25         if event is not None:
26             lines.append(f"event: {event}")
27         for data_line in data.splitlines():
28             lines.append(f"data: {data_line}")
29         if event_id is not None:
30             lines.append(f"id: {event_id}")
31         if retry is not None:
32             lines.append(f"retry: {retry}")
33         return "\n".join(lines) + "\n\n"
34
35     @dataclass(frozen=True)
36     class ServerSentEvent:
37         data: str
38         event: str = "message"
39         event_id: str | None = None
40         retry: int | None = None
41
42     def parse_sse(lines: Iterable[str]) -> Iterator[
43         ServerSentEvent]:
44         """Parse text/event-stream lines per the WHATWG
45         dispatch rules.
46
47         A blank line dispatches the buffered event; lines
48         starting with ":" are
49         comments (keep-alives) and are ignored.
50
51         """
52         data_parts: list[str] = []
53         event_type: str | None = None
54         event_id: str | None = None
55         retry: int | None = None
56
57         def dispatch() -> ServerSentEvent | None:

```

```
54     nonlocal data_parts, event_type
55     if not data_parts:
56         event_type = None
57         return None
58     parsed = ServerSentEvent("\n".join(data_parts),
59                                event_type or "message", event_id, retry)
60     data_parts, event_type = [], None
61     return parsed
62
63 for raw in lines:
64     line = raw.rstrip("\r\n")
65     if line == "":
66         dispatched = dispatch()
67         if dispatched is not None:
68             yield dispatched
69     elif line.startswith(":"):
70         continue
71     else:
72         field, _, value = line.partition(":")
73         if value.startswith("␣"):
74             value = value[1:]
75         if field == "data":
76             data_parts.append(value)
77         elif field == "event":
78             event_type = value
79         elif field == "id":
80             event_id = value
81         elif field == "retry":
82             if value.isdigit():
83                 retry = int(value)
84     dispatched = dispatch()
85     if dispatched is not None:
86         yield dispatched
87
88
89 def create_app() -> FastAPI:
90     app = FastAPI(title="Northwind␣Robotics␣Telemetry␣
91                    Stream")
92
93     async def event_generator(last_event_id: int) ->
94         AsyncIterator[str]:
95         # Announce the reconnection delay first, then
96         # resume AFTER last_event_id.
97         yield format_sse("connected", event="control",
98                          retry=RETRY_MS)
```

```

95         for event_id, event_type, data in
96             TELEMETRY_FEED:
97                 if event_id > last_event_id:
98                     yield format_sse(data, event=event_type
99                                     , event_id=event_id)
100             yield ":_keep-alive\n\n"
101
102 @app.get("/stream")
103 async def stream(request: Request) ->
104     StreamingResponse:
105         raw_last_id = request.headers.get("last-event-
106                                         id", "0")
107         last_event_id = int(raw_last_id) if raw_last_id
108             .isdigit() else 0
109         return StreamingResponse(
110             event_generator(last_event_id),
111             media_type="text/event-stream",
112             headers={"Cache-Control": "no-cache", "X-
113                     Accel-Buffering": "no"},
114         )
115
116 return app

```

The integration tests (Listing 11.5) exercise the full stack with FastAPI's in-process `TestClient` — no sockets — and prove the resume contract: reconnecting with `Last-Event-ID: 2` replays only event 3.

```

1  """Offline tests for the SSE producer, parser, and Last
2  -Event-ID resume."""
3
4  from __future__ import annotations
5
6  from fastapi.testclient import TestClient
7
8  from sse_streams import RETRY_MS, create_app,
9      format_sse, parse_sse
10
11 FIXTURE_STREAM = (
12     ":_this_is_a_comment_keep-alive\n"
13     "\n"
14     "event:_telemetry\n"
15     "data:_{\"robot\":_\"NWR-07\",_}\n"
16     "data:_{\"battery\":_0.82}\n"
17     "id:_42\n"
18     "retry:_1500\n"
19     "\n"
20     "data:_bare_message_with_default_event_type\n"
21     "\n"

```

```
19 )
20
21
22 def
    test_parser_handles_comments_multiline_data_id_and_retry
    () -> None:
23     events = list(parse_sse(FIXTURE_STREAM.splitlines(
        keepends=True)))
24     assert len(events) == 2
25     first, second = events
26     assert first.event == "telemetry"
27     assert first.data == '{"robot": "NWR-07",\n"battery
        ": 0.82}' # multi-line join
28     assert first.event_id == "42"
29     assert first.retry == 1500
30     assert second.event == "message" # default type
31     assert second.retry == 1500 # retry persists until
        replaced
32
33
34 def test_format_sse_round_trips_through_parser() ->
    None:
35     wire = format_sse("line_one\nline_two", event="
        alert", event_id=7, retry=500)
36     (event,) = list(parse_sse(wire.splitlines(keepends=
        True)))
37     assert (event.data, event.event, event.event_id,
        event.retry) == (
38         "line_one\nline_two", "alert", "7", 500,
39     )
40
41
42 def test_server_streams_all_events_with_ids() -> None:
43     client = TestClient(create_app())
44     with client.stream("GET", "/stream") as response:
45         assert response.status_code == 200
46         assert response.headers["content-type"].
            startswith("text/event-stream")
47         assert response.headers["x-accел-buffering"] ==
            "no"
48         events = list(parse_sse(response.iter_lines()))
49     ids = [e.event_id for e in events if e.event_id is
        not None]
50     assert ids == ["1", "2", "3"]
51     assert events[0].retry == RETRY_MS
52     assert events[-1].event == "alert"
```



```

53
54
55 def test_server_resumes_after_last_event_id() -> None:
56     """Reconnect with Last-Event-ID: 2-> only event 3
        is replayed."""
57     client = TestClient(create_app())
58     with client.stream("GET", "/stream", headers={"Last-Event-ID": "2"}) as response:
59         events = list(parse_sse(response.iter_lines()))
60     replayed = [e for e in events if e.event_id is not None]
61     assert [e.event_id for e in replayed] == ["3"]
62     assert replayed[0].event == "alert"

```

12.3.3 WebSockets: Handshake Math and an Echo/Broadcast Demo

Listing 11.6 proves the handshake and then drives a real (localhost-only) WebSocket session with the `websockets` library: echo to the sender, broadcast to peers, protocol-level ping/pong, and a clean 1000 close. Synchronization is via `asyncio.Event`, never sleep-based timing assertions.

```

1  """Minimal WebSocket echo/broadcast demo + the RFC 6455
   handshake math."""
2  from __future__ import annotations
3
4  import asyncio
5  import base64
6  import hashlib
7  from typing import Any
8
9  from websockets.asyncio.client import connect
10 from websockets.asyncio.server import ServerConnection,
    serve
11
12 WS_GUID = "258EAF55-E914-47DA-95CA-C5AB0DC85B11" #
    fixed by RFC 6455 section 1.3
13
14
15 def sec_websocket_accept(key: str) -> str:
16     """Derive Sec-WebSocket-Accept from the client's
        Sec-WebSocket-Key."""
17     sha1 = hashlib.sha1((key + WS_GUID).encode("ascii"))
        .digest()
18     return base64.b64encode(sha1).decode("ascii")

```

```
19
20
21 class BroadcastHub:
22     """Tracks connected robot consoles; every message
23         is echoed and broadcast."""
24
25     def __init__(self) -> None:
26         self.clients: set[ServerConnection] = set()
27
28     async def handler(self, ws: ServerConnection) ->
29         None:
30         self.clients.add(ws)
31         try:
32             async for message in ws:
33                 await ws.send(f"echo:{message}") #
34                     direct echo to the sender
35                 for peer in self.clients - {ws}:
36                     await peer.send(f"broadcast:{
37                         message}")
38         finally:
39             self.clients.discard(ws)
40
41     async def run_server(hub: BroadcastHub) -> Any:
42         return await serve(hub.handler, "127.0.0.1", 0) #
43             port 0 => OS picks a free port
44
45     async def demo() -> list[str]:
46         """Deterministic scripted session; asyncio.Event
47             provides synchronization."""
48         hub = BroadcastHub()
49         received: list[str] = []
50         peer_ready = asyncio.Event()
51
52         async with await run_server(hub) as server:
53             port = server.sockets[0].getsockname()[1]
54
55             async def peer() -> None:
56                 async with connect(f"ws://127.0.0.1:{port}"
57                     ) as ws:
58                     peer_ready.set()
59                     async for message in ws:
60                         received.append(f"peer got {message
61                             }")
```

```

57     peer_task = asyncio.create_task(peer())
58     await peer_ready.wait() # explicit
        synchronization, never sleep-based
59
60     async with connect(f"ws://127.0.0.1:{port}") as
        ws:
61         await ws.send("NWR-07_␣online")
62         received.append(await ws.recv()) # our own
            echo
63
64         pong = await ws.ping() # protocol-level
            keep-alive (opcode 0x9/0xA)
65         await asyncio.wait_for(pong, timeout=5)
66         received.append("pong_␣received")
67
68         await ws.close(code=1000, reason="demo_␣
            complete") # clean close
69     peer_task.cancel()
70
71     return received
72
73
74 if __name__ == "__main__":
75     print(sec_websocket_accept("dGhlIHNhbnBzZSBub25jZQ
        ==")) # RFC 6455 example
76     for line in asyncio.run(demo()):
77         print(line)

```

Running `python ws_demo.py` prints `s3pPLMBiTxaQ9kYGzzhZRbK+x0o=` — the exact accept value from the RFC 6455 worked example — followed by the echo, the peer's broadcast receipt, and the pong. The tests (Listing 11.7) assert all of it.

```

1  """Offline_␣tests_␣for_␣the_␣WebSocket_␣demo_␣and_␣the_␣RFC_␣
        6455_␣handshake_␣math."""
2  from __future__ import annotations
3
4  import asyncio
5
6  from ws_demo import BroadcastHub, demo,
        sec_websocket_accept
7
8
9  def test_sec_websocket_accept_matches_rfc6455_example()
    -> None:
10     # Worked example straight from RFC 6455 section
        1.3.

```

```
11     assert sec_websocket_accept("dGhlIHhnbXBsZSBub25jZQ
12         ==") == \
13         "s3pPLMBiTxaQ9kYGzZhZRbK+x0o="
14
15 def test_echo_broadcast_ping_and_clean_close() -> None:
16     received = asyncio.run(demo())
17     assert "echo:NWR-07_online" in received
18     assert "peer_got_broadcast:NWR-07_online" in
19         received # delivered to the peer
20     assert "pong_received" in received
21
22 def test_hub_disconnect_cleans_registry() -> None:
23     async def scenario() -> BroadcastHub:
24         from websockets.asyncio.client import connect
25         from websockets.asyncio.server import serve
26
27         hub = BroadcastHub()
28         async with await serve(hub.handler, "127.0.0.1"
29             , 0) as server:
30             port = server.sockets[0].getsockname()[1]
31             async with connect(f"ws://127.0.0.1:{port}"
32                 ):
33                 assert len(hub.clients) == 1
34                 # allow the server task to run its finally
35                 block deterministically
36                 for _ in range(10):
37                     await asyncio.sleep(0)
38                     if not hub.clients:
39                         break
40             return hub
41
42     hub = asyncio.run(scenario())
43     assert hub.clients == set()
```

12.3.4 The Event Contract in AsyncAPI

Listing 11.8 is the AsyncAPI 3.0 description of the same Northwind Robotics telemetry contract [Asy23]. Note what the document buys you beyond prose: the channel *address* is a templated topic string with named parameters; *operations* declare direction explicitly (*send* from the fleet's perspective, *receive* for consumers); message payloads carry JSON Schemas with patterns and ranges, so a CI gate can reject a producer that starts emitting `battery_level: 12`; and the `servers` block documents the binding

(Kafka, port 9092) without the logical contract depending on it. The same document drives code generation, documentation portals, and contract-diff gates — the OpenAPI toolchain’s event-driven mirror.

```
1  asyncapi: "3.0.0"
2  info:
3    title: Northwind Robotics Telemetry Events
4    version: "1.0.0"
5    description: >-
6      Synthetic event contract for the Northwind Robotics
7      fleet. Consumed by
8      the operations dashboard (SSE) and by customer
9      webhook endpoints.
10  servers:
11    production:
12      host: broker.northwind-robotics.example:9092
13      protocol: kafka
14      description: Fictional in-cluster broker (
15        documentation only; tests stay offline).
16  channels:
17    robotTelemetry:
18      address: northwind.robots.{robot_id}.telemetry
19      messages:
20        telemetryReading:
21          $ref: "#/components/messages/TelemetryReading"
22        batteryAlert:
23          $ref: "#/components/messages/BatteryAlert"
24  operations:
25    publishTelemetry:
26      action: send
27      channel:
28        $ref: "#/channels/robotTelemetry"
29    consumeTelemetry:
30      action: receive
31      channel:
32        $ref: "#/channels/robotTelemetry"
33  components:
34    messages:
35      TelemetryReading:
36        payload:
37          type: object
38          required: [id, robot_id, battery_level,
39            recorded_at]
40          properties:
41            id: { type: string, pattern: "^evt_[0-9]+$" }
42            robot_id: { type: string, pattern: "^NWR
```

```

    -[0-9]{2}$" }
    battery_level: { type: number, minimum: 0,
                    maximum: 1 }
    recorded_at: { type: string, format: date-
                  time }
BatteryAlert:
  payload:
    type: object
    required: [id, robot_id, threshold, recorded_at
              ]
    properties:
      id: { type: string }
      robot_id: { type: string }
      threshold: { type: number, minimum: 0,
                  maximum: 1 }
      recorded_at: { type: string, format: date-
                    time }
```

12.3.5 Contrast: Long Polling in Twelve Lines

Listing 11.9 is the long-poll alternative to Listing 11.4, kept deliberately small. The client issues `GET /notifications/next?after=N`; the server holds the request until a notification is published or a server-side deadline (always below the proxy idle timeout) expires, then answers a normal JSON document — possibly `null` — and the client immediately re-asks. Every request is an ordinary, cacheable-in-principle, auth-compatible HTTP exchange; the cost is a held worker per waiting client and a reconnect after every single event.

```
1  """Long polling, for contrast with SSE: hold the
   request until data or deadline."""
2  from __future__ import annotations
3
4  import asyncio
5
6  from fastapi import FastAPI, Query
7  from pydantic import BaseModel
8
9  HOLD_SECONDS = 15.0  # server-side deadline, always
   below the proxy idle timeout
10
11
12 class Notification(BaseModel):
13     notification_id: int
14     robot: str
```

```

15     message: str
16
17
18 class NotificationBus:
19     def __init__(self) -> None:
20         self._waiters: list[asyncio.Queue[Notification]] = []
21
22     def subscribe(self) -> asyncio.Queue[Notification]:
23         queue: asyncio.Queue[Notification] = asyncio.Queue()
24         self._waiters.append(queue)
25         return queue
26
27     def publish(self, note: Notification) -> None:
28         for queue in self._waiters:
29             queue.put_nowait(note)
30
31
32 def create_app(bus: NotificationBus) -> FastAPI:
33     app = FastAPI(title="Northwind_Robotics_Long-Poll_Notifications")
34
35     @app.get("/notifications/next", response_model=Notification | None)
36     async def next_notification(after: int = Query(0, ge=0)) -> Notification | None:
37         queue = bus.subscribe()
38         try:
39             return await asyncio.wait_for(queue.get(), timeout=HOLD_SECONDS)
40         except TimeoutError:
41             return None # 200 with null body: client reconnects immediately
42
43     return app

```

Example

Execution evidence. All listings in this section were executed offline on Python 3.13 (Windows, PowerShell) before inclusion: `pytest -q` reports **18 passed** — eleven webhook tests (tamper, stale timestamp, wrong secret, replay idempotency, rotation-header parsing, delivery success, backoff schedule, DLQ exhaustion, permanent-4xx

short-circuit, transport-error retry), four SSE tests (parser grammar, round-trip, full stream, `Last-Event-ID` resume), and three WebSocket tests (RFC 6455 accept vector, echo/broadcast/ping/close, registry cleanup). `python ws_demo.py` prints the RFC accept value `s3pPLMBiTxaQ9kYGzzhZRbK+x0o=` followed by the scripted session transcript. The AsyncAPI document parses cleanly as YAML 3.0.

12.4 Common Pitfalls & Anti-Patterns

Each entry below is a production scar, not a hypothetical.

12.4.1 Unsigned webhooks

The single most exploited webhook bug. An endpoint with no signature check is an unauthenticated “do anything” API reachable by anyone who scrapes the URL from client-side code, DNS logs, or a support forum post. Even a configured-but-unenforced check (“verify if the header is present”) fails: attackers simply omit the header. The fix is fail-closed verification — missing, malformed, stale, or mismatched all reject — as in Listing 11.1.

12.4.2 Retrying permanent failures forever

A delivery worker that retries 4xx responses indistinguishably from 5xx turns every consumer bug into an amplification loop: the consumer deploys a schema change, starts rejecting with 422, and the provider hammers it at full retry rate for days. Bound the budget, separate retryable from permanent statuses, and dead-letter the rest (Listing 11.2). The mirror-image consumer bug is returning 500 for a *payload* problem — which invites retries that can never succeed — instead of a crisp 4xx.

12.4.3 Assuming ordering

Consumers that apply events blindly in arrival order corrupt state under retries and parallelism: `robot.updated` (battery 0.79) arrives after `robot.updated` (battery 0.82) and the dashboard regresses. Either send full state per event and make consumers last-write-wins on a higher-is-newer cursor (`occurred_at` or a sequence number), or buffer and reorder. “Our provider delivers in order” is a claim about today’s implementation, not the contract.

12.4.4 SSE through buffering proxies

The stream works on the developer’s machine and stalls in production because nginx’s default `proxy_buffering on` accumulates events before forwarding. Symptoms: clients receive nothing for minutes, then a burst. Defenses: `X-Accel-Buffering: no`, `Cache-Control: no-cache`, flush per event, comment keep-alives, and — the real fix — an end-to-end test that runs *through* the production proxy path, not around it.

12.4.5 WebSocket for plain CRUD

Rebuilding `POST /robots` as a WebSocket message throws away caching, idempotency keys, status codes, tracing, and every gateway feature your organization already paid for — and adds connection lifecycle, reconnection, and backpressure code you now own. Bidirectional channels earn their keep only when both directions carry meaningful, latency-sensitive traffic.

12.4.6 Non-idempotent receivers

Processing a webhook before acknowledging it — `charge_card()` then return 200 — means every lost response re-charges the customer. Record the event id transactionally with the effect, or queue-then-ack, so redelivery is a no-op. Note the ordering subtlety: ack *before* durable recording and a crash loses the event; process *before* durable recording and a crash repeats it. Idempotency is what makes the second ordering safe.

12.4.7 Leaking event payloads in logs

Webhook payloads routinely carry PII-adjacent operational data; delivery services that log full bodies at INFO, or receivers that log the raw request “for debugging,” build an ungoverned replica of the event stream inside the log pipeline. Log event ids, types, sizes, and outcomes — never bodies, never signature secrets, never full headers.

12.4.8 Sleep-based async tests

`await asyncio.sleep(0.3)` as a synchronization mechanism makes tests flaky under CI load and slow when generous. Use `asyncio.Event`, queues, and `wait_for` with explicit timeouts (Listings 11.6–11.7): the test passes the instant the condition holds and fails loudly when it never will.

12.5 Hands-On Production Lab & Real-World Case Study

12.5.1 Lab: The Northwind Robotics Real-Time Notifications Service

You will assemble this chapter's components into one coherent service: a *notification fan-out* for the Northwind Robotics fleet that (a) serves an SSE feed to browser dashboards and (b) delivers signed webhooks to two registered customer endpoints, one of which is deliberately flaky.

Setup (PowerShell):

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install fastapi httpx uvicorn pydantic websockets
  pytest
4 mkdir lab_ch11; cd lab_ch11
```

Step 1 — the receiver. Save Listing 11.1 as `webhook_receiver.py`. This is the *customer's* endpoint; you will run two copies in-process via `TestClient`: one configured with the correct secret, one misconfigured (wrong secret) to act as the flaky endpoint.

Step 2 — the delivery worker. Save Listing 11.2 as `webhook_delivery.py`. Replace `FakeTransport` with a real transport built on `httpx` [Enc24]: a function `(url, headers, body) -> int` that POSTs with a 5s timeout and returns the status code, translating `httpx.TransportError` to `OSError`. Register it against the healthy receiver's URL and observe `DELIVERED`; register the misconfigured one and observe `REJECTED_PERMANENT` (the receiver's 400) with exactly one attempt.

Step 3 — the SSE feed. Save Listing 11.4 as `sse_streams.py` and serve it with the app factory: `uvicorn "sse_streams:create_app" --factory`. In a second terminal, consume it with `curl -N http://127.0.0.1:8000/stream` and watch events arrive with ids; kill and re-run `curl` with `-H "Last-Event-ID: 2"` and confirm only event 3 replays.

Step 4 — wire the fan-out. Write a thin orchestrator: each event in `TELEMETRY_FEED` is both appended to the SSE feed *and* handed to the `WebhookDeliverer` for each registered endpoint. Run the deterministic test suite (Listings 11.3, 11.5) as the acceptance gate.

Acceptance checks: healthy endpoint receives all three events with valid signatures; misconfigured endpoint is dead-lettered with reason `permanent 400`; SSE clients resume correctly; `pytest -q` is green with no network access (unplug it and prove it).

12.5.2 War Story 1: The Retry Loop That DDoSed a Customer

A fleet-management provider (the fictionalized composite is Northwind Robotics, the pattern is industry-common) shipped webhook delivery with a single flaw: the retry predicate was `status != 200`. A customer’s receiver began rejecting deliveries with 422 after a schema change on their side. The provider’s worker retried — immediately, then on a short fixed interval, with no cap. Queued events accumulated during the customer’s maintenance window; when the window closed, forty thousand queued deliveries retried in near-synchrony against an endpoint that rejected each one in milliseconds. From the customer’s load balancer the traffic was indistinguishable from an application-layer DDoS; their WAF rate-limited the provider’s egress IPs, which made *every* customer’s deliveries to that domain fail, which queued more events, which deepened the storm.

The postmortem produced exactly the rules encoded in Listing 11.2: classify statuses (permanent 4xx $\Rightarrow$ stop immediately), bound attempts, exponential backoff *with jitter* so recovering queues do not fire in lockstep, per-endpoint circuit breaking, and a DLQ with paging alerts. The metric that would have caught it in minutes — deliveries-per-endpoint-per-minute and 4xx ratio — became a standing dashboard.

12.5.3 War Story 2: The Unsigned-Webhook Spoofing Incident

A robotics-operations startup exposed `POST /hooks/fleet` to receive “maintenance completed” events from its CMMS vendor. The endpoint was “protected” by obscurity: a long random path. A former contractor, who knew the path from integration work, POSTed a fabricated `maintenance.completed` event for a robot that was, in reality, still faulted; the dispatcher cleared the work order automatically and the robot was scheduled back into a production shift. The resulting equipment damage was modest; the audit finding was not — the endpoint had accepted a forged state transition with zero authentication, and logs showed the vendor’s “integration guide” had suggested signature verification as *optional*.

The remediation was Listing 11.1 verbatim: shared-secret HMAC over timestamp-plus-body, a 300-second tolerance window, constant-time comparison, idempotency on event id, and — organizationally — a policy that no webhook endpoint merges without a red-team test that submits an unsigned payload and asserts rejection. Obscurity of a URL is not a control; it is a hope.

12.6 Self-Assessment Exercises & Deep-Dive Questions

- E11.1. Pattern triage.** For each requirement, pick exactly one pattern from Table 12.1 and defend the pick: (a) a stock-ticker widget on a public marketing page; (b) a two-player browser chess game; (c) “notify our ERP when an invoice is approved”; (d) a mobile app’s badge count on a metered connection; (e) a SOC wallboard that must survive the corporate proxy.
- E11.2. Signature forensics.** A receiver logs show a burst of deliveries with valid signatures but timestamps 40 minutes old. All were rejected. What two explanations fit, and what one-line config change on the provider side would distinguish them?
- E11.3. Rotation drill.** Extend Listing 11.2 so the deliverer signs each attempt with *two* secrets during a rotation window ($t=\dots, v1=\dots, v1=\dots$). Prove with a test that receivers holding either secret accept.
- E11.4. SSE grammar fuzz.** The parser in Listing 11.4 ignores fields it does not know. Per the WHATWG rules, what should happen for: a line with no colon; a `data` field with an empty value; an `id` containing a NUL; `retry: abc`? Encode your answers as four new tests.
- E11.5. Frame dissection.** By hand, write the two header bytes of: an unmasked final text frame with a 60-byte payload; a masked final binary frame with a 300-byte payload; an unfragmented ping with payload “hi”. Then verify byte two’s mask bit in each.
- E11.6. Masking math.** Client payload byte 0x48 (‘H’), masking-key byte 0x37. What goes on the wire? Why could the attacker-in-the-browser not simply choose payload bytes that survive masking unchanged?
- E11.7. Backoff analysis.** Listing 11.2 uses delays $\min(b \cdot 2^{n-1}, 60)$. For $b = 1$, $n_{\max} = 8$: total worst- case wait? Now add $\pm 20\%$ jitter: what failure mode does jitter prevent, and why is `random` seeded in the test suite if you implement it?
- E11.8. Resume-window design.** Your SSE server keeps a 500-event ring buffer. A dashboard disconnects for an hour during which 2000 events pass. Design the reconnection behavior: what does

the server send, what should the client do, and how does the user experience degrade gracefully?

E11.9. DLQ triage runbook. Write the on-call runbook entry for the alert “dead-letter rate > 10/hour”: first three diagnostic queries, the decision tree for replay-versus-discard, and the customer-communication trigger point.

E11.10. Contract gate. Add a CI check (a short Python script) that loads Listing 11.8 and fails the build if any message payload property is removed or retyped relative to a checked-in previous version. Which compatibility mode does your check implement?

12.7 Interview Q&A Vault

Junior tier

Q1. What is the difference between polling and webhooks?

Polling: the consumer repeatedly asks “anything new?” on its own schedule — simple, but most responses are empty and latency is bounded by the interval. A webhook inverts it: the consumer registers a URL and the *provider* POSTs each event as it happens. Polling costs requests; webhooks cost you a public, secured endpoint.

Q2. What does SSE stand for and what is it? Server-Sent Events: a server responds to a GET with `Content-Type: text/event-stream` and keeps the response open, appending `field: value` event blocks separated by blank lines. One-directional, server-to-client, text-only, native in browsers via `EventSource`.

Q3. Name the four SSE fields and what each does. `data` (payload; multiple lines join with newlines), `event` (named type; default message), `id` (cursor used for Last-Event-ID resume), `retry` (client reconnection delay in ms). Colon-prefixed lines are comment keep-alives.

Q4. What HTTP status begins a WebSocket connection, and what headers matter? 101 Switching Protocols. The client sends `Upgrade: websocket`, `Connection: Upgrade`, `Sec-WebSocket-Key`, `Sec-WebSocket-Version: 13`; the server replies with `Sec-WebSocket-Accept`.

Q5. Is SSE one-way or two-way? What about WebSocket? SSE is strictly server-to-client; clients respond over ordinary HTTP

requests if needed. WebSocket is full-duplex: both peers send frames at any time after the upgrade.

- Q6. Why do webhooks usually sign their payloads?** The receiver's URL is public; anyone who finds it could POST forged events. An HMAC over the body with a shared secret proves the sender holds the secret and the body is unaltered.
- Q7. What is an idempotent webhook receiver and why does it need to be one?** One where processing the same event twice has the same effect as once — typically by recording the event id and skipping duplicates. Needed because delivery is at-least-once: providers retry whenever the 2xx is lost, so duplicates are guaranteed.
- Q8. What happens when an SSE connection drops?** `EventSource` waits the last `retry` interval (default 3s), reconnects, and sends `Last-Event-ID` with the last received `id` so the server can resume the stream.
- Q9. Give two close codes you would actually use and their meanings.** 1000 normal closure (work complete); 1001 going away (server restart or page navigation). Mentioning that 1006 is reserved for *abnormal* closure and must never be sent earns extra credit.
- Q10. Webhooks versus a message queue — when which?** Webhooks cross organizational boundaries over public HTTP with zero consumer infrastructure beyond an endpoint. Queues (Kafka, SQS) are inside one trust/domain boundary, give stronger ordering/retention tooling, and require consumer connectivity to the broker.

Mid tier

- Q11. Derive `Sec-WebSocket-Accept`.** Concatenate the client's `Sec-WebSocket-Key` with the fixed GUID 258EAF5-...-C5AB0DC85B11, take SHA-1, Base64-encode. RFC vector: `dGhlIHhnbXBsZSBub25jZQ==` → `s3pPLMBiTxaQ9kYGzZhZRbK+x0o=`. Purpose: prove the server implemented WebSocket, not just header echo.
- Q12. Why are client-to-server WebSocket frames masked?** Not confidentiality — intermediaries. A hostile browser script could otherwise shape outbound bytes into something a misbehaving transparent proxy parses as a smuggled HTTP request, poisoning

shared caches. A per-frame random 32-bit XOR key chosen *after* the app supplies bytes makes the wire bytes unpredictable to the attacker.

- Q13. Walk me through the Stripe-style $t=\dots, v1=\dots$ verification.** Parse timestamp and signature(s); check $|now - t| \leq \text{tolerance}$ (replay window); recompute HMAC-SHA256 over “ t . raw-body” with the endpoint secret; constant-time compare against every $v1$; only then parse the JSON. Verify raw bytes because re-serialization is not byte-stable.
- Q14. Why multiple $v1=$ values in one header?** Secret rotation: during the overlap window the provider signs with old and new secrets; receivers configured with either still verify, so rotation needs no synchronized flag day.
- Q15. Design webhook delivery guarantees.** Durable queue per endpoint; at-least-once delivery with unique event ids; retry only retryable statuses (5xx, 429, timeouts, connection errors) with capped exponential backoff plus jitter; permanent 4xx dead-letters immediately; DLQ with alerting and replay; document best-effort ordering; consumer side: idempotent processing on event id.
- Q16. SSE versus WebSockets for a live operations dashboard — decide.** Dashboards are one-way server-to-client: SSE. You get automatic reconnection with `Last-Event-ID` resume, plain-HTTP infra compatibility (CORS, cookies, auth), trivial debugging with `curl`. Choose WebSocket only if operators must also *send* low-latency commands on the same channel.
- Q17. Your SSE stream works locally but not behind nginx. Diagnose.** Proxy buffering. Fix with `X-Accel-Buffering: no`, `Cache-Control: no-cache`, per-event flush, comment keep-alives every 10–15 s. Verify through the real proxy path.
- Q18. HTTP/1.1 versus HTTP/2 for SSE?** HTTP/1.1: browsers cap at six connections per origin; two long-lived streams can starve other requests — use a separate origin or HTTP/2. HTTP/2 multiplexes many streams over one connection, removing the cap, at the cost of shared TCP head-of-line blocking.
- Q19. How does Last-Event-ID resume work, and what is its limit?** The client replays the last seen `id` on reconnect; the server replays events after it from a buffer/log. The limit is the retention

window: if the client fell further behind than the buffer, the server must signal a resync (control event or 204) and the client re-fetches current state.

- Q20. Exactly-once webhook delivery — possible?** No; over an unreliable network the sender cannot distinguish “receiver crashed before processing” from “processed but ack lost” (Two Generals). Honest design: at-least-once transport plus idempotent consumers for effectively-once processing.
- Q21. Why does SSE get CORS/cookie auth “for free” while EventSource cannot set custom headers?** SSE *is* an HTTP response, so CORS and cookies apply natively; but the EventSource API exposes no header API, so bearer tokens must ride as cookies or (carefully) query parameters [Har12]. WebSocket’s constructor accepts subprotocols but not arbitrary headers either; token-in-first-message or ticket endpoints are the usual patterns.
- Q22. What is a subprotocol and how is it chosen?** An application-level protocol layered on WebSocket framing (e.g. `mqtt`, `graphql-ws`), offered by the client in `Sec-WebSocket-Protocol` as a preference list; the server picks at most one and returns it; a client that required one must close if the pick is absent.

Senior tier

- Q23. Dissect a WebSocket frame header.** Byte 0: FIN bit, RSV1–3, 4-bit opcode. Byte 1: MASK bit, 7-bit length. Length 0–125 literal; 126 $\Rightarrow$ next 2 bytes are a uint16; 127 $\Rightarrow$ next 8 bytes uint63 (big-endian). Then a 4-byte masking key iff MASK. Control frames (8/9/A) must be FIN with ≤ 125 -byte payloads and may interleave between data fragments.
- Q24. Design a webhook system for one million endpoints.** Partitioned durable queue (per-endpoint FIFO only if ordering is promised); worker pool with per-endpoint concurrency limits and circuit breakers; status classification; capped jittered backoff; DLQ per endpoint with replay tooling; signed delivery with rotation; endpoint verification at registration; SSRF defenses (egress proxy, DNS-pinning, private-range blocking); metrics: attempts, latency, 4xx/5xx ratio, queue depth per endpoint.
- Q25. How do you scale SSE/WebSocket fan-out horizontally?** Connections pin to a process, so introduce a backplane: each node

subscribes to a broker (Kafka/Redis pub/sub) and forwards to its local sockets; sticky routing is unnecessary if every node can serve every client; resume state (SSE) must live in the shared log, not node memory.

- Q26. Backpressure: a slow SSE consumer and a slow WebSocket consumer — what happens in each, and what do you do?** TCP flow control eventually stalls both; the server’s send buffer grows. SSE: bound a per-client queue, drop with a “resync” control event (the client can resume via `Last-Event-ID`). WebSocket: monitor buffered amount, apply app-level policies (drop deltas, send snapshots, close with 1008/1013); never let unbounded buffering become a memory-exhaustion vector.
- Q27. Ordering and deduplication strategy for a webhook consumer processing financial events.** At-least-once plus possible reordering demands: store event id and `occurred_at`/sequence transactionally with the effect; dedupe on id; make effects last-write-wins on a monotonic cursor or buffer-and-reorder within a window; reconcile periodically against the provider’s pull API (events you *missed* are the silent failure mode).
- Q28. Threat-model a public webhook receiver.** Forgery (mitigate: HMAC verification), replay (timestamp tolerance + idempotency store), tampering (HMAC over raw body), secret leak (instant rotation, multiple `v1`), log injection / PII leak (no payload logging), DoS via body size (limit bytes read before verifying), timing oracle on compare (constant-time comparison).
- Q29. When would you pick long polling over SSE in 2026?** Behind aggressive intermediaries that buffer or kill long-lived streams; when every response must be a complete, cacheable HTTP document; for very low event rates where held sockets cost more than periodic requests; or where the client platform lacks `EventSource` and you refuse a dependency.
- Q30. How do CloudEvents and AsyncAPI complement each other?** CloudEvents standardizes the *envelope* of a single event (id, source, type, time, subject) so tooling is portable; AsyncAPI standardizes the *contract* of the whole API (channels, operations, message schemas, bindings). You document channels and payloads with AsyncAPI and shape each message’s metadata as a CloudEvent [Asy23].

- Q31. What is a poison message and how does your pipeline survive one?** An event that deterministically crashes or rejects every processing attempt. Unhandled, it blocks or spins the consumer forever (head-of-line). Handling: bounded attempts with backoff, then quarantine to a dead-letter topic *with the error context*, alert, and continue with the next event; replay after a fix.
- Q32. The provider's clock and yours disagree by ten minutes — what breaks and how do you design for it?** The freshness check rejects every delivery. Design: tolerance windows sized to expected skew (300 s), NTP discipline on both fleets, monitoring on rejection-by-staleness rate, and — because the signature *covers* the timestamp — attackers cannot simply rewrite t , so the window is the only knob; keep it tight.

Staff tier

- Q33. Your org runs REST, webhooks, SSE, WebSockets, and gRPC streams. What governance keeps this coherent?** One contract repository (OpenAPI + AsyncAPI) with CI diff gates; one envelope convention (CloudEvents-style id/source/type/time); one signing/auth scheme reused across webhook providers; pattern-selection review at design time (the taxonomy matrix) so teams justify push channels; uniform consumer obligations (idempotency, DLQ handling) in a shared library; deprecation policy identical for endpoints and event types [New21; Asy23].
- Q34. Multi-region webhook delivery: what changes?** Deliver from the region nearest the endpoint to cut latency, but the queue and DLQ must survive region loss (replicated log); per-endpoint state (circuit breakers, backoff cursors) needs global consistency or bounded divergence; secrets replicate with the same rotation overlap; fail-open regions must not double-deliver *without* the consumer's idempotency absorbing it — which is why the contract promises at-least-once even in steady state.

12.8 External Bibliography & Further Reading

- **AsyncAPI Initiative**, *AsyncAPI Specification 3.0* — the contract format for channels, messages, operations, and bindings used in Listing 11.8 [Asy23].

- **IETF RFC 6455**, Fette & Melnikov, *The WebSocket Protocol* — the handshake, the SHA-1/GUID accept derivation, frame format, masking rationale, opcodes, and close-code registry.
- **WHATWG**, *Server-Sent Events* (EventSource) living standard — the authoritative `text/event-stream` parsing and reconnection algorithm implemented in Listing 11.4.
- **MDN Web Docs**, *Server-Sent Events / EventSource* — the browser API surface: named event listeners, `readyState`, `lastEventId`, and error handling.
- **Stripe**, *Webhooks documentation* — the reference implementation of the `t=...`, `v1=...` HMAC scheme, tolerance windows, and endpoint secret rotation this chapter mirrors.
- **IETF RFC 9110**, *HTTP Semantics* — methods, status codes, content negotiation, and the Upgrade mechanism both SSE and WebSocket rely on [Fie22a].
- **Adam Bellemare**, *Designing Event-Driven Systems* (O'Reilly, 2018) — event log fundamentals, consumer semantics, and the data-on-the-outside discipline behind Section 12.2.5.
- **CloudEvents (CNCF)**, *CloudEvents Specification 1.0* — the vendor-neutral event envelope (id, source, type, time) worth adopting before your event catalog outgrows ad-hoc JSON.
- **Martin Kleppmann**, *Designing Data-Intensive Applications* (O'Reilly, 2017) — the messaging-systems chapters behind at-least-once semantics and the Two Generals framing [Kle17].
- **Sam Newman**, *Building Microservices*, 2nd ed. (O'Reilly, 2021) — event-driven collaboration patterns and their operational costs [New21].
- **FastAPI** and **httpx** documentation — `StreamingResponse`, `WebSocket` routes, and streaming client usage used throughout Section 12.3 [Ram24; Enc24].

12.9 Capstone Projects

12.9.1 Capstone 11.1 [junior] — Signed Webhook Receiver for Robot Alerts

Objective: Build the consumer half of a webhook contract from scratch and prove it hostile-input-safe.

Inputs/Datasets: A fixed synthetic feed of 12 Northwind Robotics alert events (JSON files, provided); one valid signing secret; a tampered variant of three events; two events signed with a stale timestamp.

Steps:

1. Implement the `t=...`, `v1=...` verifier (reuse Listing 11.1's primitives) with a 300-second tolerance and constant-time comparison.
2. Add an idempotency store keyed on event id with a durability story you can explain (a SQLite table is fine).
3. Write tests for: valid, tampered, stale, wrong-secret, replayed, and missing-header deliveries.

Acceptance Criteria:

- ☐ All 12 valid events processed exactly once; all hostile variants rejected with 4xx problem-details bodies.
- ☐ Duplicate deliveries return 200 with `"status": "duplicate"` and cause no side effects.
- ☐ Test suite passes offline with a fixed injected clock.

Stretch Goals: support a two-secret rotation window; emit a Prometheus-style counter of accepted/rejected/duplicate deliveries.

12.9.2 Capstone 11.2 [mid] — Webhook Delivery Service with DLQ and Replay

Objective: Build the provider half: queue, bounded jittered retries, status classification, dead-letter store, and an operator replay command.

Inputs/Datasets: A registry of five synthetic customer endpoints (simulated by scripted fake transports: healthy, flaky-500, permanent-422, timeout-then-healthy, always-down); the Capstone 11.1 event feed.

Steps:

1. Extend Listing 11.2 with seeded jitter and a persistent DLQ (SQLite or JSONL) capturing event, endpoint, and attempt history.

2. Add a per-endpoint circuit breaker: after five consecutive exhausted deliveries, pause the endpoint for a cooldown.
3. Implement `replay <event_id>` re-enqueueing a dead-lettered event; prove a replayed event is delivered (and the receiver's idempotency absorbs the earlier partial processing).

Acceptance Criteria:

- ☐ The permanent-422 endpoint receives exactly one attempt per event.
- ☐ The always-down endpoint dead-letters every event; `replay` restores delivery once its transport recovers.
- ☐ Backoff schedule is asserted deterministically (injected clock and sleeper); no test sleeps.
- ☐ A one-page delivery-semantics document (at-least-once, ordering caveats) ships with the service.

Stretch Goals: SSRF guardrails on endpoint registration (scheme allowlist, private-range rejection); per-endpoint delivery metrics endpoint.

12.9.3 Capstone 11.3 [mid] — Resumable SSE Fleet Dashboard Backend

Objective: Serve a multi-channel SSE feed with correct `Last-Event-ID` resume from a bounded retention buffer, and survive a buffering proxy.

Inputs/Datasets: A deterministic event generator (seeded) emitting per-robot telemetry for five robots at scripted intervals.

Steps:

1. Generalize Listing 11.4: per-robot channels (`/stream?robot=NWR-07`) plus a combined channel, each with `id`, `retry`, and comment keep-alives every 15 seconds.
2. Implement a 500-event ring buffer per channel; resume serves the buffer after `Last-Event-ID`, and answers a `control: resync-required` event when the cursor has scrolled out.
3. Put nginx (or a small buffering reverse proxy you write yourself) in front and demonstrate the fix: `X-Accel-Buffering: no` plus flushes, verified by a client that asserts inter-event arrival gaps.

Acceptance Criteria:

- ☐ A client disconnecting and resuming within the retention window misses zero events; one resuming outside it receives the resync signal.
- ☐ Through the proxy, events arrive within one second of emission.
- ☐ Grammar tests cover comments, multi-line data, unknown fields, and empty-data blocks.

Stretch Goals: HTTP/2 serving; per-channel **retry** tuning; EventSource-compatible JavaScript demo page served by the same app.

12.9.4 Capstone 11.4 [senior] — Bidirectional Robot Control Channel over WebSockets

Objective: Build a production-shaped WebSocket service: subprotocol negotiation, per-connection auth via first-message ticket, heartbeat policy, and graceful fleet-wide shutdown.

Inputs/Datasets: The synthetic fleet (five robots); a scripted command set (dock, undock, set-speed) with per-command acknowledgements; a chaos harness that kills connections at scripted moments.

Steps:

1. Define a subprotocol `northwind.control.v1` (JSON request/response frames with correlation ids) and negotiate it in the handshake; reject connections that require an unknown subprotocol.
2. Implement ticket auth: client sends a one-time ticket in the first frame; server closes with 1008 on failure.
3. Server pings every 20s; two missed pongs $\Rightarrow$ close 1011 and reap. Client reconnects with exponential backoff and replays unacknowledged commands (deduped server-side by correlation id).
4. Fleet shutdown: server sends close 1001 (**going away**) to all connections, waits for echoes, and exits before the deploy deadline.

Acceptance Criteria:

- ☐ Commands survive connection kills exactly-once (asserted via the chaos harness with seeded scheduling).
- ☐ Zombie connections are reaped within two heartbeat intervals.
- ☐ Shutdown completes with zero unclean closes (no 1006) in the happy path.

- ☐ A short design note explains why this use case justifies WebSockets over SSE + REST commands.

Stretch Goals: permessage-deflate negotiation; a Redis-style backplane interface (stubbed in-memory) proving the design scales to N server nodes.

12.9.5 Capstone 11.5 [staff] — Event-Contract Governance Platform

Objective: Stand up the contract-governance layer for Northwind’s entire event surface: AsyncAPI documents, compatibility gates, and a consumer-obligation library shared by all receivers.

Inputs/Datasets: Three AsyncAPI documents (telemetry, alerts, maintenance) at two versions each, including one deliberately breaking change (property retype) and one compatible change (optional property added).

Steps:

1. Write an AsyncAPI diff tool classifying channel/message/schema changes as breaking or non-breaking (removal, retype, narrowing $\Rightarrow$ breaking; additive-optional $\Rightarrow$ safe), exit code non-zero on breakage.
2. Build the shared receiver library: signature verification, idempotency store, poison-message quarantine, and structured (payload-free) logging — packaged so Capstone 11.1’s receiver can adopt it in five lines.
3. Produce a governance one-pager: compatibility modes, deprecation workflow, version-support policy, and the consumer obligations every team signs up for.

Acceptance Criteria:

- ☐ The diff gate passes the compatible evolution and fails the breaking one, with human-readable diagnostics naming the offending property.
- ☐ The shared library passes this chapter’s full webhook test suite unmodified.
- ☐ The governance document covers schema evolution, secret rotation, DLQ ownership, and event-retirement criteria.

Stretch Goals: CloudEvents envelope adoption across all three contracts; a generated Markdown catalog of channels and messages from the AsyncAPI sources.

12.10 Python Dependencies Appendix

The chapter's `requirements.txt`:

```
1 fastapi>=0.110
2 websockets>=12
3 httpx>=0.27
4 pydantic>=2.7
5 pytest>=8
```

Install under PowerShell (offline machines: pre-stage a wheelhouse with `pip download -d wheelhouse -r requirements.txt` on a connected host, then `pip install --no-index --find-links wheelhouse -r requirements.txt`):

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

12.10.1 fastapi

Description: The ASGI framework used for the webhook receiver (Listing 11.1), the SSE producer (Listing 11.4), and the long-poll contrast (Listing 11.9), including `StreamingResponse` and the in-process `TestClient` that keeps every test offline [Ram24]. **Why included:** it is the book's established server framework (Chapter 6) and its response/streaming primitives map one-to-one onto the protocols in this chapter without magic. **Alternatives:** Starlette directly (FastAPI's base; fewer batteries); SSE-Starlette (adds a dedicated `EventSourceResponse` — convenient, but Listing 11.4 deliberately shows the wire format by hand, which is the pedagogical point); aiohttp or Django Channels if your organization already standardized there. **Installation:** `pip install "fastapi>=0.110"` in the activated venv; serving listings interactively additionally needs `uvicorn`.

12.10.2 websockets

Description: The reference-quality Python WebSocket library; Listing 11.6 uses its `websockets.asyncio` implementation (available since version 12) for both server and client sides of the echo/broadcast demo, ping/pong, and clean close. **Why included:** RFC 6455 correctness (handshake, masking, control frames, close handshake) is handled for you, letting the chapter focus on semantics while still exposing the wire details where they matter. **Alternatives:** python-socketio (higher-level rooms/namespaces with automatic fallbacks — pick it for product chat features, not for learning the protocol); FastAPI/Starlette's built-in WebSocket routes (fine for ASGI-

hosted endpoints); aiohttp's WS support. **Installation:** `pip install "websockets>=12"`.

12.10.3 httpx

Description: The HTTP client used as the delivery transport in the lab and as the streaming client paired with the SSE parser; powers `TestClient` under the hood [Enc24]. **Why included:** one API for sync and async, first-class streaming response iteration (`client.stream(...).iter_lines()`), explicit timeout configuration — the three properties webhook delivery and SSE consumption actually need. **Alternatives:** `urllib.request` (stdlib; workable for the delivery worker but no ergonomic streaming); `requests` (no async, no HTTP/2); `aiohttp` (async-only). **Installation:** `pip install "httpx>=0.27"`.

12.10.4 pydantic

Description: The validation layer behind FastAPI models (used directly for the long-poll `Notification` model in Listing 11.9). **Why included:** event payloads are contracts; validating them at the boundary with typed models is the same discipline Chapter 4 established for REST representations. **Alternatives:** `dataclasses` plus hand-rolled validation (what Listings 11.1–11.2 use deliberately, to keep signature logic dependency-visible); `msgspec` or `attrs` for performance-sensitive paths. **Installation:** `pip install "pydantic>=2.7"` (pulled in transitively by FastAPI).

12.10.5 pytest

Description: The test runner for all eighteen tests in this chapter. **Why included:** fixtures and parametrization keep the hostile-input matrix (tampered, stale, replayed) declarative; async behavior is tested through `asyncio.run` inside synchronous tests, so no `pytest-asyncio` plugin is required and scheduling stays explicit. **Alternatives:** `unittest` (stdlib, zero install; more boilerplate); `nose2` (largely superseded). **Installation:** `pip install "pytest>=8"`.

12.11 Chapter Summary & Key Takeaways

- **Pattern before protocol.** Choose by initiation and direction: server-to-server push $\Rightarrow$ webhooks; one-way server-to-client $\Rightarrow$ SSE; genuine two-way interaction $\Rightarrow$ WebSockets; everything else $\Rightarrow$ request/response, possibly with long polling or ETag-assisted short polling.

- **Webhooks are reverse APIs.** At-least-once delivery is the only honest semantics; the contract is signed HMAC over timestamp-plus-body, a tight tolerance window, bounded jittered retries that distinguish permanent from transient failure, idempotent receivers, and a dead-letter queue with a replay path.
- **SSE is plain HTTP with a grammar.** Four fields, comment keep-alives, blank-line dispatch; `EventSource` reconnects and resumes via `Last-Event-ID` for free. Respect buffering proxies (`X-Accel-Buffering: no`) and the HTTP/1.1 connection cap (prefer HTTP/2).
- **WebSocket abandons HTTP after the handshake.** The accept value is `Base64(SHA-1(key || GUID))`; frames carry FIN/opcode/mask/length; client masking protects intermediaries, not secrecy; ping/pong and the close handshake are your liveness and shutdown machinery. Do not pay those costs for stateless CRUD or cacheable reads.
- **Contracts govern events too.** AsyncAPI documents channels, operations, messages, and bindings; compatibility gates and idempotent, poison-tolerant consumers are as mandatory as for REST.
- **Test the failure modes, deterministically.** Injected clocks and sleepers, scripted transports, `asyncio.Event` synchronization — never wall-clock assertions — make tamper, replay, retry-storm, and resume behavior provable offline.

12.12 Quick-Reference Card

Pattern selection matrix (pocket version)

SSE field reference

WebSocket close codes

Webhook header anatomy

```
1 X-Northwind-Signature: t=<unix-seconds>,v1=<hex HMAC-  
   SHA256(secret, "t.body")>  
2 verify order: parse -> freshness (|now - t| <= 300 s)  
   -> HMAC (constant time)  
3           -> parse JSON -> idempotency check ->  
               process -> 200  
4 retry policy: 2xx = done; 408/409/425/429/5xx/network =  
   retry (bounded,
```

Need	Pick	Remember
Server → server event	Webhook	sign, retry-bounded, DLQ, idempotent receiver
Server → browser, one-way	SSE	resume via Last-Event-ID ; kill proxy buffering
Two-way low-latency	WebSocket	own reconnection, heartbeat, back-pressure
Occasional freshness, cacheable	Short poll + ETag	nearly free at scale
Simple push through hostile proxies	Long polling	hold deadline < proxy idle timeout

Line form	Meaning
data: <text>	payload; repeated lines joined with newlines
event: <name>	dispatch type (default message)
id: <text>	sets the client's last-event-id cursor
retry: <ms>	reconnection delay (digits only)
: <anything>	comment; use as keep-alive every 10–15 s
(blank line)	dispatch the buffered event

5

```
jittered exponential); other 4xx = dead-  
letter immediately
```

12.13 Standardized Evidence Addendum

12.13.1 Benchmark Snapshot Template

12.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

Code	Name	Use
1000	normal	work complete; default clean close
1001	going away	server restart, page navigation
1002	protocol error	peer violated framing rules
1003	unsupported data	received a type you cannot accept
1006	(abnormal)	reserved; never sent on the wire
1007	invalid payload	e.g. non-UTF-8 in a text frame
1008	policy violation	authz failure, generic policy refusal
1009	too big	message exceeded your limit
1011	internal error	server-side unexpected condition

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 12.2: Minimum benchmark table to keep per chapter.

12.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 12.3: Operational budget template for this chapter’s technique.

12.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

12.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

12.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

12.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 12.4: Decision card to complete per chapter.

12.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

12.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 13

GraphQL: Schema-Driven APIs

Executive Summary

REST asks the server to guess what clients need; GraphQL makes the client say it. That single inversion — the client composes the response shape, the server guarantees it against a typed schema — is the whole idea, and everything else in this chapter is a consequence of it. The consequences are larger than they look. A schema that clients query against directly is a *contract with an algebra*: fields compose, fragments factor out shared selection, nullability propagates errors along formally specified paths, and the type system is introspectable at runtime. The same property that makes GraphQL delightful for a mobile team — one endpoint, one round trip, exactly the fields requested — makes it dangerous by default: the client, not the server, decides how much work a request costs. A REST endpoint's worst case is bounded by its implementation; a GraphQL endpoint's worst case is bounded by the attacker's imagination, unless you bound it yourself.

This chapter treats GraphQL as a specification first and a library second. We work through the October 2021 specification's type system [Gra21] — scalars, objects, interfaces, unions, enums, input objects, and the list/nullability algebra — and formalize the rule that makes GraphQL error handling unlike anything in REST: when a non-null field errors, the `null` bubbles upward to the nearest nullable position, and the response arrives as *partial data* in an HTTP 200 with a populated `errors` array. We trace an operation through the parse, validate, and execute phases; we dissect the request/response wire shape; and we confront the two production taxes every GraphQL server pays. The first tax is the $N+1$ query problem, which field-level resolution creates by construction: we reproduce it with an instrumented repository and watch a single innocent query execute 11 store reads, then collapse it to 3 with per-request DataLoader batching — measured, not asserted. The second tax is cost control: depth limiting, complexity analysis, operation allow-lists, and Automatic Persisted Queries, which we

implement as a pre-execution gate that rejects expensive documents before a single resolver runs.

The reference implementation is a Strawberry-GraphQL catalog API for the fictional Northwind Robotics parts universe: Relay-style cursor connections, typed mutation errors via union payloads, field-level error extensions, an instrumented repository, a cost-control layer, an APQ-capable httpx client, and a 21-test in-process suite — all deterministic, all offline, all runnable on a Windows workstation with PowerShell. The chapter closes with an honest GraphQL-versus-REST-versus-gRPC decision matrix and a scoped primer on federation, because the correct answer to “should we use GraphQL?” is sometimes “no,” and a principal engineer should be able to say when.

Key Concept

A GraphQL schema is a *queryable contract*: clients compose responses from typed fields, and the server guarantees shape, nullability, and error semantics per the specification [Gra21]. The power inversion is real — the client chooses the work — so production GraphQL is defined less by resolvers than by its guardrails: depth and complexity budgets, persisted-query allow-lists, gated introspection, per-request DataLoaders, and an error model that treats partial success as a first-class outcome.

13.1 Learning Objectives & Prerequisites

13.1.1 Learning Objectives

After completing this chapter and its lab, you will be able to:

- LO-1.** Define every kind in the GraphQL type system — scalar, object, interface, union, enum, input object, list, and non-null — and predict the schema a Strawberry definition generates.
- LO-2.** State and apply the null-propagation rule formally: an error at a non-null position replaces that position with `null` and propagates to the nearest nullable ancestor, up to the `data` root.
- LO-3.** Explain the three operation types (query, mutation, subscription) and the parse → validate → execute pipeline, including where static cost analysis must hook in.
- LO-4.** Read and produce the GraphQL-over-HTTP wire shape: the `query/variables/operationName` request envelope and the `data+errors` response envelope, including

partial responses.

- LO-5.** Reproduce the $N+1$ query problem with an instrumented repository, count the queries, and eliminate it with DataLoader batching — including the one-DataLoader-per-request rule and its caching semantics.
- LO-6.** Implement Relay-style cursor connections (`edges/node/pageInfo`, `first/after`) with opaque, decodable cursors.
- LO-7.** Distinguish field errors from top-level errors, use `extensions.code` as the machine-readable channel, and choose between union-typed mutation payloads and raised errors.
- LO-8.** Design a cost-control layer: depth limits, argument-weighted complexity estimation, operation allow-lists, Automatic Persisted Queries, and timeouts.
- LO-9.** Enumerate the GraphQL-specific attack surface — introspection abuse, batching attacks, god-queries, resolver-side injection — and map each to its mitigation.
- LO-10.** Choose between REST, GraphQL, and gRPC on evidence: cacheability, uploads, versioning philosophy, tooling gravity, and team shape.

13.1.2 Prerequisites

This chapter assumes Chapters 0–8. Concretely:

- **Chapter 0:** reproducible-measurement discipline and the synthetic Northwind Robotics universe.
- **Chapters 1–2:** HTTP request/response mechanics, POST semantics, and Python HTTP clients [[Fie22a](#); [Enc24](#)].
- **Chapter 3:** REST resource modeling — the baseline this chapter contrasts GraphQL against.
- **Chapter 4:** JSON serialization contracts; GraphQL response envelopes are exactly such contracts.
- **Chapter 5:** client-side credential and transport hygiene.
- **Chapter 6:** building FastAPI services; the GraphQL server here mounts on FastAPI [[Ram24](#)].

- **Chapter 7:** error modeling; GraphQL’s `errors` array and `extensions` extend that chapter’s problem-details thinking into a partial-success world.
- **Chapter 8:** cursor versus offset pagination; the Relay connection spec is that chapter’s keyset idea in GraphQL clothing.

13.2 Deep Technical Mechanics & Architectural Concepts

13.2.1 What GraphQL Actually Is

Strip away the tooling and GraphQL is three documents. The *schema* is a typed, introspectable description of everything clients may ask for. An *operation* is a document the client sends, written in the same type system, selecting exactly the fields it wants. The *response* is a JSON document whose shape mirrors the operation, key for key. The server’s job is to resolve each requested field against the schema, nowhere else; the client’s job is to tolerate partial data, because a GraphQL response can be simultaneously a success and a failure.

Definition 13.1 (GraphQL request execution). Given a schema S , an operation document O , variable values V , and an initial root value R , *execution* is the function $\text{Execute}(S, O, V, R) \mapsto (\text{data}, \text{errors})$ where data mirrors O ’s selection set, field errors are collected into errors with their paths, and the null-propagation rule (Definition 13.2) governs how field errors mutate data [Gra21].

Three contrasts with REST matter architecturally. First, **one endpoint, many shapes**: REST fixes the response shape per resource; GraphQL fixes the *universe* of shapes and lets the client project. Second, **the type system is the documentation and the validator**: there is no drift between docs and behavior, because the schema that documents the API is the schema that validates the query. Third, **the client controls cost**: a REST server author bounds each endpoint’s work; a GraphQL server author bounds the *space of expressible queries*, which is the subject of Section 13.2.9 and of half this chapter’s war stories.

13.2.2 The Type System

GraphQL’s type system has eight kinds, and production fluency means knowing what each one *refuses* to express.

Scalars are leaves: the built-ins `Int`, `Float`, `String`, `Boolean`, `ID`, plus custom scalars (a `DateTime` is the canonical example) whose serialization

the server defines. ID serializes as a string and means “opaque identifier; do not parse.”

Object types are the workhorses: named sets of typed fields, where field types may themselves be objects — this recursion is what makes a GraphQL API a *graph*. **Interfaces** are abstract objects: they declare fields that every implementing object must carry, and selecting through an interface uses inline fragments for concrete-type-only fields. **Unions** are interfaces without the contract: member types share nothing, so *every* field selection goes through inline fragments — which is exactly why unions are the right tool for typed mutation results (Section 13.2.8). **Enums** are closed sets of string values, validated at the operation layer. **Input objects** are objects that flow the other way, into the server as arguments; they may contain scalars, enums, lists, and other input objects, but never unions or interfaces — input types are closed under a smaller algebra than output types, by design.

Lists and non-null are type *modifiers*, and their composition is where the specification’s subtlety lives. `[Review]` means “nullable list of nullable reviews”; `[Review!]!` means “non-null list of non-null reviews.” The four combinations are semantically distinct, and choosing among them is the schema author’s error-propagation policy expressed as syntax.

SDL type	May be	Error in one element does
<code>[Review]</code>	null, <code>[null]</code>	nulls only that element
<code>[Review!]</code>	null	nulls the whole list
<code>[Review]!</code>	<code>[null]</code>	nulls only that element; list never null
<code>[Review!]!</code>	neither	propagates past the list to the parent

Table 13.1: The four list/nullability combinations as error-containment policy.

Definition 13.2 (Null propagation on errors). Let a field position p with declared type T raise an error during execution. If T is nullable, p is set to `null` and an entry is appended to `errors`. If T is non-null, the error *bubbles*: p ’s parent object is set to `null`, and the bubbling repeats at each ancestor until a nullable position absorbs it; if no nullable position exists, the entire `data` entry becomes `null` [Gra21]. Non-null is therefore not “this cannot fail” — it is “if this fails, the failure voids my parent too.”

The practical reading: make fields non-null when a client cannot meaningfully render anything without them, and nullable when partial degradation is useful. A product page can render without a margin; a payment screen cannot render without an amount. Our reference schema encodes exactly

this: `marginPercent` is `Int!`, so its failure (cost data missing for one discontinued product) nulls the enclosing node while sibling products survive — Listing 13.7 proves both halves of that sentence.

13.2.3 Schema Definition Language and Introspection

The schema definition language (SDL) is the human-readable notation for the type system. The catalog API's core, as SDL, is:

```
1 type Query {
2   products(first: Int = 10, after: String, status:
3     ProductStatus): ProductConnection
4   product(productId: Int!): Product
5 }
6
7 type Product {
8   id: Int!
9   sku: String!
10  name: String!
11  unitPriceCents: Int!
12  stock: Int!
13  discontinued: Boolean!
14  category: Category
15  reviews: [Review!]!
16  marginPercent: Int!
17 }
18
19 type Mutation {
20   addReview(input: AddReviewInput!): AddReviewResult!
21 }
22
23 union AddReviewResult = AddReviewSuccess |
24   AddReviewError
```

SDL is one authoring style; Strawberry's is *code-first*, where Python classes generate the SDL. The choice matters less than the invariant: **the schema is the single source of truth**, and anything claiming to describe the API must be derived from it.

Introspection is the schema describing *itself* at runtime: the meta-fields `__schema` and `__type` expose every type, field, and argument, which is what powers GraphiQL's autocompletion and every code generator. It is also a complete map of your attack surface, served fresh to anyone who asks.

Warning

Introspection is not an authentication bypass — it reveals only types, not data — but it hands an adversary your entire field inventory, including the internal field you forgot carried something sensitive. Production policy: introspection **off** for unauthenticated traffic in production, **on** in development and staging, and gate it in the validation layer (an AST check for `__schema/__type`), never by substring matching — Listing 13.4 implements the AST gate that aliases and fragments cannot evade.

13.2.4 Operations: Queries, Mutations, Subscriptions

A **query** is a read with no observable side effects; a **mutation** is a write whose top-level fields execute *in series*, in order — the only ordering guarantee the specification makes, and the reason multi-step writes belong in one mutation’s sequential fields rather than in parallel mutations; a **subscription** is a query plus a stream: the server pushes a response each time an event invalidates the selection, typically over WebSockets (the `graphql-ws` protocol) or server-sent events.

Operations accept **variables**: typed, separately-serialized arguments declared in the operation header (`query Page($first: Int!)`). Variables are the only correct way to pass client input — a query is a parse tree, and variables travel in their own JSON object, so *variable injection is structurally impossible*: there is no string to escape from. **Aliases** (`active: products(...)`) rename result keys, permitting the same field twice with different arguments. **Fragments** name reusable selection sets; the directives `@include(if:)` and `@skip(if:)` condition fields on variables, letting one cached operation serve several UI states.

```

1 query CatalogPage($first: Int!, $after: String,
  $withReviews: Boolean!) {
2   products(first: $first, after: $after) {
3     edges {
4       cursor
5       node {
6         ...ProductCard
7         reviews @include(if: $withReviews) { rating }
8       }
9     }
10    pageInfo { hasNextPage endCursor }
11  }
12 }
13
14 fragment ProductCard on Product {

```

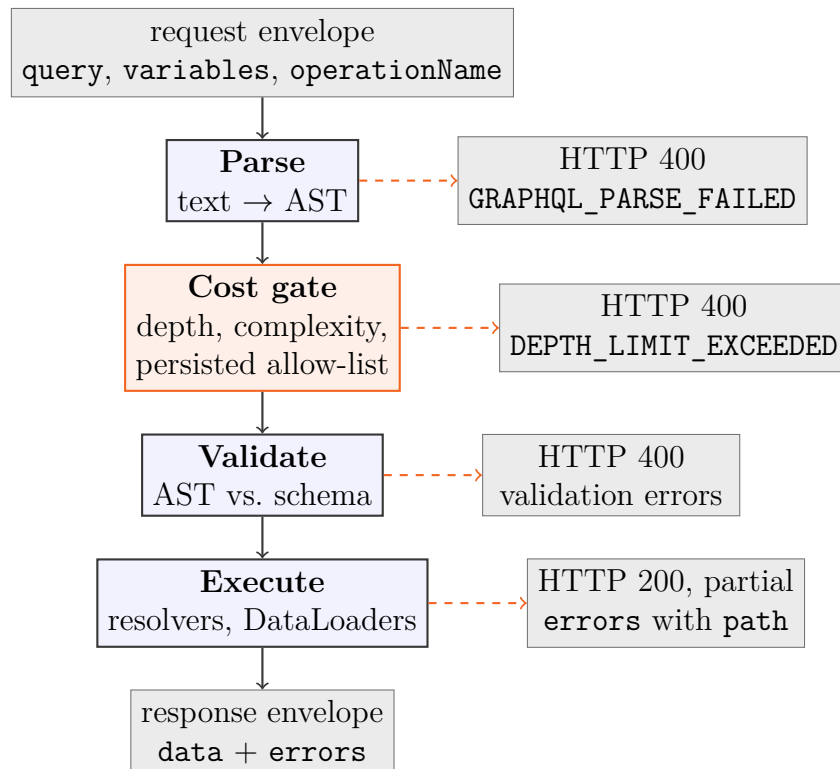
```

15   sku
16   name
17   unitPriceCents
18 }

```

13.2.5 The Execution Pipeline and the Wire Shape

Every GraphQL request walks the same three phases, and each phase has its own failure channel (Figure 13.1). **Parse** turns the document text into an AST; failure is a syntax error, HTTP 400, before any schema involvement. **Validate** checks the AST against the schema — every field exists, every argument type-checks, fragments terminate — failure again pre-execution. This is also where cost control belongs: depth and complexity are properties of the *document*, so an over-budget operation must be rejected before a resolver runs, or the meter is already spinning. **Execute** resolves fields; failures here are field errors, folded into the response under the null-propagation rule.



phases left of the dashed rejects never touch a resolver

Figure 13.1: Request lifecycle. Static cost control sits between parse and validate: cheap to compute, before any field executes.

The wire shape is austere. The request is a JSON object with `query`, optional `variables`, optional `operationName`, and optional `extensions`; over HTTP it is almost always POST [Fie22a]. The response is a JSON object with `data` (possibly partial, possibly `null`) and, on field errors, an `errors` array whose entries carry `message`, `path`, `locations`, and the open extension point `extensions` — where `code` belongs, because clients need machine-readable failure kinds, not prose.

```

1 {
2   "data": { "product": null },
3   "errors": [
4     {
5       "message": "cost data unavailable for product",
6       "path": ["product", "marginPercent"],
7       "extensions": { "code": "COST_DATA_UNAVAILABLE" }
8     }
9   ]
10  }

```

Read that response carefully, because it breaks two REST instincts at once: the HTTP status is 200 *and* the operation failed, and the failure voided exactly one field's parent, not the request. A client that checks only `resp.status_code` will render garbage; a client that treats any `errors` entry as total failure wastes the partial data the server worked to produce. Both behaviors are bugs.

13.2.6 Resolver Architecture and the N+1 Problem

Execution is field-at-a-time: for each field in the selection set, the executor calls that field's *resolver* with the parent value, the arguments, and the *context* — a per-request bag holding repositories, the authenticated principal, and (critically) `DataLoaders`. This design is what makes GraphQL composable: the `category` resolver does not know or care whether the parent product came from `products` or `product`. It is also what makes GraphQL dangerous, because resolution composes *multiplicatively* with list sizes.

Example

The query `products(first: 5) { edges { node { name category { name } reviews { rating } } } }` looks like one request. With naive resolvers — `category` calls `repo.category_by_id`, `reviews` calls `repo.reviews_for_product` — it is $1 + 5 + 5 = 11$ store reads: one for the page, then one *per product* per relationship field. At `first: 50` it is 101 reads. The *N+1* is not a bug in a

resolver; it is the default physics of field-level resolution. Listing 13.3 measures exactly these counts.

`DataLoader` collapses the fan-out by exploiting the executor’s scheduling: all product nodes’ `category` resolvers run within the same event loop tick, so each one, instead of querying, *enqueues its key* and awaits. At the tick boundary the loader calls the batch function once with the deduplicated keys — `[1, 1, 2, 2, 3]` becomes `categories_by_ids([1, 2, 3])` — and distributes results back to the waiting futures by position. Eleven reads become three: the page, one batched category read, one batched review read (Figure 13.2).

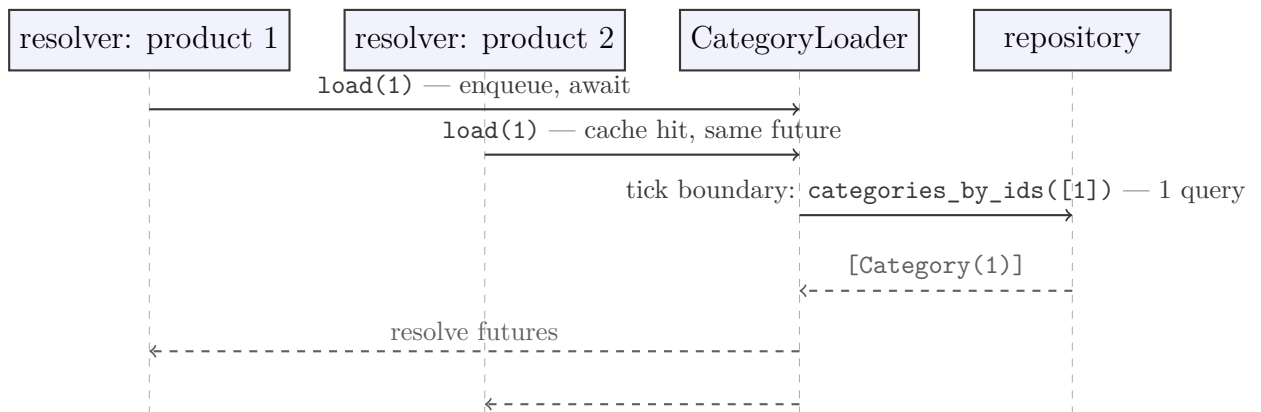


Figure 13.2: `DataLoader` batching: per-field loads within one tick collapse into a single keyed batch read; the per-request cache dedupes repeated keys.

Two rules keep `DataLoader` correct. **One loader instance per request:** its cache is request-scoped identity memoization; sharing a loader across requests serves stale data across user boundaries, which is not a performance bug but a security incident. **The batch function is the only place queries happen:** it must return results positionally aligned with the keys, with `None` for misses — never raise for one bad key, or every awaiting field in the tick fails together.

13.2.7 Pagination: The Relay Connection Spec

Chapter 8 established that value-based (cursor) pagination beats position-based (offset) pagination; the Relay connection spec is the GraphQL community’s standard encoding of that idea. A paginated field returns a *connection* object: `edges` (each with an opaque `cursor` and a `node`), `pageInfo` (with `hasNextPage`, `hasPreviousPage`, `startCursor`, `endCursor`), and forward arguments `first/after` (backward: `last/before`). The conventions that matter in production: cursors are *opaque to clients* (ours are

Base64("product:<id>"), decodable server-side, forged ones rejected with `BAD_CURSOR`; `hasNextPage` is computed with the `fetch-p+1` trick, never with a `COUNT`; and every list a client can size needs a server-side maximum — our schema caps `first` at 50 and rejects out-of-range values at the resolver.

13.2.8 The Error Model

GraphQL has exactly two error channels, and choosing between them is design. **Top-level errors** (the `errors` array) are for *exceptional* failure: the resolver could not produce the field — downstream timeout, missing cost data, invalid cursor. They compose with null propagation and partial data. **Typed mutation errors** model *expected* failure — validation, business-rule rejection — as data: the mutation returns a union of success and error payloads, and the client selects with inline fragments:

```

1 mutation Add($input: AddReviewInput!) {
2   addReview(input: $input) {
3     __typename
4     ... on AddReviewSuccess { review { id rating } }
5     ... on AddReviewError { code message }
6   }
7 }
```

The distinction is operational, not stylistic. Top-level errors page the on-call engineer; typed errors inform the end user. A rating of 6 is not an incident, and a database connection failure is not a form-validation message. Chapter 7's discipline — machine-readable codes, actionable messages — applies to both channels via `extensions.code` on the one hand and the `code` field on the other.

13.2.9 API Cost Control

Because the client chooses the work, an unguarded GraphQL endpoint has an unbounded worst case. The controls, in the order they should fire:

Depth limiting rejects documents whose longest field path exceeds a budget (ours: 7). Depth is the crudest measure but catches the classic abuse — cyclic traversals like `friends { friends { friends ... } }` — before they cost anything.

Complexity analysis weights fields so that list-returning fields multiply their subtree's cost by the requested page size: $\text{cost}(f) = 1 + m(f) \cdot \sum_{c \in \text{children}(f)} \text{cost}(c)$, where $m(f)$ is the field's `first/last` argument, a conservative default when absent, and 1 for scalar fields. The estimator runs on the parsed AST (Listing 13.4), so a `first: 50` god-query is rejected with `COMPLEXITY_LIMIT_EXCEEDED` while a `first: 2` variant of the same shape sails through — the suite proves both.

Timeouts bound what static analysis cannot predict; complexity estimates are adversarial games, and a resolver deadline is the backstop for the estimation errors you did not know you had.

Operation allow-lists invert the trust model entirely: in production the server executes only pre-registered documents, identified by hash. Nothing else parses, so depth and complexity limits become defense in depth rather than the perimeter. **Automatic Persisted Queries** (APQ) is the ergonomic half of that idea: the client sends only `extensions.persistedQuery.sha256Hash;` on `PersistedQueryNotFound` (HTTP 200 with an error entry, per the protocol) it retries once with the full query plus hash; the server verifies the hash matches the text — `PersistedQueryMismatch` otherwise — registers the document, and executes. Subsequent requests carry 64 bytes instead of kilobytes, and a CDN can cache hash-keyed GETs. Listing 13.6 implements both halves of the handshake.

13.2.10 Security: The GraphQL-Specific Attack Surface

GraphQL inherits every web-API threat and adds a few of its own.

Introspection abuse — covered in Section 13.2.3: gate it at the validation layer. **Query-based DoS** — deep, wide, or aliased-multiplied documents; mitigated by the cost gate above. Note the amplifier most teams miss: **batching abuse**. GraphQL-over-HTTP conventionally accepts an *array* of operations per request, so one HTTP request can carry a hundred god-queries, and request-count rate limiting sees “1 request.” Mitigations: disable array batching if your clients do not need it, cap the array length, and — the durable fix — **rate-limit by estimated cost, not by request count**, summing complexity across the batch. **Injection**: variable injection into the *query document* is structurally impossible (variables are a separate JSON object), but *resolver-side* injection is exactly as real as in any stack — a resolver that string-concatenates a variable into SQL is a SQL injection wearing a typed costume. Parameterized queries remain non-negotiable [Kle17]. **Field-level authorization** moves the perimeter into resolvers: the schema advertises `salaryHistory` to everyone, so authorization must live at the field, via context-carrying checks — and union payloads let you return `NOT_AUTHORIZED` as data where partial visibility is legitimate. Finally, **disable the GraphQL IDE in production**: it is a developer tool that leaks the endpoint’s shape and invites interactive probing; its presence is a config flag, and that flag belongs in the same policy file as the introspection gate.

13.2.11 GraphQL vs. REST vs. gRPC: An Honest Matrix

Dimension	REST (HTTP+JSON)	GraphQL	gRPC
Contract	OpenAPI, retrofitted	Schema, intrinsic [Gra21]	Protobuf, intrinsic
Response shape	Server-fixed	Client-composed	Server-fixed
HTTP caching	Native: GET + CDN + ETag	POST defeats it; APQ+GET partially restores	No (HTTP/2 frames)
Over/under-fetching	Common	Eliminated	Eliminated
File uploads	Native multipart	Multipart spec, awkward	Streaming, good
Versioning	/v2 or headers	Continuous evolution: deprecate, never break	Field numbers, additive
Worst-case cost	Bounded per endpoint	Unbounded without gate	Bounded per method
Tooling gravity	Ubiquitous	Strong for frontends	Strong for service mesh
Fit	Public APIs, webhooks, CDNs	Product graphs, many clients, one backend	Internal low-latency RPC

Table 13.2: Decision matrix. No column is “best”; each optimizes a different constraint.

Three rows deserve emphasis. **Cacheability:** REST’s use of GET with status codes and validators is the largest operational advantage in the table — a CDN can absorb a product launch; a GraphQL POST endpoint cannot, and the mitigations (APQ over GET, response-caching extensions) are exactly that: mitigations. **Versioning philosophy:** GraphQL’s orthodoxy is *continuous evolution* — add fields freely, deprecate with `@deprecated`, never remove — which trades REST’s explicit `/v2` flag days for an eternal compatibility obligation backed by usage telemetry on every field. Neither is cheaper; they bill different teams. **The request topology** differs structurally (Figure 13.3): REST spreads a screen’s data need across k endpoints and k round trips; GraphQL concentrates it in one request whose cost you must now govern.

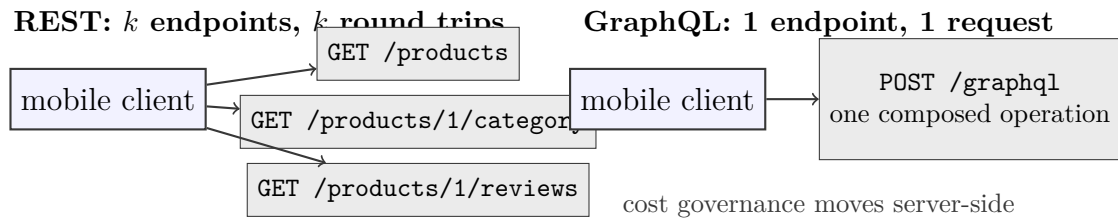


Figure 13.3: Request topology: REST distributes a screen’s data need across endpoints; GraphQL concentrates it into one operation whose cost the server must bound.

13.2.12 Federation Primer (Scope Note)

When one schema is owned by fifteen teams, the monolith reappears wearing a schema’s clothes. Federation splits the graph into *subgraphs* — each a standalone GraphQL service owning its types — that a *gateway* composes into one supergraph; *entities* are types with a key (say, `Product @key(fields: "id")`) that several subgraphs can extend, so the reviews service adds `Product.reviews` without owning products. The gateway plans each operation across subgraphs and stitches the results. That is the entire scope of this chapter’s treatment: federation is an organizational scaling tool with its own failure modes (composition conflicts, gateway as a chokepoint, entity-resolution *$N+1$ s across services*), and it deserves the same rigor this chapter gave the single graph before you adopt it.

13.3 Production-Ready Code Implementation

The reference implementation is a six-file lab, deliberately offline and deterministic. **Dependencies.** We use **Strawberry** rather than Ariadne or Graphene: it is code-first (Python type hints *are* the schema, so the listing teaches the type system instead of an SDL mapping layer), it is actively maintained on modern **graphql-core**, its DataLoader and FastAPI integration are first-class, and — decisive for a textbook — its resolvers are plain async Python that reads like the specification. Ariadne (SDL-first) and Graphene (the incumbent) are surveyed in the Dependencies Appendix.

Layout. `catalog_repository.py` (seeded store with query counter), `catalog_schema.py` (schema, connections, mutations, DataLoaders), `cost_control.py` (depth/complexity/APQ/HTTP entry point), `server.py` (FastAPI host), `graphql_client.py` (httpx client with APQ), `test_catalog.py` (21 in-process tests). Windows paths below assume the lab lives at `C:\training\graphql_lab`.

13.3.1 Listing 1: Instrumented Repository

The repository (Listing 13.1) is the lab’s ground truth: five synthetic products across three categories with five reviews, one of them (product 4, the discontinued sensor array) deliberately missing cost data to drive the null-propagation demonstration. Every read path increments `query_count`, which converts the $N+1$ problem from anecdote into an assertion a test can make.

Listing 13.1: Instrumented in-memory repository (`catalog_repository.py`).

```

1  """In-memory repository for the Northwind Robotics
   catalog.
2
3  Synthetic, deterministic, offline: the same seed data
   on every run.
4  The repository exposes an instrumentation counter so
   the N+1 problem is
5  measurable rather than anecdotal.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass
11
12
13 @dataclass(frozen=True)
14 class CategoryRow:
15     id: int
16     name: str
17     description: str
18
19
20 @dataclass(frozen=True)
21 class ProductRow:
22     id: int
23     sku: str
24     name: str
25     category_id: int
26     unit_price_cents: int
27     unit_cost_cents: int | None # None = cost data
   missing (drives null-propagation demo)
28     stock: int
29     discontinued: bool
30
31

```

```

32 @dataclass(frozen=True)
33 class ReviewRow:
34     id: int
35     product_id: int
36     rating: int
37     headline: str
38
39
40 _CATEGORIES: tuple[CategoryRow, ...] = (
41     CategoryRow(1, "Actuators", "Linear and rotary
42         motion components"),
43     CategoryRow(2, "Sensors", "LIDAR, vision, and
44         tactile sensing"),
45     CategoryRow(3, "Controllers", "Real-time motor and
46         fleet controllers"),
47 )
48
49 _PRODUCTS: tuple[ProductRow, ...] = (
50     ProductRow(1, "NWR-ACT-100", "Servo Linear Actuator
51         100mm", 1, 459_00, 210_00, 34, False),
52     ProductRow(2, "NWR-ACT-200", "Rotary Torque
53         Actuator 12Nm", 1, 899_00, 401_00, 12, False),
54     ProductRow(3, "NWR-SEN-310", "Solid-State LIDAR
55         Unit", 2, 1_299_00, 620_00, 8, False),
56     ProductRow(4, "NWR-SEN-410", "Tactile Gripper
57         Sensor Array", 2, 349_00, None, 0, True),
58     ProductRow(5, "NWR-CTL-900", "Fleet Motion
59         Controller X9", 3, 2_499_00, 1_150_00, 5, False)
60 )
61
62
63 _REVIEWS: tuple[ReviewRow, ...] = (
64     ReviewRow(1, 1, 5, "Silky smooth travel"),
65     ReviewRow(2, 1, 4, "Solid, runs warm under load"),
66     ReviewRow(3, 2, 5, "Best torque-per-dollar we
67         tested"),
68     ReviewRow(4, 3, 4, "Great range, tricky mounting"),
69     ReviewRow(5, 5, 5, "Runs our whole warehouse fleet")
70 )
71
72 class CatalogRepository:
73     """In-memory catalog store with a query counter for
74         N+1 measurement."""
75
76

```

```

66     def __init__(self) -> None:
67         self._categories = {c.id: c for c in
        _CATEGORIES}
68         self._products = {p.id: p for p in _PRODUCTS}
69         self._reviews: dict[int, list[ReviewRow]] = {}
70         for review in _REVIEWS:
71             self._reviews.setdefault(review.product_id,
              []).append(review)
72         self._next_review_id = max(r.id for r in
        _REVIEWS) + 1
73         self.query_count = 0 # incremented on every
        store read
74
75     def reset_counter(self) -> None:
76         self.query_count = 0
77
78     # --- single-record reads (the naive path uses
        these in a loop) ---
79
80     def product_by_id(self, product_id: int) ->
        ProductRow | None:
81         self.query_count += 1
82         return self._products.get(product_id)
83
84     def category_by_id(self, category_id: int) ->
        CategoryRow | None:
85         self.query_count += 1
86         return self._categories.get(category_id)
87
88     def reviews_for_product(self, product_id: int) ->
        list[ReviewRow]:
89         self.query_count += 1
90         return list(self._reviews.get(product_id, []))
91
92     # --- batched reads (the DataLoader path calls
        these once per tick) ---
93
94     def categories_by_ids(self, ids: list[int]) -> list
        [CategoryRow | None]:
95         self.query_count += 1
96         return [self._categories.get(i) for i in ids]
97
98     def reviews_for_products(self, ids: list[int]) ->
        list[list[ReviewRow]]:
99         self.query_count += 1
100        return [list(self._reviews.get(i, [])) for i in

```

```

101         ids]
102
103     # --- connection page read ---
104
105     def products_page(
106         self, after_id: int, first: int
107     ) -> tuple[list[ProductRow], bool]:
108         """Return up to 'first' products with id >
109            after_id, plus has-next."""
110         self.query_count += 1
111         ordered = sorted(self._products.values(), key=
112             lambda p: p.id)
113         page = [p for p in ordered if p.id > after_id
114            ][: first + 1]
115         return page[:first], len(page) > first
116
117     def total_products(self) -> int:
118         self.query_count += 1
119         return len(self._products)
120
121     # --- mutation ---
122
123     def add_review(self, product_id: int, rating: int,
124         headline: str) -> ReviewRow | None:
125         """Insert a review; returns None when the
126            product does not exist."""
127         if product_id not in self._products:
128             return None
129         self._next_review_id += 1
130         row = ReviewRow(self._next_review_id,
131             product_id, rating, headline)
132         self._reviews.setdefault(product_id, []).append
133             (row)
134         return row

```

Note the deliberate asymmetry: single-record reads (`category_by_id`, `reviews_for_product`) exist alongside batched reads (`categories_by_ids`, `reviews_for_products`). The naive resolvers use the first set; the `DataLoaders` use the second. Same store, same data — the only difference is *how many round trips the resolvers ask for*.

13.3.2 Listing 2: The Strawberry Schema

Listing 13.2 is the chapter in one file. Code-first type definitions with Python hints generating the SDL of Section 13.2.3; a `Context` constructed *once per request* carrying the repository and two `DataLoaders`; a Relay-style

connection with opaque Base64 cursors; a mutation returning a union of `AddReviewSuccess` and `AddReviewError` so expected failures are data; and `margin_percent` — a non-null field that raises a `GraphQLError` carrying `extensions.code` when cost data is missing, so the tests can watch null propagation happen.

Listing 13.2: Catalog schema with connections, union-typed mutations, and `DataLoaders` (`catalog_schema.py`).

```

1  """Strawberry schema for the Northwind Robotics catalog
2
3  Demonstrates: object types, enums, input types, union-
4  typed mutation
5  results, Relay-style connections, context injection,
6  per-request
7  DataLoaders, field-level errors with extensions, and
8  null propagation.
9  """
10
11 from __future__ import annotations
12
13 import base64
14 import enum
15 from typing import Annotated, Optional
16
17 import strawberry
18 from graphql import GraphQLError
19 from strawberry.dataloader import DataLoader
20 from strawberry.types import Info
21
22 from catalog_repository import CatalogRepository,
23     CategoryRow, ProductRow, ReviewRow
24
25 MAX_PAGE_SIZE = 50
26
27 #
28
29 # Enum and context
30 #
31
32 @strawberry.enum

```

```

30 class ProductStatus(enum.Enum):
31     ACTIVE = "active"
32     DISCONTINUED = "discontinued"
33
34
35 class Context:
36     """Per-request context: one repository, one set of
37         DataLoaders.
38
39     NOT a Strawberry type: it is constructed once per
40     request and handed
41     to resolvers via 'info.context'. 'use_loaders'
42     exists only so the
43     N+1 demo can switch resolvers between naive and
44     batched reads.
45
46     """
47
48     def __init__(self, repo: CatalogRepository,
49         use_loaders: bool = True) -> None:
50         self.repo = repo
51         self.use_loaders = use_loaders
52         self.category_loader = make_category_loader(
53             repo)
54         self.review_loader = make_review_loader(repo)
55
56 def get_context() -> Context:
57     return Context(CatalogRepository())
58
59 #
60
61 -----
62
63 # Object types
64 #
65 -----
66
67
68
69 @strawberry.type
70 class Category:
71     id: int
72     name: str
73     description: str
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

66 @strawberry.type
67 class Review:
68     id: int
69     rating: int
70     headline: str
71
72
73 @strawberry.type
74 class Product:
75     id: int
76     sku: str
77     name: str
78     unit_price_cents: int
79     stock: int
80     discontinued: bool
81     _category_id: strawberry.Private[int]
82
83 @strawberry.field
84 async def category(self, info: Info[Context, None])
85     -> Optional[Category]:
86     ctx = info.context
87     if ctx.use_loaders:
88         row = await ctx.category_loader.load(self.
89             _category_id)
90     else: # naive path: one store read per product
91         (the N in N+1)
92         row = ctx.repo.category_by_id(self.
93             _category_id)
94     if row is None:
95         return None
96     return Category(id=row.id, name=row.name,
97         description=row.description)
98
99 @strawberry.field
100 async def reviews(self, info: Info[Context, None])
101     -> list[Review]:
102     ctx = info.context
103     if ctx.use_loaders:
104         rows = await ctx.review_loader.load(self.id
105             )
106     else: # naive path
107         rows = ctx.repo.reviews_for_product(self.id
108             )
109     return [Review(id=r.id, rating=r.rating,
110         headline=r.headline) for r in rows]

```

```

103     @strawberry.field
104     def margin_percent(self, info: Info[Context, None])
105         -> int:
106         """NON-NULL field that errors when cost data is
107             missing.
108
109             Raising here nulls the entire Product (null
110             propagation), because
111             the schema promises this field is never null.
112             """
113         row = info.context.repo.product_by_id(self.id)
114         if row is None or row.unit_cost_cents is None:
115             raise GraphQLError(
116                 "cost data unavailable for product",
117                 extensions={"code": "COST_DATA_UNAVAILABLE"},
118             )
119         margin = row.unit_price_cents - row.
120             unit_cost_cents
121         return round(100 * margin / row.
122             unit_price_cents)
123
124 def _to_product(row: ProductRow) -> Product:
125     return Product(
126         id=row.id,
127         sku=row.sku,
128         name=row.name,
129         unit_price_cents=row.unit_price_cents,
130         stock=row.stock,
131         discontinued=row.discontinued,
132         _category_id=row.category_id,
133     )
134
135 #
136
137 # Relay-style connection
138 #
139
140
141 def encode_cursor(product_id: int) -> str:
142     raw = f"product:{product_id}".encode("ascii")

```

```

139         return base64.urlsafe_b64encode(raw).decode("ascii"
140             )
141
142     def decode_cursor(cursor: str) -> int:
143         try:
144             raw = base64.urlsafe_b64decode(cursor.encode("
145                 ascii")).decode("ascii")
146             prefix, sep, digits = raw.partition(":")
147             if prefix != "product" or sep == "":
148                 raise ValueError("bad_prefix")
149             return int(digits)
150         except (ValueError, UnicodeDecodeError) as exc:
151             raise GraphQLError(
152                 "invalid_cursor", extensions={"code": "
153                     BAD_CURSOR"})
154             ) from exc
155
156     @strawberry.type
157     class PageInfo:
158         has_next_page: bool
159         has_previous_page: bool
160         start_cursor: Optional[str]
161         end_cursor: Optional[str]
162
163     @strawberry.type
164     class ProductEdge:
165         cursor: str
166         node: Optional[Product] # nullable: a nulled node
167                                 # must not void the page
168
169     @strawberry.type
170     class ProductConnection:
171         edges: list[ProductEdge]
172         page_info: PageInfo
173
174     @strawberry.field
175     def total_count(self, info: Info[Context, None]) ->
176         int:
177         # Lazy: only costs a store read when the client
178         # asks for it.
179         return info.context.repo.total_products()

```

```
179
180 #
181 -----
182
181 # Mutation payloads: typed errors via unions
182 #
183 -----
184
183
184
185 @strawberry.input
186 class AddReviewInput:
187     product_id: int
188     rating: int
189     headline: str
190
191
192 @strawberry.type
193 class AddReviewSuccess:
194     review: Review
195
196
197 @strawberry.type
198 class AddReviewError:
199     message: str
200     code: str
201
202
203 AddReviewResult = Annotated[
204     AddReviewSuccess | AddReviewError, strawberry.union(
205         "AddReviewResult"
206     )
207 ]
208
209 #
210 -----
211
209 # Root operations
210 #
211 -----
212
211
212
213 @strawberry.type
214 class Query:
215     @strawberry.field
```

```

216     async def products(
217         self,
218         info: Info[Context, None],
219         first: int = 10,
220         after: Optional[str] = None,
221         status: Optional[ProductStatus] = None,
222     ) -> Optional[ProductConnection]: # nullable root:
223         errors stay partial
224         if not 1 <= first <= MAX_PAGE_SIZE:
225             raise GraphQLError(
226                 f"first must be between 1 and {
227                     MAX_PAGE_SIZE}",
228                 extensions={"code": "
229                     BAD_PAGINATION_ARGUMENT"},
230             )
231         after_id = decode_cursor(after) if after else 0
232         repo = info.context.repo
233         rows, has_next = repo.products_page(after_id,
234             first)
235         if status is ProductStatus.ACTIVE:
236             rows = [r for r in rows if not r.
237                 discontinued]
238         elif status is ProductStatus.DISCONTINUED:
239             rows = [r for r in rows if r.discontinued]
240         edges = [
241             ProductEdge(cursor=encode_cursor(r.id),
242                 node=_to_product(r)) for r in rows
243         ]
244         return ProductConnection(
245             edges=edges,
246             page_info=PageInfo(
247                 has_next_page=has_next,
248                 has_previous_page=after_id > 0,
249                 start_cursor=edges[0].cursor if edges
250                     else None,
251                 end_cursor=edges[-1].cursor if edges
252                     else None,
253             ),
254         )
255
256 @strawberry.field
257 async def product(self, info: Info[Context, None],
258     product_id: int) -> Optional[Product]:
259     row = info.context.repo.product_by_id(
260         product_id)
261     return _to_product(row) if row else None

```

```

252
253
254 @strawberry.type
255 class Mutation:
256     @strawberry.mutation
257     def add_review(
258         self, info: Info[Context, None], input:
259             AddReviewInput
260     ) -> AddReviewResult:
261         if not 1 <= input.rating <= 5:
262             return AddReviewError(
263                 message="rating must be between 1 and 5",
264                 code="RATING_OUT_OF_RANGE",
265             )
266         headline = input.headline.strip()
267         if not headline:
268             return AddReviewError(
269                 message="headline must not be empty",
270                 code="EMPTY_HEADLINE"
271             )
272         row = info.context.repo.add_review(input.
273             product_id, input.rating, headline)
274         if row is None:
275             return AddReviewError(
276                 message=f"product {input.product_id}
277                     not found",
278                 code="PRODUCT_NOT_FOUND",
279             )
280         return AddReviewSuccess(
281             review=Review(id=row.id, rating=row.rating,
282                 headline=row.headline)
283         )
284
285 #
286 -----
287
288 # DataLoaders: one instance per request, batch + per-
289     request cache
290 #
291 -----
292
293
294
295 def make_category_loader(repo: CatalogRepository) ->

```



```

287     DataLoader:
288         async def batch(keys: list[int]) -> list[Optional[
289             CategoryRow]]:
290             # ONE store read per tick, for all keys queued
291             # this tick.
292             return repo.categories_by_ids(keys)
293
294         return DataLoader(load_fn=batch)
295
296 def make_review_loader(repo: CatalogRepository) ->
297     DataLoader:
298     async def batch(keys: list[int]) -> list[list[
299         ReviewRow]]:
300         return repo.reviews_for_products(keys)
301
302     return DataLoader(load_fn=batch)
303
304 schema = strawberry.Schema(query=Query, mutation=
305     Mutation)

```

Two details deserve a second look. `total_count` is a resolver, not a dataclass field: it costs a store read only when the client actually selects `totalCount`, which is the whole philosophy of GraphQL cost accounting in miniature — you pay for what you select, and the schema author decides what each selection costs. And `use_loaders` exists *only* so the next listing can flip the same resolvers between naive and batched store access without changing the query; it is a lab instrument, not a production pattern.

13.3.3 Listing 3: The N+1 Demonstration

Listing 13.3 executes one operation — five products with category and reviews — twice, counting store reads. The output is the chapter’s most important four lines:

```

1 GraphQL operation asks for 5 products + category +
  reviews
2 naive resolvers      : 11 store queries (1 page + 5
  category + 5 review reads)
3 DataLoader path      : 3 store queries (1 page + 1
  batched category read + 1 batched review read)
4 N+1 reproduced and collapsed. OK

```

Listing 13.3: Measured N+1 reproduction and DataLoader collapse (`nplus1_demo.py`).

```

1  """N+1 demonstration: naive per-field store reads vs
   DataLoader batching.
2
3  Runs the same GraphQL operation twice against the same
   repository and
4  prints measured store-query counts. Deterministic and
   offline.
5
6  Run: python nplus1_demo.py
7  """
8
9  from __future__ import annotations
10
11  import asyncio
12
13  from catalog_repository import CatalogRepository
14  from catalog_schema import Context, schema
15
16  CATALOG_QUERY = """
17  query CatalogPage($first: Int!) {
18    products(first: $first) {
19      edges {
20        node {
21          name
22          category { name }
23          reviews { rating }
24        }
25      }
26    }
27  }
28  """
29
30
31  async def run_once(use_loaders: bool) -> tuple[int, int]:
32      """Execute CATALOG_QUERY; return (store_query_count,
33      products_returned)."""
34      repo = CatalogRepository()
35      context = Context(repo, use_loaders=use_loaders)
36      result = await schema.execute(
37          CATALOG_QUERY, variable_values={"first": 5},
38          context_value=context
39      )
40      assert result.errors is None, result.errors
41      assert result.data is not None
42      return repo.query_count, len(result.data["products"])

```

```

41         ]["edges"])
42
43 def main() -> None:
44     naive_count, n = run_and_count(use_loaders=False)
45     batched_count, _ = run_and_count(use_loaders=True)
46
47     print(f"GraphQL_operation_asks_for_{n}_products_+_
48           category_+_reviews")
49     print(f"__naive_resolvers__:_{naive_count:3d}_
50           store_queries_"
51           f"(1_page_+_{n}_category_+_{n}_review_reads)"
52           )
53     print(f"__DataLoader_path__:_{batched_count:3d}_
54           store_queries_"
55           "(1_page_+_1_batched_category_read_+_1_
56           batched_review_read)")
57     assert naive_count == 1 + 2 * n
58     assert batched_count == 3
59     print("N+1_reproduced_and_collapsed._OK")
60
61 def run_and_count(use_loaders: bool) -> tuple[int, int]:
62     return asyncio.run(run_once(use_loaders))
63
64 if __name__ == "__main__":
65     main()

```

Eleven reads is $1 + 2N$ at $N = 5$; at first: 50 it is 101, and each read is a network round trip against a real database. The DataLoader path is constant — 3 reads at any page size — because batching turns “one query per product” into “one query per *relationship per tick*.” The five category reads also demonstrate the cache: products 1 and 2 share category 1, and even the naive count would be worse without deduplication — the loader dedupes keys *and* batches them, which is why the batched count is 3 rather than 5.

13.3.4 Listing 4: The Cost-Control Layer

Listing 13.4 is everything that stands between a curious client and your database: an AST-walking analyzer that computes depth and argument-weighted complexity, a budget enforcer, the persisted-query registry with the server half of the APQ handshake, an AST-based introspection gate,

and `execute_graphql_http` — the single entry point that wires them in the order of Figure 13.1. Centralizing the pipeline in one function is the design decision: there is exactly one way for a document to reach a resolver, so there is exactly one place to audit.

Listing 13.4: Depth limiting, complexity estimation, persisted queries, and the execution gate (`cost_control.py`).

```
1  """Cost control for GraphQL: depth limiting, complexity
   estimation, and
2  persisted queries (allow-list + APQ protocol), plus an
   HTTP-shaped
3  execution entry point used by the FastAPI server and
   the offline tests.
4
5  Everything here operates on the *parsed document*
   before execution, so
6  expensive operations are rejected without touching a
   resolver.
7  """
8
9  from __future__ import annotations
10
11  import hashlib
12  from dataclasses import dataclass
13  from typing import Any, Awaitable, Callable, Optional
14
15  import strawberry
16  from graphql import (
17      DocumentNode,
18      FieldNode,
19      FragmentDefinitionNode,
20      FragmentSpreadNode,
21      InlineFragmentNode,
22      IntValueNode,
23      OperationDefinitionNode,
24      parse,
25  )
26  from graphql.error import GraphQLSyntaxError
27
28  MAX_DEPTH = 7
29  MAX_COMPLEXITY = 200
30  DEFAULT_LIST_MULTIPLIER = 10 # assumed page size when
   'first' is absent
31
32
33  @dataclass(frozen=True)
```

```

34 class OperationMetrics:
35     depth: int
36     complexity: int
37
38
39 class QueryCostExceeded(Exception):
40     """Raised when an operation breaches the depth or
41         complexity budget."""
42
43     def __init__(self, message: str, code: str) -> None:
44         :
45         super().__init__(message)
46         self.code = code
47
48 # -----
49 # Static analysis: depth + complexity
50 # -----
51
52 def _int_argument(field: FieldNode, name: str) ->
53     Optional[int]:
54     for arg in field.arguments:
55         if arg.name.value == name and isinstance(arg.
56             value, IntValueNode):
57             return int(arg.value.value)
58     return None
59
60 def _list_multiplier(field: FieldNode) -> int:
61     """List-returning fields multiply the cost of their
62         sub-selection.
63
64     Only the top list field multiplies (the
65         connection); edges/node are
66     structural and cost 1, otherwise nesting multiplies
67         twice.
68
69     """
70     page = _int_argument(field, "first") or
71         _int_argument(field, "last")
72     if page is not None:
73         return max(1, min(page, 50))

```

```

68     if field.name.value in ("products", "reviews"):
69         return DEFAULT_LIST_MULTIPLIER
70     return 1
71
72
73 def analyze_document(document: DocumentNode) ->
74     OperationMetrics:
75     """Compute (max_depth, total_complexity) over all
76         operations.
77
78         Depth = longest field path, fragment spreads
79         expanded (cycles rejected).
80         Complexity = field count, with list fields
81         multiplying their subtree by
82         the requested page size (or a conservative default)
83         .
84     """
85     fragments = {
86         d.name.value: d
87         for d in document.definitions
88         if isinstance(d, FragmentDefinitionNode)
89     }
90
91     def walk(
92         selections: tuple, depth: int, seen: frozenset[
93             str]
94     ) -> tuple[int, int]:
95         max_depth, cost = depth, 0
96         for sel in selections:
97             if isinstance(sel, FieldNode):
98                 child_depth, child_cost = (
99                     walk(sel.selection_set.selections,
100                         depth + 1, frozenset())
101                     if sel.selection_set
102                     else (depth + 1, 0)
103                 )
104                 cost += 1 + _list_multiplier(sel) *
105                     child_cost
106                 max_depth = max(max_depth, child_depth)
107             elif isinstance(sel, InlineFragmentNode):
108                 d, c = walk(sel.selection_set.
109                     selections, depth, seen)
110                 cost += c
111                 max_depth = max(max_depth, d)
112             elif isinstance(sel, FragmentSpreadNode):
113                 name = sel.name.value

```

```

105         if name in seen or name not in
106             fragments:
107                 raise QueryCostExceeded(
108                     f"invalid fragment spread: {
109                         name}", "BAD_FRAGMENT"
110                     )
111             d, c = walk(
112                 fragments[name].selection_set.
113                 selections, depth, seen | {name}
114             )
115             cost += c
116             max_depth = max(max_depth, d)
117         return max_depth, cost
118
119     worst = OperationMetrics(depth=0, complexity=0)
120     for definition in document.definitions:
121         if isinstance(definition,
122             OperationDefinitionNode):
123             d, c = walk(definition.selection_set.
124                 selections, 0, frozenset())
125             worst = OperationMetrics(max(worst.depth, d
126                 ), max(worst.complexity, c))
127     return worst
128
129
130 def enforce_cost_budget(
131     document: DocumentNode, max_depth: int = MAX_DEPTH,
132     max_cost: int = MAX_COMPLEXITY
133 ) -> OperationMetrics:
134     metrics = analyze_document(document)
135     if metrics.depth > max_depth:
136         raise QueryCostExceeded(
137             f"query depth {metrics.depth} exceeds limit
138             {max_depth}",
139             "DEPTH_LIMIT_EXCEEDED",
140         )
141     if metrics.complexity > max_cost:
142         raise QueryCostExceeded(
143             f"query complexity {metrics.complexity}
144             exceeds limit {max_cost}",
145             "COMPLEXITY_LIMIT_EXCEEDED",
146         )
147     return metrics
148
149 #

```

```

-----
142 # Persisted queries: allow-list registry + APQ
    resolution
143 #
    -----
144
145
146 class ApqError(Exception):
147     def __init__(self, message: str, code: str) -> None
        :
148         super().__init__(message)
149         self.code = code
150
151
152 class PersistedQueryRegistry:
153     """Allow-list of pre-registered operation documents
        keyed by SHA-256."""
154
155     def __init__(self) -> None:
156         self._by_hash: dict[str, str] = {}
157
158     def register(self, document: str) -> str:
159         digest = hashlib.sha256(document.encode("utf-8")
            ).hexdigest()
160         self._by_hash[digest] = document
161         return digest
162
163     def get(self, digest: str) -> Optional[str]:
164         return self._by_hash.get(digest)
165
166
167 def resolve_document(payload: dict[str, Any], registry:
    PersistedQueryRegistry) -> str:
168     """Resolve the operation text from a GraphQL-over-
        HTTP payload.
169
170     Implements the APQ flow server side: a hash-only
        payload must already
171     be registered; a query+hash payload is verified
        against its hash and
172     then registered.
173     """
174     query = payload.get("query")
175     extensions = payload.get("extensions") or {}

```



```

176     persisted = extensions.get("persistedQuery") or {}
177     digest = persisted.get("sha256Hash")
178
179     if digest is None:
180         if not query:
181             raise ApqError("no_query_provided", "
182                             BAD_REQUEST")
183         return query
184     if not isinstance(digest, str) or len(digest) !=
185     64:
186         raise ApqError("malformed_persistedQuery_hash",
187                         "BAD_REQUEST")
188     if query is None:
189         stored = registry.get(digest)
190         if stored is None:
191             raise ApqError("persisted_query_not_found",
192                             "PersistedQueryNotFound")
193         return stored
194     actual = hashlib.sha256(query.encode("utf-8")).
195     hexdigest()
196     if actual != digest:
197         raise ApqError("provided_hash_does_not_match_
198                         query", "PersistedQueryMismatch")
199     registry.register(query)
200     return query
201
202 def uses_introspection(document: DocumentNode) -> bool:
203     """True if any selected field is a meta field (
204         __schema / __type).
205
206     """
207     """AST-based, not substring-based: aliases and
208     fragments cannot hide it.
209     """
210     """(' __typename ' is excluded: it is cheap and used
211     by client caches.)
212     """
213     def walk(selections: tuple) -> bool:
214         for sel in selections:
215             if isinstance(sel, FieldNode):
216                 if sel.name.value in ("__schema", "
217                                         __type"):
218                     return True
219                 if sel.selection_set and walk(sel.
220                     selection_set.selections):
221                     return True
222             elif isinstance(sel, (InlineFragmentNode,

```

```

        FragmentDefinitionNode)):
211         if walk(sel.selection_set.selections):
212             return True
213     return False
214
215     for definition in document.definitions:
216         if hasattr(definition, "selection_set"):
217             if walk(tuple(definition.selection_set.
218                           selections)):
219                 return True
220     return False
221
222 #
223 -----
224 # HTTP-shaped execution entry point (used by server.py
225 # and tests)
226 #
227 -----
228
229 def _error_body(message: str, code: str) -> dict[str,
230 Any]:
231     return {"errors": [{"message": message, "extensions
232                      ": {"code": code}}]}
233
234
235 async def execute_graphql_http(
236     payload: dict[str, Any],
237     *,
238     schema: strawberry.Schema,
239     registry: PersistedQueryRegistry,
240     context_factory: Callable[[], Any],
241     max_depth: int = MAX_DEPTH,
242     max_cost: int = MAX_COMPLEXITY,
243     allow_introspection: bool = False,
244 ) -> tuple[int, dict[str, Any]]:
245     """Parse->resolve->persisted->budget-check->
246        execute.
247
248     Returns (HTTP status, response body) with GraphQL
249     semantics: execution
250     errors are HTTP 200 with a populated 'errors'
251     array; malformed or

```

```

245 over-budget_requests_are_HTTP_400.
246 """
247     try:
248         document_text = resolve_document(payload,
249                                         registry)
250     except ApqError as exc:
251         # APQ misses are HTTP 200 per the protocol: the
252         # client retries.
253         status = 200 if exc.code.startswith("
254             PersistedQuery") else 400
255         return status, _error_body(str(exc), exc.code)
256
257     try:
258         document = parse(document_text)
259     except GraphQLSyntaxError as exc:
260         return 400, _error_body(f"syntax_error:{exc.
261             message}", "GRAPHQL_PARSE_FAILED")
262
263     if not allow_introspection and uses_introspection(
264         document):
265         return 400, _error_body(
266             "introspection_is_disabled", "
267             INTROSPECTION_DISABLED"
268         )
269
270     try:
271         enforce_cost_budget(document, max_depth,
272                             max_cost)
273     except QueryCostExceeded as exc:
274         return 400, _error_body(str(exc), exc.code)
275
276     result = await schema.execute(
277         document_text,
278         variable_values=payload.get("variables"),
279         operation_name=payload.get("operationName"),
280         context_value=context_factory(),
281     )
282     body: dict[str, Any] = {"data": result.data}
283     if result.errors:
284         body["errors"] = [err.formatted for err in
285                             result.errors]
286     return 200, body

```

The complexity estimator encodes one deliberate modeling choice: `_list_multiplier` charges the *connection* field (`products`) for its requested page size but treats the structural edges/node wrappers as cost 1 — otherwise nesting multi-

plies twice and a modest `first:` 2 query reads as cost 441. Cost models are heuristics with adversaries; calibrate them against real traffic and keep the timeout backstop armed.

13.3.5 Listing 5: The FastAPI Host

The server (Listing 13.5) is thin by design: parse JSON, call `execute_graphql_http`, return. Introspection policy is an environment flag defaulting to *off*; the registry is seeded with the mobile app’s catalog operation so allow-listed traffic never pays the APQ round trip. There is no GraphQL router magic here on purpose — when the endpoint is thirty lines you own, the security review is a short meeting.

Listing 13.5: FastAPI host with introspection gating and a seeded allow-list (`server.py`).

```
1  """FastAPI host for the catalog schema.
2
3  The HTTP layer is deliberately thin: all GraphQL policy
4  (persisted-query resolution, depth/complexity budgets, execution) lives
5  in cost_control.py so it is testable without a web framework.
6  Introspection is gated by an explicit policy flag, never by "we forgot".
7
8  Run (dev only): uvicorn server:app --port 8000
9  """
10
11 from __future__ import annotations
12
13 import os
14 from typing import Any
15
16 from fastapi import FastAPI, Request
17 from fastapi.responses import JSONResponse
18
19 from catalog_repository import CatalogRepository
20 from catalog_schema import Context, schema
21 from cost_control import PersistedQueryRegistry,
22     execute_graphql_http
23
24 INTROSPECTION_ENABLED = os.environ.get("
25     NWR_INTROSPECTION", "off") == "on"
```

```

26 # Pre-register the operations the mobile app ships with
    (allow-list seed).
27 _CATALOG_OP = """
28 query CatalogPage($first:Int!){
29   products(first:$first){
30     edges{node{name category{name}}}}
31     pageInfo{hasNextPage endCursor}
32   }
33 }
34 """
35 CATALOG_OP_HASH = _registry.register(_CATALOG_OP)
36
37
38 def _context_factory() -> Context:
39     # One repository + one set of DataLoaders per
        request.
40     return Context(CatalogRepository())
41
42
43 def create_app() -> FastAPI:
44     app = FastAPI(title="Northwind_Robotics_Catalog",
        version="1.0.0")
45
46     @app.post("/graphql")
47     async def graphql_endpoint(request: Request) ->
        JSONResponse:
48         payload: dict[str, Any] = await request.json()
49         status, body = await execute_graphql_http(
50             payload,
51             schema=schema,
52             registry=_registry,
53             context_factory=_context_factory,
54             allow_introspection=INTROSPECTION_ENABLED,
55         )
56         return JSONResponse(body, status_code=status)
57
58     @app.get("/health")
59     async def health() -> dict[str, str]:
60         return {"status": "ok"}
61
62     return app
63
64
65 app = create_app()

```

13.3.6 Listing 6: The httpx Client

Listing 13.6 is the client half of the contract. It separates the three failure channels the wire shape creates: `GraphQLTransportError` for transport failure and 5xx, `GraphQLFieldErrors` for requests that never executed (4xx) and for callers who prefer exceptions, and a returned `GraphQLResponse` carrying *data* and *errors* together for the partial-success case — the client library does not get to decide whether partial data is usable; the application does. `execute_persisted` implements the APQ handshake: hash first, retry with full text only on `PersistedQueryNotFound`. The demo runs entirely in-process through `httpx.ASGITransport` [Enc24]: real HTTP semantics, zero sockets.

Listing 13.6: GraphQL-over-HTTP client with variables, error channels, and APQ (`graphql_client.py`).

```
1  """GraphQL-over-HTTP client with variables, error
   handling, and APQ.
2
3  Transport is injected, so the demo below runs fully
   offline against the
4  in-process FastAPI app (httpx.ASGITransport) -- no
   socket is opened.
5
6  Run: python graphql_client.py
7  """
8
9  from __future__ import annotations
10
11  import asyncio
12  import hashlib
13  import json
14  from dataclasses import dataclass
15  from typing import Any, Optional
16
17  import httpx
18
19  GRAPHQL_PATH = "/graphql"
20
21
22  class GraphQLTransportError(Exception):
23      """Network failure or non-GraphQL HTTP error (5xx,
   timeouts)."""
24
25
26  class GraphQLFieldErrors(Exception):
27      """The response carried a populated 'errors'"""
```

```

28         array_(HTTP_200)."""
29
30     def __init__(self, errors: list[dict[str, Any]],
31                  data: Any) -> None:
32         messages = [e.get("message", "?") for e in
33                     errors]
34         super().__init__(";".join(messages))
35         self.errors = errors
36         self.data = data # partial data may still be
37                          # present
38
39 @dataclass(frozen=True)
40 class GraphQLResponse:
41     data: Any
42     errors: list[dict[str, Any]]
43
44 class GraphQLClient:
45     def __init__(self, base_url: str, transport:
46                  Optional[httpx.AsyncBaseTransport] = None,
47                  timeout: float = 10.0) -> None:
48         self._client = httpx.AsyncClient(
49             base_url=base_url, transport=transport,
50             timeout=timeout
51         )
52
53     async def close(self) -> None:
54         await self._client.aclose()
55
56     async def __aenter__(self) -> "GraphQLClient":
57         return self
58
59     async def __aexit__(self, *exc: object) -> None:
60         await self.close()
61
62     async def execute(
63         self,
64         query: str,
65         variables: Optional[dict[str, Any]] = None,
66         operation_name: Optional[str] = None,
67     ) -> GraphQLResponse:
68         payload: dict[str, Any] = {"query": query}
69         if variables is not None:
70             payload["variables"] = variables
71         if operation_name is not None:

```

```

68         payload["operationName"] = operation_name
69         return await self._post(payload)
70
71     async def execute_persisted(
72         self, query: str, variables: Optional[dict[str, Any]] = None
73     ) -> GraphQLResponse:
74         """Automatic Persisted Queries: hash first,
75         retry with full text
76         only when the server answers
77         PersistedQueryNotFound."""
78         digest = hashlib.sha256(query.encode("utf-8")).
79            hexdigest()
80         ext = {"persistedQuery": {"version": 1, "
81             sha256Hash": digest}}
82         first = await self._post({"extensions": ext, "
83             variables": variables or {}})
84         if not self._is_apq_miss(first):
85             return first
86         # Retry once, sending query text + hash so the
87         # server can register it.
88         return await self._post(
89             {"query": query, "extensions": ext, "
90             variables": variables or {}}
91         )
92
93     @staticmethod
94     def _is_apq_miss(response: GraphQLResponse) -> bool
95     :
96         return any(
97             (e.get("extensions") or {}).get("code") ==
98             "PersistedQueryNotFound"
99             for e in response.errors
100         )
101
102     async def _post(self, payload: dict[str, Any]) ->
103         GraphQLResponse:
104         try:
105             response = await self._client.post(
106                 GRAPHQL_PATH,
107                 content=json.dumps(payload),
108                 headers={"Content-Type": "application/
109                     json"},
110             )
111         except httpx.HTTPError as exc:
112             raise GraphQLTransportError(f"transport_

```



```

102         failure: {exc}) from exc
103     if response.status_code >= 500:
104         raise GraphQLTransportError(f"server_error:
105             {HTTP_{response.status_code}}")
106     body = response.json()
107     errors = body.get("errors") or []
108     # 4xx here means the request never executed (
109     # parse/cost rejection).
110     if response.status_code >= 400:
111         raise GraphQLFieldErrors(errors, data=None)
112     return GraphQLResponse(data=body.get("data"),
113         errors=errors)
114
115 CATALOG_QUERY = """
116 query CatalogPage($first: Int!) {
117   products(first: $first) {
118     edges {
119       node {
120         name
121         category {
122           name
123         }
124       }
125     }
126     pageInfo {
127       hasNextPage
128       endCursor
129     }
130   }
131 }
132 """
133
134 async def demo() -> None:
135     import logging
136
137     from server import create_app # in-process app:
138     zero network
139
140     # Strawberry logs every field error; keep the demo
141     # output readable.
142     logging.getLogger("strawberry.execution").setLevel(
143         logging.CRITICAL)
144
145     transport = httpx.ASGITransport(app=create_app())
146     async with GraphQLClient("http://testserver",
147         transport=transport) as client:
148         # 1. Plain query with variables.
149         page = await client.execute(CATALOG_QUERY,
150             variables={"first": 2})
151         edges = page.data["products"]["edges"]
152         print(f"plain query {len(edges)} products,
153             "
154             f"hasNextPage={page.data['products']['
155                 pageInfo']['hasNextPage']}")

```

```

137
138     # 2. APQ: first call misses, second is served
139         hash-only.
140     await client.execute_persisted(CATALOG_QUERY,
141         variables={"first": 2})
142     cached = await client.execute_persisted(
143         CATALOG_QUERY, variables={"first": 3})
144     print(f"APQ_round_trip:_{len(cached.data['
145         products'] ['edges'])}_products")
146
147     # 3. Field-level error: partial data + errors,
148         HTTP 200.
149     bad = await client.execute(
150         "query_{product(productId:4)}_{name_
151         marginPercent}_{_}"
152     )
153     print(f"field_error_{data={bad.data}_
154         f"code={bad.errors[0]['extensions'] ['code
155         ']]}")
156
157     # 4. Rejected request: over-depth query is
158         refused pre-execution.
159     try:
160         deep = (
161             "query_Deep_{products_{edges_{node_{
162             _"
163             + "reviews_{_} * 12 + "rating" + "_} *
164             12
165             + "_}_{_}_{_}"
166         )
167         await client.execute(deep)
168     except GraphQLFieldErrors as exc:
169         print(f"depth_rejected:_{exc.errors[0] ['
170             extensions'] ['code']]")
171
172     if __name__ == "__main__":
173         asyncio.run(demo())

```

Run it and you get the whole pipeline in four lines of output — a paginated query, an APQ round trip (miss, register, hash-only hit), a field error arriving as partial data, and a depth rejection:

```

1 plain query      : 2 products, hasNextPage=True
2 APQ round trip: 3 products
3 field error      : data={'product': None} code=
    COST_DATA_UNAVAILABLE

```

```
4 depth rejected: DEPTH_LIMIT_EXCEEDED
```

13.3.7 Listing 7: The Test Suite

Listing 13.7 is 21 tests, all in-process via `schema.execute` or `execute_graphql_http`, driven by `asyncio.run` so no pytest plugins are required. It covers the contract surface a reviewer should demand: connection walking without gaps or duplicates, cursor forgery, aliases and `@skip`, both halves of null propagation (non-null error voids the product; the nullable list element voids only itself), all four typed mutation errors, the $N+1$ counts as *assertions* (11 naive, 3 batched), depth and complexity rejection, the full APQ handshake including hash mismatch, and introspection denied by default and served when explicitly gated on.

Listing 13.7: In-process test suite: 21 tests, zero network (`test_catalog.py`).

```
1  """In-process test suite for the Northwind Robotics GraphQL catalog.
2
3  No sockets, no network, deterministic seed data. Async execution is driven
4  with asyncio.run so no pytest plugins are required.
5
6  Run: pytest test_catalog.py -q
7  """
8
9  from __future__ import annotations
10
11  import asyncio
12  import logging
13  from typing import Any, Optional
14
15  import pytest
16
17  from catalog_repository import CatalogRepository
18  from catalog_schema import Context, schema
19  from cost_control import (
20      PersistedQueryRegistry,
21      analyze_document,
22      enforce_cost_budget,
23      QueryCostExceeded,
24      execute_graphql_http,
25  )
26  from graphql import parse
27
```

```

28 logging.getLogger("strawberry.execution").setLevel(
    logging.CRITICAL)
29
30
31 def run(
32     query: str,
33     variables: Optional[dict[str, Any]] = None,
34     repo: Optional[CatalogRepository] = None,
35 ):
36     """Execute against a fresh repository unless one is
        supplied."""
37     repo = repo or CatalogRepository()
38     context = Context(repo)
39     result = asyncio.run(
40         schema.execute(query, variable_values=variables
            , context_value=context)
41     )
42     return result, repo
43
44 #
45 -----
46 # Queries, variables, aliases, fragments
47 #
48 -----
49
50 def test_products_connection_first_page() -> None:
51     result, _ = run(
52         """
53         query Page($first: Int!) {
54             products(first: $first) {
55                 edges { cursor node { sku name } }
56                 pageInfo { hasNextPage hasPreviousPage
                    startCursor endCursor }
57                 totalCount
58             }
59         }
60         """,
61         variables={"first": 2},
62     )
63     assert result.errors is None
64     data = result.data["products"]
65     assert [e["node"]["sku"] for e in data["edges"]] ==

```

```

        ["NWR-ACT-100", "NWR-ACT-200"]
66     assert data["pageInfo"]["hasNextPage"] is True
67     assert data["pageInfo"]["hasPreviousPage"] is False
68     assert data["totalCount"] == 5
69
70
71 def test_connection_walk_is_gapless() -> None:
72     """Walking by cursor visits every product exactly
73         once."""
74     seen: list[str] = []
75     after: Optional[str] = None
76     for _ in range(10):
77         result, _ = run(
78             """
79             query Walk($first: Int!, $after: String) {
80             products(first: $first, after: $after) {
81             edges { cursor node { sku } }
82             pageInfo { hasNextPage endCursor }
83             }
84             """
85             , variables={"first": 2, "after": after},
86         )
87         assert result.errors is None
88         page = result.data["products"]
89         seen.extend(e["node"]["sku"] for e in page["edges"])
90         if not page["pageInfo"]["hasNextPage"]:
91             break
92         after = page["pageInfo"]["endCursor"]
93     assert seen == [
94         "NWR-ACT-100", "NWR-ACT-200", "NWR-SEN-310", "
95         NWR-SEN-410", "NWR-CTL-900",
96     ]
97
98 def test_invalid_cursor_returns_field_error() -> None:
99     result, _ = run(
100         """
101         query { products(first: 2, after: "%") {
102             edges { cursor } } }
103     """
104     )
105     assert result.data == {"products": None}
106     assert result.errors is not None
107     assert result.errors[0].extensions["code"] == "BAD_CURSOR"

```

```

106
107 def test_status_filter_and_alias() -> None:
108     result, _ = run(
109         """
110         query {
111             active: products(first: 10, status: ACTIVE) {
112                 edges { node { sku } }
113             }
114             retired: products(first: 10, status: DISCONTINUED) {
115                 edges { node { sku } }
116             }
117         }
118         """
119     )
120     assert result.errors is None
121     assert [e["node"]["sku"] for e in result.data["active"]["edges"]] == [
122         "NWR-ACT-100", "NWR-ACT-200", "NWR-SEN-310", "NWR-CTL-900",
123     ]
124     assert [e["node"]["sku"] for e in result.data["retired"]["edges"]] == [
125         "NWR-SEN-410"
126     ]
127
128 def test_fragments_and_skip_directive() -> None:
129     result, _ = run(
130         """
131         query F($withReviews: Boolean!) {
132             product(productId: 1) {
133                 ...ProductCard
134             }
135             fragment ProductCard on Product {
136                 name
137                 reviews @skip(if: $withReviews) { rating }
138             }
139         """
140     , variables={"withReviews": True},
141     )
142     assert result.errors is None
143     assert result.data == {"product": {"name": "Servo Linear Actuator 100mm"}}
144
145

```

```

146 #
147 # Nullability propagation on errors
148 #
149
150
151 def test_non_null_field_error_nulls_the_product() ->
    None:
152     """marginPercent is Int! and raises for product 4 (
        no cost data).
153
154     Per the spec's null-propagation rule the error
        bubbles to the nearest
155     nullable position: the whole 'product' field
        becomes null, and the
156     response is partial: HTTP 200 with data + errors.
157     """
158     result, _ = run("query { product(productId: 4) {
        name marginPercent } }")
159     assert result.data == {"product": None}
160     assert result.errors is not None
161     error = result.errors[0]
162     assert error.path == ["product", "marginPercent"]
163     assert error.extensions["code"] == "
        COST_DATA_UNAVAILABLE"
164
165
166 def test_nullable_sibling_survives_in_list() -> None:
167     """Inside a nullable list element, only that
        element nulls out."""
168     result, _ = run(
169         """
170     query {
171     products(first: 5) {
172     edges { node { sku marginPercent } }
173     }
174     }
175     """
176     )
177     assert result.errors is not None
178     nodes = [e["node"] for e in result.data["products"]
        ]["edges"]
179     assert nodes[3] is None # product 4 nulled by

```

```

180         propagation
181         assert nodes[0]["marginPercent"] == 54 # siblings
182         intact
183     #
184     -----
185     # Mutations: typed errors via union payloads
186     #
187     -----
188     ADD_REVIEW = """
189     mutation Add($input: AddReviewInput!) {
190       addReview(input: $input) {
191         __typename
192         ... on AddReviewSuccess { review { id rating
193           headline } }
194         ... on AddReviewError { code message }
195       }
196     }
197     """
198
199     def test_add_review_success() -> None:
200         result, _ = run(
201             ADD_REVIEW,
202             variables={"input": {"productId": 2, "rating":
203               4, "headline": "Solid"}},
204         )
205         assert result.errors is None
206         payload = result.data["addReview"]
207         assert payload["__typename"] == "AddReviewSuccess"
208         assert payload["review"]["rating"] == 4
209
210     @pytest.mark.parametrize(
211         "input_vars, code",
212         [
213             ({ "productId": 2, "rating": 0, "headline": "Bad
214               " }, "RATING_OUT_OF_RANGE"),
215             ({ "productId": 2, "rating": 6, "headline": "Bad
216               " }, "RATING_OUT_OF_RANGE"),
217             ({ "productId": 2, "rating": 5, "headline": "

```



```

216         "}, "EMPTY_HEADLINE"),
217         ({ "productId": 999, "rating": 5, "headline": "
218             Ghost"}, "PRODUCT_NOT_FOUND"),
219     ],
220 )
221 def test_add_review_typed_errors(input_vars: dict[str,
222     Any], code: str) -> None:
223     result, _ = run(ADD_REVIEW, variables={"input":
224         input_vars})
225     assert result.errors is None # expected failure is
226         data, not an exception
227     payload = result.data["addReview"]
228     assert payload["__typename"] == "AddReviewError"
229     assert payload["code"] == code
230
231 # -----
232 # N+1: measured query counts
233 # -----
234
235 CATALOG_QUERY = """
236 query CatalogPage($first: Int!) {
237   products(first: $first) {
238     edges {
239       node {
240         name
241         category {
242           name
243           reviews {
244             rating
245           }
246         }
247       }
248     }
249   }
250 }
251 """
252
253 def _count_queries(use_loaders: bool) -> int:
254     repo = CatalogRepository()
255     context = Context(repo, use_loaders=use_loaders)
256     result = asyncio.run(
257         schema.execute(
258             CATALOG_QUERY, variable_values={"first":
259                 5}, context_value=context
260         )
261     )
262     assert result.errors is None
263     return repo.query_count

```

```

251
252
253 def test_nplus1_naive_explosion() -> None:
254     assert _count_queries(use_loaders=False) == 11 # 1
        + 5 + 5
255
256
257 def test_dataloader_collapses_nplus1() -> None:
258     assert _count_queries(use_loaders=True) == 3 # 1 +
        1 + 1
259
260
261 #
    -----
262 # Cost control: depth, complexity, persisted queries
263 #
    -----
264
265
266 def test_depth_limiter_rejects_deep_query() -> None:
267     deep = (
268         "query_Deep_{products_{edges_{node_{
269             + "reviews_{ " * 12 + "rating" + " " * 12 + "
                }_}}_}}
270     )
271     with pytest.raises(QueryCostExceeded) as excinfo:
272         enforce_cost_budget(parse(deep))
273     assert excinfo.value.code == "DEPTH_LIMIT_EXCEEDED"
274
275
276 def test_complexity_scales_with_page_size() -> None:
277     small = analyze_document(
278         parse("query_{products(first:_2)_edges_{
                node_{name_}}_}}")
279     )
280     large = analyze_document(
281         parse("query_{products(first:_50)_edges_{
                node_{name_}}_}}")
282     )
283     assert large.complexity > small.complexity
284
285
286 def test_complexity_limit_rejects_god_query() -> None:
287     god = "query_{ " + " ".join(

```

```

288         f"p{i}:_products(first:_50)_{{_edges_{{_node_{{
289             _name_reviews_{{_rating_}}_}}_}}_}"
290     for i in range(8)
291 ) + "_}"
292 with pytest.raises(QueryCostExceeded) as excinfo:
293     enforce_cost_budget(parse(god))
294 assert excinfo.value.code == "
295     COMPLEXITY_LIMIT_EXCEEDED"
296
297 def test_persisted_query_apq_flow() -> None:
298     registry = PersistedQueryRegistry()
299     factory = lambda: Context(CatalogRepository()) #
300     noqa: E731
301
302     async def apq_round_trip() -> tuple[int, dict[str,
303         Any]]:
304         import hashlib
305
306         digest = hashlib.sha256(CATALOG_QUERY.encode("
307             utf-8")).hexdigest()
308         ext = {"persistedQuery": {"version": 1, "
309             sha256Hash": digest}}
310         # Step 1: hash only -> PersistedQueryNotFound (
311             HTTP 200)
312         status, body = await execute_graphql_http(
313             {"extensions": ext, "variables": {"first":
314                 2}},
315             schema=schema, registry=registry,
316             context_factory=factory,
317         )
318         assert status == 200
319         assert body["errors"][0]["extensions"]["code"]
320             == "PersistedQueryNotFound"
321         # Step 2: query + hash -> executes and
322             registers
323         return await execute_graphql_http(
324             {"query": CATALOG_QUERY, "extensions": ext,
325                 "variables": {"first": 2}},
326             schema=schema, registry=registry,
327             context_factory=factory,
328         )
329
330     status, body = asyncio.run(apq_round_trip())
331     assert status == 200
332     assert "errors" not in body

```

```

321     assert len(body["data"]["products"]["edges"]) == 2
322
323
324 def test_persisted_query_hash_mismatch_rejected() ->
None:
325     registry = PersistedQueryRegistry()
326
327     async def mismatch() -> tuple[int, dict[str, Any]]:
328         return await execute_graphql_http(
329             {
330                 "query": "query { product(productId: 1)
331                     { name } }",
332                 "extensions": {"persistedQuery": {"
333                     sha256Hash": "0" * 64}},
334             },
335             schema=schema,
336             registry=registry,
337             context_factory=lambda: Context(
338                 CatalogRepository()),
339
340         status, body = asyncio.run(mismatch())
341         assert status == 200
342         assert body["errors"][0]["extensions"]["code"] == "
PersistedQueryMismatch"
343
344 def test_introspection_blocked_by_default() -> None:
345     async def probe() -> tuple[int, dict[str, Any]]:
346         return await execute_graphql_http(
347             {"query": "{ __schema { types { name } } }"},
348             schema=schema,
349             registry=PersistedQueryRegistry(),
350             context_factory=lambda: Context(
351                 CatalogRepository()),
352
353         status, body = asyncio.run(probe())
354         assert status == 400
355         assert body["errors"][0]["extensions"]["code"] == "
INTROSPECTION_DISABLED"
356
357 def test_introspection_allowed_when_gated_on() -> None:
358     async def probe() -> tuple[int, dict[str, Any]]:

```

```

359         return await execute_graphql_http(
360             {"query": "{__schema{__queryType{name}}}"},
361             schema=schema,
362             registry=PersistedQueryRegistry(),
363             context_factory=lambda: Context(
364                 CatalogRepository()),
365             allow_introspection=True,
366         )
367
368     status, body = asyncio.run(probe())
369     assert status == 200
370     assert body["data"]["__schema"]["queryType"]["name"]
371         == "Query"

```

```

1 .....
   [100%]
2 21 passed in 2.78s

```

13.4 Common Pitfalls & Anti-Patterns

Each entry names the failure mode, why it happens, and the smallest durable fix. Most are defaults — GraphQL ships insecure-by-default in the specific sense that its ergonomics and its guardrails are both opt-in.

13.4.1 N+1 by Construction

Failure. Every relationship field performs its own store read; latency and database load scale with page size, not with request count. **Why.** Field-level resolution is the default physics; nothing in a naive resolver hints it is one of fifty identical calls. **Fix.** DataLoader per request, batch reads in the repository, *count queries in tests* — our suite asserts 3, not “fast.”

13.4.2 Open Introspection in Production

Failure. `__schema` answers anyone who asks with your complete type inventory. **Why.** Frameworks enable it by default because GraphiQL needs it. **Fix.** Gate it at the validation layer (AST check, not substring), default off in production, on in development; treat the schema as an asset with an audience.

13.4.3 Unlimited Depth and Complexity

Failure. A cyclic traversal or a `first: 10000` god-query pins the executor and the database. **Why.** The client chooses the work; ungarded servers accept all of it. **Fix.** Static depth and argument-weighted complexity budgets between parse and execution, plus resolver timeouts for estimation errors.

13.4.4 Treating GraphQL Errors as HTTP 500 Only

Failure. The client checks `status_code`, sees 200, renders `data.product.name` — which is `null` — and the stack trace ships to a user. Or the inverse: any `errors` entry discards a usable partial page. **Why.** Two error channels with different semantics, and REST instincts cover only one. **Fix.** Client libraries expose transport, field-error, and partial-data as distinct outcomes (Listing 13.6); clients render partial data where the schema’s nullability says it is safe.

13.4.5 Schema Equals Database Schema

Failure. Tables are exported verbatim as types; every re-normalization is a breaking API change; internal columns become public surface. **Why.** Code generation from the ORM is ten minutes of work. **Fix.** Design the schema as the *product contract* — client-shaped, not table-shaped — and let resolvers be the anti-corruption layer. Our `Product` hides `category_id` (a storage detail) behind `category` (a concept).

13.4.6 God-Queries from Your Own Clients

Failure. One team’s dashboard composes a 40-field, depth-9 operation that runs every 30 seconds from every open browser; the incident review blames “traffic.” **Why.** Composition is the feature; nobody budgets it. **Fix.** Persisted queries for first-party clients (the set of operations is known at build time), per-client cost budgets, and complexity telemetry per operation name so the expensive ones have owners.

13.4.7 DataLoader Shared Across Requests

Failure. A module-level loader caches user A’s record and serves it to user B. **Why.** The cache is the feature; its scope is the footgun — it must die with the request. **Fix.** Construct loaders in the context factory, one set per request, as Listing 13.2 does; never in module scope.

13.4.8 Bypassing the HTTP Cache Without a Plan

Failure. Everything is POST `/graphql`; the CDN is a pass-through; a product launch melts the origin that REST would have served from cache.

Why. The default GraphQL-over-HTTP mapping discards the URL-as-cache-key. **Fix.** Decide deliberately: APQ over GET restores cache keys for read-heavy public traffic; private, per-user data was never CDN-cacheable anyway — but know which you are serving before you choose the transport [Fie22a].

13.5 Hands-On Production Lab & Real-World Case Study

13.5.1 Lab: Build, Break, and Harden the Catalog API

Four hours, one Windows workstation, zero network. Work in `C:\training\graphql_lab`.

1. **Stand up the lab.** Create the six files of Section 13.3. Then:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install "strawberry-graphql[fastapi]>=0.230" "
   fastapi>=0.110" "httpx>=0.27" "pytest>=8"
4 pytest -q
```

All 21 tests pass. You now hold a working GraphQL service with its guardrails already attached — read them before you remove them.

2. **Reproduce the N+1.** Run `python nplus1_demo.py` and record the counts (11 naive, 3 batched). Then edit the schema so `use_loaders` defaults to `False` and re-run `pytest`: `test_dataloader_collapses_nplus1` fails. Revert. You have now seen the failure as a test assertion, which is the only way it will ever stay fixed.
3. **Find the threshold.** In a Python REPL, call `analyze_document(parse(...))` on `CATALOG_QUERY` with `first = 2, 10, 50` and record complexity. Then raise it until `COMPLEXITY_LIMIT_EXCEEDED` fires. Note the page size at which a *legitimate* query dies — that number is a product decision, not an implementation detail.
4. **Break the allow-list.** Using the client, POST a hash-only request for an unregistered operation and observe `PersistedQueryNotFound`; then send a mismatched hash/text pair and observe `PersistedQueryMismatch`.

Explain in one sentence why the mismatch check exists (hint: the hash is the allow-list key).

5. **Harden the endpoint.** With introspection off, attempt `{ __schema { types { name } } }` and an aliased variant; both must fail with `INTROSPECTION_DISABLED`. Then try the array-of-queries batch attack (`[{"query": ...}, {"query": ...}, ...]`) against `server.py`: it is rejected (the endpoint accepts only objects) — state which control catches it and what you would add if batching were a product requirement.
6. **Write the regression.** Add a test that walks the entire connection with `first: 2` and asserts the five SKUs in order with no duplicates — compare with `test_connection_walk_is_gapless`, which already does this. If yours is identical, write a different one: cursor reuse after a mutation that adds a review must not shift the product ordering.

13.5.2 Case Study: The Introspection-Enabled Enumeration Incident

Composite reconstruction from public incident patterns; all names, figures, and timelines fictionalized.

A mid-size robotics-parts marketplace — call it Northwind Robotics' commerce arm — shipped a GraphQL gateway in front of its catalog, pricing, and dealer-account services to feed a new mobile app. Launch went well. Nine weeks later, a security researcher's responsible disclosure landed: a complete map of every type and field in the graph, *including two that never appeared in any shipped client* — `dealerAccount { terms { creditLimitCents } }` and an undocumented `internalNotes` field on `Product`.

The disclosure's reproduction steps were four lines long, and that is the lesson. Step one: introspection was enabled in production because nobody had turned it off — GraphiQL had been “temporarily” exposed for the mobile team and the temporary flag ossified. Step two: with the field inventory in hand, the researcher composed valid queries against `dealerAccount` in minutes; field-level authorization existed on *most* account fields, but `creditLimitCents` had been added in a hurry and its resolver checked only authentication, not dealer-scoping — any authenticated dealer could page through every dealer's credit terms, with cursor pagination doing what it is designed to do: make enumeration complete and cheap. Step three: the gateway's only rate limit counted *requests*; the enumeration ran inside array-of-queries batches of fifty, so a full exfiltration of 38,000 dealer records presented to the WAF as 760 ordinary-looking POSTs over

a weekend. Step four: the cost gate did not exist. Depth and complexity limits had been ticketed as “post-launch hardening.”

The remediation reads like this chapter’s checklist, because this chapter is written from incidents like this one. Introspection and GraphQL gated behind the same explicit production flag, default off. Authorization moved into a resolver decorator with a deny-by-default audit — every field returning account data declares its required scope, and CI fails on undeclared fields. Array batching capped at five operations with cost summed across the batch. Rate limiting re-keyed from requests to estimated complexity units per token per minute. And the two shadow fields were removed from the schema, because the deepest finding was not a bug but a governance failure: the schema had drifted from the product contract, and introspection made the drift public. The total engineering cost of the remediation was about three engineer-weeks; the incident review estimated the enumeration itself at under \$50 of compute — attackers get the bulk discount that your missing cost controls hand them.

Warning

Every control in the remediation was a *default* somewhere waiting to be flipped: introspection on, batching unbounded, request-count rate limits, authorization-by-convention. A production GraphQL review is substantially a review of defaults. Write them down as explicit policy — the lab’s step 5 — or the incident review will write them down for you.

13.6 Self-Assessment Exercises & Deep-Dive Questions

13.6.1 Recall and Comprehension

1. List the eight kinds in the GraphQL type system and give a catalog-domain example of each.
2. Write the four list/nullability combinations for [Review] and state, for each, what an error in one element does to the response.
3. What are the three phases of request processing, and which failure channel does each use?
4. Why can variables not be used to inject into the query document? What injection remains possible?

5. In the APQ handshake, what does the client send first, what does the server answer on a miss, and why is the miss an HTTP 200?

13.6.2 Application

6. Extend `catalog_schema.py` with a `Category.products` field. Predict the $N+1$ count for a query selecting all categories and their products' names before you run it; then instrument and verify.
7. Our estimator charges connection fields by their `first` argument. Construct a query that is cheap by our estimator but expensive to execute. What estimator change closes the hole, and what legitimate traffic does it harm?
8. Implement `last/before` backward pagination in the repository and schema. What does `hasPreviousPage` require that forward pagination did not?
9. Add a `@deprecated` field to the schema (e.g. rename `unitPriceCents` to `priceCents` and deprecate the old one). Which client-facing guarantee of continuous evolution does this exercise?
10. Using only Listing 13.6's error channels, write client logic that renders a product card from partial data when `COST_DATA_UNAVAILABLE` is the only error, but retries the operation on `GraphQLTransportError`.

13.6.3 Deep-Dive Questions

11. The specification executes mutation root fields serially but query fields without ordering. Why is serial mutation execution the only ordering guarantee the spec can make, and what does it imply for client design?
12. Null propagation couples error containment to schema design. Argue for and against declaring `edges: [ProductEdge!]!` in a public schema. What telemetry would tell you whether you chose correctly?
13. Persisted-query allow-lists turn an open query language into a closed RPC surface. What does GraphQL still buy you under a full allow-list, and what does it cost?
14. Our complexity model multiplies by page size at the connection only. Sketch a model that also prices *resolver heterogeneity* (a field hitting a search index costs more than a field hitting memory). Where does per-field cost metadata live in a code-first schema?

15. Federation moves the $N+1$ problem from resolver-to-database to gateway-to-subgraph. Does DataLoader-style batching transfer to entity resolution across services? What breaks?

13.7 Interview Q&A Vault

Ordered junior $\rightarrow$ staff. Answers are model responses at the depth a strong candidate gives verbally.

- Q1. What is GraphQL, in one paragraph?** A query language and runtime for APIs: the server publishes a typed schema, the client sends an operation selecting exactly the fields it needs, and the response mirrors that selection. One endpoint, client-composed shapes, strong typing, introspection.
- Q2. How is a GraphQL request different from a REST request?** REST addresses a resource whose shape the server fixed; GraphQL POSTs a document describing the desired shape to one endpoint. REST spreads a screen's data across k round trips; GraphQL composes it into one — at the price of server-side cost governance.
- Q3. Name the built-in scalar types.** Int, Float, String, Boolean, ID. ID serializes as a string and means “opaque; do not parse.” Custom scalars (e.g. `DateTime`) define their own serialization.
- Q4. What is the difference between an interface and a union?** An interface declares fields every implementor must have, so you can select common fields directly; a union's members share nothing, so all selection goes through inline fragments. Unions are the right tool for typed mutation results; interfaces for genuine “is-a” hierarchies.
- Q5. What does the exclamation mark mean in `Int!`?** Non-null: the field always resolves to a value. The subtlety is the failure mode — if a non-null field errors, the null bubbles to the nearest nullable ancestor rather than stopping at the field.
- Q6. What are the three operation types?** Query (read), mutation (write; root fields execute serially, in order), subscription (selection plus an event stream, usually over WebSockets or SSE).
- Q7. What are variables and why use them?** Typed arguments serialized in their own JSON object beside the query. They keep the document static and cacheable, and they make query-string injection structurally impossible — there is no string context to escape from.

- Q8. What is a fragment?** A named, reusable selection set. With `@include(if:)` and `@skip(if:)`, one cached operation can serve several UI states without string-templating the query.
- Q9. What is an alias?** `active: products(status: ACTIVE)` — renames the result key, letting you invoke the same field twice with different arguments in one operation.
- Q10. What does a GraphQL response look like?** A JSON object with `data` (mirroring the operation, possibly partial or `null`) and, on field errors, an `errors` array whose entries carry `message`, `path`, `locations`, and `extensions`. Field errors arrive as HTTP 200.
- Q11. Explain nullability propagation on errors.** An error at a nullable field nulls that field. An error at a *non-null* field cannot be represented — the type forbids null — so the error bubbles: the parent object is nulled, and so on up until a nullable position absorbs it; if none does, `data` itself becomes `null`. Non-null is a containment policy, not a reliability claim: it says “if this fails, void my parent too.” Design nullability from the client’s rendering needs.
- Q12. What is the N+1 problem in GraphQL and why is it structural?** Field-level resolution means every product’s `category` resolver runs independently: a page of N products plus one page query is $N+1$ store reads. It is structural because composability — resolvers not knowing their caller — is the feature that causes it. Our demo measures $1+2N$ reads naive versus 3 batched at $N=5$.
- Q13. How does DataLoader solve N+1, and what is its caching scope?** Within one event-loop tick, resolvers enqueue keys instead of querying; the loader dedupes keys, issues one batch read, and distributes results positionally to the waiting futures. The cache is **per request**: one loader instance per request, built in the context factory. Sharing one across requests serves stale data across user boundaries — a security bug, not a performance bug.
- Q14. Describe the Relay connection spec.** A paginated field returns a connection: `edges` (opaque cursor + node), `pageInfo` (`hasNextPage`, `hasPreviousPage`, `startCursor`, `endCursor`), with `first/after` forward and `last/before` backward. Cursors are value-based (keyset) bookmarks, computed with `fetch-p+1` for `hasNextPage`; server caps `first`.
- Q15. Field errors vs. top-level errors — and where do typed**

mutation errors fit? Top-level `errors` entries are exceptional failures (resolver could not produce the field). Expected business failures — validation, rule rejection — should be *data*: a union payload of success and error types selected with inline fragments. Rule of thumb: typed errors inform the user; top-level errors page the on-call.

- Q16. What goes in `extensions.code`?** The machine-readable error kind — `COST_DATA_UNAVAILABLE`, `BAD_CURSOR` — so clients branch on codes, not on English `message` strings. It is the open extension point the spec deliberately leaves for exactly this.
- Q17. How do you gate introspection, and why?** Reject documents selecting `__schema` or `__type` at the validation layer — an AST check, not a substring match, because aliases and fragments defeat substrings. Why: introspection publishes your complete field inventory, including fields no client ships. Off for unauthenticated production traffic; on in development.
- Q18. Why is mutation execution serial?** It is the specification's only ordering guarantee, made precisely so multi-step writes have defined semantics: root mutation fields run in document order, each seeing the previous one's effects. Queries have no such guarantee and may resolve in parallel.
- Q19. What is a persisted query?** A pre-registered operation identified by hash. As an allow-list it closes the query language: unregistered documents never parse. As APQ it is also a bandwidth optimization — send the hash, retry with full text only on `PersistedQueryNotFound`.
- Q20. Walk through the APQ protocol flow.** Client sends `extensions.persistedQuery.sha` with no query text. Server misses: HTTP 200 with error code `PersistedQueryNotFound`. Client retries with query text plus hash; server verifies the hash matches the text (`PersistedQueryMismatch` if not), registers, executes. Steady state: 64 bytes uphill, and hash-keyed GETs become CDN-cacheable.
- Q21. Design cost limiting for a public GraphQL API.** Layers, firing in order: (1) parse limits (document size); (2) static analysis between parse and execution — depth cap, argument-weighted complexity (list fields multiply subtree cost by page size), alias and batch multipliers; (3) persisted-query allow-list for first-party clients; (4) per-field resolver timeouts as the backstop for estimation errors; (5) rate limiting keyed on *complexity units per token*, not request count, because one request

can carry a batch of fifty.

Q22. Why is request-count rate limiting insufficient for GraphQL?

The work per request varies by orders of magnitude — the client composes it — and array batching puts many operations in one HTTP request. “100 requests/minute” can mean 100 cheap reads or 100 batches of god-queries. Limit estimated cost per principal per window; count requests only as a sanity backstop.

Q23. How do you version a GraphQL API? Mostly, you don’t: continuous evolution. Additive changes are free; remove nothing — mark fields `@deprecated(reason:)` and track per-field usage telemetry until a field’s callers reach zero, then remove. When a truly breaking redesign is unavoidable, version the *endpoint* (`/graphql/v2`) as a last resort. The philosophy trades REST’s flag days for an eternal compatibility obligation — backed by telemetry, or it is a guess.

Q24. GraphQL subscriptions: transport choices? WebSockets with the `graphql-ws` subprotocol is the mainstream: full-duplex, mature client support. Server-sent events are simpler (HTTP, auto-reconnect, CDN-friendly) and suffice for server-to-client streams. In WebSocket mode, remember authn/z at connection upgrade *and* per-message, and cap concurrent subscriptions per principal — each one holds server state.

Q25. Where does authorization live in a GraphQL server? At the field, not the route — the schema advertises everything to everyone, so “the endpoint requires auth” says nothing about `salaryHistory`. Enforce in resolvers or a directive / middleware layer with the principal on the context, deny by default, and audit the schema for fields lacking a declared scope. Where partial visibility is legitimate, return `NOT_AUTHORIZED` as typed data.

Q26. “Variable injection is impossible” — true? What injection remains? True for the query document: variables are a separate JSON object, never interpolated into the parse. Resolver-side injection is fully real: a resolver concatenating a variable into SQL, a shell command, or a downstream URL is injection in a typed costume. Parameterized queries and boundary validation remain non-negotiable [\[Kle17\]](#).

Q27. When would you refuse GraphQL and ship REST? When the traffic is public, read-heavy, and CDN-cacheable — GET with ETags is an operational superpower POST cannot match; when up-

loads/downloads dominate (multipart in GraphQL is an awkward bolt-on); when consumers are countless and unknown, so cost governance cannot rely on allow-lists; or when the team lacks the appetite for the guardrail stack GraphQL demands.

- Q28. Your complexity estimator can be gamed. How, and what is the backstop?** Estimators price shapes, not data: a depth-3 query over a low-cardinality field can still fan out expensively in a resolver, and `first: 1` over a full-table scan is “cheap” only on paper. Mitigations: per-field cost weights calibrated to measured resolver latency, timeouts per resolver and per operation, and query-count telemetry per operation name. Static analysis is the sieve; timeouts are the floor.
- Q29. Design the error contract for a GraphQL platform serving thirty client teams.** Two documented channels: typed union payloads for expected, user-actionable failure (versioned like the schema, codes enumerated in the contract), and top-level errors with `extensions.code` for exceptional failure. A written nullability policy per domain (what voids the parent), client guidance on rendering partial data, and SLOs that count `errors` rates per code — otherwise every team’s alert channel becomes your schema review.
- Q30. How do you roll out a schema change that is secretly breaking?** “Breaking” hides in semantics, not syntax: reinterpreting a field, tightening a non-null, reordering enum members a client switched on. Defense is process: additive-only CI checks on the SDL, per-field usage telemetry before any deprecation completes, contract tests generated from real persisted operations, and staged rollout with per-client pinning where the platform allows it.
- Q31. When does federation pay for itself, and what new failure modes does it buy?** When schema ownership, not query volume, is the bottleneck — many teams blocked on one schema’s review queue. It buys: composition conflicts discovered at publish time, the gateway as a latency and availability chokepoint, entity resolution re-creating *N+1 across services*, and distributed tracing becoming a prerequisite rather than a nicety. Federate for organizational scaling; not because the graph is big.
- Q32. A client team demands `totalCount` on every connection. Take a position.** Refuse the default, offer the opt-in. `COUNT(*)` over a filtered relation is the offset problem in disguise — cost grows with the corpus while the value is usually decorative (“1–50 of 38,412”). Ship

`hasNextPage` by default, `totalCount` behind an explicit argument with a cost weight that prices it honestly, and approximate counts where the UI truly needs magnitude.

Q33. Design the migration of a 200-endpoint REST API to GraphQL without a flag day. Strangler pattern: the GraphQL gateway fronts the existing REST services, resolvers initially delegate to them (and inherit their cacheability problems — acknowledged, measured). Migrate consumer by consumer behind persisted operations; the allow-list doubles as the migration ledger. Deprecate REST endpoints only when telemetry shows zero traffic. The failure mode to design against is the permanent halfway state — budget for finishing.

13.8 External Bibliography & Further Reading

- **GraphQL Specification (October 2021)** [Gra21] — the normative text. Read §3 (type system), §6 (execution), and §7 (response) before trusting any tutorial, including this chapter.
- **graphql.org Learn** (graphql.org/learn) — the foundation’s tutorial track; the clearest prose introduction to schemas, queries, and mutations.
- **Relay GraphQL Cursor Connections Specification** (relay.dev/graphql/connections.htm) — the normative `edges/pageInfo` contract our connection implements.
- **Apollo Server documentation: Persisted Queries and APQ** (apollographql.com/docs) — the reference description of the hash-first handshake and its caching motivations.
- **Marc-André Giroux, *Production Ready GraphQL*** (2020) — the practitioner’s treatment of cost limiting, security, and schema design at scale; the closest book-length relative of this chapter.
- **Shopify’s GraphQL API design documentation** (shopify.dev) — a large public graph’s design conventions; notable for connection discipline and cost-based rate limiting in production.
- **Kleppmann, *Designing Data-Intensive Applications*** [Kle17] — the storage-side fundamentals that make the $N+1$ discussion precise.
- **RFC 9110, HTTP Semantics** [Fie22a] — status-code and method

semantics for the GraphQL-over-HTTP mapping, and the caching machinery GraphQL opts out of by default.

- **OpenAPI Specification v3.1** [Ope21] — the contract document GraphQL renders unnecessary, useful for contrast in the REST-versus-GraphQL decision.
- **FastAPI and httpx documentation** [Ram24; Enc24] — the server host and client transport used in this chapter’s listings.

13.9 Capstone Projects

Five projects, progressive. Each names its difficulty tier; acceptance criteria are checkable facts, not vibes. All run offline against the Northwind Robotics lab.

13.9.1 Capstone 1: Catalog Schema from Scratch [junior]

Objective. Re-implement the catalog schema without copying Listing 13.2: types, enum, input, union mutation result, and a cursor connection, over your own seeded repository.

Inputs/Datasets. Synthetic catalog of your own design (10+ products, 3+ categories, reviews), deterministic seed.

Steps.

1. Define `Product`, `Category`, `Review`, and the `ProductStatus` enum.
2. Implement `products(first, after)` as a Relay connection with opaque decodable cursors and a `first` cap.
3. Implement `addReview` returning a union of success and typed error payloads.
4. Write in-process tests for the happy paths and one typed error.

Acceptance Criteria.

- ☐ A full cursor walk returns every product exactly once.
- ☐ A forged cursor yields a field error with a machine-readable `extensions.code`.
- ☐ Out-of-range rating returns the typed error payload with `errors` absent.

- ☐ The generated SDL (printed via the schema object) contains the union and the enum.

Stretch Goals. Add `@deprecated` to one field and verify it appears in the introspected SDL; add backward pagination (`last/before`).

13.9.2 Capstone 2: N+1 Hunter [mid]

Objective. Given a deliberately naive GraphQL service (naive resolvers, no loaders), find and eliminate every $N+1$ with measured proof.

Inputs/Datasets. Your Capstone-1 schema extended with `Category.products` and `Review.author`; an instrumented repository with a query counter.

Steps.

1. Write three operations of increasing fan-out and record naive store-query counts.
2. Introduce per-request `DataLoaders` for each relationship.
3. Assert batched counts in tests — the numbers are the deliverable.
4. Demonstrate the per-request cache rule: a module-level loader serves stale data after a mutation; write the failing test, then fix by request-scoping.

Acceptance Criteria.

- ☐ Each operation's batched count is independent of page size (verified at two page sizes).
- ☐ A test proves duplicate keys are deduped into one batch read.
- ☐ The stale-cache regression test fails with a shared loader and passes with per-request loaders.

Stretch Goals. Add per-tick batch-size metrics and emit them in a response `extensions` block.

13.9.3 Capstone 3: Cost-Control Gateway [mid]

Objective. Build the static analysis gate of Listing 13.4 as a standalone, schema-agnostic library with adversarial tests.

Inputs/Datasets. A corpus of operations: legitimate pages, deep cyclic traversals, alias-multiplied god-queries, fragment-heavy documents.

Steps.

1. Implement depth and argument-weighted complexity over the parsed AST, expanding fragment spreads with cycle detection.
2. Price aliases honestly: **a: products** and **b: products** both pay.
3. Add a per-operation timeout at execution and prove it fires with a deliberately slow resolver.
4. Emit rejection reasons as structured error extensions.

Acceptance Criteria.

- ☐ Each corpus operation is accepted or rejected exactly as labeled.
- ☐ Complexity is monotonically increasing in **first** (property-tested across sizes).
- ☐ Fragment cycles are rejected with a distinct code, never a hang or crash.

Stretch Goals. Per-field cost weights driven by measured resolver latency; complexity reported to the client in response extensions.

13.9.4 Capstone 4: APQ Production Gateway [senior]

Objective. Ship the lab as an operations-locked gateway: full APQ, a strict allow-list mode, introspection policy, and a client that survives every failure channel.

Inputs/Datasets. The chapter lab; a build-time operation registry exported from a client repository (JSON hash → document).

Steps.

1. Implement APQ both directions, including `PersistedQueryMismatch` defense.
2. Add strict mode: only registered hashes execute, no query text accepted.
3. Key rate limiting on summed complexity per principal per minute; reject array batches over a cap.
4. Write the client: APQ with single retry, partial-data handling, and typed-error branching.

Acceptance Criteria.

- ☐ Strict mode rejects an unregistered document with a distinct code and zero resolver executions (proven by the query counter).

- ☐ The client completes the miss → register → hash-only sequence and sends text only once.
- ☐ Complexity-based limiting admits small queries while rejecting a burst of god-queries under the same request count.

Stretch Goals. Serve registered read operations over GET for CDN cacheability; document the cache-key design.

13.9.5 Capstone 5: Federated Robotics Graph [staff]

Objective. Split the catalog into two subgraphs — catalog (products, categories) and reputation (reviews) — with **Product** as a shared entity, and compose them behind a gateway you implement in miniature, offline.

Inputs/Datasets. Two Strawberry schemas; a hand-rolled gateway that parses operations, plans them across subgraphs, and stitches results; the seeded datasets of the earlier capstones.

Steps.

1. Define the entity boundary: catalog owns **Product**, reputation extends it with **reviews**.
2. Implement gateway query planning for the subset of the language your test operations need; say explicitly what is out of scope.
3. Reproduce the *cross-service* N+1 (entity resolution per product) and batch it.
4. Write a composition check that fails CI when subgraphs disagree on the entity key.

Acceptance Criteria.

- ☐ One client operation spanning both subgraphs returns correctly stitched data.
- ☐ Cross-service entity resolution is batched; counts asserted in tests.
- ☐ A subgraph outage degrades only its fields (partial data, typed extensions), proven by a fault-injection test.
- ☐ A written scope note lists the federation features deliberately not implemented.

Stretch Goals. Add a subscription to the reputation subgraph and carry it through the gateway; add per-subgraph cost budgets at the gateway.

13.10 Python Dependencies Appendix

13.10.1 Pinned Requirements

```

1 # requirements.txt -- Chapter 12 GraphQL lab
2 strawberry-graphql[fastapi]>=0.230
3 fastapi>=0.110
4 httpx>=0.27
5 pytest>=8

```

Install on Windows (PowerShell):

```

1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
4 pytest -q

```

13.10.2 strawberry-graphql[fastapi]

Description. A code-first GraphQL library: Python dataclasses and type hints define the schema; resolvers are plain (a)sync functions; ships a DataLoader and framework integrations, FastAPI among them (the [fastapi] extra).

Why included. Code-first keeps the chapter’s listings teaching the *type system* rather than an SDL mapping layer; it is actively maintained on modern `graphql-core`; its DataLoader and context mechanics map directly onto the specification’s execution model.

Alternatives. **Ariadne** — SDL-first: you write the schema in SDL and bind resolvers to it; choose it when the SDL must be the reviewed artifact (multi-language organizations, contract-first governance). **Graphene** — the incumbent, similar code-first style; broader legacy adoption, slower release cadence in recent years. **GraphQL-core** alone — the reference port; what Strawberry and Graphene build on; too low-level to teach from directly.

Installation (PowerShell).

```

1 pip install "strawberry-graphql[fastapi]>=0.230"

```

13.10.3 fastapi

Description. The ASGI framework used to host the GraphQL endpoint and health check [Ram24].

Why included. Consistency with Chapter 6, async-native request handling, and a testable app object that `httpx.ASGITransport` can execute in-process — the offline backbone of every demo and test in this chapter.

Alternatives. Starlette directly (FastAPI’s base; fewer conventions), or Strawberry’s Django/Flask integrations where those frameworks already own the service.

Installation (PowerShell).

```
1 pip install "fastapi>=0.110"
```

13.10.4 httpx

Description. A modern sync/async HTTP client with transport injection [\[Enc24\]](#).

Why included. `httpx.ASGITransport` speaks real HTTP to the in-process FastAPI app with zero sockets — offline-first without mocking semantics. The async client also carries the APQ handshake cleanly.

Alternatives. `requests` (sync only, no ASGI transport), `aiohttp` (async, different transport model), or a dedicated GraphQL client such as `gql` (adds subscriptions support at the price of another abstraction layer).

Installation (PowerShell).

```
1 pip install "httpx>=0.27"
```

13.10.5 pytest

Description. The test runner used across this book.

Why included. The suite’s 21 tests run in-process and offline; `asyncio.run` inside each test avoids any plugin dependency, so `pytest` alone suffices.

Alternatives. `pytest-asyncio` (idiomatic async tests at the price of a plugin), `unittest` (stdlib, more ceremony).

Installation (PowerShell).

```
1 pip install "pytest>=8"
```

13.11 Chapter Summary & Key Takeaways

- GraphQL inverts control of response shape: the client composes, the server guarantees against a typed, introspectable schema. The same inversion makes the client choose the work — cost governance is not optional.
- Nullability is error-containment policy expressed as syntax. An error at a non-null field bubbles to the nearest nullable position; the response is HTTP 200 with partial `data` and a populated `errors` array.

- Field-level resolution makes $N+1$ the default physics. DataLoader collapses it — one loader per request, batched keyed reads, results aligned positionally. Our measured collapse: 11 store reads to 3 at page size 5.
- Relay connections standardize cursor pagination: `edges/node/pageInfo`, opaque cursors, fetch- $p+1$ for `hasNextPage`, capped page sizes.
- Two error channels, chosen deliberately: typed union payloads for expected failure, top-level errors with `extensions.code` for exceptional failure.
- The guardrail stack, in firing order: depth limit, complexity budget, persisted-query allow-list (APQ for ergonomics and cacheable GETs), resolver timeouts, cost-keyed rate limiting, capped array batching, gated introspection, GraphQL off in production.
- Variables make query-document injection impossible; resolver-side injection is unchanged from every other stack — parameterized queries, always.
- REST keeps the crown for CDN-cacheable public reads, uploads, and unknown-consumer APIs; GraphQL wins for many-clients-one-backend product graphs; gRPC for internal low-latency RPC. Choose on constraints, not fashion.

13.12 Quick-Reference Card

13.12.1 Type System at a Glance

Kind	Remember
Scalar	Leaves: <code>Int</code> <code>Float</code> <code>String</code> <code>Boolean</code> <code>ID</code> (+ custom)
Object	Named field set; recursion makes the graph
Interface	Shared field contract; inline fragments for the rest
Union	No shared contract; <i>all</i> selection via fragments
Enum	Closed value set, validated pre-execution
Input object	Argument-side only; no unions/interfaces inside
List / Non-null	<code>[T]</code> , <code>T!</code> : error-containment policy
Meta fields	<code>__schema</code> , <code>__type</code> , <code>__typename</code>

Table 13.3: GraphQL type system cheat sheet.

13.12.2 Wire Shapes

```
1 // request
2 { "query": "query P($first: Int!) { products(first:
   $first) { ... } }",
3   "variables": { "first": 10 },
4   "operationName": "P",
5   "extensions": { "persistedQuery": { "version": 1, "
      sha256Hash": "..." } } }
6
7 // response: success, partial success, and pre-
   execution rejection
8 { "data": { "products": { ... } } }
9 { "data": { "product": null },
10   "errors": [ { "message": "...", "path": ["product", "
      marginPercent"],
11                 "extensions": { "code": "
      COST_DATA_UNAVAILABLE" } } ] }
12 // HTTP 400 + errors array: parse / validation / budget
   rejection
```

13.12.3 Cost-Control Checklist

- ☐ Depth limit enforced on the parsed AST, pre-execution.
- ☐ Complexity budget with page-size multipliers; aliases and batch elements counted.
- ☐ Page sizes capped at the resolver (**first** $\leq$ 50 here).
- ☐ Persisted-query allow-list for first-party clients; APQ for the rest.
- ☐ Array batching disabled or length-capped; cost summed across the batch.
- ☐ Rate limiting keyed on complexity units, not request count.
- ☐ Per-resolver and per-operation timeouts armed.
- ☐ Introspection gated (AST check), GraphiQL off in production.
- ☐ DataLoaders constructed per request, never shared.
- ☐ Field-level authorization deny-by-default, audited in CI.

13.13 Standardized Evidence Addendum

13.13.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 13.4: Minimum benchmark table to keep per chapter.

13.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

13.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 13.5: Operational budget template for this chapter’s technique.

13.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0

```

```
6 fallback_strategy: "fail_fast"  
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

13.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

13.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

13.17 Cross-Chapter Decision Card

13.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 13.6: Decision card to complete per chapter.

- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

13.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 14

gRPC & High-Performance RPC

Every API style you have studied so far — REST over JSON, GraphQL, event-driven messaging — ultimately pays the same two taxes on every call: the cost of turning an in-memory request into bytes, and the cost of moving those bytes across the network. On public, browser-facing, low-frequency paths those taxes are noise. On *hot internal paths* — service-to-service calls inside a data center, fan-out queries that touch fifty microservices per user request, telemetry streams emitting thousands of frames per second — they dominate. gRPC exists to minimize both taxes simultaneously: it pairs a compact binary serialization format (Protocol Buffers, Chapter 4) with a multiplexed binary transport (HTTP/2, Chapter 1) and generates the client and server plumbing from a single language-neutral interface definition [gRP24; Goo24].

This chapter treats gRPC with the same wire-level rigor we applied to HTTP in Chapter 1. You will learn the RPC conceptual model — stubs, skeletons, marshalling, interface definition — and precisely why REST+JSON loses on hot internal paths: payload size, text parsing cost, weak contracts, and the absence of first-class streaming. You will study gRPC’s exact message framing: every message on an HTTP/2 stream is prefixed with five bytes — a one-byte compression flag and a four-byte big-endian length — followed by the protobuf payload, with the method identified by `:path` of the form `/package.Service/Method` and the outcome reported in the `grpc-status` trailer [Tho22; gRP24]. You will master Protocol Buffers as an IDL, all four RPC modes (unary, server streaming, client streaming, bidirectional streaming), deadline propagation across hops, metadata and interceptors, the seventeen canonical status codes with rich error details, and the operational realities that textbooks love to skip: load balancing long-lived HTTP/2 connections, connection aging, keepalive, server reflection, browser exposure via `grpc-web` and JSON

transcoding, and the health-checking protocol.

All code is offline-first and deterministic: a synthetic *Northwind Robotics* telemetry service exercised entirely on localhost and in-process, with a pytest suite covering deadline exceeded, cancellation, and stream error paths.

Key Concept

gRPC is not “REST but faster.” It is a different *contract discipline*: a single `.proto` file is the source of truth from which typed clients and servers are generated in a dozen languages, the wire format is self-describing by tag number rather than by field name, and the transport multiplexes many concurrent streams over one TCP connection. You adopt gRPC to buy contract safety, streaming semantics, and wire efficiency on internal paths — and you pay for it with browser hostility, opaque-on-the-wire debugging, and load balancing subtleties that REST never had.

14.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Explain the RPC conceptual model — client stub, server skeleton, marshalling, interface definition language — and articulate precisely why REST+JSON is the wrong tool for hot internal paths (payload size, parse cost, codegen, streaming) [Kle17; New21].
- LO-2.** Decode gRPC’s wire format by hand: the 5-byte message prefix (1-byte compressed flag + 4-byte big-endian length), the `/package.Service/Method :path`, the `application/grpc` content type, trailers-only responses, and the `grpc-status` / `grpc-message` trailers [Tho22; gRP24].
- LO-3.** Author Protocol Buffers IDL with messages, services, field rules, well-known types, `oneof`, and `map`; drive the `protoc` / `grpcio-tools` codegen pipeline; and apply the schema-evolution rules (tag-number discipline, `reserved`) from Chapter 4 [Goo24].
- LO-4.** Select and implement the correct RPC mode — unary, server streaming, client streaming, bidirectional streaming — including the cancellation semantics of each.

- LO-5.** Engineer deadlines correctly: propagate remaining budget across hops via the `grpc-timeout` header, distinguish per-attempt from overall deadlines, and defend servers with `context.time_remaining()` checks.
- LO-6.** Use metadata (headers, trailers, binary `-bin` keys) and client/server interceptors for auth, logging, metrics, and retry policy.
- LO-7.** Apply the seventeen canonical gRPC status codes with sound judgment, attach rich error details (`google.rpc.Status`-style payloads in binary trailers), and map codes to HTTP statuses when transcoding.
- LO-8.** Operate gRPC in production: diagnose the L4 load-balancer hotspot, configure client-side and lookaside balancing, tune connection age and keepalive, use server reflection with `grpcurl`, expose services to browsers via `grpc-web` or JSON transcoding, and implement the standard health-checking protocol.

Prerequisites

This chapter assumes you have completed:

- **Chapters 0–2** — Python 3.11+ environment, virtual environments, `pip`/`pytest` from PowerShell, and consuming HTTP APIs.
- **Chapter 1** — the HTTP wire protocol, *especially* the HTTP/2 material: streams, frames, HPACK-compressed header blocks, `:path` and other pseudo-headers, flow control, and `RST_STREAM`. We reuse that vocabulary without re-deriving it.
- **Chapter 3** — REST’s constraints, as the point of contrast.
- **Chapter 4** — serialization and data contracts: the Protocol Buffers wire format (varints, wire types, tag computation), and the schema-evolution rules. This chapter assumes them and adds the IDL and service layer on top.

No network access is required anywhere: every listing binds to `localhost` or runs fully in-process, and all data is synthetic, drawn from the fictional Northwind Robotics warehouse-fleet universe.

Warning

gRPC's performance reputation is real but conditional. The wins come from binary framing, HPACK header compression, connection reuse, and codegen — all of which you can squander with per-call channels, unbounded streams, missing deadlines, or a naive L4 load balancer. Read Section 14.4 before you benchmark.

14.2 Deep Technical Mechanics & Architectural Concepts

14.2.1 The RPC Conceptual Model: Stubs, Skeletons, Marshalling

Definition 14.1 (Remote Procedure Call). A *remote procedure call* is an invocation that *looks* like a local function call at the call site but whose execution crosses a process or machine boundary. The illusion is constructed from four pieces: an **interface definition** that names the procedures and their parameter/return types; a **client stub** that exposes those signatures locally and converts arguments to bytes (*marshalling*); a **server skeleton** (or *servicer*) that converts bytes back to arguments and invokes the real implementation; and a **transport** that moves the bytes.

The model dates to the early 1980s and has been reincarnated as ONC RPC, DCE/RPC, CORBA, Java RMI, XML-RPC, SOAP/WSDL, Apache Thrift, and now gRPC. Every incarnation converges on the same architecture because the problem is invariant: two processes need a *shared, machine-checkable contract* and a *mechanical translation* between the contract and each language's native types. Where the generations differ is in the quality of the contract (IDL expressiveness and evolution rules), the efficiency of the encoding, and the semantics of the transport. gRPC's particular bet is: proto3 as the IDL, protobuf binary as the encoding, HTTP/2 as the transport, and code generation as the only supported consumption style [gRP24; Goo24].

Two consequences of the stub model deserve emphasis. First, **the contract precedes the implementation**: you write `telemetry.proto` before you write Python, and both sides generate from it, so client and server cannot silently disagree about field types. Second, **the call is not local**: the stub hides the network but not its physics — latency, partial failure, deadlines, and cancellation are first-class in the generated API precisely because pretending they do not exist is the classic RPC fallacy [Kle17; New21].

14.2.2 Why REST+JSON Loses on Hot Internal Paths

REST over JSON is the right default for public APIs — Chapter 3 argued this at length — but on hot internal paths it concedes four compounding disadvantages:

1. **Payload size.** JSON repeats every field name in every message, encodes numbers as decimal text, and base64-inflates binary data. A protobuf `TelemetryFrame` carries field tags as single varint bytes (Chapter 4) and is routinely 3–10× smaller than the equivalent JSON, which translates directly into bandwidth and serialization CPU.
2. **Parse cost.** JSON parsing is text scanning; protobuf decoding is varint reads and memory copies. At tens of thousands of messages per second per core, the difference is measurable in tail latency.
3. **No enforced contract.** OpenAPI describes REST contracts, but nothing forces both sides to generate from it; drift is routine. With gRPC the `.proto` is the API: both sides are generated artifacts of one file, so drift is a compile error, not an incident.
4. **No first-class streaming.** REST has one request and one response; streaming is bolted on (SSE, chunked bodies, WebSockets) with per-framework semantics. gRPC’s four modes make streaming part of the contract, with uniform cancellation, flow control, and error semantics.

The honest counterpoint — developed in Section 14.4 and the interview vault — is that gRPC’s advantages are *internal-path* advantages. Browsers cannot speak raw gRPC, humans cannot read protobuf on the wire, and the ecosystem of caches, gateways, and tooling built for HTTP/1.1+JSON mostly does not apply. Choose per path, not per organization.

14.2.3 HTTP/2 as the Transport

Chapter 1 covered HTTP/2’s mechanics: one TCP connection carries many concurrent *streams*; each stream is a sequence of *frames* (`HEADERS`, `DATA`, `RST_STREAM`, `WINDOW_UPDATE`, ...); header blocks are HPACK-compressed; flow control is per-stream and per-connection via credit windows [Tho22]. gRPC maps onto this cleanly:

$$\text{one RPC} \longleftrightarrow \text{one HTTP/2 stream.}$$

That single sentence explains most of gRPC’s performance profile. Many RPCs share one TCP connection (no head-of-line blocking between calls, no connection-pool gymnastics), streams are cheap so unary calls and long-lived bidi streams coexist, and flow control plus `RST_STREAM` give every RPC independent backpressure and cancellation.

gRPC Message Framing: The 5-Byte Prefix

HTTP/2 DATA frames are arbitrary byte containers; gRPC imposes its own message boundary inside them. Each protobuf message — request or response — is serialized on the wire as a **Length-Prefixed Message**:

```

1  byte 0          : Compressed-Flag    (0 = uncompressed, 1
   = compressed)
2  bytes 1..4      : Message-Length    (uint32, BIG-ENDIAN,
   network byte order)
3  bytes 5..N      : Message           (serialized protobuf
   , exactly Length bytes)

```

For example, a request message that serializes to the 3 protobuf bytes 0A 07 ... of length 0x0000000B would appear on the wire as:

```

1  00 00 00 00 0B 0A 07 ...  <- flag=0, length=11, then
   11 protobuf bytes

```

Three properties follow. First, framing is *length-delimited*, so a single HTTP/2 stream can carry any number of messages in order — this is what makes the three streaming modes possible with zero additional machinery. Second, the compression flag is per-message, so a server may compress opportunistically. Third, a message may span many HTTP/2 DATA frames, or many messages may share one frame: the 5-byte prefix, not the frame boundary, is the unit of meaning.

Request and Response Anatomy on the Wire

A gRPC call is an HTTP/2 *POST* with specific pseudo-headers and a specific life cycle [gRP24; Tho22]:

```

1  HEADERS (stream 7, flags: END_HEADERS)
2    :method      = POST
3    :scheme      = https
4    :path        = /northwind.telemetry.v1.
   TelemetryService/GetRobotStatus
5    :authority   = telemetry.northwind.example
6    content-type = application/grpc
7    grpc-timeout = 100m
8    authorization = Bearer ...
9    x-correlation-id = corr-000001
10 DATA      (stream 7, flags: END_STREAM)  <- 5-byte
   prefix + protobuf request

```

The `:path` is exactly `/package.Service/Method` — the fully qualified proto name. `content-type` begins with `application/grpc` (commonly

application/grpc+proto; +json exists for exotic deployments). The response:

```

1  HEADERS (stream 7)                                <- "initial metadata"
2      :status = 200
3      content-type = application/grpc
4  DATA    (stream 7)                                <- 5-byte prefix +
      protobuf response(s)
5  HEADERS (stream 7, flags: END_STREAM)              <- "trailers"
6      grpc-status = 0
7      grpc-message = ...
8      error-details-bin = <base64>

```

Note the asymmetry that surprises every newcomer: **the HTTP status is always 200 OK** (barring transport-level failures), and the *real* outcome of the RPC arrives in the **grpc-status trailer**. It must be a trailer because streaming responses may have already emitted dozens of DATA frames before the failure that determines the final status. When a call fails before any response message is produced, servers may collapse the whole response into a single HEADERS frame carrying `:status 200`, `grpc-status`, and `END_STREAM` — the **trailers-only response**, which is what most error responses look like on the wire. Figure 14.1 shows the complete anatomy.

14.2.4 Protocol Buffers as the Interface Definition Language

Chapter 4 covered the protobuf *wire format*; here we cover the protobuf *language* as the IDL from which gRPC services are built [Goo24]. The elements you will use daily:

- **Messages and field rules.** In proto3 every field is optional in the serialization sense — absent fields decode to type defaults (0, "", false, empty) — and **repeated** marks a list. proto3 deliberately removed **required**: a required field is a schema-evolution trap, because no consumer can ever stop populating it.
- **Tag numbers.** The integer after = is the field's identity on the wire (the name is for humans and codegen). Numbers 1–15 cost one byte of tag; 16–2047 cost two. *Never reuse a retired tag number* — a stale peer will silently reinterpret old bytes as the new field. Retire with **reserved 9, 10**; and **reserved "heading_deg"**; so the compiler enforces the ban (Chapter 4's evolution rules, now load-bearing).
- **Well-known types.** `google.protobuf.Timestamp`, `Duration`, `Struct`, `Any`, and wrappers like `DoubleValue` ship with `protoc`. Prefer `Timestamp`

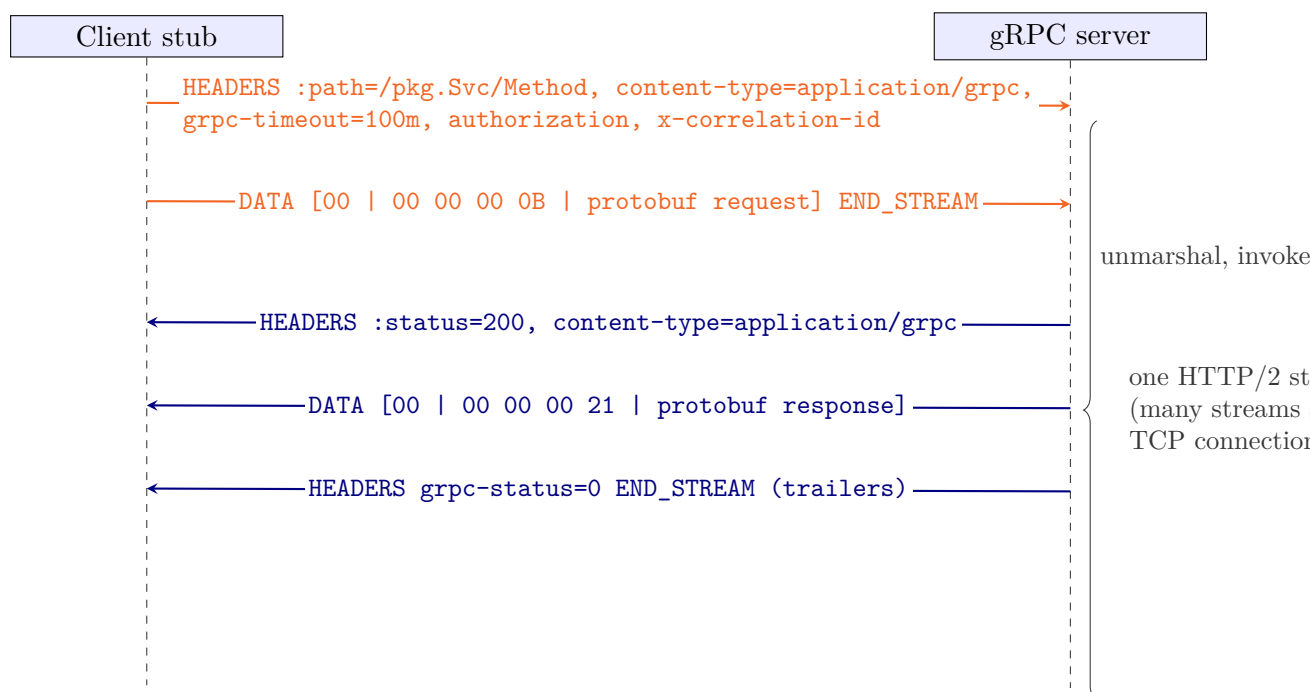


Figure 14.1: Anatomy of a unary gRPC call over one HTTP/2 stream. Client frames in orange, server frames in blue. Every protobuf message is preceded by the 5-byte prefix (compression flag + big-endian length); the definitive outcome arrives in the `grpc-status` trailer.

over “epoch seconds in an int64 named `time`” — it is self-describing and range-checked.

- **oneof.** At most one branch is set at a time; setting a branch clears the others, and presence is preserved on the wire (`WhichOneof("action")` in Python). Use it for genuine alternatives — our `location` and `action` fields — not as a general union type.
- **map<K,V>.** Sugar for a repeated entry message; keys may be integral or string types. Map ordering is *not* guaranteed on the wire — never key application logic on it.
- **Services.** A `service` block names RPCs and their cardinality: `rpc M(Req) returns (Resp)` for unary, with `stream` on either side for the streaming modes. This is the only part of the file protoc’s gRPC plugin consumes.

The codegen pipeline is mechanical and hermetic:

```
1 python -m grpc_tools.protoc -I. --python_out=. --
   grpc_python_out=. telemetry.proto
```

`grpc_tools.protoc` bundles the protoc compiler and the well-known proto files (so the `google/protobuf/timestamp.proto` import resolves offline). It emits two modules: `telemetry_pb2.py` (message classes: builders, accessors, `SerializeToString/ParseFromString`, `WhichOneof`, map containers) and `telemetry_pb2_grpc.py` (the `TelemetryServiceStub` client class, the `TelemetryServiceServicer` base class you subclass, and the `add_TelemetryServiceServicer_to_server` registration function). **Generated files are build artifacts:** regenerate, never hand-edit, and regenerate in CI so stubs can never drift from the `.proto`.

Example

Schema evolution, one paragraph: you may add fields with *fresh* tag numbers (old peers ignore unknown tags; new peers see defaults), remove fields only by reserving their tags and names first, widen `int32` to `int64` (same wire type), and add `oneof` branches. You may never reuse tag numbers, change a field’s wire type, or move an existing field into a `oneof`. Our `TelemetryFrame` reserves tags 9–10 and the names `heading_deg/speed_mps`, retired in v1.1 — the reservation makes a v1.2 reintroduction a *compile error* instead of a production data corruption (Chapter 4, Section on tag discipline) [Goo24].

14.2.5 The Four RPC Modes

Cardinality is declared in the IDL and fixed at compile time; each mode has a canonical use case and its own cancellation semantics (Figure 14.2):

1. **Unary** — one request, one response. The default for command/query shapes (`GetRobotStatus`). Cancellation: client abort sends `RST_STREAM`; server observes `context.is_active() == False`.
2. **Server streaming** — one request, many responses (`StreamTelemetry`: “subscribe to this robot’s frames”). Canonical for change feeds, tailing, and large result sets consumed incrementally. Cancellation matters most here: an unbounded stream the client abandoned is a server-side resource leak unless the handler checks `is_active()` between yields.
3. **Client streaming** — many requests, one response (`UploadTelemetry`: bulk ingestion). The server reads until end-of-stream, then computes one summary. Errors discovered mid-stream should abort with status and details, not be silently tallied.
4. **Bidirectional streaming** — two independent message sequences on one stream (`ControlLoop`: commands down, acknowledgements up). The sequences are *not* request/response pairs unless your protocol makes them so; ordering is guaranteed within each direction only. This is the mode for control planes, chat-like protocols, and interactive sessions.

14.2.6 Deadlines, Cancellation, and Metadata

Deadline mechanics. When a Python client passes `timeout=0.1`, the gRPC core computes an absolute deadline (*now* + 100 ms) and transmits the *remaining* budget in the `grpc-timeout` header as an integer with a unit suffix — H hours, M minutes, S seconds, m milliseconds, u microseconds, n nanoseconds, at most eight digits (e.g. `grpc-timeout: 99950u` after 50 μ s of transit). The server’s `context.time_remaining()` is derived from that header minus observed transit time — so **the budget propagates**: when your handler fans out to a downstream gRPC service, forwarding `time_remaining()` as the downstream timeout means the whole call tree shares one end-to-end budget instead of each hop inventing its own. This is the single most important resilience behavior in the chapter; the lab exercises it.

Per-attempt vs. overall. Retries (Section 14.3.4) spend *attempts* from the *same* overall deadline; the service config’s `timeout` field caps each RPC, while per-attempt budgets are bounded by backoff configuration. A call

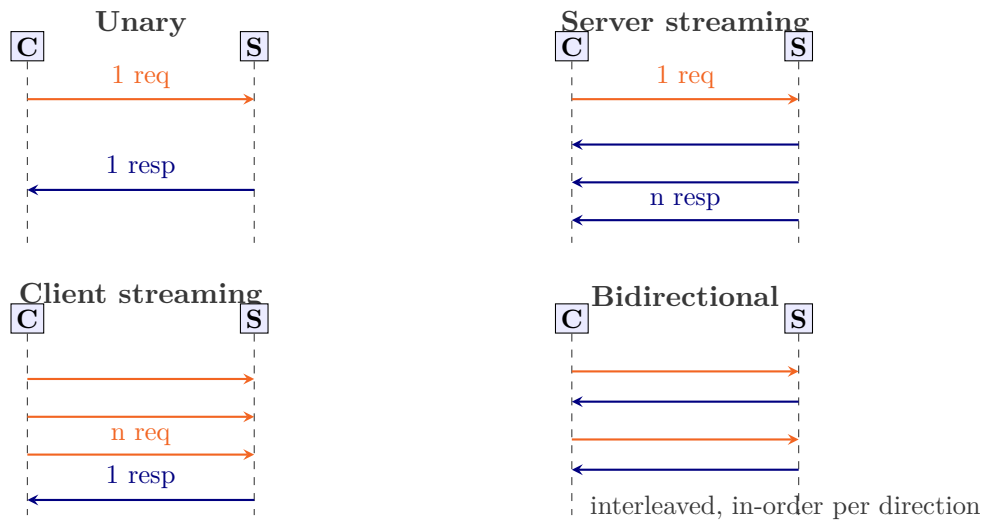


Figure 14.2: Message timelines for the four RPC modes. Client-to-server messages in orange, server-to-client in blue. Every arrow is a length-prefixed message on the same HTTP/2 stream; the stream closes with the `grpc-status` trailer.

that needs $3 \text{ attempts} \times 300 \text{ ms}$ must be given an overall deadline well above 900 ms, or the retries are decoration.

Cancellation. Client cancellation emits `RST_STREAM` (Chapter 1); the server must poll `context.is_active()` in loops and generators and stop producing. Cancellation is advisory and cooperative — a server that never checks will happily compute into the void.

Metadata. Request headers and response trailers are both *metadata*: lowercase ASCII keys, string values — *except* keys suffixed `-bin`, which carry raw binary (base64-encoded on the HTTP/2 wire, exposed as bytes in the API). Auth rides in metadata — `authorization: Bearer ...` on every call, attached by an interceptor, never hand-threaded through business logic — and rich error details ride in `-bin` trailers (Section 14.2.8). Metadata keys beginning with `grpc-` are reserved for the framework.

14.2.7 Interceptors: The Cross-Cutting Layer

Interceptors are gRPC’s middleware: functions that wrap every call on a channel (client side) or every handler on a server, for concerns that must be uniform — authentication, correlation IDs, logging, metrics, retry hedging, error mapping. They come in client/server $\times$ unary/stream varieties; Python models the client ones as the four abstract classes `grpc.UnaryUnaryClientInterceptor` ... `grpc.StreamStreamClientInterceptor`, composed with `grpc.intercept_channel`, and the server side as `grpc.ServerInterceptor.intercept_service`, which

receives the not-yet-bound handler and returns a (wrapped) handler. Figure 14.3 shows the chain; Section 14.3 implements both ends.

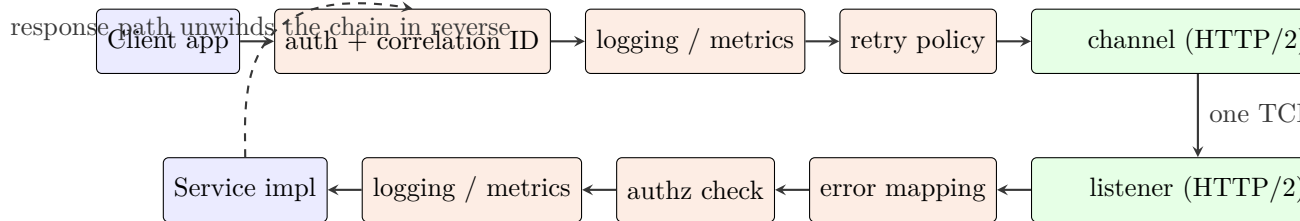


Figure 14.3: Interceptor chain. Client interceptors wrap outbound calls (attach metadata, observe latency, drive retries); server interceptors wrap inbound handlers (enforce authz, map domain errors to status codes, emit metrics).

14.2.8 The Error Model

gRPC replaces HTTP’s flat status taxonomy with seventeen canonical codes carried in the `grpc-status` trailer [gRP24]. The full set, with usage guidance:

Three judgment calls trip up even senior engineers. **INVALID_ARGUMENT vs. FAILED_PRECONDITION vs. OUT_OF_RANGE**: the first means the request is malformed no matter the system state; the second means the request is fine but the system’s state forbids it; the third is for values outside a defined range (e.g. seeking past the end of a stream). **UNAUTHENTICATED vs. PERMISSION_DENIED**: the first means “we don’t know who you are” (credentials missing/invalid — re-auth may fix it); the second means “we know who you are and the answer is no” (re-auth will not help). **NOT_FOUND vs. PERMISSION_DENIED**: when revealing existence would leak information, return `PERMISSION_DENIED` (or even `NOT_FOUND`) uniformly — but pick one policy per resource class and document it.

Rich error details. A status code plus a string is not an error *model*. Google APIs standardize a richer payload: a `google.rpc.Status` message — code, message, and repeated `google.protobuf.Any` details carrying typed payloads such as `BadRequest` (per-field violations), `QuotaFailure`, `RetryInfo`, and `ErrorInfo` (stable machine-readable reason + domain) — serialized into the binary trailer `grpc-status-details-bin`. Our listing implements the same pattern with a first-class `ErrorDetails` message in the chapter proto and an `error-details-bin` trailer, so it runs with only `grpcio/protobuf` installed; swapping in `googleapis-common-protos` and the standard key is a drop-in upgrade.

#	Code	Use when	HTTP map
0	OK	Success (not an error).	200
1	CANCELLED	Caller cancelled (or deadline-adjacent abort).	499
2	UNKNOWN	Error from another namespace / unclassified.	500
3	INVALID_ARGUMENT	Client sent malformed input; retry unchanged will fail.	400
4	DEADLINE_EXCEEDED	Budget expired (note: server-set, vs. CANCELLED for client abort).	504
5	NOT_FOUND	Named resource does not exist.	404
6	ALREADY_EXISTS	Create collided with existing resource.	409
7	PERMISSION_DENIED	Authenticated, but not allowed.	403
8	RESOURCE_EXHAUSTED	Quota / rate limit / disk full.	429
9	FAILED_PRECONDITION	System state forbids the op (e.g. non-empty dir delete).	400
10	ABORTED	Concurrency conflict; safe to retry whole op (transactions).	409
11	OUT_OF_RANGE	Value outside valid range (seek past end).	400
12	UNIMPLEMENTED	Method unknown to this server.	501
13	INTERNAL	Server bug / invariant violation.	500
14	UNAVAILABLE	Transient: server down, overloaded, restarting. <i>Retryable</i> .	503
15	DATA_LOSS	Unrecoverable data corruption.	500
16	UNAUTHENTICATED	Missing/invalid credentials.	401

Table 14.1: The 17 canonical gRPC status codes and their conventional HTTP mapping when transcoding to JSON APIs (the mapping used by Google’s `google/rpc/code.proto` conventions).

Transcoding. When a gateway translates gRPC to HTTP+JSON (Section 14.2.9), each status code maps to one HTTP status (Table 14.1); details typically serialize into the response body as a JSON `error` object. Design your codes and details so both audiences — native gRPC clients and transcoded REST clients — can act on them.

14.2.9 Operational gRPC: Load Balancing, Connections, Tooling

The L4 problem. gRPC multiplexes *many RPCs over one long-lived TCP connection*. A Layer-4 (TCP) load balancer balances *connections*, so once a client connects to a pod, every RPC it ever makes lands on that pod — autoscaling adds idle pods while one pod melts (the war story in Section 14.5 tells it with numbers). The standard remedies: **(a) client-side LB** — the client resolves all backends (DNS) and load-balances per-RPC with `round_robin` (or per-connection with the default `pick_first`); **(b) lookaside LB** — a control plane (the historical `grpc_lb` protocol) hands the client a curated server list, keeping data-plane traffic direct; **(c) xDS** — the CNCF/Envoy service-discovery API family (LDS/CDS/RDS/EDS) that modern meshes and gRPC’s “proxyless service mesh” mode use to push routing, LB policy, and config to clients; and **(d) L7 proxies** (Envoy, Linkerd) that terminate HTTP/2 and rebalance per-stream [gRP24; New21].

Connection management. Two server options prevent the pathologies of immortal connections: `grpc.max_connection_age_ms` forces periodic reconnects (clients transparently redial, landing on a rebalanced backend — set `max_connection_age_grace_ms` to drain in-flight RPCs), and `grpc.keepalive_time_ms` sends HTTP/2 PINGs so dead peers and idle connections through NATs/firewalls are detected. Client-side `grpc.keepalive_time_ms` must exceed the server’s `min_ping_interval` policy or the server will throttle you with GOAWAYS.

Server reflection and `grpcurl`. A server opting into the reflection service can enumerate its services and descriptors at runtime, which powers `grpcurl` — “curl for gRPC” — with no local `.proto` copy. In Python, enable it with the `grpcio-reflection` package; the lab shows the exact invocations.

Browsers: `grpc-web` and JSON transcoding. Browsers cannot open raw HTTP/2 streams with trailer access, so two bridges exist: **`grpc-web`** speaks a framed variant of the protocol through a proxy (Envoy has a native filter), giving web clients generated stubs and unary/server-streaming calls; **HTTP/JSON transcoding** annotates the proto (`google.api.http`) so a gateway (Envoy’s gRPC-JSON transcoder, gRPC-Gateway) exposes each RPC as a conventional REST endpoint — the standard way to keep one internal gRPC contract while publishing a public JSON API.

Health checking. The `grpc.health.v1.Health` service (`Check`, `Watch`) is the standard liveness/readiness contract: orchestrators and LBs query it per service name, and servers flip `SERVING`/`NOT_SERVING` during drains. Implement it from day one — a load balancer that cannot distinguish “up” from “serving” will route traffic into your deploys.

14.3 Production-Ready Code Implementation

The running example is the *Northwind Robotics Telemetry Service*: a fleet of warehouse robots emits telemetry frames, operators query status, and a control loop steers individual units. All six listings below were executed against Python 3.11+ with `grpcio`, `grpcio-tools`, `protobuf`, and `pytest` (Appendix, Section 14.10), entirely offline on `localhost`. Data is synthetic and deterministic: frames derive from a fixed seed and a fixed base epoch, never from the wall clock.

14.3.1 Listing 1 — Interface Definition: `telemetry.proto`

One file is the whole contract: messages with well-known types (`Timestamp`), a `map`, two `oneofs`, `reserved` tags for retired fields, and a service declaring all four RPC modes.

```
1 syntax = "proto3";
2
3 package northwind.telemetry.v1;
4
5 import "google/protobuf/timestamp.proto";
6
7 // Operating mode reported by a Northwind Robotics
8   warehouse robot.
9 enum RobotMode {
10     ROBOT_MODE_UNSPECIFIED = 0;
11     ROBOT_MODE_IDLE = 1;
12     ROBOT_MODE_MOVING = 2;
13     ROBOT_MODE_CHARGING = 3;
14     ROBOT_MODE_FAULT = 4;
15 }
16
17 message AislePosition {
18     string aisle_id = 1;
19     double offset_m = 2;
20 }
```

```
21 message GpsFix {
22     double latitude = 1;
23     double longitude = 2;
24     double hdop = 3;
25 }
26
27 // One telemetry sample emitted by a robot.
28 message TelemetryFrame {
29     string robot_id = 1;
30     google.protobuf.Timestamp observed_at = 2; // well-
        known type
31     double battery_pct = 3; // 0.0 ..
        100.0
32     double chassis_temp_celsius = 4;
33     map<string, double> joint_positions_rad = 5;
34     RobotMode mode = 6;
35
36     oneof location { //
        exactly one branch is set
37         AislePosition aisle = 7;
38         GpsFix gps = 8;
39     }
40
41     // Tag numbers and names retired in v1.1; never reuse
        them (see Chapter 4).
42     reserved 9, 10;
43     reserved "heading_deg", "speed_mps";
44 }
45
46 message RobotStatusRequest {
47     string robot_id = 1;
48 }
49
50 message RobotStatusReply {
51     string robot_id = 1;
52     RobotMode mode = 2;
53     double battery_pct = 3;
54     google.protobuf.Timestamp firmware_built_at = 4;
55     TelemetryFrame last_frame = 5;
56 }
57
58 message StreamTelemetryRequest {
59     string robot_id = 1;
60     uint32 max_frames = 2; // hard bound; the server
        never exceeds it
61     uint32 period_ms = 3; // pacing between frames
```

```

62 }
63
64 message UploadSummary {
65     uint32 accepted = 1;
66     uint32 rejected = 2;
67     double min_battery_pct = 3;
68     double max_battery_pct = 4;
69 }
70
71 message ControlCommand {
72     string robot_id = 1;
73     uint64 sequence = 2;
74
75     oneof action {
76         RobotMode set_mode = 3;
77         bool estop = 4;
78         double set_speed_mps = 5;
79     }
80 }
81
82 // Structured error payload, carried in the "error-
83 // details-bin" trailer.
84 // Mirrors the shape of google.rpc.Status details used
85 // by Google APIs.
86 message ErrorDetails {
87     string reason = 1;
88     string domain = 2; // e.g. "telemetry.northwind.
89     // example"
90     repeated FieldViolation violations = 3;
91     string correlation_id = 4;
92 }
93
94 message FieldViolation {
95     string field = 1;
96     string description = 2;
97 }
98
99 // The four RPC modes, one of each kind.
100 service TelemetryService {
101     rpc GetRobotStatus(RobotStatusRequest) returns (
102         RobotStatusReply); // unary
103     rpc StreamTelemetry(StreamTelemetryRequest) returns (
104         stream TelemetryFrame);
105     rpc UploadTelemetry(stream TelemetryFrame) returns (
106         UploadSummary);
107     rpc ControlLoop(stream ControlCommand) returns (

```

```

102         stream TelemetryFrame);
    }

```

Generate the stubs from PowerShell (the bundled `grpc_tools.protoc` resolves the well-known-type import offline):

```

1 python -m grpc_tools.protoc -I. --python_out=. --
    grpc_python_out=. telemetry.proto

```

This emits `telemetry_pb2.py` (messages) and `telemetry_pb2_grpc.py` (stub, servicer base, registration). The generated-stub usage pattern is always the same three moves: *subclass* the servicer and implement the RPCs (Listing 2); *register* the servicer on a `grpc.server`; on the client, *instantiate the stub* against a channel and call methods as if local (Listing 3).

14.3.2 Listing 2 — Server: All Four Modes with Deadline and Cancellation Discipline

Note the defensive habits: every handler checks `context.is_active()` in its loop, re-checks `context.time_remaining()` after any stall, and raises typed `DomainErrors` that Listing 4 maps to status codes with rich details. RB-SLOW simulates a slow dependency so the deadline test has something to bite.

```

1  """Northwind_Robotics_telemetry_service_--_gRPC_server_
    implementation.
2
3  Chapter_13_listing._Offline-first:_binds_to_localhost_
    only;_every_payload_is
4  synthetic_and_derived_from_a_fixed_seed,_never_from_the
    _wall_clock.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import random
10 import time
11 from concurrent import futures
12 from typing import Iterable, Iterator
13
14 import grpc
15
16 import telemetry_pb2
17 import telemetry_pb2_grpc
18
19 BASE_EPOCH_S = 1_700_000_000 # fixed synthetic epoch
    for deterministic payloads

```

```

20 KNOWN_ROBOTS = ("RB-1001", "RB-1002", "RB-1003", "RB-
    SLOW")
21 MIN_BUDGET_S = 0.005
22
23
24 class DomainError(Exception):
25     """Application error carrying its intended gRPC
        status mapping."""
26
27     status_code: grpc.StatusCode = grpc.StatusCode.
        INTERNAL
28     reason: str = "INTERNAL"
29
30     def __init__(
31         self,
32         message: str,
33         violations: list[telemetry_pb2.FieldViolation]
            | None = None,
34     ) -> None:
35         super().__init__(message)
36         self.message = message
37         self.violations = violations or []
38
39
40 class UnknownRobotError(DomainError):
41     status_code = grpc.StatusCode.NOT_FOUND
42     reason = "ROBOT_NOT_FOUND"
43
44
45 class FrameValidationError(DomainError):
46     status_code = grpc.StatusCode.INVALID_ARGUMENT
47     reason = "FRAME_VALIDATION_FAILED"
48
49
50 def _check_known(robot_id: str) -> None:
51     if robot_id not in KNOWN_ROBOTS:
52         raise UnknownRobotError(f"unknown robot id {
            robot_id!r}")
53
54
55 def _require_budget(
56     context: grpc.ServicerContext, min_s: float =
        MIN_BUDGET_S
57 ) -> None:
58     """Fail fast if the propagated deadline leaves no
        useful budget."""

```

```

59     remaining = context.time_remaining()
60     if remaining < min_s:
61         context.abort(
62             grpc.StatusCode.DEADLINE_EXCEEDED,
63             f"budget exhausted: {remaining*1000:.2f} ms remaining",
64         )
65
66
67 class TelemetryService(telemetry_pb2_grpc.
68     TelemetryServiceServicer):
69     """All four RPC modes with explicit deadline and
70     cancellation checks."""
71
72     def __init__(self, seed: int = 1337) -> None:
73         self._seed = seed
74
75     def _frame(
76         self,
77         robot_id: str,
78         seq: int,
79         mode: telemetry_pb2.RobotMode | None = None,
80     ) -> telemetry_pb2.TelemetryFrame:
81         # Per-frame seeded RNG keeps payloads
82         # deterministic and reproducible.
83         rng = random.Random(f"{self._seed}:{robot_id}:{seq}")
84         frame = telemetry_pb2.TelemetryFrame(
85             robot_id=robot_id,
86             battery_pct=round(95.0 - 0.01 * seq + rng.
87                 uniform(-0.25, 0.25), 3),
88             chassis_temp_celsius=round(32.0 + rng.
89                 uniform(-1.0, 1.0), 2),
90             mode=(
91                 telemetry_pb2.ROBOT_MODE_MOVING if mode
92                 is None else mode
93             ),
94             frame.observed_at.FromSeconds(BASE_EPOCH_S +
95                 seq * 5)
96             frame.joint_positions_rad["shoulder"] = round(
97                 rng.uniform(-1.0, 1.0), 4)
98             frame.joint_positions_rad["elbow"] = round(rng.
99                 uniform(0.0, 2.0), 4)
100             frame.aisle.CopyFrom(
101                 telemetry_pb2.AislePosition(

```



```

94         aisle_id="A-07", offset_m=round(rng.
95             uniform(0.0, 120.0), 2)
96     )
97     return frame
98
99 # -- Mode 1: unary
100 -----
101
102 def GetRobotStatus(
103     self,
104     request: telemetry_pb2.RobotStatusRequest,
105     context: grpc.ServicerContext,
106 ) -> telemetry_pb2.RobotStatusReply:
107     _require_budget(context)
108     _check_known(request.robot_id)
109     if request.robot_id == "RB-SLOW":
110         time.sleep(0.200) # simulated slow
111                             dependency
112         _require_budget(context, min_s=0.100) # re
113             -check after the stall
114     last = self._frame(request.robot_id, seq=0)
115     reply = telemetry_pb2.RobotStatusReply(
116         robot_id=request.robot_id,
117         mode=last.mode,
118         battery_pct=last.battery_pct,
119     )
120     reply.firmware_built_at.FromSeconds(
121         BASE_EPOCH_S - 86_400)
122     reply.last_frame.CopyFrom(last)
123     return reply
124
125 # -- Mode 2: server streaming
126 -----
127
128 def StreamTelemetry(
129     self,
130     request: telemetry_pb2.StreamTelemetryRequest,
131     context: grpc.ServicerContext,
132 ) -> Iterator[telemetry_pb2.TelemetryFrame]:
133     _check_known(request.robot_id)
134     if request.max_frames == 0:
135         raise FrameValidationError(
136             "max_frames must be >= 1",
137             [
138                 telemetry_pb2.FieldViolation(
139                     field="max_frames", description

```

```

133         )
134     ],
135 )
136     period_s = max(request.period_ms, 1) / 1000.0
137     for seq in range(request.max_frames):
138         if not context.is_active():
139             return # client cancelled: stop
140                 producing promptly
141                 _require_budget(context)
142                 time.sleep(period_s)
143                 yield self._frame(request.robot_id, seq)
144
145 # -- Mode 3: client streaming
146 -----
147 def UploadTelemetry(
148     self,
149     request_iterator: Iterable[telemetry_pb2.
150         TelemetryFrame],
151     context: grpc.ServicerContext,
152 ) -> telemetry_pb2.UploadSummary:
153     accepted = 0
154     rejected = 0
155     min_battery = float("inf")
156     max_battery = float("-inf")
157     for frame in request_iterator:
158         if not context.is_active():
159             break
160         if frame.robot_id not in KNOWN_ROBOTS:
161             raise FrameValidationError(
162                 "frame_references_unknown_robot",
163                 [
164                     telemetry_pb2.FieldViolation(
165                         field="robot_id",
166                         description="must_be_a_
167                             known_robot",
168                     )
169                 ],
170             )
171         if not 0.0 <= frame.battery_pct <= 100.0:
172             rejected += 1
173             continue
174         accepted += 1
175         min_battery = min(min_battery, frame.
176             battery_pct)
177         max_battery = max(max_battery, frame.

```

```

173         battery_pct)
174     return telemetry_pb2.UploadSummary(
175         accepted=accepted,
176         rejected=rejected,
177         min_battery_pct=round(min_battery, 3) if
178             accepted else 0.0,
179         max_battery_pct=round(max_battery, 3) if
180             accepted else 0.0,
181     )
182
183 # -- Mode 4: bidirectional streaming
184 -----
185
186 def ControlLoop(
187     self,
188     request_iterator: Iterable[telemetry_pb2.
189         ControlCommand],
190     context: grpc.ServicerContext,
191 ) -> Iterator[telemetry_pb2.TelemetryFrame]:
192     for command in request_iterator:
193         if not context.is_active():
194             return
195         _require_budget(context)
196         _check_known(command.robot_id)
197         action = command.WhichOneof("action")
198         if action == "set_mode":
199             mode = command.set_mode
200         elif action == "estop":
201             mode = (
202                 telemetry_pb2.ROBOT_MODE_FAULT
203                 if command.estop
204                 else telemetry_pb2.ROBOT_MODE_IDLE
205             )
206         elif action == "set_speed_mps":
207             mode = telemetry_pb2.ROBOT_MODE_MOVING
208         else:
209             raise FrameValidationError(
210                 "command_has_no_action_set",
211                 [
212                     telemetry_pb2.FieldViolation(
213                         field="action", description
214                         ="oneof_must_be_set"
215                     )
216                 ],
217             )
218     yield self._frame(command.robot_id, seq=
219         command.sequence, mode=mode)

```

```

212
213
214 def make_server(
215     interceptors: tuple[grpc.ServerInterceptor, ...] =
216         (),
217     max_workers: int = 8,
218 ) -> grpc.Server:
219     server = grpc.server(
220         futures.ThreadPoolExecutor(max_workers=
221             max_workers),
222         interceptors=interceptors,
223         options=[
224             ("grpc.max_connection_age_ms", 300_000), #
225             let L4/L7 LBs rebalance
226             ("grpc.keepalive_time_ms", 60_000),
227             ("grpc.keepalive_permit_without_calls", 0),
228         ],
229     )
230     telemetry_pb2_grpc.
231         add_TelemetryServiceServicer_to_server(
232             TelemetryService(), server
233         )
234     return server
235
236
237 def main() -> None:
238     from error_middleware import
239         ErrorMappingInterceptor
240
241     parser = argparse.ArgumentParser(description="
242         Telemetry_gRPC_server")
243     parser.add_argument("--port", type=int, default
244         =50051)
245     args = parser.parse_args()
246     server = make_server(interceptors=(
247         ErrorMappingInterceptor(),))
248     bound = server.add_insecure_port(f"localhost:{args.
249         port}")
250     server.start()
251     print(f"telemetry_server_listening_on_localhost:{
252         bound}")
253     server.wait_for_termination()
254
255 if __name__ == "__main__":
256     main()

```

14.3.3 Listing 3 — Error-Mapping Middleware with Rich Details

A `grpc.ServerInterceptor` rebuilds each handler so that any `DomainError` becomes the right status code plus a serialized `ErrorDetails` message in the binary `error-details-bin` trailer — the same transport trick Google APIs use for `google.rpc.Status` in `grpc-status-details-bin`. `decode_error_details` is the client half; the test suite asserts the round trip, including correlation-ID propagation from inbound metadata into the error payload.

```

1  """Server interceptor: map domain errors to gRPC status
   _+_rich_details.
2
3  Rich details ride in the binary trailing-metadata key "
   error-details-bin",
4  the same transport trick Google APIs use for google.rpc
   .Status in
5  "grpc-status-details-bin". A client helper decodes them
   _back.
6  """
7  from __future__ import annotations
8
9  from typing import Any, Callable
10
11  import grpc
12
13  import telemetry_pb2
14  from telemetry_server import DomainError
15
16  TRAILER_KEY = "error-details-bin"
17
18
19  def _correlation_id(context: grpc.ServicerContext) ->
   str:
20      for key, value in context.invocation_metadata():
21          if key == "x-correlation-id":
22              return str(value)
23      return ""
24
25
26  def _abort_with_details(
27      context: grpc.ServicerContext, err: DomainError
28  ) -> None:
29      details = telemetry_pb2.ErrorDetails(
30          reason=err.reason,

```

```
31         domain="telemetry.northwind.example",
32         correlation_id=_correlation_id(context),
33     )
34     details.violations.extend(err.violations)
35     context.set_trailing_metadata(
36         ((TRAILER_KEY, details.SerializeToString()),)
37     )
38     context.abort(err.status_code, err.message)
39
40
41 def _wrap_unary(behavior: Callable) -> Callable:
42     def handler(request: Any, context: grpc.
43         ServicerContext) -> Any:
44         try:
45             return behavior(request, context)
46         except DomainError as err:
47             _abort_with_details(context, err)
48
49     return handler
50
51 def _wrap_stream(behavior: Callable) -> Callable:
52     def handler(request_or_iter: Any, context: grpc.
53         ServicerContext) -> Any:
54         try:
55             yield from behavior(request_or_iter,
56                 context)
57         except DomainError as err:
58             _abort_with_details(context, err)
59
60     return handler
61
62 class ErrorMappingInterceptor(grpc.ServerInterceptor):
63     """Rebuilds each handler so DomainError becomes
64     status+rich details."""
65
66     def intercept_service(self, continuation,
67         handler_call_details):
68         handler = continuation(handler_call_details)
69         if handler is None:
70             return None
71         deserializer = handler.request_deserializer
72         serializer = handler.response_serializer
73         if handler.request_streaming and handler.
74             response_streaming:
```

```

71         return grpc.
           stream_stream_rpc_method_handler(
72             _wrap_stream(handler.stream_stream),
73             request_deserializer=deserializer,
74             response_serializer=serializer,
75         )
76     if handler.request_streaming:
77         return grpc.stream_unary_rpc_method_handler(
78             (
79                 _wrap_unary(handler.stream_unary),
80                 request_deserializer=deserializer,
81                 response_serializer=serializer,
82             )
83         )
84     if handler.response_streaming:
85         return grpc.unary_stream_rpc_method_handler(
86             (
87                 _wrap_stream(handler.unary_stream),
88                 request_deserializer=deserializer,
89                 response_serializer=serializer,
90             )
91         )
92     return grpc.unary_unary_rpc_method_handler(
93         _wrap_unary(handler.unary_unary),
94         request_deserializer=deserializer,
95         response_serializer=serializer,
96     )
97
98 def decode_error_details(
99     call_or_error: Any,
100 ) -> telemetry_pb2.ErrorDetails | None:
101     """Client-side: pull ErrorDetails out of a failed
102     call's trailers."""
103     metadata = call_or_error.trailing_metadata() or ()
104     for key, value in metadata:
105         if key == TRAILER_KEY:
106             details = telemetry_pb2.ErrorDetails()
107             details.ParseFromString(value)
108             return details
109     return None

```

14.3.4 Listing 4 — Client: Metadata, Deadline Budgets, Interceptor, Retry Policy

The interceptor attaches authorization and x-correlation-id to *every* call, of every cardinality; the channel carries a retry policy as a JSON

service config (bounded attempts, exponential backoff, UNAVAILABLE only — retrying a non-idempotent method that may have reached the server is how you double-charge a customer).

```
1  """Client for the Northwind Robotics telemetry service
   (Chapter 13).
2
3  Shows metadata, per-call deadline budgets, an auth+
   correlation-ID
4  interceptor, and a retry policy installed via gRPC
   service config.
5  Offline-first: localhost only.
6  """
7  from __future__ import annotations
8
9  import itertools
10 import json
11 from dataclasses import dataclass
12 from typing import Any, Callable, Iterable, Iterator
13
14 import grpc
15
16 import telemetry_pb2
17 import telemetry_pb2_grpc
18
19 # Retry policy delivered through the channel's service
   config. Retries are
20 # bounded (maxAttempts) and restricted to UNAVAILABLE
   -- the only status here
21 # that is provably safe to retry for these non-
   idempotent methods when the
22 # attempt never reached the server.
23 RETRY_SERVICE_CONFIG = json.dumps(
24     {
25         "methodConfig": [
26             {
27                 "name": [{"service": "northwind.
   telemetry.v1.TelemetryService"}],
28                 "retryPolicy": {
29                     "maxAttempts": 3,
30                     "initialBackoff": "0.050s",
31                     "maxBackoff": "0.500s",
32                     "backoffMultiplier": 2.0,
33                     "retryableStatusCodes": ["
   UNAVAILABLE"],
34             },
```



```

35         # Overall per-RPC timeout when the
36         caller sets none.
37         "timeout": "2.000s",
38     }
39 }
40 )
41
42
43 @dataclass(frozen=True)
44 class _ClientCallDetails:
45     """grpc.ClientCallDetails is an interface; this is
46     our concrete copy."""
47
48     method: str
49     timeout: float | None
50     metadata: tuple[tuple[str, Any], ...]
51     credentials: Any
52     wait_for_ready: bool | None
53     compression: Any
54
55 class AuthMetadataInterceptor(
56     grpc.UnaryUnaryClientInterceptor,
57     grpc.UnaryStreamClientInterceptor,
58     grpc.StreamUnaryClientInterceptor,
59     grpc.StreamStreamClientInterceptor,
60 ):
61     """Attaches auth and correlation metadata to every
62     outgoing RPC."""
63
64     def __init__(
65         self, token: str, correlation_factory: Callable[
66             [], str]
67     ) -> None:
68         self._token = token
69         self._correlation_factory = correlation_factory
70
71     def _augment(
72         self, details: grpc.ClientCallDetails
73     ) -> _ClientCallDetails:
74         metadata = tuple(details.metadata or ()) + (
75             ("authorization", f"Bearer_{self._token}"),
76             ("x-correlation-id", self.

```

```
76         return _ClientCallDetails(
77             method=details.method,
78             timeout=details.timeout,
79             metadata=metadata,
80             credentials=details.credentials,
81             wait_for_ready=details.wait_for_ready,
82             compression=details.compression,
83         )
84
85     def intercept_unary_unary(self, continuation,
86                             client_call_details, request):
87         return continuation(self._augment(
88                             client_call_details), request)
89
90     def intercept_unary_stream(self, continuation,
91                               client_call_details, request):
92         return continuation(self._augment(
93                             client_call_details), request)
94
95     def intercept_stream_unary(
96         self, continuation, client_call_details,
97         request_iterator
98     ):
99         return continuation(self._augment(
100                             client_call_details), request_iterator)
101
102     def intercept_stream_stream(
103         self, continuation, client_call_details,
104         request_iterator
105     ):
106         return continuation(self._augment(
107                             client_call_details), request_iterator)
108
109 def make_channel(
110     address: str,
111     token: str = "synthetic-dev-token",
112     correlation_factory: Callable[[], str] | None =
113         None,
114 ) -> grpc.Channel:
115     """Channel with retries and the
116     auth interceptor."""
117     if correlation_factory is None:
118         counter = itertools.count(1)
119
120     def _next_id() -> str:
```

```

112         return f"corr-{{next(counter):06d}}"
113
114         correlation_factory = _next_id
115     options = [
116         ("grpc.enable_retries", 1),
117         ("grpc.service_config", RETRY_SERVICE_CONFIG),
118         ("grpc.keepalive_time_ms", 60_000),
119     ]
120     raw = grpc.insecure_channel(address, options=
121         options)
122     return grpc.intercept_channel(
123         raw, AuthMetadataInterceptor(token,
124             correlation_factory)
125     )
126
127 def fetch_status(
128     stub: telemetry_pb2_grpc.TelemetryServiceStub,
129     robot_id: str,
130     deadline_s: float = 1.0,
131 ) -> telemetry_pb2.RobotStatusReply:
132     """Unary call with an explicit deadline budget."""
133     request = telemetry_pb2.RobotStatusRequest(robot_id
134         =robot_id)
135     return stub.GetRobotStatus(request, timeout=
136         deadline_s)
137
138 def stream_frames(
139     stub: telemetry_pb2_grpc.TelemetryServiceStub,
140     robot_id: str,
141     max_frames: int,
142     deadline_s: float = 5.0,
143 ) -> Iterator[telemetry_pb2.TelemetryFrame]:
144     request = telemetry_pb2.StreamTelemetryRequest(
145         robot_id=robot_id, max_frames=max_frames,
146         period_ms=20
147     )
148     yield from stub.StreamTelemetry(request, timeout=
149         deadline_s)
150
151 def main() -> None:
152     import argparse
153
154     parser = argparse.ArgumentParser(description="

```

```

152     Telemetry_gRPC_client_demo")
153     parser.add_argument("--port", type=int, default
154                         =50051)
155     args = parser.parse_args()
156     channel = make_channel(f"localhost:{args.port}")
157     stub = telemetry_pb2_grpc.TelemetryServiceStub(
158         channel)
159
160     reply = fetch_status(stub, "RB-1001", deadline_s
161                         =1.0)
162     mode = telemetry_pb2.RobotMode.Name(reply.mode)
163     print(f"status: {reply.robot_id} mode={mode}
164           battery={reply.battery_pct:.2f}%")
165
166     print("stream:")
167     for frame in stream_frames(stub, "RB-1001",
168                               max_frames=3):
169         print(f"{frame} battery={frame.battery_pct:.2f}
170               mode={frame.mode}")
171
172     frames = [
173         telemetry_pb2.TelemetryFrame(robot_id="RB-1001"
174                                     , battery_pct=b)
175         for b in (91.5, 90.8, 88.2)
176     ]
177     summary = stub.UploadTelemetry(iter(frames),
178                                   timeout=2.0)
179     print(f"upload: accepted={summary.accepted}
180           rejected={summary.rejected}")
181     channel.close()
182
183 if __name__ == "__main__":
184     main()

```

14.3.5 Listing 5 — Bidirectional Streaming with Flow-Control-Aware Production

A producer thread writes commands into a *bounded* queue; the gRPC request generator drains it. When the window is full, `put()` blocks — application-level backpressure stacked on top of HTTP/2's per-stream flow control. This is the canonical shape for any bidi producer fed from a source that outruns the network.

```

1  """Bidirectional_control-loop_demo_with_flow-control-
   aware_production.
2
3  A producer thread writes commands into a bounded queue;
   the gRPC request
4  generator drains it. When the queue is full the
   producer blocks, giving the
5  application its own backpressure on top of HTTP/2 flow
   control. Offline:
6  localhost only, synthetic commands, deterministic
   pacing.
7  """
8  from __future__ import annotations
9
10 import queue
11 import threading
12 import time
13 from typing import Iterator
14
15 import grpc
16
17 import telemetry_pb2
18 import telemetry_pb2_grpc
19
20 _DONE: object = object() # sentinel closing the
   command stream
21
22
23 def _producer(
24     outbox: queue.Queue,
25     robot_id: str,
26     count: int,
27     interval_s: float,
28 ) -> None:
29     try:
30         for seq in range(count):
31             if seq % 5 == 4:
32                 command = telemetry_pb2.ControlCommand(
33                     robot_id=robot_id, sequence=seq,
34                     estop=False
35                 )
36             else:
37                 command = telemetry_pb2.ControlCommand(
38                     robot_id=robot_id,
39                     sequence=seq,
40                     set_mode=telemetry_pb2.

```

```

40         ROBOT_MODE_MOVING,
41     )
42     # put() blocks while the window is full:
43     application backpressure.
44     outbox.put(command)
45     time.sleep(interval_s)
46
47 finally:
48     outbox.put(_DONE)
49
50 def _commands(
51     inbox: queue.Queue,
52 ) -> Iterator[telemetry_pb2.ControlCommand]:
53     while True:
54         item = inbox.get()
55         if item is _DONE:
56             return
57         yield item
58
59 def run_control_loop(
60     stub: telemetry_pb2_grpc.TelemetryServiceStub,
61     robot_id: str,
62     count: int,
63     interval_s: float = 0.005,
64     window: int = 8,
65     deadline_s: float = 10.0,
66 ) -> list[telemetry_pb2.TelemetryFrame]:
67     """Drive a bidi ControlLoop; returns every frame
68     the server echoed."""
69     outbox: queue.Queue = queue.Queue(maxsize=window)
70     producer = threading.Thread(
71         target=_producer,
72         args=(outbox, robot_id, count, interval_s),
73         daemon=True,
74     )
75     producer.start()
76     frames: list[telemetry_pb2.TelemetryFrame] = []
77     try:
78         for frame in stub.ControlLoop(_commands(outbox),
79                                     timeout=deadline_s):
80             frames.append(frame)
81     finally:
82         producer.join(timeout=2.0)
83     return frames

```

```

82
83 def main() -> None:
84     import argparse
85
86     from telemetry_client import make_channel
87
88     parser = argparse.ArgumentParser(description="
89         Bidirectional_control-loop_demo")
90     parser.add_argument("--port", type=int, default
91         =50051)
92     args = parser.parse_args()
93     channel = make_channel(f"localhost:{args.port}")
94     stub = telemetry_pb2_grpc.TelemetryServiceStub(
95         channel)
96     frames = run_control_loop(stub, robot_id="RB-1001",
97         count=10)
98     for frame in frames:
99         mode = telemetry_pb2.RobotMode.Name(frame.mode)
100         print(f"ack_robot={frame.robot_id}_mode={mode}_
101             t={frame.observed_at.seconds}")
102     channel.close()
103
104 if __name__ == "__main__":
105     main()

```

14.3.6 Listing 6 — Offline Test Suite: Deadline, Cancellation, and Stream Error Paths

The fixture binds the in-process server to port 0 (the OS assigns a free localhost port) — no fixed port collisions, no network, no credentials. Eight tests cover: unary happy path, rich-error round trip, deadline exceeded against the deliberately slow robot, server-stream bounds, client cancellation observed as CANCELLED, mid-stream validation abort, and the bidi round trip with fully deterministic timestamp assertions.

```

1  """Offline_pytest_suite_for_the_Chapter_13_telemetry_
2     service.
3
4  The_server_runs_in-process_on_an_ephemeral_localhost_
5     port(port_0);_no
6     network,_no_credentials,_deterministic_data._Covers_all
7     _four_RPC_modes,
8     deadline_propagation,_cancellation,_and_stream_error_
9     paths.

```

```

6  """
7  from __future__ import annotations
8
9  import grpc
10 import pytest
11
12 import telemetry_pb2
13 import telemetry_pb2_grpc
14 from bidi_demo import run_control_loop
15 from error_middleware import ErrorMappingInterceptor,
    decode_error_details
16 from telemetry_client import make_channel
17 from telemetry_server import BASE_EPOCH_S, make_server
18
19
20 @pytest.fixture()
21 def stub():
22     server = make_server(interceptors=(
23         ErrorMappingInterceptor(),))
24     port = server.add_insecure_port("localhost:0")
25     server.start()
26     channel = make_channel(
27         f"localhost:{port}", correlation_factory=lambda
28             : "corr-test-0001"
29     )
30     yield telemetry_pb2_grpc.TelemetryServiceStub(
31         channel)
32     channel.close()
33     server.stop(None)
34
35 def _frame(robot_id: str, battery: float) ->
    telemetry_pb2.TelemetryFrame:
36     frame = telemetry_pb2.TelemetryFrame(robot_id=
37         robot_id, battery_pct=battery)
38     frame.observed_at.FromSeconds(BASE_EPOCH_S)
39     return frame
40
41 def test_unary_status_happy_path(stub) -> None:
42     reply = stub.GetRobotStatus(
43         telemetry_pb2.RobotStatusRequest(robot_id="RB
44             -1001"), timeout=5.0
45     )
46     assert reply.robot_id == "RB-1001"
47     assert reply.mode == telemetry_pb2.

```



```

    ROBOT_MODE_MOVING
45     assert 0.0 <= reply.battery_pct <= 100.0
46     assert reply.last_frame.joint_positions_rad["
        shoulder"] != 0.0
47
48
49 def
    test_unknown_robot_maps_to_not_found_with_rich_details
    (stub) -> None:
50     with pytest.raises(grpc.RpcError) as excinfo:
51         stub.GetRobotStatus(
52             telemetry_pb2.RobotStatusRequest(robot_id="
                RB-9999"), timeout=5.0
53         )
54     err = excinfo.value
55     assert err.code() == grpc.StatusCode.NOT_FOUND
56     details = decode_error_details(err)
57     assert details is not None
58     assert details.reason == "ROBOT_NOT_FOUND"
59     assert details.correlation_id == "corr-test-0001"
60
61
62 def test_deadline_exceeded_propagates(stub) -> None:
63     # RB-SLOW stalls 200 ms server-side; the 50 ms
        client deadline wins.
64     with pytest.raises(grpc.RpcError) as excinfo:
65         stub.GetRobotStatus(
66             telemetry_pb2.RobotStatusRequest(robot_id="
                RB-SLOW"), timeout=0.05
67         )
68     assert excinfo.value.code() == grpc.StatusCode.
        DEADLINE_EXCEEDED
69
70
71 def test_server_stream_respects_max_frames(stub) ->
    None:
72     request = telemetry_pb2.StreamTelemetryRequest(
73         robot_id="RB-1002", max_frames=4, period_ms=5
74     )
75     frames = list(stub.StreamTelemetry(request, timeout
        =5.0))
76     assert len(frames) == 4
77     assert [f.observed_at.seconds for f in frames] == [
78         BASE_EPOCH_S + i * 5 for i in range(4)
79     ]
80

```

```
81
82 def test_client_cancellation_stops_server_stream(stub)
    -> None:
83     request = telemetry_pb2.StreamTelemetryRequest(
84         robot_id="RB-1001", max_frames=100_000,
            period_ms=5
85     )
86     call = stub.StreamTelemetry(request, timeout=30.0)
87     first = next(call)
88     assert first.robot_id == "RB-1001"
89     call.cancel()
90     with pytest.raises(grpc.RpcError) as excinfo:
91         next(call)
92     assert excinfo.value.code() == grpc.StatusCode.
        CANCELLED
93
94
95 def test_upload_rejects_out_of_range_battery(stub) ->
    None:
96     frames = [_frame("RB-1001", 90.0), _frame("RB-1001"
        , 140.0), _frame("RB-1001", 50.0)]
97     call = stub.UploadTelemetry.future(iter(frames),
        timeout=5.0)
98     summary = call.result()
99     assert (summary.accepted, summary.rejected) == (2,
        1)
100    assert summary.min_battery_pct == pytest.approx
        (50.0)
101    assert summary.max_battery_pct == pytest.approx
        (90.0)
102
103
104 def
    test_upload_unknown_robot_aborts_mid_stream_with_details
    (stub) -> None:
105     frames = [_frame("RB-1001", 90.0), _frame("RB-9999"
        , 10.0)]
106     call = stub.UploadTelemetry.future(iter(frames),
        timeout=5.0)
107     with pytest.raises(grpc.RpcError) as excinfo:
108         call.result()
109     err = excinfo.value
110     assert err.code() == grpc.StatusCode.
        INVALID_ARGUMENT
111     details = decode_error_details(err)
112     assert details is not None
```

```

113     assert details.reason == "FRAME_VALIDATION_FAILED"
114     assert details.violations[0].field == "robot_id"
115     assert details.correlation_id == "corr-test-0001"
116
117
118 def test_control_loop_bidi_round_trip(stub) -> None:
119     frames = run_control_loop(
120         stub, robot_id="RB-1001", count=10, interval_s
121         =0.001, deadline_s=10.0
122     )
123     assert len(frames) == 10
124     # Sequence numbers echo through to deterministic
125     timestamps.
126     assert [f.observed_at.seconds for f in frames] == [
127         BASE_EPOCH_S + i * 5 for i in range(10)
128     ]
129     # seq % 5 == 4 carries estop=False -> IDLE;
130     everything else MOVING.
131     assert frames[4].mode == telemetry_pb2.
132     ROBOT_MODE_IDLE
133     assert frames[0].mode == telemetry_pb2.
134     ROBOT_MODE_MOVING

```

Verified output (Python 3.13, grpcio 1.83): all eight tests pass, and the client and bidi demos print the deterministic battery and mode sequences shown in the lab (Section 14.5).

14.4 Common Pitfalls & Anti-Patterns

1. **Missing deadlines.** The default gRPC deadline is *infinity*. A client without `timeout=` will wait forever on a wedged server, and the leaked threads/sockets accumulate into an outage. Rule: *every call site sets a deadline*, and servers re-check `time_remaining()` after any blocking step. gRPC's infinity default is the single most dangerous default in the stack.
2. **Ignoring cancellation.** A server-streaming handler that never polls `context.is_active()` keeps serializing and sending to a client that left minutes ago, burning CPU and holding locks. Treat `is_active()` checks as loop hygiene, exactly like the `StreamTelemetry` listing.
3. **The L4 load-balancer hotspot.** Putting a classic TCP load balancer in front of gRPC and declaring victory. Connections, not RPCs, get balanced; see the war story in Section 14.5. Mitigate with

client-side `round_robin`, xDS/lookaside, an L7 proxy, or at minimum `max_connection_age` so clients redial.

4. **Unbounded streams.** `rpc Watch(X) returns (stream Y)` with no pagination, no max count, no heartbeat, and a consumer slower than the producer. HTTP/2 flow control will pause the sender, and your server's send buffers grow until OOM. Bound every stream (`max_frames` in our IDL), add application-level backpressure (the bounded queue in Listing 5), and set deadlines on streams too.
5. **Leaking proto field numbers.** Deleting a field and letting the next engineer reuse its tag. A stale peer then decodes old bytes as the new field — silent corruption, no error. Always **reserve** the tag and name first (Listing 1), and review `.proto` diffs like schema migrations (Chapter 4).
6. **Catching `grpc.RpcError` without reading `.code()`.** `except grpc.RpcError: retry()` conflates `INVALID_ARGUMENT` (your bug — retrying is a hot loop) with `UNAVAILABLE` (transient — retrying is correct). Branch on `err.code()`; consult Table 14.1; only `UNAVAILABLE` (and `ABORTED` for transactional ops) is generically retryable.
7. **Treating gRPC as a drop-in REST replacement for public APIs.** No browser support without a proxy, no human-readable payloads, no CDN caching, unfamiliar error surface for third parties, and payload inspection tooling that assumes JSON. Keep public edges on HTTP/JSON (transcode if you must share a contract); use gRPC where you control both ends.
8. **Per-call channels.** Creating a channel (TCP + HTTP/2 setup) per RPC throws away multiplexing and adds handshake latency to every call. Channels are heavyweight and thread-safe: build one per target, share it, close it at shutdown.
9. **Retried streaming calls without thinking.** A retry policy on a streaming RPC replays the request iterator — fine for idempotent reads, catastrophic for “append these events.” Scope `retryPolicy` per method in the service config, and prefer hedging/retries only on methods you have marked idempotent.
10. **Overstuffed metadata.** Metadata is HPACK-compressed but not free; multi-kilabyte headers on every call defeat the point. Keep metadata to credentials, correlation IDs, and small routing hints —

payloads belong in messages.

14.5 Hands-On Production Lab & Real-World Case Study

14.5.1 Lab: The Telemetry Service End to End

All commands are PowerShell, offline, from a working directory containing the six listings.

Step 1 — environment and codegen.

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install grpcio grpcio-tools protobuf pytest
4 python -m grpc_tools.protoc -I. --python_out=. --
   grpc_python_out=. telemetry.proto
```

Step 2 — run the suite. `python -m pytest -q` should report 8 passed. Read the tests as executable specifications of the deadline, cancellation, and error-path semantics.

Step 3 — exercise all four modes against a live server. In one terminal: `python telemetry_server.py -port 50051`. In a second terminal: `python telemetry_client.py -port 50051` (unary + server-streaming + client-streaming) and `python bidi_demo.py -port 50051` (bidirectional). Expected client output:

```
1 status: RB-1001 mode=ROBOT_MODE_MOVING battery=95.00%
2 stream:
3   frame battery=95.00 mode=2
4   frame battery=94.97 mode=2
5   frame battery=94.96 mode=2
6 upload: accepted=3 rejected=0
```

Step 4 — grpcurl (optional but instructive). Reflection requires `pip install grpcio-reflection` and two lines of server setup (`reflection.enable_server_reflection`); then:

```
1 grpcurl -plaintext localhost:50051 list
2 grpcurl -plaintext -d '{"robot_id": "RB-1001"}'
   localhost:50051 '
3   northwind.telemetry.v1.TelemetryService/
   GetRobotStatus
4 grpcurl -plaintext -d '{"robot_id": "RB-9999"}'
   localhost:50051 '
5   northwind.telemetry.v1.TelemetryService/
   GetRobotStatus # NOT_FOUND (code 5)
```

Step 5 — observe deadline propagation. Call the slow robot with a generous then a tight budget and compare outcomes:

```

1 import grpc, telemetry_pb2, telemetry_pb2_grpc
2 channel = grpc.insecure_channel("localhost:50051")
3 stub = telemetry_pb2_grpc.TelemetryServiceStub(channel)
4 req = telemetry_pb2.RobotStatusRequest(robot_id="RB-
    SLOW")
5 print(stub.GetRobotStatus(req, timeout=5.0).robot_id)
    # succeeds: budget ample
6 try:
7     stub.GetRobotStatus(req, timeout=0.05)
    # 50 ms < 200 ms stall
8 except grpc.RpcError as err:
9     print(err.code())
    # StatusCode.DEADLINE_EXCEEDED

```

The server-side `_require_budget` re-check after the stall is what turns “slow” into a clean `DEADLINE_EXCEEDED` instead of a late, wasted response.

14.5.2 War Story: The L4 Hotspot That Pinned All Traffic to One Pod

A logistics company (thinly fictionalized) ran a gRPC telemetry ingestion fleet behind a cloud Layer-4 load balancer: 20 pods, autoscaled, each client gateway holding one connection per target. Alerting fired at 03:40: p99 latency climbing on *one* pod, CPU at 98%, while the other nineteen idled at 8%. The autoscaler, watching *fleet-average* CPU at 12%, politely did nothing.

The mechanism was exactly Section 14.2.9’s L4 problem, with a queue-theoretic twist. The ingest clients were long-lived sidecars that dialed once at boot and kept the connection for days. All twelve sidecars had booted during a deploy when pod `telemetry-7` was the only ready endpoint behind the LB — so all twelve connections, and therefore *every RPC*, terminated on that one pod. Kubernetes service load balancing had done its job perfectly: it balanced *connections* (twelve of them, all during a one-pod window) and had no idea that each connection carried thousands of RPCs per second.

The on-call runbook failed twice. Restarting `telemetry-7` moved the hotspot: clients re-dialed, the LB again picked the lowest-connection backend — which was the freshly restarted pod — and within ninety seconds the new pod was the hotspot. Scaling from 20 to 30 pods did nothing at all, because no client ever re-dialed to discover the new backends. The incident lasted four hours and its “fix” — restarting all the sidecars simultaneously to force mass redial — was indistinguishable from an attack on their own fleet.

The durable fixes, in order of impact: **(1)** server-side `max_connection_age` of five minutes with a thirty-second grace, so every connection is periodically and gently recycled across the fleet — this alone would have capped the incident at minutes; **(2)** client-side `round_robin` with DNS resolution of all pod IPs, moving balance from per-connection to per-RPC; **(3)** alerting on *per-pod* p99 and RPC-count skew (a $10\times$ max/median ratio), not fleet averages. Postmortem quote worth memorizing: “*We load-balanced the connections and called it load balancing. Nobody owned the RPCs.*”

14.6 Self-Assessment Exercises & Deep-Dive Questions

1. **Wire decoding.** A server receives the bytes `00 00 00 00 0A` followed by ten protobuf bytes on stream 3. State the compression flag, message length, and what the receiver does if the stream ends after six of the ten payload bytes.
2. **Framing arithmetic.** A response message serializes to 300 bytes. Write out the exact 5-byte prefix in hex. What is the largest gRPC message this framing can express, and what default limit does `grpc-python` actually enforce (look up `grpc.max_receive_message_length`)?
3. **Mode selection.** For each, pick a mode and defend it: (a) pushing firmware-update progress to an operator console; (b) uploading a day of buffered sensor logs; (c) a trader-style command/ack console; (d) fetching one robot’s current pose.
4. **Deadline math.** A client sets a 500 ms deadline. The proxy hop consumes 40 ms. What `grpc-timeout` value does the downstream server see (approximately), and in what unit suffix could it be encoded? What happens if the proxy *adds* 50 ms of budget instead?
5. **Retry safety.** The retry policy in Listing 4 names only `UNAVAILABLE`. Explain precisely why retrying `DEADLINE_EXCEEDED` is dangerous for `UploadTelemetry` but harmless for `GetRobotStatus`.
6. **Evolution drill.** Product wants to replace `battery_pct` (tag 3, double) with `battery_millivolts` (uint32). Write the safe two-phase migration using `reserved` and dual-publish, referencing Chapter 4’s compatibility classes.
7. **Interceptor design.** Sketch a server interceptor that enforces a per-token rate limit (Chapter 14 previews the algorithms) and returns

`RESOURCE_EXHAUSTED` with a `RetryInfo`-style detail in a `-bin` trailer. Which of the four handler shapes need different treatment and why?

8. **Debug the symptom.** Clients see `UNAVAILABLE` spikes every five minutes, synchronized with nothing in the deploy calendar. Given the server options in Listing 2, form a hypothesis and the one-line config change that confirms it.
9. **Transcoding design.** You must expose `StreamTelemetry` to a browser dashboard. Compare `grpc-web` vs. JSON transcoding vs. a bespoke `WebSocket` shim on contract fidelity, ops cost, and browser support; pick one and justify.
10. **Deep dive.** Explain why `grpc-status` lives in trailers even for unary calls, what a trailers-only response is, and which intermediary behaviors (caches, L4 proxies, browsers) each property enables or forbids.

14.7 Interview Q&A Vault

Ordered junior → staff. Answers are calibrated to what a strong candidate says in two to six sentences.

- Q1. What is an RPC, and what are the stub and skeleton?** A remote procedure call is a network invocation dressed as a local function call. The *stub* is client-side generated code exposing the typed signature and marshalling arguments to bytes; the *skeleton* (servicer) is server-side generated glue that unmarshals and calls your implementation. Both derive from one IDL file, so the two ends cannot disagree about types.
- Q2. What two technologies does gRPC combine, and why?** Protocol Buffers as IDL and binary encoding (compact, typed, evolvable) and HTTP/2 as transport (multiplexed streams, HPACK header compression, flow control). Together they give small payloads, cheap concurrency, and generated contracts [gRP24; Goo24].
- Q3. Explain the 5-byte gRPC message prefix.** Every message on a stream is length-prefixed: byte 0 is a compression flag (0/1), bytes 1–4 are the big-endian uint32 message length, then the serialized protobuf payload. HTTP/2 DATA frame boundaries are irrelevant to message boundaries — the prefix is the delimiter, which is what lets one stream carry arbitrarily many ordered messages.

- Q4. What does the `:path` pseudo-header look like for a gRPC call?** `/package.Service/Method`, e.g. `/northwind.telemetry.v1.TelemetryService/GetTelemetry`. The method is always POST; routing keys entirely off `:path`.
- Q5. What `content-type` does gRPC use, and what HTTP status does a failed gRPC call return?** `application/grpc` (often `application/grpc+proto`). The HTTP status is 200 even for failed calls — the outcome lives in the `grpc-status` trailer, because for streaming calls the failure may be discovered after response bytes have flowed. Interviewers ask this to see if you have ever looked at the wire.
- Q6. What is a trailers-only response?** When a call fails before producing any message, the server may send a single HEADERS frame containing `:status 200`, `grpc-status`, and `END_STREAM` — headers and trailers collapsed into one frame. Most gRPC error responses look like this on the wire.
- Q7. Name the four RPC modes and a use case for each.** Unary (request/reply — `GetRobotStatus`); server streaming (subscriptions, feeds — `StreamTelemetry`); client streaming (bulk ingestion — `UploadTelemetry`); bidirectional streaming (interactive control sessions — `ControlLoop`).
- Q8. Why does proto3 have no required fields?** Because required fields can never be safely removed: every old reader rejects messages missing them, so the field is load-bearing forever. Proto3 makes everything optional-with-defaults and moves validation to application logic, which can evolve.
- Q9. What is a `oneof`?** A set of fields of which at most one is set; setting one clears the others, and presence is preserved on the wire so the receiver can ask `WhichOneof`. Use it for genuine alternatives, and note the evolution trap: moving an existing field into a `oneof` changes its wire semantics.
- Q10. Why must protobuf tag numbers never be reused?** The tag, not the name, is the field's identity on the wire. A stale peer holding old bytes will decode them into the new field silently — wrong data, no error. Retire tags with `reserved` so the compiler enforces the ban [Goo24].
- Q11. Deadline vs. timeout — is there a difference in gRPC?** In gRPC they are one mechanism seen from two ends: the client sets a

timeout (relative), the core converts it to an absolute deadline, and transmits the *remaining* budget in `grpc-timeout`. The server reads it via `context.time_remaining()`. Because only the remainder travels, a multi-hop call tree shares one end-to-end budget — that is deadline propagation.

- Q12. Walk through deadline propagation across three hops.** Client budgets 500 ms; hop 1 receives it, spends 40 ms, and calls hop 2 with `timeout=context.time_remaining()` $\approx$ 460 ms; hop 2 repeats to hop 3. Any hop may also cap the budget tighter (a stricter local policy), but must never widen it. When the budget expires, the innermost straggler surfaces `DEADLINE_EXCEEDED` and every enclosing call unwinds with the same code.
- Q13. Per-attempt vs. overall deadline — how do retries interact?** All retry attempts draw from the same overall deadline, spaced by the configured backoff. If the overall deadline is smaller than (attempts $\times$ expected attempt time + backoff), later attempts are dead on arrival. Set overall deadlines from the p99 of a *full* retry sequence, and remember gRPC only retries when it can prove the RPC never reached the server logic (or the method is idempotent).
- Q14. What is gRPC metadata and what is the `-bin` suffix?** Metadata is the name/value pairs carried in HTTP/2 header and trailer blocks — the gRPC equivalent of HTTP headers. Keys are lowercase ASCII; keys ending in `-bin` carry arbitrary binary (base64 on the wire, raw bytes in the API). Rich error details and auth tokens ride here; `grpc-*` keys are reserved.
- Q15. How do you attach authentication to gRPC calls?** As metadata — conventionally `authorization: Bearer <token>` — applied by a *client interceptor* so no call site can forget it, over TLS in production. The server validates in a *server interceptor* and rejects with `UNAUTHENTICATED` before business logic runs. Channel credentials (TLS) secure the connection; call credentials (the token) authenticate the RPC; gRPC composes the two.
- Q16. What is an interceptor and where would you use one?** Middleware for gRPC: client interceptors wrap outbound calls (auth, correlation IDs, client metrics, hedging), server interceptors wrap inbound handlers (authz, logging, error mapping, rate limiting). They come in unary and stream flavors for each side; streaming interceptors wrap iterators rather than single messages. Listing 4 and Listing 3 are working examples of both sides.

- Q17. How do you configure retries in gRPC?** Via a JSON *service config* on the channel: `retryPolicy` with `maxAttempts`, exponential backoff (`initialBackoff`, `backoffMultiplier`, `maxBackoff`), and `retryableStatusCodes` — typically only `UNAVAILABLE`. Hedging (`hedgingPolicy`) is the latency twin: fire a duplicate after a delay, take whichever answers first. Both need server-side throttling awareness (`retryThrottling`) to avoid retry storms.
- Q18. Which status codes are safe to retry?** `UNAVAILABLE` is the canonical retryable code (transient, and the framework retries only when the attempt provably never ran). `ABORTED` is retryable for transactional conflicts at the application level. `RESOURCE_EXHAUSTED` after the server-supplied delay. Everything else — `INVALID_ARGUMENT`, `NOT_FOUND`, `PERMISSION_DENIED`, `DEADLINE_EXCEEDED` — retrying unchanged is a bug amplifier.
- Q19. `INVALID_ARGUMENT` vs. `FAILED_PRECONDITION` vs. `OUT_OF_RANGE`?** `INVALID_ARGUMENT`: the request is malformed regardless of system state (bad enum, missing field). `FAILED_PRECONDITION`: the request is fine but system state forbids it (deleting a non-empty warehouse zone). `OUT_OF_RANGE`: a value lies outside a defined valid interval (seeking past the end of a stream). The distinctions matter because clients automate on them: only `FAILED_PRECONDITION` hints “fix state, then retry.”
- Q20. `UNAUTHENTICATED` vs. `PERMISSION_DENIED`?** `UNAUTHENTICATED` (16): credentials missing, expired, or invalid — re-authentication may succeed. `PERMISSION_DENIED` (7): identity known, access refused — re-auth will not help. Mapping to HTTP: 401 vs. 403. Clients key their token-refresh logic off this distinction, so getting it backwards causes refresh loops or stranded users.
- Q21. Design error semantics for a streaming RPC.** Three eras: errors *before* the first message can be trailers-only with full details; errors *mid-stream* terminate the stream with `grpc-status` + trailers, so the client must treat all received messages as a valid prefix and the status as authoritative; errors *per item* that should not kill the stream must be modeled *in the message* (an `oneof result {T ok; ItemError err;}`), not as statuses. Publish which statuses the method can return, attach rich details (`ErrorInfo`/`BadRequest`-style) in `-bin` trailers, and state idempotency so clients know

whether resume is safe.

- Q22. How do rich error details work on the wire?** A `google.rpc.Status` message — code, message, and repeated Any details holding typed payloads like `BadRequest`, `RetryInfo`, `QuotaFailure` — is serialized into the binary trailer `grpc-status-details-bin`. Clients unpack the typed details and act programmatically (field errors to forms, retry delays to backoff). Our chapter implements the identical pattern with a first-party `ErrorDetails` message and an `error-details-bin` trailer.
- Q23. Why does gRPC need client-side load balancing?** Because gRPC multiplexes many RPCs over one long-lived HTTP/2 connection, and L4 balancers balance *connections*, not RPCs: one connection pins all its RPCs to one backend forever. Client-side LB (resolve all backends via DNS, pick per-RPC with `round_robin`) moves balancing to RPC granularity with no extra network hop. The alternatives are a lookaside control plane (`grpc_lb`), xDS from a mesh control plane, or terminating HTTP/2 at an L7 proxy that rebalances per stream [New21].
- Q24. What is lookaside load balancing, and what is xDS?** Lookaside: the client queries a dedicated balancer service for a curated backend list, then connects directly — control traffic via the balancer, data traffic point-to-point (gRPC’s historical `grpc_lb`). xDS is the CNCF-standardized discovery API family from Envoy — LDS/CDS/RDS/EDS — through which a control plane pushes listeners, clusters, routes, and endpoints to data-plane clients; gRPC’s proxyless service-mesh mode consumes xDS natively. xDS is where the industry converged; `grpc_lb` is legacy context.
- Q25. How do `max_connection_age` and `keepalive` improve operability?** `max_connection_age` forces periodic reconnects so long-lived clients rediscover rebalanced backends — the cheap fix for L4 pinning — with a grace period to drain in-flight RPCs. Keepalive PINGs detect half-dead peers and keep NAT/firewall state alive on idle connections; client ping intervals must respect the server’s minimum or the server answers with GOAWAY (“too many pings”).
- Q26. How does flow control work for gRPC streams?** Two layers. HTTP/2 gives per-stream and per-connection credit windows: receivers grant bytes with `WINDOW_UPDATE`, so a slow consumer pauses the sender’s socket writes. gRPC adds application-level flow via your iterator pacing. The operational point: flow control

converts slow-consumer problems into *buffered-bytes* problems — bound queues (Listing 5), bound stream lengths, and watch memory, not just throughput.

- Q27. What happens on cancellation, end to end?** The client sends `RST_STREAM` for the call's stream; the transport tears down stream state; the server's `context.is_active()` flips false; well-written handlers observe it between iterations and return promptly. Cancellation is cooperative — nothing forcibly kills your handler — and the client observes code `CANCELLED`. Deadline expiry is the same machinery with `DEADLINE_EXCEEDED`.
- Q28. How would you test gRPC services without a network?** Bind the server to `localhost:0` (ephemeral port) inside a pytest fixture and exercise the real stack in-process, as Listing 6 does — you get genuine serialization, framing, deadlines, and cancellation with zero infrastructure. For pure unit tests, call the servicer methods with a fake `ServicerContext`. Reserve mocks of generated stubs for testing *callers*, never the service.
- Q29. What is server reflection and what does `grpcurl` need it for?** Reflection is an optional service (`grpc.reflection...`) that lets clients enumerate services and fetch file descriptors at run-time. `grpcurl` uses it to build requests without a local `.proto`: `grpcurl -plaintext host:port list`. Enable it in dev and staging; consider disabling or authenticating it in production because it advertises your full API surface.
- Q30. How do you expose a gRPC service to a browser?** Browsers cannot speak raw gRPC (no trailer access, no fine-grained HTTP/2 control), so: **grpc-web** — a framed protocol variant through a proxy filter (Envoy), with generated TS stubs, supporting unary and server streaming; or **HTTP/JSON transcoding** — `google.api.http` annotations on the proto, a gateway (Envoy transcoder, gRPC-Gateway) mapping REST paths to RPCs and statuses per Table 14.1. Transcoding doubles as your public REST facade over an internal gRPC core.
- Q31. When should a public API *not* use gRPC?** When any of these dominate: browser-first consumers (`grpc-web` adds a proxy and drops bidi); third-party developers who expect curl-able JSON and familiar HTTP semantics; a need for CDN/edge caching, which is built on HTTP semantics gRPC bypasses; firewalled enterprise networks that pass 443/HTTP but break long-lived HTTP/2; or

heavy reliance on web tooling (API gateways, WAFs, scanners) that speaks JSON. The staff answer adds: you can have both — gRPC internally, transcode at the edge — and the contract cost of maintaining that mapping is the real decision variable.

Q32. Design the load-balancing architecture for a 500-service gRPC mesh. Layer it: (1) xDS from the mesh control plane for discovery, policy, and subsetting — clients should not DNS-resolve 500 services themselves; (2) `round_robin` (or least-request where available) per-RPC within a locality, with locality weighting to keep traffic in-zone; (3) server-side `max_connection_age` as the safety net for any pinned path; (4) outlier detection to eject sick backends; (5) per-pod skew metrics as SLIs. State explicitly what you are balancing (RPCs), what each layer owns, and how you would detect imbalance before users do.

Q33. A teammate proposes gRPC streaming to replace Kafka for event delivery. Evaluate. Different guarantees. gRPC streams are ordered, flow-controlled, and low-latency, but ephemeral: no durable log, no replay, no consumer groups, no retention — a disconnected consumer simply misses events. Streams are right for *control-plane* and *live-tail* traffic; the log is right for *facts* you must not lose and must reprocess (Chapter 11). A defensible hybrid: Kafka for the durable spine, gRPC server-streaming as the fan-out edge — with sequence numbers and a resume offset modeled in your own messages.

Q34. How do you version a gRPC API across an organization? Prefer evolution over versioning: proto3's rules (add-only fields, `reserved` tags) let most changes ship in place (Chapter 9's discipline applies with tags instead of JSON keys). When a change is genuinely breaking, version the *package* (`northwind.telemetry.v2`) so both generations coexist on the wire, run dual servers during migration, and retire v1 with measured deprecation. Package-level versions in the `:path` make routing, metrics, and sunset enforcement trivial — this is why the version belongs in the package name from day one.

Q35. Debug: latency p99 doubles after moving gRPC behind a cloud L4 load balancer; CPU is low everywhere. Hypotheses? Ordered by prior: (1) connection pinning plus *unequal connection lifetimes* — a few hot tenants' RPCs concentrate; check per-pod RPC counts. (2) The LB's idle timeout silently reaping connections without keepalive — clients stall on dead sockets before reconnect; check for synchronized reconnect storms. (3) DNS TTL

vs. client resolver caching, so scale-outs are invisible for minutes. (4) MTU/fragmentation on the new path inflating DATA frame latency. The instrumentation answer: per-connection RPC counts, channel state transitions, and `grpc-timeout` distributions server-side.

Q36. Compare gRPC, REST+JSON, and GraphQL for an internal service-to-service call graph. REST+JSON: universal tooling and debuggability, weakest contract and heaviest payload; fine at low call rates. GraphQL: client-driven selection kills over/under-fetching at the *aggregation* edge, but its query planning cost and N+1 hazards make it wrong for machine-to-machine hot paths (Chapter 12). gRPC: strongest contract, smallest payload, first-class streaming, best fit for service-to-service — at the price of human-unfriendly wire format and LB complexity. Staff-level answers pick per *edge*, and can state the organization’s default and its exceptions.

Q37. What would you standardize in a “gRPC production checklist” for your company? Deadlines on every call with propagation through `time_remaining()`; `is_active()` checks in every stream loop; bounded streams and bounded queues; a documented status code policy with rich details in `-bin` trailers; interceptors for auth/correlation/metrics, never hand-threaded metadata; retry policy per method with idempotency annotation; client-side LB or xDS plus `max_connection_age`; keepalive tuned to the middle-box reality; health-check service for every server; reflection and `grpcurl` in non-prod; contract CI that regenerates stubs and rejects tag-reuse diffs. That list *is* this chapter, operationalized.

14.8 External Bibliography & Further Reading

- **gRPC documentation** — <https://grpc.io/docs/>: the authoritative guides for concepts, deadlines, metadata, interceptors, service config, load balancing, and the health-checking and reflection protocols [gRP24].
- **gRPC status codes** — <https://grpc.io/docs/guides/error/> and google/rpc/code.proto: the canonical code list and the HTTP mapping conventions used in Table 14.1.
- **Protocol Buffers documentation** — <https://protobuf.dev/>: the proto3 language guide, well-known types, the wire format (pair

with Chapter 4), and the officially documented evolution rules [Goo24].

- **RFC 9113, *HTTP/2*** — streams, frames, flow control, and HPACK context that gRPC builds on; Chapter 1’s companion reference [Tho22].
- **Kasun Indrasiri & Danesh Kuruppu, *gRPC: Up and Running*** (O’Reilly, 2020) — the standard book-length treatment: IDL design, all four modes in depth, production concerns, and the gRPC ecosystem.
- **Google AIP (API Improvement Proposals)** — <https://aip.dev/>: AIP-121 (resource-oriented design), AIP-122 (resource names), AIP-158 (pagination), AIP-193 (errors) — the design system behind Google’s gRPC APIs; steal it wholesale for internal consistency.
- **CNCF xDS / Envoy documentation** — <https://www.envoyproxy.io/docs/envoy/la> — the discovery-API family behind proxyless service mesh and modern gRPC load balancing.
- **grpc-web** — <https://github.com/grpc/grpc-web>: the browser bridge protocol and proxy model.
- **Martin Kleppmann, *Designing Data-Intensive Applications*** (O’Reilly, 2017) — the RPC fallacies, schema evolution, and distributed-systems framing underpinning this chapter [Kle17].
- **Sam Newman, *Building Microservices*, 2nd ed.** (O’Reilly, 2021) — where RPC fits (and does not fit) among inter-service communication styles [New21].

14.9 Capstone Projects

All capstones are offline-first, use the Northwind Robotics universe, and run on Windows/PowerShell with only this chapter’s dependencies.

14.9.1 Capstone 1 [junior] — Unary Status Service with Deadline Discipline

Objective. Stand up a single-RPC gRPC service and prove you understand deadlines from both ends of the wire.

Inputs/Datasets. Listing 1’s `telemetry.proto` reduced to `GetRobotStatus` only; the synthetic fleet table (RB-1001...RB-1003, RB-SLOW).

Steps.

1. Trim the proto, regenerate stubs, implement server and client.
2. Set explicit deadlines at every call site; add the `_require_budget` server check.
3. Demonstrate `DEADLINE_EXCEEDED` against `RB-SLOW` with a 50 ms client budget, and success with a 5 s budget.

Acceptance Criteria.

- ☐ `pytest` suite passes with happy-path, unknown-robot, and deadline-exceeded cases.
- ☐ No call site lacks a `timeout=`; no handler lacks a budget check.
- ☐ `grpcurl` (or a printed descriptor dump) shows the service shape.

Stretch Goals. Add `grpc.keepalive_time_ms` and log channel state transitions; measure call latency at the client interceptor.

14.9.2 Capstone 2 [mid] — Server-Streaming Telemetry Monitor

Objective. Build a resilient streaming consumer that respects cancellation, bounds, and flow control.

Inputs/Datasets. `StreamTelemetry` from Listing 1; synthetic frame generator from Listing 2.

Steps.

1. Implement the server stream with `max_frames`, `period_ms`, `is_active()` checks.
2. Build a CLI monitor that prints frames and cancels cleanly on `KeyboardInterrupt` (`call.cancel()`), verifying the server stops producing.
3. Add a client-side rendering budget: drop-and-count frames when the renderer falls behind instead of buffering unboundedly.

Acceptance Criteria.

- ☐ Cancellation test passes: after `cancel()`, the server handler exits within one period.
- ☐ Server never emits more than `max_frames`; client drop counter is observable.

- Deadline on the stream is enforced and tested.

Stretch Goals. Add a heartbeat frame every n periods; detect a stalled stream client-side via per-frame timeout.

14.9.3 Capstone 3 [mid] — Bulk Ingestion with Rich Validation Errors

Objective. Client-streaming ingestion where bad data produces *actionable* errors, not stack traces.

Inputs/Datasets. `UploadTelemetry`, `ErrorDetails`, and `FieldViolation` from Listing 1; a synthetic file of 1,000 frames, 5% deliberately invalid (unknown robots, out-of-range battery).

Steps.

1. Implement upload validation and the error-mapping interceptor (Listing 3).
2. Extend `ErrorDetails` with `first_bad_index` so clients can resume after the failure point.
3. Write the client-side decoder that renders violations as a table.

Acceptance Criteria.

- Mid-stream abort yields `INVALID_ARGUMENT` plus decoded per-field violations and the correlation ID end-to-end.
- A fix-and-resume run accepts exactly the valid 95%.
- All errors are asserted via `.code()`, never via message-string matching.

Stretch Goals. Port the details payload to `google.rpc.Status` + `grpc-status-details-bin` using `googleapis-common-protos`, and show the wire equivalence.

14.9.4 Capstone 4 [senior] — Bidirectional Control Plane with a Full Interceptor Stack

Objective. A production-shaped bidi service: windowed flow control, auth + logging + metrics interceptors on both sides, and a retry policy that provably cannot duplicate control commands.

Inputs/Datasets. `ControlLoop` (Listing 1), Listings 3–5; a scripted scenario of 500 deterministic commands including estops.

Steps.

1. Add sequence-number idempotency server-side (duplicate `sequence` $\Rightarrow$ replay cached ack).
2. Implement a metrics interceptor (counts, latency histogram buckets per method) and a server authz interceptor rejecting a synthetic bad token with `UNAUTHENTICATED`.
3. Build a two-hop deadline-propagation harness: a proxy servicer that forwards `ControlLoop` to a backend using `time_remaining()`, and show a 300 ms end-to-end budget expiring cleanly at both hops.

Acceptance Criteria.

- ☐ 500-command run: exactly-once *effects* despite forced client retries; ack order preserved.
- ☐ Metrics interceptor emits per-cardinality counters without touching business code.
- ☐ Deadline test shows the downstream hop observing a *smaller* remaining budget than the client set.
- ☐ Bad-token calls fail before any servicer code runs.

Stretch Goals. Add hedging to unary reads only, and measure the p99 delta against the retry baseline with a deterministic latency-injection hook.

14.9.5 Capstone 5 [staff] — Edge Gateway: Transcoding plus Load-Balancing Postmortem Lab

Objective. Reproduce the war story of Section 14.5 in miniature, fix it three ways, and expose the service safely to JSON clients — then write the incident report.

Inputs/Datasets. The full telemetry service; three server processes on loopback ports simulating “pods”; a connection-pinning harness that dials through a single fixed address; a synthetic load script (fixed seed, fixed schedule).

Steps.

1. Demonstrate the hotspot: all clients dialed during a one-pod window; show per-pod RPC counts 10 $\times$ skewed.
2. Apply, and measure separately: `max_connection_age` recycling; client-side `round_robin` over all three pod addresses; and a least-loaded selection policy you design.

3. Add a JSON transcoding shim (a small HTTP server mapping `GET /v1/robots/{id}/status` to `GetRobotStatus` and statuses per Table 14.1) with rich details rendered into an RFC-9457-shaped body.
4. Write the postmortem: timeline, mechanism, why restart-and-scale made it worse, and the three fixes with measured before/after skew.

Acceptance Criteria.

- ☐ Pinned-phase skew $\geq 10\times$; each fix reduces max/median RPC ratio below $1.5\times$ in the harness.
- ☐ JSON edge returns correct HTTP codes for `NOT_FOUND`, `DEADLINE_EXCEEDED`, and `INVALID_ARGUMENT` cases, with details intact.
- ☐ Postmortem identifies the balance unit (connections vs. RPCs) as the root cause and states the per-pod SLI that would have paged earlier.
- ☐ Everything runs offline with `pytest -q` green and a single PowerShell driver script.

Stretch Goals. Replace the hand-rolled `round_robin` with a DNS-name-resolver-based config; add the `grpc.health.v1` service and gate the shim's routing on `SERVING`.

14.10 Python Dependencies Appendix

requirements.txt

```
1  grpcio>=1.62           # gRPC runtime: channels, server,
    interceptors, statuses
2  grpcio-tools>=1.62     # bundled protoc + Python/gRPC
    plugins for codegen
3  protobuf>=4.25        # message runtime used by
    generated *_pb2 modules
4  pytest>=8             # test runner for the offline
    suite
```

Install from PowerShell (offline-friendly once wheels are cached):

```
1  python -m venv .venv
2  .\.venv\Scripts\Activate.ps1
3  pip install -r requirements.txt
```

14.10.1 `grpcio`

Description. The gRPC runtime for Python: channels and connections, the threaded server, all four interceptor interfaces, status codes, metadata, deadlines, and the service-config/retry machinery.

Why included. It is the subject of the chapter; every listing depends on it directly.

Alternatives. *Twirp* (Twitch) — a simpler HTTP/1.1+JSON/proto RPC with a fraction of the features (no bidi streaming, no HTTP/2 multiplexing) and a fraction of the operational complexity. *Connect-RPC* (Buf) — gRPC-compatible but also speaks plain HTTP+JSON, easing browser and curl access; a strong modern choice when one contract must serve both worlds. *Apache Thrift* — the previous generation: own transport, own IDL, mature, but no HTTP/2 semantics and a smaller mindshare today. *Plain HTTP/2* (e.g. `httpx` + handwritten framing) — you reimplement stubs, deadlines, and statuses yourself; educational, not advisable.

Installation. `pip install grpcio` (wheels for CPython 3.11+ on Windows; no compiler needed).

14.10.2 `grpcio-tools`

Description. Bundles the `protoc` compiler, the Python and gRPC-Python plugins, and the well-known `.proto` files, exposed as `python -m grpc_tools.protoc`.

Why included. Code generation is core to the gRPC workflow; bundling keeps the pipeline hermetic and offline — no separate `protoc` download.

Alternatives. A standalone `protoc` binary plus `grpc_python_plugin`; or the Buf toolchain (`buf generate`), which adds linting and breaking-change detection and is the better choice at organization scale.

Installation. `pip install grpcio-tools`; keep its version in lockstep with `grpcio`.

14.10.3 `protobuf`

Description. The Protocol Buffers runtime that generated `_pb2` modules import: message classes, serialization, well-known types, reflection over descriptors.

Why included. Transitive but load-bearing: pin it explicitly because generated code validates its runtime version, and Chapter 4’s hand-rolled codec comparisons use it as the reference implementation.

Alternatives. None within the `protobuf` ecosystem (the runtime is mandatory for generated code); the *format-level* alternatives — `MessagePack`, `Avro`, `CBOR` — are analyzed in Chapter 4.

Installation. `pip install protobuf`; match the major version `grpcio-tools` generated against.

14.10.4 `pytest`

Description. The test runner and fixture system used for the offline suite (Listing 6).

Why included. Consistent with every prior chapter; fixtures make the in-process server pattern (bind port 0, yield stub, stop server) three lines long.

Alternatives. `unittest` (stdlib, zero dependency — the fixture becomes `setUp/tearDown`); `nose2` (legacy, not recommended).

Installation. `pip install pytest`.

14.11 Chapter Summary & Key Takeaways

- gRPC = Protocol Buffers IDL + protobuf binary encoding + HTTP/2 transport + generated stubs. The `.proto` is the contract; both ends are artifacts of it.
- On the wire, one RPC is one HTTP/2 stream; every message carries a 5-byte prefix (1-byte compression flag + 4-byte big-endian length); `:path` is `/package.Service/Method`; the HTTP status is 200 and the real outcome is the `grpc-status` trailer — collapsed into a trailers-only HEADERS frame for early failures.
- Four modes — unary, server streaming, client streaming, bidirectional — are declared in the IDL and differ in cancellation and flow-control obligations; bound every stream and poll `is_active()`.
- Deadlines propagate: `grpc-timeout` carries the *remaining* budget hop to hop. Set a deadline on every call; re-check `time_remaining()` after stalls; size overall budgets for the whole retry sequence.
- The 17 status codes are the error model; rich details travel in `-bin` trailers (`google.rpc.Status` or your own typed payload). Branch on `.code()`; retry only UNAVAILABLE-class failures of idempotent-or-unreached calls.
- Interceptors — client and server, unary and stream — are where auth, correlation IDs, logging, metrics, retry, and error mapping live. Business logic never threads metadata by hand.
- Operationally: L4 balancers pin gRPC to single backends (balance

RPCs, not connections — client-side LB, xDS, or `max_connection_age`); keepalive detects dead peers; reflection + `grpcurl` restore curl-ability; `grpc-web` or JSON transcoding bridge to browsers; `grpc.health.v1` is the readiness contract.

- gRPC is for hot internal paths between services you control. Public, browser-first, cache-dependent APIs remain HTTP/JSON territory — transcode at the edge if one contract must serve both.

14.12 Quick-Reference Card

RPC Mode Selection

Mode	Pick when	Watch out for
Unary	Single request/reply; command or query.	Missing deadlines; retry only if idempotent.
Server streaming	Feeds, tailing, large incremental results.	Abandoned streams; bound with max count; check <code>is_active()</code> .
Client streaming	Bulk ingestion, uploads, aggregates.	Mid-stream validation must abort with details; buffer bounds.
Bidirectional	Control sessions, interactive protocols.	No implicit req/resp pairing; app-level backpressure queue.

Status Codes You Must Know Cold

Deadline & Cancellation Checklist

- ☐ Every call site sets `timeout=`; no infinite defaults.
- ☐ Every hop forwards `context.time_remaining()`.
- ☐ Overall budget $\geq$ attempts $\times$ attempt p99 + backoff.
- ☐ Handlers re-check budget after any stall.
- ☐ Stream loops poll `context.is_active()`.
- ☐ Clients `cancel()` abandoned streams explicitly.
- ☐ Retry policy per method; UNAVAILABLE only; idempotency known.

Code	Meaning	Retry?
OK (0)	Success.	—
CANCELLED (1)	Caller cancelled.	Only if safe to redo.
INVALID_ARGUMENT (3)	Malformed request.	No.
DEADLINE_EXCEEDED (4)	Budget expired.	No (budget already spent).
NOT_FOUND (5)	Resource absent.	No.
PERMISSION_DENIED (7)	Known caller, refused.	No.
RESOURCE_EXHAUSTED (8)	Quota/rate limit.	After server delay.
FAILED_PRECONDITION (9)	State forbids op.	After fixing state.
ABORTED (10)	Concurrency conflict.	Yes, whole operation.
UNIMPLEMENTED (12)	Method unknown here.	No.
INTERNAL (13)	Server bug.	No.
UNAVAILABLE (14)	Transient unavailability.	Yes (canonical).
UNAUTHENTICATED (16)	Bad/missing credentials.	After re-auth.

Table 14.2: Abridged; the full 17-code table with HTTP mappings is Table 14.1.

Wire Cheat Sheet

```

1 message = [compressed:1B][length:4B big-endian][
  protobuf payload]
2 :path    = /package.Service/Method      content-type =
  application/grpc
3 status   = grpc-status trailer (HTTP :status is 200
  even on errors)
4 details  = <key>-bin trailer (raw bytes; base64 on the
  HTTP/2 wire)
5 deadline = grpc-timeout header, remaining budget, unit
  suffix H/M/S/m/u/n

```


14.13 Standardized Evidence Addendum

14.13.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 14.3: Minimum benchmark table to keep per chapter.

14.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

14.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 14.4: Operational budget template for this chapter’s technique.

14.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0

```

```
6 fallback_strategy: "fail_fast"  
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

14.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

14.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

14.17 Cross-Chapter Decision Card

14.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 14.5: Decision card to complete per chapter.

- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

14.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 15

Rate Limiting, Throttling & Quota Systems

Every API has a capacity, whether or not anyone has measured it. A service that cannot say *no* does not have unlimited generosity; it has an unpriced contract with catastrophe, and the invoice arrives as a latency spike, a connection-pool exhaustion, or a 3 a.m. page. Rate limiting is the discipline of saying no *early, cheaply, predictably, and informatively* — before the expensive parts of your stack have done work they will have to abandon. This chapter treats it as what it is: a control system with mathematics, a distributed-systems problem with race conditions, and a client-facing contract with wire semantics. All three must be right, and the third is the one most often forgotten.

We begin by separating five ideas that casual usage conflates: *rate limiting* (short-window admission control per identity), *throttling* (shaping the flow, usually by delaying rather than rejecting), *quotas* (long-window budgets that bill or ration usage over hours, days, or billing periods), *concurrency limits* (how many requests may be *in flight*, not how many may start), and *load shedding* (the system protecting *itself*, dropping work by priority when capacity is exceeded regardless of who sent it). Each answers a different question, and mature platforms run all five simultaneously.

The algorithmic core is derived, not just stated. We prove that the fixed window counter admits up to $2N$ requests inside a window-length interval (Theorem 15.1) and that this bound is tight; we derive the sliding window counter’s weighted estimator and bound its error strictly by the previous window’s count (Proposition 15.3); we prove the token bucket’s burst-plus-rate envelope $A(\Delta t) \leq B + r \Delta t$ (Proposition 15.4); and we derive GCRA — the Generic Cell Rate Algorithm that powered ATM networks and now powers `nginx`, Envoy, and most serious gateways — from its theoretical arrival time update rule, proving its exact equivalence to the token bucket (Proposition 15.5). Every claim is backed by an executable test: the 2x

boundary burst is not an anecdote here, it is a passing pytest.

Distribution changes everything and nothing. The algorithms survive replication; the *state* does not. We dissect the read-modify-write race that makes naive multi-replica limiters over-admit, show why a Redis sorted set driven by a single Lua script is atomic by construction, and map the cell-based architectures (one global cell, regional cells, local limiters with gossip) onto their consistency–latency–blast-radius trade-offs, including the fail-open versus fail-closed decision you must make *before* Redis dies, not during. Client-facing semantics get equal rigor: 429 with **Retry-After** in both its legal forms, the IETF **RateLimit-*** header fields draft, 503 for shedding, and quota endpoints — because a limit a developer cannot observe is a limit they will route around [Pol24; Fie22a]. We close with multi-dimensional limiting: per-key, per-IP, per-endpoint, cost-weighted budgets, adaptive AIMD limits, and priority-aware shedding, all set in the Northwind Robotics fleet-telemetry universe.

Key Concept

A rate limiter is a **control system**, not a gate. Its setpoint is capacity, its sensor is measured load, its actuator is admission, and its contract with clients is the headers it emits. Choose the algorithm by its burst and memory envelope (proved, not vibes), keep the shared-state update atomic (one Lua script, one cell), and never reject silently: a 429 without **Retry-After** is an outage with extra steps.

15.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Distinguish rate limiting, throttling, quotas, concurrency limits, and load shedding by the question each answers, the time window each governs, and the failure each prevents; identify which of the five a given production incident actually required.
- LO-2.** Derive, not just describe, the five classic admission algorithms — fixed window, sliding window log, sliding window counter, token bucket, and GCRA — including the fixed-window 2x boundary burst (Theorem 15.1), the sliding-counter error bound (Proposition 15.3), the token-bucket envelope (Proposition 15.4), and the GCRA TAT update rule with its token-bucket equivalence (Proposition 15.5).

- LO-3.** Implement all five algorithms in typed Python 3.11 against an injected virtual clock, and prove their documented behaviors with deterministic, offline unit tests.
- LO-4.** Explain why naive read-modify-write limiting races across replicas, and implement an atomic distributed sliding-window limiter on Redis sorted sets driven by a single Lua script, verified offline against fakeredis under a thread storm.
- LO-5.** Design the failure policy for a limiter outage: fail-open, fail-closed, or degraded-local, selected per endpoint class from an explicit decision matrix rather than from adrenaline.
- LO-6.** Emit correct client-facing semantics: 429 with `Retry-After` (delay-seconds or HTTP-date), the IETF `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `RateLimit-Policy` fields, 503 for load shedding, and quota introspection endpoints [Pol24; Fie22a].
- LO-7.** Build FastAPI middleware enforcing per-API-key limits with accurate headers across window boundaries and a tested fail-open/fail-closed switch [Ram24].
- LO-8.** Compose multi-dimensional limits — per-key, per-IP, per-endpoint, cost-weighted, and adaptive (AIMD) — and prioritize critical over batch traffic under pressure.

Prerequisites

This chapter assumes Chapters 0–7: the HTTP request/response model and header semantics (Chapter 2), consuming APIs and reading error responses in Python (Chapter 3), REST resource modeling (Chapter 4), JSON payloads (Chapter 5), API-key authentication basics (Chapter 6), building and testing FastAPI services with `TestClient` (Chapter 7), and structured error responses (Chapter 8). Comfort with Big-O notation and elementary algebra (floor functions, geometric bounds) is assumed; every derivation is self-contained. The GraphQL cross-reference in the interview vault points forward to Chapter 13 but is not required here.

Setup

All listings run offline on Windows with Python 3.11+. One-time setup from PowerShell in a scratch directory:

```
1 python -m venv .venv
```

```
2 .\venv\Scripts\Activate.ps1
3 pip install fakeredis fastapi httpx pytest
```

`fakeredis` emulates Redis (including Lua script execution) entirely in-process, so the “distributed” listings run with no server, no containers, and no network; Section 15.10 notes what changes when you point the same code at a real `redis>=5` deployment.

15.2 Deep Technical Mechanics & Architectural Concepts

15.2.1 Why Limit at All: Four Distinct Failures, Five Distinct Tools

Rate limiting is not one technique but a family, and most production incidents involving “rate limits” are actually failures to distinguish its members. Four pressures make admission control non-optional:

1. **Capacity protection.** Every downstream — connection pool, worker count, database, a partner API — has a knee beyond which throughput *falls* as offered load rises (retry storms amplify this). Rejecting cheaply at the edge preserves the expensive interior for requests that will complete.
2. **Fairness.** Tenants share capacity. Without per-identity accounting, one runaway client becomes everyone’s outage; the noisy-neighbor problem is a limiting problem.
3. **Cost control.** Egress, LLM tokens, metered third-party calls, and report renders all have marginal cost. An unpriced API is a subsidy for whoever automates against it first.
4. **Abuse prevention.** Credential stuffing, scraping, and inventory hoarding are economic attacks; raising the attacker’s cost per attempt is the defense of first resort (Chapter 17).

The five tools map onto five different questions:

Definition 15.1 (Rate limit). A per-identity admission rule over a *short* window (seconds to minutes): “*how fast may this caller start requests right now?*” Enforced statefully per key; rejection is routine, cheap, and expected to be retried by well-behaved clients.

Definition 15.2 (Throttling). *Shaping* rather than rejecting: delaying, dequeuing, or degrading admitted work so the system’s *processing* rate stays at target. A leaky bucket *as a queue* throttles; a 429 *limits*. Throttling costs latency; limiting costs a rejection.

Definition 15.3 (Quota). A *long-window* budget (hour, day, billing month) that rations aggregate consumption: “*how much may this customer use in total?*” Quotas are business constructs — they encode plans, tiers, and cost — whereas rate limits are protection constructs. A client can be inside its monthly quota and still be rate-limited this second.

Definition 15.4 (Concurrency limit). A bound on requests *in flight*, not requests started per unit time. Ten requests that each hold a thread for 30 seconds stress a service more than a hundred 5-millisecond requests; only a concurrency limiter (or bulkhead, Chapter 16) sees the difference. Little’s law connects them: $\text{concurrency} = \text{throughput} \times \text{latency}$.

Definition 15.5 (Load shedding). Self-protection independent of identity: when the system is saturated, decline work — by priority class, by cheapness to serve, or at random — to keep the admitted majority healthy. Shedding answers “*which work do we drop when we cannot do it all?*”; the honest response is 503 with **Retry-After** [Fie22a].

Warning

Teams routinely deploy *one* of the five and believe they have the other four. A per-key rate limit does nothing against a legitimate, in-limit client whose requests each take 45 seconds (need: concurrency limit); a monthly quota does nothing against a 30-second burst that melts the connection pool (need: rate limit); neither protects you from a thundering herd of anonymous traffic (need: shedding). The audit question is never “do we have rate limiting?” but “which of the five questions can this system currently answer?”

15.2.2 Fixed Window Counter — and Its Provable 2x Flaw

The fixed window counter is the limiter everyone writes first: partition time into windows of length W anchored at $t = 0$, keep one integer counter per key per window index $\lfloor t/W \rfloor$, and admit while the counter is below the limit N . It is $O(1)$ memory, trivially shardable, and its **RateLimit-Reset** semantics are crisp (the window boundary). It is also wrong at the edges, and the wrongness is a theorem, not a folklore warning.

Theorem 15.1 (Fixed-window boundary burst). A fixed-window limiter configured for N requests per window W can admit $2N$ requests inside a single real-time interval of length at most W — indeed, inside an interval of length 2ε for any $\varepsilon \in (0, W/2]$ — and $2N$ is tight.

Proof. Let kW be any window boundary. Fire N requests at time $t_0 = kW - \varepsilon$: all fall in window $k - 1$, counter reaches N , all admitted. Fire N more at $t_1 = kW + \varepsilon$: window index has rolled to k , the counter reset to zero, and all N are admitted. The $2N$ admissions all lie in $[t_0, t_1]$, an interval of length $2\varepsilon \leq W$. For tightness: any interval of length W intersects at most two fixed windows, and each contributes at most N admissions, so no such interval can contain more than $2N$. $\square$

The practical reading: a client — or a synchronized fleet of mobile clients retrying after a connectivity blip, which is how this fires in real life — can double its nominal rate *every* window, forever, by straddling boundaries. Worse, the burst can be concentrated in an arbitrarily small ε , so the instantaneous rate is unbounded: $2N$ requests can arrive within milliseconds of each other. If your limit exists to protect a connection pool or a partner API with a per-second cap, the fixed window does not enforce the thing you think it enforces. Listing 15.3.2 proves Theorem 15.1 as an executable test (20 admissions in a 2-millisecond interval under a “10 per second” limit).

15.2.3 Sliding Window Log: Exactness at $O(N)$ Memory

The sliding window log removes the boundary entirely: store the timestamp of every admitted request for the key, and on each arrival at time t evict timestamps $\leq t - W$; admit if fewer than N remain. The window now *slides with the caller*, so no boundary exists to straddle, and the invariant is exact:

Proposition 15.2 (Sliding-log exactness). The sliding window log admits a request at t if and only if strictly fewer than N previously admitted requests lie in $(t - W, t]$. Consequently no interval of length W ever contains more than N admissions: the $(i+N)$ -th admission is always strictly more than W after the i -th.

The cost is memory: $O(N)$ per key, per window. At $N = 10,000$ per minute across a million API keys, the log is the state. This is why the exact algorithm is rare at platform scale and why the next two algorithms exist: they trade exactness for $O(1)$ state. A second, subtler cost is that **Retry-After** becomes the time until the *oldest* in-window request expires — perfectly computable, but it means your reset hint varies with the client’s own history rather than aligning to a wall-clock boundary (Section 15.2.10).

15.2.4 Sliding Window Counter: The Weighted Approximation

The sliding window counter keeps only two integers per key: the current fixed window's count c and the previous window's count p . At time t , a fraction $f = (t - \lfloor t/W \rfloor W)/W$ of the current window has elapsed, so a fraction $1 - f$ of the previous window still overlaps the true sliding window $[t - W, t]$. Assuming the previous window's p requests were spread uniformly across it, the overlapping portion is estimated as $p(1 - f)$, giving the estimator

$$\hat{c}(t) = c(t) + p(1 - f). \quad (15.1)$$

Admit when $\hat{c} < N$. This is the algorithm Cloudflare famously analyzed at scale, finding sub-0.01% disagreement with the exact log on real traffic — but their traffic is smooth, and the estimator's quality is a bound, not a vibe:

Proposition 15.3 (Sliding-counter error bound). Let o be the true number of previous-window requests inside $[t - W, t]$. Then $|\hat{c} - c_{\text{true}}| = |p(1 - f) - o| \leq p \cdot \max(f, 1 - f) < p \leq N$.

Proof. $0 \leq o \leq p$ by definition, and the overlap region has length $(1 - f)W$. The error $|p(1 - f) - o|$ is maximized at the extremes of o : if all p requests landed in the non-overlapping part of the previous window ($o = 0$), the estimator *overcounts* by $p(1 - f)$; if all p landed in the overlap ($o = p$), it *undercounts* by $p - p(1 - f) = pf$. The larger of the two is $p \max(f, 1 - f) < p$, and $p \leq N$ because the limiter was enforcing during the previous window. $\square$

Proposition 15.3 tells you exactly when to distrust this algorithm: error scales with p , so the estimate is worst precisely after a window that was saturated — i.e., precisely when limiting mattered. The adversarial case (all of last window's traffic bunched at its end, as a retry fleet produces) makes the counter *over-admit* early in the next window; Listing 15.3.2 exhibits exactly that over-admission as a test.

15.2.5 Token Bucket: Burst Capacity as a Conserved Quantity

The token bucket reparameterizes the problem from *counting* to *budgeting*. A bucket holds at most B tokens and refills continuously at r tokens per second; a request that finds ≥ 1 token consumes one and is admitted, else rejected. State per key is two scalars (token level, last-refill timestamp) and the update is lazy — computed on arrival, no background timers:

$$\text{tokens}(t) = \min(B, \text{tokens}(t_0) + r(t - t_0)). \quad (15.2)$$

Proposition 15.4 (Token-bucket envelope). Over any interval $[t_0, t_1]$, the number of admitted requests satisfies $A \leq B + r(t_1 - t_0)$, and the bound is tight. Hence the long-run admitted rate is at most r , while instantaneous bursts up to B are fully admitted.

Proof. Tokens are conserved: every admission removes one token, and the only source of tokens is refill at rate r into a bucket that started the interval with at most B . Tokens available in the interval $\leq B + r(t_1 - t_0)$, bounding admissions; the bound is attained by arriving exactly as tokens accumulate. Dividing by $\Delta t = t_1 - t_0$ gives average rate $\leq r + B/\Delta t \rightarrow r$ as $\Delta t \rightarrow \infty$. $\square$

Proposition 15.4 is why the token bucket is the default answer in interviews and in gateways: one parameter pair (r, B) independently controls the *sustained* rate and the *burst* tolerance, and the envelope is the tightest possible for any limiter with those two properties (any limiter that admits a burst of B and sustains r must admit at least this envelope). Figure 15.1 shows the mechanics.

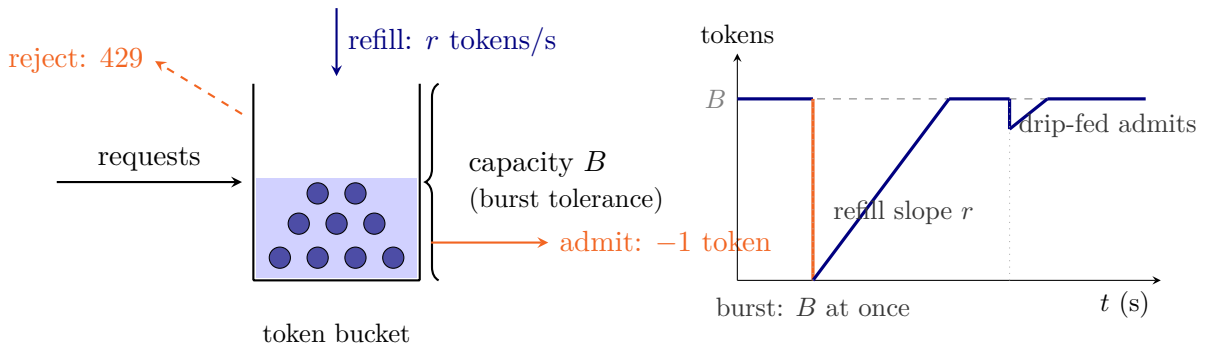


Figure 15.1: Token bucket mechanics. Left: refill at r tokens/s into a bucket capped at B ; admission consumes one token, empty buckets reject. Right: token level over time — a full bucket absorbs a burst of B instantly (vertical drop), refills linearly at r , then admits only as fast as tokens drip in. The area discipline of Proposition 15.4 is visible: burst height B , sustained slope r .

15.2.6 Leaky Bucket: Meter versus Queue

“Leaky bucket” names two different machines, and conflating them in a design review is a reliable way to build the wrong one. **As a meter**, the leaky bucket is a bucket that *drains* at a constant rate and overflows (rejects) when inflow exceeds outflow — this is precisely GCRA, derived next, and it is what `nginx`’s `limit_req` implements. **As a queue**, it is a FIFO buffer of capacity Q : requests enqueue, and a worker drains the queue at exactly r per second. The two have opposite burst postures. The

meter admits bursts (up to its tolerance) and enforces only the average; the queue *destroys* bursts entirely, presenting a perfectly smooth r -per-second stream downstream at the cost of queueing delay up to Q/r seconds — and of tying up a connection or queue slot per waiting request. Use the meter to protect a *rate*; use the queue to protect a *downstream that panics at jitter* (a legacy mainframe interface, a partner with hard pacing). The queue variant is throttling in the strict sense of Definition 14.2: it shapes rather than rejects, until the queue itself fills.

15.2.7 GCRA: One Timestamp to Rule Them All

The Generic Cell Rate Algorithm, standardized for ATM traffic policing and rediscovered by every gateway author since, reduces rate limiting to a single timestamp per key. Parameterize by the *emission interval* $T = 1/r$ (the minimum spacing between conforming requests at the sustained rate) and a *burst tolerance* τ . State is the *theoretical arrival time* TAT: when the next request would arrive if the client were perfectly paced. On an arrival at t_a :

$$\text{reject if } t_a < \text{TAT} - \tau; \quad \text{else admit and } \text{TAT} \leftarrow \max(t_a, \text{TAT}) + T. \quad (15.3)$$

Read it operationally. The condition says: a request may arrive up to τ *early* relative to the schedule; arriving earlier than that means the client has consumed more burst credit than exists. The update says: each admission pushes the schedule later by one emission interval, but if the client was idle ($t_a > \text{TAT}$) the schedule first re-anchors to the present — idle time does not accrue credit beyond τ , because $\max(t_a, \text{TAT})$ discards any schedule that has fully lapsed. Rejection yields a ready-made **Retry-After**: $\text{TAT} - \tau - t_a$.

Proposition 15.5 (GCRA $\equiv$ token bucket). GCRA with emission interval T and tolerance τ admits exactly the same request streams as a token bucket with rate $r = 1/T$ and capacity $B = 1 + \tau/T$.

Proof sketch. Define the token level from GCRA state as $L(t) = \min(1 + \tau/T, 1 + (t + \tau - \text{TAT})/T)$. The GCRA admission condition $t_a \geq \text{TAT} - \tau$ is algebraically $L(t_a) \geq 1$. On admission, $\text{TAT} \leftarrow \max(t_a, \text{TAT}) + T$ decrements L by exactly 1 (the max clamps L at capacity, matching the bucket's $\min(B, \cdot)$). Between arrivals, TAT is constant and L grows at rate $1/T = r$, matching Equation (15.2). Same state dynamics, same decisions; Listing 15.3.2 verifies the equivalence on a seeded adversarial stream. $\square$

Why keep both? The token bucket parameterizes as humans think (*rate* and *burst size*); GCRA parameterizes as schedulers think (*spacing* and *jitter tolerance*) and stores its state as one timestamp that serializes

trivially into a single Redis field — no two-field tuple, no lazy-refill ordering hazards. Most serious gateways (nginx, Envoy, Kong’s default) are GCRA underneath whatever their configuration calls it.

15.2.8 Choosing an Algorithm

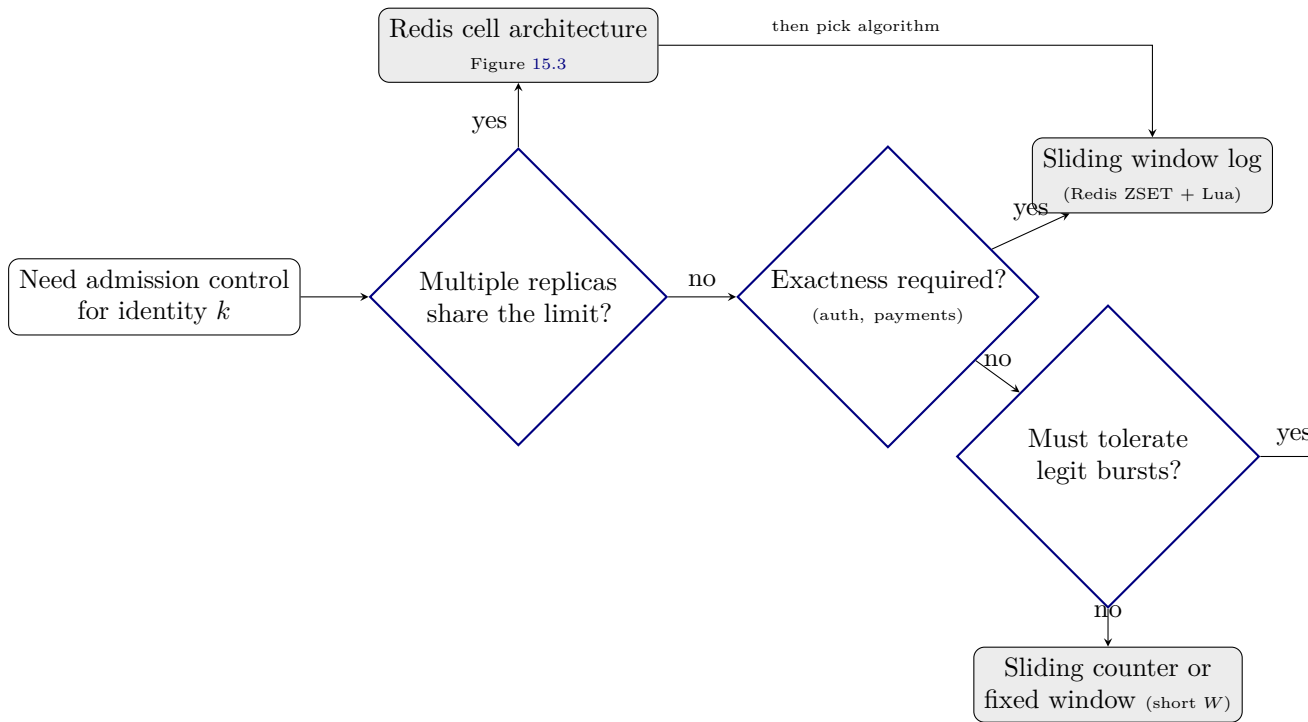


Figure 15.2: Algorithm selection flow. The first question is topological (shared state?), the second semantic (exactness?), the third behavioral (bursts?). “No bursts tolerated” with $O(1)$ memory means accepting the fixed or sliding counter’s window artifacts consciously.

15.2.9 Distribution: The Race, the Cell, and the Failure Policy

A rate limiter’s algorithm survives replication; its *state* does not. The moment two replicas enforce one logical limit, the question stops being “which algorithm?” and becomes “where does the counter live, and who can see it when?”

The read-modify-write race

The naive distributed fixed window — **GET** the counter, compare, **INCR** — over-admits under any concurrency at all. With limit N and counter at

$N - 1$, two requests landing on two replicas both read $N - 1$, both conclude “admit,” and the counter ends at $N + 1$; under a retry storm the overshoot grows with concurrency, not with request rate. Even the apparently atomic INCR-then-compare variant (increment first, reject when over) is subtly broken: rejected requests still *consume* counter increments, so an attacker (or a desperate retrying client) can poison a window for everyone including itself, and the counter can run to millions while admitting nothing. The general principle is the one Chapter 11 applied to writes and [Kle17] applies to all shared state: *check and mutate must be one indivisible step against the authoritative copy*.

The standard solution is Redis plus Lua: Redis executes each Lua script atomically on its single command-processing thread, so a script that evicts, counts, conditionally inserts, and sets expiry is a compare-and-set of arbitrary complexity with no client-side round trips (Listing 15.3.4). The sorted-set sliding log is the usual payload: timestamps as scores, a UUID per request as member, one ZREMRANGEBYSCORE/ZCARD/ZADD/PEXPIRE sequence per admission decision.

Cell architectures

A single global Redis (or Redis Cluster) holding all limiter state is a *global cell*: perfectly consistent, but every admission pays a network round trip and the cell is a blast-radius amplifier — when it dies, every replica’s limiter dies simultaneously (Section 15.2.9). The alternatives are points on a consistency–latency frontier:

- **Global cell.** Strongest semantics, highest per-request latency (colocate the cell with the edge tier to keep it sub-millisecond), largest correlated-failure domain. Right answer for low-QPS, high-stakes limits: login attempts, payment attempts, password reset.
- **Regional cells.** One cell per region; a key’s limit is enforced only within the region serving it. A roaming client gets up to $R \times$ its nominal limit across R regions — usually acceptable for abuse control, unacceptable for billing-grade quotas. Mitigate by pinning keys to home cells via consistent hashing.
- **Local in-process limiters with periodic sync.** Each replica enforces limit/ R locally (or a gossiped/share-weighted fraction) and reconciles asynchronously. Lowest latency and zero hot-path dependency; over-admission is bounded by the number of replicas times the sync lag in units of the window. This is essentially how Cloudflare’s edge limiting works: approximate at the edge, exact nowhere, correct *enough* everywhere. The error budget for this design is Proposition 15.3

thinking applied to topology instead of time.

Consistent hashing ties the pieces together: route limiter state for key k to cell $h(k) \bmod C$ so that adding or losing a cell remaps only $1/C$ of keys (their counters reset — a bounded, temporary under-limit, which fails *safe* for the system and merely generous to $1/C$ of clients). Figure 15.3 shows the shape.

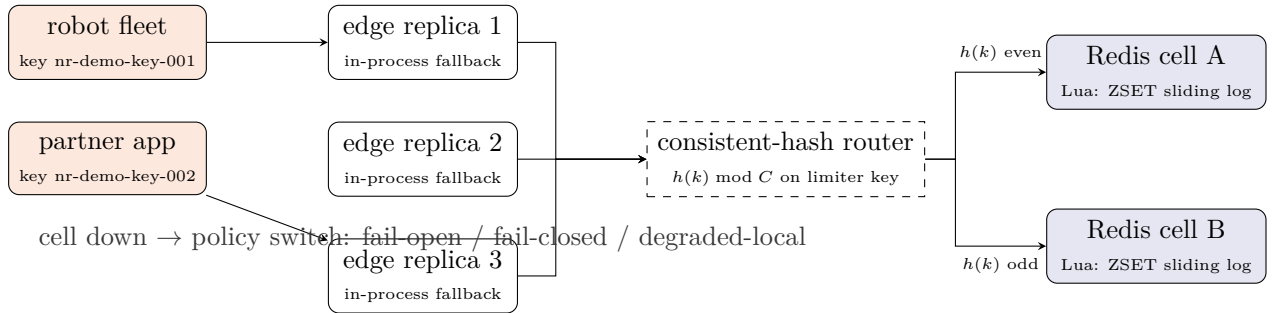


Figure 15.3: Distributed limiter architecture. Edge replicas consult one shared cell per key, chosen by consistent hashing so cell loss remaps only $1/C$ of keyspace; the Lua script makes check-and-admit atomic inside a cell. The dashed policy annotation is mandatory, not decorative: every replica must know what it does when the cell is unreachable (Section 15.2.9).

Failure modes: fail-open, fail-closed, or degrade

A limiter whose state store is unreachable cannot count. What it does next is a *business decision wearing an engineering costume*, and it must be made per endpoint class, in advance:

Warning

A fail-closed limiter with no exception is a single point of total failure: when Redis dies, the API dies with it — and because health checks usually pass through the same middleware, your load balancer may agree that everything is down and refuse to help. If you fail closed, exempt the health endpoint, keep the failure **Retry-After** short, and alert on limiter-unavailable as a severity-1 condition in its own right [Nyg18].

15.2.10 Client-Facing Semantics: Making Limits Legible

A limit developers cannot observe will be treated as a bug — theirs, then yours. The wire contract has four moving parts.

429 and Retry-After. Rejection is status 429 *Too Many Requests*. RFC 9110 defines **Retry-After** in two legal forms: a non-negative integer of *delay-seconds* (**Retry-After:** 17) or an absolute *HTTP-date* (**Retry-After:** Thu, 03 Sep 2026 21:00:00 GMT) [Fie22a]. Emit delay-seconds: HTTP-date costs you clock-sync reasoning on both sides and buys nothing a client cannot compute from the integer. Fractional waits must be ceiled — a client that retries at 0.4s when you meant 0.44s will be rejected again and will (correctly) conclude your header is noise.

The IETF RateLimit fields. The HTTPAPI working group’s draft standardizes four fields [Pol24]: **RateLimit-Limit** (the quota for the window, e.g. 100), **RateLimit-Remaining** (budget left *after* this response), **RateLimit-Reset** (seconds until the window’s quota fully resets — note: delay-seconds, *not* a timestamp; several major providers shipped epoch-seconds variants before the draft settled, so document yours), and **RateLimit-Policy** (the rule itself, e.g. "100;w=60" for 100 per 60 seconds). Emit them on *successes* too: a client that can watch **Remaining** decay can self-throttle before ever seeing a 429, which is the entire point. Headers you cannot compute truthfully (during a fail-open episode) must be omitted, never faked — Listing 15.3.5 implements exactly this rule.

503 for shedding. When the system sheds load for its own protection (not because *this client* exceeded *its* limit), the honest status is 503 *Service Unavailable* with **Retry-After** [Fie22a]. Conflating shedding with limiting (“429 for everyone during the incident”) misleads well-behaved clients into believing they did something wrong and misroutes their backoff logic: a 429 says *slow down, you specifically*; a 503 says *come back later, everyone*.

Quota endpoints. Long-window budgets deserve introspection: **GET /v1/usage** returning the period’s consumption, cap, and reset timestamp turns a billing surprise into a dashboard. Stripe-style idempotent, header-rich, self-describing limiting is what separates a platform from an obstacle course.

Example

Northwind Robotics publishes **RateLimit-Policy:** "600;w=60" on its telemetry ingest API. A fleet-management integrator watches **RateLimit-Remaining** cross 60 and linearly spaces its remaining batch across the seconds left in the window; during a regional Redis cell outage the middleware fails open *and omits* all four headers, which the integrator’s SDK reads as “limiter degraded, apply local pacing.” No page, no ticket, no surprise invoice. That is what legible limits buy.

15.2.11 Multi-Dimensional Limiting

Real systems limit along several axes at once, because no single identity captures every abuse shape:

- **Per-API-key:** the billing identity; the primary axis for fairness and plan enforcement.
- **Per-IP:** the pre-authentication axis (you must limit *before* you can know the key — otherwise login and key validation are free to attack). Behind NATs and carrier-grade NAT, one IP may be a ten-thousand-person enterprise (fairness disaster if the limit is tight) or one IP may be a botnet of thousands (useless if the limit is loose). Per-IP limits are a coarse outer net, never the only one — the case study in Section 15.5.1 is precisely this failure.
- **Per-endpoint:** a global request budget per route, defending the route's specific scarce resource regardless of which keys consume it.
- **Cost-based:** charge each request a *weight* proportional to its true cost — a bulk export might cost 10 units against a read's 1 — so the budget is denominated in work (Listing 15.3.6). GraphQL makes this mandatory rather than optional: the *query*, not the route, determines cost, so limiters there score the parsed query's field weights and depth (Chapter 13).
- **Adaptive:** AIMD — additive increase, multiplicative decrease, TCP's gift to every backpressure problem. Raise a client's limit additively while the service is healthy ($L \leftarrow L + a$), slash it multiplicatively on overload signals — p99 latency crossing target, 5xx rate, queue depth — ($L \leftarrow \beta L$, $\beta \approx 0.5\text{--}0.9$). AIMD converges to the largest limit the system can currently serve and backs off faster than it advances, which is exactly the asymmetry safety wants. It pairs naturally with adaptive *concurrency* limits on the server side.
- **Priority-aware shedding:** under pressure, admit critical traffic (health, control plane, paying interactive reads) and shed batch/analytics first. Priority must come from an authenticated attribute of the request (key tier, route class), never from a client-supplied header — a **X-Priority: high** you honor is a **X-Priority: high** your abusers send.

Expensive endpoints deserve explicit mention because they are where dimensionless limits go to die: report generation, CSV/PDF exports, bulk ingest, and search export endpoints can each cost 10–1000× a median

request. Protect them with (a) a per-endpoint cost weight, (b) a separate, tighter concurrency cap (two concurrent exports, queue the rest), and (c) where possible, an asynchronous shape instead — accept the job, return 202 with a status URL, and let the *job queue* apply its own pacing (Chapter 12).

15.3 Production-Ready Code Implementation

Everything below is typed Python 3.11, offline-first, and deterministic: every limiter takes an injected `Clock`, every timeline is seeded, and no test reads wall-clock time. The code is set in the Northwind Robotics universe — a fleet-telemetry platform whose API keys all begin `nr-demo-` (synthetic; no real credentials appear anywhere in this book). Save the listings as indicated and the printed commands reproduce every claim in this chapter.

15.3.1 Listing 14.1 — Five Algorithms, One Contract

`rate_limiters.py` implements all five admission algorithms behind a single `check(at=None) -> Decision` contract, where `Decision` carries the three things middleware will need: admit flag, remaining budget, and a `Retry-After` hint. The `VirtualClock` is the determinism backbone: tests advance time explicitly, so behavior is identical on every machine, every run.

Listing 14.1 — `rate_limiters.py`: reference implementations with injected clock.

```

1  """Reference implementations of the five classic rate-
   limiting algorithms.
2
3  Every limiter takes an injected 'Clock' so behavior
   is fully deterministic
4  under test: no wall-clock reads anywhere. Time is in
   float seconds.
5
6  Algorithms implemented:
7  ** FixedWindowLimiter — counter per floor(t /
   window) bucket
8  ** SlidingWindowLogLimiter — exact timestamp log
9  ** SlidingWindowCounterLimiter — weighted previous /
   current window estimate
10 ** TokenBucketLimiter — capacity B, refill rate r
11 ** GCRA Limiter — generic cell rate
   algorithm (leaky bucket as

```

```

12  """
13  """
14  """
15  All limiters expose the same 'allow(at=None) -> bool'
16  contract and are
17  single-process reference models; Listing 14.4 lifts the
18  sliding log to Redis.
19  """
20
21  from __future__ import annotations
22
23  import math
24  from collections import deque
25  from typing import Deque, Optional, Protocol
26
27  class Clock(Protocol):
28      """Anything that reports the current time in float
29      seconds."""
30
31      def now(self) -> float: ...
32
33  class VirtualClock:
34      """Deterministic, manually advanced clock for tests
35      and simulations."""
36
37      def __init__(self, start: float = 0.0) -> None:
38          self._t = float(start)
39
40      def now(self) -> float:
41          return self._t
42
43      def advance(self, seconds: float) -> None:
44          if seconds < 0:
45              raise ValueError("cannot move a clock
46              backwards")
47          self._t += seconds
48
49  class Decision:
50      """Result of an admission check: admit flag plus
51      header metadata."""
52
53      def __init__(self, allowed: bool, remaining: int,

```

```

51         retry_after: float) -> None:
52             self.allowed = allowed
53             self.remaining = remaining
54             self.retry_after = max(0.0, retry_after)
55
56     def __repr__(self) -> str: # pragma: no cover -
57         debugging aid
58         return (
59             f"Decision(allowed={self.allowed},
60                 remaining={self.remaining},
61                 f"retry_after={self.retry_after:.3f})"
62         )
63
64 class FixedWindowLimiter:
65     """Allow at most 'limit' requests per fixed
66         window of 'window' seconds.
67
68     Memory: O(1). Flaw: up to '2 * limit' requests
69         can be admitted inside
70         any real-time interval of length 'window' (see
71         Theorem 14.1).
72
73     """
74
75     def __init__(self, limit: int, window: float, clock
76         : Clock) -> None:
77         if limit <= 0 or window <= 0:
78             raise ValueError("limit and window must be
79                 positive")
80         self.limit = limit
81         self.window = window
82         self.clock = clock
83         self._window_index = -1
84         self._count = 0
85
86     def check(self, at: Optional[float] = None) ->
87         Decision:
88         t = self.clock.now() if at is None else at
89         index = math.floor(t / self.window)
90         if index != self._window_index:
91             self._window_index = index
92             self._count = 0
93         window_end = (index + 1) * self.window
94         if self._count < self.limit:
95             self._count += 1
96             return Decision(True, self.limit - self.

```

```

        _count, 0.0)
88     return Decision(False, 0, window_end - t)
89
90
91 class SlidingWindowLogLimiter:
92     """Exact limiter: keep the timestamp of every
93     admitted request.
94
95     Memory: O(limit) per key. Never admits more than '
96     limit' requests in
97     any interval of length 'window' -- the gold
98     standard for correctness.
99     """
100
101     def __init__(self, limit: int, window: float, clock
102     : Clock) -> None:
103         if limit <= 0 or window <= 0:
104             raise ValueError("limit and window must be
105             positive")
106         self.limit = limit
107         self.window = window
108         self.clock = clock
109         self._log: Deque[float] = deque()
110
111     def check(self, at: Optional[float] = None) ->
112     Decision:
113         t = self.clock.now() if at is None else at
114         cutoff = t - self.window
115         while self._log and self._log[0] <= cutoff:
116             self._log.popleft()
117         if len(self._log) < self.limit:
118             self._log.append(t)
119             return Decision(True, self.limit - len(self
120             ._log), 0.0)
121         oldest = self._log[0]
122         return Decision(False, 0, oldest + self.window
123         - t)
124
125 class SlidingWindowCounterLimiter:
126     """Weighted approximation of the sliding log with O
127     (1) memory.
128
129     Estimate for the true count in [t-window, t]:
130     estimate = current + previous * (1 -
131     elapsed_fraction)

```

```

123     """where elapsed_fraction is how far 't' has
        progressed into the current
124     fixed window. The absolute error versus the exact
        log is strictly less
125     than the previous window's count (Proposition 14.2)
        .
126     """
127
128     def __init__(self, limit: int, window: float, clock
        : Clock) -> None:
129         if limit <= 0 or window <= 0:
130             raise ValueError("limit and window must be
                positive")
131         self.limit = limit
132         self.window = window
133         self.clock = clock
134         self._prev_index = -1
135         self._prev_count = 0
136         self._curr_index = -1
137         self._curr_count = 0
138
139     def check(self, at: Optional[float] = None) ->
        Decision:
140         t = self.clock.now() if at is None else at
141         index = math.floor(t / self.window)
142         if index != self._curr_index:
143             self._prev_index = self._curr_index
144             self._prev_count = self._curr_count if self
                ._curr_index == index - 1 else 0
145             self._curr_index = index
146             self._curr_count = 0
147         elapsed_fraction = (t - index * self.window) /
            self.window
148         previous = self._prev_count if self._prev_index
            == index - 1 else 0
149         estimate = self._curr_count + previous * (1.0 -
            elapsed_fraction)
150         window_end = (index + 1) * self.window
151         if estimate < self.limit:
152             self._curr_count += 1
153             return Decision(True, max(0, int(self.limit
                - estimate - 1)), 0.0)
154         return Decision(False, 0, window_end - t)
155
156
157 class TokenBucketLimiter:

```

```

158         """Token_bucket: capacity 'burst' tokens,
159           refilled at 'rate' per second.
160
161         Bursts up to 'burst' are admitted instantly; the
162           long-run admitted rate
163           never exceeds 'rate' (Proposition 14.3).
164         """
165
166     def __init__(self, rate: float, burst: float, clock
167       : Clock) -> None:
168         if rate <= 0 or burst <= 0:
169             raise ValueError("rate and burst must be
170               positive")
171         self.rate = rate
172         self.burst = burst
173         self.clock = clock
174         self._tokens = float(burst)
175         self._last = clock.now()
176
177     def _refill(self, t: float) -> None:
178         if t < self._last:
179             raise ValueError("timestamps must be non-
180               decreasing")
181         self._tokens = min(self.burst, self._tokens + (
182           t - self._last) * self.rate)
183         self._last = t
184
185     def check(self, at: Optional[float] = None, cost:
186       float = 1.0) -> Decision:
187         if cost <= 0:
188             raise ValueError("cost must be positive")
189         t = self.clock.now() if at is None else at
190         self._refill(t)
191         if self._tokens >= cost:
192             self._tokens -= cost
193             return Decision(True, int(self._tokens),
194               0.0)
195         deficit = cost - self._tokens
196         return Decision(False, int(self._tokens),
197           deficit / self.rate)
198
199 class GCRA_Limiter:
200     """Generic Cell Rate Algorithm (leaky bucket used
201       as a meter).

```



```

194 Parameters: "rate" requests_per_second_sustained,
           "burst" requests
195 tolerated_instantly. Emission_interval_T = 1 / rate
           ; burst_tolerance
196 tau = (burst - 1) * T. State is a single timestamp:
           the theoretical
197 arrival_time(TAT) of the next conforming request.
198 ""
199
200 def __init__(self, rate: float, burst: int, clock:
201 Clock) -> None:
202     if rate <= 0 or burst <= 0:
203         raise ValueError("rate and burst must be
204             positive")
205     self.rate = float(rate)
206     self.emission_interval = 1.0 / rate # T
207     self.tolerance = (burst - 1) * self.
208         emission_interval # tau
209     self.clock = clock
210     self._tat = float("-inf")
211
212 def check(self, at: Optional[float] = None) ->
213 Decision:
214     t = self.clock.now() if at is None else at
215     arrival_limit = self._tat - self.tolerance
216     if t < arrival_limit:
217         return Decision(False, 0, arrival_limit - t
218             )
219     self._tat = max(t, self._tat) + self.
220         emission_interval
221     # Remaining burst: admits still possible right
222     # now before the
223     # not-before time t >= TAT - tau would be
224     # violated.
225     headroom = t + self.tolerance - self._tat
226     remaining = 1 + int(headroom // self.
227         emission_interval) if headroom >= 0 else 0
228     return Decision(True, remaining, 0.0)

```

15.3.2 Listing 14.2 — The Theorems, as Executable Tests

`test_rate_limiters.py` is the chapter’s proof sheet: Theorem 15.1 (20 admissions in 2 ms under a “10/s” limit), Proposition 15.2 (the sliding-log interval invariant), Proposition 15.3 (adversarial bunching produces exactly

the predicted over-admission), Proposition 15.4 (the envelope never violated over 2000 steps), and Proposition 15.5 (GCRA and token bucket agree on a 500-event seeded adversarial stream).

Listing 14.2 — test_rate_limiters.py: deterministic proofs of documented behavior.

```
1  """Deterministic unit tests for the five reference
   limiters.
2
3  Every test injects a VirtualClock; nothing here reads
   wall time. The
4  fixed-window double-burst flaw and the GCRA/token-
   bucket equivalence are
5  proven as executable tests, mirroring the chapter's
   theorems.
6  """
7
8  from __future__ import annotations
9
10 import pytest
11
12 from rate_limiters import (
13     FixedWindowLimiter,
14     GCRALimiter,
15     SlidingWindowCounterLimiter,
16     SlidingWindowLogLimiter,
17     TokenBucketLimiter,
18     VirtualClock,
19 )
20
21
22 def run_burst(limiter, n: int, at: float) -> int:
23     """Fire n requests at the same instant; return how
       many were admitted."""
24     return sum(1 for _ in range(n) if limiter.check(at=
       at).allowed)
25
26
27 class TestFixedWindow:
28     def test_admits_up_to_limit_per_window(self) ->
       None:
29         clock = VirtualClock()
30         limiter = FixedWindowLimiter(limit=3, window
           =10.0, clock=clock)
31         assert run_burst(limiter, 3, at=1.0) == 3
32         assert not limiter.check(at=2.0).allowed
```

```

33         assert not limiter.check(at=9.9).allowed
34         # A fresh window restores the full quota.
35         assert limiter.check(at=10.0).allowed
36
37     def test_double_burst_at_boundary(self) -> None:
38         """Executable proof of Theorem 14.1: a fixed-
39         window limiter configured
40         for NupperW admits up to 2N requests inside a
41         single real-time
42         interval of length W that straddles a boundary.
43         """
44         clock = VirtualClock()
45         limiter = FixedWindowLimiter(limit=10, window
46                                     =1.0, clock=clock)
47         admitted = 0
48         # 10 requests at the very end of window 0 ...
49         admitted += run_burst(limiter, 10, at=0.999)
50         # ... and 10 more at the very start of window
51         # 1. Both windows admit
52         # their full quota, yet all 20 requests fall
53         # inside [0.999, 1.001],
54         # a real interval of 2 ms -- far shorter than
55         # the nominal window.
56         admitted += run_burst(limiter, 10, at=1.001)
57         assert admitted == 20 # 2x the nominal limit
58         assert 1.001 - 0.999 < 1.0
59
60 class TestSlidingWindowLog:
61     def test_exactness(self) -> None:
62         clock = VirtualClock()
63         limiter = SlidingWindowLogLimiter(limit=3,
64                                           window=10.0, clock=clock)
65         # Three requests mid-window saturate it;
66         # crossing the fixed
67         # boundary at t=10 does NOT restore quota,
68         # because the window
69         # slides with the caller.
70         assert run_burst(limiter, 5, at=5.0) == 3
71         assert run_burst(limiter, 5, at=10.0) == 0
72         assert run_burst(limiter, 5, at=14.999) == 0
73         # At t=15.0 the first request has fully aged
74         # out.
75         assert limiter.check(at=15.0).allowed
76
77     def test_no_interval_exceeds_limit(self) -> None:

```

```

68         """For any request pattern, the admitted set
           never contains more
69         than 'limit' events inside any half-open
           interval of length
70         'window': the (i+limit)-th admission is
           always strictly more
71         than 'window' after the i-th."""
72         clock = VirtualClock()
73         limiter = SlidingWindowLogLimiter(limit=4,
           window=2.0, clock=clock)
74         admitted: list[float] = []
75         t = 0.0
76         for _ in range(400):
77             t += 0.05
78             if limiter.check(at=t).allowed:
79                 admitted.append(t)
80         for i in range(len(admitted) - 4):
81             assert admitted[i + 4] - admitted[i] > 2.0
               - 1e-9
82
83     def test_retry_after_points_at_oldest_expiry(self)
       -> None:
84         clock = VirtualClock()
85         limiter = SlidingWindowLogLimiter(limit=2,
           window=10.0, clock=clock)
86         limiter.check(at=1.0)
87         limiter.check(at=3.0)
88         decision = limiter.check(at=4.0)
89         assert not decision.allowed
90         assert decision.retry_after == pytest.approx
           (7.0) # 1.0 + 10.0 - 4.0
91
92
93     class TestSlidingWindowCounter:
94     def test_matches_exact_when_previous_window_empty(
       self) -> None:
95         clock = VirtualClock()
96         limiter = SlidingWindowCounterLimiter(limit=3,
           window=10.0, clock=clock)
97         assert run_burst(limiter, 5, at=15.0) == 3
98
99     def
       test_weights_previous_window_by_elapsed_fraction
       (self) -> None:
100         clock = VirtualClock()
101         limiter = SlidingWindowCounterLimiter(limit=10,

```

```

        window=1.0, clock=clock)
102     # Saturate window 0 entirely at its start.
103     assert run_burst(limiter, 10, at=0.01) == 10
104     # Halfway through window 1 the estimate is 0 +
        10 * (1 - 0.5) = 5,
105     # so exactly 5 more requests are admitted
        before t = 1.5.
106     assert run_burst(limiter, 10, at=1.5) == 5
107
108     def test_error_bound_holds(self) -> None:
109         """Executable check of Proposition 14.2: the
        absolute error of the
110         weighted estimate is strictly less than the
        previous window count."""
111         clock = VirtualClock()
112         counter = SlidingWindowCounterLimiter(limit=8,
            window=1.0, clock=clock)
113         exact = SlidingWindowLogLimiter(limit=8, window
            =1.0, clock=clock)
114         # Adversarial: all of window 0's traffic
            bunched at its very end.
115         run_burst(counter, 8, at=0.99)
116         run_burst(exact, 8, at=0.99)
117         admitted_counter = run_burst(counter, 20, at
            =1.01)
118         admitted_exact = run_burst(exact, 20, at=1.01)
119         # The exact log admits 0 here (all 8 requests
            still inside the
120         # sliding window); the counter over-admits 1
            because it assumes
121         # window-0 traffic was spread uniformly. Error
            1 < previous
122         # window count 8, as the bound predicts.
123         assert admitted_exact == 0
124         assert admitted_counter == 1
125         assert abs(admitted_counter - admitted_exact) <
            8
126
127
128     class TestTokenBucket:
129         def test_burst_equals_capacity(self) -> None:
130             clock = VirtualClock()
131             limiter = TokenBucketLimiter(rate=10.0, burst
                =10, clock=clock)
132             assert run_burst(limiter, 100, at=0.0) == 10
133             # Half a second of silence refills exactly five

```

```

tokens.
134     clock.advance(0.5)
135     assert run_burst(limiter, 100, at=clock.now())
        == 5
136
137     def test_long_run_rate_never_exceeds_refill(self)
        -> None:
138         """Executable check of Proposition 14.3: over
            any interval [t0, t1]
139         the admitted count is at most burst + rate * (
            t1 - t0)."""
140         clock = VirtualClock()
141         limiter = TokenBucketLimiter(rate=10.0, burst
            =10, clock=clock)
142         admitted = 0
143         t = 0.0
144         for _ in range(2000):
145             t += 0.02
146             if limiter.check(at=t).allowed:
147                 admitted += 1
148             assert admitted <= 10 + 10.0 * t + 1e-9
149
150     def test_retry_after_is_deficit_over_rate(self) ->
        None:
151         clock = VirtualClock()
152         limiter = TokenBucketLimiter(rate=4.0, burst=2,
            clock=clock)
153         run_burst(limiter, 2, at=0.0)
154         decision = limiter.check(at=0.0)
155         assert not decision.allowed
156         assert decision.retry_after == pytest.approx
            (0.25)
157
158
159     class TestGCRA:
160         def test_equivalent_to_token_bucket(self) -> None:
161             """GCRA with emission interval T and tolerance
                tau = (B-1)T admits
162             exactly the same streams as a token bucket with
                capacity B and rate
163             1/T. Proven by construction in Section 14.3 (
                GCRA); checked here on a
164             seeded adversarial stream."""
165             import random
166
167             rng = random.Random(14)

```

```

168         gcra = GCRA Limiter(rate=10.0, burst=10, clock=
           VirtualClock())
169         bucket = TokenBucketLimiter(rate=10.0, burst
           =10, clock=VirtualClock())
170         t = 0.0
171         for _ in range(500):
172             t += rng.choice([0.0, 0.005, 0.02, 0.2,
                               1.0])
173             assert gcra.check(at=t).allowed == bucket.
               check(at=t).allowed
174
175     def test_steady_stream_at_rate_always_conforms(self
       ) -> None:
176         clock = VirtualClock()
177         limiter = GCRA Limiter(rate=5.0, burst=3, clock=
           clock)
178         for i in range(100):
179             assert limiter.check(at=i * 0.2).allowed
180
181     def
       test_retry_after_is_tat_minus_tolerance_minus_now
       (self) -> None:
182         clock = VirtualClock()
183         limiter = GCRA Limiter(rate=10.0, burst=2, clock
           =clock) # T=0.1, tau=0.1
184         assert limiter.check(at=0.0).allowed # TAT:
           0.1
185         assert limiter.check(at=0.0).allowed # TAT:
           0.2
186         decision = limiter.check(at=0.0) #
           arrival limit = 0.2 - 0.1
187         assert not decision.allowed
188         assert decision.retry_after == pytest.approx
           (0.1)

```

Run it:

```

1 pytest test_rate_limiters.py -q
2 # 14 passed

```

15.3.3 Listing 14.3 — Simulation Harness: One Timeline, Five Verdicts

Claims about burst behavior should be measured, not asserted. `burst_simulation.py` replays one seeded synthetic timeline — a 25-request synchronized-retry

burst straddling the $t=1.0$ fixed-window boundary, followed by a steady 8 rps stream — through all five limiters configured at the same nominal 10 requests/second, and prints the per-250 ms admission counts that produce Figure 15.4.

Listing 14.3 — `burst_simulation.py`: seeded replay harness feeding the comparison chart.

```

1  """Simulation harness: replay one synthetic request
   timeline through every
2  reference limiter and print per-bucket admission counts
   .
3
4  The timeline is fully seeded and deterministic; re-
   running always prints the
5  same numbers. The printed coordinates are exactly the
   data series plotted in
6  Figure 14.4 (burst behavior comparison).
7
8  Scenario (Northwind Robotics telemetry ingest API,
   nominal plan:
9  10 requests/second per API key):
10  * t in [0.90, 1.10]: a 25-request burst straddling
   the t=1.0 fixed-window
11  boundary (a mobile fleet retrying in sync after a
   connectivity blip),
12  * t in [1.50, 4.00]: a steady stream of 8 requests/
   second with seeded
13  jitter -- well below the nominal rate, so a correct
   limiter should admit
14  all of it after the burst penalty is paid.
15  """
16
17  from __future__ import annotations
18
19  import random
20  from typing import Callable, Dict, List, Tuple
21
22  from rate_limiters import (
23      FixedWindowLimiter,
24      GCRA limiter,
25      SlidingWindowCounterLimiter,
26      SlidingWindowLogLimiter,
27      TokenBucketLimiter,
28      VirtualClock,
29  )
30

```



```

31 NOMINAL_RATE = 10 # requests per second per API key
32 WINDOW = 1.0
33 BUCKET_WIDTH = 0.25 # chart resolution
34
35
36 def build_timeline(seed: int = 14) -> List[float]:
37     """Deterministic synthetic request timeline(
38         arrival times, seconds)."""
39     rng = random.Random(seed)
40     arrivals: List[float] = []
41     # Synchronized retry burst: 25 requests spread
42     # across the t=1.0 boundary.
43     for i in range(25):
44         arrivals.append(0.90 + i * 0.008 + rng.uniform(
45             0.0, 0.002))
46     # Steady-state traffic at 8 rps with jitter, from t
47     # =1.5 to t=4.0.
48     t = 1.5
49     while t <= 4.0:
50         arrivals.append(t)
51         t += 1.0 / 8.0 + rng.uniform(-0.03, 0.03)
52     arrivals.sort()
53     return arrivals
54
55 def simulate(arrivals: List[float],
56             make_limiter: Callable[[VirtualClock],
57                                     object]) -> Dict[float, int]:
58     """Replay arrivals; return {bucket_start:
59         admitted_count}."""
60     clock = VirtualClock()
61     limiter = make_limiter(clock)
62     buckets: Dict[float, int] = {}
63     for t in arrivals:
64         decision = limiter.check(at=t) # type: ignore[
65             attr-defined]
66         if decision.allowed:
67             bucket = round(int(t / BUCKET_WIDTH) *
68                             BUCKET_WIDTH, 2)
69             buckets[bucket] = buckets.get(bucket, 0) +
70                 1
71     return buckets
72
73 def main() -> None:
74     arrivals = build_timeline()

```

```

68     factories: List[Tuple[str, Callable[[VirtualClock],
69         object]]] = [
70         ("fixed_window", lambda c: FixedWindowLimiter(
71             NOMINAL_RATE, WINDOW, c)),
72         ("sliding_log", lambda c:
73             SlidingWindowLogLimiter(NOMINAL_RATE, WINDOW
74             , c)),
75         ("sliding_counter", lambda c:
76             SlidingWindowCounterLimiter(NOMINAL_RATE,
77             WINDOW, c)),
78         ("token_bucket", lambda c: TokenBucketLimiter(
79             NOMINAL_RATE, NOMINAL_RATE, c)),
80         ("GCR", lambda c: GCRALimiter(NOMINAL_RATE,
81             NOMINAL_RATE, c)),
82     ]
83     print(f"timeline: {len(arrivals)} arrivals, "
84           f"t={arrivals[0]:.3f}..{arrivals[-1]:.3f}")
85     all_starts = [round(i * BUCKET_WIDTH, 2) for i in
86         range(int(4.0 / BUCKET_WIDTH) + 1)]
87     for name, factory in factories:
88         buckets = simulate(arrivals, factory)
89         total = sum(buckets.values())
90         coords = "".join(f"({s:.2f},{buckets.get(s, 0)})" for s in all_starts)
91         print(f"\n{name}: admitted {total}/{len(
92             arrivals)}")
93         print(coords)
94
95 if __name__ == "__main__":
96     main()

```

15.3.4 Listing 14.4 — Distributed Sliding Window: Redis + Lua

The distributed limiter lifts the sliding log of Section 15.2.3 into Redis sorted sets, with the entire evict-count-admit-expire sequence inside one Lua script so it is atomic by construction (Section 15.2.9). The script first, because it is the load-bearing artifact:

Listing 14.4a — sliding_window.lua: atomic check-and-admit (evict, count, conditionally insert, expire; returns admission, remaining budget, and retry-after in milliseconds).

```

1 local key      = KEYS[1]
2 local now      = tonumber(ARGV[1])

```

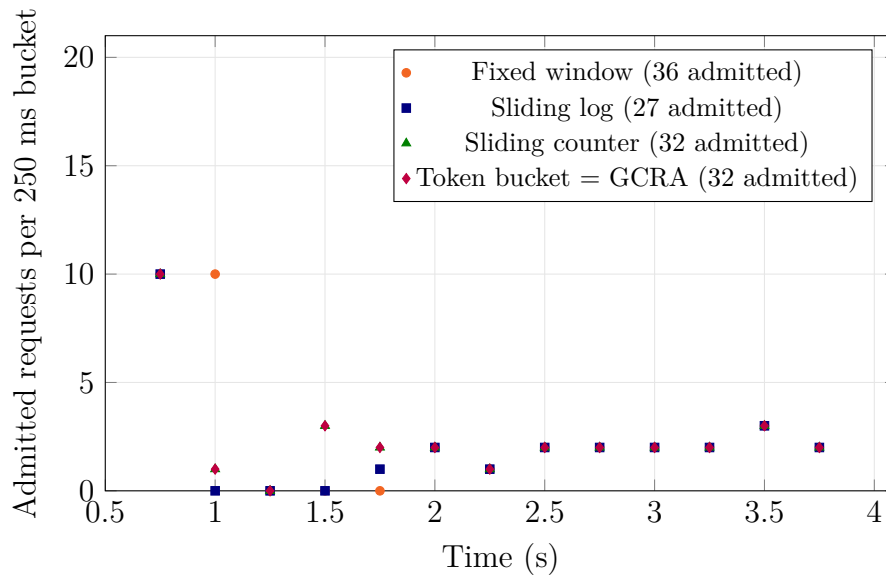


Figure 15.4: Burst behavior at the same nominal rate (10 requests/second per key), measured by Listing 15.3.3. The fixed window admits 20 requests across the $t=1.0$ boundary — Theorem 15.1 made visible — while the sliding log pays the burst penalty back in full before resuming. The token bucket and GCRA series coincide point for point, as Proposition 15.5 requires; the sliding counter’s weighted estimate coincidentally matches them on this timeline.

```

3 local window = tonumber(ARGV[2])
4 local limit  = tonumber(ARGV[3])
5 local member = ARGV[4]
6
7 redis.call('ZREMRANGEBYSCORE', key, '-inf', now -
  window)
8 local count = redis.call('ZCARD', key)
9
10 if count < limit then
11     redis.call('ZADD', key, now, member)
12     redis.call('PEXPIRE', key, math.ceil(window * 1000))
13     return {1, limit - count - 1, 0}
14 else
15     -- ZRANGE + ZSCORE (not WITHSCORES): the WITHSCORES
      return shape inside
16     -- Lua differs between Redis and emulators; two calls
      are portable.
17     local oldest = redis.call('ZRANGE', key, 0, 0)
18     local retry_after_ms = 0
19     if #oldest >= 1 then

```

```

20     local score = tonumber(redis.call('ZSCORE', key,
21         oldest[1]))
22     retry_after_ms = math.ceil((score + window - now) *
23         1000)
24 end
25 return {0, 0, retry_after_ms}
26 end

```

Two script details earn their comments. First, Lua return values cross back into RESP as *integers*, so sub-second **Retry-After** precision must travel as milliseconds and be divided down client-side. Second, the member must be unique per request — **ZADD** upserts by member, so two requests sharing a member would silently undercount. The Python wrapper guarantees uniqueness with a lock-guarded sequence counter.

Listing 14.4b — `redis_limiter.py`: the wrapper, plus an offline `fakeredis` factory.

```

1  """Distributed sliding-window limiter: Redis sorted set
2     + Lua script.
3
4  State lives in Redis so any number of API replicas
5  enforce one shared limit
6  per key. The whole check-and-admit operation is a
7  single Lua script, so it
8  executes atomically on the Redis server: no read-modify
9  -write race can split
10 it, no matter how many replicas call concurrently.
11
12 Offline-first: tests run against 'fakeredis.FakeRedis'
13 which emulates
14 RESP and executes Lua scripts atomically inside the
15 process, so the same
16 code paths are exercised without a server. For a real
17 deployment swap in
18 'redis.Redis' (redis >= 5) -- the script and calling
19 convention are
20 identical.
21 """
22
23 from __future__ import annotations
24
25 import itertools
26 import threading
27 from typing import Optional, Protocol
28
29 from rate_limiters import Clock, Decision, VirtualClock

```

```

22
23
24 class RedisLike(Protocol):
25     """The subset of the redis-py API this limiter
26         needs."""
27
28     def eval(self, script: str, numkeys: int, *
29             keys_and_args: object) -> object: ...
30
31 # Sliding-window log as one atomic Lua script (also
32 # printed standalone as
33 # Listing 14.4a; it lives inline here so this file is
34 # complete and runnable).
35 SLIDING_WINDOW_LUA = """
36 local key=KEYS[1]
37 local now=tonumber(ARGV[1])
38 local window=tonumber(ARGV[2])
39 local limit=tonumber(ARGV[3])
40 local member=ARGV[4]
41
42 redis.call('ZREMRANGEBYSCORE',key,'-inf',now-
43     window)
44 local count=redis.call('ZCARD',key)
45
46 if count<limit then
47     redis.call('ZADD',key,now,member)
48     redis.call('PEXPIRE',key,math.ceil(window*1000))
49     return{1,limit-count-1,0}
50 else
51     -- ZRANGE+ZSCORE(not WITHSCORES): the WITHSCORES
52     return_shape inside
53     -- Lua differs between Redis and emulators; two calls
54     are portable.
55     local oldest=redis.call('ZRANGE',key,0,0)
56     local retry_after_ms=0
57     if #oldest>=1 then
58         local score=tonumber(redis.call('ZSCORE',key,
59             oldest[1]))
60         retry_after_ms=math.ceil((score+window-now)*
61             1000)
62     end
63     return{0,0,retry_after_ms}
64 end
65 """
66
67
68

```

```

59
60 class RedisSlidingLogLimiter:
61     """Shared sliding-window-log limiter backed by
        Redis sorted sets.
62
63     Semantics match 'SlidingWindowLogLimiter' exactly
        :at most 'limit'
64     admissions per key inside any interval of 'window
        ' seconds. One Redis
65     key per (namespace, limiter key) pair; keys self-
        expire after 'window'
66     so idle consumers cost no memory.
67 """
68
69     def __init__(
70         self,
71         client: RedisLike,
72         namespace: str,
73         limit: int,
74         window: float,
75         clock: Clock,
76     ) -> None:
77         if limit <= 0 or window <= 0:
78             raise ValueError("limit and window must be
                positive")
79         if not namespace:
80             raise ValueError("namespace must be non-
                empty")
81         self.client = client
82         self.namespace = namespace
83         self.limit = limit
84         self.window = float(window)
85         self.clock = clock
86         self._seq = itertools.count()
87         self._seq_lock = threading.Lock()
88
89     def _member(self, t: float) -> str:
90         # ZADD upserts by member, so every admitted
            request needs a unique
91         # member id; identical scores are fine.
92         with self._seq_lock:
93             return f"{t:.6f}:{next(self._seq)}"
94
95     def check(self, key: str, at: Optional[float] =
        None) -> Decision:
96         if not key:

```

```

97         raise ValueError("limiter_key must be non-
98             empty")
99         t = self.clock.now() if at is None else at
100         redis_key = f"{self.namespace}:{key}"
101         raw = self.client.eval(
102             SLIDING_WINDOW_LUA,
103             1,
104             redis_key,
105             repr(t),
106             repr(self.window),
107             str(self.limit),
108             self._member(t),
109         )
110         allowed, remaining, retry_after_ms = (int(v)
111             for v in raw)
112         return Decision(bool(allowed), remaining,
113             retry_after_ms / 1000.0)
114
115 def build_fake_limiter(
116     limit: int = 5, window: float = 10.0, clock:
117     Optional[Clock] = None
118 ) -> RedisSlidingLogLimiter:
119     """Offline factory: a limiter against an in-process
120         faked redis server."""
121     import fakeredis
122
123     return RedisSlidingLogLimiter(
124         client=fakeredis.FakeRedis(),
125         namespace="northwind:rl",
126         limit=limit,
127         window=window,
128         clock=clock or VirtualClock(),
129     )

```

Listing 14.5 — `test_redis_limiter.py`: semantics, isolation, expiry hygiene, and a 16-thread concurrency storm.

```

1  """Tests for the Redis+Lua distributed limiter (offline
2     , fakeredis).
3
4  Atomicity note: real Redis executes each Lua script
5     atomically on its single
6     command thread, so no interleaving of ZREMRANGEBYSCORE/
7     ZCARD/ZADD is
8     possible across clients. fakeredis emulates the same
9     contract in-process

```

```

6  (scripts_run_to_completion_under_its_server_lock), so
   the concurrency test
7  below exercises the true atomicity semantics without a
   server. What fakeredis
8  does NOT emulate is wall-clock key expiry timing --
   which is why the limiter
9  never relies on EXPIRE for correctness, only for memory
   hygiene.
10 ""
11
12 from __future__ import annotations
13
14 import threading
15
16 import pytest
17
18 from rate_limiters import VirtualClock
19 from redis_limiter import RedisSlidingLogLimiter,
   build_fake_limiter
20
21
22 class TestSemantics:
23     def test_admits_limit_then_rejects(self) -> None:
24         clock = VirtualClock()
25         limiter = build_fake_limiter(limit=3, window
           =10.0, clock=clock)
26         results = [limiter.check("robot-7", at=1.0).
           allowed for _ in range(5)]
27         assert results == [True, True, True, False,
           False]
28
29     def test_window_slides_exactly(self) -> None:
30         clock = VirtualClock()
31         limiter = build_fake_limiter(limit=2, window
           =10.0, clock=clock)
32         limiter.check("robot-7", at=5.0)
33         limiter.check("robot-7", at=7.0)
34         assert not limiter.check("robot-7", at=14.999).
           allowed
35         assert limiter.check("robot-7", at=15.0).
           allowed # 5.0 has aged out
36
37     def test_keys_are_isolated(self) -> None:
38         clock = VirtualClock()
39         limiter = build_fake_limiter(limit=1, window
           =10.0, clock=clock)

```



```

40     assert limiter.check("robot-7", at=0.0).allowed
41     assert limiter.check("robot-8", at=0.0).allowed
42     assert not limiter.check("robot-7", at=0.0).
        allowed
43
44     def test_retry_after_matches_oldest_expiry(self) ->
        None:
45         clock = VirtualClock()
46         limiter = build_fake_limiter(limit=2, window
            =10.0, clock=clock)
47         limiter.check("robot-7", at=1.25)
48         limiter.check("robot-7", at=3.00)
49         decision = limiter.check("robot-7", at=4.0)
50         assert not decision.allowed
51         # Oldest (1.25) leaves the window at 11.25;
            ceil to milliseconds.
52         assert decision.retry_after == pytest.approx
            (7.25, abs=0.001)
53
54     def test_idle_keys_expire_from_redis(self) -> None:
55         import fakeredis
56
57         clock = VirtualClock()
58         client = fakeredis.FakeRedis()
59         limiter = RedisSlidingLogLimiter(client, "
            northwind:rl", 5, 10.0, clock)
60         limiter.check("robot-7", at=0.0)
61         ttl_ms = client.pttl("northwind:rl:robot-7")
62         assert 9_000 < ttl_ms <= 10_000
63
64
65     class TestConcurrency:
66         def test_no_over_admission_under_thread_storm(self)
            -> None:
67             """16 threads x 25 requests against limit 100:
                exactly 100 admissions,
68             """
            """never 101. A non-atomic check-then-set would
                intermittently fail."""
69             clock = VirtualClock()
70             limiter = build_fake_limiter(limit=100, window
                =60.0, clock=clock)
71             admitted = 0
72             admitted_lock = threading.Lock()
73
74             def worker() -> None:
75                 nonlocal admitted

```

```

76         local = 0
77         for _ in range(25):
78             if limiter.check("fleet-shared-key", at
79                               =0.0).allowed:
80                 local += 1
81             with admitted_lock:
82                 admitted += local
83
84         threads = [threading.Thread(target=worker) for
85                     _ in range(16)]
86         for thread in threads:
87             thread.start()
88         for thread in threads:
89             thread.join()
90         assert admitted == 100

```

15.3.5 Listing 14.6 — FastAPI Middleware: Headers Are the Product

The middleware implements the full client-facing contract of Section 15.2.10: per-API-key buckets, 429 with ceiled delay-seconds `Retry-After`, the four IETF `RateLimit-*` fields on successes *and* rejections, 401 for missing keys (distinct from 429 — an unauthenticated caller has no bucket to exceed), and a fail-open/fail-closed policy switch for limiter outages. It is written as raw ASGI middleware rather than `BaseHTTPMiddleware` so that rejected requests are answered without ever instantiating the downstream application’s request handling.

Listing 14.6 — `middleware_app.py`: rate-limit middleware with accurate headers and a tested failure policy.

```

1  """FastAPI␣middleware␣wiring␣a␣rate␣limiter␣to␣client-
2     facing␣HTTP␣semantics.
3
4  Semantics␣implemented␣(Section␣14.3,␣"Client-Facing
5     Semantics"):
6
7  *␣per-API-key␣buckets,␣keyed␣on␣the␣‘X-API-Key‘␣
8     header,
9
10 *␣429␣Too␣Many␣Requests␣with␣‘Retry-After‘␣(delay-
11    seconds␣form)␣on
12
13 rejection,
14
15 *␣IETF␣RateLimit␣header␣fields␣on␣every␣limited␣
16    response:
17
18 *␣‘RateLimit-Limit‘,␣‘RateLimit-Remaining‘,␣‘
19    RateLimit-Reset‘,
20
21 *␣‘RateLimit-Policy‘␣(e.g.␣‘5;w=60‘),

```

```

10  """*a fail-open/ fail-closed policy switch for limiter
    outages.
11
12  The limiter is injected behind the 'KeyedLimiter' protocol, so the same
13  middleware runs against the in-process fixed-window limiter here, or the
14  Redis limiter of Listing 14.4 in a multi-replica deployment. The clock is
15  injected, so tests are deterministic.
16  """
17
18  from __future__ import annotations
19
20  import math
21  from typing import Dict, Literal, Optional, Protocol
22
23  from fastapi import FastAPI, Request
24  from fastapi.responses import JSONResponse
25  from fastapi.testclient import TestClient
26
27  from rate_limiters import Clock, Decision, VirtualClock
28
29  FailurePolicy = Literal["fail_open", "fail_closed"]
30
31  VALID_API_KEYS = {"nr-demo-key-001", "nr-demo-key-002"}
32                      # synthetic only
33
34  class KeyedLimiter(Protocol):
35      """Anything that can admit or reject for a key and
36         report reset time."""
37
38      def check(self, key: str) -> Decision: ...
39
40      def reset_after(self, key: str) -> float:
41          """Seconds until the key's window fully resets
42             (0 if idle)."""
43
44      ...
45
46  class InProcessFixedWindowLimiter:
47      """Per-key fixed-window counters with an injected
48         clock.
49
50      """Correct 0(1)-memory semantics for a single replica;

```

```

    subject to the
48 2x boundary burst of Theorem 14.1, which is
    acceptable when the window
49 is short and the header contract (crisp RateLimit -
    Reset) matters more
50 than boundary strictness. Swap in
    RedisSlidingLogLimiter for
51 multi-replica deployments.
52 """
53
54 def __init__(self, limit: int, window: float, clock
    : Clock) -> None:
55     if limit <= 0 or window <= 0:
56         raise ValueError("limit and window must be
            positive")
57     self.limit = limit
58     self.window = window
59     self.clock = clock
60     self._buckets: Dict[str, tuple[int, int]] = {}
        # key -> (index, count)
61
62 def check(self, key: str) -> Decision:
63     t = self.clock.now()
64     index = math.floor(t / self.window)
65     bucket_index, count = self._buckets.get(key, (
        index, 0))
66     if bucket_index != index:
67         bucket_index, count = index, 0
68     window_end = (index + 1) * self.window
69     if count < self.limit:
70         self._buckets[key] = (bucket_index, count +
            1)
71         return Decision(True, self.limit - count -
            1, 0.0)
72     self._buckets[key] = (bucket_index, count)
73     return Decision(False, 0, window_end - t)
74
75 def reset_after(self, key: str) -> float:
76     t = self.clock.now()
77     index = math.floor(t / self.window)
78     bucket = self._buckets.get(key)
79     if bucket is None or bucket[0] != index or
        bucket[1] == 0:
80         return 0.0
81     return (index + 1) * self.window - t
82

```

```

83
84 class RateLimitMiddleware:
85     """ASGI middleware enforcing a per-API-key limit
86         with full headers."""
87
88     def __init__(
89         self,
90         app: FastAPI,
91         limiter: KeyedLimiter,
92         limit: int,
93         window: float,
94         on_limiter_error: FailurePolicy = "fail_open",
95     ) -> None:
96         if on_limiter_error not in ("fail_open", "fail_closed"):
97             raise ValueError("on_limiter_error must be 'fail_open' or 'fail_closed'")
98
99         self.app = app
100         self.limiter = limiter
101         self.limit = limit
102         self.window = window
103         self.on_limiter_error = on_limiter_error
104
105     async def __call__(self, scope, receive, send) -> None:
106         if scope["type"] != "http":
107             await self.app(scope, receive, send)
108             return
109
110         request = Request(scope)
111         api_key = request.headers.get("x-api-key")
112         if api_key not in VALID_API_KEYS:
113             response = JSONResponse(
114                 status_code=401,
115                 content={"detail": "missing or invalid API key"},
116             )
117             await response(scope, receive, send)
118             return
119
120         try:
121             decision = self.limiter.check(api_key)
122         except Exception:
123             if self.on_limiter_error == "fail_open":
124                 # Limiter down: serve anyway, and say
125                 # nothing we cannot
126                 # substantiate -- no RateLimit headers

```

```

123         we cannot compute.
124         await self.app(scope, receive, send)
125         return
126     response = JSONResponse(
127         status_code=503,
128         content={"detail": "rate_limiter_unavailable;_failing_closed"},
129         headers={"Retry-After": "5"},
130     )
131     await response(scope, receive, send)
132     return
133
134     headers = self._ratelimit_headers(api_key,
135         decision)
136     if not decision.allowed:
137         headers["Retry-After"] = str(max(1, math.
138             ceil(decision.retry_after)))
139     response = JSONResponse(
140         status_code=429,
141         content={"detail": "rate_limit_exceeded"},
142         headers=headers,
143     )
144     await response(scope, receive, send)
145     return
146
147     async def send_with_headers(message) -> None:
148         if message["type"] == "http.response.start":
149             :
150             extra = [(k.lower().encode(), v.encode
151                 ()) for k, v in headers.items()]
152             message["headers"] = list(message.get("
153                 headers", [])) + extra
154             await send(message)
155
156     await self.app(scope, receive,
157         send_with_headers)
158
159     def _ratelimit_headers(self, key: str, decision:
160         Decision) -> Dict[str, str]:
161         reset = decision.retry_after if not decision.
162             allowed else self.limiter.reset_after(key)
163         return {
164             "RateLimit-Limit": str(self.limit),
165             "RateLimit-Remaining": str(max(0, decision.
166                 remaining)),

```

```

157         "RateLimit-Reset": str(max(0, math.ceil(
158             reset))),
159         "RateLimit-Policy": f"{self.limit};w={int(
160             self.window)}",
161     }
162
163 def create_app(
164     limiter: Optional[KeyedLimiter] = None,
165     clock: Optional[Clock] = None,
166     on_limiter_error: FailurePolicy = "fail_open",
167     limit: int = 5,
168     window: float = 60.0,
169 ) -> FastAPI:
170     """Northwind_Robotics_telemetry_API_behind_the_rate
171     -limit_middleware."""
172     clock = clock or VirtualClock()
173     limiter = limiter or InProcessFixedWindowLimiter(
174         limit, window, clock)
175     app = FastAPI(title="Northwind_Robotics_Telemetry_
176     API")
177     app.add_middleware(
178         RateLimitMiddleware,
179         limiter=limiter,
180         limit=limit,
181         window=window,
182         on_limiter_error=on_limiter_error,
183     )
184
185     @app.get("/telemetry")
186     def telemetry() -> dict[str, str]:
187         return {"status": "ok", "fleet": "northwind-
188             robotics"}
189
190     return app
191
192 if __name__ == "__main__":
193     demo_clock = VirtualClock()
194     client = TestClient(create_app(clock=demo_clock))
195     for i in range(7):
196         resp = client.get("/telemetry", headers={"X-API
197             -Key": "nr-demo-key-001"})
198         interesting = {k: v for k, v in resp.headers.
199             items()
200             if k.startswith(("ratelimit", "

```

```

195         retry"))}
        print(f"request_{i+1}:_{resp.status_code}_{interesting}")

```

Listing 14.7 — `test_middleware_app.py`: header accuracy across window boundaries and both failure policies.

```

1  """TestClient tests: header accuracy across window
   boundaries, 429+
2  Retry-After, and the fail-open / fail-closed policy
   switch. All offline,
3  all deterministic (injected VirtualClock).
4  """
5
6  from __future__ import annotations
7
8  import pytest
9  from fastapi.testclient import TestClient
10
11 from middleware_app import create_app
12 from rate_limiters import Decision, VirtualClock
13
14 KEY = {"X-API-Key": "nr-demo-key-001"}
15 OTHER_KEY = {"X-API-Key": "nr-demo-key-002"}
16
17
18 @pytest.fixture()
19 def rig() -> tuple[TestClient, VirtualClock]:
20     clock = VirtualClock(start=0.0)
21     app = create_app(clock=clock, limit=5, window=60.0)
22     return TestClient(app), clock
23
24
25 class TestHeaderAccuracy:
26     def test_limit_remaining_and_policy_on_success(self, rig) -> None:
27         client, _ = rig
28         for expected_remaining in (4, 3, 2, 1, 0):
29             resp = client.get("/telemetry", headers=KEY)
30
31             assert resp.status_code == 200
32             assert resp.headers["RateLimit-Limit"] == "5"
33             assert resp.headers["RateLimit-Remaining"]
34                 == str(expected_remaining)
35             assert resp.headers["RateLimit-Policy"] == "5;w=60"

```



```

34
35     def test_reset_counts_down_to_window_end(self, rig)
36         -> None:
37         client, clock = rig
38         resp = client.get("/telemetry", headers=KEY)
39         assert resp.headers["RateLimit-Reset"] == "60"
40         clock.advance(25.0)
41         resp = client.get("/telemetry", headers=KEY)
42         assert resp.headers["RateLimit-Reset"] == "35"
43         # 60 - 25
44
45     def test_429_carries_retry_after_and_zero_remaining
46         (self, rig) -> None:
47         client, clock = rig
48         for _ in range(5):
49             assert client.get("/telemetry", headers=KEY
50                 ).status_code == 200
51             clock.advance(10.5) # 49.5s until window end
52             resp = client.get("/telemetry", headers=KEY)
53             assert resp.status_code == 429
54             assert resp.headers["Retry-After"] == "50" #
55                 ceil(49.5)
56             assert resp.headers["RateLimit-Remaining"] == "
57                 0"
58             assert resp.headers["RateLimit-Reset"] == "50"
59
60     def test_headers_reset_across_window_boundary(self,
61         rig) -> None:
62         client, clock = rig
63         for _ in range(5):
64             client.get("/telemetry", headers=KEY)
65             assert client.get("/telemetry", headers=KEY).
66                 status_code == 429
67             clock.advance(60.0) # cross the boundary at t
68                 =60
69             resp = client.get("/telemetry", headers=KEY)
70             assert resp.status_code == 200
71             assert resp.headers["RateLimit-Remaining"] == "
72                 4"
73             assert resp.headers["RateLimit-Reset"] == "60"
74
75     def test_buckets_are_per_api_key(self, rig) -> None
76         :
77         client, _ = rig
78         for _ in range(5):
79             client.get("/telemetry", headers=KEY)

```

```

69         assert client.get("/telemetry", headers=KEY).
           status_code == 429
70         resp = client.get("/telemetry", headers=
           OTHER_KEY)
71         assert resp.status_code == 200
72         assert resp.headers["RateLimit-Remaining"] == "
           4"
73
74     def test_missing_key_is_401_not_429(self, rig) ->
       None:
75         client, _ = rig
76         resp = client.get("/telemetry")
77         assert resp.status_code == 401
78         assert "RateLimit-Limit" not in resp.headers
79
80
81     class BrokenLimiter:
82         """A limiter that is down: every check raises, like
           a lost Redis."""
83
84         def check(self, key: str) -> Decision:
85             raise ConnectionError("redis: connection
               refused")
86
87         def reset_after(self, key: str) -> float:
88             raise ConnectionError("redis: connection
               refused")
89
90
91     class TestFailurePolicy:
92     def
           test_fail_open_serves_without_unverifiable_headers
           (self) -> None:
93         app = create_app(limiter=BrokenLimiter(),
           on_limiter_error="fail_open")
94         client = TestClient(app)
95         resp = client.get("/telemetry", headers=KEY)
96         assert resp.status_code == 200
97         assert "RateLimit-Remaining" not in resp.
           headers
98
99         def test_fail_closed_returns_503_with_retry_after(
           self) -> None:
100             app = create_app(limiter=BrokenLimiter(),
           on_limiter_error="fail_closed")
101             client = TestClient(app)

```

```

102     resp = client.get("/telemetry", headers=KEY)
103     assert resp.status_code == 503
104     assert resp.headers["Retry-After"] == "5"

```

15.3.6 Listing 14.8 — Cost-Based Limiting: Budgeting Work, Not Requests

The cost-based limiter of Section 15.2.11 is a token bucket whose `cost` parameter is looked up per endpoint: the budget is denominated in normalized work units (server CPU-milliseconds, from the observability pipeline), so an export drains it ten times faster than a median read, and `/health` is free because throttling a liveness probe is how you get load balancers to shoot healthy servers.

Listing 14.8 — `cost_limiter.py`: per-endpoint weighted token buckets.

```

1  """Cost-based (weighted) rate limiting: one token
   bucket per API key, with
2  each endpoint charging a weight proportional to its
   true server-side cost.
3
4  A flat "requests per minute" limit treats a 'GET /
   health' and a
5  'POST /reports/export' as equal, but the export scans
   and renders a
6  million-row dataset while the health check is a no-op.
   Weighted buckets make
7  the budget denominated in *work*, not in *requests*:
   expensive operations
8  drain the shared budget faster, free endpoints (health
   probes) are never
9  throttled, and cheap endpoints survive exactly as long
   as any budget
10 remains.
11
12 Northwind Robotics cost table: weights are server CPU-
   millisecond estimates
13 from the Chapter 18 observability pipeline, normalized
   so the median read
14 endpoint costs 1.0.
15 """
16
17 from __future__ import annotations
18
19 from typing import Dict, Optional

```

```

20
21 from rate_limiters import Clock, Decision,
    TokenBucketLimiter, VirtualClock
22
23 # path -> token cost per call (normalized work units)
24 ENDPOINT_COSTS: Dict[str, float] = {
25     "/health": 0.0,          # liveness probe: never
        throttle
26     "/telemetry": 1.0,      # median read
27     "/fleet/status": 1.0,   # median read
28     "/telemetry/batch": 4.0, # bulk ingest: 4x a single
        write
29     "/reports/export": 10.0, # full-fleet CSV render:
        the expensive one
30 }
31 DEFAULT_COST = 1.0
32
33
34 class CostBasedLimiter:
35     """Per-key token buckets charged by endpoint weight
        .
36
37     Budget semantics: 'rate' work-units per second
        sustained, 'burst'
38     Work-units of instantaneous headroom. With rate=10
        and burst=20, a key
39     may run 2 exports instantly (20/10) or 20 median
        reads, or any mix
40     whose total weight fits the bucket.
41     """
42
43     def __init__(self, rate: float, burst: float, clock
        : Clock,
44                 costs: Optional[Dict[str, float]] =
        None) -> None:
45         if rate <= 0 or burst <= 0:
46             raise ValueError("rate and burst must be
                positive")
47         self.rate = rate
48         self.burst = burst
49         self.clock = clock
50         self.costs = dict(ENDPOINT_COSTS if costs is
        None else costs)
51         self._buckets: Dict[str, TokenBucketLimiter] =
        {}
52

```

```

53     def cost_of(self, path: str) -> float:
54         return self.costs.get(path, DEFAULT_COST)
55
56     def check(self, key: str, path: str, at: Optional[
57         float] = None) -> Decision:
58         if not key:
59             raise ValueError("limiter_key_must_be_non-
60                 empty")
61         cost = self.cost_of(path)
62         if cost == 0.0:
63             return Decision(True, self.
64                 limit_placeholder(), 0.0)
65         bucket = self._buckets.get(key)
66         if bucket is None:
67             bucket = TokenBucketLimiter(self.rate, self.
68                 .burst, self.clock)
69             self._buckets[key] = bucket
70         return bucket.check(at=at, cost=cost)
71
72     def limit_placeholder(self) -> int:
73         # Free endpoints have no meaningful "remaining
74         # "; report the burst
75         # budget so header math stays well-formed.
76         return int(self.burst)
77
78 if __name__ == "__main__":
79     clock = VirtualClock()
80     limiter = CostBasedLimiter(rate=10.0, burst=20.0,
81         clock=clock)
82     key = "nr-demo-key-001"
83     for path in ("/reports/export", "/reports/export",
84         "/reports/export",
85             "/telemetry", "/telemetry", "/health")
86         :
87         d = limiter.check(key, path)
88         verdict = "admitted" if d.allowed else f"
89             rejected_{retry_in_{d.retry_after:.1f}s}"
90         print(f"{path:<18}_cost={limiter.cost_of(path)
91             :>4}_->_{verdict}")

```

Listing 14.9 — test_cost_limiter.py: weighted drainage, shared budgets, and edge cases.

```

1  """Tests for the cost-based limiter: weighted drainage,
2  _free_endpoints,
3  default_costs, and graceful degradation of cheap

```

```

    endpoints_under_budget
3 pressure._Deterministic_via_VirtualClock.
4 """
5
6 from __future__ import annotations
7
8 import pytest
9
10 from cost_limiter import CostBasedLimiter
11 from rate_limiters import VirtualClock
12
13 KEY = "nr-demo-key-001"
14
15
16 @pytest.fixture()
17 def limiter() -> CostBasedLimiter:
18     return CostBasedLimiter(rate=10.0, burst=20.0,
19                             clock=VirtualClock())
20
21 class TestWeightedDrainage:
22     def test_export_costs_ten_reads(self, limiter:
23         CostBasedLimiter) -> None:
24         # Two exports (2 x 10 units) exactly exhaust a
25         # 20-unit bucket.
26         assert limiter.check(KEY, "/reports/export", at
27                               =0.0).allowed
28         assert limiter.check(KEY, "/reports/export", at
29                               =0.0).allowed
30         assert not limiter.check(KEY, "/reports/export"
31                                   , at=0.0).allowed
32
33     def test_mixed_workload_shares_one_budget(self,
34         limiter: CostBasedLimiter) -> None:
35         assert limiter.check(KEY, "/reports/export", at
36                               =0.0).allowed # 10
37         for _ in range(10):
38             assert limiter.check(KEY, "/telemetry", at
39                                   =0.0).allowed # 10 x 1
40         assert not limiter.check(KEY, "/telemetry", at
41                                   =0.0).allowed # 21st unit
42
43     def test_refill_is_in_work_units(self, limiter:
44         CostBasedLimiter) -> None:
45         limiter.check(KEY, "/reports/export", at=0.0)
46         limiter.check(KEY, "/reports/export", at=0.0)

```

```

37         # 1.0s of refill = 10 work units = exactly one
           more export.
38         assert limiter.check(KEY, "/reports/export", at
           =1.0).allowed
39         assert not limiter.check(KEY, "/reports/export"
           , at=1.0).allowed
40
41     def test_retry_after_scales_with_cost(self, limiter
       : CostBasedLimiter) -> None:
42         limiter.check(KEY, "/reports/export", at=0.0)
43         limiter.check(KEY, "/reports/export", at=0.0)
44         cheap = limiter.check(KEY, "/telemetry", at
           =0.0)
45         dear = limiter.check(KEY, "/reports/export", at
           =0.0)
46         assert cheap.retry_after == pytest.approx(0.1)
           # 1 unit / 10 rps
47         assert dear.retry_after == pytest.approx(1.0)
           # 10 units / 10 rps
48
49
50     class TestEdgeCases:
51         def test_health_endpoint_is_never_throttled(self,
           limiter: CostBasedLimiter) -> None:
52             for _ in range(2):
53                 limiter.check(KEY, "/reports/export", at
                   =0.0)
54             for _ in range(100):
55                 assert limiter.check(KEY, "/health", at
                   =0.0).allowed
56
57         def test_unknown_path_defaults_to_unit_cost(self,
           limiter: CostBasedLimiter) -> None:
58             assert limiter.cost_of("/no/such/route") == 1.0
59             for i in range(20):
60                 assert limiter.check(KEY, "/no/such/route",
                   at=0.0).allowed, i
61             assert not limiter.check(KEY, "/no/such/route",
                   at=0.0).allowed
62
63         def test_budgets_are_per_key(self, limiter:
           CostBasedLimiter) -> None:
64             limiter.check(KEY, "/reports/export", at=0.0)
65             limiter.check(KEY, "/reports/export", at=0.0)
66             assert not limiter.check(KEY, "/reports/export"
           , at=0.0).allowed

```

```

67 |         assert limiter.check("nr-demo-key-002", "/"
|            reports/export", at=0.0).allowed

```

The complete suite — 35 tests across the four test listings — runs in about four seconds with no network, no containers, and no wall-clock dependence:

```

1 | pytest -q
2 | # 35 passed
3 | python burst_simulation.py      # prints the Figure 14.4
   |     coordinates
4 | python middleware_app.py        # 7-request demo: headers
   |     decay, then 429
5 | python cost_limiter.py          # weighted drainage demo

```

15.4 Common Pitfalls & Anti-Patterns

1. **Fixed window at scale, believed to be strict.** The 2x boundary burst (Theorem 15.1) is disqualifying when the limit protects something with a hard per-second ceiling (a partner API, a connection pool). If you keep the fixed window for its crisp reset headers, halve the nominal limit or shorten the window, and *document the envelope you actually enforce*.
2. **Per-IP as the only dimension.** Behind corporate NATs and carrier-grade NAT, one source IP can be a ten-thousand-employee enterprise: a tight per-IP limit punishes your best customers collectively for their network topology, while a botnet of ten thousand IPs gets ten thousand times the limit. Per-IP is a coarse outer net for pre-authentication traffic only; identity-based limits (API key, account, session) must carry the real weight — Section 15.5.1 is the cautionary tale in full.
3. **429 without Retry-After.** A rejection that does not say when to come back forces clients to guess, and the industry's guess is immediate retry — your limiter just built itself a retry storm. Always emit **Retry-After**; prefer delay-seconds; always ceil [Fie22a].
4. **Non-atomic check-then-set across replicas.** GET-compare-INCR over-admits under concurrency; INCR-then-reject lets rejected traffic poison the counter. One Lua script, one authoritative cell, or accept a documented approximation — there is no fourth option (Section 15.2.9).
5. **Silent throttling.** Queueing or delaying requests without any ob-

servable signal (no headers, no logs the client can see, no documented policy) makes your API look randomly slow and puts the debugging cost on the wrong team. If you shape, say so.

6. **One global limit for all endpoints.** A single requests-per-minute number treats `GET /health` and `POST /reports/export` as identical. Attackers and sloppy integrators will find your most expensive route and spend the whole budget there. Weight by cost (Listing 15.3.6) and cap concurrency on the heavy routes separately.
7. **Fail-closed limiter with no policy when Redis dies.** Deciding the failure posture *during* the outage means the default decides for you — usually fail-closed-by-exception, i.e., the limiter takes the whole API down with its state store. Pick per endpoint class from Table 15.2, exempt health checks, and test both paths (Listing 15.3.5 does).
8. **Wall-clock-dependent tests.** `time.sleep(1.1)` in a test suite is a flake factory: CI runners stall, laptops sleep, and the failure mode is intermittent — the worst kind. Inject a clock; advance it explicitly. Every listing in this chapter demonstrates the pattern.

15.5 Hands-On Production Lab & Real-World Case Study

Lab: From Simulation to Concurrency Storm

Goal. Verify, on your own machine, every load-bearing claim of this chapter: the algorithms’ burst envelopes, header accuracy under a live middleware, and fairness under a thread storm.

Setup. Save Listings 14.1–14.9 into one directory, create the virtual environment from Section 15.1, and install `fakeredis`, `fastapi`, `httpx`, `pytest`.

1. **Reproduce the proofs.** `pytest test_rate_limiters.py -q` — confirm 14 passes, then open `test_double_burst_at_boundary` and change the window to 2.0: the proof still holds (20 admissions, now inside a 2 ms interval of a “5/s” limit). Explain why $2N$ did not change.
2. **Run the simulation.** `python burst_simulation.py` — compare each algorithm’s per-bucket admissions to Figure 15.4. Then change the burst to land *entirely inside* one fixed window ($0.50 + i * 0.008$) and observe the fixed window’s boundary advantage disappear.

3. **Exercise the middleware.** `python middleware_app.py` shows seven requests: `RateLimit-Remaining` decaying $4 \rightarrow 0$, then 429 with `Retry-After: 60`. Now load it: run `pytest test_middleware_app.py -q`, then write a small loop that fires two interleaved API keys through the `TestClient` and assert neither key's rejections delay the other's admissions — fairness, verified.
4. **Storm the distributed limiter.** `pytest test_redis_limiter.py -q` runs 16 threads $\times$ 25 requests against a limit of 100 and asserts exactly 100 admissions. Break it deliberately: replace the Lua script with separate `zcard/zadd` calls and re-run until you catch the over-admission — the race of Section 15.2.9, reproduced on demand.
5. **Flip the policy switch.** Run both `BrokenLimiter` tests, then decide for each Northwind Robotics endpoint (`/telemetry`, `/reports/export`, `/health`, a hypothetical `/auth/login`) which failure policy belongs on it, and defend the assignment using Table 15.2.

15.5.1 Case Study: The Login Endpoint the Per-IP Limit Could Not Save

Northwind Robotics' fleet console had a textbook-looking defense: `POST /auth/login` limited to 20 requests per minute per source IP, fixed window, enforced at the edge. On a Tuesday, a credential-stuffing campaign — roughly 400,000 username/password pairs harvested from an unrelated breach — walked straight through it, and the postmortem found *three* compounding design errors rather than one.

Error one: the limit dimension was the attacker's friend. The campaign rotated through a residential proxy pool of 30,000 exit IPs, so each IP made at most 13 attempts — under the limit everywhere, forever. The per-IP ceiling divided the attacker's budget by the size of their botnet, which is to say it barely divided it at all.

Error two: the same dimension was the customers' enemy. Weeks earlier, a 6,000-employee logistics customer behind a corporate NAT had been *collectively* locked out every morning at 09:00 when their workforce logged in through two egress IPs. Support's fix — raise the per-IP limit to 2,000/min — made error one cheaper to exploit. Per-IP limits had now failed in both directions on the same endpoint.

Error three: nothing counted the thing being attacked. The asset under attack was *accounts*, but no counter was keyed by account. There was no per-username attempt limit, no global login-endpoint budget, and no velocity alert; the campaign's 0.7% success rate against 400k pairs was discovered by a customer reporting strange session emails, not by

telemetry.

The remediation layered the dimensions of Section 15.2.11: a per-account sliding log (5 attempts per 10 minutes — exactness chosen deliberately, N tiny so $O(N)$ memory is free) via the Redis cell of Listing 15.3.4; a *global* per-endpoint token bucket on `/auth/login` as a circuit-level budget; the per-IP limit retained but demoted to a loose outer net (and fail-open, because a limiter outage must never lock out all logins — fail-closed is correct for the *account* limiter instead); exponential backoff plus progressive delay on repeated per-account failures; and a velocity alert on global attempt rate. The follow-up campaign a quarter later exhausted its per-account budgets inside ninety seconds and the global bucket flagged the anomaly before support heard a word. The lesson generalizes: *choose the limiter dimension to match the asset under attack, and assume the attacker chooses their own distribution.*

15.6 Self-Assessment Exercises & Deep-Dive Questions

- E14.1. Boundary audit.** Modify Listing 15.3.2 to prove that the sliding counter, unlike the fixed window, admits *fewer* than $2N$ requests when a burst straddles a boundary — then exhibit a timeline where it still over-admits relative to the exact log. Which proposition bounds the difference?
- E14.2. Equivalence hunting.** Extend `test_equivalent_to_token_bucket` with 5,000 seeded events including multi-second idle gaps. Find (by search) a parameter pair (r, B) where GCRA and the token bucket *diverge*, or strengthen the proof sketch of Proposition 15.5 to explain why none exists.
- E14.3. Reset semantics.** The middleware computes `RateLimit-Reset` from the fixed window boundary. Specify (and implement) what it should report for the *sliding log* limiter, where “the window” never resets all at once. Which choice keeps the header monotonic and honest?
- E14.4. Clock discipline.** A teammate replaces the injected `Clock` with `time.monotonic` “to simplify the API.” List three concrete test or production failures this reintroduces, referencing specific listings.
- E14.5. Race reconstruction.** Instrument the broken non-atomic limiter from lab step 4 to *count* over-admissions across 50 seeded

storms, and plot (mentally or with pgfplots) how overshoot scales with thread count. Why does the Lua version’s atomicity eliminate the entire class rather than reduce it?

- E14.6. Quota design.** Northwind Robotics adds a monthly export quota (100 exports) on top of the per-second cost budget. Design the storage key layout, the Lua-side interaction (one script or two?), and the `GET /v1/usage` response body. What happens to a client at 99/100 whose per-second bucket is also empty?
- E14.7. AIMD tuning.** Given additive increase $a = 5$ rps and decrease $\beta = 0.5$ evaluated once per second, with a service that overloads above 300 rps, simulate (on paper or in code) the limit trajectory from $L_0 = 100$ rps. How many seconds to first contact with capacity, and what is the steady-state oscillation amplitude? Why is $\beta < 1$ the safety-critical parameter rather than a ?
- E14.8. Header archaeology.** Pick a public API you use. Capture a 429 (or its headers near the limit) and document which of `Retry-After`, `RateLimit-Limit/-Remaining/-Reset`, and `RateLimit-Policy` it emits, whether `Reset` is delay-seconds or epoch-seconds, and one concrete way its contract is better or worse than Listing 15.3.5’s.

15.7 Interview Q&A Vault

Junior (Q1–Q10)

- Q1. What is rate limiting, in one paragraph?** Per-identity admission control over a short window: a rule like “100 requests per minute per API key,” enforced statefully, whose purpose is to protect shared capacity, keep tenants fair, control cost, and raise the price of abuse. Rejection (HTTP 429) is a normal, cheap, expected outcome — not an error in the alarming sense.
- Q2. Rate limiting versus throttling versus quota — what’s the difference?** Rate limiting *rejects* excess starts in a short window; throttling *shapes* flow by delaying or dequeuing admitted work; a quota is a long-window (hour/day/month) budget that rations total consumption, usually a business/plan construct. A client can be within its monthly quota and still be rate-limited this second.
- Q3. What does HTTP 429 mean, and what header must accom-**

pany it? *Too Many Requests*: this caller exceeded its allowance; retryable later. It must carry **Retry-After** (delay-seconds or an HTTP-date) telling the client when to return; without it, clients guess, and the industry's guess is an immediate retry storm [Fie22a].

Q4. How does a fixed window counter work? Partition time at $\lfloor t/W \rfloor$, keep one counter per key per window, admit while the counter is below the limit. $O(1)$ memory, crisp reset semantics — and the 2x boundary burst of Theorem 15.1.

Q5. Explain the token bucket to a new hire. A bucket holds up to B tokens and refills at r per second; each request spends one token or is rejected. B is the burst you tolerate, r is the rate you sustain, and the two knobs are independent — that independence is the whole point.

Q6. Why not just store counters in each app server's memory? With R replicas behind a load balancer, each replica sees roughly $1/R$ of a client's traffic, so the effective limit becomes $R \times$ the configured one — and it varies with routing stickiness. Shared state (a Redis cell) or a deliberately partitioned local budget (limit/R) is required.

Q7. What is a concurrency limit and how is it different? It bounds requests *in flight*, not started per unit time. By Little's law, $\text{concurrency} = \text{throughput} \times \text{latency}$, so slow requests consume concurrency budget even when they are well within any rate limit. You need both knobs.

Q8. What information should a 429 response carry? Status 429; **Retry-After** in delay-seconds (ceiled); ideally **RateLimit-Limit**, **RateLimit-Remaining**: 0, **RateLimit-Reset**, and **RateLimit-Policy**; and a problem-details body (Chapter 8) stating which limit was hit and how to get more.

Q9. Why inject a clock into a rate limiter? Determinism. Time-advancing tests instead of **sleep**-based tests are fast, exact at boundaries, and never flaky on a loaded CI runner; the same injection lets you replay incident timelines against production code.

Q10. What is RateLimit-Policy: "100;w=60"? The IETF draft field that states the rule itself: 100 requests per 60-second window. Paired with **RateLimit-Remaining** on successes, it lets clients

self-throttle before ever being rejected [Pol24].

Mid (Q11–Q20)

- Q11. Prove the fixed-window 2x burst flaw.** Fire N requests at $kW - \varepsilon$ (window $k - 1$ admits all) and N more at $kW + \varepsilon$ (window k 's counter reset, admits all): $2N$ admissions inside an interval of length $2\varepsilon \leq W$. Tight because any W -length interval touches at most two windows (Theorem 15.1). Consequence: instantaneous rate is unbounded — the $2N$ can arrive within milliseconds.
- Q12. Token bucket versus leaky bucket — what's actually different?** “Leaky bucket” is two machines. As a *meter* (drain at r , overflow rejects) it is GCRA, which is provably the token bucket in different coordinates (Proposition 15.5): both admit bursts up to a tolerance and sustain r . As a *queue* (FIFO drained at r) it is the opposite of a token bucket: bursts are destroyed, output is perfectly smooth, and the cost is queueing delay up to Q/r . The interview trap is answering before asking which machine the question means.
- Q13. Derive GCRA's TAT update rule and its parameters.** Let $T = 1/r$ be the emission interval and τ the burst tolerance; keep TAT, the theoretical arrival time of the next conforming request. On arrival t_a : reject if $t_a < \text{TAT} - \tau$ (the request is earlier than tolerated jitter allows); else admit and set $\text{TAT} \leftarrow \max(t_a, \text{TAT}) + T$ — each admission defers the schedule by one interval, and the max re-anchors after idle time so credit never accrues beyond τ . Retry-After falls out as $\text{TAT} - \tau - t_a$.
- Q14. When is the sliding window log worth its memory?** When exactness is the product: authentication attempts, payment authorization, license seat check-in — anywhere “approximately 5 attempts” is not a defense. Note N is tiny in exactly those domains, so the $O(N)$ log is cheap precisely where you need it.
- Q15. State and bound the sliding window counter's error.** Estimator $\hat{c} = c + p(1 - f)$, with p the previous window's count and f the elapsed fraction. Error $|\hat{c} - c_{\text{true}}| \leq p \max(f, 1 - f) < p \leq N$, worst when last window's traffic was bunched at one end — i.e., worst right after the limit mattered most (Proposition 15.3).
- Q16. Why is INCR-then-reject in Redis still wrong?** It is atomic but semantically broken: rejected requests consume increments, so

an attacker (or a retrying client) inflates the counter and poisons the window for everyone, including itself; the count also stops meaning “admitted requests,” breaking your headers and metrics. Check-and-admit must be conditional inside one script.

Q17. Design the Redis+Lua sliding window. What commands, and why one script? ZREMRANGEBYSCORE to evict, ZCARD to count, conditional ZADD (unique member, timestamp score), PEXPIRE for memory hygiene, all in one Lua script because Redis runs scripts atomically — the evict-count-admit sequence can never interleave with another client’s (Listing 15.3.4). Return remaining and a millisecond retry-after so the caller can emit accurate headers.

Q18. Your limiter’s Redis dies at 2 a.m. Walk me through the options. Fail-open (serve everything; abuse window until recovery; right for read-heavy public APIs), fail-closed (reject everything; right for login/payment; must exempt health checks or the load balancer buries you), or degraded-local (per-replica limit/ R fallback; bounded over-admission; the default when a local limiter exists). The decision belongs in a per-endpoint-class matrix made before the incident (Table 15.2).

Q19. How do you choose B for a token bucket? From the legitimate burst shape of your best-behaved heavy client: e.g., a mobile fleet that syncs 20 records on reconnect needs $B \geq 20$ even if its sustained rate is 2/s. Set r from capacity, B from legitimate burst, and reject the temptation to use B to absorb *illegitimate* bursts — that is what 429 and shedding are for.

Q20. Per-IP limiting: when does it help and when does it hurt? Helps pre-authentication (login, key exchange) as a coarse outer net. Hurts behind NAT/CGN (one IP = an enterprise: collective punishment) and against distributed attacks (botnets divide the limit by pool size). Never the only dimension — see the case study (Section 15.5.1).

Senior (Q21–Q30)

Q21. Prove the token bucket’s envelope and explain why it is tight. Tokens are conserved: admissions in $[t_0, t_1]$ remove A tokens, supply is the initial $\leq B$ plus refill $r(t_1 - t_0)$, so $A \leq B + r\Delta t$; attained by arriving as tokens accumulate. Tightness matters: *any* limiter admitting bursts of B and sustaining r must admit at least this envelope, so the token bucket is the most permissive limiter

with those two guarantees — the exact frontier of the design space.

- Q22. How would you enforce one global limit across three regions with no synchronous cross-region calls?** Regional cells with home-cell pinning: consistent-hash each key to a region, enforce there; accept that a client active in two regions gets up to $2\times$ during failover windows. If the limit is billing-grade, quotas must be exact: make the quota path single-home (one authoritative region per key) and accept its latency, while rate protection stays regional. State the consistency/latency frontier explicitly in the design doc.
- Q23. Design rate limiting for a multi-tenant SaaS.** Layered: (1) per-IP coarse net pre-auth; (2) per-API-key token bucket for rate, sized by plan tier; (3) per-account monthly quota for billing; (4) per-endpoint cost weights so exports and batch jobs drain budget proportionally; (5) per-route concurrency caps on expensive endpoints; (6) global shedding by priority class when the system itself is hot. State in Redis cells, one Lua script per decision, consistent-hashed by tenant; noisy-neighbor isolation comes from (2)+(4), plan differentiation from (3), survival from (5)+(6).
- Q24. How do you rate limit GraphQL by cost?** The route is constant (`POST /graphql`); the *query* is the cost. Parse and statically score it: sum per-field weights, multiply by pagination first-arguments along list edges, penalize depth (or reject depth past a ceiling outright), charge introspection separately, and debit the score from a per-key token bucket *before* execution — ideally re-measuring after execution to calibrate weights. This is the query-cost analysis of Chapter 13 wired into this chapter's admission machinery; without it, one nested query walks through a requests-per-minute limit shaped for REST.
- Q25. What is AIMD and where does it fit in limiting?** Additive increase, multiplicative decrease: $L \leftarrow L+a$ while healthy, $L \leftarrow \beta L$ ($\beta < 1$) on overload signals (p99 latency, 5xx, queue depth). It adapts limits to a capacity you cannot measure directly, and the asymmetry — decrease faster than you increase — is what makes it safe. Same mathematics as TCP congestion control; commonly applied to concurrency limits server-side and to per-tenant ceilings during brownouts.
- Q26. Your RateLimit-Remaining says 3 but the next request is rejected. List the causes.** Multi-replica skew (client hit

a different cell or a local limiter); concurrent requests from the same key (remaining is a point-in-time read, not a reservation); cost-weighted charging (the next request costs more than 1); a second, stricter dimension (per-endpoint or global bucket) firing; clock skew between cells; or a lying middleware — which is why Listing 15.3.5 tests headers across window boundaries.

- Q27. How do you size a login endpoint's defenses?** Per-account exact sliding log (tiny N , so exactness is free), per-IP loose outer net, global endpoint token bucket as a circuit-level budget, progressive delay on repeated failures, and velocity alerting on the global attempt rate. Fail-closed for the account limiter, fail-open for the IP net, and never let any of it 429 the health check. Justify each layer against the case study's three errors (Section 15.5.1).
- Q28. Compare load shedding and rate limiting operationally.** Limiting is per-identity, steady-state, contractual: clients are told their budget and 429 means "you specifically, slow down." Shedding is identity-agnostic, incident-state, protective: 503 + **Retry-After** means "everyone, come back later." Shedding needs a priority order decided in advance (critical > interactive > batch), must shed cheaply (before parsing bodies), and must never conflate the two statuses or clients' backoff logic misfires.
- Q29. How do you test a rate limiter?** Injected clock, never sleeps; prove the documented envelope (boundary burst, long-run rate, equivalence to a reference) as executable tests; replay seeded adversarial timelines; thread-storm the distributed path to hunt over-admission; TestClient the header contract across window boundaries; and test both failure policies with a limiter double that always raises. Listings 15.3.2–15.3.6 are the worked example.
- Q30. Envoy/gateway versus in-app limiting: where should limits live?** Both, for different scopes. Edge/gateway limits (Envoy local or global rate-limit service) stop volumetric and pre-auth abuse cheaply, before your app spends anything. In-app limits enforce business semantics the gateway cannot see: per-plan quotas, cost-weighted budgets, per-account attempt counts. The anti-pattern is either alone: gateway-only cannot price work; app-only lets junk traffic consume your accept queue, TLS, and parse budget.

Staff (Q31–Q36)

- Q31. Design rate limiting for a platform at 10^5 rps across five regions, with hard per-tenant fairness and a per-request limiter latency budget of 1 ms. What falls out?** A synchronous global cell is dead on arrival (cross-region RTT alone eats the budget), so: local in-process token buckets on the hot path, budgets sharded as limit/R with gossip/cell-sync reconciliation on a 100 ms–1 s cadence, exact enforcement reserved for the narrow exactness-critical surface (auth, payments) routed to home cells. Fairness becomes a *bounded-error* claim: $\text{over-admission} \leq \text{replica count} \times \text{sync lag in window units}$ — publish the bound, monitor against it, and reserve exactness for where the business pays for it.
- Q32. A customer’s retry storm is indistinguishable from a DDoS at the edge. How do you tell them apart, and how does the answer change what the limiter does?** Intent is invisible; *shape* is not: customer storms carry valid auth, stable key sets, and requests that *would* succeed if admitted, while attack traffic fails auth or probes randomly. For the customer storm, rejection must be maximally informative (accurate **Retry-After**, jitter guidance) because they will obey it; for attack traffic, cheapness dominates — reject before auth, before parsing, ideally at a layer you do not pay per-request for. Same 429, different economics; a limiter that cannot reject junk cheaply is a cost amplifier.
- Q33. Derive the over-admission bound for R local limiters with sync interval Δ and window W , and defend or attack the design.** Each replica enforces N/R locally; between syncs a key can be admitted N/R on *each* replica, so worst-case admissions per W are N plus drift: replicas observe each other’s consumption Δ late, adding at most $R \cdot r \cdot \Delta$ where r is the attack rate; with adversarial split-brain routing the bound degrades to $R \times N/R = N$ *per unsynced window* plus the drift term. Defense: the bound is honest, tunable (shrink Δ , over-divide N), and failure-safe (a dead replica’s share is lost, not stolen). Attack: under targeted abuse the attacker routes around awareness, so exactness-critical surfaces must not use it. Both are correct; that is the point of publishing the bound.
- Q34. When should limits be contractual (documented, stable, versioned) versus dynamic (adaptive, opaque)? What does each cost the ecosystem?** Contractual limits are an API surface: SDKs codegen backoff logic against them, integrators size batch

jobs against them, and changing them is a breaking change in the Chapter 10 sense — announce, deprecate, migrate. Dynamic limits adapt to capacity you cannot promise, but opacity pushes integrators toward defensive polling and support tickets. The mature posture: contract the *floor* (“you will always get at least X ”), keep the headroom dynamic, and version the policy string itself so header semantics never shift under a pinned integration.

- Q35. Design the migration from a broken limiting system (fixed window, per-IP only, no headers) to a correct one on a live public API.** Expand-and-contract (Chapter 10): (1) emit the new headers *observationally* alongside the old limiter — log what the new algorithms *would* have done; (2) enforce the new limiter in shadow mode, comparing admit sets to bound behavioral delta; (3) tighten enforcement per tenant cohort with per-cohort kill switches, starting with internal keys; (4) publish the policy change with a deprecation window for clients whose traffic the new envelopes will newly reject; (5) retire the per-IP-only path last, after auth coverage makes it redundant. At every step, the headers tell clients what is coming before enforcement arrives.
- Q36. Is rate limiting a security control, a reliability control, or a billing control? Argue all three, then prioritize.** Reliability: it is the cheap-rejection valve that keeps offered load below the capacity knee — the primary justification. Security: it raises the cost of credential stuffing, scraping, and inventory abuse, but it is a *friction* control, not a barrier — authentication, authorization, and detection remain the barriers (Chapter 17). Billing: quotas encode the business model, and metering-grade exactness there is a different (harder, single-home) engineering problem than protection-grade approximation. Prioritize reliability — a down API has no security or revenue — but design the dimensions so security and billing ride the same machinery with their own exactness budgets.

15.8 External Bibliography & Further Reading

- IETF HTTPAPI Working Group, *RateLimit Header Fields for HTTP* (Internet-Draft) — the standardizing vocabulary for `RateLimit-Limit/-Remaining/-Reset` and `RateLimit-Policy`; read the draft’s deployment notes for the epoch-versus-delay-seconds reset history [Pol24].
- RFC 9110, *HTTP Semantics* — `Retry-After` (both forms), 429’s

relationship to the 4xx family, and 503 semantics for overload [Fie22a].

- RFC 6585, *Additional HTTP Status Codes* — where 429 is actually registered (uncited in text; short and worth reading in full).
- M. Nygard, *Release It!*, 2nd ed. — capacity knees, the stability patterns (bulkheads, circuit breakers) that rate limiting complements, and why unbounded queues fail slowly [Nyg18].
- M. Kleppmann, *Designing Data-Intensive Applications* — the shared-state and replication background behind Section 15.2.9’s atomicity and cell arguments [Kle17].
- Cloudflare, *Counting things: a lot of different things* (engineering blog) — the sliding-window-counter error analysis on real traffic that Proposition 15.3 formalizes.
- Stripe, *Rate limiters* (engineering blog) — production token-bucket fleet design, including the load-shedder priority classes that inspired Section 15.2.11.
- Kong, *How to Design a Scalable Rate Limiting Algorithm* (engineering blog) — the gateway-level view of the same algorithm zoo, including the cluster-versus-local trade-off.
- NGINX, *Rate Limiting with NGINX and NGINX Plus* — the `limit_req` documentation where GCRA hides in plain sight (the `rate` and `burst` parameters map to T and τ).
- Redis documentation, *Rate limiter patterns* — the canonical INCR/EXPIRE and sorted-set recipes, including the atomicity caveats our Lua script resolves.
- ITU-T I.371 / ATM Forum TM 4.0 — the original Generic Cell Rate Algorithm specification; historically illuminating for how little the problem has changed.
- FastAPI documentation — middleware and `TestClient` used throughout Listings 14.6–14.7 [Ram24].

15.9 Capstone Projects

Capstone 14.1 — Prove the Algorithms [junior]

Objective: Reproduce this chapter’s entire proof sheet from a standing start, then break it on purpose.

Inputs/Datasets: Listings 14.1–14.2; no datasets.

Steps: (1) Reimplement the five limiters from their mathematical definitions only (close the book; keep the **Decision** contract). (2) Port the 14 tests of Listing 15.3.2 to your implementation unmodified. (3) Introduce one deliberate bug per algorithm (e.g., forget the max in GCRA’s TAT update) and record which test catches each. (4) Write one new test: the fixed window’s 2x burst with a *non-integer* window (e.g., $W = 1.5$).

Acceptance Criteria:

- ☐ All 14 ported tests pass against your reimplementation.
- ☐ Every seeded bug is caught by at least one existing test; you can name the mapping.
- ☐ The new non-integer-window burst test proves 20 admissions in a sub-window interval.

Stretch Goals: Property-test the sliding log with random seeded timelines, asserting Proposition 15.2’s invariant on every run.

Capstone 14.2 — Header-Complete Middleware [mid]

Objective: Extend Listing 15.3.5 into middleware you would publish as a platform library.

Inputs/Datasets: Listings 14.1, 14.6, 14.7; the Northwind Robotics endpoint set (`/telemetry`, `/fleet/status`, `/reports/export`, `/health`).

Steps: (1) Swap the in-process limiter for the Redis limiter of Listing 15.3.4 behind the `KeyedLimiter` protocol (add a `reset_after` implementation). (2) Emit an RFC 9457-style problem body (Chapter 8) on 429, including the policy and a documentation link. (3) Add an `X-NR-Limiter-Degraded: true` signal for fail-open episodes (headers omitted, signal present). (4) Test: headers across a window boundary, problem body shape, degraded signal, and 401 versus 429 separation.

Acceptance Criteria:

- ☐ All Listing 15.3.5 tests pass unmodified against the Redis-backed wiring.
- ☐ 429 bodies validate against a declared problem-details schema.

- ☐ Fail-open episodes emit the degraded signal and no `RateLimit-*` fields; fail-closed returns 503 + `Retry-After`.
- ☐ Full suite is deterministic and offline.

Stretch Goals: Add per-plan policies ("`100;w=60`" vs. "`1000;w=60`") selected by API key tier.

Capstone 14.3 — Cell Architecture Simulator [mid]

Objective: Quantify, rather than assert, the over-admission of cell-based limiting.

Inputs/Datasets: Listing 15.3.3's harness pattern; a seeded multi-client timeline generator you write (10 keys, Poisson-ish arrivals, one adversarial key).

Steps: (1) Implement three enforcement topologies over the same timeline: one global limiter (exact), $R = 4$ local limiters at limit/R with no sync, and $R = 4$ local limiters with a sync every Δ seconds (reconciliation subtracts observed remote consumption). (2) Sweep $\Delta \in \{0.1, 0.5, 1.0, 5.0\}$ s and record worst-case per-window admissions for the adversarial key. (3) Produce a pgfplots-ready table of over-admission versus Δ . (4) Verify the Q33-style bound: $\text{admissions} \leq N + R \cdot r \cdot \Delta$.

Acceptance Criteria:

- ☐ Global topology never exceeds the limit; no-sync topology's overshoot matches $R \times$ prediction.
- ☐ Sync topology's measured overshoot respects the published bound at every Δ .
- ☐ All results reproducible from one seed.

Stretch Goals: Add a cell-failure event mid-timeline and show the consistent-hash remap confines counter loss to $1/C$ of keys.

Capstone 14.4 — Adaptive Limits with AIMD [senior]

Objective: Build and tune an AIMD-adaptive per-tenant limiter that tracks a moving capacity.

Inputs/Datasets: Listing 15.3.1; a synthetic "capacity signal" generator you author: service capacity oscillates between 150 and 450 rps on a seeded schedule, and overload is reported when admitted rate exceeds current capacity.

Steps: (1) Implement the controller: per-second evaluation, $L \leftarrow L + a$ when healthy, $L \leftarrow \beta L$ on overload, floored at a contractual minimum.

(2) Sweep (a, β) over a small grid; for each pair record time-to-first-contact, steady-state oscillation amplitude, and total rejected load. (3) Identify the knee: the fastest a whose β -paired oscillation never exceeds capacity by more than 10%. (4) Write the tuning memo: why multiplicative decrease is the safety parameter and additive increase merely the efficiency parameter.

Acceptance Criteria:

- ☐ Controller never lets admitted rate exceed capacity by $> 10\%$ for more than one evaluation interval, for the tuned (a, β) .
- ☐ Floor is never violated even under sustained overload.
- ☐ Grid results and the knee are reproducible from the seed; memo quantifies the asymmetry argument.

Stretch Goals: Add per-tenant fairness: divide recovered headroom proportional to recent demand rather than equally.

Capstone 14.5 — Rate-Limiting Architecture for Northwind at 100k rps [staff]

Objective: Produce the complete design document for rate limiting, quotas, and shedding across Northwind Robotics' public API surface at 10^5 rps in five regions.

Inputs/Datasets: This chapter; a synthetic endpoint inventory you author (40 endpoints across read/write/export/admin classes with measured cost weights); a synthetic tenant registry (3 tiers, 2,000 tenants, one known-abusive actor).

Steps: (1) Classify every endpoint into a protection tier (exactness-required, approximate-fair, shedding-only) and defend each classification. (2) Choose the topology per tier (global cell / regional / local-plus-sync) with latency budgets; justify the 1 ms hot-path budget's consequences. (3) Specify the header contract, quota endpoints, and the policy-versioning story. (4) Write the failure runbook: limiter-cell outage, abusive-tenant drill, capacity-brownout (AIMD kick-in), including the fail-open/fail-closed assignment per tier. (5) Model cost: Redis cell sizing from key cardinality and N , and the egress cost of rejecting cheaply versus serving slowly. (6) Red-team your own design with the Section 15.5.1 playbook.

Acceptance Criteria:

- ☐ Every endpoint has a tier, a topology, a failure policy, and a numeric limit with a derivation (not a round number from nowhere).
- ☐ Over-admission bounds are stated and monitored per approximate tier.

- ☐ Runbook covers the three named incidents with detection signals and rollback steps.
- ☐ Cost model shows limiter infrastructure < 2% of serving cost at 10⁵ rps, or explains why not.
- ☐ Red-team section names at least two residual weaknesses honestly.

Stretch Goals: Specify the GraphQL cost-scoring integration (Chapter 13) and the gRPC interceptor analogue (Chapter 14) so all three protocol surfaces share one budget.

15.10 Python Dependencies Appendix

All code runs offline after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

requirements.txt:

```
1 fakeredis>=2.23
2 fastapi>=0.110
3 httpx>=0.27
4 pytest>=8
```

For real deployments (never needed by this chapter's listings): add `redis>=5` and swap `fakeredis.FakeRedis()` for `redis.Redis(...)` — the Lua script, calling convention, and **Decision** contract are identical.

15.10.1 fakeredis (≥ 2.23)

Description: In-process emulation of the Redis server, including Lua script execution. **Why included:** lets the distributed limiter (Listings 14.4–14.5) and its thread-storm atomicity test run with zero infrastructure — the same code paths, the same script, no server, no containers. **Alternatives:** a real Redis in Docker (faithful, including wall-clock expiry timing, at the price of infrastructure); `testcontainers`-based fixtures (CI-friendly, still heavyweight); mocking `eval` at the client boundary (fast but verifies nothing about the script). **Installation:** `pip install "fakeredis>=2.23"` (Lua support requires the bundled `lupa` dependency; the pinned distribution includes it).

15.10.2 fastapi (≥ 0.110)

Description: The ASGI web framework hosting the middleware demonstrations. **Why included:** routing, dependency injection, and the `TestClient` that makes Listing 15.3.5's header assertions runnable offline [Ram24]. **Alternatives:** Starlette directly (FastAPI's base; the middleware is raw ASGI and would work unchanged); Flask (WSGI — the ASGI middleware pattern would need reworking); a gateway doing the limiting instead (Envoy local rate limit, Kong) for the edge layer. **Installation:** `pip install "fastapi>=0.110"`.

15.10.3 httpx (≥ 0.27)

Description: Sync/async HTTP client. **Why included:** it is the transport under FastAPI's `TestClient`; required for the middleware tests to execute without a server. **Alternatives:** `requests` (synchronous, not ASGI-native); running a live `uvicorn` and a real socket (faithful but no longer offline-pure). **Installation:** `pip install "httpx>=0.27"`.

15.10.4 pytest (≥ 8)

Description: Test framework. **Why included:** runs all 35 chapter tests, including the executable proofs and the concurrency storm. **Alternatives:** `unittest` (stdlib; works, but fixtures and `approx` would need hand-rolling); `nose2` (unmaintained momentum). **Installation:** `pip install "pytest>=8"`.

Ecosystem alternatives worth knowing

For application-level limiting beyond this chapter's teaching implementations: `limits` (mature strategy library: fixed/sliding algorithms over Redis/Memcached/in-memory backends), `slowapi` (FastAPI/Starlette integration built on `limits`), `django-ratelimit` (Django decorator), and Envoy's *local rate limit* filter / global rate-limit service for the edge tier. Evaluate them against Section 15.2: which algorithm, which atomicity story, which failure policy, which headers.

15.11 Chapter Summary & Key Takeaways

- Five tools, five questions: rate limiting (how fast may this caller start), throttling (shape the flow), quotas (how much in total), concurrency limits (how many in flight), load shedding (which work do we drop).

Deploying one and claiming the others is the canonical maturity failure.

- The fixed window's $2x$ boundary burst is a theorem with a tight bound, not folklore (Theorem 15.1); if the limit protects something with a hard per-second ceiling, the fixed window does not enforce it.
- The sliding log is exact at $O(N)$ memory; the sliding counter buys $O(1)$ at error $<$ previous-window count (Proposition 15.3), worst exactly after saturation; the token bucket owns the tightest burst-plus-rate envelope $A \leq B + r\Delta t$ (Proposition 15.4); GCRA is the token bucket in one timestamp (Proposition 15.5).
- Distribution moves the problem from algorithms to state: atomic check-and-admit via one Lua script in one cell; consistent hashing to bound remap on cell loss; local-plus-sync trades exactness for latency with a publishable over-admission bound.
- The failure policy is a business decision made in advance, per endpoint class: fail-open, fail-closed, or degraded-local (Table 15.2); a fail-closed limiter without a health-check exemption is a self-inflicted outage.
- Headers are the contract: 429 + **Retry-After** (delay-seconds, ceiled), **RateLimit-Limit/-Remaining/-Reset/-Policy** on successes too, 503 for shedding, quota endpoints for budgets. Omit what you cannot compute truthfully; never fake it [Pol24; Fie22a].
- Multi-dimensional is the default at maturity: per-key for fairness, per-IP as a coarse pre-auth net (never the only net — Section 15.5.1), per-endpoint and cost-weighted for expensive routes, AIMD for adapting to unmeasurable capacity, priority-aware shedding for survival.
- Test limiters with injected clocks, seeded timelines, thread storms, and a limiter double that always raises — then both failure paths are tested before the 3 a.m. page tests them for you.

15.12 Quick-Reference Card

Algorithm Matrix (one line each)

Fixed window — $O(1)$, crisp reset, $2N$ boundary burst • sliding log — exact, $O(N)$ memory, **Retry-After** = oldest expiry • sliding counter — $O(1)$, error $< p$, smooths boundaries • token bucket — burst B , sustained r , envelope $B + r\Delta t$ • GCRA — one timestamp, reject iff $t_a < \text{TAT} - \tau$,

$TAT \leftarrow \max(t_a, TAT) + T, B = 1 + \tau/T.$

Header Reference

429 *Too Many Requests* — caller over budget; always with **Retry-After:** `<delay-seconds>` (ceiled; HTTP-date is legal, worse) • **RateLimit-Limit** — window quota • **RateLimit-Remaining** — budget left after this response • **RateLimit-Reset** — delay-seconds to full reset • **RateLimit-Policy** — the rule, e.g. "100;w=60" • 503 + **Retry-After** — system shedding, not the caller's fault • 401/403 — no key, no bucket: never 429 an anonymous caller.

Decision Tree (compressed)

Shared limit across replicas? → Redis cell, one Lua script; hash keys to cells • Exactness required? → sliding log • Bursts legitimate? → token bucket / GCRA, B from client burst shape • Huge smooth keyspace? → sliding counter • Crisp headers, rough protection? → fixed window, short W • Limiter down? → per-class policy: fail-open (reads), fail-closed (auth/payment), degraded-local (default) • Expensive route? → cost weight + concurrency cap + consider 202-async.

15.13 Standardized Evidence Addendum

15.13.1 Benchmark Snapshot Template

15.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

15.13.3 Cost and Latency Budget Snapshot

15.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

15.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

15.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

15.17 Cross-Chapter Decision Card

15.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

15.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Algorithm	Memory/key	Burst behavior	Boundary	Choose when
Fixed window	$O(1)$	up to $2N$ at edges	hard, exploitable	Crisp re-set headers; rough protection; short windows
Sliding log	$O(N)$	none beyond N	none (exact)	Correctness-critical, modest N (auth attempts, payments)
Sliding counter	$O(1)$	bounded by Prop. 15.3	smoothed	Huge key space, smooth traffic, tolerance for $\pm p$ error
Token bucket	$O(1)$	exactly B , then r	none	General default; independent rate/burst knobs
GCRA	1 timestamp	exactly $1 + \tau/T$	none	Single-field state; gateway parity; pre

Policy	Behavior on limiter outage	Blast radius of the choice	Right for
Fail-open	Serve everything; emit no unverifiable headers	Abuse window until recovery; possible cascade if the outage <i>was</i> load	Read-heavy public APIs, telemetry ingest
Fail-closed	Reject (503 + Retry-After)	Total API outage for the duration; safe against abuse	Login, password reset, payment authorization
Degraded-local	Fall back to per-replica limit limit/R	Bounded over-admission ($\leq R\times$), no outage	The default for most multi-replica services

Table 15.2: Failure-policy decision matrix. Degraded-local dominates when a cheap local limiter is available; the fail-open/fail-closed binary is for endpoints where either abuse or availability is existentially worse.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 15.3: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 15.4: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 15.5: Decision card to complete per chapter.

Chapter 16

Microservices & Resilient Inter-Service Communication

Splitting a monolith into services does not remove coupling; it moves it from the compiler's reach, where it is cheap and visible, onto the network, where it is expensive, latent, and partially observable. A method call that used to take nanoseconds and fail only when the process died now takes milliseconds, fails in at least five distinguishable ways, and — this is the part that hurts — sometimes *does not fail at all*, it merely gets slow, and slow is worse than down because slow consumes the caller's resources while offering no error to react to. This chapter is about engineering that reality rather than discovering it during an incident.

We begin with the communication taxonomy: synchronous request/response (REST, gRPC) versus asynchronous messaging (queues, event streams), and orchestration versus choreography as the two ways to coordinate multi-step work. We then make coupling precise along three dimensions — *temporal* (must both parties be up simultaneously?), *spatial* (must the caller know where the callee lives?), and *logical* (must both agree on schema and semantics?) — and apply the eight fallacies of distributed computing as a checklist to every API call you write. The fallacies are not trivia; each one maps to a production failure mode with a name, a metric, and a mitigation.

The resilience core of the chapter treats five patterns with the rigor they deserve. Timeouts come first, because every other pattern is built on the assumption that failure detection is bounded; we derive the deadline budget rule $T_{\text{caller}} > \sum T_{\text{hops}}$ and show what breaks when it is violated. The circuit breaker is specified as a formal state machine — CLOSED, OPEN, HALF_OPEN, with a rolling failure window, explicit thresholds, and recovery probes — not as a vibes-based “it stops calling the broken thing.” Bulkheads isolate failure domains by partitioning thread pools, connection pools, and concurrency permits. Hedged requests attack tail latency and we price their amplification honestly. Backpressure closes the loop by giving overloaded

systems a way to say no upstream.

Two phenomena get full mathematical treatment. *Retry amplification*: we prove that an n -hop chain in which each hop issues a attempts per call multiplies one client request into $a(a^n - 1)/(a - 1)$ downstream requests — with three attempts per hop across three hops, that is $39\times$ load from a single user click (Theorem 16.1) — and we show how token-bucket retry budgets and jittered, deadline-aware backoff cap the damage. *Cascading failure*: we walk the death spiral stage by stage, from a latency blip through pool exhaustion, queue buildup, and retry storms to fleet-wide outage, with a failure-mode analysis table that maps each stage to its early signal and its countermeasure.

The architectural second half covers service discovery and load balancing (client-side, server-side, DNS, registries), the service mesh (sidecar proxies, mTLS identity, data plane versus control plane — and a blunt account of what the mesh does *not* fix, namely your application semantics), and distributed transactions without two-phase commit: sagas with compensating actions, the transactional outbox, idempotent consumers, and how to expose eventual consistency honestly in your API contracts. Every claim is executable: five typed Python 3.11 listings — a production-grade circuit breaker, a retry-budget client, a three-service chaos drill, a SQLite-backed outbox with relay, and a saga orchestrator — run fully offline with deterministic tests, in the Northwind Robotics universe.

Key Concept

Resilience is a **budget discipline**, not a library. Every outbound call spends three budgets: *time* (deadline), *concurrency* (bulkhead permits), and *retry allowance* (token bucket). A call site that cannot say how much of each budget it may spend is an unpriced risk. The circuit breaker then enforces the meta-rule: when a dependency proves it cannot honor its budget, stop paying until it recovers.

16.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Classify any inter-service interaction along the synchronous / asynchronous axis and the orchestration / choreography axis, and decompose its coupling into temporal, spatial, and logical dimensions with a concrete decoupling tactic for each.

- LO-2.** Apply the eight fallacies of distributed computing as a design review checklist, naming the failure mode and the metric that exposes each fallacy when an API call ignores it.
- LO-3.** Derive deadline budgets across multi-hop call chains ($T_{\text{caller}} > \sum T_{\text{hops}}$), choose timeout values from measured latency distributions, and explain why a timeout longer than the caller's is worse than no timeout.
- LO-4.** Derive the retry-amplification formula $S(n, a) = a(a^n - 1)/(a - 1)$ (Theorem 16.1), and design retry policies — budgets, full jitter, deadlines, server-driven throttling — that provably cap amplification.
- LO-5.** Specify, implement, and test a circuit breaker as a formal state machine: rolling failure window, minimum-call floors, open duration, half-open probe limits, and recovery thresholds.
- LO-6.** Choose and combine bulkheads, hedged requests, and back-pressure mechanisms for a given dependency, including the tail-amplification cost of hedging.
- LO-7.** Trace a cascading failure stage by stage, attach an early-warning signal and a countermeasure to each stage, and run a chaos drill that measures — not asserts — the protection a breaker buys.
- LO-8.** Evaluate service discovery and load-balancing topologies (client-side, server-side, DNS, registry) and articulate precisely which concerns a service mesh absorbs (mTLS, retries, telemetry, traffic policy) and which it cannot (idempotency, sagas, schema semantics).
- LO-9.** Replace distributed transactions with sagas, compensating actions, the transactional outbox, and idempotent consumers, and present the resulting eventual-consistency boundaries honestly in API contracts.

Prerequisites

This chapter assumes Chapters 0–14 and leans hardest on four of them: consuming HTTP APIs in Python, including `httpx` clients, timeouts, and transports (Chapter 3); idempotency keys and concurrency control, which become *load-bearing* once retries and at-least-once delivery enter the picture

(Chapter 11); gRPC, whose deadlines, status codes, and streaming flow control recur throughout (Chapter 14); and rate limiting, whose token-bucket machinery we reuse verbatim for retry budgets (Chapter 15). Familiarity with building FastAPI services (Chapter 7) and with asynchronous, event-driven API styles (Chapter 12) is assumed in the code sections. All listings run offline on Python 3.11+; install the dependencies from Section 16.10 once, from PowerShell.

16.2 Deep Technical Mechanics & Architectural Concepts

16.2.1 A Taxonomy of Inter-Service Communication

Every interaction between two services answers four design questions, whether or not anyone asked them. First, *does the caller block?* Synchronous communication (REST over HTTP, gRPC) holds the caller’s execution until a response or a timeout arrives; asynchronous communication (queues, event streams) hands the work to an intermediary and returns. Second, *who knows the workflow?* In **orchestration** a designated coordinator issues commands in sequence and owns the process state; in **choreography** each service reacts to events and emits new ones, and the workflow exists only as the emergent sum of those reactions. Third, *how many consumers?* Request/response is inherently point-to-point; event streams are naturally one-to-many. Fourth, *what is the consistency promise?* Synchronous chains pretend toward immediacy; asynchronous pipelines admit, and should advertise, eventual consistency.

Definition 16.1 (Orchestration and choreography). A multi-service workflow is **orchestrated** when one component (the orchestrator) explicitly invokes each participant in a known order and tracks overall progress. It is **choreographed** when no component knows the whole workflow: each participant subscribes to domain events, acts, and publishes its own events, and the workflow is the composition of those local reactions [New21].

Neither coordination style is a resilience pattern by itself; they differ in *where failure is managed*. An orchestrator can retry, compensate, and report a single workflow status — at the cost of being a stateful component you must make highly available. Choreography removes the coordinator as a bottleneck and a single point of failure, but scatters workflow state across event logs and inboxes, making “what is the status of order ORD-7?” a query you must deliberately engineer (a read model, a status API) rather than a row you can **SELECT**. Production systems mix both: orchestration inside a bounded context (the order saga of Listing 16.3.5), choreography

across context boundaries (order-placed events fanning out to analytics, notifications, and fraud screening).

Dimension	Synchronous (REST/gRPC)	Asynchronous (queue/stream)
Caller blocked	Yes, until re- sponse/timeout	No; hand-off to broker
Temporal coupling	High: both must be up	Low: broker buffers down- time
Failure surface	Timeouts, 5xx, partial re- sponse	Duplicates, reordering, lag, poison messages
Latency profile	Additive across hops	Broker hop + consumer pacing
Workflow visibility	Trace IDs across calls	Event correlation, read models
Best for	Queries, user-facing com- mands	Fan-out, long work, smoothing load

Table 16.1: Synchronous versus asynchronous inter-service communication.

16.2.2 Coupling Dimensions and the Fallacies of Distributed Computing

“Loosely coupled” is a slogan until you decompose it. Three dimensions cover most of what matters between services.

Definition 16.2 (Coupling dimensions). Two services are **temporally coupled** if a call requires both to be healthy at the same instant; **spatially coupled** if the caller must know the callee’s network location (host, port, instance identity); **logically coupled** if they must agree on schemas, semantics, and deployment cadence to interoperate.

Asynchronous messaging attacks temporal coupling directly — the broker holds the message while the consumer is down — and, combined with a registry, spatial coupling too. Logical coupling is the stubborn one: schema registries, consumer-driven contracts, and explicit versioning (Chapter 10) manage it but never eliminate it, because semantics live in code on both sides. A useful design-review question for every integration: *which of the three couplings does this call site have, and is that deliberate?*

The eight fallacies of distributed computing, first catalogued by Peter Deutsch and colleagues at Sun Microsystems, read like a list of things engineers believe precisely until their first major incident. Applied to inter-service API calls:

1. *The network is reliable.* Packets vanish; TCP connections half-open; the callee may process your request and lose the response. Consequence: you often cannot distinguish “not done” from “done but unacknowledged” — which is why idempotency keys (Chapter 11) are a prerequisite for safe retries, not an optional garnish.
2. *Latency is zero.* In-process calls cost nanoseconds; network calls cost milliseconds, with a tail that is 10–100× the median. Tail latency, not the mean, sets your timeout budget.
3. *Bandwidth is infinite.* Chatty fine-grained APIs that were free inside a process become serialization and egress bills across a network. Coarse-grained payloads and pagination (Chapter 9) exist because of this fallacy.
4. *The network is secure.* Every hop is a trust boundary; mTLS and service identity (Section 16.2.9) replace the fiction of a trusted LAN.
5. *Topology doesn’t change.* Autoscaling, rolling deploys, and container rescheduling move your callees constantly; hardcoded addresses are latent outages (spatial coupling again).
6. *There is one administrator.* Your dependency is run by another team with its own deploy cadence, SLOs, and incidents; version skew is the steady state, not an anomaly.
7. *Transport cost is zero.* Serialization, encryption, and proxying burn CPU on both sides; meshes add hops. Measure the tax.
8. *The network is homogeneous.* Mixed client libraries, proxy generations, and TLS stacks mean the wire contract — headers, status codes, HTTP semantics (Chapter 2) — is the only thing you can truly rely on.

Warning

The most dangerous sentence in microservice design is “it is just an internal call.” Internality does not repeal the fallacies; it usually means *fewer* timeouts, retries, and contracts than the public API gets. Apply the same discipline inward that you would demand of a third-party integration — your VPC is still a network.

16.2.3 Timeouts Everywhere, and How to Budget Them

A call without a timeout is a promise to wait forever, and “forever” is the one deadline production will always accept. Timeouts are the foundation every other pattern in this chapter stands on: the circuit breaker’s failure counter is fed mostly by timeout expirations; bulkheads only help if blocked workers are eventually released; retry budgets assume each attempt has a bounded cost.

Timeout selection is a measurement exercise, not a guess. Take the dependency’s latency distribution under realistic load — not the mean, the tail — and set the timeout at a high percentile with headroom, e.g. $p_{99.9}$ plus a margin, then validate with the chaos drill of Section 16.5. Too tight and you convert slow successes into failures (and retries, and amplification); too loose and you hold resources past the point where the caller still cares.

The structural rule is the **deadline budget**: a caller’s timeout must exceed the sum of the budgets it grants downstream, or the chain guarantees wasted work.

Definition 16.3 (Deadline budget rule). For a call chain $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n$ where T_i is the timeout C_{i-1} applies to its call to C_i , correct budgeting requires $T_i > \sum_{j>i} T_j$ for every hop — equivalently, budgets shrink strictly toward the leaves. gRPC makes this explicit by propagating a *deadline* (an absolute timestamp) in metadata, so every hop derives its remaining budget instead of re-guessing [gRP24].

Concretely, for the Northwind Robotics catalog chain: the public API grants the catalog request a 2 s end-to-end deadline; catalog budgets 800 ms for its inventory call; inventory budgets 400 ms for warehouse; warehouse budgets 200 ms for its database. Each layer keeps headroom for serialization, queuing, and its own logic. Section 16.4 catalogs what happens when a middle hop violates the rule (a timeout *longer* than the caller’s): the caller abandons the request while the callee keeps working — the definition of waste, and under load, of cascading failure.

16.2.4 The Mathematics of Retry Amplification

Retries are the first resilience tool everyone reaches for and the first amplifier of every outage. The amplification is multiplicative across hops, and it is worth deriving once, carefully, because the numbers are startling enough to change design instincts.

Theorem 16.1 (Retry amplification in an n -hop chain). Consider a call chain of n hops $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n$. Suppose each service C_i ($i < n$), upon receiving a call, issues up to a attempts to C_{i+1} when attempts fail

($a = 1 + r$ for r retries). Then one client request at C_0 generates up to a^k calls arriving at hop k , and the total number of downstream requests in the system is

$$S(n, a) = \sum_{k=1}^n a^k = a \frac{a^n - 1}{a - 1}.$$

Proof. By induction on the hop index. One request arrives at C_0 (base of the chain receives the client call). Each call arriving at C_i independently triggers up to a calls to C_{i+1} , so if L_k denotes the number of calls arriving at hop k , then $L_{k+1} = a \cdot L_k$ with $L_0 = 1$, giving $L_k = a^k$. The system-wide total is the geometric sum $\sum_{k=1}^n a^k = a(a^n - 1)/(a - 1)$. $\square$

The canonical numbers: a three-hop chain (edge $\rightarrow$ catalog $\rightarrow$ inventory $\rightarrow$ warehouse) in which every hop issues up to three attempts ($a = 3$: one initial try plus two retries) produces up to $3 + 9 + 27 = 39$ downstream requests per client request — a $39\times$ load multiplier from a single user action, with the leaf service absorbing $27\times$. If “three retries” means three retries *beyond* the first attempt ($a = 4$), the total is $4 + 16 + 64 = 84$. During an outage, when failure rates approach 100%, these maxima are not worst-case theory; they are the expected case.

Proposition 16.2 (Retry budgets cap amplification). Let each client process hold a token bucket of capacity B where every retry (not every attempt) consumes one token. Then regardless of configured `max_retries`, the attempts issued per logical request are bounded by $1 + B$, and the steady-state retry rate is bounded by the refill rate. Amplification becomes a controlled parameter, not an emergent property.

The proof is immediate — a token must exist for a retry to start — but the consequences are deep: the budget converts retries from a per-request decision into a *per-process allocation problem*, which is the only granularity at which “how much extra load may this client add” is answerable. Listing 16.3.2 implements exactly this, and Listing 16.3.2 proves the cap: configured for five retries against an always-failing dependency, a client with a two-token budget issues three requests and stops.

Three more disciplines complete the anti-amplification toolkit. **Jitter**: if N clients fail together at time t and all sleep the same deterministic backoff, they re-arrive together at $t + \Delta$ and re-collide; *full jitter*, drawing the sleep uniformly from $[0, \min(c, b \cdot 2^k)]$, decorrelates the herd while preserving the exponential envelope. **Deadline awareness**: a retry whose backoff would overshoot the caller’s remaining deadline is a retry that cannot help; skip it (Listing 16.3.2 raises `RetryDeadlineExceededError` rather than donating doomed work downstream). **Server-driven throttling**: when the server says 429 or 503 with `Retry-After`, that signal outranks the client’s local schedule — the server has information the client lacks, and honoring it is what turns a thundering herd into an orderly queue (Chapter 15).

16.2.5 The Circuit Breaker, Specified Formally

The circuit breaker, named by Nygard [Nyg18], wraps a dependency in a state machine that substitutes fast, cheap, honest failure for slow, expensive, resource-consuming failure. Everyone can draw the three states; production-grade breakers are distinguished by what the diagram leaves out: *what counts* as a failure, *how many* observations are required before the statistics mean anything, *who* is allowed to test recovery, and *when* old evidence expires.

Definition 16.4 (Circuit breaker). A circuit breaker is a per-dependency state machine over $\{\text{CLOSED}, \text{OPEN}, \text{HALF_OPEN}\}$ with parameters (θ, W, N, d, p, s) : failure-rate threshold $\theta \in (0, 1]$, rolling window W seconds, minimum call count N , open duration d , probe concurrency limit p , and probe successes s required to close. Transitions: $\text{CLOSED} \rightarrow \text{OPEN}$ when the window holds at least N calls with failure rate $\geq \theta$; $\text{OPEN} \rightarrow \text{HALF_OPEN}$ when d seconds have elapsed since opening; $\text{HALF_OPEN} \rightarrow \text{CLOSED}$ after s consecutive probe successes; $\text{HALF_OPEN} \rightarrow \text{OPEN}$ on any probe failure. While OPEN , calls are rejected immediately without touching the dependency.

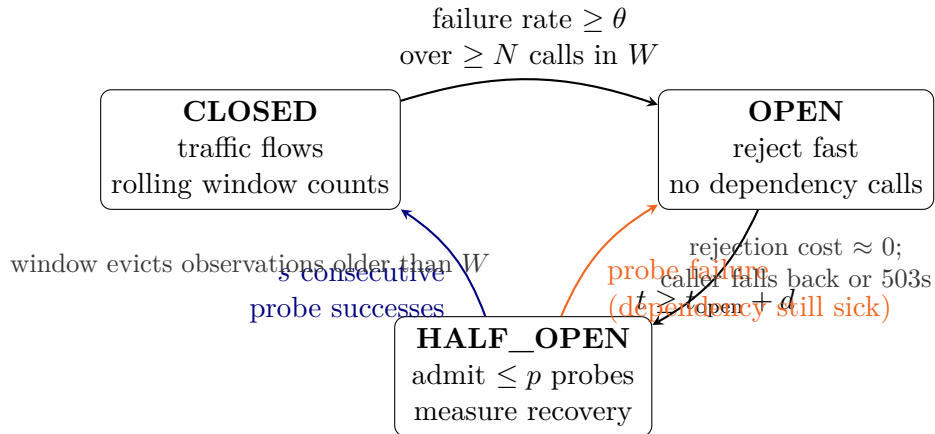


Figure 16.1: Circuit breaker state machine with the parameters of Definition 16.4. The half-open state is the recovery probe mechanism: a strictly limited number of trial calls decide whether the dependency has healed before normal traffic resumes.

Four specification details do the real work. **Minimum calls** N prevents statistical noise from tripping the breaker: two failures out of two calls at 3 a.m. must not open a circuit that carries ten thousand calls a minute at noon. **The rolling window** W makes the decision a statement about the *recent* past; a tumbling window would let a burst of failures at the boundary hide in one window and vanish in the next. **Probe limits** p and s make recovery gradual — one probe at a time, several successes before closing

— because a dependency that has just recovered is usually fragile, and a stampede of pent-up traffic is how recoveries become second outages. **One breaker per dependency:** the breaker’s state is a hypothesis about *a specific* downstream resource; sharing one breaker across two dependencies makes a sick dependency poison a healthy one (Section 16.4).

While open, the breaker reduces load on the sick dependency from this caller to approximately zero — p probe calls per open duration d — which is precisely the headroom the dependency needs to finish recovery. The caller, meanwhile, must decide what rejection *means*: fail the request (503 with a problem-details body, Chapter 8), degrade gracefully (serve a cached or partial answer), or queue for later. The breaker decides *when* to stop calling; the application still owns *what happens instead*.

16.2.6 Bulkheads, Hedged Requests, and Backpressure

Bulkheads partition resources so that one dependency’s failure cannot consume the capacity needed by everything else — named, like the breaker, for shipbuilding, via [Nyg18]. In practice a bulkhead is any of: a dedicated connection pool per dependency (an `httpx` client per downstream, with its own `Limits`), a semaphore bounding concurrent calls into one dependency (the `BULKHEAD_PERMITS` of Listing 16.3.3), or a separate worker pool per workload class. The sizing rule: permits for dependency d should be at most what d can healthily serve, and the sum of all permit pools must leave capacity for everything else the process does — health checks, metrics, other endpoints. A bulkhead that is larger than the process’s total capacity is decorative.

Hedged requests attack tail latency: if a call has not answered within the latency budget’s $p95$ – $p99$, issue a second, identical call to a different replica and take whichever answers first. The trade-off is explicit — you trade a bounded amount of extra load for a large reduction in tail latency. If the hedge fires when the first call exceeds delay h , and $\Pr[T > h] = q$, expected load per request becomes $1 + q$; with $q = 0.05$ that is a 5% tax for cutting the $p99$ dramatically. The corollaries are mandatory: hedge only idempotent operations (or carry idempotency keys), cancel or disregard the loser, and never hedge while the breaker is open — hedging into a sick dependency is amplification with a marketing budget.

Backpressure is the mechanism by which an overloaded component pushes load back upstream instead of collapsing silently. Its forms: explicit rejection (429/503 with `Retry-After`), bounded queues that fail fast when full (an unbounded queue is not a buffer; it is a latency generator with a memory leak), credit-based flow control (HTTP/2 stream windows, gRPC flow control [gRP24]), and consumer-side pacing in event systems (consumer

lag *is* the backpressure signal; Chapter 12). A system without backpressure converts overload into latency for everyone; a system with backpressure converts it into rejection for some — which is what makes the difference between a degraded service and a dead one.

16.2.7 Anatomy of a Cascading Failure

Cascading failures are the reason this chapter exists. They are also boringly predictable: the same stages, in the same order, in incident review after incident review. Figure 16.2 shows the spiral; Table 16.2 attaches signals and countermeasures.

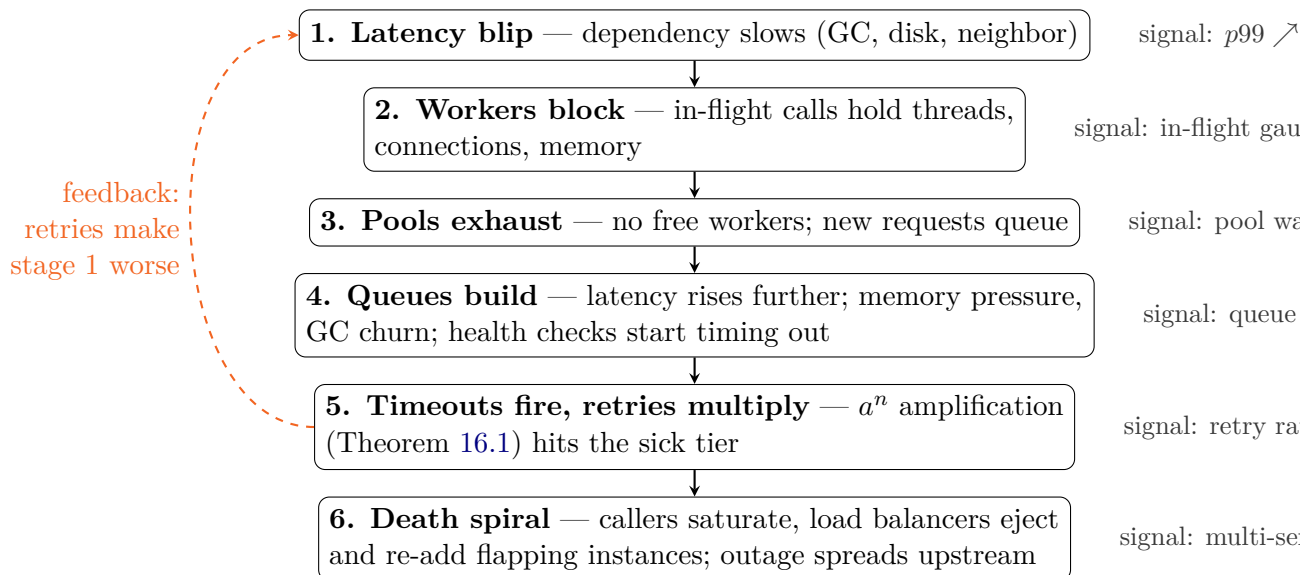


Figure 16.2: The cascading-failure death spiral. Each stage feeds the next, and the retry stage feeds back into the first. Every stage has an observable early-warning signal and a pattern that breaks the chain (Table 16.2).

Two properties make the spiral treacherous. First, *each stage is locally rational*: waiting a little longer, queueing a little more, and retrying once are all defensible decisions in isolation; the catastrophe is a property of the composition. Second, *recovery is not symmetric with collapse*: a service that fell over under load L cannot usually be restarted into load L — it must be brought back behind a breaker, a load shedder, or a traffic ramp, or it will fall over again immediately. The Google SRE book’s chapter on cascading failures makes the same point with considerably larger blast radii; the mechanics are identical at Northwind Robotics scale.

Stage	Early signal	Countermeasure	Where
Latency blip	<i>p</i> 99 latency rises, errors flat	tight per-hop timeouts	16.2.3
Workers block	in-flight calls gauge climbs	bounded concurrency	16.2.6
Pools exhaust	pool checkout wait > 0	bulkheads per dependency	16.2.6
Queues build	queue depth, GC time climb	bounded queues, load shedding	16.2.6
Retry storm	downstream QPS $\gg$ client QPS	retry budgets, jitter, breakers	16.2.4
Death spiral	correlated multi-service burn	all of the above, drilled	16.5

Table 16.2: Failure-mode analysis: each stage of the spiral, its observable precursor, and the pattern that breaks the chain.

16.2.8 Service Discovery and Load Balancing

Resilience patterns assume a *pool* of interchangeable backends behind each logical dependency; discovery and load balancing are how that pool is maintained and used. The design space has three axes: who resolves addresses, who picks an instance, and how membership health is known.

Server-side load balancing puts a dedicated load balancer between callers and the pool: clients resolve one stable virtual address (usually via DNS) and the balancer owns health checking, instance selection, and connection management. Simple for clients, operationally centralized — and an extra network hop and a component that must itself be made redundant. **Client-side load balancing** gives every client the instance list (from a registry such as Consul, Eureka, or Kubernetes’ API) and lets the client library pick: fewer hops, per-client policy (weighted, zone-aware, hedged), at the cost of embedding the logic — and its bugs — into every client, in every language you support. **DNS-based discovery** is the lowest common denominator: SRV or A records per service, resolved by the client’s stub resolver. It works everywhere and updates nowhere near fast enough for rapid membership churn, because TTLs cache staleness into the system by design.

Whichever topology you choose, the resilience-critical property is *health-check fidelity*: an instance must be removed from rotation when it is actually failing to serve (not merely when it fails to respond to a ping), and re-added gradually, because a freshly started instance hit with a full share of traffic is stage one of Figure [16.2](#) all over again.

	Server-side LB	Client-side LB	DNS-based
Extra hop	Yes	No	No
Client complexity	None	High	Low
Policy flexibility	Centralized	Per-client, rich	Nearly none
Membership freshness	Fast (active health checks)	Fast (registry watch)	Slow (TTL-bound)
Failure domain	LB tier	Each client	Resolver cache

Table 16.3: Discovery and load-balancing topologies.

16.2.9 Service Mesh: Data Plane versus Control Plane

A service mesh extracts cross-cutting communication concerns — discovery, load balancing, mTLS, retries, timeouts, telemetry — from application code into infrastructure. The canonical architecture is a fleet of **sidecar proxies** (Envoy being the reference implementation) forming the **data plane**: every packet in or out of a service transits its local sidecar, so policy executes one hop from the application, without the application’s cooperation or knowledge. Above them sits the **control plane** (Istio, Linkerd’s control components), which distributes configuration — routes, retry policies, certificates, load-balancing subsets — and aggregates telemetry, but never touches a request byte.

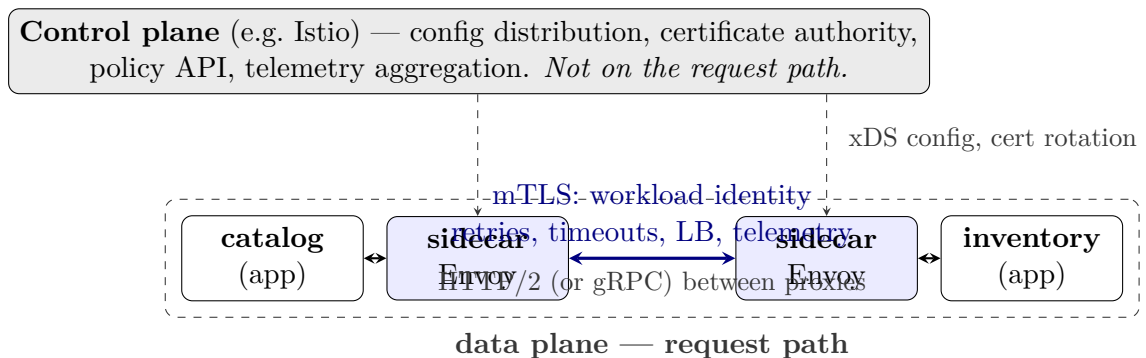


Figure 16.3: Service mesh anatomy. The data plane (sidecar proxies) executes traffic policy on the request path: mTLS identity, retries, timeouts, load balancing, telemetry. The control plane distributes configuration and certificates but never touches a request.

Three mesh capabilities matter for this chapter. **mTLS identity**: each workload receives a short-lived cryptographic identity (a SPIFFE-style verifiable ID), and the sidecars mutually authenticate and encrypt every hop — fallacy four handled by default, with identity-based authorization (“catalog

may call inventory”) replacing network-location-based trust. **Data-plane traffic policy**: retries, timeouts, circuit breaking (Envoy calls it outlier detection plus connection-pool circuit breakers), and load balancing execute identically for every service regardless of language, which eliminates the “every team re-implements resilience slightly wrong” problem. **Uniform telemetry**: golden signals per hop, with consistent labels, emitted by the proxy rather than by application diligence.

Warning

The mesh does **not** fix application semantics. It cannot make your endpoints idempotent, cannot choose your saga compensations, cannot version your schemas, cannot know which failures are safe to retry (mesh-level retries of a non-idempotent POST are exactly as dangerous as client-level ones), and cannot budget your deadlines end-to-end — it sees hops, not user requests. Teams that adopt a mesh and delete Chapter-10–15 discipline from their services get a beautifully observable, mutually authenticated cascading failure.

16.2.10 Distributed Transactions without 2PC

A single database gives you ACID; five services give you a choice of which letter to sacrifice. Two-phase commit (2PC) — a coordinator asks all participants to *prepare*, then commits only if all agree — is theoretically tidy and operationally avoided: it is blocking (a coordinator crash leaves participants holding locks in-doubt until recovery), it couples the availability of the whole to the availability of every participant, and its chatty prepare/commit rounds multiply latency across exactly the network boundaries this chapter has spent its length mistrusting [Kle17]. Over HTTP, where participants cannot even hold a lock-shaped resource across stateless requests, 2PC is an anti-pattern on arrival (Section 16.4).

The pragmatic replacement is the **saga**: a sequence of local transactions $T_1, \dots, T_n$, each with a **compensating action** C_i that semantically undoes T_i — *semantically*, because the world has moved on: you cannot un-send an email, but you can send a correction; you cannot un-charge a card, but you can refund it.

Definition 16.5 (Saga). A saga is a sequence of local transactions $T_1, \dots, T_n$ with compensating actions $C_1, \dots, C_{n-1}$ such that if T_k fails, the system executes $C_{k-1}, \dots, C_1$ in reverse order. The saga provides *atomicity of outcome* (all steps complete, or all completed steps are compensated) without isolation: intermediate states are visible to other requests between steps.

Sagas come in the two coordination flavors of Definition 16.1. A *choreographed* saga has each step emit events that trigger the next participant —

simple for two or three steps, then increasingly difficult to reason about, because the compensation graph lives in nobody’s code. An *orchestrated* saga puts the sequence, the state, and the compensation logic in one explicit component (Listing 16.3.5), which is why orchestration is the default recommendation once a flow has more than a couple of steps or any step with a non-trivial compensation.

Two plumbing patterns make sagas reliable rather than aspirational. The **transactional outbox** solves the dual-write problem — “update my database *and* emit an event” cannot be done atomically against two systems, so you don’t: in one local transaction you write the domain row *and* an outbox row; a separate relay reads the outbox and publishes, marking each event published only after the broker accepts it (Listing 16.3.4). The guarantee is **at-least-once** delivery: a crash between publish and mark replays the event, which is why the second pattern, the **idempotent consumer**, is not optional — the consumer records processed event IDs in the same transaction as its state change, so replays are absorbed (Chapter 11 generalizes this to request scopes).

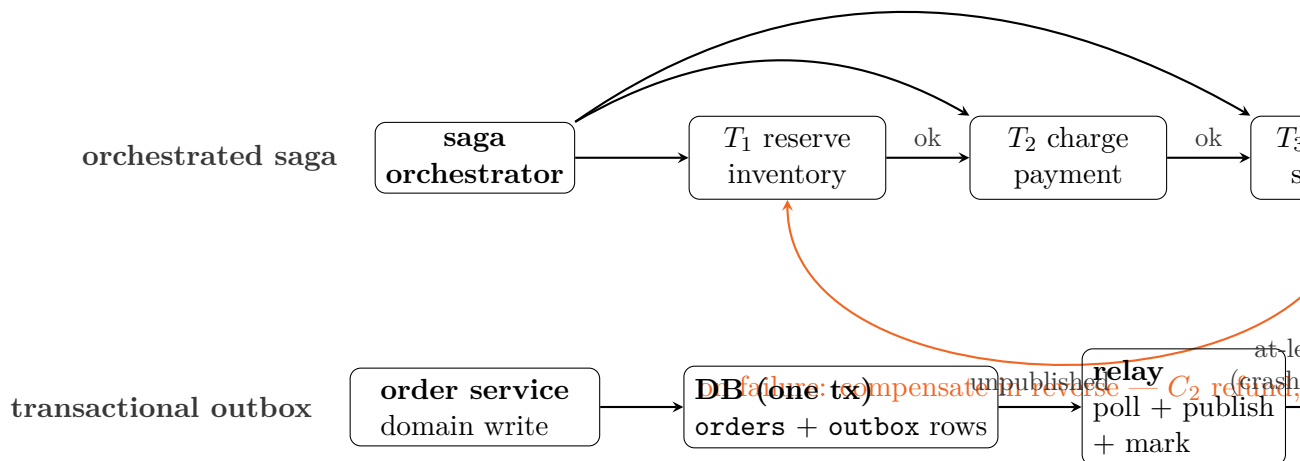


Figure 16.4: Saga orchestration and the transactional outbox. Top: the orchestrator drives steps in order and compensates in reverse on failure. Bottom: the domain write and the outbox insert commit atomically; the relay publishes at-least-once; the idempotent consumer absorbs replays.

Example

Presenting eventual consistency in an API. A client POST `/orders` against the saga system gets 202 Accepted with a `Location: /orders/ORD-7` header and a body containing `"status": "PLACED"` and a `"status_url"` poll target. The order resource exposes

its saga state machine honestly: `PLACED → PAID → SHIPPED`, or `COMPENSATED` with a machine-readable `failure_reason` and a `refund_status`. What the API must never do is return 201 `Created` with `"status": "CONFIRMED"` while shipment is still pending — consistency boundaries you hide become incident pages someone else writes.

16.3 Production-Ready Code Implementation

All five subsystems below run offline on Python 3.11+ against in-process ASGI transports — no sockets, no containers, no cloud. Determinism is structural: time enters through injected clocks, concurrency is bounded by semaphores and coordinated with events and barriers, and every assertion is on counts and states, never on wall-clock duration. Install once, from PowerShell, using the requirements of Section 16.10:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
4 pytest -q
5 python demo_services.py
```

16.3.1 Circuit Breaker: Full State Machine, Sync and Async

Listing 16.3.1 implements Definition 16.4 without compromise: a rolling deque of timestamped outcomes with lazy eviction, a minimum-call floor before the failure rate is even evaluated, lazy `OPEN → HALF_OPEN` transition driven by the injected clock, bounded concurrent probes, a configurable number of consecutive probe successes to close, and a metrics hook that emits `call_success`, `call_failure`, `rejected`, and `state_change` events — the hook is how you will watch the drill in Section 16.5. The sync `call` and async `acall` share one state machine; late completions from calls admitted before a transition are ignored so probe accounting stays exact. Note the deliberate absence of wall-clock reads: `SystemClock` is the production default, and tests inject `ManualClock`.

Listing 15.1 — `circuit_breaker.py`: production-grade circuit breaker with injected clock and metrics hooks.

```
1 """Production-grade circuit breaker: full state machine
   , rolling failure
```



```

2  window, _half-open_probes, _metrics_hooks, _sync + _async
   support.
3
4  Deterministic: _time comes from an injected 'Clock';
   tests drive a manual
5  clock. _No_wall-clock_reads_inside_the_state_machine.
6  """
7  from __future__ import annotations
8
9  import threading
10 import time
11 from collections import deque
12 from collections.abc import Awaitable, Callable
13 from dataclasses import dataclass
14 from enum import Enum
15 from typing import Any, Protocol, TypeVar
16
17 T = TypeVar("T")
18
19
20 class Clock(Protocol):
21     """Monotonic time source, in seconds."""
22
23     def now(self) -> float: ...
24
25
26 class SystemClock:
27     """Production clock backed by 'time.monotonic'."""
28
29     def now(self) -> float:
30         return time.monotonic()
31
32
33 class BreakerState(str, Enum):
34     CLOSED = "closed" # traffic flows; failures
35                     # are counted
36     OPEN = "open" # traffic is rejected
37                     # without touching the dependency
38     HALF_OPEN = "half_open" # a limited number of
39                             # probe calls are admitted
40
41
42 class CircuitOpenError(RuntimeError):
43     """Raised by 'call'/'acall' when the breaker
44     refuses the call."""

```

```

41
42
43 @dataclass(frozen=True, slots=True)
44 class BreakerConfig:
45     """Tuning knobs for one circuit breaker.
46
47     failure_rate_threshold: open when failures /
48         calls >= this, in (0, 1]
49     window_seconds: rolling observation
50         window length
51     minimum_calls: window is not
52         evaluated until this many calls
53     open_duration: seconds to stay open
54         before the first probe
55     half_open_max_probes: concurrent probe calls
56         admitted in half-open
57     half_open_success_threshold: consecutive probe
58         successes required to close
59     """
60
61     failure_rate_threshold: float = 0.5
62     window_seconds: float = 30.0
63     minimum_calls: int = 10
64     open_duration: float = 5.0
65     half_open_max_probes: int = 1
66     half_open_success_threshold: int = 1
67
68     def __post_init__(self) -> None:
69         if not 0.0 < self.failure_rate_threshold <=
70             1.0:
71             raise ValueError("failure_rate_threshold
72                 must be in (0, 1]")
73         if self.window_seconds <= 0 or self.
74             open_duration <= 0:
75             raise ValueError("window_seconds and
76                 open_duration must be > 0")
77         if self.minimum_calls < 1 or self.
78             half_open_max_probes < 1:
79             raise ValueError("minimum_calls and
80                 half_open_max_probes must be >= 1")
81         if self.half_open_success_threshold < 1:
82             raise ValueError("
83                 half_open_success_threshold must be >= 1
84                 ")

```

```

73 MetricsHook = Callable[[str, dict[str, Any]], None]
74
75
76 class CircuitBreaker:
77     """A circuit breaker guarding exactly ONE
78         dependency.
79
80     States and transitions (Section 3.5 of the chapter)
81     :
82     CLOSED --[window failure rate >= threshold]-->
83         OPEN
84     OPEN --[open_duration elapsed]----->
85         HALF_OPEN
86     HALF_OPEN --[probe success x threshold]----->
87         CLOSED
88     HALF_OPEN --[probe failure]----->
89         OPEN
90
91     Thread-safe for the sync path; the async path is
92     safe on a single
93     event loop. Late completions of calls admitted
94     before a state change
95     are ignored, which keeps probe accounting exact.
96     """
97
98     def __init__(
99         self,
100         name: str,
101         config: BreakerConfig | None = None,
102         *,
103         clock: Clock | None = None,
104         on_event: MetricsHook | None = None,
105         is_failure: Callable[[BaseException], bool] |
106             None = None,
107     ) -> None:
108         if not name:
109             raise ValueError("breaker name is required
110                               (one breaker per dependency)")
111
112         self._name = name
113         self._config = config or BreakerConfig()
114         self._clock: Clock = clock if clock is not None
115             else SystemClock()
116         self._on_event = on_event
117         # Caller-fault exceptions (e.g. ValueError from
118             validation) can be

```

```

107         # excluded by supplying is_failure; they count
           as successes here.
108         self._is_failure = is_failure or (lambda exc:
           True)
109         self._state = BreakerState.CLOSED
110         self._opened_at = 0.0
111         self._window: deque[tuple[float, bool]] = deque
           ()
112         self._probes_in_flight = 0
113         self._probe_successes = 0
114         self._lock = threading.Lock()
115
116     # ---- observability
           -----
117
118     @property
119     def state(self) -> BreakerState:
120         with self._lock:
121             self._maybe_half_open(self._clock.now())
122             return self._state
123
124     def _emit(self, event: str, **fields: Any) -> None:
125         if self._on_event is not None:
126             self._on_event(event, {"breaker": self.
127                 _name, **fields})
128
129     # ---- state machine internals (call with self.
           _lock held) -----
130
131     def _transition(self, to: BreakerState, now: float)
           -> None:
132         if to is self._state:
133             return
134         previous = self._state
135         self._state = to
136         if to is BreakerState.OPEN:
137             self._opened_at = now
138         if to is BreakerState.HALF_OPEN:
139             self._probes_in_flight = 0
140             self._probe_successes = 0
141         self._emit("state_change", previous=previous.
142             value, current=to.value)
143
144     def _maybe_half_open(self, now: float) -> None:
145         if (
146             self._state is BreakerState.OPEN
147             and now - self._opened_at >= self._config.

```

```

144         open_duration
145     ):
146         self._transition(BreakerState.HALF_OPEN,
147                         now)
148
149     def _evict(self, now: float) -> None:
150         horizon = now - self._config.window_seconds
151         while self._window and self._window[0][0] <=
152             horizon:
153             self._window.popleft()
154
155     def _admit(self) -> bool:
156         """Return True if the admitted call is a half-
157         open probe."""
158         with self._lock:
159             now = self._clock.now()
160             self._maybe_half_open(now)
161             if self._state is BreakerState.OPEN:
162                 self._emit("rejected", state=self.
163                             _state.value)
164                 raise CircuitOpenError(
165                     f"circuit_{self._name!r} is OPEN;
166                     call_rejected_fast"
167                 )
168             if self._state is BreakerState.HALF_OPEN:
169                 if self._probes_in_flight >= self.
170                     _config.half_open_max_probes:
171                     self._emit("rejected", state=self.
172                                 _state.value)
173                     raise CircuitOpenError(
174                         f"circuit_{self._name!r} is
175                         HALF_OPEN; probe_limit
176                         reached"
177                     )
178                 self._probes_in_flight += 1
179                 return True
180             return False
181
182     def _after_call(self, *, success: bool, probe: bool
183 ) -> None:
184         with self._lock:
185             now = self._clock.now()
186             self._emit(
187                 "call_success" if success else "
188                 call_failure",
189                 probe=probe,

```

```

178         state=self._state.value,
179     )
180     if probe:
181         self._probes_in_flight -= 1
182         if success:
183             self._probe_successes += 1
184             if (
185                 self._probe_successes
186                 >= self._config.
187                     half_open_success_threshold
188             ):
189                 self._window.clear()
190                 self._transition(BreakerState.
191                     CLOSED, now)
192             else:
193                 # One failed probe is enough: the
194                 # dependency is still sick.
195                 self._transition(BreakerState.OPEN,
196                     now)
197             return
198         if self._state is not BreakerState.CLOSED:
199             return # admitted before a transition;
200                 late completion ignored
201         self._window.append((now, success))
202         self._evict(now)
203         if len(self._window) >= self._config.
204             minimum_calls:
205             failures = sum(1 for _, ok in self.
206                 _window if not ok)
207             if failures / len(self._window) >= self
208                 ._config.failure_rate_threshold:
209                 self._transition(BreakerState.OPEN,
210                     now)
211
212 # ---- public API
213 -----
214
215 def call(self, fn: Callable[..., T], *args: Any, **
216     kwargs: Any) -> T:
217     """Run 'fn' through the breaker (synchronous
218         path)."""
219     probe = self._admit()
220     try:
221         result = fn(*args, **kwargs)
222     except BaseException as exc:
223         self._after_call(success=not self.

```

```

211         _is_failure(exc), probe=probe)
212         raise
213     self._after_call(success=True, probe=probe)
214     return result
215
216 async def acall(self, fn: Callable[..., Awaitable[T]], *args: Any, **kwargs: Any) -> T:
217     """Run an awaitable through the breaker(
218         asyncio.path)."""
219     probe = self._admit()
220     try:
221         result = await fn(*args, **kwargs)
222     except BaseException as exc:
223         self._after_call(success=not self._
224             _is_failure(exc), probe=probe)
225         raise
226     self._after_call(success=True, probe=probe)
227     return result

```

Listing 16.3.1 proves the documented behavior with a manual clock: threshold opening at exactly N calls, fail-fast rejection (the dependency is never invoked while open — asserted via a counting stub), the probe protocol (early probe rejected, success closes, failure reopens and restarts the open duration), the single-concurrent-probe limit enforced with a real thread blocked on an event, rolling-window eviction, caller-fault classification, event ordering, and the async path.

Listing 15.2 — test_circuit_breaker.py: deterministic state-machine proofs.

```

1  """Deterministic tests for the circuit breaker(
2     injected manual clock)."""
3
4  from __future__ import annotations
5
6  import asyncio
7  import threading
8
9  import pytest
10
11 from circuit_breaker import (
12     BreakerConfig,
13     BreakerState,
14     CircuitBreaker,
15     CircuitOpenError,
16 )

```

```

17 class ManualClock:
18     """Virtual clock: nothing happens unless the test
        advances it."""
19
20     def __init__(self, start: float = 1_000.0) -> None:
21         self._now = start
22
23     def now(self) -> float:
24         return self._now
25
26     def advance(self, seconds: float) -> None:
27         if seconds < 0:
28             raise ValueError("time only moves forward")
29         self._now += seconds
30
31
32 def make_breaker(**overrides) -> tuple[CircuitBreaker,
ManualClock, list]:
33     clock = ManualClock()
34     events: list[tuple[str, dict]] = []
35     config = BreakerConfig(
36         failure_rate_threshold=overrides.get("
            failure_rate_threshold", 0.5),
37         window_seconds=overrides.get("window_seconds",
            10.0),
38         minimum_calls=overrides.get("minimum_calls", 4)
39         ,
40         open_duration=overrides.get("open_duration",
            5.0),
41         half_open_max_probes=overrides.get("
            half_open_max_probes", 1),
42         half_open_success_threshold=overrides.get("
            half_open_success_threshold", 1),
43     )
44     breaker = CircuitBreaker(
45         "inventory",
46         config,
47         clock=clock,
48         on_event=lambda event, fields: events.append((
49             event, fields)),
50     )
51     return breaker, clock, events
52
53 def boom() -> None:
54     raise ConnectionError("inventory_unreachable")

```



```

54
55
56 def test_closed_breaker_passes_calls_through() -> None:
57     breaker, _, _ = make_breaker()
58     assert breaker.state is BreakerState.CLOSED
59     assert breaker.call(lambda: 42) == 42
60
61
62 def test_opens_when_failure_rate_threshold_is_reached()
63     -> None:
64     breaker, _, events = make_breaker()
65     # 4 calls, 2 failures -> exactly 0.5 failure rate
66     # at minimum_calls.
67     breaker.call(lambda: "ok")
68     with pytest.raises(ConnectionError):
69         breaker.call(boom)
70     with pytest.raises(ConnectionError):
71         breaker.call(boom)
72     assert breaker.state is BreakerState.CLOSED # 2/3
73     < window minimum
74     breaker.call(lambda: "ok")
75     assert breaker.state is BreakerState.OPEN # 2/4 =
76     0.5 -> open
77     assert any(e == "state_change" and f["current"] ==
78         "open" for e, f in events)
79
80 def
81     test_open_breaker_rejects_without_calling_the_dependency
82     () -> None:
83     breaker, _, _ = make_breaker()
84     for _ in range(2):
85         breaker.call(lambda: "ok")
86         with pytest.raises(ConnectionError):
87             breaker.call(boom)
88     assert breaker.state is BreakerState.OPEN
89     calls = 0
90
91     def counted() -> int:
92         nonlocal calls
93         calls += 1
94         return calls
95
96     for _ in range(10):
97         with pytest.raises(CircuitOpenError):
98             breaker.call(counted)

```

```

93     assert calls == 0 # fail-fast: the dependency was
       never touched
94
95
96 def test_half_open_probe_success_closes_breaker() ->
  None:
97     breaker, clock, _ = make_breaker()
98     for _ in range(2):
99         with pytest.raises(ConnectionError):
100             breaker.call(boom)
101         with pytest.raises(ConnectionError):
102             breaker.call(boom)
103     assert breaker.state is BreakerState.OPEN
104     clock.advance(4.9)
105     with pytest.raises(CircuitOpenError):
106         breaker.call(lambda: "too_early")
107     clock.advance(0.2) # now past open_duration
108     assert breaker.state is BreakerState.HALF_OPEN
109     assert breaker.call(lambda: "recovered") == "
       recovered"
110     assert breaker.state is BreakerState.CLOSED
111
112
113 def test_half_open_probe_failure_reopens() -> None:
114     breaker, clock, _ = make_breaker(minimum_calls=2,
       failure_rate_threshold=1.0)
115     for _ in range(2):
116         with pytest.raises(ConnectionError):
117             breaker.call(boom)
118     assert breaker.state is BreakerState.OPEN
119     clock.advance(5.0)
120     with pytest.raises(ConnectionError):
121         breaker.call(boom) # probe fails
122     assert breaker.state is BreakerState.OPEN
123     # Reopened: the full open_duration restarts from
       now.
124     clock.advance(4.9)
125     with pytest.raises(CircuitOpenError):
126         breaker.call(lambda: "still_closed_to_traffic")
127
128
129 def test_half_open_admits_only_one_concurrent_probe()
  -> None:
130     breaker, clock, _ = make_breaker(minimum_calls=2,
       failure_rate_threshold=1.0)
131     for _ in range(2):

```

```

132         with pytest.raises(ConnectionError):
133             breaker.call(boom)
134     clock.advance(5.0)
135
136     release = threading.Event()
137     probe_started = threading.Event()
138
139     def slow_probe() -> str:
140         probe_started.set()
141         assert release.wait(timeout=5.0)
142         return "healed"
143
144     result: list[str] = []
145     worker = threading.Thread(
146         target=lambda: result.append(breaker.call(
147             slow_probe)), daemon=True
148     )
149     worker.start()
150     assert probe_started.wait(timeout=5.0) # probe is
151     in flight
152     with pytest.raises(CircuitOpenError):
153         breaker.call(lambda: "second_probe") # probe
154         limit = 1
155     release.set()
156     worker.join(timeout=5.0)
157     assert result == ["healed"]
158     assert breaker.state is BreakerState.CLOSED
159
160 def test_rolling_window_ages_out_old_failures() -> None:
161     breaker, clock, _ = make_breaker(window_seconds
162         =10.0, minimum_calls=4)
163     # Two failures, then the window slides past them.
164     for _ in range(2):
165         with pytest.raises(ConnectionError):
166             breaker.call(boom)
167     clock.advance(11.0)
168     breaker.call(lambda: "ok")
169     breaker.call(lambda: "ok")
170     breaker.call(lambda: "ok")
171     breaker.call(lambda: "ok")
172     assert breaker.state is BreakerState.CLOSED #
173     failures evicted: 0/4 rate

```

```

172 def test_caller_fault_exceptions_count_as_successes()
    -> None:
173     clock = ManualClock()
174     breaker = CircuitBreaker(
175         "catalog",
176         BreakerConfig(minimum_calls=2,
177             failure_rate_threshold=0.5),
177         clock=clock,
178         is_failure=lambda exc: isinstance(exc,
179             ConnectionError),
179     )
180     for _ in range(5):
181         with pytest.raises(ValueError):
182             breaker.call(lambda: (_ for _ in ()).throw(
183                 ValueError("bad_input")))
184     assert breaker.state is BreakerState.CLOSED
185
186 def test_metrics_hook_receives_events_in_order() ->
    None:
187     breaker, _, events = make_breaker(minimum_calls=2,
188         failure_rate_threshold=1.0)
189     with pytest.raises(ConnectionError):
190         breaker.call(boom)
191     with pytest.raises(ConnectionError):
192         breaker.call(boom)
193     with pytest.raises(CircuitOpenError):
194         breaker.call(lambda: "rejected")
195     names = [name for name, _ in events]
196     assert names == ["call_failure", "call_failure", "
197         state_change", "rejected"]
198
199 def test_async_path_uses_the_same_state_machine() ->
    None:
200     clock = ManualClock()
201
202     async def main() -> None:
203         breaker = CircuitBreaker(
204             "warehouse",
205             BreakerConfig(minimum_calls=2,
206                 failure_rate_threshold=1.0),
207             clock=clock,
208         )
209
210     async def ok() -> str:

```

```

209         return "fine"
210
211     async def bad() -> str:
212         raise ConnectionError("down")
213
214     with pytest.raises(ConnectionError):
215         await breaker.acall(bad)
216     with pytest.raises(ConnectionError):
217         await breaker.acall(bad)
218     assert breaker.state is BreakerState.OPEN
219     with pytest.raises(CircuitOpenError):
220         await breaker.acall(ok)
221     clock.advance(5.0)
222     assert await breaker.acall(ok) == "fine"
223     assert breaker.state is BreakerState.CLOSED
224
225     asyncio.run(main())

```

16.3.2 Retry-Budget Client: Token Bucket, Full Jitter, Deadlines

Listing 16.3.2 turns Proposition 16.2 into code. The first attempt is always free; each retry spends one token from a process-wide bucket; the bucket refills at a configured rate, so *sustained* retry traffic is rate-limited even when *bursts* are tolerated. Backoff ceilings grow exponentially and are jittered by an injectable jitter function (the default production choice is full jitter over a seeded `random.Random`); `no_jitter` exists for deterministic tests. The deadline rule of Definition 16.3 is enforced client-side: if the next backoff would overshoot the call deadline, the client raises `RetryDeadlineExceededError` instead of donating doomed work downstream.

Listing 15.3 — `retry_budget.py`: budgeted, jittered, deadline-aware retry client.

```

1  """Retry-budgeted HTTP client: token-bucket retry
   budget+jittered
2  backoff+deadline awareness.
3
4  Design rules implemented (Section 3.4):
5  ** The first attempt is free; every RETRY spends one
   budget token.
6  ** Tokens refill at a fixed rate, so steady-state
   retry traffic cannot
7  exceed 'budget_refill_per_second' retries/s
   process-wide.

```

```

8  """Backoff is exponential with full jitter from a
   seeded RNG.
9  """
10 """A call-level deadline caps total elapsed (virtual)
   time; a retry that
11 """
12 """would overshoot the deadline is never attempted.
   """
13
14 from __future__ import annotations
15
16 import random
17 import time
18 from collections.abc import Callable
19 from dataclasses import dataclass, field
20 from typing import Any, Protocol
21
22 import httpx
23
24 class Clock(Protocol):
25     def now(self) -> float: ...
26     def sleep(self, seconds: float) -> None: ...
27
28 class SystemClock:
29     def now(self) -> float:
30         return time.monotonic()
31
32     def sleep(self, seconds: float) -> None:
33         time.sleep(seconds)
34
35 class ManualClock:
36     """Virtual clock for deterministic tests; 'sleep'
37        advances it."""
38
39     def __init__(self, start: float = 0.0) -> None:
40         self._now = start
41
42     def now(self) -> float:
43         return self._now
44
45     def sleep(self, seconds: float) -> None:
46         if seconds < 0:
47             raise ValueError("cannot sleep for a
48                               negative duration")
49         self._now += seconds

```

```

50
51 class RetryBudgetExhaustedError(RuntimeError):
52     """The token bucket is empty: this retry would
        exceed the retry budget."""
53
54
55 class RetryDeadlineExceededError(RuntimeError):
56     """The next backoff sleep would overshoot the call
        deadline."""
57
58
59 @dataclass(frozen=True, slots=True)
60 class RetryPolicy:
61     max_retries: int = 3                # retries
        , not attempts
62     base_backoff: float = 0.05          # seconds
        ; delay = base * 2**attempt
63     max_backoff: float = 1.0
64     deadline_seconds: float = 5.0       # whole-
        call deadline
65     budget_tokens: float = 10.0         # bucket
        capacity (burst)
66     budget_refill_per_second: float = 2.0 #
        sustained retry allowance
67     retry_statuses: frozenset[int] = frozenset({429,
        502, 503, 504})
68
69     def __post_init__(self) -> None:
70         if self.max_retries < 0:
71             raise ValueError("max_retries must be >= 0")
72         if self.base_backoff <= 0 or self.max_backoff <
            self.base_backoff:
73             raise ValueError("require 0 < base_backoff
                <= max_backoff")
74         if self.deadline_seconds <= 0:
75             raise ValueError("deadline_seconds must be
                > 0")
76         if self.budget_tokens < 0 or self.
            budget_refill_per_second <= 0:
77             raise ValueError("budget parameters must be
                non-negative/positive")
78
79
80 class TokenBucketRetryBudget:
81     """Process-wide retry budget. Tokens are spent ONLY

```

```

    by_retries."""
82
83     def __init__(self, capacity: float,
84                   refill_per_second: float, clock: Clock) -> None:
85         if capacity <= 0 or refill_per_second <= 0:
86             raise ValueError("capacity_and_
87                               refill_per_second_must_be_>0")
88         self._capacity = capacity
89         self._rate = refill_per_second
90         self._clock = clock
91         self._tokens = capacity
92         self._last = clock.now()
93
94     def _refill(self) -> None:
95         now = self._clock.now()
96         self._tokens = min(self._capacity, self._tokens
97                             + (now - self._last) * self._rate)
98         self._last = now
99
100     def consume(self) -> bool:
101         self._refill()
102         if self._tokens >= 1.0:
103             self._tokens -= 1.0
104             return True
105         return False
106
107     @property
108     def tokens(self) -> float:
109         self._refill()
110         return self._tokens
111
112 # Full jitter (AWS Architecture Blog style): uniform in
113 [0, computed cap].
114 def full_jitter(rng: random.Random) -> Callable[[float
115 ], float]:
116     def apply(ceiling: float) -> float:
117         return rng.uniform(0.0, ceiling)
118
119     return apply
120
121 def no_jitter(ceiling: float) -> float:
122     return ceiling

```



```

122 @dataclass(slots=True)
123 class RetryBudgetClient:
124     """Wraps an httpx.Client -shaped object with
125         budgeted retries."""
126
127     client: Any # httpx.Client, or a stub with .
128         request(method, url, **kw)
129     budget: TokenBucketRetryBudget
130     clock: Clock
131     policy: RetryPolicy = field(default_factory=
132         RetryPolicy)
133     jitter: Callable[[float], float] = no_jitter
134     on_event: Callable[[str, dict[str, Any]], None] |
135         None = None
136
137     def _emit(self, event: str, **fields: Any) -> None:
138         if self.on_event is not None:
139             self.on_event(event, fields)
140
141     def backoff_ceiling(self, attempt: int) -> float:
142         return min(self.policy.max_backoff, self.policy
143             .base_backoff * (2**attempt))
144
145     def request(self, method: str, url: str, **kwargs:
146         Any) -> httpx.Response:
147         deadline = self.clock.now() + self.policy.
148             deadline_seconds
149         attempt = 0
150         while True:
151             self._emit("attempt", attempt=attempt, url=
152                 url)
153             response: httpx.Response | None
154             last_error: httpx.TransportError | None =
155                 None
156             try:
157                 response = self.client.request(method,
158                     url, **kwargs)
159             except httpx.TransportError as exc:
160                 response = None
161                 last_error = exc
162                 retryable = True
163             else:
164                 retryable = response.status_code in
165                     self.policy.retry_statuses
166             if not retryable:
167                 return response

```

```

157         if attempt >= self.policy.max_retries:
158             self._emit("retries_exhausted",
159                 attempts=attempt + 1, url=url)
160             if response is not None:
161                 return response # caller sees the
162                     final 5xx
163             raise last_error # type: ignore[misc]
164
165         if not self.budget.consume():
166             self._emit("budget_exhausted", attempts
167                 =attempt + 1, url=url)
168             raise RetryBudgetExhaustedError(
169                 "retry_budget_exhausted; failing
170                     fast instead of amplifying"
171             ) from last_error
172
173         delay = self.jitter(self.backoff_ceiling(
174             attempt))
175         if self.clock.now() + delay > deadline:
176             self._emit("deadline_exceeded",
177                 attempts=attempt + 1, url=url)
178             raise RetryDeadlineExceededError(
179                 f"next backoff of {delay:.3f}s
180                     would overshoot the deadline"
181             ) from last_error
182
183         self._emit("retry", attempt=attempt + 1,
184             delay=delay, url=url)
185         self.clock.sleep(delay)
186         attempt += 1

```

The tests in Listing 16.3.2 are the executable core of this chapter’s anti-retry-storm argument: `test_retry_budget_caps_amplification` configures five permitted retries against an always-failing dependency with a two-token budget and asserts the client issues exactly three requests — the budget, not the retry count, is the binding constraint.

Listing 15.4 — `test_retry_budget.py`: budget-cap, refill, deadline, and jitter determinism proofs.

```

1  """Deterministic tests for the retry-budget client.
2
3  The amplification test is the heart of the chapter's
4  anti-retry-storm
5  argument: with a budget of 2 tokens, a client
6  configured for up to 5

```

```

5  retries_issues_AT_MOST_3_requests_against_an_always-
   failing_dependency.
6  """
7  from __future__ import annotations
8
9  import random
10
11 import httpx
12 import pytest
13
14 from retry_budget import (
15     ManualClock,
16     RetryBudgetClient,
17     RetryBudgetExhaustedError,
18     RetryDeadlineExceededError,
19     RetryPolicy,
20     TokenBucketRetryBudget,
21     full_jitter,
22     no_jitter,
23 )
24
25
26 class StubClient:
27     """httpx.Client-shaped stub; serves queued statuses
   or transport errors."""
28
29     def __init__(self, script: list[int | type[
   Exception]]) -> None:
30         self._script = list(script)
31         self.calls = 0
32
33     def request(self, method: str, url: str, **kwargs)
   -> httpx.Response:
34         self.calls += 1
35         outcome = self._script.pop(0) if self._script
   else 200
36         if isinstance(outcome, type) and issubclass(
   outcome, Exception):
37             raise outcome("injected_transport_failure",
   request=None)
38         return httpx.Response(outcome, request=httpx.
   Request(method, url))
39
40
41 def make_client(
42     script: list[int | type[Exception]],

```

```

43     *,
44     tokens: float = 10.0,
45     policy: RetryPolicy | None = None,
46     jitter=no_jitter,
47     clock: ManualClock | None = None,
48 ) -> tuple[RetryBudgetClient, StubClient, ManualClock,
49 list]:
50     clock = clock or ManualClock()
51     stub = StubClient(script)
52     events: list[tuple[str, dict]] = []
53     client = RetryBudgetClient(
54         client=stub,
55         budget=TokenBucketRetryBudget(tokens,
56             refill_per_second=1.0, clock=clock),
57         clock=clock,
58         policy=policy or RetryPolicy(max_retries=5,
59             deadline_seconds=60.0),
60         jitter=jitter,
61         on_event=lambda event, fields: events.append((
62             event, fields)),
63     )
64     return client, stub, clock, events
65
66 def test_success_first_try_spends_no_budget() -> None:
67     client, stub, _, _ = make_client([200], tokens=2.0)
68     response = client.request("GET", "http://inventory.
69         test/availability/SKU-1")
70     assert response.status_code == 200
71     assert stub.calls == 1
72     assert client.budget.tokens == 2.0
73
74 def test_transient_failures_then_success() -> None:
75     client, stub, _, events = make_client([503, 503,
76         200], tokens=5.0)
77     response = client.request("GET", "http://inventory.
78         test/availability/SKU-1")
79     assert response.status_code == 200
80     assert stub.calls == 3
81     retries = [f for e, f in events if e == "retry"]
82     assert [r["attempt"] for r in retries] == [1, 2]
83     assert retries[0]["delay"] == pytest.approx(0.05)
84     assert retries[1]["delay"] == pytest.approx(0.10)

```

```

82 def test_retry_budget_caps_amplification() -> None:
83     """5 allowed retries, budget of 2 tokens -> at most
84         3 requests, then fail fast."""
85     client, stub, _, events = make_client(
86         [503] * 100,
87         tokens=2.0,
88         policy=RetryPolicy(max_retries=5,
89                             deadline_seconds=60.0),
90     )
91     with pytest.raises(RetryBudgetExhaustedError):
92         client.request("GET", "http://inventory.test/
93             availability/SKU-1")
94     assert stub.calls == 3 # 1 original + 2 budgeted
95         retries, NOT 6
96     assert any(e == "budget_exhausted" for e, _ in
97         events)
98
99 def test_budget_refills_over_virtual_time() -> None:
100     clock = ManualClock()
101     client, stub, _, _ = make_client(
102         [503] * 100, tokens=1.0, clock=clock,
103         policy=RetryPolicy(max_retries=5,
104                             deadline_seconds=60.0),
105     )
106     with pytest.raises(RetryBudgetExhaustedError):
107         client.request("GET", "http://inventory.test/x"
108             )
109     assert stub.calls == 2
110     clock.sleep(2.0) # refill 1 token/s -> 2 attempts
111         possible again
112     with pytest.raises(RetryBudgetExhaustedError):
113         client.request("GET", "http://inventory.test/x"
114             )
115     assert stub.calls == 4
116
117 def test_deadline_stops_retries_before_overshoot() ->
118     None:
119     # base 0.05s backoff; deadline 0.07s: first retry
120         (0.05) fits,
121     # second retry's 0.10s sleep would overshoot ->
122         deadline error.
123     client, stub, _, events = make_client(
124         [503] * 100,
125         tokens=10.0,

```

```

116         policy=RetryPolicy(max_retries=5, base_backoff
117                             =0.05, deadline_seconds=0.07),
118     )
119     with pytest.raises(RetryDeadlineExceededError):
120         client.request("GET", "http://inventory.test/
121                         availability/SKU-1")
122     assert stub.calls == 2
123     assert any(e == "deadline_exceeded" for e, _ in
124               events)
125
126 def test_non_retryable_status_returns_immediately() ->
127     None:
128     client, stub, _, _ = make_client([404], tokens=5.0)
129     response = client.request("GET", "http://inventory.
130                               test/missing")
131     assert response.status_code == 404
132     assert stub.calls == 1
133     assert client.budget.tokens == 5.0
134
135 def
136     test_transport_errors_are_retryable_and_final_error_reraises
137     () -> None:
138     client, stub, _, _ = make_client(
139         [httpx.ConnectError, httpx.ConnectError],
140         tokens=5.0,
141         policy=RetryPolicy(max_retries=1,
142                           deadline_seconds=60.0),
143     )
144     with pytest.raises(httpx.ConnectError):
145         client.request("GET", "http://inventory.test/x"
146                       )
147     assert stub.calls == 2 # 1 original + the single
148                           permitted retry
149
150 def
151     test_full_jitter_is_deterministic_under_a_seeded_rng
152     () -> None:
153     rng_a = random.Random(42)
154     rng_b = random.Random(42)
155     jitter_a = full_jitter(rng_a)
156     jitter_b = full_jitter(rng_b)
157     ceilings = [0.05, 0.10, 0.20]
158     draws_a = [jitter_a(c) for c in ceilings]

```

```

150     draws_b = [jitter_b(c) for c in ceilings]
151     assert draws_a == draws_b
152     for ceiling, draw in zip(ceilings, draws_a):
153         assert 0.0 <= draw <= ceiling

```

16.3.3 Three-Service Chaos Drill: Catalog → Inventory → Warehouse

Listing 16.3.3 wires three FastAPI services — catalog, inventory, warehouse — with `httpx.AsyncClient` over `ASGITransport`: real ASGI request/response cycles and real `httpx` timeout machinery, zero network. The catalog edge owns the resilience posture: a per-hop timeout tighter than its caller’s budget, a dedicated client with a bounded connection pool (bulkhead one), a semaphore bounding concurrent calls into inventory (bulkhead two), and — switched by a flag so the drill can measure the difference — the Listing 16.3.1 circuit breaker. The `ChaosConfig` knob flips inventory into a hard-down failure mode; `run_drill` then fires thirty concurrent catalog requests and counts what happens.

Listing 15.5 — `demo_services.py`: three in-process services plus a chaos harness with before/after measurement.

```

1  """Three-service Northwind Robotics demo: catalog->
   inventory->warehouse.
2
3  All services are in-process FastAPI apps wired together
   with
4  ‘‘httpx.AsyncClient‘‘ over ‘‘ASGITransport‘‘: real ASGI
   request/response
5  cycles, real timeouts, zero sockets, zero network.
6
7  Resilience posture at the catalog edge:
8  **per-hop timeouts (catalog->inventory tighter than
   the caller’s budget),
9  **a circuit breaker around the inventory dependency (
   optional, so the
10  chaos drill can measure before/after),
11  **a bulkhead: a semaphore bounding concurrent calls
   into inventory.
12
13  The chaos harness flips the inventory hop into a
   failure or latency mode
14  and measures how many requests actually reach it, with
   and without the
15  breaker.
16  """

```

```

17 from __future__ import annotations
18
19 import asyncio
20 import time
21 from dataclasses import dataclass, field
22 from typing import Any
23
24 import httpx
25 from fastapi import FastAPI, Request
26 from fastapi.responses import JSONResponse
27
28 from circuit_breaker import (
29     BreakerConfig,
30     CircuitBreaker,
31     CircuitOpenError,
32 )
33
34 CATALOG_TIMEOUT = 0.25          # seconds; the caller's
    budget is larger
35 INVENTORY_TIMEOUT = 0.20
36 WAREHOUSE_TIMEOUT = 0.15
37 BULKHEAD_PERMITS = 8            # max concurrent catalog ->
    inventory calls
38
39 CATALOG = {
40     "NR-1001": {"sku": "NR-1001", "name": "Warehouse_
    Picker_Bot", "price": 18499.00},
41     "NR-2002": {"sku": "NR-2002", "name": "Conveyor_
    Vision_Kit", "price": 3299.50},
42 }
43
44
45 @dataclass
46 class ChaosConfig:
47     """Mutable_fault-injection_knobs_for_the_inventory_
    service."""
48
49     always_fail: bool = False
50     latency_seconds: float = 0.0
51
52
53 @dataclass
54 class ServiceStats:
55     inventory_calls: int = 0
56     warehouse_calls: int = 0
57

```



```

58
59 #
    -----

60 # Warehouse (leaf service)
61 #
    -----

62 def build_warehouse_app(stats: ServiceStats) -> FastAPI:
63     app = FastAPI(title="warehouse")
64
65     @app.get("/stock/{sku}")
66     async def stock(sku: str) -> dict[str, Any]:
67         stats.warehouse_calls += 1
68         return {"sku": sku, "on_hand": 7, "reserved":
69                 2}
70
71     return app
72
73 #
    -----

74 # Inventory (middle service; chaos target)
75 #
    -----

76 def build_inventory_app(
77     warehouse_client: httpx.AsyncClient,
78     chaos: ChaosConfig,
79     stats: ServiceStats,
80 ) -> FastAPI:
81     app = FastAPI(title="inventory")
82
83     @app.get("/availability/{sku}")
84     async def availability(sku: str) -> dict[str, Any]:
85         stats.inventory_calls += 1
86         if chaos.latency_seconds > 0:
87             await asyncio.sleep(chaos.latency_seconds)
88         if chaos.always_fail:
89             return JSONResponse(
90                 status_code=500,
91                 content={"detail": "chaos: inventory
92                             database unreachable"},

```

```

93     stock = await warehouse_client.get(f"/stock/{
94         sku}", timeout=WAREHOUSE_TIMEOUT)
95     body = stock.json()
96     return {
97         "sku": sku,
98         "available": body["on_hand"] - body["
99             reserved"],
100     }
101
102     return app
103
104 # -----
105 # Catalog (edge service; owns the resilience policies)
106 # -----
107
108 def build_catalog_app(
109     inventory_client: httpx.AsyncClient,
110     breaker: CircuitBreaker | None,
111     bulkhead: asyncio.Semaphore,
112 ) -> FastAPI:
113     app = FastAPI(title="catalog")
114
115     @app.get("/products/{sku}")
116     async def product(sku: str, request: Request) ->
117         Any:
118         product_row = CATALOG.get(sku)
119         if product_row is None:
120             return JSONResponse(status_code=404,
121                                 content={"detail": "unknown_ sku"})
122
123         async def fetch_availability() -> dict[str, Any
124             ]:
125             async with bulkhead: # bulkhead: bounded
126                 concurrency into inventory
127                 response = await inventory_client.get(
128                     f"/availability/{sku}", timeout=
129                         CATALOG_TIMEOUT
130                 )
131             response.raise_for_status()
132             return response.json()
133
134     try:

```

```

128         if breaker is not None:
129             availability = await breaker.acall(
130                 fetch_availability)
131         else:
132             availability = await fetch_availability
133             ()
134     except CircuitOpenError:
135         return JSONResponse(
136             status_code=503,
137             content={"detail": "inventory_
138                     unavailable_(circuit_open)",
139                     "circuit_open": True},
140         )
141     except (httpx.HTTPError, asyncio.TimeoutError)
142     as exc:
143         return JSONResponse(
144             status_code=503,
145             content={"detail": f"inventory_call_
146                     failed:{type(exc).__name__}",
147                     "circuit_open": False},
148         )
149     return {**product_row, "availability":
150            availability}
151
152     return app
153
154 #
155
156 # Wiring + chaos drill harness
157 #
158
159 @dataclass
160 class DrillResult:
161     requests: int
162     catalog_errors: int
163     circuit_open_rejections: int
164     inventory_calls: int
165     warehouse_calls: int
166     wall_seconds: float
167
168 @dataclass
169 class DemoSystem:

```

```

164     apps: dict[str, FastAPI]
165     chaos: ChaosConfig
166     stats: ServiceStats
167     events: list[tuple[str, dict]] = field(
168         default_factory=list)
169
170 def build_system(*, use_breaker: bool) -> DemoSystem:
171     chaos = ChaosConfig()
172     stats = ServiceStats()
173     events: list[tuple[str, dict]] = []
174
175     warehouse_app = build_warehouse_app(stats)
176     warehouse_client = httpx.AsyncClient(
177         transport=httpx.ASGITransport(app=warehouse_app
178         ),
179         base_url="http://warehouse.test",
180     )
181     inventory_app = build_inventory_app(
182         warehouse_client, chaos, stats)
183     inventory_client = httpx.AsyncClient(
184         transport=httpx.ASGITransport(app=inventory_app
185         ),
186         base_url="http://inventory.test",
187         # Bulkhead on the client side too: a small,
188         # dedicated pool.
189         limits=httpx.Limits(max_connections=
190             BULKHEAD_PERMITS),
191     )
192     breaker = None
193     if use_breaker:
194         breaker = CircuitBreaker(
195             "inventory",
196             BreakerConfig(
197                 failure_rate_threshold=0.5,
198                 window_seconds=60.0,
199                 minimum_calls=4,
200                 open_duration=30.0, # >> drill
201                 duration: no probes mid-drill
202                 half_open_max_probes=1,
203             ),
204             on_event=lambda event, fields: events.
205                 append((event, fields)),
206         )
207     catalog_app = build_catalog_app(
208         inventory_client, breaker, asyncio.Semaphore(

```

```

202         BULKHEAD_PERMITS)
203     )
204     return DemoSystem(
205         apps={"catalog": catalog_app, "inventory":
206             inventory_app,
207             "warehouse": warehouse_app},
208         chaos=chaos,
209         stats=stats,
210         events=events,
211     )
212
213 async def run_drill(
214     requests: int = 30, *, use_breaker: bool, sku: str
215     = "NR-1001"
216 ) -> DrillResult:
217     """Fire 'requests' concurrent catalog calls at a
218     broken inventory."""
219     system = build_system(use_breaker=use_breaker)
220     system.chaos.always_fail = True # inventory is
221     hard-down for the drill
222     started = time.perf_counter()
223     async with httpx.AsyncClient(
224         transport=httpx.ASGITransport(app=system.apps["
225             catalog"]),
226         base_url="http://catalog.test",
227     ) as catalog:
228         responses = await asyncio.gather(
229             *(catalog.get(f"/products/{sku}") for _ in
230                 range(requests))
231         )
232     elapsed = time.perf_counter() - started
233     errors = sum(1 for r in responses if r.status_code
234                 >= 500)
235     open_rejections = sum(
236         1 for r in responses if r.json().get("
237             circuit_open") is True
238     )
239     return DrillResult(
240         requests=requests,
241         catalog_errors=errors,
242         circuit_open_rejections=open_rejections,
243         inventory_calls=system.stats.inventory_calls,
244         warehouse_calls=system.stats.warehouse_calls,
245         wall_seconds=elapsed,
246     )

```

```

239
240
241 async def _main() -> None:
242     print("Chaos_drill: inventory_hard-down, 30_
        concurrent_catalog_requests")
243     for use_breaker in (False, True):
244         result = await run_drill(use_breaker=
            use_breaker)
245         label = "WITH_breaker" if use_breaker else "
            WITHOUT_breaker"
246         print(
247             f"{label}| catalog_5xx: {result.
                catalog_errors:>2}/ "
248             f"{result.requests}| circuit-open_fast-
                fails: "
249             f"{result.circuit_open_rejections:>2}| |
                calls_reaching_inventory: "
250             f"{result.inventory_calls:>2}| wall: {
                result.wall_seconds:.3f}s"
251         )
252
253
254 if __name__ == "__main__":
255     asyncio.run(_main())

```

Running the drill (`python demo_services.py`) produces the measured before/after comparison of Table 16.4. The breaker does not make requests succeed — inventory is genuinely down, and all thirty catalog calls return 503 either way. What changes is *where the load goes*: without the breaker, all thirty requests reach the sick service; with it, four do (the minimum-calls floor plus the concurrent admissions before the window evaluates) and twenty-six fail fast and cheap. Those twenty-six unanswered invitations are the headroom in which inventory recovers.

Listing 15.6 — test_demo_services.py: drill assertions on counts and breaker events.

```

1  """Deterministic_tests_for_the_three-service_chaos_
    drill.
2
3  Assertions_are_on_COUNTS_(how_many_requests_reached_the_
    broken_inventory),
4  never_on_wall-clock_latency. The_30_catalog_calls_run_
    concurrently_via
5  ‘asyncio.gather’; the_failure_mode_under_test_(
    inventory_hard-down,
6  immediate_500) is_fully_deterministic.

```

Configuration	Calls reaching inventory	Catalog 5xx	Fast-fail rejections
Without breaker	30 / 30	30	0 — every request pays full failure latency
With breaker	4 / 30	30	26 — rejected in microseconds, dependency spared

Table 16.4: Measured chaos-drill results, Listing 16.3.3 (deterministic: counts, not wall-clock). Resilience patterns do not convert failure into success; they convert expensive failure into cheap failure and buy recovery headroom.

```

7  """
8  from __future__ import annotations
9
10 import asyncio
11
12 import httpx
13
14 from demo_services import (
15     BULKHEAD_PERMITS,
16     build_system,
17     run_drill,
18 )
19
20
21 def
22     test_happy_path_catalog_joins_inventory_and_warehouse
23     () -> None:
24         async def main() -> None:
25             system = build_system(use_breaker=True) #
26                 chaos OFF
27             async with httpx.AsyncClient(
28                 transport=httpx.ASGITransport(app=system.
29                     apps["catalog"]),
30                 base_url="http://catalog.test",
31             ) as client:
32                 response = await client.get("/products/NR

```

```

        -1001")
29     assert response.status_code == 200
30     body = response.json()
31     assert body["name"] == "Warehouse_Picker_Bot"
32     assert body["availability"] == {"sku": "NR-1001", "available": 5}
33     assert system.stats.inventory_calls == 1
34     assert system.stats.warehouse_calls == 1
35
36     asyncio.run(main())
37
38
39 def
    test_without_breaker_every_request_reaches_the_broken_inventory
    () -> None:
40     result = asyncio.run(run_drill(requests=30,
        use_breaker=False))
41     assert result.catalog_errors == 30
42     assert result.inventory_calls == 30    # full load
        hits the sick dependency
43     assert result.circuit_open_rejections == 0
44
45
46 def test_with_breaker_most_requests_fail_fast() -> None
    :
47     result = asyncio.run(run_drill(requests=30,
        use_breaker=True))
48     assert result.catalog_errors == 30    # still
        errors: the dependency IS down
49     # ...but almost none touch inventory: breaker opens
        after 4 failures
50     # (minimum_calls, 100% failure rate >= 0.5
        threshold).
51     assert result.inventory_calls < result.requests
52     assert result.circuit_open_rejections == 30 -
        result.inventory_calls
53     assert result.circuit_open_rejections >= 30 -
        BULKHEAD_PERMITS
54
55
56 def
    test_breaker_state_transitions_are_emitted_during_drill
    () -> None:
57     system = build_system(use_breaker=True)
58     system.chaos.always_fail = True
59

```



```

60     async def main() -> None:
61         async with httpx.AsyncClient(
62             transport=httpx.ASGITransport(app=system.
63                 apps["catalog"]),
64             base_url="http://catalog.test",
65             ) as client:
66             await asyncio.gather(*(client.get("/
67                 products/NR-1001") for _ in range(30)))
68
69     asyncio.run(main())
70     transitions = [f for e, f in system.events if e ==
71         "state_change"]
72     assert {"previous": "closed", "current": "open"} in
73         [
74             {"previous": t["previous"], "current": t["
75                 current"]} for t in transitions
76         ]
77     # Inventory hard-down and breaker open-duration (30
78         s) >> drill duration:
79     # exactly one transition, and no half-open probes
80         mid-drill.
81     assert len(transitions) == 1

```

16.3.4 The Transactional Outbox with Relay and Idempotent Consumer

Listing 16.3.4 implements the outbox pattern end to end against SQLite. `OrderService.place_order` commits the domain row and the outbox row in one `with conn` transaction — atomicity is the entire point, and the first test proves rollback leaves no orphan outbox row. `OutboxRelay.relay_once` publishes each pending event and marks it published strictly *after* the publish call returns: the crash window between those two statements is where at-least-once delivery comes from, and `CrashAfterPublish` exists to stand inside it. `InventoryConsumer.handle` records the event ID and applies the stock decrement in one transaction, so a replayed event is absorbed, not reapplied.

Listing 15.7 — outbox.py: atomic domain write + outbox, relay loop, idempotent consumer.

```

1  """Transactional outbox: atomic domain write + outbox -
2  insert, a relay loop
3  with at-least-once delivery, and an idempotent consumer
4  .

```

```

4 SQLite-backed, fully in-process. The "broker" is a
    Python callable, which
5 keeps the pattern's GUARANTEES visible: the relay marks
    an event published
6 only after the publish call returns, so a crash in
    between replays the
7 event -- and the consumer must deduplicate (Chapter 10
    idempotency).
8 """
9 from __future__ import annotations
10
11 import json
12 import sqlite3
13 import uuid
14 from dataclasses import dataclass
15 from typing import Any, Protocol
16
17 PRODUCER_SCHEMA = """
18 CREATE TABLE IF NOT EXISTS orders(
19     order_id TEXT PRIMARY KEY,
20     sku TEXT NOT NULL,
21     quantity INTEGER NOT NULL CHECK(quantity > 0),
22     status TEXT NOT NULL
23 );
24 CREATE TABLE IF NOT EXISTS outbox(
25     seq INTEGER PRIMARY KEY AUTOINCREMENT,
26     event_id TEXT NOT NULL UNIQUE,
27     aggregate_id TEXT NOT NULL,
28     event_type TEXT NOT NULL,
29     payload TEXT NOT NULL,
30     published INTEGER NOT NULL DEFAULT 0
31 );
32 """
33
34 CONSUMER_SCHEMA = """
35 CREATE TABLE IF NOT EXISTS processed_events(
36     event_id TEXT PRIMARY KEY
37 );
38 CREATE TABLE IF NOT EXISTS stock(
39     sku TEXT PRIMARY KEY,
40     on_hand INTEGER NOT NULL
41 );
42 """
43
44
45 class Publisher(Protocol):

```

```

46     """Anything that can accept one event. A real
47         deployment: Kafka, etc."""
48
49     def publish(self, event: dict[str, Any]) -> None:
50         ...
51
52 class InProcessBroker:
53     """Deterministic stand-in broker: records every
54         published event."""
55
56     def __init__(self) -> None:
57         self.messages: list[dict[str, Any]] = []
58
59     def publish(self, event: dict[str, Any]) -> None:
60         self.messages.append(event)
61
62 class CrashAfterPublish:
63     """Fault-injection publisher: delivers, THEN
64         crashes before the relay
65         can mark the event published -- the classic at-
66         least-once window."""
67
68     def __init__(self, inner: Publisher) -> None:
69         self._inner = inner
70
71     def publish(self, event: dict[str, Any]) -> None:
72         self._inner.publish(event)
73         raise RuntimeError("simulated relay crash after
74             publish")
75
76 class OrderService:
77     """Producer side: domain write and outbox insert in
78         ONE transaction."""
79
80     def __init__(self, conn: sqlite3.Connection) ->
81         None:
82         self._conn = conn
83         self._conn.executescript(PRODUCER_SCHEMA)
84
85     def place_order(
86         self, order_id: str, sku: str, quantity: int,
87         *, event_id: str | None = None
88     ) -> str:

```

```

83         event_id = event_id or uuid.uuid4().hex
84         payload = json.dumps({"sku": sku, "quantity":
85                                quantity})
86         with self._conn: # atomic: both rows or
87                             neither
88             self._conn.execute(
89                 "INSERT INTO orders (order_id, sku,
90                                     quantity, status)"
91                 "VALUES (?, ?, ?, 'PLACED')",
92                 (order_id, sku, quantity),
93             )
94             self._conn.execute(
95                 "INSERT INTO outbox (event_id,
96                                     aggregate_id, event_type, payload)"
97                 "VALUES (?, ?, 'order.placed', ?)",
98                 (event_id, order_id, payload),
99             )
100         return event_id
101
102     def pending_events(self) -> list[dict[str, Any]]:
103         rows = self._conn.execute(
104             "SELECT event_id, aggregate_id, event_type,
105                  payload"
106             "FROM outbox WHERE published = 0 ORDER BY
107                  seq"
108         ).fetchall()
109         return [
110             {
111                 "event_id": row[0],
112                 "aggregate_id": row[1],
113                 "event_type": row[2],
114                 "payload": json.loads(row[3]),
115             }
116             for row in rows
117         ]
118
119 @dataclass
120 class OutboxRelay:
121     """Polls the outbox, publishes, marks published. At
122         -least-once: the
123         mark happens strictly AFTER publish returns."""
124
125     conn: sqlite3.Connection
126     publisher: Publisher
127     batch_size: int = 100

```

```

122
123     def relay_once(self) -> int:
124         rows = self.conn.execute(
125             "SELECT event_id, aggregate_id, event_type,
126                payload"
127             "FROM outbox WHERE published = 0 ORDER BY seq LIMIT ?",
128             (self.batch_size,),
129         ).fetchall()
130         delivered = 0
131         for event_id, aggregate_id, event_type, payload
132             in rows:
133             event = {
134                 "event_id": event_id,
135                 "aggregate_id": aggregate_id,
136                 "event_type": event_type,
137                 "payload": json.loads(payload),
138             }
139             self.publisher.publish(event) # crash here
140             -> replay later
141             with self.conn:
142                 self.conn.execute(
143                     "UPDATE outbox SET published = 1
144                     WHERE event_id = ?",
145                     (event_id,),
146                 )
147                 delivered += 1
148             return delivered
149
150
151     class InventoryConsumer:
152         """Idempotent consumer: effect applied at most once
153            , tracked in the
154            same transaction as the dedup record."""
155
156         def __init__(self, conn: sqlite3.Connection) ->
157             None:
158             self._conn = conn
159             self._conn.executescript(CONSUMER_SCHEMA)
160
161         def seed_stock(self, sku: str, on_hand: int) ->
162             None:
163             with self._conn:
164                 self._conn.execute(
165                     "INSERT INTO stock (sku, on_hand)
166                     VALUES (?, ?)"

```

```

159         "ON_CONFLICT(sku) DO UPDATE SET
160             on_hand=excluded.on_hand",
161         (sku, on_hand),
162     )
163
164     def on_hand(self, sku: str) -> int:
165         row = self._conn.execute(
166             "SELECT on_hand FROM stock WHERE sku=?",
167             (sku,)
168         ).fetchone()
169         return row[0] if row else 0
170
171     def handle(self, event: dict[str, Any]) -> bool:
172         """Apply the event; return False for duplicates
173         (already processed)."""
174         if event["event_type"] != "order.placed":
175             raise ValueError(f"unsupported event type {
176                 event['event_type']!r}")
177         with self._conn:
178             cursor = self._conn.execute(
179                 "INSERT OR IGNORE INTO processed_events
180                 (event_id) VALUES (?)",
181                 (event["event_id"],),
182             )
183             if cursor.rowcount == 0:
184                 return False # duplicate delivery:
185                             # deduped
186             self._conn.execute(
187                 "UPDATE stock SET on_hand=on_hand-?
188                 WHERE sku=?",
189                 (event["payload"]["quantity"], event["
190                     payload"]["sku"]),
191             )
192             return True

```

Listing 15.8 — test_outbox.py: atomicity, crash-mid-publish re-play, and dedup proofs.

```

1  """Tests for the transactional outbox, relay crash, and
2  consumer dedup."""
3
4  from __future__ import annotations
5
6  import sqlite3
7
8  import pytest
9
10 from outbox import (

```

```

9      CrashAfterPublish,
10     InProcessBroker,
11     InventoryConsumer,
12     OrderService,
13     OutboxRelay,
14 )
15
16
17 @pytest.fixture()
18 def producer_conn() -> sqlite3.Connection:
19     return sqlite3.connect(":memory:")
20
21
22 @pytest.fixture()
23 def consumer() -> InventoryConsumer:
24     consumer = InventoryConsumer(sqlite3.connect(":memory:"))
25     consumer.seed_stock("NR-1001", 100)
26     return consumer
27
28
29 def test_domain_write_and_outbox_insert_are_atomic(
30     producer_conn) -> None:
31     service = OrderService(producer_conn)
32     service.place_order("ORD-1", "NR-1001", 3, event_id="evt-1")
33     # Second insert with the same order_id violates the
34     # PK; the outbox row
35     # from the failed transaction must be rolled back
36     # too.
37     with pytest.raises(sqlite3.IntegrityError):
38         service.place_order("ORD-1", "NR-1001", 5,
39                             event_id="evt-2")
40     pending = service.pending_events()
41     assert [e["event_id"] for e in pending] == ["evt-1"]
42
43
44 def test_relay_delivers_pending_events_and_marks_them(
45     producer_conn) -> None:
46     service = OrderService(producer_conn)
47     service.place_order("ORD-1", "NR-1001", 3, event_id="evt-1")
48     service.place_order("ORD-2", "NR-2002", 1, event_id="evt-2")
49     broker = InProcessBroker()

```

```

45     relay = OutboxRelay(producer_conn, broker)
46     assert relay.relay_once() == 2
47     assert [m["event_id"] for m in broker.messages] ==
48         ["evt-1", "evt-2"]
49     assert relay.relay_once() == 0 # nothing left
50     pending
51
52 def test_relay_crash_mid_publish_replays_event(
53     producer_conn) -> None:
54     service = OrderService(producer_conn)
55     service.place_order("ORD-1", "NR-1001", 3, event_id
56         = "evt-1")
57     broker = InProcessBroker()
58
59     crashing = OutboxRelay(producer_conn,
60         CrashAfterPublish(broker))
61     with pytest.raises(RuntimeError, match="simulated_
62         relay_crash"):
63         crashing.relay_once()
64     assert len(broker.messages) == 1 # delivered
65     ...
66     assert len(service.pending_events()) == 1 # ...but
67     NOT marked published
68
69     recovered = OutboxRelay(producer_conn, broker)
70     assert recovered.relay_once() == 1 # at-least-
71     once: replayed
72     assert len(broker.messages) == 2 # duplicate
73     on the wire
74     assert len(service.pending_events()) == 0
75
76 def test_idempotent_consumer_dedups_replayed_events(
77     consumer) -> None:
78     event = {
79         "event_id": "evt-1",
80         "aggregate_id": "ORD-1",
81         "event_type": "order.placed",
82         "payload": {"sku": "NR-1001", "quantity": 3},
83     }
84     assert consumer.handle(event) is True
85     assert consumer.on_hand("NR-1001") == 97
86     assert consumer.handle(event) is False # duplicate
87     : no double decrement
88     assert consumer.handle(event) is False

```



```

79     assert consumer.on_hand("NR-1001") == 97
80
81
82 def
83     test_end_to_end_order_to_stock_under_crash_and_replay(
84         producer_conn, consumer
85     ) -> None:
86         service = OrderService(producer_conn)
87         service.place_order("ORD-1", "NR-1001", 3, event_id
88                             ="evt-1")
89         service.place_order("ORD-2", "NR-1001", 2, event_id
90                             ="evt-2")
91         broker = InProcessBroker()
92
93         crashing = OutboxRelay(producer_conn,
94                                CrashAfterPublish(broker))
95         with pytest.raises(RuntimeError):
96             crashing.relay_once() # evt-1 delivered, crash
97                                   before mark
98
99         OutboxRelay(producer_conn, broker).relay_once() #
100            replays evt-1, adds evt-2
101         assert [m["event_id"] for m in broker.messages] ==
102            ["evt-1", "evt-1", "evt-2"]
103
104         applied = [consumer.handle(m) for m in broker.
105                    messages]
106         assert applied == [True, False, True]
107         assert consumer.on_hand("NR-1001") == 95 # 3 + 2
108            deducted exactly once

```

The end-to-end test deserves a slow reading because it *is* the pattern's guarantee: two orders are placed; the relay delivers `evt-1` and crashes before marking it; the recovered relay replays `evt-1` and delivers `evt-2`, so the wire carries `[evt-1, evt-1, evt-2]`; the consumer applies `[True, False, True]`; and stock ends at exactly $100 - 3 - 2 = 95$. At-least-once transport plus idempotent consumption equals effectively-once processing — no two-phase commit involved.

16.3.5 Saga Orchestrator with Compensating Actions

Listing 16.3.5 implements the orchestrated saga of Definition 16.5 for Northwind Robotics order fulfillment: reserve inventory, charge payment, schedule shipment, each with its compensation (release, refund, cancel). The orches-

trator is deliberately small — the value is in the explicitness: steps in order, compensations in reverse, every transition emitted to the event hook, and a `COMPENSATION_FAILED` status that is *surfaced*, never swallowed, because a failed compensation means the world is in a state only an operator can reconcile. The in-memory domain services take a `fail_on` set so tests can break any operation by name.

Listing 15.9 — `saga.py`: orchestrated order-fulfillment saga with compensations and failure injection.

```

1  """Saga_orchestrator_for_Northwind_Robotics_order_
   fulfillment.
2
3  Flow: reserve_inventory->charge_payment->schedule_
   shipment.
4  Each_step_declares_a_compensating_action; on_failure_
   the_orchestrator_runs
5  compensations_for_completed_steps_in_REVERSE_order_and_
   reports_the_saga_as
6  COMPENSATED(not_rolled_back"--the_real_world_
   already_happened;_we
7  compensate,_we_do_not_time-travel).
8
9  Failure_injection:_each_domain_service_takes_`fail_on`
   `--a_set_of
10 operation_names_that_raise_on_invocation.
11 """
12 from __future__ import annotations
13
14 from collections.abc import Callable
15 from dataclasses import dataclass, field
16 from enum import Enum
17 from typing import Any
18
19
20 class SagaStatus(str, Enum):
21     COMPLETED = "completed"
22     COMPENSATED = "compensated"
23     COMPENSATION_FAILED = "compensation_failed"
24
25
26 class CompensationError(RuntimeError):
27     """A_compensating_action_itself_failed;_requires_
   operator_attention."""
28
29
30 @dataclass(frozen=True, slots=True)

```

```

31 class SagaStep:
32     name: str
33     action: Callable[[dict[str, Any]], None]
34     compensate: Callable[[dict[str, Any]], None] | None
35         = None
36
37 @dataclass(frozen=True, slots=True)
38 class SagaResult:
39     status: SagaStatus
40     failed_step: str | None
41     compensated: tuple[str, ...]
42     context: dict[str, Any]
43
44
45 class SagaOrchestrator:
46     """Executes steps in order; compensates in reverse
47     on failure."""
48
49     def __init__(
50         self,
51         steps: list[SagaStep],
52         on_event: Callable[[str, dict[str, Any]], None]
53         | None = None,
54     ) -> None:
55         if not steps:
56             raise ValueError("a saga needs at least one
57                 step")
58         self._steps = steps
59         self._on_event = on_event
60
61     def _emit(self, event: str, **fields: Any) -> None:
62         if self._on_event is not None:
63             self._on_event(event, fields)
64
65     def run(self, context: dict[str, Any] | None = None)
66         -> SagaResult:
67         context = dict(context or {})
68         completed: list[SagaStep] = []
69         for step in self._steps:
70             self._emit("step_started", step=step.name)
71             try:
72                 step.action(context)
73             except Exception as exc:
74                 self._emit("step_failed", step=step.
75                     name, error=type(exc).__name__)

```

```

71         return self._compensate(completed,
72                                 failed=step.name, context=context)
73         completed.append(step)
74         self._emit("step_completed", step=step.name
75                     )
76     return SagaResult(
77         status=SagaStatus.COMPLETED,
78         failed_step=None,
79         compensated=(),
80         context=context,
81     )
82
83     def _compensate(
84         self, completed: list[SagaStep], *, failed: str
85         , context: dict[str, Any]
86     ) -> SagaResult:
87         compensated: list[str] = []
88         try:
89             for step in reversed(completed):
90                 if step.compensate is None:
91                     continue
92                 self._emit("compensation_started", step
93                             =step.name)
94                 step.compensate(context)
95                 compensated.append(step.name)
96                 self._emit("compensation_completed",
97                             step=step.name)
98             except Exception as exc:
99                 self._emit(
100                     "compensation_failed", step=step.name,
101                     error=type(exc).__name__
102                 )
103             return SagaResult(
104                 status=SagaStatus.COMPENSATION_FAILED,
105                 failed_step=failed,
106                 compensated=tuple(compensated),
107                 context=context,
108             )
109         return SagaResult(
110             status=SagaStatus.COMPENSATED,
111             failed_step=failed,
112             compensated=tuple(compensated),
113             context=context,
114         )

```

```

111 #
112 # Northwind Robotics domain services (in-memory,
113 # deterministic)
114 #
115
116 class InventoryService:
117     def __init__(self, stock: dict[str, int], fail_on:
118         set[str] | None = None) -> None:
119         self.stock = dict(stock)
120         self.reserved: dict[str, int] = {}
121         self.fail_on = fail_on or set()
122         self.calls: list[str] = []
123
124     def reserve(self, sku: str, quantity: int) -> None:
125         self.calls.append("reserve")
126         if "reserve" in self.fail_on:
127             raise ConnectionError("inventory_service_
128                 unavailable")
129         if self.stock.get(sku, 0) < quantity:
130             raise ValueError(f"insufficient_stock_for_{
131                 sku}")
132         self.stock[sku] -= quantity
133         self.reserved[sku] = self.reserved.get(sku, 0)
134             + quantity
135
136     def release(self, sku: str, quantity: int) -> None:
137         self.calls.append("release")
138         if "release" in self.fail_on:
139             raise ConnectionError("inventory_service_
140                 unavailable")
141         self.reserved[sku] = self.reserved.get(sku, 0)
142             - quantity
143         self.stock[sku] = self.stock.get(sku, 0) +
144             quantity
145
146 class PaymentService:
147     def __init__(self, fail_on: set[str] | None = None)
148         -> None:
149         self.charges: list[tuple[str, float]] = []
150         self.refunds: list[tuple[str, float]] = []
151         self.fail_on = fail_on or set()
152         self.calls: list[str] = []

```

```

144
145     def charge(self, order_id: str, amount: float) ->
146         None:
147         self.calls.append("charge")
148         if "charge" in self.fail_on:
149             raise ConnectionError("payment_gateway_
150                 timeout")
151         self.charges.append((order_id, amount))
152
153     def refund(self, order_id: str, amount: float) ->
154         None:
155         self.calls.append("refund")
156         if "refund" in self.fail_on:
157             raise ConnectionError("payment_gateway_
158                 timeout")
159         self.refunds.append((order_id, amount))
160
161 class ShipmentService:
162     def __init__(self, fail_on: set[str] | None = None)
163         -> None:
164         self.shipments: list[str] = []
165         self.cancellations: list[str] = []
166         self.fail_on = fail_on or set()
167         self.calls: list[str] = []
168
169     def schedule(self, order_id: str) -> None:
170         self.calls.append("schedule")
171         if "schedule" in self.fail_on:
172             raise ConnectionError("carrier_API_error")
173         self.shipments.append(order_id)
174
175     def cancel(self, order_id: str) -> None:
176         self.calls.append("cancel")
177         if "cancel" in self.fail_on:
178             raise ConnectionError("carrier_API_error")
179         self.cancellations.append(order_id)
180
181     def build_order_fulfillment_saga(
182         inventory: InventoryService,
183         payments: PaymentService,
184         shipments: ShipmentService,
185         on_event: Callable[[str, dict[str, Any]], None] |
186             None = None,
187     ) -> SagaOrchestrator:

```

```

184     """The canonical three-step order saga with
185         compensations."""
186     steps = [
187         SagaStep(
188             name="reserve_inventory",
189             action=lambda ctx: inventory.reserve(ctx["sku"], ctx["quantity"]),
190             compensate=lambda ctx: inventory.release(
191                 ctx["sku"], ctx["quantity"]),
192         ),
193         SagaStep(
194             name="charge_payment",
195             action=lambda ctx: payments.charge(ctx["order_id"], ctx["amount"]),
196             compensate=lambda ctx: payments.refund(ctx["order_id"], ctx["amount"]),
197         ),
198         SagaStep(
199             name="schedule_shipment",
200             action=lambda ctx: shipments.schedule(ctx["order_id"]),
201             compensate=lambda ctx: shipments.cancel(ctx["order_id"]),
202         ),
203     ]
204     return SagaOrchestrator(steps, on_event=on_event)

```

Listing 15.10 — `test_saga.py`: failure-injection proofs, including reverse-order compensation and surfaced compensation failure.

```

1  """Failure-injection tests for the saga orchestrator."""
2
3  from __future__ import annotations
4
5  from saga import (
6      InventoryService,
7      PaymentService,
8      SagaOrchestrator,
9      SagaStatus,
10     SagaStep,
11     ShipmentService,
12     build_order_fulfillment_saga,
13 )
14
15 ORDER = {"order_id": "ORD-7", "sku": "NR-1001", "quantity": 3, "amount": 184.50}

```

```

16
17 def build(fail_on: dict[str, set[str]] | None = None):
18     fail_on = fail_on or {}
19     inventory = InventoryService({"NR-1001": 10},
20                                   fail_on.get("inventory"))
21     payments = PaymentService(fail_on.get("payments"))
22     shipments = ShipmentService(fail_on.get("shipments")
23                                 )
24     events: list[tuple[str, dict]] = []
25     saga = build_order_fulfillment_saga(
26         inventory, payments, shipments,
27         on_event=lambda event, fields: events.append((
28             event, fields)),
29     )
30     return saga, inventory, payments, shipments, events
31
32 def test_happy_path_completes_all_steps_in_order() ->
33     None:
34     saga, inventory, payments, shipments, events =
35         build()
36     result = saga.run(ORDER)
37     assert result.status is SagaStatus.COMPLETED
38     assert result.failed_step is None
39     assert result.compensated == ()
40     assert inventory.stock["NR-1001"] == 7
41     assert inventory.reserved["NR-1001"] == 3
42     assert payments.charges == [("ORD-7", 184.50)]
43     assert shipments.shipments == ["ORD-7"]
44     started = [f["step"] for e, f in events if e == "
45                 step_started"]
46     assert started == ["reserve_inventory", "
47                         charge_payment", "schedule_shipment"]
48
49 def
50     test_payment_failure_compensates_inventory_in_reverse
51     () -> None:
52     saga, inventory, payments, shipments, _ = build(
53         {"payments": {"charge"}})
54     result = saga.run(ORDER)
55     assert result.status is SagaStatus.COMPENSATED
56     assert result.failed_step == "charge_payment"
57     assert result.compensated == ("reserve_inventory",)
58     assert inventory.stock["NR-1001"] == 10 #

```



```

    reservation released
53  assert inventory.reserved["NR-1001"] == 0
54  assert payments.charges == []          # charge
    never landed
55  assert shipments.shipments == []      # never
    reached
56
57
58  def
    test_shipment_failure_compensates_payment_then_inventory
    () -> None:
59      saga, inventory, payments, shipments, _ = build(
60          {"shipments": {"schedule"}}
61      )
62      result = saga.run(ORDER)
63      assert result.status is SagaStatus.COMPENSATED
64      assert result.failed_step == "schedule_shipment"
65      # Reverse order of completion: refund BEFORE
        release.
66      assert result.compensated == ("charge_payment", "
        reserve_inventory")
67      assert payments.charges == [("ORD-7", 184.50)]    #
        happened, then refunded
68      assert payments.refunds == [("ORD-7", 184.50)]
69      assert inventory.stock["NR-1001"] == 10
70      assert shipments.shipments == []
71
72
73  def test_first_step_failure_needs_no_compensation() ->
    None:
74      saga, inventory, payments, shipments, _ = build(
75          {"inventory": {"reserve"}}
76      )
77      result = saga.run(ORDER)
78      assert result.status is SagaStatus.COMPENSATED
79      assert result.failed_step == "reserve_inventory"
80      assert result.compensated == ()
81      assert payments.charges == []
82      assert shipments.shipments == []
83
84
85  def test_compensation_failure_is_surfaced_not_swallowed
    () -> None:
86      saga, inventory, payments, shipments, events =
        build(
87          {"payments": {"charge", "refund"}}

```

```

88     )
89     # charge fails -> saga compensates reserve (refund
    is never reached
90     # because charge never completed). To exercise a
    FAILING compensation,
91     # fail schedule + refund.
92     saga, inventory, payments, shipments, events =
    build(
93         {"shipments": {"schedule"}, "payments": {"
            refund"}}
94     )
95     result = saga.run(ORDER)
96     assert result.status is SagaStatus.
    COMPENSATION_FAILED
97     assert result.failed_step == "schedule_shipment"
98     # Refund failed, so inventory release never ran:
    partial compensation
99     # is explicit in the result for an operator to
    reconcile.
100    assert result.compensated == ()
101    assert any(e == "compensation_failed" for e, _ in
    events)
102    assert inventory.stock["NR-1001"] == 7 # still
    reserved: needs a human
103
104
105    def test_orchestrator_rejects_empty_step_list() -> None
    :
106        import pytest
107
108        with pytest.raises(ValueError):
109            SagaOrchestrator([])
110
111
112    def test_saga_steps_without_compensation_are_skipped()
    -> None:
113        log: list[str] = []
114
115        def fail(_ctx) -> None:
116            raise ConnectionError("down")
117
118        saga = SagaOrchestrator(
119            [
120                SagaStep("fire_and_forget_email", lambda
                    ctx: log.append("email")),
121                SagaStep("charge", fail, compensate=lambda

```

```

122         ctx: log.append("refund")),
123     ]
124 )
125 result = saga.run({})
126 assert result.status is SagaStatus.COMPENSATED
127 assert result.compensated == () # email step has
    no compensation; charge never completed
    assert log == ["email"]

```

The tests cover the state space that matters: the happy path's step ordering; payment failure compensated by exactly one release; shipment failure compensated refund-then-release (reverse order, asserted on the tuple, not on vibes); first-step failure requiring no compensation at all; and the uncomfortable one — a compensation that itself fails, leaving stock reserved and the saga in `COMPENSATION_FAILED` with the partial compensation list intact for reconciliation. That last test is the difference between a saga implementation and a saga demo.

16.4 Common Pitfalls & Anti-Patterns

Each anti-pattern below has been the proximal cause of a real outage class. Format: what it looks like, why it fails, the fix.

P15.1 — The Distributed Monolith via Synchronous Chains

Looks like: eight services, and rendering one page calls all of them, in sequence, synchronously. *Why it fails:* availability multiplies downward — a chain of n services each at 99.9% availability offers the caller at most $0.999^n \approx 99.2\%$ for $n = 8$, before latency tails and retry amplification are counted. You have deployed a monolith with network faults inside its function calls. *Fix:* shorten synchronous chains ruthlessly; move non-critical work behind events (Section 16.2.1); replicate read data locally instead of fetching it per request; ask of every hop whether the response is needed *before* the caller's deadline or merely *eventually*.

P15.2 — Retries without Budget, Jitter, or Idempotency

Looks like: `for attempt in range(5): try: ...` at every layer, fixed 100 ms backoff, applied to every endpoint including non-idempotent POSTs. *Why it fails:* Theorem 16.1 — you have built a $39\times$ load multiplier that

engages precisely when the system is least able to absorb load, synchronized across clients by the shared fixed backoff, and applied to operations that must not run twice. *Fix:* the Listing 16.3.2 discipline: retry only idempotent operations or those carrying idempotency keys (Chapter 11), spend from a process-wide token bucket, jitter fully, respect deadlines, honor `Retry-After`.

P15.3 — One Breaker Shared across Dependencies

Looks like: a single global `CircuitBreaker("downstream")` wrapping calls to inventory, payments, and recommendations alike. *Why it fails:* the breaker's state is a hypothesis about one resource; a sick recommendations service now opens the circuit for payments — converting a partial degradation into a total one. *Fix:* one breaker per dependency per caller (Listing 16.3.1 enforces this by requiring a name), each with thresholds tuned to that dependency's traffic volume and SLO.

P15.4 — A Timeout Longer Than the Caller's

Looks like: the edge times out at 2s; catalog calls inventory with a 5s timeout “to be safe.” *Why it fails:* after the edge abandons the request, catalog keeps working for three more seconds — burning bulkhead permits, connections, and inventory capacity on work nobody will read. Under load this is stage 2–3 of Figure 16.2. *Fix:* Definition 16.3: budgets shrink strictly toward the leaves; propagate deadlines (gRPC does this on the wire [gRP24]); verify with a drill.

P15.5 — Ignoring Bulkheads

Looks like: all endpoints share one connection pool, one thread pool, one event loop's worth of unbounded concurrency into every dependency. *Why it fails:* the slowest dependency consumes the shared pool, and now healthy dependencies fail too — the failure domain is the whole process. *Fix:* per-dependency clients with their own `Limits`, semaphores per dependency (Listing 16.3.3), and reserve capacity for health checks and metrics so the control surfaces keep working during the incident.

P15.6 — Two-Phase Commit over HTTP

Looks like: `POST /prepare` on three services, then `POST /commit` on all, orchestrated over stateless HTTP. *Why it fails:* HTTP gives you no way to hold a prepare lock across requests, no coordinator-crash protocol, and no answer when the commit response is lost; you have re-implemented

2PC’s blocking, availability- coupling core with none of its correctness machinery. *Fix*: Section 16.2.10: sagas with compensations, the outbox for event emission, idempotent consumers. Accept eventual consistency as the design input it is.

P15.7 — Assuming the Mesh Fixes Application Semantics

Looks like: “we have Istio, so we can delete the retry logic, the idempotency keys, and the saga code.” *Why it fails*: the mesh sees packets, not meaning. It will happily retry your non-idempotent charge endpoint three times (now at infrastructure level, where your code review can’t see it), and it cannot express “compensate reserve-inventory with release.” *Fix*: treat the mesh as the transport-hygiene layer (mTLS, telemetry, per-hop timeouts, LB) and keep application-level correctness — idempotency, sagas, deadline budgets — in the application, where the semantics live.

P15.8 — Breakers and Budgets without Metrics

Looks like: a breaker that opens in production, and the first anyone learns of it is the 503 spike in a customer screenshot. *Why it fails*: every pattern in this chapter is a control system; a control system you cannot observe is a superstition. *Fix*: wire the `on_event` hooks of Listings 16.3.1 and 16.3.2 to your metrics pipeline; alert on `state_change` to open, on sustained `budget_exhausted`, and on probe-failure loops (open → half-open → open flapping).

Warning

The composite anti-pattern is *pattern theater*: breakers, retries, and timeouts present in code review but untested under failure. If you have never watched your breaker open in a drill, you do not have a breaker; you have a hope. Section 16.5 is the remedy.

16.5 Hands-On Production Lab & Real-World Case Study

Lab: Run the Chaos Drill, Watch the Breaker, Tune the Thresholds

Work in the chapter code directory. Everything runs offline.

1. **Baseline.** Create and populate the environment, then run the full suite and the drill:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
4 pytest -q
5 python demo_services.py
```

Confirm the drill reproduces Table 16.4: 30 calls reach the broken inventory without the breaker, 4 with it.

2. **Observe state transitions.** In `demo_services.py`, the breaker is constructed with an `on_event` hook feeding `DemoSystem.events`. Extend `_main` to print every `state_change` and `rejected` event with its payload, and re-run. You should see exactly one closed $\rightarrow$ open transition and no half-open probes, because the drill is shorter than `open_duration`.
3. **Watch a probe.** Shorten `open_duration` to 0.0-ish (use 0.05) and insert `await asyncio.sleep(0.1)` into the drill loop after the first batch; flip `chaos.always_fail` off mid-drill. Now a half-open probe succeeds and the breaker closes — observe the open $\rightarrow$ half_open $\rightarrow$ closed sequence in the events, and catalog returning 200s again.
4. **Tune thresholds.** Sweep `minimum_calls` over $\{2, 4, 8\}$ and `failure_rate_threshold` over $\{0.25, 0.5, 1.0\}$. Record calls-reaching-inventory for each combination. Lower floors open sooner (better protection, more false positives under transient single-failure noise); higher thresholds tolerate flaky-but-alive dependencies.
5. **Break the budget rule.** Set `CATALOG_TIMEOUT` larger than the caller's deadline and re-run the drill with latency chaos (`chaos.latency_seconds = 0.4`). Count abandoned work. Restore the rule.

War Story: The 200 ms Blip That Took Down Three Services

Northwind Robotics, a Tuesday. The warehouse service's storage layer hiccupped: *p99* latency jumped from 40 ms to 200 ms for ninety seconds — a blip, by any dashboard's standards. Here is what the composition did with it.

Inventory's calls to warehouse, budgeted at 100 ms, now timed out; inventory's client retried twice with a fixed 50 ms backoff. Catalog called inventory with retries of its own. Within forty seconds the warehouse was

receiving roughly $27\times$ its normal request rate — Theorem 16.1 with $a = 3$, $n = 3$, and the feedback loop of Figure 16.2 closing: each retry wave deepened the warehouse’s queue, which raised latency further, which converted more slow successes into timeout “failures,” which justified more retries. The warehouse, which had been *slow*, was now genuinely down; inventory’s shared connection pool exhausted waiting on it, so inventory stopped serving *all* requests, including the ones that never touched warehouse; catalog followed. Three services down, customer- visible, from a 200 ms blip. The blip had healed on its own within ninety seconds; the outage took forty minutes, because each restart walked straight back into $27\times$ load.

The postmortem fixes read as this chapter’s table of contents: per-hop deadline budgets that shrink toward the leaves; full jitter on every backoff; a process-wide retry budget of ten percent of request volume; one circuit breaker per dependency with a minimum-call floor; a dedicated warehouse client pool so inventory’s other endpoints kept their own capacity; and a standing chaos drill so the next 200 ms blip is a line in a metrics dashboard instead of an incident review. The blips did not stop. The outages did.

16.6 Self-Assessment Exercises & Deep-Dive Questions

1. **Deadline budgets.** A public API promises $p99 < 1.5\text{ s}$. The chain is $\text{edge} \rightarrow \text{orders} \rightarrow \text{payments} \rightarrow \text{ledger}$, and ledger’s measured $p99.9$ is 300 ms. Propose per-hop budgets satisfying Definition 16.3 with headroom at each layer, and state what you would measure to validate them.
2. **Amplification audit.** Your platform’s call graph has four hops; each hop currently retries up to twice with no budget. Compute the worst-case downstream load multiplier. Now impose a one-token retry budget per process at the two middle hops: recompute. Which hop’s budget matters most, and why?
3. **Breaker parameters.** A dependency serves 2,000 req/s to your fleet at 99.95% success, and its team asks you not to exceed 20 req/s during its recoveries. Design (θ, W, N, d, p, s) from Definition 16.4 and justify each number.
4. **Extend the breaker.** Add a *slow-call* dimension to Listing 16.3.1: calls slower than a threshold count as failures even when they succeed. Prove with a manual-clock test that five slow successes open the breaker.

5. **Hedge, honestly.** Extend the demo's catalog $\rightarrow$ inventory call to hedge after the measured $p95$. Instrument the extra-call rate. At what injected $q = \Pr[T > h]$ does your measured hedge rate match $1 + q$? What must be true of the endpoint for hedging to be safe?
6. **Outbox ordering.** The relay of Listing 16.3.4 publishes in `seq` order, but a crash mid-batch plus a retry can interleave replays with new events. Specify (and test) a per-aggregate ordering guarantee: what changes in the outbox schema, the relay, and the consumer?
7. **Choreograph the saga.** Re-express Listing 16.3.5's flow as choreography over the in-process broker: inventory emits `inventory.reserved`, payments consumes it, and so on. Where does the compensation graph now live? Write the test that catches a missing compensation subscription.
8. **Mesh threat model.** Your mesh provides mTLS everywhere. Enumerate three application-level failure or misuse modes that mTLS does nothing for, and name the chapter pattern that addresses each.

16.7 Interview Q&A Vault

Junior (Q1–Q10)

- Q1. **Synchronous versus asynchronous communication between services — what's the difference and when do you pick each?** Synchronous (REST, gRPC): the caller blocks for a response; simple mental model, tight temporal coupling, additive latency. Asynchronous (queues, streams): the caller hands work to a broker; low temporal coupling, natural fan-out and load smoothing, at the price of duplicates, lag, and harder status queries. Pick synchronous for user-facing queries that need an answer now; asynchronous for side effects, fan-out, and anything whose consumer can be down.
- Q2. **What is a timeout, and why is “no timeout” never the right default?** A timeout is an upper bound on how long a call may occupy the caller's resources. Without one, a slow dependency holds your threads, connections, and memory indefinitely — slow is worse than down, because down gives you an error to react to and slow gives you a resource leak.
- Q3. **What are the three states of a circuit breaker?** CLOSED: calls flow and failures are counted. OPEN: calls are rejected immediately without touching the dependency. HALF_OPEN: a

limited number of probe calls are admitted to test recovery; success closes, failure reopens.

- Q4. Why does a circuit breaker need a minimum number of calls before it can open?** Statistics. Two failures out of two overnight calls must not trip a circuit that carries thousands of calls per minute at noon. The minimum-call floor N ensures the failure rate is evaluated only once the window contains enough evidence to mean something.
- Q5. What is a bulkhead, in software?** A partition of resources — connection pools, thread pools, concurrency permits — so that one failing dependency cannot consume the capacity everything else needs. Named for ship compartments: a breach floods one compartment, not the hull.
- Q6. What is the difference between orchestration and choreography?** Orchestration: one coordinator explicitly drives each step and owns workflow state. Choreography: services react to events and emit their own; the workflow is emergent. Orchestration centralizes failure handling and status queries; choreography removes the coordinator as a bottleneck and single point of failure.
- Q7. What does it mean for a consumer to be idempotent, and why does at-least-once delivery require it?** An idempotent consumer processes the same event many times with the effect of processing it once — typically by recording processed event IDs in the same transaction as the state change. At-least-once delivery guarantees replays; without dedup, replays double-apply effects (charging twice, decrementing stock twice).
- Q8. What is service discovery?** The mechanism by which callers learn the current network locations of a logical service's instances — via DNS, a registry with health checking, or a platform API — replacing spatially coupled hardcoded addresses that break under autoscaling and rescheduling.
- Q9. Name three of the fallacies of distributed computing and a consequence of believing each.** “The network is reliable” → you cannot distinguish unprocessed from unacknowledged requests, so retries need idempotency. “Latency is zero” → tail latency, not the mean, must set your timeout budgets. “Topology doesn't change” → hardcoded addresses become outages the first time autoscalers run.

- Q10. What is a compensating action?** The semantic undo of a saga step: release the reservation, refund the charge, cancel the shipment. It is not a database rollback — the original step really happened and the world saw it; the compensation applies a forward action that restores the business invariant.

Mid (Q11–Q20)

- Q11. Derive the retry amplification of an n -hop chain where each hop makes up to a attempts.** Let L_k be calls arriving at hop k ; $L_0 = 1$ (the client request) and $L_{k+1} = a \cdot L_k$, so $L_k = a^k$ and the total downstream load is $S(n, a) = \sum_{k=1}^n a^k = a(a^n - 1)/(a - 1)$. With $a = 3$, $n = 3$: $3 + 9 + 27 = 39$ requests per client call — $39\times$ amplification, $27\times$ at the leaf (Theorem 16.1).
- Q12. How do retry budgets cap amplification?** By making retries spend from a process-wide token bucket where only retries (never first attempts) consume tokens. Attempts per logical request are then bounded by $1 + B$ regardless of configured `max_retries`, and sustained retry rate by the refill rate (Proposition 16.2). Listing 16.3.2 proves it: five permitted retries, two tokens, three requests total.
- Q13. Why jitter, and why *full* jitter?** Failures correlate clients in time; identical deterministic backoffs re-correlate them into waves that re-collide with the recovering service. Full jitter draws each sleep uniformly from $[0, \min(c, b \cdot 2^k)]$, decorrelating the herd while keeping the exponential envelope — at the cost of occasionally short delays, which is what the budget is for.
- Q14. Explain the deadline budget rule for a call chain.** Every hop's timeout must be smaller than its caller's remaining budget, with headroom: $T_i > \sum_{j>i} T_j$. Violation means the caller gives up while the callee keeps working — pure waste, and under load, pool exhaustion. gRPC propagates an absolute deadline in metadata so each hop derives its remaining budget instead of re-guessing [gRP24].
- Q15. Design breaker thresholds for a dependency with a 99.9% success SLO serving you 500 req/s.** Work backward from the SLO and volume. Baseline failure rate is 0.1%; set $\theta = 0.5$ so the breaker opens only when failure is unambiguously systemic, not at SLO noise. Window $W = 10$ s covers $\approx 5,000$ calls; set $N = 20$ (a floor far below normal volume so statistics are meaningful, far

above noise). Open duration $d = 5\text{--}30$ s, tuned to the dependency's observed recovery time. Probes: $p = 1\text{--}3$ concurrent, $s = 3$ consecutive successes — recovery is when probes succeed repeatedly, not once. Validate every number with the chaos drill, not with hope.

Q16. What is the tail-amplification trade-off of hedged requests?

Hedging fires a second identical call when the first exceeds a delay h ; with $q = \Pr[T > h]$, expected load per request rises to $1+q$ while the tail (e.g. $p99$) falls sharply. Constraints: hedge only idempotent operations (or carry idempotency keys), disregard the loser, and never hedge into an open circuit — hedging a sick dependency is amplification, not resilience.

Q17. Client-side versus server-side load balancing?

Server-side: a dedicated LB owns health checking and instance selection; clients stay simple; cost is an extra hop and a component to make redundant. Client-side: every client watches a registry and picks instances; fewer hops, richer per-client policy; cost is embedding correct LB logic into every client in every language. Meshes move client-side logic into the sidecar to get both.

Q18. Explain the outbox pattern and its delivery guarantee.

Domain write and event-record insert commit in one local transaction (atomicity); a relay polls the outbox, publishes, and marks published only after the broker accepts. A crash between publish and mark replays the event, so the guarantee is **at-least-once** — which is precisely why consumers must be idempotent. Together they yield effectively-once processing without distributed transactions (Listing 16.3.4).

Q19. What is backpressure, and name three concrete mechanisms.

The ability of an overloaded component to push load upstream instead of collapsing: explicit rejection (429/503 with **Retry-After**), bounded queues that fail fast when full, and credit-based flow control (HTTP/2 stream windows, gRPC). Unbounded queues are not backpressure; they are delayed failure.

Q20. Why is a slow dependency more dangerous than a dead one?

A dead dependency fails fast: errors are immediate and cheap, the breaker opens quickly, callers degrade. A slow dependency fails *expensively*: every call occupies a worker, a connection, and memory for the full timeout, so the caller's pools exhaust — stages 1–3 of Figure 16.2 — and the failure propagates to healthy functionality

sharing those pools.

Senior (Q21–Q27)

- Q21. Walk through a cascading failure from a latency blip to a fleet outage, and name the intervention at each stage.** Latency blip → blocked workers (intervene: per-hop timeouts) → pool exhaustion (bulkheads, bounded concurrency) → queue buildup (bounded queues, load shedding) → timeouts plus retries (budgets, jitter, breakers) → death spiral with flapping instances (gradual re-add, traffic ramps, staged restarts). Each stage’s signal and countermeasure: Table 16.2.
- Q22. Saga versus 2PC — when and why?** 2PC gives isolation at the price of blocking participants, coupled availability, and a coordinator whose crash leaves everyone in-doubt; over stateless HTTP it is unimplementable correctly. Sagas give atomicity of *outcome* via local transactions plus compensating actions, accepting visible intermediate states (no isolation). Choose sagas for cross-service business flows in always-available systems — which is to say, almost always; choose 2PC essentially only inside a single tightly administered database tier [Kle17].
- Q23. Choreographed versus orchestrated saga — trade-offs?** Choreography keeps each participant autonomous and removes the orchestrator as a bottleneck, but the workflow — and worse, the compensation graph — exists nowhere in code; debugging “where is order ORD-7?” requires reconstructing it from event logs. Orchestration centralizes sequence, state, and compensation, making status queries and failure handling explicit, at the cost of operating one more stateful component. Beyond two or three steps, orchestration is usually the honest choice.
- Q24. What does a service mesh actually solve — and what does it not solve?** Solves, uniformly and language-independently: mTLS workload identity, transport encryption, discovery, load balancing, per-hop retries/timeouts/circuit breaking, and golden-signal telemetry — moved from application libraries into the data plane. Does not solve: idempotency of your endpoints, saga compensation logic, schema evolution, end-to-end deadline budgets across business operations, or knowing which of your failures are retryable. The mesh owns transport hygiene; the application still owns semantics.

- Q25. How do you present eventual consistency honestly in an API contract?** Return 202 Accepted with Location to a status resource; model the workflow’s state machine as explicit, documented resource states (PLACED, PAID, SHIPPED, COMPENSATED) with machine-readable failure reasons; give clients poll or webhook completion signals; never report a terminal state you have not reached. Consistency boundaries hidden from clients become their incidents.
- Q26. Your breakers keep flapping open-closed-open after recovery. Diagnose.** Classic causes: $s = 1$ (one lucky probe closes onto a still-fragile dependency, re-fail, reopen), d shorter than the dependency’s real recovery time, and no traffic ramp — the closed breaker admits a thundering herd of pent-up traffic. Fixes: require several consecutive probe successes, lengthen d past measured recovery, and shed or ramp traffic on close. Alert on the flap rate itself.
- Q27. How do bulkheads interact with async event loops?** An event loop makes threads cheap but does not make *the dependency’s* capacity infinite: without semaphores per dependency you can still issue ten thousand concurrent calls into a service that can serve two hundred, buying you its collapse. The async bulkhead is a semaphore (plus per-dependency connection limits); Listing 16.3.3 uses both. Concurrency is cheaper, not free.

Staff (Q28–Q32)

- Q28. Design the resilience posture for a new user-facing endpoint with four downstream dependencies, two of which are non-critical.** Critical dependencies: deadline budgets derived from the endpoint SLO, one breaker each with thresholds justified from their SLOs and volumes, budgeted and jittered retries on idempotent calls only, per-dependency bulkheads sized below their healthy capacity. Non-critical: same machinery, plus designed degradation — the endpoint renders without them (cached or partial responses), their budget share is small, and their breakers’ open state is a silent fallback, not a 503. Then: a chaos drill proving the posture before launch, metrics hooks wired from day one, and an explicit answer for every dependency to “what does the user get when this is down?”
- Q29. When would you deliberately choose *not* to retry?** When the operation is non-idempotent and carries no idempotency key;

when the caller's remaining deadline cannot absorb the backoff (retrying produces work nobody awaits); when the error is deterministic (4xx validation, auth); when the budget is exhausted and the marginal retry's expected value is below its contribution to a developing retry storm; and in the middle of an incident, fleet-wide, where the kindest thing a client can do is fail fast and quietly.

Q30. How would you set organization-wide defaults for timeouts, retries, and breakers without creating false confidence?

Ship a thin client library encoding the chapter's machinery — budgets, jitter, per-dependency breakers, metrics hooks — with defaults derived from measured platform latency distributions, not folklore. Require per-service overrides to carry a justification and an expiration. Instrument everything and publish the flapping/open-rate dashboards. And run shared chaos drills, because a default nobody has seen fire is decoration (Section 16.5). Policy encodes the floor; drills prove the behavior.

Q31. A 200 ms latency blip in one leaf service took down three tiers. Reconstruct the mechanism and the minimal fix set.

Theorem 16.1 with synchronized backoffs: the blip converted slow successes into timeouts; retries multiplied load $\approx a^n$; the leaf's queues deepened, raising latency further (the feedback arrow of Figure 16.2); shared pools carried the failure sideways to healthy endpoints. Minimal fixes, in order of leverage: per-hop deadline budgets; full jitter; process-wide retry budgets; per-dependency breakers and bulkheads; gradual recovery (probes, traffic ramps). Any one helps; the composition is the fix, because the outage was a property of the composition.

Q32. Design the migration from in-code resilience libraries to a service mesh for a forty-service platform — what moves, what stays, and how do you avoid a regression mid-migration?

Moves to the mesh: mTLS and identity, discovery, load balancing, per-hop timeout/retry/breaker enforcement, telemetry. Stays in the application: idempotency, saga orchestration and compensation, deadline budgets across business operations, degradation/fallback semantics, retryability classification. Migration discipline: shadow first (mesh enforcing telemetry only), then enforce with in-code policy as the safety net — and make the two policies *identical* during overlap, because mesh retries stacked on library retries are amplification you chose on purpose. Remove in-code transport policy only after drills against the mesh configuration reproduce Table 16.4's behavior. Treat it as a protocol

migration (Chapter 10’s discipline), not an install.

16.8 External Bibliography & Further Reading

- Sam Newman, *Building Microservices*, 2nd ed., O’Reilly, 2021 — the reference for communication styles, orchestration versus choreography, and sagas in organizational context [New21].
- Michael T. Nygard, *Release It!*, 2nd ed., Pragmatic Bookshelf, 2018 — the origin text for circuit breakers, bulkheads, timeouts, and the stability patterns/anti-patterns vocabulary this chapter formalizes [Nyg18].
- Martin Kleppmann, *Designing Data-Intensive Applications*, O’Reilly, 2017 — chapters on distributed systems faults, consistency, and exactly-why-2PC-is-hard; the theoretical foundation for Section 16.2.10 [Kle17].
- Uwe Friedrichsen, *Patterns of Resilience* article series (uwe-friedrichsen.de) — a deep, pragmatic taxonomy of resilience patterns and their combinations; the best long-form companion to Sections 16.2.3–16.2.7.
- Betsy Beyer et al., *Site Reliability Engineering*, O’Reilly, 2016 — the chapters “Addressing Cascading Failures” and “Managing Load” (overload, load shedding, criticality) at Google scale.
- Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns*, Addison-Wesley, 2003 — the canonical vocabulary for messaging, idempotent receivers, and the patterns underlying Section 16.2.10.
- Envoy Proxy documentation (envoyproxy.io) — circuit breaking, outlier detection, retries, and the xDS APIs the mesh figure (Figure 16.3) abstracts.
- Istio documentation (istio.io) — control-plane architecture, traffic management, and mTLS identity model.
- The gRPC documentation on deadlines and cancellation [gRP24]; the FastAPI [Ram24] and httpx [Enc24] documentation for the toolkit used throughout the listings; and the twelve-factor methodology [Wig17] for the operational posture this chapter’s services assume.

16.9 Capstone Projects

Capstone 15.1 — Breaker Under a Microscope [junior]

Objective: Reimplement Listing 16.3.1 from Definition 16.4 alone, then attack it.

Inputs/Datasets: Listings 15.1–15.2; no datasets.

Steps: (1) Reimplement the breaker from the definition without consulting the listing. (2) Run Listing 16.3.1 unmodified against your implementation. (3) Seed three deliberate bugs — drop the minimum-call floor, forget window eviction, allow unlimited probes — and record which tests catch each. (4) Add one new test: `half_open_success_threshold=3` with a probe sequence of success–success–failure, asserting the breaker reopens and requires the full open duration again.

Acceptance Criteria:

- ☐ All Listing 16.3.1 tests pass against your reimplementation.
- ☐ Each seeded bug is caught by at least one existing test; you can name the mapping.
- ☐ The new probe-threshold test passes and fails against the seeded unlimited-probe bug.

Stretch Goals: Add the slow-call failure dimension (Exercise 4 of Section 16.6) with a manual-clock proof.

Capstone 15.2 — Latency Chaos and the Price of Hedging [mid]

Objective: Extend the drill to latency faults and measure hedged requests honestly.

Inputs/Datasets: Listings 15.5–15.6; the `ChaosConfig` latency knob.

Steps: (1) Drive inventory with `latency_seconds` drawn from a seeded bimodal distribution (95% fast, 5% slow). (2) Measure catalog *p*99 with no hedging. (3) Add hedging at the measured fast-mode *p*95 and re-measure *p*99 and the extra-call rate. (4) Verify the extra load matches $1 + q$ within sampling error across seeded runs.

Acceptance Criteria:

- ☐ Hedged *p*99 is materially below unhedged *p*99 on the same seed.
- ☐ Measured hedge rate equals $1 + q$ within the tolerance you derived.
- ☐ Hedging is disabled while the breaker is open; a test proves it.

- ☐ Everything is seeded and offline; no wall-clock assertions.

Stretch Goals: Cancel the loser request and measure the downstream savings; add a budget so hedges spend retry tokens too.

Capstone 15.3 — Production Outbox: Retry, DLQ, Ordering [mid]

Objective: Harden Listing 16.3.4 into the relay you would actually run.

Inputs/Datasets: Listings 15.7–15.8; a seeded event generator you write (Northwind Robotics orders, 10k events).

Steps: (1) Add relay retry with the Listing 16.3.2 budget for publish failures. (2) Add a dead-letter column/table after k failed attempts, with a requeue operation. (3) Enforce per-aggregate ordering: no event for aggregate A is published while an earlier event for A is unpublished after a failure — implement via per-aggregate sequencing in the outbox. (4) Tests: poison event goes to DLQ after exactly k attempts; aggregates without the poison event are unaffected; requeue restores flow exactly once.

Acceptance Criteria:

- ☐ Publish-failure retries respect the budget cap (proved with a counting publisher).
- ☐ DLQ after exactly k attempts; requeue is idempotent.
- ☐ Per-aggregate ordering holds under crash/replay injection; cross-aggregate throughput is unaffected.
- ☐ The Listing 16.3.4 suite still passes unmodified.

Stretch Goals: Add a lag metric (unpublished count, oldest unpublished age) to the relay’s event hook and alert thresholds to the lab’s dashboard list.

Capstone 15.4 — Measure Theorem 16.1 [senior]

Objective: Build a four-hop amplification testbed and verify the theorem empirically, then deploy the countermeasures and verify them too.

Inputs/Datasets: Listings 15.1, 15.3, 15.5; a fourth FastAPI tier you add downstream of warehouse.

Steps: (1) Wire edge $\rightarrow$ catalog $\rightarrow$ inventory $\rightarrow$ warehouse, each hop with a counting stub for its downstream and configurable retry count. (2) With the leaf hard-down and three attempts per hop, measure calls arriving at each hop; compare to 3, 9, 27 and the total to 39. (3) Add per-process one-token retry budgets at the two middle hops; re-measure;

derive the closed form for the budgeted case and check it. (4) Add full jitter and confirm (statistically, over seeds) that retry burst correlation drops.

Acceptance Criteria:

- ☐ Measured per-hop counts match a^k exactly in the unbudgeted run.
- ☐ Your closed-form prediction for the budgeted run matches measurement.
- ☐ A written paragraph explains any residual gap between theory and measurement.
- ☐ Fully offline, seeded, no wall-clock assertions.

Stretch Goals: Replace counting stubs with the real ASGI chain and show the theorem survives real HTTP machinery.

Capstone 15.5 — Resilience Architecture for Fleet Telemetry [staff]

Objective: Produce and defend a complete resilience design for Northwind Robotics' fleet-telemetry platform: ingestion API → normalizer → enrichment → storage, plus a user-facing dashboard read path.

Inputs/Datasets: This chapter; the Northwind Robotics endpoint set from prior chapters; SLO sheet you write (ingestion 99.95%, dashboard 99.9% at $p_{99} < 800$ ms).

Steps: (1) Draw the call graph and classify every edge: sync/async, coupling dimensions, retryability, criticality. (2) Assign deadline budgets, breaker parameters, bulkhead sizes, retry budgets, and degradation behavior per edge, each justified from the SLOs and a stated traffic model. (3) Write the failure-mode analysis table (extend Table 16.2) for your graph's top five scenarios, including one poisoning attack and one slow-dependency case. (4) Define the chaos-drill program: which faults, which cadence, which abort criteria. (5) Record the mesh boundary: which policies move to the mesh, which stay in the application, and why.

Acceptance Criteria:

- ☐ Every edge carries a justified, SLO-derived parameter set; no “default because default.”
- ☐ Deadline budgets satisfy Definition 16.3 on every path.
- ☐ The failure-mode table has a signal and a countermeasure per scenario; at least one scenario requires degradation, not just failure.
- ☐ The drill program includes abort criteria and a rollback story.

- The mesh boundary list contains at least three application-owned concerns with correct justifications.

Stretch Goals: Cost the design: extra requests from retries and hedges, engineering hours, and drill operational cost, against the incident cost model from Section 16.5’s war story.

16.10 Python Dependencies Appendix

All code runs offline after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

requirements.txt:

```
1 fastapi>=0.110
2 httpx>=0.27
3 pydantic>=2.7
4 pytest>=8
5 tenacity>=8.3
```

16.10.1 fastapi (≥ 0.110)

Description: The ASGI framework hosting the catalog, inventory, and warehouse services of Listing 16.3.3. **Why included:** real routing, dependency injection, and response handling over real ASGI request cycles — the chaos drill exercises genuine framework machinery, not a mock of it [Ram24].

Alternatives: Starlette directly (the demo would work nearly unchanged); Flask (WSGI — loses the async concurrency the drill depends on); a live multi-process deployment under uvicorn (faithful, but no longer offline-pure).

Installation: `pip install "fastapi>=0.110"` in the activated virtual environment.

16.10.2 httpx (≥ 0.27)

Description: The sync/async HTTP client used for every inter-service call in this chapter [Enc24]. **Why included:** `ASGITransport` wires clients to in-process FastAPI apps with the real request/response and timeout code paths — no sockets, no network; `Limits` provides the connection-pool bulkhead; the `TransportError` hierarchy is what Listing 16.3.2 classifies.

Alternatives: `requests` (sync only, no ASGI transport); `aiohttp` (fine, but its client/timeout model differs from this book’s other chapters); gRPC

with its own deadline propagation for the RPC variant (Chapter 14). **Installation:** `pip install "httpx>=0.27"`.

16.10.3 pydantic (≥ 2.7)

Description: Data validation and settings library; the model layer under FastAPI. **Why included:** it is FastAPI's contract surface — request/response schemas, validation errors — and the natural home for the typed configuration objects (breaker and retry policies) you will extract when these listings grow configuration files. **Alternatives:** `dataclasses` with hand-rolled validation (what the listings use for their internal value types, to keep the dependency surface visible); `attrs`; `msgspec` (fast, narrower ecosystem). **Installation:** `pip install "pydantic>=2.7"`.

16.10.4 pytest (≥ 8)

Description: The test framework running all 34 chapter tests. **Why included:** `fixtures`, `raises`, and `approx` keep the deterministic proofs (manual clocks, counting stubs, event-order assertions) short enough to read as specifications. **Alternatives:** `unittest` (stdlib; workable, more boilerplate); `nose2` (unmaintained momentum). **Installation:** `pip install "pytest>=8"`.

16.10.5 tenacity (≥ 8.3)

Description: The production-grade Python retry library: declarative stop/wait/retry conditions, jittered exponential backoff, async support. **Why included:** the chapter's Listing 16.3.2 deliberately teaches the machinery by building it; in real services you should reach for tenacity's battle-tested implementation and add this chapter's two contributions — the process-wide token-bucket budget and the deadline guard — on top. **Alternatives:** `pybreaker` for the circuit-breaker half (mature, sync-first; compare its state machine against Definition 16.4 as an exercise); `resilience4j` concepts if your estate is JVM (the patterns are identical; the Java implementation is the genre's reference); mesh-level retry policy (Envoy/Istio) for the transport tier — with Section 16.4 P15.7's caveat. **Installation:** `pip install "tenacity>=8.3"`.

16.11 Chapter Summary & Key Takeaways

- Distribution moves coupling from the compiler to the network. Classify every interaction by sync/async and orchestration/choreography, and

by its temporal, spatial, and logical coupling — then make each coupling a decision, not a default (Section 16.2.1).

- The fallacies of distributed computing are a checklist, not a trivia list; each ignored fallacy has a named failure mode, an early signal, and a metric (Section 16.2.2).
- Timeouts are the foundation: every call gets one, and budgets shrink strictly toward the leaves (Definition 16.3). A timeout longer than the caller's manufactures wasted work.
- Retries amplify multiplicatively: $S(n, a) = a(a^n - 1)/(a - 1)$ downstream requests per client request — $39\times$ for three attempts across three hops (Theorem 16.1). Token-bucket budgets cap it (Proposition 16.2); jitter decorrelates it; deadlines and **Retry-After** discipline it.
- The circuit breaker is a state machine with parameters (θ, W, N, d, p, s) , one per dependency: minimum-call floors against noise, rolling windows against staleness, bounded probes against stampedes (Definition 16.4, Listing 16.3.1).
- Bulkheads bound failure domains; hedged requests buy tail latency at an explicit $1 + q$ load tax; backpressure converts overload into cheap rejection instead of expensive collapse.
- Cascading failure is a six-stage spiral with a feedback loop; every stage has an observable precursor and a countermeasure (Figure 16.2, Table 16.2). Recovery must be engineered: probes, ramps, and staged restarts, because a fallen service cannot stand back up into the load that felled it.
- Discovery and load balancing decide *who talks to whom*; the mesh moves transport hygiene (mTLS, retries, timeouts, telemetry) into the data plane while the control plane configures it — and neither touches application semantics, which remain yours (Section 16.2.9).
- Distributed transactions: sagas with compensating actions instead of 2PC; the transactional outbox for atomic write-plus-publish; idempotent consumers to absorb at-least-once replays; and eventual consistency presented honestly in the API contract.
- Patterns you have not drilled are decoration. The chaos harness exists so you can watch the state machine fire before the incident does (Section 16.5).

16.12 Quick-Reference Card

Pattern selection

Symptom / risk	Reach for
Calls hang when dependency slows	per-hop timeouts + deadline budgets
Retry volume exploding during incidents	token-bucket retry budget + full jitter
One dependency poisoning the process	circuit breaker (one per dependency) + bulkhead
Tail latency dominates user experience	hedged requests (idempotent calls only)
Overload converting to collapse	backpressure: bounded queues, 429/503 + Retry-After
Dual-write inconsistency (DB + events)	transactional outbox + relay
Replayed events double-apply effects	idempotent consumer (dedup in same tx)
Multi-step flow fails halfway	saga with compensating actions (orchestrated)
Polyglot transport policy drift	service mesh data plane (keep semantics in-app)

Table 16.5: Symptom-to-pattern selection.

Breaker parameter cheat-sheet

Anti-amplification checklist

- ☐ Every outbound call has a timeout; budgets shrink toward the leaves on every path.
- ☐ Retries only on idempotent operations or with idempotency keys (Chapter 11).
- ☐ Every retry spends a token from a process-wide budget; first attempts are free.
- ☐ Backoff is exponential with full jitter; no fixed sleeps anywhere in the fleet.
- ☐ Call-level deadlines bound total retry time; overshoot skips the retry.
- ☐ **Retry-After** from 429/503 responses overrides local schedules.

Parameter	Start at	Tune by
θ failure rate	0.5	lower for flaky-but-critical, never below SLO noise
W window	10–60 s	dependency’s failure-burst duration
N minimum calls	10–20	high-volume deps: higher; low-volume: use duration windows
d open duration	5–30 s	measured recovery time of the dependency
p probe concurrency	1	raise only with a traffic-ramp story
s probe successes	2–3	raise if you observe flap (open/close oscillation)

Table 16.6: Starting points for Definition 16.4’s parameters. Every number is a hypothesis until a chaos drill confirms it.

- ☐ One breaker per dependency; bulkheads sized below each dependency’s healthy capacity.
- ☐ Breaker and budget events flow to metrics; opening and flapping are alertable.
- ☐ The chaos drill runs on a schedule, with abort criteria — and someone watches the state transitions.

16.13 Standardized Evidence Addendum

16.13.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 16.7: Minimum benchmark table to keep per chapter.

16.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.

- **Result:** what changed in correctness, resilience, latency, and operability.

16.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 16.8: Operational budget template for this chapter’s technique.

16.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

16.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

16.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.

2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

16.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 16.9: Decision card to complete per chapter.

16.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

16.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.

- Test restores regularly; an untested backup is not a backup.

Chapter 17

API Security Engineering (OWASP API Security Top 10)

Every API you ship is an adversary-facing product surface. The friendly consumers integrate once and file tickets; the unfriendly ones enumerate your object identifiers at machine speed, replay your tokens, probe your legacy routes, and ask your server to fetch `169.254.169.254` on their behalf. The uncomfortable fact of the last decade of breach reports is that the dominant failures are not exotic cryptographic breaks — they are *authorization* failures: the server correctly identified the caller and then handed over someone else’s object. The OWASP API Security Top 10 (2023) [OWA23] is the industry’s distilled incident ledger, and this chapter treats it the way an incident ledger deserves: item by item, each with its mechanism, a realistic exploit narrative set in the Northwind Robotics universe, a concrete mitigation, and the detection signal that tells you the exploit is being attempted against *you*.

The chapter’s pedagogical engine is adversarial and executable. We build a deliberately vulnerable FastAPI service that seeds all ten flaws — each fenced with a comment naming its OWASP item — then build the hardened counterpart, then run a ten-test exploit suite that proves both directions: the exploit lands on the vulnerable build and is repelled by the hardened one. Security claims you cannot execute are marketing; here every claim is a pytest. Around that centerpiece we build the server-side identity stack with equal rigor: the OAuth 2.0/OIDC token-validation pipeline in its only safe order (structure, algorithm allow-list, key resolution, signature, `iss/aud/exp`, then claims-based authorization) [Jon15; Har12], scope design from coarse to fine, token exchange and audience restriction for service-to-service calls, and mTLS with SPIFFE/SPIRE for workload identity where tokens are the wrong tool [Res18]. We then widen the aperture to the disciplines that keep the ten items closed permanently: STRIDE threat modeling per endpoint with a worked table, attack trees and abuse-

case writing, injection and canonicalization attacks, SSRF defense-in-depth, response security (headers, CORS, CSRF), and secrets management with scanning in CI.

Key Concept

Authentication is not authorization, and neither is a substitute for validation. Every request must cross three independent gates, in order: (1) *authenticate* — verify the credential cryptographically; (2) *authorize* — check function scope, then the *object*, then the *properties*; (3) *validate* — treat all input and all upstream output as hostile. The OWASP API Top 10 is, to first order, the catalog of what happens when one of the three gates is missing, and the defense is never a single wall: $P(\text{breach}) = \prod_i P(\text{layer } i \text{ fails})$, so independent layers multiply protection while dependent layers merely add ceremony.

17.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter you will be able to:

1. Explain each OWASP API Security Top 10 (2023) item — its mechanism, a realistic exploit, the concrete mitigation, and the detection signal — and map real incidents to the taxonomy.
2. Implement object-level authorization (BOLA defense) and property-level authorization (mass-assignment and excessive-exposure defense) as explicit, testable guards rather than conventions.
3. Build a production-grade token-validation pipeline: JWKS key resolution, algorithm allow-listing, issuer/audience/expiry enforcement, and claims-to-principal mapping.
4. Design OAuth scopes (coarse versus fine) and apply audience restriction and token exchange to service-to-service calls.
5. Decide when mTLS, bearer tokens, or both are the right carrier of service identity, and operate certificate rotation safely.
6. Threat-model an API with STRIDE per endpoint, attack trees, and abuse cases, and convert threats into acceptance criteria.
7. Defend against injection, canonicalization, and path-traversal attacks,

and engineer SSRF defense-in-depth (allow-lists, DNS pinning, redirect re-validation, metadata-endpoint blocking).

8. Operate secrets correctly: vault/KMS patterns, rotation, and leak-scanning in CI.

Prerequisites

This chapter assumes Chapters 0–15, most directly: HTTP semantics and headers (Chapter 2), consuming APIs in Python with `httpx` (Chapter 3), client-side credential handling and JWT internals (Chapter 6), FastAPI routing and dependency injection (Chapter 7), validation and `application/problem+json` errors (Chapter 8), and rate limiting (Chapter 15), whose token bucket reappears here as the API6 defense. Familiarity with versioning (Chapter 10) helps for the inventory discussion (API9). All code is Python 3.11+, typed, offline-first: a simulated network and a local JWKS fixture replace the internet and the identity provider.

17.2 Deep Technical Mechanics & Architectural Concepts

17.2.1 The Trust Model: Where Requests Cross Boundaries

Security engineering begins by deciding what you refuse to trust. An API request crosses at least four trust boundaries before it touches data: the network edge (TLS termination, WAF), the authentication boundary (who are you?), the authorization boundary (may you do this to *this* object?), and the domain boundary (is the input well-formed, is the upstream response sane?). The classic failure is collapsing these into one: a gateway that authenticates and then lets the service assume authorization happened, or a service that validates input but trusts upstream JSON blindly. Figure 17.1 shows the pipeline this chapter builds in code.

Definition 17.1 (Trust boundary). A *trust boundary* is any edge in the data-flow graph across which the provenance guarantees of data change: from unauthenticated network input to an authenticated principal, from a principal to an authorized one, from a process to its data store, or from your service to a third-party API. Validation and authorization checks belong *at* the boundary, not somewhere downstream of it.

Definition 17.2 (Authentication, authorization, validation). *Authentication* (authN) establishes *who* the caller is, with cryptographic evidence.

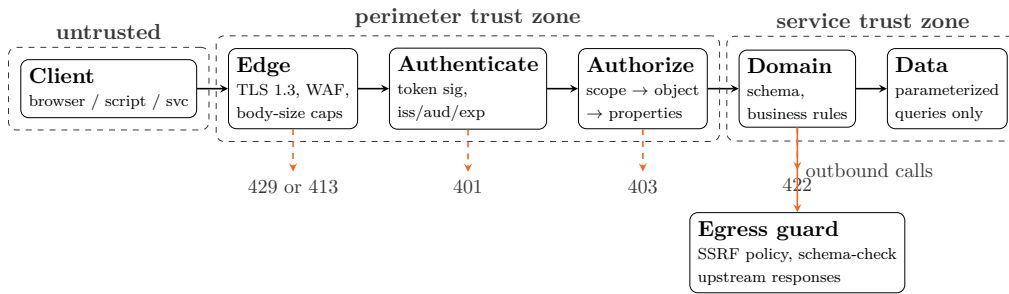


Figure 17.1: Request trust-boundary and validation pipeline. Each gate is independent and can reject on its own evidence; the egress guard applies the same suspicion to traffic *leaving* the service (API7, API10).

Authorization (authZ) decides whether that caller may perform *this function*, on *this object*, touching *these properties*. *Validation* decides whether a payload is well-formed and within contract, regardless of who sent it. The three are orthogonal: a perfectly authenticated admin can still send `' ; DROP TABLE robots ; -` as a robot name.

17.2.2 The OWASP API Security Top 10 (2023) at a Glance

Table 17.1 compresses the taxonomy; the subsections that follow expand each row into mechanism, exploit, mitigation, and detection. Notice the shape of the list: five of ten items (API1, API3, API5, API6, API9) are *authorization and inventory* failures, not cryptographic ones. Teams that spend their entire security budget on TLS and JWT libraries are armoring the wrong gate.

17.2.3 API1:2023 — Broken Object Level Authorization (BOLA/IDOR)

Mechanism. The endpoint resolves an object identifier from the path or body (`/v1/robots/1002`) and returns or mutates the object after checking *that* the caller is authenticated but not *whether* the caller may touch this object. Authorization, if it exists at all, lives at the route level; the per-object decision is missing. Sequential or predictable identifiers turn the flaw into bulk enumeration.

Exploit narrative. An attacker at Northwind Robotics captures their own fleet app’s traffic, sees `GET /v1/robots/1001`, and changes the id to 1002: a competitor’s robot, full record. A five-line loop from `id=1` to `id=90000` exfiltrates every robot in the fleet — names, locations, notes, and

(thanks to API3) their service keys. No credential is stolen; no alarm trips, because every request is a well-formed, authenticated 200 OK.

Mitigation. Make every object access carry its subject: fetch the record, then evaluate `guard.require_read(record)` where the guard compares `record["owner_id"]` (and `tenant_id`) against the authenticated principal (Listing 17.3.6). Two design choices shrink the blast radius further: unguessable identifiers (UUIDv4/UUIDv7) as *defense in depth* — never as the control — and default-deny policies evaluated server-side on every request, including list endpoints (filter *then* paginate; see Chapter 9).

Detection signal. Per-principal object-access entropy: one subject reading objects owned by many distinct owners, monotonically increasing ids, or 4xx-then-200 sweeps. Alert on `owner_id != sub` successes, not on failures alone — the failures are the attacker’s rehearsal.

17.2.4 API2:2023 — Broken Authentication

Mechanism. Anything that lets a caller become someone else without their credential: unsigned or unverified tokens (our teaching app’s base64 “tokens”), acceptance of `alg=none` or RS256→HS256 confusion, missing expiry checks, weak password reset flows, credential stuffing without throttling, or hardcoded master keys shipped in the binary.

Exploit narrative. The attacker base64-encodes `user:mallory` — or reads the hardcoded master key `sk_test_DEMO_masterkey_invalid` out of a leaked debug endpoint — and the API obligingly becomes anyone. No brute force required; the protocol itself is the vulnerability.

Mitigation. Adopt the validation pipeline of Section 17.2.13 verbatim: asymmetric signatures (RS256/ES256), an algorithm allow-list, kid-keyed JWKS resolution, mandatory `iss/aud/exp`, short token lifetimes, and refresh rotation. Rate-limit and lock out authentication endpoints (Chapter 15); hash stored secrets with a password KDF; and scan the repo so a master key can never reach production (Section 17.2.19).

Detection signal. Sudden 401 spikes (credential stuffing), one IP cycling many subjects, tokens with unexpected `kid` or `alg` values arriving at all, and any successful auth whose token predates the latest key rotation.

17.2.5 API3:2023 — Broken Object Property Level Authorization

Mechanism. Two halves of one flaw. On write, *mass assignment*: the handler binds the request body directly onto the model (`record.update(patch)`), so clients can set server-owned fields — `owner_id`, `role`, `is_admin`, `price`. On read, *excessive data exposure*: the endpoint serializes the internal model

wholesale and relies on the client to hide fields, so `service_key` and internal notes ride along in every 200.

Exploit narrative. The attacker sends `PATCH /v1/robots/1001 {"owner_id": "mallory"}` and now owns the robot; a follow-up `GET` conveniently includes the robot's service key, harvested for lateral movement. Both directions come from the same root cause: the wire schema was never distinguished from the storage schema.

Mitigation. Explicit contracts both ways. On write: a request model with `extra="forbid"` (Pydantic v2 [Col24]) so unknown fields are 422 before logic runs, *plus* a role-keyed write policy (`ROBOT_WRITE_POLICY`) so even known fields require the role; then re-assert invariants (`id`, `owner_id`, `tenant_id`) from the stored record. On read: serialize through a role-keyed field filter (`filter_fields`); the response is a projection, never the row. Section 17.3 implements both.

Detection signal. 422 spikes with unknown-field errors (someone is probing the schema), patch bodies containing server-owned field names in logs, and response-size anomalies on read endpoints.

17.2.6 API4:2023 — Unrestricted Resource Consumption

Mechanism. Any unbounded knob: page size, simulation steps, file upload size, concurrency, or third-party billable calls (SMS, geocoding). The attacker does not need to break in; they merely need to make the service do 10^7 units of work per request, or one request per second forever, until CPU, memory, connection pools, or the cloud bill becomes the outage.

Exploit narrative. `POST /v1/fleet/simulate?steps=40000000` from a dozen free-tier accounts. Each request pins a worker for minutes; the pool starves; legitimate dispatch calls queue behind the bonfire. In a variant, the attacker enumerates `/v1/robots?limit=1000000` and lets serialization do the damage.

Mitigation. Ceilings at the schema layer (`Query(le=10_000)`), body-size limits at the edge, per-principal and global rate limits with `429/Retry-After` (Chapter 15), timeouts on every outbound call, and cost-weighted budgets for billable third-party calls. Reject early and cheaply: a 422 computed from the query string costs microseconds.

Detection signal. Latency and queue-depth percentiles (p95/p99) by endpoint, per-principal work-unit totals, and 4xx/429 rate anomalies — the attacker's probing phase is visible before the denial of service is.

17.2.7 API5:2023 — Broken Function Level Authorization

Mechanism. Administrative or privileged functions (`/v1/admin/users`, `/internal/export`) protected by nothing but the assumption that only admins know the URL — or by a front-end that hides the button while the route stays open. Group/role checks are missing or derived from client-supplied claims.

Exploit narrative. The attacker’s ordinary operator token calls `GET /v1/admin/users`; the server checks that *a* user exists, never *which* role, and returns the directory. From there: targeted phishing, password-reset abuse, and role mining.

Mitigation. Function-level checks in the dependency layer, not by convention: `require_scope("admin:users")` *and* a role check, default-deny on every non-public route, and denial tests in CI for every privileged route with every non-privileged role. Obscurity is not a control; route tables are public knowledge the day the OpenAPI spec leaks.

Detection signal. 403s on admin routes clustered per subject (probing), successful admin calls from principals outside the admin group (alarm, not log line), and drift between the route table and the role matrix.

17.2.8 API6:2023 — Unrestricted Access to Sensitive Business Flows

Mechanism. The flow works exactly as designed — redemption, dispatch, purchase, password reset — but nothing limits *how often* or *by how many identities* it can be driven. The vulnerability is economic: the legitimate flow, automated, at scale.

Exploit narrative. Northwind launches `POST /v1/promo/redeem` with code `NWR-LAUNCH`. A reseller scripts twelve redemptions per second across two hundred free accounts; the entire campaign budget is gone in ninety seconds, and every request was a valid, authenticated, schema-conformant call.

Mitigation. Velocity limits per principal *and* per flow (token bucket, Chapter 15), device fingerprinting and proof-of-work or CAPTCHA on unauthenticated flows, one-time idempotency keys where semantics allow (Chapter 11), and business invariants as code: “one welcome bonus per verified identity” is a constraint the system must enforce, not a hope.

Detection signal. Flow-level funnel anomalies: redemption rate by principal, by IP ASN, by account age; bursts that track campaign launches; success rates far above the human baseline.

17.2.9 API7:2023 — Server-Side Request Forgery (SSRF)

Mechanism. The server fetches a caller-influenced URL — webhooks, “fetch favicon”, diagnostics pings, PDF-from-URL — and the attacker aims it inward: `http://169.254.169.254/latest/meta-data` (cloud instance credentials), `http://inventory.internal.nwr/`, or a redirect chain that starts at an allowed host and ends inside the network.

Exploit narrative. The attacker posts a JSON body whose `url` field names `http://169.254.169.254/latest/meta-data` to the diagnostics endpoint and receives the fleet-worker IAM credentials in the response body. When the team adds a naive “block 169.254.*” string check, the attacker switches to a redirect: an allowed shortener host that 302s to the metadata endpoint — most naive fetchers follow redirects blindly.

Mitigation. Defense in depth, each layer independently sufficient (Section 17.2.17): scheme allow-list, exact host allow-list, port restriction, DNS resolution with private/link-local range rejection *before* connecting (defeats DNS rebinding), redirect re-validation per hop, and response size/type caps. Egress firewalling at the platform layer makes the application bug survivable.

Detection signal. Outbound destinations by host histogram (a diagnostics feature has a tiny, stable destination set); any egress to link-local or RFC 1918 space from application pods; 302-chasing in egress logs.

17.2.10 API8:2023 — Security Misconfiguration

Mechanism. Everything that is “on” that should be “off”: `/v1/debug/config` in production, stack traces in 500s, wildcard CORS with credentials, directory listing, default credentials, permissive TLS versions, unpatched framework CVEs, open cloud storage buckets. Misconfiguration is the compost heap where the other nine items grow.

Exploit narrative. The attacker requests `/v1/debug/config` — linked from a two-year-old internal wiki page — and receives the master API key and the database DSN. Alternatively they trigger a 500 and read the traceback: absolute paths, dependency versions, ORM internals — a free reconnaissance report.

Mitigation. Hardened defaults as code: debug surfaces compiled out of production builds (not merely “unlinked”), opaque 500s with the detail in server logs, exact-origin CORS, secure response headers on every route (Section 17.2.18), pinned and scanned dependencies, and a repeatable environment build so “it was only staging” cannot drift into production.

Detection signal. Internet scan traffic hitting `/debug`, `/actuator`, `/.env`; response-header audits in CI; config drift alerts between declared and observed posture.

17.2.11 API9:2023 — Improper Inventory Management

Mechanism. APIs you forgot you run: `/v0/robots` from 2023 still routable, a shadow endpoint that never entered the spec, a staging deployment reachable from the internet, third-party integrations nobody owns. Every forgotten surface runs forgotten — meaning absent — controls.

Exploit narrative. The attacker guesses `/v0/robots` (or finds it in an archived client SDK) and gets the entire fleet, no authentication — `v0` predates the auth gateway migration. The hardened `v1` is irrelevant; the attacker shops where the lights are off.

Mitigation. Inventory as an operational artifact: a single source of truth for deployed versions and routes (generated from the OpenAPI spec, Chapter 21), sunset headers and enforced end-of-life for every version (Chapter 10), gateways that default-deny routes not in the inventory, and documented data-flow for every third-party integration.

Detection signal. Requests to routes absent from the inventory (gateway-level diff), traffic share by API version (nonzero `v0` traffic *is* the alert), and certificate/DNS enumeration of your own estate.

17.2.12 API10:2023 — Unsafe Consumption of APIs

Mechanism. The trust boundary you forgot is the one facing your *providers*: upstream responses deserialized without schema validation, upstream redirects followed into internal space, upstream payloads merged into your models, upstream “errors” trusted as success. Third parties get breached, misconfigured, or merely sloppy; your parser inherits all three.

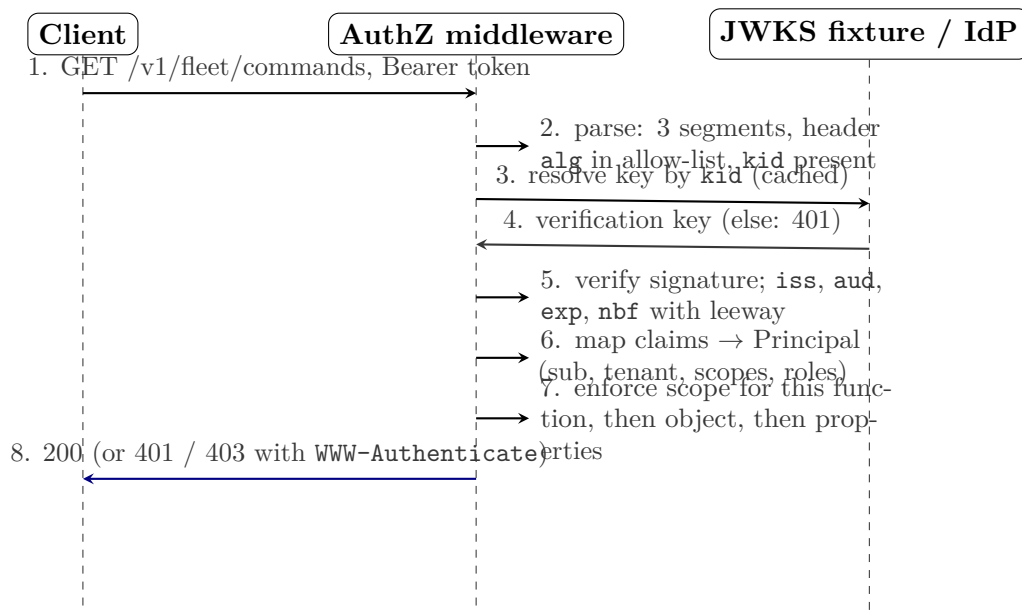
Exploit narrative. The fleet-enrichment job merges whatever the mapping partner returns into the robot record. A compromised (or simply hostile) partner returns `{"maintenance_mode": true}` and the fleet quietly parks itself. No credential was stolen; the trust was the exploit.

Mitigation. Treat upstream output like user input: parse against an explicit schema with `extra="ignore"` (or `"forbid"`), copy only vetted fields into domain objects, cap sizes and timeouts, never follow upstream-supplied URLs without the SSRF gauntlet, and record upstream provenance for audits. Evaluate third parties’ security posture as part of procurement — their incident is your incident.

Detection signal. Schema-validation failures on upstream payloads (a spike means compromise or contract drift), unexpected fields in upstream responses, and behavioral diffs after partner releases.

17.2.13 Server-Side OAuth 2.0 & OIDC: The Token Validation Pipeline

Chapter 6 treated tokens as a client-side possession problem. Here the shoe is on the other foot: you are the resource server, and every arriving bearer token is an accusation you must verify. The verification order is itself a security property — claim inspection before signature verification is how `alg=none` and key-confusion bugs happen. Figure 17.2 shows the pipeline that Listing 17.3.5 implements.



Any failure before step 6 discards the token without reading claims.
“Trust, then verify” inverted is “verify, then trust.”

Figure 17.2: Token validation sequence. Steps 2–5 are *authentication*; only after them may claims be read; step 7 is *authorization* and is a separate decision per function, object, and property.

Scope Design: Coarse versus Fine

Scopes are the function-level authorization vocabulary of OAuth [Har12]. The design spectrum runs from *coarse* (`read/write`) through *resource-scoped* (`fleet:read`, `fleet:command`) to *fine-grained* (`robots:1001:command`). Coarse scopes keep tokens small and policies centralized but over-privilege every holder; ultra-fine scopes push object-level decisions into the token issuer, which explodes token count and still cannot express *dynamic* ownership (the issuer does not know who bought robot 1002 yesterday). The pragmatic middle, used throughout this chapter:

- Scopes express *capability classes*: `fleet:read`, `fleet:write`, `fleet:command`, `promo:redeem`, `admin:users`.
- Roles express *organizational standing*: `owner`, `technician`, `admin`.
- Object and property decisions stay in the resource server’s guard layer (Listing 17.3.6), which knows the data.

Claims-Based Authorization

Once — and only once — the token verifies, claims become your typed principal: `sub` (subject), `tenant` (tenancy boundary), `scope` (capability set), `roles`. Treat claim parsing as untrusted-input handling: type-check every claim, reject malformed ones with 401, and never accept a claim the IdP did not promise to control (user-editable profile fields are not authorization claims). The `Principal` dataclass in Listing 17.3.5 freezes this mapping: downstream code sees immutable, validated facts, not a raw JWT payload dict.

Audience Restriction and Token Exchange

A token is a letter addressed to someone. `aud` is the address: the fleet API must reject tokens issued for the billing API even when both are signed by the same IdP, or a low-value integration becomes a lateral-movement vehicle. For service-to-service calls, prefer *token exchange* (RFC 8693 semantics): service A exchanges the inbound user token for a new token with audience `service-b` and the *minimum* scopes B needs — preserving the user’s identity for audit (the `act` claim) while enforcing least privilege per hop. Impersonation (acting *as* the user) versus delegation (acting *on behalf of*, visibly) is a deliberate audit choice; default to delegation.

Warning

The five ways token validation goes wrong in production: (1) reading claims before verifying the signature; (2) accepting `alg` from the token instead of a server-side allow-list; (3) fetching JWKS from a URL *inside the token* (`jku` injection) rather than a pinned issuer location; (4) skipping `aud` because “we only have one API today”; (5) treating expiry as the only temporal check while ignoring `nbf` and clock skew in the other direction. Listing 17.3.5 closes all five; its tests attack all five.

17.2.14 mTLS for Service Identity

Bearer tokens answer “what may this call do”; mTLS answers “what *is* this workload” — at the transport layer, before a byte of application data flows [Res18]. In a mutual TLS handshake the server sends `CertificateRequest` after its own certificate message; the client responds with its certificate chain and a `CertificateVerify` signature over the handshake transcript, proving possession of the private key. Both sides authenticate; a stolen token without the client certificate is useless at the socket.

Operational realities. Certificate *rotation* is the whole game: short-lived workload certificates (hours, not years) issued by an internal CA, overlapped trust bundles so old and new chains verify during rollover, and automated renewal — a manually rotated certificate fleet is a future outage. **SPIFFE** standardizes the workload identity document (a SPIFFE ID like `spiffe://nwr.example/ns/fleet/sa/dispatcher` carried in a X.509 SVID or JWT-SVID); **SPIRE** is the runtime that attests workloads (node/process attestation) and delivers SVIDs to them. The combination removes the “where does the bootstrap secret live” problem that plagues token-based service identity.

When mTLS, when tokens. Use mTLS for *machine* identity on the wire: mesh-internal service calls, gateway-to-service, anywhere a sidecar or platform can own certificate lifecycle. Use bearer tokens for *user/delegated* identity and cross-boundary calls where semantics (scopes, audience, expiry) must travel with the request. Mature platforms run both: mTLS authenticates the workload channel; the token layered inside it carries the end-user context — and neither is accepted as a substitute for the other.

17.2.15 Threat Modeling APIs

Threat modeling converts paranoia into a checklist. STRIDE classifies threats by effect — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege — and applying it *per endpoint* keeps the analysis concrete: each route is a small system with inputs, outputs, state changes, and an actor model. Table 17.2 is the worked example for `PATCH /v1/robots/{id}`; Listing 17.3.8 generates the skeleton mechanically from an OpenAPI document so the worksheet cannot drift from the shipped surface.

Attack trees complement STRIDE’s breadth with depth: root goal (“read another tenant’s robot record”), intermediate nodes (“obtain a valid token” OR “bypass object check” OR “read via legacy v0”), leaves mapped to concrete controls. The tree’s value is the OR-branches — they force you to close *every* path, not the fashionable one. **Abuse cases** write the attacker’s user story in the same backlog grammar as features: “As an

attacker, I enumerate sequential robot ids so that I can map the entire fleet,” with acceptance criteria inverted into security requirements.

Figure 17.3 maps the same six categories onto the sample fleet API’s attack surface: every arrow into a component is a place where a STRIDE category can materialize, and the labels carry the OWASP item that operationalizes it.

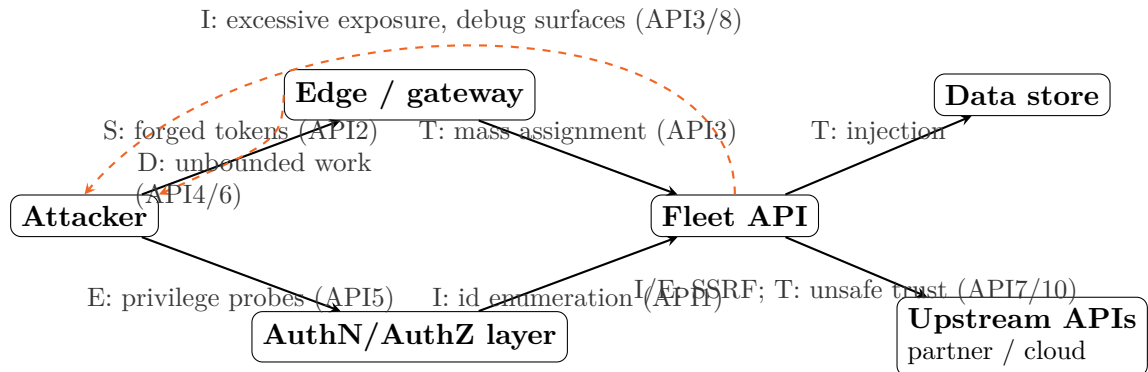


Figure 17.3: STRIDE attack-surface map for the Northwind Robotics fleet API. Edges are labeled by the STRIDE category they expose and the OWASP item that weaponizes it.

Security requirements as acceptance criteria is where modeling pays rent. Each abuse case lands in the definition of done as an executable assertion: “*Given alice’s token, when she requests robot 1002, then the response is 403*” is not a sentence in a wiki — in this chapter it is `test_api1_bola_cross_owner_read`. The exploit suite (Section 17.3) is the acceptance-criteria document, compiled.

Example

Abuse case to acceptance criterion, in one move.

Abuse case: “As a reseller bot, I redeem the launch promo thousands of times across fresh accounts to drain the campaign budget.”

Acceptance criterion: `test_api6_promo_redemption_farming` — twelve consecutive redemptions from one principal must yield at most three 200s and a 429 with `Retry-After`. The product owner can read it; the CI runner can execute it; the attacker cannot argue with it.

17.2.16 Injection and Input Attacks

APIs concentrate injection risk because every parameter is remote input by design. The taxonomy is one idea with four costumes: a string the caller controls is later interpreted as *code* by some engine.

SQL injection survives the ORM era through raw fragments: `f"WHERE owner = '{owner}'"`, “just this once” report queries, and ORM escape hatches. The defense is non-negotiable: parameterized queries, always; where identifiers (table/column names) must be dynamic, map them through a server-side allow-list — never interpolate. **NoSQL injection** trades quotes for operators: a JSON body `{"username": {"$gt": ""}}` turns a login lookup into an always-true match when the driver accepts operator objects from the wire. The defense is schema validation that pins *types*, not just fields (Pydantic’s strict types [Col24]), and rejecting operator objects at the boundary. **Command injection** appears in “run this diagnostic” features: any path from request data to `os.system` or `subprocess` with `shell=True` is a pre-existing breach; pass argument lists, never strings. **Server-side template injection** hides in features that render caller-controlled strings through a template engine (email subjects, report headers): `{{7*7}}` returning 49 is the canary; the fix is to render caller data as *data*, never as template source.

Canonicalization pitfalls are the filter-bypass division. Unicode normalization folds visually distinct identifiers together (NFKC-collapse), so an allow-list on `username` is bypassed by a confusable; percent-decoding twice (`%252e%252e%252f` → `../`) walks straight past a single-decode path check; and overlong UTF-8 encodings historically smuggled / past naive scanners. Rules: decode *once*, validate on the canonical form, compare with normalization you chose deliberately, and reject — never “clean” — input that fails.

Path traversal via IDs is the API-shaped cousin: a download endpoint keyed by filename (`/v1/firmware/{name}`) that joins the id onto a base directory. The defense is a strict identifier grammar (`[A-Za-z0-9-]{1,64}`), `Path.resolve()` plus a containment check against the base directory, and — better — indirection: ids that are keys into a manifest, not filenames at all.

17.2.17 SSRF Defense-in-Depth

SSRF deserves its own blueprint because single-layer defenses keep falling. The layers, all implemented in Listing 17.3.7:

1. **Scheme allow-list.** `https` (plus `http` only where the lab demands it). This alone kills `file://`, `gopher://`, `dict://`.

2. **Exact host allow-list.** Not suffix matching — `example.com.evil.example` ends with your string too — and no raw IP literals or userinfo (`user@host` tricks).
3. **Port restriction.** 80/443/8443 only; the metadata endpoint speaks HTTP on 80 but internal admin panels speak 8080/9000.
4. **DNS pinning.** Resolve the host yourself, reject any answer in private/loopback/link-local ranges, and connect to the validated address. This closes the *DNS rebinding* race, where the name resolves publicly at validation time and privately at connect time (TOCTOU). The fetcher validates resolution and never hands the raw hostname back to a resolver it does not control.
5. **Redirect re-validation.** `follow_redirects=False`; every 30x target goes back through layers 1–4. The allowed shortener that 302s to `169.254.169.254` dies here.
6. **Response caps.** Size and content-type limits so the fetch cannot become a memory-amplification or parser-confusion channel.
7. **Platform egress.** Network policy at the cluster/firewall layer denying pods any route to `169.254.169.254` and RFC 1918 space: the bug becomes survivable even when code fails.

17.2.18 Response Security

Security is also what you *return*. Excessive data exposure (API3’s read half) is the headline item — serialize projections, never rows. Beyond the body:

Security headers. `X-Content-Type-Options: nosniff` blocks MIME confusion on error payloads; `X-Frame-Options: DENY` and a `frame-ancestors` CSP directive stop clickjacking of any HTML you serve (Swagger UI included — give docs a real CSP: `default-src 'self'` rather than disabling CSP for the docs route); `Cache-Control: no-store` keeps authenticated responses out of shared caches and back buttons; `Strict-Transport-Security` once TLS is universal. The hardened app sets these in one middleware (Listing 17.3.4).

CORS rigor. CORS is a *browser-enforced relaxation* mechanism, not server security — but misconfigured, it deputizes every browser to attack you. Exact origin lists only; `*` never combines with credentials (browsers refuse, and Starlette will not emit the pair — but “reflect the Origin header” hand-rolled middlewares reintroduce it silently); keep `allow_methods` and `allow_headers` minimal; and remember CORS does nothing at all for non-browser clients, so it is never a substitute for authentication.

CSRF for cookie-based APIs. If your API authenticates with

cookies (first-party SPAs), browsers attach them cross-site automatically: `SameSite=Lax` (or `Strict`) is the baseline; defense in depth adds a synchronizer token header or the double-submit pattern for state-changing methods. Bearer-token APIs in `Authorization` headers are CSRF-immune by construction — the browser will not attach them cross-origin — one reason tokens displaced cookies for API auth.

17.2.19 Secrets Management

Secrets are radioactive inventory: their half-life is the time to the next leak. The operational pattern: a *vault* (HashiCorp Vault, cloud KMS, or a secrets manager) holds the secret; workloads receive short-lived, scoped credentials at runtime via their platform identity (SPIFFE ID, IAM role); *rotation* is a scheduled non-event, not an incident ritual — dual-credential windows so old and new both verify during rollout. **No-secrets-in-code** is a policy with teeth only when enforced mechanically: `gitleaks`-style scanning in pre-commit *and* CI (history included — a deleted commit still contains the key), with detected secrets treated as compromised and rotated, never merely removed. Log hygiene is part of the same discipline: redact `Authorization` headers and token-shaped strings at the logging boundary; Chapter 5’s client-side rules (Chapter 6) apply verbatim on the server. And version-pin your security dependencies — an unpatched JWT library is an unpatched authentication layer (Section 17.4).

17.3 Production-Ready Code Implementation

All listings live under `code\ch16\`, are Python 3.11+, fully typed, and run offline: `fake_network.py` replaces the internet, `jwtks_fixture.py` replaces the identity provider. Run everything with:

```
1 cd code\ch16
2 python -m pip install -r requirements.txt
3 python -m pytest -q                      # 44 tests, all
   green
4 python -m pytest test_exploit_suite.py -q --target=
   vulnerable    # 10 REDS
5 python -m pytest test_exploit_suite.py -q --target=
   hardened      # 10 GREENS
```

Warning

Listing 17.3.3 is deliberately insecure teaching material. It exists to be exploited by Listing 17.3.10 and then retired. Never deploy it, never copy patterns out of it, and never let it listen on a real network interface. Every flaw is fenced with a comment naming its OWASP item.

17.3.1 Listing 16.1 — Simulated Network (fake_network.py)

Every SSRF and unsafe-consumption exploit needs an internet; the lab supplies a deterministic one. The host table models the three destinations that matter: an internal service, the cloud metadata endpoint, and an allowed partner — plus one redirect that turns an allowed host into a metadata fetch.

```

1  """Simulated_network_for_offline_SSRF/API10_teaching.
2
3  In_production, 'httpx' sockets reach the real
   internet. In this workbook
4  every outbound request is routed through a
   deterministic in-process host
5  table so the exploit suite runs with zero network
   access. The table models
6  the hosts that matter for the security story: an
   internal-only inventory
7  service, the cloud instance-metadata endpoint, and an
   allowed partner API.
8  """
9
10 from __future__ import annotations
11
12 import httpx
13
14 # Hosts that exist inside the simulated world. RFC 1918
   / link-local /
15 # loopback addresses stand in for Northwind Robotics'
   private network.
16 HOST_TABLE: dict[str, tuple[int, dict[str, str], str]]
   = {
17     "inventory.internal.nwr": (
18         200,
19         {"content-type": "application/json"},
20         '{"service": "inventory", "db_dsn": "postgres
           ://svc:FAKE_PASS_invalid@db.internal:5432/
           inv"}',

```

```

21     ),
22     "169.254.169.254": (
23         200,
24         {"content-type": "application/json"},
25         '{"iam": {"role": "nwr-fleet-worker", "AccessKeyId": "AKIAFAKE_INVALID", '
26         '"SecretAccessKey": "sk_test_DEMO_9f8e7d6c5b4a3210_invalid"}}',
27     ),
28     "telemetry.partner-nwr.example": (
29         200,
30         {"content-type": "application/json"},
31         '{"firmware": "4.2.1", "channels": ["stable"]}',
32     ),
33     "untrusted.maps.example": (
34         200,
35         {"content-type": "application/json"},
36         '{"label": "Dock7", "telemetry_channel": "beta", "maintenance_mode": true}',
37     ),
38 }
39
40 # One simulated redirect: the "safe-looking" shortener
41 # bounces to metadata.
42 REDIRECTS: dict[str, str] = {
43     "short.partner-nwr.example": "http
44     ://169.254.169.254/latest/meta-data",
45 }
46
47 def simulated_dns(host: str) -> list[str]:
48     """Deterministic stand-in for DNS resolution."""
49     table = {
50         "inventory.internal.nwr": ["10.20.0.14"],
51         "169.254.169.254": ["169.254.169.254"],
52         "telemetry.partner-nwr.example": ["203.0.113.10"],
53         "untrusted.maps.example": ["198.51.100.77"],
54         "short.partner-nwr.example": ["203.0.113.99"],
55     }
56     return table.get(host, ["192.0.2.1"]) # TEST-NET-1
57     for anything unknown
58
59 def mock_transport(request: httpx.Request) -> httpx.

```

```

Response:
59     """httpx_transport_handler_implementing_the_
        simulated_network."""
60     host = request.url.host or ""
61     if host in REDIRECTS:
62         return httpx.Response(302, headers={"location":
            REDIRECTS[host]})
63     if host in HOST_TABLE:
64         status, headers, body = HOST_TABLE[host]
65         return httpx.Response(status, headers=headers,
            text=body)
66     return httpx.Response(404, text="host_not_found_in_
        simulated_network")
67
68
69 def make_client(**kwargs: object) -> httpx.Client:
70     """Build_an_httpx_client_bound_to_the_simulated_
        network."""
71     return httpx.Client(transport=httpx.MockTransport(
        mock_transport), **kwargs) # type: ignore[arg-
        type]

```

17.3.2 Listing 16.2 — Local JWKS Fixture (jwks_fixture.py)

The identity provider, offline. A synthetic RSA keypair (generated for this workbook, signing only test tokens, protecting nothing) backs a JWKS-style kid → key map, and `mint_token` issues signed JWTs with injectable claims so tests can forge issuers, audiences, key ids, and expired lifetimes on demand.

```

1  """Local_JWKS-style_fixture:_synthetic_RSA_keys+_token
        _minting_for_tests.
2
3  The_key_below_is_a_SYNTHETIC_teaching_fixture_generated
        _for_this_workbook.
4  It_protects_nothing,_signs_only_test_tokens,_and_must_
        never_be_deployed.
5  """
6
7  from __future__ import annotations
8
9  import time
10 from typing import Any
11
12 import jwt

```

```

13
14 ISSUER = "https://id.northwind-robotics.example"
15 AUDIENCE = "nwr-fleet-api"
16 KEY_ID = "nwr-demo-2026-01"
17
18 SYNTHETIC_PRIVATE_PEM = "-----BEGIN PRIVATE KEY-----
19 MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAcwggSjAgEAAoIBAQC+/+
    W8y3oqhIdS
20 2
    pI3BHb1vJwcD1T3StnITyGpBxlZGh39711Ql5g1ie0FMbTVEad1kLHDZFAAVnFa
21 1WrWyEFxYLRzlZP9+zSan30hj4BRwsvIfWG08i9f4SaUeWk+FE+
    JhsxETpZwPVw9
22 +++i0QfEIQK5Z0XY39k01P79ewIo+TNJV6j3q0ar4sDvwVgu9K1z+
    LL2JHZ6PKXI
23 HQ4iN9wdAMbDy0AzgED1z+kqX8UV2o95chIh1HbH0+/XsNKPITNri/
    p8pSgtuHeh
24 jGChkb1xi82MEC1SpnHwDuBqMZmzslMKbsplHjSPZBRwbXZRNPB+
    FunH0gJs4H4C
25 LI1FwyqpAgMBAAECggEABmhTjb+d7I+YYKUSMnM5Qf1R+
    WQmhVokZLazNfcjarGh
26 7Q4qKoq3H1WpwSesUjR5Y10JTG6rzA+
    f0n4oHyha5PahEeShLrhgZhtCfPDVmlDy
27 SX2ivbTY6IRvHQvzpzJwy5eVfcApVXV/2
    q3GgTHztNoPZwZC2NH7HlyzUehVzLQF
28 KqFV+/3waevnx+GTEkCik04fBHHQ6sv4Mgp0huIJTF+
    u6xb1uAC0zoo8bimVVd9/
29 rKOK5ZbTZpKmpBgCv6NCzTDYeAv+vwpEz5qtSluiyUsUd3z6SLwL+
    nyCJDMdc84G
30 anXuqCLxKgKPVTeM4FYdh0W0RR/9kDvL/1KE7AEazwKBgQDPe9h+
    QAFtnPsnWyW9
31 OSM2eRUvOAKkVDLY9FYnklPeNadNywM0y/sf9MwbNfNoIb5lym/
    escXL0Byr4fMj
32 XCLvFgxQd9YLpsU8JGTZG5W0qaPND5IL9zF85k1t9y7yP/
    HQrF6KLMkymh2j12gg
33 IvCNLIQlYCRGFu46VoJ3DwtJwKBgQDH4VSjx7gpKmTfTxy4lq0n4k/
    oa8zJWQ78
34 WIq3gEfJUsjks0WXqZSvkgCX+5
    xBWs2LwRfPJr1X0t37HyVAwWLSxbl0pPNPhCq7
35 tQlrE8s/9JuirMZg67Haohv65bXN9av6fiih/
    BiBJeiJFEA0TRsvKQa0yF6qyKs3
36 oGuyQDIrrwKBgQCyvMSWlfrk+6
    vcjoenR7a08aYPRFf6SlJ3VZ12f3biYSQcPvwn
37 GmXedJr0AJKtjQwhU1Am7swvNLv0U1qLJo8tmbfIBkQNS4BzvAJoiXvAJ2F1gLLW
38 t38ZUqh3R85Ygd+HfUYAEG70ubc48pLPxGmnpCa+r+

```

```

    DvxEc7WFURzZMRVwKBgH4H
39 w2Gdra4vL/
    lqHbb6MuZCGZZ4WmDeycctYRIBTcJQc6FXNP0jDUB5BZePK+A9i/
    tB
40 3mxch eh5krwj0E57YY/fwE8pTM1njbZbmTut+Gs0Jeo1vMQh+
    TvdGX1i1/asoCrK
41 3337wcu1BmFgpncT3yXu3W6iJKbU7ridayqyta+7AoGAMfYURmN+
    K4jEoZbzA/Ap
42 1VRy9JmQUzE/
    UtA8QcK1VvZzn6ESUNe1G9gN6cR8HNVRZi1qaSCvgZzft3e7eQ2A
43 zfFTHQzFM9qTLeUlvwLFOIMMxuUBym4xY01vjJCiS4h/
    IkuKqxGbgSzUL270rGY
44 8KUnb5+M+Z5JoWmaVNItxcw=
45 -----END_PRIVATE_KEY-----"""
46
47 SYNTHETIC_PUBLIC_PEM = """-----BEGIN_PUBLIC_KEY-----
48 MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAof/
    lvMt6KoSHUtqSNwR2
49 9bycHA9U90rZyE8hqQcZWRod/
    e9dUJeYNYntBTG01RGndZCw2RQAFZxWpVq1shB
50 cWC0c5WT/fs0mp99IY+AUcLLyH1hjvIvX+
    EmlHlpPhRPiYbMRE6WcD1cPfvvotEH
51 xCECuWdF2N/ZDpT+/XsCKPkzSVeo96tGq+LA78FYLvStc/
    iy9iR2ejylyB00Ijfc
52 HQDGw8tAM4BA5c/pK1/
    FFdqPeXISIdR2x9Pv17DSjyEza4v6fKUoLbh3oYxgoZG9
53 cYvNjBAUqZx8A7gajGZs7JTCm7KZR40j2QUcG12UTTwhbpxzoCb0B
    +AiyNRcMq
54 qQIDAQAB
55 -----END_PUBLIC_KEY-----"""
56
57 # JWKS-style resolution table: kid -> verification
    material. A real service
58 # loads this from the IdP's JWKS endpoint and refreshes
    on unknown kid.
59 JWKS: dict[str, str] = {KEY_ID: SYNTHETIC_PUBLIC_PEM}
60
61
62 def mint_token(
63     subject: str,
64     scopes: list[str],
65     *,
66     audience: str = AUDIENCE,
67     issuer: str = ISSUER,
68     ttl_seconds: int = 300,
69     kid: str = KEY_ID,

```

```

70     tenant: str = "tenant-nwr",
71     now: int | None = None,
72     algorithm: str = "RS256",
73     extra: dict[str, Any] | None = None,
74 ) -> str:
75     """Mint a signed JWT for tests. 'now' is injectable for determinism."""
76     issued = int(time.time()) if now is None else now
77     claims: dict[str, Any] = {
78         "iss": issuer,
79         "aud": audience,
80         "sub": subject,
81         "scope": " ".join(scopes),
82         "tenant": tenant,
83         "iat": issued,
84         "nbf": issued - 1,
85         "exp": issued + ttl_seconds,
86     }
87     if extra:
88         claims.update(extra)
89     return jwt.encode(
90         claims,
91         SYNTHETIC_PRIVATE_PEM,
92         algorithm=algorithm,
93         headers={"kid": kid},
94     )

```

17.3.3 Listing 16.3 — The Deliberately Vulnerable API (vulnerable_app.py)

The centerpiece's dark half: every OWASP API Security Top 10 flaw seeded in one service, each fenced by a comment naming its item. Read it as an incident report written in advance — the flaws are one or two lines each, which is exactly why they ship so often.

```

1  """
   =====
2  INTENTIONALLY_INSECURE_TEACHING_MATERIAL--DO_NOT_
   DEPLOY.DO_NOT_IMITATE.
3  =====
4  Northwind_Robotics_fleet_API, seeded with all ten OWASP
   API_Security_Top_10

```



```

5  (2023) flaws. Every flaw is fenced with a comment
   naming its OWASP item.
6  The hardened counterpart is hardened_app.py; the
   exploit suite is
7  test_exploit_suite.py. All network I/O goes through
   fake_network.py, so the
8  whole file runs offline.
9  """
10
11 from __future__ import annotations
12
13 import base64
14 import traceback
15 from typing import Any
16
17 from fastapi import Depends, FastAPI, Request
18 from fastapi.middleware.cors import CORSMiddleware
19 from fastapi.responses import JSONResponse
20
21 from fake_network import make_client
22
23 # A synthetic "master key" used by the broken
   authenticator below.
24 # OWASP API2 (hardcoded credential). Clearly fake;
   never a real secret.
25 MASTER_KEY = "sk_test_DEMO_masterkey_invalid"
26
27 USERS: dict[str, dict[str, Any]] = {
28     "alice": {"username": "alice", "role": "operator",
29              "tenant": "tenant-nwr"},
29     "bob": {"username": "bob", "role": "operator", "
30            tenant": "tenant-nwr"},
30     "carol": {"username": "carol", "role": "admin", "
31              tenant": "tenant-nwr"},
31 }
32
33 ROBOTS: dict[int, dict[str, Any]] = {
34     1001: {
35         "id": 1001, "name": "Welder-7", "model": "NR
36             -220", "status": "idle",
37         "battery_pct": 88, "last_seen": "2026-08-30T09
38             :14:00Z",
39         "notes": "dock", "owner_id": "alice", "
40             tenant_id": "tenant-nwr",
41         "firmware_version": "4.2.0", "service_key": "
42             sk_test_DEMO_robot1001_invalid",

```

```

39         "service_key_last4": "1001", "maintenance_mode"
40             : False,
41         "telemetry_channel": "stable",
42     },
43     1002: {
44         "id": 1002, "name": "Lifter-2", "model": "NR
45             -410", "status": "active",
46         "battery_pct": 61, "last_seen": "2026-08-30T09
47             :15:00Z",
48         "notes": "bay_9", "owner_id": "bob", "tenant_id
49             ": "tenant-nwr",
50         "firmware_version": "4.1.3", "service_key": "
51             sk_test_DEMO_robot1002_invalid",
52         "service_key_last4": "1002", "maintenance_mode"
53             : False,
54         "telemetry_channel": "stable",
55     },
56 }
57
58 PROMO_BALANCE = {"NWR-LAUNCH": 100} # code ->
59     remaining units
60
61 # -----
62 # OWASP API2:2023 -- Broken Authentication
63 # "Tokens" are unsigned base64 of "user:<name>"; anyone
64 # can mint one for any
65 # user. A hardcoded master key bypasses everything. No
66 # expiry, no signature.
67 # -----
68 def vulnerable_auth(request: Request) -> dict[str, Any
69     ]:
70     header = request.headers.get("authorization", "")
71     token = header.removeprefix("Bearer_").strip()
72     if token == MASTER_KEY: # hardcoded backdoor
73         credential
74         return {"username": "master", "role": "admin",
75             "tenant": "tenant-nwr"}
76     try:
77         decoded = base64.b64decode(token).decode()
78         username = decoded.split(":", 1)[1]
79         return USERS[username]
80     except Exception:
81         # Fail-open flavoured: anonymous requests still
82         # get a context object,
83         # and several endpoints below forget to check

```

```

        it at all.
72         return {"username": "anonymous", "role": "none"
73                 , "tenant": "tenant-nwr"}
74
75 def forge_token(username: str) -> str:
76     """Helper used by the exploit suite: mint a 'valid'
77     token for anyone."""
78     return base64.b64encode(f"user:{username}".encode()
79                             ).decode()
79
80 def create_app() -> FastAPI:
81     app = FastAPI(title="NWR Fleet API (VULNERABLE_
82                   TEACHING_BUILD)")
83
84     # -----
85     # OWASP API8:2023 -- Security Misconfiguration
86     # Wildcard CORS with credentials; stack traces
87     # returned to clients;
88     # a debug endpoint that dumps configuration
89     # including secrets.
90     # -----
91     app.add_middleware(
92         CORSMiddleware,
93         allow_origins=["*"],
94         allow_credentials=True,
95         allow_methods=["*"],
96         allow_headers=["*"],
97     )
98
99     @app.exception_handler(Exception)
100     async def verbose_errors(request: Request, exc:
101                               Exception) -> JSONResponse:
102         # Leaks internals (paths, library versions) to
103         # any caller.
104         return JSONResponse(
105             status_code=500,
106             content={"error": str(exc), "trace":
107                     traceback.format_exc()},
108         )
109
110     @app.get("/v1/debug/config")
111     def debug_config() -> dict[str, Any]:
112         return {
113             "env": "prod",

```

```

108         "master_key": MASTER_KEY,
109         "db_dsn": "postgres://svc:
            FAKE_PASS_invalid@db.internal:5432/fleet
            ",
110         "flags": {"debug": True, "stack_traces":
            True},
111     }
112
113     # -----
114     # OWASP API1:2023 -- Broken Object Level
        Authorization
115     # OWASP API3:2023 -- Excessive Data Exposure (
        service_key, owner fields,
116     # tenant internals all serialized straight back).
117     # Sequential integer ids make enumeration trivial:
        1001, 1002, ...
118     # -----
119     @app.get("/v1/robots/{robot_id}")
120     def get_robot(
121         robot_id: int, user: dict[str, Any] = Depends(
            vulnerable_auth)
122     ) -> dict[str, Any]:
123         # user is fetched but never checked against the
            object. Anyone
124         # authenticated (or 'anonymous') reads anyone's
            robot, secrets included.
125         return dict(ROBOTS[robot_id])
126
127     # -----
128     # OWASP API3:2023 -- Broken Object Property Level
        Authorization
129     # (mass assignment): the request body is merged
        wholesale into the record,
130     # so clients can set owner_id, service_key,
        maintenance_mode, anything.
131     # -----
132     @app.patch("/v1/robots/{robot_id}")
133     def patch_robot(
134         robot_id: int,
135         patch: dict[str, Any],
136         user: dict[str, Any] = Depends(vulnerable_auth)
137     ) -> dict[str, Any]:
138         record = ROBOTS[robot_id]
139         record.update(patch) # <-- the entire flaw in
            one line

```

```

140         return dict(record)
141
142     # -----
143     # OWASP API4:2023 -- Unrestricted Resource
144     # Consumption
145     # No pagination ceiling; unbounded simulation
146     # parameter; no body limits.
147     # -----
148     @app.get("/v1/robots")
149     def list_robots(limit: int = 10) -> list[dict[str,
150         Any]]:
151         # limit=1000000 is accepted happily; there is
152         # no cap.
153         return [dict(r) for r in list(ROBOTS.values())
154             [:limit]]
155
156     @app.post("/v1/fleet/simulate")
157     def simulate(steps: int = 100) -> dict[str, Any]:
158         # The 2025 incident ran steps=40_000_000 per
159         # request until the pool
160         # starved. Loop truncated in the lab so the
161         # test suite stays fast;
162         # the flaw is that 'steps' is accepted and
163         # trusted at all.
164         work = 0
165         for _ in range(min(steps, 20_000)):
166             work += 1
167         return {"requested_steps": steps, "
168             executed_steps": work}
169
170     # -----
171     # OWASP API5:2023 -- Broken Function Level
172     # Authorization
173     # The admin surface checks authentication, never
174     # the role.
175     # -----
176     @app.get("/v1/admin/users")
177     def admin_users(user: dict[str, Any] = Depends(
178         vulnerable_auth)) -> Any:
179         return list USERS.values() # any
180             authenticated (or anon) caller
181
182     # -----
183     # OWASP API6:2023 -- Unrestricted Access to
184     # Sensitive Business Flows
185     # Promo redemption has no rate limit, no CAPTCHA,

```

```

no velocity checks:
172 # a script drains the campaign budget in seconds.
173 # -----
174 @app.post("/v1/promo/redeem")
175 def redeem_promo(
176     body: dict[str, Any], user: dict[str, Any] =
        Depends(vulnerable_auth)
177 ) -> dict[str, Any]:
178     code = str(body.get("code", ""))
179     if PROMO_BALANCE.get(code, 0) <= 0:
180         return JSONResponse(status_code=409,
            content={"error": "exhausted"}) # type:
            ignore[return-value]
181     PROMO_BALANCE[code] -= 1
182     return {"redeemed": code, "remaining":
        PROMO_BALANCE[code]}
183
184 # -----
185 # OWASP API7:2023 -- Server-Side Request Forgery
186 # The "diagnostics" feature fetches any caller-
        supplied URL server-side:
187 # internal hosts, the cloud metadata endpoint,
        anything.
188 # -----
189 @app.post("/v1/diagnostics/fetch")
190 def diagnostics_fetch(
191     body: dict[str, Any], user: dict[str, Any] =
        Depends(vulnerable_auth)
192 ) -> dict[str, Any]:
193     url = str(body.get("url", ""))
194     with make_client() as client: # simulated
        network; real one in prod
195         response = client.get(url, follow_redirects
            =True)
196     return {"url": url, "status": response.
        status_code, "body": response.text}
197
198 # -----
199 # OWASP API9:2023 -- Improper Inventory Management
200 # Legacy v0, shipped "temporarily" in 2023, still
        live, no auth at all.
201 # An internal export route that never made it into
        the OpenAPI spec.
202 # -----
203 @app.get("/v0/robots")
204 def legacy_robots() -> list[dict[str, Any]]:

```

```

205         return [dict(r) for r in ROBOTS.values()] #
           unauthenticated
206
207     @app.get("/internal/export")
208     def internal_export() -> dict[str, Any]:
209         return {"users": list(USERS.values()), "robots"
           : list(ROBOTS.values())}
210
211     # -----
212     # OWASP API10:2023 -- Unsafe Consumption of APIs
213     # The enrichment job trusts the upstream partner
           completely: every field
214     # it returns is merged into our record -- including
           maintenance_mode.
215     # -----
216     @app.post("/v1/fleet/enrich/{robot_id}")
217     def enrich_robot(
218         robot_id: int, user: dict[str, Any] = Depends(
           vulnerable_auth)
219     ) -> dict[str, Any]:
220         with make_client() as client:
221             upstream = client.get("http://untrusted.
           maps.example/site")
222             payload = upstream.json() # no schema, no
           validation, total trust
223             ROBOTS[robot_id].update(payload)
224             return dict(ROBOTS[robot_id])
225
226     return app
227
228
229 app = create_app()

```

17.3.4 Listing 16.4 — The Hardened API (hardened_app.py)

The repaired service. Every fix names the item it closes: scope + object + property authorization on robot routes (API1/API3), real RS256 authentication (API2), schema ceilings (API4), role-checked admin surface (API5), per-principal velocity limits on the promo flow (API6), the SSRF gauntlet on outbound fetches (API7), no debug surface and secure headers (API8), no legacy or shadow routes (API9), and schema-validated upstream consumption (API10).

```

1  """Hardened_Northwind_Robotics_fleet_API--the_exploit
   suite's_green_target.

```

```

2
3 Every endpoint is the repaired counterpart of
4   vulnerable_app.py. Fixes are
5   commented with the OWASP item they close. Runs fully
6   offline: tokens come
7   from jwks_fixture.py, outbound I/O from fake_network.py
8   .
9   """
10
11
12 from __future__ import annotations
13
14 from typing import Any, Literal
15
16 from fastapi import Depends, FastAPI, HTTPException,
17     Query, Request
18 from fastapi.middleware.cors import CORSMiddleware
19 from fastapi.responses import JSONResponse
20 from pydantic import BaseModel, Field
21
22 from authz import (
23     ROBOT_READ_POLICY,
24     ROBOT_WRITE_POLICY,
25     ForbiddenError,
26     Guard,
27     apply_patch,
28     filter_fields,
29 )
30
31 from fake_network import make_client, simulated_dns
32 from jwks_fixture import AUDIENCE, ISSUER, JWKS
33 from ssrf_fetcher import OutboundPolicy, SafeFetcher,
34     SsrfBlockedError
35 from token_validation import Principal, TokenValidator,
36     build_auth_dependency
37
38 # -----
39 # Data (same fictional universe as the vulnerable build
40 # ; secrets never leave
41 # the store layer now).
42 # -----
43
44 USERS: dict[str, dict[str, Any]] = {
45     "alice": {"username": "alice", "role": "operator",
46             "tenant": "tenant-nwr"},
47     "bob": {"username": "bob", "role": "operator",
48            "tenant": "tenant-nwr"},
49     "carol": {"username": "carol", "role": "admin",
50             "tenant": "tenant-nwr"},

```



```

38 }
39
40 ROBOTS: dict[int, dict[str, Any]] = {
41     1001: {
42         "id": 1001, "name": "Welder-7", "model": "NR
43             -220", "status": "idle",
44         "battery_pct": 88, "last_seen": "2026-08-30T09
45             :14:00Z",
46         "notes": "dock_3", "owner_id": "alice", "
47             tenant_id": "tenant-nwr",
48         "firmware_version": "4.2.0", "service_key": "
49             sk_test_DEMO_robot1001_invalid",
50         "service_key_last4": "1001", "maintenance_mode"
51             : False,
52         "telemetry_channel": "stable",
53     },
54     1002: {
55         "id": 1002, "name": "Lifter-2", "model": "NR
56             -410", "status": "active",
57         "battery_pct": 61, "last_seen": "2026-08-30T09
58             :15:00Z",
59         "notes": "bay_9", "owner_id": "bob", "tenant_id
60             ": "tenant-nwr",
61         "firmware_version": "4.1.3", "service_key": "
62             sk_test_DEMO_robot1002_invalid",
63         "service_key_last4": "1002", "maintenance_mode"
64             : False,
65         "telemetry_channel": "stable",
66     },
67 }
68
69 PROMO_BALANCE = {"NWR-LAUNCH": 100}
70 PROMO_LIMIT_PER_PRINCIPAL = 3 # lab stand-in for the
71     Chapter 14 token bucket
72
73 OUTBOUND_POLICY = OutboundPolicy(
74     allowed_hosts=frozenset({"telemetry.partner-nwr.
75         example"})
76 )
77
78 class PatchRobotRequest(BaseModel):
79     """API3 fix(write_side): the contract is an
80         explicit schema, and the
81         authz layer re-checks every field against the
82         caller's role."""

```

```

70
71     model_config = {"extra": "forbid"}
72     name: str | None = Field(default=None, max_length
73                               =80)
74     notes: str | None = Field(default=None, max_length
75                               =500)
76     status: Literal["idle", "active", "charging"] |
77             None = None
78
79 class FetchRequest(BaseModel):
80     model_config = {"extra": "forbid"}
81     url: str = Field(max_length=2048)
82
83 class PromoRequest(BaseModel):
84     model_config = {"extra": "forbid"}
85     code: str = Field(max_length=32)
86
87 class UpstreamSiteInfo(BaseModel):
88     """API10 fix: upstream payloads are parsed against
89       a schema with
90       extra fields ignored; only vetted fields ever touch
91       our records."""
92
93     model_config = {"extra": "ignore"}
94     label: str = Field(max_length=120)
95     telemetry_channel: Literal["stable", "beta"] = "
96       stable"
97
98 def create_app() -> FastAPI:
99     app = FastAPI(title="NWR Fleet API (hardened)")
100     validator = TokenValidator(JWKS, issuer=ISSUER,
101                               audience=AUDIENCE)
102     principal_dep = build_auth_dependency(validator)
103     redeem_counters: dict[str, int] = {}
104
105     # API8 fix: exact origins, no credentials; never
106       the wildcard pair.
107     app.add_middleware(
108         CORSMiddleware,
109         allow_origins=["https://portal.northwind-
110                       robotics.example"],
111         allow_credentials=False,

```

```

107     allow_methods=["GET", "POST", "PATCH"],
108     allow_headers=["Authorization", "Content-Type"
109                   ],
110 )
111
112 @app.middleware("http")
113 async def security_headers(request: Request,
114                             call_next: Any) -> Any:
115     # API8 fix: secure defaults on every response;
116     # no stack traces.
117     response = await call_next(request)
118     response.headers.setdefault("X-Content-Type-Options", "nosniff")
119     response.headers.setdefault("X-Frame-Options", "DENY")
120     response.headers.setdefault(
121         "Content-Security-Policy", "default-src 'none'; frame-ancestors 'none'"
122     )
123     response.headers.setdefault("Cache-Control", "no-store")
124     return response
125
126 @app.exception_handler(ForbiddenError)
127 async def forbidden(request: Request, exc:
128                     ForbiddenError) -> JSONResponse:
129     return JSONResponse(status_code=403, content={"detail": "forbidden"})
130
131 @app.exception_handler(SsrfBlockedError)
132 async def ssrf_blocked(request: Request, exc:
133                       SsrfBlockedError) -> JSONResponse:
134     return JSONResponse(status_code=403, content={"detail": "destination_blocked"})
135
136 @app.exception_handler(Exception)
137 async def opaque_errors(request: Request, exc:
138                       Exception) -> JSONResponse:
139     return JSONResponse(status_code=500, content={"detail": "internal_error"})
140
141 def guard_for(principal: Principal) -> Guard:
142     return Guard(
143         actor_id=principal.subject,
144         tenant_id=principal.tenant,
145         roles=principal.roles,

```

```

140         )
141
142     def view_roles(principal: Principal) -> list[str]:
143         # Owners authenticate with a fleet scope and no
144         # role claim; they get
145         # the owner field policy. Named roles widen the
146         # view per policy.
147         return sorted(principal.roles) if principal.
148             roles else ["owner"]
149
150     # API1 fix: fetch, then authorize the *object*;
151     # API3 fix (read side):
152     # serialize through the role's field policy --
153     # never the raw record.
154     @app.get("/v1/robots/{robot_id}")
155     def get_robot(
156         robot_id: int, principal: Principal = Depends(
157             principal_dep)
158     ) -> dict[str, Any]:
159         principal.require_scope("fleet:read")
160         record = ROBOTS.get(robot_id)
161         if record is None:
162             raise HTTPException(status_code=404, detail
163                                 ="not_found")
164         guard_for(principal).require_read(record)
165         return filter_fields(record, ROBOT_READ_POLICY,
166                             view_roles(principal))
167
168     @app.patch("/v1/robots/{robot_id}")
169     def patch_robot(
170         robot_id: int,
171         patch: PatchRobotRequest,
172         principal: Principal = Depends(principal_dep),
173     ) -> dict[str, Any]:
174         principal.require_scope("fleet:write")
175         record = ROBOTS.get(robot_id)
176         if record is None:
177             raise HTTPException(status_code=404, detail
178                                 ="not_found")
179         guard_for(principal).require_write(record)
180         body = patch.model_dump(exclude_none=True)
181         updated = apply_patch(record, body,
182                               ROBOT_WRITE_POLICY, principal.roles)
183         ROBOTS[robot_id] = updated
184         return filter_fields(updated, ROBOT_READ_POLICY,
185                             view_roles(principal))

```

```

175
176     # API4 fix: hard ceilings at the schema layer;
177     # anything bigger is a 422
178     # before a single byte of work happens.
179     @app.get("/v1/robots")
180     def list_robots(
181         limit: int = Query(default=10, ge=1, le=100),
182         principal: Principal = Depends(principal_dep),
183     ) -> list[dict[str, Any]]:
184         principal.require_scope("fleet:read")
185         mine = [
186             r for r in ROBOTS.values() if guard_for(
187                 principal).can_read(r)
188         ]
189         return [
190             filter_fields(r, ROBOT_READ_POLICY,
191                 view_roles(principal))
192             for r in mine[:limit]
193         ]
194
195     @app.post("/v1/fleet/simulate")
196     def simulate(
197         steps: int = Query(default=100, ge=1, le=10_000),
198         principal: Principal = Depends(principal_dep),
199     ) -> dict[str, Any]:
200         principal.require_scope("fleet:command")
201         work = 0
202         for _ in range(steps):
203             work += 1
204         return {"executed_steps": work, "cap": 10_000}
205
206     # API5 fix: the admin surface requires the admin
207     # role AND scope,
208     # checked in the dependency layer, not by
209     # convention.
210     @app.get("/v1/admin/users")
211     def admin_users(
212         principal: Principal = Depends(principal_dep),
213     ) -> list[dict[str, Any]]:
214         principal.require_scope("admin:users")
215         if "admin" not in principal.roles:
216             raise HTTPException(status_code=403, detail=
217                 "admin_role_required")
218         return [
219             {"username": u["username"], "role": u["role"]

```

```

214         "]} for u in USERS.values()
215     ]
216     # API6 fix: the sensitive flow is velocity-limited
217     # per authenticated
218     # principal before business logic runs.
219     @app.post("/v1/promo/redeem")
220     def redeem_promo(
221         body: PromoRequest, principal: Principal =
222         Depends(principal_dep)
223     ) -> dict[str, Any]:
224         principal.require_scope("promo:redeem")
225         used = redeem_counters.get(principal.subject,
226                                     0)
227         if used >= PROMO_LIMIT_PER_PRINCIPAL:
228             return JSONResponse( # type: ignore[return
229                                   -value]
230                                   status_code=429,
231                                   content={"detail": "rate_limit_exceeded"},
232                                   headers={"Retry-After": "60"},
233                               )
234         if PROMO_BALANCE.get(body.code, 0) <= 0:
235             raise HTTPException(status_code=409, detail=
236                                   "promotion_exhausted")
237         redeem_counters[principal.subject] = used + 1
238         PROMO_BALANCE[body.code] -= 1
239         return {"redeemed": body.code, "remaining":
240                 PROMO_BALANCE[body.code]}
241
242     # API7 fix: outbound fetches pass the full SSRF
243     # gauntlet -- allow-list,
244     # scheme/port checks, DNS pinning, redirect re-
245     # validation.
246     @app.post("/v1/diagnostics/fetch")
247     def diagnostics_fetch(
248         body: FetchRequest, principal: Principal =
249         Depends(principal_dep)
250     ) -> dict[str, Any]:
251         principal.require_scope("diagnostics:run")
252         with make_client() as client:
253             fetcher = SafeFetcher(OUTBOUND_POLICY,
254                                   client, resolve=simulated_dns)
255             text = fetcher.fetch_text(body.url)
256         return {"url": body.url, "body": text}

```

17.3. Production-Ready Code Implementation

```
248 # API8 fix: no /v1/debug/config exists. API9 fix: /  
    v0 and /internal/*  
249 # are gone -- every route is in the OpenAPI spec  
    and inventory.  
  
250  
251 # API10 fix: upstream data is parsed against  
    UpstreamSiteInfo; only the  
252 # vetted telemetry_channel field may flow into our  
    records.  
253 @app.post("/v1/fleet/enrich/{robot_id}")  
254 def enrich_robot(  
255     robot_id: int, principal: Principal = Depends(  
        principal_dep)  
256 ) -> dict[str, Any]:  
257     principal.require_scope("fleet:write")  
258     record = ROBOTS.get(robot_id)  
259     if record is None:  
260         raise HTTPException(status_code=404, detail=  
            ="not_found")  
261     guard_for(principal).require_write(record)  
262     with make_client() as client:  
263         upstream = client.get("http://untrusted.  
            maps.example/site")  
264     try:  
265         info = UpstreamSiteInfo.model_validate(  
            upstream.json())  
266     except ValueError as exc:  
267         raise HTTPException(  
268             status_code=502, detail="upstream_  
                payload_failed_validation"  
269         ) from exc  
270     record["telemetry_channel"] = info.  
        telemetry_channel  
271     return filter_fields(record, ROBOT_READ_POLICY,  
        view_roles(principal))  
  
272  
273 return app  
274  
275  
276 app = create_app()
```

17.3.5 Listing 16.5 — Token Validation Middleware (token_validation.py)

The authentication gate as a reusable component. Note the order — header structure and `alg` allow-list, `kid` resolution, signature and claims verification, *then* mapping to an immutable `Principal` — and that every failure mode raises 401 before a single claim is trusted. Scope failures raise 403 with a WWW-Authenticate hint [Fie22a].

```

1  """Token validation middleware: JWS-style key
   resolution, scope/audience
2  enforcement, and claim-to-principal mapping.
3
4  Validation order is a security invariant: never trust a
   claim before the
5  signature, issuer, audience, and expiry have all been
   verified.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass, field
11 from typing import Any, Mapping
12
13 import jwt
14 from fastapi import Depends, FastAPI, HTTPException,
   Request
15 from fastapi.security import
   HTTPAuthorizationCredentials, HTTPBearer
16
17 ALLOWED_ALGORITHMS = ("RS256",) # allow-list: 'none'
   and HS-confusion rejected
18 CLOCK_LEEWAY_SECONDS = 30
19
20
21 @dataclass(frozen=True)
22 class Principal:
23     """The authenticated caller, mapped from verified
   claims only."""
24
25     subject: str
26     tenant: str
27     scopes: frozenset[str] = field(default_factory=
   frozenset)
28     roles: frozenset[str] = field(default_factory=
   frozenset)

```



```

29
30     def has_scope(self, scope: str) -> bool:
31         return scope in self.scopes
32
33     def require_scope(self, scope: str) -> None:
34         if not self.has_scope(scope):
35             raise HTTPException(
36                 status_code=403,
37                 detail=f"missing required scope '{scope}' ",
38                 headers={"WWW-Authenticate": f'Bearer scope="{scope}"'},
39             )
40
41
42 class TokenValidator:
43     """Verifies Bearer tokens against a JWKS-style key map."""
44
45     def __init__(
46         self,
47         jwks: Mapping[str, str],
48         *,
49         issuer: str,
50         audience: str,
51         leeway: int = CLOCK_LEEWAY_SECONDS,
52     ) -> None:
53         if not jwks:
54             raise ValueError("JWKS key map must not be empty")
55         self._jwks = dict(jwks)
56         self._issuer = issuer
57         self._audience = audience
58         self._leeway = leeway
59
60     def validate(self, token: str) -> Principal:
61         header = self._unverified_header(token)
62         key = self._resolve_key(header)
63         claims = self._verify_claims(token, key)
64         return self._to_principal(claims)
65
66     def _unverified_header(self, token: str) -> dict[
67         str, Any]:
68         try:
69             return jwt.get_unverified_header(token)
70         except jwt.DecodeError as exc:

```

```

70         raise HTTPException(status_code=401, detail
71                               = "malformed_token") from exc
72
73     def _resolve_key(self, header: dict[str, Any]) ->
74         str:
75         alg = header.get("alg")
76         if alg not in ALLOWED_ALGORITHMS:
77             raise HTTPException(status_code=401, detail
78                                   = "disallowed_algorithm")
79         kid = header.get("kid")
80         key = self._jwks.get(kid) if isinstance(kid,
81                                                 str) else None
82         if key is None:
83             # Real deployments: one rate-limited JWKS
84             # refetch, then fail.
85             raise HTTPException(status_code=401, detail
86                                   = "unknown_key_id")
87         return key
88
89     def _verify_claims(self, token: str, key: str) ->
90         dict[str, Any]:
91         try:
92             claims: dict[str, Any] = jwt.decode(
93                 token,
94                 key,
95                 algorithms=list(ALLOWED_ALGORITHMS),
96                 audience=self._audience,
97                 issuer=self._issuer,
98                 leeway=self._leeway,
99                 options={"require": ["exp", "iat", "iss",
100                                     "aud", "sub"]},
101             )
102         except jwt.ExpiredSignatureError as exc:
103             raise HTTPException(status_code=401, detail
104                                   = "token_expired") from exc
105         except jwt.InvalidTokenError as exc:
106             raise HTTPException(status_code=401, detail
107                                   = "invalid_token") from exc
108         return claims
109
110     @staticmethod
111     def _to_principal(claims: dict[str, Any]) ->
112         Principal:
113         scope_claim = claims.get("scope", "")
114         if not isinstance(scope_claim, str):
115             raise HTTPException(status_code=401, detail

```

```

105         ="malformed_scope_claim")
106     roles_claim = claims.get("roles", [])
107     if not isinstance(roles_claim, list):
108         raise HTTPException(status_code=401, detail
109                             ="malformed_roles_claim")
110     tenant = claims.get("tenant")
111     if not isinstance(tenant, str) or not tenant:
112         raise HTTPException(status_code=401, detail
113                             ="missing_tenant_claim")
114     return Principal(
115         subject=str(claims["sub"]),
116         tenant=tenant,
117         scopes=frozenset(s for s in scope_claim.
118                           split() if s),
119         roles=frozenset(str(r) for r in roles_claim
120                           ),
121     )
122
123 def build_auth_dependency(
124     validator: TokenValidator,
125 ) -> Any:
126     """Wire a TokenValidator into FastAPI's dependency
127     system."""
128     bearer = HTTPBearer(auto_error=True)
129
130     def current_principal(
131         credentials: HTTPAuthorizationCredentials =
132             Depends(bearer),
133     ) -> Principal:
134         return validator.validate(credentials.
135                                   credentials)
136
137     return current_principal
138
139 def demo_app(validator: TokenValidator) -> FastAPI:
140     """Minimal app proving scope enforcement end to end
141     ."""
142     app = FastAPI(title="NWR Token Validation Demo")
143     principal_dep = build_auth_dependency(validator)
144
145     @app.get("/v1/whoami")
146     def whoami(principal: Principal = Depends(
147         principal_dep)) -> dict[str, Any]:
148         return {

```

```

141         "sub": principal.subject,
142         "tenant": principal.tenant,
143         "scopes": sorted(principal.scopes),
144     }
145
146     @app.get("/v1/fleet/commands")
147     def fleet_commands(
148         request: Request,
149         principal: Principal = Depends(principal_dep),
150     ) -> dict[str, Any]:
151         principal.require_scope("fleet:command")
152         return {"accepted": True, "by": principal.subject}
153
154     return app

```

17.3.6 Listing 16.6 — Authorization Guards (authz.py)

The authorization gate as pure functions over mappings: Guard answers “may this actor touch *this* object” (API1); filter_fields and apply_patch answer “which *properties* may this role see or change” (API3). Because the guards are framework-free, they are unit-testable without HTTP and reusable across REST, GraphQL (Chapter 13), and gRPC (Chapter 14) surfaces.

```

1  """Authorization guards: object-level (BOLA defense)
   and property-level
2  (BOPLA defense) checks, independent of any web
   framework.
3
4  Rule of thumb: authentication answers "who are you";
   these guards answer
5  "may you touch *this* object" and "may you see/write *
   these* fields".
6  Records are plain mappings, so the guards work with ORM
   rows, dict stores,
7  or deserialized payloads alike.
8  """
9
10 from __future__ import annotations
11
12 from dataclasses import dataclass, field
13 from typing import Any, Callable, Iterable, Mapping
14
15

```

```

16 class ForbiddenError(PermissionError):
17     """Raised when a principal may not act on a
        resource."""
18
19
20 def _owner(resource: Mapping[str, Any]) -> str:
21     try:
22         return str(resource["owner_id"])
23     except KeyError as exc:
24         raise ForbiddenError("resource has no ownership
        metadata") from exc
25
26
27 def _tenant(resource: Mapping[str, Any]) -> str:
28     try:
29         return str(resource["tenant_id"])
30     except KeyError as exc:
31         raise ForbiddenError("resource has no tenant
        metadata") from exc
32
33
34 @dataclass(frozen=True)
35 class Guard:
36     """Object-level authorization: every access names
        its subject."""
37
38     actor_id: str
39     tenant_id: str
40     roles: frozenset[str] = field(default_factory=
        frozenset)
41
42     def can_read(self, resource: Mapping[str, Any]) ->
        bool:
43         if "admin" in self.roles:
44             return _tenant(resource) == self.tenant_id
45         return (
46             _tenant(resource) == self.tenant_id
47             and _owner(resource) == self.actor_id
48         )
49
50     def can_write(self, resource: Mapping[str, Any]) ->
        bool:
51         # Writes are owner-only; even admins must use
        the audited admin API.
52         return (
53             _tenant(resource) == self.tenant_id

```

```

54         and _owner(resource) == self.actor_id
55     )
56
57     def require_read(self, resource: Mapping[str, Any])
58     -> None:
59         if not self.can_read(resource):
60             raise ForbiddenError("read_denied: not your
61             _object")
62
63     def require_write(self, resource: Mapping[str, Any
64     ]) -> None:
65         if not self.can_write(resource):
66             raise ForbiddenError("write_denied: not
67             your_object")
68
69 # Property-level policy: role -> fields the role may
70 see or change.
71 # Absent roles get nothing; server-owned fields appear
72 in NO write policy.
73 FieldPolicy = Mapping[str, frozenset[str]]
74
75 ROBOT_READ_POLICY: FieldPolicy = {
76     "owner": frozenset(
77         {"id", "name", "model", "status", "battery_pct",
78          "last_seen", "notes"}
79     ),
80     "technician": frozenset({"id", "name", "model", "
81     status", "battery_pct"}),
82     "auditor": frozenset({"id", "name", "status", "
83     firmware_version"}),
84     "admin": frozenset(
85         {
86             "id", "name", "model", "status", "
87             battery_pct", "last_seen",
88             "notes", "firmware_version", "
89             service_key_last4",
90         }
91     ),
92 }
93
94 ROBOT_WRITE_POLICY: FieldPolicy = {
95     "owner": frozenset({"name", "notes"}),
96     "technician": frozenset({"status", "notes"}),
97     "admin": frozenset({"name", "status", "notes", "
98     firmware_version"}),

```

```

88 }
89
90 _ROLE_PRECEDENCE = ("admin", "technician", "auditor", "
    owner")
91
92
93 def role_for(roles: Iterable[str]) -> str | None:
94     """Pick the most privileged policy role the actor
        holds."""
95     held = set(roles)
96     for candidate in _ROLE_PRECEDENCE:
97         if candidate in held:
98             return candidate
99     return None
100
101
102 def filter_fields(
103     record: Mapping[str, Any], policy: FieldPolicy,
104     roles: Iterable[str]
105 ) -> dict[str, Any]:
106     """Return only the fields the actor's role may read
        (BOPLA defense)."""
107     role = role_for(roles)
108     allowed = policy.get(role, frozenset()) if role
109         else frozenset()
110     return {key: value for key, value in record.items()
111             if key in allowed}
112
113
114 def apply_patch(
115     record: Mapping[str, Any],
116     patch: Mapping[str, Any],
117     policy: FieldPolicy,
118     roles: Iterable[str],
119 ) -> dict[str, Any]:
120     """Apply only writable fields; reject anything else
        .
121
122     """
123     role = role_for(roles)
124     allowed = policy.get(role, frozenset()) if role

```

```

125         else frozenset()
126         rejected = sorted(set(patch) - allowed)
127         if rejected:
128             raise ForbiddenError(f"fields_not_writable_by_
129                                 this_role:{rejected}")
130         updated = dict(record)
131         for key in allowed & set(patch):
132             updated[key] = patch[key]
133         for invariant in ("id", "owner_id", "tenant_id"):
134             updated[invariant] = record[invariant]
135         return updated
136
137 def require_readable_factory(
138     get_actor: Callable[[], Guard],
139 ) -> Callable[[Mapping[str, Any]], Mapping[str, Any]]:
140     """Adapt the guard to DI: fetch-then-authorize in
141     one step."""
142
143     def require_readable(resource: Mapping[str, Any])
144         -> Mapping[str, Any]:
145         get_actor().require_read(resource)
146         return resource
147
148     return require_readable

```

17.3.7 Listing 16.7 — SSRF-Safe Outbound Fetcher (ssrf_fetcher.py)

Seven layers, each independently sufficient against the lab exploits: scheme allow-list, exact host allow-list, port restriction, userinfo/IP-literal rejection, DNS pinning against private ranges (rebinding-safe), redirect re-validation, and response size caps.

```

1  """SSRF-safe_outbound_fetcher.
2
3  Defense_layers, each independently sufficient to block
4  the_lab_exploits:
5  1. Scheme allow-list (https only by default).
6  2. Host allow-list (exact match; no suffix tricks).
7  3. Port restriction.
8  4. DNS pinning: resolve first, reject private/link-
   local/loopback ranges,
   then connect to the *validated* address (closes
   DNS-rebinding TOCTOU).

```



```

9  """5. Redirect re-validation: every hop is checked like
   the original URL.
10 """6. Size and content-type caps on the response.
11 """
12
13 from __future__ import annotations
14
15 import ipaddress
16 from dataclasses import dataclass
17 from typing import Callable
18
19 import httpx
20
21 MAX_BODY_BYTES = 64 * 1024
22 ALLOWED_SCHEMES = frozenset({"https", "http"}) # http
   only for the offline lab
23 ALLOWED_PORTS = frozenset({80, 443, 8443})
24
25
26 class SsrfBlockedError(PermissionError):
27     """Raised when a URL fails any layer of the SSRF
   policy."""
28
29
30 # Explicit blacklist (stable across Python versions,
   unlike the shifting
31 # semantics of 'is_private'). In production, prefer
   'ip.is_global'
32 # from a maintained egress-control library instead of
   hand-rolling this.
33 _BLOCKED_NETWORKS = tuple(
34     ipaddress.ip_network(net)
35     for net in (
36         "0.0.0.0/8",          # "this host"
37         "10.0.0.0/8",         # RFC 1918
38         "100.64.0.0/10",      # carrier-grade NAT
39         "127.0.0.0/8",        # loopback
40         "169.254.0.0/16",     # link-local: home of
   169.254.169.254 metadata
41         "172.16.0.0/12",      # RFC 1918
42         "192.168.0.0/16",     # RFC 1918
43         "224.0.0.0/4",        # multicast
44         "240.0.0.0/4",        # reserved
45         "::/128",             # unspecified
46         "::1/128",            # IPv6 loopback
47         "fc00::/7",           # IPv6 unique-local

```

```

48         "fe80::/10",          # IPv6 link-local
49         "ff00::/8",          # IPv6 multicast
50         "::ffff:0:0/96",      # IPv4-mapped IPv6 (re-
                                checked after mapping)
51     )
52 )
53
54
55 def _is_public_address(address: str) -> bool:
56     try:
57         ip = ipaddress.ip_address(address)
58     except ValueError:
59         return False
60     if isinstance(ip, ipaddress.IPv6Address) and ip.
        ipv4_mapped is not None:
61         ip = ip.ipv4_mapped # canonicalize ::ffff
                                :10.0.0.1 -> 10.0.0.1
62     return not any(ip in net for net in
        _BLOCKED_NETWORKS)
63
64
65 @dataclass(frozen=True)
66 class OutboundPolicy:
67     allowed_hosts: frozenset[str]
68     allowed_schemes: frozenset[str] = ALLOWED_SCHEMES
69     allowed_ports: frozenset[int] = ALLOWED_PORTS
70     max_redirects: int = 3
71
72     def check_url(self, url: str) -> httpx.URL:
73         parsed = httpx.URL(url)
74         scheme = parsed.scheme.lower()
75         if scheme not in self.allowed_schemes:
76             raise SsrfBlockedError(f"scheme_{scheme}'_not_allowed")
77         host = (parsed.host or "").lower()
78         if host not in self.allowed_hosts:
79             raise SsrfBlockedError(f"host_{host}'_not_on_allow-list")
80         port = parsed.port or (443 if scheme == "https"
                                else 80)
81         if port not in self.allowed_ports:
82             raise SsrfBlockedError(f"port_{port}_not_allowed")
83         # Canonicalization guard: reject userinfo and
            raw-IP literals.
84         if parsed.username or parsed.password:

```

```

85         raise SsrfBlockedError("userinfo_in_URL_is_
            not_allowed")
86     if _looks_like_ip(host):
87         raise SsrfBlockedError("raw_IP_literals_are
            not_allowed")
88     return parsed
89
90     def check_resolved(
91         self, parsed: httpx.URL, resolve: Callable[[str
92             ], list[str]]
93     ) -> None:
94         """DNS pinning: every resolved address must be
95             public."""
96         for address in resolve(parsed.host or ""):
97             if not _is_public_address(address):
98                 raise SsrfBlockedError(f"resolved_
99                     address_{address}_is_not_public")
100
101     def _looks_like_ip(host: str) -> bool:
102         try:
103             ipaddress.ip_address(host)
104             return True
105         except ValueError:
106             return False
107
108     class SafeFetcher:
109         """Fetches only URLs that survive the full policy
110             gauntlet."""
111
112         def __init__(
113             self,
114             policy: OutboundPolicy,
115             client: httpx.Client,
116             *,
117             resolve: Callable[[str], list[str]],
118         ) -> None:
119             self._policy = policy
120             self._client = client
121             self._resolve = resolve
122
123         def fetch_text(self, url: str) -> str:
124             current = url
125             for _hop in range(self._policy.max_redirects +
126                             1):

```

```

124         parsed = self._policy.check_url(current)
125         self._policy.check_resolved(parsed, self._resolve)
126         # follow_redirects is OFF: we re-validate every hop ourselves.
127         response = self._client.get(parsed, follow_redirects=False)
128         if response.is_redirect:
129             location = response.headers.get("location")
130             if not location:
131                 raise SsrfBlockedError("redirect without Location header")
132             current = str(parsed.join(location))
133             continue
134         response.raise_for_status()
135         body = response.content
136         if len(body) > MAX_BODY_BYTES:
137             raise SsrfBlockedError("response exceeds size cap")
138         return body.decode("utf-8", errors="strict")
139     raise SsrfBlockedError("too many redirects")

```

17.3.8 Listing 16.8 — STRIDE Worksheet Generator (stride_worksheet.py)

Threat modeling that cannot drift from the shipped surface: feed it the OpenAPI document, receive a per-endpoint STRIDE checklist skeleton with heuristic flags (path ids → BOLA review, write methods → mass-assignment review, missing 429 → DoS review, admin paths → elevation review). The generator is deliberately simple — its value is that it runs in CI on every spec change.

```

1  """STRIDE_worksheet_generator.
2
3  Reads an OpenAPI 3.x document (JSON or the small YAML subset PyYAML-free
4  tooling can handle: pass pre-parsed dicts) and emits a per-endpoint threat
5  checklist skeleton. Pure stdlib, offline, deterministic: endpoints are
6  sorted, heuristics are explicit rules you can audit.
7  """
8

```

```

9  from __future__ import annotations
10
11  import json
12  from typing import Any, Iterable
13
14  STRIDE = ("Spoofing", "Tampering", "Repudiation", "
15           Information_Disclosure",
16           "Denial_of_Service", "Elevation_of_Privilege"
17           )
18
19  WRITE_METHODS = {"post", "put", "patch", "delete"}
20  ADMIN_HINTS = ("/admin", "/internal", "/debug")
21  ID_PARAM_HINTS = ("{id}", "{robot_id}", "{user_id}", "{
22                    order_id}")
23
24  def _heuristics(method: str, path: str, operation: dict
25                  [str, Any]) -> dict[str, list[str]]:
26      """Flag the STRIDE categories that need human
27      attention and why."""
28      flagged: dict[str, list[str]] = {s: [] for s in
29      STRIDE}
30      secured = bool(operation.get("security"))
31      has_id = any(hint in path for hint in
32      ID_PARAM_HINTS) or any(
33      p.get("in") == "path" and str(p.get("name", ""))
34      ).endswith("id")
35      for p in operation.get("parameters", [])
36      ):
37          if not secured:
38              flagged["Spoofing"].append("no_security_scheme_
39              declared_on_operation")
40          if has_id:
41              flagged["Information_Disclosure"].append(
42              "path_id_present: verify object-level
43              authorization (BOLA)"
44              )
45          flagged["Tampering"].append(
46          "path_id_present: confirm ids are not
47          enumerable or are scoped"
48          )
49      if method in WRITE_METHODS:
50          flagged["Tampering"].append(
51          "write_method: schema must forbid server-
52          owned fields (mass assignment)"
53          )
54      )

```

```

43         flagged["Repudiation"].append(
44             "write_method: require an audit record with
              actor, before/after"
45         )
46     if any(hint in path for hint in ADMIN_HINTS):
47         flagged["Elevation of Privilege"].append(
48             "admin/internal surface: function-level
              role check required"
49         )
50     responses = operation.get("responses", {})
51     if "429" not in responses:
52         flagged["Denial of Service"].append(
53             "no 429 documented: confirm rate/size
              limits exist"
54         )
55     if method == "get":
56         flagged["Information Disclosure"].append(
57             "read method: review response fields for
              excessive data exposure"
58         )
59     return {k: v for k, v in flagged.items() if v}
60
61
62 def generate_worksheet(spec: dict[str, Any]) -> str:
63     """Emit a Markdown STRIDE checklist skeleton, one
        block per endpoint."""
64     title = spec.get("info", {}).get("title", "Untitled
        API")
65     lines = [
66         f"# STRIDE Worksheet -- {title}",
67         "",
68         "Fill one row per flagged category: threat,
        abuse case, mitigation,",
69         "detection signal, owner, status.",
70         "",
71     ]
72     for path in sorted(spec.get("paths", {})):
73         for method in sorted(spec["paths"][path]):
74             if method not in {"get", "post", "put", "
                patch", "delete"}:
75                 continue
76             operation = spec["paths"][path][method]
77             summary = operation.get("summary", "")
78             lines.append(f"#{method.upper()} {path}")
79             if summary:
80                 lines.append(f"_{summary}_")

```

```

81         lines.append("")
82         flagged = _heuristics(method, path,
83                                operation)
84         if not flagged:
85             lines.append("-[ ] No heuristic flags;
86                           still review manually.")
87         for category in STRIDE:
88             for reason in flagged.get(category, []):
89                 lines.append(f"-[ ] **{category}**
90                               --{reason}")
91         lines.append("")
92         lines.append("| STRIDE | Threat | Abuse |
93                       case | Mitigation | Detection | Owner |
94                       Status |")
95         lines.append("| --- | --- | --- | --- | --- | --- | --- |")
96         for category in STRIDE:
97             mark = "FLAGGED" if category in flagged
98             else ""
99             lines.append(f"| {category} | {mark} |
100                           | | | | |")
101         lines.append("")
102     return "\n".join(lines)
103
104 def load_spec(path: str) -> dict[str, Any]:
105     """Load an OpenAPI document from a JSON file."""
106     with open(path, encoding="utf-8") as handle:
107         data: dict[str, Any] = json.load(handle)
108     if "paths" not in data:
109         raise ValueError("not an OpenAPI document:
110                           missing 'paths'")
111     return data
112
113 def main(argv: Iterable[str] | None = None) -> int:
114     import argparse
115
116     parser = argparse.ArgumentParser(description="
117         OpenAPI -> STRIDE worksheet")
118     parser.add_argument("spec", help="path to an
119         OpenAPI JSON document")
120     parser.add_argument("-o", "--out", default=None,
121                         help="output.md path")
122     args = parser.parse_args(list(argv) if argv is not

```

```

    None else None)
114     worksheet = generate_worksheet(load_spec(args.spec)
    )
115     if args.out:
116         with open(args.out, "w", encoding="utf-8") as
            handle:
117             handle.write(worksheet)
118     else:
119         print(worksheet)
120     return 0
121
122
123 if __name__ == "__main__":
124     raise SystemExit(main())

```

17.3.9 Listing 16.9 — Exploit Suite Wiring (conftest.py)

One option switches the suite between *demonstration* mode (prove exploit and fix) and *contract* mode (assert the hardened contract against a chosen build — the lab’s red/green gauntlet).

```

1  """Pytest wiring for the exploit suite.
2
3  Default mode ("demo"): each test proves the exploit
    lands on the vulnerable
4  app AND is repelled by the hardened app -- 10 greens.
5
6  Lab mode: 'pytest --target=vulnerable' asserts the *
    hardened contract*
7  against the vulnerable build (10 reds); '--target=
    hardened' re-runs the
8  same contract (10 greens). That is the exploit-then-fix
    gauntlet.
9  """
10
11 from __future__ import annotations
12
13 import pytest
14
15
16 def pytest_addoption(parser: pytest.Parser) -> None:
17     parser.addoption(
18         "--target",
19         action="store",
20         default="demo",
21         choices=["demo", "vulnerable", "hardened"],

```



```

22         help="demo: prove exploit+fix; vulnerable/
           hardened: assert contract",
23     )
24
25
26 @pytest.fixture()
27 def target(request: pytest.FixtureRequest) -> str:
28     return str(request.config.getoption("--target"))

```

17.3.10 Listing 16.10 — The Exploit Suite (test_exploit_suite.py)

Ten tests, one per OWASP item. Each demonstrates the exploit against the vulnerable build (the assertion that it *succeeds* is part of the test — an exploit that does not land proves nothing) and its failure against the hardened build.

```

1  """Exploit suite: ten OWASP API Security Top 10 flaws,
   exploited then fixed.
2
3  Default run('pytest') proves both halves per item:
   the exploit succeeds
4  against the vulnerable build and is repelled by the
   hardened build.
5  'pytest --target=vulnerable' asserts the security
   contract against the
6  vulnerable build (10 reds). 'pytest --target=hardened' runs
   the same contract
7  against the fixed build (10 greens). That is the
   exploit-then-fix gauntlet.
8  """
9
10 from __future__ import annotations
11
12 from fastapi.testclient import TestClient
13
14 import hardened_app
15 import vulnerable_app
16 from jwks_fixture import mint_token
17
18 BOBS_ROBOT = 1002 # owned by bob; alice must never
   reach it
19
20
21 def _clients() -> tuple[TestClient, TestClient]:
22     return (
23         TestClient(vulnerable_app.create_app()),

```

```

24         TestClient(hardened_app.create_app()),
25     )
26
27
28 def _vuln_headers(user: str = "alice") -> dict[str, str]:
29     return {"Authorization": f"Bearer_{vulnerable_app.forge_token(user)}"}
30
31
32 def _jwt_headers(
33     user: str, scopes: list[str], roles: list[str] |
34     None = None
35 ) -> dict[str, str]:
36     token = mint_token(user, scopes, extra={"roles":
37         roles or ["owner"]})
38     return {"Authorization": f"Bearer_{token}"}
39
40 def check(
41     target: str, exploit_ok: bool, holds_vuln: bool,
42     holds_hard: bool
43 ) -> None:
44     """demo: prove exploit lands AND fix holds; target
45     modes: contract only."""
46     if target == "demo":
47         assert exploit_ok, "exploit should succeed
48             against the vulnerable build"
49         assert not holds_vuln, "vulnerable build must
50             fail the security contract"
51         assert holds_hard, "hardened build must hold
52             the security contract"
53     elif target == "vulnerable":
54         assert holds_vuln # expected to FAIL: 10 reds
55             in the lab
56     else:
57         assert holds_hard # expected to PASS: 10
58             greens
59
60
61 # --- API1:2023 Broken Object Level Authorization
62 -----
63 def test_api1_bola_cross_owner_read(target: str) ->
64     None:
65     vuln, hard = _clients()
66     exploit = vuln.get(f"/v1/robots/{BOBS_ROBOT}",

```

```

        headers=_vuln_headers("alice"))
57 exploit_ok = (
58     exploit.status_code == 200 and exploit.json()["
        owner_id"] == "bob"
59 )
60 holds_vuln = exploit.status_code == 403
61 defense = hard.get(
62     f"/v1/robots/{BOBS_ROBOT}", headers=
        _jwt_headers("alice", ["fleet:read"])
63 )
64 check(target, exploit_ok, holds_vuln, defense.
        status_code == 403)
65
66
67 # --- API2:2023 Broken Authentication
    -----
68 def test_api2_forged_unsigned_token(target: str) ->
    None:
69     vuln, hard = _clients()
70     forged = vulnerable_app.forge_token("mallory") #
        unsigned, self-minted
71     exploit = vuln.get(
72         f"/v1/robots/1001", headers={"Authorization": f"
            Bearer_{forged}"}}
73 )
74 exploit_ok = exploit.status_code == 200
75 holds_vuln = exploit.status_code == 401
76 defense = hard.get(
77     f"/v1/robots/1001", headers={"Authorization": f"
            Bearer_{forged}"}}
78 )
79 check(target, exploit_ok, holds_vuln, defense.
        status_code == 401)
80
81
82 # --- API3:2023 Broken Object Property Level
    Authorization
    -----
83 def test_api3_mass_assignment_and_excessive_exposure(
    target: str) -> None:
84     vuln, hard = _clients()
85     exploit = vuln.patch(
86         f"/v1/robots/1001",
87         json={"owner_id": "mallory", "service_key": "
            sk_test_DEMO_pwned"},
            headers=_vuln_headers("alice"),
88

```

```

89     )
90     leak = vuln.get("/v1/robots/1001", headers=
91         _vuln_headers("alice"))
92     exploit_ok = (
93         exploit.status_code == 200
94         and exploit.json()["owner_id"] == "mallory" #
95             server-owned field set!
96         and "service_key" in leak.json() # secret
97             serialized to any caller
98     )
99     holds_vuln = (
100         exploit.status_code in (403, 422) and "
101             service_key" not in leak.json()
102     )
103     defense = hard.patch(
104         "/v1/robots/1001",
105         json={"owner_id": "mallory"},
106         headers=_jwt_headers("alice", ["fleet:write"]),
107     )
108     read_back = hard.get(
109         "/v1/robots/1001", headers=_jwt_headers("alice"
110             , ["fleet:read"])
111     )
112     holds_hard = (
113         defense.status_code == 422 # schema forbids
114             the field outright
115         and "service_key" not in read_back.json()
116         and "owner_id" not in read_back.json()
117     )
118     check(target, exploit_ok, holds_vuln, holds_hard)
119
120 # --- API4:2023 Unrestricted Resource Consumption
121 -----
122 def test_api4_unbounded_simulation(target: str) -> None
123 :
124     vuln, hard = _clients()
125     exploit = vuln.post("/v1/fleet/simulate", params={"
126         steps": 10_000_000})
127     exploit_ok = exploit.status_code == 200 # 10M
128         steps accepted, no cap
129     holds_vuln = exploit.status_code == 422
130     defense = hard.post(
131         "/v1/fleet/simulate",
132         params={"steps": 10_000_000},
133         headers=_jwt_headers("alice", ["fleet:command"]

```

```

    ]),
125     )
126     check(target, exploit_ok, holds_vuln, defense.
        status_code == 422)
127
128
129 # --- API5:2023 Broken Function Level Authorization
    -----
130 def test_api5_admin_endpoint_without_role(target: str)
    -> None:
131     vuln, hard = _clients()
132     exploit = vuln.get("/v1/admin/users", headers=
        _vuln_headers("alice"))
133     exploit_ok = exploit.status_code == 200 and len(
        exploit.json()) == 3
134     holds_vuln = exploit.status_code == 403
135     defense = hard.get(
136         "/v1/admin/users",
137         headers=_jwt_headers("alice", ["fleet:read"],
            roles=["owner"]),
138     )
139     check(target, exploit_ok, holds_vuln, defense.
        status_code == 403)
140
141
142 # --- API6:2023 Unrestricted Access to Sensitive
    Business Flows
    -----
143 def test_api6_promo_redemption_farming(target: str) ->
    None:
144     vuln, hard = _clients()
145     successes = sum(
146         vuln.post(
147             "/v1/promo/redeem",
148             json={"code": "NWR-LAUNCH"},
149             headers=_vuln_headers("alice"),
150         ).status_code
151         == 200
152         for _ in range(12)
153     )
154     exploit_ok = successes == 12 # no velocity check:
        farmed a dozen times
155     holds_vuln = successes <= 3
156     codes = [
157         hard.post(
158             "/v1/promo/redeem",

```

```

159         json={"code": "NWR-LAUNCH"},
160         headers=_jwt_headers("alice", ["promo:
           redeem"]),
161     ).status_code
162     for _ in range(12)
163 ]
164 holds_hard = 429 in codes and codes.count(200) <= 3
165 check(target, exploit_ok, holds_vuln, holds_hard)
166
167
168 # --- API7:2023 Server-Side Request Forgery
    -----
169 def test_api7_ssrf_to_metadata_endpoint(target: str) ->
    None:
170     vuln, hard = _clients()
171     exploit = vuln.post(
172         "/v1/diagnostics/fetch",
173         json={"url": "http://169.254.169.254/latest/
           meta-data"},
174         headers=_vuln_headers("alice"),
175     )
176     exploit_ok = (
177         exploit.status_code == 200 and "SecretAccessKey
           " in exploit.json()["body"]
178     )
179     holds_vuln = exploit.status_code == 403
180     defense = hard.post(
181         "/v1/diagnostics/fetch",
182         json={"url": "http://169.254.169.254/latest/
           meta-data"},
183         headers=_jwt_headers("alice", ["diagnostics:run
           "]),
184     )
185     check(target, exploit_ok, holds_vuln, defense.
        status_code == 403)
186
187
188 # --- API8:2023 Security Misconfiguration
    -----
189 def test_api8_debug_config_exposed(target: str) -> None
    :
190     vuln, hard = _clients()
191     exploit = vuln.get("/v1/debug/config")
192     exploit_ok = exploit.status_code == 200 and "
           master_key" in exploit.json()
193     holds_vuln = exploit.status_code == 404

```

```

194     defense = hard.get("/v1/debug/config")
195     check(target, exploit_ok, holds_vuln, defense.
196           status_code == 404)
197
198 # --- API9:2023 Improper Inventory Management
199 -----
200 def test_api9_legacy_unauthenticated_version(target:
201     str) -> None:
202     vuln, hard = _clients()
203     exploit = vuln.get("/v0/robots") # no credentials
204     at all
205     exploit_ok = exploit.status_code == 200 and len(
206         exploit.json()) == 2
207     holds_vuln = exploit.status_code == 404
208     defense = hard.get("/v0/robots")
209     check(target, exploit_ok, holds_vuln, defense.
210           status_code == 404)
211
212 # --- API10:2023 Unsafe Consumption of APIs
213 -----
214 def test_api10_blind_trust_of_upstream(target: str) ->
215     None:
216     vuln, hard = _clients()
217     exploit = vuln.post("/v1/fleet/enrich/1001",
218         headers=_vuln_headers("alice"))
219     exploit_ok = (
220         exploit.status_code == 200
221         and exploit.json()["maintenance_mode"] is True
222         # upstream said jump
223     )
224     holds_vuln = (
225         exploit.status_code != 200 or "maintenance_mode"
226         " not in exploit.json()
227     )
228     defense = hard.post(
229         "/v1/fleet/enrich/1001",
230         headers=_jwt_headers("alice", ["fleet:write"]),
231     )
232     after = hard.get(
233         "/v1/robots/1001", headers=_jwt_headers("alice",
234         ["fleet:read"])
235     )
236     holds_hard = (
237         defense.status_code == 200

```

```

228         and "maintenance_mode" not in defense.json()
229         and "maintenance_mode" not in after.json()
230     )
231     check(target, exploit_ok, holds_vuln, holds_hard)

```

17.3.11 Listings 16.11–16.13 — Library Test Suites

The supporting libraries carry their own adversarial tests: `test_token_validation.py` attacks the validator with expired tokens, wrong audiences, wrong issuers, `alg=none`, unknown kids, and tampered signatures; `test_authz.py` proves the guard matrix (owner/stranger/admin/cross-tenant, read/write, field filtering, invariant re-assertion); `test_ssrf_fetcher.py` attacks the fetcher with the metadata IP, internal hosts, `file://`, userinfo tricks, DNS re-binding, and the allowed-host redirect to metadata. The full files ship in `code\ch16\`; they are reproduced here in full because each test encodes an attack.

```

1  """Tests for the token validation middleware and claim
   ->principal mapping."""
2
3  from __future__ import annotations
4
5  import jwt
6  import pytest
7  from fastapi import HTTPException
8  from fastapi.testclient import TestClient
9
10 from jwks_fixture import (
11     AUDIENCE,
12     ISSUER,
13     JWKS,
14     SYNTHETIC_PRIVATE_PEM,
15     mint_token,
16 )
17 from token_validation import TokenValidator, demo_app
18
19 VALIDATOR = TokenValidator(JWKS, issuer=ISSUER,
20     audience=AUDIENCE)
21 # Tokens that must validate are minted at the current
22 # instant (now=None);
23 # only the expired-token test pins a past epoch. Either
24 # way every run of
25 # the suite exercises identical code paths and
26 # assertions.
27 PAST = 1_700_000_000

```



```

24
25
26 def test_valid_token_maps_to_principal() -> None:
27     token = mint_token(
28         "alice", ["fleet:read", "fleet:write"], now=
29         None,
30         extra={"roles": ["owner"]},
31     )
32     principal = VALIDATOR.validate(token)
33     assert principal.subject == "alice"
34     assert principal.tenant == "tenant-nwr"
35     assert principal.has_scope("fleet:read")
36     assert not principal.has_scope("admin:users")
37
38 def test_expired_token_rejected() -> None:
39     token = mint_token("alice", ["fleet:read"], now=
40         PAST, ttl_seconds=60)
41     with pytest.raises(HTTPException) as err:
42         VALIDATOR.validate(token)
43     assert err.value.status_code == 401
44
45 def test_wrong_audience_rejected() -> None:
46     token = mint_token("alice", ["fleet:read"], now=
47         None, audience="nwr-billing-api")
48     with pytest.raises(HTTPException) as err:
49         VALIDATOR.validate(token)
50     assert err.value.status_code == 401
51
52 def test_wrong_issuer_rejected() -> None:
53     token = mint_token(
54         "alice", ["fleet:read"], now=None, issuer="
55         https://evil.example"
56     )
57     with pytest.raises(HTTPException) as err:
58         VALIDATOR.validate(token)
59     assert err.value.status_code == 401
60
61 def test_alg_none_rejected() -> None:
62     import time
63
64     issued = int(time.time())
65     token = jwt.encode(

```

```

66         {"iss": ISSUER, "aud": AUDIENCE, "sub": "alice"
67           , "exp": issued + 300,
68           "iat": issued, "scope": "fleet:read", "tenant"
69             : "tenant-nwr"},
70         key=None,
71         algorithm="none",
72     )
73     with pytest.raises(HTTPException) as err:
74         VALIDATOR.validate(token)
75     assert err.value.status_code == 401
76
77 def test_unknown_kid_rejected() -> None:
78     token = mint_token("alice", ["fleet:read"], now=
79         None, kid="nwr-rotated-2099")
80     with pytest.raises(HTTPException) as err:
81         VALIDATOR.validate(token)
82     assert err.value.status_code == 401
83     assert err.value.detail == "unknown_key_id"
84
85 def test_tampered_signature_rejected() -> None:
86     token = mint_token("alice", ["fleet:read"], now=
87         None)
88     head, payload, _sig = token.split(".")
89     tampered = f"{head}.{payload}.AAAA{token.split('.')[2][4:]}"
90     with pytest.raises(HTTPException):
91         VALIDATOR.validate(tampered)
92
93 def test_scope_enforcement_end_to_end() -> None:
94     client = TestClient(demo_app(VALIDATOR))
95     reader = mint_token("alice", ["fleet:read"], now=
96         None)
97     commander = mint_token("bob", ["fleet:command"],
98         now=None)
99     assert client.get(
100         "/v1/fleet/commands", headers={"Authorization":
101             f"Bearer_{reader}"})
102     ).status_code == 403
103     response = client.get(
104         "/v1/fleet/commands", headers={"Authorization":
105             f"Bearer_{commander}"})
106     )
107     assert response.status_code == 200

```

```

103     assert response.json()["by"] == "bob"
104     assert client.get("/v1/whoami").status_code == 401
105         # no header at all
106
107 def test_validator_requires_nonempty_jwks() -> None:
108     with pytest.raises(ValueError):
109         TokenValidator({}, issuer=ISSUER, audience=
110             AUDIENCE)
111
112 def test_minted_key_fixture_is_self_consistent() ->
113     None:
114     # Guards the fixture itself: the synthetic keypair
115     # must sign/verify.
116     token = jwt.encode({"sub": "selftest"},
117         SYNTHETIC_PRIVATE_PEM, algorithm="RS256")
118     decoded = jwt.decode(
119         token, next(iter(JWKS.values())), algorithms=["
120             RS256"]
121     )
122     assert decoded["sub"] == "selftest"

```

```

1  """Tests for the authorization guards in authz.py."""
2
3  from __future__ import annotations
4
5  import pytest
6
7  from authz import (
8      ROBOT_READ_POLICY,
9      ROBOT_WRITE_POLICY,
10     ForbiddenError,
11     Guard,
12     apply_patch,
13     filter_fields,
14 )
15
16 ALICE_ROBOT = {
17     "id": 1001,
18     "name": "Welder-7",
19     "model": "NR-220",
20     "status": "idle",
21     "battery_pct": 88,
22     "last_seen": "2026-08-30T09:14:00Z",
23     "notes": "dock_3",

```

```

24     "owner_id": "alice",
25     "tenant_id": "tenant-nwr",
26     "firmware_version": "4.2.0",
27     "service_key": "sk_test_DEMO_robot1001_invalid",
28     "service_key_last4": "1001",
29     "maintenance_mode": False,
30 }
31
32 OWNER = Guard(actor_id="alice", tenant_id="tenant-nwr",
33               roles=frozenset({"owner"}))
34 STRANGER = Guard(actor_id="mallory", tenant_id="tenant-
35 nwr", roles=frozenset({"owner"}))
36 ADMIN = Guard(actor_id="carol", tenant_id="tenant-nwr",
37               roles=frozenset({"admin"}))
38 CROSS_TENANT = Guard(actor_id="alice", tenant_id="
39 tenant-other", roles=frozenset({"admin"}))
40
41
42 def test_owner_reads_own_robot() -> None:
43     OWNER.require_read(ALICE_ROBOT)
44
45
46 def test_stranger_denied_read_and_write() -> None:
47     with pytest.raises(ForbiddenError):
48         STRANGER.require_read(ALICE_ROBOT)
49     with pytest.raises(ForbiddenError):
50         STRANGER.require_write(ALICE_ROBOT)
51
52
53 def test_admin_reads_but_does_not_own_write() -> None:
54     ADMIN.require_read(ALICE_ROBOT)
55     with pytest.raises(ForbiddenError):
56         ADMIN.require_write(ALICE_ROBOT)
57
58
59 def test_cross_tenant_admin_denied() -> None:
60     with pytest.raises(ForbiddenError):
61         CROSS_TENANT.require_read(ALICE_ROBOT)
62
63
64 def test_filter_fields_hides_secrets_from_owner() ->
65 None:
66     view = filter_fields(ALICE_ROBOT, ROBOT_READ_POLICY
67 , ["owner"])
68     assert "service_key" not in view
69     assert "owner_id" not in view

```

```

64     assert view["name"] == "Welder-7"
65
66
67 def test_filter_fields_admin_gets_last4_not_secret() ->
None:
68     view = filter_fields(ALICE_ROBOT, ROBOT_READ_POLICY
        , ["admin"])
69     assert view["service_key_last4"] == "1001"
70     assert "service_key" not in view
71
72
73 def test_filter_fields_unknown_role_gets_nothing() ->
None:
74     assert filter_fields(ALICE_ROBOT, ROBOT_READ_POLICY
        , ["intruder"]) == {}
75
76
77 def test_apply_patch_allows_writable_fields() -> None:
78     updated = apply_patch(
79         ALICE_ROBOT, {"name": "Welder-7b"},
80         ROBOT_WRITE_POLICY, ["owner"]
81     )
82     assert updated["name"] == "Welder-7b"
83     assert updated["owner_id"] == "alice"
84
85 def test_apply_patch_rejects_server_owned_fields() ->
None:
86     with pytest.raises(ForbiddenError):
87         apply_patch(
88             ALICE_ROBOT,
89             {"owner_id": "mallory"},
90             ROBOT_WRITE_POLICY,
91             ["owner"],
92         )
93
94
95 def
test_apply_patch_reinstates_invariants_even_for_admin
() -> None:
96     forged = dict(ALICE_ROBOT)
97     forged["owner_id"] = "mallory"
98     updated = apply_patch(
99         forged, {"status": "charging"},
100         ROBOT_WRITE_POLICY, ["admin"]
    )

```

```

101     assert updated["owner_id"] == "mallory" # stored
        value preserved verbatim
102     assert updated["status"] == "charging"

1  """Tests for the SSRF-safe outbound fetcher against the
    simulated network."""
2
3  from __future__ import annotations
4
5  import pytest
6
7  from fake_network import make_client, simulated_dns
8  from ssrf_fetcher import OutboundPolicy, SafeFetcher,
    SsrfBlockedError
9
10 POLICY = OutboundPolicy(
11     allowed_hosts=frozenset({"telemetry.partner-nwr.
        example"})
12 )
13
14
15 def _fetcher() -> SafeFetcher:
16     return SafeFetcher(POLICY, make_client(), resolve=
        simulated_dns)
17
18
19 def test_allowed_host_fetches() -> None:
20     body = _fetcher().fetch_text("https://telemetry.
        partner-nwr.example/v1/manifest")
21     assert "firmware" in body
22
23
24 def test_metadata_ip_blocked_by_host_allowlist() ->
    None:
25     with pytest.raises(SsrfBlockedError):
26         _fetcher().fetch_text("http://169.254.169.254/
            latest/meta-data")
27
28
29 def test_internal_host_blocked() -> None:
30     with pytest.raises(SsrfBlockedError):
31         _fetcher().fetch_text("http://inventory.
            internal.nwr/status")
32
33
34 def test_scheme_restriction_blocks_file_and_gopher() ->

```

```

None:
35 with pytest.raises(SsrfBlockedError):
36     _fetcher().fetch_text("file:///etc/passwd")
37 with pytest.raises(SsrfBlockedError):
38     _fetcher().fetch_text("gopher://telemetry.
        partner-nwr.example/")
39
40
41 def test_userinfo_trick_blocked() -> None:
42     with pytest.raises(SsrfBlockedError):
43         _fetcher().fetch_text(
44             "http://telemetry.partner-nwr.example@169
                .254.169.254/latest"
45         )
46
47
48 def
test_dns_rebinding_blocked_when_host_resolves_private
() -> None:
49     fetcher = SafeFetcher(
50         POLICY,
51         make_client(),
52         resolve=lambda host: ["10.20.0.14"], # rebind:
            good name, bad address
53     )
54     with pytest.raises(SsrfBlockedError):
55         fetcher.fetch_text("https://telemetry.partner-
            nwr.example/v1/manifest")
56
57
58 def test_redirect_target_is_revalidated() -> None:
59     policy = OutboundPolicy(
60         allowed_hosts=frozenset(
61             {"telemetry.partner-nwr.example", "short.
                partner-nwr.example"}
62         )
63     )
64     fetcher = SafeFetcher(policy, make_client(),
        resolve=simulated_dns)
65     # short.partner-nwr.example 302s to the metadata
        endpoint: the hop is
66     # re-validated and blocked even though the *first*
        URL was allowed.
67     with pytest.raises(SsrfBlockedError):
68         fetcher.fetch_text("https://short.partner-nwr.
            example/x")

```

17.4 Common Pitfalls & Anti-Patterns

Authentication without authorization.

The gateway validates the JWT and the service calls it “secure.” But API1, API3, API5, and API6 all fire *after* perfect authentication. If your authorization logic fits in the gateway config, you do not have authorization logic.

Trusting client-supplied IDs.

Any identifier from path, query, or body is a *request*, not a *fact*. Fetch the object, then check the object against the principal. “The client would never send that” is the BOLA creed; clients are attackers who have not been caught yet.

PATCH merging the request into the model.

`record.update(body)` is mass assignment as a one-liner. The wire schema and the storage schema are different contracts — Pydantic gives you `extra="forbid"`; use it, and layer the role-keyed write policy beneath it.

Debug endpoints left in production.

“Unlinked” is not “unreachable.” If the route is compiled into the production build, an attacker will enumerate it. Debug surfaces must be absent from production artifacts, verified by a route-inventory diff in CI (API9’s discipline applied to API8’s problem).

Wildcard CORS with credentials.

Browsers refuse the literal pair, so developers “fix” it by reflecting the `Origin` header — which is wildcard CORS with extra steps. Exact origins only, credentials only where the first-party app genuinely needs cookies.

Security by obscurity.

Undocumented routes, “secret” URLs, unguessable-looking but sequential ids, hope as a patch process. The route table leaks the day an SDK, an archived spec, or a curious proxy log escapes. Obscurity may slow an attacker; it must never be the control.

Logging tokens and secrets.

Authorization headers in access logs, request bodies in debug logs, “temporary” print of the JWT payload. Treat logs as public within the org and one mis-shipped export away from public *tout court*; redact at the logging boundary.

Unversioned security dependencies.

Your authentication layer is exactly as strong as the JWT library's patch state. Pin versions, track CVEs, and treat security-library upgrades as fast-track changes — an `alg=none` acceptance CVE is a breach with a version number.

Fail-open error handling.

`except Exception: return anonymous_context` converts every authenticator bug into silent access. AuthN/authZ code fails *closed*: on any ambiguity, the answer is no, loudly logged.

Rate limits only on “expensive-looking” routes.

Attackers price your endpoints better than you do. The promo redemption looked cheap; the campaign budget disagreed. Limit by *business impact*, not by CPU intuition (API6).

17.5 Hands-On Production Lab & Real-World Case Study

17.5.1 Lab: The Exploit-Then-Fix Gauntlet

Objective. Feel the red/green rhythm that production security work actually has: first prove the system is broken, then prove it is fixed, and never confuse the two directions.

Setup (PowerShell).

```
1 cd "C:\Training\Training Areas\Data jobs learning\APIs
  - Usage and development\code\ch16"
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 python -m pip install -r requirements.txt
```

Phase 1 — watch ten reds. Assert the *hardened contract* against the *vulnerable* build:

```
1 python -m pytest test_exploit_suite.py -q --target=
  vulnerable
2 # 10 failed -- every OWASP item demonstrably
  exploitable
```

Open two or three failures and read them as an attacker: the BOLA response body contains `"owner_id": "bob"`; the SSRF response body contains `SecretAccessKey`; the promo counter happily reports eleven successes. Reds in this phase are evidence, not problems.

Phase 2 — exploit interactively. Fire up the vulnerable app and drive it by hand:

```
1 python -c "import vulnerable_app, uvicorn; uvicorn.run(  
    vulnerable_app.app, port=8765)"  
2 # in another shell:  
3 curl.exe -s http://127.0.0.1:8765/v1/debug/config  
4 curl.exe -s -X POST http://127.0.0.1:8765/v1/  
    diagnostics/fetch '  
5 -H "Content-Type: application/json" '  
6 -d '{"url": "http://169.254.169.254/latest/meta-data  
    "}'
```

Phase 3 — watch ten greens. The identical contract, the hardened build:

```
1 python -m pytest test_exploit_suite.py -q --target=  
    hardened  
2 # 10 passed
```

Phase 4 — prove both directions at once.

```
1 python -m pytest -q  
2 # 44 passed: exploit demonstrations, token gauntlet,  
    authz matrix,  
3 # SSRF gauntlet, STRIDE generator
```

Phase 5 — break it yourself. Pick one fix in `hardened_app.py` (delete the `guard.require_read` call, or change `extra="forbid"` to `"ignore"`), re-run the demo suite, and watch exactly one test catch exactly your sabotage. If none does, the suite — not the fix — is wrong: add the missing test.

17.5.2 War Story: The BOLA Breach That Walked the Primary Keys

A fleet-management provider (think Northwind Robotics with worse luck) shipped `GET /api/devices/{id}` behind a perfectly modern stack: TLS everywhere, RS256 access tokens, scopes checked at the gateway, SOC 2 badge gleaming. The handler looked up the device and returned it; ownership was “handled by the mobile app,” which only ever requested devices the user owned. The ids were sequential integers starting at 10 000.

A security researcher registered a free account, captured one request, decremented the id, and received another customer’s device — location history, owner email, and an MQTT password. A weekend script walking id downward harvested 1.4 million devices before a finance analyst asked why the API bill tripled. There had been no alarm: every request was

authenticated, schema-valid, and a 200 OK. The postmortem’s action items map one-to-one onto this chapter: object-level checks in the handler (API1), unguessable ids as depth (API1), removal of internal fields from responses (API3), per-principal enumeration anomaly alerts (detection), and a regression test asserting the 403 — the exploit suite pattern you have now run yourself. The company’s cryptography was never attacked. It never mattered.

17.6 Self-Assessment Exercises & Deep-Dive Questions

Exercises

- E16.1. Close the redirect hole by hand.** Extend `test_ssrf_fetcher.py` with a second simulated redirect: an allowed host that chains two 302s before landing on `inventory.internal.nwr`. Prove the fetcher blocks the final hop, then lower `max_redirects` to 1 and prove the chain dies earlier. What does each failure mode tell the operator?
- E16.2. Add a role.** Add a `support` role to `authz.py` that may read `notes` but not `battery_pct`, and may write nothing. Add the policy rows and three tests, including “support plus owner” union behavior.
- E16.3. Break the token pipeline safely.** In `token_validation.py`, move the claims mapping ahead of signature verification (do not keep this code). Which existing test catches it? If none does, write one that mints an unsigned token with `alg=HS256` and an attacker-chosen `sub`.
- E16.4. Inventory audit.** Use `app.routes` on both builds to list every route; diff them against the OpenAPI document. Which vulnerable-build routes are absent from its own spec, and which OWASP item does that instantiate?
- E16.5. Property-level read diff.** Capture the JSON returned for robot 1001 by *each* role against the hardened build and diff the field sets against `ROBOT_READ_POLICY`. Then do the same against the vulnerable build and count the excess fields.

Deep-Dive Questions

- D16.1.** Why is aud enforcement still mandatory when your IdP only issues tokens to one audience today? Trace the failure eighteen months later when a second API appears.
- D16.2.** The chapter claims dependent layers “add ceremony.” Give a concrete pair of defenses against mass assignment that are *dependent* (share a failure mode) and a pair that are *independent*.
- D16.3.** DNS pinning validates the resolved address and connects to it. What breaks about TLS hostname verification when you connect by IP, and how do production SSRF guards resolve that tension?
- D16.4.** In the token-exchange pattern, what should happen to the **sub** and **act** claims across two hops, and what do you lose if hop two impersonates instead of delegates?
- D16.5.** Which of the ten OWASP items are *invisible* to a Web Application Firewall at the edge, and why? (Hint: anything requiring knowledge of object ownership.)
- D16.6.** The promo rate limiter in the hardened app is a per-process counter. What changes when the service runs twelve replicas, and which Chapter 15 architecture applies?

17.7 Interview Q&A Vault

Junior (Q1–Q10)

Q1. Explain BOLA and its mitigation.

A. Broken Object Level Authorization: the server checks *that* you’re logged in but not *whether* you may touch the requested object, so `/v1/robots/1002` returns someone else’s robot. Mitigate in the handler: fetch the object, compare its `owner_id/tenant_id` to the authenticated principal, default-deny, and add unguessable ids as depth — never as the control.

Q2. What is the difference between authentication and authorization?

A. Authentication proves identity with cryptographic evidence; authorization decides what that identity may do — function (scope/role), object (ownership), and properties (fields). Conflating them is how “authenticated” APIs leak other tenants’ data.

Q3. What are the three segments of a JWT, and which are protected?

A. Header, payload, signature, base64url-joined. Only the signature is integrity-protected; header and payload are readable by anyone, so never put secrets in them [Jon15].

Q4. Why is `alg=none` acceptance so dangerous?

A. A “JWT” with algorithm `none` has no signature at all — anyone can mint one claiming any `sub`. Validators must enforce a server-side algorithm allow-list; never honor the `alg` the attacker supplies.

Q5. What does a 401 mean versus a 403?

A. 401: identity unknown or invalid (missing/bad token) — authenticate, then retry. 403: identity known, action refused — retrying unchanged will never succeed. 401 carries `WWW-Authenticate` [Fie22a].

Q6. What is mass assignment?

A. Binding the request body directly onto the model (`record.update(patch)`), letting clients set server-owned fields like `owner_id` or `is_admin`. Defense: explicit request schemas with `extra="forbid"` plus a role-keyed writable-field policy.

Q7. Why must API responses not serialize the database row directly?

A. Rows contain fields clients must never see — service keys, internal notes, other tenants’ metadata. The client cannot be trusted to hide them. Serialize a role-appropriate projection.

Q8. What is SSRF in one paragraph?

A. The server fetches an attacker-chosen URL, so the attacker reaches destinations only the server can: internal services, localhost, and the cloud metadata endpoint `169.254.169.254`, which hands out instance credentials.

Q9. Name three security headers and what each blocks.

A. `X-Content-Type-Options: nosniff` (MIME confusion), `X-Frame-Options: DENY` (clickjacking), `Cache-Control: no-store` (sensitive data in caches). `Strict-Transport-Security` and CSP belong on anything serving HTML.

Q10. Where do API keys belong in a request, and why?

A. In a header (`Authorization` or a dedicated key header). Query strings land in access logs, browser history, and `Referer`; headers do not.

Mid-Level (Q11–Q22)

Q11. Walk the safe JWT validation order.

A. Structure (three segments) → `alg` against a server allow-list → `kid` resolved against your trusted JWKS → signature → `iss`, `aud`, `exp/nbf` with leeway → *only then* map claims to a principal. Any earlier claim use is a vulnerability.

Q12. How does mass assignment differ from BOLA?

A. BOLA is the wrong *object*; mass assignment is the wrong *properties* on the right object. They compose: a BOLA read plus mass-assignment write is a full account takeover. Different guards, same family — OWASP groups mass assignment under API3 with excessive data exposure [OWA23].

Q13. Coarse versus fine scopes — how do you choose?

A. Coarse scopes (`read/write`) are simple but over-privilege; per-object scopes explode and still cannot express dynamic ownership. Middle path: scopes name capability classes (`fleet:command`), roles name standing, and the resource server enforces object-level rules against its own data.

Q14. What is audience restriction and what breaks without it?

A. `aud` names the intended resource server; without enforcement, a token minted for a low-risk API replays against your high-risk one — lateral movement with valid signatures. Validate `aud` exactly, on every service.

Q15. Explain token exchange for service-to-service calls.

A. Service A exchanges the inbound user token at the IdP for a new token with audience `service-b` and minimum scopes, keeping the user visible via an actor claim (delegation, not impersonation). Least privilege per hop, auditable chain.

Q16. Design SSRF defenses in layers.

A. Scheme allow-list; exact host allow-list (no suffix matches, no raw IPs, no userinfo); port restriction; resolve-then- validate DNS against private/link-local ranges and connect to the validated address (anti-rebinding); re-validate every redirect hop; size/type caps; platform egress rules as the backstop.

Q17. What is DNS rebinding and how does pinning stop it?

A. The attacker's domain resolves to a public IP at your validation moment, then to 127.0.0.1 or a link-local address at connect moment (short TTL, race). Pinning: resolve once, validate every returned address, and connect to *that* address — the TOCTOU window disappears.

Q18. Why is “filter the response fields in the serializer” safer than “don’t put secrets in the model”?

A. The model must hold secrets for its own work (signing, rotation). The serializer is the boundary where context (the caller's role) is available, so the allow-list projection belongs there. Secrets-in-model is unavoidable; secrets-on-wire is the bug.

Q19. How do you threat-model one endpoint with STRIDE?

A. Enumerate the route's inputs, state changes, and outputs; walk Spoofing/Tampering/Repudiation/Information Disclosure/DoS/Elevation asking “how could this happen here?”; each credible threat gets a mitigation and a detection signal; each abuse case becomes an executable acceptance test.

Q20. Wildcard CORS with credentials — what's the trap?

A. Browsers reject `Access-Control-Allow-Origin: *` with credentials, so developers reflect the request's `Origin` instead — which is universal cross-origin read access for any malicious site. Exact allow-lists only; and CORS is browser-only, never an authN/Z substitute.

Q21. When does a cookie-authenticated API need CSRF defenses, and which?

A. Whenever browsers attach credentials automatically. `SameSite=Lax/Strict` baseline; synchronizer-token or double-submit header for state-changing methods; custom-header requirement as depth. Bearer tokens in `Authorization` headers are CSRF-immune by construc-

tion.

Q22. What’s wrong with sequential integer ids?

A. Nothing cryptographically — but they turn any BOLA into bulk enumeration and leak business volume. Use random UUIDv4/UUIDv7 externally *as defense in depth* while the ownership check remains the actual control.

Senior & Staff (Q23–Q33)

Q23. Design authorization for a multi-tenant API.

A. Tenant in the token (signed claim, not a header the client picks); every stored object carries `tenant_id`; the guard requires `record.tenant_id == principal.tenant` before any role logic; cross-tenant admin operations go through a separate audited elevation path; tests include cross-tenant access for every route. Add tenant-scoped rate limits so one tenant’s abuse cannot starve others.

Q24. Scope design for a payments API?

A. Capability classes per risk tier: `payments:read`, `payments:create`, `payments:refund`, `payouts:configure` — the last two step-up protected (fresh auth, MFA evidence in `acr/amr`). Amount limits as *policy*, not scope combinatorics; object-level checks (this merchant’s payments only) in the resource server; every mutation idempotent (Chapter 11) and audited.

Q25. mTLS versus bearer tokens for service identity — decide and defend.

A. mTLS binds identity to the transport and the workload’s key possession — nothing reusable crosses the wire — but user semantics (scopes, delegation) don’t fit certificates. Bearer tokens carry rich, expiring, audience-restricted context but are copyable. Production answer: both — mTLS (SPIFFE/SPIRE SVIDs, auto-rotated) authenticates the workload channel; a narrowly-scoped token inside carries the user context.

Q26. Your JWKS endpoint is unavailable during a deploy. Design the failure mode.

A. Cache keys with TTL well beyond token lifetime; on unknown `kid`, one rate-limited refetch, then 401 (fail closed) — never accept

on cache miss. Pre-warm caches on rotation; overlap old/new keys in the published set so in-flight tokens verify through rollover.

Q27. How would you detect BOLA exploitation in production?

A. Per-principal object-access metrics: distinct owners touched per subject, sequential-id walk detection, 403:200 ratio shifts, enumeration velocity. Alert on *successful* cross-owner reads, not just denied ones — denials are the rehearsal; successes are the breach.

Q28. Design the redaction boundary for logs.

A. A logging middleware/filter with a deny-list (`Authorization`, cookies, fields matching key patterns like `sk_*`) applied at emission, plus body capture off by default in production. Treat centralized logs as a breach surface: least-privilege access, retention limits, and periodic gitleaks-style scans of the log store itself.

Q29. API6 says the flow worked “as designed.” How do you specify abuse-resistance for a business flow?

A. Write the economic abuse case first (“drain the budget via fresh accounts”); derive quantitative invariants (redemptions per verified identity ≤ 1 , per-IP velocity, per-campaign budget circuit breaker); implement as layered controls (token bucket per principal, device/identity correlation, anomaly alerts); express each as an executable test — this chapter’s `test_api6` is the template.

Q30. A partner API you consume is compromised. What did your architecture already assume?

A. Upstream responses are untrusted input: schema validation with `extra="ignore"`, field-level allow-listing into your domain model, timeouts and size caps, no following of upstream-supplied URLs without the SSRF gauntlet, provenance logging, and a kill switch per integration. Their breach becomes your 502 spike, not your data corruption (API10).

Q31. How do you keep the API inventory honest at scale?

A. The deployed route table *is* the inventory: generate it from OpenAPI specs in CI, diff gateway-observed routes against it (any unlisted route is an incident), enforce default-deny at the gateway, attach owners and end-of-life dates to every version (Chapter 10,

Chapter 21), and alert on any traffic to retired versions — nonzero v0 traffic is the smoke alarm.

Q32. Critique: “We put authorization in the API gateway, so services trust gateway headers.”

A. Fine only if the network is provably closed (mTLS everywhere, no path to services except the gateway, header stripping at the edge) — one misconfigured ingress and callers forge `X-User-Role: admin`. Depth says: gateway enforces coarse policy, services still verify a signed token and enforce object-level rules against their own data.

Q33. Design certificate rotation that cannot page you at 3 a.m.

A. Short-lived workload certs issued by an internal CA (SPIRE), renewed automatically at half-life; trust bundles served with old+new during overlap; clients reload without restarts; alerts on *renewal failures*, not on expiry; a documented break-glass procedure whose drill runs quarterly. Manual rotation is an outage you have scheduled.

Q34. Which OWASP API items does a WAF not solve, and how do you explain that to an executive?

A. API1, API3, API5, most of API6 and API9 — they are *logic* failures indistinguishable from legitimate traffic at the edge: every malicious request was authenticated, schema-valid, and rate-reasonable. Executive framing: the WAF is the building’s front door; these attacks use a valid key and walk to the wrong office. The controls live in the application and the inventory, and they are testable — here is the suite.

17.8 External Bibliography & Further Reading

- **OWASP API Security Top 10 (2023)** [OWA23] — the taxonomy this chapter is organized around; read the project page’s per-item “Is the API Vulnerable?” sections alongside Section 17.2.3–17.2.12.
- **OWASP Application Security Verification Standard (ASVS)** — the control checklist for turning this chapter into an audit; sections V1 (architecture), V3 (session/token management), and V13 (API verification) map directly onto our pipeline.

- **OWASP Cheat Sheet Series** — especially the *REST Security*, *JSON Web Token*, *Server Side Request Forgery Prevention*, and *Cross-Site Request Forgery Prevention* sheets; the pragmatic counterparts to the standards.
- **RFC 6749** (OAuth 2.0) [Har12] and **RFC 7519** (JWT) [Jon15] — read the Security Considerations sections first; they anticipate half the exploits in this chapter.
- **RFC 8446** (TLS 1.3) [Res18] — the handshake that mTLS extends with `CertificateRequest`; worth reading for the transcript-authentication design.
- **RFC 9110** (HTTP Semantics) [Fie22a] — the precise meanings of 401/403, `WWW-Authenticate`, caching and `Vary` rules your response-security posture depends on.
- ***API Security in Action*** (Neil Madden, Manning) — the book-length treatment: capability tokens, macaroons, fine-grained authorization, and hardened OAuth patterns.
- **PortSwigger Web Security Academy** — free, hands-on labs for SSRF, access control, and injection; the closest public equivalent of this chapter's exploit gauntlet.
- **FastAPI** [Ram24] and **Pydantic** [Col24] documentation — security utilities, dependency injection, and `extra/strict` model configuration used throughout Section 17.3.
- ***Release It!*** (Nygard) [Nyg18] — the stability patterns (bulkheads, circuit breakers, fail-fast) that keep API4/API6 controls from becoming outages themselves.

17.9 Capstone Projects

Capstone 16.1 — BOLA Hunter [junior]

Objective. Build a small FastAPI notes API (`GET/PATCH /notes/{id}`) with an ownership model, then write an exploit script and the fix.

Inputs/Datasets. A synthetic in-memory store of 20 notes owned by 4 fictional users; unsigned-lab tokens acceptable here (this capstone predates the JWT requirement).

Steps. (1) Implement the API with the ownership check deliberately missing on `GET`. (2) Write `exploit.py` that enumerates ids 1–20 with one user's credential and dumps every note. (3) Add the `Guard`-style object

check. (4) Re-run: the exploit must now produce 20 denials. (5) Add a regression test asserting the 403.

Acceptance Criteria.

- ☐ Exploit script retrieves a foreign note against the vulnerable build.
- ☐ Hardened build answers 403 for every cross-owner access.
- ☐ Pytest suite: exploit-succeeds and fix-holds both asserted, all offline.
- ☐ README explains why unguessable ids alone would not fix it.

Stretch Goals. Add ETag-based conditional writes (Chapter 11); make ids UUIDv7 and show the enumeration script fail (then explain why that is not the control).

Capstone 16.2 — Scope Forge & Token Gauntlet [mid]

Objective. Stand up a toy resource server with the full token-validation pipeline and a scope model, then attack it with a forgery battery.

Inputs/Datasets. This chapter’s `jwt_fixtures.py` and `token_validation.py`; a scope vocabulary you design for a three-resource fleet domain.

Steps. (1) Design coarse/fine scope sets; document the trade-off. (2) Wire the validator into three endpoints with different required scopes. (3) Write adversarial tests: expired, wrong `aud`, wrong `iss`, `alg=none`, unknown `kid`, tampered payload, missing scope. (4) Add WWW-Authenticate diagnostics on failures.

Acceptance Criteria.

- ☐ Seven forgery tests all produce 401/403, none 200.
- ☐ Validation order provably never reads claims before signature verification (test with an unsigned token carrying admin claims).
- ☐ Scope design document justifies each scope’s granularity.

Stretch Goals. Implement token exchange to a second “service” with narrowed scope and audience; add clock-skew boundary tests.

Capstone 16.3 — SSRF Bunker [mid]

Objective. Harden a “webhook tester” feature against SSRF with the full seven-layer gauntlet and a simulated adversary network.

Inputs/Datasets. `fake_network.py` plus two hosts you add: one that redirects twice into link-local space, one serving a 1 MB body.

Steps. (1) Build the naive fetcher and exploit it three ways (metadata IP, internal host, redirect chain). (2) Layer in the `SafeFetcher` policy one

control at a time, re-running exploits after each. (3) Add DNS-pinning and redirect re-validation tests. (4) Document which layer stops which exploit, including the layers that stop *nothing* alone but matter in composition.

Acceptance Criteria.

- ☐ All naive exploits succeed pre-fix, all blocked post-fix, demonstrated in one pytest run.
- ☐ DNS-rebinding and redirect tests included.
- ☐ A layer-by-layer blame table in the README.

Stretch Goals. IPv6-mapped IPv4 bypass attempts; egress-rule simulation as a second, independent transport wrapper.

Capstone 16.4 — Threat-Model Pipeline [senior]

Objective. Turn threat modeling into a build artifact: extend `stride_worksheet.py` into a CI gate that fails when new endpoints ship without completed STRIDE rows.

Inputs/Datasets. The hardened app’s OpenAPI document; a completed worksheet file per endpoint checked into the repo.

Steps. (1) Extend the generator with heuristics for query-string filters, file uploads, and webhook registrations. (2) Add a `-check` mode that diffs generated skeletons against committed, human-completed worksheets. (3) Wire it as a CI step; demonstrate a red build by adding an undocumented route. (4) Write two abuse cases with executable acceptance tests.

Acceptance Criteria.

- ☐ `-check` exits nonzero on undocumented or unmodeled endpoints, zero when worksheets are current.
- ☐ At least three new heuristics with unit tests.
- ☐ Two abuse cases expressed as passing security tests.

Stretch Goals. Emit attack-tree DOT files from the worksheet annotations; score endpoint risk by flag count and data sensitivity.

Capstone 16.5 — Full Red Team / Blue Team Exercise [staff]

Objective. Run a two-sided exercise on a multi-service Northwind Robotics topology: a red team exploits a seeded environment; a blue team ships fixes without breaking the contract test suite.

Inputs/Datasets. This chapter’s vulnerable app extended with a second service (billing), a JWKS fixture with two audiences, and the simulated network; a scoring rubric mapping findings to OWASP items.

Steps. (1) Seed at least ten flaws across both services, including one cross-service token-confusion flaw (billing tokens accepted by fleet). (2) Red team: document each exploit as a failing contract test before showing it. (3) Blue team: fix until the contract suite is green and the Chapter 20 functional suite stays green. (4) Retro: map every finding to an OWASP item, a detection signal, and a control that would have prevented it by construction.

Acceptance Criteria.

- ☐ Every red finding exists as an executable contract test (no slide-deck-only findings).
- ☐ Cross-service audience-confusion exploit demonstrated and closed.
- ☐ Functional test suite remains green after hardening (security fixes must not break the contract).
- ☐ Retro document: findings → OWASP items → detection → permanent control.

Stretch Goals. Add mTLS between the two services via a local CA and show the token-confusion exploit’s remaining path; introduce a deliberate regression and measure time-to-detect with the new alerts.

17.10 Python Dependencies Appendix

The chapter’s `requirements.txt` (also in `code\ch16\requirements.txt`):

```
1 fastapi>=0.110
2 pyjwt[crypto]>=2.8
3 httpx>=0.27
4 pytest>=8
5 pydantic>=2.7
```

17.10.1 fastapi

Description. The ASGI framework hosting both builds; its dependency-injection system is where authentication and authorization compose, and its `TestClient` makes the whole suite in-process and offline.

Why included. Every listing targets FastAPI [Ram24]; pinning ≥ 0.110 keeps the Pydantic v2 integration and the security utilities used here.

Alternatives. Starlette directly (FastAPI’s substrate), Flask + flask-login-style extensions, or Litestar — the guard patterns in `authz.py` port unchanged.

Installation (PowerShell).

```
1 python -m pip install "fastapi>=0.110"
```

17.10.2 pyjwt[crypto]

Description. The de-facto Python JWT library: encode/decode with algorithm allow-lists, claim validation (`iss`, `aud`, `exp`, `nbf`), leeway, and — via the `[crypto]` extra — RSA/ECDSA verification through `cryptography`.

Why included. Listing 17.3.5 builds the JWKS-style pipeline on it: `algorithms=["RS256"]` is mandatory, `none` is rejected by construction, and `leeway` is a first-class parameter.

Alternatives. `authlib` (full OAuth/OIDC client and server — the better choice when you must *issue* tokens, not just verify); `python-jose` (historically common, less actively maintained).

Installation (PowerShell).

```
1 python -m pip install "pyjwt[crypto]>=2.8"
```

17.10.3 httpx

Description. The HTTP client (and FastAPI’s test transport substrate). Its pluggable `transport` seam is what lets the chapter simulate the internet with `MockTransport`.

Why included. Both SSRF halves depend on it: the vulnerable `fetcher` and the policy-wrapped `SafeFetcher` share one client interface, so the defense is a drop-in layer, not a rewrite.

Alternatives. `requests` (no transport injection, no HTTP/2, sync-only); `aiohttp` (async but a different ecosystem).

Installation (PowerShell).

```
1 python -m pip install "httpx>=0.27"
```

17.10.4 pytest

Description. The test runner; the exploit suite’s `-target` option uses `pytest_addoption` to switch between demonstration and contract modes.

Why included. Security requirements as acceptance criteria is the chapter’s thesis; `pytest` is where the criteria execute (Chapter 20 treats the harness itself).

Alternatives. `unittest` (stdlib, no fixtures/parametrize ergonomics); `nose2` (effectively unmaintained). For property-based adversarial input generation, add `hypothesis` later.

Installation (PowerShell).

```
1 python -m pip install "pytest>=8"
```

17.10.5 pydantic

Description. The validation library underneath FastAPI: typed models, `extra="forbid"/"ignore"` policies, strict types, and constraint fields (`le`, `max_length`) that turn resource ceilings into schema facts.

Why included. API3's fix (explicit wire schemas), API4's fix (schema-level ceilings), and API10's fix (upstream payload validation) are all Pydantic models [Col24]; pinning ≥ 2.7 keeps the v2 `model_validate/model_dump` API used here.

Alternatives. `marshmallow` (schema-first, more ceremony); `attrs+cattrs`; for policy-as-code beyond field filtering, look at policy engines in the `oso/Cedar` style — valuable when authorization rules outgrow per-role tables and need centralized, auditable evaluation.

Installation (PowerShell).

```
1 python -m pip install "pydantic>=2.7"
```

17.11 Chapter Summary & Key Takeaways

- The OWASP API Security Top 10 is mostly an *authorization* catalog: API1, API3, API5, API6, and API9 all fire after perfect authentication. Armor the object, the property, the function, and the flow — not just the credential.
- Token validation has one safe order: structure, algorithm allow-list, kid-keyed JWKS resolution, signature, `iss/aud/exp`, *then* claims-based authorization. Reading claims early is the root of the classic JWT exploits.
- Scopes are capability classes, not object permissions; object and property decisions belong in the resource server next to the data.
- mTLS (ideally SPIFFE/SPIRE-issued, auto-rotated) authenticates *workloads*; tokens carry *user/delegated* context. Strong platforms run both, each for its own job.

- SSRF is a systems problem: scheme/host/port allow-lists, DNS pinning, per-hop redirect re-validation, response caps, and platform egress rules — each layer independently sufficient.
- Threat modeling pays rent only when abuse cases become executable acceptance criteria; the exploit suite *is* the security requirements document.
- Secrets are rotated, vaulted, and scanned for — never committed, never logged, and treated as compromised the moment a scanner trips.
- Every claim in this chapter runs: 10 exploit demonstrations, 10 reds, 10 greens, 44 tests, zero network.

17.12 Quick-Reference Card

17.13 Standardized Evidence Addendum

17.13.1 Benchmark Snapshot Template

17.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

17.13.3 Cost and Latency Budget Snapshot

17.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

17.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

17.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

17.17 Cross-Chapter Decision Card

17.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

17.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Item	Name	One-line mechanism
API1	Broken Object Level Authorization	Object id accepted without an ownership/tenancy check.
API2	Broken Authentication	Weak, forgeable, replayable, or unvalidated credentials.
API3	Broken Object Property Level Authorization	Mass assignment on write; excessive data exposure on read.
API4	Unrestricted Resource Consumption	No ceilings on size, rate, CPU, memory, or third-party cost.
API5	Broken Function Level Authorization	Privileged routes guarded by obscurity, not role checks.
API6	Unrestricted Access to Sensitive Business Flows	Automatable high-value flows (redeem, dispatch, purchase).
API7	Server-Side Request Forgery	Server fetches attacker-chosen URLs into private networks.
API8	Security Misconfiguration	Debug surfaces, wildcard CORS, stack traces, defaults.
API9	Improper Inventory Management	Shadow, legacy, and unpatched versions still routable.
API10	Unsafe Consumption of APIs	Third-party responses trusted without validation.

Table 17.1: OWASP API Security Top 10 (2023) [OWA23]: mechanism summary. Every row is executable in this chapter’s exploit suite.

STRIDE	Threat to /v1/robots/{id}	PATCH	Mitigation (this chapter)
S	Forged or replayed bearer token.		RS256 + JWKS + iss/aud/exp pipeline.
T	Mass assignment of <code>owner_id</code> , <code>service_key</code> .		<code>extra="forbid"</code> schema + role-keyed write policy.
R	Owner denies changing robot state.		Audit log: actor, before/after, request id.
I	Response leaks secret fields of the object.		Role-keyed read projection; no raw-row serialization.
D	Patch floods; oversized bodies.		Body-size caps; per-principal token bucket.
E	Operator token calling admin functions.		Function-level scope + role checks, default-deny.

Table 17.2: STRIDE per endpoint: worked row set for the robot patch route. Every mitigations cell names an executable control, not a policy aspiration.

OWASP item	Mitigation matrix (control → detection)
API1 BOLA	Guard every object access; unguessable ids as depth → alert on cross-owner reads and id walks.
API2 Broken AuthN	RS256/ES256 + JWKS + iss/aud/exp; throttle auth endpoints → 401 spikes, alg/kid anomalies.
API3 BOPLA	<code>extra="forbid"</code> writes + role-keyed read/write field policies → unknown-field 422 spikes.
API4 Resource	Schema ceilings, body caps, cost budgets, 429s → p99 latency and per-principal work units.
API5 BFLA	Scope+role checks in dependencies; default-deny → 403 clusters on admin routes.
API6 Flow abuse	Per-principal velocity limits; business invariants as code → funnel anomalies per flow.
API7 SSRF	Allow-lists, DNS pinning, redirect re-validation, egress rules → egress destination histograms.
API8 Misconfig	Debug compiled out, opaque 500s, exact CORS, security headers → route/config drift scans.
API9 Inventory	Spec-driven inventory, gateway default-deny, EOL enforcement → traffic to unlisted routes.
API10 Upstream	Schema-validate upstream, copy vetted fields only → upstream validation-failure spikes.

Table 17.3: OWASP API Top 10 mitigation matrix [OWA23].

#	Token validation checklist (in order)
1	Three segments; base64url decodes; header/payload are JSON.
2	alg in server allow-list; none rejected outright.
3	kid resolves in your trusted JWKS map (one rate-limited refetch on miss).
4	Signature verifies against the resolved key.
5	exp present and in the future within leeway; nbf/iat sane.
6	iss exact match; aud contains this service.
7	Claims type-checked; only now mapped to a principal.
8	Scope check for the function; guard check for the object; field policy for the properties.

Table 17.4: The only safe token-validation order.

Header	API value
X-Content-Type-Options	nosniff
X-Frame-Options	DENY (or CSP frame-ancestors 'none')
Content-Security-Policy	default-src 'none' for JSON; default-src 'self' on docs routes
Cache-Control	no-store on authenticated responses
Strict-Transport-Security	max-age=63072000; includeSubDomains (once TLS is universal)
Access-Control-Allow-Origin	Exact origins only; never * with credentials

Table 17.5: Secure-headers table for JSON APIs.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 17.6: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 17.7: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 17.8: Decision card to complete per chapter.

Chapter 18

Deployment & Infrastructure: Containers, Gateways & Edge

Every chapter before this one ended at the same invisible boundary: the API worked *on your machine*. This chapter is about the far harsher question of whether it keeps working while the machine underneath it is being replaced, scaled, drained, upgraded, or killed. Deployment is not a soup of YAML that ops people sprinkle on finished code; it is the continuation of the API contract by other means. A client does not experience your code — it experiences the behavior of your code *while infrastructure is changing around it*. The difference between a platform and a pile of services is whether a routine Tuesday-afternoon deploy is indistinguishable, from the caller’s side, from no deploy at all.

Three ideas organize everything that follows. First, the *immutable artifact*: a container image is the unit of deployment, and its quality is a security and reliability property, not a cosmetic one. Multi-stage builds, pinned digests, non-root users, and read-only filesystems are not hygiene theater; they shrink the attack surface to a fraction of a convenience image and make every byte in production traceable to a commit. Second, *process discipline*: inside the container your process is PID 1, which makes signal handling your responsibility, and termination is a four-act protocol — stop accepting, drain in-flight work, time out, exit — that must be choreographed with the platform’s *preStop* hooks, `terminationGracePeriodSeconds`, and the embarrassingly long lag between a Pod dying and the rest of the cluster noticing. Third, *progressive delivery*: you never actually deploy a version; you migrate traffic between versions, gated by metrics, with a rollback path rehearsed before it is needed [Nyg18; Bey+16].

We treat the API gateway as what it operationally is — a policy enforcement point that absorbs cross-cutting concerns (JWT validation, rate limiting, request transformation, canary routing) so the service code does not — and we draw the boundary lines between gateway, ingress, and service

mesh precisely, because muddling them produces double TLS terminations, duplicated rate limits, and unowned failures. On Kubernetes we go past the tour and into the semantics that bite: the exact failure behavior of startup versus readiness versus liveness probes (demonstrated with a deterministic simulator, not asserted), the CPU-throttling latency trap that `limits` create, HPA stabilization windows, PodDisruptionBudgets, and the rolling-update arithmetic of `maxSurge` and `maxUnavailable`. We close the loop at the edge: TLS termination placement, certificate automation, CDN caching rules for APIs, WAF essentials, and the north-south/east-west distinction that decides which of these boxes a packet even visits.

Everything runnable is offline, deterministic, and localhost-only: a graceful-shutdown proof that catches a termination signal *during* a slow request and shows the drain completing; a probe-semantics simulator that replays a boot, a dependency outage, and a deadlock against the kubelet's rules; and a canary analyzer that runs Wald's sequential probability ratio test over synthetic metric series and promotes or rolls back on evidence, not vibes. The Dockerfile and the Kubernetes manifests are complete and production-shaped but are *reviewed line by line*, not applied — reading infrastructure critically is the skill this chapter exists to teach.

18.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Author a production multi-stage Dockerfile for a FastAPI service: separate build and runtime stages, choose between slim and distroless bases, run as a non-root `USER`, control the build context with `.dockerignore`, order instructions for layer-cache reuse, pin base images by digest, and wire `HEALTHCHECK` and read-only filesystems — and justify every line as attack-surface or reproducibility engineering.
- LO-2.** Explain the PID 1 problem and implement the graceful-shutdown sequence — stop accepting, drain, timeout, exit — including uvicorn's `SIGTERM` behavior, `preStop` hooks, and load-balancer connection draining, and prove drain semantics empirically with the chapter's signal-driven demo.
- LO-3.** Compare Kong, Envoy, and NGINX as API gateways across routing, authentication offload, rate limiting, transformation, and canary support; position gateway versus ingress

versus service mesh; and argue for the gateway as the policy enforcement point for north-south traffic.

- LO-4.** Design Kubernetes workloads for APIs with correct probe semantics (startup gates liveness/readiness; readiness removes endpoints without restarts; liveness restarts only process-local death), resource requests and limits that avoid throttle-induced latency, HPA metrics and stabilization windows, PodDisruptionBudgets, and rolling-update parameters chosen from replica-count arithmetic.
- LO-5.** Select and operate a progressive-delivery strategy — blue-green, canary with traffic-splitting math and metric-gated promotion, or feature flags — including automated rollback signals and the expand-and-contract database discipline that makes rollbacks possible at all (Chapter 10).
- LO-6.** Place TLS termination deliberately (edge, re-encrypt, passthrough), automate certificate lifecycles, define CDN cache behavior for APIs (what to cache, cache keys, purge semantics), apply WAF essentials, and classify any hop as north-south or east-west.

Prerequisites

This chapter assumes Chapters 0–16, with four load-bearing dependencies. Chapter 7 (Building APIs with FastAPI) supplies the Orders API we containerize and the `TestClient`/uvicorn mechanics the shutdown demo leans on [Ram24]. Chapter 15 (Rate Limiting) supplies the algorithms the gateway offloads and the header contract the edge must preserve. Chapter 16 (Microservices & Resilience) supplies timeouts, retries, and circuit breakers — the behaviors that make connection draining and endpoint-removal lag survivable rather than fatal [New21]. Chapter 17 (API Security Engineering) supplies the JWT validation, secret handling, and transport-security vocabulary reused at the gateway and the edge. The expand-and-contract migration discussion calls back to Chapter 10 (Versioning & Evolution). The Twelve-Factor App’s process and config factors [Wig17] are referenced throughout; a passing familiarity with the release-engineering chapter of the Google SRE book [Bey+16] helps but is not required.

Setup

All runnable listings execute offline on Windows with Python 3.11+ and localhost only. One-time setup from PowerShell in a scratch directory:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install fastapi uvicorn httpx pytest pyyaml
```

The Dockerfile and Kubernetes manifests are reviewed, not applied, so no Docker daemon or cluster is required. If you want to go further, Section 18.10 gives `winget` install lines for the optional toolchain (`docker`, `kubect1`, `k6`).

18.2 Deep Technical Mechanics & Architectural Concepts

18.2.1 The Immutable Artifact: What a Container Image Actually Is

A container image is not a lightweight virtual machine and it is not a zip file of your app. It is a *content-addressed stack of filesystem layers* plus a small JSON manifest: each layer is a diff, each diff is addressed by the SHA-256 of its contents, and the image itself is addressed by the digest of the manifest. Two operational facts follow immediately, and both are security facts before they are convenience facts. First, layers are shared and cached: two images built from the same base literally share those bytes on disk, which is why base-image choice dominates image size and why instruction ordering dominates build time. Second, a *tag* like `3.12-slim` is a mutable pointer a registry can retarget at any time, while a *digest* (`python@sha256:...`) names exactly one manifest forever. Pinning by digest is the difference between “we deployed what we tested” and “we deployed whatever the tag pointed at that morning.”

Definition 18.1 (Immutable artifact). An *immutable artifact* is a deployable unit whose contents are fixed at build time and identified by a cryptographic digest, so that every running instance can be traced to exactly one build, and a rollback is a pointer change to a previous digest rather than a new build.

Multi-stage anatomy. A multi-stage Dockerfile contains at least two `FROM` instructions. The first stage — the *build* stage — is allowed to be fat: compilers, headers, package managers, and your full dependency resolver live there, because that stage is discarded. The final stage — the *runtime* stage — receives only copied artifacts: a virtual environment or a `site-packages` tree, your source, nothing else. The attack-surface math is simple and brutal: every binary in the runtime image is a thing an attacker with remote code execution can invoke, so a runtime stage containing `gcc`,

`curl`, and a shell is a gift; a runtime stage containing `python`, your wheels, and no shell at all forces an attacker to bring their own tooling through whatever hole they already have.

Base-image spectrum. The practical spectrum runs from `python:3.12` (full Debian, ~1 GB, everything present, everything exposable) through `python:3.12-slim` (~150 MB, no compiler, a shell and `coreutils` remain) to *distroless* (gcr.io/distroless/python3: Python, `glibc`, CA certificates, and *nothing else* — no shell, no package manager, typically ~50–80 MB for a FastAPI service). Distroless is the strongest default for interpreters that need no OS packages; slim is the pragmatic choice when you need one or two system libraries or when your organization’s vulnerability scanner needs an OS package database to reason about. The wrong default is the full image “because something might need it” — something might also exploit it.

Non-root, read-only, health-checked. Three more lines of the same argument. Containers on Docker and Kubernetes run as UID 0 by default; a process that escapes as container-root is far closer to host root than one that first has to escalate from an unprivileged UID, so the runtime stage ends with `USER` set to a created non-root account. A `readOnlyRootFilesystem` (or `docker run --read-only`) turns whole exploit classes — webshell drops, config tampering, binary replacement — into no-ops, at the price of explicitly mounting writable `tmpfs`s for the few paths that genuinely need writes. `HEALTHCHECK` (or its Kubernetes successor, the probe, which subsumes it there) encodes the service’s own opinion of its health into the artifact instead of leaving it tribal.

Layer-caching discipline. BuildKit caches each instruction’s result and invalidates the cache from the first changed instruction downward. The discipline is therefore: order instructions from least-frequently-changed to most-frequently-changed (base image, system packages, dependency manifests, dependency install, *then* source), and control what `COPY . .` can even see with `.dockerignore` — excluding `.git`, virtual environments, test artifacts, and any secrets that wandered into the working tree. An unignored `.env` copied into an image layer is a credential leak that survives deletion, because the layer remains in the image history.

Warning

A file deleted in a later `RUN rm` is still present in the earlier layer and recoverable from the image. “Install, use, delete in one `RUN`” is a size optimization; “copy secret, delete secret in a later layer” is a breach. Secrets never enter the build context or a layer — use BuildKit `--mount=type=secret` for build-time needs and runtime secret mounts for deploy-time needs (Chapter 17).

18.2.2 Process Discipline: PID 1, Signals, and the Four-Act Shutdown

In a Linux namespace, PID 1 is special in two ways that matter to an API author. First, orphaned processes are re-parented to PID 1, which is responsible for reaping zombies; second, and more important here, a process only receives a signal it has not installed a handler for if that signal’s default disposition applies — and for PID 1 the kernel *only delivers signals for which a handler is explicitly installed*. An application that spawns a shell wrapper that spawns the server, or a naive process that never calls `signal.signal`, can therefore be unreachable by `SIGTERM`: Kubernetes asks politely, waits `terminationGracePeriodSeconds`, and then issues `SIGKILL` — every in-flight request severed mid-write. The fixes are boring and absolute: use `exec` (or no shell at all — the `exec` form of `ENTRYPOINT/CMD`) so your server *is* PID 1, and let the server framework install proper handlers. `uvicorn` does: on `SIGTERM` it sets `should_exit`, stops accepting new connections, waits for in-flight requests up to `timeout_graceful_shutdown`, runs ASGI lifespan shutdown, and exits.

Definition 18.2 (Graceful shutdown sequence). *Graceful shutdown* is the ordered protocol: (1) **stop accepting** — close listeners and fail readiness so no new traffic is routed; (2) **drain** — allow in-flight requests to finish up to a budget; (3) **timeout** — abort stragglers that exceed the budget so the process cannot hang forever; (4) **exit** — release resources and terminate with a status the supervisor can interpret. Skipping act (1) routes new work to a dying instance; skipping act (2) drops requests clients paid for; skipping act (3) turns a deploy into a wedged rollout.

The platform choreography has two sharp edges. First, the **preStop hook**: when Kubernetes decides to terminate a Pod, it runs the container’s `preStop` hook *before* sending `SIGTERM`, and endpoint removal (updating the Service’s Endpoint objects, propagating to kube-proxy/iptables or the gateway’s load balancer) happens *concurrently* — not before, not after. A Pod can therefore receive `SIGTERM` while stale endpoints still route it fresh traffic for several seconds. The standard mitigation is a deliberate **sleep in preStop** (5–10 seconds is common) so the process keeps serving while the endpoint update fans out, then accepts the signal into a quieted listener. Second, **connection draining at the load balancer** is the same idea one tier up: cloud LBs and gateways mark the target “draining,” stop sending new connections, and wait for existing ones — your `terminationGracePeriodSeconds` must exceed the LB drain timeout plus your slowest request, or the platform kills the process with work still in flight. Section 18.5 replays the war story in which exactly one missing `preStop` sleep turned a routine rolling deploy into dropped checkouts.

Key Concept

Termination is a *race you must rig*, not an event you can order: SIGTERM, endpoint removal, and LB draining start concurrently. You rig it by making your server tolerant of late traffic (preStop sleep), tolerant of early signals (drain logic), and explicit about the budget (terminationGracePeriodSeconds > slowest request + preStop delay).

18.2.3 The API Gateway: Policy Enforcement Point for North-South Traffic

An API gateway is a reverse proxy with opinions. It terminates client connections at the edge of your platform and applies the cross-cutting concerns that must be uniform across services: authentication offload, rate limiting, request/response transformation, routing, and telemetry. The architectural argument is the one from Chapter 16: any policy implemented in N services is implemented N times, differently, and audited never; a policy enforced once at the gateway is uniform, versioned, and testable. **JWT validation at the edge** is the canonical example — the gateway verifies the signature, expiry, and issuer against cached JWKS and rejects forged or expired tokens before they consume a single cycle of service CPU, while services receive an already-authenticated identity in a trusted header (Chapter 17). **Rate limiting at the gateway** (Chapter 15) protects every upstream uniformly with one configuration language instead of N middleware copies.

Gateway vs. ingress vs. mesh. These three are conflated constantly and are distinguished by *what they terminate and where their policy boundary sits*. *Ingress* (the Kubernetes resource and its controllers) is the narrowest: a cluster-standard way to route north-south HTTP(S) to Services, with TLS termination and host/path rules — a gateway subset with a portable vocabulary. An *API gateway* is a product-tier edge: everything ingress does plus auth offload, rate limiting, transformation, developer portals, and analytics — the **policy enforcement point** for north-south traffic. A *service mesh* (sidecar or ambient proxies on every workload) governs *east-west* traffic: service-to-service mTLS, retries, circuit breaking, and telemetry *inside* the platform. The common production shape is all three in sequence: CDN/WAF at the provider edge, gateway at the platform edge, mesh between services — and the failure mode to avoid is enforcing the same concern in two tiers (e.g., rate limiting at gateway *and* mesh with different budgets) so neither is the authoritative budget.

18.2.4 Kubernetes for APIs: The Semantics That Bite

Anatomy in one paragraph. A *Deployment* declares desired state — N identical Pods from one Pod template — and a controller continuously reconciles reality toward it. A *Pod* is the scheduling unit: one or more containers sharing a network namespace (one IP) and volumes. A *Service* is a stable virtual IP and DNS name that load balances across the Pods matching its selector, via an endpoints controller that tracks *ready* Pods. Everything below is about the gaps between these declarations — the gaps are where requests die.

Probes: three questions, three consequences. The kubelet asks your container three distinct questions, and the failure consequence of each is different — this is the most cargo-culted area of Kubernetes, so we state it as a table and then *prove* it with the simulator in Section 18.3.

The **startup probe** exists because boot time and liveness budget are unrelated quantities. Without it, a slow-starting application (JVM warmup, model load, cache prime) is indistinguishable from a deadlocked one, and the liveness budget becomes a boot deadline you never meant to set; the simulator’s Scenario B crashes exactly this way. The **readiness/liveness split** encodes a recovery theory: readiness says “wait” (the problem may be elsewhere — restarting me cannot fix the database), liveness says “kill me” (the problem *is* me). **Endpoint-removal lag** completes the picture: readiness and termination only update the endpoints object; kube-proxy programming, DNS caches, and gateway target lists converge seconds later, which is why Section 18.2.2 insists on the preStop sleep and why clients still need retries (Chapter 16).

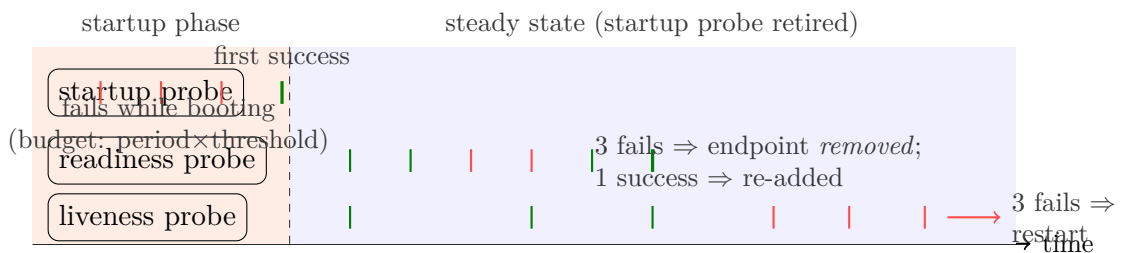


Figure 18.1: Probe lifecycle. The startup probe alone is evaluated during boot, protecting slow starts from the liveness budget. After its first success, readiness (routing consequence) and liveness (restart consequence) run on their own periods.

Resources: requests schedule, limits throttle. Kubernetes uses **requests** for scheduling (bin-packing: a node must have the requested CPU/memory free) and as the QoS input, and **limits** as hard ceilings with two very different enforcement mechanisms. Exceed the memory limit

and the kernel OOM-kills the container — violent, visible, logged. Exceed the CPU *limit* and nothing fails: the kernel’s Completely Fair Scheduler *throttles* the cgroup for the remainder of each 100 ms quota period. The pitfall is that throttling is quantized: a service with 8 worker threads and a 400m CPU limit can consume its 40 ms-of-quota-per-period in the first 5 ms of a burst, then freeze every thread — including the one holding a request — for up to 95 ms. The symptom is p99 latency inflated by *exactly* multiples of 100 ms while average CPU sits far below the limit, which is why CPU-limit throttling is the classic cause of “mystery tail latency” in Go and Python services alike. Operational guidance: always set CPU *requests* (they also weight CFS shares under contention), set memory requests = limits (memory cannot be throttled, so a limit without a matching request is pure risk), and either omit CPU limits on latency-sensitive services or set them generously above p99 usage and alert on `container_cpu_cfs_throttled_periods_total`.

HPA: metrics, windows, and flapping. The Horizontal Pod Autoscaler computes `desired = ⌈current × currentMetric/targetMetric⌉` every sync period from resource metrics (CPU/memory utilization against *requests*), custom metrics (requests per second per pod), or external metrics (queue depth). Two details separate operators from tourists: utilization is computed against *requests*, so a service with inflated requests never autoscales no matter how hot it runs; and the default `scaleDown` stabilization window (300 s) exists because scaling down kills in-flight connections — the window makes downscale decisions from the *highest* recommendation in the window, trading money for connection stability. Downscale *rate* limits (*policies*: e.g., remove at most 1 pod or 10% per 60 s) are the second half of the same idea.

PDBs and rolling updates. A `PodDisruptionBudget` states how many Pods of a Service may be *voluntarily* unavailable (`minAvailable` or `maxUnavailable`); node drains and cluster upgrades respect it, crashes do not (they are involuntary). Without a PDB, `kubectl drain` will happily evict every replica of your API simultaneously. Rolling updates trade capacity for safety via two parameters: `maxSurge` (extra Pods above desired allowed during the rollout — needs quota headroom) and `maxUnavailable` (desired Pods allowed to be down). With $R=4$ replicas, `maxSurge: 1`, `maxUnavailable: 0` keeps serving capacity at ≥ 4 throughout at the cost of one extra pod’s resources; with `maxUnavailable: 1` you never exceed R but serve a window at $R-1$. The interaction that bites: HPA plus a bad PDB (`minAvailable` equal to replicas) deadlocks drains, and a readiness gate that never passes (new version’s probe path renamed) turns `maxUnavailable: 0` into a rollout that cannot proceed — which is the safe failure, and why `progressDeadlineSeconds` exists to fail the rollout loudly instead of wedging it silently.

18.2.5 Progressive Delivery: Migrating Traffic, Not Versions

Blue-green keeps two full environments; the cutover is an atomic traffic switch (DNS, LB target group, or Service selector), and rollback is the same switch flipped back. Its virtues are instant rollback and a fully warm target; its costs are 2x infrastructure and the cold-start problem merely moved (the idle environment rots unless exercised). **Canary** releases the new version to a fraction of traffic and promotes through steps — 1%, 5%, 25%, 50%, 100% is a common ladder — comparing SLIs at each step. The traffic-splitting math has two subtleties. First, error budgets are ratios, and ratios need volume: at a 0.4% baseline error rate, a 1% canary serving 100 requests shows zero errors with probability $\approx 67\%$ even if the canary is perfectly healthy — small slices are statistically blind, so canary steps must be sized by *requests*, not just percentages, which is why the analyzer in Section 18.3 works in fixed-size batches. Second, weighted splitting must be *stable* for stateful-feeling flows: session-affinity-aware hashing (on API key or tenant) prevents one client oscillating between versions mid-workflow, which matters whenever response shapes differ (Chapter 10).

Definition 18.3 (Metric-gated promotion). A *metric-gated promotion* advances traffic to the next canary step only when a pre-registered analysis over a fixed window passes — and rolls back automatically when regression evidence crosses a threshold. The gate is a statistical decision procedure (here: Wald’s SPRT with error budgets α for false-promote and β for false-rollback), not a human watching a dashboard; the human designed the procedure.

Automated rollback signals worth wiring: error-rate delta versus baseline (the SPRT in Listing 18.3.6), latency quantile guardrails (p95/p99 against a hard factor of baseline), and saturation signals (CPU throttling, connection-pool exhaustion). Signals that *look* useful and are not: average latency (hides tails), absolute error counts (denominator-blind), and business metrics on short windows (too noisy to gate on minute timescales).

Feature flags vs. deployment strategies are complements, not rivals: deployments change *which binary* runs, flags change *which code paths* execute inside one binary. Flags decouple release from deploy (ship dark, enable per-tenant, kill-switch instantly) at the price of combinatorial code paths and flag-debt; canaries decouple blast radius from deploy speed at the price of two contract versions in flight simultaneously. Both, therefore, rest on the same foundation: **database compatibility**. The expand-and-contract discipline of Chapter 10 is what makes any of this safe: migrations run in the *expand* step (add column/table, never rename or drop), both binary versions tolerate both shapes, and *contract* (drop, rename) ships only after every

old binary is gone. Deploying a destructive migration *before* the compatible code is the one rollout error rollback cannot fix: the old binary comes back, the column it needs does not. We name this rule *migrate-expand*, *code*, *migrate-contract* and treat violations as release blockers.

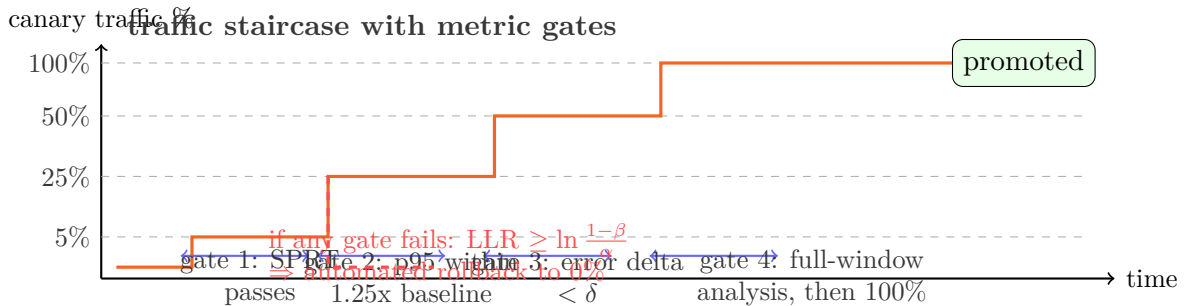


Figure 18.2: Canary rollout timeline. Each step holds for a fixed analysis window while the gate (error-rate SPRT plus latency guardrails, Listing 18.3.6) evaluates; evidence crossing the upper boundary rolls back automatically, the lower boundary promotes.

18.2.6 Edge & TLS: Where Bytes Get Terminated

TLS termination placement is a three-way decision with real consequences, not a default. *Edge termination* (decrypt at CDN/LB/gateway, plaintext inside) minimizes certificate sprawl and enables L7 policy, but leaves the last hop plaintext — acceptable only on networks you genuinely control and increasingly non-compliant. *Re-encryption* (terminate at the edge, open a *second* TLS session to the backend) keeps policy visibility and encrypted transit at the price of two certificate lifecycles. *Passthrough* (L4 forwarding of the encrypted stream to the service) gives end-to-end encryption with zero edge visibility — no WAF, no L7 routing, no JWT offload. The common production answer for APIs: terminate at the edge with automated public certificates, re-encrypt to services with an internal CA (a mesh gives you this as mTLS “for free”), and reserve passthrough for regulated payloads. **Certificate automation** is non-negotiable either way: ACME (Let’s Encrypt) at the edge, cert-manager or mesh-issued certificates inside — a manually renewed certificate is an outage with a calendar date (Chapter 17).

CDN behavior for APIs is mostly about what *not* to cache. Cacheable: truly anonymous, truly idempotent GETs with explicit **Cache-Control** (public reference data, OpenAPI documents, JWKS). Never cacheable: authenticated responses (unless the cache key includes the auth scope — rarely worth it), anything with **Set-Cookie**, and personalized content.

The **cache key** discipline: default keys ignore query strings and headers, so filtered list endpoints (Chapter 9) must either opt out or list their query/header inputs in the key; **Vary** mishandling is the classic tenant-data-leak vector. **Purge** semantics matter more than TTLs for mutable resources: if you cannot purge by key or tag on write, keep TTLs in seconds and accept staleness as a documented contract (Chapter 11’s caching half). **WAF essentials**: managed rule sets (OWASP core rules) in front of the gateway, tuned to block mode gradually (detect first — API payloads trip SQLi rules on legitimate JSON), plus body-size limits and request-rate anomaly rules as the coarse outer net before the gateway’s precise per-key limiting (Chapter 15).

Definition 18.4 (North-south vs. east-west traffic). *North-south* traffic crosses the platform boundary: client to edge, edge to gateway, gateway to first service. *East-west* traffic stays inside: service to service, service to database. The distinction selects the control point: gateways, WAFs, and CDNs see only north-south; meshes, internal firewalls, and mTLS see east-west. A policy applied at the wrong tier (e.g., per-client rate limits enforced only east-west) protects nothing, because the abuse never reaches it.

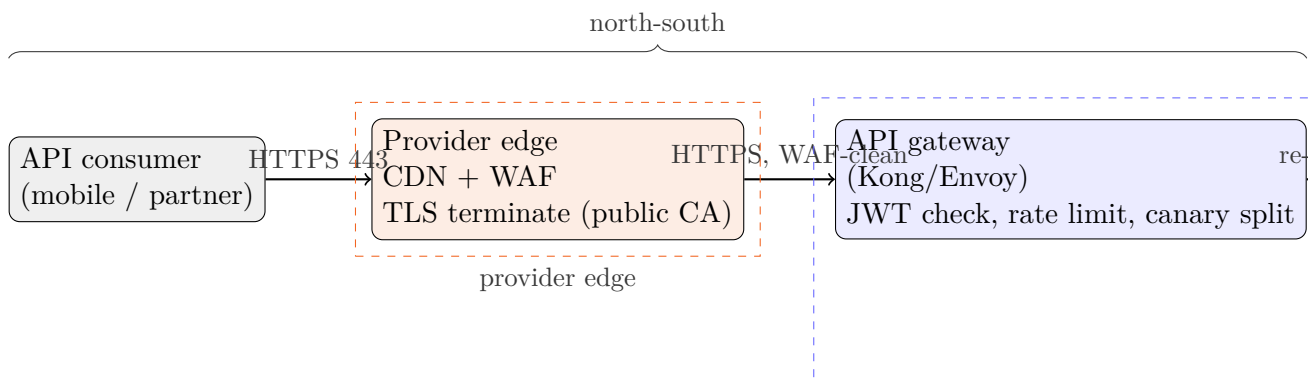


Figure 18.3: Request path through edge, gateway, and mesh. TLS terminates at the provider edge (public certificate, WAF inspection), is re-encrypted to the gateway, and becomes mTLS east-west; each box owns exactly the policies its position can see.

18.3 Production-Ready Code Implementation

This section contains every artifact the chapter discusses: the Dockerfile and its build-context guard, the full Kubernetes manifest set (reviewed line by line, never applied), and the three runnable, offline, deterministic programs — the graceful-shutdown proof, the probe-semantics simulator, and the canary analyzer.

18.3.1 Listing 17.1 — Production Multi-Stage Dockerfile

```

1  # ----- Stage 1: build (discarded) -----
2  FROM python:3.12-slim AS build
3  ENV PIP_NO_CACHE_DIR=1 \
4      PIP_DISABLE_PIP_VERSION_CHECK=1
5  WORKDIR /build
6  RUN python -m venv /opt/venv
7  ENV PATH="/opt/venv/bin:$PATH"
8  COPY requirements.txt .
9  RUN pip install -r requirements.txt
10
11 # ----- Stage 2: runtime (shipped) -----
12 # Pin by digest for real releases; resolve it with:
13 #   docker buildx imagetools inspect python:3.12-slim
14 FROM python:3.12-slim AS runtime
15 ENV PYTHONDONTWRITEBYTECODE=1 \
16     PYTHONUNBUFFERED=1 \
17     PATH="/opt/venv/bin:$PATH"
18 RUN groupadd --system app \
19     && useradd --system --gid app --uid 10001 --no-create-
20         home app
21 WORKDIR /app
22 COPY --from=build /opt/venv /opt/venv
23 COPY app/ ./app/
24 USER app
25 EXPOSE 8080
26 HEALTHCHECK --interval=30s --timeout=3s --start-period
27     =15s --retries=3 \
28     CMD ["python", "-c", "import urllib.request,sys; \
29         sys.exit(0 if urllib.request.urlopen('http
30             ://127.0.0.1:8080/healthz', timeout=2).status
31             == 200 else 1)"]

```

```

28 ENTRYPOINT ["python", "-m", "uvicorn", "app.main:app",
    \
29     "--host", "0.0.0.0", "--port", "8080"]

```

Line-by-line, with the reason each line earns its place:

- **Stage split** (AS build / AS runtime): the build stage owns `pip` and the resolver; only the finished virtual environment crosses into the runtime stage via `COPY --from=build`. Compilers, caches, and package managers never ship.
- `PIP_NO_CACHE_DIR=1`: the pip cache is pure dead weight in a layer — every cached wheel doubles the cost of the layer that installed it.
- **venv at a fixed path** (`/opt/venv`): a virtual environment is a relocatable, root-owned-by-build artifact; the runtime stage sets `PATH` once and never invokes pip again. `PYTHONDONTWRITEBYTECODE` plus a read-only root filesystem agree: nothing writes `.pyc` at runtime.
- **Cache ordering**: `COPY requirements.txt` and the `pip install` precede `COPY app/` so that a source-only change reuses the entire dependency layer; a dependency change correctly invalidates from the manifest copy downward.
- **Digest pinning** (commented): tags are mutable registry pointers; a digest is a content hash. Production pipelines resolve the digest at build time (`imagetools inspect`) and record it, so “what we tested” and “what is running” are the same manifest.
- **Non-root**: a dedicated system user (fixed UID 10001 so Kubernetes `runAsUser` can assert the same number) is the last instruction before `ENTRYPOINT`; everything after `USER app` runs unprivileged.
- **HEALTHCHECK**: the image carries its own health opinion — `stdlib urllib` against `/healthz`, no `curl` dependency (slim images lack it; installing it to run a health check would grow the attack surface to check the attack surface). Kubernetes ignores `HEALTHCHECK` in favor of probes; Docker and compose honor it.
- **Exec-form ENTRYPOINT**: the JSON array form means `uvicorn` is PID 1 — no shell wrapper to swallow `SIGTERM` (Section 18.2.2). Binding `0.0.0.0` is correct *inside* a container: the network namespace is already the isolation boundary.

The build-context guard, without which the cache ordering above is

fiction (a stray log file changing would invalidate the source layer) and secrets can leak into layers:

```

1 # .dockerignore
2 .git
3 .venv
4 __pycache__/
5 *.pyc
6 .pytest_cache/
7 tests/
8 datasets/
9 *.md
10 .env
11 .env.*
12 Dockerfile
13 .dockerignore

```

Runtime hardening completes the artifact: `docker run -read-only-tmpfs /tmp:rw,noexec,nosuid,size=16m ...` gives the process a read-only root filesystem with a tiny, non-executable scratch space; the Kubernetes equivalent (`readOnlyRootFilesystem: true` plus an `emptyDir` mounted at `/tmp`) appears in Listing 18.3.2.

Example

Size as a report card. The same Orders API built three ways on the authors' reference machine: full `python:3.12` base: ~1.02 GB, 3 high-severity and 41 lower findings from a standard scanner. `python:3.12-slim` multi-stage (Listing 17.1): ~165 MB, 0 high, 9 low. Distroless runtime: ~78 MB, 0 high, 2 low — and no shell, which flips several scanner findings from “exploitable” to “unreachable.” Image size is not vanity; it is transfer time, cold-start time, and vulnerability count in one number.

18.3.2 Listing 17.2 — Kubernetes Manifest Set (Reviewed, Not Applied)

The set below deploys the Orders API with a stable track and a canary track. Read each block with the justifications; the design review in Section 18.5 walks the same fields as a meeting exercise.

Deployment (stable track)

```

1 apiVersion: apps/v1
2 kind: Deployment

```

```
3 metadata:
4   name: orders-api
5   labels: {app: orders-api, track: stable}
6 spec:
7   replicas: 4
8   revisionHistoryLimit: 5
9   progressDeadlineSeconds: 600
10  strategy:
11    type: RollingUpdate
12    rollingUpdate:
13      maxSurge: 1
14      maxUnavailable: 0
15  selector:
16    matchLabels: {app: orders-api, track: stable}
17  template:
18    metadata:
19      labels: {app: orders-api, track: stable, version:
20        "2.4.1"}
21    spec:
22      terminationGracePeriodSeconds: 45
23      securityContext:
24        runAsNonRoot: true
25        runAsUser: 10001
26        fsGroup: 10001
27      containers:
28        - name: api
29          image: registry.northwind-robotics.example/
30            orders-api:2.4.1
31          ports:
32            - {name: http, containerPort: 8080}
33          env:
34            - {name: UVICORN_WORKERS, value: "2"}
35            - {name: UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN,
36              value: "30"}
37            - name: DATABASE_URL
38              valueFrom:
39                secretKeyRef: {name: orders-api-secrets
40                  , key: database-url}
41          resources:
42            requests: {cpu: 250m, memory: 256Mi}
43            limits: {memory: 256Mi}      # no CPU limit:
44              throttle pitfall
45          securityContext:
46            allowPrivilegeEscalation: false
47            readOnlyRootFilesystem: true
48            capabilities: {drop: ["ALL"]}
```



```

44     volumeMounts:
45       - {name: tmp, mountPath: /tmp}
46     lifecycle:
47       preStop:
48         sleep: {seconds: 8}          # cover
49         endpoint-removal lag
50     startupProbe:
51       httpGet: {path: /healthz/startup, port:
52         http}
53       periodSeconds: 5
54       failureThreshold: 12          # budget: 60 s
55       to boot
56     readinessProbe:
57       httpGet: {path: /healthz/ready, port: http}
58       periodSeconds: 5
59       failureThreshold: 3
60     livenessProbe:
61       httpGet: {path: /healthz/live, port: http}
62       periodSeconds: 10
63       failureThreshold: 3
64   volumes:
65     - name: tmp
66       emptyDir: {}

```

Field by field: **replicas:** 4 pairs with **maxSurge:** 1, **maxUnavailable:** 0 — capacity never dips below 4 during a rollout, at the price of one surplus pod; the arithmetic for other sizes is in Table 18.3. **progressDeadlineSeconds:** 600 makes a wedged rollout (e.g., a readiness path renamed in the new version) *fail loudly* after ten minutes instead of hanging silently. **revisionHistoryLimit:** 5 keeps five old ReplicaSets for instant rollback without unbounded clutter. **terminationGracePeriodSeconds:** 45 is the shutdown budget: 8 s preStop sleep + 30 s unicorn graceful timeout + margin; the two values must be set together — a 30 s default grace against a 45 s drain need is a dropped-request generator. The pod-level **securityContext** pins the UID the image was built with (mutual assertion: Kubernetes refuses a container whose image tries to run as root). **resources:** memory request equals limit (memory cannot be throttled; a mismatch is pure OOM risk), CPU has a request (scheduling and CFS shares) but *no limit* — the throttling pitfall of Section 18.2.4. **DATABASE_URL** comes from a Secret reference; nothing secret is ever a literal in a manifest (Chapter 17). **readOnlyRootFilesystem:** true plus the **emptyDir** at /tmp is the runtime-fleet version of the Dockerfile's discipline. The **preStop** sleep uses the native sleep action (Kubernetes ≥ 1.30); on older clusters the equivalent is **exec:** {**command:** ["sh", "-c", "sleep 8"]}, which requires a shell in the image — one concrete reason to choose slim over distroless. The three probes implement

Table 18.2 verbatim: startup gates the others with a 60-second boot budget; readiness is the only probe allowed to remove endpoints; liveness checks a dependency-free path so only process-local death can restart the pod.

Service

```

1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: orders-api
5 spec:
6   selector:
7     app: orders-api          # no 'track' label: spans
7     stable AND canary
8   ports:
9     - {name: http, port: 80, targetPort: http}

```

The selector’s omission is the canary mechanism: the Service load balances across both tracks in proportion to ready pod count, so one canary pod against four stable pods receives $\approx 20\%$ of traffic. Pod-count splitting is the coarse, zero-infrastructure option; arbitrary weights (and header/tenant-based pinning) require gateway weighted clusters — Envoy’s first-class feature in Table 18.1.

HorizontalPodAutoscaler

```

1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: orders-api
5 spec:
6   scaleTargetRef: {apiVersion: apps/v1, kind:
6     Deployment, name: orders-api}
7   minReplicas: 4
8   maxReplicas: 20
9   metrics:
10    - type: Resource
11      resource:
12        name: cpu
13        target: {type: Utilization, averageUtilization:
13          70}
14   behavior:
15     scaleUp:
16       stabilizationWindowSeconds: 0
17     policies:

```

```

18         - {type: Percent, value: 100, periodSeconds:
19           60}
19     scaleDown:
20       stabilizationWindowSeconds: 300
21     policies:
22       - {type: Pods, value: 1, periodSeconds: 60}

```

Utilization targets CPU *request* (70% of 250m), which is why the request must reflect reality. The asymmetric behavior is the point: scale up instantly (a flash crowd does not wait five minutes) but scale down through a 300-second stabilization window and at most one pod per minute, because every scale-down is a termination event with in-flight requests attached — the HPA’s politeness and the preStop hook are two halves of the same courtesy. `minReplicas: 4` interacts with the PDB below: voluntary disruptions always leave three.

PodDisruptionBudget

```

1 apiVersion: policy/v1
2 kind: PodDisruptionBudget
3 metadata:
4   name: orders-api
5 spec:
6   minAvailable: 3
7   selector:
8     matchLabels: {app: orders-api}

```

`minAvailable: 3` means node drains and cluster upgrades must leave three pods serving; an eviction that would violate the budget *blocks* (the drain waits, it does not force). The classic misconfiguration is `minAvailable` equal to the replica count, which deadlocks every drain; the second classic is omitting the PDB entirely, which lets `kubect1 drain` evict the whole fleet of a single-replica API during a kernel upgrade.

Canary Deployment (variant)

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: orders-api-canary
5 spec:
6   replicas: 1                                # 1 canary vs 4
7     stable = ~20% traffic
7   strategy:
8     type: RollingUpdate

```

```

 9     rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
10 selector:
11   matchLabels: {app: orders-api, track: canary}
12 template:
13   metadata:
14     labels: {app: orders-api, track: canary, version:
15             "2.5.0-rc.1"}
16   spec:
17     # identical pod spec to the stable Deployment
18     except:
19     containers:
20       - name: api
21         image: registry.northwind-robotics.example/
22           orders-api:2.5.0-rc.1
23       # ... probes, resources, securityContext,
24       lifecycle unchanged ...

```

The canary Deployment differs in exactly three places: name, replica count, and the `track/version` labels that keep the two ReplicaSets distinct and let observability slice metrics by version (Chapter 19). Promotion is `kubectl scale` steps on the two Deployments (2/3, 3/2, 4/1) gated by Listing 18.3.6’s analyzer; rollback is scaling the canary to zero — fast precisely because the expand-and-contract schema discipline of Chapter 10 guaranteed the old binary still understands the data.

18.3.3 Listing 17.3 — The Server Under Test (`orders_server.py`)

The FastAPI Orders API in its deployment-relevant form: in-flight request tracking via middleware (the ledger that lets us *prove* a drain), a fast write path, a deliberately slow five-second report endpoint to be caught mid-flight, and a probe endpoint [Ram24].

```

1  """NorthwindRoboticsOrdersAPI--used to demonstrate
   graceful shutdown.
2
3  Run standalone('python orders_server.py') or under
   the driver in
4  'shutdown_demo.py'. Every response carries three
   diagnostic headers so an
5  outside observer can prove what happened to in-flight
   work during a
6  termination signal:
7
8  * 'X-Request-Id' -- monotonically increasing request
   identifier.

```

```

9  * _ 'X-In-Flight-Remaining' _ -- _requests _still _in _flight
   _after _this _one.
10 * _ 'X-Elapsed-Ms' _ -- _server-side _wall _time _for _the _
    _request.
11 """
12
13 from __future__ import annotations
14
15 import asyncio
16 import itertools
17 import time
18
19 from fastapi import FastAPI, Request, Response
20
21 app = FastAPI(title="Northwind_Robotics_Orders_API")
22
23 IN_FLIGHT: dict[int, str] = {}
24 _request_ids = itertools.count(1)
25
26
27 @app.middleware("http")
28 async def track_in_flight(request: Request, call_next)
29     -> Response:
30     """Ledger_middleware: _register _on _entry, _deregister
31        _on _exit."""
32     rid = next(_request_ids)
33     IN_FLIGHT[rid] = f"{request.method}_{request.url.
34        path}"
35     started = time.perf_counter()
36     try:
37         response = await call_next(request)
38     finally:
39         IN_FLIGHT.pop(rid, None)
40         response.headers["X-Request-Id"] = str(rid)
41         response.headers["X-In-Flight-Remaining"] = str(len
42             (IN_FLIGHT))
43         response.headers["X-Elapsed-Ms"] = f"{{(time.
44             perf_counter() _- _started) _*_1000:.0f}}"
45     return response
46
47
48 @app.get("/healthz")
49 async def healthz() -> dict[str, bool]:
50     """Cheap_probe_endpoint: _process _is _alive _and _the _
51        event _loop _turns."""
52     return {"ok": True}

```

```

47
48
49 @app.post("/orders", status_code=201)
50 async def create_order() -> dict[str, str]:
51     """Fast path: a normal order write."""
52     await asyncio.sleep(0.05) # simulated database
53     round trip
54     return {"order_id": "ord-1042", "status": "accepted"}
55
56 @app.get("/orders/export")
57 async def export_orders() -> dict[str, object]:
58     """Slow path: a five-second report, one simulated
59     chunk per second.
60     This endpoint exists to be *in flight* when the
61     termination signal
62     arrives; whether it completes is the whole point of
63     the demo.
64     """
65     chunks = 0
66     for _ in range(5):
67         await asyncio.sleep(1.0) # one chunk of report
68         work per second
69         chunks += 1
70     return {
71         "report": "orders",
72         "chunks_completed": chunks,
73         "in_flight_at_finish": len(IN_FLIGHT),
74     }
75
76 if __name__ == "__main__":
77     import uvicorn
78
79     uvicorn.run(app, host="127.0.0.1", port=8571,
80                 log_level="warning")

```

18.3.4 Listing 17.4 — The Graceful-Shutdown Proof (shutdown_demo.py)

The driver starts the server as a child process, puts the five-second export in flight, sends the termination signal, and then verifies the four-act protocol of Section 18.2.2 from the *client's* side: a new request must be refused (stop

accepting), the in-flight request must complete with HTTP 200 (drain), and the process must exit on its own. On Linux the signal is `SIGTERM`; Windows has no POSIX signal delivery between processes, so the driver sends `CTRL_BREAK_EVENT` to the child's console process group — uvicorn installs a `SIGBREAK` handler on Windows that enters exactly the same graceful-exit path `SIGTERM` triggers on Linux, which is the path that matters, because the production artifact is a Linux container receiving `SIGTERM` from the kubelet.

```

1  """Prove uvicorn's drain semantics: terminate a server
   mid-request.
2
3  Offline and deterministic in behavior: the driver
   starts
4  'orders_server.py' as a child process, launches a
   slow '/orders/export'
5  request (about five seconds), sends the termination
   signal roughly one
6  second in, and then verifies the three properties a
   graceful shutdown must
7  have, in order:
8
9  1. STOP ACCEPTING -- a brand-new request after the
   signal is refused.
10 2. DRAIN ----- the in-flight export finishes with
   HTTP 200.
113. EXIT ----- the server process terminates on
   its own.
12
13 Usage (PowerShell): python shutdown_demo.py
14 Exit code 0 means all three properties held; 1 means a
   drain violation.
15
16
17 from __future__ import annotations
18
19 import json
20 import os
21 import signal
22 import subprocess
23 import sys
24 import threading
25 import time
26 import urllib.error
27 import urllib.request
28 from pathlib import Path

```

```

29
30 HOST, PORT = "127.0.0.1", 8571
31 BASE = f"http://{HOST}:{PORT}"
32 SERVER_SCRIPT = Path(__file__).resolve().parent / "
    orders_server.py"
33
34 _step = 0
35
36
37 def log(message: str) -> None:
38     """Print a numbered timeline line; numbering makes
        ordering explicit."""
39     global _step
40     _step += 1
41     print(f"[{_step:02d}] {message}")
42
43
44 def get(path: str) -> tuple[int, dict[str, str], dict]:
45     """Plain-stdlib GET returning (status, headers,
        parsed JSON body).
46
47     Header keys are normalized to lowercase because
        ASGI servers emit
48     lowercase field names on the wire.
49     """
50     with urllib.request.urlopen(f"{BASE}{path}",
        timeout=25.0) as resp:
51         headers = {k.lower(): v for k, v in resp.
            headers.items()}
52         return resp.status, headers, json.loads(resp.
            read())
53
54
55 def wait_ready(server: subprocess.Popen, timeout: float
    = 15.0) -> None:
56     """Poll /healthz until the server answers or dies
        trying."""
57     deadline = time.monotonic() + timeout
58     while time.monotonic() < deadline:
59         if server.poll() is not None:
60             raise RuntimeError(f"server exited early,
                code {server.returncode}")
61         try:
62             status, _, _ = get("/healthz")
63             if status == 200:
64                 return

```



```

65         except (urllib.error.URLError, ConnectionError,
66                 OSError):
67             time.sleep(0.25)
68             raise TimeoutError("server did not become ready in
69                                 time")
70
71 def main() -> int:
72     creationflags = 0
73     if os.name == "nt":
74         # Give the child its own console process group
75         # so CTRL_BREAK_EVENT
76         # reaches it (and only it) on Windows.
77         creationflags = subprocess.
78             CREATE_NEW_PROCESS_GROUP
79     server = subprocess.Popen(
80         [sys.executable, str(SERVER_SCRIPT)],
81         stdout=subprocess.DEVNULL,
82         stderr=subprocess.DEVNULL,
83         creationflags=creationflags,
84     )
85     try:
86         wait_ready(server)
87         log("server ready; /healthz answers 200")
88
89         outcome: dict[str, object] = {}
90
91     def slow_call() -> None:
92         try:
93             status, headers, body = get("/orders/
94                                     export")
95             outcome["status"] = status
96             outcome["elapsed_ms"] = headers.get("x-
97                                     elapsed-ms", "?")
98             outcome["chunks"] = body.get("
99                                     chunks_completed")
100         except (urllib.error.URLError,
101                 ConnectionError, OSError) as exc:
102             outcome["error"] = f"{type(exc).
103                                 __name__}: {exc}"
104
105     worker = threading.Thread(target=slow_call,
106                             daemon=True)
107     worker.start()
108     time.sleep(1.2) # the five-second export is
109                     now mid-flight

```

```

100
101     if os.name == "nt":
102         sig_name, sig = "CTRL_BREAK_EVENT", signal.
            CTRL_BREAK_EVENT
103     else:
104         sig_name, sig = "SIGTERM", signal.SIGTERM
105     log(f"sending_{sig_name}_while_the_export_is_in
        _flight")
106     os.kill(server.pid, sig)
107
108     time.sleep(0.3)
109     log("attempting_a_NEW_request_after_the_signal"
        )
110     try:
111         get("/healthz")
112         new_blocked = False
113         log("UNEXPECTED:_a_new_request_was_accepted
            _after_the_signal")
114     except (urllib.error.URLError, ConnectionError,
        OSError) as exc:
115         new_blocked = True
116         log(f"new_request_refused_as_expected_{
            type(exc).__name__}")
117
118     worker.join(timeout=25.0)
119     if "error" in outcome:
120         log(f"in-flight_export_FAILED:_{outcome['
            error']}")
121     else:
122         log(
123             "in-flight_export_completed:_",
124             f"HTTP_{outcome.get('status')},_",
125             f"elapsed_{outcome.get('elapsed_ms')}
                ms,_"
126             f"chunks_{outcome.get('chunks')}/5"
127         )
128
129     returncode = server.wait(timeout=25.0)
130     log(f"server_process_exited_on_its_own_(code_{
        returncode})")
131
132     drained = outcome.get("status") == 200 and
        outcome.get("chunks") == 5
133     print()
134     print("VERDICT")
135     print(f"1._stop_accepting_first:_{'PASS' if _

```

```

136         new_blocked_else='FAIL'})
137     print(f"2. drain in-flight work: {'PASS' if drained else 'FAIL'}")
138     print(f"3. clean process exit: {'PASS' if returncode is not None else 'FAIL'}")
139     return 0 if (new_blocked and drained) else 1
140 finally:
141     if server.poll() is None: # never leave an
142         orphaned server behind
143         server.kill()
144         server.wait(timeout=10)
145 if __name__ == "__main__":
146     raise SystemExit(main())

```

Observed output (Windows 11, Python 3.13, uvicorn 0.49; on Linux the signal name reads SIGTERM and the exit code is 0):

```

1 [01] server ready; /healthz answers 200
2 [02] sending CTRL_BREAK_EVENT while the export is in
   flight
3 [03] attempting a NEW request after the signal
4 [04] new request refused as expected (URLError)
5 [05] in-flight export completed: HTTP 200, elapsed 5036
   ms, chunks 5/5
6 [06] server process exited on its own (code 3)
7
8 VERDICT
9     1. stop accepting first : PASS
10    2. drain in-flight work : PASS
11    3. clean process exit   : PASS

```

Read line [05] carefully: the export was signaled at roughly 1.2 seconds into a five-second job and still reports all five chunks and an elapsed time of 5036 ms — the drain completed *after* the refusal of new work, which is the only ordering that is both safe and useful. The exit code (3 on Windows, 0 on Linux) is platform plumbing; the contract is that the process exits *itself*, before the supervisor's patience (`terminationGracePeriodSeconds`) runs out.

18.3.5 Listing 17.5 — Probe-Semantics Simulator (`probe_simulator.py`)

Table 18.2 stated the rules; this simulator enforces them. It models one Pod under the kubelet's probe logic over a virtual one-second-tick clock and a scripted fault timeline (slow boot, then a 35-second dependency outage,

then a transient deadlock), and prints the two things a caller can observe: endpoint membership (does this Pod get traffic?) and restarts. Scenario B removes the startup probe — the liveness-as-boot-check anti-pattern — and demonstrates CrashLoopBackOff on the same world.

```

1  """Kubernetes_probe-semantics_simulator_(offline,
2     stdlib-only, deterministic).
3
4  Two_scenarios_run_over_the_same_world:
5
6  A. WITH a startup_probe: slow boots are tolerated; a
7     deadlock is
8     detected by liveness and fixed by a restart; a
9     dependency outage
10    only removes the Pod from Service endpoints (no
11    restart).
12
13    B. WITHOUT a startup_probe (the copy-paste anti-
14    pattern where the
15    liveness probe doubles as the startup check): a
16    45-second cold boot
17    exceeds the liveness budget, so the kubelet kills
18    the container
19    before it ever serves -- CrashLoopBackOff.
20
21  The clock is virtual (one-second ticks), so every run
22  prints byte-for-byte
23  identical output.
24  """
25
26  from __future__ import annotations
27
28  from dataclasses import dataclass, field
29  from enum import Enum
30
31  SIM_SECONDS = 240
32  MAX_RESTARTS = 2 # afterwards the kubelet backs off (
33     CrashLoopBackOff)
34
35  class Mode(Enum):
36      BOOTING = "booting" # process alive, app
37          still initializing
38      HEALTHY = "healthy"
39      DEP_DOWN = "dep_down" # dependency down: /
40          ready fails, /live fine
41      DEADLOCKED = "deadlocked" # event loop stuck:

```

```

        every probe times out
31
32
33 @dataclass(frozen=True)
34 class ProbeSpec:
35     name: str
36     period_seconds: int
37     failure_threshold: int
38     success_threshold: int = 1
39
40
41 STARTUP = ProbeSpec("startup", period_seconds=5,
42                     failure_threshold=12) # 60 s
43 READINESS = ProbeSpec("readiness", period_seconds=5,
44                       failure_threshold=3)
45 LIVENESS = ProbeSpec("liveness", period_seconds=10,
46                      failure_threshold=3)
47
48 # The fault environment: (start_s, end_s, mode once
49 # booted).
50 TIMELINE: list[tuple[int, int, Mode]] = [
51     (0, 60, Mode.HEALTHY),
52     (60, 95, Mode.DEP_DOWN),          # 35 s database
53     (95, 150, Mode.HEALTHY),          outage
54     (150, 10_000, Mode.DEADLOCKED),   # transient
55     (10_000, 10_000, Mode.HEALTHY),   deadlock; a restart cures it
56 ]
57
58 def timeline_mode(t: int) -> Mode:
59     for start, end, mode in TIMELINE:
60         if start <= t < end:
61             return mode
62     return Mode.HEALTHY
63
64 @dataclass
65 class SimResult:
66     label: str
67     events: list[str] = field(default_factory=list)
68     restarts: int = 0
69     traffic_seconds: int = 0
70     ready_at_end: bool = False

```

```

70 def run_scenario(label: str, use_startup_probe: bool,
71    boot_seconds: int,
72    reboot_seconds: int) -> SimResult:
73     """Simulate one Pod. Returns an event log plus
74     routing statistics."""
75     result = SimResult(label=label)
76
77     boot_started = 0
78     boot_len = boot_seconds
79     booted = False # startup check has passed
80     at least once
81     ready = False # Pod is a member of the
82     Service endpoints
83     deadlock_cleared = False # restarts cure process-
84     local deadlocks
85     crashlooped = False
86
87     startup_fail = ready_fail = ready_ok = live_fail =
88     0
89
90     def mode_at(t: int) -> Mode:
91         if t - boot_started < boot_len:
92             return Mode.BOOTING
93         mode = timeline_mode(t)
94         if mode is Mode.DEADLOCKED and deadlock_cleared
95         :
96             return Mode.HEALTHY
97         return mode
98
99     def restart(t: int, why: str) -> None:
100         nonlocal boot_started, boot_len, booted, ready
101         nonlocal startup_fail, ready_fail, ready_ok,
102         live_fail
103         nonlocal deadlock_cleared, crashlooped
104         result.restarts += 1
105         if result.restarts > MAX_RESTARTS:
106             crashlooped = True
107             result.events.append(
108                 f"t={t:3d}s restart budget exhausted ->
109                 CrashLoopBackOff;"
110                 "kubenet backs off, Pod never becomes
111                 ready"
112             )
113             return
114         result.events.append(f"t={t:3d}s RESTART#{
115             result.restarts}_{why}")

```

```

105     boot_started, boot_len = t, reboot_seconds
106     booted = ready = False
107     startup_fail = ready_fail = ready_ok =
108         live_fail = 0
109     deadlock_cleared = True
110
111     def set_ready(t: int, value: bool, why: str) ->
112         None:
113         nonlocal ready
114         if ready != value:
115             ready = value
116             action = "ADDED_TO" if value else "REMOVED_FROM"
117             result.events.append(
118                 f"t={t:3d}s endpoints: Pod {action} Service rotation ({why})"
119             )
120
121     for t in range(1, SIM_SECONDS + 1):
122         if crashlooped:
123             break
124         mode = mode_at(t)
125
126         # --- startup probe (gates the other two)
127         -----
128         if use_startup_probe and not booted and t %
129             STARTUP.period_seconds == 0:
130             if mode is not Mode.BOOTING:
131                 booted = True
132                 startup_fail = 0
133                 result.events.append(
134                     f"t={t:3d}s startup probe: first success->"
135                     "liveness/readiness probes begin"
136                 )
137             else:
138                 startup_fail += 1
139                 if startup_fail >= STARTUP.failure_threshold:
140                     restart(t, "startup probe exhausted its budget")
141
142     probes_active = booted if use_startup_probe
143         else True
144
145     # --- readiness probe: endpoint membership only

```

```

-----
141     if probes_active and t % READINESS.
142         period_seconds == 0:
143             if mode is Mode.HEALTHY:
144                 ready_fail = 0
145                 ready_ok += 1
146                 if ready_ok >= READINESS.
147                     success_threshold:
148                         set_ready(t, True, "readiness_
149                             success")
150             else:
151                 ready_ok = 0
152                 ready_fail += 1
153                 if ready_fail >= READINESS.
154                     failure_threshold:
155                         set_ready(t, False, "readiness_
156                             failing_--_no_restart!")
157
158     # --- liveness probe: restart authority
159     -----
160     if probes_active and t % LIVENESS.
161         period_seconds == 0:
162             if mode in (Mode.HEALTHY, Mode.DEP_DOWN):
163                 live_fail = 0
164             else: # BOOTING (no startup probe) or
165                 DEADLOCKED: timeout
166                 live_fail += 1
167                 result.events.append(
168                     f"t={t:3d}s_{}_liveness_probe:_
169                         failure_{live_fail}/"
170                     f"{LIVENESS.failure_threshold}_("
171                         mode={mode.value})"
172                 )
173             if live_fail >= LIVENESS.
174                 failure_threshold:
175                     why = ("deadlock_detected" if mode
176                         is Mode.DEADLOCKED
177                         else "killed_while_still_
178                             booting")
179                     restart(t, why)
180
181     if ready:
182         result.traffic_seconds += 1
183
184     result.ready_at_end = ready
185     return result

```



```

173
174
175 def report(result: SimResult) -> None:
176     print("=" * 72)
177     print(f"SCENARIO_{result.label}")
178     print("=" * 72)
179     for line in result.events:
180         print("_", line)
181     print("-" * 72)
182     print(f"_{result.restarts}_:_{result.restarts}")
183     print(f"_{result.traffic_seconds}_/_{SIM_SECONDS}")
184     print(f"_{result.ready_at_end}_")
185     print()
186
187
188 def main() -> None:
189     print("Kubernetes_probe-semantics_simulator")
190     print(f"probes:_{STARTUP.period_seconds}s_{STARTUP.failure_threshold},_"
191           f"_{READINESS.period_seconds}s_{READINESS.failure_threshold},_"
192           f"_{LIVENESS.period_seconds}s_{LIVENESS.failure_threshold}")
193     print(f"faults:_{18}s_{boot}_dep_outage_t=60..95_|_"
194           f"_{deadlock}_at_t=150_"
195           f"_{cured_by_restart}")
196     print()
197
198     with_startup = run_scenario(
199         "A_--_WITH_startup_probe_(boot_18s_tolerated)",
200         use_startup_probe=True, boot_seconds=18,
201         reboot_seconds=12,
202     )
203     report(with_startup)
204
205     without_startup = run_scenario(
206         "B_--_WITHOUT_startup_probe_(liveness_doubles_"
207         f"_{as_boot_check}_boot_45s)",
208         use_startup_probe=False, boot_seconds=45,
209         reboot_seconds=45,
210     )
211     report(without_startup)

```

```

209     print("LESSONS")
210     print("__*__DEP_DOWN__caused__readiness__removal__but__
        ZERO__restarts__--")
211     print("____restarting__cannot__fix__someone__else's__
        outage.")
212     print("__*__DEADLOCKED__caused__a__restart__after__3__
        consecutive__liveness")
213     print("____failures__--__liveness__exists__for__process-
        local__death.")
214     print("__*__Without__a__startup__probe__,__a__slow__boot__is__
        indistinguishable__from")
215     print("____a__deadlock__,__and__the__liveness__budget__
        becomes__a__boot__deadline.")
216
217
218 if __name__ == "__main__":
219     main()

```

Observed output (identical on every run — the clock is virtual):

```

1  =====
2  SCENARIO A -- WITH startup probe (boot 18s tolerated)
3  =====
4
5  t= 20s  startup probe: first success -> liveness/
6         readiness probes begin
7
8  t= 20s  endpoints: Pod ADDED TO Service rotation (
9         readiness success)
10
11 t= 70s  endpoints: Pod REMOVED FROM Service rotation
12        (readiness failing -- no restart!)
13
14 t= 95s  endpoints: Pod ADDED TO Service rotation (
15        readiness success)
16
17 t=150s  liveness probe: failure 1/3 (mode=deadlocked)
18
19 t=160s  endpoints: Pod REMOVED FROM Service rotation
20        (readiness failing -- no restart!)
21
22 t=160s  liveness probe: failure 2/3 (mode=deadlocked)
23
24 t=170s  liveness probe: failure 3/3 (mode=deadlocked)
25
26 t=170s  RESTART #1 (deadlock detected)
27
28 t=185s  startup probe: first success -> liveness/
29         readiness probes begin
30
31 t=185s  endpoints: Pod ADDED TO Service rotation (
32         readiness success)
33
34 -----
35
36 restarts                                : 1
37 seconds receiving traffic                : 171 / 240

```

```

18   ready at end of simulation   : True
19
20  =====
21  SCENARIO B -- WITHOUT startup probe (liveness doubles
    as boot check, boot 45s)
22  =====
23
24  t= 10s  liveness probe: failure 1/3 (mode=booting)
25  t= 20s  liveness probe: failure 2/3 (mode=booting)
26  t= 30s  liveness probe: failure 3/3 (mode=booting)
27  t= 30s  RESTART #1 (killed while still booting)
28  t= 40s  liveness probe: failure 1/3 (mode=booting)
29  t= 50s  liveness probe: failure 2/3 (mode=booting)
30  t= 60s  liveness probe: failure 3/3 (mode=booting)
31  t= 60s  RESTART #2 (killed while still booting)
32  t= 70s  liveness probe: failure 1/3 (mode=booting)
33  t= 80s  liveness probe: failure 2/3 (mode=booting)
34  t= 90s  liveness probe: failure 3/3 (mode=booting)
35  t= 90s  restart budget exhausted -> CrashLoopBackOff;
    kubelet backs off,
    Pod never becomes ready
36  -----
37
38  restarts                               : 3
39  seconds receiving traffic              : 0 / 240
    ready at end of simulation           : False

```

Three readings. In Scenario A the 35-second dependency outage produced a *routing* change (endpoint removed at $t=70$ s, three readiness periods after the fault began) and *zero* restarts — correct, because restarting cannot fix the database; the readiness `failureThreshold: 3` is what keeps a single flaky probe from flapping the endpoint. The deadlock, a process-local fault, earned the only restart, at $t=170$ s — three liveness periods after it began. In Scenario B the *identical* liveness configuration is fatal for a different reason: a 45-second boot exceeds the $10\text{ s} \times 3$ budget, so the kubelet kills the container at $t=30$ s, mid-boot, forever. The configuration did not change between scenarios; the *meaning* of the liveness probe did. That is why probes are a design artifact and not boilerplate.

18.3.6 Listing 17.6 — Canary Analyzer (`canary_analysis.py`)

The analyzer consumes two metric series — a long baseline and a batched canary — and makes the promote/rollback call with Wald’s Sequential Probability Ratio Test. For Bernoulli error events with hypotheses $H_0 : p =$

p_0 (baseline error rate) and $H_1 : p = p_1 = p_0 + \delta$ (an intolerable regression), the log-likelihood ratio accumulates per observation:

$$\Lambda_n = \sum_{i=1}^n \left[x_i \ln \frac{p_1}{p_0} + (1-x_i) \ln \frac{1-p_1}{1-p_0} \right], \quad \Lambda_n \geq \ln \frac{1-\beta}{\alpha} \Rightarrow \text{ROLLBACK}, \quad \Lambda_n \leq \ln \frac{\beta}{1-\alpha} \Rightarrow \text{PROMOTE}$$

with α the false-promote budget and β the false-rollback budget (both 0.05 here, so both boundaries sit at $\pm \ln 19 \approx \pm 2.944$). The SPRT's virtue for rollouts is that it is *sequential*: it decides as soon as the evidence allows — a badly regressed canary is caught in two or three batches instead of after the full window — with controlled error rates, unlike repeated peeking at a fixed-horizon test. A hard latency gate (batch p95 $\leq$ baseline p95 $\times 1.25$) runs alongside, because some regressions are pure latency.

```

1  """Canary_analysis:error-rate_delta+Wald_SPRT_
   promote/rollback_decision.
2
3  Given_a_stable_baseline_metric_series_and_a_canary_
   metric_series(synthetic
4  fixtures_generated_with_fixed_seeds),_this_script_
   computes,_per_analysis
5  batch:
6
7  *_the_canary/baseline_error-rate_delta_,
8  *_a_Wald_Sequential_Probability_Ratio_Test_(SPRT)_on_
   Bernoulli_error
9  _events,_which_decides_PROMOTE_(canary_no_worse)_or_
   ROLLBACK_(canary
10 _regressed_by_at_least_delta)_as_soon_as_the_evidence_
   crosses_a_boundary_,
11 *_a_hard_latency_gate:_batch_p95_must_stay_within_25%_
   of_baseline_p95.
12
13 Stdlib_only,_fully_deterministic:_every_run_prints_
   identical_output.
14 """
15
16 from __future__ import annotations
17
18 import math
19 import random
20 from dataclasses import dataclass
21
22 # Configuration (the numbers an SRE would set in the
   rollout policy)
23 BATCH_SIZE = 200          # canary requests analyzed
   per gate evaluation

```

```

24 MAX_BATCHES = 10          # give up (hold) if still
    undecided after this many
25 DELTA = 0.010             # H1: canary error rate
    worse by >= 1 percentage pt
26 ALPHA = 0.05             # false-promote probability
    budget
27 BETA = 0.05               # false-rollback probability
    budget
28 LATENCY_P95_FACTOR = 1.25 # hard gate: canary p95 <=
    baseline p95 * 1.25
29
30
31 @dataclass(frozen=True)
32 class Event:
33     latency_ms: float
34     error: bool
35
36
37 def synth_series(n: int, mean_latency: float, jitter:
    float,
38                 error_rate: float, seed: int) -> list[
    Event]:
39     """Deterministic synthetic request series(
        lognormal-ish latencies)."""
40     rng = random.Random(seed)
41     series: list[Event] = []
42     for _ in range(n):
43         latency = mean_latency + 10.0 * rng.
            lognormvariate(0.0, jitter)
44         series.append(Event(round(latency, 1), rng.
            random() < error_rate))
45     return series
46
47
48 def percentile(sorted_values: list[float], pct: float)
    -> float:
49     """Nearest-rank percentile; input must already be
        sorted ascending."""
50     if not sorted_values:
51         raise ValueError("percentile of empty series")
52     rank = max(1, math.ceil(pct / 100.0 * len(
        sorted_values)))
53     return sorted_values[rank - 1]
54
55
56 class BernoulliSPRT:

```

```

57     """Wald's sequential test: H0 (p=p0) vs H1 (p=
58         p1=p0+delta)."""
59
60     def __init__(self, p0: float, p1: float, alpha:
61         float, beta: float) -> None:
62         if not (0.0 < p0 < p1 < 1.0):
63             raise ValueError("require 0<p0<p1<1")
64         self.p0, self.p1 = p0, p1
65         self.upper = math.log((1.0 - beta) / alpha) #
66             rollback boundary
67         self.lower = math.log(beta / (1.0 - alpha)) #
68             promote boundary
69         self.llr = 0.0
70         self.errors = 0
71         self.total = 0
72
73     def observe(self, error: bool) -> None:
74         self.total += 1
75         if error:
76             self.errors += 1
77             self.llr += math.log(self.p1 / self.p0)
78         else:
79             self.llr += math.log((1.0 - self.p1) / (1.0
80                 - self.p0))
81
82     def verdict(self) -> str:
83         if self.llr >= self.upper:
84             return "ROLLBACK"
85         if self.llr <= self.lower:
86             return "PROMOTE"
87         return "CONTINUE"
88
89     def analyze(label: str, baseline: list[Event], canary:
90         list[Event]) -> str:
91         base_errors = sum(1 for e in baseline if e.error)
92         p0 = base_errors / len(baseline)
93         base_lat = sorted(e.latency_ms for e in baseline)
94         base_p95 = percentile(base_lat, 95)
95         p95_limit = base_p95 * LATENCY_P95_FACTOR
96
97         print("=" * 74)
98         print(f"FIXTURE: {label}")
99         print(f"baseline: {len(baseline)} reqs, {
100             base_errors} errors"
101             f"({p0:.2%}), p50={percentile(base_lat, 50)}

```

```

        :.1f}\_ms,\_"
    f"p95={base_p95:.1f}\_ms")
96
97 print(f"H0:\_p\_canary\_={p0:.2%}\_\_\_\_H1:\_p\_canary\_>=\_{
    p0+\_DELTA:.2%}\_\_\_\_"
98
99     f"alpha=beta={ALPHA}\_\_\_\_latency\_gate:\_p95\_<=\_{
        p95_limit:.1f}\_ms")
100
101 print("-" * 74)
102 print(f"{'batch':>5}\_\{'reqs':>5}\_\{'errors':>6}\_\{'
    err-rate':>9}\_"
103
104     f"{'delta':>8}\_\{'p95\_ms':>8}\_\{'lat-gate':>8}\_\_
        {'LLR':>8}\_\_\_decision")
105
106 print("-" * 74)
107
108 sprt = BernoulliSPRT(p0, p0 + DELTA, ALPHA, BETA)
109 final = "HOLD\_(inconclusive\_after\_max\_batches)"
110 for batch_no in range(1, MAX_BATCHES + 1):
111     lo, hi = (batch_no - 1) * BATCH_SIZE, batch_no
112         * BATCH_SIZE
113     batch = canary[lo:hi]
114     if not batch:
115         break
116     latency_breach = False
117     for event in batch:
118         sprt.observe(event.error)
119     batch_lat = sorted(e.latency_ms for e in batch)
120     batch_p95 = percentile(batch_lat, 95)
121     if batch_p95 > p95_limit:
122         latency_breach = True
123     err_rate = sprt.errors / sprt.total
124     delta = err_rate - p0
125     verdict = sprt.verdict()
126     print(f"{'batch_no':>5}\_\{sprt.total:>5}\_\{sprt.
        errors:>6}\_"
127
128         f"{'err_rate':>9.2%}\_\{'delta':>+8.2%}\_\{
            batch_p95:>8.1f}\_"
129
130         f"{'BREACH'\_if\_latency\_breach\_else\_ 'ok'
            ':>8}\_"
131
132         f"{'sprt.llr':>8.3f}\_\_\_{'verdict'}")
133
134     if latency_breach:
135         final = f"ROLLBACK\_(latency\_gate\_breached\_
            at\_batch\_\{batch_no})"
136         break
137     if verdict == "ROLLBACK":
138         final = (f"ROLLBACK\_(error\_regression\_
            detected\_at\_batch\_
139
            f"{'batch_no'},\_\{sprt.total}\_\_canary\_

```

```

131         requests)")
132         break
133         if verdict == "PROMOTE":
134             final = (f"PROMOTE_(error_parity_proven_at_
135                     batch_{batch_no},_"
136                     f"{sprt.total}_canary_requests)")
137             break
138         print("-" * 74)
139         print(f"FINAL_DECISION:_{final}")
140         print()
141         return final
142
143 def main() -> None:
144     print("Canary_analysis_--_Northwind_Robotics_Orders
145           _API_rollout")
146     print(f"policy:_batches_of_{BATCH_SIZE},_error_
147           delta_>=_{DELTA:.1%}_")
148     f"rolls_back,_p95_latency_x{
149         LATENCY_P95_FACTOR}_rolls_back")
150     print()
151
152     baseline = synth_series(4000, mean_latency=45.0,
153                             jitter=0.6,
154                             error_rate=0.004, seed
155                                 =2026)
156
157     good = synth_series(MAX_BATCHES * BATCH_SIZE,
158                        mean_latency=47.0,
159                        jitter=0.6, error_rate=0.004,
160                        seed=1701)
161     analyze("canary_v2.4.1_--_healthy_(error_parity,_
162            latency_parity)",
163            baseline, good)
164
165     bad = synth_series(MAX_BATCHES * BATCH_SIZE,
166                       mean_latency=58.0,
167                       jitter=0.6, error_rate=0.014,
168                       seed=1702)
169     analyze("canary_v2.4.2_--_regressed_(error_rate_3.5
170            x_baseline)",
171            baseline, bad)
172
173 if __name__ == "__main__":
174     main()

```


Observed output:

```

1  FIXTURE: canary v2.4.1 -- healthy (error parity,
    latency parity)
2  baseline: 4000 reqs, 19 errors (0.47%), p50=54.9 ms,
    p95=71.2 ms
3  H0: p_canary = 0.47%    H1: p_canary >= 1.47%    alpha=
    beta=0.05    latency gate: p95 <= 89.0 ms
4  -----
5  batch  reqs errors  err-rate    delta    p95 ms lat-gate
    LLR  decision
6  -----
7      1    200      1    0.50%    +0.03%    76.5      ok
    -0.877  CONTINUE
8      2    400      1    0.25%    -0.22%    70.7      ok
    -2.896  CONTINUE
9      3    600      2    0.33%    -0.14%    74.4      ok
    -3.773  PROMOTE
10
11 FINAL DECISION: PROMOTE (error parity proven at batch
    3, 600 canary requests)
12
13 FIXTURE: canary v2.4.2 -- regressed (error rate 3.5x
    baseline)
14 (baseline and policy header lines identical to the
    first fixture)
15 batch  reqs errors  err-rate    delta    p95 ms lat-gate
    LLR  decision
16      1    200      2    1.00%    +0.53%    88.0      ok
    0.267  CONTINUE
17      2    400      5    1.25%    +0.78%    88.7      ok
    1.677  CONTINUE
18      3    600     11    1.83%    +1.36%    85.9      ok
    6.516  ROLLBACK
19
20 FINAL DECISION: ROLLBACK (error regression detected at
    batch 3, 600 canary requests)

```

Both fixtures resolve in three batches — 600 canary requests — but in opposite directions, and the trace shows why the sequential test is the right tool: batch 1 of the regressed canary (+0.53 points of delta, LLR 0.267) is exactly the evidence a dashboard-watcher would shrug at (“could be noise”), and batch 3 crosses the rollback boundary with the error budget still honored. Note also what the analyzer deliberately leaves to you: the

latency gate never fired, because the regressed fixture’s mean-latency rise lands just under the gate ($88.0 \leq 89.0$ ms) — retune `LATENCY_P95_FACTOR` to 1.20 in Exercise 7.4 and the same fixture falls to the latency gate at the very first batch, two batches before the error SPRT speaks.

18.4 Common Pitfalls & Anti-Patterns

:latest tags in anything that matters

`latest` is not a version; it is a pointer that moves when the publisher rebuilds. Deployments referencing it are unreproducible (two clusters running “the same” manifest run different bytes), rollbacks are impossible (there is no previous pointer to return to), and Kubernetes’ default `imagePullPolicy: IfNotPresent` means two nodes can run *different* images under one tag after a republish. Pin tags to release versions; pin digests for anything you would have to defend in an incident review.

Root containers and writable filesystems

“It needs root” almost always means “we never checked.” A FastAPI service needs to bind a high port, read its own files, and write to one scratch directory — none of which is UID 0. Running as root with a writable root filesystem converts every remote-code-execution bug into a full container takeover with persistence (dropped binaries, modified config). `USER` in the image, `runAsNonRoot` in the pod, `readOnlyRootFilesystem: true`, `drop: ["ALL"]` capabilities: four lines, most of a container-escape kill chain.

Liveness = readiness copy-paste

Duplicating the readiness probe’s path into the liveness probe is the single most common probe bug in the wild, and it inverts the recovery theory of Table 18.2: a dependency outage (readiness correctly fails) now also restarts every pod simultaneously, which hammers the recovering dependency with reconnection storms and produces a fleet-wide outage from a partial one. Rule of thumb: the liveness handler should be *cheaper* than the readiness handler and touch *fewer* things — ideally just “event loop turns.”

CPU limits causing throttle-induced latency

Section 18.2.4 derived the mechanism; the anti-pattern is the cargo-culted `limits.cpu` copied from a tutorial. The signature: p99 latency glued to multiples of 100 ms, average CPU well below the limit, `container_cpu_cfs_throttled_periods_to`

climbing. The fix is not “raise the limit” blindly; it is “decide whether the limit’s protection (noisy-neighbor containment) is worth its cost (tail latency) for *this* service” — latency-sensitive APIs usually drop the CPU limit and rely on requests plus HPA.

HPA without a PDB (and PDBs without thought)

An HPA will happily scale to `minReplicas` during a node drain, and without a PDB the drain evicts *all* of them — autoscaling governs desired count, drains govern eviction, and only the PDB sits between them. The symmetric bug: `minAvailable: <replicas>` or `maxUnavailable: 0` on a 1-replica deployment, which makes every cluster upgrade hang until a human notices. PDBs are arithmetic, not decoration: `budget = replicas you can lose and still serve the SLO`, minus one for the rollout window.

Hardcoded secrets in manifests

`DATABASE_URL: postgres://user:password@...` in a committed YAML file is a credential in version control — replicated to every clone, every CI log that echoes the apply, every `kubectl get -o yaml`. `secretKeyRef` is the floor; external-secrets operators or CSI secret stores are the grown-up version. Also: `kubectl create secret` values are base64, not encryption — enable encryption at rest and RBAC accordingly (Chapter 17).

TLS terminated once, plaintext to the pod

“The LB does TLS” plus plaintext HTTP from LB to pod means every internal hop — node network, mesh-less sidecar gap, packet mirror — carries bearer tokens in the clear, and a single misconfigured security group exposes the service port directly. Defense in depth is cheap here: re-encrypt (edge cert outside, internal CA inside) or run a mesh and let mTLS cover east-west by default (Section 18.2.6).

Migration deployed before compatible code

The one rollout mistake rollback cannot fix: a destructive migration (drop/rename) applied while old binaries still run, followed by a failed rollout that restores old code against a schema it no longer understands. Expand-and-contract exists precisely to make schema and code independently deployable (Chapter 10): expand first, deploy code that tolerates both shapes, contract only after every old pod is gone. Pipeline-enforced, not remembered.

18.5 Hands-On Production Lab & Real-World Case Study

18.5.1 Lab: Signals, Probes, and a Manifest Design Review

Time: 90 minutes. Requirements: the venv from Section 18.1; no Docker or cluster needed for parts 1–2.

Part 1 — Prove the drain (30 min). Save Listings 17.3 and 17.4 side by side, then run `python shutdown_demo.py`. Confirm all three PASS lines, then break things deliberately: (a) edit `orders_server.py` to add `timeout_graceful_shutdown=1` to the `uvicorn.run` call and re-run — the export now fails mid-flight (act 3’s timeout amputating act 2’s drain); (b) restore it, and note that no edit can make the drain work if the process never receives the signal — on Linux, wrap the entrypoint in a shell (`sh -c "uvicorn ..."`) and `SIGTERM` dies in the shell. Record both failure signatures.

Part 2 — Replay the probes (25 min). Run `python probe_simulator.py`. Then edit the timeline: move the `DEP_DOWN` window to overlap the deadlock, and predict — before running — the restart count and traffic seconds. Now set `READINESS.failure_threshold = 1` and observe the endpoint flapping during the dependency outage (removed at the first failure, oscillating with the probe period). Explain, in one sentence each, why flapping hurts (connection churn, LB convergence cost) and why threshold 3 was the right default.

Part 3 — Manifest design review (35 min). Print Listing 18.3.2. Walk it as a design-review meeting, field by field, answering aloud: What happens to in-flight requests when this pod is evicted? When the node dies? When the database is down for a minute? What is the worst-case traffic loss during a rollout (`maxUnavailable: 0` $\Rightarrow$ none below 4 ready) and what does it cost (quota for one surge pod)? Why does CPU have no limit? Where does the database password live, and who can read it? If any answer is “I don’t know,” the manifest has a bug you haven’t found yet — keep digging until every field has a failure story.

18.5.2 Case Study: The Rolling Deploy That Dropped In-Flight Requests

Context. Northwind Robotics’ checkout-adjacent Orders API: 4 replicas behind a cloud load balancer, Kubernetes rolling updates, `maxSurge: 1`, `maxUnavailable: 0` — a configuration review had declared it “zero-downtime safe.” Every Friday deploy nonetheless produced a 30–60-second spike of HTTP 502s at exactly the rollout minutes, roughly 0.1% of requests

— small enough to lose in the graphs, large enough to be a standing client complaint.

Diagnosis. The 502s correlated with pod terminations, and the terminated pods’ logs showed active requests severed mid-flight. The mechanism was the race of Section 18.2.2: the kubelet sent **SIGTERM** and *concurrently* updated endpoints, but the cloud load balancer’s target list converged seconds later — during which it kept opening new connections to a process that had already stopped accepting. Worse, the app’s longest endpoint (the order export) ran up to 40 s while `terminationGracePeriodSeconds` was the default 30 s: even requests that *were* in flight at SIGTERM were SIGKILLED at the 30-second wall. Two gaps, one symptom. There was no `preStop` hook at all — the missing line the whole outage hung on.

Fix. Three manifest lines and one server flag: `preStop.sleep.seconds: 8` (the pod keeps serving while endpoint removal fans out), `terminationGracePeriodSeconds: 60` (slowest request 40 s + `preStop` 8 s + margin), unicorn’s graceful timeout raised to match, and an SLO alert on 502 rate so the class of bug could never again hide at 0.1%. The next deploy’s 502 count was zero, and has been since.

Lessons. (1) “Zero downtime” is a property of the *termination path*, not the update strategy — `maxSurge` arithmetic protects capacity, not connections. (2) Grace periods are budgets to be engineered from measured request durations, not defaults to be inherited. (3) The correct test is exactly Listing 18.3.4: signal the server mid-request and watch from the client side; had that test existed, the config review’s “safe” stamp would have died in CI instead of in production.

18.6 Self-Assessment Exercises & Deep-Dive Questions

Exercise 7.1 Layer forensics. Rewrite Listing 17.1’s Dockerfile so the source `COPY` precedes the dependency install. Explain exactly which layers rebuild on a one-line source edit now, and quantify the cost for a 200 MB dependency layer.

Exercise 7.2 Break the drain. In `orders_server.py`, set `timeout_graceful_shutdown=1` and capture the demo’s new failure signature. Which of the three verdict lines flips, and which act of the four-act shutdown did you just amputate?

Exercise 7.3 Probe arithmetic. A service boots in 75 seconds. Using `periodSeconds: 5`, what is the minimum `failureThreshold` for a startup probe that tolerates this with 25% head-

room? What goes wrong if you instead raise the liveness `failureThreshold` to accommodate it?

Exercise 7.4 Retune the gate. Run `canary_analysis.py` with `LATENCY_P95_FACTOR = 1.20` and confirm the regressed fixture is rolled back at batch 1 by the latency gate. Then set `DELTA = 0.02`: what happens to the regressed fixture’s detection latency, and what does that teach about choosing δ relative to your error budget?

Exercise 7.5 CrashLoop replay. In `probe_simulator.py`, give Scenario B an 18-second boot (same as A). Does it still crash-loop? Now 35 seconds. Derive the exact boot-time boundary between the two outcomes from the probe parameters alone.

Exercise 7.6 Endpoint lag budget. Your cloud LB converges target removals in up to 12 s and your p99.9 request duration is 9 s. Choose `preStop` sleep, `uvicorn` graceful timeout, and `terminationGracePeriodSeconds`, and defend each number.

Exercise 7.7 Design review script. Take Listing 18.3.2 and remove the PDB. Write the exact sequence of events (drain, evict, HPA response) that takes the API fully offline during a routine node upgrade.

Exercise 7.8 Canary slice sizing. Baseline error rate 0.4%, $\delta = 1$ point, $\alpha = \beta = 0.05$. Using the expected per-request LLR drift, estimate the expected number of canary requests to rollback when the true canary rate is 1.4%, and to promote when it equals baseline. Verify with the simulator.

Deep-dive questions for further study: How does `containerd`’s image pull interact with digest pinning across a node pool with heterogeneous pull times during a surge? What changes in every section of this chapter when the mesh moves from sidecars to ambient mode? How would you express Listing 18.3.6’s gate in PromQL-based analysis instead of batch files? What does the `PodLifecycleSleepAction` feature gate’s graduation history tell you about depending on alpha features in lifecycle hooks?

18.7 Interview Q&A Vault

Junior (Q1–Q10)

- Q1. What is a container image, in one paragraph?** A content-addressed stack of filesystem layers plus a manifest: each layer is a diff addressed by its hash, the image is addressed by the manifest's digest, and a running container adds one thin writable layer on top. The properties that matter are immutability (bytes never change after build) and traceability (the digest names exactly one build forever).
- Q2. What problem does a multi-stage Dockerfile solve?** The runtime image inherits everything the build needed — compilers, package managers, caches — unless you build in one stage and copy only artifacts into a fresh final stage. The win is triple: smaller images (faster pulls, faster cold starts), smaller attack surface (fewer binaries for an attacker to reuse), and cleaner caching (dependency layers invalidate only when manifests change).
- Q3. Why should containers not run as root?** Container root is a UID on the host separated only by namespace configuration; a container-escape or a misconfigured mount turns application-root into something dangerously close to host-root. Running as a fixed unprivileged UID forces an attacker to win twice: exploit the app, *then* escalate. It is also a deploy-time assertion: `runAsNonRoot: true` makes Kubernetes refuse an image that regresses.
- Q4. What does `.dockerignore` protect?** Two things: build correctness (junk churn like `.git` or local logs would otherwise bust the cache on every build) and secrets (a stray `.env` copied by `COPY .` persists in the layer history even if a later layer deletes it).
- Q5. What is the difference between a tag and a digest?** A tag (`:3.12-slim`) is a mutable registry pointer that can be retargeted at any time; a digest (`@sha256:...`) is the content hash of one manifest and can never name anything else. Tags are for humans; digests are for deployments you may have to defend in an incident review.
- Q6. What happens, in order, when Kubernetes terminates a Pod?** The Pod is marked terminating and removed from endpoints (asynchronously); the `preStop` hook runs; then `SIGTERM` to PID 1; the grace period (`terminationGracePeriodSeconds`, default 30 s)

counts down; then **SIGKILL**. Note the trap: endpoint removal is concurrent, not first — traffic can still arrive after **SIGTERM**.

Q7. What is a liveness probe? A periodic check (HTTP, TCP, or exec) whose consecutive failures — **failureThreshold** of them — make the kubelet *restart the container in place*. It answers one question: is this process unrecoverably stuck? It should therefore check only process-local health and never a dependency.

Q8. What is an API gateway? A reverse proxy at the platform edge that owns cross-cutting north-south policy — routing, JWT validation offload, rate limiting, transformation, canary splitting — so those concerns are configured once instead of reimplemented per service.

Q9. What is blue-green deployment? Two complete environments; traffic switches atomically from one to the other (selector, LB target, or DNS), and rollback is the switch flipped back. Instant rollback and a warm target, at the price of double infrastructure and an idle environment that rots unless exercised.

Q10. What is a PodDisruptionBudget? A policy object stating how many pods of a service may be *voluntarily* unavailable (**minAvailable** or **maxUnavailable**); drains and cluster upgrades respect it by blocking, while crashes (involuntary) ignore it. Without one, a routine node drain can evict your entire fleet at once.

Mid (Q11–Q20)

Q11. Liveness vs. readiness vs. startup probes — failure behavior of each? Startup: only probe evaluated during boot; exhausting **periodSeconds** × **failureThreshold** restarts the container *before* it ever served — it exists to give slow starters a budget liveness never sees. Readiness: failure removes the Pod from Service endpoints — a routing consequence, no restart — and the first success re-adds it; it answers “should I get traffic *now*?” Liveness: **failureThreshold** consecutive failures restart the container in place; it answers “am I dead in a way only a restart can fix?” The simulator in Section 18.3.5 demonstrates all three consequences over one fault timeline.

Q12. Why is copying the readiness probe into liveness dangerous? Readiness legitimately fails for external reasons (dependency down); making liveness fail for the same reason converts a partial

outage into a fleet-wide restart storm — every pod restarts simultaneously and then reconnection-storms the dependency that was trying to recover. Readiness says “wait”; liveness says “kill me”; merging them deletes the distinction that recovery depends on.

- Q13. Why can CPU limits *hurt* latency?** CPU limits are enforced by CFS quota throttling in (default) 100 ms periods: consume the quota in the first few milliseconds of a burst and every thread in the container freezes until the next period. The signature is p99 latency quantized to 100 ms multiples while average CPU sits far below the limit. Memory limits OOM-kill instead — violent but honest. Mitigation: requests always (scheduling and CFS shares), CPU limits omitted or generous on latency-sensitive services, and an alert on the throttled-periods metric.
- Q14. Explain `maxSurge` and `maxUnavailable`.** Rolling-update trade-off knobs. `maxSurge`: extra pods above the replica count allowed during rollout (needs quota). `maxUnavailable`: replicas allowed to be down. With $R=4$: `surge 1 / unavailable 0` never serves below 4 and costs one extra pod; `surge 0 / unavailable 1` costs nothing but serves a window at 3. `0/0` cannot roll at all — a common copy-paste wedge.
- Q15. What is endpoint-removal lag during scale-down?** “Pod terminating” and “endpoints object updated” are concurrent, and downstream convergence — kube-proxy iptables programming, DNS TTLs, cloud LB target lists, gateway discovery — takes seconds more. So a pod can stop accepting (post-SIGTERM) while the LB still opens new connections to it: instant 502s. Mitigation: `preStop` sleep so the process keeps serving through the lag, plus client retries for the residue (Chapter 16).
- Q16. How does an HPA compute replicas, and what is the stabilization window?** Every sync period: $\lceil \text{current} \times \text{currentMetric} / \text{targetMetric} \rceil$, where resource utilization is measured against *requests* (inflated requests silently disable autoscaling). The scale-down stabilization window (default 300 s) takes the *highest* recommendation in the window, because every scale-down terminates connections; scale-up is typically immediate. Downscale policies cap pods removed per period.
- Q17. Design a canary release with automated rollback — the sketch?** Small traffic steps (1/5/25/50/100% or request-budget batches), stable splitting (hash on API key/tenant so a client

doesn't oscillate between versions), a pre-registered gate per step — error-rate SPRT with budgets α/β plus a hard latency-quantile guardrail, as in Listing 18.3.6 — and rollback defined as the same weight change in reverse. Prerequisites: expand-and-contract schema so old code still reads new data, and per-version metric labels so the gate can see the canary at all.

- Q18. Gateway vs. ingress vs. service mesh?** Ingress is the Kubernetes-standard vocabulary for north-south HTTP routing plus TLS termination — a gateway subset. An API gateway is the product tier at the platform edge: auth offload, rate limiting, transformation, analytics — the policy enforcement point for north-south traffic. A mesh puts proxies on every workload to govern *east-west*: mTLS, retries, telemetry between services. Production commonly runs all three in sequence; the failure mode is enforcing one concern in two tiers with different budgets.
- Q19. Where should TLS terminate for a public API?** At the provider edge with automated public certificates (ACME), then *re-encrypt* to backends with an internal CA — or let a mesh's mTLS cover it. Edge-only termination leaves the last hop plaintext (bearer tokens on the wire inside your VPC); passthrough gives end-to-end encryption but blinds the WAF and L7 routing entirely. The decision is: which tiers need to *see* the traffic versus merely *carry* it.
- Q20. Why does a container need to care about PID 1?** The kernel delivers signals to PID 1 only if a handler is installed, and orphans re-parent to PID 1 for reaping. A shell wrapper as PID 1 swallows SIGTERM (the shell has no handler); Kubernetes then waits the grace period and SIGKILLs, severing in-flight work. Fix: exec-form ENTRYPOINT so the server *is* PID 1, and a server (uvicorn) that installs real handlers.

Senior (Q21–Q28)

- Q21. Walk me through every system that must cooperate for a request to survive a rolling deploy.** Endpoint controller removes the pod (async); preStop sleeps through LB convergence; SIGTERM; uvicorn stops accepting, drains within its graceful timeout; `terminationGracePeriodSeconds` exceeds preStop + slowest request; LB draining stops new connections; clients retry the residue with backoff and idempotency keys (Chapter 11); the new pod's startup probe tolerates cold boot; readiness gates its traffic.

Any link missing and some fraction of requests dies.

Q22. Why is the SPRT better than “watch the error rate for ten minutes”? A fixed-horizon eyeball has uncontrolled error rates and no early stopping: a catastrophically bad canary burns ten minutes of damage, a healthy one waits ten minutes for nothing. The SPRT pre-registers hypotheses (p_0 , $p_0+\delta$) and error budgets (α , β), then decides the moment cumulative evidence crosses a boundary — typically a few hundred requests either way — with the false-promote and false-rollback rates *guaranteed* by construction. It also forces the honest conversation upfront: what regression size δ do we actually care about?

Q23. Your p99 spiked after you “hardened” resources. No deploy. Diagnose. The fingerprint: p99 quantized near 100 ms multiples, mean CPU far below limit, throttle counters climbing — you added a CPU limit and CFS throttling is freezing all threads mid-request for the remainder of each quota period. Verify with `container_cpu_cfs_throttled_periods_total`. Fix: drop the CPU limit (keep requests) or raise it well above p99 burst usage; consider fewer worker processes so per-period quota lasts longer. Memory “hardening” would have shown OOMKills instead — different graph, same meeting.

Q24. How do you size a canary’s traffic steps? By *requests*, not percentages: the gate needs enough canary volume to detect the regression you care about. At baseline p_0 and intolerance δ , expected requests-to-decide scale roughly with $1/\delta^2$; at $p_0=0.4\%$ and $\delta=1$ pt that is a few hundred requests per step (Listing 18.3.6 resolves in 600), while a 1% slice of a low-traffic API might take hours — so either accept coarser δ , lengthen windows, or synthesize load. Step count trades blast radius against total rollout time.

Q25. Feature flags or canary deploys? Different axes. Deployments change binaries; flags change code paths within one binary. Flags give per-tenant targeting and instant kill-switches but accrue combinatorial debt and live in the release path forever; canaries bound blast radius with zero code changes but run two contract versions simultaneously. Mature orgs compose them: canary the binary rollout, flag the risky feature *within* the new binary, and never let a flag outlive its rollout.

Q26. The migration must ship with the code. Order them. Expand-and-contract, never destructive-first: (1) expand migra-

tion (add the new column/table; nothing renamed or dropped); (2) deploy code that writes both shapes and reads tolerantly; (3) backfill if needed; (4) after *every* old binary is gone, contract migration (drop the old shape). Violating the order makes rollback impossible: restored old code meets a schema it cannot read. This is Chapter 10's discipline enforced by the deployment pipeline, not by memory.

Q27. Why does the Service selector in Listing 18.3.2 omit the `track` label, and what does that buy and cost? Omitting `track` makes one Service span stable and canary pods, splitting traffic by ready-pod ratio — free canary weights with zero infrastructure. Cost: weights are quantized to pod counts (1/5, 2/6, ...), there is no session stability (one client can oscillate between versions), and autoscaling the stable track silently moves the split. Gateway weighted clusters (Envoy) fix all three at the price of a control plane.

Q28. What belongs in a readiness check — and what must never be in a liveness check? Readiness may shallowly check *critical* dependencies (can I reach the database with a sub-100-ms probe?) because its consequence is only traffic removal. Liveness must check nothing external: its consequence is restart, and restarting cannot fix someone else's outage — it only multiplies it. Deepest-check discipline: liveness < readiness < nothing at request time; a dependency-deep liveness probe is a cascading-failure machine (Chapter 16).

Staff (Q29–Q36)

Q29. Design the deployment safety system for a platform with 200 API services. Paved road, not gate: one blessed base image (distroless, digest-pinned, rebuilt on CVE), one manifest template with the probe/preStop/PDB/resource decisions pre-made and parameterized, progressive delivery as the *only* rollout mechanism with platform-wide gates (error SPRT, latency guardrails, SLO burn rate), schema-change pipelines that physically cannot run contract before code, and automatic rollback as default with humans paged on gate *failure to decide*, not on every rollout. Measure: change-failure rate, time-to-rollback, and percentage of services off-template.

Q30. When is a service mesh *not* worth it? When the east-west concerns it buys (mTLS, L7 retry policy, traffic telemetry, fine-

grained splits) are already solved or not needed: few services, one team, flat compliance requirements, or a gateway already owning the splits you need. Costs are real: a proxy on every hop's latency budget, a control plane to operate, and a debugging story that now includes someone else's iptables. The 5-service monorepo does not need Istio; the 200-service regulated platform probably cannot live without it.

- Q31. How do you make rollbacks boring?** Every property that makes rollbacks rare makes them safe: immutable digest-pinned artifacts (rollback is a pointer move, never a rebuild); expand-and-contract schemas (old code always reads current data); forward-compatible APIs (Chapter 10) so consumers tolerate either version; automated rollback triggers rehearsed on every deploy rather than saved for emergencies; and fast rollbacks measured and SLO'd like any other operation. The SRE framing: release engineering is risk management, and rollback is the cheapest risk control you own [Bey+16].
- Q32. A vendor gateway, NGINX, or Envoy-as-gateway for a new platform — how do you decide?** Score on the matrix of Table 18.1 plus two columns the table can't hold: operational ownership (who pages for it at 3 a.m. — a vendor SLA or your team) and extensibility trajectory (plugin ecosystems and Wasm beat config-file reloads the moment policy gets bespoke). Default heuristic: if you will run a mesh within two years, standardize on Envoy-flavored everything now; if you need a developer portal and monetization, that's gateway-product territory (Kong class); if the traffic profile is simple and the team's depth is NGINX, boring wins.
- Q33. How do you set error budgets α , β , and δ for an automated canary gate?** From the SLO, backwards: δ is the regression size that would burn meaningful error budget if it reached 100% for one detection window; α (false promote) is priced against rollback cost — cheap rollback buys aggressive α ; β (false rollback) is priced against developer velocity — flaky gates train teams to bypass gates, which is how gates die. Then verify with the simulator: replay historical incident metric series through the gate and confirm it catches every real regression *and* passes the last twenty healthy deploys. A gate that fails the replay is a gate you have not designed.
- Q34. What is the blast-radius math behind progressive delivery?** Expected harm of a bad deploy $\approx$ (probability of regression) $\times$ (af-

fect fraction) $\times$ (time to detect + time to mitigate). Progressive delivery attacks all three factors: canary slices shrink the fraction, sequential gates shrink detection time from SLO-burn to seconds-of-evidence, and weight-based rollback shrinks mitigation to a pointer change. The complementary lever is shrink-the-unit: smaller, more frequent releases reduce the probability term, which is why release cadence and deployment safety are the same investment [Nyg18].

- Q35. How do you handle stateful connections (WebSockets, gRPC streams) under everything in this chapter?** Long-lived connections break every assumption above: draining is hours not seconds, L7 load balancing pins streams to pods, and HPA-on-CPU misses connection-count imbalance. The playbook: advertise *max connection age* and make clients honor it (gRPC `MAX_CONNECTION_AGE`, WS renegotiation), so drains complete in bounded time; rebalance deliberately on scale events; size `terminationGracePeriodSeconds` to connection lifetime, not request latency; and consider a dedicated connection-tier deployment with its own, gentler rollout policy (Chapter 12, Chapter 14).
- Q36. Regulated industry: what changes in this chapter?** Everything gets evidence. Images: signed (cosign) and admitted by policy, digests everywhere, SBOMs per release. Probes and rollout params: change-controlled, reviewed like code. TLS: passthrough or re-encrypt with an internal CA — plaintext last-hops stop being a judgment call. Secrets: external vault with rotation and audit, never cluster Secrets alone. Canary gates: thresholds become compliance artifacts, and the analyzer’s decision log is a record you produce in audits. The engineering is the same; the difference is that “why” must be written down *before* the deploy, not reconstructed after the incident.
- Q37. What would you instrument to know, a year from now, that this chapter’s discipline is working?** Leading indicators: percentage of deploys that are progressive with gates (target 100%), rollback mean-time and rollback *frequency* (both should be boring), gate false-positive rate (drives bypass behavior), probe-related incidents (flapping endpoints, boot-deadline crash loops — target zero), CFS-throttling alerts on latency-critical services, deploy-time 5xx rate (the case study’s metric), and drift of running digests from declared releases. The meta-metric: whether the on-call can explain any of these numbers without opening a dashboard first.

18.8 External Bibliography & Further Reading

- **Kubernetes documentation — Pod lifecycle and probes:** *Container Probes* and *Pod Lifecycle* (<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>). The authoritative semantics for startup/readiness/liveness behavior, termination ordering, and `terminationGracePeriodSeconds` that Section 18.2.4 compresses.
- **Kubernetes documentation — Horizontal Pod Autoscaler:** <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. The scaling algorithm, metric types, and the behavior stabilization windows and policies used in Listing 18.3.2.
- **Kubernetes documentation — Disruptions:** <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>. Voluntary vs. involuntary disruption and PDB arithmetic — required reading before your first node-pool upgrade.
- **Docker — best practices for writing Dockerfiles:** https://docs.docker.com/develop/develop-images/dockerfile_best-practices/. Layer caching, instruction ordering, and multi-stage builds; pair with the *Overview of best practices for building images* page for the attack-surface framing of Section 18.2.1.
- **Envoy proxy documentation:** <https://www.envoyproxy.io/docs/envoy/latest/>. Weighted clusters, the JWT authentication filter, local/global rate limiting, and the xDS APIs that make Envoy the substrate of modern gateways and meshes.
- **Kong Gateway documentation:** <https://docs.konghq.com/gateway/>. Plugin reference (JWT, rate-limiting, request-transformer, canary) and the declarative `deck` configuration model.
- **Google SRE book, chapter 8 — *Release Engineering*:** Betsy Beyer et al. ([Bey+16], also at <https://sre.google/sre-book/release-engineering/>). The canonical argument that releases are a risk-management discipline, not an afterthought.
- **Bilgin Ibryam & Roland Huß, *Kubernetes Patterns* (O'Reilly, 2nd ed.):** the pattern vocabulary behind this chapter's manifests — *Predictable Demands* (requests/limits), *Self Awareness*, health probes, and the lifecycle patterns that justify `preStop` and termination budgets.

- **The Twelve-Factor App** [Wig17]: factors III (config in the environment), IV (backing services as attached resources), and IX (disposability: fast startup, graceful shutdown) are this chapter’s process discipline in manifesto form.
- **OWASP Cheat Sheet Series — Docker Security and Kubernetes Security**: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html and the Kubernetes Security cheat sheet. The checklist form of Section 18.4’s container and manifest anti-patterns.
- **Michael Nygard, *Release It!* (2nd ed.)** [Nyg18]: stability patterns and antipatterns whose production war stories motivate the connection-draining and rollout-safety sections.
- **Mark Richards & Neal Ford’s deployment chapters in *Fundamentals of Software Architecture*** and Sam Newman’s deployment discussion in *Building Microservices* [New21]: progressive delivery placed in its architectural context.
- **Wald, *Sequential Analysis* (1947)** — for the mathematically inclined, the original SPRT construction behind Listing 18.3.6; a gentler modern treatment is any mathematical-statistics text’s chapter on sequential testing.

18.9 Capstone Projects

Capstone 17.1 — Harden the Image [junior]

Objective: Take a naive single-stage Dockerfile for the Orders API and rebuild it to this chapter’s standard.

Inputs/Datasets: Listing 17.1; a “naive” Dockerfile you write first (full base image, root, COPY . . ., shell entrypoint); no datasets.

Steps: (1) Write the naive version and record its image size and layer count. (2) Apply, one commit at a time: multi-stage split, slim or distroless runtime, non-root USER with fixed UID, .dockerignore, cache-ordered COPY, HEALTHCHECK, exec-form ENTRYPOINT. (3) Record size and (if a scanner is available) finding count after each step. (4) Produce a table of the deltas with one sentence of justification per step.

Acceptance Criteria:

- Final image is $\leq 20\%$ of the naive image’s size (or you document precisely why not).

- ☐ `docker inspect` shows a non-root user and an exec-form entrypoint.
- ☐ A one-line source edit rebuilds only the source layer (build log evidence).
- ☐ No `.env`, `.git`, or test artifacts in any layer (`docker history / dive` evidence).

Stretch Goals: Produce both slim and distroless variants and document the one operational capability (preStop `exec sleep`, in-container debugging) you traded away with distroless.

Capstone 17.2 — Prove the Drain Under Load [mid]

Objective: Extend the graceful-shutdown demo from one in-flight request to a realistic mix, and make the demo CI-grade.

Inputs/Datasets: Listings 17.3–17.4; synthetic traffic mix: 70% fast POST `/orders`, 25% `/healthz`, 5% slow `/orders/export`.

Steps: (1) Rewrite the driver to run N concurrent workers generating the mix at a fixed request rate while the signal arrives at a random-but-seeded moment. (2) Count and classify outcomes: completed, refused-after-signal (expected), and *severed* (the failure class). (3) Sweep `timeout_graceful_shutdown` $\in \{0, 1, 10\}$ s and tabulate severed counts. (4) Wrap the whole experiment as a pytest that asserts zero severed requests at the production timeout.

Acceptance Criteria:

- ☐ Three verdict classes reported per run; totals reconcile exactly with requests attempted.
- ☐ Zero severed requests at the production timeout; nonzero at `timeout=0`; you can explain both numbers.
- ☐ Seeded and offline: two runs produce identical classification counts.
- ☐ pytest integration exits nonzero on any severed request.

Stretch Goals: Add a preStop-equivalent delay to the driver (keep serving k seconds post-signal) and show the refused-after-signal count drop to zero.

Capstone 17.3 — Manifest Review Board [mid]

Objective: Run Listing 17.2’s manifest set through a formal design review and fix what you find.

Inputs/Datasets: Listing 18.3.2 and its companions, saved as YAML files; `pyyaml` for the linting script; a seeded list of injected faults (you create:

remove the PDB, set `minAvailable: 4`, add `limits.cpu: 100m`, delete the `preStop` hook, hardcode `DATABASE_URL`, set `terminationGracePeriodSeconds: 5`, copy readiness into liveness).

Steps: (1) Write `manifest_lint.py`: parse each YAML document and assert this chapter’s rules (probes distinct paths; memory request = limit; no CPU limit or a justified one; PDB present and arithmetic consistent with replicas; no literal secret-looking strings; `preStop` present; grace period > slowest endpoint budget). (2) Verify the linter passes the clean set and catches every injected fault, naming the rule violated. (3) Write the review minutes: for each caught fault, the failure story it prevents.

Acceptance Criteria:

- ☐ Linter catches all seven injected faults with rule-specific messages.
- ☐ Linter passes the clean manifest set.
- ☐ Every lint rule cites its chapter section.
- ☐ Review minutes contain a failure story per fault, not restatements of the rule.

Stretch Goals: Add a second reviewer mode that diffs two manifest revisions and flags only *safety-relevant* changes (probes, resources, lifecycle, grace periods).

Capstone 17.4 — Gate Tuner

[senior]

Objective: Turn Listing 18.3.6 from a demo into a defensible rollout policy.

Inputs/Datasets: `canary_analysis.py`; a seeded fixture generator you extend with a third case: a canary whose regression is exactly 0.5δ (below the intolerance line — the gate *should* promote it, eventually).

Steps: (1) Sweep $\delta \in \{0.5, 1.0, 2.0\}$ points and $\alpha = \beta \in \{0.01, 0.05, 0.10\}$ over the three fixtures; record requests-to-decision and verdict for each cell. (2) Add a latency SPRT (Gaussian approximation on per-batch means) alongside the hard p95 gate and compare detection times. (3) Derive, on paper, the expected LLR drift per request as a function of the true canary rate, and verify the simulator’s detection latencies match within a batch. (4) Write the one-page rollout policy the numbers justify.

Acceptance Criteria:

- ☐ Sweep table shows the $\delta/\alpha/\beta$ trade-off: tighter intolerance costs more requests per decision.
- ☐ The 0.5δ canary is promoted (never rolled back) in every cell.

- ☐ Measured detection latency matches the drift derivation within one batch size.
- ☐ The policy page names δ , α , β , batch size, and the rollback action — and cites the evidence for each.

Stretch Goals: Add a seasonal-baseline mode: compare the canary against a time-shifted baseline (same hour last week) and quantify what seasonality does to the SPRT’s assumptions.

Capstone 17.5 — Zero-Downtime Reference Architecture [staff]

Objective: Design and document the complete deployment architecture for Northwind Robotics’ public API platform — the chapter, applied end to end.

Inputs/Datasets: All chapter listings; the platform brief: 12 services (3 latency-critical), mixed REST and gRPC, one WebSocket telemetry service, PCI-adjacent payments path, two regions.

Steps: (1) Choose and justify the edge stack (CDN, WAF mode, TLS posture per traffic class) and the gateway/mesh split; draw the request-path diagram for the payments path. (2) Define the manifest template (probes, resources policy, preStop, grace periods) with per-service-class parameters. (3) Define the progressive-delivery standard: steps, gates (δ , α , β per SLO tier), rollback semantics, and the expand-and-contract pipeline stage that blocks destructive migrations. (4) Specify the long-lived-connection strategy for WebSocket/gRPC (max connection age, rebalancing, drain budgets). (5) Write the incident runbook for “canary gate refuses to decide.” (6) Deliver as an architecture decision record set: one ADR per load-bearing choice, each with the rejected alternatives.

Acceptance Criteria:

- ☐ Every ADR names the rejected alternatives and the reversal cost.
- ☐ The payments path has an explicit TLS-termination decision with a compliance rationale (no plaintext last hop).
- ☐ The rollout standard includes per-tier gate parameters *and* the evidence (simulator runs) that they detect the stated regression sizes.
- ☐ The WebSocket service’s drain budget is derived from measured connection lifetimes, not defaults.
- ☐ A peer can run Capstone 17.3’s linter against your template and find zero violations.

Stretch Goals: Add the multi-region failover half: health-based DNS shifting, what “rollback” means when the failure is a region, and where the canary gates live when traffic itself moves.

18.10 Python Dependencies Appendix

All runnable code is offline and localhost-only after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

requirements.txt:

```
1 fastapi>=0.110
2 uvicorn>=0.29
3 httpx>=0.27
4 pytest>=8
5 pyyaml>=6.0
```

18.10.1 fastapi (≥ 0.110)

Description: The ASGI framework the Orders API is written in. **Why included:** Listings 17.3–17.4 containerize and terminate a FastAPI service; its middleware is the in-flight ledger the drain proof relies on [Ram24]. **Alternatives:** Starlette directly (the middleware pattern is raw ASGI and ports unchanged); Flask (WSGI — no async lifespan semantics, weaker fit for graceful-shutdown teaching). **Installation:** `pip install "fastapi>=0.110"` in the activated venv (PowerShell: keep the quotes so `>=` is not parsed as redirection).

18.10.2 uvicorn (≥ 0.29)

Description: The ASGI server whose signal handling is the subject of Listing 18.3.4: on `SIGTERM` (or `SIGBREAK` via `CTRL_BREAK_EVENT` on Windows) it stops accepting, waits for in-flight requests up to `timeout_graceful_shutdown`, runs lifespan shutdown, and exits. **Why included:** graceful shutdown cannot be taught as folklore — this chapter proves it against the real server. **Alternatives:** hypercorn (comparable lifespan semantics); gunicorn with uvicorn workers (the classic production shape whose own signal choreography — master vs. worker — is an interview topic in itself). **Installation:** `pip install "uvicorn>=0.29"`.

18.10.3 httpx (≥ 0.27)

Description: Sync/async HTTP client. **Why included:** it backs FastAPI's `TestClient` for any tests you write around the listings, and is the natural client for extending the lab's load mix. (The driver itself deliberately uses `stdlib urllib` so the proof depends on nothing but Python.) **Alternatives:** `requests` (synchronous only); raw `urllib` (what the driver uses — proof that you can, not that you should). **Installation:** `pip install "httpx>=0.27"`.

18.10.4 pytest (≥ 8)

Description: Test framework. **Why included:** the lab and capstones wrap the demos as executable assertions (e.g., “zero severed requests”); exit codes from the demo driver map cleanly onto pytest outcomes. **Alternatives:** `unittest` (`stdlib`; serviceable, more ceremony). **Installation:** `pip install "pytest>=8"`.

18.10.5 pyyaml (≥ 6.0)

Description: YAML parser. **Why included:** Capstone 17.3's manifest linter parses Listing 17.2's YAML documents; parsing is the honest way to review manifests — grepping YAML for `readOnlyRootFilesystem` misses indentation-level semantics that `yaml.safe_load` gets right. **Alternatives:** `ruamel.yaml` (round-trips comments; heavier); `kubeconform` / `confest` (real schema/policy validation when you graduate from teaching lint rules). **Installation:** `pip install "pyyaml>=6.0"`.

Toolchain (not pip — install with winget)

The chapter's reviewed-not-applied artifacts come alive with three tools, all optional here: **Docker Desktop** (`winget install Docker.DockerDesktop`) to build Listing 17.1 and feel the layer cache; **kubectl** (`winget install Kubernetes.kubectl`) plus a local cluster (`kind` — `winget install Kubernetes.kind` — or Docker Desktop's bundled Kubernetes) to actually apply Listing 17.2 and watch probes and rollouts in `kubectl get pods -w`; and **k6** (`winget install k6.k6`) to put real load on the canary endpoints instead of synthetic fixtures. None is required: every behavior this chapter claims is either proven by the runnable listings or stated as platform semantics you will verify on first contact with a cluster.

18.11 Chapter Summary & Key Takeaways

- A container image is a content-addressed layer stack: pin digests, split build from runtime, drop root, make the filesystem read-only, and treat image size as attack-surface accounting — every line of Listing 17.1 is a security decision.
- Inside the container you are PID 1: exec-form entrypoints and real signal handlers are what make **SIGTERM** mean “drain” instead of “die.” The four-act sequence — stop accepting, drain, timeout, exit — is provable offline (Listings 17.3–17.4), and each act has a platform knob (readiness, graceful timeout, `terminationGracePeriodSeconds`) that must agree with the others.
- The gateway is the policy enforcement point for north-south traffic: JWT offload, rate limiting, transformation, and canary routing configured once (Table 18.1); ingress is its portable subset; the mesh owns east-west. Enforce each concern in exactly one tier.
- Probes are a recovery theory, not boilerplate: startup budgets boot time, readiness moves traffic without restarts, liveness restarts only process-local death — and Scenario B of the simulator shows the `CrashLoopBackOff` that results from conflating them.
- Requests schedule, limits throttle: CPU limits quantize latency into 100 ms stalls; memory limits OOM-kill. Set memory request = limit, CPU request always, CPU limit deliberately.
- HPA scales on metrics against requests; stabilization windows and downscale policies are connection-protection, and PDBs are the arithmetic that keeps voluntary disruptions inside the SLO.
- Progressive delivery migrates traffic, not versions: size canary steps by requests, gate them with a sequential test with pre-registered δ , α , β (Listing 18.3.6), and make rollback a pointer move — which requires expand-and-contract schemas (Chapter 10).
- TLS placement is a visibility-versus-encryption decision per hop: terminate at the edge, re-encrypt inside, reserve passthrough for regulated payloads; automate every certificate; let the CDN cache only what is anonymous and keyed correctly.
- Termination is a concurrent race among **SIGTERM**, endpoint removal, and LB draining; the `preStop` sleep exists to rig it, and the missing `preStop` in Section 18.5.2 is what 0.1% dropped requests looks like

from the inside.

18.12 Quick-Reference Card

Probe Behavior Table

Gating: startup runs alone until first success; readiness and liveness begin only afterwards. Endpoint removal lags by seconds (kube-proxy, DNS, LB convergence) — rig the race with `preStop` sleep.

Rollout & Autoscaling Cheat-Sheet

`maxSurge: 1, maxUnavailable: 0` $\Rightarrow$ capacity never dips, costs one surge pod • `maxUnavailable: 1, maxSurge: 0` $\Rightarrow$ no extra quota, serves at $R-1$ windows • `progressDeadlineSeconds` fails wedged rollouts loudly • `revisionHistoryLimit: 5` keeps rollbacks instant • `terminationGracePeriodSeconds` = `preStop` + slowest request + margin • HPA: $\lceil \text{current} \times \text{metric} / \text{target} \rceil$ against *requests* • `scaleDown` window 300 s + ≤ 1 pod/60 s; `scaleUp` immediate • PDB: `minAvailable = R - 1` for rollouts to proceed; never $= R$ (deadlocks drains); canary weights by pod ratio: canary pods / total pods.

TLS Termination Matrix

Certificates: ACME (Let's Encrypt) at the edge; cert-manager or mesh issuance inside; a manually renewed certificate is an outage with a calendar date.

18.13 Standardized Evidence Addendum

18.13.1 Benchmark Snapshot Template

18.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

18.13.3 Cost and Latency Budget Snapshot

18.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

18.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

18.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

18.17 Cross-Chapter Decision Card

18.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

18.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Capability	Kong	Envoy	NGINX (OSS / Plus)
Core proxy engine	NGINX OpenResty (Lua)	+ purpose-built C++ L4/L7 proxy	NGINX
Routing	rich: host/path/method/header/ plugins	very rich: L7 match, weights, retries	host/path/regex map; headers via config
Auth offload (JWT)	first-class plu- gin (JWKS, claims checks)	JWT authn filter (JWKS, claims)	Plus native; OSS via njs/Lua mod- ules
Rate limiting	plugin (lo- cal/Redis, per con- sumer/route)	local token bucket + global rate-limit ser- vice	<code>limit_req</code> (GCRA); Plus adds key-value store
Request transformation	plugin ecosys- tem (Lua)	Lua/Wasm filters; header/body mutation	<code>sub_filter</code> , headers, njs scripting
Canary routing	traffic-split plu- gin, weighted upstreams	native weighted clusters (first- class)	<code>split_clients</code> / Plus API
Extensibility model	plugin hub + custom Lua/Go/JS/Python	Wasm/Lua/ext_p- modules at the mesh data- plane standard	modules at compile time; njs runtime
Config style	declarative (DB or deck YAML), dynamic API	xDS APIs (built for con- trol planes)	static files + reload (Plus: dynamic API)
Operational posture	gateway-first, admin API, enterprise suite	building block: gate- ways/meshes embed it	ubiquitous, minimal deps, least dynamic

Table 18.1: Gateway capabilities matrix. Envoy is the substrate many gateways and meshes (Istio, Consul, Contour, Gloo) are built from; Kong is a batteries-included gateway on NGINX; NGINX itself is the minimal, static-configuration baseline.

	Startup	Readiness	Liveness
Question	Has the app finished initializing?	Should this Pod receive traffic <i>right now</i> ?	Is the process unrecoverably stuck?
On failure	Retried within budget (period $\times$ failureThreshold)no restart, no budget exhausted $\Rightarrow$ container restarted	Pod removed from Service endpoints ; no restart, no reschedule; re-added on first success	Failure streak $\Rightarrow$ container restarted in place (same Pod, new container)
Gates	Disables liveness & readiness until first success	Nothing; evaluated only after startup succeeds	Nothing; evaluated only after startup succeeds
Should fail when	Still booting	A <i>recoverable</i> /external problem: dependency down, overloaded, draining	A <i>process-local</i> death: deadlock, wedged event loop
Typical check	/healthz/startup (cheap, no deps)	/healthz/ready (may check critical deps, shallowly)	/healthz/live (no deps — deadlocks must be the only failure)

Table 18.2: Probe semantics. The load-bearing row is the second: readiness failure changes *routing*, liveness failure changes *process state*. A readiness probe that also gates liveness restarts Pods for other services’ outages; a liveness probe that checks dependencies turns a database blip into a fleet-wide restart storm.

maxSurge	maxUnavailable	Serving floor ($R=4$)	Cost / risk
1	0	4	one surge pod's quota; zero capacity loss — the default for user-facing APIs
0	1	3	no extra quota; 25% capacity dip per rollout wave
1	1	3	fastest rollout; both costs at once
0	0	—	cannot roll at all; a copy-paste wedge, not a strategy

Table 18.3: Rolling-update arithmetic for a 4-replica Deployment. Percentages are allowed for both fields (rounded: surge up, unavailable down), which keeps the policy correct as HPA moves R .

Probe	Failure consequence	Checks
Startup	container restarted after period $\times$ threshold budget	boot complete (cheap, no deps)
Readiness	endpoint removed; re-added on first success; no restart	may touch critical deps, shallowly
Liveness	container restarted in place after threshold streak	process-local only; never dependencies

Mode	Edge sees	Last hop	Use when
Terminate at edge	full L7 (WAF, JWT, routing)	plaintext	private network you truly control; legacy
Re-encrypt	full L7	TLS to backend (internal CA / mesh mTLS)	default for production APIs
Passthrough	L4 only (no WAF, no L7 route)	end-to-end encrypted	regulated payloads; L7 policy unnecessary

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 18.4: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 18.5: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 18.6: Decision card to complete per chapter.

Chapter 19

Observability & SRE for APIs

Every chapter until now has asked a design-time question: how should the API be shaped, secured, versioned, and deployed? This chapter asks the runtime question that decides whether all of that work matters: *when the API misbehaves in production — and it will — can you see it, measure it, and respond before your error budget is gone?* An API without observability is a system whose failures are reported by customers, whose latency regressions are discovered in quarterly reviews, and whose outages are debugged by intuition. An API without SLOs has observability data but no judgment: dashboards nobody pages on, alerts everybody ignores, and no principled answer to “is this bad enough to stop the release?”

We build the answer in three layers. First, the **three pillars** applied concretely to APIs: structured JSON logs with levels, sampling, and a hard rule against PII and secrets; metrics organized as RED (rate, errors, duration) for services and USE (utilization, saturation, errors) for resources; and distributed traces with spans and context propagation. Second, the **machinery**: OpenTelemetry’s API/SDK/exporter/collector architecture [Ope24], the W3C **traceparent** header parsed byte by byte (version, trace-id, span-id, flags), **tracestate** and baggage, span kinds and HTTP semantic conventions, and the head-versus-tail sampling decision. Third, the **engineering of numbers**: histogram buckets and why percentiles must come from histograms (including a proof by example of the p99-of-averages fallacy), cardinality discipline with cost arithmetic, exemplars, SLI formulas, error budgets (43.2 minutes per month at 99.9%), and multi-window multi-burn-rate alerting with the full window-pair mathematics [Bey+16].

The operational half closes the loop: correlation IDs end to end (connecting the problem-details work of Chapter 8), per-endpoint dashboards anchored to SLOs rather than vibes, a systematic timeline-decomposition method for debugging production latency, and the incident-response apparatus — severity levels, roles, runbooks, and blameless postmortems with a template. All code is typed Python 3.11, offline-first (the OTLP

exporter is replaced by a file exporter; the collector is simulated), deterministic (seeded ID generators and timelines), and set in the Northwind Robotics fleet-telemetry universe. The war story is the invisible retry storm: a latency regression that multiplied traffic fivefold while every individual log line looked perfectly healthy — found through trace fan-out metrics, invisible in logs.

Key Concept

Observability answers “*what is the system doing?*”; SRE answers “*is that good enough, and what do we do when it is not?*” The two are useless apart: telemetry without SLOs is data hoarding, and SLOs without telemetry are wishes. The contract that binds them is the error budget — the only number in this chapter that is allowed to page a human being — and the mechanism that makes telemetry coherent across service boundaries is context propagation via the W3C `traceparent` header.

19.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Apply the three pillars to API services concretely: emit structured JSON logs with levels and sampling and no PII or secrets; define RED metrics for services and USE metrics for resources; construct traces with correct span kinds and context propagation.
- LO-2.** Explain the OpenTelemetry architecture — API, SDK, exporter, collector — and justify the collector as the boundary between application code and vendor backends [Ope24].
- LO-3.** Parse and emit the W3C `traceparent` header by hand (version–trace-id–span-id–flags), explain `tracestate` and baggage, and state exactly why context does *not* propagate itself through queues.
- LO-4.** Engineer latency metrics correctly: choose histogram buckets, compute percentiles from histograms, refute the p99-of-averages fallacy, budget label cardinality with explicit arithmetic, and use exemplars to jump from a metric to a trace.

- LO-5.** Design SLIs for availability, latency, and freshness; derive error budgets (43.2 minutes per month at 99.9%); and implement multi-window multi-burn-rate alerting with the fast/slow window-pair mathematics of the SRE Workbook [[Bey+16](#)].
- LO-6.** Run operational practice end to end: correlation IDs propagated to problem-details responses, SLO-anchored per-endpoint dashboards, timeline decomposition of a slow request, and incident response with severities, roles, runbooks, and blameless postmortems.
- LO-7.** Instrument a FastAPI service with traces, metrics, and correlated logs offline, and verify propagation with executable tests [[Ram24](#)].

Prerequisites

This chapter assumes Chapters 0–17, especially: building and testing FastAPI services (Chapter 7), structured errors and correlation-friendly problem details (Chapter 8), retries and resilience patterns (Chapter 16), API security boundaries — you cannot judge what may be logged without them (Chapter 17), and deployment mechanics (Chapter 18). Elementary probability (quantiles, ratios) and algebra are assumed; every formula is derived in place.

Setup

All listings run offline on Windows with Python 3.11+. One-time setup from PowerShell in a scratch directory:

```
1 python -m venv .venv
2 .\venv\Scripts\Activate.ps1
3 pip install "opentelemetry-sdk>=1.25" '
4     "opentelemetry-instrumentation-fastapi
5         >=0.46b0" '
6     "fastapi>=0.110" "pytest>=8"
```

No collector, no network, no credentials: spans are exported to a local JSON-lines file, metrics are read back from an in-memory reader, and every timeline is seeded. Section 19.10 notes what changes when you point the same code at a real OpenTelemetry Collector.

19.2 Deep Technical Mechanics & Architectural Concepts

19.2.1 The Three Pillars, Applied to APIs

“Logs, metrics, traces” is a slogan; the engineering content is in what each pillar is *for*, what it *costs*, and how they *join*.

Structured logs: events with a schema. A log line is a timestamped record of something that happened once. The moment an API serves real traffic, free-text logs stop being searchable and start being forensics-resistant, so the discipline is: emit JSON, one object per line, with a fixed schema — timestamp, level, logger, message, and *correlation fields* (trace id, span id). Levels are a contract with the reader: **ERROR** means a human should care, **WARN** means a human should eventually care, **INFO** means normal operation worth keeping, **DEBUG** means sampled or off in production. Volume control is part of the design: a hot endpoint logging every request at **INFO** is a denial-of-wallet attack on yourself, so high-frequency paths are sampled (keep 1 in N) while errors are always kept. And the hard rule, discussed again in Section 19.4: **no PII, no credentials, no tokens, no request bodies** — logs outlive their retention assumptions, leak into support tickets and vendor UIs, and are readable by people who should not see user data.

Metrics: aggregatable numbers. A metric is a number observed repeatedly and aggregated cheaply: counters (requests served), histograms (durations in buckets), gauges (queue depth). For API services the working vocabulary is **RED**: *Rate* (requests per second), *Errors* (failures per second, or error ratio), *Duration* (latency distribution). For the resources under the service, the complementary vocabulary is **USE**: *Utilization* (how busy), *Saturation* (how queued), *Errors* (how broken) — connection pools, worker processes, disk. RED tells you your users are hurting; USE usually tells you why.

Traces: causality with timestamps. A trace records one request’s journey as a tree of *spans* — named, timed intervals with a parent pointer. Where logs answer “what happened here?” and metrics answer “how often, how fast?”, traces answer the question microservices make urgent: “*where* did the time go, across the three services this request touched?” The trace/span model and its wire format get their own subsections below.

Definition 19.1 (Span, Trace, and Context Propagation). A *span* is a named, timed operation with attributes, a status, a span id, and an optional parent span id. A *trace* is the set of spans sharing one trace id, forming a

tree. *Context propagation* is the mechanism by which a caller transmits the trace id, its own span id, and sampling flags to a callee — for HTTP, via the W3C `traceparent` header — so the callee’s spans attach to the same tree.

Key Concept

The pillars join on **exemplars and correlation ids**: a metric histogram bucket carries an exemplar pointing at one trace; every log line emitted inside a span carries that span’s trace id; every problem-details response (Chapter 8) carries the same id for the client to quote back. One identifier, three pillars, zero archaeology.

19.2.2 OpenTelemetry Architecture: API, SDK, Exporter, Collector

OpenTelemetry (OTel) is the industry-converged instrumentation standard, and its layering is the first architecture decision to get right [Ope24]:

- **API** — the instrumentation surface your code calls: `tracer.start_as_current_span(...)` `meter.create_counter(...)`. Depending on the API is free; it does nothing until an SDK is installed.
- **SDK** — the implementation: span processors, samplers, metric aggregations, resource attribution (`service.name`). Applications configure the SDK; libraries must never.
- **Exporter** — the egress adapter: OTLP over gRPC/HTTP to a collector in production; in this chapter, a deliberately offline JSON-lines file exporter implementing the same interface, so the telemetry pipeline is testable with zero infrastructure.
- **Collector** — a standalone process receiving OTLP, processing (batching, redaction, tail sampling), and re-exporting to backends (Prometheus, Jaeger/Tempo, Loki/Elasticsearch, or a vendor). Putting the collector between app and backend is what lets you switch vendors by editing one YAML file instead of a fleet redeploy.

Two SDK details matter for API services. First, **span processors**: the production default is the *batch* processor (queue and flush every few seconds; the hot path only enqueues); this chapter’s offline listings use the *simple* processor (export synchronously on span end) because deterministic tests beat throughput here. Second, **metric views**: the SDK lets you re-shape an instrument’s aggregation — we use a view to pin the latency histogram

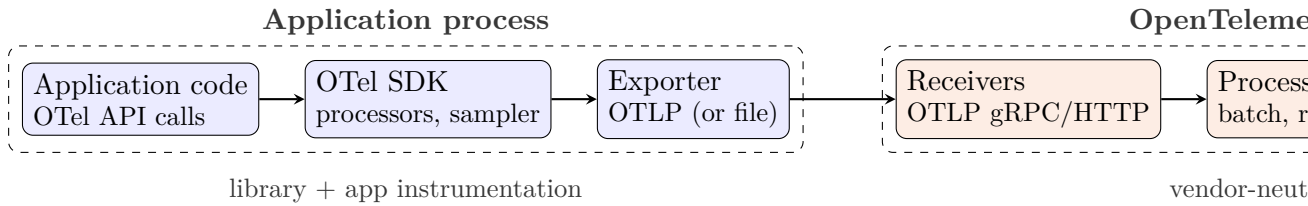


Figure 19.1: The observability pipeline. Application code depends only on the OTel API; the SDK assembles spans and metric aggregations; the exporter ships OTLP to the collector, which batches, redacts, optionally tail-samples, and fans out to one backend per signal. Vendor changes happen at the collector, never in application code.

to explicit millisecond buckets, which is what makes percentile math and backend cost predictable (Section 19.2.6).

19.2.3 The W3C Trace Context: **traceparent**, Byte by Byte

Context propagation across HTTP services is standardized by the W3C Trace Context recommendation. The **traceparent** header is exactly four dash-separated lowercase-hex fields:

Example

Decoding a live header. 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01 means: version 00; trace id 4bf92f3577b34da6a3ce929d0e0e4736 (128 bits); parent span id 00f067aa0ba902b7 (64 bits); flags 01 — bit 0 set, so this trace is *sampled*. Listing 18.1 parses this format strictly and Listing 18.2 proves the round trip.

Three companion mechanisms complete the picture. **tracestate** is a comma-separated list of at most 32 vendor **key=value** members carrying vendor-specific sampling or routing state; it is mutable hop by hop under strict size and syntax rules. **Baggage** is a separate W3C header carrying user-defined key/value pairs (e.g. tenant or plan tier) that propagate with the trace — powerful, dangerous (it travels *everywhere*, so never put PII in it), and intentionally not part of the span identity. And **queues**: nothing about an HTTP header helps you across a message broker. Propagation through a queue requires *explicitly* writing the trace context into the message metadata on publish and extracting it on consume (the same inject/extract pair, different carrier) — the async patterns of Chapter 12 do not do this for you. Section 19.4 lists the broken trace tree as a first-class anti-pattern.

Field	Width	Example	Rules
version	2 hex	00	<code>ff</code> is forbidden; version 00 forbids trailing fields; future versions may append fields, which parsers must tolerate.
trace-id	32 hex	4bf9...4736	All-zero is invalid. Identifies the whole trace; constant across every hop.
span-id	16 hex	00f067aa0ba902b7	All-zero is invalid. Identifies the caller's span; the callee records it as its <i>parent</i> .
flags	2 hex	01	Only bit 0 (0x01, <i>sampled</i>) is defined; other bits are reserved and must not be echoed as set.

Table 19.1: `traceparent` field reference (W3C Trace Context).

19.2.4 Span Kinds and Semantic Conventions for HTTP

OTel assigns every span a *kind* that states its relationship to its parent, and getting kinds right is what makes waterfall views and service-graph aggregations correct [Ope24]:

- **SERVER** — entry point of a remote call handled by this process (an inbound HTTP request). Parent came from headers.
- **CLIENT** — an outgoing remote call (an HTTP request this process makes). Its span id becomes the child's parent via the injected `traceparent`.
- **INTERNAL** — work inside the process (a cache lookup, a pricing calculation).
- **PRODUCER / CONSUMER** — asynchronous messaging; the consumer's parent may not be on the critical path of the producer's latency, which is why they are distinct kinds.

Semantic conventions then standardize attribute names so dashboards and backends are portable: `http.request.method`, `url.path` (templated route, never the raw path with ids), `http.response.status_code`, `server.address`, `http.route`. The single most consequential rule for APIs: **attributes use route templates** (`/fleet/{id}`), never concrete paths — a concrete path per request is a cardinality bomb, the subject of Section 19.2.6.

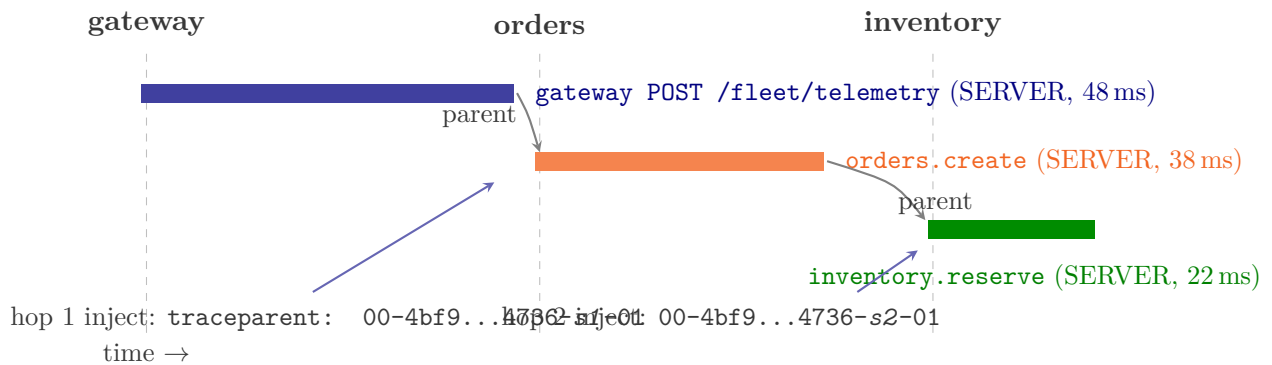


Figure 19.2: One trace across three services. Bars are spans on a shared time axis; every span carries trace id `4bf9...4736`; each hop injects a fresh span id into the outgoing `traceparent`, which the next service records as its parent. This is exactly the tree Listing 18.3 exports and asserts.

19.2.5 Sampling: Head, Tail, and the Errors You Cannot Afford to Lose

At 10,000 requests per second, keeping every trace is a storage and egress decision most platforms lose. Sampling reduces volume; where the decision happens determines what you keep.

Head sampling decides at the trace’s first span: flip a weighted coin (keep $p\%$), record the verdict in the `sampled` flag, and propagate it so every downstream hop agrees. It is cheap, deterministic per trace, and statistically unbiased — and it throws away almost all of the rare, interesting traces: at $p = 1\%$, you keep roughly 1% of your *error* traces too. **Tail sampling** decides at the collector after the whole trace is buffered: keep every trace containing an error or a p99-plus latency, sample the healthy rest. It keeps exactly the traces you debug with, at the price of collector memory, decision latency, and the operational burden of buffering entire traces. The pragmatic hybrid: head-sample at a few percent for unbiased statistics, tail-sample at the collector to guarantee error and slow traces survive, and always record counters *before* sampling so metrics stay exact regardless.

Warning

Sampling happens *after* metrics, never before. If your request counters and latency histograms are derived from sampled spans, your RED metrics are wrong by the sampling factor — silently. Metrics must be aggregated from every request; traces may be sampled.

19.2.6 Metrics Engineering: Histograms, Percentiles, Cardinality

Why latency is a histogram, never an average. API latency is long-tailed: most requests are fast, a few are catastrophically slow, and the slow ones are your most valuable users doing the largest operations. The average hides exactly that. The fix is to record durations into *histogram buckets* — counters for $(0, 5]$, $(5, 10]$, $(10, 25]$, $(25, 50]$, $(50, 100]$, $(100, 250]$, $(250, 500]$, $(500, 1000]$, $(1000, 2500]$, $(2500, \infty)$ milliseconds — from which percentiles can be estimated at query time.

Definition 19.2 (Histogram percentile). Given cumulative bucket counts $C_0 \leq C_1 \leq \dots \leq C_k = N$ with upper bounds $u_0 < u_1 < \dots < u_k$, the q -th percentile estimate is the value interpolated within the first bucket where $C_j \geq \lceil qN \rceil$:

$$\hat{p}_q = u_{j-1} + (u_j - u_{j-1}) \frac{\lceil qN \rceil - C_{j-1}}{C_j - C_{j-1}}. \quad (19.1)$$

Accuracy is bounded by bucket width: fine buckets where your SLO lives, coarse buckets in the tail. Percentiles are *estimated from counts*, never averaged from other percentiles.

Proposition 19.1 (The p99-of-averages fallacy, by counterexample). Per-minute averages of a latency distribution can lie arbitrarily far below the true p99. Take one minute with 1,000 requests: 999 at 10 ms and one at 10,000 ms. The minute’s average is $\frac{999 \cdot 10 + 10000}{1000} \approx 20$ ms. If every minute looks like this, the “p99 latency” computed from per-minute averages is ≈ 20 ms, while the true per-request p99 is 10,000 ms — a $500\times$ error, in the reassuring direction. Averaging first destroys the tail; percentiles must be computed from per-request data (histograms), and percentiles of percentiles or of averages are statistically void.

The same argument forbids averaging precomputed per-instance p99s across a fleet: quantiles do not compose under averaging. Merging is legal only at the histogram level (add the bucket counts, then apply Equation (19.1)) — which is precisely why the histogram is the exportable unit.

Cardinality discipline, with the cost math. Every distinct combination of label values is a new *time series* that must be stored, indexed, and scanned. Series count is multiplicative:

$$\text{series} = \prod_{\text{label } \ell} |\text{values}(\ell)|. \quad (19.2)$$

A request counter labeled by route, status class, and region with $5 \times 6 \times 3 = 90$ series is free. Add `user_id` with 50,000 tenants and you have

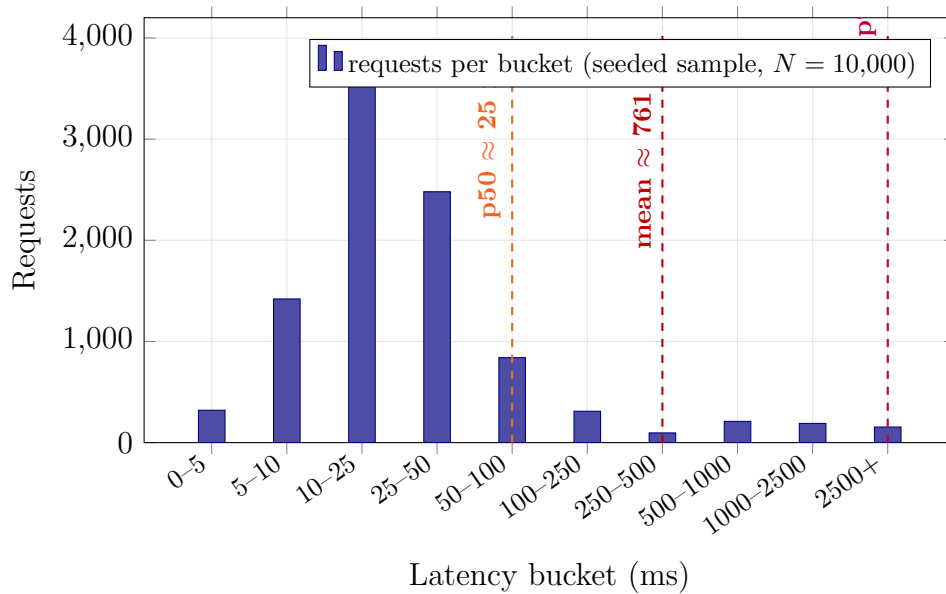


Figure 19.3: The long tail, plotted from Listing 18.4’s seeded distribution. Half of all requests finish in 25 ms, yet the *average* is 761 ms — pulled $30\times$ above the median by a tail the naked eye almost misses — and p99 is three buckets past the average. Any dashboard or SLO keyed on the mean is measuring a user nobody has.

$90 \times 50,000 = 4.5$ million series; at a few KB of index and chunk overhead per series, that is tens of GB of memory and a query planner that hates you — before counting ingestion cost per active series per scrape interval. The rule: labels may only be dimensions you would group by on a dashboard; anything unbounded (user ids, request ids, full URLs, error messages) belongs in logs and trace attributes, never in metric labels.

Definition 19.3 (Exemplar). An *exemplar* is a trace id pinned onto a specific histogram bucket observation. It is the bridge from “p99 spiked” (a metric) to “here is one concrete slow trace” (a waterfall) — the cheapest debugging accelerator in the entire stack, and the reason metrics and traces must share an ID space.

19.2.7 SLO Engineering: SLIs, Error Budgets, Burn Rates

Definition 19.4 (SLI, SLO, error budget). A *service level indicator* (SLI) is a quantitative measure of service behavior, expressed as a ratio of good events to total events. A *service level objective* (SLO) is a target on an SLI over a rolling window (typically 30 days). The *error budget* is the tolerated shortfall: $\text{budget} = 1 - \text{SLO}$. For a 99.9% availability SLO over a 30-day

month, the budget is $0.001 \times 43,200 \text{ min} = 43.2 \text{ minutes}$ of equivalent full outage per month [Bey+16].

Choosing SLIs for APIs is choosing a proxy for user pain. The three that cover nearly every API:

- **Availability:** $\text{SLI}_{\text{avail}} = \frac{\# \text{ requests with status} < 500}{\# \text{ requests}}$ per route. Count 5xx as failure; 4xx is the client’s budget problem, not yours (Chapter 8 responses excluded).
- **Latency:** $\text{SLI}_{\text{lat}} = \frac{\# \text{ requests faster than } T}{\# \text{ requests}}$, computed from histogram buckets against the SLO threshold T — e.g. “99% of `GET /fleet/status` under 300ms.” This is a threshold-count ratio, which is exactly what Equation (19.1)’s histogram answers.
- **Freshness** (data-serving APIs): $\text{SLI}_{\text{fresh}} = \frac{\# \text{ reads served with data newer than } D}{\# \text{ reads}}$ — the right SLI when the API’s job is recency (fleet positions, prices), where “fast but stale” is a silent failure mode.

SLO	Budget / 30 days	What it permits
99%	7 h 12 min	A bad deploy <i>and</i> a slow rollback, monthly.
99.9%	43.2 min	One memorable incident; change freezes near zero.
99.95%	21.6 min	Pages must be rare and mitigation rehearsed.
99.99%	4.32 min	Automation mitigates; humans mostly supervise.

Table 19.2: Error budgets by SLO target. The budget is a *spending allowance* shared by outages, risky deploys, and experiments — when it is spent, the roadmap tilts to reliability [Bey+16].

Burn-rate alerting: the math. Alerting on raw error counts pages for trivia and sleeps through budget catastrophes. The SRE Workbook’s answer is to alert on *burn rate* — how fast the budget is being consumed relative to its allotted pace [Bey+16]:

$$\text{burn rate} = \frac{\text{observed error rate}}{\text{budget rate}} = \frac{e_w}{1 - \text{SLO}}, \quad \text{budget consumed} = \text{burn rate} \times \frac{w}{43,200 \text{ min}} \quad (19.3)$$

A burn rate of 1 exhausts the budget exactly as the period ends; 14.4 exhausts it in $720/14.4 = 50$ hours — i.e. 2% of the monthly budget gone in one hour. Alerting on a single window forces a bad trade: a long window is precise but slow, a short window is fast but noisy. The *multi-window multi-burn-rate* pattern fires only when **both** a long window and a short window exceed the same burn-rate threshold:

$$\text{alert} \iff (\text{burn}_{\text{long}}(t) \geq b) \wedge (\text{burn}_{\text{short}}(t) \geq b). \quad (19.4)$$

The long window certifies the burn is sustained; the short window certifies it is happening *now* (so the alert auto-resolves and does not page on stale history). The canonical table:

Policy	Burn	Long	Short	Meaning
Fast page	14.4×	1 h	5 m	2% of budget in an hour: page now.
Slow page	6×	6 h	30 m	5% of budget in six hours: page soon.
Ticket	1×	3 d	6 h	10% of budget in three days: work-queue.

Table 19.3: Multi-window multi-burn-rate alerting for a 99.9% SLO (SRE Workbook parameters) [Bey+16]. Listing 18.5 implements and tests exactly this table.

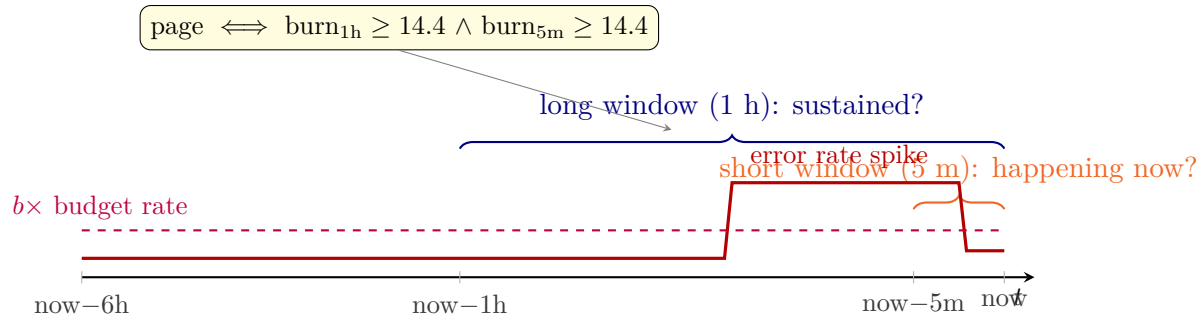


Figure 19.4: Burn-rate alert logic. The long window (blue brace) filters noise by requiring a sustained burn; the short window (orange brace) requires the burn to be active *at this instant*, so alerts clear promptly after mitigation instead of paging on history. Both must hold (Equation (19.4)).

Alert fatigue is a system property, not a personality flaw. Every page that does not require immediate human action teaches the on-call to ignore pages; enough false positives and a real page gets snoozed. The discipline: page only on user-facing symptoms measured as budget burn (fast/slow policies above); route everything else to tickets or dashboards;

delete or demote any alert that fired three times without action; and review the page budget monthly like any other budget. An alert that cannot name the SLO it protects is a dashboard panel wearing a pager costume.

19.2.8 Operational Practice: From Telemetry to Response

Correlation IDs end to end. Chapter 8 gave errors a `traceId`; this chapter gives it a source. The discipline: accept or mint a `traceparent` at the edge, propagate it through every internal call (headers for HTTP, message metadata for queues), stamp it on every log line, echo it in every problem-details body and support-facing error. The debugging loop becomes: client quotes id → one log query returns the whole request’s story → one trace lookup returns the cross-service waterfall.

Dashboards per endpoint, anchored to SLOs. A service dashboard that cannot answer “are we burning budget, and which endpoint is burning it?” is decoration. The layout that survives contact: row one is the SLO (28-day SLI, budget remaining, burn-rate sparkline); row two is RED per endpoint (rate, error ratio, duration percentiles from histograms); row three is USE for the resources (pool utilization, queue depth); row four is dependency health. Every panel cites the query; every query aggregates only low-cardinality labels.

Debugging production latency systematically. When p99 for POST /fleet/telemetry jumps from 180 ms to 900 ms, resist the log dive. Decompose the timeline with the trace waterfall: total span (900 ms) minus child spans ($\approx$ time in gateway code) → which child dominates? If `orders.create` grew, recurse; if children are unchanged but *gaps* between them grew, the time went to queuing, connection acquisition, TLS, or scheduling — none of which appear as spans unless you instrument them. The sequence is always: exemplar from the violated bucket → waterfall → dominant span or gap → USE metrics for that span’s resources → only then, correlated logs.

Incident response. Severity is declared in user-impact units, roles are assigned before diagnosis, and the postmortem is blameless or it is fiction [Bey+16]. Northwind Robotics’ table:

The roles that matter at 3 a.m.: the *incident commander* owns coordination and decisions (and does not debug); the *operations lead* debugs and mitigates; the *communications lead* updates stakeholders on a cadence. Runbooks — per service, with SLOs, triage steps, rollback commands, and

escalation paths — turn the first ten minutes from improvisation into checklist. And the blameless postmortem: timeline, mechanism-level root cause (“the deploy changed the connection-pool default,” never “Alex broke the pool”), what went well, what went poorly, action items with owners, and an explicit lesson for the SLO itself — did the alerts fire at the right time? Listing 18.6 generates both skeletons from metadata, because structure enforced by code survives stress better than structure enforced by memory.

19.3 Production-Ready Code Implementation

Everything below is typed Python 3.11, offline-first, and deterministic: the OTLP exporter is replaced by a JSON-lines file exporter, metrics are read back from an in-memory reader, trace ids come from a seeded generator, and every timeline is seeded. The universe is Northwind Robotics fleet telemetry. Save the listings under the indicated filenames and the printed commands reproduce every claim in this chapter.

19.3.1 Listing 18.1 — The W3C Trace Context, by Hand

`trace_context.py` is a `stdlib`-only parser and injector for `traceparent`, `tracestate`, and baggage. The design rule is strictness: a malformed header raises `ValueError` naming the offending field — silently repairing a broken header would fork the trace, and silently dropping it would hide a producer bug.

Listing 18.1 — `trace_context.py`: W3C header mechanics, `stdlib` only.

```
1  """Hand-rolled W3C Trace Context: parse and inject '
   traceparent' '.
2
3  Stdlib-only reference implementation of the wire format
   every API engineer
4  should be able to read cold:
5
6  traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00
   f067aa0ba902b7-01
7  |-----|
   |-----|
8  |trace-id(32hex, !=0) parent/span-id flags
```

```

9  version = (2 if hex, never "ff")
   (16 if hex, != 0)
   bit 0 = sampled

10

11  Design notes:
12  * Parsing is strict: malformed headers raise
   ValueError with the
13  offending field named. Silently "fixing" a broken
   header would fork
14  the trace; dropping it (returning 'None') hides
   producer bugs.
15  * Version "00" headers must be exactly 55 characters
   / 4 fields. Future
16  versions may append fields, so for version != "00"
   we accept extra
17  trailing fields and ignore them (forward
   compatibility).
18  * 'tracestate' and 'baggage' parsers are included
   because the three
19  headers travel together in production.
20
21  No network, no dependencies, no randomness:
   deterministic by construction.
22  ""
23
24  from __future__ import annotations
25
26  import secrets
27  from dataclasses import dataclass
28
29  HEX = frozenset("0123456789abcdef")
30  MAX_TRACESTATE_MEMBERS = 32 # per the W3C Trace
   Context spec
31
32
33  @dataclass(frozen=True)
34  class TraceParent:
35      """An immutable, validated 'traceparent' value."""
36
37      version: str
38      trace_id: str
39      span_id: str
40      sampled: bool
41
42      def __post_init__(self) -> None:
43          if len(self.version) != 2 or not set(self.

```

```

        version) <= HEX:
44         raise ValueError(f"version must be 2
            lowercase hex chars, got {self.version!r}
            ")
45     if self.version == "ff":
46         raise ValueError("version 'ff' is reserved
            (invalid) by the spec")
47     _check_hex_field("trace_id", self.trace_id, 32)
48     _check_hex_field("span_id", self.span_id, 16)
49     if self.trace_id == "0" * 32:
50         raise ValueError("trace_id of all zeros is
            invalid")
51     if self.span_id == "0" * 16:
52         raise ValueError("span_id of all zeros is
            invalid")
53
54     @property
55     def flags(self) -> str:
56         return "01" if self.sampled else "00"
57
58     def header(self) -> str:
59         """Render the canonical 'traceparent' header
            value."""
60         return f"{self.version}-{self.trace_id}-{self.
            span_id}-{self.flags}"
61
62
63     def _check_hex_field(name: str, value: str, length: int
        ) -> None:
64         if len(value) != length or not set(value) <= HEX:
65             raise ValueError(
66                 f"{name} must be exactly {length} lowercase
                    hex chars, got {value!r}"
67             )
68
69
70     def parse_traceparent(header: str) -> TraceParent:
71         """Parse a 'traceparent' header value, raising a
            ValueError if bad."""
72         if not isinstance(header, str) or not header:
73             raise ValueError("traceparent header must be a
                non-empty string")
74         if header != header.strip():
75             raise ValueError("traceparent must not carry
                surrounding whitespace")
76         parts = header.split("-")

```

```

77     if len(parts) < 4:
78         raise ValueError(
79             f"traceparent_needs_at_least_4_dash-
              separated_fields,_got_{len(parts)}"
80         )
81     version, trace_id, span_id, flags = parts[0], parts
      [1], parts[2], parts[3]
82     if len(flags) != 2 or not set(flags) <= HEX:
83         raise ValueError(f"flags_must_be_2_lowercase_
              hex_chars,_got_{flags!r}")
84     if version == "00" and len(parts) != 4:
85         raise ValueError("version_'00'_forbids_extra_
              fields")
86     sampled = bool(int(flags, 16) & 0b0000_0001) #
              only bit 0 is defined
87     return TraceParent(version=version, trace_id=
              trace_id,
88                        span_id=span_id, sampled=sampled
89                        )
90
91 def new_traceparent(*, sampled: bool = True) ->
      TraceParent:
92     """Mint_a_fresh_root_context_(16-byte_trace_id,_8-
              byte_span_id)."""
93     return TraceParent(
94         version="00",
95         trace_id=secrets.token_hex(16),
96         span_id=secrets.token_hex(8),
97         sampled=sampled,
98     )
99
100
101 def child_of(parent: TraceParent, *, sampled: bool |
      None = None) -> TraceParent:
102     """Derive_the_outgoing_context_for_a_child_hop:_
              same_trace,_new_span_id."""
103     return TraceParent(
104         version=parent.version,
105         trace_id=parent.trace_id,
106         span_id=secrets.token_hex(8),
107         sampled=parent.sampled if sampled is None else
              sampled,
108     )
109
110

```

```

111 def parse_tracestate(header: str) -> dict[str, str]:
112     """Parse a 'tracestate' header into an ordered dict.
113
114     Raises 'ValueError' on more than 32 members or malformed entries.
115     """
116     if not header.strip():
117         return {}
118     members = [m.strip() for m in header.split(",")]
119     if len(members) > MAX_TRACESTATE_MEMBERS:
120         raise ValueError(f"tracestate allows at most {MAX_TRACESTATE_MEMBERS} members")
121     out: dict[str, str] = {}
122     for member in members:
123         if "=" not in member:
124             raise ValueError(f"tracestate member lacks '=': {member!r}")
125         key, _, value = member.partition("=")
126         if not key or key in out:
127             raise ValueError(f"tracestate duplicate/empty key: {key!r}")
128         out[key] = value
129     return out
130
131
132 def parse_baggage(header: str) -> dict[str, str]:
133     """Parse a W3C 'baggage' header; unknown per-entry properties dropped. """
134     out: dict[str, str] = {}
135     for member in header.split(","):
136         member = member.strip()
137         if not member:
138             continue
139         pair = member.split(";")[0].strip() # drop ; property annotations
140         if "=" not in pair:
141             raise ValueError(f"baggage member lacks '=': {member!r}")
142         key, _, value = pair.partition("=")
143         out[key.strip()] = value.strip()
144     return out
145
146
147 def _demo() -> None:
148     """Round-trip one trace through three synthetic

```



```

149     hops_(deterministic)."""
150     print("==_W3C_Trace_Context_round-trip_demo_(
151         Northwind_Robotics)_==")
152     root = parse_traceparent(
153         "00-4bf92f3577b34da6a3ce929d0e0e4736-00
154         f067aa0ba902b7-01"
155     )
156     print(f"incoming_{root.header()}_{root.sampled}")
157     hop1 = child_of(root)
158     print(f"gateway->orders_{hop1.header()}")
159     hop2 = child_of(hop1)
160     print(f"orders->inventory_{hop2.header()}")
161     echoed = parse_traceparent(hop2.header())
162     assert echoed.trace_id == root.trace_id, "trace_id
163         must_survive_every_hop"
164     assert echoed.span_id == hop2.span_id, "round-trip
165         must_preserve_span_id"
166     print(f"trace_id_stable_across_hops:", echoed.
167         trace_id == root.trace_id)
168     print(parse_tracestate("nr=region:eu-1,congo=
169         t61rcWkgMzE"))
170     print(parse_baggage("tenant=northwind-robotics,plan
171         =fleet-pro;property=x"))
172
173 if __name__ == "__main__":
174     _demo()

```

19.3.2 Listing 18.2 — The Header Mechanics, Proven

Round-trip, rejection, forward-compatibility, and propagation tests — the wire format as an executable specification.

Listing 18.2 — test_trace_context.py: 19 tests for the trace-context contract.

```

1  """Executable_proof_sheet_for_trace_context.py(run:
2     pytest-q)."""
3
4  from __future__ import annotations
5
6  import pytest
7
8  from trace_context import (
9     TraceParent,

```

```
9     child_of ,
10     new_traceparent ,
11     parse_baggage ,
12     parse_traceparent ,
13     parse_tracestate ,
14 )
15
16 VALID = "00-4bf92f3577b34da6a3ce929d0e0e4736-00
17         f067aa0ba902b7-01"
18
19 def test_round_trip_preserves_every_field() -> None:
20     tp = parse_traceparent(VALID)
21     assert tp.header() == VALID
22     assert tp.version == "00"
23     assert tp.trace_id == "4
24         bf92f3577b34da6a3ce929d0e0e4736"
25     assert tp.span_id == "00f067aa0ba902b7"
26     assert tp.sampled is True
27
28 def test_not_sampled_flag() -> None:
29     tp = parse_traceparent(VALID[:-2] + "00")
30     assert tp.sampled is False
31     assert tp.header().endswith("-00")
32
33
34 def test_flags_bitmask_ignores_undefined_bits() -> None:
35     # only bit 0 (0x01) is the sampled flag; other bits
36     # must not leak in
37     tp = parse_traceparent(VALID[:-2] + "03")
38     assert tp.sampled is True
39     assert tp.flags == "01" # canonical rendering
40     # keeps only defined bits
41
42 def test_child_keeps_trace_id_and_gets_fresh_span_id()
43 -> None:
44     root = parse_traceparent(VALID)
45     child = child_of(root)
46     assert child.trace_id == root.trace_id
47     assert child.span_id != root.span_id
48     assert len(child.span_id) == 16
49     assert child.sampled == root.sampled
```

```

49
50 def test_new_contexts_are_well_formed() -> None:
51     tp = new_traceparent()
52     again = parse_traceparent(tp.header())
53     assert again == tp
54
55
56 @pytest.mark.parametrize(
57     "bad",
58     [
59         "", # empty
60         VALID.upper(), #
61             uppercase hex forbidden
62         "ff-4bf92f3577b34da6a3ce929d0e0e4736-00
63             f067aa0ba902b7-01", # ff version
64         "00-00000000000000000000000000000000-00
65             f067aa0ba902b7-01", # zero trace id
66         "00-4bf92f3577b34da6a3ce929d0e0e4736
67             -0000000000000000-01", # zero span id
68         "00-4bf92f3577b34da6a3ce929d0e0e473-00
69             f067aa0ba902b7-01", # short trace id
70         "00-4bf92f3577b34da6a3ce929d0e0e4736-00
71             f067aa0ba902b7", # missing flags
72         "00-4bf92f3577b34da6a3ce929d0e0e4736-00
73             f067aa0ba902b7-0g", # bad hex flag
74         "00-4bf92f3577b34da6a3ce929d0e0e4736-00
75             f067aa0ba902b7-01-extra", # v00 extra field
76         "␣00-4bf92f3577b34da6a3ce929d0e0e4736-00
77             f067aa0ba902b7-01", # whitespace
78     ],
79 )
80
81 def test_rejects_malformed_headers(bad: str) -> None:
82     with pytest.raises(ValueError):
83         parse_traceparent(bad)
84
85
86 def test_future_version_may_append_fields() -> None:
87     # forward compatibility: version 01 with a trailing
88     # field parses fine
89     tp = parse_traceparent("01-" + VALID[3:] + "-congo"
90 )
91     assert tp.version == "01"
92
93
94 def test_tracestate_round_trip_and_limits() -> None:
95     state = parse_tracestate("nr=region:eu-1,congo="

```

```

84         t61rcWkgMzE")
85     assert state == {"nr": "region:eu-1", "congo": "
86         t61rcWkgMzE"}
87     with pytest.raises(ValueError):
88         parse_tracestate(",".join(f"k{i}=v" for i in
89             range(33)))
90     with pytest.raises(ValueError):
91         parse_tracestate("nr=one,nr=two") # duplicate
92         key
93
94 def test_baggage_drops_properties() -> None:
95     bag = parse_baggage("tenant=northwind-robotics;ttl
96         =30,plan=fleet-pro")
97     assert bag == {"tenant": "northwind-robotics", "
98         plan": "fleet-pro"}
99     with pytest.raises(ValueError):
100         parse_baggage("no-equals-sign")
101
102 def test_immutability() -> None:
103     tp = parse_traceparent(VALID)
104     with pytest.raises(AttributeError):
105         tp.trace_id = "0" * 32 # type: ignore[misc]

```

19.3.3 Listing 18.3 — An Instrumented FastAPI Service, Offline

`instrumented_app.py` wires all three pillars into one service. A seeded ID generator makes span ids reproducible; `JsonlSpanExporter` implements the real `SpanExporter` interface but writes spans to a local file (the offline OTLP replacement); an `InMemoryMetricReader` holds RED metrics (counter plus explicit-bucket latency histogram); and a logging filter stamps the active span's ids onto every record. The three-hop chain — gateway → orders → inventory — propagates context by injecting and extracting real `traceparent` strings between hops, and the demo *asserts* the exported span tree: one trace id, correct parent chain, correct span kinds. The FastAPI app itself is fully wired (instrumentor plus correlation middleware); the demo drives the service layer directly so the whole thing runs with no server and no network.

Listing 18.3 — `instrumented_app.py`: traces, metrics, and correlated logs with propagation proof.

```

1  """Instrumented FastAPI service: traces, metrics, and
    correlated logs.

```

```

2
3 Northwind_Robotics_fleet-telemetry_platform, offline_
  edition. The_OTLP
4 exporter you would point at a real collector is
  replaced by
5 ‘ ‘JsonlSpanExporter‘ ‘ (spans to a local JSON-lines file
  ) and an
6 ‘ ‘InMemoryMetricReader‘ ‘ (metrics read back from the
  SDK), so the whole
7 demo runs with no collector, no network, and no
  credentials.
8
9 Telemetry_wiring:
10 **one ‘ ‘TracerProvider‘ ‘ with a seeded ID generator (
  deterministic tests)
11 **one ‘ ‘MeterProvider‘ ‘ with explicit-bucket
  histogram views (RED metrics)
12 **a ‘ ‘logging.Formatter‘ ‘ that stamps every record
  with trace_id/span_id
13 ****of the *active* span, so a log line and a trace
  never drift apart
14
15 Propagation_demo: a synthetic incoming ‘ ‘traceparent‘ ‘
  header is extracted
16 at the gateway, and each internal hop injects the
  current context into a
17 fresh carrier dict --- the exact bytes that would ride
  the wire in
18 production. The exported span tree proves all three
  hops share one
19 trace_id and nest parent->child correctly.
20
21 Run: python instrumented_app.py
22 Serve(optional, needs uvicorn): uvicorn
  instrumented_app:app
23 """
24
25 from __future__ import annotations
26
27 import json
28 import logging
29 import random
30 import time
31 from dataclasses import dataclass, field
32 from pathlib import Path
33 from typing import Mapping, Sequence

```

```
34
35 from fastapi import FastAPI, Request
36 from fastapi.responses import JSONResponse
37 from opentelemetry import trace
38 from opentelemetry.instrumentation.fastapi import
   FastAPIInstrumentor
39 from opentelemetry.sdk.metrics import MeterProvider
40 from opentelemetry.sdk.metrics.export import
   InMemoryMetricReader
41 from opentelemetry.sdk.metrics.view import
   ExplicitBucketHistogramAggregation, View
42 from opentelemetry.sdk.resources import Resource
43 from opentelemetry.sdk.trace import TracerProvider
44 from opentelemetry.sdk.trace.export import (
45     SimpleSpanProcessor,
46     SpanExporter,
47     SpanExportResult,
48 )
49 from opentelemetry.sdk.trace.id_generator import
   IdGenerator
50 from opentelemetry.sdk.trace.sampling import ALWAYS_ON
51 from opentelemetry.trace.propagation.tracecontext
   import (
52     TraceContextTextMapPropagator,
53 )
54 from opentelemetry.trace import SpanKind
55
56 LATENCY_BUCKETS_MS = [5.0, 10.0, 25.0, 50.0, 100.0,
   250.0, 500.0, 1000.0, 2500.0]
57 PROPAGATOR = TraceContextTextMapPropagator()
58
59 #
60 -----
61
62 # Telemetry plumbing
63 #
64 -----
65
66 class SeededIdGenerator(IdGenerator):
67     """Deterministic trace/span IDs so tests assert
   exact span trees."""
68
69     def __init__(self, seed: int) -> None:
70         self._rng = random.Random(seed)
```

```

69     def generate_trace_id(self) -> int:
70         return max(1, self._rng.getrandbits(128)) #
            all-zero id is invalid
71
72     def generate_span_id(self) -> int:
73         return max(1, self._rng.getrandbits(64))
74
75
76 class JsonlSpanExporter(SpanExporter):
77     """Offline OTLP replacement: append finished spans
78         to a JSONL file."""
79
80     def __init__(self, path: Path) -> None:
81         self._path = path
82         self._path.write_text("", encoding="utf-8")
83
84     def export(self, spans: Sequence[trace.Span]) ->
85         SpanExportResult:
86         with self._path.open("a", encoding="utf-8") as
87             fh:
88                 for span in spans:
89                     fh.write(json.dumps(_span_to_dict(span)
90                                     ) + "\n")
91                 return SpanExportResult.SUCCESS
92
93     def shutdown(self) -> None: # nothing buffered;
94         file handles are scoped
95         return None
96
97 def _span_to_dict(span: trace.Span) -> dict[str, object
98 ]:
99     ctx = span.get_span_context()
100     parent = span.parent
101     return {
102         "name": span.name,
103         "kind": span.kind.name if span.kind else "
            INTERNAL",
104         "trace_id": f"{ctx.trace_id:032x}",
105         "span_id": f"{ctx.span_id:016x}",
106         "parent_span_id": f"{parent.span_id:016x}" if
            parent else None,
107         "duration_ms": round((span.end_time - span.
            start_time) / 1e6, 3),
108         "attributes": {k: v for k, v in (span.
            attributes or {}).items()},

```

```

104         "status": span.status.status_code.name,
105     }
106
107
108 class TraceContextFilter(logging.Filter):
109     """Stamps trace_id/span_id of the active span onto
110     every record."""
111
112     def filter(self, record: logging.LogRecord) -> bool:
113         :
114         ctx = trace.get_current_span().get_span_context()
115         record.trace_id = f"{ctx.trace_id:032x}" if ctx
116             .trace_id else "-"
117         record.span_id = f"{ctx.span_id:016x}" if ctx.
118             span_id else "-"
119         return True
120
121 class JsonFormatter(logging.Formatter):
122     """One JSON object per line; never log headers,
123     tokens, or payloads."""
124
125     def format(self, record: logging.LogRecord) -> str:
126         return json.dumps({
127             "ts": self.formatTime(record, "%Y-%m-%dT%H
128                 :%M:%S"),
129             "level": record.levelname,
130             "logger": record.name,
131             "msg": record.getMessage(),
132             "trace_id": getattr(record, "trace_id", "-"),
133             "span_id": getattr(record, "span_id", "-"),
134         })
135
136 @dataclass
137 class Telemetry:
138     """Everything an instrumented service needs, wired
139     offline."""
140
141     provider: TracerProvider
142     tracer: trace.Tracer
143     exporter_path: Path
144     meter_reader: InMemoryMetricReader
145     request_counter: object

```



```

141     request_duration: object
142     logger: logging.Logger
143     log_lines: list[str] = field(default_factory=list)
144
145
146 def build_telemetry(seed: int = 18, spans_path: Path =
Path("spans.jsonl")) -> Telemetry:
147     resource = Resource.create({"service.name": "nr-
fleet-telemetry"})
148     provider = TracerProvider(
149         sampler=ALWAYS_ON,
150         id_generator=SeededIdGenerator(seed),
151         resource=resource,
152     )
153     provider.add_span_processor(SimpleSpanProcessor(
JsonlSpanExporter(spans_path)))
154
155     reader = InMemoryMetricReader()
156     view = View(
157         instrument_name="http.server.request.duration",
158         aggregation=ExplicitBucketHistogramAggregation(
boundaries=LATENCY_BUCKETS_MS),
159     )
160     meter_provider = MeterProvider(
161         metric_readers=[reader], views=[view], resource
=resource
162     )
163     meter = meter_provider.get_meter("nr.fleet")
164
165     logger = logging.getLogger("nr.fleet")
166     logger.setLevel(logging.INFO)
167     logger.handlers.clear()
168     logger.propagate = False
169
170     telemetry = Telemetry(
171         provider=provider,
172         tracer=provider.get_tracer("nr.fleet"),
173         exporter_path=spans_path,
174         meter_reader=reader,
175         request_counter=meter.create_counter(
176             "http.server.request.count",
177             description="Total HTTP requests (RED: rate
, errors)",
178         ),
179         request_duration=meter.create_histogram(
180             "http.server.request.duration",

```

```

181         unit="ms",
182         description="Request latency histogram (RED
           : duration)",
183     ),
184     logger=logger,
185 )
186 return telemetry
187
188 #
189 -----
190 # Service layer: a three-hop internal call chain
191 #
192 -----
193
194 def inventory_reserve(telemetry: Telemetry, carrier:
dict[str, str]) -> dict[str, object]:
195     """Hop 3 (deepest): check stock. Extracts context
           from its carrier."""
196     ctx = PROPAGATOR.extract(carrier=carrier)
197     with telemetry.tracer.start_as_current_span(
           "inventory.reserve", context=ctx, kind=SpanKind
           .SERVER
198     ) as span:
199         span.set_attribute("peer.service", "inventory")
200         span.set_attribute("warehouse", "eu-1")
201         time.sleep(0.004) # deterministic-ish work
           stand-in; tests don't time it
202         telemetry.logger.info("stock reserved sku=NR-
           GRIP-4 qty=2")
203         return {"sku": "NR-GRIP-4", "reserved": 2}
204
205 def orders_create(telemetry: Telemetry, carrier: dict[
str, str]) -> dict[str, object]:
206     """Hop 2: create the resupply order; injects
           context for hop 3."""
207     ctx = PROPAGATOR.extract(carrier=carrier)
208     with telemetry.tracer.start_as_current_span(
           "orders.create", context=ctx, kind=SpanKind.
           SERVER
209     ) as span:
210         span.set_attribute("peer.service", "orders")
211         span.set_attribute("order.kind", "resupply")
212         outbound: dict[str, str] = {}
213

```

```

214     PROPAGATOR.inject(outbound) # traceparent
        bytes for the next hop
215     reservation = inventory_reserve(telemetry,
        outbound)
216     telemetry.logger.info("order_created_id=NR
        -2026-0917")
217     return {"order_id": "NR-2026-0917", "
        reservation": reservation}
218
219
220 def gateway_handle(telemetry: Telemetry, headers:
    Mapping[str, str]) -> dict[str, object]:
221     """Hop 1 (edge): extract the client-supplied
        traceparent, record RED."""
222     started = time.perf_counter()
223     status = 200
224     ctx = PROPAGATOR.extract(carrier=dict(headers))
225     with telemetry.tracer.start_as_current_span(
        "gateway_POST/fleet/telemetry", context=ctx,
226         kind=SpanKind.SERVER
    ) as span:
227         span.set_attribute("http.request.method", "POST
        ")
228         span.set_attribute("url.path", "/fleet/
        telemetry")
229         span.set_attribute("http.response.status_code",
        status)
230         outbound: dict[str, str] = {}
231         PROPAGATOR.inject(outbound)
232         try:
233             result = orders_create(telemetry, outbound)
234         except Exception:
235             status = 500
236             span.set_attribute("http.response.
        status_code", status)
237             raise
238         finally:
239             elapsed_ms = (time.perf_counter() - started
        ) * 1000.0
240             attrs = {"route": "/fleet/telemetry", "
        status": status}
241             telemetry.request_counter.add(1, attrs)
242             telemetry.request_duration.record(
        elapsed_ms, attrs)
243             telemetry.logger.info("request_complete_status
        =200")

```

```
245         return result
246
247
248 #
249 -----
250 # FastAPI wiring (thin adapters over the instrumented
251     service layer)
252 #
253 -----
254
255 telemetry = build_telemetry()
256 app = FastAPI(title="Northwind_Robotics_Fleet_Telemetry")
257
258 FastAPIInstrumentor.instrument_app(
259     app, tracer_provider=telemetry.provider,
260     excluded_urls="/health"
261 )
262
263 @app.middleware("http")
264 async def correlate_logs(request: Request, call_next):
265     # noqa: ANN001
266     # The instrumentor's server span is active here, so
267     # logs pick up its ids.
268     response: JSONResponse = await call_next(request)
269     telemetry.logger.info(
270         "http_request_method=%s_path=%s_status=%s",
271         request.method, request.url.path, response.
272         status_code,
273     )
274     return response
275
276 @app.post("/fleet/telemetry")
277 async def ingest(request: Request) -> dict[str, object]:
278     return gateway_handle(telemetry, dict(request.
279         headers))
280
281 @app.get("/health")
282 async def health() -> dict[str, str]:
283     return {"status": "ok"}
```

```

279 #
    -----
280 # Offline demo: drive the service layer with a
    synthetic traceparent
281 #
    -----

282 def _demo() -> None:
283     telemetry = build_telemetry(seed=18, spans_path=
        Path("spans.jsonl"))

284
285     handler = logging.StreamHandler()
286     handler.setFormatter(JsonFormatter())
287     handler.addFilter(TraceContextFilter())
288     telemetry.logger.addHandler(handler)
289
290     incoming = {
291         "traceparent": "00-4
            bf92f3577b34da6a3ce929d0e0e4736-00
            f067aa0ba902b7-01",
292         "tracestate": "nr=region:eu-1",
293     }
294     print(f"==_three-hop_trace,_one_synthetic_incoming_
        traceparent_==")
295     result = gateway_handle(telemetry, incoming)
296     telemetry.provider.force_flush()
297
298     lines = telemetry.exporter_path.read_text(encoding=
        "utf-8").splitlines()
299     spans = [json.loads(line) for line in lines if line
        .strip()]
300     print(f"exported_{len(spans)}_spans:")
301     for s in sorted(spans, key=lambda s: s["
        parent_span_id"] is not None):
302         print(
303             f"_{s['name']}<32}_{kind={s['kind']}<8}"
304             f"trace={s['trace_id'][:12]}..._span={s['
                span_id']}"
305             f"parent={s['parent_span_id']}"
306         )
307     trace_ids = {s["trace_id"] for s in spans}
308     by_name = {s["name"]: s for s in spans}
309     assert len(trace_ids) == 1, "all_hops_must_share_
        one_trace_id"
310     assert trace_ids == {"4

```

```

311         bf92f3577b34da6a3ce929d0e0e4736"}, (
312             "incoming_trace_id_must_survive_every_hop"
313         )
314         assert by_name["orders.create"]["parent_span_id"]
315             == by_name[
316                 "gateway_POST_fleet/telemetry"]["span_id"]
317         assert by_name["inventory.reserve"]["parent_span_id"]
318             == by_name["orders.create"]["span_id"]
319         print("propagation_verified: one_trace_id, correct_
320             parent_chain")
321
322         data = telemetry.meter_reader.get_metrics_data()
323         for rm in data.resource_metrics:
324             for sm in rm.scope_metrics:
325                 for m in sm.metrics:
326                     pts = list(m.data.data_points)
327                     print(f"metric_{m.name}: points={len(
328                         pts)}")
329                 print(f"result: {json.dumps(result)}")
330
331 if __name__ == "__main__":
332     _demo()

```

Running it prints the proof of propagation (ids are seeded and reproduce exactly):

```

1 exported 3 spans:
2   inventory.reserve                kind=SERVER    trace
3     =4bf92f3577b3... parent=72e63ac7a9538322
4   orders.create                   kind=SERVER    trace
5     =4bf92f3577b3... parent=1f70d5dc2e675fc7
6   gateway POST /fleet/telemetry    kind=SERVER    trace
7     =4bf92f3577b3... parent=00f067aa0ba902b7
8 propagation verified: one trace_id, correct parent
9 chain

```

Note the gateway's parent: 00f067aa0ba902b7 — the span id from the synthetic incoming `traceparent` of Listing 18.1's example. The client's trace context survived the whole chain.

19.3.4 Listing 18.4 — Histograms: Percentiles That Survive the Tail

`latency_histograms.py` re-implements the `HdrHistogram` layout in `stdlib`: power-of-two buckets with linear sub-buckets give bounded relative error (two significant figures $\Rightarrow$ bucket width under 1% of the value) at $O(1)$ memory per recording. The embedded `pytest` suite proves the two properties that matter: percentiles match exact rank-based quantiles where resolution permits, and larger values stay within the resolution bound. The demo prints the long-tail comparison behind Figure 19.3: mean 761 ms, p50 25 ms, p99 23,680 ms — the average lying about a $31\times$ gap.

Listing 18.4 — `latency_histograms.py`: log-bucket histogram with embedded proof suite.

```

1  """Log-bucket latency histograms: percentiles that
   survive the long tail.
2
3  A stdlib-only re-implementation of the HdrHistogram
   layout (power-of-two
4  buckets with linear sub-buckets), because "compute p99"
   is a sentence every
5  API engineer says and few can defend. Properties:
6
7  *  $O(1)$  memory per recording, fixed-size counts array
8  * bounded relative error: 2 significant figures  $\Rightarrow$ 
   every recorded value
9  * lands in a bucket whose width is at most  $\sim 0.8\%$  of
   the value
10 * percentiles are read from cumulative bucket counts,
   never from a
11 * sample list, so cost is  $O(\text{buckets})$ , not  $O(\text{samples})$ 
12
13 The demo generates a seeded long-tail distribution (
   typical API latency:
14 fast median, brutal tail) and prints the comparison
   that motivates this
15 whole chapter: the naive average sits comfortably below
   p99 --- the average
16 is lying to you about your users' pain.
17
18 Run: python latency_histograms.py (demo)
19 pytest latency_histograms.py -q (embedded
   test suite)
20 """
21
22 from __future__ import annotations

```

```
23
24 import math
25 import random
26 from dataclasses import dataclass, field
27
28
29 @dataclass
30 class LogHistogram:
31     """HdrHistogram-style integer histogram (values in
32         milliseconds)."""
33
34     lowest: int = 1
35     highest: int = 3_600_000 # one hour in ms
36     significant_figures: int = 2
37     total: int = 0
38     sum_ms: float = 0.0
39     counts: list[int] = field(init=False)
40
41     def __post_init__(self) -> None:
42         if not 1 <= self.significant_figures <= 5:
43             raise ValueError("significant_figures must
44                 be in 1..5")
45         if self.lowest < 1 or self.highest < 2 * self.
46             lowest:
47             raise ValueError("need 1 <= lowest <=
48                 highest/2")
49         # sub_bucket_count: smallest power of two >= 2
50             * 10^sig
51         self._sub_mag = math.ceil(math.log2(2 * 10 **
52             self.significant_figures))
53         self._sub_count = 1 << self._sub_mag
54         self._half = self._sub_count // 2
55         self._unit_mag = (self.lowest).bit_length() - 1
56         # bucket_count: enough doublings to reach '
57             highest'
58         self._bucket_count = 1
59         trackable = self._sub_count << self._unit_mag
60         while trackable < self.highest:
61             trackable <<= 1
62             self._bucket_count += 1
63         self.counts = [0] * ((self._bucket_count + 1) *
64             self._half)
65
66     # -- recording
67     -----
```



```

59     def _counts_index(self, value: int) -> int:
60         if value < 0 or value > self.highest:
61             raise ValueError(f"value_{value}_outside_
62                               [0,_{self.highest}]")
63         v = max(value >> self._unit_mag, 1)
64         bucket = max(0, v.bit_length() - self._sub_mag)
65         sub_bucket = v >> bucket
66         return (bucket + 1) * self._half + (sub_bucket
67         - self._half)
68
69     def record(self, value: int) -> None:
70         self.counts[self._counts_index(value)] += 1
71         self.total += 1
72         self.sum_ms += value
73
74     # -- reading
75     -----
76
77     def _value_at(self, counts_index: int) -> int:
78         bucket = (counts_index >> (self._sub_mag - 1))
79         - 1
80         sub_bucket = (counts_index & (self._half - 1))
81         + self._half
82         if bucket < 0:
83             bucket = 0
84             sub_bucket -= self._half
85         return sub_bucket << (bucket + self._unit_mag)
86
87     def percentile(self, q: float) -> int:
88         """Smallest recorded-bucket value at which q%
89           of samples fall."""
90         if not 0 < q <= 100:
91             raise ValueError("q must be in (0,100]")
92         if self.total == 0:
93             raise ValueError("no samples recorded")
94         rank = max(1, math.ceil(q / 100.0 * self.total)
95         )
96         cumulative = 0
97         for idx, count in enumerate(self.counts):
98             cumulative += count
99             if cumulative >= rank:
100                 return self._value_at(idx)
101         raise AssertionError("unreachable: rank <=
102           total")
103
104     def mean(self) -> float:

```

```
96         if self.total == 0:
97             raise ValueError("no samples recorded")
98         return self.sum_ms / self.total
99
100
101 def exact_quantile(data: list[int], q: float) -> int:
102     """Ground truth on small data: the rank-based
103         quantile of a sorted list."""
104     if not data:
105         raise ValueError("empty data")
106     ordered = sorted(data)
107     rank = max(1, math.ceil(q / 100.0 * len(ordered)))
108     return ordered[rank - 1]
109
110 def synthetic_latencies(seed: int = 18, n: int = 10_000
111 ) -> list[int]:
112     """Seeded long-tail: 95% of requests are healthy,
113         5% hit the tail."""
114     rng = random.Random(seed)
115     out: list[int] = []
116     for _ in range(n):
117         if rng.random() < 0.95: # healthy path: tight
118             lognormal around ~25 ms
119             out.append(max(1, int(rng.lognormvariate(
120                 math.log(25), 0.35))))
121         else: # tail path: GC pauses
122             , retries, cold caches
123             out.append(rng.randint(800, 30_000))
124     return out
125
126 def _demo() -> None:
127     data = synthetic_latencies()
128     hist = LogHistogram()
129     for ms in data:
130         hist.record(ms)
131     print(f"samples={hist.total} (seeded, long-tail)")
132     print(f"naive average: {hist.mean():9.1f} ms
133         <-- the lie")
134     for q in (50, 95, 99, 99.9):
135         print(f"p{q}<5} {hist.percentile(q)
136             :9} ms")
137     ratio = hist.percentile(99) / hist.mean()
138     print(f"p99 / average: {ratio:9.1f}x <-- what
139         the average was hiding")
```

```

133
134
135 #
    -----

136 # Embedded test suite: pytest latency_histograms.py -q
137 #
    -----

138 def
    test_percentiles_match_exact_quantiles_on_small_data
    () -> None:
139     # values below sub_bucket_count (256) each get
        their own bucket,
140     # so histogram percentiles must equal exact rank-
        based quantiles
141     data = list(range(1, 256))
142     hist = LogHistogram()
143     for v in data:
144         hist.record(v)
145     for q in (1, 25, 50, 75, 90, 95, 99, 99.9, 100):
146         assert hist.percentile(q) == exact_quantile(
            data, q)
147
148
149 def test_larger_values_stay_within_resolution_bound()
    -> None:
150     data = list(range(1, 5001))
151     hist = LogHistogram()
152     for v in data:
153         hist.record(v)
154     for q in (25, 50, 75, 90, 99):
155         approx, exact = hist.percentile(q),
            exact_quantile(data, q)
156         assert abs(approx - exact) / exact < 0.01 # 2
            significant figures
157
158
159 def test_mean_is_exact() -> None:
160     hist = LogHistogram()
161     for v in [10, 20, 30]:
162         hist.record(v)
163     assert hist.mean() == 20.0
164
165
166 def test_rank_rounding_uses_ceiling() -> None:

```

```
167     data = [5] * 9 + [900]    # p90 rank = ceil(9.0) = 9
    -> 5; p95 -> 900
168     hist = LogHistogram()
169     for v in data:
170         hist.record(v)
171     assert hist.percentile(90) == 5
172     assert hist.percentile(95) == 900
173
174
175 def test_relative_error_is_bounded_by_resolution() ->
    None:
176     hist = LogHistogram(significant_figures=2)
177     hist.record(12_345)    # bucket width at this
    magnitude is 128 ms
178     approx = hist.percentile(50)
179     assert abs(approx - 12_345) / 12_345 < 0.01
180
181
182 def test_out_of_range_and_empty_rejected() -> None:
183     hist = LogHistogram()
184     import pytest
185
186     with pytest.raises(ValueError):
187         hist.record(hist.highest + 1)
188     with pytest.raises(ValueError):
189         hist.percentile(99)
190     with pytest.raises(ValueError):
191         hist.percentile(0)
192
193
194 def test_long_tail_demo_is_deterministic() -> None:
195     assert synthetic_latencies() == synthetic_latencies
    ()
196     hist = LogHistogram()
197     for ms in synthetic_latencies():
198         hist.record(ms)
199     assert hist.percentile(99) > 10 * hist.mean()    #
    the tail really is long
200
201
202 if __name__ == "__main__":
203     _demo()
```

19.3.5 Listing 18.5 — The Burn-Rate Simulator

`burn_rate_sim.py` implements Equations (19.3) and (19.4) against a seeded three-day timeline with a scripted incident (50 minutes at an $80\times$ burn — a wedged dependency). Window queries run over prefix sums, so evaluation is $O(n)$. The demo prints the policy table and the alert timeline; the tests prove the budget arithmetic (43.2 minutes; 2%/5%/10% consumption), that healthy traffic never pages, that the incident pages fast then slow but never tickets, and that a sustained $3\times$ burn tickets without paging.

Listing 18.5 — `burn_rate_sim.py`: multi-window multi-burn-rate alerting, simulated offline.

```

1  """Multi-window, multi-burn-rate SLO alerting simulator
   |     (offline, seeded).
2
3  Feeds a synthetic minute-by-minute traffic timeline for
   |     the Northwind
4  Robotics fleet API into the alerting logic from the
   |     Google SRE Workbook's
5  "Alerting on SLOs" chapter: an alert fires only when
   |     BOTH a long window
6  and a short window show a burn rate above the policy
   |     threshold.
7
8  burn_rate(t, w) == error_rate(t, w) / budget_rate
9  budget_rate == 1 - SLO (0.001 for a 99.9% SLO)
10 budget_consumed == burn_rate * w / period (% of
   |     the 30-day budget)
11
12 Policy table (budget consumed if the burn persists for
   |     the long window):
13 fast_page: burn 14.4 long 1h short 5m -> 2%
   |     of budget
14 slow_page: burn 6.0 long 6h short 30m -> 5%
   |     of budget
15 ticket: burn 1.0 long 3d short 6h -> 10%
   |     of budget
16
17 Run: python burn_rate_sim.py (scripted incident
   |     demo)
18 pytest burn_rate_sim.py -q (embedded test
   |     suite)
19 """
20
21 from __future__ import annotations

```

```
22
23 import random
24 from dataclasses import dataclass
25
26 SLO = 0.999
27 BUDGET_RATE = 1.0 - SLO # 0.001
28 PERIOD_MINUTES = 30 * 24 * 60 # 30-day rolling
    window: 43,200 min
29 BUDGET_MINUTES_EQUIVALENT = BUDGET_RATE *
    PERIOD_MINUTES # 43.2 min
30
31
32 @dataclass(frozen=True)
33 class MinuteSample:
34     minute: int
35     requests: int
36     errors: int
37
38
39 @dataclass(frozen=True)
40 class BurnRatePolicy:
41     name: str
42     burn_rate: float
43     long_window_min: int
44     short_window_min: int
45
46     @property
47     def budget_consumed_pct(self) -> float:
48         """% of the period budget burned if this rate
            lasts the long window."""
49         return 100.0 * self.burn_rate * self.
            long_window_min / PERIOD_MINUTES
50
51
52 POLICIES = [
53     BurnRatePolicy("fast-page", 14.4, 60, 5),
54     BurnRatePolicy("slow-page", 6.0, 360, 30),
55     BurnRatePolicy("ticket", 1.0, 3 * 24 * 60, 360),
56 ]
57
58
59 @dataclass(frozen=True)
60 class Alert:
61     policy: str
62     fired_at: int
63     long_burn: float
```

```

64     short_burn: float
65
66
67 def window_rate(samples: list[MinuteSample], t: int,
68 window: int) -> float:
69     """Error_rate_over_samples_in_(t-window,t]; 0.0
70     when_no_traffic."""
71     total = errors = 0
72     for s in samples:
73         if t - window < s.minute <= t:
74             total += s.requests
75             errors += s.errors
76     return errors / total if total else 0.0
77
78
79 def burn_rate(samples: list[MinuteSample], t: int,
80 window: int) -> float:
81     return window_rate(samples, t, window) /
82     BUDGET_RATE
83
84
85 def _prefix_sums(samples: list[MinuteSample]) -> tuple[
86 list[int], list[int]]:
87     """Cumulative_requests/errors_per_minute_for_0(1)
88     window_queries."""
89     horizon = samples[-1].minute
90     cum_req = [0] * (horizon + 1)
91     cum_err = [0] * (horizon + 1)
92     for s in samples:
93         cum_req[s.minute] += s.requests
94         cum_err[s.minute] += s.errors
95     for t in range(1, horizon + 1):
96         cum_req[t] += cum_req[t - 1]
97         cum_err[t] += cum_err[t - 1]
98     return cum_req, cum_err
99
100
101 def _burn(cum_req: list[int], cum_err: list[int], t:
102 int, window: int) -> float:
103     lo = max(0, t - window) # window (t - window, t]
104     total = cum_req[t] - cum_req[lo]
105     errors = cum_err[t] - cum_err[lo]
106     return (errors / total / BUDGET_RATE) if total else
107     0.0

```

```

102 def evaluate(
103     samples: list[MinuteSample],
104     policies: list[BurnRatePolicy] | None = None,
105 ) -> list[Alert]:
106     """First firing of each policy; reset requires the
107         long window to clear."""
108     fired: list[Alert] = []
109     active: set[str] = set()
110     if not samples:
111         return fired
112     policies = policies or POLICIES
113     cum_req, cum_err = _prefix_sums(samples)
114     for t in range(1, samples[-1].minute + 1):
115         for policy in policies:
116             if t < policy.long_window_min:
117                 continue # window not yet fully
118                     populated; don't alert on cold data
119             long_burn = _burn(cum_req, cum_err, t,
120                             policy.long_window_min)
121             short_burn = _burn(cum_req, cum_err, t,
122                               policy.short_window_min)
123             breached = (
124                 long_burn >= policy.burn_rate
125                 and short_burn >= policy.burn_rate
126             )
127             if breached and policy.name not in active:
128                 active.add(policy.name)
129                 fired.append(Alert(policy.name, t,
130                                   long_burn, short_burn))
131             elif long_burn < policy.burn_rate and
132                  policy.name in active:
133                 active.remove(policy.name) # recovered
134                 ; may re-fire later
135     return fired
136
137 def synthetic_timeline(seed: int = 18, minutes: int = 3
138                       * 24 * 60) -> list[MinuteSample]:
139     """Three days of traffic; a scripted bad deploy
140         burns hot for 50 minutes.
141
142     Baseline: ~120 req/min with a healthy 0.05% error
143         rate (below budget).
144     Incident: minutes 2000..2049 at 8% errors --- an 80
145         x burn rate, the
146     classic "deploy wedged a dependency" shape.

```



```

137     Everything seeded, so the
138     alert timeline below reproduces on every machine.
139     """
140     rng = random.Random(seed)
141     out: list[MinuteSample] = []
142     for t in range(minutes):
143         requests = 120 + rng.randint(-10, 10)
144         error_rate = 0.08 if 2000 <= t < 2050 else
145             0.0005
146         # errors are a deterministic quota plus seeded
147         # jitter around it
148         errors = int(requests * error_rate) + (1 if rng
149             .random() < 0.02 else 0)
150         out.append(MinuteSample(t, requests, errors))
151     return out
152
153 def _demo() -> None:
154     samples = synthetic_timeline()
155     print(f"timeline: {len(samples)} minutes, SLO={SLO}
156         ",
157         f"budget={BUDGET_MINUTES_EQUIVALENT:.1f} min-
158         equivalent/30d")
159     print(f"{'policy':<10}{'burn':>5}{'long':>6}{'
160         short':>6}{'budget%':>8}")
161     for p in POLICIES:
162         print(f"{p.name:<10}{p.burn_rate:>5}{p.
163             long_window_min:>5}m")
164         print(f"{p.short_window_min:>5}m {p.
165             budget_consumed_pct:>7.1f}%")
166     print("\nalert timeline (minute=minutes since
167         window start):")
168     for alert in evaluate(samples):
169         hh, mm = divmod(alert.fired_at, 60)
170         print(f"[t={alert.fired_at:>5}]({hh:02d}h{mm
171             :02d}m) PAGE {alert.policy}"
172             f"long-burn={alert.long_burn:.1f}x
173             short-burn={alert.short_burn:.1f}x")
174
175 #
176
177 -----
178
179 # Embedded test suite: pytest burn_rate_sim.py -q
180 #
181 -----

```

```
168 def test_error_budget_minutes() -> None:
169     assert abs(BUDGET_MINUTES_EQUIVALENT - 43.2) < 1e-9
170         # the famous 43.2 min/month
171
172 def
173     test_budget_consumed_percentages_match_workbook_table
174     () -> None:
175         assert abs(POLICIES[0].budget_consumed_pct - 2.0) <
176             1e-9
177         assert abs(POLICIES[1].budget_consumed_pct - 5.0) <
178             1e-9
179         assert abs(POLICIES[2].budget_consumed_pct - 10.0)
180             < 1e-9
181
182 def test_healthy_traffic_never_pages() -> None:
183     samples = synthetic_timeline()
184     healthy = [s for s in samples if not 1950 <= s.
185                 minute < 2100]
186     assert evaluate(healthy) == []
187
188 def test_incident_fires_both_pages_but_no_ticket() ->
189     None:
190         alerts = evaluate(synthetic_timeline())
191         fast = [a for a in alerts if a.policy == "fast-page"]
192         assert fast, "an 80x burn for 50 minutes must page the fast policy"
193         assert 2005 <= fast[0].fired_at <= 2060 # fires
194             during/right after burst
195         # 50 min at 80x still averages 11x over the 6h
196             window -> slow page too
197         assert any(a.policy == "slow-page" for a in alerts)
198         # but 50 bad minutes out of 3 days stays under the
199             1x ticket threshold
200         assert not any(a.policy == "ticket" for a in alerts)
201
202 def test_sustained_low_burn_fires_ticket_only() -> None
203     :
204         # 0.3% errors = 3x burn, sustained for the whole
205             timeline
```

```

197     samples = [
198         MinuteSample(t, requests=1000, errors=3) for t
199         in range(3 * 24 * 60 + 400)
200     ]
201     alerts = evaluate(samples)
202     names = {a.policy for a in alerts}
203     assert names == {"ticket"}, f"3x burn should page ticket only, got {names}"
204
205 def
206     test_window_rate_excludes_boundary_and_handles_empty
207     () -> None:
208         samples = [MinuteSample(0, 100, 10), MinuteSample
209                     (5, 100, 0)]
210         assert window_rate(samples, 5, 5) == 0.0
211             # (0, 5] holds minute 5 only
212         assert window_rate(samples, 5, 6) == 10 / 200
213             # (-1, 5] holds both minutes
214         assert window_rate(samples, 10, 5) == 0.0
215             # (5, 10] holds neither
216         assert window_rate([], 10, 5) == 0.0
217
218 if __name__ == "__main__":
219     _demo()

```

Its alert timeline for the scripted incident:

```

1 alert timeline (minute = minutes since window start):
2   [t= 2010 (33h30m)] PAGE fast-page long-burn=14.5x
3   short-burn=76.1x
4   [t= 2027 (33h47m)] PAGE slow-page long-burn=6.2x
5   short-burn=71.7x

```

19.3.6 Listing 18.6 — Runbook and Postmortem Generator

`incident_templates.py` turns incident metadata into markdown skeletons with every mandatory section present: the runbook carries SLOs, alert entry points, a ten-minute triage checklist, mitigation commands, and escalation; the postmortem carries the blameless preamble, timeline, impact, mechanism-level root cause, action items, and an explicit “lessons for the SLO” section. Tests assert the sections exist and that invalid severities and empty service lists are rejected.

Listing 18.6 — `incident_templates.py`: structure enforced by code.

```

1  """Runbook and blameless-postmortem skeleton generator
   (stdlib only).
2
3  Incident response fails in predictable ways: nobody
   remembers the rollback
4  command at 3 a.m., and postmortems devolve into blame
   or vanish unwritten.
5  Both are structure problems, and structure is exactly
   what code is good at.
6  Given incident metadata, this module emits markdown
   skeletons with every
7  mandatory section present and every placeholder
   explicit, so the human
8  work is analysis, not formatting.
9
10 Run: python incident_templates.py (writes demo
   artifacts)
11 pytest incident_templates.py -q (embedded
   test suite)
12 """
13
14 from __future__ import annotations
15
16 from dataclasses import dataclass, field
17 from pathlib import Path
18
19 SEVERITIES = {
20     "SEV1": "total outage or data loss; all-hands, page
        immediately",
21     "SEV2": "major degradation of a core SLO; page the
        owning team",
22     "SEV3": "minor degradation or single-tenant impact;
        ticket, next business day",
23     "SEV4": "cosmetic or internal-only; backlog",
24 }
25
26
27 @dataclass(frozen=True)
28 class ServiceProfile:
29     name: str
30     slo_target: float
31     owner: str
32     endpoints: tuple[str, ...]

```

```

33     dashboards: tuple[str, ...]
34     rollback_command: str
35
36
37 @dataclass(frozen=True)
38 class IncidentMetadata:
39     incident_id: str
40     title: str
41     severity: str
42     commander: str
43     services: tuple[str, ...]
44     started_utc: str
45     detected_by: str
46     slo_impact: str
47     action_items: tuple[str, ...] = field(default=())
48
49     def __post_init__(self) -> None:
50         if self.severity not in SEVERITIES:
51             raise ValueError(f"severity must be one of {sorted(SEVERITIES)}")
52         if not self.services:
53             raise ValueError("an incident must name at least one service")
54
55
56 def render_runbook(profile: ServiceProfile) -> str:
57     endpoints = "\n".join(f"|_{ep}'_{_|_|_|" for ep in
58                             profile.endpoints)
59     boards = "\n".join(f"-_{d}" for d in profile.
60                         dashboards)
61     return f"""#_Runbook:_{profile.name}
62
63 Owner:_{profile.owner}|_SL0:_{profile.slo_target:.3%}_
64     availability_(30d_rolling)
65
66 ##_1._Service_summary
67 One_paragraph:_{what_{profile.name}_does,_{who_calls_it,_{
68     what_breaks_when_it
69 breaks.
70
71 ##_2._SL0s_and_alert_entry_points
72 |_Endpoint_|_SLI_|_Alert_|
73 |---|---|---|
74 {endpoints}
75
76 Dashboards:

```

```

73 {boards}
74
75 ## 3. Triage checklist (first 10 minutes)
76 1. Confirm the alert is real: check the SLO dashboard,
    not one log line.
77 2. Declare severity (see severity table) and open an
    incident channel.
78 3. Capture the current trace exemplars before
    mitigation flushes them.
79
80 ## 4. Mitigation
81 - Rollback: {profile.rollback_command}
82 - Feature flag: <flag name and console path>
83 - Traffic shift / drain: <load balancer procedure>
84
85 ## 5. Escalation
86 - Primary on-call -> secondary (15 min ack) ->
    engineering manager.
87 ""
88
89
90 def render_postmortem(meta: IncidentMetadata) -> str:
91     items = "\n".join(f"- [{a}]" for a in meta.
92         action_items) or "- <action>"
93     services = ", ".join(meta.services)
94     return f"""# Postmortem: {meta.incident_id} -- {
95         meta.title}
96
97 **Severity:** {meta.severity} ({SEVERITIES[meta.
98     severity]})
99 **Incident commander:** {meta.commander}
100 **Services:** {services}
101 **Started (UTC):** {meta.started_utc}
102 **Detected by:** {meta.detected_by}
103 **SLO impact:** {meta.slo_impact}
104
105 > This document is blameless. We analyze systems, not
106     people: any engineer
107 > in the same situation, with the same information,
108     might have done the
109 > same thing. Naming a person as a cause voids the
110     review.
111
112 ## Summary
113 Two sentences a VP can read.
114

```

```

109 ##_Impact
110 Who_was_affected, _for_how_long, _in_what_units(requests
    _failed, _SLO_budget
111 burned, _revenue_or_contractual_exposure).
112
113 ##_Timeline_(all_times_UTC)
114 |_Time_|_Event_|
115 |---|---|
116 |_|_detection_(alert_or_human?)_|
117 |_|_mitigation_applied_|
118 |_|_full_recovery_|
119
120 ##_Root_cause
121 The_mechanism, _not_the_culprit:_what_change, _what_
    assumption_failed, _why
122 existing_guardrails_did_not_catch_it.
123
124 ##_What_went_well/_what_went_poorly
125 Detection_latency, _mitigation_latency, _runbook_accuracy
    , _paging_quality.
126
127 ##_Action_items
128 {items}
129
130 ##_Lessons_for_the_SLO
131 Did_the_alerts_fire_at_the_right_time?_Was_the_burn-
    rate_table_tuned
132 correctly, _or_did_this_incident_reveal_a_gap_(add_a_row
    , _adjust_a_window)?
133 ""
134
135
136 def write_artifacts(out_dir: Path, profile:
    ServiceProfile, meta: IncidentMetadata) -> None:
137     out_dir.mkdir(parents=True, exist_ok=True)
138     (out_dir / f"runbook_{profile.name}.md").write_text
139     (render_runbook(profile), encoding="utf-8")
140     (out_dir / f"postmortem_{meta.incident_id}.md").
        write_text(
141         render_postmortem(meta), encoding="utf-8"
142     )
143
144 def _demo() -> None:
145     profile = ServiceProfile(
146         name="fleet-telemetry",

```

```

147     slo_target=0.999,
148     owner="fleet-platform@northwind-robotics.
        example",
149     endpoints=("POST_/fleet/telemetry", "GET_/fleet
        /status", "GET_/health"),
150     dashboards=("SLO_/fleet-telemetry_(Grafana_uid
        =nr-fleet-slo)",
151                 "RED_/upper-endpoint_(Grafana_uid=nr
        -fleet-red)"),
152     rollback_command="kubectl rollout undo deploy/
        fleet-telemetry",
153 )
154 meta = IncidentMetadata(
155     incident_id="NR-2026-0142",
156     title="Retry_storm_after_inventory_latency_
        regression",
157     severity="SEV2",
158     commander="a.reyes",
159     services=("fleet-gateway", "orders", "inventory
        "),
160     started_utc="2026-08-19T14:03Z",
161     detected_by="fast-burn_SLO_alert_(14.4x, 1h/5m)
        ",
162     slo_impact="41%_of_monthly_availability_budget_
        in_52_minutes",
163     action_items=(
164         "Cap_gateway_retries_at_2_with_jitter_(
            owner:_fleet-platform)",
165         "Add_fan-out_panel_(requests_per_client_
            request)_to_the_SLO_board",
166     ),
167 )
168 write_artifacts(Path("incident_docs"), profile,
        meta)
169 print(render_postmortem(meta).splitlines()[0])
170 print("wrote_incident_docs/runbook_fleet-telemetry.
        md_and",
171       "incident_docs/postmortem_NR-2026-0142.md")
172
173
174 #
        -----
175 # Embedded test suite: pytest incident_templates.py -q
176 #
        -----

```



```

177 def _profile() -> ServiceProfile:
178     return ServiceProfile(
179         name="orders", slo_target=0.9995, owner="
180             orders@northwind-robotics.example",
181             endpoints=("POST_/orders",), dashboards=("SLO_/
182                 _orders",),
183             rollback_command="kubectl_rollback_undo_deploy/
184                 orders",
185     )
186
187 def _meta() -> IncidentMetadata:
188     return IncidentMetadata(
189         incident_id="NR-2026-0001", title="Test_
190             incident", severity="SEV3",
191             commander="oncall", services=("orders",),
192             started_utc="2026-01-01T00:00Z",
193             detected_by="test", slo_impact="none",
194     )
195
196 def test_runbook_has_all_mandatory_sections() -> None:
197     text = render_runbook(_profile())
198     for section in ("Service_summary", "SLOs_and_alert_
199         entry_points",
200         "Triage_checklist", "Mitigation", "
201             Escalation"):
202         assert f"#{section}" in text and section in text, f"
203             missing_section:_{section}"
204     assert "kubectl_rollback_undo_deploy/orders" in text
205     assert "99.950%" in text # SLO rendered as a
206         percentage
207
208 def test_postmortem_is_blameless_and_complete() -> None
209 :
210     text = render_postmortem(_meta())
211     for section in ("Summary", "Impact", "Timeline", "
212         Root_cause",
213         "Action_items", "Lessons_for_the_
214             SLO"):
215         assert section in text, f"missing_section:_{
216             section}"
217     assert "blameless" in text.lower()
218     assert "SEV3" in text

```

```
209
210
211 def test_invalid_severity_rejected() -> None:
212     import pytest
213
214     with pytest.raises(ValueError):
215         IncidentMetadata(
216             incident_id="X", title="t", severity="SEV9"
217             , commander="c",
218             services=("s",), started_utc="2026-01-01T00
219             :00Z",
220             detected_by="test", slo_impact="none",
221         )
222     with pytest.raises(ValueError):
223         IncidentMetadata(
224             incident_id="X", title="t", severity="SEV1"
225             , commander="c",
226             services=(), started_utc="2026-01-01T00:00Z
227             ",
228             detected_by="test", slo_impact="none",
229         )
230
231 def test_write_artifacts(tmp_path: Path) -> None:
232     write_artifacts(tmp_path, _profile(), _meta())
233     assert (tmp_path / "runbook_orders.md").exists()
234     assert (tmp_path / "postmortem_NR-2026-0001.md").
235         exists()
```

19.4 Common Pitfalls & Anti-Patterns

1. **Logging PII, tokens, or request bodies.** One `logger.info("request %s", body)` and your log backend — with its generous retention, broad ACLs, and vendor UIs — is now a shadow database of everything Chapter 17 told you to protect. Log identifiers and outcomes, not payloads; enforce it with an allowlist formatter field set and a redaction processor in the collector as the second line of defense.
2. **High-cardinality metric labels.** `user_id`, `request_id`, concrete URL paths, or error-message strings as label values are a memory

and billing incident on a timer (Equation (19.2): 90 series become 4.5 million). Rule of thumb: if you would not put the label on a dashboard group-by, it does not belong on a metric. Use trace attributes and logs for unbounded dimensions.

3. **Averaging percentiles, or percentiling averages.** Both are statistical category errors (Proposition 19.1): quantiles do not average, and averaging first destroys the tail you were measuring. Merge histograms by adding bucket counts; estimate percentiles from the merged histogram. If your metrics pipeline cannot do that, fix the pipeline before trusting the dashboards.
4. **Alerting on every 5xx spike.** A raw error-count page fires for a 30-second blip and sleeps through a slow 30-day bleed. Alert on budget burn with the window pairs of Table 19.3: fast burn pages, slow burn tickets, blips appear on dashboards.
5. **Sampling away the errors you need.** A 1% head sampler keeps 1% of your error traces too — exactly the traces you will beg for during an incident. Sample errors at 100% (tail sampling at the collector, or a keep-all-errors sampler), and aggregate metrics *before* sampling, never from sampled spans.
6. **Dashboard sprawl without SLO anchoring.** Sixty panels that no alert references are a museum. Build dashboards outward from the SLO row; any panel that has never been cited in an incident review or a runbook is a deletion candidate. Dashboard count is not a maturity metric — decision latency is.
7. **Tracing without propagation through queues.** The `traceparent` header dies at the broker unless you explicitly inject the context into message metadata on publish and extract it on consume (Chapter 12). The symptom is a forest of single-span traces and a service map with severed edges; the fix is one inject/extract pair per messaging boundary.
8. **Mean-keyed latency dashboards and SLOs.** “Average latency 761 ms” sounds acceptable for the distribution of Figure 19.3 while 1% of users wait 24 seconds. SLOs are threshold ratios (“99% under 300 ms”); dashboards show p50/p95/p99 from histograms; the mean survives only as a capacity planning input.

19.5 Hands-On Production Lab & Real-World Case Study

Lab: Propagation, Percentiles, and Pages

Goal. Reproduce this chapter’s three load-bearing claims on your own machine: trace context survives a three-hop chain, percentiles beat averages on a long tail, and burn-rate policies page at the mathematically predicted minutes.

Setup. Save Listings 18.1–18.6 into one directory and create the virtual environment from Section 19.1.

1. **Prove the wire format.** `pytest test_trace_context.py -q` — 19 passes. Then flip the flags nibble in one valid header to 02 and confirm the parser accepts it but reports `sampled=False`: bit 0 is the only defined bit.
2. **Run the instrumented service.** `python instrumented_app.py` — three spans export to `spans.jsonl`, all sharing the incoming trace id 4bf9...4736, with the parent chain asserted. Open `spans.jsonl` and find the two RED metrics in the demo output. Then break propagation: comment out the `PROPAGATOR.inject(outbound)` in `orders_create` and re-run — the assertion fails and `inventory.reserve` becomes a root span of a new trace. You have just manufactured the most common production tracing bug.
3. **Measure the lying average.** `python latency_histograms.py` — confirm mean ≈ 761 ms against $p99 \approx 23.7$ s, then `pytest latency_histograms.py -q` (7 passes). Edit `synthetic_latencies` to drop the tail branch to 1% and watch the mean collapse toward the median while $p95$ barely moves: the mean is a tail detector, not a typical-experience metric.
4. **Watch the alerts fire.** `python burn_rate_sim.py` — the fast page fires at $t=2010$, the slow page at $t=2027$. Then edit the incident to a sustained 0.3% error rate with no spike and confirm only the ticket policy fires (`pytest burn_rate_sim.py -q` already proves both directions: 6 passes). Explain, using Equation (19.3), why the fast page fired 10 minutes into the incident and not at its first minute.
5. **Generate the paperwork.** `python incident_templates.py` — inspect the runbook and postmortem skeletons, then fill the postmortem template for the scripted incident as if you had paged on it: timeline from the alert log, root cause, and two action items.

19.5.1 Case Study: The Invisible Retry Storm

Northwind Robotics’ fleet gateway calls `orders`, which calls `inventory`. On a Thursday deploy, an `inventory` connection pool was resized from 50 to 5 (a configuration typo, reviewed and green). Inventory p50 rose from 22ms to 140ms and p99 to 1.8s — slow, but every individual request *succeeded*. The gateway’s resilience layer (Chapter 16) treated the slowness-plus-occasional-timeout as transient and retried: two retries with jittered backoff, exactly as configured.

The logs were a wall of health. Every line was a well-formed request that completed: 200s everywhere, INFO level, correct correlation ids. Aggregate error rate: under 0.1%. Nobody paged on errors, correctly.

What actually happened was visible only in the traces. Spans per trace for `POST /fleet/telemetry` jumped from a steady 3 (one per hop) to a median of 7 — one gateway span, then *three* `orders.create` CLIENT spans where there should be one, each retry fanning into its own `inventory.reserve` subtree. The trace fan-out metric — spans per trace, per route — had no dashboard panel, but the trace backend’s service-graph view showed `orders` receiving 5× its usual request rate while *client-facing* rate was flat. That mismatch was the smoking gun: retries multiply downstream load, so the victim service’s traffic diverges from the front door’s. The retry storm then deepened the latency that triggered it — a self-reinforcing loop that error-rate logs cannot see because nothing, technically, was failing.

The slow-burn SLO alert (6×, 6h/30m) paged at minute 41 of hour two: latency SLI burn, not errors. The fix was threefold: the pool typo reverted; the gateway’s retry policy capped at one retry with a per-route retry *budget* (retries themselves rate-limited, Chapter 15); and two permanent additions to the service dashboard — a *fan-out panel* (downstream requests per client request, per route) and a spans-per-trace alert at 2× baseline. The postmortem’s headline: *your retry policy is a load multiplier, and multipliers are invisible to logs*. War-story rules worth keeping: alert on the topology of traffic (fan-out), not only its volume and failures; and treat every resilience mechanism as something that can itself become the incident [Nyg18].

19.6 Self-Assessment Exercises & Deep-Dive Questions

1. **SLI selection.** Northwind Robotics ships a new endpoint, `GET /fleet/route-plan`, that computes asynchronously and returns 202 + a polling URL (Chapter 12). Write SLIs for availability, latency, *and* freshness of the finished plan. Which events count as “good” for each, and why is “request returned 202” not an availability SLI for

the plan itself?

2. **Budget arithmetic.** A 99.95% SLO service burned 14 minutes of budget by day 10. What average burn rate does that represent, and what is the latest day on which a $2\times$ sustained burn can start without exhausting the month's budget?
3. **Window pairs.** Using Equation (19.3), show that a 100% error burst lasting t minutes raises the 1-hour burn rate to $t/(0.001 \cdot 60)$. From that, derive the shortest full-outage that fires the $14.4\times$ fast page, and check your answer against Listing 18.5's simulator.
4. **Histogram design.** Choose bucket boundaries for an endpoint with a 300 ms latency SLO and explain why at least two buckets must straddle 300 ms. What does the percentile estimate's maximum error become at the SLO threshold under your layout?
5. **Cardinality audit.** A teammate proposes labels `route`, `status`, `tenant_id` (4,000 tenants), and `client_version` (30 versions) on the request histogram. Compute the series count, decide which labels survive, and say where the rejected dimensions go instead.
6. **Propagation through a queue.** Sketch the inject/extract code path for telemetry published to a message broker, including what happens to the trace when a consumer batches 50 messages into one processing span. (Hint: span *links*, not parents.)
7. **Fallacy patrol.** Your weekly report shows "average p99 latency across instances: 210 ms." List two distinct ways this number can understate user pain, and give the correct aggregation.
8. **Sampling design.** You may keep 5% of trace volume. Design the sampling policy (head + tail) for a fleet API where errors cost you money but slow-and-successful requests cost your *customers* money. State precisely which traces are guaranteed to survive.
9. **Debugging drill.** p99 for `POST /orders` tripled; per-span durations in sampled traces are unchanged, but the gap between the `SERVER` span's start and its first child span grew. Name three candidate causes and the USE metric that confirms or clears each.
10. **Postmortem review.** Given a postmortem whose root-cause section reads "on-call engineer misconfigured the pool," rewrite it blamelessly: what mechanism questions replace the person question, and which action items follow?

19.7 Interview Q&A Vault

Junior (Q1–Q10)

- Q1. What are the three pillars of observability, and what question does each answer?** Logs: what happened, once, here (events). Metrics: how often and how fast, aggregated cheaply (numbers over time). Traces: where did this one request's time go across services (causality). They join on shared identifiers — trace ids in logs, exemplars on metrics.
- Q2. What makes a log line “structured,” and why do you care?** It is a machine-parseable object (one JSON per line) with a fixed schema — timestamp, level, logger, message, correlation ids — rather than free text. You care because parsing regexes over prose during an incident is how minutes of error budget become hours.
- Q3. What may never appear in logs?** PII, credentials, tokens, session ids, request/response bodies, and anything Chapter 17 classifies as sensitive. Logs have wide read access and long retention; treat them as a broadcast medium.
- Q4. Define RED and USE.** RED — Rate, Errors, Duration: the service-level view (are users hurting?). USE — Utilization, Saturation, Errors: the resource-level view (which pool, queue, or disk is the bottleneck?). You page on RED and diagnose with USE.
- Q5. What is a span? A trace?** A span is a named, timed operation with attributes, a status, and a parent pointer. A trace is all spans sharing one trace id — the tree of one request's work across every service it touched.
- Q6. What does the `traceparent` header look like?** Four dash-separated lowercase-hex fields: `version-traceid-spanid-flags`, e.g. `00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`. Version `ff` is invalid; all-zero ids are invalid; flags bit 0 is *sampled*.
- Q7. What is an SLO, in one sentence?** A measurable target on a service level indicator over a rolling window — e.g. “99.9% of requests succeed over 30 days” — against which you define an error budget and alerting.
- Q8. How much downtime does a 99.9% monthly SLO allow?** $0.001 \times 43,200$ minutes = **43.2 minutes** per month. (99.99% allows 4.32; 99% allows 7 h 12 min.)

- Q9. Why is average latency a bad dashboard number?** Latency is long-tailed; the mean is dragged into the tail and describes no actual user. Use histogram-based p50/p95/p99 — Listing 18.4’s demo shows a 761 ms mean over a 25 ms median and a 23.7 s p99.
- Q10. What is a correlation id and where does it live?** One identifier (here: the W3C trace id) attached to a request at the edge and echoed in every log line, span, metric exemplar, and problem-details response, so one id retrieves the request’s whole story.

Mid (Q11–Q20)

- Q11. Walk through the OpenTelemetry architecture.** Application code calls the vendor-neutral *API*; the *SDK* (configured by the application only) runs samplers, span processors, and metric aggregations; *exporters* ship OTLP to the *collector*, which batches, redacts, tail-samples, and fans out to per-signal backends. Vendor changes stop at collector config [Ope24].
- Q12. Why can’t you average p99s across instances or minutes?** Quantiles are order statistics, not additive measures; the average of per-minute p99s is not any percentile of the underlying distribution and systematically understates the tail (Proposition 19.1). Correct path: merge histogram bucket counts, then compute the percentile from the merged histogram.
- Q13. Explain multi-window multi-burn-rate alerting.** Alert on burn rate (observed error rate $\div$ budget rate), and fire only when a *long* window (precision: the burn is real and sustained) and a *short* window (currency: it is happening now) both exceed the threshold — Equation (19.4). Canonical 99.9% table: 14.4 $\times$ with 1h+5m (2% budget), 6 $\times$ with 6h+30m (5%), 1 $\times$ with 3d+6h (10%, ticket).
- Q14. Head vs tail sampling trade-offs?** Head sampling decides at trace start: cheap, unbiased statistics, but it discards rare error traces at the sampling rate. Tail sampling decides at the collector after buffering whole traces: keeps all errors and slow traces, at the cost of collector memory, decision latency, and operational complexity. Hybrids head-sample the healthy bulk and tail-sample the interesting tail.
- Q15. What is cardinality and why does it bankrupt metrics systems?** Series count is the product of label cardinalities (Equa-

tion (19.2)); each series costs index and chunk memory plus ingestion on every scrape. One unbounded label (`user_id`) turns 90 series into 4.5 million. Labels must be dashboard group-bys; unbounded dimensions belong in logs and trace attributes.

- Q16. What are exemplars?** Trace ids pinned to specific histogram observations. They convert “p99 spiked” into one clickable slow trace — the metric-to- waterfall bridge — and they only work because metrics and traces share the trace-id namespace.
- Q17. Design SLIs for an asynchronous (202 + polling) API.** Three ratios: submission availability (accepted vs. rejected submissions), completion latency (fraction of jobs finishing within T of submission, measured at the poller or job store), and freshness/correctness of results where applicable. The HTTP 202 latency is a client-ergonomics metric, not the job’s SLI — the unit of good is the *finished job*, not the acknowledgment.
- Q18. How does traceparent propagate through a queue?** It does not — by itself. There are no HTTP headers on a broker message unless you put them there: inject the context into message metadata on publish, extract it on consume, and start the consumer span as a `CONSUMER` kind (with span links when batching). The OTEL propagator API is transport-agnostic; the discipline is not automatic.
- Q19. What is the difference between tracestate and baggage?** `tracestate`: vendor-specific propagation state (sampling decisions, vendor ids), a ≤ 32 -member `key=value` list with strict syntax, managed by tracers. `Baggage`: user-defined key/value pairs propagated with the trace for application use (tenant, plan tier) — deliberately separate, and a PII leak vector if abused because it rides every outbound call.
- Q20. Your p99 latency alert fires. First three moves?** (1) Check the SLO dashboard: is this budget burn or a blip — which endpoint, which burn window? (2) Pull an exemplar trace from the violated histogram bucket and decompose the waterfall: dominant span or inter-span gap? (3) Check USE metrics for the implicated resource (pool utilization, queue depth) before reading a single log line.

Senior (Q21–Q30)

- Q21. Derive the fast-page window pair from first principles.**
A 99.9% SLO allows budget rate 0.001. To page when 2% of the monthly budget burns in one hour, require burn rate b with $b \times 1\text{h}/720\text{h} = 0.02$, so $b = 14.4$. The 1-hour long window estimates the burn with enough samples; the 5-minute short window confirms it is still active, so the alert resolves after mitigation instead of paging on history.
- Q22. Why do 4xx responses not count against an availability SLO?** The SLI measures *service* behavior; a 400 means the client sent an invalid request — counting it would let one buggy integrator burn your budget and page you for their bug. Track 4xx as a separate indicator (sudden 4xx shifts do reveal bad deploys and doc drift), but the availability SLI counts 5xx and transport failures.
- Q23. Your traces show healthy spans but the gaps between spans grew. Where did the time go?** Between-span time is work nobody instrumented: connection acquisition from an exhausted pool, TLS handshakes on cold connections, DNS, thread/executor queueing, GC pauses, or client-side retries (check fan-out first — see Section 19.5.1). Confirm with USE metrics on the suspected resource, then add spans for the confirmed stage.
- Q24. How do you keep metrics correct while sampling traces?**
Aggregate metrics from every request *before* sampling — counters and histograms are unconditional; only span export is sampled. Never derive request rates or latency percentiles from sampled spans: your RED metrics would be off by the sampling factor, silently.
- Q25. Design a latency SLO for an endpoint with a heavy tail of legitimate work (large fleet exports).** Split the SLI by request class (separate export vs. interactive routes, different thresholds), pick the threshold from the histogram at the knee rather than from a wish, and express the SLO as a threshold ratio (“99% of interactive requests under 300 ms”). If one route genuinely has two populations, one SLI will lie to one of them.
- Q26. What breaks when you attach `http.route` as the raw path instead of the template?** `/fleet/robot-4821` instead of `/fleet/{id}` makes every id a new label value: per-request series creation, index blowup, dashboards that cannot aggregate, and

eventually a metrics backend that drops or bills you into noticing. Route templates are a cardinality control, not a cosmetic choice.

- Q27. Explain the alert-fatigue failure mode and its fix.** Pages that do not require immediate action train the on-call to ignore pages; response latency to *real* pages degrades — fatigue is a system output, not a character flaw. Fix: page only on symptom-level budget burn (fast/slow policies), ticket the rest, delete or demote alerts that fire unactionably, and review page volume as a budget.
- Q28. How would you verify a third-party client’s claim that “your API was down for us at 14:03”?** Ask for the correlation id or `traceparent` they sent (you taught them to, via problem-details `traceId`); query traces and logs for that id; then check the SLI window around 14:03 for their route and region. If their requests never appear in server telemetry, the failure is between them and your edge — which is also a finding.
- Q29. When is a gauge the wrong instrument?** For anything you will want to aggregate across instances or percentile over time: gauges capture instantaneous values at scrape time and lose everything between scrapes. Rates come from counters, distributions from histograms; gauges are for levels that are meaningful instantaneously (queue depth, pool in-use).
- Q30. What belongs in a runbook entry for one alert?** The SLO it protects; the dashboard that confirms it; the triage checklist (ten minutes, ordered); mitigation with exact commands (rollback, flag, drain); escalation path with ack timeouts; and the date it was last exercised. A runbook that has never been drilled is a hypothesis.

Staff (Q31–Q36)

- Q31. Design the observability architecture for a 40-service API platform. What do you standardize centrally vs. leave to teams?** Central: the collector tier (endpoints, redaction, tail sampling, per-tenant quotas), the trace-context and baggage policy, metric naming and label vocabularies (route templates, status classes), the SLO framework and burn-rate alerting library, exemplar wiring, and the dashboard/slo-as-code templates. Teams: SLI thresholds and SLO targets for their services, domain attributes on spans, and their runbooks. The invariant: propagation and cardinality discipline are platform concerns; target-setting is an ownership concern.

- Q32. When would you deliberately *lower* an SLO?** When the budget is consistently unspent and users cannot perceive the marginal reliability — an over-tight SLO is a tax on velocity (frozen releases, rejected experiments) buying nothing. SLOs are set from user pain and business tolerance, not from what the architecture happens to achieve; ratcheting to the observed maximum is the stealth failure mode.
- Q33. How do error budgets change organizational behavior, and how do you keep them honest?** The budget converts reliability from a vibe into a shared currency: unspent budget authorizes risk (faster releases, experiments); exhausted budget shifts the roadmap to reliability by policy, not by argument [Bey+16]. Keeping it honest: the SLI must be user-visible (no internal-health proxies), the measurement window and query are versioned and reviewed, and budget policy changes are signed off like contracts — otherwise teams relitigate the ruler instead of fixing the service.
- Q34. Critique: “we sample 10% of traces and compute our SLO from the sampled spans.”** Two independent errors. First, metrics from sampled spans are biased by the sampler (and catastrophically so under tail sampling, which *over-represents* errors and slow traces — your SLI would read worse than reality, or better under head sampling if errors correlate with sampling bugs). Second, SLOs must be computed from complete event data: unsampled request logs, load-balancer records, or pre-sampling metric aggregations. Traces are for diagnosis; SLIs are for accounting; never mix the supplies.
- Q35. A queue-based pipeline drops trace continuity at three broker boundaries. Design the fix and its failure modes.** Inject `traceparent`/`tracestate` into message metadata at every publish (one carrier-agnostic propagator call), extract at every consume, start `PRODUCER/CONSUMER` spans, and use span *links* for fan-in batches so one consumer span can reference fifty parents without lying about causality. Failure modes to design for: metadata size limits on the broker, consumers on old library versions that drop unknown fields (contract-test the metadata schema), and the temptation to log the baggage — which now travels everywhere.
- Q36. Your CEO asks: “why did we invest in all this telemetry when the outage still lasted 40 minutes?” Defend the program with numbers.** Compare against the counterfactual the telemetry itself records: detection time (alert fired at minute

4 of burn, not at a customer ticket at minute 40), diagnosis time (exemplar-to-root-cause in minutes via the waterfall vs. hours of log archaeology), and budget math (40 minutes consumed vs. 43.2 allowed — the month survived). Observability does not prevent incidents; it compresses detection and diagnosis, which are the two phases engineering actually controls. Then say what will shrink mitigation: better runbooks and rehearsed rollbacks — the program is how you know that.

19.8 External Bibliography & Further Reading

- B. Beyer, C. Jones, J. Petoff, N. Murphy (eds.), *Site Reliability Engineering* (O’Reilly, 2016) — the source of SLOs, error budgets, and the elimination-of-toil philosophy this chapter operationalizes [Bey+16].
- B. Beyer, N. Murphy, D. Rensin, K. Kawahara, S. Thorne (eds.), *The Site Reliability Workbook* (O’Reilly, 2018) — chapter 4 (“Implementing SLOs”) and chapter 5 (“Alerting on SLOs”) are the origin of the multi-window multi-burn-rate table implemented in Listing 18.5 (uncited in text; the practical companion to the SRE book).
- OpenTelemetry project documentation — API/SDK concepts, the collector, and the HTTP semantic conventions referenced throughout Section 19.2 [Ope24].
- W3C, *Trace Context* (Recommendation) and *Baggage* — the normative specifications for `traceparent` and `tracestate` that Listings 18.1–18.2 implement by hand (uncited in text; short, readable, and worth reading in full).
- C. Majors, L. Fong-Jones, G. Miranda, *Observability Engineering* (O’Reilly, 2022) — the high-cardinality, high-dimensionality argument for event-first observability and the clearest treatment of exemplar-style debugging loops (uncited in text).
- Prometheus documentation, *Histograms and Summaries* — why percentiles must come from histogram buckets, the `histogram_quantile` interpolation behind Equation (19.1), and the unaggregability of summary quantiles (uncited in text).
- J. Turnbull, *The Art of Monitoring* — the opinionated end-to-end framing of metrics, alerting, and dashboard design that shaped Sec-

tion [19.2.8](#) (uncited in text).

- RFC 9110, *HTTP Semantics* — status-code classes that define “good” versus “bad” events in availability SLIs, and the header semantics trace context rides on [[Fie22a](#)].
- M. Kleppmann, *Designing Data-Intensive Applications* — the messaging and stream-processing background for the propagation-through-queues discussion [[Kle17](#)].
- M. Nygard, *Release It!*, 2nd ed. — the stability patterns whose interactions with telemetry produced Section [19.5.1](#)’s retry storm [[Nyg18](#)].
- FastAPI documentation — middleware and the ASGI surface instrumented in Listing 18.3 [[Ram24](#)].
- A. Hétu, *The HdrHistogram* project documentation — the log-bucket layout and its bounded-relative-error guarantee re-implemented in Listing 18.4 (uncited in text).

19.9 Capstone Projects

Capstone 18.1 — Trace Context Conformance Pack [junior]

Objective: Prove you own the wire format by extending the parser into a conformance checker.

Inputs/Datasets: Listings 18.1–18.2; a hand-written file of 20 `traceparent` headers (10 valid, 10 invalid in distinct ways) plus 5 `tracestate` headers.

Steps: (1) Write the header corpus as a data file, one header per line with an expected verdict. (2) Build a checker that runs every corpus line through `parse_traceparent` and reports pass/fail per case. (3) Add version-01 headers with trailing fields and verify forward-compatible parsing. (4) Add a property test: for 500 seeded random contexts, `parse(header(x)) == x`.

Acceptance Criteria:

- ☐ All 25 corpus cases produce the expected verdict.
- ☐ The seeded round-trip property test passes 500/500.
- ☐ Every rejection message names the offending field.

Stretch Goals: Fuzz with random truncations and case flips; assert the parser never throws anything but `ValueError`.

Capstone 18.2 — RED Metrics for a Real Route Set [mid]

Objective: Extend Listing 18.3 into a per-endpoint RED story for Northwind Robotics' four public routes.

Inputs/Datasets: Listing 18.3; routes `POST /fleet/telemetry`, `GET /fleet/status`, `GET /fleet/{id}`, `GET /health`.

Steps: (1) Record counter and histogram with attributes `route` (template only), `status_class` (2xx/4xx/5xx), nothing else. (2) Drive 2,000 seeded synthetic requests with a per-route latency model and a 0.2% error rate on one route. (3) Read back the `InMemoryMetricReader` and render a per-route RED table (rate, error ratio, p50/p99 from bucket counts). (4) Prove the cardinality bound: assert the exported series count equals `routes × status classes`, exactly.

Acceptance Criteria:

- ☐ RED table prints with per-route error ratios matching the seeded model within sampling jitter.
- ☐ Series-count assertion passes; adding a `user_id` label makes it fail loudly (demonstrate, then revert).
- ☐ Everything deterministic under one seed; no network.

Stretch Goals: Attach exemplars (one trace id per histogram bucket observation) and print one exemplar id for the p99 bucket.

Capstone 18.3 — SLO Workbench [mid]

Objective: Turn Listing 18.5 into a policy design tool and defend one design change with data.

Inputs/Datasets: Listing 18.5; three scripted incidents you write (a 3-minute total outage, a 6-hour 2% error bleed, a flapping 30-seconds-on/30-seconds-off failure).

Steps: (1) Encode the three incidents as timeline generators with distinct seeds. (2) Run the stock policy table against each; record time-to-page and false positives. (3) Propose one modification (e.g. a third window pair at $3\times/1d+2h$, or a flapping-damping reset rule) and show its effect on all three incidents. (4) Write a one-page memo: which table ships, and why, in budget-consumption terms.

Acceptance Criteria:

- ☐ All three incidents produce a documented, reproducible alert timeline.
- ☐ The flapping incident is shown to reset-and-refire under the stock table, and your modification addresses it.

- The memo states budget math for every policy row.

Stretch Goals: Add alert-resolution tracking (when would each page auto-resolve?) and report mean time to resolve per policy.

Capstone 18.4 — Queue Propagation End to End [senior]

Objective: Close the tracing gap this chapter warns about: one trace across HTTP *and* a message broker.

Inputs/Datasets: Listing 18.3; an in-process fake broker you write (dict of queues; no network, deterministic).

Steps: (1) Publish telemetry from the gateway to the fake broker, injecting `traceparent`/`tracestate` into message metadata. (2) A consumer extracts context and starts a `CONSUMER` span parented correctly; a batch consumer uses span links for its 50 parents. (3) Export and assert: one trace id spans HTTP ingress, publish, and consume; batch consumption produces one span with 50 links. (4) Write the failure-mode test: a legacy consumer that drops unknown metadata fields produces a detectably orphaned span (assert and log a warning).

Acceptance Criteria:

- The exported tree crosses the broker boundary with one trace id and correct parentage.
- Batch consumption uses links, not a false single parent.
- The legacy-consumer test detects the orphaned span programmatically.

Stretch Goals: Add a redaction pass that strips baggage at the queue boundary and prove no baggage key survives into consumer spans.

Capstone 18.5 — Incident Program in a Box [staff]

Objective: Stand up the complete operational loop for the fleet-telemetry platform and run a game day against it.

Inputs/Datasets: Listings 18.3–18.6; the case study of Section 19.5.1 as the game-day scenario.

Steps: (1) Define SLOs (availability + latency + freshness) for three services with per-endpoint thresholds. (2) Generate runbooks from Listing 18.6 for all three services and fill every placeholder. (3) Script the retry-storm incident into the simulator’s timeline plus a fan-out metric; verify the fast/slow pages and show that a spans-per-trace alert fires *before* the latency burn pages. (4) Run the game day: one teammate injects the fault, another follows only the runbook; measure detection, mitigation,

and resolution times. (5) Produce the blameless postmortem from the generator, including the “lessons for the SLO” section with at least one concrete alerting change.

Acceptance Criteria:

- ☐ SLOs, runbooks, and dashboards-as-tables exist for all three services and cite one another.
- ☐ The game day produces measured detection/mitigation/ resolution times, all reproducible from seeds.
- ☐ The postmortem is complete, blameless, and its action items include one alerting-topology change with an owner.
- ☐ A second run by a different pair reproduces the timelines within one minute of simulated time.

Stretch Goals: Add a budget-policy gate: when the simulator shows >75% budget consumed, the deploy script refuses to ship and explains why.

19.10 Python Dependencies Appendix

All code runs offline after a one-time install. From PowerShell:

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
```

requirements.txt:

```
1 opentelemetry-sdk>=1.25
2 opentelemetry-instrumentation-fastapi>=0.46b0
3 fastapi>=0.110
4 pytest>=8
```

For real deployments (never needed by this chapter’s listings): add the OTLP exporter package (`opentelemetry-exporter-otlp`) and point it at a collector endpoint — the `SpanExporter` interface Listing 18.3 implements is the same one OTLP satisfies, so the swap is constructor-deep.

19.10.1 opentelemetry-sdk (≥ 1.25)

Description: The OpenTelemetry SDK for Python: tracer and meter providers, span processors, samplers, metric views, and the propagator API. **Why included:** it is the entire telemetry backbone of Listing 18.3 — and its `SpanExporter` interface is what lets the chapter replace OTLP

with an offline JSON-lines exporter without touching application code.

Alternatives: the Prometheus `prometheus_client` (metrics only — no traces, no logs, and a pull model that changes your deployment shape); the Datadog SDK / `ddtrace` (vendor-locked but zero-collector); `structlog` for structured logging specifically (excellent, and composable with the `stdlib` handler used here — but it is a logging library, not a telemetry pipeline).

Installation: `pip install "opentelemetry-sdk>=1.25"`.

19.10.2 `opentelemetry-instrumentation-fastapi` ($\geq 0.46b0$)

Description: Auto-instrumentation that wraps every FastAPI route in a `SERVER` span with semantic-convention attributes and extracts incoming `traceparent` headers. **Why included:** it is the difference between teaching propagation as a diagram and having it happen on every request for free; Listing 18.3 instruments the app with it and then demonstrates the manual inject/extract path it automates. **Alternatives:** hand-written ASGI middleware (full control, full maintenance); the `opentelemetry-instrumentation-asgi` package it builds on (framework-agnostic, more manual); vendor agents (locked). **Installation:** `pip install "opentelemetry-instrumentation-fastapi>=0.46b0"` — the pre-1.0 instrumentation packages version in lockstep with the SDK (0.46b0 pairs with SDK 1.25).

19.10.3 `fastapi` (≥ 0.110)

Description: The ASGI framework hosting the instrumented service. **Why included:** the chapter's service is a FastAPI app because the book's services are FastAPI apps (Chapter 7); the framework supplies routing, middleware, and the request objects the instrumentation wraps [Ram24]. **Alternatives:** Starlette directly (the instrumentation targets ASGI and would work unchanged); Flask via `opentelemetry-instrumentation-flask` (WSGI; identical concepts); a gateway-level service mesh (sidecar telemetry without code changes, at the price of losing in-process spans). **Installation:** `pip install "fastapi>=0.110"`.

19.10.4 `pytest` (≥ 8)

Description: Test framework running every embedded suite in this chapter (36 tests across four files). **Why included:** the chapter's claims are executable — the header round-trips, histogram percentile proofs, and burn-rate policy tests all run under it. **Alternatives:** `unittest` (`stdlib`; the suites would survive a port, but `parametrize` and `tmp_path` would

need hand-rolling); `nose2` (unmaintained momentum). **Installation:** `pip install "pytest>=8"`.

Ecosystem alternatives worth knowing

For production telemetry beyond this chapter's offline exporters: the OpenTelemetry Collector itself (the binary this chapter simulates), Prometheus + `prometheus_client` for metrics-only deployments, Grafana Tempo/Jaeger for trace storage, Loki or Elasticsearch for logs, and `structlog` when you want richer structured logging pipelines than a `stdlib` formatter. Evaluate each against Section 19.2: which signal, which cardinality story, which sampling point, which failure mode when the backend is down.

19.11 Chapter Summary & Key Takeaways

- The three pillars answer different questions and join on one identifier: structured JSON logs (levels, sampling, never PII or secrets), RED metrics for services and USE for resources, and traces as parent-linked span trees propagated by context.
- OpenTelemetry's layering is the architecture: code to the API, configure the SDK, export to a collector, and let the collector own vendor coupling — the offline file exporter in Listing 18.3 is the same interface OTLP satisfies [Ope24].
- `traceparent` is four hex fields — version, trace id, span id, flags — with all-zero ids and version `ff` invalid and bit 0 meaning *sampled* (Table 19.1); queues do not propagate it for you — inject and extract at every messaging boundary.
- Latency is a histogram. Percentiles come from bucket counts (Equation (19.1)); averaging first destroys the tail (Proposition 19.1); cardinality is multiplicative and priced per series (Equation (19.2)); exemplars bridge metrics to traces.
- SLOs turn telemetry into judgment: availability, latency, and freshness SLIs as good/total ratios; error budgets (43.2 minutes per month at 99.9%); and multi-window multi-burn-rate alerts — $14.4\times$ over 1h+5m, $6\times$ over 6h+30m, $1\times$ over 3d+6h — that page on budget burn instead of noise (Table 19.3) [Bey+16].
- Operations is a discipline, not a mood: correlation ids end to end, dashboards anchored to SLOs, latency debugged by timeline decom-

position (span or gap first, logs last), incidents run with severities, roles, runbooks, and blameless postmortems.

- The retry storm of Section 19.5.1 is the exam question: resilience mechanisms multiply load invisibly to logs; alert on the topology of traffic (fan-out, spans per trace), not only its volume and errors.

19.12 Quick-Reference Card

SLI Formulas

Availability = $\frac{\# \text{status} < 500}{\# \text{requests}}$ • Latency = $\frac{\# \text{requests} < T}{\# \text{requests}}$ (from histogram buckets)
• Freshness = $\frac{\# \text{reads newer than } D}{\# \text{reads}}$ • Error budget = $(1 - \text{SLO}) \times \text{window}$:
99.9%/30d = 43.2 min; 99.95% = 21.6 min; 99.99% = 4.32 min • Burn rate
= $\frac{\text{error rate}}{1 - \text{SLO}}$ • Budget consumed = burn $\times w / 43,200$ min.

Burn-Rate Window Table (99.9% SLO)

Fast page: 14.4 $\times$, long 1h + short 5m, 2% budget • slow page: 6 $\times$, long 6h + short 30m, 5% budget • ticket: 1 $\times$, long 3d + short 6h, 10% budget •
fire $\iff \text{burn}_{\text{long}} \geq b \wedge \text{burn}_{\text{short}} \geq b$; reset when the long window clears.

traceparent Field Reference

version-traceid-spanid-flags • version: 2 hex, ff forbidden, 00 forbids extra fields • trace id: 32 hex, nonzero, constant across hops • span id: 16 hex, nonzero, fresh per hop, recorded as the callee's parent • flags: 2 hex, bit 0x01 = sampled; echo only defined bits • **tracestate**: ≤ 32 vendor **key=value** members • baggage: user key/values, propagates everywhere — never PII.

Field Discipline (compressed)

Logs: JSON, levels mean something, sample hot paths, no PII/tokens/bodies • Metrics: RED per route template + status class only; histograms for latency; exemplars on; series = product of label cardinalities • Traces: SERVER/CLIENT/INTERNAL/PRODUCER/CONSUMER kinds; metrics aggregated *before* sampling; errors kept at 100% • Debug order: SLO dashboard $\rightarrow$ exemplar waterfall $\rightarrow$ dominant span or gap $\rightarrow$ USE metrics $\rightarrow$ correlated logs • Incidents: severity with a response contract, commander coordinates (never debugs), runbook first ten minutes, postmortem blames mechanisms.

19.13 Standardized Evidence Addendum

19.13.1 Benchmark Snapshot Template

19.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

19.13.3 Cost and Latency Budget Snapshot

19.14 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

19.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

19.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

19.17 Cross-Chapter Decision Card

19.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

19.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Severity	Definition	Response
SEV1	Total All-hands page; incident commander appointed in minutes. out-age or data loss; SLO budget in-cin-er-at-ing.	
SEV2	MajorPage owning team; commander + comms lead. degra-da-tion of a core SLO; sus-tained fast burn.	
SEV3	MinorTicket; next business day. degra-da-tion or single-tenant im-pact.	
SEV4	CosmeBacklog. or internal-only.	

Table 19.4: Severity levels. The response column is part of the definition: a severity without a response contract is a label, not a level.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 19.5: Minimum benchmark table to keep per chapter.

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 19.6: Operational budget template for this chapter’s technique.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 19.7: Decision card to complete per chapter.

Chapter 20

Testing & Quality Engineering for APIs

An API you have not tested is a rumor. Worse, an API tested only by its own developers is a rumor the provider tells itself: every chapter of this book so far has built artifacts — HTTP semantics (Chapter 2), serializers (Chapter 5), FastAPI services (Chapter 7), RFC 9457 error shapes (Chapter 8), versioned evolvable contracts (Chapter 10), resilience policies (Chapter 16) — and each artifact makes promises that *someone else* will depend on. This chapter is about turning those promises into executable evidence, and about building the quality gates that refuse to ship when the evidence goes missing.

We begin with the **API test pyramid**: unit tests for validators and serializers, component tests that run an endpoint against real in-process dependencies, contract tests that pin the consumer-provider agreement, integration tests that cross process boundaries, and a thin crust of end-to-end journeys. For each layer we state precisely which defect classes it can and cannot catch, its cost and speed profile, and where the “testing trophy” critique of the classic pyramid is right and wrong. We then go deep on **consumer-driven contract testing** with the Pact model [Pac24]: the consumer records interactions into a pact file, a broker distributes it, the provider replays it against real code using named provider states, and a **can-i-deploy** gate blocks incompatible releases. We contrast this with OpenAPI-diff schema checking [Ope21] and give a decision rule for when each applies.

Next come the two most under-used techniques in API engineering. **Property-based testing** replaces hand-picked examples with generated universes of inputs and machine-checked invariants — never 5xx, schema-conformant 2xx, problem-shaped 4xx — with automatic *shrinking* of counterexamples and rule-based *stateful* testing of API sequences. **Offline-first mocking** gives us a precise fake/stub/mock taxonomy, `httpx.MockTransport-`

style interception, record–replay cassettes, and — critically — *virtual clocks* that let us test retries, timeouts, and circuit breakers deterministically without ever calling `sleep`. Then we build **load and performance testing** from first principles: open versus closed workload models, coordinated omission, a derivation and application of Little’s Law $L = \lambda W$, saturation sweeps and knee-point analysis, and the load/stress/soak/spike taxonomy [Nyg18; Bey+16]. **Chaos engineering** follows: a fault taxonomy (latency, errors, aborts, partitions, resource exhaustion), blast-radius control, and steady-state hypotheses, implemented as deterministic fault-injection middleware. Finally we assemble everything into **CI quality gates as code**: coverage floors, contract gates, performance regression budgets, fuzz smoke tests, and disciplined flake management.

Every technique is backed by runnable, offline, deterministic Python 3.11 code against the synthetic Northwind Robotics robot registry: a mini contract-testing framework that *proves* it catches a breaking field rename, a Hypothesis suite plus a stdlib OpenAPI fuzzer that finds and shrinks a planted bug, an `asyncio` load harness that prints a knee-point table, fault-injection middleware exercised entirely on virtual time, and a single `ci_gate.py` that runs the whole battery and exits 0 or 1.

Key Concept

A test suite is not a pile of checks; it is a *supply chain of evidence*. Each layer manufactures a different kind of confidence at a different unit cost: units catch logic bugs in milliseconds, contracts catch cross-team breakage without a shared environment, fuzzers explore input spaces humans cannot enumerate, load harnesses locate the saturation knee before your customers do, and chaos experiments prove your resilience claims instead of assuming them. The craft of quality engineering is allocating budget across these layers so that the *cheapest* layer that can catch each defect class is the one that does.

20.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Place any given test on the API test pyramid (unit, component, contract, integration, end-to-end), state which defect classes that layer catches, and defend the cost/speed trade-off of the placement, including the testing-trophy critique of the classic pyramid.

- LO-2.** Implement the consumer-driven contract loop end to end: record consumer interactions into a pact file, distribute it, replay it on the provider against real code using named provider states, and gate deployment on verification — and articulate when Pact-style contracts beat OpenAPI-diff schema checking and vice versa [[Pac24](#); [Ope21](#)].
- LO-3.** Write property-based API tests with explicit strategies and invariants (never 5xx; 2xx validates against the response schema; 4xx is RFC 9457 shaped), explain how shrinking minimizes counterexamples, and design rule-based stateful tests that catch sequence-dependent bugs.
- LO-4.** Distinguish fakes, stubs, mocks, and record–replay cassettes with precise definitions, and test retries, timeouts, and circuit breakers deterministically using virtual clocks and scripted fault injection instead of wall-clock sleeps [[Enc24](#)].
- LO-5.** Build an offline load harness implementing open and closed workload models, derive and apply Little’s Law $L = \lambda W$ to capacity questions, run a saturation sweep, and read a knee-point table to set performance budgets.
- LO-6.** Define a fault taxonomy (latency, error, abort, partition, resource exhaustion), state a steady-state hypothesis, bound a blast radius, and implement deterministic fault-injection middleware for tests.
- LO-7.** Assemble CI quality gates as code — coverage floors, contract verification, fuzz smoke, performance smoke with structural (not wall-clock) assertions — and operate a flake-management policy that keeps gates trustworthy.

Prerequisites

This chapter assumes you have completed:

- **Chapters 0–5** — Python 3.11+ environment, HTTP wire semantics, consuming APIs with `httpx`, serialization contracts, and client-side security hygiene (no credentials in tests, ever).
- **Chapter 6** (Building APIs with FastAPI) — application factories, dependency injection, Pydantic models, and running an app in-process through an ASGI transport [[Ram24](#)].

- **Chapter 7** (Errors, Validation & Problem Details) — RFC 9457 application/problem+json shape; our fuzz gate asserts it for every 4xx.
- **Chapter 9** (Versioning & Evolution) — additive versus breaking change; the contract-diff gate connects directly to the compatibility rules defined there.
- **Chapter 15** (Microservices & Resilience) — retries, timeouts, and circuit breakers, which we now learn to test deterministically.
- **Chapter 18** (Observability & SRE) — latency percentiles and SLIs; the load harness histogram reuses that chapter’s percentile machinery.

All code runs offline on Windows with PowerShell: services are exercised in-process via ASGI transports, pact files live on local disk, and every random process is seeded or derandomized. Reported performance numbers are illustrative, never asserted.

20.2 Deep Technical Mechanics & Architectural Concepts

20.2.1 The API Test Pyramid

The test pyramid is a portfolio theory, not a geometry lesson. Each layer answers a different question, at a different speed, at a different cost per defect found:

Definition 20.1 (Test layers for APIs). A **unit test** exercises a pure unit of logic — a validator, a serializer, a pagination-token codec — with no I/O. A **component test** exercises one whole service in-process: real routing, real validation, real serialization, with external dependencies replaced or embedded. A **contract test** checks that a provider still satisfies the recorded expectations of its consumers (or that a spec change is backward compatible). An **integration test** crosses a real process or infrastructure boundary: service to database, service to queue, service to service. An **end-to-end (E2E) test** drives a user journey through the deployed system, front door to database and back.

The economics are unforgiving. A defect caught by a unit test costs a millisecond rerun; the same defect caught by an E2E test costs a distributed-systems debugging session across three teams’ dashboards. The inverted pyramid (“ice-cream cone”) — a thin unit layer and a fat E2E layer — is the single most common structural failure of API test suites: it is slow,

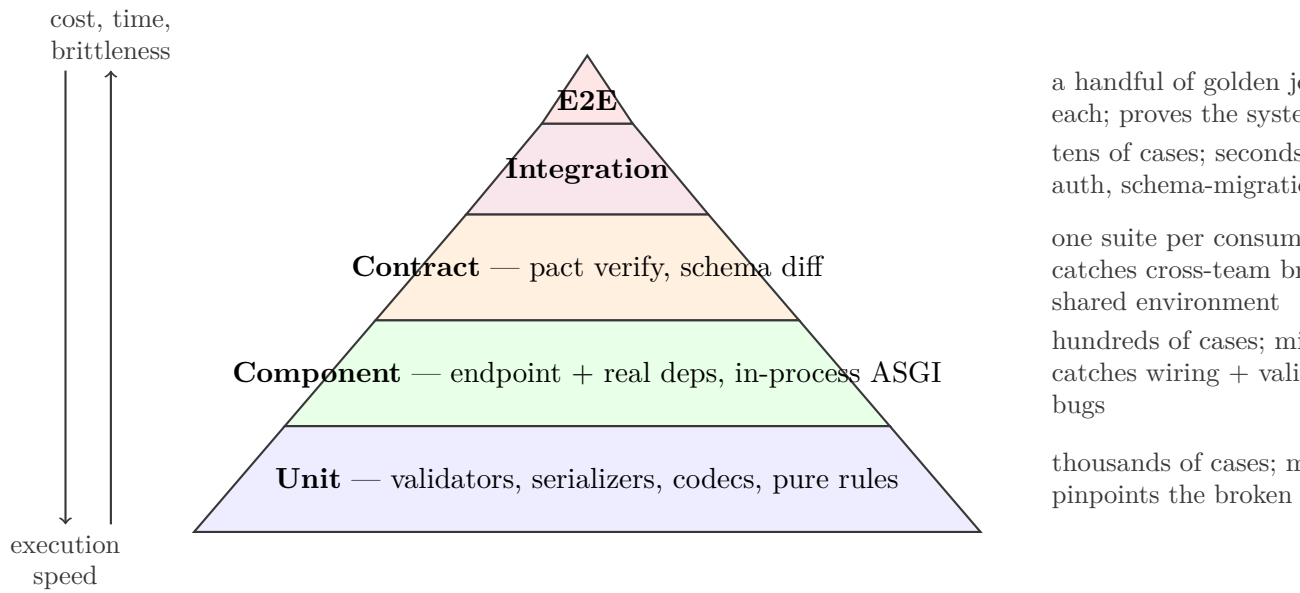


Figure 20.1: The API test pyramid. Width encodes recommended *volume*; height encodes cost and environmental fidelity. Contract tests sit above component tests because they assume a working provider component, yet far below integration tests in environmental cost.

flaky, and worst of all *ambiguous*, because a red E2E test tells you that *something* is wrong somewhere in five services.

Remark (The testing trophy debate). Kent C. Dodds’ “testing trophy” argues the pyramid under-weights *integration* (in our terms, component) tests: modern in-process transports and embedded fakes make whole-service tests cheap enough to carry most of the suite. Both shapes agree on the actual invariant: push verification *down* to the cheapest layer that can catch the defect, and keep the top thin. For HTTP APIs specifically, the component layer is extraordinarily powerful because ASGI transports let us exercise the entire service — routing, validation, serialization, error shaping — with zero network, which is why most listings in this chapter live there.

A useful discipline is to declare, per defect class, the *designated catcher*: validation bugs belong to unit and property tests; wiring and serialization bugs to component tests; cross-team contract drift to contract tests; infrastructure misconfiguration to integration tests; and only genuine emergent, whole-system behavior to E2E. When a bug escapes to production, the post-mortem question is never “why did no test catch it” but “which designated catcher failed, and why”.

20.2.2 Consumer-Driven Contract Testing

Schema tells you what a provider *can* serve; a consumer-driven contract tells you what consumers *actually rely on*. The gap between those two sets is where deploys go to die: a provider renames a field it believes unused, every one of its own tests stays green, and three consumer services fail in production.

Definition 20.2 (Consumer-driven contract). A **consumer-driven contract** is a machine-readable artifact, authored from the consumer’s perspective, recording the interactions the consumer depends on — request shape, expected status, and the subset of the response the consumer reads — together with the *provider states* under which those expectations hold. The provider’s build *verifies* the contract by replaying every interaction against real provider code.

Definition 20.3 (Provider state). A **provider state** is a named precondition (“a robot with id 7 exists”) that the provider must arrange before replaying an interaction. Good states are *minimal* (they arrange exactly what the interaction needs), *independent* (any order, any subset), and *semantic* (named by domain meaning, not by fixture mechanics). A state that seeds the entire universe is a smell: it hides which interactions need which data and makes failures undebuggable.

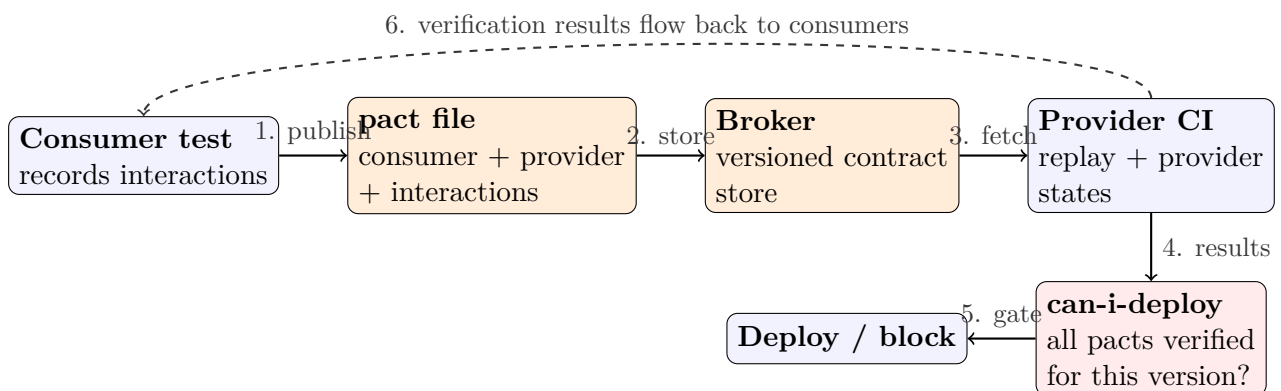


Figure 20.2: The consumer-driven contract loop (Pact model [Pac24]). The pact file is written by the *consumer*, owned by *both* teams, and executed by the *provider’s* build.

Three properties make the loop work in organizations, not just in diagrams. First, **the artifact crosses team boundaries**: the pact file is a message from the consumer team to the provider team, and the broker makes that message durable, versioned, and queryable. Second, **verification runs on the provider’s real code**, typically in-process against the real

application with real serialization — a contract verified against a mock of the provider verifies nothing. Third, **deployment is gated: can-i-deploy** answers “does version v of the provider satisfy every pact published against it?” before v ships, per environment.

Pact versus OpenAPI-diff. An OpenAPI-diff gate (connecting to Chapter 10) compares two spec revisions and flags structural incompatibilities: removed operations, removed response fields, tightened constraints. It needs no consumer participation and works for public APIs with thousands of unknown consumers — but it can only flag what is *structurally* breaking, and it must conservatively treat every published field as load-bearing. Pact-style contracts are usage-based: a field nobody reads can be renamed without tripping any pact, and a *semantically* breaking change inside a structurally identical shape (same field, new meaning) can still be caught by an explicit consumer expectation. The decision rule:

	Contract tests (Pact)	OpenAPI diff
Consumers	few, known, internal	many / unknown / public
Detects	usage-level breakage, semantic drift	structural breakage only
Setup cost	broker, provider states, both teams	spec in repo, diff tool
False safety	uncovered consumers	“compatible” spec, changed meaning

Mature platforms run both: schema diff as the coarse public-safety net, consumer contracts as the precise internal gate.

20.2.3 Property-Based Testing and API Fuzzing

Example-based tests enumerate inputs the author imagined; property-based tests enumerate inputs the author *did not*. A property-based test consists of three parts: a **strategy** that generates a universe of inputs (valid *and* invalid), an **invariant** that must hold for every generated input, and a **shrinker** that, on failure, automatically minimizes the counterexample to the smallest reproducing case. For HTTP APIs, the killer invariants are remarkably uniform:

1. **Never 5xx.** Whatever the payload — wrong types, missing fields, boundary values, pathological lengths — the server must answer 2xx or a well-formed 4xx. A 5xx is an unhandled exception, full stop.

2. **2xx validates against the published schema.** If the response does not conform to the contract you advertised, your successful responses are bugs.
3. **4xx is RFC 9457 shaped.** Every client-side error carries `type`, `title`, `status`, `detail`, `instance` (Chapter 8), so consumers can handle errors programmatically.

Shrinking is what makes failures actionable. A naive fuzzer reports “500 on payload of 4,000 random keys”; a shrinker systematically deletes keys and minimizes values while the failure reproduces, converging to something like `{"price_cents": 666}` — which is a bug report a human can act on. Hypothesis performs this automatically; our mini fuzzer implements a greedy version by hand so the mechanics are visible.

Stateful (rule-based) testing extends properties across *sequences*: generate random interleavings of create/read/update operations, maintain a local model of expected state, and assert the API agrees with the model after every step. This catches the bugs single-call tests structurally cannot: lost updates, read-your-writes violations, idempotency-key reuse bugs (Chapter 11), and leakage between resources.

Schemathesis-style fuzzing derives the strategy automatically from the OpenAPI document: every operation, every parameter, boundary and malformed values per schema constraint, executed against the live service with the same three invariants above [Ope21]. In Section 20.3 we build a stdlib miniature of this pipeline so that the generation logic — candidate values at `minLength-1`, `maxLength+1`, `minimum-1`, wrong types, `null`, missing required keys, extra keys, non-object bodies — is explicit rather than magical.

20.2.4 Test Doubles, Record–Replay, and Virtual Clocks

The word “mock” is used for five different things; imprecision here causes real design damage, so we fix the taxonomy (after Meszaros’ xUnit patterns):

Definition 20.4 (Test doubles). A **dummy** is passed to satisfy a signature and never used. A **stub** returns canned answers to calls the test triggers. A **fake** is a working implementation with shortcuts (an in-memory repository, an ASGI in-process transport). A **spy** records calls so the test can assert on them afterward. A **mock** is programmed with *expectations* up front and fails on unexpected calls.

For HTTP clients, the practical hierarchy is: prefer the *fake* (in-process ASGI transport to the real app) for anything you own; use *stubs* (`httpx.MockTransport` handlers, or `respx` routes) for third-party edges with fixed canned responses;

use **record–replay cassettes** (VCR-style: record real interactions once, replay byte-identical responses forever after) when the third party is too complex to hand-stub — with two disciplines: cassettes must be *sanitized* (no secrets, no real PII — synthetic Northwind Robotics data only) and *re-recorded on a schedule*, because a cassette is a contract claim that silently rots. WireMock-style standalone HTTP servers are the same stub idea hoisted into a separate process for polyglot test suites. The classic anti-pattern is mocking *what you do not own* and then asserting on the mock’s call list: you have then tested that your code calls your imagination of the vendor’s API, and the test stays green while the real integration rots.

Virtual clocks solve the hardest determinism problem in client testing: time. Any test that calls `time.sleep(2)` to exercise a retry backoff is slow, flaky, and unscalable. The fix is to inject the sleeper: production wires `asyncio.sleep`; tests wire a `VirtualClock` that records the requested delay and advances a counter instantly. The test then asserts on the *recorded schedule* (`[0.1, 0.2, 0.4]` for exponential backoff) — which is the actual behavioral contract — in microseconds of real time. The same injection point serves timeouts and circuit breakers: fault injection decides *what* fails, the virtual clock decides *how long* anything takes, and no assertion ever touches the wall clock.

20.2.5 Load and Performance Testing

Performance testing has one question: *where is the knee, and how much headroom separates us from it?* Everything else — tools, models, charts — serves that question.

Open versus closed workload models. In a **closed model**, a fixed population of N workers each issues a request, waits for the response, then issues the next; offered load adapts to the system’s speed, so an overloaded system sees *less* load — which hides overload. In an **open model**, requests arrive at a fixed rate λ regardless of completions; overload manifests as queueing and latency growth, which is what real traffic does to real APIs. `k6` (via arrival-rate executors) and `Locust` (via fixed user counts versus custom arrival control) embody the same distinction. The classic methodological sin is **coordinated omission**: a closed-model load generator that schedules by *completion* time silently skips the load that should have been offered during a latency spike, and reports a healthier system than reality.

Little’s Law.

Theorem 20.1 (Little’s Law). For a stable system, the mean number of requests in residence L , the arrival rate λ , and the mean residence time W satisfy $L = \lambda W$.

Derivation. Observe the system over a long interval $[0, T]$ in which N requests complete and the number in residence $L(t)$ returns to its initial value. Each request i contributes exactly its residence time W_i to the area under the curve $L(t)$:

$$\int_0^T L(t) dt = \sum_{i=1}^N W_i. \quad (20.1)$$

Divide both sides by T :

$$L = \frac{1}{T} \int_0^T L(t) dt = \frac{N}{T} \cdot \frac{1}{N} \sum_{i=1}^N W_i = \lambda \cdot W, \quad (20.2)$$

since $\lambda = N/T$ is the throughput (equal to the arrival rate in a stable system) and $W = \frac{1}{N} \sum_i W_i$ is the mean residence time. $\square$

Little’s Law is the Swiss army knife of capacity reasoning. If your API serves $\lambda = 500$ req/s at a mean latency of $W = 40$ ms, then on average $L = 500 \times 0.04 = 20$ requests are in flight — your worker pool, connection pool, and async concurrency limits must all exceed 20 with headroom, or queueing begins. Conversely, in a closed-model test with N workers you can *predict* throughput: $\lambda = N/W$. Our harness performs exactly this cross-check numerically.

Saturation and the knee. Let S be mean service time and c the effective concurrency (workers, event-loop capacity, pool size). Utilization is $\rho = \lambda S/c$, and queueing theory gives the qualitative shape $W \approx S/(1 - \rho)$: latency is flat while $\rho \ll 1$, then explodes hyperbolically as $\rho \rightarrow 1$. The **knee** is the offered rate at which the curve turns — operationally, the first load level where tail latency degrades superlinearly (we flag $p99 > 10\times$ baseline) or errors appear. Sustainable capacity is provisioned *below* the knee, typically at 60–70% of it, because arrival bursts and deploys eat the remainder [Bey+16; Nyg18].

The four load test shapes.

Shape	Question answered	Duration
Load	Does the system meet SLOs at <i>expected</i> peak?	15–60 min
Stress	Where is the knee? What fails first past it?	ramp to failure
Soak	Do leaks, fragmentation, or log growth degrade us over time?	hours–days
Spike	Does the system survive a $10\times$ burst (cache cold, autoscaler lag)?	seconds–minutes

Soak tests catch what ramps cannot: memory leaks, file-descriptor exhaustion, token-cache stampede alignment, and disk growth (Chapter 19 covers the telemetry). Spike tests expose *elasticity* defects — the load balancer scales in minutes, the traffic arrives in seconds.

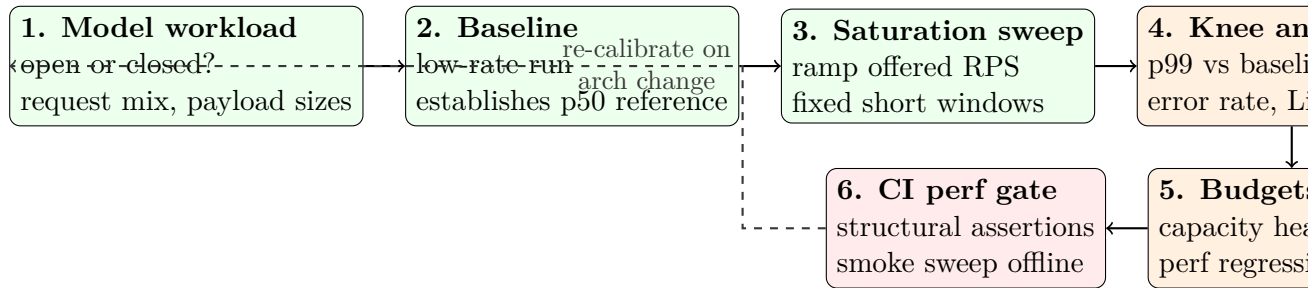


Figure 20.3: Load-test methodology. The CI gate runs a short structural smoke; deep sweeps are scheduled offline and re-baselined on architectural change.

20.2.6 Chaos Engineering Basics

Chaos engineering is the experimental validation of resilience claims. Chapter 16 taught you to *build* retries, timeouts, and breakers; chaos asks: do they actually work, in combination, under realistic failure?

Definition 20.5 (Steady-state hypothesis and blast radius). The **steady state** is a measurable, behavioral definition of “the system is fine” — e.g., $p99 < 250$ ms and error rate $< 0.1\%$ over five minutes — never an internal one (“CPU looks low”). A **steady-state hypothesis** asserts the system remains in steady state under a specified fault. The **blast radius** is the maximal scope the experiment can affect; experiments start at the smallest blast radius (one in-process middleware in a test) and widen only as confidence grows.

Fault class	Mechanism	Reveals
Latency	inject delay before/around dependency	missing timeouts, thread/connection starvation
Error (5xx)	synthesize error responses	retry storms, missing backoff/jitter, error-mapping bugs
Abort	drop/reset the connection with no response	retry-on-exception paths, hung-read behavior
Partition	make a dependency unreachable entirely	failover, cache fallback, graceful degradation
Resource exhaustion	cap CPU/memory/threads/FD	queue growth, priority inversion, GC death spirals

Distributed systems fail in partial, correlated, and Byzantine ways [Kle17]; the taxonomy above is ordered roughly by increasing subtlety. The critical engineering move for *testing* is determinism: our fault-injection middleware is scripted by request index, not by random dice, so a failing resilience test reproduces byte-for-byte. Randomized chaos is for staging environments and game days; scripted chaos is for CI.

20.2.7 CI Quality Gates as Code

A quality gate is a function from change to evidence, executed on every pull request, whose failure blocks merge. The design rules that keep gates healthy:

1. **Hermetic.** No network, no cloud, no shared environment. Everything in this chapter runs in-process; a gate that needs staging is a gate that will be disabled.
2. **Fast.** Minutes, not hours. Deep load sweeps and long soaks are scheduled jobs that update budgets; the PR gate runs structural smoke.
3. **Self-checking.** A gate that cannot fail proves nothing. Our gate runs each check against a deliberately broken variant and requires the check to catch it — mutation testing applied to the pipeline itself.
4. **Deterministic.** Derandomized property seeds, scripted faults, structural perf assertions. A flaky gate trains the team to click “re-run” without reading, which is worse than no gate.

5. **Legible.** The report names the layer, the interaction, and the mismatch: `$.sku: missing`. “1 test failed” is not a diagnosis.

Typical API gate battery: unit + property suites with a coverage floor (floors guard against suite deletion, not for their own sake — an 80% floor with the invariants of Section 20.2.3 beats a 95% floor of example tests); contract verification against every published pact plus an OpenAPI diff against the previous released spec; a fuzz smoke (a bounded case budget, not an open-ended campaign); a perf smoke that asserts completion and zero 5xx rather than latency (CI machines are noisy neighbors; latency budgets belong to scheduled runs on quiet hardware); and flake management: quarantine list with SLA, automatic bisection of derandomized failures, and a hard rule that a test flaky twice in a week is either fixed or deleted. Delivery-performance research consistently finds that fast, trusted automated gates correlate with both throughput and stability — the gate is a delivery feature, not a tax [Bey+16].

20.3 Production-Ready Code Implementation

All listings live under `code\ch19\` and share one app under test: the Northwind Robotics robot registry. They are fully offline (in-process ASGI), deterministic (derandomized generators, scripted faults, virtual clocks), and use synthetic data only. Install dependencies once:

```
1 cd code\ch19
2 pip install -r requirements.txt
```

20.3.1 Listing 19.1 — The App Under Test with Deliberate-Defect Knobs

The registry (Listing 19.1) is a plain Chapter 6 FastAPI service with two teaching additions. First, every error is RFC 9457 problem-shaped (Chapter 8), because later listings *assert* on that shape. Second, `create_app` accepts two keyword-only defect knobs: `breaking=True` renames `sku` to `part_number` in responses (a contract break), and `bug_at_price=N` returns an unstructured 500 when `price_cents == N` (a planted bug for the fuzzer). The module also provides `SyncASGITransport`: since `httpx` 0.28 the bundled `ASGITransport` is async-only [Enc24], so we wrap it in a five-line shim that runs the app on one dedicated event loop and re-wraps the body as a sync byte stream.

```

1  """Northwind_Robotics_robot_registry: the single API
   under test in Chapter 19.
2
3  'create_app' ships with two deliberate-defect knobs
   so the lab can *break*
4  the API on demand:
5
6  * 'breaking=True' renames the 'sku' field to '
   part_number' in every
7  response body -- the classic contract-breaking change
   a provider team
8  ships believing it is "internal".
9  * 'bug_at_price=N' makes 'POST/robots' return an
   unhandled 500 (not
10 RFC 9457 shaped) whenever 'price_cents==N' -- a
   planted bug for the
11 fuzzer to find and shrink.
12
13 Offline-first: every consumer of this module talks to
   the app in-process
14 through 'httpx.ASGITransport'; no socket is ever
   opened.
15 """
16 from __future__ import annotations
17
18 import asyncio
19 from typing import Any
20
21 import httpx
22 from fastapi import FastAPI, HTTPException, Request
23 from fastapi.exceptions import RequestValidationError
24 from fastapi.responses import JSONResponse
25 from pydantic import BaseModel, Field
26
27 PROBLEM_BASE = "https://northwind-robotics.example/
   problems"
28
29
30 class SyncASGITransport(httpx.BaseTransport):
31     """Sync shim over 'httpx.ASGITransport' (async-
   only since httpx 0.28).
32
33     Runs the ASGI app on one dedicated event loop, in-
   process: no sockets,
34     no network, fully offline.

```

```

35 """
36
37 def __init__(self, app: Any) -> None:
38     self._inner = httpx.ASGITransport(app=app)
39     self._loop = asyncio.new_event_loop()
40
41 def handle_request(self, request: httpx.Request) ->
42     httpx.Response:
43     async_resp = self._loop.run_until_complete(
44         self._inner.handle_async_request(request)
45     )
46     content = self._loop.run_until_complete(
47         async_resp.aread()
48     )
49     # Rebuild with a byte body so the sync client
50     # gets a SyncByteStream.
51     return httpx.Response(
52         status_code=async_resp.status_code,
53         headers=async_resp.headers,
54         content=content,
55     )
56
57 def make_sync_client(app: Any, base_url: str = "http://
58     testserver") -> httpx.Client:
59     """Build a synchronous httpx client wired straight
60     into the ASGI app."""
61     return httpx.Client(transport=SyncASGITransport(app
62         ), base_url=base_url)
63
64 class RobotCreate(BaseModel):
65     """Payload accepted by 'POST /robots'."""
66
67     sku: str = Field(min_length=3, max_length=32)
68     name: str = Field(min_length=1, max_length=80)
69     price_cents: int = Field(ge=0, le=10_000_000)
70
71 class Robot(RobotCreate):
72     """Stored representation: everything in '
73     RobotCreate' plus a server id."""
74
75     id: int
76
77 def _problem(status: int, title: str, detail: str,

```

```
instance: str) -> JSONResponse:
74     """Build an RFC 9457 problem-details response."""
75     slug = title.lower().replace("_", "-")
76     return JSONResponse(
77         status_code=status,
78         media_type="application/problem+json",
79         content={
80             "type": f"{PROBLEM_BASE}/{slug}",
81             "title": title,
82             "status": status,
83             "detail": detail,
84             "instance": instance,
85         },
86     )
87
88
89 def create_app(*, breaking: bool = False, bug_at_price:
90     int | None = None) -> FastAPI:
91     """Application factory; defect knobs are keyword-
92     only and default to off."""
93     app = FastAPI(title="Northwind Robotics -- Robot
94         Registry")
95     app.state.db = {} # type: dict[int, dict[str, Any
96         ]]
97     app.state.next_id = 1
98
99     @app.exception_handler(RequestValidationError)
100     async def on_validation_error(
101         request: Request, exc: RequestValidationError
102     ) -> JSONResponse:
103         errors = exc.errors()
104         first = errors[0] if errors else {}
105         loc = ".".join(str(part) for part in first.get(
106             "loc", []))
107         detail = f"{loc}: {first.get('msg', 'invalid
108             request')}"
109         return _problem(422, "Validation Error", detail
110             , request.url.path)
111
112     @app.exception_handler(HTTPException)
113     async def on_http_error(request: Request, exc:
114         HTTPException) -> JSONResponse:
115         title = "Not Found" if exc.status_code == 404
116             else "HTTP Error"
117         return _problem(exc.status_code, title, str(exc
118             .detail), request.url.path)
```



```

109
110     def present(record: dict[str, Any]) -> dict[str,
111         Any]:
112         """Serialize a stored record, applying the
113             deliberate break if asked."""
114         body = dict(record)
115         if breaking:
116             body["part_number"] = body.pop("sku")
117         return body
118
119 @app.post("/robots", status_code=201)
120 async def create_robot(payload: RobotCreate) -> Any
121 :
122     if bug_at_price is not None and payload.
123         price_cents == bug_at_price:
124         # Planted bug: an unstructured, non-RFC
125             -9457 server error.
126         return JSONResponse(status_code=500,
127             content={"oops": "price_demon"})
128     robot_id = app.state.next_id
129     app.state.next_id += 1
130     record = {"id": robot_id, **payload.model_dump
131         ()}
132     app.state.db[robot_id] = record
133     return present(record)
134
135 @app.get("/robots/{robot_id}")
136 async def get_robot(robot_id: int) -> dict[str, Any
137     ]:
138     if robot_id not in app.state.db:
139         raise HTTPException(status_code=404, detail
140             =f"robot_{robot_id}_not_found")
141     return present(app.state.db[robot_id])
142
143 return app

```

20.3.2 Listing 19.2 — A Mini Pact: Recorder, Pact File, Provider Verifier

Listing 19.2 implements the full consumer-driven loop of Figure 20.2 in about 200 lines. The consumer side (PactRecorder) executes each interaction, fails the consumer test immediately if the API already violates the expectation, and freezes the interaction — method, path, request body, expected status, and a *body subset* (what the consumer actually reads) — into JSON. The

provider side (`ProviderVerifier`) builds a *fresh* application per interaction, applies the named provider state, replays the request against real code in-process, and reports recursive subset mismatches with JSONPath-style locations. `PROVIDER_STATES` is the provider team’s half of the bargain: the implementation of the consumer’s named preconditions.

```

1  """A mini consumer-driven contract testing framework,
   Pact-inspired.
2
3  The moving parts mirror the real Pact loop:
4
5  1. Consumer side: class: 'PactRecorder' wraps an '
   httpx.Client'; every
6  interaction the consumer relies on is executed,
   validated on the spot,
7  and frozen into a JSON *pact file*.
8  2. The pact file is the contract artifact. In
   production it travels through
9  a broker; here it is a plain JSON file under 'pacts
   \'.
10 3. Provider side: class: 'ProviderVerifier' replays
   each interaction against
11 a *fresh* provider application, applying the named *
   provider state*
12 first, and checks status + a recursive body subset
   match.
13
14 Everything is offline: provider replay runs through '
   httpx.ASGITransport'.
15 """
16 from __future__ import annotations
17
18 import json
19 from dataclasses import dataclass, field
20 from pathlib import Path
21 from typing import Any, Callable
22
23 import httpx
24
25 from robot_registry import create_app, make_sync_client
26
27 StateHandler = Callable[[Any], None] # receives the
   provider FastAPI app
28 AppFactory = Callable[[], Any]
29
30

```

```

31 @dataclass
32 class Interaction:
33     """One request/response pair the consumer depends
34         on."""
35
36     description: str
37     provider_state: str | None
38     method: str
39     path: str
40     request_body: dict[str, Any] | None
41     expected_status: int
42     expected_body_subset: dict[str, Any]
43
44     def to_json(self) -> dict[str, Any]:
45         return {
46             "description": self.description,
47             "providerState": self.provider_state,
48             "request": {"method": self.method, "path":
49                 self.path, "body": self.request_body},
50             "response": {
51                 "status": self.expected_status,
52                 "bodySubset": self.expected_body_subset
53             },
54         }
55
56     @classmethod
57     def from_json(cls, raw: dict[str, Any]) -> "
58         Interaction":
59         return cls(
60             description=raw["description"],
61             provider_state=raw.get("providerState"),
62             method=raw["request"]["method"],
63             path=raw["request"]["path"],
64             request_body=raw["request"].get("body"),
65             expected_status=raw["response"]["status"],
66             expected_body_subset=raw["response"]["
67                 bodySubset"],
68         )
69
70 def subset_mismatches(expected: Any, actual: Any, path:
71     str = "$") -> list[str]:
72     """Recursive subset match: every expected leaf must
73         exist and be equal."""
74     if isinstance(expected, dict):

```

```

70         if not isinstance(actual, dict):
71             return [f"{path}:_expected_object,_got_{
72                 type(actual).__name__}"]
73         problems: list[str] = []
74         for key, sub in expected.items():
75             if key not in actual:
76                 problems.append(f"{path}.{key}:_missing
77                     _(expected_{sub!r})")
78             else:
79                 problems.extend(subset_mismatches(sub,
80                     actual[key], f"{path}.{key}"))
81         return problems
82     if isinstance(expected, list):
83         if not isinstance(actual, list) or len(actual)
84             != len(expected):
85             return [f"{path}:_expected_list_of_length_{
86                 len(expected)}"]
87         problems = []
88         for i, sub in enumerate(expected):
89             problems.extend(subset_mismatches(sub,
90                 actual[i], f"{path}[{i}]"))
91         return problems
92     if expected != actual:
93         return [f"{path}:_expected_{expected!r},_got_{
94             actual!r}"]
95     return []
96
97 class PactRecorder:
98     """Consumer_side:_exercise_the_API_and_freeze_the_
99     interactions_relied_on."""
100
101     def __init__(self, client: httpx.Client, *,
102         consumer: str, provider: str) -> None:
103         self._client = client
104         self.consumer = consumer
105         self.provider = provider
106         self.interactions: list[Interaction] = []
107
108     def record(
109         self,
110         description: str,
111         method: str,
112         path: str,
113         *,
114         provider_state: str | None = None,

```

```

107         json_body: dict[str, Any] | None = None,
108         expected_status: int,
109         expect_subset: dict[str, Any],
110     ) -> httpx.Response:
111         """Execute one interaction; fail the consumer if
112            the API already violates the expectation, then
113            freeze it."""
114         resp = self._client.request(method, path, json=
115             json_body)
116         if resp.status_code != expected_status:
117             raise AssertionError(
118                 f"[{description}] expected {
119                     expected_status}, got "
120                 f"{resp.status_code}: {resp.text}"
121             )
122         problems = subset_mismatches(expect_subset,
123             resp.json())
124         if problems:
125             raise AssertionError(f"[{description}]
126                 response mismatches: {problems}")
127         self.interactions.append(
128             Interaction(
129                 description=description,
130                 provider_state=provider_state,
131                 method=method,
132                 path=path,
133                 request_body=json_body,
134                 expected_status=expected_status,
135                 expected_body_subset=expect_subset,
136             )
137         )
138         return resp
139
140     def write_pact(self, path: Path) -> Path:
141         path.parent.mkdir(parents=True, exist_ok=True)
142         document = {
143             "consumer": {"name": self.consumer},
144             "provider": {"name": self.provider},
145             "interactions": [i.to_json() for i in self.
146                 interactions],
147         }
148         path.write_text(json.dumps(document, indent=2)
149             + "\n", encoding="utf-8")
150         return path

```

```

145
146 @dataclass
147 class VerificationReport:
148     """Outcome of replaying every pact interaction
149     against the provider."""
150
151     results: list[tuple[str, bool, list[str]]] = field(
152         default_factory=list)
153
154     @property
155     def ok(self) -> bool:
156         return all(passed for _, passed, _ in self.
157             results)
158
159     def failures(self) -> list[str]:
160         return [f"{desc}: {problems}" for desc, passed,
161             problems in self.results if not passed]
162
163     def render(self) -> str:
164         lines = []
165         for desc, passed, problems in self.results:
166             mark = "PASS" if passed else "FAIL"
167             lines.append(f" [{mark}] {desc}")
168             for problem in problems:
169                 lines.append(f" {problem}")
170         return "\n".join(lines)
171
172 class ProviderVerifier:
173     """Provider side: replay a pact file against a
174     fresh application instance."""
175
176     def __init__(
177         self,
178         app_factory: AppFactory,
179         pact_path: Path,
180         states: dict[str, StateHandler],
181     ) -> None:
182         self._app_factory = app_factory
183         self._pact_path = pact_path
184         self._states = states
185
186     def verify(self) -> VerificationReport:
187         document = json.loads(self._pact_path.read_text(
188             encoding="utf-8"))
189         report = VerificationReport()

```

```

185     for raw in document["interactions"]:
186         interaction = Interaction.from_json(raw)
187         problems: list[str] = []
188         app = self._app_factory()
189         if interaction.provider_state is not None:
190             handler = self._states.get(interaction.
191                                     provider_state)
192             if handler is None:
193                 problems.append(f"unknown_provider_
194                               state: {interaction.
195                               provider_state!r}")
196             else:
197                 handler(app)
198         if not problems:
199             client = make_sync_client(app, base_url
200                                     = "http://provider.test")
201             resp = client.request(
202                 interaction.method,
203                 interaction.path,
204                 json=interaction.request_body,
205             )
206             if resp.status_code != interaction.
207                 expected_status:
208                 problems.append(
209                     f"status: {expected} {interaction
210                     .expected_status}, "
211                     f"got {resp.status_code}"
212                 )
213             if interaction.expected_body_subset:
214                 try:
215                     body = resp.json()
216                 except json.JSONDecodeError:
217                     problems.append("response_body_
218                                     is not JSON")
219                 else:
220                     problems.extend(
221                         subset_mismatches(
222                             interaction.
223                                 expected_body_subset,
224                                 body
225                         ))
226             report.results.append((interaction.
227                                   description, not problems, problems))
228     return report

```

```

220 #
221 # Demo consumer: the Northwind Robotics fulfillment
222 # service.
223
224 def _seed_robot_7(app: Any) -> None:
225     """Provider state: 'a robot with id 7 exists'."""
226     app.state.db[7] = {
227         "id": 7,
228         "sku": "NWR-ARM-6",
229         "name": "A6 Manipulator Arm",
230         "price_cents": 129900,
231     }
232
233 PROVIDER_STATES: dict[str, StateHandler] = {
234     "a robot with id 7 exists": _seed_robot_7,
235 }
236
237
238
239 def record_demo_pact(pact_dir: Path) -> Path:
240     """Run the fulfillment service's expectations and
241     freeze them as a pact."""
242     app = create_app()
243     _seed_robot_7(app) # consumer designs the 'GET'
244     # interaction against known data
245     client = make_sync_client(app, base_url="http://
246     consumer.test")
247     recorder = PactRecorder(
248         client, consumer="fulfillment-service",
249         provider="robot-registry"
250     )
251     recorder.record(
252         "create a robot",
253         "POST",
254         "/robots",
255         json_body={"sku": "NWR-PLT-2", "name": "Pallet
256         Mover 2", "price_cents": 540000},
257         expected_status=201,
258         expect_subset={"sku": "NWR-PLT-2", "name": "
259         Pallet Mover 2", "price_cents": 540000},
260     )

```



```

255 recorder.record(
256     "fetch_robot_7",
257     "GET",
258     "/robots/7",
259     provider_state="a_robot_with_id_7_exists",
260     expected_status=200,
261     expect_subset={"id": 7, "sku": "NWR-ARM-6", "
        name": "A6_Manipulator_Arm"},
262 )
263 return recorder.write_pact(pact_dir / "fulfillment-
    service-robot-registry.json")

```

The pact file itself (excerpt) is intentionally boring — boring is what crosses team boundaries well:

```

1 {
2   "consumer": {"name": "fulfillment-service"},
3   "provider": {"name": "robot-registry"},
4   "interactions": [
5     {"description": "fetch robot 7",
6      "providerState": "a robot with id 7 exists",
7      "request": {"method": "GET", "path": "/robots/7",
8                  "body": null},
9      "response": {"status": 200,
10                   "bodySubset": {"id": 7, "sku": "NWR-
11                                   ARM-6"}}}
12 ]
13 }

```

20.3.3 Listing 19.3 — Contract Tests, Including the Breaking Change

Four tests prove the framework works *and has teeth*: the pact file is well-formed; the healthy provider satisfies it; renaming `sku` to `part_number` makes verification fail with a message naming the missing field; and an unknown provider state fails loudly rather than silently passing.

```

1 """Contract tests for the mini pact framework.
2
3 Run: pytest test_pact_mini.py -q
4 """
5 from __future__ import annotations
6
7 import json
8 from pathlib import Path
9

```

```

10 from pact_mini import PROVIDER_STATES, ProviderVerifier
    , record_demo_pact
11 from robot_registry import create_app
12
13
14 def test_recorded_pact_file_is_wellformed(tmp_path:
    Path) -> None:
15     pact = record_demo_pact(tmp_path)
16     document = json.loads(pact.read_text(encoding="utf
        -8"))
17     assert document["consumer"]["name"] == "fulfillment
        -service"
18     assert document["provider"]["name"] == "robot-
        registry"
19     assert [i["description"] for i in document["
        interactions"]] == [
20         "create_a_robot",
21         "fetch_robot_7",
22     ]
23     assert document["interactions"][1]["providerState"]
        == "a_robot_with_id_7_exists"
24
25
26 def test_provider_satisfies_the_pact(tmp_path: Path) ->
    None:
27     pact = record_demo_pact(tmp_path)
28     report = ProviderVerifier(create_app, pact,
        PROVIDER_STATES).verify()
29     print("\n" + report.render())
30     assert report.ok, report.failures()
31
32
33 def test_breaking_field_rename_is_caught(tmp_path: Path
    ) -> None:
34     """The provider renames 'sku' -> 'part_number':
        verification must fail."""
35     pact = record_demo_pact(tmp_path)
36     report = ProviderVerifier(
37         lambda: create_app(breaking=True), pact,
            PROVIDER_STATES
38     ).verify()
39     print("\n" + report.render())
40     assert not report.ok
41     rendered = report.render()
42     assert "sku" in rendered # the missing field is
        named in the report

```

```

43
44
45 def test_unknown_provider_state_fails_loudly(tmp_path:
    Path) -> None:
46     pact = record_demo_pact(tmp_path)
47     document = json.loads(pact.read_text(encoding="utf
        -8"))
48     document["interactions"][1]["providerState"] = "a_
        robot_with_id_99_exists"
49     pact.write_text(json.dumps(document), encoding="utf
        -8")
50     report = ProviderVerifier(create_app, pact,
        PROVIDER_STATES).verify()
51     assert not report.ok
52     assert any("unknown_provider_state" in f for f in
        report.failures())

```

20.3.4 Listing 19.4 — Property-Based Suite with Hypothesis

Listing 19.4 encodes the three universal API invariants from Section 20.2.3. The payload strategy is deliberately hostile: wrong types, `None`, out-of-range integers, over- and under-length strings. Whatever Hypothesis generates, the service must never 5xx, must return schema-valid JSON on 201, and must return an RFC 9457 body on 422. `derandomize=True` replays the identical examples on every run — determinism is a feature, not a limitation, in CI. The rule-based `RobotRegistryMachine` then generates random create/read *sequences* and checks the API against a local model after every step, catching sequence-dependent bugs single-call properties cannot see.

```

1  """Property-based tests for the robot registry, plus a
    stateful rule suite.
2
3  Invariants under test:
4
5  * 'POST /robots' never returns 5xx, whatever the
    payload.
6  * A 201 response always validates against the published
    response schema.
7  * A 4xx response is always RFC 9457 problem-shaped.
8  * 'GET /robots/{id}' never returns 5xx for arbitrary
    integer ids.
9  * Rule-based sequences (create->read) stay consistent
    with a local model.
10

```

```

11 Deterministic: ``derandomize=True`` replays the same
    examples every run and
12 no Hypothesis example database is consulted.
13
14 Run: pytest test_pbt_robots.py -q
15 """
16 from __future__ import annotations
17
18 from typing import Any
19
20 import jsonschema
21 from hypothesis import given, settings, strategies as
    st
22 from hypothesis.stateful import (
23     Bundle,
24     RuleBasedStateMachine,
25     invariant,
26     rule,
27 )
28
29 from robot_registry import create_app, make_sync_client
30
31 CLIENT = make_sync_client(create_app(), base_url="http
    ://pbt.test")
32
33 ROBOT_SCHEMA: dict[str, Any] = {
34     "type": "object",
35     "required": ["id", "sku", "name", "price_cents"],
36     "properties": {
37         "id": {"type": "integer", "minimum": 1},
38         "sku": {"type": "string", "minLength": 3, "
            maxLength": 32},
39         "name": {"type": "string", "minLength": 1, "
            maxLength": 80},
40         "price_cents": {"type": "integer", "minimum":
            0, "maximum": 10_000_000},
41     },
42     "additionalProperties": False,
43 }
44
45 PROBLEM_MEMBERS = {"type", "title", "status", "detail",
    "instance"}
46
47 # Deliberately hostile: wrong types, None, out-of-range
    ints, over/under strings.
48 HOSTILE_PAYLOADS = st.fixed_dictionaries(

```

```

49     {
50         "sku": st.one_of(st.text(min_size=0, max_size
51                             =40), st.integers(), st.none()),
52         "name": st.one_of(st.text(max_size=100), st.
53                             booleans(), st.none()),
54         "price_cents": st.one_of(
55             st.integers(min_value=-1_000_000, max_value
56                             =20_000_000),
57             st.text(max_size=10),
58             st.none(),
59         ),
60     }
61 )
62
63 VALID_PAYLOADS = st.fixed_dictionaries(
64     {
65         "sku": st.text(
66             alphabet=st.characters(categories=("Lu", "
67                                     Ll", "Nd"), include_characters="-"),
68             min_size=3,
69             max_size=32,
70         ),
71         "name": st.text(
72             alphabet=st.characters(categories=("Lu", "
73                                     Ll", "Nd", "Zs"), include_characters="-"
74                                     ),
75             min_size=1,
76             max_size=80,
77         ),
78         "price_cents": st.integers(min_value=0,
79                                     max_value=10_000_000),
80     }
81 )
82
83 DETERMINISTIC = dict(derandomize=True, database=None,
84                         deadline=None)
85
86 def assert_problem_shape(body: Any, status: int) ->
87     None:
88     assert isinstance(body, dict), f"4xx body must be a
89         JSON object, got {body!r}"
90     missing = PROBLEM_MEMBERS - body.keys()
91     assert not missing, f"problem details missing
92         members: {missing}"
93     assert body["status"] == status

```

```

84
85
86 @given(payload=HOSTILE_PAYLOADS)
87 @settings(max_examples=60, **DETERMINISTIC)
88 def
89     test_create_robot_never_5xx_and_always_schema_conformant
90     (payload: dict) -> None:
91         resp = CLIENT.post("/robots", json=payload)
92         assert resp.status_code < 500, f"5xx_for_payload_{
93             payload!r}:_{resp.text}"
94         if resp.status_code == 201:
95             jsonschema.validate(resp.json(), ROBOT_SCHEMA)
96         else:
97             assert resp.status_code == 422, f"unexpected_
98                 status_{resp.status_code}"
99             assert resp.headers["content-type"].startswith(
100                 "application/problem+json")
101             assert_problem_shape(resp.json(), 422)
102
103 @given(robot_id=st.integers(min_value=-1_000, max_value
104                             =1_000))
105 @settings(max_examples=50, **DETERMINISTIC)
106 def test_get_robot_never_5xx(robot_id: int) -> None:
107     resp = CLIENT.get(f"/robots/{robot_id}")
108     assert resp.status_code in (200, 404), f"unexpected
109         _status_{resp.status_code}"
110     if resp.status_code == 200:
111         jsonschema.validate(resp.json(), ROBOT_SCHEMA)
112     else:
113         assert_problem_shape(resp.json(), 404)
114
115 class RobotRegistryMachine(RuleBasedStateMachine):
116     """Rule-based_stateful_test:_random_create/read_
117         sequences_checked_against
118         a_local_model_of_the_registry."""
119
120     created_ids: Bundle[int] = Bundle("created_ids")
121
122     def __init__(self) -> None:
123         super().__init__()
124         self.client = make_sync_client(create_app(),
125                                         base_url="http://pbt-stateful.test")
126         self.model: dict[int, dict[str, Any]] = {}

```

```

121 @rule(target=created_ids, payload=VALID_PAYLOADS)
122 def create_robot(self, payload: dict[str, Any]) ->
123     int:
124         resp = self.client.post("/robots", json=payload
125                                 )
126         assert resp.status_code == 201, resp.text
127         body = resp.json()
128         jsonschema.validate(body, ROBOT_SCHEMA)
129         self.model[body["id"]] = body
130         return body["id"]
131
132 @rule(robot_id=created_ids)
133 def read_back_matches_model(self, robot_id: int) ->
134     None:
135         resp = self.client.get(f"/robots/{robot_id}")
136         assert resp.status_code == 200
137         assert resp.json() == self.model[robot_id]
138
139 @invariant()
140 def every_created_robot_is_readable(self) -> None:
141     for robot_id in self.model:
142         resp = self.client.get(f"/robots/{robot_id}
143                                 ")
144         assert resp.status_code == 200, f"robot_{
145             robot_id}_vanished"
146
147 RobotRegistryMachine.TestCase.settings = settings(
148     max_examples=15, stateful_step_count=20, **
149     DETERMINISTIC
150 )
151 TestRobotRegistry = RobotRegistryMachine.TestCase

```

20.3.5 Listing 19.5 — A Stdlib OpenAPI Fuzzer with Shrinking

Where Hypothesis owns the strategy machinery, Listing 19.5 exposes it: from a request-body schema (a stand-in for an excerpt of the published OpenAPI document [Ope21]) it derives boundary and malformed candidates per property — `minLength-1`, `maxLength+1`, `minimum-1`, wrong types, `None`, pathological lengths — plus missing-required, extra-property, and non-object-body cases. Every response is classified: 5xx is always a failure, 4xx must be problem-shaped, 2xx must validate. The greedy `shrink` first tries dropping keys, then minimizes values to the smallest schema-valid

stand-ins — the second move matters, because dropping a *required* key flips a 500 into a 422 and stalls a naive shrinker.

```
1  """Stdlib-only OpenAPI-driven fuzzer: Schemathesis in
   miniature.
2
3  Given the request-body schema of 'POST /robots' (a
   stand-in for an excerpt
4  of the published OpenAPI document), generate boundary
   and malformed
5  payloads, fire them at the app in-process, and classify
   every response:
6
7  * 2xx -> must validate against the response schema;
8  * 4xx -> must be RFC 9457 problem-shaped;
9  * 5xx -> always a failure.
10
11 Includes a greedy shrinker: when a failing payload is
   found, strip it down
12 to a minimal reproducer so the report is actionable.
13
14 Run the demo: python fuzz_openapi_mini.py
15 """
16 from __future__ import annotations
17
18 import copy
19 import json
20 from dataclasses import dataclass
21 from typing import Any, Callable
22
23 import httpx
24 import jsonschema
25
26 from robot_registry import create_app, make_sync_client
27
28 # Excerpt of the published OpenAPI 3.1 document: POST /
   robots request body.
29 REQUEST_SCHEMA: dict[str, Any] = {
30     "type": "object",
31     "required": ["sku", "name", "price_cents"],
32     "properties": {
33         "sku": {"type": "string", "minLength": 3, "
34                 maxLength": 32},
35         "name": {"type": "string", "minLength": 1, "
36                  maxLength": 80},
37         "price_cents": {"type": "integer", "minimum":
```



```

36         0, "maximum": 10_000_000},
37     },
38     "additionalProperties": False,
39 }
40 RESPONSE_SCHEMA: dict[str, Any] = {
41     "type": "object",
42     "required": ["id", "sku", "name", "price_cents"],
43     "properties": {
44         "id": {"type": "integer", "minimum": 1},
45         "sku": {"type": "string", "minLength": 3, "
46             maxLength": 32},
47         "name": {"type": "string", "minLength": 1, "
48             maxLength": 80},
49         "price_cents": {"type": "integer", "minimum":
50             0, "maximum": 10_000_000},
51     },
52     "additionalProperties": False,
53 }
54
55 PROBLEM_MEMBERS = {"type", "title", "status", "detail",
56     "instance"}
57 VALID_BASE: dict[str, Any] = {"sku": "NWR-ARM-6", "name
58     ": "A6□Manipulator□Arm", "price_cents": 129900}
59
60 def boundary_candidates(prop_schema: dict[str, Any]) ->
61     list[Any]:
62     """Boundary□and□malformed□candidates□for□one□
63     property□schema."""
64     ptype = prop_schema.get("type")
65     if ptype == "string":
66         lo = prop_schema.get("minLength", 0)
67         hi = prop_schema.get("maxLength", 16)
68         candidates = ["A" * lo, "A" * hi, "", None,
69             123, True]
70         if lo > 0:
71             candidates.append("A" * (lo - 1))
72             candidates.append("A" * (hi + 1))
73             candidates.append("Z" * 512) # pathological
74             length
75         return candidates
76     if ptype == "integer":
77         lo = prop_schema.get("minimum", 0)
78         hi = prop_schema.get("maximum", 100)
79         return [lo, hi, lo - 1, hi + 1, 0, -1, 666,

```

```

72         2**31, "10", None, True, 1.5]
73     return [None]
74
75 def fuzz_cases(schema: dict[str, Any]) -> list[tuple[
76     str, Any]]:
77     """All (label, payload) cases derived from the
78     request schema."""
79     cases: list[tuple[str, Any]] = []
80     properties = schema["properties"]
81     for name, prop_schema in properties.items():
82         for value in boundary_candidates(prop_schema):
83             payload = dict(VALID_BASE, **{name: value})
84             cases.append((f"{name}={value!r}"[:60],
85                           payload))
86     for name in schema.get("required", []):
87         payload = {k: v for k, v in VALID_BASE.items()
88                     if k != name}
89         cases.append((f"missing_required_{name!r}",
90                       payload))
91     cases.append(("extra_property", dict(VALID_BASE,
92                                           legacy_field="x")))
93     cases.append(("empty_object", {}))
94     cases.append(("body_is_a_list", [1, 2, 3]))
95     cases.append(("body_is_a_string", "not-an-object"))
96     cases.append(("body_is_null", None))
97     return cases
98
99 @dataclass
100 class FuzzFailure:
101     label: str
102     payload: Any
103     status: int
104     reason: str
105
106 def classify(label: str, payload: Any, resp: httpx.
107 Response) -> FuzzFailure | None:
108     if resp.status_code >= 500:
109         return FuzzFailure(label, payload, resp.
110                             status_code, "server_error(5xx)")
111     if resp.status_code >= 400:
112         body = resp.json()
113         missing = PROBLEM_MEMBERS - set(body)
114         if missing or body.get("status") != resp.

```

```

109         status_code:
110             return FuzzFailure(
111                 label, payload, resp.status_code, f"4xx
112                 ␣not␣RFC␣9457␣shaped:␣{body!r}"
113             )
114         return None
115     try:
116         jsonschema.validate(resp.json(),
117                             RESPONSE_SCHEMA)
118     except jsonschema.ValidationError as exc:
119         return FuzzFailure(label, payload, resp.
120                             status_code, f"2xx␣violates␣schema:␣{exc.
121                             message}")
122     return None
123
124 def run_fuzz(
125     client: httpx.Client, cases: list[tuple[str, Any]]
126 ) -> tuple[int, list[FuzzFailure]]:
127     failures: list[FuzzFailure] = []
128     for label, payload in cases:
129         resp = client.post("/robots", json=payload)
130         failure = classify(label, payload, resp)
131         if failure is not None:
132             failures.append(failure)
133     return len(cases), failures
134
135 def minimal_values(prop_schema: dict[str, Any]) -> list
136 [Any]:
137     """Smallest␣schema-valid␣stand-ins␣for␣a␣property␣(
138     value␣minimization)."""
139     ptype = prop_schema.get("type")
140     if ptype == "string":
141         return ["A" * prop_schema.get("minLength", 0)]
142     if ptype == "integer":
143         return [prop_schema.get("minimum", 0)]
144     return [None]
145
146 def shrink(
147     client: httpx.Client,
148     payload: dict[str, Any],
149     is_failure: Callable[[httpx.Response], bool],
150     schema: dict[str, Any] | None = None,
151 ) -> dict[str, Any]:

```

```

148     """Greedy minimization: drop keys, then minimize
        values, while the
149     failure still reproduces. Dropping a *required* key
        usually turns a 500
150     into a 422, so value minimization is what isolates
        the trigger field."""
151     minimal = copy.deepcopy(payload)
152     for key in sorted(minimal):
153         candidate = {k: v for k, v in minimal.items()
                        if k != key}
154         if is_failure(client.post("/robots", json=
                                    candidate)):
155             minimal = candidate
156     if schema is not None:
157         for key, prop_schema in schema.get("properties"
        , {}).items():
158             if key not in minimal:
159                 continue
160             for candidate_value in minimal_values(
                prop_schema):
161                 trial = dict(minimal, **{key:
                    candidate_value})
162                 if is_failure(client.post("/robots",
                    json=trial)):
163                     minimal = trial
164                     break
165     return minimal
166
167
168 def demo() -> None:
169     cases = fuzz_cases(REQUEST_SCHEMA)
170     healthy = make_sync_client(create_app(), base_url="
        http://fuzz.test")
171     total, failures = run_fuzz(healthy, cases)
172     print(f"healthy app: {total} cases, {len(
        failures)} failures")
173
174     buggy = make_sync_client(create_app(bug_at_price
        =666), base_url="http://fuzz.test")
175     total, failures = run_fuzz(buggy, cases)
176     print(f"buggy app: {total} cases, {len(
        failures)} failures")
177     if failures:
178         first = failures[0]
179         minimal = shrink(
180             buggy, first.payload, lambda r: r.

```

```

181         status_code >= 500, REQUEST_SCHEMA
182     )
183     print(f"first_failure: {first.label} -> {first.status} ({first.reason}")
184     print(f"shrunk to: {json.dumps(minimal, sort_keys=True)}")
185
186 if __name__ == "__main__":
187     demo()

```

Its tests (the companion test file) pin the report-hygiene rules of Section 20.4: prove the fuzzer actually exercised the validation boundary (count the 4xxs), prove it finds a planted bug, and prove the shrinker converges to the minimal reproducer.

```

1  """Tests for the mini OpenAPI fuzzer.
2
3  Run: pytest test_fuzz_openapi_mini.py -q
4  """
5  from __future__ import annotations
6
7  import httpx
8
9  from fuzz_openapi_mini import REQUEST_SCHEMA,
10     fuzz_cases, run_fuzz, shrink
11  from robot_registry import create_app, make_sync_client
12
13  def _client(**app_kwargs: object) -> httpx.Client:
14      return make_sync_client(
15          create_app(**app_kwargs), # type: ignore[arg-
16              type]
17          base_url="http://fuzz.test",
18      )
19
20  def test_fuzzer_finds_no_failures_on_healthy_app() ->
21      None:
22      cases = fuzz_cases(REQUEST_SCHEMA)
23      total, failures = run_fuzz(_client(), cases)
24      assert total == len(cases) and total > 30 # report
25          hygiene: prove coverage
26      assert failures == [], [f"{f.label}: {f.reason}"
27          for f in failures]

```

```

27 def test_fuzzer_finds_and_shrinks_the_planted_bug() ->
    None:
28     cases = fuzz_cases(REQUEST_SCHEMA)
29     client = _client(bug_at_price=666)
30     _, failures = run_fuzz(client, cases)
31     assert failures, "fuzzer must find the planted 500"
32     assert all(f.status == 500 for f in failures)
33     first = failures[0]
34     minimal = shrink(client, first.payload, lambda r: r
        .status_code >= 500, REQUEST_SCHEMA)
35     # Required keys cannot be dropped (that flips 500
        -> 422), so the minimal
36     # reproducer isolates the trigger field with
        minimal valid stand-ins:
37     assert minimal == {"sku": "AAA", "name": "A", "
        price_cents": 666}
38
39
40 def test_every_4xx_is_problem_shaped() -> None:
41     """A focused regression of the RFC 9457 invariant
        on the healthy app."""
42     cases = fuzz_cases(REQUEST_SCHEMA)
43     client = _client()
44     saw_4xx = 0
45     for label, payload in cases:
46         resp = client.post("/robots", json=payload)
47         if 400 <= resp.status_code < 500:
48             saw_4xx += 1
49             body = resp.json()
50             assert body["status"] == resp.status_code,
                label
51             assert resp.headers["content-type"].
                startswith("application/problem+json")
52     assert saw_4xx > 10 # the fuzzer really exercised
        the validation boundary

```

20.3.6 Listing 19.6 — An asyncio Load Harness with Knee-Point Sweep

Listing 19.6 implements both workload models against the in-process app: `open_loop` schedules sends at fixed virtual offsets (offered load is independent of completions — no coordinated omission), while `closed_loop` runs a fixed worker pool. A fixed-bucket histogram estimates percentiles without storing every sample (the same machinery as Chapter 19). The demo work-

load hides simulated 20 ms of async work behind a semaphore of four slots, so its saturation point is real and findable: capacity $\approx c/S = 4/0.02 = 200$ req/s, exactly where the printed knee appears. `littles_law_check` closes the loop with Theorem 20.1: with eight closed-model workers it measures $L_{\text{est}} = \lambda \bar{W} \approx 8$. The sweep prints a table; nothing is asserted about wall-clock numbers.

```

1  """Stdlib+asyncio load harness for offline API load
   testing.
2
3  Implements both classic workload models against an in-
   process ASGI app:
4
5  * open model -- requests are scheduled at a fixed
   rate (requests/sec),
6  independent of how fast responses
   come back;
7  * closed model -- a fixed number of workers each run
   request-response
8  loops (think: a fixed pool of users).
9
10 Includes a bucketed latency histogram with percentile
   estimation, a
11 saturation sweep that prints a knee-point table, and a
   Little's Law
12 cross-check ( $L = \lambda * W$ ) for the closed model.
13
14 Latency numbers are *illustrative* (they depend on your
   machine); the
15 harness never asserts on wall-clock values.
16
17 Run the demo: python load_harness.py
18 """
19 from __future__ import annotations
20
21 import asyncio
22 import bisect
23 import time
24 from dataclasses import dataclass, field
25 from typing import Any
26
27 import httpx
28 from fastapi import FastAPI
29
30 BOUNDS_MS: tuple[int, ...] = (1, 2, 5, 10, 25, 50, 100,
   250, 500, 1_000, 2_500, 5_000)

```

```
31
32
33 @dataclass
34 class Histogram:
35     """Fixed-bucket latency histogram; percentiles read
36         the bucket ceiling."""
37
38     bounds: tuple[int, ...] = BOUNDS_MS
39     counts: list[int] = field(init=False)
40     errors: int = 0
41     total_ms: float = 0.0
42
43     def __post_init__(self) -> None:
44         self.counts = [0] * (len(self.bounds) + 1)
45
46     def record_ms(self, ms: float) -> None:
47         self.counts[bisect.bisect_right(self.bounds, ms)
48                     ] += 1
49         self.total_ms += ms
50
51     @property
52     def total(self) -> int:
53         return sum(self.counts)
54
55     def percentile_ms(self, p: float) -> float:
56         if self.total == 0:
57             return 0.0
58         rank = max(1, int(p / 100.0 * self.total +
59                           0.999999))
60         seen = 0
61         for i, count in enumerate(self.counts):
62             seen += count
63             if seen >= rank:
64                 return float(self.bounds[i]) if i < len
65                     (self.bounds) else float("inf")
66         return float("inf")
67
68     def mean_ms(self) -> float:
69         return self.total_ms / self.total if self.total
70             else 0.0
71
72     async def _send(client: httpx.AsyncClient, hist:
73         Histogram, path: str) -> None:
74         started = time.perf_counter()
75         try:
```



```

71         resp = await client.get(path)
72         await resp.aread()
73         if resp.status_code >= 500:
74             hist.errors += 1
75     except (httpx.HTTPError, OSError):
76         hist.errors += 1
77     finally:
78         hist.record_ms((time.perf_counter() - started)
79                        * 1_000.0)
80
81 async def open_loop(
82     client: httpx.AsyncClient, *, rps: float,
83     duration_s: float, path: str = "/work"
84 ) -> Histogram:
85     """Open_model: schedule_sends_at_fixed_virtual_
86         offsets_from_the_start."""
87     hist = Histogram()
88     period = 1.0 / rps
89     start = time.perf_counter()
90     tasks: list[asyncio.Task[None]] = []
91     k = 0
92     while k * period < duration_s: # deterministic
93         scheduled count
94         scheduled = start + k * period
95         delay = scheduled - time.perf_counter()
96         if delay > 0:
97             await asyncio.sleep(delay)
98         tasks.append(asyncio.create_task(_send(client,
99             hist, path)))
100         k += 1
101     if tasks:
102         await asyncio.gather(*tasks)
103     return hist
104
105 async def closed_loop(
106     client: httpx.AsyncClient, *, workers: int,
107     duration_s: float, path: str = "/work"
108 ) -> Histogram:
109     """Closed_model: each_worker_issues_the_next_
110         request_only_after_the
111         previous_response_arrives."""
112     hist = Histogram()
113     deadline = time.perf_counter() + duration_s

```

```
110     async def worker() -> None:
111         while time.perf_counter() < deadline:
112             await _send(client, hist, path)
113
114     await asyncio.gather(*(worker() for _ in range(
115         workers)))
116     return hist
117
118 def create_workload_app(*, work_ms: int = 20,
119     max_concurrent: int = 4) -> FastAPI:
120     """Demo workload: simulated async work behind a
121         concurrency semaphore,
122         so the service has a real, findable saturation
123         point."""
124     app = FastAPI(title="Load harness demo workload")
125     semaphore = asyncio.Semaphore(max_concurrent)
126
127     @app.get("/work")
128     async def work() -> dict[str, str]:
129         async with semaphore:
130             await asyncio.sleep(work_ms / 1_000.0)
131             return {"status": "ok"}
132
133     return app
134
135 @dataclass
136 class SweepRow:
137     rps: int
138     completed: int
139     p50_ms: float
140     p99_ms: float
141     error_rate: float
142     is_knee: bool
143
144 async def saturation_sweep(
145     app: Any,
146     levels: tuple[int, ...] = (25, 50, 100, 150, 200,
147         300, 400),
148     duration_s: float = 0.3,
149 ) -> list[SweepRow]:
150     """Ramp offered RPS; flag the knee: first level
151         where p99 explodes
152         (10x baseline) or errors appear."""
```

```

150     rows: list[SweepRow] = []
151     baseline_p50: float | None = None
152     async with httpx.AsyncClient(
153         transport=httpx.ASGITransport(app=app),
154         base_url="http://load.test"
155     ) as client:
156         for rps in levels:
157             hist = await open_loop(client, rps=rps,
158                                     duration_s=duration_s)
159             p50 = hist.percentile_ms(50)
160             p99 = hist.percentile_ms(99)
161             error_rate = hist.errors / hist.total if
162                 hist.total else 1.0
163             if baseline_p50 is None:
164                 baseline_p50 = p50 if p50 > 0 else 1.0
165             is_knee = (p99 > 10.0 * baseline_p50) or
166                 error_rate > 0.0
167             rows.append(SweepRow(rps, hist.total, p50,
168                                   p99, error_rate, is_knee))
169         return rows
170
171 def render_sweep(rows: list[SweepRow]) -> str:
172     lines = ["rps|completed|p50ms|p99ms|err%|knee",
173             "-----+-----+-----+-----+-----+-----"]
174     for row in rows:
175         lines.append(
176             f"{row.rps:5}|{row.completed:9}|{row.p50_ms:7.0f}|{row.p99_ms:8.0f}|{100*row.error_rate:5.1f}|{row.is_knee}"
177         )
178     return "\n".join(lines)
179
180 async def littlesLawCheck(workers: int = 8,
181                           duration_s: float = 0.5) -> str:
182     """Closed model: verify L = lambda * W against the configured concurrency."""
183     app = create_workload_app(work_ms=20,
184                               max_concurrent=10_000) # unthrottled
185     async with httpx.AsyncClient(

```

```

183         transport=httpx.ASGITransport(app=app),
184         base_url="http://load.test"
185     ) as client:
186         hist = await closed_loop(client, workers=
187             workers, duration_s=duration_s)
188         lam = hist.total / duration_s          #
189         throughput lambda (requests/s)
190         w = hist.mean_ms() / 1_000.0          # mean
191         residence time W (s)
192         l_est = lam * w
193         return (
194             f"closed_model: workers={workers} throughput={
195                 lam:.0f} req/s"
196             f"mean={hist.mean_ms():.1f} ms -> L= lambda*W
197                 = {l_est:.1f}"
198             f"(configured concurrency: {workers})"
199         )
200
201 async def _demo() -> None:
202     print("saturation_sweep (illustrative numbers,
203         machine-dependent):")
204     rows = await saturation_sweep(create_workload_app()
205         )
206     print(render_sweep(rows))
207     print(await littles_law_check())
208
209 if __name__ == "__main__":
210     asyncio.run(_demo())

```

Illustrative output (your numbers will differ; the shape will not):

	rps	completed	p50 ms	p99 ms	err %	knee
1	25	8	50	50	0.0	
2	100	30	50	50	0.0	
3	200	60	250	250	0.0	
4	400	120	500	1000	0.0	<<
5	KNEE					
6	closed model: workers=8 throughput=270 req/s mean=30.5 ms					
7	-> L = lambda*W = 8.2 (configured concurrency: 8)					

20.3.7 Listing 19.7 — Fault-Injection Middleware and Resilient Client

Listing 19.7 is the chaos toolkit. `FaultScript` maps request index to fault — deterministic by construction, no dice. The pure ASGI `FaultInjectionMiddleware` implements three fault classes from the taxonomy: `LATENCY` (sleep on the *virtual* clock, then serve), `ERROR` (answer directly with an RFC 9457-shaped 503), `ABORT` (raise before any response, simulating a reset connection). The client under test, `ResilientTransport`, is a real `httpx` transport: bounded retries with exponential backoff on the injected sleeper, and a circuit breaker that fails fast after `breaker_threshold` consecutive exhausted calls.

```

1  """Deterministic fault-injection middleware and a
   resilient client transport.
2
3  Fault classes (see the chapter's fault taxonomy):
4
5  * 'LATENCY' -- sleep (on a *virtual* clock) before
   calling the app;
6  * 'ERROR' -- answer directly with an RFC 9457-
   shaped 5xx;
7  * 'ABORT' -- drop the connection (raise before any
   response).
8
9  The class: 'VirtualClock' sleeper never waits in real
   time: tests exercise
10 multi-second retry backoffs in microseconds and assert
   on *recorded*
11 virtual delays, never on the wall clock.
12
13 The class: 'ResilientTransport' is the client under
   test: bounded retries
14 with exponential backoff plus a circuit breaker that
   fails fast after
15 'breaker_threshold' consecutive exhausted calls.
16 """
17 from __future__ import annotations
18
19 from dataclasses import dataclass
20 from enum import StrEnum
21 from typing import Any, Awaitable, Callable, Sequence
22
23 import httpx
24
25 Sleeper = Callable[[float], Awaitable[None]]
26

```

```
27
28 class FaultKind(StrEnum):
29     LATENCY = "latency"
30     ERROR = "error"
31     ABORT = "abort"
32
33
34 @dataclass(frozen=True)
35 class Fault:
36     kind: FaultKind
37     delay_ms: int = 0
38     status: int = 503
39
40
41 class FaultScript:
42     """Deterministic fault schedule: fault_for(i) depends only on the
43     request index i; beyond the script the service is healthy."""
44
45     def __init__(self, entries: Sequence[Fault | None])
46         -> None:
47         self._entries = list(entries)
48
49     def fault_for(self, index: int) -> Fault | None:
50         return self._entries[index] if index < len(self
51             ._entries) else None
52
53
54 class FaultInjectionMiddleware:
55     """Pure ASGI middleware applying the scripted fault
56     per request."""
57
58     def __init__(self, app: Any, script: FaultScript,
59         sleeper: Sleeper) -> None:
60         self._app = app
61         self._script = script
62         self._sleeper = sleeper
63         self.requests_seen = 0
64
65     async def __call__(self, scope: dict, receive:
66         Callable, send: Callable) -> None:
67         if scope["type"] != "http":
68             await self._app(scope, receive, send)
69             return
70         fault = self._script.fault_for(self
```

```

        requests_seen)
66     self.requests_seen += 1
67     if fault is None or fault.kind is FaultKind.
        LATENCY:
68         if fault is not None:
69             await self._sleeper(fault.delay_ms / 1
                _000.0)
70         await self._app(scope, receive, send)
71         return
72     if fault.kind is FaultKind.ERROR:
73         body = (
74             b'{"type": "https://northwind-robotics.
                example/problems/injected", '
75             b'"title": "Injected failure", "status": %
                d, '
76             b'"detail": "chaos middleware", "instance
                ": "%s"}'
77             % (fault.status, scope.get("path", "/")
                .encode())
78         )
79         await send({
80             "type": "http.response.start",
81             "status": fault.status,
82             "headers": [(b"content-type", b"
                application/problem+json")],
83         })
84         await send({"type": "http.response.body", "
            body": body})
85         return
86         raise ConnectionResetError("injected abort") #
            FaultKind.ABORT
87
88
89 class VirtualClock:
90     """Sleeper that never waits: records delays,
        advances virtual time."""
91
92     def __init__(self) -> None:
93         self.now = 0.0
94         self.sleeps: list[float] = []
95
96     async def sleep(self, seconds: float) -> None:
97         self.sleeps.append(seconds)
98         self.now += seconds
99
100

```

```
101 class BreakerOpenError(httpx.TransportError):
102     """Raised when the circuit breaker short-circuits a
        call."""
103
104
105 @dataclass(frozen=True)
106 class RetryPolicy:
107     retries: int = 3
108     base_backoff_s: float = 0.1
109     breaker_threshold: int = 3
110
111
112 class ResilientTransport(httpx.AsyncBaseTransport):
113     """Bounded retries with virtual-clock backoff +
        circuit breaker."""
114
115     def __init__(
116         self,
117         inner: httpx.AsyncBaseTransport,
118         policy: RetryPolicy | None = None,
119         sleeper: Sleeper | None = None,
120     ) -> None:
121         self._inner = inner
122         self._policy = policy or RetryPolicy()
123         self._sleeper = sleeper or _real_sleep
124         self._consecutive_failures = 0
125
126     async def handle_async_request(self, request: httpx
        .Request) -> httpx.Response:
127         if self._consecutive_failures >= self._policy.
            breaker_threshold:
128             raise BreakerOpenError("circuit breaker
                open: failing fast")
129         policy = self._policy
130         last_resp: httpx.Response | None = None
131         last_exc: Exception| None = None
132         for attempt in range(policy.retries + 1):
133             try:
134                 resp = await self._inner.
                    handle_async_request(request)
135             except (httpx.TransportError, OSError) as
                exc:
136                 last_exc, resp = exc, None # ABORT
                    class: no response at all
137             if resp is not None and resp.status_code <
                500:
```



```

138         self._consecutive_failures = 0
139         return resp
140     if resp is not None:
141         await resp.aread()
142         last_resp = resp
143     if attempt < policy.retries:
144         await self._sleeper(policy.
145                               base_backoff_s * (2**attempt))
146     self._consecutive_failures += 1
147     if last_resp is not None:
148         return last_resp
149     assert last_exc is not None
150     raise last_exc
151
152 async def _real_sleep(seconds: float) -> None:
153     import asyncio
154
155     await asyncio.sleep(seconds)

```

The tests (the companion test file) run multi-second backoff schedules in microseconds and assert only on recorded virtual time and call counts: recovery from two injected 5xxs with backoff [0.1, 0.2], a 250 ms latency fault that never touches the wall clock, abort storms surfacing as transport errors, a breaker that opens after two exhausted calls and short-circuits the third (proven by the middleware's request counter), and a healthy passthrough that burns zero retries.

```

1  """Deterministic resilience tests: every fault class,
2     zero wall-clock waits.
3
4  Run: pytest test_fault_injection.py -q
5  """
6
7  from __future__ import annotations
8
9  import asyncio
10
11  import httpx
12  import pytest
13
14  from fault_injection import (
15      BreakerOpenError,
16      Fault,
17      FaultInjectionMiddleware,
18      FaultKind,
19      FaultScript,

```

```
18     ResilientTransport,
19     RetryPolicy,
20     VirtualClock,
21 )
22 from robot_registry import create_app
23
24 SEEDED_ROBOT = {"id": 1, "sku": "NWR-ARM-6", "name": "
    A6_Manipulator_Arm", "price_cents": 129900}
25
26
27 def make_rig(
28     script: FaultScript, policy: RetryPolicy | None =
        None
29 ) -> tuple[httpx.AsyncClient, VirtualClock,
    FaultInjectionMiddleware]:
30     clock = VirtualClock()
31     app = create_app()
32     app.state.db[1] = dict(SEEDED_ROBOT)
33     middleware = FaultInjectionMiddleware(app, script,
        clock.sleep)
34     inner = httpx.ASGITransport(app=middleware)
35     transport = ResilientTransport(inner, policy or
        RetryPolicy(), clock.sleep)
36     client = httpx.AsyncClient(transport=transport,
        base_url="http://faults.test")
37     return client, clock, middleware
38
39
40 def run(coroutine): # keep pytest asyncio-plugin free
41     return asyncio.run(coroutine)
42
43
44 def test_recovers_from_transient_errors() -> None:
45     async def scenario() -> tuple[int, list[float]]:
46         client, clock, _ = make_rig(FaultScript([Fault(
            FaultKind.ERROR), Fault(FaultKind.ERROR),
            None]))
47         async with client:
48             resp = await client.get("/robots/1")
49             return resp.status_code, clock.sleeps
50
51     status, sleeps = run(scenario())
52     assert status == 200
53     assert sleeps == [0.1, 0.2] # exponential backoff
        on the virtual clock
54
```

```

55
56 def test_latency_fault_is_injected_on_virtual_time_only
    () -> None:
57     async def scenario() -> tuple[int, float]:
58         client, clock, _ = make_rig(
59             FaultScript([Fault(FaultKind.LATENCY,
60                               delay_ms=250)]),
61             RetryPolicy(retries=0),
62         )
63         async with client:
64             resp = await client.get("/robots/1")
65             return resp.status_code, clock.now
66
67     status, virtual_now = run(scenario())
68     assert status == 200
69     assert virtual_now == 0.25 # recorded, not slept
70
71 def test_abort_retries_then_surfaces_transport_error()
    -> None:
72     async def scenario() -> list[float]:
73         client, clock, _ = make_rig(
74             FaultScript([Fault(FaultKind.ABORT), Fault(
75                               FaultKind.ABORT)]),
76             RetryPolicy(retries=1),
77         )
78         async with client:
79             with pytest.raises(ConnectionResetError):
80                 await client.get("/robots/1")
81             return clock.sleeps
82
83     assert run(scenario()) == [0.1]
84
85 def test_circuit_breaker_fails_fast_after_threshold()
    -> None:
86     async def scenario() -> tuple[int, int]:
87         client, _, middleware = make_rig(
88             FaultScript([Fault(FaultKind.ERROR)] * 5),
89             RetryPolicy(retries=0, breaker_threshold=2),
90         )
91         async with client:
92             first = await client.get("/robots/1") #
93             503 -> failures = 1
94             second = await client.get("/robots/1") #

```

```

94         503 -> failures = 2 (threshold)
          assert (first.status_code, second.
                status_code) == (503, 503)
95         with pytest.raises(BreakerOpenError):
96             await client.get("/robots/1")          #
                short-circuited
97         return middleware.requests_seen, client is not
                None and 1
98
99     requests_seen, _ = run(scenario())
100     assert requests_seen == 2  # third call never
        reached the middleware
101
102
103 def test_healthy_passthrough_needs_no_retries() -> None
    :
104     async def scenario() -> tuple[int, list[float], int
        ]:
105         client, clock, middleware = make_rig(
            FaultScript([]))
106         async with client:
107             resp = await client.get("/robots/1")
108             return resp.status_code, clock.sleeps,
                middleware.requests_seen
109
110     status, sleeps, seen = run(scenario())
111     assert (status, sleeps, seen) == (200, [], 1)

```

20.3.8 Listing 19.8 — The Quality Gate

`ci_gate.py` composes the chapter into one blocking check. Each sub-gate is *self-checking*: the contract gate additionally verifies that the deliberately broken provider variant is rejected, and the fuzz gate requires that the planted-bug variant produces failures — a gate that cannot fail is not a gate. The perf smoke asserts structure (all scheduled requests completed, zero 5xx), never wall-clock latency. Exit code 0 means merge; 1 means read the report.

```

1  """CI quality gate: contract verify + fuzz smoke + perf
   smoke -> exit code.
2
3  Each sub-gate returns (passed, detail). The contract
   and fuzz gates include
4  a *self-check*: they also run against a deliberately
   broken variant and

```

```

5  require_that_the_gate_catches_it -- a gate that cannot
   fail is not a gate.
6
7  Run: python ci_gate.py (exit code 0 = PASS, 1 =
   FAIL)
8  """
9  from __future__ import annotations
10
11  import asyncio
12  import sys
13  from pathlib import Path
14
15  import httpx
16
17  from fuzz_openapi_mini import REQUEST_SCHEMA,
   fuzz_cases, run_fuzz
18  from load_harness import create_workload_app, open_loop
19  from pact_mini import PROVIDER_STATES, ProviderVerifier
   , record_demo_pact
20  from robot_registry import create_app, make_sync_client
21
22  PACT_DIR = Path(__file__).resolve().parent / "pacts"
23
24
25  def contract_gate() -> tuple[bool, str]:
26      pact = record_demo_pact(PACT_DIR)
27      healthy = ProviderVerifier(create_app, pact,
   PROVIDER_STATES).verify()
28      broken = ProviderVerifier(lambda: create_app(
   breaking=True), pact, PROVIDER_STATES).verify()
29      detail = (
30          f"{len(healthy.results)} interactions, {len(
   healthy.failures())} mismatches;"
31          f"self-check: {breaking} variant rejected with "
32          f"{len(broken.failures())} mismatch(es)"
33      )
34      return healthy.ok and not broken.ok, detail
35
36
37  def fuzz_gate() -> tuple[bool, str]:
38      cases = fuzz_cases(REQUEST_SCHEMA)
39      healthy_client = make_sync_client(create_app(),
   base_url="http://gate.test")
40      total, failures = run_fuzz(healthy_client, cases)
41      buggy_client = make_sync_client(create_app(
   bug_at_price=666), base_url="http://gate.test")

```

```

42     _, self_check_failures = run_fuzz(buggy_client,
43         cases)
44     detail = (
45         f"{total}_cases,_{len(failures)}_failures;"
46         f"self-check:_planted_bug_found_{len(self_check_failures)}_failing_case(s))"
47     )
48     return not failures and bool(self_check_failures),
49     detail
50
51 def perf_gate() -> tuple[bool, str]:
52     """Perf_smoke:_short_open-model_burst._Asserts*_
53     structure*_(_all_scheduled
54     requests_completed, _zero_5xx), _never_wall-clock_
55     latency_thresholds."""
56
57     async def burst() -> tuple[int, int]:
58         app = create_workload_app()
59         async with httpx.AsyncClient(
60             transport=httpx.ASGITransport(app=app),
61             base_url="http://gate.test"
62         ) as client:
63             hist = await open_loop(client, rps=50,
64                 duration_s=0.4)
65             return hist.total, hist.errors
66
67     total, errors = asyncio.run(burst())
68     expected = int(50 * 0.4) # deterministic scheduled
69     count modulo float edge
70     ok = errors == 0 and abs(total - expected) <= 1
71     return ok, f"{total}_requests_(~{expected}_
72     scheduled),_{errors}_server_errors"
73
74
75 GATES = (
76     ("contract_verify", contract_gate),
77     ("fuzz_smoke", fuzz_gate),
78     ("perf_smoke", perf_gate),
79 )
80
81 def main() -> int:
82     print("==_API_quality_gate_==")
83     all_ok = True
84     for name, gate in GATES:

```

```

79         try:
80             passed, detail = gate()
81         except Exception as exc: # a crashing gate is
            a failing gate
82             passed, detail = False, f"gate_crashed:_{
                exc!r}"
83             all_ok &= passed
84             print(f"[{'PASS' if passed else 'FAIL'}]_{name
                :<16}_{detail}")
85             print(f"GATE_RESULT:_{{'PASS' if all_ok else 'FAIL'}
                ")
86             return 0 if all_ok else 1
87
88
89 if __name__ == "__main__":
90     sys.exit(main())

```

```

1 == API quality gate ==
2 [PASS] contract verify 2 interactions, 0 mismatches;
   self-check: breaking
3                               variant rejected with 2
                               mismatch(es)
4 [PASS] fuzz smoke        38 cases, 0 failures; self-
   check: planted bug
5                               found (1 failing case(s))
6 [PASS] perf smoke        20 requests (~20 scheduled), 0
   server errors
7 GATE RESULT: PASS

```

20.3.9 Listing 19.9 — The Pipeline as Code

The GitHub Actions workflow is PowerShell-native (Windows runner, pwsh shells, backslash paths — no POSIX assumptions):

```

1 name: api-quality-gates
2 on:
3   pull_request:
4   push:
5     branches: [main]
6
7 jobs:
8   quality:
9     runs-on: windows-latest
10    steps:
11      - uses: actions/checkout@v4

```

```
12     - uses: actions/setup-python@v5
13       with:
14         python-version: "3.11"
15     - name: Install dependencies
16       shell: pwsh
17       run: |
18         python -m venv .venv
19         .\.venv\Scripts\Activate.ps1
20         pip install -r code\ch19\requirements.txt
21     - name: Unit, property, contract, fuzz,
22       resilience tests
23       shell: pwsh
24       run: pytest code\ch19 -q
25     - name: Blocking quality gate
26       shell: pwsh
27       run: python code\ch19\ci_gate.py
```

20.4 Common Pitfalls & Anti-Patterns

Each of these is a production scar, not a stylistic preference.

- P-1. Testing implementation instead of behavior.** A test that asserts `validator()` was called with `X` breaks on every refactor and catches nothing. Assert on the wire contract: status, body shape, headers. The refactor-safety of a suite is its behavioral surface area.
- P-2. The inverted pyramid.** An E2E-heavy suite is slow, flaky, and diagnostically mute — a red test says “something, somewhere”. Every E2E failure should be convertible into a missing lower-layer test; if it cannot, your layers have a hole.
- P-3. Mocking what you do not own, naively.** A hand-built stub of a third-party API encodes your *guess* about that API. The stub stays green while the vendor changes. Mitigations, in order: contract tests with the vendor’s participation, record–replay cassettes re-recorded on a schedule, and at minimum a nightly smoke against the real sandbox.
- P-4. Asserting on wall-clock.** `assert elapsed < 2.0` fails on a loaded CI runner and passes on an idle one — the assertion measures the machine, not the code. Inject clocks and sleepers; assert on recorded virtual schedules. Reserve real latency assertions for

scheduled runs on quiet hardware, and even then compare ratios (candidate vs. baseline), not absolutes.

- P-5. Load-testing production without isolation.** A stress test against shared production is an outage you scheduled. Isolate: dedicated environment, synthetic tenants, synthetic data, and a blast-radius budget agreed with whoever gets paged.
- P-6. Flaky retries masking real bugs.** “Re-run and it passes” applied blindly converts race conditions into invisible inventory. Policy: two flakes in a week means fix or delete; automatic retry in CI is allowed only with failure-cause logging and a flake dashboard.
- P-7. Fuzzing without shrinking or report hygiene.** A 4,000-key failing payload is not a bug report; shrink it. And always emit coverage of the campaign — cases run, classes hit — otherwise a crash-on-startup fuzzer reports “0 failures” forever.
- P-8. Verifying contracts against a mock provider.** If the “provider side” of contract verification stubs the provider, the only thing verified is the pact file’s internal consistency. Replay against the real application, in-process at minimum.
- P-9. Provider states that seed the universe.** A state named “standard test data” that loads 400 fixtures hides which interaction needs what. Minimal, semantic, independent states — or debugging a failure becomes archaeology.
- P-10. Coverage floors as quality theater.** A 95% line-coverage gate can be satisfied by tests that assert nothing. Floors guard against suite deletion; they do not manufacture confidence. Pair every floor with invariant-based tests that must fail when behavior breaks.

Warning

The two most expensive words in API quality are “it passed”. Passed *what*, running *where*, against *which* version of the dependency, with *which* seed? A gate whose inputs are not pinned — versions, seeds, pacts, specs — manufactures confidence, not evidence.

20.5 Hands-On Production Lab & Real-World Case Study

20.5.1 Lab: Run the Gate, Then Break the API

Work from the book repository root in PowerShell. Estimated time: 60–90 minutes.

```
1 cd code\ch19
2 pip install -r requirements.txt          # once
3 pytest -q                               # 15 tests: pact
    , pbt, fuzz, faults
```

1. **Baseline.** Run `pytest -q` (all green), then `python fuzz_openapi_mini.py` and note: 38 cases, zero failures on the healthy app; the buggy variant yields one failure shrunk to `price_cents = 666`. Run `python load_harness.py` and read the sweep: locate the knee level and compare the Little’s Law estimate to the configured worker count.
2. **Run the gate.** `python ci_gate.py`; confirm all three sub-gates pass and `$LASTEXITCODE` is 0. Read `pacts\fulfillment-service-robot-registry.json` — this is the contract the next step will break.
3. **Break the contract.** In `ci_gate.py`, change the healthy verifier factory from `create_app` to `lambda: create_app(breaking=True)` and re-run. The contract gate fails and names the missing field: `$.sku: missing`. Restore the file.
4. **Break the error contract.** Change the fuzz gate’s healthy client to `create_app(bug_at_price=666)` and re-run: the fuzz smoke fails with a 5xx report. Restore.
5. **Break capacity.** In `ci_gate.py`, drop the perf smoke to `rps=50, duration_s=0.4` against `create_workload_app(work_ms=400)` and observe queueing in a manual sweep; reason with Little’s Law about which pool exhausts first.
6. **Reflect.** For each break: which gate caught it, at which pyramid layer, and what would the same break have cost as a production incident?

20.5.2 War Story: The Deploy That Passed Everything Except the Contract

A fulfillment platform team (names changed; the failure is painfully common) ran a genuinely good pipeline: 90% coverage, integration environments, canary deploys. The robot-registry team shipped what their OpenAPI review called a “cosmetic normalization”: the response field `sku` became `part_number`, matching the new ERP system’s vocabulary. Every provider-side test passed — they had all been updated in the same commit, which is exactly the problem. The staging integration suite passed too, because the only consumer exercised there was the provider team’s own demo client.

The fulfillment service — which read `sku` on every order line — did not fail at deploy time. It failed eleven minutes later, at the peak of the hourly batch, as a tide of `KeyError: 'sku'` that the canary analysis could not see because the canary received no fulfillment traffic. Rollback took forty minutes; order backlog recovery took the weekend.

The postmortem’s uncomfortable finding was that the consumer team *had* written a pact — months earlier, in a spike — and it sat in a gist nobody owned. It was never published to the broker, the provider team never knew it existed, and no `can-i-deploy` gate consumed it. The failure was not technical; the loop of Figure 20.2 was never closed organizationally. The fix was equally organizational: pact publication became a required artifact of consumer CI, provider verification became a required step of provider CI, and the deploy gate learned to answer “which consumers have verified this exact provider version?” The technology in this chapter is the easy half; wiring the artifact across the team boundary is the half that fails.

Example

Mini-case, fully reproducible on your machine: the lab’s step 3 is this war story in miniature. The provider’s own tests (Listing 19.1’s app is self-consistent in both modes) stay green under the rename; only the consumer-recorded pact detects it. If you want the full emotional experience, delete the pact file first and watch the gate turn green — then decide who on your team would have caught that in review.

20.6 Self-Assessment Exercises & Deep-Dive Questions

- E-1. Layer audit.** Take the fifteen tests in `code\ch19` and classify each into exactly one pyramid layer of Definition 20.1. Which layer is over-weighted relative to its defect-catching power, and

why is that acceptable here?

- E-2. Designated catcher.** For each defect — (a) SKU validation regex wrong, (b) response field renamed, (c) database connection string mispointed, (d) retry backoff missing jitter, (e) p99 regression after a serializer change — name the cheapest pyramid layer that can catch it and sketch the test.
- E-3. Contract authoring.** Add a third consumer, `billing-service`, that reads only `id` and `price_cents`. Record its pact. Prove that renaming `name` breaks no consumer while renaming `price_cents` breaks exactly one — the usage-sensitivity schema diffing cannot express.
- E-4. Provider-state discipline.** Refactor the seeded state into two independent states (“an empty registry”, “a robot with id 7 exists”) and replay. What breaks if two interactions share one app instance instead of one fresh app per interaction?
- E-5. Metamorphic properties.** Extend Listing 19.4 with a metamorphic relation: creating the same payload twice (minus `id`) must yield equal records up to the `id` field. Why is this stronger than any single example assertion?
- E-6. Coordinated omission.** Modify `open_loop` to schedule by completion time (send next 1/rps after the previous finishes). Re-run the sweep at 400 rps and explain why the table now lies about the knee.
- E-7. Blast radius.** Write the steady-state hypothesis and abort-criteria paragraph for running Listing 19.7’s latency fault against a shared staging environment. What telemetry proves the experiment stayed in bounds?
- E-8. Gate economics.** Your CI budget is four minutes. Allocate it across the gates of Listing 19.8 plus coverage, and defend the cuts. What moves to a nightly job, and what evidence do you lose?

20.7 Interview Q&A Vault

- Q1. *[junior]* What are the layers of the API test pyramid, and what does each catch?** Unit: pure logic (validators, serializers) — logic bugs, in microseconds. Component: whole

service in-process — wiring, validation, error-shape bugs. Contract: consumer-provider agreement — cross-team breakage. Integration: real infrastructure boundaries — configuration and migration drift. E2E: golden journeys — emergent whole-system failures only.

- Q2. [junior] Unit versus component test for an HTTP endpoint?** A unit test calls the validation function directly; a component test sends a real HTTP request through the real router and serializer, in-process. The component test catches URL mapping, content-negotiation, and error-shaping bugs the unit test cannot see.
- Q3. [junior] What is a consumer-driven contract?** An artifact authored by the *consumer* recording the exact interactions it depends on, replayed by the *provider's* build against real code. The direction of authorship is the whole point: it inverts who defines “correct”.
- Q4. [junior] What is inside a pact file?** Consumer and provider names, and a list of interactions: request (method, path, body), expected response (status, the body subset actually read), and the named provider state required for replay.
- Q5. [junior] Soak versus stress versus spike?** Stress: ramp until failure to find the knee. Soak: expected load for hours to catch leaks and slow degradation. Spike: sudden 10x burst to test elasticity and cold caches.
- Q6. [junior] Why p99 and not average latency?** Averages hide tails; users experience tails. At 1% bad requests and a million requests a day, ten thousand users had a bad day while the average looked fine.
- Q7. [mid] What makes a good provider state?** Minimal (arranges exactly what the interaction needs), independent (any order or subset), and semantic (named by domain meaning: “a robot with id 7 exists”, not “fixture set B”). States that seed the universe hide coupling and make failures undebuggable.
- Q8. [mid] Consumer-driven contracts versus OpenAPI schema diffing — when each?** Contracts are usage-based: they catch breakage in fields consumers actually read, including semantic drift inside unchanged shapes, but cover only enrolled consumers. Schema diff is structural: it guards public APIs with unknown

consumers but flags only syntactic incompatibility. Internal microservices with few consumers: Pact. Public API: diff plus both if possible.

- Q9.** *[mid]* **Fake versus stub versus mock?** Fake: working implementation with a shortcut (in-memory store, ASGI transport). Stub: canned answers. Mock: pre-programmed expectations that fail on unexpected calls. Prefer fakes for what you own; stubs at third-party edges; mocks sparingly, since they couple tests to interaction detail.
- Q10.** *[mid]* **When is mocking what you don't own acceptable?** At a hard third-party boundary, with canned responses derived from the vendor's documented contract — and always paired with either a vendor-verified contract, a re-recorded cassette, or a scheduled sandbox smoke. The unforgivable version asserts on the mock's call list and calls it integration testing.
- Q11.** *[mid]* **What is property-based testing?** Instead of hand-picked examples, you define a generator (strategy) over the input space and an invariant that must hold for every generated input; the framework explores, and on failure shrinks to a minimal counterexample.
- Q12.** *[mid]* **Name three invariants every API should satisfy under generated inputs.** Never 5xx; every 2xx validates against the published response schema; every 4xx is RFC 9457 problem-shaped.
- Q13.** *[mid]* **What is shrinking and why does it matter?** Automatic minimization of a failing input while the failure still reproduces. It converts “500 on this 4,000-key blob” into “500 when `price_cents` is 666” — the difference between noise and a bug report.
- Q14.** *[mid]* **How do you test timeouts and retries without sleeping?** Inject the sleeper. Production wires the real `sleep`; tests wire a virtual clock that records requested delays and advances instantly. Assert on the recorded backoff schedule (`[0.1, 0.2, 0.4]`), which is the real behavioral contract, in microseconds of wall time.
- Q15.** *[mid]* **How do you test a circuit breaker deterministically?** Scripted fault injection indexed by request number (not randomness): N consecutive exhausted calls, then assert the $(N+1)$ -th

call fails fast by counting requests that reached the dependency. Then inject one success and assert the breaker resets.

- Q16.** *[mid]* **Risks of record–replay cassettes?** Secrets and PII baked into fixtures (sanitize); silent rot as the real service evolves (re-record on schedule, or at least smoke against reality); and false confidence because replay cannot diverge — it can only be stale.
- Q17.** *[mid]* **What belongs in a CI quality gate for an API?** Fast hermetic layers: unit + property suites with a coverage floor, contract verification per published pact, OpenAPI diff versus the released spec, a bounded fuzz smoke, and a structural perf smoke. Deep load and soak move to scheduled jobs.
- Q18.** *[mid]* **What is the value — and the failure mode — of coverage floors?** Value: they make suite deletion visible and force new code to arrive with *some* test. Failure mode: they reward assertion-free tests that touch lines. Pair floors with invariant-based tests and review test *assertions*, not just diffs.
- Q19.** *[mid]* **How do you handle flaky tests?** Track flake rate per test; two flakes in a week means fix or delete; quarantine with an SLA, never as a graveyard; make CI retries log failure causes; prefer determinism (derandomized seeds, virtual clocks) over retry budgets.
- Q20.** *[senior]* **Open versus closed workload models, and what is coordinated omission?** Closed: N workers each wait for the response before sending again — offered load shrinks when the system slows. Open: fixed arrival rate regardless of completions. Coordinated omission is the closed-model pathology where load scheduled *during* a latency spike is silently skipped, producing healthy-looking results for an unhealthy system. Latency and saturation work demands open models.
- Q21.** *[senior]* **Derive Little’s Law and apply it to sizing.** Over a long window, the area under $L(t)$ equals the sum of residence times; dividing by T gives $L = \lambda W$. Sizing: at 800 req/s and 30 ms mean latency, $L = 24$ requests are resident — every pool and concurrency limit must exceed 24 with burst headroom, or queueing begins.
- Q22.** *[senior]* **What is the knee point and how do you find it operationally?** The offered load where latency stops growing

linearly and turns hyperbolic ($W \approx S/(1 - \rho)$). Find it with an open-model sweep: fixed windows at increasing RPS, plotting p99 and error rate; the knee is the first level where p99 exceeds roughly 10x baseline or errors appear. Provision at 60–70% of the knee.

- Q23. [senior] How does Schemathesis-style fuzzing differ from naive random fuzzing?** It is schema-guided: candidates are drawn from the OpenAPI constraints (`minLength-1`, `maxLength+1`, `minimum-1`, type confusions, missing/extra keys), so it concentrates on validation boundaries where bugs live, and it checks contract invariants (never 5xx, schema-valid 2xx, problem-shaped 4xx) instead of merely watching for crashes.
- Q24. [senior] What does stateful rule-based testing catch that single-call properties cannot?** Sequence-dependent bugs: lost updates, read-your-writes violations, idempotency-key reuse corruption, cross-resource leakage. You generate random operation interleavings, maintain a local model, and assert agreement after every step.
- Q25. [senior] Define steady-state hypothesis and blast radius.** Steady state: a measurable behavioral definition of “fine” (p99 and error rate over a window), never an internal proxy. Hypothesis: the system stays in steady state under a specified fault. Blast radius: the maximal scope the experiment can harm — start at one in-process middleware, widen only as confidence and abort automation mature.
- Q26. [senior] Give the fault taxonomy for HTTP APIs and one resilience defect each class reveals.** Latency (missing timeouts), synthesized 5xx (retry storms without jitter), abort (unhandled exception paths), partition (failed failover), resource exhaustion (queue growth, priority inversion).
- Q27. [senior] Performance regression budgets in CI when runners are noisy?** Do not assert wall-clock latency on shared runners: assert structure (completion, zero 5xx) in the PR gate, and run ratio-based comparisons (candidate versus same-run baseline) on dedicated quiet hardware as a scheduled job with archived histograms.
- Q28. [senior] A provider’s tests all pass, the deploy ships, consumers break. What process failed?** The provider tested against itself. No consumer contract was verified for the de-

ployed version: pact publication, provider-side replay, and the `can-i-deploy` gate were missing or not blocking. Schema diff may also have missed it if the change was semantic.

- Q29.** *[staff]* **Design a load test that finds the knee of a new checkout API.** Model the workload (open model; realistic request mix and payload sizes); isolate an environment; baseline at trivial load; sweep offered RPS in fixed short windows past predicted capacity (c/S); record p50/p99/error rate per level; flag the knee at 10x-baseline p99 or first error; cross-check with Little's Law ($L = \lambda W$ versus pool sizes); set the budget at 60–70% of the knee; convert the sweep into a scheduled regression job and a structural PR smoke.
- Q30.** *[staff]* **How would you roll out consumer-driven contracts across forty microservices?** Start with the three highest-change-frequency provider/consumer pairs; stand up a broker; make publication a CI artifact and verification a provider CI step; run gates advisory-first for a month, publish the catch statistics, then make them blocking; invest in provider-state libraries per domain; measure pact coverage of production traffic, not of repositories.
- Q31.** *[staff]* **What does the delivery-performance evidence (Accelerate, SRE practice) say about test automation?** Fast, trusted, automated gates correlate with *both* deployment throughput and stability — quality gates are a delivery feature. The organizational failure mode is slow, flaky suites that train engineers to bypass them; the fix is engineering discipline in the gates themselves, not more process around them.
- Q32.** *[staff]* **Your pbt suite passes for months, then fails on a seed nobody can reproduce. Diagnose.** Check for hidden nondeterminism: unseeded randomness, wall-clock reads, dict/set ordering, environment dependence, leaked state between examples, or a live dependency sneaking into the “offline” suite. Then pin everything (derandomize, inject clocks, fresh app per test) and add the failing example as a regression test.

20.8 External Bibliography & Further Reading

- **Pact documentation** — *Pact: Consumer-Driven Contracts*, <https://docs.pact.io> [Pac2] The reference for the consumer/pact-file/broker/verification loop, provider states, and `can-i-deploy`.
- **OpenAPI 3.1 Specification** — the schema source our fuzzer consumes; pair with any OpenAPI-diff tooling for structural compatibility gates [Ope21].
- **Steve Freeman & Nat Pryce**, *Growing Object-Oriented Software, Guided by Tests* (Addison-Wesley, 2009). The canonical treatment of mock-based design, the doubles taxonomy in practice, and why tests should express behavior.
- **Gerard Meszaros**, *xUnit Test Patterns: Refactoring Test Code* (Addison-Wesley, 2007). The precise dummy/stub/fake/ spy/mock vocabulary used in Section 20.2.4.
- **Hypothesis documentation** — <https://hypothesis.readthedocs.io>. Strategies, shrinking, `derandomize`, and `RuleBasedStateMachine` for stateful API testing.
- **Schemathesis documentation** — <https://schemathesis.readthedocs.io>. Production-grade OpenAPI-driven fuzzing whose miniature we built in Listing 19.5.
- **k6 documentation** — <https://grafana.com/docs/k6>. Arrival-rate (open-model) executors, thresholds, and checks; Locust (<https://docs.locust.io>) is the Python-native alternative.
- **Betsy Beyer et al.** (eds.), *Site Reliability Engineering* (O'Reilly, 2016) [Bey+16] — the testing and release-engineering chapters, plus SLO thinking for load-test budgets.
- **Michael T. Nygard**, *Release It!, 2nd ed.* (Pragmatic Bookshelf, 2018) [Nyg18] — stability patterns and anti-patterns; the conceptual home of circuit breakers and capacity disasters.
- **Martin Kleppmann**, *Designing Data-Intensive Applications* (O'Reilly, 2017) [Kle17] — partial failure, partitions, and why distributed systems need fault injection.
- **Nicole Forsgren, Jez Humble, Gene Kim**, *Accelerate* (IT Revolu-

tion, 2018) — the empirical link between fast, trusted test automation and delivery performance.

- **FastAPI and httpx documentation** [Ram24; Enc24] — application factories, ASGI transports, and `MockTransport` interception patterns.
- **RFC 9457**, *Problem Details for HTTP APIs* — the error shape our invariants assert on every 4xx.
- **Gil Tene**, “How NOT to Measure Latency” (talk, 2011) — the definitive coordinated-omission rant behind our open-model insistence.

20.9 Capstone Projects

Capstone 19.1 [junior] — A Second Consumer

Objective: Extend the chapter’s contract suite with a new consumer to see usage-based verification in action.

Inputs/Datasets: `code\ch19\` (robot registry, `pact_mini.py`); synthetic Northwind Robotics data only.

Steps:

1. Invent `billing-service`, which reads only `id` and `price_cents` from `GET /robots/{id}`.
2. Record its pact with its own provider state (“a robot with id 11 exists”).
3. Verify both pacts against the healthy app, then against `create_app(breaking=True)`.
4. Introduce a third variant that renames `name` only, and show no consumer pact fails.

Acceptance Criteria:

- ☐ Two pact files exist and verify green against the healthy app.
- ☐ The `sku` rename fails exactly one pact; the `name` rename fails none.
- ☐ Reports name the failing field with a JSONPath-style location.

Stretch Goals: Add a `can-i-deploy(provider_version)` function consulting a local JSON “broker” mapping versions to verified pacts.

Capstone 19.2 [mid] — Property Suite for a New Resource

Objective: Bring a second resource under property-based and fuzz coverage.

Inputs/Datasets: `code\ch19\`; a new `/warehouses` resource you add to the registry (fields: `code`, `city`, `capacity_m2`).

Steps:

1. Implement the resource with RFC 9457 error handling.
2. Write hostile-payload Hypothesis tests encoding the three universal invariants.
3. Extend the fuzzer's schema excerpt and candidate generation for the new fields.
4. Plant two distinct bugs (a 500 trigger; a schema-violating 201) and confirm both are found and shrunk.

Acceptance Criteria:

- ☐ `derandomize=True` suites pass repeatably.
- ☐ Both planted bugs produce minimal shrunk reproducers.
- ☐ Zero 5xx and zero non-problem 4xx across the campaign.

Stretch Goals: Add a rule-based stateful machine over create/read/update sequences for warehouses, with a local model oracle.

Capstone 19.3 [mid] — Record–Replay Cassette Library

Objective: Build a minimal VCR for `httpx` and learn where cassettes rot.

Inputs/Datasets: `code\ch19\`; a JSON cassette format you design (request key → recorded status/headers/body).

Steps:

1. Implement a recording transport that writes cassettes and a replaying transport that serves them.
2. Add sanitization hooks (strip `Authorization`, redact fields).
3. Record the robot-registry interactions; replay with the app offline.

4. Mutate the app (change a field) and demonstrate that replay cannot detect it — then add a cassette “freshness date” check.

Acceptance Criteria:

- ☐ Replay works with zero application code.
- ☐ No credential-like header ever reaches disk.
- ☐ A stale cassette is detected and reported by date or hash.

Stretch Goals: Request matching on method+path+canonical body; strict versus lax matching modes.

Capstone 19.4 [senior] — Capacity Report from a Knee Sweep

Objective: Produce a capacity analysis for a service using the harness, defended with Little’s Law.

Inputs/Datasets: `code\ch19\` load harness; a workload app with tunable `work_ms` and `max_concurrent`.

Steps:

1. Predict capacity analytically: c/S , with $\rho \rightarrow 1$ latency shape.
2. Run open-model sweeps at three `work_ms` settings; tabulate knee points; compare with prediction.
3. Cross-check with a closed-model run and $L = \lambda W$.
4. Write the one-page report: knee, recommended provisioning headroom, and the perf budget to encode in CI.

Acceptance Criteria:

- ☐ Predicted and measured knees agree within a factor of two, with the discrepancy explained.
- ☐ The report includes a knee table, a Little’s Law cross-check, and an explicit headroom recommendation.
- ☐ No wall-clock assertion anywhere; numbers labeled illustrative.

Stretch Goals: Add a coordinated-omission buggy scheduler variant and quantify how much it overstates capacity.

Capstone 19.5 [staff] — Quality-Gate Architecture for a Fleet

Objective: Design the gate architecture for a (fictional) ten-service Northwind Robotics fleet.

Inputs/Datasets: The chapter's gate; a written fleet topology you invent (services, consumers, deploy cadences).

Steps:

1. Allocate pyramid layers per service with designated catchers per defect class.
2. Specify the broker topology: pact storage, version tagging, `can-i-deploy` semantics per environment.
3. Budget CI minutes per gate; move deep sweeps and soaks to scheduled jobs with archived evidence.
4. Write the flake policy (quarantine SLA, deletion rule, dashboards) and the advisory-to-blocking rollout plan for contract gates.

Acceptance Criteria:

- ☐ Every defect class in your taxonomy has exactly one designated cheapest catcher.
- ☐ The gate budget fits four minutes per PR with a written justification for each cut.
- ☐ The rollout plan includes measurable adoption metrics (pact coverage of production traffic) and a self-check requirement for every gate.

Stretch Goals: Add mutation testing of the gates themselves (deliberately broken variants) as a scheduled meta-gate.

20.10 Python Dependencies Appendix

```
1 # code\ch19\requirements.txt -- Chapter 19, Python
   3.11+, offline-first
2 pytest>=8.0
3 hypothesis>=6.100
4 fastapi>=0.110
5 httpx>=0.27
6 pydantic>=2.7
7 jsonschema>=4.21
```

Installation (PowerShell, from the track folder):

```
1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install -r code\ch19\requirements.txt
4 pytest code\ch19 -q
```

20.10.1 pytest

Description: The test runner and fixture framework used by every suite in the chapter. **Why included:** Discovery, `tmp_path` fixtures, rich assertion introspection, and exit codes compose into CI gates without glue code. **Alternatives:** `unittest` (stdlib, heavier boilerplate); `nose2` (largely dormant). **Installation:** `pip install "pytest>=8.0"`.

20.10.2 hypothesis

Description: Property-based testing library: strategies, shrinking, and rule-based state machines. **Why included:** Powers Listing 19.4's invariant suites; `derandomize=True` gives deterministic CI replays. **Alternatives:** `schemathesis` (OpenAPI-driven API fuzzing built on Hypothesis — use it when you may add a dependency per spec); hand-rolled generators (Listing 19.5 shows the floor of that path). **Installation:** `pip install "hypothesis>=6.100"`.

20.10.3 fastapi

Description: The ASGI framework implementing the robot registry under test [Ram24]. **Why included:** Real routing, validation, and serialization give component and contract tests something true to verify. **Alternatives:** Starlette (lower level, same ASGI base); Flask (WSGI — needs a different in-process testing story). **Installation:** `pip install "fastapi>=0.110"`.

20.10.4 httpx

Description: HTTP client with both sync and async APIs and, crucially, transport injection [Enc24]. **Why included:** `ASGITransport` runs everything in-process; `MockTransport` underpins stub-based client tests; custom `AsyncBaseTransport` hosts the resilient transport of Listing 19.7. **Alternatives:** `respx` (declarative route mocking on top of `httpx`); `requests` + `responses` (sync-only, older idiom). **Installation:** `pip install "httpx>=0.27"`.

20.10.5 pydantic

Description: Validation and serialization models behind FastAPI. **Why included:** Field constraints generate the 422 boundary our fuzzer explores; `model_json_schema()` is the bridge between code and contract. **Alternatives:** `dataclasses` + manual validation (`stdlib`, no schema generation); `attrs` + `cattrs`. **Installation:** `pip install "pydantic>=2.7"`.

20.10.6 jsonschema

Description: JSON Schema validation for Python. **Why included:** Asserts the “2xx validates against the published schema” invariant independently of the server’s own serializer — the checker must not share the checker’s bugs. **Alternatives:** `pydantic.TypeAdapter` (faster, but shares implementation lineage with the app — weaker independence); `fastjsonschema`. **Installation:** `pip install "jsonschema>=4.21"`.

Notable tools intentionally not required. `schemathesis` (we build its core idea by hand for instruction), `pact-python` (we implement the loop explicitly), `locust` and `k6` (we implement open/closed models in 100 lines), `respx` (`MockTransport` suffices), and `pytest-asyncio` (we call `asyncio.run` directly to keep the dependency surface at six packages).

20.11 Chapter Summary & Key Takeaways

- The test pyramid is portfolio allocation: every defect class gets a *designated cheapest catcher*; E2E stays thin because it is slow, flaky, and diagnostically mute.
- Consumer-driven contracts invert correctness: consumers record what they rely on, providers replay it against real code with minimal named states, and `can-i-deploy` turns verification into a release gate [Pac24]. Schema diffing guards public surfaces; contracts guard known consumers; use both.
- Property-based testing and schema-driven fuzzing buy coverage of input spaces you cannot enumerate; the universal invariants are never-5xx, schema-valid 2xx, RFC 9457 4xx; shrinking turns crashes into bug reports; stateful rules catch sequence bugs.
- Test doubles have precise meanings; prefer fakes for what you own, stubs at third-party edges, sanitized cassettes with freshness dates; never let a mock of someone else’s API be your integration evidence [Enc24].

- Determinism is engineered: injected sleepers and virtual clocks test retries, timeouts, and breakers in microseconds; scripted fault injection makes chaos reproducible in CI.
- Load testing is the search for the knee: open models avoid coordinated omission; Little's Law $L = \lambda W$ connects throughput, latency, and concurrency; provision at 60–70% of the knee; load/stress/soak/spike answer different questions [Nyg18; Bey+16].
- Quality gates are code: hermetic, fast, deterministic, legible, and *self-checking* — every gate in this chapter proves it can fail by running against a deliberately broken variant.

20.12 Quick-Reference Card

Layer	Catches	Tooling (this chapter)
Unit	validation/serialization logic	pytest on pure functions
Component	routing, wiring, error shapes	httpx + ASGI transport (in-process)
Contract	cross-team breakage	pact recorder + verifier, OpenAPI diff
Integration	config, auth, migration drift	real infra in ephemeral env
E2E	emergent whole-system failures	a handful of golden journeys
Property/fuzz	input-space and sequence bugs	Hypothesis, mini OpenAPI fuzzer
Load	capacity, knee, regression	asyncio harness, budgets
Chaos	resilience claims	scripted fault middleware

Table 20.1: Test-layer responsibility matrix.

Fault	Assertion pattern in tests
Latency	virtual clock recorded the delay; no wall-clock assertion
Error 5xx	retried with recorded backoff schedule; breaker trips at threshold
Abort	surfaced as transport error after bounded retries
Partition	failover/fallback engaged; steady state held
Exhaustion	queue bounds respected; load-shedding, not collapse

Table 20.2: Fault taxonomy and how to assert on it deterministically.

Quantity	Rule
Little's Law	$L = \lambda W$; size every pool above λW with burst headroom
Utilization	$\rho = \lambda S/c$; latency $\sim S/(1 - \rho)$ as $\rho \rightarrow 1$
Knee	first sweep level with p99 > 10× baseline p50, or any errors
Provisioning	run production at 60–70% of the knee
Perf in CI	structural assertions on PRs; ratio budgets on quiet scheduled runs

Table 20.3: Little's Law and saturation cheat sheet.

20.13 Standardized Evidence Addendum

20.13.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 20.4: Minimum benchmark table to keep per chapter.

20.13.2 Request Trace Template

- **Request:** one representative API call for this chapter's topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

20.13.3 Cost and Latency Budget Snapshot

20.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0

```

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 20.5: Operational budget template for this chapter’s technique.

```

6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

20.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

20.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 20.6: Decision card to complete per chapter.

20.17 Cross-Chapter Decision Card

20.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

20.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 21

API Lifecycle, Governance & Developer Experience

Every API you have built in this book is immortal until proven otherwise. The moment a consumer integrates against an endpoint — the FastAPI services of Chapter 7, the versioned contracts of Chapter 10, the rate-limited public surfaces of Chapter 15 — you have created an obligation that outlives the enthusiasm that created it. Teams change, roadmaps pivot, funding moves on, but the consumer’s production workload keeps calling. The difference between an API organization and a pile of endpoints is not technical talent; it is *lifecycle management*: knowing which states an API can be in, what evidence is required to move between states, who is allowed to decide, and how the people on the other side of the contract experience every transition.

This chapter treats the API lifecycle as a **formal state machine**: **design** → **review** → **published** (operate) → **deprecated** → **sunset** → **retired**, with explicit entry and exit criteria per state and a small set of legal transitions. On top of that machine we build **design-first governance**: a style guide that standardizes exactly the things consumers can observe (naming, pagination, errors, authentication, versioning), enforced not by review heroics but by linting — Spectral-style rulesets executed as data in CI — plus a design-review process with service-level expectations, API decision records, and the honesty to say that governance is a trade-off between fleet-wide consistency and team autonomy. The resolution of that trade-off is the *paved path*: make the compliant way the easiest way, and measure how many teams choose it voluntarily.

We then turn the lens around to **developer experience (DX) engineering**. A developer portal with a searchable catalog, docs-as-code pipelines that render OpenAPI into reference documentation [Ope21], SDK generation (and a clear-eyed account of when generated SDKs hurt more than they help), deterministic sandbox environments, and above all **time-to-**

first-call (TTFC) — the single activation metric that predicts whether an API will be adopted or abandoned. Because APIs are **products**, we cover consumer research, tiering and pricing models, support models, changelog discipline, and the SLA/SLO/SLI relationship [Bey+16], including the reasoning behind the rule that a contractual SLA must be strictly looser than the internal SLO that defends it. **Deprecation and sunseting at scale** gets a full treatment: discovering who actually calls what from gateway telemetry (including the shadow APIs nobody owns), migration campaigns with communication cadences, kill dates, brownouts, and escape hatches, and the very different policies that apply to internal versus external consumers. Finally we address **regulatory and data governance** (PII boundaries in payloads and logs, retention, audit trails, GDPR-style deletion propagation, classification labels in schemas) and **platform economics** (build-versus-buy, funding models for platform teams [New21], and measuring platform ROI in adoption, TTFC, and incident reduction).

Everything is backed by five complete, offline, deterministic Python 3.11 tools against the synthetic Northwind Robotics universe: a mini Spectral-style linter with rules-as-data and CI exit codes, an OpenAPI → Markdown docs generator, a TTFC funnel analyzer over synthetic onboarding events, a deprecation campaign tracker that prints retirement readiness and the communication schedule, and a catalog manifest generator that emits Backstage-style API entries from a folder of specs.

Key Concept

Governance that lives in documents is a wish; governance that lives in code is a system. A style guide nobody can execute is enforced by the scarcest resource you have — senior reviewer attention — and decays the moment that attention is needed elsewhere. The platform-engineering move is to convert every “should” into a lint rule, a template, a generated artifact, or a default, so that compliance is the path of least resistance and reviewer attention is reserved for the genuinely hard questions: Is this the right API? Should it exist at all? What happens to its consumers when it dies?

21.1 Learning Objectives & Prerequisites

Learning Objectives

After completing this chapter, its lab, and its exercises, you will be able to:

- LO-1.** Model an API’s life as a state machine with named states, legal transitions, and evidence-based entry/exit criteria, and

identify which transition a given organizational event represents.

- LO-2.** Write an API style guide that standardizes consumer-observable surfaces (naming, pagination, errors, authentication, versioning), express it as an executable Spectral-style ruleset with custom rules, and wire it into CI with meaningful exit codes.
- LO-3.** Run a design-review process that respects team autonomy — paved paths, API decision records, review SLAs — and articulate precisely which decisions are centralized and which are not.
- LO-4.** Engineer developer experience as a measurable system: portals and catalogs, docs-as-code, SDK generation policy, sandboxes, and TTFC instrumentation with per-step funnel conversion.
- LO-5.** Operate an API as a product: tiering and pricing, support models, changelog discipline, and the SLA < SLO rule with its error-budget rationale [Bey+16].
- LO-6.** Design and execute a deprecation campaign at scale: consumer discovery from gateway telemetry, communication cadence, brownouts as escape hatches, kill dates, and internal-versus-external policy differences (extending Chapter 10).
- LO-7.** Apply data-governance controls to API ecosystems: PII boundaries, retention, audit trails, deletion propagation, and schema-level classification labels.
- LO-8.** Evaluate platform economics: build-versus-buy for portals and gateways, platform team funding models, and an ROI scorecard in adoption, activation, and reliability terms.

Prerequisites

This chapter assumes you have completed Chapters 0–19 and are comfortable with:

- **Chapter 7** (building APIs with FastAPI) — the services being governed here are the ones you learned to build; **Chapter 10** (versioning & evolution) — deprecation and sunset policy extend the compatibility algebra developed there; **Chapter 20** (testing & quality engineering) — lint gates are another quality gate in the same CI supply chain.

- OpenAPI 3.1 document structure [Ope21] from Chapters 7 and 20: `info`, `paths`, `operations`, `components.schemas`, and extension properties (`x-*`).
- RFC 9457 problem details (Chapter 8), cursor pagination (Chapter 9), and rate-limit/deprecation headers (Chapters 15 and 10), since the style guide standardizes exactly these surfaces.
- SLI/SLO vocabulary and error budgets from Chapter 19 [Bey+16].
- Python 3.11+, PyYAML, and pytest; all code runs offline on Windows/PowerShell with synthetic Northwind Robotics data only.

21.2 Deep Technical Mechanics & Architectural Concepts

21.2.1 The API Lifecycle as a State Machine

An “API lifecycle diagram” drawn as a circle of buzzwords is decoration. A lifecycle you can *operate* is a state machine: a finite set of states, a small alphabet of events, legal transitions, and — the part everyone skips — the **evidence required** to take each transition.

Definition 21.1 (API Lifecycle State). An *API lifecycle state* is a tuple (s, O_s, E_s, X_s) where s is the state name, O_s the set of operational obligations in force while in s (e.g., SLA coverage, on-call, changelog duty), E_s the entry criteria (evidence that must exist to enter), and X_s the exit criteria (evidence required to leave). A *transition* $s \xrightarrow{e} s'$ is an event e fired by a named role and recorded in an audit log.

The seven states we operate at Northwind Robotics are **design**, **review**, **published**, **deprecated**, **sunset**, and **retired** — with “operate” being the normal dwell mode of **published**. The states are not a maturity gradient; an API in **deprecated** is not “worse” than one in **published**, it is *under a different contract*. Each state changes what consumers may assume, what the provider must do, and what the platform automates.

Table 21.1 gives the entry/exit criteria. Two details matter more than the rest. First, **entry into published requires evidence, not opinion**: a clean lint run, a completed design review, an owner in the catalog, and a declared SLO. Second, **deprecated is a first-class operational state with real machinery**: the contract is frozen (no new features, security fixes only), every response carries a **Sunset** header (Chapter 10), and the migration campaign (Section 21.2.5) is running against a public kill date.

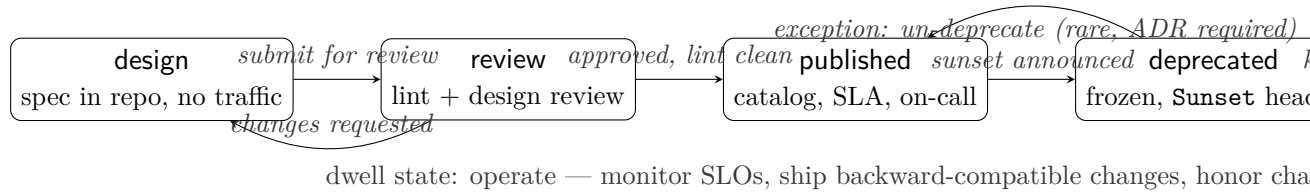


Figure 21.1: The API lifecycle as a state machine. Each transition is an audited event with named preconditions; **published** is the dwell state where normal operation (and only backward-compatible evolution) happens.

State	Entry criteria (evidence)	Exit criteria (evidence)
design	problem statement + ADR approved to proceed	spec in repo; lint clean; review packet complete
review	lint gate green in CI; owner identified	review board sign-off; open questions resolved or ADR'd
published (operate)	catalog entry, rendered docs, declared SLO, on-call rotation, sandbox live	announcement of deprecation date + replacement path
deprecated	deprecation ADR; consumer census from gateway telemetry; Sunset/Deprecation headers live	kill date reached; residual traffic below policy floor
sunset	brownout schedule executed; final warnings sent	post-sunset audit: 410 traffic ≈ 0 for one audit window
retired	audit clean; catalog entry archived; runbooks deleted	terminal state — no exit

Table 21.1: Lifecycle states with evidence-based entry/exit criteria.

The state machine earns its keep in three ways. It makes **ownership answerable at any moment**: every state names an accountable role (in **design** the proposing team; from **published** onward the owning team of record in the catalog). It makes **tooling targetable**: lint gates fire on the **design**→**review** transition, catalog rendering on **review**→**published**, telemetry diffing on **published**→**deprecated**. And it makes **audits trivial**: “which APIs entered **deprecated** without a consumer census?” is a log query, not an inquisition.

21.2.2 Design-First Governance

What a Style Guide Standardizes — and What It Leaves Alone

A style guide that tries to standardize everything standardizes nothing, because teams route around it. The discipline is to standardize only the **consumer-observable seam** and leave internals to the teams. Our canonical list, built up through the book:

Surface	Standard (Northwind Robotics)	Chapter
Path naming	kebab-case segments, plural nouns, no verbs	4
Operation IDs	camelCase, unique, verb+noun (<code>listRobots</code>)	7
Pagination	cursor-based, <code>limit</code> + <code>cursor</code> + <code>nextCursor</code>	9
Errors	RFC 9457 <code>application/problem+json</code> on every 4xx/5xx	8
Authentication	OAuth2 client credentials; scopes named <code>domain:action</code>	17
Versioning	URI major version; <code>info.version</code> strict semver; <code>Sunset</code> header on deprecation	10
Idempotency	<code>Idempotency-Key</code> on all unsafe operations	11
Rate limiting	standard <code>RateLimit-*</code> headers + 429 with <code>Retry-After</code>	15

Table 21.2: The consumer-observable surfaces a style guide must pin down.

Everything not in the table — language, framework, datastore, internal module layout — is deliberately ungoverned. This is the **consistency-versus-autonomy trade-off** made explicit: consistency buys fleet-wide learnability (a developer who knows one Northwind API knows them all) and tooling leverage (one docs renderer, one SDK generator, one linter); autonomy buys team speed and local fit. The mistake is paying for consistency you cannot cash: mandating internal choices produces compliance theater, while under-specifying the seam produces fifty dialects of pagination.

Spectral-Style Linting: Rules as Data

A style guide becomes real when it compiles. Spectral (the open-source OpenAPI linter this section’s code imitates) expresses rules as *data*: a YAML ruleset where each rule names a target (a JSONPath-ish expression over the document), a severity, and a function (truthy, pattern, casing, schema...). The anatomy generalizes:

- **Rule identity and severity.** Stable IDs (`operation-id-camel-case`) so violations can be referenced, suppressed, and trended; severities mapped to CI semantics (`error` fails the build, `warn` does not — yet).
- **Targets.** Where the rule looks: every operation, every path segment, a single pointer like `/info/contact`.
- **Functions.** The check itself: casing, regex match, required-field presence, or a domain predicate like “every operation declares an `application/problem+json` error response”.
- **Messages with JSON paths.** A violation that cannot point at the offending node (`$.paths["/RobotParts"].get`) doubles the cost of fixing it.

Custom rules are where governance encodes *your* history. The rule `pagination-required-on-lists` exists because a 2019 incident (Chapter 9’s war story) traced an outage to an unpaginated collection endpoint. A ruleset is an institutional memory with a `main()` function.

Design Review: Process, Not Tribunal

With lint absorbing the mechanical checks, human review is reserved for judgment: Is the resource model right? Is this operation idempotent? Does this API deserve to exist? A healthy review process has four properties: (1) **review SLA** — feedback within two business days, or the review is waived by default (a slow gate teaches teams to bypass gates); (2) **a written packet** — the spec, the lint report, and a short ADR for each non-obvious decision; (3) **advisory-by-default verdicts** with a short list of blocking conditions (security, data classification, contract-breaking changes to published APIs); (4) **appeals** — a route past the reviewer to an architecture council, used rarely and publicly.

API Decision Records

An ADR for APIs is a one-page document: *context* (forces and constraints), *decision* (what we will do), *status* (proposed/accepted/superseded), *consequences* (what becomes easier and harder), and *alternatives considered*. Two

ADR habits pay outsized dividends in API programs: recording **rejected alternatives** (they are the answers to next year’s “why not GraphQL?” mail; see Chapter 13), and linking ADRs **from the spec** via an **x-decisions** extension so the rationale travels with the contract.

Paved Paths (Golden Paths)

The synthesis of style guide, linter, and review is the **paved path**: a blessed, fully tooled route from idea to published API — repository template with a compliant OpenAPI skeleton, pre-wired CI with the ruleset, generated docs, automatic catalog registration, a sandbox, and dashboards from Chapter 19. Teams may leave the paved path, but then they own the paving. Figure 21.2 shows the flow; the essential property is that *every arrow is automation*, and the humans only appear where judgment is irreducible.

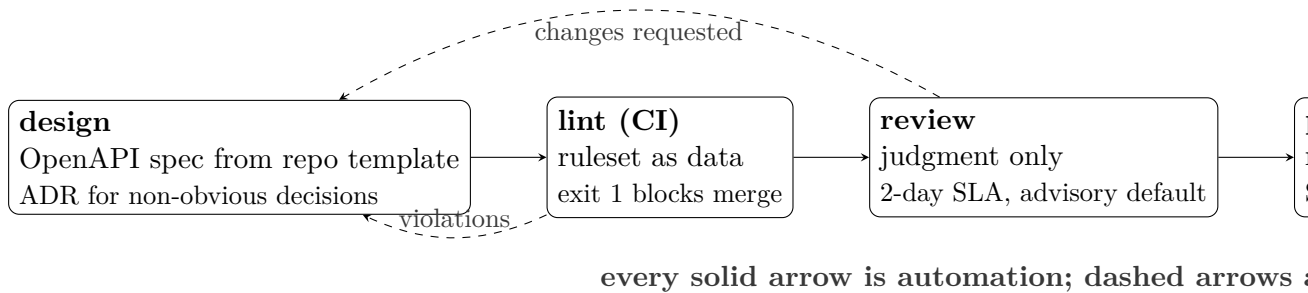


Figure 21.2: The paved path: design → lint → review → publish → portal. Compliance is the default because the compliant route is the one with tooling.

21.2.3 Developer Experience Engineering

Developer experience is not a documentation gloss applied at launch; it is an engineering discipline with instrumentation, funnels, and budgets. The unit of analysis is the *developer journey*: discover → evaluate → activate → integrate → scale. Most API programs instrument only the last two stages, which is why they cannot explain their adoption numbers.

Developer Portals and Catalogs

The portal’s foundation is a **catalog of record**: every API, its owner, its lifecycle state, its spec, and its SLO — Backstage’s **kind**: API entity being the reference shape. A catalog answers the two questions that otherwise consume engineering weeks: *does an API for X already exist?* and *who owns API Y?* (the war story in Section 21.5 is what happens when the

second question has no answer). The catalog is also where the lifecycle state machine becomes visible: filtering by state turns governance from a spreadsheet ritual into a query. Catalog entries must be *generated* from the spec repo (Listing 20.7), never hand-maintained — a hand-maintained catalog is a rumor with a search box.

Docs-as-Code

Docs-as-code means the reference documentation is a *build artifact of the contract*: OpenAPI in, rendered reference out (Redoc, Stoplight Elements, or the stdlib renderer in Listing 20.4), versioned with the spec, deployed by the same pipeline. The invariant worth tattooing: **humans edit the spec; machines edit the docs**. The moment someone hot-fixes the published HTML, docs and contract diverge and trust in both collapses (Pitfall 21.4.7). Guides and tutorials remain hand-written, but they too live in the repo and are reviewed like code.

SDK Generation from OpenAPI

The openapi-generator flow is seductive: one spec, N languages, clients regenerated in CI on every spec change. When it works, it is a superpower — typed models, retry hooks, and auth wiring for free. The caveats are real: generated code is idiomatic in *no* language (Java-flavored Python and Python-flavored Java), generated clients lag generator bugs and language evolution, and an unmaintained SDK is *worse than none* because it carries your logo into its own bit-rot. Our decision rule: **generate SDKs when** the spec is stable, the consumer count justifies maintenance, and regeneration is wired to CI with publish automation; **ship a great OpenAPI document plus a 15-minute quickstart instead** when the API is young, consumers are few, or the team cannot commit to owning the SDK as a product. An interactive sandbox plus copy-paste `httpx` snippets (Chapter 3) covers the long tail at near-zero marginal cost.

Sandbox Environments

A sandbox is a deterministic, self-service environment with synthetic data — the same offline-first discipline this book applies to itself. Its purpose is to compress evaluation time: a developer should be able to make a realistic call within minutes, against data that never changes under their feet (seeded fixtures, fixed clocks where relevant), with no sales call and no approval queue. Sandbox keys are issued instantly and self-service; production keys may carry vetting. A sandbox that requires a ticket is not a sandbox; it is a waiting room with a URL.

TTFC: The Activation Metric

Definition 21.2 (Time to First Call (TTFC)). For developer d , $\text{TTFC}(d) = t_d(\text{first successful 2xx}) - t_d(\text{signup})$, where “successful” means the developer received a *useful* response, not merely a response. Reported fleet-wide as the median: $\text{TTFC} = \text{median}_d \text{TTFC}(d)$.

We instrument TTFC as a funnel — **signup** → **key issued** → **first call** → **first 200** — because the aggregate number hides *where* developers stall. With per-step counts N_i and conversions $C_i = N_i/N_{i-1}$, the funnel localizes the friction: a weak **signup**→**key** step points at key-issuance bureaucracy; weak **key**→**first call** points at documentation and quickstart quality; weak **first call**→**first 200** points at authentication friction and error-message quality (this is where Chapter 8’s **detail** field pays rent). TTFC is *the* activation metric because it is the earliest signal that correlates with long-term adoption: developers who succeed in the first hour almost never churn in the first month. Every DX investment — portal search, quickstarts, sandboxes, SDKs — is justified, and prioritized, by its modeled effect on one funnel step. Listing 20.5 computes the funnel from synthetic event logs.

API Analytics for Product Decisions

Beyond TTFC, the analytics that change roadmap decisions are: **endpoint adoption** (which operations are used at all — candidates for deprecation are born here), **error mix per consumer** (a consumer whose 4xx rate spikes after a release is a support case you can open *before* the email arrives), **version distribution** (Chapter 10’s migration telemetry), and **top consumers by revenue and by support cost** — the ratio of which is the most honest pricing input you own. The ethical boundary: analytics observe *traffic*, not payload content, unless the consumer contract says otherwise (Section 21.2.6).

21.2.4 API as a Product

Consumer Research and Tiering

API product management begins with consumer research that is boringly concrete: interview ten consumers, read their integration code, and count their workarounds. Pricing and packaging then follow the value metric — the unit that scales with the consumer’s success (calls, seats, robots managed). A canonical three-tier structure: **free** (sandbox plus low-rate production, community support, best-effort availability — the adoption engine), **usage-based** (pay per unit of the value metric, business-hours support, contractual SLA), and **enterprise** (committed volumes, dedicated support channel, custom SLOs, contractual sunset protection). The tier

boundary that matters most technically is rate limits (Chapter 15): tiers must be enforceable at the gateway, or the pricing page is fiction.

SLA, SLO, SLI — and Why $SLA < SLO$

Recall the stack from Chapter 19 [Bey+16]: an **SLI** is a measurement (fraction of requests succeeding in under 300 ms), an **SLO** is an internal target on that SLI (99.9% monthly), and an **SLA** is the contractual promise with penalties (99.5%, service credits below). The invariant is $SLA < SLO$, and the rationale is the error budget. If the internal objective is tighter than the external promise, the gap $1 - SLO$ versus $1 - SLA$ is a buffer in which the team can *detect, decide, and act* — page humans, roll back, freeze deploys — before any contractual breach. With monthly SLI p measured against an SLA floor p_{SLA} , budget remaining is $(p - p_{SLA}) / (1 - p_{SLA})$; it must still be positive when the SLO budget $(p - p_{SLO}) / (1 - p_{SLO})$ is exhausted, which holds iff $SLA < SLO$. An SLA equal to or tighter than the SLO means customers discover your incidents before your error-budget policy does. (The reversed mistake is Pitfall 21.4.5.)

Support Models and Changelog Discipline

Support scales by self-service: docs, a searchable issue history, status page, and only then humans — community forum for free tier, ticketed queue for paid, named contacts for enterprise. Changelog discipline is the cheapest trust instrument in the product: every consumer-visible change, classified (added/changed/deprecated/removed/fixed/security), dated, and linked from the **Deprecation** machinery of Chapter 10. A changelog that lags the deploy train teaches consumers to distrust every other signal you send.

Developer Relations as a Feedback Loop

DevRel closes the loop between consumers and governance: funnel analytics and support tickets flow into style-guide and paved-path evolution; governance changes flow out through changelog, portal, and migration guides. Figure 21.3 shows the loop. The failure mode to guard against is DevRel as *megaphone* — outbound only — which produces beautiful launches of APIs nobody can integrate.

21.2.5 Deprecation & Sunsetting at Scale

Chapter 10 taught the mechanics of signaling change (**Deprecation** and **Sunset** headers, version negotiation); this section is about the *campaign*: retiring a version that hundreds of consumers depend on, without breaking their production or your reputation.

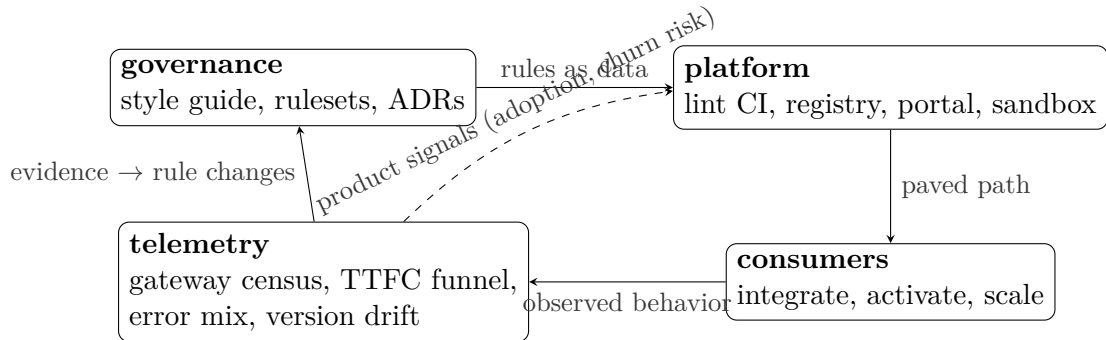


Figure 21.3: The governance feedback loop: standards flow to consumers through tooling; observed consumer behavior flows back as evidence that revises the standards.

Consumer Discovery: Who Uses What?

You cannot migrate consumers you cannot name. The discovery order is: (1) **gateway telemetry** — join API keys/client IDs to consumer identities and aggregate calls per (consumer, version, endpoint); this is ground truth because packets do not lie; (2) **the catalog of record** — the declared mapping, useful for owners and escalation contacts; (3) **contract registries** from Chapter 20. The gap between (1) and (2) is where **shadow consumers** live: integrations nobody declared, often built by departed employees, and — worse — *shadow APIs*: provider endpoints serving real traffic with no catalog entry at all. Both are found by diffing observed traffic against declared registrations, which is why consumer discovery is a telemetry problem before it is a process problem.

Migration Campaigns

A campaign is a dated communication plan with a hard end state, run like a release. The skeleton (implemented in Listing 20.6): **T-180** announce (changelog, portal banner, email to registered owners, **Deprecation** header on); **T-90/T-60** reminders to non-migrated consumers, escalating to their engineering leads; **T-30** first **brownout** — a short, announced window where the old version returns 410/404, the escape hatch that flushes out consumers who never read email; **T-14** final warning with sponsor escalation; **T-7** second, longer brownout; **T-0** permanent sunset (HTTP 410 with a problem+json body pointing at the migration guide); **T+7** post-sunset audit of residual traffic. Escape hatches: a *documented* extension process (time-boxed, signed by the consumer’s director, carrying an explicit new kill date) and a per-consumer allowlist so a laggard can be carved out of a

brownout without delaying the campaign for everyone else.

The Organizational Cost of Breaking Changes

Breaking changes externalize cost onto every consumer simultaneously: each must schedule, implement, test, and deploy a migration — work with zero feature value to them. The total cost of a $v1 \rightarrow v2$ break scales with consumer count, which is why the calculus of Chapter 10 (expand–migrate–contract, parallel change) is not politeness but economics: every backward-compatible expansion you ship instead of a break is N consumer migrations you did not force. Budget for the campaign itself too: provider-side migration tooling, shims, and the support surge around brownouts are real engineering weeks.

Internal versus External Policy

Policy lever	Internal consumers	External consumers
Notice period	60–90 days; sprint-plannable	6–12 months; often contractual
Enforcement	platform can mandate via CI/deploy gates	persuasion, pricing, brownouts; contract terms govern
Escape hatch	CTO-approved exception with expiry	paid extended-support tier, time-boxed
Discovery	catalog + telemetry + org chart	telemetry + contracts; no org chart to consult
Sunset mechanics	hard kill dates, brownouts acceptable	brownouts announced per contract; 410 body with migration link mandatory

Table 21.3: Deprecation policy: internal versus external consumers.

21.2.6 Regulatory & Data Governance

API governance that ignores data governance is a compliance incident on a timer. Five controls, each cheap at design time and brutal retrofitted:

1. **PII boundaries in payloads and logs.** Classify every schema field; payloads carry only what the consumer’s purpose requires (data minimization), and logs capture metadata — never payload bodies by default. Chapter 19’s structured logging must enforce redaction lists, and Chapter 17’s scopes should be classification-aware.

2. **Classification labels in schemas.** Machine-readable labels such as `x-classification: pii|internal|public` on schema properties (used by our specs and surfaced by the docs generator in Listing 20.4) let linters fail builds that log, export, or widen access to classified fields.
3. **Retention policies.** Every persisted representation has a retention clock and an owner; API audit logs, analytics events, and backups each get explicit, possibly different, windows.
4. **Audit trails.** Lifecycle transitions (who moved this API to `deprecated`, on what evidence), admin operations, and data-access events are append-only, timestamped, and attributable.
5. **Deletion propagation.** A GDPR-style erasure request is a *workflow across the API ecosystem*, not a row delete: primary store, derived stores, analytics events, caches (Chapter 11), backups (handled by expiry, not surgical edits), and downstream consumers you pushed data to via async APIs (Chapter 12) — each acknowledged and auditable. Designing erasure as an event with tombstones and fan-out receipts, rehearsed quarterly, beats discovering your propagation gaps during a regulator’s deadline.

21.2.7 Platform Economics

Build versus buy. Buy (or adopt managed) when the capability is undifferentiated and mature — gateways, key management, hosted docs. Build when the capability encodes *your* conventions — the ruleset, the paved-path templates, the catalog enrichment that knows your org chart. The composite answer is usual: a managed gateway, Backstage-style open-source portal core, and thin bespoke glue (like this chapter’s manifest generator) that stamps your conventions onto commodity infrastructure.

Funding models. Platform teams fail when funded as ticket-taking cost centers; they succeed as *product teams with internal customers* [New21] — funded on a multi-year horizon, with adoption as their success metric and voluntary uptake as the forcing function (a platform that must mandate usage is a platform whose paved path is not paved well enough).

Measuring ROI. The honest scorecard has three columns: *adoption* (APIs onboarded to the paved path, share of traffic behind the gateway), *activation* (TTFC median and funnel conversion, trended), and *risk removed* (lint-blocked non-compliant designs, incidents attributed to governed versus ungoverned APIs, sunset campaigns completed without escalation). Report it quarterly; a platform that cannot show its value eventually meets someone who cannot see it.

21.3 Production-Ready Code Implementation

All listings live under `code\ch20\` and together form a *mini platform*: lint a spec, generate its docs, publish its catalog entry, measure onboarding, and run a deprecation campaign. Everything is offline, deterministic (the tracker takes `-today` explicitly; the funnel reads a fixed synthetic fixture), and uses the Northwind Robotics synthetic universe only. Install once:

```
1 cd code\ch20
2 pip install -r requirements.txt
3 python -m pytest -q
```

21.3.1 Listing 20.1 — The Ruleset as Data

The style guide of Table 21.2 compiles into a YAML ruleset. Six rules cover contact ownership, semver versioning, path and operation naming, RFC 9457 error coverage, and pagination on collection endpoints. Adding a rule is a *data* change, reviewed in a pull request like any other — governance as pull request, not as meeting.

```
1 # Northwind Robotics API style ruleset (rules as data).
2 # Each rule: id, severity (error|warn), kind, and kind-
   specific params.
3 rules:
4   - id: info-contact-required
5     severity: error
6     kind: require-field
7     pointer: /info/contact
8     message: info.contact must name an owning team or
       alias.
9
10  - id: semver-info-version
11    severity: error
12    kind: match
13    pointer: /info/version
14    pattern: '^\\d+\\.\\d+\\.\\d+$'
15    message: info.version must be strict semver (MAJOR.
       MINOR.PATCH).
16
17  - id: path-segment-kebab-case
18    severity: error
19    kind: casing
20    target: path_segment
21    style: kebab
```

```

22     message: Path segments must be kebab-case.
23
24 - id: operation-id-camel-case
25   severity: error
26   kind: casing
27   target: operation_id
28   style: camel
29   message: operationId must be camelCase.
30
31 - id: problem-json-errors
32   severity: error
33   kind: problem-json-errors
34   message: >-
35     Every operation must declare a 4xx/5xx or default
36     response carrying
37     application/problem+json (RFC 9457).
38
39 - id: pagination-required-on-lists
40   severity: warn
41   kind: pagination-required
42   params: [limit, cursor]
43   message: >-
44     Collection GET endpoints must accept limit and
45     cursor query
46     parameters.

```

21.3.2 Listing 20.2 — mini_spectral.py: A Spectral-Style Linter

The linter loads the spec and ruleset, evaluates each rule *kind* (*require-field*, *match*, *casing*, *problem-json-errors*, *pagination-required*), and reports violations with JSON paths. The exit code is the CI contract: 0 clean, 1 error-severity violations, 2 usage or parse failure — so a pipeline can distinguish “your API is non-compliant” from “the gate itself broke”.

```

1 r"""mini_spectral.py -- a miniature Spectral-style
   OpenAPI linter.
2
3 Loads an OpenAPI YAML document and a ruleset expressed
   as data(YAML),
4 evaluates every rule, reports violations with JSON
   paths, and returns a
5 CI-friendly exit code:
6
7 0 -- no error-severity violations

```

```

8 11111111--at least one error-severity violation
9 11111222--usage error, unreadable file, or malformed
   document/ruleset
10
11 Usage(PowerShell):
12 11111python mini_spectral.py .\specs\northwind_robots.
   yaml--ruleset.\ruleset.yaml
13 ""
14
15 from __future__ import annotations
16
17 import argparse
18 import re
19 import sys
20 from dataclasses import dataclass
21 from pathlib import Path
22 from typing import Any, Callable
23
24 import yaml
25
26 HTTP_METHODS = frozenset(
27     {"get", "put", "post", "delete", "options", "head",
28      "patch", "trace"}
29 )
30
31 CASING_PATTERNS: dict[str, re.Pattern[str]] = {
32     "kebab": re.compile(r"^[a-z0-9]+(-[a-z0-9]+)*$"),
33     "snake": re.compile(r"^[a-z0-9]+(_[a-z0-9]+)*$"),
34     "camel": re.compile(r"^[a-z][a-zA-Z0-9]*$"),
35 }
36
37 SEVERITIES = ("error", "warn")
38
39 @dataclass(frozen=True)
40 class Violation:
41     """A single rule breach located by a JSON-path-ish
42     string."""
43
44     rule_id: str
45     severity: str
46     path: str
47     message: str
48
49     def render(self) -> str:
50         return (

```

```

50         f"[{self.severity.upper():5}]_{self.rule_id}"
51         f"at_{self.path}:_{self.message}"
52     )
53
54
55 #
56
57 # Document navigation helpers
58 #
59
60 def resolve_pointer(doc: Any, pointer: str) -> Any:
61     """Resolve a minimal JSON Pointer ('/info/contact')
62     ; None if absent."""
63     node = doc
64     for token in pointer.strip("/").split("/"):
65         if not isinstance(node, dict) or token not in node:
66             return None
67         node = node[token]
68     return node
69
70 def json_path(*parts: str) -> str:
71     """Build a readable JSON path, quoting keys that
72     contain slashes."""
73     out = "$"
74     for part in parts:
75         if re.fullmatch(r"[A-Za-z0-9_-]+", part):
76             out += f".{part}"
77         else:
78             out += f'["{part}"]'
79     return out
80
81 def iter_operations(doc: dict[str, Any]):
82     """Yield (json_path, path_key, method, operation)
83     for every operation."""
84     paths = doc.get("paths") or {}
85     if not isinstance(paths, dict):
86         return
87     for path_key, path_item in paths.items():
88         if not isinstance(path_item, dict):

```

```

87         continue
88     for method, operation in path_item.items():
89         if method.lower() in HTTP_METHODS and
90            isinstance(operation, dict):
91             yield (
92                 json_path("paths", path_key, method
93                     .lower()),
94                 path_key,
95                 method.lower(),
96                 operation,
97             )
98
99 # -----
100 # Rule kinds -- each takes (doc, rule) and yields
101 # Violations
102 # -----
103
104 def check_require_field(
105     doc: dict[str, Any], rule: dict[str, Any]
106 ) -> list[Violation]:
107     pointer = str(rule["pointer"])
108     if resolve_pointer(doc, pointer) is None:
109         return [
110             Violation(
111                 rule["id"],
112                 rule["severity"],
113                 "$" + pointer.replace("/", "."),
114                 rule["message"],
115             )
116         ]
117     return []
118
119 def check_match(doc: dict[str, Any], rule: dict[str,
120     Any]) -> list[Violation]:
121     pointer = str(rule["pointer"])
122     value = resolve_pointer(doc, pointer)
123     pattern = re.compile(str(rule["pattern"]))
124     if value is None or not pattern.fullmatch(str(value)):
125         return [

```

```

124         Violation(
125             rule["id"],
126             rule["severity"],
127             "$" + pointer.replace("/", "."),
128             f"{rule['message']}_{found:_{value!r}} "
129         ),
130     ]
131     return []
132
133
134 def check_casing(doc: dict[str, Any], rule: dict[str,
135 Any]) -> list[Violation]:
136     style = str(rule["style"])
137     pattern = CASING_PATTERNS[style]
138     violations: list[Violation] = []
139
140     if rule["target"] == "operation_id":
141         for op_path, _pk, _m, operation in
142             iter_operations(doc):
143                 op_id = operation.get("operationId")
144                 if op_id is None or not pattern.fullmatch(
145                     str(op_id)):
146                     violations.append(
147                         Violation(
148                             rule["id"],
149                             rule["severity"],
150                             f"{op_path}.operationId",
151                             f"{rule['message']}_{found:_{
152                                 op_id!r}} ",
153                         )
154                     )
155
156     elif rule["target"] == "path_segment":
157         paths = doc.get("paths") or {}
158         for path_key in paths:
159             for segment in str(path_key).split("/"):
160                 if not segment or (segment.startswith("
161                     {") and segment.endswith("}")):
162                     continue
163                 if not pattern.fullmatch(segment):
164                     violations.append(
165                         Violation(
166                             rule["id"],
167                             rule["severity"],
168                             json_path("paths", path_key
169                                     ),

```



```

163         f"{rule['message']}\n(
164             segment:\n{segment!r})\"",
165     )
166     else: # defensive: ruleset is data too, and data
167           can be wrong
168         raise ValueError(f"unknown_casing_target:\n{rule
169           ['target']!r}")
170     return violations
171
172 def check_problem_json_errors(
173     doc: dict[str, Any], rule: dict[str, Any]
174 ) -> list[Violation]:
175     violations = []
176     for op_path, _pk, _m, operation in iter_operations(
177         doc):
178         responses = operation.get("responses") or {}
179         ok = False
180         for status, response in responses.items():
181             code = str(status)
182             if not (code.startswith(("4", "5")) or code
183                 == "default"):
184                 continue
185             content = (response or {}).get("content")
186             or {}
187             if "application/problem+json" in content:
188                 ok = True
189                 break
190         if not ok:
191             violations.append(
192                 Violation(
193                     rule["id"],
194                     rule["severity"],
195                     f"{op_path}.responses",
196                     rule["message"],
197                 )
198             )
199     return violations
200
201 def check_pagination_required(
202     doc: dict[str, Any], rule: dict[str, Any]
203 ) -> list[Violation]:
204     required_params = [str(p) for p in rule.get("params
205         ", ["limit", "cursor"])]

```

```

202     violations = []
203     for op_path, path_key, method, operation in
204         iter_operations(doc):
205         is_collection_get = method == "get" and not str
206             (path_key).rstrip("/").endswith("}")
207         if not is_collection_get:
208             continue
209         present = {
210             str(p.get("name"))
211             for p in operation.get("parameters") or []
212             if isinstance(p, dict) and p.get("in") == "
213                 query"
214         }
215         missing = [p for p in required_params if p not
216             in present]
217         if missing:
218             violations.append(
219                 Violation(
220                     rule["id"],
221                     rule["severity"],
222                     f"{op_path}.parameters",
223                     f"{rule['message']}_{(missing:_{', _
224                         '.join(missing)})}",
225                 )
226             )
227     return violations
228
229
230 RULE_KINDS: dict[
231     str, Callable[[dict[str, Any], dict[str, Any]],
232         list[Violation]]
233 ] = {
234     "require-field": check_require_field,
235     "match": check_match,
236     "casing": check_casing,
237     "problem-json-errors": check_problem_json_errors,
238     "pagination-required": check_pagination_required,
239 }
240
241 #
242
243 # Engine
244 #

```

```

239
240 def load_yaml_file(path: Path) -> Any:
241     with path.open("r", encoding="utf-8") as handle:
242         return yaml.safe_load(handle)
243
244
245 def lint(spec: dict[str, Any], ruleset: dict[str, Any])
246     -> list[Violation]:
247     """Evaluate every rule in the ruleset against the
248     spec document."""
249     if not isinstance(spec, dict) or "openapi" not in
250     spec:
251         raise ValueError("document does not look like
252         an OpenAPI spec")
253     rules = (ruleset or {}).get("rules") or []
254     violations: list[Violation] = []
255     for rule in rules:
256         kind = rule.get("kind")
257         handler = RULE_KINDS.get(str(kind))
258         if handler is None:
259             raise ValueError(f"unknown rule kind: {kind
260             !r}")
261         if rule.get("severity") not in SEVERITIES:
262             raise ValueError(f"rule {rule.get('id')!r}:
263             bad severity")
264         violations.extend(handler(spec, rule))
265     return violations
266
267
268 def main(argv: list[str] | None = None) -> int:
269     parser = argparse.ArgumentParser(description="Mini
270     Spectral-style linter.")
271     parser.add_argument("spec", type=Path, help="
272     OpenAPI YAML file")
273     parser.add_argument(
274         "--ruleset", type=Path, required=True, help="
275         Ruleset YAML file"
276     )
277     args = parser.parse_args(argv)
278
279     try:
280         spec = load_yaml_file(args.spec)
281         ruleset = load_yaml_file(args.ruleset)
282     except (OSError, yaml.YAMLError) as exc:
283         print(f"error: cannot load input: {exc}", file=

```

```

275         sys.stderr)
276     return 2
277
278     try:
279         violations = lint(spec, ruleset)
280     except ValueError as exc:
281         print(f"error: {exc}", file=sys.stderr)
282         return 2
283
284     for violation in violations:
285         print(violation.render())
286
287     errors = sum(1 for v in violations if v.severity ==
288                 "error")
289     warnings = sum(1 for v in violations if v.severity
290                  == "warn")
291     print(f"\n{args.spec.name}: {errors} error(s), {
292           warnings} warning(s)")
293     return 1 if errors else 0
294
295 if __name__ == "__main__":
296     raise SystemExit(main())

```

The deliberately non-compliant legacy warehouse spec (`fixtures\noncompliant_spec.yaml`) trips five of the six rules:

```

1 openapi: 3.1.0
2 info:
3   title: Northwind Robotics Warehouse Legacy API
4   version: v2-final # NOT semver: violates semver-
5     info-version
6   description: Legacy warehouse endpoints kept alive
7     for one partner.
8   # no contact block: violates info-contact-required
9 paths:
10   /RobotParts: # PascalCase segment: violates
11     path-segment-kebab-case
12   get:
13     operationId: list_parts # snake_case: violates
14     operation-id-camel-case
15     summary: List all parts without pagination
16     responses:
17       '200':
18         description: Every part in one giant payload
19         content:
20           application/json:

```

```

17         schema:
18             type: array
19             items:
20                 $ref: '#/components/schemas/Part'
21     '500':
22         description: Boom, plain text
23         content:
24             text/plain:
25                 schema:
26                     type: string
27 /robot-parts/{partId}:
28     delete:
29         operationId: deletePart
30         summary: Delete a part
31         parameters:
32             - name: partId
33               in: path
34               required: true
35               schema:
36                   type: string
37     responses:
38         '204':
39             description: Deleted
40         '404':
41             description: Not found, bare JSON
42             content:
43                 application/json:
44                     schema:
45                         type: object
46 components:
47     schemas:
48         Part:
49             type: object
50             properties:
51                 sku:
52                     type: string
53                 bin:
54                     type: string

```

Running the linter against it produces one violation per breach, each pinpointed by JSON path, and exit code 1:

```

1 [ERROR] info-contact-required at $.info.contact: info.
   contact must name an owning team or alias.
2 [ERROR] semver-info-version at $.info.version: info.
   version must be strict semver (MAJOR.MINOR.PATCH). (
   found: 'v2-final')

```

```

3 [ERROR] path-segment-kebab-case at $.paths["/RobotParts
   ": Path segments must be kebab-case. (segment: '
   RobotParts')
4 [ERROR] operation-id-camel-case at $.paths["/RobotParts
   "].get.operationId: operationId must be camelCase. (
   found: 'list_parts')
5 [ERROR] problem-json-errors at $.paths["/RobotParts"].
   get.responses: Every operation must declare a 4xx/5
   xx or default response carrying application/problem+
   json (RFC 9457).
6 [ERROR] problem-json-errors at $.paths["/robot-parts/{
   partId}"].delete.responses: Every operation must
   declare a 4xx/5xx or default response carrying
   application/problem+json (RFC 9457).
7 [WARN ] pagination-required-on-lists at $.paths["/
   RobotParts"].get.parameters: Collection GET
   endpoints must accept limit and cursor query
   parameters. (missing: limit, cursor)
8
9 noncompliant_spec.yaml: 6 error(s), 1 warning(s)

```

The compliant fleet spec passes clean (exit 0):

```

1
2 northwind_robots.yaml: 0 error(s), 0 warning(s)

```

21.3.3 Listing 20.3 — Linter Tests, Including the Non-Compliant Spec

The test suite treats the exit-code contract as first-class behavior: the compliant spec exits 0, the non-compliant one exits 1 with every expected rule ID in the output, a missing file exits 2, and warn-severity violations provably do not fail the build — the property that lets you roll out new rules as warnings before flipping them to errors.

```

1 """Tests for mini_spectral.py, including a deliberately
   non-compliant spec."""
2
3 from __future__ import annotations
4
5 from pathlib import Path
6
7 import pytest
8 import yaml
9
10 import mini_spectral as ms

```

```

11
12 HERE = Path(__file__).parent
13 RULESET = HERE / "ruleset.yaml"
14 COMPLIANT = HERE / "specs" / "northwind_robots.yaml"
15 NONCOMPLIANT = HERE / "fixtures" / "noncompliant_spec.
    yaml"
16
17
18 def load(path: Path) -> dict:
19     return yaml.safe_load(path.read_text(encoding="utf
        -8"))
20
21
22 #
    -----
23 # End-to-end CLI behaviour (exit codes are the CI
    contract)
24 #
    -----
25
26 def test_compliant_spec_exits_zero(capsys: pytest.
    CaptureFixture[str]) -> None:
27     code = ms.main([str(COMPLIANT), "--ruleset", str(
        RULESET)])
28     out = capsys.readouterr().out
29     assert code == 0
30     assert "0_error(s)" in out
31
32
33 def test_noncompliant_spec_exits_one_and_names_rules(
34     capsys: pytest.CaptureFixture[str],
35 ) -> None:
36     code = ms.main([str(NONCOMPLIANT), "--ruleset", str(
        RULESET)])
37     out = capsys.readouterr().out
38     assert code == 1
39     for rule_id in (
40         "info-contact-required",
41         "semver-info-version",
42         "path-segment-kebab-case",
43         "operation-id-camel-case",
44         "problem-json-errors",
45         "pagination-required-on-lists",
46     ):

```

```
47         assert rule_id in out
48     # violations carry JSON paths
49     assert '$.paths["/RobotParts"].get.operationId' in
50         out
51     assert "6␣error(s),␣1␣warning(s)" in out
52
53 def test_missing_file_exits_two(capsys: pytest.
54     CaptureFixture[str]) -> None:
55     code = ms.main([str(HERE / "nope.yaml"), "--ruleset
56         ", str(RULESET)])
57     assert code == 2
58     assert "cannot␣load␣input" in capsys.readouterr().
59         err
60
61 def test_not_an_openapi_doc_exits_two(
62     tmp_path: Path, capsys: pytest.CaptureFixture[str]
63 ) -> None:
64     bogus = tmp_path / "bogus.yaml"
65     bogus.write_text("just:␣a␣mapping\n", encoding="utf
66         -8")
67     code = ms.main([str(bogus), "--ruleset", str(
68         RULESET)])
69     assert code == 2
70     assert "OpenAPI" in capsys.readouterr().err
71
72 #
73 -----
74
75 # Unit behaviour of individual rule kinds
76 #
77 -----
78
79 def test_casing_patterns() -> None:
80     assert ms.CASING_PATTERNS["kebab"].fullmatch("robot
81         -parts")
82     assert not ms.CASING_PATTERNS["kebab"].fullmatch("
83         RobotParts")
84     assert ms.CASING_PATTERNS["camel"].fullmatch("
85         listRobots")
86     assert not ms.CASING_PATTERNS["camel"].fullmatch("
87         list_parts")
```



```

79
80 def test_resolve_pointer_missing() -> None:
81     assert ms.resolve_pointer({"info": {}}, "/info/
      contact") is None
82     assert ms.resolve_pointer({"info": {"contact":
      {}}, "/info/contact") == {}
83
84
85 def
      test_template_path_params_are_not_linted_as_segments
      () -> None:
86     doc = load(COMPLIANT)
87     ruleset = {
88         "rules": [
89             {
90                 "id": "path-segment-kebab-case",
91                 "severity": "error",
92                 "kind": "casing",
93                 "target": "path_segment",
94                 "style": "kebab",
95                 "message": "kebab␣only",
96             }
97         ]
98     }
99     assert ms.lint(doc, ruleset) == []
100
101
102 def test_unknown_rule_kind_is_a_ruleset_error() -> None
      :
103     doc = load(COMPLIANT)
104     with pytest.raises(ValueError, match="unknown␣rule␣
      kind"):
105         ms.lint(doc, {"rules": [{"id": "x", "severity":
      "error",
106                                     "kind": "telepathy"
      }]])
107
108
109 def test_warn_severity_does_not_fail_build(
110     capsys: pytest.CaptureFixture[str],
111 ) -> None:
112     # The compliant spec trips the pagination rule if
      we demand a
113     # non-existent parameter -- warnings must not
      change the exit code.
114     ruleset_text = RULESET.read_text(encoding="utf-8").

```

```

115         replace(
            "params: [limit, cursor]", "params: [limit, cursor, locale]"
116     )
117     violations = ms.lint(load(COMPLIANT), yaml.safe_load(ruleset_text))
118     assert violations and all(v.severity == "warn" for v in violations)

```

21.3.4 Listing 20.4 — docs_generator.py: OpenAPI → Markdown

Docs-as-code in eighty lines of stdlib-plus-PyYAML: endpoints with parameter and response tables, the first response example as a fenced JSON block, and schema tables that surface `x-classification` labels so data governance is visible in the published reference. Because the docs are generated, they cannot drift from the spec without the pipeline noticing.

```

1  r"""docs_generator.py -- render an OpenAPI YAML spec as
   Markdown reference docs.
2
3  Stdlib+PyYAML only. Docs-as-code in miniature: the spec
   is the single
4  source of truth and the rendered reference is a build
   artifact, so docs
5  can never drift from the contract without the build
   noticing.
6
7  Usage (PowerShell):
8  python docs_generator.py .\specs\northwind_robots.
   yaml
9  python docs_generator.py .\specs\northwind_robots.
   yaml --out fleet-api.md
10 """
11
12  from __future__ import annotations
13
14  import argparse
15  import json
16  import sys
17  from pathlib import Path
18  from typing import Any
19
20  import yaml
21

```

```

22 HTTP_METHODS = ("get", "put", "post", "delete", "patch"
23                 , "options", "head")
24
25 def schema_type(schema: dict[str, Any]) -> str:
26     """Human-readable type label, resolving local $refs
27     to their name."""
28     if "$ref" in schema:
29         return str(schema["$ref"]).rsplit("/", 1)[-1]
30     base = schema.get("type", "any")
31     if base == "array":
32         return f"array[{schema_type(schema.get('items'))
33                        or {} }]"
34     if "enum" in schema:
35         return "enum(" + ", ".join(str(v) for v in
36                                     schema["enum"]) + ")"
37     if "format" in schema:
38         return f"{base}_{schema['format']}"
39     return str(base)
40
41 def render_parameters(operation: dict[str, Any]) ->
42     list[str]:
43     params = operation.get("parameters") or []
44     if not params:
45         return []
46     lines = [
47         "",
48         "|_Name_|_In_|_Required_|_Type_|",
49         "|---|---|---|---|",
50     ]
51     for param in params:
52         schema = param.get("schema") or {}
53         lines.append(
54             f"|_{param.get('name', '?')}|_{param.get(
55                 'in', '?')}|_{
56                 f{'yes' if param.get('required') else 'no'}
57                 }_|_"
58             f"{schema_type(schema)}_|_"
59         )
60     return lines
61
62 def render_responses(operation: dict[str, Any]) -> list
63     [str]:
64     responses = operation.get("responses") or {}

```

```

60     lines = ["", "|_Status_|_Meaning_|_Body_|", "
        |---|---|---|"]
61     for status, response in responses.items():
62         content = (response or {}).get("content") or {}
63         bodies = ",_".join(
64             f"{media}_->_{schema_type((cfg_or_{}).get('
                schema')_or_{}))}"
65             for media, cfg in content.items()
66         ) or "-"
67         lines.append(
68             f"|_{status}_|_{(response_or_{}).get('
                description',_''')}_|"
69             f"|_{bodies}_|"
70         )
71     return lines
72
73
74 def render_example(operation: dict[str, Any]) -> list[
    str]:
75     """Emit the first response example found, as a
        fenced JSON block."""
76     for _status, response in (operation.get("responses"
        ) or {}).items():
77         for _media, cfg in ((response or {}).get("
            content") or {}).items():
78             if isinstance(cfg, dict) and "example" in
                cfg:
79                 return [
80                     "",
81                     "Example:",
82                     "",
83                     "```json",
84                     json.dumps(cfg["example"], indent
                        =2),
85                     "```",
86                 ]
87     return []
88
89
90 def render_markdown(spec: dict[str, Any]) -> str:
91     info = spec.get("info") or {}
92     lines: list[str] = [
93         f"#_{info.get('title',_ 'Untitled API')}",
94         "",
95         f"**Version:**_{info.get('version',_ 'n/a')}",
96         "",

```

```

97         (info.get("description") or "").strip(),
98         "",
99     ]
100     for server in spec.get("servers") or []:
101         lines.append(f"-_Server:_{{server.get('url')}}_
102                     f"({server.get('description', '_')}
103                        })")
104     lines.append("")
105     lines.append("##_Endpoints")
106
107     paths = spec.get("paths") or {}
108     for path_key, path_item in paths.items():
109         for method in HTTP_METHODS:
110             operation = (path_item or {}).get(method)
111             if not isinstance(operation, dict):
112                 continue
113             lines += [
114                 "",
115                 f"###_{{method.upper()}}_{{path_key}}",
116                 "",
117                 operation.get("summary", ""),
118             ]
119             lines += render_parameters(operation)
120             lines += render_responses(operation)
121             lines += render_example(operation)
122             lines.append("")
123
124     schemas = ((spec.get("components") or {}).get("
125                schemas")) or {}
126     if schemas:
127         lines.append("##_Schemas")
128         for name, schema in schemas.items():
129             required = set(schema.get("required") or [])
130             lines += [
131                 "",
132                 f"###_{{name}}",
133                 "",
134                 "|_Property_|_Type_|_Required_|_Notes_|",
135                 "|---|---|---|---|",
136             ]
137             for prop, prop_schema in (schema.get("
138                properties") or {}).items():

```

```

139         + (
140             f"[classification:_{prop_schema['x-  
classification']}]"
141         )
142         lines.append(
143             f"|_{prop}'_{schema_type(prop_schema  
)}_|_"
144             f"{'yes' if prop in required else 'no'}  
_|_{notes}|"
145         )
146         lines.append("")
147         return "\n".join(lines)
148
149 def main(argv: list[str] | None = None) -> int:
150     parser = argparse.ArgumentParser(description="
151         OpenAPI->Markdown docs.")
152     parser.add_argument("spec", type=Path, help="
153         OpenAPI YAML file")
154     parser.add_argument("--out", type=Path, default=
155         None,
156         help="Write Markdown here (
157             default: stdout)")
158     args = parser.parse_args(argv)
159
160     try:
161         spec = yaml.safe_load(args.spec.read_text(
162             encoding="utf-8"))
163     except (OSError, yaml.YAMLError) as exc:
164         print(f"error: cannot load spec: {exc}", file=
165             sys.stderr)
166         return 2
167     if not isinstance(spec, dict) or "openapi" not in
168         spec:
169         print("error: not an OpenAPI document", file=
170             sys.stderr)
171         return 2
172
173     markdown = render_markdown(spec)
174     if args.out:
175         args.out.write_text(markdown, encoding="utf-8")
176         print(f"wrote {args.out} ({len(markdown)} chars
177             ")
178     else:
179         print(markdown)
180     return 0

```

```

172
173
174 if __name__ == "__main__":
175     raise SystemExit(main())

```

Demo output (truncated to the first endpoint; the full document covers all operations and schemas):

```

1 # Northwind Robotics Fleet API
2
3 **Version:** 2.4.0
4
5 Manages the Northwind Robotics synthetic robot fleet:
6     registration, health telemetry summaries, and
7     decommissioning workflows.
8
9 - Server: 'https://api.northwind-robotics.example/fleet
10    /v2' (Production (synthetic))
11
12 ## Endpoints
13
14 ### 'GET /robots'
15
16 List robots in the fleet
17
18 | Name | In | Required | Type |
19 |---|---|---|---|
20 | 'limit' | query | no | integer |
21 | 'cursor' | query | no | string |
22
23 | Status | Meaning | Body |
24 |---|---|---|
25 | '200' | A page of robots | application/json ->
26     RobotPage |
27 | 'default' | Unexpected error | application/problem+
28     json -> Problem |
29
30 Example:
31
32 ```json
33 {
34     "items": [
35         {
36             (...)

```

21.3.5 Listing 20.5 — `ttfc_funnel.py`: Activation Analytics

The funnel analyzer reads a JSONL event log (one synthetic onboarding event per line), reconstructs per-developer journeys, and reports the ordered funnel with per-step and cumulative conversion plus median inter-step gaps. It is defensive by design: malformed lines are counted and skipped, non-funnel events (like `docs_viewed`) are ignored, and a developer is only counted at a step if all prior steps are present — out-of-order telemetry cannot fabricate conversions. The fixture (head shown below) holds 222 events for 60 synthetic developers:

```

1 {"ts": "2026-03-02T09:07:00Z", "developer_id": "dev
   -001", "event": "signup", "channel": "docs-site"}
2 {"ts": "2026-03-02T09:09:00Z", "developer_id": "dev
   -001", "event": "docs_viewed", "page": "quickstart"}
3 {"ts": "2026-03-02T09:14:00Z", "developer_id": "dev
   -002", "event": "signup", "channel": "docs-site"}
4 {"ts": "2026-03-02T09:16:00Z", "developer_id": "dev
   -002", "event": "docs_viewed", "page": "quickstart"}
5 {"ts": "2026-03-02T09:21:00Z", "developer_id": "dev
   -003", "event": "signup", "channel": "conference"}
6 {"ts": "2026-03-02T09:23:00Z", "developer_id": "dev
   -003", "event": "docs_viewed", "page": "quickstart"}
7 {"ts": "2026-03-02T09:28:00Z", "developer_id": "dev
   -004", "event": "signup", "channel": "docs-site"}
8 {"ts": "2026-03-02T09:30:00Z", "developer_id": "dev
   -004", "event": "docs_viewed", "page": "quickstart"}
9 (...)

```

```

1 r"""ttfc_funnel.py--onboarding_funnel_and_time-to-
   first-call_analytics.
2
3 Reads_a_JSONL_event_log(one_synthetic_onboarding_event
   per_line)and
4 computes_the_activation_funnel
5
6 signup->key_issued->first_call->first_200
7
8 with_per-step_absolute_counts,step-to-step_conversion,
   and_median
9 elapsed_minutes_between_steps.TTFC_is_reported_as_the
   median_time_from
10 signup_to_first_200--the_first_moment_a_developer
   received_a_useful*
11 response.

```



```

12
13 Usage_(PowerShell):
14 python_ttfc_funnel.py.\fixtures\onboarding_events.
    jsonl
15 """
16
17 from __future__ import annotations
18
19 import argparse
20 import json
21 import statistics
22 import sys
23 from dataclasses import dataclass, field
24 from datetime import datetime
25 from pathlib import Path
26
27 FUNNEL_STEPS = ("signup", "key_issued", "first_call", "
    first_200")
28
29
30 @dataclass
31 class DeveloperJourney:
32     """First occurrence of each funnel step for one
        developer."""
33
34     first_seen: dict[str, datetime] = field(
35         default_factory=dict)
36
37 def parse_ts(raw: str) -> datetime:
38     return datetime.strptime(raw, "%Y-%m-%dT%H:%M:%SZ")
39
40
41 def load_journeys(path: Path) -> tuple[dict[str,
    DeveloperJourney], int]:
42     """Build per-developer journeys; returns (journeys,
        skipped_lines)."""
43     journeys: dict[str, DeveloperJourney] = {}
44     skipped = 0
45     with path.open("r", encoding="utf-8") as handle:
46         for line in handle:
47             line = line.strip()
48             if not line:
49                 continue
50             try:
51                 record = json.loads(line)

```

```

52         dev = str(record["developer_id"])
53         event = str(record["event"])
54         ts = parse_ts(str(record["ts"]))
55     except (ValueError, KeyError, TypeError):
56         skipped += 1 # never let one bad line
57         # kill the report
58         continue
59     if event not in FUNNEL_STEPS:
60         continue # telemetry like docs_viewed
61         # is not a funnel step
62     journey = journeys.setdefault(dev,
63                                   DeveloperJourney())
64     journey.first_seen.setdefault(event, ts)
65     return journeys, skipped
66
67 def reached(journeys: dict[str, DeveloperJourney], step
68 : str) -> set[str]:
69     """Developers who completed this step AND every
70     step before it."""
71     index = FUNNEL_STEPS.index(step)
72     required = FUNNEL_STEPS[: index + 1]
73     return {
74         dev
75         for dev, journey in journeys.items()
76         if all(s in journey.first_seen for s in
77             required)
78     }
79
80 def median_gap_minutes(
81     journeys: dict[str, DeveloperJourney], start: str,
82     end: str
83 ) -> float | None:
84     gaps = [
85         (j.first_seen[end] - j.first_seen[start]).
86         total_seconds() / 60.0
87         for j in journeys.values()
88         if start in j.first_seen and end in j.
89             first_seen
90     ]
91     return statistics.median(gaps) if gaps else None
92
93 def build_report(journeys: dict[str, DeveloperJourney])
94     -> list[str]:

```

```

88     lines = [
89         "Onboarding_funnel(unique_developers, ordered_
90             steps)",
91         "=" * 68,
92         f"{'step':<14}{'reached':>9}{'step_conv
93             .':>12}{'cum_conv.':>12}"
94         f"{'median_gap(min)':>18}",
95         "-" * 68,
96     ]
97     total = len(reached(journeys, FUNNEL_STEPS[0])) or
98         1 # avoid div-by-zero
99     previous_count: int | None = None
100     previous_step: str | None = None
101     for step_name in FUNNEL_STEPS:
102         count = len(reached(journeys, step_name))
103         step_conv = (
104             100.0 * count / previous_count
105             if previous_count
106             else 100.0
107         )
108         gap = (
109             median_gap_minutes(journeys, previous_step,
110                 step_name)
111             if previous_step
112             else None
113         )
114         lines.append(
115             f"{'step_name':<14}{'count':>9}{'step_conv:>11.1
116                 f}%"
117             f"{'100.0*count/total:>11.1f}%"
118             f"({'n/a' if gap is None else f'{gap:.0f}'}
119                 :>18)"
120         )
121         previous_count, previous_step = count,
122             step_name
123
124     ttfc = median_gap_minutes(journeys, "signup", "
125         first_200")
126     lines += [
127         "-" * 68,
128         f"TTFC(median_signup->first_200):_"
129         f"({'n/a' if ttfc is None else f'{ttfc:.0f}'_
130             minutes'})_"
131         f"({' ' if ttfc is None else f'{ttfc/60:.1f}'_
132             h'})_"
133     ]

```

```

124     return lines
125
126
127 def main(argv: list[str] | None = None) -> int:
128     parser = argparse.ArgumentParser(description="TTFC_
129         funnel_analytics.")
130     parser.add_argument("events", type=Path, help="
131         JSONL_onboarding_events")
132     args = parser.parse_args(argv)
133
134     try:
135         journeys, skipped = load_journeys(args.events)
136     except OSError as exc:
137         print(f"error: cannot read events: {exc}", file
138             =sys.stderr)
139         return 2
140
141     if not journeys:
142         print("error: no usable events found", file=sys
143             .stderr)
144         return 2
145
146     for line in build_report(journeys):
147         print(line)
148     if skipped:
149         print(f"note: skipped {skipped} malformed line(
150             s)", file=sys.stderr)
151     return 0
152
153 if __name__ == "__main__":
154     raise SystemExit(main())

```

The printed funnel table is the artifact you put in front of the DX review board:

1	Onboarding funnel (unique developers, ordered steps)					
2	=====					
3	step	reached	step conv.	cum. conv.	median	
4	gap (min)					

5	signup	60	100.0%	100.0%		
		n/a				
6	key_issued	45	75.0%	75.0%		
		69				
7	first_call	33	73.3%	55.0%		

		194		
8	first_200	24	72.7%	40.0%
		93		
9	-----			
10	TTFC (median signup -> first_200): 431 minutes (7.2 h)			

Read it as a product manager: 75% of signups obtain a key in a median 69 minutes, but only 73% of those make a first call, and the key→call gap (194 minutes median) is the largest in the funnel — the quickstart, not key issuance, is where Northwind Robotics loses developers. One instrumentation subtlety: **first call** and **first 200** are separate steps because an 11% slice of first calls in this fixture return 401 (expired sandbox keys), which is exactly the kind of friction TTFC exists to expose.

21.3.6 Listing 20.6 — `deprecation_tracker.py`: Campaign Management

The tracker models a campaign as a registry — API, version, replacement, announce and kill dates, and a consumer census with per-consumer status, traffic, and notes — and renders two artifacts: the retirement-readiness report (residual traffic, blockers, verdict) and the full communication schedule derived from the kill date. The exit code is 1 while any non-migrated consumer still has traffic, so the retirement runbook can be gated in CI. The registry (note the last entry — a shadow integration found by gateway telemetry, with no owner in the catalog):

```

1 # Synthetic deprecation campaign registry for the
   legacy Fleet API v1.
2 api: fleet
3 version: v1
4 replacement: v2
5 announce_date: "2026-03-02"
6 kill_date: "2026-09-01"    # hard sunset: version
   returns HTTP 410 after this
7 consumers:
8   - name: fleet-dashboard
9     owner: team-telemetry
10    tier: internal
11    status: migrated
12    weekly_calls: 0
13    last_contact: "2026-05-04"
14    notes: Cut over to v2 in sprint 12.
15   - name: maintenance-planner
16     owner: team-operations

```

```

17     tier: internal
18     status: in_progress
19     weekly_calls: 8200
20     last_contact: "2026-06-29"
21     notes: Blocked on v2 bulk endpoint; ETA 2026-08-01.
22 - name: partner-robologix
23   owner: external:robologix
24   tier: external
25   status: in_progress
26   weekly_calls: 41000
27   last_contact: "2026-06-21"
28   notes: Contracted migration window ends 2026-08-15.
29 - name: warehouse-scanner
30   owner: team-warehouse
31   tier: internal
32   status: not_started
33   weekly_calls: 1300
34   last_contact: "2026-04-10"
35   notes: Team dissolved; ownership transferred to
        team-fleet-platform.
36 - name: legacy-billing-hook
37   owner: unknown
38   tier: internal
39   status: unresponsive
40   weekly_calls: 95
41   last_contact: null
42   notes: Discovered via gateway telemetry; no owner
        in catalog. Shadow API!

```

```

1  r"""deprecation_tracker.py -- retirement readiness +
    communication schedule.
2
3  Reads a deprecation campaign registry (YAML) and
    produces:
4
5  1. a retirement-readiness report (per consumer, plus
    traffic-weighted
6     migration percentage), and
7  2. the communication schedule relative to the kill
    date
8     (announcement, reminders, brownouts, sunset, post-
    sunset audit).
9
10 The report is deterministic: the evaluation date is
    supplied explicitly
11 (--today) so CI output and this book's examples never

```

```

    drift.
12
13 Usage_(PowerShell):
14 python_deprecation_tracker.py.\fixtures\
    deprecation_registry.yaml --today 2026-07-15
15 """
16
17 from __future__ import annotations
18
19 import argparse
20 import sys
21 from dataclasses import dataclass
22 from datetime import date, timedelta
23 from pathlib import Path
24 from typing import Any
25
26 import yaml
27
28 # Statuses that block retirement even at zero reported
    traffic.
29 ACTIVE_STATUSES = ("in_progress", "not_started")
30
31 # Communication cadence, as offsets (in days) from the
    kill date.
32 # Brownouts (scheduled short 410 windows) are the
    escape hatch that flushes
33 # out consumers who ignore email -- see Chapters 9 and
    14.
34 CADENCE: tuple[tuple[int, str, str], ...] = (
35     (-180, "announcement", "all_consumers_email_and_
        changelog_portal_banner"),
36     (-90, "reminder", "all_consumers_not_yet_migrated"),
37     (-60, "reminder", "non-migrated_consumers_and_their_
        owning_teams_leads"),
38     (-30, "brownout_#1", "1-hour_410_window_business_
        hours_with_notice"),
39     (-14, "final_warning", "non-migrated_consumers_and_
        escalation_to_sponsors"),
40     (-7, "brownout_#2", "4-hour_410_window_with_notice"
        ),
41     (0, "sunset", "permanent_HTTP_410_and_final_changelog_
        entry"),
42     (7, "post-sunset_audit", "gateway_log_sweep_for_
        residual_410_traffic"),
43 )

```

```

44
45
46 @dataclass(frozen=True)
47 class Consumer:
48     name: str
49     owner: str
50     tier: str
51     status: str
52     weekly_calls: int
53     notes: str = ""
54
55
56 def load_registry(path: Path) -> dict[str, Any]:
57     data = yaml.safe_load(path.read_text(encoding="utf
58         -8"))
59     if not isinstance(data, dict) or "consumers" not in
60         data:
61         raise ValueError("registry must contain a
62             consumers' list")
63     return data
64
65
66 def parse_consumers(registry: dict[str, Any]) -> list[
67     Consumer]:
68     consumers = []
69     for raw in registry["consumers"]:
70         consumers.append(
71             Consumer(
72                 name=str(raw["name"]),
73                 owner=str(raw.get("owner", "unknown")),
74                 tier=str(raw.get("tier", "internal")),
75                 status=str(raw.get("status", "
76                     not_started")),
77                 weekly_calls=int(raw.get("weekly_calls"
78                     , 0)),
79                 notes=str(raw.get("notes", "")),
80             )
81         )
82     return consumers
83
84
85 def readiness_report(
86     registry: dict[str, Any], consumers: list[Consumer
87         ], today: date
88 ) -> list[str]:
89     kill = date.fromisoformat(str(registry["kill_date"]

```



```

    )))
83 # Migrated consumers no longer call v1, so the
    meaningful readiness
84 # metric is residual traffic: calls still arriving
    from consumers that
85 # have NOT finished migrating.
86 residual = [c for c in consumers if c.status != "
    migrated"]
87 residual_calls = sum(c.weekly_calls for c in
    residual)
88
89 lines = [
90     f"Retirement_readiness:_{registry['api']}_{
        registry['version']}"
91     f"(replacement:_{registry.get('replacement','n
        /a')})",
92     "-" * 72,
93     f"today:_{today.isoformat()}_{kill_date:_{kill
        .isoformat()}"
94     f"days_remaining:_{(kill-_today).days}",
95     "",
96     f"{'consumer':<22}{tier':<10}{status':<14}{
        calls/wk':>10}_{owner}",
97     "-" * 72,
98 ]
99 for c in sorted(consumers, key=lambda c: -c.
    weekly_calls):
100     lines.append(
101         f"{c.name:<22}{c.tier:<10}{c.status:<14}{c.
            weekly_calls:>10}_{
102             f"{c.owner}"
103         )
104     lines += [
105         "-" * 72,
106         f"residual_v1_traffic:_{residual_calls}_{calls/
            wk_across}"
107         f"{len(residual)}_non-migrated_consumer(s)",
108         "",
109     ]
110
111 blockers = [
112     c for c in consumers
113     if c.status in ACTIVE_STATUSES
114     or (c.status == "unresponsive" and c.
        weekly_calls > 0)
115 ]

```

```

116     if blockers:
117         lines.append("verdict:_NOT_READY_TO_RETIRE_--_
            blockers:")
118         for c in blockers:
119             lines.append(f"_{c.name}_{c.status},_{c}
                .weekly_calls_"
                f"calls/wk):_{c.notes}")
120
121     else:
122         lines.append("verdict:_READY_TO_RETIRE_--_no_
            live_traffic_on_"
            "non-migrated_consumers.")
123
124     return lines
125
126
127 def schedule_report(registry: dict[str, Any]) -> list[
    str]:
128     kill = date.fromisoformat(str(registry["kill_date"
        ]))
129     lines = [
130         "",
131         f"Communication_schedule_(kill_date_{kill.
            isoformat()})",
132         "-" * 72,
133         f"{'date':<12}{'T-minus':>9} _ _action_/ _audience
            ",
134         "-" * 72,
135     ]
136     for offset, action, audience in CADENCE:
137         when = kill + timedelta(days=offset)
138         label = f"T{offset:+d}" if offset else "T-0"
139         lines.append(f"{{when.isoformat():<12}}{label:>9}
            _ _{{action}}:_{{audience}}")
140     return lines
141
142
143 def main(argv: list[str] | None = None) -> int:
144     parser = argparse.ArgumentParser(description="
        Deprecation_campaign_tracker.")
145     parser.add_argument("registry", type=Path, help="
        campaign_registry_YAML")
146     parser.add_argument(
147         "--today",
148         required=True,
149         help="evaluation_date,_ISO_format_(kept_
            explicit_for_determinism)",
150     )

```

```

151     args = parser.parse_args(argv)
152
153     try:
154         today = date.fromisoformat(args.today)
155     except ValueError:
156         print("error: --today must be ISO format, e.g. 2026-07-15",
157               file=sys.stderr)
158         return 2
159     try:
160         registry = load_registry(args.registry)
161         consumers = parse_consumers(registry)
162     except (OSError, ValueError, KeyError, yaml.YAMLError) as exc:
163         print(f"error: cannot parse registry: {exc}",
164               file=sys.stderr)
165         return 2
166
167     for line in readiness_report(registry, consumers, today):
168         print(line)
169     for line in schedule_report(registry):
170         print(line)
171
172     # Non-zero exit while blocked so CI can gate the retirement runbook.
173     blocked = any(
174         c.status in ACTIVE_STATUSES
175         or (c.status == "unresponsive" and c.weekly_calls > 0)
176         for c in consumers
177     )
178     return 1 if blocked else 0
179
180 if __name__ == "__main__":
181     raise SystemExit(main())

```

```

1 Retirement readiness: fleet v1 (replacement: v2)

```

```

2 =====

```

```

3 today: 2026-07-15    kill date: 2026-09-01    days
4     remaining: 48

```

```

5 consumer            tier        status        calls/
6     wk  owner

```

```

6 -----

```

```

7 partner-robologix      external  in_progress
  41000 external:robologix
8 maintenance-planner   internal  in_progress
  8200 team-operations
9 warehouse-scanner      internal  not_started
  1300 team-warehouse
10 legacy-billing-hook    internal  unresponsive
  95  unknown
11 fleet-dashboard        internal  migrated
  0   team-telemetry
12 -----

13 residual v1 traffic: 50595 calls/wk across 4 non-
  migrated consumer(s)
14
15 verdict: NOT READY TO RETIRE -- blockers:
16 - maintenance-planner (in_progress, 8200 calls/wk):
  Blocked on v2 bulk endpoint; ETA 2026-08-01.
17 - partner-robologix (in_progress, 41000 calls/wk):
  Contracted migration window ends 2026-08-15.
18 - warehouse-scanner (not_started, 1300 calls/wk):
  Team dissolved; ownership transferred to team-
  fleet-platform.
19 - legacy-billing-hook (unresponsive, 95 calls/wk):
  Discovered via gateway telemetry; no owner in
  catalog. Shadow API!
20
21 Communication schedule (kill date 2026-09-01)
22 -----

23 date          T-minus  action / audience
24 -----

25 2026-03-05      T-180  announcement: all consumers:
  email + changelog + portal banner
26 2026-06-03      T-90   reminder: all consumers not yet
  migrated
27 2026-07-03      T-60   reminder: non-migrated consumers
  + their owning teams' leads
28 2026-08-02      T-30   brownout #1: 1-hour 410 window,
  business hours, with notice
29 2026-08-18      T-14   final warning: non-migrated
  consumers, escalation to sponsors
30 2026-08-25      T-7    brownout #2: 4-hour 410 window
  with notice

```

```

31 2026-09-01          T-0  sunset: permanent HTTP 410 +
    final changelog entry
32 2026-09-08          T+7  post-sunset audit: gateway log
    sweep for residual 410 traffic

```

21.3.7 Listing 20.7 — `catalog_manifest.py`: Backstage-Style Entries

The manifest generator scans a directory of OpenAPI specs and emits one catalog document per API, pulling lifecycle state and owner from the spec's `info.x-lifecycle` block so the catalog is generated from the contract of record rather than curated by hand. Non-OpenAPI files in the folder are skipped with a warning, not a crash.

```

1  r"""catalog_manifest.py -- generate Backstage-style API
   catalog entries.
2
3  Scans a directory of OpenAPI YAML specs and emits one
   catalog manifest
4  document per spec (apiVersion backstage.io/v1alpha1,
   kind: API), with
5  ownership and lifecycle read from the spec's info.x-
   lifecycle block so
6  the catalog, like the docs, is generated from the
   contract rather than
7  maintained by hand.
8
9  Usage (PowerShell):
10 python catalog_manifest.py .\specs
11 python catalog_manifest.py .\specs --out catalog-
   apis.yaml
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import re
18 import sys
19 from pathlib import Path
20 from typing import Any
21
22 import yaml
23
24 DEFAULT_OWNER = "platform-team"
25 DEFAULT_LIFECYCLE = "production"

```

```
26
27
28 def slugify(title: str) -> str:
29     slug = re.sub(r"[^a-z0-9]+", "-", title.lower()).
        strip("-")
30     return slug or "unnamed-api"
31
32
33 def manifest_for(spec_path: Path, spec: dict[str, Any],
34                 root: Path) -> dict[str, Any]:
35     info = spec.get("info") or {}
36     lifecycle_block = info.get("x-lifecycle") or {}
37     rel_path = spec_path.relative_to(root.parent).
        as_posix()
38     return {
39         "apiVersion": "backstage.io/v1alpha1",
40         "kind": "API",
41         "metadata": {
42             "name": slugify(str(info.get("title",
43             spec_path.stem))),
44             "description": "␣".join(
45                 str(info.get("description", "")).split
46                 ()
47                 )[:200],
48             "annotations": {
49                 "northwind/spec-version": str(info.get(
50                 "version", "0.0.0")),
51             },
52         },
53         "spec": {
54             "type": "openapi",
55             "lifecycle": str(lifecycle_block.get("state",
56             DEFAULT_LIFECYCLE)),
57             "owner": str(lifecycle_block.get("owner",
58             DEFAULT_OWNER)),
59             "definition": {"$text": f"./{rel_path}"},
60         },
61     }
62
63 def scan(spec_dir: Path) -> tuple[list[dict[str, Any]],
64     list[str]]:
65     manifests: list[dict[str, Any]] = []
66     warnings: list[str] = []
67     candidates = sorted(
68         [*spec_dir.glob("*.yaml"), *spec_dir.glob("*.yml")]
```

```

        yaml"")]
63     )
64     if not candidates:
65         warnings.append(f"no .yaml/.yml files found in
            {spec_dir}")
66     for spec_path in candidates:
67         try:
68             spec = yaml.safe_load(spec_path.read_text(
                encoding="utf-8"))
69         except yaml.YAMLError as exc:
70             warnings.append(f"{spec_path.name}: YAML
                parse error: {exc}")
71             continue
72         if not isinstance(spec, dict) or "openapi" not
            in spec:
73             warnings.append(f"{spec_path.name}: not an
                OpenAPI doc, skipped")
74             continue
75         manifests.append(manifest_for(spec_path, spec,
            spec_dir))
76     return manifests, warnings
77
78
79 def main(argv: list[str] | None = None) -> int:
80     parser = argparse.ArgumentParser(
81         description="Emit Backstage-style API manifests
            for a spec folder."
82     )
83     parser.add_argument("spec_dir", type=Path, help="
        directory of OpenAPI YAML")
84     parser.add_argument("--out", type=Path, default=
        None,
85                         help="write manifests here (
                            default: stdout)")
86     args = parser.parse_args(argv)
87
88     if not args.spec_dir.is_dir():
89         print(f"error: not a directory: {args.spec_dir}
            ", file=sys.stderr)
90         return 2
91
92     manifests, warnings = scan(args.spec_dir)
93     for warning in warnings:
94         print(f"warning: {warning}", file=sys.stderr)
95
96     text = "---\n" + "---\n".join(

```

```
97         yaml.safe_dump(m, sort_keys=False) for m in
           manifests
98     )
99     if args.out:
100         args.out.write_text(text, encoding="utf-8")
101         print(f"wrote {len(manifests)} manifest(s) to {
           args.out}")
102     else:
103         print(text, end="")
104     return 0 if manifests else 1
105
106
107 if __name__ == "__main__":
108     raise SystemExit(main())
```

```
1  ---
2  apiVersion: backstage.io/v1alpha1
3  kind: API
4  metadata:
5    name: northwind-robotics-fleet-api
6    description: 'Manages the Northwind Robotics
7      synthetic robot fleet: registration,
8      health telemetry summaries, and decommissioning
9      workflows.'
10   annotations:
11     northwind/spec-version: 2.4.0
12 spec:
13   type: openapi
14   lifecycle: published
15   owner: team-fleet-platform
16   definition:
17     $text: ./specs/northwind_robots.yaml
18 ---
19 apiVersion: backstage.io/v1alpha1
20 kind: API
21 metadata:
22   name: northwind-robotics-telemetry-api
23   description: Batch ingestion and querying of
24     synthetic robot telemetry streams (motor
25     current, battery voltage, wheel odometry).
26   annotations:
27     northwind/spec-version: 1.1.0
28 spec:
29   type: openapi
30   lifecycle: published
31   owner: team-telemetry
```



```

29     definition:
30         $text: ./specs/northwind_telemetry.yaml

```

21.3.8 Listing 20.8 — Smoke Tests for the Platform Tools

A compact suite pins the cross-tool behaviors the lab relies on: docs render classification labels, the funnel counts 60 developers and computes the median TTFC of the fixture, malformed event lines are tolerated, the tracker flags the shadow consumer and renders the T-0 sunset, and the catalog scan emits one entry per spec.

```

1  """Smoke tests for the docs, funnel, deprecation, and
   catalog tools."""
2
3  from __future__ import annotations
4
5  from datetime import date
6  from pathlib import Path
7
8  import pytest
9  import yaml
10
11  import catalog_manifest
12  import deprecation_tracker
13  import docs_generator
14  import ttfc_funnel
15
16  HERE = Path(__file__).parent
17
18
19  def test_docs_generator_renders_endpoints_and_schemas()
20      -> None:
21      spec = yaml.safe_load(
22          (HERE / "specs" / "northwind_robots.yaml").
23              read_text(encoding="utf-8")
24      )
25      md = docs_generator.render_markdown(spec)
26      assert "# Northwind Robotics Fleet API" in md
27      assert "### 'GET /robots'" in md
28      assert "application/problem+json-> Problem" in md
29      assert "[classification: internal]" in md # data
   classification surfaced

```

```

30 def test_funnel_counts_and_ttfc() -> None:
31     journeys, skipped = ttfc_funnel.load_journeys(
32         HERE / "fixtures" / "onboarding_events.jsonl"
33     )
34     assert skipped == 0
35     assert len(journeys) == 60
36     assert len(ttfc_funnel.reached(journeys, "first_200
37         ")) == 24
38     ttfc = ttfc_funnel.median_gap_minutes(journeys, "
39         signup", "first_200")
40     assert ttfc == pytest.approx(431.0)
41
42 def test_funnel_survives_malformed_lines(tmp_path: Path
43 ) -> None:
44     bad = tmp_path / "events.jsonl"
45     bad.write_text(
46         '{ "ts": "2026-03-02T09:00:00Z", "developer_id":
47         "d1",
48         "event": "signup"}\nnot json at all\n',
49         encoding="utf-8",
50     )
51     journeys, skipped = ttfc_funnel.load_journeys(bad)
52     assert skipped == 1 and len(journeys) == 1
53
54 def test_deprecation_tracker_blocked_and_schedule() ->
55     None:
56     registry = deprecation_tracker.load_registry(
57         HERE / "fixtures" / "deprecation_registry.yaml"
58     )
59     consumers = deprecation_tracker.parse_consumers(
60         registry)
61     report = "\n".join(
62         deprecation_tracker.readiness_report(
63             registry, consumers, today=date(2026, 7,
64             15)
65         )
66     )
67     assert "NOT_READY_TO_RETIRE" in report
68     assert "legacy-billing-hook" in report # the
69         shadow API shows up
70     schedule = "\n".join(deprecation_tracker.
71         schedule_report(registry))
72     assert "T-0" in schedule and "brownout_#2" in
73         schedule

```

```

66
67
68 def test_catalog_manifest_scans_both_specs() -> None:
69     manifests, warnings = catalog_manifest.scan(HERE /
70         "specs")
71     assert warnings == []
72     names = {m["metadata"]["name"] for m in manifests}
73     assert names == {
74         "northwind-robotics-fleet-api",
75         "northwind-robotics-telemetry-api",
76     }
77     assert all(m["kind"] == "API" for m in manifests)
78     assert manifests[0]["spec"]["owner"].startswith("
    team-")

```

Full suite:

```

1 .....
   [100%]
2 14 passed in 0.59s

```

21.4 Common Pitfalls & Anti-Patterns

21.4.1 Governance as Gatekeeping Theater

A review board that meets fortnightly, demands slide decks, and rules on naming by taste is not governance; it is theater with a queue. Symptoms: review wait times measured in weeks, “shadow reviews” in hallway conversations, teams shipping unreviewed APIs and begging forgiveness. The fix is to move everything mechanical into lint (advisory first, blocking later), cap review scope at judgment calls, and give the review itself an SLA — a gate slower than the bypass will be bypassed.

21.4.2 Style Guide Without Linter Enforcement

A wiki page of conventions decays the day it is published; enforcement falls on senior reviewers, who become bottlenecks and then, inevitably, inconsistent ones — rule application becomes a function of which reviewer you drew. If a rule cannot be expressed as executable data, the honest options are to drop it or to reclassify it as advisory guidance. Rules-as-data plus CI exit codes is the only enforcement mechanism that scales past a handful of teams.

21.4.3 The Portal Nobody Can Search

A portal whose catalog is hand-curated decays into a museum: half the entries are stale, search ranks marketing pages above reference docs, and developers learn to ask in chat instead — after which the portal team declares “developers don’t use portals.” Generate catalog entries from the spec repo (Listing 20.7), render docs from the spec (Listing 20.4), and instrument search queries with zero results; an unsearchable portal is worse than none because it teaches developers to stop looking.

21.4.4 SDKs Promised but Unmaintained

Announcing “SDKs for seven languages” at launch is a mortgage: every spec change, generator bug, and language release comes due, in public, under your brand. An unmaintained SDK with stale models teaches consumers that your API cannot be trusted — worse than no SDK, because the failure arrives wearing your logo. Generate only what CI regenerates and publishes automatically, and sunset SDKs with the same campaign discipline as APIs.

21.4.5 SLA Looser Than SLO, Reversed

Promising a 99.95% SLA while engineering to a 99.9% SLO means the contract runs out of budget before the team does: customers hit breach before your alerting policy fires, and every incident becomes a legal event. The invariant is $SLA < SLO$, with the gap sized as reaction time in error-budget terms [Bey+16]. If sales demands $SLA \geq SLO$, the correct response is to raise the SLO *and the engineering investment*, not to sign the number.

21.4.6 Sunsetting Without Telemetry

Announcing a kill date from a spreadsheet of “known consumers” retires only the consumers you already knew about. The first brownout then pages a team nobody mapped to an API key — or worse, a production workload whose owner left the company two reorganizations ago. Consumer discovery *must* start from gateway telemetry; the registry of declared consumers is the escalation directory, not the census. No census, no kill date.

21.4.7 Docs Drifting From Spec

Hand-maintained reference docs diverge from the contract on the first hotfix, and every divergence costs a consumer hours and costs you trust. Docs-as-code (generate from OpenAPI, deploy from CI) makes drift a build failure instead of a slow betrayal; the linter can even enforce that every operation carries a `summary` so the generated docs are never vacuous.

Warning

The compound anti-pattern is **governance without telemetry**: style rules nobody measures, a portal nobody instruments, deprecations announced to a census nobody verified. Each pitfall above is survivable alone; together they produce the organization that discovers its real API surface during an incident. If you fund only two things from this chapter, fund the gateway consumer census and the TTFC funnel — they are the instruments every other control depends on.

21.5 Hands-On Production Lab & Real-World Case Study

21.5.1 Lab: Stand Up the Mini Platform

Work from `code\ch20\` in PowerShell. Expected duration: 60–90 minutes.

1. **Install and baseline.** `pip install -r requirements.txt`, then `python -m pytest -q` — all 14 tests pass.
2. **Lint the fleet.** Run `mini_spectral.py` against both specs in `specs\` and the non-compliant fixture; confirm exit codes 0, 0, 1 (use `$LASTEXITCODE`). Identify which rule catches the most violations.
3. **Author a rule.** Add a warn-severity rule `summary-required` to `ruleset.yaml` (hint: new rule kind not needed — extend `check_casing`-style iteration with a `require-operation-field` kind, or reuse `problem-json-errors` as a template). Verify the non-compliant fixture still exits 1 and the rule appears in output.
4. **Generate docs.** Render the fleet spec with `docs_generator.py -out fleet-api.md`; confirm the `x-classification` label appears in the Robot schema table.
5. **Publish the catalog.** Run `catalog_manifest.py .\specs -out catalog-apis.yaml` and verify two entries with correct owners and lifecycle states.
6. **Measure activation.** Run `ttfc_funnel.py` on the fixture; write one sentence identifying the weakest funnel step and one DX investment that targets it.
7. **Run the campaign.** Execute the deprecation tracker at `-today`

2026-07-15 (expect exit 1, blocked) and again at `-today` 2026-09-02 after editing all consumers to `migrated` (expect exit 0). Note how the communication schedule dates shift with the kill date.

21.5.2 Case Study: The Shadow API Nobody Owned

Example

Northwind Robotics, incident INC-2291 (synthetic but typical). At 14:07 on a Tuesday, the fulfillment team deploys a routine change to the `/shipments` service. Eight minutes later, checkout error rates spike — but checkout never calls `/shipments`. Forty minutes of war-room archaeology later, gateway telemetry reveals the coupling: an undocumented endpoint, `/shipments/internal-quote`, deployed eighteen months earlier by a contractor, serving 3,000 calls a day from a billing integration whose owner field in the wiki pointed to a team dissolved in the last reorganization. No catalog entry, no linter history (the spec was never in the spec repo), no SLO, no on-call — and, fatally, no entry in the impact-assessment step of the fulfillment team’s change checklist, because as far as the platform was concerned the endpoint did not exist.

The postmortem actions read like this chapter’s table of contents: (1) diff observed gateway routes against the catalog weekly and quarantine unregistered traffic behind an explicit registration step; (2) require a catalog entry — and therefore an owner — as an *entry criterion for published*; (3) run the deprecation tracker’s census step against telemetry, not the wiki; (4) treat “unknown owner with nonzero traffic” as a severity-2 finding in its own right. The endpoint was eventually owned, spec’d, linted, and published properly; the campaign to retire it took four months and appears, disguised, as `legacy-billing-hook` in Listing 20.6’s registry.

The lab’s through-line is the loop of Figure 21.3: lint and catalog generation are the outward flow of governance; the funnel and the campaign census are the telemetry flowing back. Running them side by side makes the feedback tangible in under two hours.

21.6 Self-Assessment Exercises & Deep-Dive Questions

1. **State machine fluency.** For each event, name the transition and the evidence that must accompany it: (a) a team opens a PR adding a new

endpoint to a published spec; (b) security flags an unauthenticated endpoint in a spec under review; (c) a brownout reveals an unknown consumer three weeks before the kill date.

2. **Rule authoring.** Write (in YAML, using the Listing 20.2 rule kinds) a rule that requires every `POST` operation to document a 201 or 202 response. Which kind did you need, and what gap in the kind taxonomy did you find?
3. **Severity economics.** Argue both sides: should `pagination-required-on-lists` be error or warn severity for a company at 40 APIs? At 400?
4. **Funnel diagnosis.** The fixture's key→first-call step converts at 73.3% with a 194-minute median gap. Propose two distinct hypotheses and the instrument (not the fix) you would add to distinguish them.
5. **SLA math.** An API measures 99.93% monthly success. Its SLO is 99.9%, its SLA 99.5%. Compute both error budgets remaining. A change would spend another 0.05%; which budget governs the decision and why?
6. **Design the exception.** Draft the one-paragraph ADR for exempting a legacy partner endpoint from `problem-json-errors` for six months. What makes the exemption time-boxed and auditable rather than permanent?
7. **Docs drift forensics.** Your generated docs and your hand-written quickstart disagree about a parameter name. Trace the failure: which artifact is wrong, which process allowed it, and what lint rule prevents recurrence?
8. **Campaign design.** Using Listing 20.6's cadence, schedule a campaign for an API announced today with a 6-month notice period, one contracted enterprise consumer, and twelve internal consumers. Which cadence rows change for the enterprise consumer?
9. **Shadow-API hunt.** Describe the exact telemetry query (fields, joins, thresholds) that surfaces an unregistered endpoint with real traffic, and the runbook for the first 24 hours after it fires.
10. **Platform ROI.** Your platform team of six costs \$1.4M/year. Construct the strongest honest one-slide ROI case using only adoption, TTFC, and incident-reduction evidence — and identify the weakest assumption in your own slide.

21.7 Interview Q&A Vault

- Q1. *[junior]* Name the API lifecycle states and what fundamentally changes between published and deprecated.** Design, review, published (operate), deprecated, sunset, retired. At deprecation the contract freezes — no new features, security fixes only — the provider starts signaling (**Deprecation/Sunset** headers), and a dated migration campaign begins; the API still serves traffic under its SLA until the kill date.
- Q2. *[junior]* What belongs in an API style guide?** Only consumer-observable surfaces: naming, pagination, error shapes, authentication, versioning, idempotency, rate-limit signaling. Internal implementation choices stay with the team — standardizing everything guarantees the guide is routed around.
- Q3. *[junior]* What is Spectral?** An open-source linter for OpenAPI: rulesets written as YAML data target parts of the spec (JSONPath-ish) with functions (casing, pattern, truthy, custom), run in CI so style-guide compliance is checked by machines, not reviewer memory.
- Q4. *[junior]* What is a developer portal?** The self-service front door: searchable API catalog, rendered docs, key issuance, sandbox access, status, and changelog. Its foundation is a machine-generated catalog of record; a hand-maintained portal decays into a museum.
- Q5. *[junior]* What is docs-as-code?** Documentation generated from the contract of record (OpenAPI), versioned and deployed by the same pipeline as the spec. Humans edit the spec; machines edit the docs — so reference docs can never silently diverge from what the API does.
- Q6. *[junior]* What is TTFC and how do you reduce it?** Time-to-first-call: median elapsed time from signup to first successful (2xx) call. Reduce it by attacking the funnel step with the worst conversion: self-service instant keys (signup→key), a 15-minute quickstart and sandbox (key→call), and actionable error messages (call→200). Instrument first; never optimize the funnel blind.
- Q7. *[junior]* SLA vs SLO vs SLI — relationship?** SLI is the measurement, SLO the internal target on it, SLA the contractual promise with penalties. Invariant: SLA strictly looser than SLO,

so the error-budget gap gives the team room to detect and fix before breaching the contract [Bey+16].

- Q8. [junior] Deprecation vs sunset vs retirement?** Deprecation is the announcement (signal, freeze, campaign start); sunset is the enforcement window (brownouts, then permanent 410 at the kill date); retirement is the aftermath (audit clean, catalog archived, runbooks removed).
- Q9. [mid] How do you enforce a style guide without killing velocity?** Convert rules to lint executed in CI (fast, impartial, self-service feedback in minutes), roll new rules out as warnings before errors, keep human review scoped to judgment with a two-day SLA, and invest the savings in the paved path so compliance is the easiest route — enforcement should feel like autocomplete, not customs inspection.
- Q10. [mid] What makes a good custom lint rule?** A stable ID, an honest severity, a precise target, an actionable message with a JSON path to the offender, and a story — the incident or inconsistency the rule exists to prevent. Rules without stories get litigated every review; rules with stories get quoted.
- Q11. [mid] When do generated SDKs hurt DX?** When the team cannot commit to owning them: generated code is unidiomatic in every language, lags language evolution, and a stale SDK under your brand is worse than none. Rule of thumb: generate when the spec is stable and CI regenerates and publishes automatically; otherwise ship an excellent OpenAPI doc, a quickstart, and a sandbox.
- Q12. [mid] Your funnel shows strong key issuance but weak first-call conversion. Diagnosis path?** The failure is between “has credential” and “made a call”: quickstart quality, missing base-URL clarity, confusing scopes, or no sandbox. Instrument docs-viewed events and quickstart completion, time five developers through the getting-started flow, and check the first call→first 200 step too — if 401s dominate first calls, the auth docs are the real culprit.
- Q13. [mid] What goes into a design-review packet?** The spec diff, the lint report, the resource model rationale, and an ADR per non-obvious decision (versioning strategy, pagination choice, error model deviations). The packet exists so the reviewer spends the

two-day SLA window on judgment, not archaeology.

- Q14. [mid] Why ADRs for APIs specifically?** API decisions are unusually permanent — consumers hard-code around them — and unusually relitigated, because every new team re-asks “why not GraphQL/why cursor pagination/why problem+json”. Recording rejected alternatives answers next year’s mail without a meeting.
- Q15. [mid] How do you find out who consumes an API?** Gateway telemetry joined from API key/client ID to consumer identity — ground truth, because packets don’t lie. The catalog of record supplies owners and escalation contacts; the diff between observed and declared consumers is where shadow integrations live. Never run a census from a wiki.
- Q16. [mid] What is a paved path and how do you measure it?** The blessed, fully toolled route from idea to published API: repo template, pre-wired lint CI, generated docs, auto-catalog registration, sandbox, dashboards. Measure voluntary adoption (share of new APIs on the path) and time-to-publish versus the unpaved route; if teams need mandates, the path isn’t paved well enough.
- Q17. [mid] Changelog discipline — what does it look like operationally?** Every consumer-visible change, classified (added/changed/deprecated/removed), shipped with the deploy, and machine-linked to deprecation headers. The changelog is a trust instrument: one lagging release teaches consumers to distrust every other signal you send.
- Q18. [mid] What makes a sandbox good?** Self-service instant keys, deterministic seeded synthetic data, no approvals, realistic failure modes (you can trigger a 429 and a 422 on purpose), and isolation from production. Its success metric is TTFC; a sandbox requiring a ticket is a waiting room with a URL.
- Q19. [senior] Design a sunset campaign for a v1 API with 200 consumers.** Census from gateway telemetry (expect unknowns); segment internal/external and by traffic; announce at T-180 via changelog, portal banner, email, and **Deprecation** headers; publish the kill date and a migration guide with a shim where feasible; reminders at T-90/60 escalating to engineering leads; brownouts at T-30 (1 hour) and T-7 (4 hours) as escape hatches for non-readers; a documented, time-boxed extension process signed by the laggard’s director; permanent 410 with a problem+json body at T-0;

post-sunset traffic audit at T+7. Track readiness traffic-weighted and gate the retirement runbook on it.

- Q20. [senior] Why must the SLA be strictly looser than the SLO?** Because the gap between the internal budget (1 – SLO) and the contractual budget (1 – SLA) is reaction time measured in errors: alerting, rollback, and deploy freezes must consume SLO budget *before* SLA budget is touched. $SLA \geq SLO$ means customers discover your incidents before your policy does, and every incident becomes a contract negotiation [Bey+16].
- Q21. [senior] Brownouts: purpose, mechanics, risks?** Short, announced windows where the old version returns errors, flushing out consumers who ignore email before the permanent kill. Mechanics: gateway-level 410/404 with a problem body, per-consumer allowlist for carve-outs, business-hours timing so the right humans are awake. Risks: a brownout that pages the *wrong* team (your census failed), and crying wolf — brownouts that are always rolled back teach consumers to ignore them.
- Q22. [senior] Where do you draw the consistency-versus-autonomy line?** At the consumer-observable seam: everything a caller can see is standardized; everything behind the route table is the team’s business. Revisit the line only with telemetry — if an ungoverned internal choice keeps causing cross-team incidents, it earns promotion into the guide via ADR, not via decree.
- Q23. [senior] Kill date arrives; one consumer is unresponsive but still calling. What do you do?** Follow the pre-agreed policy, which existed precisely for this moment: confirm the escape-hatch criteria (is this a carve-out case or an abandonment case?), attempt final escalation through the census owner chain, brownout on schedule, and sunset — with a documented reactivation path. Bending the kill date for silence teaches 199 other consumers that deadlines are negotiable.
- Q24. [senior] Design GDPR-style erasure propagation across an API ecosystem.** Erasure is an event, not a DELETE: a tombstone workflow fans out to the primary store, derived stores, caches, analytics, and downstream consumers you pushed data to, with per-hop acknowledgments; backups expire data by retention schedule rather than surgical edits. Schema-level classification labels tell you what to erase; the audit trail proves you did; quarterly rehearsals prove the workflow still works.

- Q25. [senior] How do you keep PII out of API logs at platform scale?** Don't rely on developer discipline: classify fields in the schema (**x-classification**), log only allow-listed metadata fields at the gateway and framework middleware level (never bodies by default), lint specs for classification coverage, and run periodic payload-sampling audits with automated detectors. The control lives in the pipeline, not in the style guide.
- Q26. [senior] Internal vs external deprecation policy — what differs and why?** Notice period (sprint-plannable 60–90 days vs contractual 6–12 months), enforcement lever (CI and deploy mandates vs persuasion, pricing, and brownouts within contract terms), and discovery (org chart available vs contracts and telemetry only). The substance is identical — census, cadence, brownouts, kill date — but external consumers hold contractual rights you must design around, not through.
- Q27. [senior] What telemetry should revise your style guide?** Lint violations by rule (a rule everyone suppresses is wrong or mistargeted), review findings that recur (recurrence means the guide is silent or unclear), support-ticket themes, funnel step conversions after docs changes, and error-mix shifts per consumer after releases. Governance without this feedback loop ossifies into theater.
- Q28. [staff] Build vs buy for portals and gateways — your framework?** Buy the undifferentiated and mature (gateway, key management, hosted docs rendering); build what encodes your conventions (rulesets, paved-path templates, catalog enrichment glued to your org chart); default to composing: managed gateway, open-source portal core (Backstage-style), thin bespoke glue. The decisive question is not features but *who operates it at 3 a.m.* and what it costs to leave.
- Q29. [staff] How should a platform team be funded, and why do cost-center models fail?** Fund as a product team with internal customers on a multi-year horizon, success measured by voluntary adoption. Cost-center funding turns the team into a ticket queue: roadmap by loudest stakeholder, no investment in self-service (which would “reduce demand” for the team), and eventual mandate-driven adoption that destroys the feedback loop that keeps the paved path paved [New21].
- Q30. [staff] How do you measure platform ROI?** Three columns:

adoption (APIs on the paved path, traffic share behind the gateway), activation (TTFC median and funnel conversion, trended), and risk removed (lint-blocked non-compliant designs, incident rates of governed vs ungoverned APIs, sunsets completed without escalation). Report quarterly; translate to money only where the counterfactual is defensible.

- Q31.** *[staff]* **You inherit 500 APIs across 50 teams with zero governance. First 90 days?** Days 1–30: instrument — gateway census, catalog skeleton generated from whatever specs exist, TTFC baseline. Days 31–60: publish the v1 style guide (consumer-observable surfaces only), ship the linter as *advisory*, and stand up one paved path for *new* APIs. Days 61–90: flip two high-value rules to blocking, run the first review-SLA'd design reviews, and publish the first transparency report. Never begin with a mandate to retrofit all 500.
- Q32.** *[staff]* **A strong team demands an exception to the error-shape rule “just this once.” Your process?** Exceptions are ADRs, not conversations: written context, explicit time-boxing, a named owner, a suppression comment in the spec referencing the ADR, and a review date. Permanent undocumented exceptions compound; time-boxed documented ones are governance working as intended — the rule learns where its edges are.
- Q33.** *[staff]* **Your largest revenue account refuses to migrate off v1. Sunset anyway?** This is a business decision the campaign should have surfaced at T-90, not T-7. Options, in order: dedicated migration engineering (often cheaper than the standoff), a paid extended-support tier with a contractual final date, or accepting v1 as a frozen, ring-fenced deployment with its own cost model charged back. What you don't do is quietly extend forever — an undead v1's maintenance cost is invisible precisely because nobody is measuring it.
- Q34.** *[staff]* **What organizational signals tell you governance has become theater?** Review queues measured in weeks while ship rates stay flat (teams are bypassing); lint suppressions clustering on the same rules; a catalog whose entries disagree with gateway telemetry; style-guide pages last edited by someone who left; and — the tell — governance metrics that report *activity* (reviews held) instead of *outcomes* (consistency, adoption, TTFC, sunset success rate).

21.8 External Bibliography & Further Reading

- **OpenAPI Specification v3.1.0** [Ope21] — the contract of record this chapter’s linter, docs generator, and catalog all consume.
- **Gehl & Higginbotham, *API Design Patterns* (Manning, 2021)** [GH21] — pattern-level grounding for the style-guide surfaces: naming, pagination, versioning, and long-running operations.
- **Beyer et al., *Site Reliability Engineering* (O’Reilly, 2016)** [Bey+16] — SLOs, error budgets, and the SLA < SLO doctrine.
- **Newman, *Building Microservices*, 2nd ed. (O’Reilly, 2021)** [New21] — platform and team-boundary thinking behind paved paths.
- **The Twelve-Factor App** [Wig17] — config-in-environment discipline that sandbox and portal key issuance builds on.
- **Spectral documentation**, <https://docs.stoplight.io/docs/spectral> — ruleset anatomy, built-in functions, custom rules; the production counterpart of Listing 20.2.
- **Backstage documentation**, <https://backstage.io/docs> — software catalog and kind: API entity model behind Listing 20.7.
- **openapi-generator**, <https://openapi-generator.tech> — the reference SDK generation pipeline, including its templating and CI regeneration story.
- **Redocly / Redoc** (<https://redocly.com/docs>) and **Stoplight Elements** — production docs-as-code renderers whose role Listing 20.4 miniaturizes.
- **Skelton & Pais, *Team Topologies* (IT Revolution, 2019)** — the platform-team-as-product funding and interaction models of Section 21.2.7.
- **Google AIP program**, <https://google.aip.dev> — a mature, public, lint-enforced API design standard; study its AIP-122 (resource names) and AIP-158 (pagination) alongside Table 21.2.
- **Stripe, Slack, and Twilio engineering blogs** (<https://stripe.com/blog>, <https://slack.engineering>, <https://www.twilio.com/blog>) — primary sources on DX engineering at scale: Stripe’s API version-

ing and changelog discipline, Slack’s developer-platform iterations, Twilio’s helper-library (SDK) maintenance economics.

- **RFC 8594 (the **Sunset** header)** and the deprecation-header draft — the wire-level machinery behind Section 21.2.5, introduced in Chapter 10.

21.9 Capstone Projects

Capstone 20.1 [junior] — Three New Rules for the Fleet

Objective: Extend the Northwind ruleset with three custom rules and prove each one bites.

Inputs/Datasets: `code\ch20\` (`mini_spectral.py`, `ruleset.yaml`, both spec fixtures).

Steps:

1. Add rules: `summary-required` (every operation has a non-empty `summary`), `no-verbs-in-paths` (path segments match a noun-only denylist: `get`, `create`, `delete`, `fetch`), `cursor-type-string` (any `cursor` parameter is `type: string`).
2. Extend the linter with any new rule kinds needed, keeping rules as data.
3. Add one violating case per rule to a copy of the non-compliant fixture; assert exit codes and rule IDs in new tests.

Acceptance Criteria:

- ☐ Compliant specs still exit 0; the extended fixture exits 1 with all three new rule IDs.
- ☐ Each violation carries a JSON path pinpointing the offending node.
- ☐ New rule kinds are unit-tested, including a bad-ruleset (unknown kind) test exiting 2.

Stretch Goals: Add per-rule suppression comments (`x-lint-ignore: [rule-id]`) honored by the engine and logged as warnings.

Capstone 20.2 [mid] — Docs-as-Code Pipeline

Objective: Assemble `lint` → `docs` → `catalog` into one offline pipeline script with a CI gate.

Inputs/Datasets: `code\ch20\`; the two specs under `specs\`.

Steps:

1. Write `pipeline.py`: lint every spec (block on errors), render Markdown docs to `docs\`, and emit `catalog-apis.yaml`.
2. Add a staleness check: if a spec changed but its rendered docs did not regenerate (compare content hashes stored in a manifest), exit 1 with a clear message.
3. Record a one-page ADR describing the pipeline's blocking policy.

Acceptance Criteria:

- ☐ One command produces docs and catalog for all specs; a non-compliant spec blocks the run.
- ☐ Editing a spec without rerunning the generator fails the staleness gate.
- ☐ Everything runs offline; output is byte-deterministic across runs.

Stretch Goals: Add a Markdown link checker (`stdlib`) over the generated docs.

Capstone 20.3 [mid] — TTFC Instrumentation Pack

Objective: Extend the funnel analyzer from report to decision tool.

Inputs/Datasets: `ttfc_funnel.py`, `fixtures\onboarding_events.jsonl` (222 synthetic events, 60 developers).

Steps:

1. Add a `-by channel` breakdown (the fixture's `channel` field) computing per-channel funnels.
2. Add a weekly cohort table (signups grouped by ISO week) with per-cohort TTFC medians.
3. Write a "DX experiment" report: propose one intervention, state which funnel step it targets, and the metric movement that would confirm it.

Acceptance Criteria:

- ☐ Per-channel funnels show conversion deltas with counts (no percentage without an n).

- ☐ Cohort table is deterministic and sorted; malformed events are still counted and skipped, never fatal.
- ☐ The experiment report names one metric, one step, and one falsifiable threshold.

Stretch Goals: Flag small- n cohorts ($n < 10$) in the output to discourage over-reading noise.

Capstone 20.4 [senior] — Sunset Campaign for Fleet API v1

Objective: Design and simulate the full retirement of a legacy version with heterogeneous consumers.

Inputs/Datasets: `deprecation_tracker.py`, `fixtures\deprecation_registry.yaml`; a 12-consumer registry you extend it into (add tiers, contract dates, one shadow consumer).

Steps:

1. Extend the registry schema with `notice_days` per tier and `extension_until` per consumer; validate defensively.
2. Add a simulation mode: given a migration-velocity assumption per tier, project residual traffic weekly until the kill date.
3. Produce the campaign plan: cadence rows adjusted per tier, brownout scope, carve-out list, and the go/no-go checklist for T-0.

Acceptance Criteria:

- ☐ The projection flags, in advance, the consumers who will miss the date under stated velocity.
- ☐ The plan differentiates internal vs external policy per Table 21.3.
- ☐ Exit codes remain CI-meaningful; all dates are explicit CLI inputs (deterministic).

Stretch Goals: Model one extension request and show its effect on the brownout schedule for non-extended consumers.

Capstone 20.5 [staff] — Platform ROI Scorecard & Governance Operating Model

Objective: Design the quarterly platform review for a fictional 50-team Northwind Robotics.

Inputs/Datasets: This chapter’s tools plus invented-but-plausible baselines (document all assumptions); no real company data.

Steps:

1. Define the scorecard: adoption, activation (TTFC funnel), risk removed; a data source and an owner per metric.
2. Build `scorecard.py`: reads the funnel fixture, a lint history CSV you synthesize, and an incident log you synthesize; renders the one-page scorecard.
3. Write the operating model: rule lifecycle (propose → advisory → blocking → retire), review SLA, exception process, and the platform team’s funding ask with the ROI narrative.

Acceptance Criteria:

- ☐ Every scorecard number traces to a reproducible computation from synthetic inputs.
- ☐ The ROI narrative states its weakest assumption explicitly.
- ☐ The operating model fits on two pages and names decision rights for every governance event.

Stretch Goals: Add a counterfactual section: estimated incident cost avoided using governed-vs-ungoverned incident rates, with the selection-bias caveat addressed.

21.10 Python Dependencies Appendix

The chapter’s code targets Python 3.11+ and needs four third-party packages (only PyYAML and pytest are exercised at runtime; the other two support the program-level patterns referenced below):

```
1 pyyaml>=6.0
2 jsonschema>=4.21
3 pydantic>=2.7
4 pytest>=8
```

Install on Windows/PowerShell (virtual environment recommended):

```
1 cd code\ch20
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 pip install -r requirements.txt
```

21.10.1 PyYAML (pyyaml>=6.0)

Description: The de-facto YAML parser for Python. **Why included:** OpenAPI documents and rulesets in this chapter are YAML; `yaml.safe_load` parses specs, rulesets, registries, and catalog emission uses `yaml.safe_dump`. Safe loaders only — `yaml.load` without a safe loader is a code-execution hazard on untrusted input. **Alternatives:** `ruamel.yaml` (round-trip editing with comments preserved, useful if you want a linter that *fixes* specs); the real Spectral CLI (Node.js) for production-grade linting; Redocly CLI for lint-plus-bundle. **Installation:** `pip install pyyaml>=6.0` (PowerShell; quote as "pyyaml>=6.0" if your shell interprets >).

21.10.2 jsonschema (jsonschema>=4.21)

Description: JSON Schema validation for Python; OpenAPI 3.1 schemas are JSON Schema dialects [Ope21]. **Why included:** The natural next layer after structural linting is *schema-aware* rules — e.g., validating that a spec's example payloads actually satisfy their declared schemas, or that a registry YAML conforms to a published registry schema (the defensive parsing in Listings 20.2 and 20.6 is the hand-rolled floor; `jsonschema` is the ceiling). **Alternatives:** `openapi-spec-validator` and `openapi-schema-validator` (OpenAPI-aware validation); Spectral/Redocly for schema checks embedded in a lint pipeline. **Installation:** `pip install "jsonschema>=4.21"`.

21.10.3 Pydantic (pydantic>=2.7)

Description: Type-annotated data modeling and validation. **Why included:** The chapter's tools deliberately use `stdlib` dataclasses for transparency, but a production platform models its catalog entries, registry records, and ruleset schema as `pydantic` models — gaining validation errors with field paths, JSON Schema export for free, and parity with the FastAPI services of Chapter 7 [Ope21]. **Alternatives:** `stdlib dataclasses` (as used here) plus `jsonschema` for boundary validation; `msgspec` for speed-critical parsing; `attrs` for lightweight class ergonomics. **Installation:** `pip install "pydantic>=2.7"`.

21.10.4 pytest (pytest>=8)

Description: The standard Python test runner. **Why included:** Both test suites (Listings 20.3 and 20.8) use `pytest`'s `tmp_path` and `capsys` fixtures to exercise CLI exit codes — the CI contract of the linter and tracker — without shelling out, keeping tests offline and deterministic per Chapter 20. **Alternatives:** `unittest` (`stdlib`, more ceremony); `nox/tox` as session runners on top of `pytest`; MkDocs is *not* a test alternative but is

the natural companion for publishing this chapter’s generated Markdown as a docs site. **Installation:** `pip install "pytest>=8"`.

21.11 Chapter Summary & Key Takeaways

- **The lifecycle is a state machine, not a poster.** Seven states with evidence-based entry/exit criteria make ownership, tooling, and audits mechanical. The `deprecated` state is first-class: frozen contract, `Sunset` headers, live campaign.
- **Governance compiles or it doesn’t exist.** Style guides standardize only the consumer-observable seam; Spectral-style rulesets enforce them as data in CI with exit codes that distinguish “non-compliant” from “gate broken”; review keeps a two-day SLA and advisory-by-default verdicts; ADRs make exceptions and rejected alternatives permanent public record.
- **DX is a measured funnel.** Portals and catalogs are generated, not curated; docs are build artifacts of the spec; SDKs are generated only when CI regenerates them forever; TTFC (signup → key → call → 200) is the activation metric, and every DX investment names the funnel step it moves.
- **APIs are products with contract discipline.** Tier on a value metric enforceable at the gateway; keep SLA strictly looser than SLO so error budget is reaction time [Bey+16]; changelogs are trust instruments.
- **Sunsetting is a campaign run on telemetry.** The census comes from the gateway, not the wiki; cadence runs T-180 to T+7 with brownouts as escape hatches; internal and external consumers get different levers but the same dates.
- **Data governance rides the contract.** Classification labels in schemas, payload minimization, redaction by default, retention clocks, and rehearsed deletion propagation across stores, caches, and downstream consumers.
- **Platform economics justify the platform.** Buy the undifferentiated, build the conventional; fund the platform team as a product team [New21]; report ROI in adoption, TTFC, and risk removed — or watch the budget leave.

21.12 Quick-Reference Card

State	Consumer may assume	Provider must do
design	nothing	spec in repo, ADR for hard choices
review	nothing	lint green; review within SLA
published	stable contract, SLO, docs	operate, changelog, compatible evolution only
deprecated	works until kill date; no new features	headers on, campaign running, security fixes only
sunset	brownouts then 410	final warnings, 410 + problem body with migration link
retired	nothing (404/410 forever)	audit clean, catalog archived

Table 21.4: Lifecycle states: the contract each side is under.

Surface	Core rule (lint-enforced)
Naming	kebab-case path segments; camelCase <code>operationId</code>
Versioning	<code>info.version</code> strict semver; URI major version
Errors	every operation declares <code>application/problem+json</code> on 4xx/5xx/default
Pagination	collection GETs accept <code>limit</code> + <code>cursor</code>
Ownership	<code>info.contact</code> + <code>x-lifecycle.owner</code> required
Governance hygiene	rules as data; warn → error rollout; suppressions are ADR-linked

Table 21.5: Style-guide core rules and their enforcement.

21.13 Standardized Evidence Addendum

21.13.1 Benchmark Snapshot Template

21.13.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.

Metric	Definition / target practice
TTFC	median $t(\text{first } 200) - t(\text{signup})$; the activation metric
Funnel conversion	$C_i = N_i/N_{i-1}$ per step; fix the weakest step first
Census coverage	% of gateway traffic mapped to cataloged consumers; target 100%, diff weekly
Paved-path adoption	% of new APIs via the paved path; voluntary, trended
SLA/SLO gap	$SLA < SLO$; gap = reaction budget in errors [Bey+16]
Campaign readiness	residual traffic on non-migrated consumers; T-0 gate
Portal health	zero-result searches, docs-to-spec staleness hash

Table 21.6: DX and governance metrics cheat sheet.

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	—	—	—	1.00x
Candidate A	—	—	—	—
Candidate B	—	—	—	—

Table 21.7: Minimum benchmark table to keep per chapter.

- **Result:** what changed in correctness, resilience, latency, and operability.

21.13.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	—	handshake round trips, keep-alive hit rate
Request validation	—	schema complexity, CPU per request
Business logic and I/O	—	downstream fan-out, query cost
Serialization and egress	—	payload size, compression ratio

Table 21.8: Operational budget template for this chapter's technique.

21.14 Configuration Preset Template

```

1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"

```

21.15 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

21.16 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

21.17 Cross-Chapter Decision Card

Dimension	Choose this approach when	Prefer an alternative when
Use case	–	–
Latency budget	–	–
Operational cost	–	–
Team maturity	–	–

Table 21.9: Decision card to complete per chapter.

21.18 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

21.19 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Chapter 22

Synthesis, Reference Architectures & Future Directions

Executive Summary

Every chapter so far has taken one API concern — the wire protocol, a framework, an error model, a scaling tactic, a governance discipline — and treated it with the depth it deserves. Real systems, however, are never about one concern. A production API platform is a *composition*: HTTP semantics (Chapter 2), REST resource design (Chapter 4), GraphQL selection sets (Chapter 13), gRPC streaming (Chapter 14), event delivery (Chapter 12), and the operational disciplines of security, observability, testing, and lifecycle governance (Chapters 17–21) all coexist inside one coherent architecture. This final chapter is about that composition.

The chapter delivers four capstone artifacts. First, the **Master Decision Card**: a rigorous, constraint-driven selection framework for choosing between REST, GraphQL, gRPC, event-driven/webhook, and WebSocket/SSE interaction profiles — expressed as decision tables, a decision tree, and worked scenarios, and then *encoded as executable software*. Second, the **Northwind Robotics API Platform reference architecture**: one integrated design that uses every concept in the book, with per-layer commentary mapping each architectural element back to the chapter that taught it. Third, the **Grand Capstone**: a complete, buildable specification of that platform — scope, domains, API surfaces, non-functional requirements, a four-milestone delivery plan, acceptance criteria, and an evaluation rubric. Fourth, an honest assessment of the **emerging frontiers** — HTTP/3 and QUIC, AI-agent-consumed APIs and the Model Context Protocol, AI-assisted design, edge-native deployment, post-quantum TLS, and security automation — so that you can separate deployable reality from

conference-stage enthusiasm.

The chapter closes the loop on your own development: a cross-cutting synthesis of the book’s failure taxonomy, one-page-per-module summary tables, career pathways from junior to staff API engineer mapped to chapters, a consolidated bibliography, and five integrative capstone projects. The code in this chapter is deliberately integrative: a protocol-selection advisor with a full rationale trace, a single FastAPI application that exposes REST, GraphQL, and SSE surfaces with RFC 9457 errors, idempotency keys, and rate-limit headers, and an architecture conformance checker that turns the book’s governance rules into executable policy.

22.1 Learning Objectives & Prerequisites

After completing this chapter, you will be able to:

- Select an API interaction profile (REST, GraphQL, gRPC, event-driven, or streaming) from stated constraints, and defend the selection with an explicit, auditable rationale.
- Assemble a complete reference architecture — edge, gateway, services, data, observability, and governance — and justify every layer against the non-functional requirements it serves.
- Specify a multi-protocol API platform to the standard a staff engineer would expect: SLOs, security posture, rate tiers, delivery milestones, and measurable acceptance criteria.
- Diagnose which chapter’s discipline applies to any concrete failure mode observed in production.
- Assess emerging API technologies (HTTP/3, MCP, edge-native compute, post-quantum TLS) with a sober adopt/trial/wait classification.
- Map your own competency growth from junior to staff engineer onto concrete, chapter-anchored skills.
- Encode architectural governance as executable policy that runs offline in CI, and prove platform surfaces work together with in-process smoke tests.

Prerequisites

This is the synthesis chapter: it assumes the whole book. You should be comfortable with HTTP message semantics and connection manage-

ment (Chapter 2), client-side consumption patterns (Chapter 3), REST maturity and resource modeling (Chapter 4), serialization contracts (Chapter 5), building services with FastAPI (Chapter 7), RFC 9457 error design (Chapter 8), pagination and versioning (Chapters 9–10), idempotency and caching (Chapter 11), asynchronous and event-driven APIs (Chapter 12), GraphQL and gRPC (Chapters 13–14), rate limiting (Chapter 15), microservice resilience (Chapter 16), security engineering (Chapter 17), deployment (Chapter 18), observability (Chapter 19), testing (Chapter 20), and lifecycle governance (Chapter 21). Sections below reference these chapters liberally rather than re-teaching them.

22.2 Deep Technical Mechanics & Architectural Concepts

22.2.1 The Master Decision Card: Choosing an Interaction Profile

The most consequential API decision a team makes is rarely *how* to implement a protocol — it is *which* protocol to implement. Choosing poorly is expensive in a way that refactoring cannot fix: the protocol is the contract your consumers build against, and contracts accrete (Chapter 21). The industry conversation about this choice is unfortunately dominated by fashion. The antidote is to treat protocol selection as **constraint satisfaction**, not taste.

Definition 22.1 (Interaction Profile). An *interaction profile* is the combination of transport semantics, message format, delivery model, and contract style through which a consumer and provider exchange data. This book covers five: **REST** (resource oriented, HTTP semantics, Chapter 4), **GraphQL** (selection sets over a typed graph, Chapter 13), **gRPC** (contract-first RPC over HTTP/2 with Protobuf, Chapter 14), **event-driven** (brokered events or signed webhooks, Chapter 12), and **streaming** (Server-Sent Events or WebSockets, Chapter 12).

The Six Criteria

The decision card evaluates a candidate interaction against six criteria. Each criterion is a question with a small set of allowed answers — which is precisely what makes the framework encodable as the questionnaire of Listing 1 (Section 22.3.1).

Criterion	Question	Why it dominates
Consumer type	Who calls this API?	Browser-based third parties cannot speak raw gRPC; machines do not care about human-readable payloads; mobile clients feel every over-fetched byte on a metered radio.
Payload shape	Fixed resources or a nested graph?	Deep object graphs make REST chatty (the N+1 endpoint problem) and make GraphQL's selection sets pay for themselves.
Latency budget	What p99 must you hold?	Sub-50 ms service-to-service budgets favor gRPC's binary framing and HTTP/2 multiplexing [Tho22; gRP24]; relaxed budgets tolerate store-and-forward asynchrony.
Delivery model	Sync, push, or fire-and-forget?	Structural constraint: unary request-response protocols cannot push. If the server must initiate, you need SSE/WebSockets or events.
HTTP caching	Must shared caches help?	Only REST GETs are addressable, cacheable resources under HTTP semantics [Fie22a]; POST-based GraphQL bypasses the CDN layer entirely.
Team maturity	Exploratory or disciplined?	GraphQL and gRPC pay off only with schema governance discipline; REST has the shallowest tooling curve [GH21].

Table 22.1: The six criteria of the Master Decision Card.

The Comparison Table

Table 22.2 condenses twenty chapters of trade-offs into one grid. Read it as *tendencies*, not laws — each row has an entire chapter of nuance behind it.

	REST	GraphQL	gRPC	Events/webhooks	SSO/SSO/SSO
Primary consumer	public, partner	mobile, SPA, data	internal services	machines, partners	dashboards, tickers
Delivery	sync	sync	sync + streams	async, at-least-once	server push
Payload	JSON resources [Bra17]	client-shaped JSON [Fie22a]	binary Protobuf [Goo24]	event envelopes	text frames / JSON
Caching	full HTTP caching [Fie22a]	persisted queries only	none	n/a (push)	no-cache streams
Contract	OpenAPI [Ope21]	SDL schema [Gra21]	proto files [gRP24]	AsyncAPI [Asy23]	AsyncAPI
Tooling curve	shallow	medium	medium–steep	medium	shallow–medium
Failure mode	N+1 hops	N+1 resolvers	coupling via stubs	silent event loss	connection storms
Book chapter	4	13	14	12	12

Table 22.2: Interaction profile comparison matrix (tendencies, not laws).

Key Concept

Constraints choose protocols; engineers choose implementations. If your stated constraints (consumer, payload, latency, delivery, caching, maturity) do not uniquely determine a profile, your constraints are not yet stated sharply enough. When two profiles genuinely tie, prefer the one your organization already operates well: operational maturity beats theoretical fit.

The Decision Tree

Figure 22.1 renders the card as a tree. The delivery model comes first because it is a *structural* constraint — no amount of engineering makes a unary REST endpoint push data to the client.

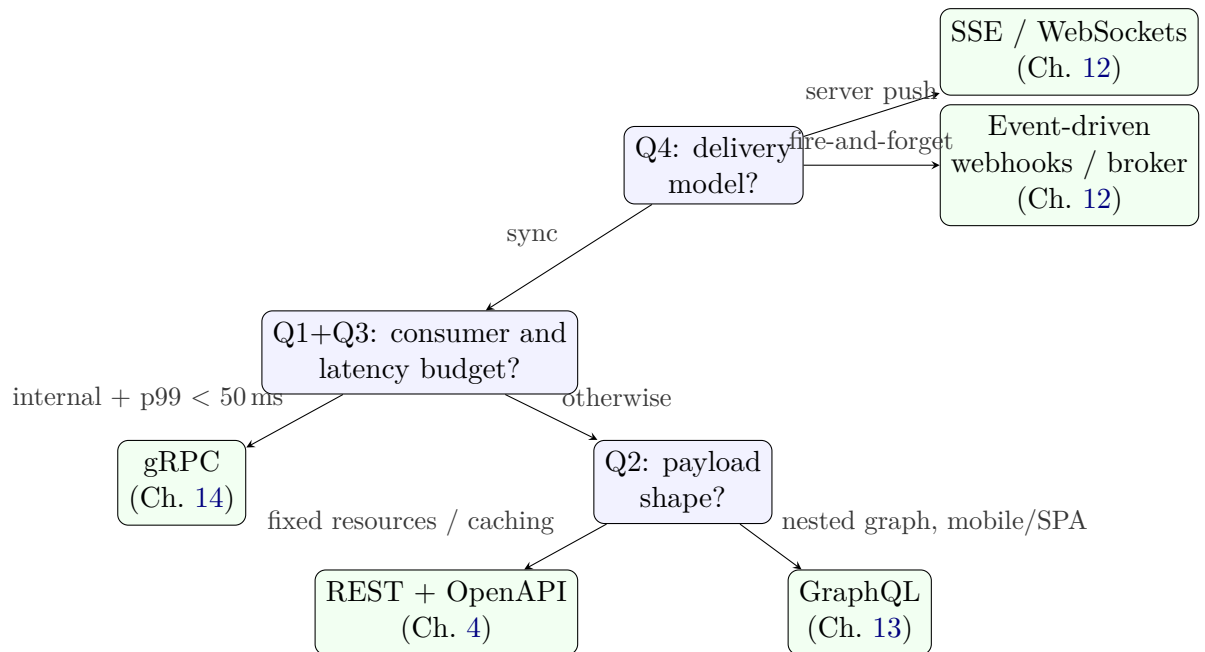


Figure 22.1: The Master Decision Card as a decision tree. Question numbers refer to Table 22.1; delivery is tested first because it is structural. Edge cases (gRPC-Web for browsers, GraphQL subscriptions over WebSockets) are refinements, not new branches.

Worked Scenarios

The framework earns its keep on realistic, messy scenarios. Each of the five below is encoded verbatim as a canonical test case in the advisor of Section 22.3.1, so the prose and the executable policy cannot drift apart.

Example

Scenario A — Public SaaS API. A vendor exposes its product catalog to unknown third-party developers. Consumer: public. Payload: fixed resources. Latency: relaxed (hundreds of milliseconds fine). Delivery: synchronous. Caching: catalog reads must be CDN-cacheable. Team: disciplined. **Recommendation: REST + OpenAPI.** Third parties expect plain HTTPS and JSON they can explore with `curl`; shared caching [Fie22a] absorbs read amplification; OpenAPI gives the portal its source of truth [Ope21]. GraphQL adds support burden with no caching upside; gRPC is unreachable from browsers without a proxy layer.

Scenario B — Internal microservice mesh. Forty services exchange inventory and pricing calls inside one trust boundary. Consumer: internal.

Latency: extreme (p99 < 50ms budgets on hot paths). Delivery: sync, with some server-streaming feeds. **Recommendation: gRPC.** Strongly typed contracts, code generation, HTTP/2 multiplexing, and binary framing [Tho22; gRP24] dominate at this latency budget, and the team controls both ends of every call.

Scenario C — Mobile app backend. A field-technician app renders deeply nested work-order screens over metered connections. Consumer: mobile. Payload: nested graph. Latency: tight. Caching: personalized, so no shared cache. **Recommendation: GraphQL.** One round trip per screen with exactly the fields rendered [Gra21]; an aggregating BFF-style REST layer is the acceptable fallback when GraphQL governance is not yet in place.

Scenario D — IoT telemetry. Three thousand robots stream sensor readings. Consumer: machine. Payload: high-volume streams. Delivery: fire-and-forget. **Recommendation: event-driven** ingestion to a broker with an event envelope (`event_id`, `occurred_at`); gRPC bidirectional streaming is the runner-up when devices speak HTTP/2 and the operator wants backpressure-aware streams (Chapter 12).

Scenario E — Partner integrations. Business partners must learn within seconds that an order shipped. Consumer: public, delivery: fire-and-forget to *their* endpoints. **Recommendation: signed webhooks** with delivery logs and replay, per Chapter 12. Polling a REST endpoint is the graceful degradation path, not the primary design.

Warning

The decision card is a floor, not a ceiling. It prevents *indefensible* choices (public gRPC to browsers, unary APIs for push, uncached GraphQL for a read-heavy public catalog). It does not certify that the chosen design is good — that still requires the discipline of the relevant chapter.

22.2.2 The Northwind Robotics Reference Architecture

We now integrate the entire book into one design: the **Northwind Robotics API Platform**, a synthetic-but-realistic platform for a robotics manufacturer. Northwind sells robots and spare parts (catalog, orders), runs a connected fleet (telemetry), and integrates with resellers (partner APIs). Figure 22.2 shows the full picture; the commentary after it maps every layer to the chapters that justify it.

Layer-by-Layer Commentary

Consumers. Four consumer classes drive four interaction profiles, exactly as the decision card predicts: partners (REST + webhooks), mobile (GraphQL + SSE), the web storefront (REST with CDN caching), and the robot fleet (event-driven ingestion). One platform, four profiles — chosen per consumer, not per fashion.

Edge. A CDN terminates TLS 1.3 [Res18], serves cached catalog GETs, and absorbs cheap abuse; a WAF filters known-bad signatures before they cost compute. This is where HTTP semantics pay rent: correct **Cache-Control**, **ETag**, and **Vary** headers (Chapters 2, 11) translate directly into origin shielding [Fie22a].

Gateway. The gateway is a policy enforcement point, not a junk drawer. It offloads token verification (OAuth 2.0 bearer tokens with JWTs [Har12; Jon15], PKCE for public clients [Sak15]), enforces rate tiers and emits IETF **RateLimit** headers [Pol24] (Chapter 15), and stamps every request with a correlation ID for tracing (Chapter 19). Business logic never lives here.

Services. Each service owns its profile: FastAPI REST for catalog and orders with keyset pagination (Chapter 9), date-based versioning (Chapter 10), RFC 9457 problem details [Not23] (Chapter 8), and idempotency keys on order creation (Chapter 11). GraphQL fronts the read model for the mobile app with depth limits and persisted queries (Chapter 13). gRPC handles the internal inventory hot path (Chapter 14). The dispatcher owns webhook signing, delivery logs, and SSE fan-out (Chapter 12).

Data. PostgreSQL is the system of record; every state change that must be published is written *in the same transaction* to an outbox table, then relayed to the broker — the transactional outbox pattern of Chapter 12, which eliminates dual-write anomalies [Kle17]. Redis carries rate-limit counters and short-lived cache entries; it is never a source of truth.

Observability. OpenTelemetry instrumentation in every hop produces traces joined by the gateway’s correlation ID, RED/USE metrics, and structured logs [Ope24] (Chapter 19). SLOs and error budgets follow the SRE discipline of Chapter 19 [Bey+16].

Governance. OpenAPI and AsyncAPI specs are the source of truth, linted in CI, rendered to the developer portal, and checked by the conformance policy engine of Listing 3 (Section 22.3.3) [Ope21; Asy23] (Chapter 21). Deprecations follow the **Sunset** header convention [Wil19].

Request Lifecycle Walkthrough

A partner’s `POST /v1/orders` traverses the platform like this:

1. **Edge:** TLS 1.3 handshake (resumed), WAF pass, cache bypass (POST is not cacheable [Fie22a]).

2. **Gateway:** JWT verified locally against cached JWKS [Jon15]; rate tier `partner-standard` admits the request, `RateLimit-Remaining: 97` stamped [Pol24]; correlation ID `rid-9f2` assigned.
3. **Service:** FastAPI validates the payload with Pydantic [Col24; Ram24]; the `Idempotency-Key` deduplicates retries (Chapter 11); a gRPC call to inventory reserves stock within the 50 ms budget (Chapter 14); order row and outbox row commit in one transaction.
4. **Async fan-out:** the outbox relay publishes `order.created` to the broker; the dispatcher signs and POSTs the webhook to the partner and pushes an SSE event to the operations dashboard (Chapter 12).
5. **Observe:** one trace, four spans, zero log lines containing credentials (Chapters 19, 17).

Every failure along this path has a named, rehearsed response: validation fails with RFC 9457 422 [Not23]; rate exhaustion returns 429 with `Retry-After`; inventory timeout trips a circuit breaker [Nyg18] (Chapter 16); webhook delivery retries with backoff and lands in the delivery log for replay.

22.2.3 The Grand Capstone: Platform Specification

This subsection is the full buildable specification of the platform in Figure 22.2. It extends every per-chapter capstone into one integrative build, and it is the [staff] project of Section 22.9.

Scope and Domains

Build the Northwind Robotics API Platform as a set of deployable services plus the governance toolchain that keeps them honest. Four domains: **catalog** (products, categories, pricing), **orders** (placement, status, partner notification), **inventory** (stock reservation, restock), and **telemetry** (fleet readings, operator dashboards). Synthetic data only; all services must run offline on one machine with in-process fakes for the broker and payment gateway.

API Surfaces per Protocol

Non-Functional Requirements

- **SLOs:** catalog read p99 < 200 ms at 99.9 % monthly availability; order write p99 < 500 ms; inventory gRPC p99 < 50 ms; webhook delivery 99.5 % within 60 s. Error budgets govern release pace [Bey+16].

Surface	Profile	Contract
/v1/products, /v1/orders	REST + OpenAPI	keyset pagination, date version, RFC 9457 errors, idempotency keys
/graphql	GraphQL	read model over catalog + telemetry; depth ≤ 5 ; persisted queries
inventory.v1.Inventory	gRPC	Reserve/Release/WatchStock (server stream)
order.created, order.shipped	webhooks	signed (HMAC), delivery log, replay endpoint, AsyncAPI spec
/v1/telemetry/stream	SSE	operator dashboard feed, Last-Event-ID resume

Table 22.3: Grand Capstone API surfaces. Each surface cites its governing spec artifact.

- **Security posture:** OAuth 2.0 + JWT bearer at the gateway [Har12; Jon15]; API keys for partners; webhook signatures; OWASP API Top 10 checklist in CI [OWA23]; no credential material in logs; TLS 1.3 only [Res18].
- **Rate tiers:** public 60/min, partner-standard 600/min, partner-premium 6000/min; IETF headers on every response [Pol24] (Chapter 15).
- **Data integrity:** order + outbox atomicity; idempotent order placement; replayable webhook log retained 30 days [Kle17].
- **Operability:** OTel traces end to end [Ope24]; twelve-factor configuration [Wig17]; health endpoints; benchmark snapshot per release.

Phased Delivery Plan

Acceptance Criteria and Evaluation Rubric

Acceptance is binary and evidence-based: every surface in Table 22.3 responds per its spec in an offline in-process test run; the conformance checker passes on the manifests folder; the SLO dashboard shows green against a recorded load test; a chaos drill (kill inventory mid-order) produces

Milestone	Theme	Exit criteria
M1 (weeks 1–4)	REST core	catalog + orders REST surface green on contract tests; RFC 9457 everywhere; keyset pagination; idempotency replay test passing.
M2 (weeks 5–8)	Read model + internal RPC	GraphQL read model live with depth limiting; inventory gRPC service with streaming <code>WatchStock</code> ; gateway auth + rate tiers.
M3 (weeks 9–12)	Async platform	outbox relay, signed webhooks with delivery log and replay, SSE telemetry stream; AsyncAPI specs linted in CI.
M4 (weeks 13–16)	Production hardening	OTel traces across all surfaces; SLO dashboards; conformance checker gating merges; load and failure-mode test evidence; portal published.

Table 22.4: Grand Capstone four-milestone delivery plan.

the designed circuit-breaker behavior [Nyg18].

The rubric scores five dimensions at 0–4 each: **correctness** (contract conformance), **resilience** (failure-mode behavior), **operability** (telemetry, runbooks), **governance** (specs, lint, portal), and **craft** (code quality, typing, tests). A passing Grand Capstone scores ≥ 16 with no dimension below 3.

22.2.4 Cross-Cutting Synthesis

The Failure Taxonomy, Revisited

Chapter by chapter, this book has built a taxonomy of how APIs fail. Table 22.5 consolidates it: for each failure class, where it is taught, how you detect it, and the designed mitigation.

One Page per Module

22.2.5 Emerging Frontiers, Honestly Assessed

A capstone chapter owes you more than enthusiasm: it owes you an adopt/trial/wait posture for each frontier, with the maturity caveats that vendor keynotes omit.

HTTP/3 and QUIC

HTTP/3 [Bis22] reimplements HTTP semantics over QUIC [Iye21], which replaces TCP + TLS 1.3 [Res18] with a UDP-based transport that integrates the handshake, provides per-stream loss recovery (eliminating TCP’s head-of-line blocking that HTTP/2 [Tho22; Peo15] merely mitigates), and supports connection migration across network changes — a direct win for mobile clients moving between radios. **Posture: trial.** Browsers and major CDNs already negotiate HTTP/3 via **Alt-Svc**, so public REST and SSE surfaces inherit it for free at the edge; but QUIC’s CPU cost at the server, UDP filtering in enterprise networks, and thinner debugging tooling mean you should measure before mandating it. gRPC stays on HTTP/2 in most stacks today.

APIs Consumed by AI Agents (MCP)

The newest consumer class is not a human developer or a mobile app but an AI agent invoking tools. The Model Context Protocol (MCP) standardizes how hosts discover and call such tools: a server publishes *tool schemas* — typed names, descriptions, and JSON Schema argument contracts [JSO22] — and the agent selects and invokes them at runtime. The implication for API design is profound: **your schema is now read by a model, not just by humans.** Descriptions, field naming, error shapes, and idempotency

Failure class	Chapter	Detection	Mitigation
Protocol misuse	2	semantic lint, packet capture	teach HTTP, not frameworks [Fie22a]
Client brittleness	3	contract tests	retries with backoff, timeouts [Enc24]
Ambiguous errors	8	error-shape lint	RFC 9457 problem details [Not23]
Page drift	9	duplicate/skipped rows	keyset cursors
Breaking change	10	spec diff in CI	date versions, Sunset [Wil19]
Duplicate side effects	11	replay tests	idempotency keys
Lost events	12	outbox lag, delivery log	transactional outbox [Kle17]
N+1 resolvers	13	query tracing	dataloaders, depth limits
Tight RPC coupling	14	stub fan-out audit	contract-first, versioned protos
Quota abuse	15	429 rate, tier stats	tiers + IETF headers [Pol24]
Cascading failure	16	latency waterfall	timeouts, breakers, bulkheads [Nyg18]
Credential leaks	17	log scans, secret scans	OWASP discipline [OWA23]
Snowflake deploys	18	drift detection	twelve-factor config [Wig17]
Blind operation	19	missing spans	OTel everywhere [Ope24]
Untested contracts	20	consumer complaints	consumer-driven contracts [Pac24]
Spec rot	21	portal/spec drift	executable governance [Ope21; Asy23]

Table 22.5: The consolidated failure taxonomy: one row per chapter’s core enemy.

Module I — Foundations (Ch. 1–6)		
Chapter	Core idea	Artifact you must produce
1	APIs as products and contracts	platform vision + consumer map
2	messages, semantics, connections [Fie22a; Fie22b; Tho22]	annotated request/response traces
3	disciplined clients	retrying, timeout-bound client [Enc24; Pyt24]
4	resources, uniformity, maturity [Fie00; RAR13]	resource model + OpenAPI draft
5	contracts over bytes [Bra17; JSO22]	versioned schemas
6	secrets, TLS, token handling [Res18; Sak15]	secure client checklist

Table 22.6: Module I synthesis: speak the protocol fluently before building.

Module II — Building APIs (Ch. 7–11)		
Chapter	Core idea	Artifact you must produce
7	typed, fast service scaffolding [Ram24; Col24]	working FastAPI service
8	errors as a designed interface [Not23]	problem-details catalog
9	stable traversal of changing data [Not17]	keyset-paginated collections
10	evolve without breaking	date versioning + Sunset policy [Wil19]
11	retries must be safe	idempotency-key middleware

Table 22.7: Module II synthesis: correctness is a designed property.

Module III — Protocols & Scale (Ch. 12–16)		
Chapter	Core idea	Artifact you must produce
12	decouple with events and push [Asy23]	outbox + webhook pipeline
13	client-driven selection [Gra21]	governed schema + persisted queries
14	typed, streaming RPC [gRP24; Goo24]	proto contracts + stubs
15	protect capacity, signal policy [Pol24]	tiered limiter + headers
16	resilience is engineered [Nyg18; New21]	breaker/timeout budget map

Table 22.8: Module III synthesis: pick profiles per constraint, scale deliberately.

Module IV — Production & Governance (Ch. 17–21)		
Chapter	Core idea	Artifact you must produce
17	defense in depth for APIs [OWA23; Har12; Jon15]	threat model + auth design
18	reproducible, portable services [Wig17]	container + CI pipeline
19	you cannot run what you cannot see [Ope24; Bey+16]	traces, SLOs, runbooks
20	contracts are the test surface [Pac24]	contract + chaos suites
21	governance as code [Ope21; GH21]	linter specs, portal, policy CI

Table 22.9: Module IV synthesis: production is a discipline, not a destination.

semantics become prompt-adjacent surface area. Tool schemas should be derived from the same OpenAPI/JSON Schema source of truth [Ope21] rather than hand-maintained, or the two contracts will drift — exactly the spec-rot failure of Chapter 21 in a new costume. RFC 9457 problem details [Not23] turn out to be agent-friendly errors: structured, typed, and self-describing. **Posture:** **trial** for internal tool ecosystems; **wait** before exposing MCP surfaces publicly until authorization and auditing patterns settle.

AI-Assisted API Design and Review

LLM assistants now draft OpenAPI documents, generate SDKs, and review spec diffs competently. The durable value is not generation — it is *review at scale*: an assistant can flag missing error responses, inconsistent pagination, or non-idiomatic naming across hundreds of endpoints. The durable risk is confident invention of plausible-but-wrong contracts. The professional pattern: AI proposes, executable policy (the conformance checker of Listing 3 (Section 22.3.3)) disposes. **Posture:** **adopt** for drafting and review, always behind deterministic CI gates.

Edge-Native APIs

Edge runtimes (worker isolates, regional endpoints) move request handling closer to the consumer, cutting TLS and TTFB latency and enabling regional data affinity for compliance. The architectural constraint is state: edge functions shine for read-heavy, cache-friendly, or geographically partitioned workloads and hurt when every request must reach a centralized strongly consistent store. Regional affinity — routing a tenant’s writes to their home region — is the pragmatic middle. **Posture:** **trial** for read-dominated surfaces (catalog!); **wait** for write-heavy cores.

Post-Quantum TLS Readiness

Harvest-now-decrypt-later attacks make cryptographic agility a present-tense concern. The ecosystem’s answer is hybrid key exchange in TLS 1.3 [Res18] (classical + post-quantum KEM in one handshake), already rolling out across browsers and CDNs. For API platforms the work is unglamorous: inventory where TLS is terminated, ensure libraries track current hybrids, watch handshake-size regressions that can break QUIC amplification limits, and keep certificate automation healthy so algorithm rotation is an upgrade, not a migration. **Posture:** **adopt** at the edge (inherited from your CDN); **trial** for east-west service mesh TLS.

API Security Automation

Attack surface discovery, schema-aware fuzzing, and runtime anomaly detection are converging into continuous API security testing keyed off your OpenAPI specs [Ope21; OWA23]. The spec-first discipline of Chapter 21 is the prerequisite: tools can only fuzz what they can see. **Posture: adopt** spec-driven scanning in CI; **trial** runtime behavioral detection.

Frontier	Posture	Prerequisite before adoption
HTTP/3 / QUIC [Bis22; Iye21]	trial	edge support inherited; measure server CPU
MCP tool schemas	trial (internal)	spec-first JSON Schema source of truth [JSO22]
AI-assisted design/review	adopt	deterministic lint gates behind every suggestion
Edge-native APIs	trial	read-heavy or regionally partitionable data
Post-quantum TLS	adopt (edge)	TLS inventory, automation, current libraries [Res18]
Security automation	adopt (CI)	complete, linted OpenAPI coverage [OWA23]

Table 22.10: Frontier adoption postures. Nothing here exempts you from the operational maturity of Module IV.

Warning

Frontiers amplify your existing maturity. HTTP/3 does not fix a missing caching strategy; MCP does not fix an unversioned schema; edge compute does not fix unbounded fan-out. Adopt frontiers *after* the boring disciplines of Chapters 17–21 are routine, or you will simply fail faster, in more places.

22.2.6 Career Pathways and the Mastery Checklist

API engineering is a T-shaped career: broad literacy across protocols and operations, deep expertise that grows with each level. Figure 22.3 maps the levels onto the book’s modules; Table 22.11 makes the progression checkable.

Staying current. The durable sources are the specification bodies and their errata, not social media: the IETF HTTP and QUIC working groups (RFCs 9110–9114 [Fie22a; Bis22]), the OpenAPI Initiative [Ope21], the

increasing scope: endpoint → service → system → organization

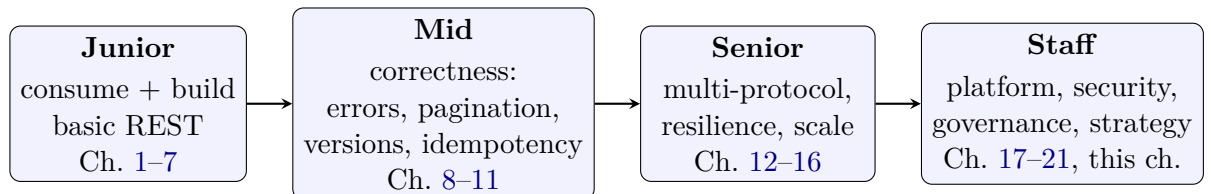


Figure 22.3: The API engineer mastery pathway. Levels are scopes of responsibility, not years of tenure.

Level	You can be trusted to...
Junior	read a packet capture; consume any documented REST API with retries and timeouts; build a typed FastAPI CRUD service; keep secrets out of code and logs.
Mid	design RFC 9457 error catalogs; implement keyset pagination and date versioning; make every mutation idempotent; write contract tests; explain your API's caching headers to a CDN engineer.
Senior	choose protocols from constraints and defend the choice; design outbox-based event pipelines; operate rate tiers; break production safely (timeouts, breakers, bulkheads) and prove it with chaos drills [Nyg18].
Staff	own a multi-protocol platform strategy; set SLOs and error budgets [Bey+16]; run governance as code; evaluate frontiers with adopt/trial/wait rigor; grow seniors through design review [GH21].

Table 22.11: The mastery checklist, by level.

AsyncAPI Initiative [Asy23], the GraphQL Foundation [Gra21], the CNCF (gRPC [gRP24], OpenTelemetry [Ope24]), and OWASP [OWA23]. Conferences worth your time: API World, APIDays, KubeCon/CloudNativeCon, QCon, and SREcon. Communities: vendor-neutral spec discussion lists and the conference talk archives. Budget one hour a week; reread foundational texts [Fie00; Kle17; New21] yearly — they reward rereading as your scope grows.

22.3 Production-Ready Code Implementation

Three integrative, complete programs implement this chapter’s ideas. All are offline-first (no network, no credentials), deterministic (no wall-clock or randomness on decision paths), fully typed, and shipped with executable tests. Setup (PowerShell, run once per machine):

```

1 cd "code\ch21"
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 pip install fastapi "strawberry-graphql[fastapi]"
   pydantic pyyaml httpx pytest
5 python -m pytest -q

```

22.3.1 Listing 1 — The Protocol-Selection Advisor

The advisor encodes the Master Decision Card of Section 22.2.1 as a scored questionnaire. Each criterion emits *signals*: weighted, human-readable point contributions to one or more profiles. The score for profile p is simply

$$\text{score}(p) = \sum_{s \in S, s.\text{profile}=p} s.\text{points} \quad (22.1)$$

and the recommendation is the argmax with a deterministic tie-break by profile declaration order. Because every signal carries its rationale, the recommendation ships with an *auditable trace* — you can show a reviewer exactly which constraints moved which profile, which is how the framework stays teachable instead of becoming an oracle.

```

1 """Protocol-selection advisor for the Master Decision
   Card.
2
3 Encodes the Chapter 21 decision framework as a scored
   questionnaire:
4 weighted criteria, deterministic arithmetic, and a full
   rationale trace

```

```

5 so_the_recommendation_can_be_audited_and_taught_from.
6
7 Offline, deterministic, standard library only. Python
8 3.11+.
9
10 from __future__ import annotations
11
12 from dataclasses import dataclass, field
13 from enum import Enum
14
15
16 class Profile(str, Enum):
17     """API interaction profiles compared by the advisor
18     ."""
19
20     REST = "REST"
21     GRAPHQL = "GraphQL"
22     GRPC = "gRPC"
23     EVENTS = "Event-driven(webhooks/broker)"
24     STREAM = "WebSockets/SSE"
25
26 class Consumer(str, Enum):
27     PUBLIC = "public-third-party"
28     MOBILE = "mobile-spa"
29     INTERNAL = "internal-service"
30     MACHINE = "machine-telemetry"
31
32
33 class Payload(str, Enum):
34     FIXED = "fixed-resources"
35     GRAPH = "nested-graph"
36     HIGH_VOLUME = "high-volume-streams"
37
38
39 class Latency(str, Enum):
40     RELAXED = "relaxed" # p99 >= 200 ms is fine
41     TIGHT = "tight" # interactive, p99 < 200
42         ms
43     EXTREME = "extreme" # p99 < 50 ms service-to-
44         service
45
46 class Delivery(str, Enum):
47     SYNC = "sync-request-response"

```

```

47     PUSH = "server-push-realtime"
48     ASYNC = "async-fire-and-forget"
49
50
51     class Team(str, Enum):
52         EXPLORATORY = "exploratory"
53         DISCIPLINED = "disciplined"
54
55
56     @dataclass(frozen=True)
57     class Requirements:
58         """Answers to the six-question selection
59         questionnaire."""
60
61         consumer: Consumer
62         payload: Payload
63         latency: Latency
64         delivery: Delivery
65         http_caching: bool
66         team: Team
67
68     @dataclass(frozen=True)
69     class Signal:
70         """One weighted contribution to a profile's score."
71         """
72
73         profile: Profile
74         points: int
75         criterion: str
76         rationale: str
77
78     @dataclass(frozen=True)
79     class Recommendation:
80         """Advisor output: ranked scores plus an auditable
81         trace."""
82
83         winner: Profile
84         scores: dict[Profile, int]
85         signals: tuple[Signal, ...] = field(default_factory=tuple)
86
87         def rationale(self, profile: Profile | None = None)
88             -> list[str]:
89             """Return the trace lines for one profile(

```

```

88         default: the_winner)."""
89     target = profile or self.winner
90     return [
91         f"[{s.points:+d}] {s.criterion}: {s.
92             rationale}"
93         for s in self.signals
94         if s.profile == target
95     ]
96
97 def _score(req: Requirements) -> tuple[Signal, ...]:
98     """Apply every rule of the Master Decision Card to
99     the answers."""
100     signals: list[Signal] = []
101
102     def add(profile: Profile, points: int, criterion:
103         str, rationale: str) -> None:
104         signals.append(Signal(profile, points,
105             criterion, rationale))
106
107     # --- Criterion 1: consumer type
108     -----
109     if req.consumer is Consumer.PUBLIC:
110         add(Profile.REST, 3, "consumer",
111             "public third parties expect plain HTTPS +
112             JSON they can curl")
113         add(Profile.EVENTS, 2, "consumer",
114             "webhooks are the standard public async
115             contract")
116         add(Profile.GRAPHQL, 1, "consumer",
117             "powerful for public developers but raises
118             the support burden")
119         add(Profile.GRPC, -3, "consumer",
120             "browser clients cannot speak raw HTTP/2
121             gRPC without a proxy")
122         add(Profile.STREAM, 1, "consumer",
123             "SSE/WebSocket endpoints are feasible but
124             operationally niche")
125     elif req.consumer is Consumer.MOBILE:
126         add(Profile.GRAPHQL, 3, "consumer",
127             "clients pick exactly the fields they
128             render; no over-fetching")
129         add(Profile.REST, 2, "consumer",
130             "aggregating BFF-style REST resources also
131             solve over-fetching")
132         add(Profile.STREAM, 1, "consumer",

```

```

121         "push_channels_pair_well_with_mobile_UX")
122     add(Profile.GRPC, -2, "consumer",
123         "gRPC-Web adds a proxy hop that mobile
124         teams rarely want")
125     elif req.consumer is Consumer.INTERNAL:
126         add(Profile.GRPC, 3, "consumer",
127             "strongly typed, streaming, low-overhead
128             service-to-service RPC")
129         add(Profile.REST, 1, "consumer",
130             "REST remains the default when tooling
131             maturity is low")
132         add(Profile.EVENTS, 1, "consumer",
133             "async integration decouples internal
134             services")
135         add(Profile.GRAPHQL, 1, "consumer",
136             "federated graphs can front many internal
137             services")
138     else: # MACHINE telemetry
139         add(Profile.EVENTS, 3, "consumer",
140             "telemetry is naturally asynchronous and
141             bursty")
142         add(Profile.GRPC, 2, "consumer",
143             "bidirectional streaming fits high-rate
144             device channels")
145         add(Profile.STREAM, 1, "consumer",
146             "persistent sockets work where HTTP/2 is
147             unavailable")
148         add(Profile.REST, -1, "consumer",
149             "per-reading REST calls waste headers and
150             connections")
151
152     # --- Criterion 2: payload shape
153     -----
154     if req.payload is Payload.GRAPH:
155         add(Profile.GRAPHQL, 3, "payload",
156             "nested graph selection sets are GraphQL's
157             home turf")
158         add(Profile.REST, -1, "payload",
159             "deep graphs force N+1 endpoint hops in
160             REST")
161     elif req.payload is Payload.HIGH_VOLUME:
162         add(Profile.GRPC, 2, "payload",
163             "compact binary frames and HTTP/2 streams
164             suit high volume")
165         add(Profile.EVENTS, 2, "payload",
166             "brokered batches absorb volume spikes")

```



```

154         add(Profile.REST, -1, "payload",
155             "textual_JSON_per_request_is_the_most_
              expensive_option")
156     else:
157         add(Profile.REST, 2, "payload",
158             "fixed_resource_shapes_map_one-to-one_onto_
              REST_resources")
159
160     # --- Criterion 3: latency budget
161     -----
162     if req.latency is Latency.EXTREME:
163         add(Profile.GRPC, 3, "latency",
164             "HTTP/2_multiplexing_plus_ProtoBuf_minimize_
              per-call_overhead")
165         add(Profile.GRAPHQL, -1, "latency",
166             "query_planning_and_JSON_add_milliseconds_
              per_request")
167         add(Profile.REST, 0, "latency",
168             "acceptable_with_keep-alive_and_HTTP/2,_but_
              not_minimal")
169     elif req.latency is Latency.TIGHT:
170         add(Profile.GRPC, 1, "latency", "comfortable_
              within_tight_budgets")
171         add(Profile.REST, 1, "latency", "comfortable_
              within_tight_budgets")
172     else:
173         add(Profile.EVENTS, 2, "latency",
174             "async_delivery_tolerates_relaxed_latency_
              by_design")
175
176     # --- Criterion 4: delivery model (hard structural
177     signals) -----
178     if req.delivery is Delivery.PUSH:
179         add(Profile.STREAM, 4, "delivery",
180             "server_push_requires_SSE_or_WebSockets;_
              unary_APIs_cannot_push")
181         add(Profile.GRPC, 1, "delivery",
182             "server-streaming_RPC_works_for_non-browser_
              clients")
183         add(Profile.REST, -3, "delivery",
184             "unary_REST_has_no_server-push_channel")
185         add(Profile.GRAPHQL, -2, "delivery",
186             "GraphQL_subscriptions_need_a_socket_
              transport_anyway")
187     elif req.delivery is Delivery.ASYNC:
188         add(Profile.EVENTS, 4, "delivery",

```

```

187         "fire-and-forget_delivery_is_the_defining_
           event-driven_case")
188     else:
189         add(Profile.REST, 1, "delivery",
190             "synchronous_request-response_is_REST's_
              native_shape")
191         add(Profile.GRPC, 1, "delivery",
192             "unary_RPC_is_also_synchronous_request-
              response")
193
194     # --- Criterion 5: HTTP caching
195     -----
196     if req.http_caching:
197         add(Profile.REST, 3, "caching",
198             "only_REST_leverages_shared/CDN_caching_of_
              GET_responses")
199         add(Profile.GRAPHQL, -2, "caching",
200             "POST-based_queries_bypass_standard_HTTP_
              caches")
201         add(Profile.GRPC, -2, "caching",
202             "gRPC_calls_are_not_addressable_cacheable_
              resources")
203     else:
204         add(Profile.GRAPHQL, 1, "caching",
205             "no_penalty_when_edge_caching_is_not_
              required")
206
207     # --- Criterion 6: team maturity
208     -----
209     if req.team is Team.EXPLORATORY:
210         add(Profile.REST, 2, "team",
211             "REST_has_the_shallowest_learning_curve_and_
              richest_tooling")
212         add(Profile.GRPC, -1, "team",
213             "Protobuf_toolchains_and_reflection_quirks_
              slow_new_teams")
214         add(Profile.GRAPHQL, -1, "team",
215             "schema_governance_and_N+1_discipline_
              require_experience")
216     else:
217         add(Profile.GRAPHQL, 1, "team",
218             "disciplined_teams_can_exploit_schema-first_
              workflows")
219         add(Profile.GRPC, 1, "team",
220             "disciplined_teams_can_run_contract-first_
              RPC_safely")

```

```

219         return tuple(signals)
220
221
222
223 def recommend(req: Requirements) -> Recommendation:
224     """Score all profiles and return the winner with a
225         rationale trace.
226
227     Ties break deterministically by profile enum order,
228         so the same
229     answers always produce the same recommendation.
230     """
231     signals = _score(req)
232     scores = {profile: 0 for profile in Profile}
233     for signal in signals:
234         scores[signal.profile] += signal.points
235     winner = max(Profile, key=lambda p: (scores[p], -
236         list(Profile).index(p)))
237     return Recommendation(winner=winner, scores=scores,
238         signals=signals)
239
240
241 # --- Canonical scenarios from the chapter's worked
242     examples -----
243
244 PUBLIC_SAAS = Requirements(
245     consumer=Consumer.PUBLIC,
246     payload=Payload.FIXED,
247     latency=Latency.RELAXED,
248     delivery=Delivery.SYNC,
249     http_caching=True,
250     team=Team.DISCIPLINED,
251 )
252
253 INTERNAL_MESH = Requirements(
254     consumer=Consumer.INTERNAL,
255     payload=Payload.FIXED,
256     latency=Latency.EXTREME,
257     delivery=Delivery.SYNC,
258     http_caching=False,
259     team=Team.DISCIPLINED,
260 )
261
262 MOBILE_BACKEND = Requirements(
263     consumer=Consumer.MOBILE,
264     payload=Payload.GRAPH,

```

```

260     latency=Latency.TIGHT,
261     delivery=Delivery.SYNC,
262     http_caching=False,
263     team=Team.DISCIPLINED,
264 )
265
266 IOT_TELEMETRY = Requirements(
267     consumer=Consumer.MACHINE,
268     payload=Payload.HIGH_VOLUME,
269     latency=Latency.RELAXED,
270     delivery=Delivery.ASYNC,
271     http_caching=False,
272     team=Team.DISCIPLINED,
273 )
274
275 PARTNER_NOTIFICATIONS = Requirements(
276     consumer=Consumer.PUBLIC,
277     payload=Payload.FIXED,
278     latency=Latency.RELAXED,
279     delivery=Delivery.ASYNC,
280     http_caching=False,
281     team=Team.EXPLORATORY,
282 )
283
284
285 if __name__ == "__main__":
286     scenarios = {
287         "Public_SaaS_API": PUBLIC_SAAS,
288         "Internal_microservice_mesh": INTERNAL_MESH,
289         "Mobile_app_backend": MOBILE_BACKEND,
290         "IoT_telemetry": IOT_TELEMETRY,
291         "Partner_notifications": PARTNER_NOTIFICATIONS,
292     }
293     for name, reqs in scenarios.items():
294         rec = recommend(reqs)
295         print(f"{name}: {rec.winner.value}")
296         for line in rec.rationale():
297             print(f"    {line}")

```

The unit tests lock the five canonical worked scenarios to their expected recommendations and assert the framework invariants: determinism, score arithmetic (scores equal the sum of their signals), and trace integrity.

```

1  """Unit tests for the protocol-selection advisor.
2
3  Covers the five canonical scenarios from the chapter
   plus framework

```

```
4 invariants_(determinism, scoring_arithmetic, rationale_  
5     trace).  
6  
7 from __future__ import annotations  
8  
9 from protocol_advisor import (  
10     IOT_TELEMETRY,  
11     INTERNAL_MESH,  
12     MOBILE_BACKEND,  
13     PARTNER_NOTIFICATIONS,  
14     PUBLIC_SAAS,  
15     Consumer,  
16     Delivery,  
17     Latency,  
18     Payload,  
19     Profile,  
20     Requirements,  
21     Team,  
22     recommend,  
23 )  
24  
25  
26 def test_public_saas_recommends_rest() -> None:  
27     assert recommend(PUBLIC_SAAS).winner is Profile.  
28     REST  
29  
30 def test_internal_mesh_recommends_grpc() -> None:  
31     assert recommend(INTERNAL_MESH).winner is Profile.  
32     GRPC  
33  
34 def test_mobile_backend_recommends_graphql() -> None:  
35     assert recommend(MOBILE_BACKEND).winner is Profile.  
36     GRAPHQL  
37  
38 def test_iot_telemetry_recommends_events() -> None:  
39     assert recommend(IOT_TELEMETRY).winner is Profile.  
40     EVENTS  
41  
42 def test_partner_notifications_recommend_events() ->  
43     None:  
44     assert recommend(PARTNER_NOTIFICATIONS).winner is
```

```

44         Profile.EVENTS
45
46 def test_push_delivery_selects_streams() -> None:
47     reqs = Requirements(
48         consumer=Consumer.PUBLIC,
49         payload=Payload.FIXED,
50         latency=Latency.TIGHT,
51         delivery=Delivery.PUSH,
52         http_caching=False,
53         team=Team.DISCIPLINED,
54     )
55     assert recommend(reqs).winner is Profile.STREAM
56
57
58 def test_recommendation_is_deterministic() -> None:
59     first = recommend(PUBLIC_SAAS)
60     second = recommend(PUBLIC_SAAS)
61     assert first == second
62
63
64 def test_scores_equal_sum_of_signals() -> None:
65     rec = recommend(MOBILE_BACKEND)
66     for profile in Profile:
67         total = sum(s.points for s in rec.signals if s.
68                     profile is profile)
69         assert rec.scores[profile] == total
70
71
72 def test_winner_has_strictly_highest_score() -> None:
73     rec = recommend(IOT_TELEMETRY)
74     best = max(rec.scores.values())
75     assert rec.scores[rec.winner] == best
76     assert list(rec.scores.values()).count(best) >= 1
77
78 def test_rationale_trace_references_winner_only() ->
79     None:
80     rec = recommend(INTERNAL_MESH)
81     trace = rec.rationale()
82     assert trace, "winner must have at least one
83         rationale line"
84     for line in trace:
85         assert ":" in line and line.startswith("[")

```

Running the demo driver prints the recommendation and the rationale

trace for each canonical scenario:

```

1 Public SaaS API: REST
2   [+3] consumer: public third parties expect plain
      HTTPS + JSON they can curl
3   [+2] payload: fixed resource shapes map one-to-one
      onto REST resources
4   [+1] delivery: synchronous request-response is REST
      's native shape
5   [+3] caching: only REST leverages shared/CDN
      caching of GET responses
6 Internal microservice mesh: gRPC
7   [+3] consumer: strongly typed, streaming, low-
      overhead service-to-service RPC
8   [+3] latency: HTTP/2 multiplexing plus Protobuf
      minimize per-call overhead
9   [+1] delivery: unary RPC is also synchronous
      request-response
10  [+1] team: disciplined teams can run contract-first
      RPC safely
11 Mobile app backend: GraphQL
12  [+3] consumer: clients pick exactly the fields they
      render; no over-fetching
13  [+3] payload: nested graph selection sets are
      GraphQL's home turf
14  [+1] caching: no penalty when edge caching is not
      required
15  [+1] team: disciplined teams can exploit schema-
      first workflows
16 IoT telemetry: Event-driven (webhooks/broker)
17  [+3] consumer: telemetry is naturally asynchronous
      and bursty
18  [+2] payload: brokered batches absorb volume spikes
19  [+2] latency: async delivery tolerates relaxed
      latency by design
20  [+4] delivery: fire-and-forget delivery is the
      defining event-driven case
21 Partner notifications: Event-driven (webhooks/broker)
22  [+2] consumer: webhooks are the standard public
      async contract
23  [+2] latency: async delivery tolerates relaxed
      latency by design
24  [+4] delivery: fire-and-forget delivery is the
      defining event-driven case

```

22.3.2 Listing 2 — The Northwind Platform Slice

One FastAPI application [Ram24; Col24] assembling the book’s patterns into a single deployable unit: a keyset-paginated REST catalog (Chapters 7, 9), a GraphQL read model joining catalog rows to inventory availability (Chapter 13), an SSE telemetry stream (Chapter 12), RFC 9457 problem-details errors from every failure path [Not23] (Chapter 8), idempotency keys on order creation (Chapter 11), and IETF `RateLimit` headers from a fixed-window limiter [Pol24] (Chapter 15). State is injected through the application factory so every test gets an isolated platform — the deterministic-test discipline of Chapter 20.

```

1  """Northwind_Robotics_API_Platform--integrative_slice
   _ (Chapter_21) .
2
3  One_FastAPI_application_assembling_the_book's_core_
   patterns_into_a
4  single_deployable_unit:
5
6  *_REST_catalog_with_keyset_pagination_ (Ch._6, _
   Ch._8)
7  *_GraphQL_read_model_over_the_same_domain_data_ (Ch._12)
8  *_SSE_telemetry_stream_ (Ch._11)
9  *_RFC_9457_problem-details_errors_ (Ch._7)
10 *_Idempotency_keys_on_order_creation_ (Ch._10)
11 *_IETF_rate-limit_response_headers_ (Ch._14)
12
13 Offline, _deterministic, _synthetic_data_only. _Python_
   3.11+.
14 """
15
16 from __future__ import annotations
17
18 import base64
19 import threading
20 from collections.abc import AsyncIterator
21 from dataclasses import dataclass, field
22
23 import strawberry
24 import uvicorn
25 from fastapi import FastAPI, Query, Request
26 from fastapi.exceptions import RequestValidationError
27 from fastapi.responses import JSONResponse,
   StreamingResponse
28 from pydantic import BaseModel, Field
29 from strawberry.fastapi import GraphQLRouter

```



```

30
31 # --- Domain data (synthetic Northwind Robotics
    universe) -----
32
33 PRODUCTS: list[dict[str, object]] = [
34     {
35         "id": 1,
36         "sku": "NWR-ARM-6X",
37         "name": "Northwind_6-Axis_Arm",
38         "category": "manipulators",
39         "unit_price_cents": 12_500_00,
40     },
41     {
42         "id": 2,
43         "sku": "NWR-LDR-360",
44         "name": "LiDAR_360_Scanner",
45         "category": "sensors",
46         "unit_price_cents": 3_499_00,
47     },
48     {
49         "id": 3,
50         "sku": "NWR-DRV-BLDC",
51         "name": "BLDC_Motor_Driver",
52         "category": "actuators",
53         "unit_price_cents": 189_99,
54     },
55 ]
56
57 AVAILABILITY: dict[str, int] = {
58     "NWR-ARM-6X": 4,
59     "NWR-LDR-360": 12,
60     "NWR-DRV-BLDC": 61,
61 }
62
63 ROBOT_TELEMETRY: dict[str, dict[str, float | int]] = {
64     "R-101": {"motor_temp_c": 41.2, "cycles": 18233},
65     "R-102": {"motor_temp_c": 38.7, "cycles": 12004},
66     "R-103": {"motor_temp_c": 44.9, "cycles": 25077},
67 }
68
69 DEMO_API_KEY = "dev-key-northwind"
70 RATE_LIMIT = 30
71 API_VERSION = "2026-09-01"
72
73
74 # --- Errors: RFC 9457 problem details (Ch. 7)

```

```

-----
75
76 def problem(status: int, title: str, detail: str, type_
: str = "about:blank") -> JSONResponse:
77     """Build an RFC 9457 'application/problem+json'
response."""
78     return JSONResponse(
79         status_code=status,
80         media_type="application/problem+json",
81         content={"type": type_, "title": title, "status
": status, "detail": detail},
82     )
83
84
85 # --- Platform state: rate limiting (Ch. 14) and
idempotency (Ch. 10) -----
86
87 @dataclass
88 class PlatformState:
89     """Mutable application state guarded by one lock (
demo-scale)."""
90
91     lock: threading.Lock = field(default_factory=
threading.Lock)
92     remaining: int = RATE_LIMIT
93     orders: dict[str, dict[str, object]] = field(
default_factory=dict)
94     next_order_id: int = 1
95
96
97 class CursorError(ValueError):
98     """Raised when a keyset pagination cursor cannot be
decoded."""
99
100
101 # --- Schemas
-----
102
103 class ProductOut(BaseModel):
104     id: int
105     sku: str
106     name: str
107     category: str
108     unit_price_cents: int
109

```

```

110
111 class ProductPage(BaseModel):
112     items: list[ProductOut]
113     next_cursor: str | None = None
114     api_version: str
115
116
117 class OrderIn(BaseModel):
118     sku: str
119     quantity: int = Field(ge=1, le=100)
120
121
122 class OrderOut(BaseModel):
123     order_id: str
124     sku: str
125     quantity: int
126     status: str
127     api_version: str
128
129
130 def _encode_cursor(product_id: int) -> str:
131     return base64.urlsafe_b64encode(f"cursor:{
132         product_id}".encode()).decode()
133
134
135 def _decode_cursor(cursor: str) -> int:
136     raw = base64.urlsafe_b64decode(cursor.encode()).
137         decode()
138     prefix, _, value = raw.partition(":")
139     if prefix != "cursor" or not value.isdigit():
140         raise ValueError(f"malformed cursor: {cursor!r}")
141     return int(value)
142
143
144 # --- GraphQL read model (Ch. 12)
145     -----
146
147 @strawberry.type
148 class CatalogItem:
149     sku: str
150     name: str
151     category: str
152     unit_price_cents: int
153     availability: int

```

```

152
153 @strawberry.type
154 class RobotStatus:
155     robot_id: str
156     motor_temp_c: float
157     cycles: int
158
159
160 @strawberry.type
161 class GraphQLQuery:
162     @strawberry.field
163     def catalog(self, category: str | None = None) ->
164         list[CatalogItem]:
165         rows = PRODUCTS
166         if category is not None:
167             rows = [p for p in rows if p["category"] ==
168                 category]
169         return [
170             CatalogItem(
171                 sku=str(p["sku"]),
172                 name=str(p["name"]),
173                 category=str(p["category"]),
174                 unit_price_cents=int(p["
175                     unit_price_cents"]),
176                 availability=AVAILABILITY.get(str(p["
177                     sku"]), 0),
178             )
179             for p in rows
180         ]
181
182     @strawberry.field
183     def robot_status(self, robot_id: str) ->
184         RobotStatus | None:
185         row = ROBOT_TELEMETRY.get(robot_id)
186         if row is None:
187             return None
188         return RobotStatus(
189             robot_id=robot_id,
190             motor_temp_c=float(row["motor_temp_c"]),
191             cycles=int(row["cycles"]),
192         )
193
194 graphql_router = GraphQLRouter(strawberry.Schema(query=
195     GraphQLQuery))
196

```

```

192
193 # --- Application factory
194
195 def create_app(state: PlatformState | None = None) ->
FastAPI:
196     """Build the platform app; inject state to keep
tests isolated."""
197     app = FastAPI(title="Northwind Robotics API
Platform", version="1.0.0")
198     platform = state or PlatformState()
199
200     @app.middleware("http")
201     async def platform_middleware(request: Request,
call_next): # type: ignore[no-untyped-def]
202         if request.url.path in ("/healthz", "/docs", "/
openapi.json"):
203             return await call_next(request)
204         if request.headers.get("X-API-Key") !=
DEMO_API_KEY:
205             return problem(401, "Unauthorized", "
missing or invalid X-API-Key header")
206         with platform.lock:
207             if platform.remaining <= 0:
208                 return JSONResponse(
209                     status_code=429,
210                     media_type="application/problem+
json",
211                     headers={"Retry-After": "30", "
RateLimit": f'limit={RATE_LIMIT
}'},
212                     content={
213                         "type": "about:blank",
214                         "title": "Too Many Requests",
215                         "status": 429,
216                         "detail": "quota exhausted;
retry after the Retry-After
window",
217                     },
218                 )
219             platform.remaining -= 1
220             remaining = platform.remaining
221             response = await call_next(request)
222             response.headers["RateLimit"] = f'limit={
RATE_LIMIT}, remaining={remaining}'

```

```

223         response.headers["RateLimit-Remaining"] = str(
224             remaining)
225         return response
226
227 @app.exception_handler(RequestValidationError)
228 async def validation_handler(
229     _request: Request, exc: RequestValidationError
230 ) -> JSONResponse:
231     detail = ";".join(
232         f"{'.'.join(str(p) for p in err['loc'])}: {err['msg']}" for err in exc.errors()
233     )
234     return problem(422, "Unprocessable Entity",
235                    detail)
236
237 # --- REST catalog (Ch. 6 + Ch. 8)
238 -----
239
240 @app.get("/v1/products", response_model=ProductPage)
241 )
242 def list_products(
243     category: str | None = None,
244     limit: int = Query(default=2, ge=1, le=50),
245     cursor: str | None = None,
246 ) -> ProductPage:
247     rows = [p for p in PRODUCTS if category is None
248             or p["category"] == category]
249     rows.sort(key=lambda p: int(p["id"]))
250     if cursor is not None:
251         try:
252             after = _decode_cursor(cursor)
253         except ValueError as exc:
254             raise CursorError(str(exc)) from exc
255     rows = [p for p in rows if int(p["id"]) >
256            after]
257     page = rows[:limit]
258     next_cursor = (
259         _encode_cursor(int(page[-1]["id"])) if len(
260             rows) > limit else None
261     )
262     return ProductPage(
263         items=[ProductOut(**p) for p in page], #
264         type: ignore[arg-type]
265         next_cursor=next_cursor,
266         api_version=API_VERSION,
267     )
268
269

```

```

260 @app.exception_handler(CursorError)
261 async def cursor_handler(_request: Request, exc:
      CursorError) -> JSONResponse:
262     return problem(400, "Invalid_Cursor", str(exc))
263
264 # --- Orders with idempotency keys (Ch. 10)
      -----
265 @app.post("/v1/orders", response_model=OrderOut,
      status_code=201)
266 def create_order(order: OrderIn, request: Request)
      -> OrderOut | JSONResponse:
267     key = request.headers.get("Idempotency-Key")
268     if not key:
269         return problem(
270             400,
271             "Missing_Idempotency-Key",
272             "POST_v1/orders_requires_an_
                Idempotency-Key_header",
273         )
274     with platform.lock:
275         existing = platform.orders.get(key)
276         if existing is not None:
277             if existing["payload"] != order.
                model_dump():
278                 return problem(
279                     422,
280                     "Idempotency_Conflict",
281                     "key_reused_with_a_different_
                        request_payload",
282                 )
283             return OrderOut(
284                 order_id=str(existing["order_id"]),
285                 sku=order.sku,
286                 quantity=order.quantity,
287                 status="accepted",
288                 api_version=API_VERSION,
289             )
290         if order.sku not in AVAILABILITY:
291             return problem(404, "Unknown_SKU", f"no
                _catalog_entry_for_{order.sku!r}")
292         order_id = f"ORD-{platform.next_order_id:04
                d}"
293         platform.next_order_id += 1
294         platform.orders[key] = {
295             "order_id": order_id,
296             "payload": order.model_dump(),

```

```

297         }
298         return OrderOut(
299             order_id=order_id,
300             sku=order.sku,
301             quantity=order.quantity,
302             status="accepted",
303             api_version=API_VERSION,
304         )
305
306     # --- SSE telemetry stream (Ch. 11)
307     -----
308     @app.get("/v1/telemetry/stream")
309     async def telemetry_stream() -> StreamingResponse:
310         async def events() -> AsyncIterator[str]:
311             event_id = 0
312             for robot_id in sorted(ROBOT_TELEMETRY):
313                 row = ROBOT_TELEMETRY[robot_id]
314                 event_id += 1
315                 yield (
316                     f"id: {event_id}\n"
317                     f"event: robot-reading\n"
318                     f'data: {{"robot_id": "{robot_id}",\n'
319                     f'               "\n'
320                     f'               "motor_temp_c": {row["\n'
321                     f'               motor_temp_c"]},\n'
322                     f'               "cycles": {row["cycles"]}}}\n\n'
323                 )
324             )
325         return StreamingResponse(
326             events(),
327             media_type="text/event-stream",
328             headers={"Cache-Control": "no-cache"},
329         )
330
331     @app.get("/healthz")
332     def healthz() -> dict[str, str]:
333         return {"status": "ok"}
334
335     app.include_router(graphql_router, prefix="/graphql")
336
337     return app
338
339 if __name__ == "__main__":
340     uvicorn.run(create_app(), host="127.0.0.1", port=8121)

```


The smoke suite proves the surfaces work *together* — through one middleware stack, with one state — using an in-process ASGI client, so no sockets, servers, or network are involved:

```

1  """In-process smoke tests for the Northwind platform
   slice.
2
3  Every request runs through httpx + ASGITransport: no
   sockets, no
4  network, fully deterministic. The suite proves the four
   surfaces
5  (REST, GraphQL, SSE, problem-details errors) work
   together on one app.
6  """
7
8  from __future__ import annotations
9
10 import json
11 from collections.abc import Iterator
12
13 import pytest
14 from fastapi.testclient import TestClient
15
16 from northwind_platform import DEMO_API_KEY, create_app
17
18 AUTH = {"X-API-Key": DEMO_API_KEY}
19
20
21 @pytest.fixture()
22 def client() -> Iterator[TestClient]:
23     """Fresh app + in-process client per test (isolated
       rate limiter)."""
24     with TestClient(create_app()) as c:
25         yield c
26
27
28 # --- Auth and rate limiting
   -----
29
30 def test_missing_api_key_is_problem_json(client:
   TestClient) -> None:
31     r = client.get("/v1/products")
32     assert r.status_code == 401
33     assert r.headers["content-type"] == "application/
       problem+json"
34     body = r.json()

```

```

35     assert body["status"] == 401
36     assert {"type", "title", "status", "detail"} <=
        body.keys()
37
38
39 def test_rate_limit_headers_present(client: TestClient)
    -> None:
40     r = client.get("/v1/products", headers=AUTH)
41     assert r.status_code == 200
42     assert "RateLimit" in r.headers
43     assert int(r.headers["RateLimit-Remaining"]) < 30
44
45
46 def test_rate_limit_exhaustion_returns_429() -> None:
47     with TestClient(create_app()) as c:
48         last = None
49         for _ in range(31):
50             last = c.get("/v1/products", headers=AUTH)
51             assert last is not None and last.status_code ==
                429
52             assert last.headers["Retry-After"] == "30"
53
54
55 # --- REST catalog
    -----

56
57 def test_product_page_one_and_cursor(client: TestClient
    ) -> None:
58     page1 = client.get("/v1/products", headers=AUTH,
        params={"limit": 2}).json()
59     assert len(page1["items"]) == 2
60     assert page1["next_cursor"] is not None
61     assert page1["api_version"]
62
63     page2 = client.get(
64         "/v1/products",
65         headers=AUTH,
66         params={"limit": 2, "cursor": page1["
            next_cursor"]},
67     ).json()
68     assert len(page2["items"]) == 1
69     assert page2["next_cursor"] is None
70     ids = [p["id"] for p in page1["items"] + page2["
        items"]]
71     assert ids == [1, 2, 3]

```

```

72
73
74 def test_bad_cursor_is_problem_400(client: TestClient)
    -> None:
75     r = client.get(
76         "/v1/products", headers=AUTH, params={"cursor":
            "not-a-cursor"}
77     )
78     assert r.status_code == 400
79     assert r.json()["title"] == "Invalid_Cursor"
80
81
82 def test_validation_error_is_problem_422(client:
    TestClient) -> None:
83     r = client.get("/v1/products", headers=AUTH, params
        ={"limit": 0})
84     assert r.status_code == 422
85     assert r.headers["content-type"] == "application/
        problem+json"
86
87
88 # --- Orders + idempotency
    -----
89
90 def test_order_requires_idempotency_key(client:
    TestClient) -> None:
91     r = client.post("/v1/orders", headers=AUTH, json={"
        sku": "NWR-ARM-6X", "quantity": 1})
92     assert r.status_code == 400
93
94
95 def test_order_replay_returns_same_order_id(client:
    TestClient) -> None:
96     headers = {**AUTH, "Idempotency-Key": "smoke-key-1"
        }
97     payload = {"sku": "NWR-LDR-360", "quantity": 2}
98     first = client.post("/v1/orders", headers=headers,
        json=payload)
99     second = client.post("/v1/orders", headers=headers,
        json=payload)
100     assert first.status_code == 201
101     assert second.json()["order_id"] == first.json()["
        order_id"]
102
103

```

```

104 def test_idempotency_key_conflict_is_422(client:
    TestClient) -> None:
105     headers = {**AUTH, "Idempotency-Key": "smoke-key-2"}
106     client.post("/v1/orders", headers=headers, json={
        "sku": "NWR-ARM-6X", "quantity": 1})
107     conflict = client.post(
108         "/v1/orders", headers=headers, json={"sku": "
            NWR-ARM-6X", "quantity": 9}
109     )
110     assert conflict.status_code == 422
111     assert conflict.json()["title"] == "Idempotency_
        Conflict"
112
113
114 def test_unknown_sku_is_problem_404(client: TestClient)
    -> None:
115     headers = {**AUTH, "Idempotency-Key": "smoke-key-3"}
116     r = client.post("/v1/orders", headers=headers, json=
        {"sku": "NOPE-1", "quantity": 1})
117     assert r.status_code == 404
118     assert r.json()["status"] == 404
119
120
121 # --- GraphQL read model
    -----

122
123 def test_graphql_catalog_joins_availability(client:
    TestClient) -> None:
124     query = "{_catalog_{_sku_name_availability_}_}"
125     r = client.post("/graphql", headers=AUTH, json={"
        query": query})
126     assert r.status_code == 200
127     rows = {item["sku"]: item for item in r.json()["
        data"]["catalog"]}
128     assert rows["NWR-LDR-360"]["availability"] == 12
129
130
131 def test_graphql_robot_status(client: TestClient) ->
    None:
132     query = '{_robotStatus(robotId:_R-101)_{_robotId_
        motorTempC_cycles_}_}'
133     r = client.post("/graphql", headers=AUTH, json={"
        query": query})

```

```

134     assert r.status_code == 200
135     status = r.json()["data"]["robotStatus"]
136     assert status["robotId"] == "R-101"
137     assert status["motorTempC"] == pytest.approx(41.2)
138
139
140 # --- SSE telemetry
141 -----
142
143 def test_sse_stream_yields_three_robot_events(client:
144     TestClient) -> None:
145     with client.stream("GET", "/v1/telemetry/stream",
146         headers=AUTH) as r:
147         assert r.status_code == 200
148         assert r.headers["content-type"].startswith("
149             text/event-stream")
150         body = "".join(r.iter_text())
151         events = [chunk for chunk in body.strip().split("\n
152             \n") if chunk]
153         assert len(events) == 3
154         assert all(chunk.startswith("id:") for chunk in
155             events)
156         payload = events[0].split("data:", 1)[1]
157         assert json.loads(payload)["robot_id"] == "R-101"

```

22.3.3 Listing 3 — The Architecture Conformance Checker

Chapter 21 argued that governance must be executable or it is decorative. This checker is the proof: it loads every OpenAPI [Ope21] and AsyncAPI [Asy23] manifest in a folder and verifies the platform’s invariants — security schemes with descriptions and a global `security` requirement (O-001), RFC 9457 error shapes on every 4xx/5xx response (O-002) [Not23], keyset pagination conventions on collection GETs (O-003), date-based version identifiers (O-004), and the event envelope on AsyncAPI publish payloads (A-001). Local `$ref` resolution keeps the checker honest against real spec style.

```

1  """Architecture_conformance_checker_(Chapter_21) .
2
3  Turns_the_book's_governance_rules_into_executable_
4  policy._Given_a
5  manifests_folder_of_OpenAPI_(YAML/JSON)_and_AsyncAPI_
6  documents,_the

```

```

5 checker verifies the platform invariants that Chapter
   20 establishes
6 as linted, CI-enforced rules:
7
8 *0-001: every spec declares a security scheme with a
   non-empty
9   'description' and applies 'security' globally.
10 *0-002: every 4xx/5xx response references the shared
11   'ProblemDetails' component (RFC 9457).
12 *0-003: every collection GET documents 'limit' and
   'cursor'
13   query parameters and a 'nextCursor' response field.
14 *0-004: the 'info.version' header version matches
15   'YYYY-MM-DD' (the platform's date-based versioning
   rule).
16 *A-001: every AsyncAPI publish payload message defines
   'event_id'
17   and 'occurred_at' (the platform's event envelope).
18
19 Offline, deterministic. Requires only PyYAML. Python
   3.11+.
20 """
21
22 from __future__ import annotations
23
24 import re
25 import sys
26 from dataclasses import dataclass
27 from pathlib import Path
28 from typing import Any
29
30 import yaml
31
32 DATE_VERSION = re.compile(r"^\d{4}-\d{2}-\d{2}$")
33 ENVELOPE_FIELDS = ("event_id", "occurred_at")
34
35
36 @dataclass(frozen=True)
37 class Violation:
38     """A single governance rule breach in one manifest.
39     """
40     rule: str
41     spec: str
42     detail: str
43

```

```

44     def render(self) -> str:
45         return f"[{self.rule}]_{self.spec}:_{self.
            detail}"
46
47
48 def _load_specs(folder: Path) -> dict[str, Any]:
49     specs: dict[str, Any] = {}
50     for path in sorted(folder.rglob("*")):
51         if path.suffix.lower() in {".yaml", ".yml", ".
            json"}:
52             specs[str(path.relative_to(folder))] = yaml
                .safe_load(
53                 path.read_text(encoding="utf-8")
54             )
55     return specs
56
57
58 def _resolve_local_ref(document: Any, node: Any, depth:
    int = 0) -> Any:
59     """Resolve one local '#/...' reference against
        its own document."""
60     if depth > 10 or not isinstance(node, dict) or "
        $ref" not in node:
61         return node
62     ref = str(node["$ref"])
63     if not ref.startswith("#/"):
64         return node # external refs are out of scope
            for the offline checker
65     target: Any = document
66     for part in ref[2:].split("/"):
67         if not isinstance(target, dict) or part not in
            target:
68             return node
69         target = target[part]
70     return _resolve_local_ref(document, target, depth +
        1)
71
72
73 def _schema_has_property(document: Any, schema: Any,
    name: str) -> bool:
74     """True when 'name' appears in the schema or any
        composed/ref'd part."""
75     schema = _resolve_local_ref(document, schema)
76     if not isinstance(schema, dict):
77         return False
78     if name in (schema.get("properties") or {}):

```

```

79         return True
80     for key in ("allOf", "anyOf", "oneOf"):
81         if any(
82             _schema_has_property(document, part, name)
83             for part in schema.get(key, [])
84         ):
85             return True
86     return False
87
88 def _collect_refs(document: Any, schema: Any, depth:
89     int = 0) -> list[str]:
90     """Collect every '$ref' target reachable from a
91     schema (any depth)."""
92     schema = _resolve_local_ref(document, schema)
93     if depth > 10 or not isinstance(schema, dict):
94         return []
95     refs: list[str] = []
96     raw = schema.get("$ref")
97     if isinstance(raw, str):
98         refs.append(raw)
99     for key in ("allOf", "anyOf", "oneOf"):
100         for part in schema.get(key, []):
101             refs.extend(_collect_refs(document, part,
102                                     depth + 1))
103     return refs
104
105 def _content_schemas(content: Any) -> list[Any]:
106     if not isinstance(content, dict):
107         return []
108     return [media.get("schema") for media in content.
109             values() if isinstance(media, dict)]
110
111 def _check_openapi(name: str, spec: Any) -> list[
112     Violation]:
113     violations: list[Violation] = []
114     if not isinstance(spec, dict) or "openapi" not in
115         spec:
116         return violations
117
118     # 0-001: auth scheme presence and global security
119     requirement.
120     schemes = (
121         (spec.get("components") or {}).get("

```



```

117         securitySchemes") or {}
118     )
119     if not schemes:
120         violations.append(Violation("0-001", name, "no_
121             securitySchemes_defined"))
122     for scheme_name, scheme in schemes.items():
123         if not (scheme or {}).get("description"):
124             violations.append(
125                 Violation("0-001", name, f"scheme_{
126                     scheme_name!r}_lacks_a_description")
127             )
128     if not spec.get("security"):
129         violations.append(
130             Violation("0-001", name, "no_global_
131                 security_requirement_applied")
132         )
133
134     # 0-002: error responses must reference
135     # ProblemDetails.
136     paths = spec.get("paths") or {}
137     for path, item in paths.items():
138         for method, operation in (item or {}).items():
139             if method not in ("get", "post", "put", "
140                 patch", "delete"):
141                 continue
142             responses = (operation or {}).get("
143                 responses") or {}
144             for status, response in responses.items():
145                 if not str(status).startswith(("4", "5"
146                     )):
147                     continue
148                 schemas = _content_schemas((response or
149                     {}).get("content"))
150                 refs: list[str] = []
151                 for schema in schemas:
152                     raw = (schema or {}).get("$ref") if
153                         isinstance(schema, dict) else
154                         None
155                     if raw:
156                         refs.append(raw)
157                     else:
158                         refs.extend(_collect_refs(spec,
159                             schema))
160                 if not any(r.endswith("/ProblemDetails"
161                     ) for r in refs):
162                     violations.append(

```

```

150         Violation(
151             "0-002",
152             name,
153             f"{method.upper()}_{path}_{status}_does_not_{reference}"
154             "ProblemDetails",
155         )
156     )
157
158     # 0-003: collection GETs must document limit/cursor
159     # pagination.
160     for path, item in paths.items():
161         get_op = (item or {}).get("get")
162         if not get_op or not path.rstrip("/").endswith("s"):
163             continue
164         params = {
165             (p or {}).get("name")
166             for p in get_op.get("parameters") or []
167             if isinstance(p, dict) and p.get("in") == "query"
168         }
169         missing = {"limit", "cursor"} - params
170         if missing:
171             violations.append(
172                 Violation(
173                     "0-003",
174                     name,
175                     f"GET_{path}_missing_query_params:{sorted(missing)}",
176                 )
177             )
178         ok_schemas = _content_schemas(
179             ((get_op.get("responses") or {}).get("200") or {}).get("content")
180             )
181         )
182         if not any(
183             _schema_has_property(spec, s, "nextCursor")
184             for s in ok_schemas
185         ):
186             violations.append(
187                 Violation("0-003", name, f"GET_{path}_200_schema_lacks_nextCursor")
188             )

```

```

187 # 0-004: date-based version header convention.
188 version = str((spec.get("info") or {}).get("version
189         ", ""))
190 if not DATE_VERSION.match(version):
191     violations.append(
192         Violation(
193             "0-004",
194             name,
195             f"info.version_{version!r}_is_not_a_
196             YYYY-MM-DD_date_version",
197         )
198     )
199 return violations
200
201 def _check_asyncapi(name: str, spec: Any) -> list[
202     Violation]:
203     violations: list[Violation] = []
204     if not isinstance(spec, dict) or "asyncapi" not in
205     spec:
206         return violations
207     channels = spec.get("channels") or {}
208     for channel, item in channels.items():
209         publish = (item or {}).get("publish")
210         if not publish:
211             continue
212         message = _resolve_local_ref(spec, publish.get(
213             "message") or {})
214         payload = _resolve_local_ref(spec, (message or
215             {}).get("payload") or {})
216         missing = [
217             f for f in ENVELOPE_FIELDS if not
218             _schema_has_property(spec, payload, f)
219         ]
220         if missing:
221             violations.append(
222                 Violation(
223                     "A-001",
224                     name,
225                     f"channel_{channel!r}_publish_
226                     payload_missing_envelope_
227                     f"fields:_{missing}",
228                 )
229             )
230     return violations

```

```

225
226 def check_folder(folder: Path) -> list[Violation]:
227     """Check every spec under 'folder' and return all
        violations."""
228     specs = _load_specs(folder)
229     if not specs:
230         raise FileNotFoundError(f"no YAML/JSON
            manifests found under {folder}")
231     violations: list[Violation] = []
232     for name, spec in specs.items():
233         violations.extend(_check_openapi(name, spec))
234         violations.extend(_check_asyncapi(name, spec))
235     return violations
236
237
238 def main(argv: list[str]) -> int:
239     folder = Path(argv[1]) if len(argv) > 1 else Path("
        manifests")
240     try:
241         violations = check_folder(folder)
242     except FileNotFoundError as exc:
243         print(f"error: {exc}", file=sys.stderr)
244         return 2
245     if violations:
246         print(f"FAIL: {len(violations)} governance
            violation(s)")
247         for v in violations:
248             print(f"{v.render()}")
249         return 1
250     print(f"PASS: all governance invariants hold under
        {folder}")
251     return 0
252
253
254 if __name__ == "__main__":
255     raise SystemExit(main(sys.argv))

```

The checker is validated against two synthetic manifest folders shipped with the lab: a compliant Northwind folder and a legacy folder that breaches every rule family. A dedicated test proves that \$ref-based AsyncAPI payloads resolve instead of false-positiving.

```

1  """Tests for the architecture conformance checker.
2
3  Runs against two synthetic fixture folders: '
    fixtures_pass' (a
4  compliant Northwind manifests folder) and '

```

```

    fixtures_fail '''(a_legacy
5 folder_that_breaches_every_rule_family).
6 '''
7
8 from __future__ import annotations
9
10 from pathlib import Path
11
12 import pytest
13
14 from conformance_checker import check_folder, main
15
16 HERE = Path(__file__).parent
17 PASS_DIR = HERE / "fixtures_pass"
18 FAIL_DIR = HERE / "fixtures_fail"
19
20
21 def test_compliant_folder_passes() -> None:
22     assert check_folder(PASS_DIR) == []
23
24
25 def test_legacy_folder_fails() -> None:
26     violations = check_folder(FAIL_DIR)
27     assert violations, "legacy_folder_must_produce_violations"
28
29
30 def test_every_rule_family_fires_on_legacy_folder() ->
    None:
31     rules = {v.rule for v in check_folder(FAIL_DIR)}
32     assert {"0-001", "0-002", "0-003", "0-004", "A-001"
33             } <= rules
34
35
36 def test_main_exit_codes() -> None:
37     assert main(["prog", str(PASS_DIR)]) == 0
38     assert main(["prog", str(FAIL_DIR)]) == 1
39
40
41 def test_missing_folder_exit_code() -> None:
42     assert main(["prog", str(HERE / "no_such_dir")]) ==
43         2
44
45
46 def test_checker_is_deterministic() -> None:
47     assert check_folder(FAIL_DIR) == check_folder(

```

```

46         FAIL_DIR)
47
48 def test_asyncapi_ref_payload_is_resolved(tmp_path:
49     Path) -> None:
50     """A publish payload referenced via components must
51         not be false-positive."""
52     spec = tmp_path / "ref.asyncapi.yaml"
53     spec.write_text(
54         "\n".join(
55             [
56                 'asyncapi: "2.6.0"',
57                 "info: {title: Ref Events, version: '1.0.0'}",
58                 "channels:",
59                 "  robot.readings:",
60                 "    publish:",
61                 "      message: {ref: '#/components/
62                     messages/Reading'}",
63                 "    components:",
64                 "      messages:",
65                 "        Reading:",
66                 "          payload: {ref: '#/components/
67                     schemas/Envelope'}",
68                 "          schemas:",
69                 "            Envelope:",
70                 "              type: object",
71                 "              properties:",
72                 "                event_id: {type: string}",
73                 "                occurred_at: {type: string,
74                     format: date-time}",
75             ]
76         ),
77         encoding="utf-8",
78     )
79     assert check_folder(tmp_path) == []
80
81 if __name__ == "__main__":
82     raise SystemExit(pytest.main([__file__, "-q"]))

```

The compliant manifests folder (abridged here to the catalog spec; the AsyncAPI telemetry spec follows the same shape):

```

1 openapi: 3.1.0
2 info:
3   title: Northwind Catalog API

```

```
4   version: "2026-09-01"
5 security:
6   - ApiKeyAuth: []
7 paths:
8   /v1/products:
9     get:
10      summary: List catalog products
11      parameters:
12        - name: limit
13          in: query
14          schema: { type: integer, minimum: 1, maximum:
15                    50, default: 20 }
16        - name: cursor
17          in: query
18          schema: { type: string }
19      responses:
20        "200":
21          description: Page of products
22          content:
23            application/json:
24              schema:
25                type: object
26                properties:
27                  items:
28                    type: array
29                    items: { $ref: "#/components/
30                              schemas/Product" }
31                  nextCursor: { type: string }
32        "400":
33          description: Invalid cursor
34          content:
35            application/problem+json:
36              schema: { $ref: "#/components/schemas/
37                          ProblemDetails" }
38        "401":
39          description: Missing or invalid API key
40          content:
41            application/problem+json:
42              schema: { $ref: "#/components/schemas/
43                          ProblemDetails" }
44 components:
45   securitySchemes:
46     ApiKeyAuth:
47       type: apiKey
48       in: header
49       name: X-API-Key
```

```
46         description: Partner API key issued via the
47             Northwind developer portal.
48     schemas:
49         Product:
50             type: object
51             properties:
52                 id: { type: integer }
53                 sku: { type: string }
54                 name: { type: string }
55                 category: { type: string }
56                 unit_price_cents: { type: integer }
57         ProblemDetails:
58             type: object
59             properties:
60                 type: { type: string }
61                 title: { type: string }
62                 status: { type: integer }
63                 detail: { type: string }
```

22.4 Common Pitfalls & Anti-Patterns

The failure modes below are synthesis-level: they appear only when you combine everything, which is exactly why they are common at platform scale.

22.4.1 Resume-Driven Protocol Selection

Choosing GraphQL or gRPC because it is fashionable, then discovering the public partner audience cannot consume it, is the canonical platform mistake. The decision card exists precisely to make this failure awkward: if you cannot fill in the six criteria, you have not done the analysis. The repair — a BFF shim, a gRPC-Web proxy, a parallel REST surface — always costs more than the analysis would have.

22.4.2 One-Protocol-Fits-All Mandates

The mirror image of fashion is the organizational mandate: “everything is REST” or “everything is gRPC.” Mandates feel like governance but are actually the refusal to govern. The reference architecture of Section 22.2.2 runs four profiles simultaneously *because the constraints differ per consumer*; a mandate forces the worst profile onto at least one consumer class. Govern invariants (errors, auth, pagination, observability), not protocols.

22.4.3 Big-Bang Platform Builds

Attempting all four milestones of the Grand Capstone in one heroic push produces a platform that is 90 % built and 0 % operable. The phased plan of Table 22.4 is ordered so that each milestone is independently shippable and teaches the organization something the next one needs. If M1’s REST core cannot pass its contract tests, M3’s event pipeline will not save you.

22.4.4 Skipping Governance Until Fifty APIs Exist

Governance feels premature with three APIs and impossible with fifty. The cheap moment is the beginning: three rules (error shape, auth scheme, pagination convention) in a conformance checker like Listing 3 (Section 22.3.3) cost an afternoon when you have three specs and a quarter when you have fifty — and retrofitting means negotiating with fifty teams’ shipped contracts [GH21].

22.4.5 Adopting Frontiers Without Operational Maturity

HTTP/3, edge runtimes, and MCP tool surfaces amplify whatever you already are. Teams without SLOs, traces, and rollback discipline do not get faster from QUIC; they get *less debuggable*. Table 22.10 lists a prerequisite per frontier; treat it as a gate, not a suggestion.

22.4.6 Synthesis Without Ownership

A reference architecture diagram that no team owns decays into a poster. Every box in Figure 22.2 must have a named owning team, an SLO, and a runbook, or the architecture is fiction [New21].

22.5 Hands-On Production Lab & Real-World Case Study

22.5.1 Lab: Run the Full Toolkit

Goal: exercise all three listings end to end, offline, in under ten minutes.

Steps (PowerShell):

```
1 cd "code\ch21"  
2 .\.venv\Scripts\Activate.ps1  
3  
4 # 1. Prove all 30 tests pass (advisor + platform +  
   checker).
```

```
5 python -m pytest -q
6
7 # 2. Run the decision advisor against the five
   canonical scenarios.
8 python protocol_advisor.py
9
10 # 3. Check governance: compliant folder passes, legacy
    folder fails.
11 python conformance_checker.py fixtures_pass
12 python conformance_checker.py fixtures_fail
13
14 # 4. Boot the platform slice and probe every surface.
15 python northwind_platform.py # serves http
    ://127.0.0.1:8121
```

In a second PowerShell window:

```
1 $h = @{ "X-API-Key" = "dev-key-northwind" }
2
3 # REST catalog with keyset pagination (note RateLimit-*
   headers).
4 curl.exe -s -D - "http://127.0.0.1:8121/v1/products?
   limit=2" -H "X-API-Key: dev-key-northwind"
5
6 # Problem-details error: no auth header.
7 curl.exe -s "http://127.0.0.1:8121/v1/products"
8
9 # Idempotent order placement: run twice, compare
   order_id.
10 $body = '{"sku":"NWR-LDR-360","quantity":2}'
11 curl.exe -s -X POST "http://127.0.0.1:8121/v1/orders" -
   H "X-API-Key: dev-key-northwind" -H "Idempotency-Key
   : lab-1" -H "Content-Type: application/json" -d
   $body
12 curl.exe -s -X POST "http://127.0.0.1:8121/v1/orders" -
   H "X-API-Key: dev-key-northwind" -H "Idempotency-Key
   : lab-1" -H "Content-Type: application/json" -d
   $body
13
14 # GraphQL read model.
15 $q = '{"query":{"{ catalog { sku name availability }
   }"}}'
16 curl.exe -s -X POST "http://127.0.0.1:8121/graphql" -H
   "X-API-Key: dev-key-northwind" -H "Content-Type:
   application/json" -d $q
17
18 # SSE telemetry stream (three robot events, then the
```

```

demo stream ends).
19 curl.exe -s -N "http://127.0.0.1:8121/v1/telemetry/
    stream" -H "X-API-Key: dev-key-northwind"

```

Expected results: 30 tests pass; the advisor prints REST, gRPC, GraphQL, event-driven, and event-driven for scenarios A–E with rationale traces; the checker prints **PASS** for `fixtures_pass` and seven violations for `fixtures_fail`; the live server returns a paginated product page with `RateLimit-Remaining`, an `application/problem+json` 401 without the key, the same `order_id` on both POSTs, three catalog rows from GraphQL with joined availability, and three `robot-reading` SSE events.

Extensions: (a) flip one scenario’s constraints in the advisor (e.g., make scenario C cacheable) and watch the recommendation change; (b) add a violating spec to `fixtures_fail` and predict the rule codes before running; (c) exhaust the 30-request rate limit against the live server and confirm the 429 carries `Retry-After`.

22.5.2 Case Study: Three Years of an Enterprise API Platform

Year 1 — the product year. Northwind Robotics ships a public REST catalog and orders API to resellers: OpenAPI-first, RFC 9457 errors from day one, keyset pagination, idempotency keys on orders. The team debates GraphQL for the partner audience and rejects it with the decision card: unknown third parties plus a cacheable catalog is a REST scenario, full stop. A two-rule conformance linter (error shape, auth scheme) runs in CI from the first spec. Year-end incident count: one — a pagination drift bug caught by a contract test, not a partner.

Year 2 — the platform year. The mobile app launches, and its nested screens justify a GraphQL read model (scenario C) with depth limits and persisted queries. Inventory moves to internal gRPC when the checkout hot path misses its latency budget under REST-over-JSON. Order events go through a transactional outbox to signed partner webhooks after one painful week in which a dual-write bug (database committed, webhook never sent) teaches the team why Chapter 12 insists on the outbox [Kle17]. Rate tiers launch with IETF headers; the noisy top-1% integrators get quota offers instead of silent 429s. The conformance checker, now five rules, blocks two specs that would have shipped undocumented breaking changes.

Year 3 — the strategy year. OTel traces span edge to outbox; SLOs with error budgets replace uptime folklore [Bey+16]. The platform team evaluates HTTP/3 (adopted free at the CDN edge; internal gRPC stays on HTTP/2), runs a pilot exposing read-only catalog tools via MCP to an internal support copilot with tool schemas generated from the same OpenAPI

source, and declines an edge-compute rewrite of the order write path — regional affinity could not satisfy the single-region consistency requirement, a `wait` decision the frontier table predicts. The year’s most quoted internal document is not a design doc but the conformance checker’s violation log: governance-as-code turned fifty APIs into one coherent platform.

Lessons: protocols were chosen per consumer and revisited only when constraints changed; governance was cheapest on day one; the outbox and idempotency investments paid out every incident; and every frontier decision had an explicit adopt/trial/wait rationale tied to measured constraints.

22.6 Self-Assessment Exercises & Deep-Dive Questions

Work these without rereading the chapter; then check your reasoning against the decision card and the reference architecture.

22.6.1 Concept Review

1. Why is the delivery model the *first* question in the decision tree rather than the consumer type? Give an example where ordering them the other way produces a wrong recommendation.
2. The comparison matrix calls its entries “tendencies, not laws.” Pick one cell and construct a legitimate exception.
3. In the reference architecture, why does the outbox relay read from the database instead of the service publishing to the broker directly? What exactly can go wrong otherwise [Kle17]?
4. Which single header pair makes a public REST catalog dramatically cheaper to operate, and why do GraphQL endpoints not benefit equally [Fie22a]?
5. The conformance checker’s O-002 rule requires a `ProblemDetails` reference on every 4xx/5xx response. What failure class of Table 22.5 does this prevent, and for whom?
6. Why does the advisor break ties deterministically instead of reporting “either is fine”? What property of the test suite depends on it?
7. Explain why MCP tool schemas should be *generated from* the OpenAPI source of truth rather than maintained by hand.

8. The case study's year-3 team declined an edge-compute rewrite of the order path. Reconstruct their reasoning using Table 22.10.

22.6.2 Architecture-Design Drills

Each drill gives a consumer, a constraint set, and asks for a defended design. Use the decision card, name the profile(s), sketch the surfaces, and state the top two failure modes with mitigations.

1. **Drill 1 — Hospital device monitoring.** Bedside devices emit vitals every second; nurses view live dashboards; auditors pull historical ranges. Constraints: no public internet exposure, p99 dashboard freshness < 2 s, audit reads cacheable for 24 h, small disciplined team. Design all three surfaces.
2. **Drill 2 — Marketplace payout API.** Sellers trigger payouts; duplicates are catastrophic; partner banks are slow and flaky. Constraints: public consumers, strict exactly-once semantics at the boundary, audit trail, 100 requests/min typical with $10\times$ month-end bursts. Which chapters' patterns combine here?
3. **Drill 3 — Multi-tenant analytics backend.** A SPA renders arbitrary user-configured dashboards over 40 entity types. Constraints: payload shape unknowable at design time, metered mobile variant planned next year, team of four. Defend or reject GraphQL using the card, and design the guardrails whichever way you land.
4. **Drill 4 — Fleet firmware update orchestration.** 50 000 devices must receive signed update commands and report progress; connectivity is intermittent. Constraints: server-initiated delivery, resume across disconnects, no inbound connections to devices allowed on some networks. Choose between SSE, WebSockets, gRPC streaming, and polling-with-ETags — and defend the choice.
5. **Drill 5 — Your own platform.** Take a system you know professionally. Fill in the six criteria for each of its API surfaces. Does the card reproduce the choices that were actually made? Where it does not, was the card wrong or was the choice?

22.7 Interview Q&A Vault

Thirty-two synthesis-level questions, ordered junior $\rightarrow$ staff. Answers are deliberately terse; the chapters cited hold the depth.

Junior

- Q1. When would you choose REST over GraphQL for a new public API?** — When consumers are unknown third parties, payloads are fixed resources, and CDN caching matters: REST’s addressable GETs are the only profile that exploits shared HTTP caches [Fie22a], and OpenAPI tooling is universally understood [Ope21]. (Ch. 4)
- Q2. What is RFC 9457 and why should every error use it?** — A standard JSON problem-details shape (`type`, `title`, `status`, `detail`) so clients handle errors programmatically instead of parsing prose [Not23]. (Ch. 8)
- Q3. Offset or keyset pagination — and why?** — Keyset: stable under concurrent inserts, $O(1)$ deep pages, and it composes with cursors; offset drifts and scans [Not17]. (Ch. 9)
- Q4. What does an idempotency key buy you?** — Safe retries: the server deduplicates replays of the same key, so a flaky network cannot double-charge or double-order. (Ch. 11)
- Q5. REST vs. gRPC in one sentence?** — REST optimizes for reach and cacheability over standard HTTP; gRPC optimizes for typed, low-latency service-to-service RPC over HTTP/2 [Tho22; gRP24]. (Ch. 14)
- Q6. What headers belong on a rate-limited response?** — 429 with `Retry-After` plus the IETF `RateLimit` family so well-behaved clients can self-throttle [Pol24]. (Ch. 15)
- Q7. Why pin timeouts on every outbound call?** — An unbounded wait turns a slow dependency into your outage; timeouts plus retries with backoff bound the blast radius [Nyg18]. (Ch. 3)
- Q8. What is the OpenAPI document’s role in a team?** — The machine-readable source of truth for the contract: docs, tests, lint, and SDKs all derive from it [Ope21]. (Ch. 21)

Mid

- Q9. How do you version an API without breaking consumers?** — Additive changes within a version; date-based versions for breaking ones; `Sunset` headers and a published deprecation policy [Wil19]. (Ch. 10)

- Q10. Explain the transactional outbox in ninety seconds.** — Write the state change *and* the event row in one transaction; a relay publishes from the outbox; consumers get at-least-once delivery and the dual-write anomaly disappears [Kle17]. (Ch. 12)
- Q11. When is GraphQL the wrong answer?** — Cacheable public reads, teams without schema-governance maturity, and payloads that are actually fixed resources; you pay resolver complexity and lose HTTP caching for nothing [Gra21]. (Ch. 13)
- Q12. How do you secure a webhook?** — HMAC signatures with rotated secrets, timestamp tolerance against replay, delivery logs, and a replay endpoint; treat inbound signature failures as security events [OWA23]. (Ch. 12)
- Q13. SSE or WebSockets?** — SSE when the flow is server-to-client text events: it rides plain HTTP, auto-reconnects, and resumes via `Last-Event-ID`; WebSockets when you need bidirectional or binary. (Ch. 12)
- Q14. What belongs in a health endpoint vs. a readiness endpoint?** — Liveness: process is up. Readiness: dependencies (database, broker) verified, safe to route traffic. Conflating them turns dependency blips into restart storms. (Ch. 18)
- Q15. How do circuit breakers and retries interact?** — Retries handle transient faults; the breaker stops hammering a down dependency; retry budgets prevent retry storms from becoming a self-inflicted DDoS [Nyg18]. (Ch. 16)
- Q16. Walk through what the gateway should and should not do.** — Should: auth offload [Har12; Jon15], rate tiers, correlation IDs, request validation. Should not: business logic, data transformation that services then cannot evolve independently. (Ch. 16, Ch. 15)

Senior

- Q17. Design the API surface for a real-time bidding system under a 50 ms budget.** — gRPC streaming between participants and the auction core for latency; events for post-trade fan-out; REST only for management; the decision card lands here via `consumer=internal`, `latency=extreme`. (Ch. 14, this chapter)
- Q18. When would you expose gRPC publicly?** — Rarely, and only

for machine consumers you influence: SDK-shipped partners, not browsers. Budget for a gRPC-Web/JSON transcoding proxy, dual documentation, and firewall pain; if any of those is unacceptable, it was a REST API. (Ch. 14)

Q19. How do you make four protocol stacks feel like one platform?

— Govern the invariants, not the protocols: one error model (RFC 9457 [Not23]), one auth story, one pagination convention, one tracing context, one portal — enforced by executable policy. (Ch. 21, this chapter)

Q20. Your GraphQL gateway is melting under expensive queries.

Now what? — Persisted queries, depth and cost limits, dataloaders for N+1, per-field tracing to find the hot resolvers, and caching at the edge only for truly public anonymous queries [Gra21]. (Ch. 13, Ch. 19)

Q21. How do you roll out a breaking change to 200 partners?

— You do not: you add the new shape alongside, emit **Sunset** and **Link** headers [Wil19; Not17], measure old-version traffic, contact the long tail, and only then schedule removal. (Ch. 10)

Q22. What does your error budget buy you operationally?

— A neutral throttle: budget remaining $\Rightarrow$ ship; budget burned $\Rightarrow$ freeze features and invest in reliability [Bey+16]. It converts reliability arguments into arithmetic. (Ch. 19)

Q23. Design the telemetry pipeline for 3 000 robots.

— Event-driven ingestion to a broker with envelope fields (`event_id`, `occurred_at`), consumer-side idempotency, SSE fan-out to dashboards, and AsyncAPI contracts linted in CI [Asy23]. (Ch. 12)

Q24. How do you test a platform, not a service?

— Consumer-driven contract tests per surface [Pac24], in-process smoke suites proving surfaces coexist, conformance policy on every spec, and chaos drills on the integration points. (Ch. 20)

Staff

Q25. Design the API strategy for a marketplace startup that will become a platform.

— Year one: REST + OpenAPI public core, RFC 9457 and pagination conventions as day-one lint rules. Year two: add the profiles constraints demand (webhooks for partners, GraphQL if app payload graphs justify it), never by mandate. Year

three: governance as code, rate tiers as a product, SLOs and error budgets, and a conformance checker gating every spec [GH21]. (This chapter)

- Q26. Walk me through your reference architecture’s request life-cycle.** — Edge (TLS, WAF, cache) → gateway (JWT verify, rate tier, correlation ID) → service (validate, idempotency check, gRPC inventory call, transaction + outbox) → relay → broker → signed webhook + SSE fan-out, with one OTEL trace across all of it [Ope24]. (Section 22.2.2)
- Q27. How do MCP tool schemas change API design?** — The consumer is now a model choosing tools by reading your schema: descriptions, naming, error structure, and idempotency semantics become behavioral surface area. Generate tool schemas from the OpenAPI source of truth [Ope21; JSO22], keep error shapes structured [Not23], and gate agent-callable surfaces behind the same authZ and audit discipline as human ones. (Section 22.2.5)
- Q28. Your CEO read about HTTP/3 and wants it “everywhere by Friday.” Respond.** — Public surfaces already inherit it at the CDN via Alt-Svc [Bis22]; east-west gRPC gains little and risks UDP filtering and CPU regressions; propose measuring p99 and CPU per request before and after an edge-only rollout. (Section 22.2.5)
- Q29. How do you keep 50 teams’ APIs coherent without a review board?** — Executable policy: a handful of linted invariants (errors, auth, pagination, versioning) in CI, a self-service portal with templates that make the right way the easy way, and office hours for the genuinely hard cases [Ope21; Asy23]. (Ch. 21)
- Q30. What is your post-quantum readiness plan?** — Inventory TLS termination points, track hybrid key-exchange support in your edge/library stack [Res18], automate certificate rotation so algorithm changes are routine, and watch handshake-size effects on QUIC amplification limits. (Section 22.2.5)
- Q31. When is a second protocol stack worth its operational cost?** — When a consumer class’s constraints fail on the existing stack in a way money and caching cannot fix — e.g., mobile over-fetching that no amount of BFF trimming solves, or a p99 that only binary framing meets. Quantify the constraint first; “two stacks” is a conclusion, never a starting point. (This chapter)

Q32. How do you evaluate an AI-generated API design? — The same way as a human one, plus distrust: run it through the decision card, lint it with executable policy, contract-test it against consumers, and treat its plausible-but-wrong inventions as the default case. AI drafts; deterministic gates decide. (Section [22.2.5](#))

22.8 External Bibliography & Further Reading

Consolidated across the book, organized by module. All entries are real, primary, and freely available unless noted as books.

Module I — Foundations

- Fielding’s dissertation [[Fie00](#)] — the REST constraints from the source; reread it after Chapter [4](#).
- HTTP semantics and wire formats: RFC 9110 [[Fie22a](#)], RFC 9112 [[Fie22b](#)], and for the modern transports RFC 9113 [[Tho22](#)] with HPACK [[Peo15](#)], and RFC 9114 over QUIC [[Bis22](#); [Iye21](#)].
- JSON [[Bra17](#)] and JSON Schema [[JSO22](#)] — the serialization contract layer of Chapter [5](#).
- REST in practice: Richardson’s maturity model book [[RAR13](#)]; API design craft in [[GH21](#)].
- Python HTTP clients: `httpx` [[Enc24](#)] and `requests` [[Pyt24](#)].
- TLS 1.3 [[Res18](#)] for the transport security underpinning of Chapter [6](#).

Module II — Building APIs

- FastAPI [[Ram24](#)] and Pydantic [[Col24](#)] — the reference stack of Chapter [7](#).
- RFC 9457 problem details [[Not23](#)] — the error interface of Chapter [8](#).
- RFC 8288 Web Linking [[Not17](#)] and RFC 8594 **Sunset** [[Wil19](#)] — pagination links and deprecation signaling.

Module III — Protocols & Scale

- AsyncAPI [Asy23] — the event-contract counterpart of OpenAPI for Chapter 12.
- The GraphQL specification [Gra21] — read the execution section, not just the schema language.
- gRPC [gRP24] and Protocol Buffers [Goo24] — including the versioning rules for `proto` files.
- The IETF RateLimit header draft [Pol24] — the emerging standard behind Chapter 15.
- Kleppmann [Kle17] (book) — the data-systems foundation for out-boxes, logs, and consistency reasoning.
- Nygard [Nyg18] (book) — the failure-mode catalog behind Chapter 16.
- Newman [New21] (book) — service boundaries and decomposition strategy.

Module IV — Production & Governance

- OAuth 2.0 [Har12], PKCE [Sak15], and JWT [Jon15] — the token stack of Chapters 6 and 17.
- OWASP API Security Top 10 [OWA23] — the standing checklist; reread at every major release.
- OpenTelemetry [Ope24] and the Google SRE book [Bey+16] — instrumentation and SLO discipline for Chapter 19.
- The twelve-factor methodology [Wig17] — configuration and deployment hygiene for Chapter 18.
- Pact [Pac24] — consumer-driven contract testing for Chapter 20.
- OpenAPI 3.1 [Ope21] — the governance source of truth for Chapter 21.

22.9 Capstone Projects

Five integrative projects. Each builds on multiple chapters; datasets and services are synthetic and offline.

22.9.1 [junior] Project 1: Extend the Decision Advisor

Objective: deepen Listing 1 (Section 22.3.1) into a usable team tool.

Inputs/Datasets: `protocol_advisor.py`, its test file, and Table 22.1.

Steps:

1. Add an interactive questionnaire mode that prompts for the six criteria and prints the recommendation with its rationale trace.
2. Add a seventh criterion (data-residency constraint) with signals that penalize public-CDN-dependent profiles when residency forbids edge caching.
3. Add a `-explain` flag that prints the full score table for all five profiles, not only the winner.
4. Extend the tests: the new criterion must change at least one canonical scenario’s runner-up margin without flipping any existing winner.

Acceptance Criteria:

- ☐ Questionnaire mode runs offline and rejects invalid answers with a clear message.
- ☐ All pre-existing advisor tests pass unmodified.
- ☐ New tests cover the residency criterion and `-explain` output.
- ☐ Rationale traces mention the residency criterion when it moved a score.

Stretch Goals: emit the score table as JSON for CI consumption; add a “confidence” metric based on the winner’s margin.

22.9.2 [mid] Project 2: Governance Toolchain

Objective: grow the conformance checker of Listing 3 (Section 22.3.3) into a team-grade lint gate.

Inputs/Datasets: `conformance_checker.py`, both fixture folders, and the invariants of Chapter 21.

Steps:

1. Add three new rules: every operation has an `operationId` matching `camelCase`; every error schema is registered in `components`, not inlined; AsyncAPI channel names follow `domain.noun.verb`.
2. Produce both a console report and a machine-readable JSON report with stable rule codes.

3. Add a `-fix-hints` mode that prints the smallest suggested remediation per violation.
4. Wire the checker into a script that fails CI on any violation in a changed spec only (accept a folder pair: baseline vs. candidate).

Acceptance Criteria:

- ☐ All five original rule families plus the three new rules fire on `fixtures_fail` and pass on `fixtures_pass`.
- ☐ JSON report parses and round-trips deterministically.
- ☐ Changed-only mode ignores pre-existing violations in untouched files.
- ☐ Tests cover each new rule with a positive and negative fixture.

Stretch Goals: severity levels (error/warning) with per-rule overrides in a policy file; SARIF output for code-scanning integration.

22.9.3 [senior] Project 3: Multi-Protocol Read/Write Split

Objective: build a domain where writes are REST, reads are GraphQL, and internal coordination is gRPC — over one shared store — and prove consistency.

Inputs/Datasets: Listing 2 (Section 22.3.2) as the starting skeleton; synthetic catalog/orders datasets from the Northwind universe.

Steps:

1. Split Listing 2 (Section 22.3.2) into a write service (REST, idempotent commands) and a read service (GraphQL projections).
2. Propagate writes to the read store through an outbox-backed event stream (in-process broker fake is fine; keep the interface broker-shaped).
3. Model the internal inventory check as a gRPC contract (write the `.proto`; an in-process stub implementation is acceptable offline).
4. Write a consistency test: concurrent orders never oversell stock; the GraphQL read model converges within a bounded number of polls.

Acceptance Criteria:

- ☐ REST writes return RFC 9457 errors and honor idempotency keys.

- ☐ GraphQL reads never expose uncommitted state.
- ☐ The concurrency test passes deterministically in 100 runs.
- ☐ OpenAPI and GraphQL SDL artifacts both exist and pass the Project-2 checker.

Stretch Goals: add a CDC-style lag metric to the read model and an SLO on projection freshness; simulate broker outage and show recovery without loss.

22.9.4 [senior] Project 4: Webhook Notification Pipeline with Outbox Replay

Objective: implement the partner-notification subsystem of the reference architecture end to end.

Inputs/Datasets: the outbox pattern of Chapter 12; synthetic partner endpoint (a local FastAPI receiver you also build).

Steps:

1. Add an outbox table and relay loop to Listing 2 (Section 22.3.2) (in-process, polling, with exponential backoff).
2. Sign every webhook with HMAC; build the receiving endpoint to verify signatures and reject replays outside a timestamp window.
3. Persist a delivery log with attempt counts and last error; expose `POST /v1/webhooks/{id}/replay`.
4. Emit AsyncAPI documentation for the event catalog; extend the conformance checker's A-001 to cover it.
5. Add an SSE fallback stream carrying the same events for partners who cannot receive inbound webhooks.

Acceptance Criteria:

- ☐ No event is lost across a simulated broker/receiver outage (test proves it).
- ☐ Signature failures and stale timestamps are rejected with 401/422 problem details.
- ☐ Replay re-sends exactly the logged event payload.
- ☐ AsyncAPI spec passes the conformance checker.

Stretch Goals: dead-letter queue with operator triage endpoint; delivery-latency histogram exported as an OTel-style metric.

22.9.5 [staff] Project 5: The Grand Capstone — Northwind Robotics API Platform

Objective: build the complete platform of Section 22.2.3: all five surfaces of Table 22.3, the four milestones of Table 22.4, and the acceptance rubric — as a coherent, governed, observable system.

Inputs/Datasets: the full chapter (reference architecture, NFRs, milestone plan); synthetic Northwind datasets; in-process fakes for broker, payments, and identity provider.

Steps:

1. M1: REST catalog + orders core with keyset pagination, date versioning, RFC 9457 errors, idempotency keys, and contract tests.
2. M2: GraphQL read model with depth limits and persisted queries; inventory gRPC service; gateway-simulating middleware for auth and rate tiers with IETF headers [Pol24].
3. M3: outbox relay, signed webhooks with delivery log and replay (Project 4 folded in), SSE telemetry stream; OpenAPI + AsyncAPI manifests under conformance CI.
4. M4: OTel-style trace context propagated across all surfaces; SLO definitions with a load test harness; chaos drill (kill inventory mid-order) demonstrating circuit-breaker behavior; developer-portal pages generated from the specs.
5. Final: a written design review — decision-card rationale per surface, frontier assessment for HTTP/3 and MCP, and a one-year evolution plan.

Acceptance Criteria:

- ☐ All five surfaces respond per spec in one offline in-process test run.
- ☐ Conformance checker passes on the full manifests folder.
- ☐ SLOs from Section 22.2.3 measured green against a recorded load test.
- ☐ Chaos drill produces the designed failure behavior with zero data loss (outbox replay verified).
- ☐ Rubric score $\geq 16/20$ with no dimension below 3.

Stretch Goals: canary deployment simulation with traffic splitting; MCP tool surface generated from the OpenAPI spec for the read-only catalog; regional-affinity design document for a hypothetical EU expansion.

22.10 Python Dependencies Appendix

This chapter's code targets Python 3.11+ and pins minimum versions only, per the book's convention:

```
1 # requirements.txt
2 fastapi>=0.110
3 strawberry-graphql[fastapi]>=0.230
4 pydantic>=2.7
5 pyyaml>=6.0
6 httpx>=0.27
7 pytest>=8
```

All installation commands below assume an activated virtual environment in PowerShell (`.\.venv\Scripts\Activate.ps1`).

22.10.1 fastapi

Description: the ASGI web framework used for every HTTP surface in the platform slice [\[Ram24\]](#).

Why included: it is the book's reference framework (Chapter 7): type-driven validation, automatic OpenAPI generation, and first-class streaming responses for SSE.

Alternatives: Flask (simpler, weaker typing), Django REST Framework (heavier, ORM-coupled), Litestar (comparable, smaller ecosystem).

Installation:

```
1 pip install "fastapi>=0.110"
```

22.10.2 strawberry-graphql[fastapi]

Description: a code-first GraphQL library; the `[fastapi]` extra provides the ASGI router used to mount the read model at `/graphql`.

Why included: implements the GraphQL surface of the platform slice with typed Python resolvers instead of a separate schema language (Chapter 13); the extra keeps integration to one line.

Alternatives: Graphene (older API style), Ariadne (schema-first), or hand-rolled JSON-over-POST (not recommended).

Installation:

```
1 pip install "strawberry-graphql[fastapi]>=0.230"
```


22.10.3 pydantic

Description: the data-validation library underlying FastAPI’s request/response models [Col24].

Why included: all request and response schemas in Listing 2 (Section 22.3.2) are Pydantic models; version 2’s validator performance and stricter typing matter at platform scale.

Alternatives: dataclasses + manual validation (verbose), msgspec (faster, smaller ecosystem), attrs + cattrs (mature, more boilerplate).

Installation:

```
1 pip install "pydantic>=2.7"
```

22.10.4 pyyaml

Description: the YAML parser used by the conformance checker to load OpenAPI and AsyncAPI manifests.

Why included: real-world API manifests are overwhelmingly YAML; the checker needs nothing more than `yaml.safe_load` plus structural traversal, so a heavy spec parser would be over-engineering.

Alternatives: ruamel.yaml (round-trip preservation, heavier), strictyaml (schema-restricted subset), or JSON-only manifests (loses comments).

Installation:

```
1 pip install "pyyaml>=6.0"
```

22.10.5 httpx

Description: the modern sync/async HTTP client [Enc24]; pulled in by FastAPI’s `TestClient` and used throughout the book’s client chapters.

Why included: the smoke suite’s in-process client is built on it; its transport abstraction is what makes network-free tests possible (Chapter 3).

Alternatives: requests [Pyt24] (sync-only, no ASGI transport), aiohttp (async-only, larger API surface).

Installation:

```
1 pip install "httpx>=0.27"
```

22.10.6 pytest

Description: the test runner and fixture framework for all three listings’ suites.

Why included: fixtures give each test an isolated platform instance; `pytest.approx` and parametrization keep the suites terse; it is the ecosystem default (Chapter 20).

Alternatives: `unittest` (stdlib, more boilerplate), `nose2` (fading), `doctest` (documentation tests only).

Installation:

```
1 pip install "pytest>=8"
```

22.11 Chapter Summary & Key Takeaways

- Protocol selection is constraint satisfaction. Six criteria — consumer, payload, latency, delivery, caching, team maturity — determine the interaction profile; the Master Decision Card and its executable advisor make the choice auditable and teachable.
- Production platforms are multi-protocol by design. The Northwind reference architecture runs REST, GraphQL, gRPC, webhooks, and SSE simultaneously, unified by governed invariants rather than mandates.
- The transactional outbox, idempotency keys, and RFC 9457 errors are the three cheapest reliability investments in the entire book — and the three most often deferred [Kle17; Not23].
- Governance is cheapest on day one and must be executable; a conformance checker turns architectural intent into CI reality [Ope21; Asy23].
- Frontiers (HTTP/3, MCP, edge, post-quantum TLS, security automation) amplify existing maturity; adopt/trial/wait with explicit prerequisites, never enthusiasm alone.
- Mastery is a pathway: consume and build (junior), correctness (mid), scale and resilience (senior), strategy and governance (staff). The book's twenty-one chapters map one-to-one onto it.

22.12 Quick-Reference Card

22.13 Closing Thoughts

Twenty chapters ago, an API was a URL you fetched. It is now — if this book did its job — a contract you design, a system you operate, a product you govern, and a career you build deliberately. The tools will

THE MASTER DECISION CARD — pin this page.		
If the constraint says...	Then choose...	And remember...
unknown public consumers	REST + OpenAPI [Ope21]	curl-ability is a feature; govern with lint from day one
cacheable reads at scale	REST GETs [Fie22a]	POST-based GraphQL bypasses every shared cache
nested graphs, mobile/SPA	GraphQL [Gra21]	depth limits + persisted queries are not optional
internal, p99 < 50 ms	gRPC [gRP24]	contract-first protos; version fields, never renumber
fire-and-forget, machines	events / webhooks [Asy23]	transactional outbox or lose events silently
server must push to browsers	SSE (WebSockets if bidirectional)	unary APIs cannot push — structural, not preferential
exploratory team	REST	maturity is a constraint, not an insult
Always	RFC 9457 errors [Not23], idempotent writes, IETF rate headers [Pol24], OTel traces [Ope24], linted specs	these are protocol-independent invariants — the actual platform

Table 22.12: The one-page Master Decision Card. Photocopy freely.

keep changing: HTTP/3 will finish arriving, agents will become first-class consumers, something not yet named will be next year’s conference darling. The disciplines will not change: say what you mean in a machine-readable contract, fail in ways callers can handle, make retries safe, watch your systems with honest telemetry, and govern invariants rather than tastes. Engineers who internalize those sentences will never be behind the frontier — they will be the ones defining it. Go build platforms worth depending on.

22.14 Standardized Evidence Addendum

22.14.1 Benchmark Snapshot Template

Config	p50 latency	p99 latency	Error rate	Throughput
Baseline	–	–	–	1.00x
Candidate A	–	–	–	–
Candidate B	–	–	–	–

Table 22.13: Minimum benchmark table to keep per chapter.

22.14.2 Request Trace Template

- **Request:** one representative API call for this chapter’s topic.
- **Before:** behavior (headers, payload, latency, failure mode) with the naive approach.
- **After:** behavior with the technique in this chapter.
- **Result:** what changed in correctness, resilience, latency, and operability.

22.14.3 Cost and Latency Budget Snapshot

Stage	Latency budget	Cost driver
TLS / connection setup	–	handshake round trips, keep-alive hit rate
Request validation	–	schema complexity, CPU per request
Business logic and I/O	–	downstream fan-out, query cost
Serialization and egress	–	payload size, compression ratio

Table 22.14: Operational budget template for this chapter’s technique.

22.15 Configuration Preset Template

```
1 # api_policy.yaml
2 policy_version: "v1.0.0"
3 component: "<chapter_topic>"
4 timeout_ms: 0
5 retry_budget: 0
6 fallback_strategy: "fail_fast"
7 rollback_trigger: "error_budget_burn_gt_threshold"
```

22.16 CI Quality Gate Specification

- contract tests green against the pinned OpenAPI/AsyncAPI/Protobuf spec,
- no breaking schema changes without a version bump,
- error responses conform to RFC 9457 problem-details shape,
- benchmark deltas within approved regression bounds (latency and throughput),
- policy version and rollback plan present in release metadata.

22.17 Incident Runbook Template

1. Detect regression from SLO burn-rate alerts or consumer reports.
2. Scope impacted endpoints, versions, and consumer segments.
3. Contain impact (rollback, feature flag, rate-limit tightening, or traffic shift).
4. Diagnose with traces, structured logs, and contract diffs.
5. Recover with validated fix and verified rollout procedure.
6. Prevent recurrence via new regression tests and CI gates.

Dimension	Choose this approach when	Prefer an alternative when
Use case	—	—
Latency budget	—	—
Operational cost	—	—
Team maturity	—	—

Table 22.15: Decision card to complete per chapter.

22.18 Cross-Chapter Decision Card

22.19 ROI Model and Build-vs-Buy

- **Build when:** the capability is differentiating, compliance forbids vendors, or scale makes vendor pricing irrational.
- **Buy when:** the capability is commodity (auth, gateways, observability), time-to-market dominates, or staffing is thin.
- **Hidden costs to model:** on-call load, upgrade cadence, expertise ramp, data egress fees.

22.20 Disaster Recovery Notes (RPO/RTO)

- Define recovery point objective (RPO): maximum acceptable data loss window.
- Define recovery time objective (RTO): maximum acceptable restoration time.
- Test restores regularly; an untested backup is not a backup.

Appendix A

Additional Resources

A.1 Glossary of Terms

- **API (Application Programming Interface):** A versioned contract between a provider and consumers defining operations, representations, and failure semantics over a transport protocol.
- **ALPN (Application-Layer Protocol Negotiation):** TLS extension by which client and server agree on the application protocol (e.g., h2 vs http/1.1) during the handshake.
- **APQ (Automatic Persisted Queries):** GraphQL protocol in which clients send a query hash first and the full query only on a cache miss.
- **AsyncAPI:** A specification format for describing event-driven and asynchronous APIs (channels, messages, bindings).
- **Backpressure:** A flow-control mechanism by which a slow consumer signals a producer to reduce its rate.
- **BOLA (Broken Object Level Authorization):** OWASP API1:2023 — accessing another user’s object by manipulating an object identifier; also called IDOR.
- **Bulkhead:** Isolation of resources (pools, threads, queues) per dependency so one failure cannot exhaust shared capacity.
- **Burn Rate:** The speed at which a service consumes its error budget relative to the budget’s allocation window.
- **Circuit Breaker:** A resilience pattern (closed/open/half-open state machine) that halts calls to a failing dependency to prevent cascading

failure.

- **Content Negotiation:** Selection of a response representation via `Accept`-family headers and server-side quality-value scoring.
- **Correlation ID:** An identifier propagated across service boundaries to tie one logical request's logs, metrics, and traces together.
- **Cursor Pagination:** Pagination keyed on an opaque token encoding position, immune to the drift and deep-scan costs of offset pagination.
- **DataLoader:** A per-request batching/caching utility that collapses $N+1$ resolver queries into single batched lookups.
- **Deadline Propagation:** Passing the remaining time budget of a request across downstream calls so no hop outlives the caller's patience.
- **ETag:** A strong or weak validator used with `If-None-Match`/`If-Match` for caching (304) and optimistic concurrency (412).
- **Error Budget:** The tolerated unreliability derived from an SLO (e.g., 43.2 minutes/month at 99.9% availability).
- **GCRA (Generic Cell Rate Algorithm):** The virtual-scheduling rate-limiting algorithm equivalent to a token bucket; used by cell-based limiters.
- **GraphQL Federation:** Composing multiple GraphQL subgraph schemas into one supergraph behind a gateway.
- **HATEOAS:** Hypermedia as the Engine of Application State — REST's constraint that representations carry the links/affordances for next actions.
- **Hedged Request:** Issuing a duplicate request after a latency threshold and using whichever response arrives first, to shave tail latency.
- **HOL Blocking (Head-of-Line):** When one stalled message/byte-stream delays all subsequent ones sharing the same ordered channel.
- **HPACK/QPACK:** Header compression schemes for HTTP/2 (static+dynamic table with Huffman coding) and HTTP/3 respectively.
- **Idempotency:** The property that repeating an operation produces the same observable effect as performing it once ($f(f(x)) = f(x)$).

- **Idempotency Key:** A client-generated unique key sent with a mutating request so the server can detect and safely replay duplicates.
- **JWT (JSON Web Token):** A compact signed token format (header.payload.signature, base64url) carrying claims; signed, not encrypted.
- **Keyset (Seek) Pagination:** Pagination via a `WHERE (sort_key, id) > (:last)` predicate aligned to a composite index.
- **Leaky Bucket:** Rate-limiting algorithm modeling requests as fluid leaking at a constant rate (as meter) or draining from a queue (as queue).
- **mTLS (Mutual TLS):** TLS where both client and server present certificates, providing workload identity.
- **N+1 Problem:** Executing one query for a list plus one per element; solved by batching (DataLoader) or join-eager loading.
- **OIDC (OpenID Connect):** Identity layer on OAuth 2.0 adding ID tokens, discovery, and standard claims.
- **OpenAPI:** A language-agnostic specification format for describing HTTP APIs (v3.1 aligns with JSON Schema 2020-12).
- **Outbox Pattern:** Writing domain state and an event row in one local transaction, then relaying events asynchronously for guaranteed at-least-once publication.
- **PKCE (Proof Key for Code Exchange):** OAuth extension binding an authorization code to the client that initiated the flow via a code verifier/challenge pair.
- **Problem Details (RFC 9457):** The standard `application/problem+json` error body: `type`, `title`, `status`, `detail`, `instance`, `extensions`.
- **Protobuf (Protocol Buffers):** Google's binary serialization IDL; messages encode as tag-length-value with varint/zigzag integers.
- **Rate Limiting:** Controlling request throughput per consumer using algorithms such as token bucket, leaky bucket, sliding window, or GCRA.
- **RED / USE:** Metrics methodologies — Rate, Errors, Duration for services; Utilization, Saturation, Errors for resources.

- **Retry Budget:** A cap on the fraction of requests that may be retries, preventing retry amplification storms.
- **Saga:** A sequence of local transactions with compensating actions, used instead of distributed transactions across services.
- **SLI/SLO/SLA:** Service Level Indicator (measured behavior), Objective (internal target), Agreement (external contractual commitment, looser than the SLO).
- **SSRF (Server-Side Request Forgery):** Tricking a server into fetching attacker-chosen URLs; defended with allow-lists, redirect re-validation, and metadata-endpoint blocking.
- **SSE (Server-Sent Events):** The `text/event-stream` push protocol: unidirectional, HTTP-native, with automatic reconnection via `Last-Event-ID`.
- **Token Bucket:** Rate limiter holding up to B tokens refilled at rate r ; burst tolerance equals B .
- **traceparent:** The W3C Trace Context header (`version-traceid-spanid-flags`) propagating distributed trace identity.
- **TTFC (Time To First Call):** Developer-experience activation metric: elapsed time from signup to a first successful API call.
- **Webhook:** A provider-initiated HTTP callback delivering events to a consumer-registered URL, typically HMAC-signed with retry semantics.

A.2 Recommended Learning Path

1. Start with **Chapter 0** (Introduction) and **Chapters 1–2** (HTTP wire protocol, consuming APIs).
2. Progress sequentially through Module 1, then Module 2, for foundational mastery.
3. Jump to Module 3 and Module 4 chapters only after completing Modules 1 and 2.
4. Use capstone projects to practice immediately after each chapter.

Appendix B

Solutions, Model Answers & Grading Rubrics

This appendix collects model answers for every self-assessment exercise in Chapters 1–22 and grading rubrics for all 110 capstone projects. Three rules govern its use.

Key Concept

Attempt first, then compare. Every answer here is *reference-grade*, not *exclusive*: an alternative answer that is correct, justified, and evidence-backed deserves full credit. Rubrics are grading contracts — they define what full marks require, not the only path to them.

Warning

Rubrics are operational, not cosmetic. For senior and staff capstones, correctness alone never earns full marks: the weights deliberately reserve 30–45% for tests, determinism, observability, and rollback discipline. Grade against the criteria, not against your first impression of the code.

Numbering conventions. Exercise numbering follows each chapter’s own scheme (e.g. E0.1, E14.2, E-3, (7.1), Exercise 7.1). A few chapters number their exercises or capstones without a chapter prefix (“Capstone 1–5”); in this appendix every capstone is cited with a chapter prefix (C6.1, C12.3, ...) for unambiguous reference, and a note flags each such case. Chapter 17 numbers its exercises “Exercise 7.1–7.8” (a template artifact); this appendix preserves that numbering. Where a chapter splits questions into named lists (Exercises, Deep-Dive Questions, Concept Review, Drills), the same split and order is kept here.

Example

How to grade with a rubric. Score each criterion independently at 0%, 50%, or 100% of its weight (no interpolation needed for classroom use), multiply by the weight, and sum. A submission missing any “full marks when...” condition drops that criterion to 50% only if the gap is demonstrably minor (e.g. one untested edge case); systemic gaps (no test suite at all) score 0% for that criterion.

B.1 Chapter 0: Orientation, Failure Taxonomies & Engineering Judgment

B.1.1 Self-Assessment Model Answers

The readiness table (Table 1.7) is self-scored and has no per-row key: award yourself a 2 only if you can perform the skill unprompted (e.g. compute a percentile by hand on ten numbers, or explain the bytes between `urlopen` and the response). Any honest 0 names your first study topic; the target total ≥ 14 stands.

E0.1. (a) 502 during a deploy: *availability* failure in the operational layer; detect via per-status-code rate monitoring correlated with deployment markers. (b) field type change: *contract* (compatibility) failure; detect with schema-diff and contract tests in CI. (c) wrong currency with 200: *semantic* failure — syntactically valid, wrong meaning; detect with invariant checks, canary metric comparison, and consumer-side validation. (d) daily p99 spike: *performance* (tail-latency) failure correlated with the batch window; detect with percentile alerts plus event correlation. (e) ID-increment invoice access: *security* failure (broken object-level authorization); detect with authorization regression tests and access-pattern anomaly audits.

E0.2. Sorted: 4, 5, 5, 6, 7, 9, 12, 18, 40, 220. Nearest-rank $p_{50} = 7$ ms (rank 5), $p_{90} = 40$ ms (rank 9), mean = 32.6 ms. Put p_{99} (or p_{90} with this little data) on the dashboard: the mean (32.6) is $4\times$ the median and describes no real request — pain concentrates in the tail. Always annotate sample size and offered load.

E0.3. Whitelist the query-key set (`in_stock` only); any other key returns 400 with an `application/problem+json` body [Not23] naming the offending parameter. Parse the value strictly (`true/false`), filter the catalog in the handler, keep type hints and stdlib-only dependencies, and extend the self-test to cover both branches.

- E0.4.** A 99.95% monthly SLO allows $0.0005 \times 43,200 = 21.6$ minutes of downtime per 30-day month, i.e. $0.0005 \times 10^6 = 500$ failed responses per million requests. The per-request unit alerts better: it is traffic-normalized and computable on rolling windows, whereas “minutes down” requires an arbitrary definition of *down*.
- E0.5.** Interleave the three payload sizes from a seeded PRNG (e.g. `random.Random(42)`) and bucket samples per size before computing percentiles. Seeding makes the load mix identical across runs, so differences between runs reflect the system, not sampling noise; unseeded randomness makes results irreproducible and runs incomparable.
- E0.6.** The mini API scores roughly L1 (works, partial tests, no formal contract). Cheapest three upgrades: (1) explicit timeouts on every outbound call; (2) structured request logging with a correlation ID; (3) a checked-in OpenAPI document with one generated contract test in CI. Together they reach L2 without architectural change.
- E0.7.** Timestamp at socket accept, handler entry, and response flush; aggregate 500 requests into per-stage p50/p99 columns and show the stages sum to the end-to-end budget of Equation (1.1). Tighten the stage with the largest share *and* variance first — typically the downstream-dependency wait, which is also where a timeout buys the most.
- E0.8.** Little’s Law: $\lambda_{\max} = C/L = 128/30 \approx 4.3$ rps with 30-second stalls; with a 2-second timeout, $\lambda_{\max} = 128/2 = 64$ rps. A single configuration change buys $\sim 15\times$ capacity — the cheapest scaling decision in the book.
- E0.9.** **S**: no authentication — ignored. **T**: no integrity controls beyond transport — partially ignored. **R**: no audit log — ignored. **I**: error bodies may leak internals — partially controlled. **D**: no rate limiting, no timeouts on inbound work — ignored. **E**: input validation on paths and bodies — partially controlled. The code today controls only fragments of I and E; the exercise’s point is how much a “working” API leaves open.
- E0.10.** Fail the build on: missing/invalid OpenAPI document, contract-test failure, breaking-change diff without a major bump, missing error-schema conformance (RFC 9457), absent timeouts in client code. Warn on: coverage regression, missing runbook link, undocumented error codes. Override: platform-team approval with a time-boxed, logged exception (compare the addendum’s CI quality-gate template

— the gate fails closed, the exception process is the only escape valve).

Deep-Dive Questions

- D0.1.** *For:* retryability is failure semantics — part of the contract by Definition 1.1; a fleet that starts retrying POST X multiplies write traffic, and if the endpoint is not idempotency-safe the provider has silently accepted duplicate-mutation risk. *Against:* it is additive guidance; no schema changes and clients may ignore it. Verdict: treat it as breaking *unless* the endpoint is provably idempotent (keys or natural idempotence) — then it is a minor contract expansion.
- D0.2.** Example: a platform with rigorous design governance (catalog, linting, review boards) whose services run single-region with no timeouts, bulkheads, or breakers. The risk is compounded: governance certifies the APIs, so consumers trust them, so the first real infrastructure incident cascades through a fleet of “compliant” but fragile services — and the governance program absorbs the blame.
- D0.3.** A retry storm violates stationarity and independence: arrivals are no longer exogenous (retries are generated by failures), and L inflates as queues build. Applying $C = \lambda L$ with the original offered rate then *understates* true concurrency — the law reassures you precisely when the system has left the regime where it applies.
- D0.4.** Count shed load as failure at the edge: $SLI = (\text{requests served correctly within the latency bound}) / (\text{all offered requests})$, measured *before* load shedding, with a separate alert on the shed ratio itself. Fast 503s then burn budget instead of gaming the percentile.
- D0.5.** Offline labs cannot teach: real TLS/certificate failure modes, CDN and proxy behavior, provider-specific rate-limit shapes, DNS and network partitions, genuine latency distributions, and identity-provider edge cases. Compensate with sandbox accounts, recorded-traffic replay, chaos shims that mimic provider faults, provider status-page archaeology, and small-blast-radius canaries before depending on any real provider.

B.1.2 Capstone Grading Rubrics

Capstone 0.1 — Contract Archaeology [junior]

Criterion	Weight	Full marks when...
Contract coverage	35%	every path, method, status, and error body of the mini API appears in the written contract.
Failure semantics	25%	each operation documents its error codes, retryability, and idempotence posture.
Methodology evidence	20%	probe transcripts/logs are included and reproducible by re-running the probe script.
Reflection	20%	the writeup names at least two things black-box probing cannot discover (rate limits, auth rules, SLAs, deprecation policy).

Reference approach. Build a probe matrix: enumerate routes from hints and 404 behavior, exercise happy path plus malformed, unknown-ID, and wrong-method cases per route, capture raw request/response pairs, then tabulate the contract from evidence only. The stretch (a hand-written OpenAPI 3.1 document validated offline [Ope21]) converts the table into a machine-checkable artifact.

Capstone 0.2 — Honest Benchmarks [mid]

Criterion	Weight	Full marks when...
Experimental design	30%	3 load shapes $\times$ 3 repetitions, seeded, with warmup discarded and raw CSVs committed.
Statistical analysis	30%	per-config p50/p95/p99 show where the tail detaches, attributed to a budget stage.
Reproducibility	20%	one command regenerates every table and figure bit-for-bit.
Interpretation	20%	the writeup explains quantitatively why the mean understated user pain.

Reference approach. Extend the chapter benchmark harness with seeded load shapes (steady, bursty, ramp); write one CSV row per request; analyze offline with a small script that computes percentiles per configuration and reconciles stage timings with the latency-budget equation. The stretch fits $L = a + b \cdot \text{bytes}$ from a fourth variable.

Capstone 0.3 — Failure Taxonomy Field Guide [mid]

Criterion	Weight	Full marks when...
Taxonomy coverage	30%	five runnable demos, one per failure category, each offline and deterministic.
Detection evidence	30%	each demo prints the symptom <i>and</i> the monitoring signal/alert that would catch it.
Playbook quality	20%	on-call entries follow the incident format: symptom, cause class, detection, first action.
Determinism & hygiene	20%	seeded randomness only; scripts exit non-zero on unexpected outcomes.

Reference approach. One script per category (crash, timeout, contract drift, semantic error, tail-latency), each driving the mini API against a seeded chaos stub; every run ends with a symptom → category → detection mapping line. The stretch packages the flaky dependency as a reusable seeded stub for later chapters' drills.

Capstone 0.4 — Budgets and Breakers for the Mini API [senior]

Criterion	Weight	Full marks when...
Protection correctness	30%	timeouts, bounded jittered retries, and a three-state circuit breaker each have focused tests [Nyg18].
Controlled experiment	30%	the <i>same</i> scripted fault runs with and without protections; p99 and error rate both improve when protected.
SLO arithmetic	20%	error budget computed from raw samples, not assumed.
Operational writeup	20%	post-incident report in war-story form: timeline, trigger, mechanism, fix, follow-ups.

Reference approach. Wrap the dependency call in timeout → retry-budget → breaker layers; inject a deterministic stall fault; graph unprotected vs protected latency distributions; compute budget burn from the sample CSVs. The stretch adds a bulkhead and shows an unrelated endpoint surviving the fault.

Capstone 0.5 — API Platform Governance Starter [staff]

Criterion	Weight	Full marks when...
Catalog completeness	25%	every lab API registered with owner, tier, lifecycle state, and contract link.
Automated gates	30%	offline linter exits non-zero on violations; gate runs in CI with a documented exception workflow.
Policy measurability	20%	deprecation policy defines sunset in observable terms (usage at zero for n weeks).
Validation & audit	25%	automated maturity report agrees with a manual audit on at least one API; discrepancies analyzed.

Reference approach. Define a YAML/JSON catalog schema; write a stdlib linter over the registered OpenAPI documents; wire it into CI with a time-boxed waiver mechanism; write the deprecation policy against telemetry keys; produce the quarterly maturity report by script. The stretch generates a static HTML portal from the same catalog — governance that ships value gets adopted (Chapter 21).

B.2 Chapter 1: HTTP on the Wire

B.2.1 Self-Assessment Model Answers

- E1.1** The server reads exactly `Content-Length` bytes, leaving one stray body byte in the TCP receive buffer; the next response parse on the keep-alive connection begins at that byte, so the status line is garbage or the framing splits mid-header. This is the desynchronization class behind request smuggling: `Content-Length` is the only framing authority, and writer/reader disagreement corrupts every subsequent message on the connection [Fie22b].
- E1.2** The client's `pending` buffer must accumulate bytes until a complete chunk-size line, then a full chunk plus CRLF, have arrived — TCP delivers byte streams, not chunks. Lowering `MAX_CHUNK` below an advertised chunk size must raise the defensive `ValueError` *before* allocating or reading, bounding memory against hostile or buggy peers.
- E1.3** `POST /robots`: 201 + `Location` on success; 400 (malformed body), 422 (semantically invalid), 409 (duplicate serial), 401/403. `GET /robots/{id}`: 200; 304 with `If-None-Match`; 404. `PUT /robots/{id}/firmware`: 200/204 (or 202 if the update is deferred); 400/422; 404; 412 on a

failed `If-Match` precondition. `DELETE /robots/{id}`: 204; 404 on repeat (still idempotent); 409 if a referenced resource blocks deletion [Fie22a].

- E1.4** Default `SETTINGS_MAX_FRAME_SIZE` is $2^{14} = 16,384$ octets. A 70,000-octet header block emits $\lceil 70000/16384 \rceil = 5$ frames: one `HEADERS` (without `END_HEADERS`) followed by four `CONTINUATION` frames, the last carrying `END_HEADERS` (and `END_STREAM` if the request has no body) [Tho22].
- E1.5** HTTP/1.1 resends the full header block every request: at roughly 400 bytes each, 100 requests cost ≈ 40 KB. With HPACK, request 1 encodes literally and populates the dynamic table; requests 2–100 encode as indexed references plus one literal `x-trace-id` (~ 20 bytes), totaling $\approx 2\text{--}3$ KB — a $>90\%$ reduction, assuming a 4,096-octet dynamic table that never evicts the reusable entries [Peo15].
- E1.6** Only `DATA` frames consume flow-control credit, so `HEADERS`, `SETTINGS`, `PING`, `RST_STREAM`, `PRIORITY`, `WINDOW_UPDATE`, and `GOAWAY` still flow — legal and deliberate, so the connection stays diagnosable and recoverable. A server that never sends `WINDOW_UPDATE` leaves the client with a permanently zero window: requests are accepted, responses never start, and users see hangs ending in client timeouts.
- E1.7** Cleartext: `ClientHello`, `ServerHello`. Encrypted under handshake keys: `EncryptedExtensions`, `Certificate`, `CertificateVerify`, `Finished` [Res18]. On resumption, 0-RTT early data can be sent *immediately after the ClientHello* (encrypted under PSK-derived early traffic keys), before any server message arrives.
- E1.8** CL.TE layout: the front-end trusts `Content-Length: 6` and forwards six body bytes; the back-end trusts `Transfer-Encoding: chunked`, sees a terminating zero chunk, and treats the leftover bytes (the smuggled `G...` prefix) as the start of the *next* request. Mitigations: (1) the front-end must reject or normalize any request carrying both headers; (2) use HTTP/2 end-to-end, whose binary length-prefixed framing removes the ambiguity — and never downgrade `h2`→`h1` without strict normalization.
- E1.9** On a lossy link (1% loss, 50 ms RTT), one dropped segment stalls *all ten* HTTP/2 streams behind TCP's ordered delivery while retransmission takes a full RTT; six HTTP/1.1 connections lose only the unlucky connection's request. `h2` loses exactly when the mul-

tiplexing gain is smaller than the HOL stall probability. HTTP/3 differs because QUIC recovers loss per stream: a lost packet stalls only its own stream [Iye21; Bis22].

E1.10 0-RTT data is encrypted under resumption keys with no server round-trip, so there is no freshness exchange — a network attacker can capture and replay it verbatim. Replayed `POST /payments` means duplicate charges. Defences: restrict early data to safe/idempotent methods via an allowlist, use single-use session tickets, keep anti-replay windows tight, and require application-level idempotency keys for anything money-moving (Chapter 11).

B.2.2 Capstone Grading Rubrics

Capstone 1.1 — From-Scratch HTTP/1.1 Server [junior]

Criterion	Weight	Full marks when...
Wire correctness	35%	<code>curl -v</code> shows byte-correct status line, CRLF headers, and exact <code>Content-Length</code> framing.
Keep-alive	20%	ten sequential requests on one connection all succeed.
Error semantics	20%	malformed request-line → 400, unknown path → 404, POST → 405, each byte-correct.
HEAD correctness	15%	HEAD returns GET-identical headers with zero body bytes.
Code quality	10%	typed, stdlib-only, readable parser state machine.

Reference approach. A `socketserver` request handler loops: read until CRLF CRLF, parse request line and headers, dispatch `/parts` and `/parts/{id}` from a route table, serialize the JSON body once to compute `Content-Length`, honor `Connection: close` by exiting the loop. The stretch adds `ETag/If-None-Match` → 304.

Capstone 1.2 — Chunked Streaming Proxy [mid]

Criterion	Weight	Full marks when...
Relay fidelity	30%	decoded body is byte-identical to a direct fetch, re-chunked in 256-octet frames.
Defensive parsing	25%	oversized or malformed chunks rejected with 502 and a log line naming the violation.
Header hygiene	20%	hop-by-hop headers stripped per RFC 9110 §7.6.1 [Fie22a].
Offline determinism	15%	deterministic test script runs the whole matrix with no network.
Observability	10%	per-request byte counts and timings via logging.

Reference approach. Threaded TCP proxy: forward the request head verbatim, then run a bounded chunk-decoding state machine over origin bytes, re-emitting fixed-size chunks to the client. Stretch: trailer support and bounded buffers for backpressure.

Capstone 1.3 — HPACK-Lite Header Encoder [mid]

Criterion	Weight	Full marks when...
Integer codec	25%	prefix integer coding round-trips every corpus value including prefix overflow [Peo15].
Indexing & dynamic table	30%	a repeated header block encodes as a single indexed reference; table never exceeds its size bound (eviction tested).
Compression ratio	25%	≥60% byte reduction on the repeat-heavy corpus, measured in the test.
Test quality	20%	golden hex vectors plus property-style round-trips, fully offline.

Reference approach. Implement the integer prefix primitive first with golden vectors, then a static-table dictionary, then a dynamic table with FIFO eviction; the encoder prefers indexed → literal-with-incremental-indexing. Stretch: static Huffman and a decoder that round-trips the encoder.

Capstone 1.4 — Protocol Fingerprinting & Frame Dissection
[senior]

Criterion	Weight	Full marks when...
Classification accuracy	30%	every fixture classified (h1 text, h2 preface, TLS records) with zero false positives.
Anomaly detection	25%	planted CL.TE desync and truncated frame both flagged with byte offsets.
Output contract	20%	structured report plus <code>-json</code> mode; deterministic across runs.
Engineering	15%	fixture generator and tests committed; CLI usable without reading the source.
Depth of dissection	10%	frame-level decode (type, flags, stream id) rather than heuristic string matching.

Reference approach. Layered recognizers: TLS record-header check first, then the h2 client preface, then an HTTP/1.1 token grammar; each recognized stream feeds a frame dissector emitting structured records. Stretch: HPACK state reconstruction across HEADERS/CONTINUATION boundaries.

Capstone 1.5 — HTTP Version Latency Profiler [staff]

Criterion	Weight	Full marks when...
Harness rigor	30%	one-command offline run, seeded randomness, warmup excluded, raw samples kept.
Measurement validity	25%	warm-vs-cold keep-alive benefit quantified per payload size; delay shim demonstrates HOL growth under injected latency.
Reporting	20%	output table conforms to the addendum's benchmark template with units and <i>n</i> .
Extensibility	15%	pluggable transport interface so a real h2 client drops in without harness changes.
Operational notes	10%	README states noise controls, known confounders, and how results should drive decisions.

Reference approach. Insert a delay/loss shim between client and server sockets; sweep payload sizes $\times$ connection reuse; summarize with percentiles, not means. The staff signal is the analysis: results must explain *which* budget stage each configuration moves and when the conclusion would flip on a lossy network.

B.3 Chapter 2: Consuming APIs in Python

B.3.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–6 list; numbers below match it.

1. Each un-annotated call leaves at least the connect or read dimension unbounded; a silent peer then parks a worker forever, and at scale the pool exhausts and the outage cascades — the Chapter 2 war-story mechanism. For every hit, name which of connect/read/write/pool is unbounded and the failure it permits.
2. Deterministic exponential waits: 0.5, 1, 2, 4 s $\rightarrow$ 7.5 s total. Full jitter draws $U[0, \text{cap}_i]$, halving the expectation: $E \approx 3.75$ s. Worst case equals the deterministic total (7.5 s); the $c = 8$ s cap never binds because no individual wait exceeds 4 s.
3. GET `/parts/NR-1001`: safe and idempotent — retry blindly. POST `/orders`: unsafe — a read timeout is ambiguous (the order may exist); retry only with an idempotency key. PUT `.../status`: idempotent — safe. POST `/payments` with `Idempotency-Key`: safe *iff* the server honors the key; verify that contract before retrying.
4. Worst case is $32 \times 2 = 64$ simultaneous upstream calls, so `max_connections=64` (headroom, not average) and `max_keepalive_connections` between 32 and 64 so warm connections survive between bursts without unbounded idle growth.
5. `ApiAuthError` is raised for 401/403 inside `_raise_for_status` *before* retry classification, so the retry policy maps it to non-retryable. The `MockTransport` test returns 401, asserts the typed exception, and asserts the transport saw exactly one request — auth failures must never be retried.
6. Loop pages with `offset += limit`; terminate when a page returns fewer than `limit` items (or is empty); raise on `max_pages` as a tripwire against infinite loops from a buggy server. `MockTransport` serves scripted page sequences to cover both termination branches.

Deep-Dive Questions

1. The pool timeout bounds waiting for a *free connection slot*; the read timeout bounds waiting for *response bytes* after the request is sent. `PoolTimeout` therefore signals pool undersizing or upstreams

stalling while holding connections — a topology/configuration fact that `ReadTimeout` can never reveal.

2. For the individual caller, halved expected wait is a latency *saving*. The fleet-level win holds regardless of the mean: jitter decorrelates retry instants across clients, preventing the synchronized retry herd that re-collapses a recovering service [Nyg18].
3. `Retry-After` is authoritative for the server’s state, but your own latency budget is authoritative for your caller. On irreconcilable conflict (120 s > 20 s budget): stop retrying, fail fast with a typed error that preserves the server’s hint, and let the caller decide — never silently exceed your budget.
4. `requests` is built on `urllib3`’s synchronous HTTP/1.1 connection core; multiplexed framing would require rewriting that core. `httpx` separated the transport behind an interface, so HTTP/2 became a pluggable option rather than a fork [Enc24; Pyt24].
5. Reader fetches page 1 (rows 1–5); a writer inserts one row at the front; page 2 at offset 5 re-reads row 5 (duplicate) — and the mirror image, deleting a read row, skips an unread one. A cursor pins the walk to key values, so inserts or deletes *before* the cursor cannot shift positions (Chapter 9).
6. `MockTransport` legitimately excluded the socket/TLS layer. Minimum offline addition: a test that builds the real TLS context and completes a handshake against a local self-signed `ssl` socket server, asserting hostname verification behavior — no network required.

B.3.2 Capstone Grading Rubrics

Capstone 2.1 — Robust CLI Downloader [junior]

Criterion	Weight	Full marks when...
Timeout discipline	25%	a test asserts no code path issues a request without an explicit timeout.
Artifact integrity	25%	a killed download leaves no partial file (write to <code>.tmp</code> , atomic rename).
Fault-matrix tests	30%	200, 404, 500-then-200, and malformed body covered via <code>MockTransport</code> with the stub stopped.
Log hygiene	20%	log lines carry method, path, status, latency — and no headers or tokens.

Reference approach. Thin CLI over an `httpx` client with a four-dimension timeout; stream the body to a temp file, `fsync`, rename; map status classes to typed errors. Stretch: `Range` resume and a `-dry-run` mode.

Capstone 2.2 — Rate-Limit-Aware Catalog Crawler [mid]

Criterion	Weight	Full marks when...
Rate conformance	30%	full crawl completes with zero steady-state 429s at ≤ 5 req/s, evidenced by the timestamp CSV.
Retry-After correctness	25%	a forced-429 <code>MockTransport</code> test proves Retry-After overrides computed jitter.
Determinism	25%	seeded RNG and injected sleeper; no wall-clock assertions anywhere.
Cursor correctness	20%	cursor-paginated walk visits every catalog item exactly once.

Reference approach. Client-side token bucket paces requests below the published limit; on 429, sleep exactly **Retry-After** (injected clock); walk cursors until `next` is absent. Stretch: AIMD bucket adaptation and a process-wide bucket shared by parallel crawlers (Chapter 15).

Capstone 2.3 — Typed SDK Wrapper with Full Test Matrix [mid]

Criterion	Weight	Full marks when...
Fault matrix breadth	30%	≥ 15 scenarios spanning every taxonomy branch (transport, 4xx classes, 5xx, malformed bodies).
Retry/idempotency policy	25%	contract errors never retried; transient errors retried within budget; mutations gated on idempotency keys.
Type safety	20%	public API fully typed; <code>mypy</code> clean; pydantic models validate at the boundary [Col24].
Docs & packaging	25%	installable local package; PowerShell-only README covering install, usage, and error semantics.

Reference approach. Layer as models $\rightarrow$ transport policy $\rightarrow$ resource methods $\rightarrow$ pagination iterators; express the retry policy as a table mapping error class to action. Stretch: generate models from JSON Schema at build time and add an async facade over `httpx.AsyncClient`.

Capstone 2.4 — Chaos-Client Harness [senior]

Criterion	Weight	Full marks when...
Fault coverage	30%	≥ 8 fault classes, each with an executable expectation (not just an injection).
Reproducibility	25%	every run replays from its seed with zero wall-clock dependence.
Assertion rigor	25%	at least one fault provably exhausts the retry budget and fails fast with the correct typed error.
Safety guards	20%	decompression-bomb test proves the size guard engages.

Reference approach. A chaos transport wraps MockTransport with a seeded fault schedule (latency spikes, resets, truncation, 5xx bursts, gzip bombs); the resilient client runs unmodified on top. Stretch: fake-clock latency injection and chaos schedules replayed from synthetic incident timelines.

Capstone 2.5 — Async Concurrent Fetcher [staff]

Criterion	Weight	Full marks when...
Correctness under load	25%	10,000-item queue drains with zero unhandled exceptions and zero pool timeouts.
Rate conformance	25%	sustained ≤ 50 req/s with $< 0.1\%$ 429s after warm-up.
Cancellation hygiene	20%	cancellation test shows no orphan tasks and no partial artifacts.
Determinism	15%	MockTransport plus injected async sleeps; suite is reproducible.
Design memo	15%	every knob (pool, concurrency, backoff, budget) quantified with its trade-off.

Reference approach. An `asyncio` queue feeding a bounded worker pool; semaphore-limited concurrency; one token bucket for pacing; structured cancellation via task groups so `close()` always runs. Stretch: AIMD adaptive concurrency and per-host fairness buckets.

B.4 Chapter 3: The REST Architectural Style

B.4.1 Self-Assessment Model Answers

Numbering note: the chapter numbers its exercises (7.1)–(7.8); this appendix preserves that scheme.

- (7.1) A lint-clean route table uses plural-noun resources (`/parts`, `/parts/{id}`, `/orders/{id}/lines`) with methods mapped to semantics (GET read, POST create, PUT replace, PATCH partial update, DELETE remove), correct status codes (200/201/204/ 404/409/412), and an explicit cache policy per GET route [Fie00]. Controller-style routes (`/getPart`) fail the lint rules.
- (7.2) Session cookie: *not* stateless if the server must look up server-side session state — the request alone is not self-contained. Bearer token: stateless — the request carries all context. OAuth token validated by introspection: the request itself is self-contained, but validation reintroduces a hidden server-to-server dependency; strictly stateless per the definition, operationally a coupling to watch.
- (7.3) URI `/promotions/today` (or date-keyed `/promotions/2026-09-03`); freshness via `Cache-Control: max-age` bounded by time-to-midnight, plus `ETag` and `Last-Modified` validators. Eight hours stale in a shared cache means customers see yesterday’s prices — direct revenue and trust damage, caused by a caching-policy bug, not a data bug.
- (7.4) HEAD derives from the GET handler minus the body (strengthening the uniform interface and cacheability — validators become usable); `Idempotency-Key` on POST `/parts` strengthens the stateless uniform contract’s reliability by making the one non-idempotent method safely retryable (Chapter 11).
- (7.5) The JSON Patch evaluator applies `add/remove/replace/test` atomically (all-or-nothing, with `test` as precondition). Media type `application/json-patch+json` selects it; `application/merge-patch+json` keeps the merge evaluator — content negotiation picks the semantics.
- (7.6) Scenario: pushing firmware to 50,000 robots one resource-PUT at a time vs one bespoke RPC broadcast. The uniform interface costs 50,000 round trips where one would do; what the inefficiency purchases is evolvability — every intermediary, cache, tool, and new client understands the uniform semantics without new code [Fie00].
- (7.7) *For* DELETE on `/warehouses/{wid}/stock/{sku}`: uniform, automation-friendly, and honest — 410 can signal deliberate retirement. *Against*: it conflates “zero stock” (a state of an existing record) with “no record” (absence), and invites accidental data loss. 404 communicates “currently absent, may return”; 410 communicates “gone by policy” [Fie22a].

(7.8) JSON:API decides for you: document shape, URL conventions, error objects, sparse fieldsets, and pagination parameters — entire Chapter 4 bikesheds vanish. It forbids custom envelopes and bespoke error formats; its opinion costs flexibility exactly where domain-specific representations or non-CRUD operations matter.

B.4.2 Capstone Grading Rubrics

Capstone 3.1 — Model the Library Domain [junior]

Criterion	Weight	Full marks when...
Resource purity	35%	zero lint findings (R001–R005); no controller archetype without a written reification-failure analysis.
Model completeness	25%	books, copies, members, loans, holds, and fines all modeled with lifecycle states.
URI discipline	20%	nesting depth ≤ 2 ; every collection/member relationship explicit.
Cache policy	20%	every GET route has a stated validator and freshness policy.

Reference approach. List nouns, not verbs; give loans and holds their own resources (they have lifecycles); express actions as state transitions on resources. Stretch: sparse fieldsets and a Merge Patch document renewing a loan.

Capstone 3.2 — Run and Extend the Stdlib Catalog [junior]

Criterion	Weight	Full marks when...
Constraint demonstrations	40%	seven curl transcripts captured: 200+ETag, 304, 201+Location, 409, 412, 406, CSV via Accept .
Extension correctness	25%	the “batteries” category works end-to-end and the selftest still passes.
Analysis	25%	the memo correctly identifies statelessness as the first constraint server-side sessions would break.
Hygiene	10%	transcripts reproducible; no code changes beyond the extension.

Reference approach. Drive the listing with scripted curl; capture raw exchanges; add the category through the public surface only. Stretch: HEAD-only verification and Sunset/Deprecation headers on a retired part.

Capstone 3.3 — Productionize the URI Linter [mid]

Criterion	Weight	Full marks when...
Exit-code contract	20%	documented 0/1/2 semantics (clean / violations / tool error) honored exactly.
Waiver mechanism	25%	waivers suppress only the cited rule+path and expire; expired waivers fail the build.
Rule test coverage	30%	every rule has positive and negative deterministic tests.
Machine-readable output	25%	JSON report validates against a committed schema.

Reference approach. Keep the rule engine pure (routes in, findings out); layer CLI, waivers file, and JSON serializer around it. Stretch: OpenAPI 3.1 as an input adapter and a route-table diff classifying deltas as breaking vs additive (Chapter 10).

Capstone 3.4 — Port an RPC API to Resource Orientation [senior]

Criterion	Weight	Full marks when...
Migration completeness	30%	all 12 legacy endpoints mapped to resources, or each exception has a written reification-failure analysis.
Compatibility	30%	shims preserve behavior for all 12 legacy calls, proven by before/after transcripts.
Contract quality	20%	design-first OpenAPI lints clean.
Migration plan	20%	dates, owners, and kill criteria keyed to a synthetic access log; no “someday” items.

Reference approach. Design the resource model first, generate the contract, then implement shims that translate legacy RPC calls onto the new resources so consumers migrate on their own schedule [RAR13]. Stretch: hypermedia transition links and a payload/round-trip delta quantification.

Capstone 3.5 — Design-Review Report on a Public API [staff]

Criterion	Weight	Full marks when...
Evidence discipline	30%	every claim cites observable evidence (docs, captured exchanges); no speculation presented as fact.
Balance	20%	at least one finding credits a sophisticated design choice — no pure hatchet job.
Remediation quality	30%	the backlog obeys the no-rename-and-restructure-in-one-step doctrine with sequencing.
Communication	20%	≤8 pages, executive-readable, constraint-by-constraint verdicts.

Reference approach. Score the API against each REST constraint with captured evidence; classify findings by evolvability impact; propose a remediation sequence that never breaks existing consumers. Stretch: repeat the review five years apart from archived docs to test evolvability claims.

B.5 Chapter 4: Serialization & Contracts

B.5.1 Self-Assessment Model Answers

- E4.1** Varints: $1 \rightarrow 01$; $127 \rightarrow 7F$; $128 \rightarrow 80\ 01$ (continuation bit set on the low seven bits first); $16,384 \rightarrow 80\ 80\ 01$. Verify each against the codec's `encode_varint` [Goo24].
- E4.2** $\text{zigzag}(-64) = 127$ (negative n maps to $-2n-1$); $\text{zigzag}^{-1}(127) = -64$. The point: small-magnitude negatives get small varints, which plain two's-complement cannot deliver.
- E4.3** `double` is wire type 1 (fixed 64-bit), `int64` is wire type 0 (varint). Old consumers see tag 4 with an unexpected wire type: they cannot interpret the bytes as their known field and skip it — silent data loss. This is why the evolution rules forbid changing a field's wire type; deprecate tag 4 and add a new tag instead [Goo24].
- E4.4** Add if: `{properties: {status: {const: "active"}}}`, required: `[status]}`, then: `{required: [qty]}` to the schema. Validate three instances: `active-with-qty` (pass), `active-without-qty` (fail), `inactive-without-qty` (pass) [JSO22].
- E4.5** `additionalProperties: false` inside one `allOf` branch is evaluated *per subschema*, so fields defined only in a sibling branch look “addi-

tional” and legitimate extensions are rejected. `unevaluatedProperties: false` at the top level is evaluated after all subschemas merge, fixing the composition; before/after validator output proves it [JSO22].

- E4.6** `text/html` matches its explicit range at $q = 0.7$ (most specific wins); `application/json` matches only `*/*` at $q = 0.5$. Server score: `html` 0.7, `json` 0.5 — `html` is served. The negotiation selector must confirm exactly this.
- E4.7** Count records per second over a deterministically generated 1,000,000-line file with a buffered reader. The early stop after 100 records demonstrates generator laziness: the reader never touches line 102 onward — streaming gives constant memory and early-exit safety.
- E4.8** Torn stream: a final line truncated mid-JSON by naive concatenation (`{"a":1}\n{"b":2}`). The chapter’s reader yields every complete record and treats the trailing partial line as incomplete (skip/quarantine), never crashing — the property that makes NDJSON safe for append-oriented shipping.
- E4.9** `object_pairs_hook` receives the raw pairs list; flag duplicates when key count $\neq$ unique key count. On `{"a": 1, "a": 2}` a first-wins parser returns 1 and a last-wins parser returns 2 — a parser differential that attackers use to slip values past one layer of a system [Bra17].
- E4.10** `float(2**53 + 1) == float(2**53 + 2)` evaluates `True` in Python while the integers differ — both collapse to the same IEEE-754 double. This simulates JSON parsers (JavaScript’s `JSON.parse`) that decode all numbers as doubles; IDs above 2^{53} must travel as strings.
- E4.11** Avro resolution permits `int`→`long` because every 32-bit value fits in 64 bits; the reverse is rejected because the *value space*, not the current data, defines compatibility — nothing guarantees tomorrow’s writer stays in range. Forward compatibility is a property of schemas, not of observed datasets.
- E4.12** The missing header is `Vary: Accept`; without it the CDN keys the cache entry on the URI alone and serves the cached v1 representation to clients that negotiated `v=2`. The origin must emit `Vary: Accept` on every negotiated response [Fie22a].

B.5.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Capstone 1–5”; they are cited here as C4.1–C4.5.

Capstone 4.1 — NDJSON Inventory Log Shipper [junior]

Criterion	Weight	Full marks when...
Validation accuracy	30%	all 9,950 clean records shipped; exactly the poisoned set quarantined — no more, no fewer.
Atomicity	25%	a mid-batch kill leaves only a <code>.tmp</code> file; no torn line ever lands in <code>.ndjson</code> .
Bounded memory	20%	flat RSS across the full run (streaming, not load-all).
Reproducibility	15%	same seed → same shipped/quarantined split.
Error reporting	10%	quarantine summary names line numbers and schema violations.

Reference approach. Stream the input line by line; validate each against the JSON Schema; write valid lines to a staging `.tmp`, quarantine invalid ones with reasons; atomically rename on batch completion. Stretch: gzip rotation and an RFC 9457 problem summary [Not23].

Capstone 4.2 — Schema-Validation Middleware [mid]

Criterion	Weight	Full marks when...
Error contract	30%	every invalid body returns 422 <code>application/problem+json</code> with a complete <code>errors[]</code> array of JSON Pointers.
Zero false rejections	25%	all 400 valid corpus bodies pass byte-identical.
Performance	25%	schemas compiled once at startup; measured per-request overhead documented.
Reusability	20%	route-to-schema mapping is configuration, not code edits.

Reference approach. WSGI middleware intercepting request bodies for mapped routes, running a pre-compiled validator, and short-circuiting with the problem document on failure. Stretch: content-negotiate the error body (JSON vs HTML) using the chapter’s selector.

Capstone 4.3 — Hand-Rolled Protobuf Codec Extension [senior]

Criterion	Weight	Full marks when...
Wire correctness	30%	byte-level golden tests (committed hex) for ≥ 5 messages covering packed repeated, enum, and embedded message [Goo24].
Unknown-field retention	25%	unknown fields survive a decode/encode round-trip byte-identical.
Evolution governance	25%	reserved-tag rules documented; a lint rejects tag reuse.
Engineering	20%	codec stays dependency-free; tests deterministic.

Reference approach. Extend the mini codec with a second message type and nested-message framing (length-prefixed), packed varint runs for repeated fields, and an unknown-fields bag carried on the decoded object. Stretch: zigzag `sint64` deltas and a packed-vs-unpacked benchmark.

Capstone 4.4 — Negotiation-Aware Cache Key Tool [senior]

Criterion	Weight	Full marks when...
Detection accuracy	35%	every seeded mis-Vary'd response flagged; zero false positives.
Key stability	25%	variant keys deterministic and canonicalized (case, q-value normalization).
Explanation quality	25%	report explains each collision via the negotiation algorithm, not just “mismatch”.
Offline rigor	15%	recorded-exchange fixtures; one-command run.

Reference approach. Parse recorded exchanges; compute the cache key as URI plus the Vary-listed request headers (normalized); simulate lookups across the corpus and report collisions and misses. Stretch: quantify hit-rate loss from an overly broad Vary: `User-Agent`.

Capstone 4.5 — Format-Migration CLI [staff]

Criterion	Weight	Full marks when...
Conversion completeness	25%	all 4×4 format pairs stream offline in bounded memory.
Compatibility gate	25%	seeded backward-incompatible edit blocked with precise reasons; compatible edit permitted.
Semantic verification	25%	verify mode proves semantic equality (including float tolerance), not byte equality.
Reporting & ops	25%	dry-run impact report matches the benchmark harness within noise; versioned manifest committed; failure behavior documented.

Reference approach. Define a canonical in-memory record model; write one streaming reader and writer per format; a schema manifest with explicit compatibility classes drives the gate; verify mode replays both datasets through the canonical model. Stretch: a machine-readable registry manifest and a chaos mode that truncates output mid-stream, which verify must catch.

B.6 Chapter 5: Client-Side Security**B.6.1 Self-Assessment Model Answers**

Numbering note: the chapter uses plain 1–7 exercises and 1–5 deep-dive questions; numbering below matches.

1. 22 base64url chars from `secrets`: $22 \times 6 = 132$ bits — acceptable. UUIDv4: 122 bits — acceptable. 32 hex chars from `random.Random(42)`: zero real entropy (deterministic PRNG, known seed) — reject. `sha256(hostname+date)`: the input space is brute-forceable, so effective entropy is a few dozen bits — reject. Only CSPRNG output counts; hashing low-entropy input does not create entropy.
2. `canonical_query` must sort parameters by key, then by value, and sign *every* value of a repeated key; keeping only the first or last value means client and server sign different queries — an interop break, and worse, a smuggling gap where an attacker appends an unsigned second `filter`. SigV4 requires sorted, fully enumerated pairs.
3. Leeway must exceed worst-case clock skew plus latency: $20 + 8 \approx 28$ s, so 30–60 s is defensible. Within the leeway window a captured token

replays successfully; a `jti` cache with TTL equal to the leeway window closes that residual risk [Jon15].

4. Without `state`, an attacker can inject their own authorization code into the victim's redirect (login CSRF/code substitution), binding the victim's session to the attacker's account. PKCE protects the token *exchange* from interception; it does not bind the redirect response to the browser that initiated the flow — that is exactly `state`'s job [Sak15; Har12].
5. `exp` as a JSON string: reject — claims must be type-checked before comparison. `aud` as a list containing your audience plus a foreign one: safest is reject unless multi-audience is explicitly supported (and then require `azp` binding); accepting blindly invites audience confusion. `crit: ["exp"]`: reject — an unrecognized critical extension must fail closed, never be ignored.
6. With 64 threads and a 50 ms fake refresh, instrumentation must show exactly one refresh: the double-checked lock admits one thread to refresh while the rest find the fresh token. Removing the inner check lets every queued thread refresh after acquiring the lock — a thundering herd against the token endpoint.
7. Effective adversarial lines: secrets split across two log fields, URL-encoded tokens, JSON with irregular spacing. Regex redaction misses these; defense in depth: structured logging that never emits raw headers/bodies, allowlist field logging, and a DLP scan at log-ship time.

Deep-Dive Questions

1. Signing a payload hash prevents *tampering*, not *replay*: a captured signed request can be resent verbatim. Timestamp + nonce bounds validity; the server-side replay cache is keyed by (signature or nonce) with TTL equal to the allowed clock window, and deduplication runs before any side effect.
2. The front channel (browser redirect) is observable and modifiable — URLs land in history, logs, and referrer headers — so it must never carry tokens. The back channel (server-to-server) is authenticated and confidential. The split exists because the browser is a confused deputy: it transports, but cannot be trusted with secrets [Har12].
3. Rotation reuse detection: presenting a retired refresh token revokes the whole family. False positive: the client crashed after receiving the

new token but before persisting it, so it replays the old one. A robust client persists-then-discards (write the new token before deleting the old) and offers a re-auth recovery; a lenient server may accept the immediately-previous token once within a short grace window and flag it for review.

4. Stateless JWT verification buys zero-lookup verification, horizontal scale, and latency; the price is a revocation gap until `exp` and stale claims (a revoked role lingers). Introspection wins when revocation immediacy matters (payments, admin tooling), tokens are long-lived, or per-request policy freshness is required — it reintroduces the lookup deliberately.
5. Auditor answer, in three moves: (1) a mobile app is a public client — any embedded secret is extractable from the binary regardless of obfuscation, so client credentials would be one shared secret whose extraction compromises every installation; (2) authorization code + PKCE binds the exchange to the initiating device and user, issuing per-user tokens with consent; (3) the flow preserves user identity for audit and per-user revocation, which a shared client secret cannot [Sak15; OWA23].

B.6.2 Capstone Grading Rubrics

Capstone 5.1 — HMAC-Signed Webhook Consumer [junior]

Criterion	Weight	Full marks when...
Verification correctness	35%	17 valid events accepted; 3 corrupted events rejected with distinct reasons.
Replay defense	25%	replayed event rejected by the replay cache within the tolerance window.
Constant-time comparison	25%	<code>hmac.compare_digest</code> used throughout — <code>grep</code> finds no <code>==</code> on signatures.
Offline determinism	15%	byte-reproducible output; no network.

Reference approach. Parse `t=...,v1=...`, recompute HMAC over `t.body` with the shared secret, compare in constant time, enforce the timestamp tolerance, and record seen event IDs. Stretch: two-secret rollover window and constant-time measurement.

Capstone 5.2 — Full PKCE CLI [mid]

Criterion	Weight	Full marks when...
Flow correctness	30%	complete auth-code+PKCE exchange against the stub IdP; <code>state</code> and verifier validated [Sak15].
Token storage	20%	tokens stored owner-only-permission; zero token material in any log.
Refresh discipline	25%	proactive refresh; exactly one refresh under 8 concurrent invocations (single-flight).
Rotation handling	25%	retired-token replay triggers family revocation and a clean re-authentication path.

Reference approach. Loopback redirect listener for the code; verifier/challenge from `secrets`; a token cache file with 0600 permissions guarded by a process-wide lock for single-flight refresh. Stretch: device-code mode and OS keychain with file fallback.

Capstone 5.3 — JWT Security Linter [mid]

Criterion	Weight	Full marks when...
Rule coverage	35%	≥ 6 anti-pattern categories detected with file/line attribution (algorithm confusion, <code>==</code> comparison, missing <code>aud/exp</code> checks, ...).
Precision	25%	zero false positives on the clean corpus.
Traceability	20%	each rule cites a chapter subsection or OWASP category [OWA23].
Determinism	20%	sorted, byte-stable output across runs.

Reference approach. Walk the `ast` for JWT-related call sites; match call patterns against rule predicates; report through a stable, sorted finding model. Stretch: auto-fix mode and pre-commit integration.

Capstone 5.4 — Token-Broker Service [senior]

Criterion	Weight	Full marks when...
End-to-end flow	30%	login $\rightarrow$ broker $\rightarrow$ scoped call works; scope escalation rejected.
Secret hygiene	25%	no long-lived secret in any client process or file.
Audit trail	25%	audit log carries <code>sub/jti/scope/expiry</code> and zero token material.
Failure behavior	20%	broker outage yields bounded client failure (one retry, exit 2) — no hang, no credential fallback.

Reference approach. The broker holds the IdP credential, mints short-lived task-scoped tokens, and logs issuance; clients authenticate to the broker with their developer identity. Stretch: a `jti` blocklist endpoint with propagation measurement and RS256 issuance.

Capstone 5.5 — Red-Team Your Own Client [staff]

Criterion	Weight	Full marks when...
Threat model	25%	≥ 6 abuse cases traced to concrete code locations.
Findings & fixes	30%	≥ 3 genuine findings in the chapter listings, each with a landed fix and regression test.
CI gate	25%	gate fails when a vulnerability is reintroduced and passes on the fixed tree.
Residual-risk report	20%	accepted findings quantified; roll-back/exception process stated.

Reference approach. Work the attack surface methodically (token handling, signature verification, storage, logging); convert every exploit into a failing test, then fix; wire the corpus into CI. Stretch: differential-fuzz the verifier against a reference library over 10,000 seeded tokens.

B.7 Chapter 6: Building APIs with FastAPI

B.7.1 Self-Assessment Model Answers

Numbering note: the chapter labels exercises E1–E8 (no chapter prefix); numbering below matches.

- E1.** `def get_api_version(x_api_version: str = Header("1"))` exposed through a typed alias (`Annotated[str, Depends(...)]`); tests assert the default "1" without the header and the override with it. The alias keeps every signature self-documenting [Ram24].
- E2.** Store (`key` $\rightarrow$ `body hash`, `stored response`); same `key` + same `body` replays the original 201 without re-executing; same `key` + different `body` returns 409 (fingerprint mismatch). This is the idempotency contract of Chapter 11 applied to `POST /orders`.
- E3.** The middleware schedules `asyncio.sleep(0)` and measures the delay until it runs — that delay *is* event-loop lag. A deliberately blocking endpoint (CPU loop or `time.sleep`) spikes the metric for *all* concurrent requests; moving the work to a threadpool (or making it genuinely async) flattens it. Lag is the observable; blocking is the cause.

- E4.** The counter must live on `app.state` because middleware instances are not the canonical shared store — the application object owns lifespan-managed state, and counters on middleware attributes can be recreated or fragmented across instances. Pure ASGI middleware keeps the count below the framework layer, so it sees every request.
- E5.** Duplicate-SKU is a persistence invariant: only the repository can see all existing rows, and a Pydantic `model_validator` couples the rule to one transport DTO (a second write path would bypass it). Moving it to `insert` as a domain error mapped to 422 is correct, and it implies cross-aggregate invariants (max five open orders per customer) also belong in the repository/service layer, never in DTOs.
- E6.** Paging the repository in limit/offset batches of 100 inside the generator bounds memory at one batch and lets the stream observe committed writes between batches. Snapshotting prevents iterator invalidation and mid-yield mutation, but at the cost of pinning the entire result set in memory for the stream’s lifetime — the failure mode this exercise trades away deliberately.
- E7.** Exiting the `TestClient` context runs the lifespan shutdown, which calls the repository’s `close()` — asserted by a flag on the repo. Uvicorn differs by being signal-driven: lifespan shutdown runs on `SIGTERM` during graceful shutdown (documented in uvicorn’s lifespan section), so the same code path must be verified under a real server before production (Chapter 18).
- E8.** The test diffs live `/openapi.json` against a checked-in snapshot; renaming a response field changes the document and fails CI. The point: your OpenAPI output *is* the contract, and silent drift becomes a build failure rather than a customer surprise [Ope21].

B.7.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Capstone 1–5”; they are cited here as C6.1–C6.5.

Capstone 6.1 — Todo API with a Full Test Suite [junior]

Criterion	Weight	Full marks when...
CRUD correctness	30%	all routes behave per contract, including the complete transition, with deterministic IDs.
Response discipline	25%	every route declares <code>response_model</code> ; no internal field leaks in any response.
Contract documentation	20%	OpenAPI documents a 422 for every body-accepting route.
Test suite	25%	pytest passes offline from a fresh venv; in-memory repo behind a protocol.

Reference approach. Repository protocol + in-memory implementation injected via dependency overrides; TestClient covering every route and error path. Stretch: correlation-ID middleware with header echo and an NDJSON streaming endpoint with a streaming test.

Capstone 6.2 — Inventory API with a SQLite Repository [mid]

Criterion	Weight	Full marks when...
Repository parity	30%	the identical suite passes against <code>memory</code> and <code>sqlite</code> with zero code changes.
Conflict semantics	25%	oversubscription returns a 409 problem document naming part and shortfall [Not23].
Readiness honesty	25%	readiness returns 503 when the SQLite path is a directory — a real failure, not a mock.
Settings plumbing	20%	backend selection via settings only; no branching in route code.

Reference approach. Rebuild the Orders pattern for parts, stock, and reservations; the readiness probe opens the configured database for real. Stretch: optimistic concurrency with a `version` field and 412 on mismatch (Chapter 11).

Capstone 6.3 — Plugin-Style Dependency-Injection Architecture [senior]

Criterion	Weight	Full marks when...
Plugin ergonomics	30%	a new plugin is one new file plus one registry line — no route or factory edits.
Fail-fast config	25%	unknown plugin name fails at startup with an actionable error.
Test replaceability	25%	<code>dependency_overrides</code> tests prove per-test plugin swapping.
Composition clarity	20%	<code>lifespan</code> composes the selected plugins explicitly; startup logs the effective set.

Reference approach. Define protocols for repositories, notifiers, and pricing policies; a name $\rightarrow$ factory registry resolved in the `lifespan`; routes depend on protocols only. Stretch: expose plugin versions in `/health` and a contract test that every notifier satisfies a shared behavioral suite.

Capstone 6.4 — Async Migration of a Blocking Service [senior]

Criterion	Weight	Full marks when...
Baseline proof	25%	loop-lag middleware shows >5 s lag under the 200-way concurrency load before the fix.
Fix efficacy	30%	both fixes (<code>def-threadpool</code> and <code>asyncio.to_thread</code>) bring lag under 50 ms; p99 improves by an order of magnitude over three reproducible runs.
Evidence quality	25%	decision memo cites pool-occupancy and lag measurements, not folklore.
Reproducibility	20%	benchmark harness is deterministic and offline.

Reference approach. Keep the blocking implementation behind the repository protocol, swap implementations, and let the lag middleware plus the concurrency benchmark adjudicate. Stretch: a genuinely async store and threadpool queue-depth instrumentation.

Capstone 6.5 — OpenAPI-First Generator Round-Trip [staff]

Criterion	Weight	Full marks when...
Gate correctness	30%	seeded compatible and breaking changes classified correctly with readable reasons; running service passes its own gate with zero round-trip diffs.
Round-trip fidelity	25%	generated typed client works against the service in a TestClient integration test without hand edits.
Governance policy	25%	written policy states who may bump which version number and how breaking changes ship (Chapter 10).
Operational rollout	20%	gate wired into CI with documented override and rollback path.

Reference approach. Hand-author the OpenAPI 3.1 spec as the source of truth [Ope21]; generate server skeleton and typed client; implement; build the diff gate over normalized specs; then execute one governed compatible change and demonstrate one blocked breaking change. Stretch: diff report as an RFC 9457 problem document and undocumented-endpoint detection.

B.8 Chapter 7: Errors, Validation & Problem Details

B.8.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–10 list; numbering below matches.

1. Malformed JSON at POST `/shipments` fails at the *transport/syntax* layer of the taxonomy: the body never becomes a data structure, so Pydantic is never invoked. The result is 400 (bad request — the bytes are at fault), not 422, and the problem body should say the JSON itself could not be parsed [Not23].
2. 404-for-unauthorized is correct when resource *existence* is sensitive (private tenant data — IDOR probing must learn nothing). It still leaks when existence is inferable anyway (enumerable public IDs, timing differences between “exists but forbidden” and “absent”, or when a 403-only endpoint elsewhere confirms the namespace). Then 404 only confuses legitimate clients while fooling no one.

3. Safe retry of a conditionally-retriable 409: (1) on 409, re-read the current resource state; (2) re-evaluate the precondition locally; (3) if the conflict is semantic (duplicate key), abort — no retry helps; (4) otherwise resubmit with a fresh precondition (**If-Match**/version); (5) bound attempts with jittered backoff. Pseudo-code must show the re-read — blind resubmission just re-loses.
4. `unit_weight_kg` is `strict=False` because real clients send JSON strings like "1.5"; removing the opt-out rejects those payloads (a compatibility break). Removing the companion validator as well drops the range/unit checks, so negative or absurd weights pass — the two knobs guard different failure classes and both are needed.
5. JSON Pointer: `/lines/2/dimensions/width` (array indices are decimal positions). Per RFC 6901 escaping, `~` becomes `~0` and `/` becomes `~1`, so a key `a/b` is addressed as `/a~1b`. The pointer must survive this escaping round-trip exactly.
6. Misreading breaker-503s as application 500s sent the incident to the wrong team with the wrong hypothesis (code bug vs dependency/capacity), burning the golden first hour. A metric label carrying the failure *origin* (e.g. `error_source="circuit_breaker"`) alongside `status_code` prevents recurrence.
7. *For* localizing `title`: developer experience for global consumers. *Against*: it breaks log aggregation, alerting rules, and support search that key on the stable string. The invariant that makes localization safe: `type` (and any `code` extension) remain constant and machine-consumable; only human-readable fields localize [Not23].
8. Apply the scrubber per chunk with a sliding overlap window so secrets spanning a chunk boundary are still caught; accept the trade-off: boundary guarantees are approximate (window-bound) and over-redaction near boundaries is possible — in exchange for never buffering a 50 MB stream.
9. `additionalProperties: false` (or `unevaluatedProperties` across composed schemas) on the problem schema makes any undocumented extension member fail validation in CI [JSO22]. Implied workflow: registry-first — the schema and the emitter change land in the same PR, so the gate and the contract move together.
10. Map alert thresholds onto error-budget burn rates (Chapter 19): page on fast-burn 5xx, ticket on slow-burn. The code that should never

page is 4xx (except authentication-abuse spikes): client errors are caller-owned, and paging on them trains the on-call to ignore pages.

B.8.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Project 1–5”; they are cited here as C7.1–C7.5.

Capstone 7.1 — Problem-Details Retrofit [junior]

Criterion	Weight	Full marks when...
Uniformity	30%	every error — including framework-generated 404s — is schema-valid <code>application/problem+json</code> [Not23].
Success-path preservation	25%	no success-path test changed or broken.
Validation errors	25%	422s carry JSON-Pointer-located <code>errors[]</code> .
Runnability	20%	pytest green from a fresh venv, offline.

Reference approach. Install exception handlers for the framework’s request/validation/HTTP exceptions plus a catch-all; centralize the problem serializer; prove with a route-by-route error matrix. Stretch: `Accept-Language`-localized `title` for two locales.

Capstone 7.2 — Error-Code Registry & Documentation Generator [mid]

Criterion	Weight	Full marks when...
Registry validation	30%	import-time validation catches an injected duplicate code and a missing remediation field.
Generated artifacts	25%	Markdown docs and <code>errors.json</code> validate against the published schema.
Orphan detection	25%	the CI check fails on a service emitting an unregistered code.
Determinism	20%	version-diff output is byte-stable and sorted.

Reference approach. Declarative registry (one record per code: status, title, remediation, since-version) validated at import; generators render docs and JSON from the same records; the orphan check runs the service’s error paths against the registry. Stretch: per-code deep links as `type` URIs and a mark-never-delete deprecation workflow.

Capstone 7.3 — Redaction Library with Fuzz Corpus [mid]

Criterion	Weight	Full marks when...
Corpus coverage	30%	100% of the 50+ adversarial corpus redacted (split tokens, encodings, odd spacing).
False-positive discipline	25%	documented bounded FP rate with a CI regression gate.
Extensibility	20%	new rules added without editing library code.
Determinism	25%	seeded mutation sweeps; fully offline; reproducible reports.

Reference approach. Harden the chapter scrubber behind a rule pipeline (pattern + replacement + confidence); attack it with the curated corpus plus seeded mutation; publish the FP budget. Stretch: streaming redaction with boundary windows and JSON-aware field-name scrubbing.

Capstone 7.4 — Error-Observability Dashboard Specification [senior]

Criterion	Weight	Full marks when...
Incident detection	30%	replay of the seeded week detects all three injected incidents with zero false pages on baseline.
Declarative rules	25%	alert rules are data (thresholds, windows, codes), not code.
Traceability	25%	every dashboard panel traces to a registry error code.
Burn-rate correctness	20%	the burn-rate panel matches a hand-computed 100-request example.

Reference approach. Derive the metric set from the error registry (rate per code, burn-rate windows); replay a seeded log week through the alert evaluator; render the dashboard spec as data. Stretch: fast/slow burn-rate windows per Chapter 19 and per-code anomaly baselines.

Capstone 7.5 — Chaos Error-Injection Test Harness [staff]

Criterion	Weight	Full marks when...
Chain integrity	30%	every fault yields a schema-valid problem response in the correct 5xx family — never a raw 500 or HTML.
Correlation	25%	trace IDs join both services' logs for every scenario.
Determinism & speed	20%	fixed seed, fully offline, full run under one minute.
Coverage matrix	25%	a fault $\times$ contract matrix demonstrates no blind spots; rollback of injected state verified between scenarios.

Reference approach. In-process fault injection (timeouts, malformed upstream payloads, resets, slow leaks) between two toy services; assert the whole error chain — status mapping, problem shape, redaction, correlation, metrics — under each fault. Stretch: composed faults and a “contract score” trending across registry versions.

B.9 Chapter 8: Pagination, Filtering & Sorting

B.9.1 Self-Assessment Model Answers

- E8.1.** Page k costs $c_{\text{seek}} + k c_s + p c_f$; summing over n/p pages gives total row visits $\sum_k kp \approx p \frac{(n/p)(n/p+1)}{2} \approx \frac{n^2}{2p} = \Theta(n^2/p)$. The worst page size from the server's viewpoint is $p=1$: maximal pages, maximal total work ($\Theta(n^2)$).
- E8.2.** One valid schedule (page size 5): page 1 emits r1–r5; page 2 (offset 5) emits r6–r10; then the writer *moves* r7 to the end of the table (position 20) and *moves* r20 into position 7 (e.g. via sort-key updates). Page 3 (offset 10) now reads r11–r15; page 4 (offset 15) reads r16, r17, r18, r19, and the relocated r7. Final multiset: r1–r19 with r7 twice; r20, sitting in the already-read region, is never emitted.
- E8.3.** Re-reading the previous page's last row detects overlap (duplicates) at the boundary, but a skipped row leaves no positional trace — the position stream carries no identity information, so detection without recovery is the best any positional scheme can do; identity-bearing cursors are the fix.

- E8.4.** Embed a SHA-256 hash of the active filter/sort parameters inside the signed cursor; a continuation whose presented parameters no longer match the hash is rejected with 400. This closes cursor-replay and confused-deputy bugs where a cursor minted under one filter is replayed against another — the server would otherwise silently return a different result set than the client believes it is walking.
- E8.5.** Cap page size and maximum page depth; cache or approximate the COUNT with a TTL (exact counts are a separate, expensive product); back the grid with a covering index; offer background export for deep access. Tell the PM the physics: deep offset pages cost $\Theta(n^2/p)$ server work — page 47 is a filter/search problem, not a browsing problem.
- E8.6.** By trichotomy: if $a > x$ the tuple is greater regardless of b ; if $a = x$ the comparison reduces to $b > y$; if $a < x$ both sides are false — hence $(a, b) > (x, y) \iff a > x \vee (a = x \wedge b > y)$. Six-row counterexample for the hand-rolled **a >= x AND b > y** with cursor (1, 2) over rows (1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3): the buggy predicate drops (2, 1) and (2, 2) (skipped rows), while the mirror bug **a > x OR b > y** re-admits already-seen rows (duplicates) — the two distinct error modes of offset-free predicates done wrong.
- E8.7.** Order by the null-flag first: `ORDER BY (discontinued_at IS NULL), discontinued_at, id` and build the keyset predicate over the triple — the boolean null flag becomes the leading tuple element, so the disjunctive expansion handles the NULL boundary exactly like any tie-break; verify against the filter_compiler demo database with `EXPLAIN QUERY PLAN`.
- E8.8.** Expansion policy: cap depth at 2, batch expansions with per-resource fan-out limits (e.g. 50), set `Cache-Control` to `private/no-store` when expansions embed per-user data and short `max-age` otherwise. When selections become arbitrary or deeper than the cap, redirect the consumer to the GraphQL gateway — that is the workload GraphQL exists for (Chapter 13).
- E8.9.** Create an index whose column order cannot serve both the range predicate and the ORDER BY (e.g. (id, posted_at) where the keyset needs (posted_at, id)), or one leading with a low-cardinality column: the plan flips from `SEARCH . . . USING INDEX` to `SCAN` plus a temp B-tree. The planner is saying no available index satisfies filter + ordering together — the defining requirement of keyset pagination.
- E8.10.** Two architectures: (1) transactionally maintained count tables (fail-

ure: drift and write amplification — mitigate with reconciliation jobs);
 (2) an asynchronous export/audit service running exact `COUNT(*)`
 over the indexed predicate with statement timeouts and as-of times-
 tamps (failure: latency and staleness — bounded by reporting the
 count’s observation time). Either way, exact totals are isolated from
 the list API so metadata cost never taxes interactive reads.

B.9.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Capstone 1–5”; they are cited here as C8.1–C8.5.

Capstone 8.1 — Cursor-Paginated Ledger API [junior]

Criterion	Weight	Full marks when...
Walk correctness	30%	a scripted full walk emits every one of the 100k postings exactly once (set comparison).
Index discipline	25%	EXPLAIN shows SEARCH using the composite (<code>posted_at</code> , <code>ledger_id</code>) index on every page.
Cursor security	25%	forged, expired, and version-mismatched cursors each receive the same uniform 400 problem body.
Tie handling	20%	timestamp ties across page boundaries produce no duplicates or skips.

Reference approach. Order by the composite key; the cursor is an HMAC-signed payload of the last row’s key tuple; the predicate is the disjunctive tuple expansion. Stretch: `rel="prev"` via reversed-order query and per-account filtering on an index-led equality prefix.

Capstone 8.2 — Filter Compiler with Property Tests [mid]

Criterion	Weight	Full marks when...
Injection safety	30%	property tests prove no raw input ever reaches SQL; all 50 attack strings rejected or provably inert when executed.
Grammar & OR support	25%	documented grammar; OR-groups compile correctly.
Namespace hygiene	20%	no public parameter name maps to an unintended internal column.
Parameter binding	25%	inspection test shows compiled queries use bound parameters exclusively.

Reference approach. Parse to an AST, compile the AST to SQL with placeholders, never interpolate; the property test mutates inputs and asserts

the compiled SQL text is input-free. Stretch: case-insensitive `contains` with correct collation and an EXPLAIN-driven index-recommendation report.

Capstone 8.3 — Pagination Benchmark Report [mid]

Criterion	Weight	Full marks when...
Reproducibility	25%	one command regenerates every table from fixtures.
Asymptotic evidence	30%	offset growth demonstrably linear (fit reported); keyset flat within noise; cursor overhead constant with depth.
Index evidence	25%	every query named with its EXPLAIN plan.
Scale coverage	20%	100k / 200k / 1M rows, p50/p95, cold-cache variant.

Reference approach. Seed the three table sizes; sweep page depths under each scheme; fit offset latency vs depth to a line and report residuals. Stretch: a covering-index variant showing the offset penalty shrink but not vanish, and a PowerShell runner for the matrix.

Capstone 8.4 — Infinite-Scroll Client over Signed Cursors [senior]

Criterion	Weight	Full marks when...
Opaque-cursor discipline	25%	client source contains no offset arithmetic and no cursor parsing — it follows <code>Link rel="next"</code> only [Not17].
Resume correctness	30%	kill-and-resume completes the walk with zero duplicates using on-disk state.
Failure drills	25%	expired and forged cursors get controlled, user-visible recovery; server-restart drill passes.
Model fidelity	20%	mid-walk-insert behavior note matches the chapter's drift model.

Reference approach. Terminal browser persisting the last `next` link after every page; on restart, resume from it; on cursor rejection, offer refetch-from-filter with a clear notice. Stretch: per-page ETag conditional requests and silent offset-to-cursor link negotiation.

Capstone 8.5 — Multi-Tenant Keyset Design [staff]

Criterion	Weight	Full marks when...
Isolation	25%	cross-tenant cursor replay is impossible, proven by test (tenant id inside the signed cursor and the predicate).
Depth independence	25%	all tenants (1k / 200k / 5M rows) show depth-independent page latency, measured and tabulated.
Resilience	20%	whale-tenant export resumes after a simulated crash with no loss.
ADR quality	30%	every rejected trade-off (offsets, per-page counts, transparent cursors) named with reasons; operational criteria (index ownership, monitoring, rollback) included.

Reference approach. Tenant-led composite index (`tenant_id`, `sort key`...); the cursor binds tenant and position; per-tenant policy table for page size and export quotas; the ADR carries the decision record for the org. Stretch: row-level security via views/roles and snapshot-id cursors for compliance exports.

B.10 Chapter 9: Versioning & Evolution**B.10.1 Self-Assessment Model Answers**

E9.1. Optional response field added: *non-breaking* (additive). Optional request field made required: *breaking*. `price` → `unit_price_cents` (rename + retype): *breaking* — twice. JSON member reordering: *non-breaking* — object member order is insignificant [Bra17]. Changed 404 **detail** message: formally non-breaking (**detail** is human-readable), but *semantically hazardous* if clients parse it — a contract-test finding, not a schema one. New 410 for retired resources: a *behavioral* change clients must handle; announce it like a breaking change even though the schema is untouched.

E9.2. The gate flags as breaking: a newly *added required* path/query parameter and any *removed* parameter; fixtures prove both directions (old consumers cannot call the new shape; new definition no longer accepts old calls). Optional added parameters and added optional request body fields pass.

E9.3. With Accept: `application/vnd.nr.v2+json;q=0.3`, `application/vnd.nr.*;q=0.9`, the higher-q wildcard wins and `negotiate` returns the default ver-

sion — surprising but correct q-value semantics. Reverse the weights (explicit 0.9, wildcard 0.3) and v2 is chosen: explicit intent must outbid the wildcard.

- E9.4.** A multi-currency price list cannot be back-translated into v1's scalar price. The three honest options: (1) ship the field in v2 only (v1 consumers simply do not get the feature); (2) expose a documented lossy approximation in v1 (e.g. base-currency only, flagged); (3) schedule v1 retirement. Expand-and-contract prescribes (1) with a managed path to (3): add alongside, never mutate meaning in place, and let usage telemetry drive the sunset.
- E9.5.** Required response headers: **Vary**: **X-API-Version** on every response, plus a deliberate **Cache-Control** per versioned resource. If a CDN ignores **Vary**, it keys entries on URI alone and serves a cached v1 representation to v2 requesters — a contract violation that looks, maddeningly, like an application bug [Fie22a].
- E9.6.** *Defend* media-type versioning here: the ecosystem is closed (you ship the clients), generated models tolerate exact content types, and URIs stay stable. The six-month release tail dictates the timeline: support N-2 versions for at least 6–9 months, instrument per-version traffic, and use a minimum-version killswitch only with a consumer notification runway.
- E9.7.** Publish a security-exception clause *before* the incident: severe vulnerabilities may force immediate breaking change within a version, with (a) simultaneous notification on every channel, (b) minimal blast radius (revoke/compensate rather than redesign), (c) parallel-run or extended overlap wherever technically possible, and (d) a public post-incident review. The clause converts a contract violation into a governed emergency procedure.

B.10.2 Capstone Grading Rubrics

Capstone 9.1 — Add v2 to an Existing API with an Adapter Layer [junior]

Criterion	Weight	Full marks when...
Dual-version correctness	30%	v1 and v2 both respond correctly for every seeded record after the rename+retype.
Deprecation signaling	25%	v1 responses carry Deprecation , Sunset , and Link headers; v2 carries none [Wil19].
Adapter purity	25%	<code>to_v1</code> is a pure function; no duplicated business logic; adapter tests deterministic and offline.
Test coverage	20%	both surfaces covered for identical underlying state.

Reference approach. Evolve the internal model to v2; implement `to_v1` as a pure projection; mount both routers over one repository. Stretch: a third experimental version behind an opt-in header and an adapter-overhead timing loop.

Capstone 9.2 — Media-Type Version Negotiation Middleware [mid]

Criterion	Weight	Full marks when...
Negotiation correctness	35%	every outcome matches the published decision table (explicit, wildcard, q-values, default).
Error quality	20%	406 bodies enumerate the supported version=N values.
Cache safety	25%	Vary: Accept present on every 200, proven by test.
Suite health	20%	full pytest suite passes offline.

Reference approach. Generalize the chapter’s negotiation listing into middleware that resolves the vendor media type once per request and dispatches to versioned serializers on a single `/catalog` URI. Stretch: negotiate a second wire format (`+json` vs `+cbor`-style suffix) orthogonal to version.

Capstone 9.3 — Production Breaking-Change CI Gate [mid]

Criterion	Weight	Full marks when...
Verdict correctness	35%	exit code 1 exactly when breaking changes lack a major bump; all five fixture pairs produce expected verdicts.
Diff completeness	25%	local <code>\$ref</code> resolution and parameter diffing implemented.
Reporting	20%	JSON report round-trips through <code>json.loads</code> ; enum-growth is a warning tier, not a failure.
Escape hatch	20%	<code>-allow-breaking-with=VERSION_BUMP</code> works and is audit-logged.

Reference approach. Normalize both specs (resolve local refs), diff operations/parameters/schemas against the chapter’s compatibility classes, and classify findings into error vs warning tiers. Stretch: semantic-risk heuristics (format changes, pattern tightening, `additionalProperties: false` introductions).

Capstone 9.4 — Deprecation Campaign Plan & Retirement Analytics [senior]

Criterion	Weight	Full marks when...
Policy completeness	25%	timeline, SLAs, channels, and exceptions all documented.
Projection accuracy	30%	per-consumer trend analytics project a zero-crossing date and name the stragglers — deterministically over the 6-month/12-consumer fixture.
Operational checklist	25%	go/no-go items are all checkable from telemetry (usage at zero, brownout results), not sentiment.
Communication plan	20%	outreach keyed to the contact list with brownout simulation evidence.

Reference approach. Parse the synthetic access log into per-consumer weekly series; extrapolate linearly to a retirement date; simulate brownouts; gate the kill date on the checklist. Stretch: “what-if” mode showing the date shift if the top two consumers migrate next month (Chapter 21).

Capstone 9.5 — Version Governance at Platform Scale [staff]

Criterion	Weight	Full marks when...
Policy codifiability	25%	the policy is complete enough to lint mechanically — no judgment calls left implicit.
Linter breadth	25%	≥ 4 violation classes flagged on the seeded 40-internal/6-external registry.
Cost model	25%	carrying-cost report ranks versions reproducibly offline and identifies when the quadratic term bites.
Decision record	25%	RFC-style document defends the strategy mix against ≥ 2 alternatives, with roll-back/exception governance.

Reference approach. Write the policy as lint rules first, prose second; build the registry linter over a synthetic catalog; model carrying cost as teams $\times$ versions $\times$ time; the decision record compares URI/header/media-type mixes. Stretch: a ten-year portfolio simulation showing the policy-change trigger point.

B.11 Chapter 10: Idempotency, Caching & Concurrency

B.11.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–10 list; numbering below matches.

1. GET: safe and idempotent. PUT: idempotent, not safe. DELETE: idempotent, not safe. POST: neither. PATCH with `{"op": "set", ..., "value": 5}`: idempotent (absolute write), not safe. PATCH with `{"op": "inc", ...}`: neither — n retries add n times [Fie22a].
2. The claim is wrong: **no-cache** permits storage but mandates *revalidation with the origin before every reuse*; it is the freshness guarantee, not a storage ban. The never-persist directive is **no-store** — quoting the wrong one either leaks sensitive data or destroys cache efficiency.
3. With **max-age=60** and storage at $t=0$: $t=30$ is a hit (fresh). $t=60$ is the boundary — age $\geq$ max-age means stale, so revalidate (a 304 can still save the body). $t=61$: stale, revalidate. Verify all three against Listing 10.5.

4. First request: processed, response stored under the key. Every subsequent request with the same key but a different payload: fingerprint mismatch $\rightarrow$ 409/422, *no* re-execution. The batch job's logic bug (key reuse) is surfaced loudly instead of silently replaying the first transfer n times — exactly-once is preserved, and the client is forced to mint one key per logical operation.
5. *Store/replay*: a deterministic 422 will fail identically forever; replaying saves downstream load and gives the client a stable answer. *Abandon*: stored errors can fossilize a transient validation bug and complicate TTL policy. Stripe's published design persists and replays the first result including client errors (validation is deterministic) but does not replay transient 5xx — matching Listing 10.1's abandonment of the key row on downstream 5xx.
6. Extend the cache key with **Accept-Language** and emit **Vary: Accept-Language**. Timeline of the bug without it: a French request primes the shared cache; an English request for the same URI is served the French representation — a cross-user correctness leak caused by a missing header, not by application code [Fie22a].
7. One table: `idempotency_keys(key PK, request_fingerprint, status, response_snapshot, created_at)` with `status` $\in$ `{in_flight, completed}`. The business mutation and the key-row upsert commit in *one* transaction; a crash rolls both back, leaving neither an orphan key nor a partial mutation. A concurrent retry seeing `in_flight` waits or receives 409 — the row is the serialization point.
8. At a 95% 412 rate, expected attempts per success ≈ 20 , so optimistic throughput collapses to $\sim 5\%$ even with jitter; bounded retry only bounds the waste. Pessimistic row locks serialize correctly but head-of-line block every reader of the row. A per-SKU command queue serializes only the hot key while the rest of the catalog stays concurrent — pick the queue (or locks if simplicity dominates), with the $20\times$ waste figure as the quantitative justification.
9. Tag every cached representation with surrogate keys of the entities it embeds (**part-123** on the product page, search cards, and recommendation slots); a price change on part 123 purges exactly **part-123** across all three surfaces — no URL-pattern purges, no over-invalidation.
10. *Refuted*. Idempotency keys have a TTL (hours to days); after expiry a late retry re-executes and duplicates. Unique constraints are the timeless integrity backstop that knows nothing of TTLs; keys are a

dedup/UX layer at the transport, constraints are the correctness layer in storage. Both, always.

B.11.2 Capstone Grading Rubrics

Capstone 10.1 — Idempotency-Key Client SDK [junior]

Criterion	Weight	Full marks when...
Exactly-once effect	35%	up to 5 retries produce exactly one server-side mutation, proven against Listing 10.1's server.
Retry classification	25%	422 never retried; 409 retried with backoff; the policy table is explicit.
Key discipline	25%	a second logical operation mints a fresh key and executes a second transfer.
Determinism	15%	tests pass offline, seeded, no wall-clock sleeps.

Reference approach. An `httpx` wrapper minting one UUID key per logical operation, attaching `Idempotency-Key`, and classifying responses by retryability. Stretch: persist in-flight operations so a crashed client resumes with the same key.

Capstone 10.2 — Conditional-GET Sync Agent [junior]

Criterion	Weight	Full marks when...
Revalidation correctness	35%	unchanged polls return 304 and transfer zero representation bytes.
State fidelity	25%	local state equals the server representation after every poll.
Savings evidence	25%	bytes-saved report vs naive always-GET, reproducible from the scripted schedule.
Scheduling hygiene	15%	poll loop deterministic in tests (injected clock/sleeper).

Reference approach. Store the ETag beside each synced representation; send `If-None-Match`; on 304 touch nothing, on 200 replace. Stretch: Last-Modified/If-Modified-Since fallback when no ETag is present.

Capstone 10.3 — Optimistic-Locking Order Service [mid]

Criterion	Weight	Full marks when...
Conservation under storm	30%	100-worker storm preserves quantity exactly — no lost updates.
Status semantics	25%	blind writes get 428; stale writes get 412; correct retries succeed.
Client loop	25%	read–decide–write–retry loop with bounded attempts and jitter.
Reproducibility	20%	contention report identical across runs.

Reference approach. Extend Listing 10.3 so reserve/release are conditional writes on a version token (**If-Match**); the client re-reads on 412 before re-deciding. Stretch: adaptive escalation to a serialized per-SKU queue past a 412-rate threshold, with the throughput/latency trade-off quantified.

Capstone 10.4 — Cache-Policy Workbench [senior]

Criterion	Weight	Full marks when...
Evaluation correctness	30%	candidate policies ranked on origin load, staleness, and error resilience over the synthetic timeline; table bit-reproducible.
Outage resilience	25%	at least one policy serves zero failed requests during the scripted outage; the memo explains the mechanism (stale-while-revalidate / grace).
Safety classification	25%	account page correctly assigned no-store/private with justification.
Analysis depth	20%	trade-offs quantified, not adjectives.

Reference approach. Turn Listing 10.5 into a policy engine: a scripted request timeline drives each candidate **Cache-Control** policy; metrics are counted offline. Stretch: model a shared cache with a **Vary** dimension and demonstrate a misconfiguration leaking across user segments.

Capstone 10.5 — Correctness Audit of a Retry-Capable Gateway [staff]

Criterion	Weight	Full marks when...
Audit precision	25%	linter flags exactly the seeded violations — zero false positives, zero misses.
Remediation rigor	30%	every remediation names its enforcing mechanism (unique constraint, fingerprint, fencing token).
Telemetry plan	20%	one alert per war-story failure mode, with thresholds.
Blast-radius analysis	25%	the plan states what happens when the idempotency-key store is down (fail open vs closed, per endpoint) and how it rolls back.

Reference approach. Enumerate every retry source (clients, gateways, queues); classify each mutation endpoint by idempotency posture; the linter encodes the posture rules; the report sequences remediations by risk. Stretch: extend the audit across the async boundary (queue redelivery plus consumer-side dedup windows, Chapter 12).

B.12 Chapter 11: Async & Event-Driven APIs

B.12.1 Self-Assessment Model Answers

- E11.1.** Public stock-ticker widget: **SSE** — one-way server push over plain HTTP. Two-player browser chess: **WebSockets** — bidirectional, low latency. ERP notification on invoice approval: **webhook** — server-to-server with retries and signatures. Mobile badge count on a metered connection: **polling with ETags** (or platform push) — cheapest bytes, no standing connection. SOC wallboard behind a corporate proxy: **SSE** — survives proxies that terminate WebSocket upgrades.
- E11.2.** Two explanations fit valid signatures with 40-minute-old timestamps: (1) receiver (or sender) clock skew beyond the tolerance window; (2) genuine delayed/replayed deliveries after a long outage. The one-line distinguisher: have the provider include its current timestamp (or per-attempt IDs) in the delivery — or log the receiver's clock at rejection — so skew vs delay is decidable from one log line.
- E11.3.** During rotation the deliverer emits `t=...,v1=sig_new, v1=sig_old` (two `v1` values); the receiver accepts if *any* `v1` verifies against a known

secret. The test proves holders of either secret accept, and a receiver with neither rejects.

- E11.4.** Per the WHATWG grammar: a line with no colon is treated as a field name with an empty value; **data:** with empty value appends an empty line (the event still dispatches); an **id** containing NUL causes the field to be ignored; **retry: abc** is ignored because the value is not all digits. Encode each as a parser test.
- E11.5.** Unmasked final text frame, 60-byte payload: **81 3C**. Masked final binary frame, 300-byte payload: **82 FE** (FIN+opcode 2; mask bit + 126 extended length), then **01 2C**, then the 4-byte masking key. Ping with payload “hi”: **8A 02 68 69**. Byte two’s top bit is the mask flag in each case.
- E11.6.** **0x48 XOR 0x37 = 0x7F** goes on the wire. An in-browser attacker cannot choose payload bytes that survive masking unchanged because the *browser* generates a fresh masking key per frame and the attacker never observes it — masking exists precisely to stop attacker-controlled bytes from confusing intermediaries.
- E11.7.** Delays $\min(2^{n-1}, 60)$ for $n = 1..8$: $1+2+4+8+16+32+60+60 = 183$ s worst case. $\pm 20\%$ jitter prevents synchronized reconnection herds after a shared failure (thundering herd). The suite seeds **random** so every delay sequence is reproducible.
- E11.8.** With 2,000 missed events against a 500-event ring buffer, resume is impossible: the server answers the stale **Last-Event-ID** with a **resync-required** control event; the client drops its state, refetches a full snapshot via REST, then resumes the stream fresh. UX: a “reconnecting” notice followed by a clean full refresh — degraded, never corrupt.
- E11.9.** Runbook for “dead-letter rate > 10/hour”: (1) DLQ grouped by endpoint and error class; (2) last successful delivery per endpoint; (3) recent deploys/secret rotations. Decision tree: poison payload (schema mismatch) → fix producer, then replay; consumer down → wait for recovery, then replay; expired credentials → rotate, then replay; truly obsolete events → discard with a logged reason. Customer communication triggers when a contracted endpoint’s freshness SLO is breached, not on the alert itself.
- E11.10.** The CI script loads old and new AsyncAPI documents, walks channels/messages/payload properties, and fails on any removed or re-

typed property — a *backward-compatibility* gate (the new provider must still emit everything old consumers parse) [Asy23].

B.12.2 Capstone Grading Rubrics

Capstone 11.1 — Signed Webhook Receiver for Robot Alerts [junior]

Criterion	Weight	Full marks when...
Verification correctness	30%	all 12 valid events processed exactly once under the <code>t=...,v1=...</code> scheme with 300s tolerance.
Hostile-input handling	30%	tampered, stale, wrong-secret, replayed, and missing-header variants each rejected with a 4xx problem body.
Idempotent duplicates	25%	duplicates return 200 <code>{"status":"duplicate"}</code> with no side effects.
Determinism	15%	fixed injected clock; offline tests.

Reference approach. Verify HMAC over `t.body` in constant time, enforce tolerance, dedupe on event ID in a small store. Stretch: two-secret rotation window and Prometheus-style counters.

Capstone 11.2 — Webhook Delivery Service with DLQ and Replay [mid]

Criterion	Weight	Full marks when...
Status classification	25%	permanent 422 gets exactly one attempt per event; transient failures retry.
Retry & DLQ correctness	30%	bounded jittered backoff asserted with injected clock/sleeper (no test sleeps); always-down endpoint dead-letters everything.
Replay operator path	25%	replay restores delivery after recovery without duplicates.
Documentation & ops	20%	one-page delivery-semantics doc; per-endpoint circuit breaker behavior stated.

Reference approach. Queue → deliverer with classification table → dead-letter store → operator replay command; the circuit breaker protects the fleet from a slow consumer. Stretch: SSRF guardrails on endpoint registration and a per-endpoint metrics endpoint.

Capstone 11.3 — Resumable SSE Fleet Dashboard Backend [mid]

Criterion	Weight	Full marks when...
Resume correctness	30%	resume within the retention window misses zero events; outside it, a resync-required control event is emitted.
Proxy survival	25%	through a buffering proxy, events arrive within 1s of emission (the X-Accel-Buffering fix demonstrated).
Grammar robustness	25%	tests cover comments, multi-line data, unknown fields, and empty data.
Channel model	20%	multi-channel subscription with clean disconnect handling.

Reference approach. A 500-event ring buffer per channel keyed by monotonic event IDs; **Last-Event-ID** lookup decides resume vs resync; heartbeat comments keep intermediaries honest. Stretch: HTTP/2 serving, per-channel retry tuning, and an EventSource demo page.

Capstone 11.4 — Bidirectional Robot Control Channel [senior]

Criterion	Weight	Full marks when...
Exactly-once commands	30%	commands survive connection kills exactly-once under the seeded chaos harness.
Liveness discipline	20%	zombie connections reaped within two heartbeat intervals.
Graceful shutdown	20%	fleet-wide shutdown completes with zero unclean (1006) closes.
Auth & negotiation	15%	subprotocol negotiation and first-message ticket auth enforced before any command.
Design justification	15%	written note defends WebSockets over SSE+REST for this workload.

Reference approach. Ticket-issued first message authenticates; sequence numbers with server-side dedup give idempotent commands; ping/pong heartbeats drive reaping; shutdown sends close frames and drains. Stretch: permessage-deflate and a stubbed backplane proving N -node scalability.

Capstone 11.5 — Event-Contract Governance Platform [staff]

Criterion	Weight	Full marks when...
Compatibility gate	30%	diff gate passes the compatible change and fails the breaking retype, with diagnostics naming the property [Asy23].
Shared library	25%	the consumer-obligation receiver library passes the chapter's full webhook suite unmodified.
Governance coverage	25%	one-pager covers schema evolution, secret rotation, DLQ ownership, and event retirement.
Operational adoption	20%	AsyncAPI docs generated in CI; gate wired with a documented exception path.

Reference approach. Treat the AsyncAPI documents as the contract of record; the diff gate enforces consumer-driven compatibility; the shared receiver library encodes verification/idempotency obligations so no team re-implements them. Stretch: CloudEvents envelopes across all three contracts and a generated Markdown catalog.

B.13 Chapter 12: GraphQL

B.13.1 Self-Assessment Model Answers

Numbering note: the chapter uses one running 1–15 list split into Recall (1–5), Application (6–10), and Deep-Dive (11–15); numbering below matches.

1. The eight type-system kinds with catalog examples: Scalar (`SKU: String`), Object (`Product`), Interface (`Node`), Union (`SearchResult = Product | Category`), Enum (`StockStatus`), InputObject (`ProductFilter`), List (`[Product]`), NonNull (`id: ID!`) [Gra21].
2. `[Review]` (all nullable): an element error nulls that element only. `[Review!]`: an element error nulls the whole list. `[Review]!`: a null list propagates to the parent field. `[Review!]!`: any element error nulls the list *and* propagates to the parent — potentially `data: null` at the root. Nullability *is* error-containment policy.
3. Phases: (1) parse/validate — failures are request errors, no execution, no `data` key; (2) execution — per-field failures land in `errors[]` alongside partial `data`; (3) response assembly — transport-level failures (the channel the client must also handle distinctly).

4. Variables are *values* substituted after parsing; they can never alter the query document's structure, so GraphQL-injection in the SQL sense is impossible through variables. What remains injectable: the *values themselves* when a resolver concatenates them into SQL or shell — injection moved one layer down, not away.
5. APQ handshake: the client sends only the query hash; on a cache miss the server answers `PersistedQueryNotFound`; the client then resends hash + full text, and subsequent requests go hash-only. The miss is HTTP 200 because it is a GraphQL-protocol outcome inside a valid response envelope, not a transport failure.
6. Without batching: 1 query for categories plus one per category — $1 + C$ queries. With the DataLoader, the product fetches collapse into one batched read: 2 queries total, independent of C ; the instrumentation must show exactly this.
7. A query with `first: 1` on a connection whose resolver runs a full search-index scan is cheap to the estimator (charged 1) and expensive to execute. Fix: per-field base costs plus argument multipliers, not multipliers alone. Collateral damage: legitimate small-page consumers of genuinely expensive fields get throttled — the cost model must be calibrated against measured resolver latency.
8. Backward pagination runs the window query in reverse order (`last/before`) and re-reverses the page. `hasPreviousPage` requires probing for rows *before* the returned window (a limit+1 fetch in the reverse direction) — the mirror of forward pagination's `hasNextPage` probe, with cursor direction semantics that are easy to invert.
9. Add `priceCents`, mark `unitPriceCents @deprecated`, keep resolving both. This exercises continuous evolution: additive change plus machine-readable deprecation, never in-place removal — the GraphQL analogue of expand-and-contract [Gra21].
10. If `errors == [COST_DATA_UNAVAILABLE]` and `data` is present, render the card with a fallback cost block (partial data is a feature). On `GraphQLTransportError` (no `data` at all), retry the operation with backoff — the two channels have opposite handling.
11. The spec can order mutation root fields because they arrive in one document on one connection; cross-document ordering would require global coordination no transport-neutral spec can mandate. Client implication: never depend on ordering between separate operations —

sequence dependent mutations in one document or await each response.

12. *For edges:* `[ProductEdge!]!`: simpler client invariants, no null checks. *Against:* null propagation couples error containment to schema design — one bad edge nulls the whole connection, turning a single-row fault into a page-level outage. Validating telemetry: per-path field-error rates before and after tightening nullability.
13. Under a full persisted-query allow-list, GraphQL still buys the typed schema, developer tooling, deprecation workflow, and partial-field selection *within* approved documents. What it costs: the open query language’s flexibility — you have built closed RPC with a sophisticated type system, which may be exactly right for a public API under abuse pressure.
14. Price each field as base cost $\times$ argument multiplier, with base costs sourced from measured resolver latency (search-index hit vs memory hit differ by orders of magnitude). In a code-first schema the metadata lives in per-field extensions/directives consumed by the static analyzer.
15. DataLoader batching transfers to cross-service entity resolution (`_entities` batching per subgraph per request), but three things break or degrade: cross-request cache invalidation is now distributed; partial subgraph failure mid-batch must degrade per-field; and the batching window adds latency to the critical path — the N+1 became network fan-out, which is better but not free [Kle17].

B.13.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Capstone 1–5”; they are cited here as C12.1–C12.5.

Capstone 12.1 — Catalog Schema from Scratch [junior]

Criterion	Weight	Full marks when...
Schema completeness	30%	types, enum, input, union mutation result, and a cursor connection all present; generated SDL shows them.
Pagination correctness	25%	full cursor walk returns every product exactly once.
Error discipline	25%	forged cursor → field error with machine-readable <code>extensions.code</code> ; out-of-range rating → typed union payload with <code>errors</code> absent.
Repository quality	20%	own seeded repository; deterministic offline tests.

Reference approach. Re-implement schema-first from the SDL outward; keep mutation results as union types so domain errors are typed payloads, not exceptions. Stretch: `@deprecated` verified in introspected SDL and backward pagination.

Capstone 12.2 — N+1 Hunter [mid]

Criterion	Weight	Full marks when...
Measured elimination	35%	store-query counts independent of page size, verified at two sizes.
Dedup correctness	25%	a test proves duplicate keys collapse into one batch read.
Loader lifecycle	25%	regression test fails with a module-level (stale-cache) loader and passes with per-request loaders.
Reporting	15%	before/after query counts tabulated per operation.

Reference approach. Instrument the store with a counting wrapper; find every resolver doing per-item fetches; introduce request-scoped DataLoaders. Stretch: per-tick batch-size metrics in a response `extensions` block.

Capstone 12.3 — Cost-Control Gateway [mid]

Criterion	Weight	Full marks when...
Gate accuracy	30%	every corpus operation accepted/rejected exactly as labeled.
Cost model soundness	25%	complexity monotonically increasing in first (property-tested).
Adversarial robustness	25%	fragment cycles rejected with a distinct code — never hang, never crash.
Enforcement	20%	per-operation timeout proven to fire.

Reference approach. Schema-agnostic AST walker computing depth and argument-weighted complexity before execution; reject over budget with a machine-readable code. Stretch: per-field cost weights from measured resolver latency and complexity reported in response extensions.

Capstone 12.4 — APQ Production Gateway [senior]

Criterion	Weight	Full marks when...
Allow-list strictness	30%	strict mode rejects unregistered documents with a distinct code and zero resolver executions (proven).
Handshake fidelity	25%	client completes miss → register → hash-only, sending full text exactly once.
Cost-based limiting	25%	complexity-based rate limiting admits small queries while rejecting god-query bursts at equal request count.
Introspection policy	20%	introspection disabled/governed per environment with tests.

Reference approach. APQ cache + persisted-document registry ahead of validation; complexity scoring feeds the rate limiter (Chapter 15). Stretch: serve registered reads over GET for CDN cacheability with a documented cache-key design.

Capstone 12.5 — Federated Robotics Graph [staff]

Criterion	Weight	Full marks when...
Cross-subgraph correctness	30%	one operation spanning catalog + reputation returns correctly stitched data.
Batched resolution	25%	cross-service entity resolution batched, counts asserted.
Failure isolation	25%	subgraph outage degrades only its fields (fault-injection test), never the whole response.
Scope honesty & ops	20%	written scope note lists unimplemented federation features; gateway has metrics and a documented roll-back.

Reference approach. Two subgraph services share the `Product` entity; the hand-rolled gateway plans queries, fans out, batches `_entities` calls, and merges with per-field error containment. Stretch: a subscription carried through the gateway and per-subgraph cost budgets.

B.14 Chapter 13: gRPC & High-Performance RPC

B.14.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–10 list; numbering below matches.

1. The 5-byte prefix `00 00 00 00 0A` decodes as compression flag 0 (uncompressed) and message length 10 octets; the ten protobuf bytes are the message on stream 3. If the stream ends after six payload bytes, the message is truncated — the receiver must fail the RPC (INTERNAL / stream error), never deliver a partial message [gRP24].
2. 300 bytes = `0x12C`, so the prefix is `00 00 00 01 2C`. The 32-bit length field can express up to $2^{32} - 1$ bytes (≈ 4 GiB), but `grpc-python` enforces `grpc.max_receive_message_length` at 4 MiB by default — the wire format is not the binding constraint.
3. (a) Firmware-update progress push: **server-streaming** — one request, many progress messages. (b) Uploading a day of buffered sensor logs: **client-streaming** — many chunks, one summary response. (c) Trader-style command/ack console: **bidirectional** — independent streams

both ways. (d) One robot's current pose: **unary** — request/response, simplest possible surface.

4. The downstream server sees `grpc-timeout` of $500 - 40 = 460$ ms, encoded "460m". If the proxy *adds* 50 ms, the downstream keeps working after the original client has already abandoned the call — wasted work and misleading success signals; deadlines must only shrink as they propagate, never grow [gRP24].
5. `UploadTelemetry` mutates server state: retrying `DEADLINE_EXCEEDED` risks applying the upload twice (the first attempt may have succeeded after the deadline). `GetRobotStatus` is a read with no side effects, so retrying it is harmless — retry safety is a property of the method's semantics, not the status code (Chapter 11).
6. Two-phase migration: (1) add `battery_millivolts` (new tag) and dual-publish both fields while consumers migrate; (2) after usage hits zero, mark tag 3 and the old name `reserved` and stop publishing. Never retype tag 3 in place — wire-type/type changes are the incompatibility class of Chapter 5 [Goo24].
7. The interceptor reads the token from metadata, checks a per-token bucket, and on exhaustion aborts with `RESOURCE_EXHAUSTED` plus a `RetryInfo`-style detail packed in a `-bin` trailer (binary metadata). Streaming handlers differ from unary: the limiter must decide whether to charge per call or per message, and must not break long-lived streams mid-flight — four handler shapes, four charging decisions.
8. Hypothesis: the server's connection-recycling option (e.g. `max_connection_age` at 300 s) forces reconnects every five minutes, and reconnect storms surface as `UNAVAILABLE` spikes. Confirm by changing `grpc.max_connection_age_ms` to a different value — the spike interval follows the setting, proving causation [gRP24].
9. `grpc-web`: full contract fidelity, browser-safe, but needs a proxy and has limited streaming. JSON transcoding: annotation-driven, cheap ops, weakest for streams. Bespoke `WebSocket` shim: maximal flexibility, maximal contract drift. For a browser dashboard over `StreamTelemetry`, pick `grpc-web` when streaming fidelity matters, transcoding when the surface is mostly unary — justify on contract fidelity and ops cost, not fashion.
10. `grpc-status` lives in trailers because the definitive outcome of a streaming RPC is knowable only after the last message; a *trailers-only* re-

sponse is HEADERS carrying the status with no DATA — an error before any message. Trailers let intermediaries terminate failures without parsing bodies; conversely gRPC traffic is opaque to HTTP caches, and browsers need translation (grpc-web) because they cannot expose trailers mid-fetch [Tho22; gRP24].

B.14.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Capstone 1–5”; they are cited here as C13.1–C13.5.

Capstone 13.1 — Unary Status Service with Deadline Discipline [junior]

Criterion	Weight	Full marks when...
Path coverage	30%	pytest covers happy path, unknown robot, and deadline-exceeded cases.
Deadline discipline	30%	no call site without <code>timeout=</code> ; no handler without a remaining-budget check.
Service shape	20%	grpcurl/descriptor dump demonstrates the service contract.
Error mapping	20%	domain errors map to correct status codes, asserted via <code>.code()</code> .

Reference approach. One proto, one servicer, one test module; handlers check `context.time_remaining()` before work. Stretch: keepalive configuration with channel-state logging and a latency-measuring client interceptor.

Capstone 13.2 — Server-Streaming Telemetry Monitor [mid]

Criterion	Weight	Full marks when...
Cancellation correctness	30%	after <code>cancel()</code> the server handler exits within one period (proven).
Flow-control bounds	25%	server never exceeds <code>max_frames</code> ; client drop counter observable.
Deadline enforcement	25%	stream deadline enforced and tested.
CLI quality	20%	bounded rendering; clean shutdown; readable output.

Reference approach. Server loop checks cancellation each period; client renders into a bounded buffer with an explicit drop counter. Stretch: heartbeat frames and per-frame stalled-stream detection.

Capstone 13.3 — Bulk Ingestion with Rich Validation Errors [mid]

Criterion	Weight	Full marks when...
Error richness	30%	mid-stream abort yields INVALID_ARGUMENT with decoded per-field violations from the <code>-bin</code> trailer and an end-to-end correlation ID.
Resume correctness	25%	fix-and-resume accepts exactly the valid 95%.
Assertion discipline	25%	all errors asserted via <code>.code()</code> , never message strings.
Interceptor design	20%	error mapping lives in the interceptor, not the servicer.

Reference approach. Client streams rows; the server validates per message and aborts with packed details; the client decodes the trailer, fixes the rows, and resumes from the acknowledged offset. Stretch: port to `google.rpc.Status` and show wire equivalence.

Capstone 13.4 — Bidirectional Control Plane with Interceptor Stack [senior]

Criterion	Weight	Full marks when...
Exactly-once effects	30%	500-command run has exactly-once effects despite forced retries, with ack order preserved (sequence-number idempotency).
Interceptor separation	25%	auth/logging/metrics interceptors emit per-cardinality counters without touching business code; bad tokens fail before servicer code.
Deadline propagation	25%	deadline test shows the downstream hop observing a smaller remaining budget.
Flow control	20%	windowed flow control prevents unbounded buffering under a slow consumer.

Reference approach. Bidi stream with per-command sequence numbers and a server-side dedup table; interceptors cross-cut auth, logging, metrics on both peers. Stretch: hedged unary reads with a measured p99 delta vs the retry baseline under deterministic latency injection (Chapter 16).

Capstone 13.5 — Edge Gateway: Transcoding + Load-Balancing Postmortem [staff]

Criterion	Weight	Full marks when...
Reproduction	25%	pinned-phase skew $\geq 10\times$ reproduced deterministically.
Fix efficacy	30%	each of the three fixes (<code>max_connection_age</code> , <code>round_robin</code> , <code>least-loaded</code>) reduces the max/median RPC ratio below $1.5\times$.
Transcoding correctness	20%	JSON edge returns correct HTTP codes for <code>NOT_FOUND/DEADLINE_EXCEEDED/INVALID_ARGUMENT</code> with details intact.
Postmortem quality	25%	identifies connections-vs-RPCs as root cause, names the per-pod SLI, and includes detection and rollback notes; <code>pytest -q</code> green via one PowerShell driver.

Reference approach. Recreate the war story: one L4-balanced connection per client pins load; fix at the server (connection aging), the client (`round_robin`), and the policy layer (`least-loaded`); write the incident report with the measured evidence. Stretch: DNS-based resolver config and `grpc.health.v1` gating the shim's routing on `SERVING`.

B.15 Chapter 14: Rate Limiting & Throttling

B.15.1 Self-Assessment Model Answers

- E14.1.** The boundary test shows the sliding-window counter admits fewer than $2N$ requests across a window edge (unlike fixed window, which admits exactly $2N$ in an infinitesimal span). It still over-admits relative to the exact sliding log when the previous window's traffic clustered at its end and the query happens mid-window: the linear weight then under-credits the true count. Bounding proposition: the estimate's error is at most the previous window's count, so admissions stay below $2N$.
- E14.2.** With correct parameterization (emission interval $T = 1/r$, tolerance $\tau = B/r$), no divergent (r, B) pair exists — the strengthened proof is by induction over the event stream: bucket level and $(\tau - (\text{TAT} -$

$t)/T$ are invariantly equal, including across multi-second idle gaps (both refill fully). The 5,000-event seeded sweep is the empirical confirmation of the equivalence theorem.

- E14.3.** A sliding log has no single reset instant; the honest, monotonic choice is to report the seconds until the *oldest request still blocking capacity* expires — i.e. the delay until at least one more request would be admitted. This decreases monotonically and never promises a cliff-edge reset that does not exist [Pol24].
- E14.4.** Three concrete failures from swapping the injected `Clock` for `time.monotonic`: (1) window-rollover unit tests become real-time sleeps (slow, flaky); (2) seeded simulation/replay harnesses (AIMD tuning, incident replay) become impossible — they need virtual time; (3) boundary-condition tests (burst exactly at a window edge) become nondeterministic, so regressions pass or fail by timing luck.
- E14.5.** Instrumented check-then-increment admits overshoot that grows with thread count: the TOCTOU window between read and write widens under contention, so 50-thread storms show systematically more over-admissions than 5-thread storms. A Lua script executes read-check-increment atomically *inside* Redis, so the entire race class is eliminated by construction — not probabilistically reduced.
- E14.6.** Keys: `quota:{tenant}:YYYY-MM` (INCR with month-end EXPIRE) and `rate:{tenant}` (token bucket). One Lua script checking both atomically is preferred; two scripts invite a quota/rate ordering hazard. `GET /v1/usage` returns `{quota_limit, quota_used, quota_remaining, resets_at, rate_limit, rate_remaining}`. A client at 99/100 with an empty per-second bucket gets 429 with `Retry-After` from the rate layer while the response body/headers still show the quota layer's remaining 1 — the two layers must be distinguishable.
- E14.7.** From $L_0 = 100$ with $a=5$ rps per 1 s evaluation: first contact with 300 rps capacity at $(300 - 100)/5 = 40$ s. Steady state is a sawtooth: reach 300, multiplicative cut to $\beta \cdot 300 = 150$, climb back — amplitude 150 rps peak-to-trough. $\beta < 1$ is safety-critical because multiplicative decrease sheds load *proportionally* at any level (fast relief when far over capacity), while the additive term only probes slowly upward.
- E14.8.** A good capture documents: presence/absence of `Retry-After`, the `RateLimit-Limit/-Remaining/-Reset` triple and `RateLimit-Policy`, and critically whether `Reset` is delay-seconds or epoch-seconds (a historical interop split) [Pol24]. Contract comparison: a provider

better than the chapter’s middleware publishes both header families plus a problem body; worse is 429 with no headers at all — clients must guess the retry instant.

B.15.2 Capstone Grading Rubrics

Capstone 14.1 — Prove the Algorithms [junior]

Criterion	Weight	Full marks when...
Reimplementation fidelity	35%	all 14 ported tests pass on the from-definition reimplementation of the five algorithms.
Bug-detection mapping	30%	each seeded bug is caught by at least one existing test, with the bug→test mapping named.
Extension proof	20%	the new non-integer-window ($W=1.5$) burst test proves 20 admissions in a sub-window interval.
Determinism	15%	seeded, offline, no wall-clock assertions.

Reference approach. Implement each algorithm from its mathematical definition against an injected clock; port the proof sheet first, then break the implementations deliberately. Stretch: property-test the sliding log’s exactness invariant over random seeded timelines.

Capstone 14.2 — Header-Complete Middleware [mid]

Criterion	Weight	Full marks when...
Contract completeness	30%	429 bodies validate against the declared RFC 9457 schema; the full header family emitted [Not23; Pol24].
Degradation honesty	30%	fail-open episodes emit <code>X-NR-Limiter-Degraded</code> and no <code>RateLimit-*</code> fields; fail-closed returns <code>503 + Retry-After</code> .
Drop-in compatibility	25%	all existing middleware tests pass unmodified against the Redis wiring.
Determinism	15%	suite deterministic and offline.

Reference approach. Package the ASGI middleware with a Redis-backed limiter behind the same interface; make degradation modes explicit in the response contract. Stretch: per-plan policies selected by API-key tier.

Capstone 14.3 — Cell Architecture Simulator [mid]

Criterion	Weight	Full marks when...
Topology bounds	35%	global topology never exceeds the limit; no-sync overshoot matches the $R\times$ prediction; sync topology respects the published bound at every interval Δ .
Reproducibility	25%	all results regenerate from one seed.
Modeling fidelity	25%	the simulator's event model matches the chapter's cell semantics (local counts + sync).
Report quality	15%	over-admission quantified per topology with a recommendation.

Reference approach. Discrete-event simulation over cells with local counters and periodic gossip/sync; sweep topology and Δ ; tabulate overshoot. Stretch: inject a cell failure and show consistent-hash remapping confines counter loss to $1/C$ of keys.

Capstone 14.4 — Adaptive Limits with AIMD [senior]

Criterion	Weight	Full marks when...
Control quality	30%	tuned controller never exceeds the moving capacity signal by $>10\%$ for more than one evaluation interval; the contractual floor is never violated.
Tuning evidence	30%	(a, β) grid sweep and the identified “knee” reproducible from seed.
Analysis	25%	tuning memo quantifies the multiplicative-decrease asymmetry argument.
Determinism	15%	no wall-clock dependence; capacity signal replayed from fixture.

Reference approach. AIMD controller per tenant evaluated on a fixed cadence against a replayed capacity signal; sweep parameters; defend the chosen (a, β) with the oscillation data. Stretch: per-tenant fairness dividing recovered headroom proportional to recent demand.

Capstone 14.5 — Rate-Limiting Architecture for Northwind at 100k rps [staff]

Criterion	Weight	Full marks when...
Coverage & derivation	25%	every endpoint has a tier, topology, failure policy, and a numerically derived limit — no defaults.
Bounds & monitoring	20%	over-admission bounds stated and monitored per approximate tier.
Runbooks	25%	limiter-cell outage, abusive-tenant drill, and AIMD brownout runbooks each include detection and rollback steps.
Cost realism	15%	cost model shows limiter infrastructure <2% of serving cost.
Red-team honesty	15%	≥ 2 residual weaknesses named without spin.

Reference approach. Tier the surface (strict/exact vs approximate), assign topologies per tier (centralized, cell, AIMD), derive limits from capacity data, and write the operational runbooks as part of the design — not after it. Stretch: GraphQL cost-scoring and gRPC interceptor integration so all three protocol surfaces share one budget.

B.16 Chapter 15: Microservices & Resilience

B.16.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–8 list; numbering below matches.

1. Budgets must shrink down the chain with headroom for each hop’s own work: ledger ≤ 300 ms (its measured p99.9), payments ≤ 600 ms, orders $\leq 1,000$ ms, edge $\leq 1,300$ – $1,500$ ms, satisfying the rule that each hop’s timeout is strictly smaller than the caller’s remaining budget. Validate by measuring per-hop p99/p99.9 under load and by deadline-propagation tests showing downstream hops abandon work the client already gave up on.
2. Unbudgeted, each of the 4 hops retries twice: attempts multiply as $3^4 = 81$ downstream requests per original request at the deepest hop (total $3 + 9 + 27 + 81 = 120$). With a one-token retry budget at the two middle hops: $3 \times 1 \times 1 \times 3 = 9$. The budget at the *earliest* hop matters most, because amplification compounds — one retry near the client re-executes everything downstream.

3. Defensible set: error-rate threshold $\theta = 50\%$ over a rolling window $W = 30\text{ s}$; minimum call volume $N = 100$ (about 0.05 s of traffic at 2,000 rps — the window is always statistically meaningful); open duration $d = 30\text{ s}$; half-open admits p probes paced to stay under the 20 rps recovery allowance (e.g. 1 probe/s); close after $s = 5$ consecutive successes. Every number traces to the dependency’s stated success rate and recovery budget [Nyg18].
4. Add a slow-call dimension: a call whose duration exceeds the threshold counts as a failure in the window even on success. The manual-clock test injects five slow successes and asserts the window error rate reaches θ and the breaker opens — slow failures are exactly the ones that exhaust pools while looking “healthy”.
5. Hedging after the measured p95 fires the second call on the q tail; the measured total call rate should converge to $1 + q$ within derived tolerance as the injected tail probability q varies. Hedging is safe only when the operation is idempotent (or key-deduplicated) *and* the dependency has capacity headroom for the extra q fraction of load.
6. Per-aggregate ordering: outbox rows carry (`aggregate_id`, `seq`); the relay publishes in seq order per aggregate; the consumer tracks the last applied seq per aggregate and buffers/refetches on a gap. Crash-mid-batch replays then cannot interleave with new events — the consumer enforces monotonic application per aggregate regardless of publisher timing.
7. In the choreographed version, each service subscribes to domain events and emits its own (`inventory.reserved` $\rightarrow$ payments consumes, etc.); the compensation graph no longer lives in one orchestrator — it is *distributed across the subscriptions* of each participant. The catching test: fail one step and assert the compensating event is consumed and state reverted; a missing compensation subscription fails the test loudly.
8. mTLS authenticates services, not outcomes; it does nothing for: (1) object-level authorization (BOLA) — the chapter’s policy/authorization pattern; (2) semantic abuse over a legitimate identity (injection, mass assignment) — input validation and contract enforcement; (3) business-flow abuse from a credentialed caller — rate limiting and anomaly detection (Chapter 15).

B.16.2 Capstone Grading Rubrics

Capstone 15.1 — Breaker Under a Microscope [junior]

Criterion	Weight	Full marks when...
Reimplementation fidelity	35%	all existing breaker tests pass on the from-definition reimplementation.
Bug hunting	30%	each seeded bug (no min-call floor, no window eviction, unlimited probes) is caught by a named existing test.
New-test quality	25%	the probe-threshold test ($s=3$, success-success-failure) passes on the correct breaker and fails against the unlimited-probe bug.
Determinism	10%	manual clock only; no sleeps.

Reference approach. Implement the closed/open/half-open state machine from the formal definition over an injected clock; run the existing suite; then mutate. Stretch: the slow-call failure dimension with a manual-clock proof.

Capstone 15.2 — Latency Chaos and the Price of Hedging [mid]

Criterion	Weight	Full marks when...
Benefit evidence	30%	hedged p99 materially below unhedged p99 on the same seed under bimodal latency faults.
Cost accounting	30%	measured hedge rate equals $1 + q$ within the derived tolerance — the price is quantified, not asserted.
Breaker interaction	25%	hedging is disabled while the breaker is open, proven by test.
Determinism	15%	fully seeded/offline; no wall-clock assertions.

Reference approach. Chaos stub with bimodal latency; hedge timer at measured p95; count issued vs completed calls. Stretch: loser-request cancellation with measured downstream savings, and hedges spending retry-budget tokens.

Capstone 15.3 — Production Outbox: Retry, DLQ, Ordering [mid]

Criterion	Weight	Full marks when...
Retry discipline	25%	publish-failure retries respect the budget cap (counting-publisher proof).
Dead-lettering	25%	DLQ after exactly k attempts with idempotent requeue.
Ordering guarantee	30%	per-aggregate ordering holds under crash/replay injection without hurting cross-aggregate throughput.
Regression safety	20%	the original outbox suite still passes unmodified.

Reference approach. Harden the teaching relay: budget wrapper, dead-letter table, (`aggregate_id`, `seq`) ordering with consumer-side gap handling. Stretch: lag metrics (unpublished count, oldest unpublished age) with alert thresholds.

Capstone 15.4 — Measure the Retry-Amplification Theorem [senior]

Criterion	Weight	Full marks when...
Unbudgeted verification	30%	per-hop attempt counts match a^k exactly (3, 9, 27; total 39).
Budgeted verification	30%	closed-form prediction for the budgeted case matches measurement.
Gap analysis	20%	a written paragraph explains any residual theory-vs-measurement gap.
Methodology	20%	offline, seeded, counting stubs; no wall-clock assertions.

Reference approach. Four-hop chain of counting stubs, each retrying per its budget token; count attempts per hop and compare against $a(a^n - 1)/(a - 1)$. Stretch: swap the stubs for the real ASGI chain and show the theorem survives real HTTP machinery.

Capstone 15.5 — Resilience Architecture for Fleet Telemetry [staff]

Criterion	Weight	Full marks when...
Parameter justification	25%	every edge has SLO-derived parameters — zero defaults anywhere.
Deadline correctness	20%	deadline budgets satisfy the formal rule on every path (ingestion and dashboard read).
Failure-mode coverage	25%	table has signal + countermeasure per scenario, including at least one <i>degradation</i> (not just failure) case.
Operational program	30%	chaos-drill program includes abort criteria and rollback; mesh boundary list correctly assigns ≥ 3 concerns to the application.

Reference approach. Walk every edge of ingestion $\rightarrow$ normalizer $\rightarrow$ enrichment $\rightarrow$ storage (plus the read path); derive timeouts, retry budgets, and breaker parameters from the SLO arithmetic; tabulate failure modes with signals and countermeasures [Bey+16; New21]. Stretch: cost the design (retry/hedge load, engineering hours, drill cost) against an incident cost model.

B.17 Chapter 16: API Security Engineering

B.17.1 Self-Assessment Model Answers

- E16.1.** The fetcher must re-validate the resolved IP of *every* redirect hop, not just the first URL: the two-302 chain to an internal host must be blocked at the final hop. With `max_redirects=1` the same chain dies one hop earlier. Operationally, the first failure proves policy correctness end-to-end; the second proves defense in depth and leaves a clean audit signal of where the chain was cut [OWA23].
- E16.2.** Policy rows for `support`: read `notes` allowed, read `battery_pct` denied, all writes denied. Tests: allowed read, denied field, denied write — plus the union case: a principal that is both `support` and `owner` gets the union of permissions (and nothing beyond it).
- E16.3.** Moving claims mapping ahead of signature verification means attacker bytes become trusted claims before authenticity is established. The catching test mints an unsigned `alg=HS256` token with an attacker-chosen `sub` and asserts rejection — verification order is a security invariant, not a style choice [Jon15].

- E16.4.** `app.routes` on the vulnerable build lists `debug/admin` routes absent from its own OpenAPI document — undocumented attack surface. This instantiates OWASP API9 (Improper Inventory Management): you cannot protect what your inventory does not know about [OWA23].
- E16.5.** Capture robot 1001’s JSON per role on the hardened build and diff field sets against the read policy — exact match required. On the vulnerable build the same diff counts the excess fields leaked (property-level authorization failure, OWASP API3).

Deep-Dive Questions

- D16.1.** `aud` enforcement is mandatory *now* because the moment a second API exists (18 months later), tokens minted for API A replay cleanly against API B — audience confusion. Retrofitting `aud` then is a breaking change for every client; enforcing it from day one is free.
- D16.2.** Dependent pair: a DTO allowlist and Pydantic `extra=forbid` — both fail identically when a new sensitive field is added to the shared model (one omission defeats both; ceremony without independence). Independent pair: DTO allowlist *plus* database-layer column permissions — different mechanisms, different failure modes.
- D16.3.** Connecting by pinned IP breaks SNI and hostname verification: the certificate’s SANs name hosts, not IPs, and a shared IP may serve the wrong vhost. Production SSRF guards resolve the tension by resolving → validating the IP → connecting with the original `Host/SNI` pinned, or by pushing egress through an allowlisting proxy that sees the hostname.
- D16.4.** Across two hops, `sub` must remain the original user while `act` accumulates each acting party (delegation chain). If hop two impersonates instead of delegates, the audit trail loses the agent — logs can no longer distinguish what the service did on the user’s behalf from what the user did.
- D16.5.** Invisible to an edge WAF: API1 (BOLA), API3 (property-level authorization), API6 (unrestricted sensitive business flows), API9 (inventory) — everything requiring object-ownership or business context, which no payload-inspecting appliance can possess [OWA23].
- D16.6.** A per-process limiter at twelve replicas admits twelve times the intended limit — the limit silently scales with the fleet. The fix is Chapter 15’s distributed architecture: a shared store (Redis cell/token

bucket) so the limit is a property of the service, not of one process.

B.17.2 Capstone Grading Rubrics

Capstone 16.1 — BOLA Hunter [junior]

Criterion	Weight	Full marks when...
Exploit demonstration	30%	the exploit script retrieves a foreign note on the vulnerable build.
Fix completeness	30%	the hardened build returns 403 for <i>every</i> cross-owner access, proven by test.
Bidirectional tests	25%	pytest asserts both exploit-succeeds (vulnerable) and fix-holds (hardened), offline.
Conceptual understanding	15%	README explains why unguessable IDs alone are not the control.

Reference approach. Ownership column on notes; exploit walks sequential IDs; fix enforces `owner_id == caller` in the query itself. Stretch: ETag conditional writes, and the UUIDv7 demonstration that enumeration resistance is not authorization.

Capstone 16.2 — Scope Forge & Token Gauntlet [mid]

Criterion	Weight	Full marks when...
Forgery battery	35%	all seven forgery tests (expired, wrong <code>aud</code> , wrong <code>iss</code> , <code>alg=none</code> , unknown <code>kid</code> , tampered payload, missing scope) yield 401/403 — never 200.
Verification order	25%	provably no claim is read before signature verification.
Scope design	25%	the scope document justifies each scope's granularity.
Test hygiene	15%	deterministic offline token minting for tests.

Reference approach. Fixed pipeline: parse header → resolve key by `kid` → verify signature → validate `exp/iss/aud` → map scopes; the gauntlet is a parameterized test. Stretch: token exchange to a second service with narrowed scope/audience and clock-skew boundary tests [Jon15].

Capstone 16.3 — SSRF Bunker [mid]

Criterion	Weight	Full marks when...
Exploit/fix symmetry	30%	all naive exploits (metadata IP, internal host, redirect chain) succeed pre-fix and are blocked post-fix in one pytest run.
Layer coverage	25%	DNS-rebinding and redirect re-validation tests included — all seven layers exercised.
Documentation	25%	README layer-by-layer blame table maps each exploit to its catching control.
Determinism	20%	simulated adversary network; fully offline.

Reference approach. Apply the seven-layer gauntlet: scheme allowlist, host allowlist, DNS resolution with IP validation, pinned connect, redirect re-validation, response limits, egress policy. Stretch: IPv6-mapped IPv4 bypass attempts and an independent egress-rule transport wrapper.

Capstone 16.4 — Threat-Model Pipeline [senior]

Criterion	Weight	Full marks when...
Gate enforcement	30%	<code>-check</code> exits non-zero on undocumented/unmodeled endpoints and zero when worksheets are current.
Generator heuristics	25%	≥ 3 new STRIDE heuristics, each unit-tested.
Abuse-case tests	25%	two abuse cases expressed as passing security tests.
CI integration	20%	gate wired into CI with documented exception/expiry handling.

Reference approach. Treat threat models as build artifacts: a worksheet per endpoint, generated skeletons from the route table, staleness detection by hashing the route surface. Stretch: attack-tree DOT output and risk scoring by flag count and data sensitivity.

Capstone 16.5 — Full Red Team / Blue Team Exercise [staff]

Criterion	Weight	Full marks when...
Executable findings	25%	every red finding exists as an executable contract test — no prose-only findings.
Cross-service exploit	25%	audience-confusion exploit demonstrated <i>and</i> closed, with regression test.
Functional safety	20%	the functional suite stays green after all hardening.
Retro quality	30%	retro maps findings → OWASP items → detection signals → permanent controls; roll-back plan and residual risk stated.

Reference approach. Two-sided exercise on the multi-service topology: red exploits seeded flaws, blue fixes behind contract tests, and the retro institutionalizes every finding as a permanent control or alert [OWA23]. Stretch: mTLS via a local CA showing the token-confusion exploit’s remaining path, and time-to-detect measurement on a deliberate regression.

B.18 Chapter 17: Deployment & Infrastructure

B.18.1 Self-Assessment Model Answers

Numbering note: the chapter numbers its exercises “Exercise 7.1–7.8” (a template artifact); this appendix preserves that numbering. The chapter’s four deep-dive questions are unnumbered prose; they are answered here as D17.1–D17.4 in order of appearance.

Exercise 7.1 With source COPY before dependency install, a one-line source edit invalidates every later layer — the 200 MB dependency layer rebuilds (and re-pushes) on *every commit*. Correct order: copy manifests, install dependencies, then copy source; a source edit then rebuilds only the final thin layer. The cost quantified: 200 MB × every build vs a few KB.

Exercise 7.2 With `timeout_graceful_shutdown=1`, in-flight requests are severed after one second: the verdict line counting severed/reset requests flips from zero to nonzero. The amputated act of the four-act shutdown is the *drain* — waiting for accepted work to finish.

Exercise 7.3 Boot 75s with `periodSeconds: 5` needs $75/5 = 15$ failures tolerated; with 25% headroom, `failureThreshold` ≥ 19 (say 20).

Raising the *liveness* failureThreshold instead is wrong: liveness exists to detect deadlocks, and a tolerant liveness probe will happily restart slow-booting pods mid-boot — the CrashLoopBackOff the startup probe was invented to prevent.

Exercise 7.4 With latency p95 factor 1.20 the regressed fixture is rolled back at batch 1 by the latency gate. Setting $\delta = 0.02$ widens the intolerance band: fewer false rollbacks, but detection needs more evidence, so detection latency grows. Choosing δ is choosing how much error budget you are willing to burn *while deciding* — δ must stay well below the budget a full rollout would spend.

Exercise 7.5 With an 18-second boot (matching A), Scenario B survives; at 35 seconds it crash-loops. The exact boundary is derivable from probe parameters alone: boot time must not exceed `initialDelaySeconds + periodSeconds × failureThreshold` of the startup probe — beyond that, the kubelet kills the container before it finishes booting, forever.

Exercise 7.6 `preStop sleep` $\geq$ LB target-removal convergence: choose 15s for a 12s bound. Uvicorn graceful timeout $\geq$ p99.9 request duration: choose 10–12s for a 9s bound. `terminationGracePeriodSeconds` $\geq$ `preStop` + drain + margin: choose 30–35s. Each number is justified by the mechanism it protects — endpoint propagation, in-flight drain, and the kubelet’s hard kill.

Exercise 7.7 Without the PDB, a routine node upgrade evicts *all* replicas simultaneously (nothing enforces `minAvailable`); the drain completes before replacements schedule; the HPA sees no CPU signal to react to; the API is fully offline until new pods schedule, pull, and pass readiness — an outage caused entirely by a routine, healthy operation.

Exercise 7.8 Method: compute the expected per-request log-likelihood-ratio increment under each hypothesis (baseline 0.4% vs alternative 1.4% with $\delta = 1$ pt), then $E[N_{\text{rollback}}] \approx \log((1-\beta)/\alpha) / E[\text{LLR} \mid H_1]$ and $E[N_{\text{promote}}] \approx \log((1-\alpha)/\beta) / E[\text{LLR} \mid H_0]$. Verify both against the simulator — the point is that decision latency is a *derived* quantity, not a guess.

Deep-Dive Questions

D17.1. Digest pinning guarantees every node pulls identical bits, but pull time is on the scale-out critical path: during a surge, heterogeneous pull

times stagger readiness across the node pool exactly when capacity is most needed. Mitigations: pre-pulled images on the node image, registry mirrors near the pool, and surge headroom that does not depend on cold pulls.

- D17.2.** Ambient mesh removes sidecars: pod resource requests shrink and lose a container-start ordering concern; L4 moves to ztunnel, L7 policy to waypoints (changing where mTLS terminates and where authorization policy lives); drain and lifecycle hooks no longer coordinate with an injected proxy; observability and HPA metrics now come from mesh components outside the pod.
- D17.3.** In PromQL: compute canary-vs-baseline error-rate (or latency) ratios over rolling windows and express the gate as an alert expression with `for: dwell` — the batch-file SPRT becomes a recording rule plus an abort condition the rollout controller watches.
- D17.4.** PodLifecycleSleepAction’s graduation history ($\alpha \rightarrow \beta \rightarrow \text{GA}$ over multiple releases) is the cautionary tale: α features can change shape or vanish between releases, so lifecycle-correctness mechanisms (drain!) must never depend on them — wait for GA, or implement the sleep portably.

B.18.2 Capstone Grading Rubrics

Capstone 17.1 — Harden the Image [junior]

Criterion	Weight	Full marks when...
Size & layering	25%	final image $\leq 20\%$ of the naive size (or a documented reason); one-line source edit rebuilds only the source layer (build-log evidence).
Runtime hardening	30%	<code>docker inspect</code> shows a non-root user and exec-form entrypoint.
Hygiene	25%	no <code>.env</code> , <code>.git</code> , or test artifacts in any layer.
Reproducibility	20%	pinned bases/dependencies; clean-build instructions work offline.

Reference approach. Multi-stage build: deps from lockfile first, source last, runtime stage minimal, non-root user, exec entrypoint [Wig17]. Stretch: slim vs distroless variants with the traded operational capability (shell/debuggability) documented.

Capstone 17.2 — Prove the Drain Under Load [mid]

Criterion	Weight	Full marks when...
Verdict accounting	30%	three verdict classes reported per run reconcile exactly with attempted requests.
Contrast demonstration	30%	zero severed requests at production timeout, nonzero at timeout 0 — both explained mechanistically.
CI gating	25%	pytest integration exits non-zero on any severed request.
Reproducibility	15%	seeded, offline, stable across runs.

Reference approach. Drive a concurrent traffic mix through the shutdown demo; classify every request outcome; run the drill at both timeout extremes. Stretch: a preStop-equivalent delay showing refused-after-signal drop to zero.

Capstone 17.3 — Manifest Review Board [mid]

Criterion	Weight	Full marks when...
Fault detection	35%	the linter catches all seven injected faults with rule-specific messages; clean set passes.
Rule traceability	25%	every rule cites its chapter section.
Review realism	25%	minutes give a failure story per fault (blast radius, mechanism) — not rule restatements.
Engineering	15%	YAML parsing, deterministic output, documented exit codes.

Reference approach. Encode the chapter's rules (probes, PDB, resources, non-root, image pinning, ...) as linter rules over parsed manifests; run the formal review; record minutes. Stretch: a second-reviewer mode diffing two manifest revisions for safety-relevant changes only.

Capstone 17.4 — Gate Tuner [senior]

Criterion	Weight	Full marks when...
Sweep evidence	30%	the $\delta/\alpha/\beta$ sweep table shows the trade-off: tighter intolerance costs more requests per decision.
Promotion safety	25%	the 0.5δ canary is promoted (never rolled back) in every sweep cell.
Theory-measurement agreement	25%	measured detection latency matches the LLR-drift derivation within one batch.
Policy artifact	20%	the policy page names δ , α , β , batch size, and rollback action with cited evidence.

Reference approach. Parameterize the SPRT gate; sweep; derive expected drift per request; reconcile measurement with derivation. Stretch: seasonal-baseline mode and an analysis of seasonality’s effect on the SPRT assumptions.

Capstone 17.5 — Zero-Downtime Reference Architecture [staff]

Criterion	Weight	Full marks when...
ADR quality	25%	every ADR names rejected alternatives and reversal cost.
Compliance decisions	20%	the payments path has an explicit TLS-termination decision with compliance rationale — no plaintext last hop.
Gate calibration	25%	per-tier gate parameters come with simulator evidence they detect the stated regression sizes.
Operational completeness	30%	WebSocket drain budget derived from measured connection lifetimes; C17.3’s linter finds zero violations in the template; rollback semantics defined per tier.

Reference approach. ADR set covering image supply, rollout, drain, probes, and traffic policy across the 12-service, two-region platform; every number traced to measurement or derivation [Wig17; Bey+16]. Stretch: the multi-region failover half — health-based DNS shifting and where canary gates live when traffic itself moves.

B.19 Chapter 18: Observability & SRE

B.19.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–10 list; numbering below matches.

1. Availability SLI: fraction of polling URLs that eventually yield a completed plan (good = terminal success within a deadline). Latency SLI: time from POST to plan-ready, not the 202's return time. Freshness SLI: age of the plan's input data at completion. A 202 only means *accepted* — nothing about the finished plan — so it can never be the availability SLI for the async result [Bey+16].
2. Monthly budget = $0.0005 \times 43,200 = 21.6$ min. Burned 14 by day 10: average burn rate $\approx 1.94\times$ (consumed fraction $14/21.6$ in one third of the month). Budget left: 7.6 min; at sustained $2\times$ burn the budget drains at $2 \times 21.6/30 = 1.44$ min/day, covering $7.6/1.44 \approx 5.3$ days — the latest a $2\times$ burn can start is about day 24.7 (day 24 to be safe).
3. A 100% error burst of t minutes contributes error ratio $t/60$ to the 1-hour window, so burn rate = $(t/60)/0.001 = 16.67t$. The $14.4\times$ fast page fires when $16.67t \geq 14.4$, i.e. $t \geq 0.864$ min ≈ 52 s — the shortest full outage that pages. Verify against the chapter simulator.
4. Choose boundaries so the 300 ms SLO threshold is bracketed tightly — e.g. a bucket boundary *at* 300 ms plus neighbors on both sides (≥ 2 buckets straddling the threshold) — because percentile estimates interpolate within a bucket. With a boundary exactly at the threshold, threshold classification is exact; otherwise the estimate error at the SLO is bounded by the containing bucket's relative width.
5. Series count: routes $\times$ statuses $\times$ 4,000 tenants $\times$ 30 versions $\approx$ millions of series — cardinality explosion. Keep `route` and `status`; reject `tenant_id` outright and demote `client_version` (or cap its values); rejected dimensions belong in logs and traces (exemplars), where cardinality is free [Ope24].
6. Inject the trace context into message metadata on publish; extract and continue the trace on consume. A consumer batching 50 messages into one span must use *span links* to the 50 producers — a single parent would falsely claim causal descent from one message and orphan the other 49 [Ope24].
7. “Average p99 across instances” understates pain two ways: (1) instances

with the worst tails are diluted by quiet instances; (2) it weights instances equally regardless of request volume, so a hot instance serving most users counts once. Correct aggregation: merge the per-instance histograms and compute the global percentile over all requests — never average percentiles.

8. Head sampling keeps a uniform 5% baseline; tail rules add: keep 100% of error traces (they cost money) and 100% of traces exceeding the latency SLO (they cost customers), plus a sampled slice of slow-but-successful requests near the threshold. Guaranteed survivors: every error trace and every SLO-breaching trace.
9. A growing gap between SERVER span start and the first child span is queueing/middleware time, not work time. Three candidates, each with a USE check: (1) connection/thread-pool exhaustion — pool utilization and queue depth (saturation); (2) event-loop or CPU blockage — CPU utilization/saturation; (3) a slow synchronous middleware call (auth, config) — that dependency’s latency metric.
10. Blameless rewrite: replace “the engineer misconfigured the pool” with mechanism questions — what validation allowed the misconfiguration to deploy, what signal would have caught it pre-impact, and why the change process permitted an unreviewed pool change. Action items: startup-time config validation, canary/announce for config changes, and a pool-saturation alert — fixes aimed at the system that made the error easy [Bey+16].

B.19.2 Capstone Grading Rubrics

Capstone 18.1 — Trace Context Conformance Pack [junior]

Criterion	Weight	Full marks when...
Corpus conformance	35%	all 25 hand-written corpus cases yield the expected verdict.
Property testing	25%	seeded round-trip property test passes 500/500.
Diagnostic quality	25%	every rejection message names the offending field.
Robustness	15%	parser only ever raises <code>ValueError</code> — no crashes on any input.

Reference approach. Extend the `traceparent` parser into a checker with a corpus-driven test table; round-trip encode/decode under a seed. Stretch: fuzz with random truncations and case flips.

Capstone 18.2 — RED Metrics for a Real Route Set [mid]

Criterion	Weight	Full marks when...
Metric accuracy	35%	per-route rate/errors/duration table matches the seeded model within jitter.
Cardinality discipline	25%	series-count assertion passes, fails loudly when a <code>user_id</code> label is added, and passes again when reverted.
Determinism	25%	one seed, no network, reproducible table.
Instrumentation quality	15%	per-route labeling consistent; no high-cardinality leaks.

Reference approach. Extend the instrumentation listing to label per route; replay a seeded traffic model; assert the RED table. Stretch: exemplars attaching one trace id per bucket.

Capstone 18.3 — SLO Workbench [mid]

Criterion	Weight	Full marks when...
Incident replay	30%	three scripted incidents produce reproducible alert timelines.
Flapping analysis	25%	the flapping incident is shown to reset-and-refire under the stock table, and your modification demonstrably addresses it.
Budget math	25%	the memo states the budget arithmetic for every policy row.
Policy defense	20%	one policy change defended with the generated data, not intuition [Bey+16].

Reference approach. Drive the burn-rate simulator with scripted incident timelines; tabulate alert fire/reset times per policy; modify one window/threshold and show the delta. Stretch: alert-resolution tracking with mean-time-to-resolve per policy.

Capstone 18.4 — Queue Propagation End to End [senior]

Criterion	Weight	Full marks when...
Trace continuity	30%	the exported tree crosses the broker boundary with one trace id and correct parentage.
Batch semantics	25%	batch consumption uses span links, never a false single parent [Ope24].
Legacy detection	25%	the legacy-consumer test (drops metadata) detects the orphaned span programmatically.
Determinism	20%	in-process fake broker; seeded; offline.

Reference approach. Inject context into message metadata on publish; extract on consume; link batch spans to all producers; the orphan detector flags spans whose parent never arrives. Stretch: baggage redaction at the queue boundary with proof no baggage key survives.

Capstone 18.5 — Incident Program in a Box [staff]

Criterion	Weight	Full marks when...
Artifact completeness	25%	SLOs, runbooks, and dashboards exist for three services and cite one another.
Game-day rigor	30%	reproducible detection/mitigation/resolution times; a second pair reproduces the timeline within one simulated minute.
Postmortem quality	25%	blameless postmortem includes an alerting-topology change with a named owner.
Operational loop	20%	rollback criteria for every mitigation; alert fatigue considered; follow-ups tracked.

Reference approach. Define SLOs from user journeys; derive burn-rate alerts; write runbooks that link to the dashboards that justify them; run the scripted game day end to end [Bey+16]. Stretch: a budget-policy gate blocking deploys when $>75\%$ of the error budget is consumed.

B.20 Chapter 19: Testing & Quality Engineering

B.20.1 Self-Assessment Model Answers

E-1. Classify each of the fifteen tests into exactly one layer (unit / component-service / end-to-end). The service layer is typically over-weighted relative to its defect-catching power — acceptable here because an API's contract surface *is* where its defects live; the pyramid becomes a honeycomb deliberately, not by drift.

E-2. Cheapest catchers: wrong SKU regex $\rightarrow$ unit property test over generated SKUs; renamed response field $\rightarrow$ contract test against the pact/schema; mispointed DB connection string $\rightarrow$ deployment smoke test (readiness against the real target); missing retry jitter $\rightarrow$ unit test of the policy with a fake clock asserting decorrelated delays; p99 regression after a serializer change $\rightarrow$ microbenchmark gate in CI.

- E-3.** The billing pact pins only `id` and `price_cents`. Renaming `name` breaks no consumer (no pact references it); renaming `price_cents` breaks exactly billing-service. This is *usage sensitivity* — a dimension schema diffing alone cannot express, because it cannot see which fields consumers actually read [Pac24].
- E-4.** Split the seeded provider state into two independent states and re-play. If two interactions share one app instance, state leaks across interactions: execution order changes outcomes, failures misattribute, and passes can be false. A fresh app (or transactional reset) per interaction restores independence.
- E-5.** Metamorphic relation: creating the same payload twice (minus `id`) must yield records equal up to the `id` field. It is stronger than any example assertion because it quantifies over *all* generated payloads — one counterexample anywhere in the generated space fails the property.
- E-6.** In the closed model, clients wait for completion before issuing the next request, so offered load self-throttles exactly when the server saturates: the queueing that defines the knee is omitted (coordinated omission), the measured knee shifts right, and capacity is overstated. The open-loop 400 rps sweep lies less because it keeps offering load the server cannot absorb.
- E-7.** Steady-state hypothesis: p99 latency and error rate of the shared staging environment remain within baseline bands under the latency-fault injection. Abort criteria: error budget burn $>2\times$, p99 beyond the band, or any page. Telemetry proving the experiment stayed in bounds: canary-cohort vs control metrics plus blast-radius scoping labels on every injected fault.
- E-8.** A defensible 4-minute split: unit tests ~ 60 s, contract verification ~ 90 s, schema-diff/lint ~ 30 s, a selected service-test slice ~ 60 s. Moved to nightly: fuzz/property campaigns and load sweeps. The evidence lost: long-run flakiness and performance regressions surface a day later — accepted knowingly, with the nightly results gated on release rather than on PR.

B.20.2 Capstone Grading Rubrics

Capstone 19.1 — A Second Consumer [junior]

Criterion	Weight	Full marks when...
Pact health	30%	both pact files verify green against the healthy app [Pac24].
Selective sensitivity	35%	the <code>sku</code> rename fails exactly one pact while the <code>name</code> rename fails none.
Diagnostic quality	20%	reports name the failing field with a JSONPath-style location.
Reproducibility	15%	deterministic, offline, seeded provider states.

Reference approach. Author the billing pact from real usage; run both renames as experiments and record which pact fails. Stretch: a `can-i-deploy(provider_version)` function consulting a local JSON “broker”.

Capstone 19.2 — Property Suite for a New Resource [mid]

Criterion	Weight	Full marks when...
Bug detection	30%	both planted bugs (500 trigger; schema-violating 201) are found and shrunk to minimal reproducers.
Invariant coverage	25%	zero 5xx and zero non-problem 4xx asserted across the whole campaign.
Determinism	25%	derandomized (seeded) suites pass repeatably.
Resource modeling	20%	<code>/warehouses</code> lifecycle fully covered by generators.

Reference approach. Generators for valid and near-valid payloads; properties over create/read/update; shrinking on failure. Stretch: a rule-based stateful machine with a local model oracle.

Capstone 19.3 — Record–Replay Cassette Library [mid]

Criterion	Weight	Full marks when...
Transparency	30%	replay works with zero application-code changes (transport-level interception).
Secret hygiene	30%	no credential-like header ever reaches disk — proven by test.
Staleness handling	25%	stale cassettes detected and reported by date or hash.
Matching correctness	15%	request matching on method+path+canonical body behaves per spec.

Reference approach. A VCR-style transport for `httpx`: record mode writes sanitized interactions; replay mode matches and returns them; a scrubber runs before persistence. Stretch: strict vs lax matching modes.

Capstone 19.4 — Capacity Report from a Knee Sweep [senior]

Criterion	Weight	Full marks when...
Prediction-measurement agreement	30%	predicted and measured knees agree within a factor of two, with the discrepancy explained.
Little’s Law cross-check	25%	concurrency at the knee reconciles with $C = \lambda L$.
Recommendation quality	25%	explicit headroom recommendation with the assumptions listed.
Methodology	20%	open-model sweep; no wall-clock assertions anywhere.

Reference approach. Sweep offered load in open model; locate the knee where latency departs linearity; cross-check with Little’s Law; report with headroom. Stretch: add the coordinated-omission buggy scheduler variant and quantify how much it overstates capacity.

Capstone 19.5 — Quality-Gate Architecture for a Fleet [staff]

Criterion	Weight	Full marks when...
Defect-catcher mapping	25%	every defect class has exactly one designated cheapest catcher — no duplicated or orphaned classes.
Budget discipline	25%	the PR gate fits four minutes with written justification per cut.
Adoption metrics	25%	rollout plan includes measurable adoption (pact coverage of production traffic) and a self-check requirement per gate.
Operational sustainability	25%	gate ownership, exception process, and rollback of a bad gate defined; flaky-gate handling stated.

Reference approach. Build the defect-class inventory first; assign each class its cheapest layer; budget the PR pipeline by measurement; instrument the gates themselves. Stretch: scheduled mutation testing of the gates as a meta-gate.

B.21 Chapter 20: Lifecycle, Governance & Developer Experience

B.21.1 Self-Assessment Model Answers

Numbering note: the chapter uses a plain 1–10 list; numbering below matches.

1. (a) PR adding an endpoint to a published spec: no state transition if the spec is still draft; if published, the change requires a new minor/major version — evidence: the spec diff plus the compatibility report. (b) security flag on an unauthenticated endpoint: published → restricted/remediation workflow — evidence: the threat-model worksheet. (c) brownout revealing an unknown consumer three weeks before the kill date: deprecated → sunset-blocked (extension) — evidence: access logs identifying the consumer.
2. The rule needs a *function/custom* kind: “every POST operation documents a 201 or 202 response” is an any-of-keys existential that schema-style rule kinds cannot express — that gap (no anyOf-keys kind) is the finding; the rule walks `paths.*.post.responses` and asserts the intersection with `{201, 202}` is non-empty.

3. With 40 APIs, **error**: the fix cost is low, humans absorb the findings, and the standard actually binds. With 400 APIs, **warn** plus a ratchet (error only on new or changed files): error-everywhere creates thousands of findings, alert fatigue, and a bypass culture that destroys the gate's authority — governance that only says “no” gets routed around (Chapter 21).
4. Hypotheses for the 73.3% key→first-call conversion with a 194-minute median gap: (a) *onboarding friction* — instrument step-level funnel events with timestamps to find the drop-off step; (b) *first-call failure* (docs mismatch, TLS, auth) — instrument first-call outcomes with error reason codes. The instrument, not the fix, is the deliverable.
5. SLO budget: 0.1% allowed, 0.07% consumed (99.93% measured vs 99.9% SLO) → 0.03% remains. SLA budget: 0.5% allowed → 0.43% remains. A change spending another 0.05% exceeds the remaining *SLO* budget, so the SLO governs: defer the change, shrink its blast radius, or take a formal exception — the stricter internal contract exists precisely to protect the looser external one.
6. The ADR paragraph names: scope (the one legacy partner endpoint), reason (contracted consumer, migration cost), expiry (six months, a date, not a condition), owner, and removal criteria. It is time-boxed and auditable because the lint waiver carries an **expires** field that CI enforces — the exemption fails the build the day it lapses.
7. Generated docs derive from the spec, so if the hand-written quickstart disagrees, the quickstart is wrong — unless the spec itself is wrong, in which case both are and the spec fix regenerates the docs. The process gap: the quickstart is never tested against the spec. Preventing lint rule: a docs-snippet conformance check (code samples executed or schema-checked in CI).
8. Standard cadence rows: announce (with date) → Deprecation/Sunset headers → staged brownouts → kill [Wil19]. For the contracted enterprise consumer the changed rows: extended notice per contract, named-contact outreach instead of broadcast channels, a documented extension clause, and brownout exemptions — the twelve internal consumers follow the standard schedule with owner-assigned migration tasks.
9. Telemetry query: join gateway access logs (path, method, consumer id, volume) against the catalog registry; alert on any path with traffic above a threshold that has no registry entry. First-24-hours runbook:

identify the owning team via deploy logs and request patterns, register or block the endpoint deliberately, and assess what data the shadow surface exposed.

10. The honest slide: adoption (teams/APIs onboarded to paved paths), TTFC reduction (days $\rightarrow$ hours, from funnel telemetry), and incident reduction (P1 count and MTTR delta priced at incident cost). Weakest assumption: counterfactual attribution — teams that adopt platform tooling were likely the better-instrumented teams anyway (selection bias); say so on the slide.

B.21.2 Capstone Grading Rubrics

Capstone 20.1 — Three New Rules for the Fleet [junior]

Criterion	Weight	Full marks when...
Rule efficacy	35%	compliant specs still exit 0; the extended fixture exits 1 with all three new rule IDs firing.
Diagnostic quality	25%	every violation carries a JSON path to the offender.
Engineering	25%	new rule kinds unit-tested, including a bad-ruleset exit-2 test.
Convention fit	15%	rules follow the existing engine's kinds and style.

Reference approach. Extend the mini-Spectral engine with one rule per gap; test each rule positive and negative. Stretch: `x-lint-ignore` suppression comments honored and logged as warnings.

Capstone 20.2 — Docs-as-Code Pipeline [mid]

Criterion	Weight	Full marks when...
Single-command pipeline	30%	lint $\rightarrow$ docs $\rightarrow$ catalog for all specs in one run; non-compliant specs block the run.
Staleness gate	30%	editing a spec without regenerating docs fails CI.
Determinism	25%	fully offline, byte-deterministic output.
Artifact quality	15%	generated docs are navigable and the catalog is complete.

Reference approach. Chain the chapter's linter, doc generator, and catalog builder in one script; the staleness check hashes spec inputs into the generated output and compares in CI. Stretch: a stdlib Markdown link checker over the generated docs.

Capstone 20.3 — TTFC Instrumentation Pack [mid]

Criterion	Weight	Full marks when...
Funnel integrity	30%	per-channel funnels show conversion deltas with counts — never a percentage without n .
Cohort determinism	25%	weekly cohort table deterministic and sorted; malformed events counted-and-skipped, never fatal.
Decision orientation	25%	the experiment report names one metric, one funnel step, one falsifiable threshold.
Small-sample honesty	20%	small- n cohorts flagged so noise is not over-read.

Reference approach. Extend the funnel analyzer from report to decision tool: per-channel breakdowns, cohort tables, and an experiment template. Stretch: the $n < 10$ flag.

Capstone 20.4 — Sunset Campaign for Fleet API v1 [senior]

Criterion	Weight	Full marks when...
Projection accuracy	30%	the model flags in advance which consumers will miss the kill date under the stated migration velocity.
Policy differentiation	25%	internal vs external consumers follow the chapter's differentiated policy table.
Campaign mechanics	25%	headers, brownouts, and communications scheduled; dates are explicit CLI inputs.
CI semantics	20%	exit codes remain CI-meaningful throughout the campaign.

Reference approach. Replay a heterogeneous consumer log; model migration velocity per consumer; schedule brownouts against the projection [Wil19]. Stretch: one extension request and its effect on the brownout schedule for non-extended consumers.

Capstone 20.5 — Platform ROI Scorecard & Governance Operating Model [staff]

Criterion	Weight	Full marks when...
Reproducible numbers	30%	every scorecard figure traces to a reproducible computation from synthetic inputs.
Narrative honesty	25%	the ROI narrative states its weakest assumption explicitly.
Operating model	25%	two pages naming decision rights for every governance event (exceptions, sunsets, standard changes).
Quarterly cadence	20%	the review is runnable end-to-end as a quarterly ritual with owners and artifacts.

Reference approach. Build the scorecard from the funnel, incident, and adoption data pipelines; write the operating model as a decision-rights table; rehearse the quarterly review once. Stretch: a counterfactual section estimating incident cost avoided, with the selection-bias caveat addressed head-on.

B.22 Chapter 21: Synthesis & Future Directions

B.22.1 Self-Assessment Model Answers

Concept Review

1. Delivery model comes first because it prunes whole protocol families: a streaming requirement eliminates plain request/response options before consumer type is even examined. Reversed-order failure: choosing “internal consumers → gRPC” for a fleet telemetry *push* leads to bolting streaming semantics onto the wrong conversation — delivery shape dominates consumer identity.
2. Any cell is a tendency; a legitimate exception to “gRPC is for internal service-to-service”: several major public platforms expose gRPC (or gRPC+transcoding) externally with great success — the tendency encodes tooling/browser friction, not a law [gRP24].
3. The outbox relay reads from the database because a service cannot atomically write its database *and* publish to the broker: crash between the two writes yields either a lost event (DB committed, publish never sent) or a phantom event (publish sent, DB rolled back). The outbox

makes the database the single atomic commit point and the relay a replayable follower — the dual-write anomaly is designed out [Kle17].

4. **ETag + Cache-Control** (conditional GETs and shared caching) make a public REST catalog dramatically cheaper: repeat reads cost headers only or hit a CDN [Fie22a]. GraphQL endpoints do not benefit equally because queries are POSTed bodies to one endpoint — HTTP caches have nothing to key on without persisted-query-over-GET gymnastics.
5. O-002 (a ProblemDetails reference on every 4xx/5xx) prevents the *contract/observability* class of failure: unstructured errors consumers cannot parse programmatically. It protects consumers (stable error handling) and on-call (classifiable failures) alike [Not23].
6. Deterministic tie-breaking makes the advisor a pure function of its inputs — “either is fine” is an unstable output that breaks the snapshot-style tests asserting byte-identical recommendations per answer set; determinism is what makes the tool testable and its output diffable in review.
7. MCP tool schemas generated from the OpenAPI source of truth cannot drift from the human-facing contract; hand-maintained tool schemas rot silently, and an agent acting on a stale schema produces confident, wrong calls — the same contract-drift failure class as any codegen target, now with an autonomous consumer.
8. The case-study reasoning: the frontiers table prices edge compute as immature tooling plus new operational burden (observability, deploys, data residency), while the order path’s bottleneck was measurable elsewhere; the marginal latency win did not pay the migration risk for a small team — being early is a cost, not a virtue.

Architecture-Design Drills

- Drill 1.** *Hospital device monitoring.* Profiles: internal, regulated, no public internet. Surfaces: device ingestion over gRPC streaming (efficient, schema-strict); nurse dashboards over SSE (freshness <2s, proxy-friendly, resumable); auditor reads over REST with **Cache-Control: max-age=86400** and ETags. Top failure modes: device-data loss on disconnect (buffer at the edge + outbox on ingestion) and dashboard staleness (heartbeats, Last-Event-ID resume, freshness SLI).
- Drill 2.** *Marketplace payout API.* Profiles: public, financial, exactly-once at the boundary. Surfaces: REST POST with **Idempotency-Key** plus unique constraints (Chapter 11); async bank confirmation via signed

webhooks with DLQ (Chapter 12); RFC 9457 errors (Chapter 8). Top failure modes: duplicate payout (keys + constraints + replayed stored responses) and slow/flaky partner banks (deadlines, circuit breaker, saga with compensation); bursts handled by a queue-backed admission tier with rate limits (Chapter 15).

- Drill 3.** *Multi-tenant analytics backend.* Defend GraphQL: forty entity types with unknowable, user-configured payload shapes are its home territory. Guardrails either way: persisted queries for production traffic, depth/complexity limits, per-tenant cost-based rate limiting, per-request DataLoaders, and APQ-over-GET for the metered mobile variant (Chapter 13).
- Drill 4.** *Fleet firmware orchestration.* Some networks forbid inbound device connections, so server-push alone is disqualified: choose **device-initiated long-polling with ETags/signed cursors** — resumable, NAT-friendly, cacheable; offer SSE where inbound is permitted. Commands signed and expiring; jittered poll intervals to avoid a post-outage thundering herd of 50,000 devices. Failure modes: herd after outage (jitter + backoff), stale/replayed commands (signatures + expiry + per-device cursors).
- Drill 5.** *Your own platform.* Apply the six criteria surface by surface; where the card disagrees with history, decide whether the card missed a constraint actually present (data residency, team skills, regulatory) — extend the card — or whether the historical choice was fashion-driven — document the drift. The deliverable is the annotated disagreement list, not agreement.

B.22.2 Capstone Grading Rubrics

Numbering note: the chapter labels its capstones “Project 1–5”; they are cited here as C21.1–C21.5.

Capstone 21.1 — Extend the Decision Advisor [junior]

Criterion	Weight	Full marks when...
Questionnaire robustness	25%	interactive mode runs offline and rejects invalid answers clearly.
Regression safety	25%	all pre-existing advisor tests pass unmodified.
New-criterion correctness	25%	data-residency tests cover scoring and -explain mentions residency when it moved a score.
Explainability	25%	-explain output traces every recommendation to criterion scores.

Reference approach. Add the seventh criterion to the scoring table, thread it through the rationale trace, and wrap the questionnaire around the existing pure scoring function. Stretch: JSON score output for CI and a confidence metric from the winner's margin.

Capstone 21.2 — Governance Toolchain [mid]

Criterion	Weight	Full marks when...
Rule coverage	30%	all five original rule families plus three new rules fire on failing fixtures and pass on clean ones, with positive/negative tests each.
Reporting	25%	JSON report parses and round-trips deterministically; fix-hints attached.
Changed-only mode	25%	CI mode ignores pre-existing violations in untouched files.
Engineering	20%	offline, deterministic, documented exit codes.

Reference approach. Grow the conformance checker: rule registry, finding model with fix hints, diff-aware file selection. Stretch: severity levels with per-rule policy-file overrides and SARIF output.

Capstone 21.3 — Multi-Protocol Read/Write Split [senior]

Criterion	Weight	Full marks when...
Write-path discipline	25%	REST writes return RFC 9457 errors and honor idempotency keys [Not23].
Read-path consistency	25%	GraphQL reads never expose uncommitted state.
Concurrency proof	25%	the no-overselling concurrency test passes deterministically in 100 runs.
Contract governance	25%	OpenAPI and GraphQL SDL artifacts both pass the C21.2 checker.

Reference approach. One shared store behind three surfaces: transactional REST writes, GraphQL read models with DataLoaders, gRPC for internal coordination; consistency proven by the seeded concurrency harness. Stretch: CDC-style lag metric with a freshness SLO and a broker-outage lossless-recovery drill.

Capstone 21.4 — Webhook Notification Pipeline with Outbox Replay [senior]

Criterion	Weight	Full marks when...
Losslessness	30%	no event lost across the simulated outage, proven by test (outbox → relay → delivery log).
Security	25%	signature failures and stale timestamps rejected with 401/422 problem details.
Replay fidelity	25%	replay re-sends exactly the logged payload — byte-identical.
Contract & fallback	20%	AsyncAPI spec passes the conformance checker; SSE fallback demonstrated [Asy23].

Reference approach. Compose the book’s reference pieces: outbox relay, HMAC signing, delivery log, operator replay endpoint, AsyncAPI documentation, SSE fallback for push-averse consumers. Stretch: DLQ with operator triage endpoint and a delivery-latency histogram metric.

Capstone 21.5 — The Grand Capstone: Northwind Robotics API Platform [staff]

Criterion	Weight	Full marks when...
Surface completeness	25%	all five surfaces respond per spec in one offline in-process test run.
Governance	20%	the conformance checker passes on the full manifests folder.
Measured SLOs	20%	SLOs measured green against a recorded load test — asserted, not claimed [Bey+16].
Resilience proof	20%	the chaos drill (kill inventory mid-order) produces the designed failure behavior with zero data loss.
Coherence & score	15%	rubric score $\geq 16/20$ with no dimension below 3; rollback and observability designed in, not bolted on.

Reference approach. Build in the four milestones (contracts, surfaces, resilience, governance), each milestone shippable; reuse every prior capstone’s

hardened component rather than rewriting; the final system is graded as a *platform* — coherent contracts, measured SLOs, rehearsed failure, and a working governance loop [New21; Kle17]. Stretch: canary simulation with traffic splitting, an MCP tool surface generated from the OpenAPI spec, and a regional-affinity design document for EU expansion.

Bibliography

- [Asy23] AsyncAPI Initiative. *AsyncAPI Specification*. <https://www.asyncapi.com/docs/reference/specification/latest>. 2023.
- [Bey+16] Betsy Beyer et al., eds. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [Bis22] Bishop, Mike. *HTTP/3*. RFC 9114. <https://www.rfc-editor.org/rfc/rfc9114>. IETF, 2022.
- [Bra17] Bray, Tim. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. <https://www.rfc-editor.org/rfc/rfc8259>. IETF, 2017.
- [Col24] Colvin, Samuel and contributors. *Pydantic Documentation*. <https://docs.pydantic.dev/>. 2024.
- [Enc24] Encode. *HTTPX Documentation*. <https://www.python-httpx.org/>. 2024.
- [Fie00] Roy Thomas Fielding. “Architectural Styles and the Design of Network-based Software Architectures”. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>. PhD thesis. University of California, Irvine, 2000.
- [Fie22a] Fielding, Roy T. and Nottingham, Mark and Reschke, Julian. *HTTP Semantics*. RFC 9110. <https://www.rfc-editor.org/rfc/rfc9110>. IETF, 2022.
- [Fie22b] Fielding, Roy T. and Nottingham, Mark and Reschke, Julian. *HTTP/1.1*. RFC 9112. <https://www.rfc-editor.org/rfc/rfc9112>. IETF, 2022.
- [GH21] Matthias Gehl and Daniel Higginbotham. *API Design Patterns*. See also: JJ Geewax, *API Design Patterns*, Manning, 2021. Manning, 2021.
- [Goo24] Google. *Protocol Buffers Documentation*. <https://protobuf.dev/>. 2024.
- [Gra21] GraphQL Foundation. *GraphQL Specification*. <https://spec.graphql.org/>. 2021.

- [gRP24] gRPC Authors. *gRPC Documentation*. <https://grpc.io/docs/>. 2024.
- [Har12] Hardt, Dick. *The OAuth 2.0 Authorization Framework*. RFC 6749. <https://www.rfc-editor.org/rfc/rfc6749>. IETF, 2012.
- [Iye21] Iyengar, Jana and Thomson, Martin. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. <https://www.rfc-editor.org/rfc/rfc9000>. IETF, 2021.
- [Jon15] Jones, Michael and Bradley, John and Sakimura, Nat. *JSON Web Token (JWT)*. RFC 7519. <https://www.rfc-editor.org/rfc/rfc7519>. IETF, 2015.
- [JSO22] JSON Schema Team. *JSON Schema Specification (2020-12)*. <https://json-schema.org/specification>. 2022.
- [Kle17] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [New21] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2nd ed. O'Reilly Media, 2021.
- [Not17] Nottingham, Mark. *Web Linking*. RFC 8288. <https://www.rfc-editor.org/rfc/rfc8288>. IETF, 2017.
- [Not23] Nottingham, Mark and Wilde, Erik and Dalal, Sanjay. *Problem Details for HTTP APIs*. RFC 9457. <https://www.rfc-editor.org/rfc/rfc9457>. IETF, 2023.
- [Nyg18] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. 2nd ed. Pragmatic Bookshelf, 2018.
- [Ope21] OpenAPI Initiative. *OpenAPI Specification v3.1.0*. <https://spec.openapis.org/oas/v3.1.0>. 2021.
- [Ope24] OpenTelemetry Authors. *OpenTelemetry Documentation*. <https://opentelemetry.io/docs/>. 2024.
- [OWA23] OWASP. *OWASP API Security Top 10 – 2023*. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>. 2023.
- [Pac24] Pact Foundation. *Pact Contract Testing Documentation*. <https://docs.pact.io/>. 2024.
- [Peo15] Peon, Roberto and Ruellan, Herve. *HPACK: Header Compression for HTTP/2*. RFC 7541. <https://www.rfc-editor.org/rfc/rfc7541>. IETF, 2015.
- [Pol24] Polli, Roberto and Pario, Alejandro Martinez. *RateLimit Header Fields for HTTP (Internet-Draft)*. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>. IETF, 2024.

- [Pyt24] Python Software Foundation. *Requests: HTTP for Humans*. <https://requests.readthedocs.io/>. 2024.
- [Ram24] Ramírez, Sebastián. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>. 2024.
- [Res18] Rescorla, Eric. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. <https://www.rfc-editor.org/rfc/rfc8446>. IETF, 2018.
- [RAR13] Leonard Richardson, Mike Amundsen, and Sam Ruby. *RESTful Web APIs*. O'Reilly Media, 2013.
- [Sak15] Sakimura, Nat and Bradley, John and Agarwal, Naveen. *Proof Key for Code Exchange by OAuth Public Clients*. RFC 7636. <https://www.rfc-editor.org/rfc/rfc7636>. IETF, 2015.
- [Tho22] Thomson, Martin and Benfield, Cory. *HTTP/2*. RFC 9113. <https://www.rfc-editor.org/rfc/rfc9113>. IETF, 2022.
- [Wig17] Wiggins, Adam. *The Twelve-Factor App*. <https://12factor.net/>. 2017.
- [Wil19] Wilde, Erik. *The Sunset HTTP Header Field*. RFC 8594. <https://www.rfc-editor.org/rfc/rfc8594>. IETF, 2019.